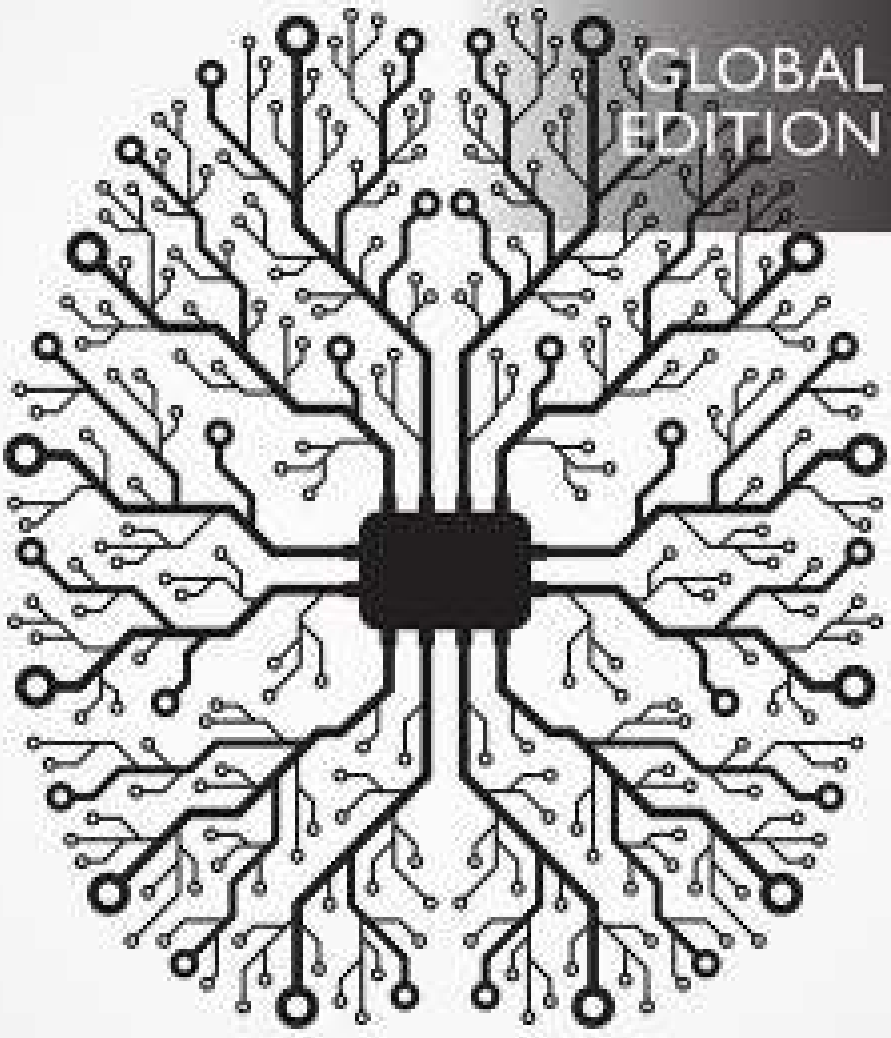
GLOBAL EDITION



# Operating Systems

## *Internals and Design Principles*

NINTH EDITION

William Stallings

Pearson

# Operating Systems

*This page intentionally left blank*

# Operating Systems

## *Internals and Design Principles*

## Ninth Edition

## Global Edition

William Stallings

Pearson

Senior Vice President Courseware Portfolio Management: Marcia J. Horton
Director, Portfolio Management: Engineering, Computer Science & Global Editions: Julian Partridge
Higher Ed Portfolio Management: Tracy Johnson (Dunkelberger)
Acquisitions Editor, Global Editions: Sourabh Maheshwari
Portfolio Management Assistant: Kristy Alaura
Managing Content Producer: Scott Disanno
Content Producer: Robert Engelhardt
Project Editor, Global Editions: K.K. Neelakantan
Web Developer: Steve Wright
Rights and Permissions Manager: Ben Ferrini
Manufacturing Buyer, Higher Ed, Lake Side Communications Inc (LSC): Maura Zaldivar-Garcia
Senior Manufacturing Controller, Global Editions: Trudy Kimber
Media Production Manager, Global Editions: Vikram Kumar
Inventory Manager: Ann Lam
Marketing Manager: Demetrius Hall
Product Marketing Manager: Yvonne Vannatta
Marketing Assistant: Jon Bryant
Cover Designer: Lumina Datamatics
Cover Art: Shai_Halud/Shutterstock
Full-Service Project Management: Bhanuprakash Sherla, SPi Global

Credits and acknowledgments borrowed from other sources and reproduced, with permission, in this textbook appear on page CL-1.

Many of the designations by manufacturers and seller to distinguish their products are claimed as trademarks. Where those designations appear in this book, and the publisher was aware of a trademark claim, the designations have been printed in initial caps or all caps.

Pearson Education Limited
Edinburgh Gate
Harlow
Essex CM20 2JE
England

and Associated Companies throughout the world

Visit us on the World Wide Web at:
**www.pearsonglobaleditions.com**

© Pearson Education Limited 2018

The right of William Stallings to be identified as the author of this work has been asserted by him in accordance with the Copyright, Designs and Patents Act 1988.

*Authorized adaptation from the United States edition, entitled Operating Systems: Internals and Design Principles, 9th Edition, ISBN 978-0-13-467095-9, by William Stallings published by Pearson Education © 2018.*

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without either the prior written permission of the publisher or a license permitting restricted copying in the United Kingdom issued by the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS.

All trademarks used herein are the property of their respective owners. The use of any trademark in this text does not vest in the author or publisher any trademark ownership rights in such trademarks, nor does the use of such trademarks imply any affiliation with or endorsement of this book by such owners.

**British Library Cataloguing-in-Publication Data**
A catalogue record for this book is available from the British Library

10 9 8 7 6 5 4 3 2 1

ISBN 10: 1-292-21429-5
ISBN 13: 978-1-292-21429-0

Typeset by SPi Global

Printed and bound in Malaysia.

*For Tricia*

*This page intentionally left blank*

# CONTENTS

**Online Chapters and Appendices 13**

**VideoNotes 15**

**Preface 17**

**About the Author 27**

**PART 1 BACKGROUND 29**

**Chapter 1 Computer System Overview 29**

**1.1** Basic Elements 30
**1.2** Evolution of the Microprocessor 32
**1.3** Instruction Execution 32
**1.4** Interrupts 35
**1.5** The Memory Hierarchy 46
**1.6** Cache Memory 49
**1.7** Direct Memory Access 53
**1.8** Multiprocessor and Multicore Organization 54
**1.9** Key Terms, Review Questions, and Problems 58
**1A** Performance Characteristics of Two-Level Memories 61

**Chapter 2 Operating System Overview 68**

**2.1** Operating System Objectives and Functions 69
**2.2** The Evolution of Operating Systems 73
**2.3** Major Achievements 83
**2.4** Developments Leading to Modern Operating Systems 92
**2.5** Fault Tolerance 95
**2.6** OS Design Considerations for Multiprocessor and Multicore 98
**2.7** Microsoft Windows Overview 101
**2.8** Traditional UNIX Systems 108
**2.9** Modern UNIX Systems 110
**2.10** Linux 113
**2.11** Android 118
**2.12** Key Terms, Review Questions, and Problems 127

**PART 2 PROCESSES 129**

**Chapter 3 Process Description and Control 129**

**3.1** What is a Process? 131
**3.2** Process States 133
**3.3** Process Description 148

**3.4** Process Control 157
**3.5** Execution of the Operating System 163
**3.6** UNIX SVR4 Process Management 166
**3.7** Summary 171
**3.8** Key Terms, Review Questions, and Problems 171

**Chapter 4 Threads 176**

**4.1** Processes and Threads 177
**4.2** Types of Threads 183
**4.3** Multicore and Multithreading 190
**4.4** Windows Process and Thread Management 195
**4.5** Solaris Thread and SMP Management 202
**4.6** Linux Process and Thread Management 206
**4.7** Android Process and Thread Management 211
**4.8** Mac OS X Grand Central Dispatch 215
**4.9** Summary 217
**4.10** Key Terms, Review Questions, and Problems 218

**Chapter 5 Concurrency: Mutual Exclusion and Synchronization 223**

**5.1** Mutual Exclusion: Software Approaches 226
**5.2** Principles of Concurrency 232
**5.3** Mutual Exclusion: Hardware Support 241
**5.4** Semaphores 244
**5.5** Monitors 257
**5.6** Message Passing 263
**5.7** Readers/Writers Problem 270
**5.8** Summary 274
**5.9** Key Terms, Review Questions, and Problems 275

**Chapter 6 Concurrency: Deadlock and Starvation 289**

**6.1** Principles of Deadlock 290
**6.2** Deadlock Prevention 299
**6.3** Deadlock Avoidance 300
**6.4** Deadlock Detection 306
**6.5** An Integrated Deadlock Strategy 308
**6.6** Dining Philosophers Problem 309
**6.7** UNIX Concurrency Mechanisms 313
**6.8** Linux Kernel Concurrency Mechanisms 315
**6.9** Solaris Thread Synchronization Primitives 324
**6.10** Windows Concurrency Mechanisms 326
**6.11** Android Interprocess Communication 330
**6.12** Summary 331
**6.13** Key Terms, Review Questions, and Problems 332

**PART 3 MEMORY 339**

**Chapter 7 Memory Management 339**

**7.1** Memory Management Requirements 340
**7.2** Memory Partitioning 344
**7.3** Paging 355
**7.4** Segmentation 358
**7.5** Summary 360
**7.6** Key Terms, Review Questions, and Problems 360
**7A** Loading and Linking 363

**Chapter 8 Virtual Memory 370**

**8.1** Hardware and Control Structures 371
**8.2** Operating System Software 388
**8.3** UNIX and Solaris Memory Management 407
**8.4** Linux Memory Management 413
**8.5** Windows Memory Management 417
**8.6** Android Memory Management 419
**8.7** Summary 420
**8.8** Key Terms, Review Questions, and Problems 421

**PART 4 SCHEDULING 425**

**Chapter 9 Uniprocessor Scheduling 425**

**9.1** Types of Processor Scheduling 426
**9.2** Scheduling Algorithms 430
**9.3** Traditional UNIX Scheduling 452
**9.4** Summary 454
**9.5** Key Terms, Review Questions, and Problems 455

**Chapter 10 Multiprocessor, Multicore, and Real-Time Scheduling 460**

**10.1** Multiprocessor and Multicore Scheduling 461
**10.2** Real-Time Scheduling 474
**10.3** Linux Scheduling 489
**10.4** UNIX SVR4 Scheduling 492
**10.5** UNIX FreeBSD Scheduling 494
**10.6** Windows Scheduling 498
**10.7** Summary 500
**10.8** Key Terms, Review Questions, and Problems 500

**PART 5 INPUT/OUTPUT AND FILES 505**

**Chapter 11 I/O Management and Disk Scheduling 505**

**11.1** I/O Devices 506
**11.2** Organization of the I/O Function 508
**11.3** Operating System Design Issues 511

**11.4** I/O Buffering 514
**11.5** Disk Scheduling 517
**11.6** RAID 524
**11.7** Disk Cache 533
**11.8** UNIX SVR4 I/O 537
**11.9** Linux I/O 540
**11.10** Windows I/O 544
**11.11** Summary 546
**11.12** Key Terms, Review Questions, and Problems 547

**Chapter 12 File Management 550**

**12.1** Overview 551
**12.2** File Organization and Access 557
**12.3** B-Trees 561
**12.4** File Directories 564
**12.5** File Sharing 569
**12.6** Record Blocking 570
**12.7** Secondary Storage Management 572
**12.8** UNIX File Management 580
**12.9** Linux Virtual File System 585
**12.10** Windows File System 589
**12.11** Android File Management 594
**12.12** Summary 595
**12.13** Key Terms, Review Questions, and Problems 596

**PART 6 EMBEDDED SYSTEMS 599**

**Chapter 13 Embedded Operating Systems 599**

**13.1** Embedded Systems 600
**13.2** Characteristics of Embedded Operating Systems 605
**13.3** Embedded Linux 609
**13.4** TinyOS 615
**13.5** Key Terms, Review Questions, and Problems 625

**Chapter 14 Virtual Machines 627**

**14.1** Virtual Machine Concepts 628
**14.2** Hypervisors 631
**14.3** Container Virtualization 635
**14.4** Processor Issues 642
**14.5** Memory Management 644
**14.6** I/O Management 645
**14.7** VMware ESXi 647
**14.8** Microsoft Hyper-V and Xen Variants 650
**14.9** Java VM 651
**14.10** Linux Vserver Virtual Machine Architecture 652
**14.11** Summary 655
**14.12** Key Terms, Review Questions, and Problems 655

**Chapter 15 Operating System Security 657**

**15.1** Intruders and Malicious Software 658
**15.2** Buffer Overflow 662
**15.3** Access Control 670
**15.4** UNIX Access Control 678
**15.5** Operating Systems Hardening 681
**15.6** Security Maintenance 685
**15.7** Windows Security 686
**15.8** Summary 691
**15.9** Key Terms, Review Questions, and Problems 692

**Chapter 16 Cloud and IoT Operating Systems 695**

**16.1** Cloud Computing 696
**16.2** Cloud Operating Systems 704
**16.3** The Internet of Things 720
**16.4** IoT Operating Systems 724
**16.5** Key Terms and Review Questions 731

**APPENDICES**

**Appendix A Topics in Concurrency A-1**

**A.1** Race Conditions and Semaphores A-2
**A.2** A Barbershop Problem A-9
**A.3** Problems A-14

**Appendix B Programming and Operating System Projects B-1**

**B.1** Semaphore Projects B-2
**B.2** File Systems Project B-3
**B.3** OS/161 B-3
**B.4** Simulations B-4
**B.5** Programming Projects B-4
**B.6** Research Projects B-6
**B.7** Reading/Report Assignments B-7
**B.8** Writing Assignments B-7
**B.9** Discussion Topics B-7
**B.10** BACI B-7

**References R-1**

**Credits CL-1**

**Index I-1**

*This page intentionally left blank*

# ONLINE CHAPTERS AND APPENDICES[1]


**Chapter 17 Network Protocols**

**17.1** The Need for a Protocol Architecture 17-3
**17.2** The TCP/IP Protocol Architecture 17-5
**17.3** Sockets 17-12
**17.4** Linux Networking 17-16
**17.5** Summary 17-18
**17.6** Key Terms, Review Questions, and Problems 17-18
**17A** The Trivial File Transfer Protocol 17-21

**Chapter 18 Distributed Processing, Client/Server, and Clusters**

**18.1** Client/Server Computing 18-2
**18.2** Distributed Message Passing 18-12
**18.3** Remote Procedure Calls 18-16
**18.4** Clusters 18-19
**18.5** Windows Cluster Server 18-25
**18.6** Beowulf and Linux Clusters 18-27
**18.7** Summary 18-29
**18.8** References 18-29
**18.9** Key Terms, Review Questions, and Problems 18-30

**Chapter 19 Distributed Process Management**

**19.1** Process Migration 19-2
**19.2** Distributed Global States 19-9
**19.3** Distributed Mutual Exclusion 19-14
**19.4** Distributed Deadlock 19-23
**19.5** Summary 19-35
**19.6** References 19-35
**19.7** Key Terms, Review Questions, and Problems 19-37

**Chapter 20 Overview of Probability and Stochastic Processes**

**20.1** Probability 20-2
**20.2** Random Variables 20-7
**20.3** Elementary Concepts of Stochastic Processes 20-12
**20.4** Problems 20-20

**Chapter 21 Queueing Analysis**

**21.1** How Queues Behave—A Simple Example 21-3
**21.2** Why Queueing Analysis? 21-8


[1]Online chapters, appendices, and other documents are Premium Content, available via the access card at the front of this book.

**21.3** Queueing Models 21-10
**21.4** Single-Server Queues 21-17
**21.5** Multiserver Queues 21-20
**21.6** Examples 21-20
**21.7** Queues With Priorities 21-26
**21.8** Networks of Queues 21-27
**21.9** Other Queueing Models 21-31
**21.10** Estimating Model Parameters 21-32
**21.11** References 21-35
**21.12** Problems 21-35

**Programming Project One Developing a Shell**

**Programming Project Two The HOST Dispatcher Shell**

**Appendix C Topics in Concurrency C-1**

**Appendix D Object-Oriented Design D-1**

**Appendix E Amdahl's Law E-1**

**Appendix F Hash Tables F-1**

**Appendix G Response Time G-1**

**Appendix H Queueing System Concepts H-1**

**Appendix I The Complexity of Algorithms I-1**

**Appendix J Disk Storage Devices J-1**

**Appendix K Cryptographic Algorithms K-1**

**Appendix L Standards Organizations L-1**

**Appendix M Sockets: A Programmer's Introduction M-1**

**Appendix N The International Reference Alphabet N-1**

**Appendix O BACI: The Ben-Ari Concurrent Programming System O-1**

**Appendix P Procedure Control P-1**

**Appendix Q ECOS Q-1**

**Glossary**

# VIDEONOTES

**Locations of VideoNotes**

**http://www.pearsonglobaleditions.com/stallings**


**Chapter 5 Concurrency: Mutual Exclusion and Synchronization 223**

- **5.1** Mutual Exclusion Attempts 227
- **5.2** Dekker's Algorithm 230
- **5.3** Peterson's Algorithm for Two Processes 231
- **5.4** Illustration of Mutual Exclusion 238
- **5.5** Hardware Support for Mutual Exclusion 242
- **5.6** A Definition of Semaphore Primitives 246
- **5.7** A Definition of Binary Semaphore Primitives 247
- **5.9** Mutual Exclusion Using Semaphores 249
- **5.12** An Incorrect Solution to the Infinite-Buffer Producer/Consumer Problem Using Binary Semaphores 252
- **5.13** A Correct Solution to the Infinite-Buffer Producer/Consumer Problem Using Binary Semaphores 254
- **5.14** A Solution to the Infinite-Buffer Producer/Consumer Problem Using Semaphores 255
- **5.16** A Solution to the Bounded-Buffer Producer/Consumer Problem Using Semaphores 256
- **5.17** Two Possible Implementations of Semaphores 257
- **5.19** A Solution to the Bounded-Buffer Producer/Consumer Problem Using a Monitor 260
- **5.20** Bounded-Buffer Monitor Code for Mesa Monitor 262
- **5.23** Mutual Exclusion Using Messages 268
- **5.24** A Solution to the Bounded-Buffer Producer/Consumer Problem Using Messages 269
- **5.25** A Solution to the Readers/Writers Problem Using Semaphore: Readers Have Priority 271
- **5.26** A Solution to the Readers/Writers Problem Using Semaphore: Writers Have Priority 273
- **5.27** A Solution to the Readers/Writers Problem Using Message Passing 274
- **5.28** An Application of Coroutines 277

**Chapter 6 Concurrency: Deadlock and Starvation 289**

- **6.9** Deadlock Avoidance Logic 305
- **6.12** A First Solution to the Dining Philosophers Problem 311
- **6.13** A Second Solution to the Dining Philosophers Problem 311
- **6.14** A Solution to the Dining Philosophers Problem Using a Monitor 312
- **6.18** Another Solution to the Dining Philosophers Problem Using a Monitor 337

**Chapter 13 Embedded Operating Systems 599**

- **13.12** Condition Variable Example Code 626

*This page intentionally left blank*

# PREFACE

## WHAT'S NEW IN THE NINTH EDITION

Since the eighth edition of this book was published, the field of operating systems has seen continuous innovations and improvements. In this new edition, I have tried to capture these changes while maintaining a comprehensive coverage of the entire field. To begin the process of revision, the eighth edition of this book was extensively reviewed by a number of professors who teach the subject and by professionals working in the field. The result is that, in many places, the narrative has been clarified and tightened, and illustrations have been improved.

Beyond these refinements to improve pedagogy and user friendliness, the technical content of the book has been updated throughout to reflect the ongoing changes in this exciting field, and the instructor and student support has been expanded. The most noteworthy changes are as follows:

- **Updated Linux coverage:** The Linux material has been updated and expanded to reflect changes in the Linux kernel since the eighth edition.
- **Updated Android coverage:** The Android material has been updated and expanded to reflect changes in the Android kernel since the eighth edition.
- **New Virtualization coverage:** The chapter on virtual machines has been completely rewritten to provide better organization and an expanded and more up-to-date treatment. In addition, a new section has been added on the use of containers.
- **New Cloud operating systems:** New to this edition is the coverage of cloud operating systems, including an overview of cloud computing, a discussion of the principles and requirements for a cloud operating system, and a discussion of a OpenStack, a popular open-source Cloud OS.
- **New IoT operating systems:** New to this edition is the coverage of operating systems for the Internet of Things. The coverage includes an overview of the IoT, a discussion of the principles and requirements for an IoT operating system, and a discussion of a RIOT, a popular open-source IoT OS.
- **Updated and Expanded Embedded operating systems:** This chapter has been substantially revised and expanded including:
  - The section on embedded systems has been expanded and now includes discussions of microcontrollers and deeply embedded systems.
  - The overview section on embedded OSs has been expanded and updated.
  - The treatment of embedded Linux has been expanded, and a new discussion of a popular embedded Linux system, $\mu$Clinux, has been added.
- **Concurrency:** New projects have been added to the Projects Manual to better help the student understand the principles of concurrency.

## OBJECTIVES

This book is about the concepts, structure, and mechanisms of operating systems. Its purpose is to present, as clearly and completely as possible, the nature and characteristics of modern-day operating systems.

This task is challenging for several reasons. First, there is a tremendous range and variety of computer systems for which operating systems are designed. These include embedded systems, smart phones, single-user workstations and personal computers, medium-sized shared systems, large mainframe and supercomputers, and specialized machines such as real-time systems. The variety is not just confined to the capacity and speed of machines, but in applications and system support requirements. Second, the rapid pace of change that has always characterized computer systems continues without respite. A number of key areas in operating system design are of recent origin, and research into these and other new areas continues.

In spite of this variety and pace of change, certain fundamental concepts apply consistently throughout. To be sure, the application of these concepts depends on the current state of technology and the particular application requirements. The intent of this book is to provide a thorough discussion of the fundamentals of operating system design, and to relate these to contemporary design issues and to current directions in the development of operating systems.

## EXAMPLE SYSTEMS

This text is intended to acquaint the reader with the design principles and implementation issues of contemporary operating systems. Accordingly, a purely conceptual or theoretical treatment would be inadequate. To illustrate the concepts and to tie them to real-world design choices that must be made, four operating systems have been chosen as running examples:

- **Windows:** A multitasking operating system for personal computers, workstations, servers, and mobile devices. This operating system incorporates many of the latest developments in operating system technology. In addition, Windows is one of the first important commercial operating systems to rely heavily on object-oriented design principles. This book covers the technology used in the most recent version of Windows, known as Windows 10.
- **Android:** Android is tailored for embedded devices, especially mobile phones. Focusing on the unique requirements of the embedded environment, the book provides details of Android internals.
- **UNIX:** A multiuser operating system, originally intended for minicomputers, but implemented on a wide range of machines from powerful microcomputers to supercomputers. Several flavors of UNIX are included as examples. FreeBSD is a widely used system that incorporates many state-of-the-art features. Solaris is a widely used commercial version of UNIX.
- **Linux:** An open-source version of UNIX that is widely used.

These systems were chosen because of their relevance and representativeness. The discussion of the example systems is distributed throughout the text rather than assembled as a single chapter or appendix. Thus, during the discussion of concurrency, the concurrency mechanisms of each example system are described, and the motivation for the individual design choices is discussed. With this approach, the design concepts discussed in a given chapter are immediately reinforced with real-world examples. For convenience, all of the material for each of the example systems is also available as an online document.

## SUPPORT OF ACM/IEEE COMPUTER SCIENCE CURRICULA 2013

The book is intended for both an academic and a professional audience. As a textbook, it is intended as a one-semester or two-semester undergraduate course in operating systems for computer science, computer engineering, and electrical engineering majors. This edition is designed to support the recommendations of the current (December 2013) version of the ACM/IEEE Computer Science Curricula 2013 (CS2013). The CS2013 curriculum recommendation includes Operating Systems (OS) as one of the Knowledge Areas in the Computer Science Body of Knowledge. CS2013 divides all course work into three categories: Core-Tier 1 (all topics should be included in the curriculum), Core-Tier 2 (all or almost all topics should be included), and Elective (desirable to provide breadth and depth). In the OS area, CS2013 includes two Tier 1 topics, four Tier 2 topics, and six Elective topics, each of which has a number of subtopics. This text covers all of the topics and subtopics listed by CS2013 in these three categories.

Table P.1 shows the support for the OS Knowledge Areas provided in this textbook. A detailed list of subtopics for each topic is available as the file CS2013-OS .pdf at box.com/OS9e.

## PLAN OF THE TEXT

The book is divided into six parts:

1. Background
2. Processes
3. Memory
4. Scheduling
5. Input/Output and files
6. Advanced topics (embedded OSs, virtual machines, OS security, and cloud and IoT operating systems)

The book includes a number of pedagogic features, including the use of animations and videonotes and numerous figures and tables to clarify the discussion. Each chapter includes a list of key words, review questions, and homework problems. The book also includes an extensive glossary, a list of frequently used acronyms, and a bibliography. In addition, a test bank is available to instructors.

**Table P.1** Coverage of CS2013 Operating Systems (OSs) Knowledge Area

| Topic | Coverage in Book |
|---|---|
| **Overview of Operating Systems (Tier 1)** | Chapter 2: Operating System Overview |
| **Operating System Principles (Tier 1)** | Chapter 1: Computer System Overview<br>Chapter 2: Operating System Overview |
| **Concurrency (Tier 2)** | Chapter 5: Mutual Exclusion and Synchronization<br>Chapter 6: Deadlock and Starvation<br>Appendix A: Topics in Concurrency<br>Chapter 18: Distributed Process Management |
| **Scheduling and Dispatch (Tier 2)** | Chapter 9: Uniprocessor Scheduling<br>Chapter 10: Multiprocessor and Real-Time Scheduling |
| **Memory Management (Tier 2)** | Chapter 7: Memory Management<br>Chapter 8: Virtual Memory |
| **Security and Protection (Tier 2)** | Chapter 15: Operating System Security |
| **Virtual Machines (Elective)** | Chapter 14: Virtual Machines |
| **Device Management (Elective)** | Chapter 11: I/O Management and Disk Scheduling |
| **File System (Elective)** | Chapter 12: File Management |
| **Real Time and Embedded Systems (Elective)** | Chapter 10: Multiprocessor and Real-Time Scheduling<br>Chapter 13: Embedded Operating Systems<br>Material on Android throughout the text |
| **Fault Tolerance (Elective)** | Section 2.5: Fault Tolerance |
| **System Performance Evaluation (Elective)** | Performance issues related to memory management, scheduling, and other areas addressed throughout the text |

## INSTRUCTOR SUPPORT MATERIALS

The major goal of this text is to make it as effective a teaching tool as possible for this fundamental yet evolving subject. This goal is reflected both in the structure of the book and in the supporting material. The text is accompanied by the following supplementary material to aid the instructor:

- **Solutions manual:** Solutions to end-of-chapter Review Questions and Problems.
- **Projects manual:** Suggested project assignments for all of the project categories listed in this Preface.
- **PowerPoint slides:** A set of slides covering all chapters, suitable for use in lecturing.
- **PDF files:** Reproductions of all figures and tables from the book.
- **Test bank:** A chapter-by-chapter set of questions with a separate file of answers.




- **VideoNotes on concurrency:** Professors perennially cite concurrency as perhaps the most difficult concept in the field of operating systems for students to grasp. The edition is accompanied by a number of VideoNotes lectures discussing the various concurrency algorithms defined in the book. This icon appears next to each algorithm definition in the book to indicate that a VideoNote is available:
- **Sample syllabuses:** The text contains more material that can be conveniently covered in one semester. Accordingly, instructors are provided with several sample syllabuses that guide the use of the text within limited time. These samples are based on real-world experience by professors with the seventh edition.

All of these support materials are available at the **Instructor Resource Center (IRC)** for this textbook, which can be reached through the publisher's website http://www.pearsonglobaleditions.com/stallings. To gain access to the IRC, please contact your local Pearson sales representative.

## PROJECTS AND OTHER STUDENT EXERCISES

For many instructors, an important component of an OS course is a project or set of projects by which the student gets hands-on experience to reinforce concepts from the text. This book has incorporated a projects component in the course as a result of an overwhelming support it received. In the online portion of the text, two major programming projects are defined. In addition, the instructor's support materials available through Pearson not only includes guidance on how to assign and structure the various projects, but also includes a set of user's manuals for various project types plus specific assignments, all written especially for this book. Instructors can assign work in the following areas:

- **OS/161 projects:** Described later.
- **Simulation projects:** Described later.
- **Semaphore projects:** Designed to help students understand concurrency concepts, including race conditions, starvation, and deadlock.
- **Kernel projects:** The IRC includes complete instructor support for two different sets of Linux kernel programming projects, as well as a set of kernel programming projects for Android.
- **Programming projects:** Described below.
- **Research projects:** A series of research assignments that instruct the student to research a particular topic on the Internet and write a report.
- **Reading/report assignments:** A list of papers that can be assigned for reading and writing a report, plus suggested assignment wording.
- **Writing assignments:** A list of writing assignments to facilitate learning the material.

- **Discussion topics:** These topics can be used in a classroom, chat room, or message board environment to explore certain areas in greater depth and to foster student collaboration.

In addition, information is provided on a software package known as BACI that serves as a framework for studying concurrency mechanisms.

This diverse set of projects and other student exercises enables the instructor to use the book as one component in a rich and varied learning experience and to tailor a course plan to meet the specific needs of the instructor and students. See Appendix B in this book for details.

## OS/161

This edition provides support for an active learning component based on OS/161. OS/161 is an educational operating system that is becoming increasingly recognized as the preferred teaching platform for OS internals. It aims to strike a balance between giving students experience in working on a real operating system, and potentially overwhelming students with the complexity that exists in a full-fledged operating system, such as Linux. Compared to most deployed operating systems, OS/161 is quite small (approximately 20,000 lines of code and comments), and therefore it is much easier to develop an understanding of the entire code base.

The IRC includes:

1. A packaged set of html files that the instructor can upload to a course server for student access.
2. A getting-started manual to be distributed to students to help them begin using OS/161.
3. A set of exercises using OS/161, to be distributed to students.
4. Model solutions to each exercise for the instructor's use.
5. All of this will be cross-referenced with appropriate sections in the book, so the student can read the textbook material then do the corresponding OS/161 project.

## SIMULATIONS

The IRC provides support for assigning projects based on a set of seven **simulations** that cover key areas of OS design. The student can use a set of simulation packages to analyze OS design features. The simulators are written in Java and can be run either locally as a Java application or online through a browser. The IRC includes specific assignments to give to students, telling them specifically what they are to do and what results are expected.

## ANIMATIONS

This edition also incorporates animations. Animations provide a powerful tool for understanding the complex mechanisms of a modern OS. A total of 53 animations are used to illustrate key functions and algorithms in OS design. The animations are used for Chapters 3, 5, 6, 7, 8, 9, and 11.

## PROGRAMMING PROJECTS

This edition provides support for programming projects. Two major programming projects, one to build a shell, or command line interpreter, and one to build a process dispatcher are described in the online portion of this textbook. The IRC provides further information and step-by-step exercises for developing the programs.

As an alternative, the instructor can assign a more extensive series of projects that cover many of the principles in the book. The student is provided with detailed instructions for doing each of the projects. In addition, there is a set of homework problems, which involve questions related to each project for the student to answer.

Finally, the project manual provided at the IRC includes a series of programming projects that cover a broad range of topics and that can be implemented in any suitable language on any platform.

## ONLINE DOCUMENTS AND VIDEONOTES FOR STUDENTS

For this new edition, a substantial amount of original supporting material for students has been made available online, at two online locations. The **book's website**, at http://www.pearsonglobaleditions.com/stallings (click on *Student Resources* link), includes a list of relevant links organized by chapter and an errata sheet for the book.

Purchasing this textbook new also grants the reader twelve months of access to the **Companion Website**, which includes the following materials:

- **Online chapters:** To limit the size and cost of the book, 5 chapters of the book, covering security, are provided in PDF format. The chapters are listed in this book's table of contents.
- **Online appendices:** There are numerous interesting topics that support material found in the text, but whose inclusion is not warranted in the printed text. A total of 15 online appendices cover these topics for the interested student. The appendices are listed in this book's table of contents.
- **Homework problems and solutions:** To aid the student in understanding the material, a separate set of homework problems with solutions is available.

- **Animations:** Animations provide a powerful tool for understanding the complex mechanisms of a modern OS. A total of 53 animations are used to illustrate key functions and algorithms in OS design. The animations are used for Chapters 3, 5, 6, 7, 8, 9, and 11.
- **VideoNotes:** VideoNotes are step-by-step video tutorials specifically designed to enhance the programming concepts presented in this textbook. The book is accompanied by a number of VideoNotes lectures discussing the various concurrency algorithms defined in the book.

To access the Premium Content site, click on the Companion website link at www.pearsonglobaleditions.com/stallings and enter the student access code found on the card in the front of the book.

## ACKNOWLEDGMENTS

I would like to thank the following for their contributions. Rami Rosen contributed most of the new material on Linux. Vineet Chadha made a major contribution to the new chapter on virtual machines. Durgadoss Ramanathan provided the new material on Android ART.

Through its multiple editions this book has benefited from review by hundreds of instructors and professionals, who generously spared their precious time and shared their expertise. Here I acknowledge those whose help contributed to this latest edition.

The following instructors reviewed all or a large part of the manuscript for this edition: Jiang Guo (California State University, Los Angeles), Euripides Montagne (University of Central Florida), Kihong Park (Purdue University), Mohammad Abdus Salam (Southern University and A&M College), Robert Marmorstein (Longwood University), Christopher Diaz (Seton Hill University), and Barbara Bracken (Wilkes University).

Thanks also to all those who provided detailed technical reviews of one or more chapters: Nischay Anikar, Adri Jovin, Ron Munitz, Fatih Eyup Nar, Atte Peltomaki, Durgadoss Ramanathan, Carlos Villavieja, Wei Wang, Serban Constantinescu and Chen Yang.

Thanks also to those who provided detailed reviews of the example systems. Reviews of the Android material were provided by Kristopher Micinski, Ron Munitz, Atte Peltomaki, Durgadoss Ramanathan, Manish Shakya, Samuel Simon, Wei Wang, and Chen Yang. The Linux reviewers were Tigran Aivazian, Kaiwan Billimoria, Peter Huewe, Manmohan Manoharan, Rami Rosen, Neha Naik, and Hualing Yu. The Windows material was reviewed by Francisco Cotrina, Sam Haidar, Christopher Kuleci, Benny Olsson, and Dave Probert. The RIOT material was reviewed by Emmanuel Baccelli and Kaspar Schleiser, and OpenStack was reviewed by Bob Callaway. Nick Garnett of eCosCentric reviewed the material on eCos; and Philip Levis, one of the developers of TinyOS reviewed the material on TinyOS. Sid Young reviewed the material on container virtualization.

Andrew Peterson of the University of Toronto prepared the OS/161 supplements for the IRC. James Craig Burley authored and recorded the VideoNotes.

Adam Critchley (University of Texas at San Antonio) developed the simulation exercises. Matt Sparks (University of Illinois at Urbana-Champaign) adapted a set of programming problems for use with this textbook.

Lawrie Brown of the Australian Defence Force Academy produced the material on buffer overflow attacks. Ching-Kuang Shene (Michigan Tech University) provided the examples used in the section on race conditions and reviewed the section. Tracy Camp and Keith Hellman, both at the Colorado School of Mines, developed a new set of homework problems. In addition, Fernando Ariel Gont contributed a number of homework problems; he also provided detailed reviews of all of the chapters.

I would also like to thank Bill Bynum (College of William and Mary) and Tracy Camp (Colorado School of Mines) for contributing Appendix O; Steve Taylor (Worcester Polytechnic Institute) for contributing the programming projects and reading/report assignments in the instructor's manual; and Professor Tan N. Nguyen (George Mason University) for contributing the research projects in the instruction manual. Ian G. Graham (Griffith University) contributed the two programming projects in the textbook. Oskars Rieksts (Kutztown University) generously allowed me to make use of his lecture notes, quizzes, and projects.

Finally, I thank the many people responsible for the publication of this book, all of whom did their usual excellent job. This includes the staff at Pearson, particularly my editor Tracy Johnson, her assistant Kristy Alaura, program manager Carole Snyder, and project manager Bob Engelhardt. Thanks also to the marketing and sales staffs at Pearson, without whose efforts this book would not be in front of you.

## ACKNOWLEDGMENTS FOR THE GLOBAL EDITION

Pearson would like to thank and acknowledge Moumita Mitra Manna (Bangabasi College) for contributing to the Global Edition, and A. Kannamal (Coimbatore Institute of Technology), Kumar Shashi Prabh (Shiv Nadar University), and Khyat Sharma for reviewing the Global Edition.

*This page intentionally left blank*

# About the Author

**Dr. William Stallings** has authored 18 titles, and including the revised editions, over 40 books on computer security, computer networking, and computer architecture. His writings have appeared in numerous publications, including the *Proceedings of the IEEE, ACM Computing Reviews* and *Cryptologia*.

He has received the Best Computer Science textbook of the Year award 13 times from the Text and Academic Authors Association.

In over 30 years in the field, he has been a technical contributor, technical manager, and an executive with several high-technology firms. He has designed and implemented both TCP/IP-based and OSI-based protocol suites on a variety of computers and operating systems, ranging from microcomputers to mainframes. As a consultant, he has advised government agencies, computer and software vendors, and major users on the design, selection, and use of networking software and products.

He created and maintains the *Computer Science Student Resource Site* at ComputerScienceStudent.com. This site provides documents and links on a variety of subjects of general interest to computer science students (and professionals). He is a member of the editorial board of *Cryptologia*, a scholarly journal devoted to all aspects of cryptology.

Dr. Stallings holds a Ph.D. from M.I.T. in Computer Science and a B.S. from Notre Dame in electrical engineering.

*This page intentionally left blank*

# PART 1 Background

## CHAPTER 1

# COMPUTER SYSTEM OVERVIEW


1.1 **Basic Elements**

1.2 **Evolution of the Microprocessor**

1.3 **Instruction Execution**

1.4 **Interrupts**
- Interrupts and the Instruction Cycle
- Interrupt Processing
- Multiple Interrupts

1.5 **The Memory Hierarchy**

1.6 **Cache Memory**
- Motivation
- Cache Principles
- Cache Design

1.7 **Direct Memory Access**

1.8 **Multiprocessor and Multicore Organization**
- Symmetric Multiprocessors
- Multicore Computers

1.9 **Key Terms, Review Questions, and Problems**

APPENDIX 1A **Performance Characteristics of Two-Level Memories**
- Locality
- Operation of Two-Level Memory
- Performance

### Learning Objectives

After studying this chapter, you should be able to:

- Describe the basic elements of a computer system and their interrelationship.
- Explain the steps taken by a processor to execute an instruction.
- Understand the concept of interrupts, and how and why a processor uses interrupts.
- List and describe the levels of a typical computer memory hierarchy.
- Explain the basic characteristics of multiprocessor systems and multicore computers.
- Discuss the concept of locality and analyze the performance of a multilevel memory hierarchy.
- Understand the operation of a stack and its use to support procedure call and return.

An operating system (OS) exploits the hardware resources of one or more processors to provide a set of services to system users. The OS also manages secondary memory and I/O (input/output) devices on behalf of its users. Accordingly, it is important to have some understanding of the underlying computer system hardware before we begin our examination of operating systems.

This chapter provides an overview of computer system hardware. In most areas, the survey is brief, as it is assumed that the reader is familiar with this subject. However, several areas are covered in some detail because of their importance to topics covered later in the book. Additional topics are covered in Appendix C. For a more detailed treatment, see [STAL16a].

## 1.1 BASIC ELEMENTS

At a top level, a computer consists of processor, memory, and I/O components, with one or more modules of each type. These components are interconnected in some fashion to achieve the main function of the computer, which is to execute programs. Thus, there are four main structural elements:

- **Processor:** Controls the operation of the computer and performs its data processing functions. When there is only one processor, it is often referred to as the **central processing unit** (CPU).
- **Main memory:** Stores data and programs. This memory is typically volatile; that is, when the computer is shut down, the contents of the memory are lost. In contrast, the contents of disk memory are retained even when the computer system is shut down. Main memory is also referred to as *real memory* or *primary memory*.

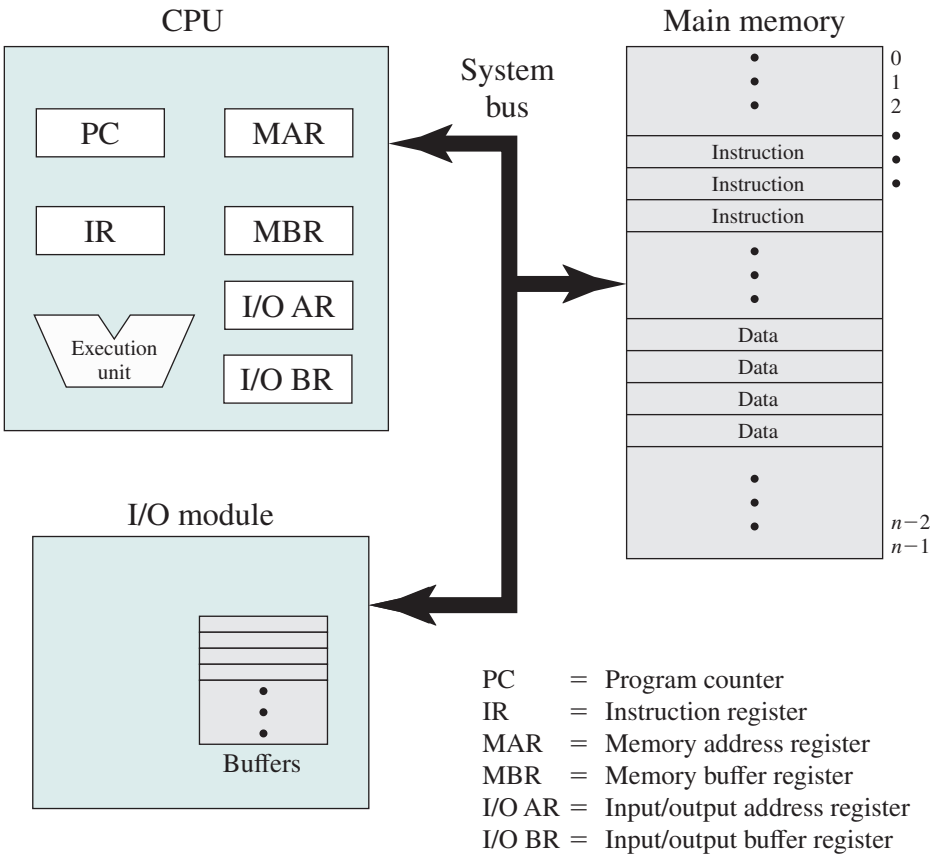



**Figure 1.1** **Computer Components: Top-Level View**

- **I/O modules:** Move data between the computer and its external environment. The external environment consists of a variety of devices, including secondary memory devices (e.g., disks), communications equipment, and terminals.
- **System bus:** Provides for communication among processors, main memory, and I/O modules.

Figure 1.1 depicts these top-level components. One of the processor's functions is to exchange data with memory. For this purpose, it typically makes use of two internal (to the processor) registers: a memory address register (MAR), which specifies the address in memory for the next read or write; and a memory buffer register (MBR), which contains the data to be written into memory, or receives the data read from memory. Similarly, an I/O address register (I/OAR) specifies a particular I/O device. An I/O buffer register (I/OBR) is used for the exchange of data between an I/O module and the processor.

A memory module consists of a set of locations, defined by sequentially numbered addresses. Each location contains a bit pattern that can be interpreted as either

an instruction or data. An I/O module transfers data from external devices to processor and memory, and vice versa. It contains internal buffers for temporarily storing data until they can be sent on.

## 1.2 EVOLUTION OF THE MICROPROCESSOR

The hardware revolution that brought about desktop and handheld computing was the invention of the microprocessor, which contained a processor on a single chip. Though originally much slower than multichip processors, microprocessors have continually evolved to the point that they are now much faster for most computations due to the physics involved in moving information around in sub-nanosecond timeframes.

Not only have microprocessors become the fastest general-purpose processors available, they are now multiprocessors; each chip (called a socket) contains multiple processors (called cores), each with multiple levels of large memory caches, and multiple logical processors sharing the execution units of each core. As of 2010, it is not unusual for even a laptop to have 2 or 4 cores, each with 2 hardware threads, for a total of 4 or 8 logical processors.

Although processors provide very good performance for most forms of computing, there is increasing demand for numerical computation. Graphical Processing Units (GPUs) provide efficient computation on arrays of data using Single-Instruction Multiple Data (SIMD) techniques pioneered in supercomputers. GPUs are no longer used just for rendering advanced graphics, but they are also used for general numerical processing, such as physics simulations for games or computations on large spreadsheets. Simultaneously, the CPUs themselves are gaining the capability of operating on arrays of data–with increasingly powerful vector units integrated into the processor architecture of the x86 and AMD64 families.

Processors and GPUs are not the end of the computational story for the modern PC. Digital Signal Processors (DSPs) are also present for dealing with streaming signals such as audio or video. DSPs used to be embedded in I/O devices, like modems, but they are now becoming first-class computational devices, especially in handhelds. Other specialized computational devices (fixed function units) co-exist with the CPU to support other standard computations, such as encoding/decoding speech and video (codecs), or providing support for encryption and security.

To satisfy the requirements of handheld devices, the classic microprocessor is giving way to the System on a Chip (SoC), where not just the CPUs and caches are on the same chip, but also many of the other components of the system, such as DSPs, GPUs, I/O devices (such as radios and codecs), and main memory.

## 1.3 INSTRUCTION EXECUTION

A program to be executed by a processor consists of a set of instructions stored in memory. In its simplest form, instruction processing consists of two steps: The processor reads (*fetches*) instructions from memory one at a time and executes each instruction. Program execution consists of repeating the process of instruction fetch

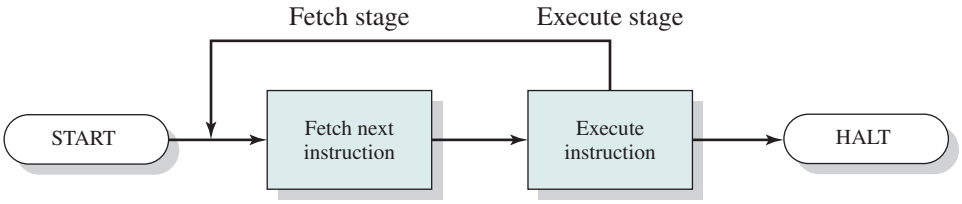



**Figure 1.2 Basic Instruction Cycle**

and instruction execution. Instruction execution may involve several operations and depends on the nature of the instruction.

The processing required for a single instruction is called an *instruction cycle.* Using a simplified two-step description, the instruction cycle is depicted in Figure 1.2. The two steps are referred to as the *fetch stage* and the *execute stage.* Program execution halts only if the processor is turned off, some sort of unrecoverable error occurs, or a program instruction that halts the processor is encountered.

At the beginning of each instruction cycle, the processor fetches an instruction from memory. Typically, the program counter (PC) holds the address of the next instruction to be fetched. Unless instructed otherwise, the processor always increments the PC after each instruction fetch so it will fetch the next instruction in sequence (i.e., the instruction located at the next higher memory address). For example, consider a simplified computer in which each instruction occupies one 16-bit word of memory. Assume that the program counter is set to location 300. The processor will next fetch the instruction at location 300. On succeeding instruction cycles, it will fetch instructions from locations 301, 302, 303, and so on. This sequence may be altered, as explained subsequently.

The fetched instruction is loaded into the instruction register (IR). The instruction contains bits that specify the action the processor is to take. The processor interprets the instruction and performs the required action. In general, these actions fall into four categories:

- **Processor-memory:** Data may be transferred from processor to memory, or from memory to processor.
- **Processor-I/O:** Data may be transferred to or from a peripheral device by transferring between the processor and an I/O module.
- **Data processing:** The processor may perform some arithmetic or logic operation on data.
- **Control:** An instruction may specify that the sequence of execution be altered. For example, the processor may fetch an instruction from location 149, which specifies that the next instruction be from location 182. The processor sets the program counter to 182. Thus, on the next fetch stage, the instruction will be fetched from location 182 rather than 150.

An instruction's execution may involve a combination of these actions.

Consider a simple example using a hypothetical processor that includes the characteristics listed in Figure 1.3. The processor contains a single data register, called




(a) Instruction format




(b) Integer format

Program counter (PC) = Address of instruction
Instruction register (IR) = Instruction being executed
Accumulator (AC) = Temporary storage

(c) Internal CPU registers

0001 = Load AC from memory
0010 = Store AC to memory
0101 = Add to AC from memory

(d) Partial list of opcodes

**Figure 1.3 Characteristics of a Hypothetical Machine**

the accumulator (AC). Both instructions and data are 16 bits long, and memory is organized as a sequence of 16-bit words. The instruction format provides 4 bits for the opcode, allowing as many as $2^4 = 16$ different opcodes (represented by a single hexadecimal[1] digit). The opcode defines the operation the processor is to perform. With the remaining 12 bits of the instruction format, up to $2^{12} = 4{,}096$ (4K) words of memory (denoted by three hexadecimal digits) can be directly addressed.

Figure 1.4 illustrates a partial program execution, showing the relevant portions of memory and processor registers. The program fragment shown adds the contents of the memory word at address 940 to the contents of the memory word at address 941 and stores the result in the latter location. Three instructions, which can be described as three fetch and three execute stages, are required:

1. The PC contains 300, the address of the first instruction. This instruction (the value 1940 in hexadecimal) is loaded into the IR and the PC is incremented. Note that this process involves the use of a memory address register (MAR) and a memory buffer register (MBR). For simplicity, these intermediate registers are not shown.
2. The first 4 bits (first hexadecimal digit) in the IR indicate that the AC is to be loaded from memory. The remaining 12 bits (three hexadecimal digits) specify the address, which is 940.
3. The next instruction (5941) is fetched from location 301 and the PC is incremented.

[1]A basic refresher on number systems (decimal, binary, hexadecimal) can be found at the Computer Science Student Resource Site at ComputerScienceStudent.com.

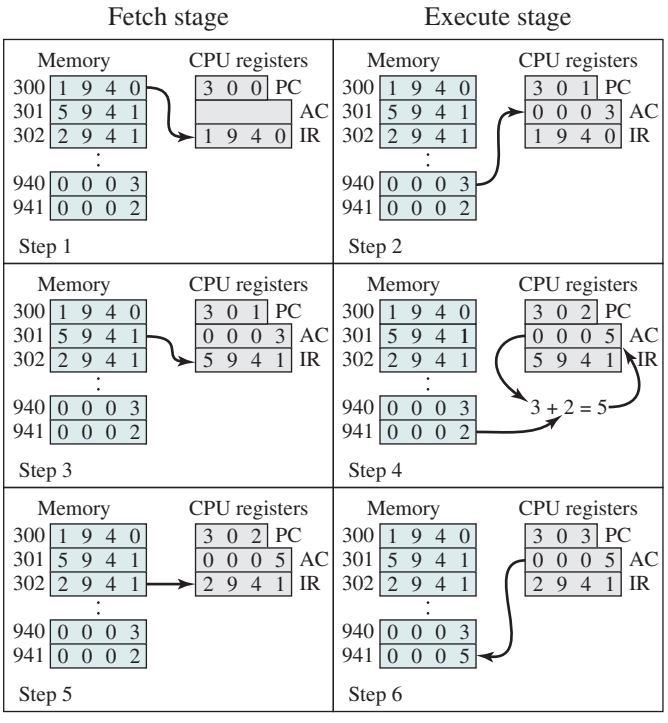



**Figure 1.4** **Example of Program Execution (contents of memory and registers in hexadecimal)**

4. The old contents of the AC and the contents of location 941 are added, and the result is stored in the AC.
5. The next instruction (2941) is fetched from location 302, and the PC is incremented.
6. The contents of the AC are stored in location 941.

In this example, three instruction cycles, each consisting of a fetch stage and an execute stage, are needed to add the contents of location 940 to the contents of 941. With a more complex set of instructions, fewer instruction cycles would be needed. Most modern processors include instructions that contain more than one address. Thus, the execution stage for a particular instruction may involve more than one reference to memory. Also, instead of memory references, an instruction may specify an I/O operation.

## 1.4 INTERRUPTS

Virtually all computers provide a mechanism by which other modules (I/O, memory) may interrupt the normal sequencing of the processor. Table 1.1 lists the most common classes of interrupts.

**Table 1.1** Classes of Interrupts

| | |
|---|---|
| **Program** | Generated by some condition that occurs as a result of an instruction execution, such as arithmetic overflow, division by zero, attempt to execute an illegal machine instruction, or reference outside a user's allowed memory space. |
| **Timer** | Generated by a timer within the processor. This allows the operating system to perform certain functions on a regular basis. |
| **I/O** | Generated by an I/O controller, to signal normal completion of an operation or to signal a variety of error conditions. |
| **Hardware failure** | Generated by a failure, such as power failure or memory parity error. |

Interrupts are provided primarily as a way to improve processor utilization. For example, most I/O devices are much slower than the processor. Suppose that the processor is transferring data to a printer using the instruction cycle scheme of Figure 1.2. After each write operation, the processor must pause and remain idle until the printer catches up. The length of this pause may be on the order of many thousands or even millions of instruction cycles. Clearly, this is a very wasteful use of the processor.

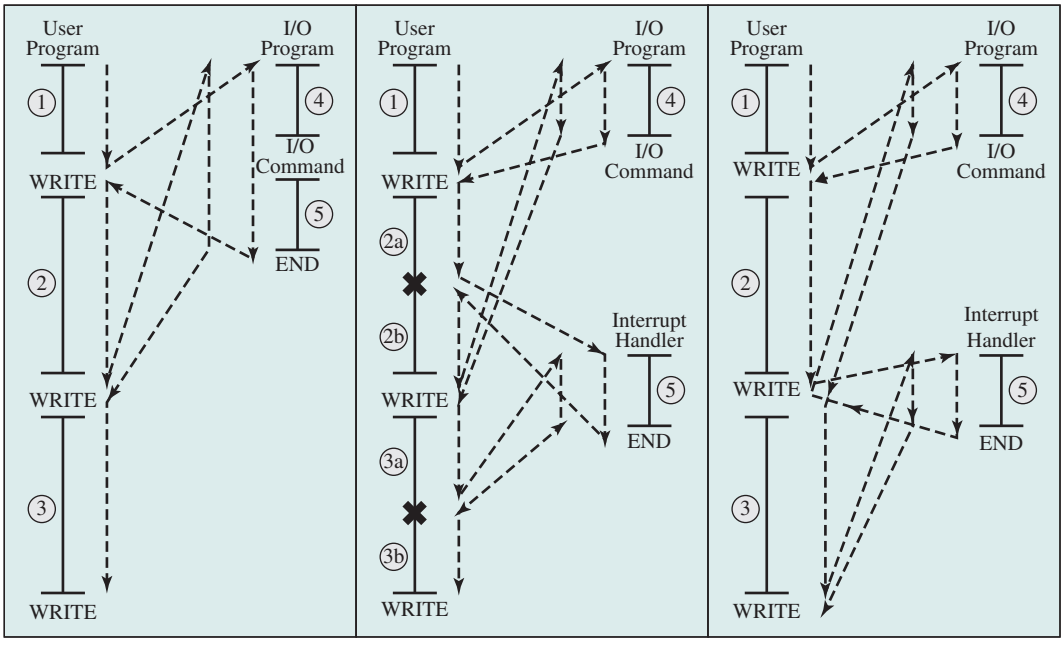
To give a specific example, consider a PC that operates at 1 GHz, which would allow roughly $10^9$ instructions per second.[2] A typical hard disk has a rotational speed of 7200 revolutions per minute for a half-track rotation time of 4 ms, which is 4 million times slower than the processor.

Figure 1.5a illustrates this state of affairs. The user program performs a series of WRITE calls interleaved with processing. The solid vertical lines represent segments of code in a program. Code segments 1, 2, and 3 refer to sequences of instructions that do not involve I/O. The WRITE calls are to an I/O routine that is a system utility and will perform the actual I/O operation. The I/O program consists of three sections:

- A sequence of instructions, labeled 4 in the figure, to prepare for the actual I/O operation. This may include copying the data to be output into a special buffer and preparing the parameters for a device command.
- The actual I/O command. Without the use of interrupts, once this command is issued, the program must wait for the I/O device to perform the requested function (or periodically check the status of, or poll, the I/O device). The program might wait by simply repeatedly performing a test operation to determine if the I/O operation is done.
- A sequence of instructions, labeled 5 in the figure, to complete the operation. This may include setting a flag indicating the success or failure of the operation.

The dashed line represents the path of execution followed by the processor; that is, this line shows the sequence in which instructions are executed. Thus, after the first

[2] A discussion of the uses of numerical prefixes, such as giga and tera, is contained in a supporting document at the Computer Science Student Resource Site at ComputerScienceStudent.com.




(a) No interrupts (b) Interrupts; short I/O wait (c) Interrupts; long I/O wait

✖ = interrupt occurs during course of execution of user program

**Figure 1.5 Program Flow of Control Without and With Interrupts**

WRITE instruction is encountered, the user program is interrupted and execution continues with the I/O program. After the I/O program execution is complete, execution resumes in the user program immediately following the WRITE instruction.

Because the I/O operation may take a relatively long time to complete, the I/O program is hung up waiting for the operation to complete; hence, the user program is stopped at the point of the WRITE call for some considerable period of time.

## Interrupts and the Instruction Cycle

With interrupts, the processor can be engaged in executing other instructions while an I/O operation is in progress. Consider the flow of control in Figure 1.5b. As before, the user program reaches a point at which it makes a system call in the form of a WRITE call. The I/O program that is invoked in this case consists only of the preparation code and the actual I/O command. After these few instructions have been executed, control returns to the user program. Meanwhile, the external device is busy accepting data from computer memory and printing it. This I/O operation is conducted concurrently with the execution of instructions in the user program.

When the external device becomes ready to be serviced (that is, when it is ready to accept more data from the processor) the I/O module for that external device sends an *interrupt request* signal to the processor. The processor responds by suspending operation of the current program; branching off to a routine to service

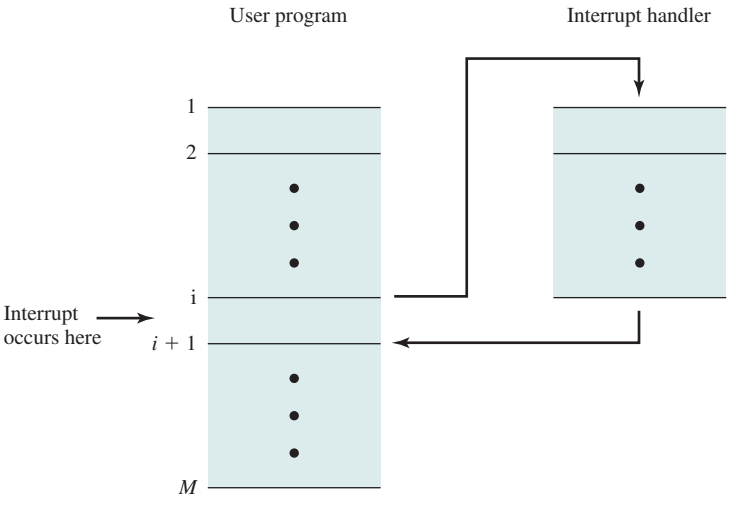



**Figure 1.6** **Transfer of Control via Interrupts**

that particular I/O device (known as an interrupt handler); and resuming the original execution after the device is serviced. The points at which such interrupts occur are indicated by ✖ in Figure 1.5b. Note that an interrupt can occur at any point in the main program, not just at one specific instruction.

For the user program, an interrupt suspends the normal sequence of execution. When the interrupt processing is completed, execution resumes (see Figure 1.6). Thus, the user program does not have to contain any special code to accommodate interrupts; the processor and the OS are responsible for suspending the user program, then resuming it at the same point.

To accommodate interrupts, an *interrupt stage* is added to the instruction cycle, as shown in Figure 1.7 (compare with Figure 1.2). In the interrupt stage, the processor checks to see if any interrupts have occurred, indicated by the presence of an

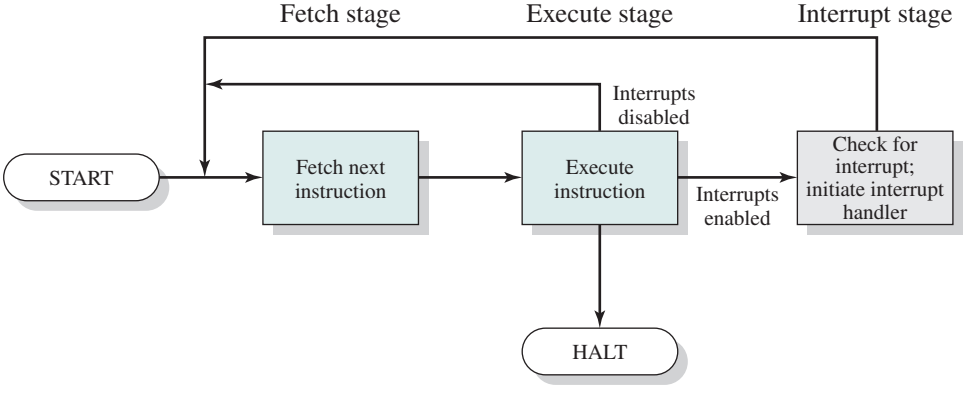



**Figure 1.7** **Instruction Cycle with Interrupts**

interrupt signal. If no interrupts are pending, the processor proceeds to the fetch stage and fetches the next instruction of the current program. If an interrupt is pending, the processor suspends execution of the current program and executes an *interrupt-handler* routine. The interrupt-handler routine is generally part of the OS. Typically, this routine determines the nature of the interrupt and performs whatever actions are needed. In the example we have been using, the handler determines which I/O module generated the interrupt, and may branch to a program that will write more data out to that I/O module. When the interrupt-handler routine is completed, the processor can resume execution of the user program at the point of interruption.

It is clear that there is some overhead involved in this process. Extra instructions must be executed (in the interrupt handler) to determine the nature of the interrupt and to decide on the appropriate action. Nevertheless, because of the relatively large amount of time that would be wasted by simply waiting on an I/O operation, the processor can be employed much more efficiently with the use of interrupts.

To appreciate the gain in efficiency, consider Figure 1.8, which is a timing diagram based on the flow of control in Figures 1.5a and 1.5b. Figures 1.5b and 1.8 assume

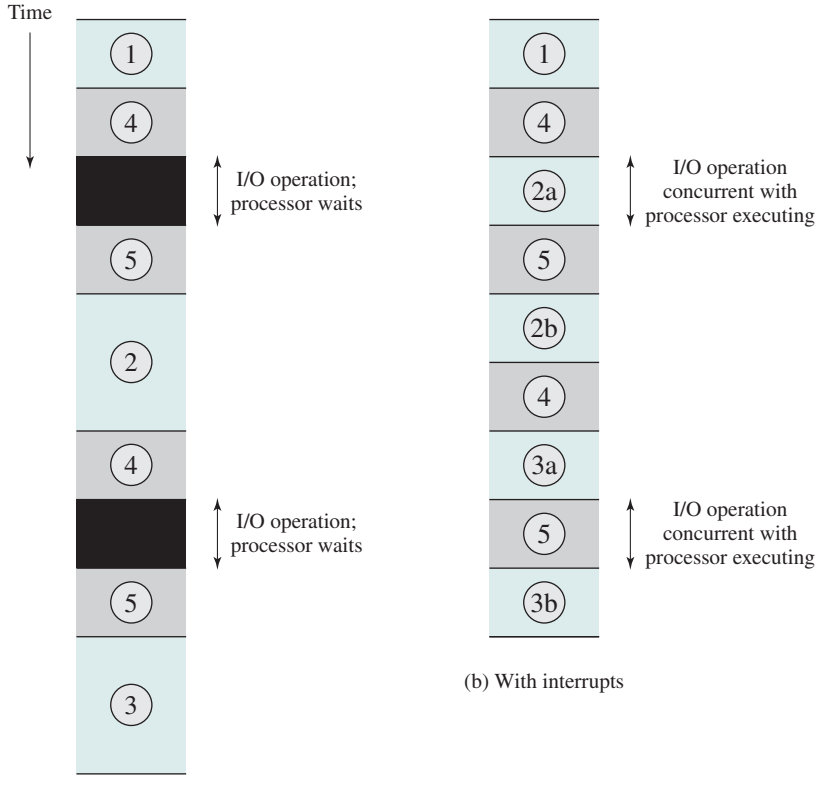



**Figure 1.8** **Program Timing: Short I/O Wait**

that the time required for the I/O operation is relatively short: less than the time to complete the execution of instructions between write operations in the user program. The more typical case, especially for a slow device such as a printer, is that the I/O operation will take much more time than executing a sequence of user instructions. Figure 1.5c indicates this state of affairs. In this case, the user program reaches the second WRITE call before the I/O operation spawned by the first call is complete. The result is that the user program is hung up at that point. When the preceding I/O operation is completed, this new WRITE call may be processed, and a new I/O operation may be started. Figure 1.9 shows the timing for this situation with and without the use of interrupts. We can see there is still a gain in efficiency, because part of the time during which the I/O operation is underway overlaps with the execution of user instructions.

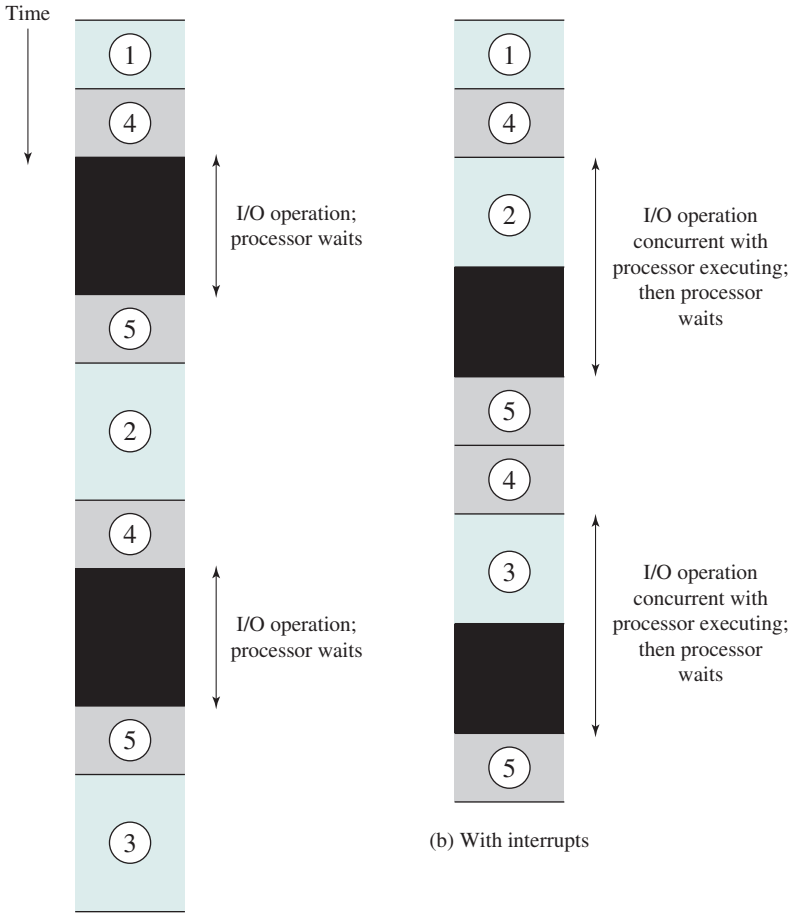



**Figure 1.9** **Program Timing: Long I/O Wait**

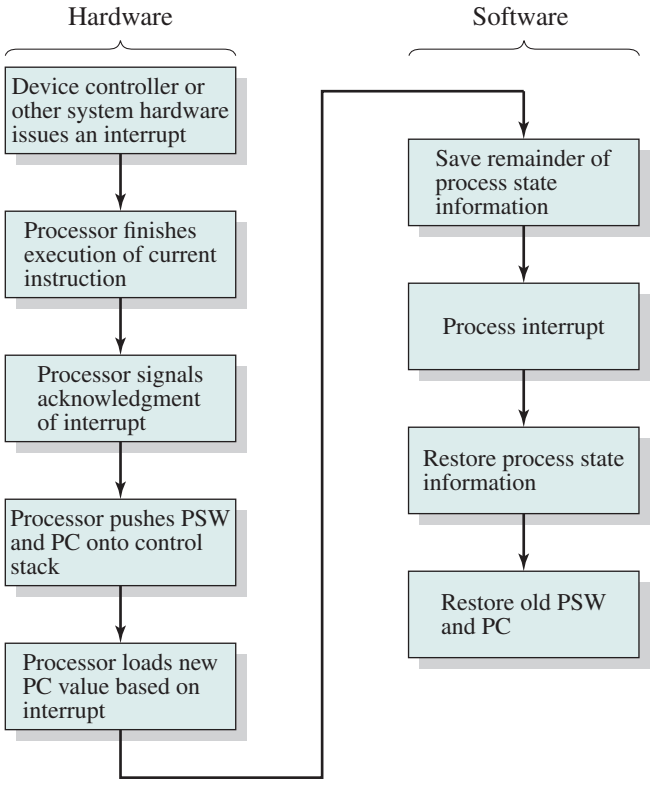



**Figure 1.10** **Simple Interrupt Processing**

## Interrupt Processing

An interrupt triggers a number of events, both in the processor hardware and in software. Figure 1.10 shows a typical sequence. When an I/O device completes an I/O operation, the following sequence of hardware events occurs:

1. The device issues an interrupt signal to the processor.
2. The processor finishes execution of the current instruction before responding to the interrupt, as indicated in Figure 1.7.
3. The processor tests for a pending interrupt request, determines there is one, and sends an acknowledgment signal to the device that issued the interrupt. The acknowledgment allows the device to remove its interrupt signal.
4. The processor next needs to prepare to transfer control to the interrupt routine. To begin, it saves information needed to resume the current program at the point of interrupt. The minimum information required is the program status word[3] (PSW) and the location of the next instruction to be executed, which is

[3]The PSW contains status information about the currently running process, including memory usage information, condition codes, and other status information such as an interrupt enable/disable bit and a kernel/user-mode bit. See Appendix C for further discussion.

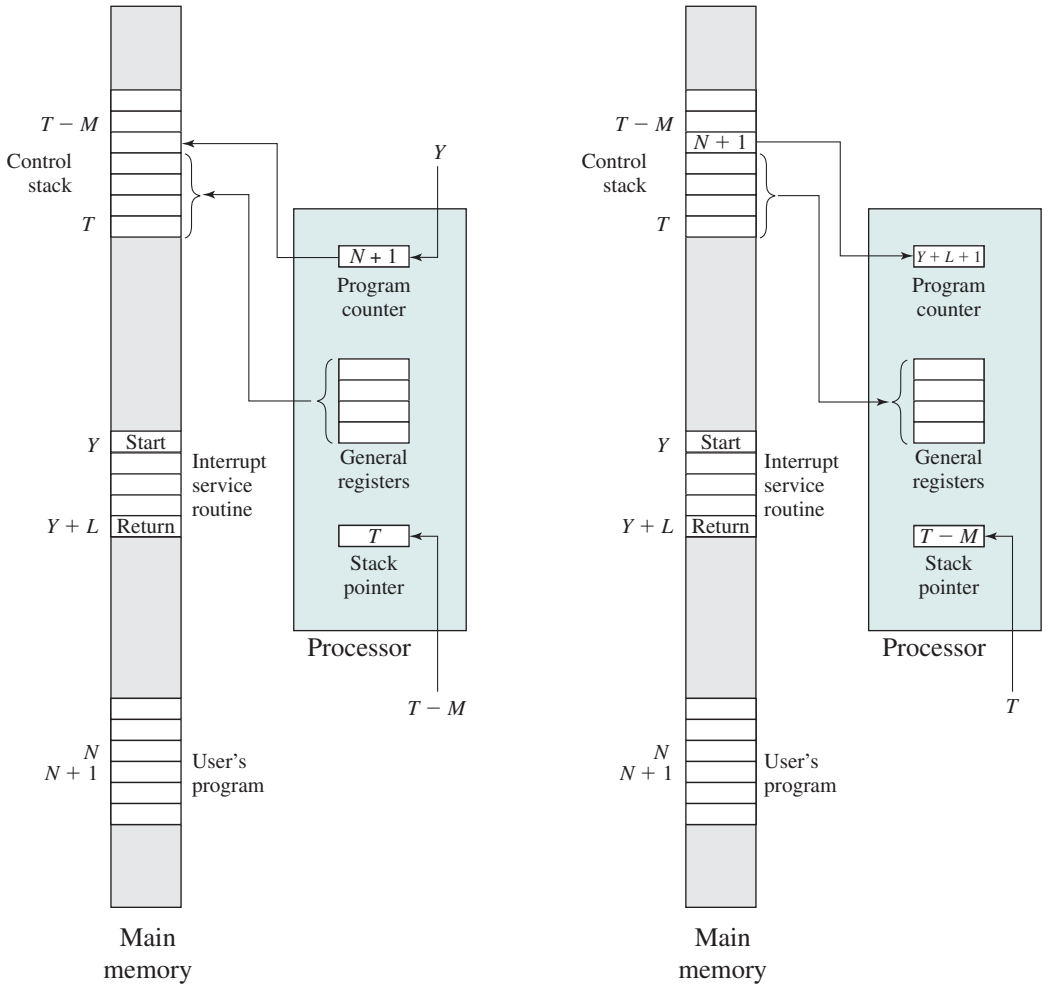
contained in the program counter (PC). These can be pushed onto a control stack (see Appendix P).

5. The processor then loads the program counter with the entry location of the interrupt-handling routine that will respond to this interrupt. Depending on the computer architecture and OS design, there may be a single program, one for each type of interrupt, or one for each device and each type of interrupt. If there is more than one interrupt-handling routine, the processor must determine which one to invoke. This information may have been included in the original interrupt signal, or the processor may have to issue a request to the device that issued the interrupt to get a response that contains the needed information.

Once the program counter has been loaded, the processor proceeds to the next instruction cycle, which begins with an instruction fetch. Because the instruction fetch is determined by the contents of the program counter, control is transferred to the interrupt-handler program. The execution of this program results in the following operations:

6. At this point, the program counter and PSW relating to the interrupted program have been saved on the control stack. However, there is other information that is considered part of the state of the executing program. In particular, the contents of the processor registers need to be saved, because these registers may be used by the interrupt handler. So all of these values, plus any other state information, need to be saved. Typically, the interrupt handler will begin by saving the contents of all registers on the stack. Other state information that must be saved will be discussed in Chapter 3. Figure 1.11a shows a simple example. In this case, a user program is interrupted after the instruction at location *N*. The contents of all of the registers plus the address of the next instruction ($N + 1$), a total of $M$ words, are pushed onto the control stack. The stack pointer is updated to point to the new top of stack, and the program counter is updated to point to the beginning of the interrupt service routine.
7. The interrupt handler may now proceed to process the interrupt. This includes an examination of status information relating to the I/O operation or other event that caused an interrupt. It may also involve sending additional commands or acknowledgments to the I/O device.
8. When interrupt processing is complete, the saved register values are retrieved from the stack and restored to the registers (see Figure 1.11b).
9. The final act is to restore the PSW and program counter values from the stack. As a result, the next instruction to be executed will be from the previously interrupted program.

It is important to save all of the state information about the interrupted program for later resumption. This is because the interrupt is not a routine called from the program. Rather, the interrupt can occur at any time, and therefore at any point in the execution of a user program. Its occurrence is unpredictable.




(a) Interrupt occurs after instruction at location $N$

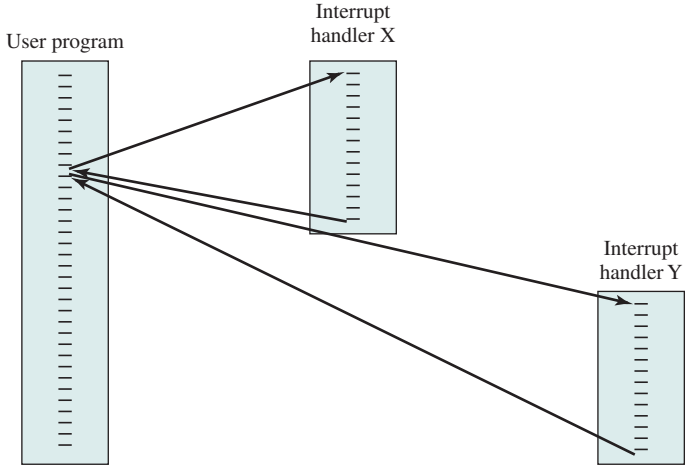
(b) Return from interrupt

**Figure 1.11 Changes in Memory and Registers for an Interrupt**

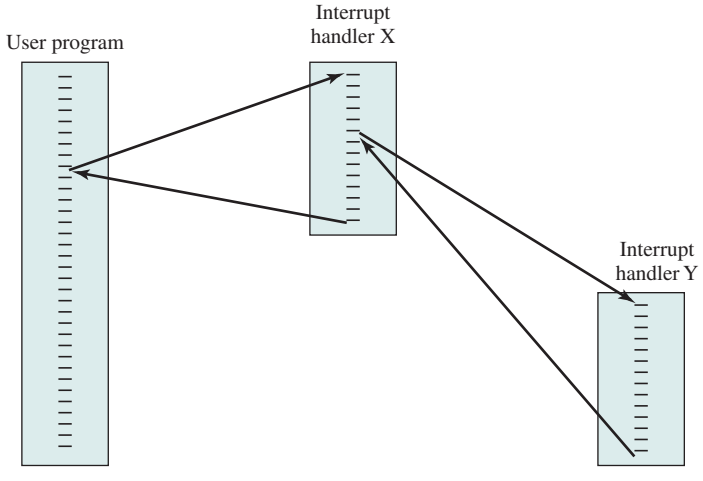
## Multiple Interrupts

So far, we have discussed the occurrence of a single interrupt. Suppose, however, that one or more interrupts can occur while an interrupt is being processed. For example, a program may be receiving data from a communications line, and printing results at the same time. The printer will generate an interrupt every time it completes a print operation. The communication line controller will generate an interrupt every time a unit of data arrives. The unit could either be a single character or a block, depending on the nature of the communications discipline. In any case, it is possible for a communications interrupt to occur while a printer interrupt is being processed.

Two approaches can be taken to dealing with multiple interrupts. The first is to disable interrupts while an interrupt is being processed. A *disabled interrupt* simply means the processor ignores any new interrupt request signal. If an interrupt occurs during this time, it generally remains pending and will be checked by the processor after the processor has reenabled interrupts. Thus, if an interrupt occurs when a user program is executing, then interrupts are disabled immediately. After the interrupt-handler routine completes, interrupts are reenabled before resuming the user program, and the processor checks to see if additional interrupts have occurred. This approach is simple, as interrupts are handled in strict sequential order (see Figure 1.12a).




(a) Sequential interrupt processing




(b) Nested interrupt processing

**Figure 1.12 Transfer of Control with Multiple Interrupts**

The drawback to the preceding approach is that it does not take into account relative priority or time-critical needs. For example, when input arrives from the communications line, it may need to be absorbed rapidly to make room for more input. If the first batch of input has not been processed before the second batch arrives, data may be lost because the buffer on the I/O device may fill and overflow.

A second approach is to define priorities for interrupts and to allow an interrupt of higher priority to cause a lower-priority interrupt handler to be interrupted (see Figure 1.12b). As an example of this second approach, consider a system with three I/O devices: a printer, a disk, and a communications line, with increasing priorities of 2, 4, and 5, respectively. Figure 1.13 illustrates a possible sequence. A user program begins at $t = 0$. At $t = 10$, a printer interrupt occurs; user information is placed on the control stack and execution continues at the printer interrupt service routine (ISR). While this routine is still executing, at $t = 15$ a communications interrupt occurs. Because the communications line has higher priority than the printer, the interrupt request is honored. The printer ISR is interrupted, its state is pushed onto the stack, and execution continues at the communications ISR. While this routine is executing, a disk interrupt occurs ($t = 20$). Because this interrupt is of lower priority, it is simply held, and the communications ISR runs to completion.

When the communications ISR is complete ($t = 25$), the previous processor state is restored, which is the execution of the printer ISR. However, before even a single instruction in that routine can be executed, the processor honors the higher-priority disk interrupt and transfers control to the disk ISR. Only when that routine is complete ($t = 35$) is the printer ISR resumed. When that routine completes ($t = 40$), control finally returns to the user program.

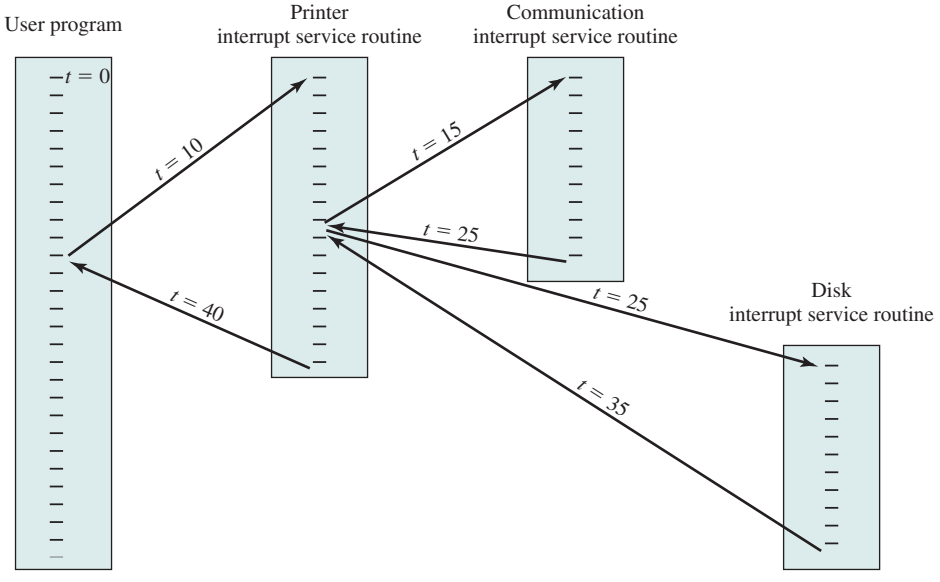



**Figure 1.13** Example Time Sequence of Multiple Interrupts

## 1.5 THE MEMORY HIERARCHY

The design constraints on a computer's memory can be summed up by three questions: How much? How fast? How expensive?

The question of how much is somewhat open-ended. If the capacity is there, applications will likely be developed to use it. The question of how fast is, in a sense, easier to answer. To achieve greatest performance, the memory must be able to keep up with the processor. That is, as the processor is executing instructions, we would not want it to have to pause waiting for instructions or operands. The final question must also be considered. For a practical system, the cost of memory must be reasonable in relationship to other components.

As might be expected, there is a trade-off among the three key characteristics of memory: capacity, access time, and cost. A variety of technologies are used to implement memory systems, and across this spectrum of technologies, the following relationships hold:

- Faster access time, greater cost per bit
- Greater capacity, smaller cost per bit
- Greater capacity, slower access speed

The dilemma facing the designer is clear. The designer would like to use memory technologies that provide for large-capacity memory, both because the capacity is needed and because the cost per bit is low. However, to meet performance requirements, the designer needs to use expensive, relatively lower-capacity memories with fast access times.

The way out of this dilemma is to not rely on a single memory component or technology, but to employ a **memory hierarchy**. A typical hierarchy is illustrated in Figure 1.14. As one goes down the hierarchy, the following occur:

**a.** Decreasing cost per bit

**b.** Increasing capacity

**c.** Increasing access time

**d.** Decreasing frequency of access to the memory by the processor

Thus, smaller, more expensive, faster memories are supplemented by larger, cheaper, slower memories. The key to the success of this organization is the decreasing frequency of access at lower levels. We will examine this concept in greater detail later in this chapter when we discuss the cache, and when we discuss virtual memory later in this book. A brief explanation is provided at this point.

Suppose the processor has access to two levels of memory. Level 1 contains 1000 bytes and has an access time of 0.1 $\mu$s; level 2 contains 100,000 bytes and has an access time of 1 $\mu$s. Assume that if a byte to be accessed is in level 1, then the processor accesses it directly. If it is in level 2, the byte is first transferred to level 1, then accessed by the processor. For simplicity, we ignore the time required for the processor to determine whether the byte is in level 1 or level 2. Figure 1.15 shows the general shape of the curve that models this situation. The figure shows the average access time to a two-level memory as a function of the **hit ratio** $H$, where $H$ is defined

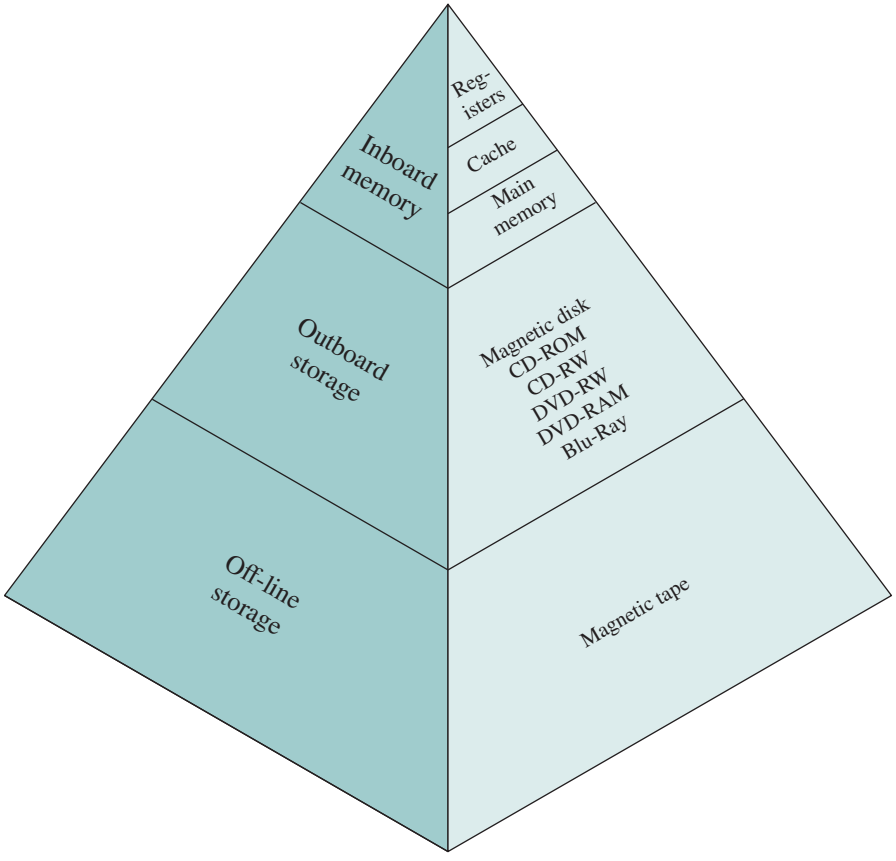



**Figure 1.14** **The Memory Hierarchy**

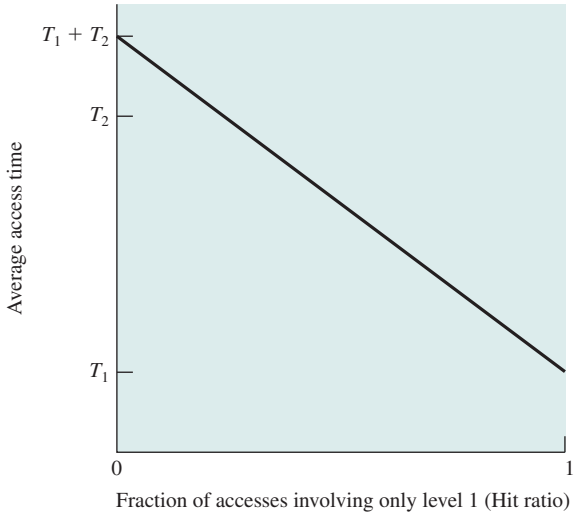



**Figure 1.15** **Performance of a Simple Two-Level Memory**

as the fraction of all memory accesses that are found in the faster memory (e.g., the cache), $T_1$ is the access time to level 1, and $T_2$ is the access time to level 2.[4] As can be seen, for high percentages of level 1 access, the average total access time is much closer to that of level 1 than that of level 2.

In our example, suppose 95% of the memory accesses are found in the cache ($H = 0.95$). Then, the average time to access a byte can be expressed as

$$(0.95)(0.1\ \mu s) + (0.05)(0.1\ \mu s + 1\ \mu s) = 0.095 + 0.055 = 0.15\ \mu s$$

The result is close to the access time of the faster memory. So the strategy of using two memory levels works in principle, but only if conditions (a) through (d) in the preceding list apply. By employing a variety of technologies, a spectrum of memory systems exists that satisfies conditions (a) through (c). Fortunately, condition (d) is also generally valid.

The basis for the validity of condition (d) is a principle known as **locality of reference** [DENN68]. During the course of execution of a program, memory references by the processor, for both instructions and data, tend to cluster. Programs typically contain a number of iterative loops and subroutines. Once a loop or subroutine is entered, there are repeated references to a small set of instructions. Similarly, operations on tables and arrays involve access to a clustered set of data bytes. Over a long period of time, the clusters in use change, but over a short period of time, the processor is primarily working with fixed clusters of memory references.

Accordingly, it is possible to organize data across the hierarchy such that the percentage of accesses to each successively lower level is substantially less than that of the level above. Consider the two-level example already presented. Let level 2 memory contain all program instructions and data. The current clusters can be temporarily placed in level 1. From time to time, one of the clusters in level 1 will have to be swapped back to level 2 to make room for a new cluster coming in to level 1. On average, however, most references will be to instructions and data contained in level 1.

This principle can be applied across more than two levels of memory. The fastest, smallest, and most expensive type of memory consists of the registers internal to the processor. Typically, a processor will contain a few dozen such registers, although some processors contain hundreds of registers. Skipping down two levels, main memory is the principal internal memory system of the computer. Each location in main memory has a unique address, and most machine instructions refer to one or more main memory addresses. Main memory is usually extended with a higher-speed, smaller cache. The cache is not usually visible to the programmer or, indeed, to the processor. It is a device for staging the movement of data between main memory and processor registers to improve performance.

The three forms of memory just described are typically volatile and employ semiconductor technology. The use of three levels exploits the fact that semiconductor memory comes in a variety of types, which differ in speed and cost. Data are stored more permanently on external mass storage devices, of which the most common are hard disk and removable media, such as removable disk, tape, and optical

[4]If the accessed word is found in the faster memory, that is defined as a **hit**. A **miss** occurs if the accessed word is not found in the faster memory.

storage. External, nonvolatile memory is also referred to as **secondary memory** or **auxiliary memory**. These are used to store program and data files, and are usually visible to the programmer only in terms of files and records, as opposed to individual bytes or words. A hard disk is also used to provide an extension to main memory known as virtual memory, which will be discussed in Chapter 8.

Additional levels can be effectively added to the hierarchy in software. For example, a portion of main memory can be used as a buffer to temporarily hold data that are to be read out to disk. Such a technique, sometimes referred to as a disk cache (to be examined in detail in Chapter 11), improves performance in two ways:

1. Disk writes are clustered. Instead of many small transfers of data, we have a few large transfers of data. This improves disk performance and minimizes processor involvement.
2. Some data destined for write-out may be referenced by a program before the next dump to disk. In that case, the data are retrieved rapidly from the software cache rather than slowly from the disk.

Appendix 1A examines the performance implications of multilevel memory structures.

## 1.6 CACHE MEMORY

Although cache memory is invisible to the OS, it interacts with other memory management hardware. Furthermore, many of the principles used in virtual memory schemes (to be discussed in Chapter 8) are also applied in cache memory.

### Motivation

On all instruction cycles, the processor accesses memory at least once, to fetch the instruction, and often one or more additional times, to fetch operands and/or store results. The rate at which the processor can execute instructions is clearly limited by the memory cycle time (the time it takes to read one word from or write one word to memory). This limitation has been a significant problem because of the persistent mismatch between processor and main memory speeds. Over the years, processor speed has consistently increased more rapidly than memory access speed. We are faced with a trade-off among speed, cost, and size. Ideally, main memory should be built with the same technology as that of the processor registers, giving memory cycle times comparable to processor cycle times. This has always been too expensive a strategy. The solution is to exploit the principle of locality by providing a small, fast memory between the processor and main memory, namely the cache.

### Cache Principles

Cache memory is intended to provide memory access time approaching that of the fastest memories available, and at the same time support a large memory size that has the price of less expensive types of semiconductor memories. The concept is illustrated in Figure 1.16a. There is a relatively large and slow main memory together with a smaller, faster cache memory. The cache contains a copy of a portion of main

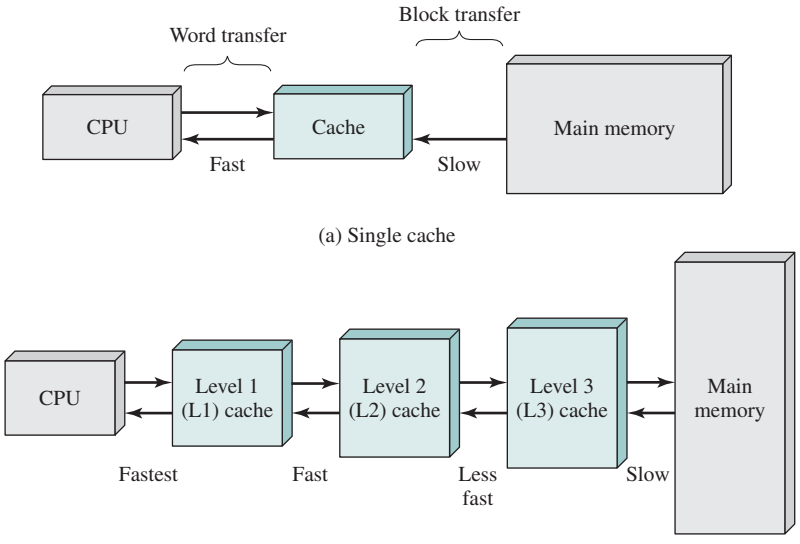



**Figure 1.16 Cache and Main Memory**

memory. When the processor attempts to read a byte or word of memory, a check is made to determine if the byte or word is in the cache. If so, the byte or word is delivered to the processor. If not, a block of main memory, consisting of some fixed number of bytes, is read into the cache then the byte or word is delivered to the processor. Because of the phenomenon of locality of reference, when a block of data is fetched into the cache to satisfy a single memory reference, it is likely that many of the near-future memory references will be to other bytes in the block.

Figure 1.16b depicts the use of multiple levels of cache. The L2 cache is slower and typically larger than the L1 cache, and the L3 cache is slower and typically larger than the L2 cache.

Figure 1.17 depicts the structure of a cache/main memory system. Main memory consists of up to $2^n$ addressable words, with each word having a unique $n$-bit address. For mapping purposes, this memory is considered to consist of a number of fixed-length **blocks** of $K$ words each. That is, there are $M = 2^n/K$ blocks. Cache consists of $C$ **slots** (also referred to as *lines*) of $K$ words each, and the number of slots is considerably less than the number of main memory blocks ($C << M$).[5] Some subset of the blocks of main memory resides in the slots of the cache. If a word in a block of memory that is not in the cache is read, that block is transferred to one of the slots of the cache. Because there are more blocks than slots, an individual slot cannot be uniquely and permanently dedicated to a particular block. Therefore, each slot includes a tag that identifies which particular block is currently being stored. The tag is usually some number of higher-order bits of the address, and refers to all addresses that begin with that sequence of bits.

[5]The symbol << means *much less than.* Similarly, the symbol >> means *much greater than.*

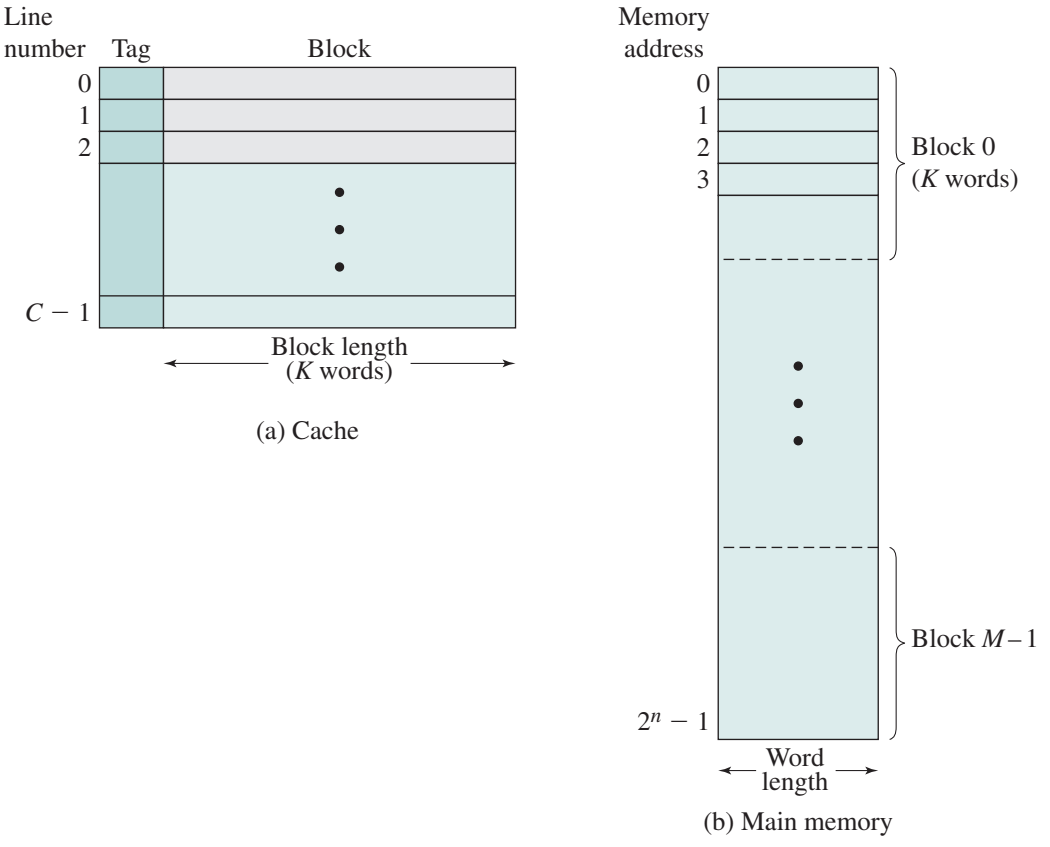



**Figure 1.17** **Cache/Main Memory Structure**

As a simple example, suppose we have a 6-bit address and a 2-bit tag. The tag 01 refers to the block of locations with the following addresses: 010000, 010001, 010010, 010011, 010100, 010101, 010110, 010111, 011000, 011001, 011010, 011011, 011100, 011101, 011110, 011111.

Figure 1.18 illustrates the read operation. The processor generates the address, RA, of a word to be read. If the word is contained in the cache, it is delivered to the processor. Otherwise, the block containing that word is loaded into the cache, and the word is delivered to the processor.

## Cache Design

A detailed discussion of cache design is beyond the scope of this book. Key elements are briefly summarized here. We will see that similar design issues must be addressed in dealing with virtual memory and disk cache design. They fall into the following categories:

- Cache size
- Block size

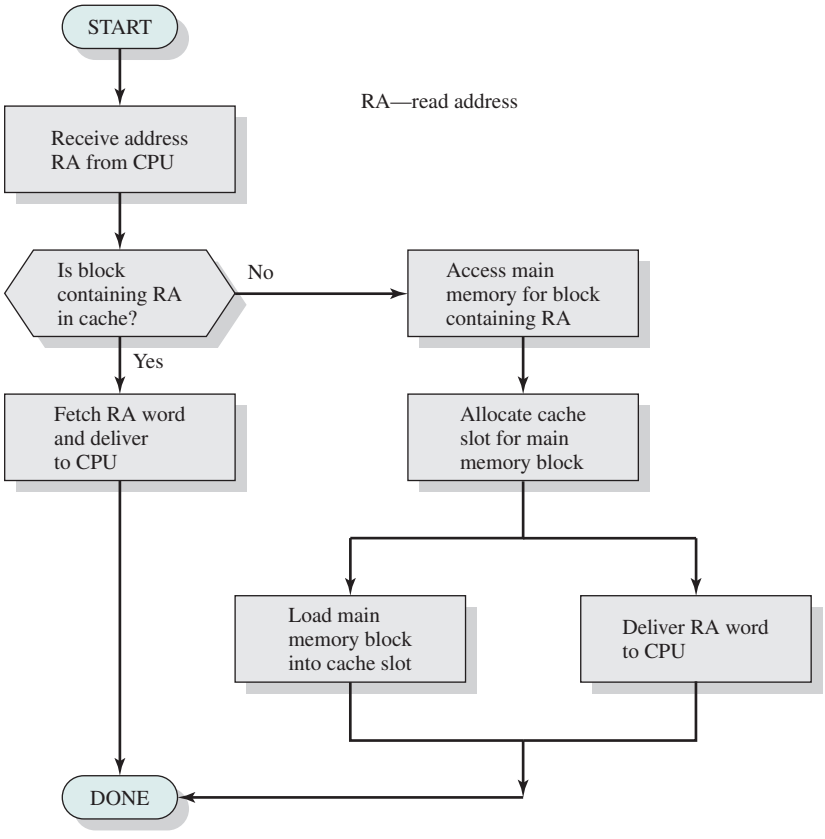



**Figure 1.18** **Cache Read Operation**

- Mapping function
- Replacement algorithm
- Write policy
- Number of cache levels

We have already dealt with the issue of **cache size**. It turns out that reasonably small caches can have a significant impact on performance. Another size issue is that of **block size**: the unit of data exchanged between cache and main memory. Consider beginning with a relatively small block size, then increasing the size. As the block size increases, more useful data are brought into the cache with each block transfer. The result will be that the hit ratio increases because of the principle of locality: the high probability that data in the vicinity of a referenced word are likely to be referenced in the near future. The hit ratio will begin to decrease, however, as the block becomes even bigger, and the probability of using the newly fetched data becomes less than the probability of reusing the data that have to be moved out of the cache to make room for the new block.

When a new block of data is read into the cache, the **mapping function** determines which cache location the block will occupy. Two constraints affect the design

of the mapping function. First, when one block is read in, another may have to be replaced. We would like to do this in such a way as to minimize the probability that we will replace a block that will be needed in the near future. The more flexible the mapping function, the more scope we have to design a replacement algorithm to maximize the hit ratio. Second, the more flexible the mapping function, the more complex is the circuitry required to search the cache to determine if a given block is in the cache.

The **replacement algorithm** chooses (within the constraints of the mapping function) which block to replace when a new block is to be loaded into the cache and the cache already has all slots filled with other blocks. We would like to replace the block that is least likely to be needed again in the near future. Although it is impossible to identify such a block, a reasonably effective strategy is to replace the block that has been in the cache longest with no reference to it. This policy is referred to as the least-recently-used (LRU) algorithm. Hardware mechanisms are needed to identify the least-recently-used block.

If the contents of a block in the cache are altered, then it is necessary to write it back to main memory before replacing it. The **write policy** dictates when the memory write operation takes place. At one extreme, the writing can occur every time that the block is updated. At the other extreme, the writing occurs only when the block is replaced. The latter policy minimizes memory write operations, but leaves main memory in an obsolete state. This can interfere with multiple-processor operation, and with direct memory access by I/O hardware modules.

Finally, it is now commonplace to have multiple levels of cache, labeled L1 (cache closest to the processor), L2, and in many cases L3. A discussion of the performance benefits of multiple cache levels is beyond our current scope (see [STAL16a] for a discussion).

## 1.7 DIRECT MEMORY ACCESS

Three techniques are possible for I/O operations: programmed I/O, interrupt-driven I/O, and direct memory access (DMA). Before discussing DMA, we will briefly define the other two techniques; see Appendix C for more detail.

When the processor is executing a program and encounters an instruction relating to I/O, it executes that instruction by issuing a command to the appropriate I/O module. In the case of **programmed I/O**, the I/O module performs the requested action, then sets the appropriate bits in the I/O status register but takes no further action to alert the processor. In particular, it does not interrupt the processor. Thus, after the I/O instruction is invoked, the processor must take some active role in determining when the I/O instruction is completed. For this purpose, the processor periodically checks the status of the I/O module until it finds that the operation is complete.

With programmed I/O, the processor has to wait a long time for the I/O module of concern to be ready for either reception or transmission of more data. The processor, while waiting, must repeatedly interrogate the status of the I/O module. As a result, the performance level of the entire system is severely degraded.

An alternative, known as **interrupt-driven I/O**, is for the processor to issue an I/O command to a module then go on to do some other useful work. The I/O module

will then interrupt the processor to request service when it is ready to exchange data with the processor. The processor then executes the data transfer, as before, and resumes its former processing.

Interrupt-driven I/O, though more efficient than simple programmed I/O, still requires the active intervention of the processor to transfer data between memory and an I/O module, and any data transfer must traverse a path through the processor. Thus, both of these forms of I/O suffer from two inherent drawbacks:

1. The I/O transfer rate is limited by the speed with which the processor can test and service a device.
2. The processor is tied up in managing an I/O transfer; a number of instructions must be executed for each I/O transfer.

When large volumes of data are to be moved, a more efficient technique is required: **direct memory access (DMA)**. The DMA function can be performed by a separate module on the system bus, or it can be incorporated into an I/O module. In either case, the technique works as follows. When the processor wishes to read or write a block of data, it issues a command to the DMA module by sending the following information:

- Whether a read or write is requested
- The address of the I/O device involved
- The starting location in memory to read data from or write data to
- The number of words to be read or written

The processor then continues with other work. It has delegated this I/O operation to the DMA module, and that module will take care of it. The DMA module transfers the entire block of data, one word at a time, directly to or from memory without going through the processor. When the transfer is complete, the DMA module sends an interrupt signal to the processor. Thus, the processor is involved only at the beginning and end of the transfer.

The DMA module needs to take control of the bus to transfer data to and from memory. Because of this competition for bus usage, there may be times when the processor needs the bus and must wait for the DMA module. Note this is not an interrupt; the processor does not save a context and do something else. Rather, the processor pauses for one bus cycle (the time it takes to transfer one word across the bus). The overall effect is to cause the processor to execute more slowly during a DMA transfer when processor access to the bus is required. Nevertheless, for a multiple-word I/O transfer, DMA is far more efficient than interrupt-driven or programmed I/O.

## 1.8 MULTIPROCESSOR AND MULTICORE ORGANIZATION

Traditionally, the computer has been viewed as a sequential machine. Most computer programming languages require the programmer to specify algorithms as sequences of instructions. A processor executes programs by executing machine instructions in sequence and one at a time. Each instruction is executed in

a sequence of operations (fetch instruction, fetch operands, perform operation, store results).

This view of the computer has never been entirely true. At the micro-operation level, multiple control signals are generated at the same time. Instruction pipelining, at least to the extent of overlapping fetch and execute operations, has been around for a long time. Both of these are examples of performing functions in parallel.

As computer technology has evolved and as the cost of computer hardware has dropped, computer designers have sought more and more opportunities for parallelism, usually to improve performance and, in some cases, to improve reliability. In this book, we will examine three approaches to providing parallelism by replicating processors: symmetric multiprocessors (SMPs), multicore computers, and clusters. SMPs and multicore computers are discussed in this section; clusters will be examined in Chapter 16.

## Symmetric Multiprocessors

***DEFINITION*** An SMP can be defined as a stand-alone computer system with the following characteristics:

1. There are two or more similar processors of comparable capability.
2. These processors share the same main memory and I/O facilities and are interconnected by a bus or other internal connection scheme, such that memory access time is approximately the same for each processor.
3. All processors share access to I/O devices, either through the same channels or through different channels that provide paths to the same device.
4. All processors can perform the same functions (hence the term *symmetric*).
5. The system is controlled by an integrated operating system that provides interaction between processors and their programs at the job, task, file, and data element levels.

Points 1 to 4 should be self-explanatory. Point 5 illustrates one of the contrasts with a loosely coupled multiprocessing system, such as a cluster. In the latter, the physical unit of interaction is usually a message or complete file. In an SMP, individual data elements can constitute the level of interaction, and there can be a high degree of cooperation between processes.

An SMP organization has a number of potential advantages over a uniprocessor organization, including the following:

- **Performance:** If the work to be done by a computer can be organized such that some portions of the work can be done in parallel, then a system with multiple processors will yield greater performance than one with a single processor of the same type.
- **Availability:** In a symmetric multiprocessor, because all processors can perform the same functions, the failure of a single processor does not halt the machine. Instead, the system can continue to function at reduced performance.

- **Incremental growth:** A user can enhance the performance of a system by adding an additional processor.
- **Scaling:** Vendors can offer a range of products with different price and performance characteristics based on the number of processors configured in the system.

It is important to note these are potential, rather than guaranteed, benefits. The operating system must provide tools and functions to exploit the parallelism in an SMP system.

An attractive feature of an SMP is that the existence of multiple processors is transparent to the user. The operating system takes care of scheduling of tasks on individual processors, and of synchronization among processors.

***ORGANIZATION*** Figure 1.19 illustrates the general organization of an SMP. There are multiple processors, each of which contains its own control unit, arithmetic-logic unit, and registers. Each processor typically has two dedicated levels of cache, designated L1 and L2. As Figure 1.19 indicates, each processor and its dedicated caches are housed on a separate chip. Each processor has access to a shared main memory and the I/O devices through some form of interconnection mechanism; a shared bus is a common facility. The processors can communicate with each other through

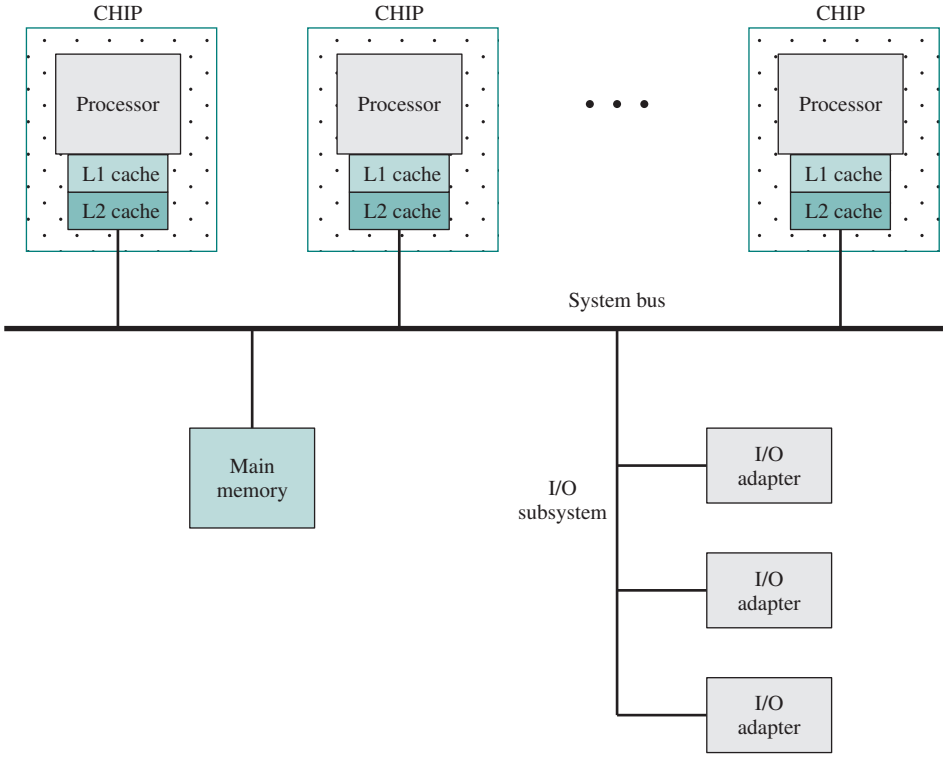



**Figure 1.19 Symmetric Multiprocessor Organization**

memory (messages and status information left in shared address spaces). It may also be possible for processors to exchange signals directly. The memory is often organized so multiple simultaneous accesses to separate blocks of memory are possible.

In modern computers, processors generally have at least one level of cache memory that is private to the processor. This use of cache introduces some new design considerations. Because each local cache contains an image of a portion of main memory, if a word is altered in one cache, it could conceivably invalidate a word in another cache. To prevent this, the other processors must be alerted that an update has taken place. This problem is known as the cache coherence problem, and is typically addressed in hardware rather than by the OS.[6]

## Multicore Computers

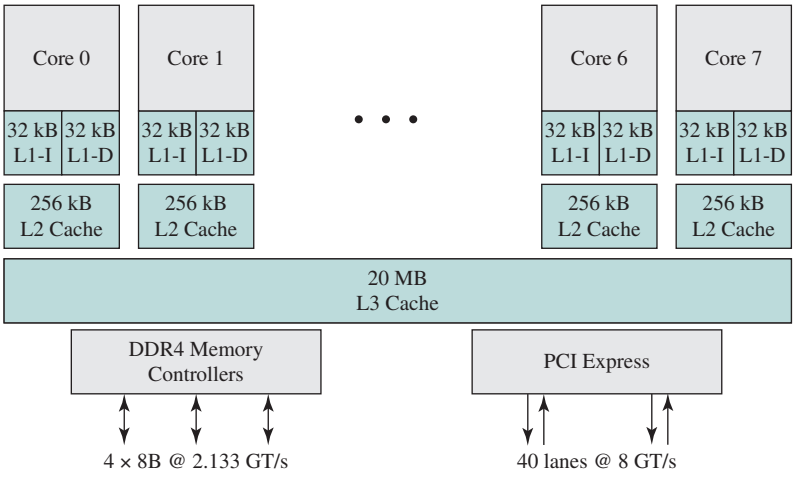
A **multicore** computer, also known as a **chip multiprocessor**, combines two or more processors (called cores) on a single piece of silicon (called a die). Typically, each core consists of all of the components of an independent processor, such as registers, ALU, pipeline hardware, and control unit, plus L1 instruction and data caches. In addition to the multiple cores, contemporary multicore chips also include L2 cache and, in some cases, L3 cache.

The motivation for the development of multicore computers can be summed up as follows. For decades, microprocessor systems have experienced a steady, usually exponential, increase in performance. This is partly due to hardware trends, such as an increase in clock frequency and the ability to put cache memory closer to the processor because of the increasing miniaturization of microcomputer components. Performance has also been improved by the increased complexity of processor design to exploit parallelism in instruction execution and memory access. In brief, designers have come up against practical limits in the ability to achieve greater performance by means of more complex processors. Designers have found that the best way to improve performance to take advantage of advances in hardware is to put multiple processors and a substantial amount of cache memory on a single chip. A detailed discussion of the rationale for this trend is beyond our current scope, but is summarized in Appendix C.

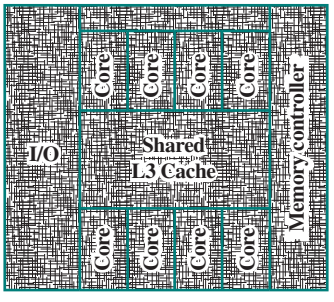
An example of a multicore system is the Intel Core i7-5960X, which includes six x86 processors, each with a dedicated L2 cache, and with a shared L3 cache (see Figure 1.20a). One mechanism Intel uses to make its caches more effective is prefetching, in which the hardware examines memory access patterns and attempts to fill the caches speculatively with data that's likely to be requested soon. Figure 1.20b shows the physical layout of the 5960X in its chip.

The Core i7-5960X chip supports two forms of external communications to other chips. The **DDR4 memory controller** brings the memory controller for the DDR (double data rate) main memory onto the chip. The interface supports four channels that are 8 bytes wide for a total bus width of 256 bits, for an aggregate data rate of up to 64 GB/s. With the memory controller on the chip, the Front Side Bus is eliminated. The **PCI Express** is a peripheral bus and enables high-speed communications among connected processor chips. The PCI Express link operates at 8 GT/s (transfers per second). At 40 bits per transfer, that adds up to 40 GB/s.

[6] A description of hardware-based cache coherency schemes is provided in [STAL16a].




(a) Block diagram




(b) Physical layout on chip

**Figure 1.20** **Intel Core i7-5960X Block Diagram**

## 1.9 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

### Key Terms

| | | |
|---|---|---|
| address register | instruction cycle | programmed I/O |
| auxiliary memory | instruction register | register |
| block | interrupt | replacement algorithm |
| cache memory | interrupt-driven I/O | secondary memory |
| cache slot | I/O module | slot |
| central processing unit | locality of reference | spatial locality |
| chip multiprocessor | main memory | stack |
| data register | memory hierarchy | stack frame |
| direct memory access (DMA) | miss | stack pointer |
| hit | multicore | system bus |
| hit ratio | multiprocessor | temporal locality |
| input/output | processor | |
| instruction | program counter | |

## Review Questions

**1.1.** List and briefly define the four main elements of a computer.
**1.2.** Define the two main categories of processor registers.
**1.3.** In general terms, what are the four distinct actions that a machine instruction can specify?
**1.4.** What is an interrupt?
**1.5.** How can multiple interrupts be serviced by setting priorities?
**1.6.** What characteristics are observed while going up the memory hierarchy?
**1.7.** What are the trade-offs that determine the size of the cache memory?
**1.8.** What is the difference between a multiprocessor and a multicore system?
**1.9.** What is the distinction between spatial locality and temporal locality?
**1.10.** In general, what are the strategies for exploiting spatial locality and temporal locality?

## Problems

**1.1.** Suppose the hypothetical processor of Figure 1.3 also has two I/O instructions:

0011 = Load AC from I/O
0100 = SUB from AC

In these cases, the 12-bit address identifies a particular external device. Show the program execution (using the format of Figure 1.4) for the following program:

1. Load AC from device 7.
2. SUB from AC contents of memory location 880.
3. Store AC to memory location 881.

Assume that the next value retrieved from device 7 is 6 and that location 880 contains a value of 5.

**1.2.** The program execution of Figure 1.4 is described in the text using six steps. Expand this description to show the use of the MAR and MBR.

**1.3.** Consider a hypothetical 64-bit microprocessor having 64-bit instructions composed of two fields. The first 4 bytes contain the opcode, and the remainder an immediate operand or an operand address.

**a.** What is the maximum directly addressable memory capacity?
**b.** What ideal size of microprocessor address buses should be used? How will system speed be affected for data buses of 64 bits, 32 bits and 16 bits?
**c.** How many bits should the instruction register contain if the instruction register is to contain only the opcode, and how many if the instruction register is to contain the whole instruction?

**1.4.** Consider a hypothetical microprocessor generating a 16-bit address (e.g., assume the program counter and the address registers are 16 bits wide) and having a 16-bit data bus.

**a.** What is the maximum memory address space that the processor can access directly if it is connected to a "16-bit memory"?
**b.** What is the maximum memory address space that the processor can access directly if it is connected to an "8-bit memory"?
**c.** What architectural features will allow this microprocessor to access a separate "I/O space"?
**d.** If an input and an output instruction can specify an 8-bit I/O port number, how many 8-bit I/O ports can the microprocessor support? How many 16-bit I/O ports? Explain.

**1.5.** Consider a 64-bit microprocessor, with a 32-bit external data bus, driven by a 16 MHz input clock. Assume that this microprocessor has a bus cycle whose minimum duration equals four input clock cycles. What is the maximum data transfer rate across the bus that this microprocessor can sustain in bytes/s? To increase its performance, would it be better to make its external data bus 64 bits or to double the external clock frequency supplied to the microprocessor? State any other assumptions you make and explain. *Hint:* Determine the number of bytes that can be transferred per bus cycle.

**1.6.** Consider a computer system that contains an I/O module controlling a simple keyboard/printer Teletype. The following registers are contained in the CPU and connected directly to the system bus:

| | |
|---|---|
| INPR: | Input Register, 8 bits |
| OUTR: | Output Register, 8 bits |
| FGI: | Input Flag, 1 bit |
| FGO: | Output Flag, 1 bit |
| IEN: | Interrupt Enable, 1 bit |

Keystroke input from the Teletype and output to the printer are controlled by the I/O module. The Teletype is able to encode an alphanumeric symbol to an 8-bit word and decode an 8-bit word into an alphanumeric symbol. The Input flag is set when an 8-bit word enters the input register from the Teletype. The Output flag is set when a word is printed.

**a.** Describe how the CPU, using the first four registers listed in this problem, can achieve I/O with the Teletype.

**b.** Describe how the function can be performed more efficiently by also employing IEN.

**1.7.** In virtually all systems that include DMA modules, DMA access to main memory is given higher priority than processor access to main memory. Why?

**1.8.** A DMA module is transferring characters to main memory from an external device transmitting at 10800 bits per second (bps). The processor can fetch instructions at the rate of 1 million instructions per second. By how much will the processor be slowed down due to the DMA activity?

**1.9.** A computer consists of a CPU and an I/O device *D* connected to main memory *M* via a shared bus with a data bus width of one word. The CPU can execute a maximum of 106 instructions per second. An average instruction requires five processor cycles, three of which use the memory bus. A memory read or write operation uses one processor cycle. Suppose that the CPU is continuously executing "background" programs that require 95% of its instruction execution rate but not any I/O instructions. Assume that one processor cycle equals one bus cycle. Now suppose that very large blocks of data are to be transferred between *M* and *D*.

**a.** If programmed I/O is used and each one-word I/O transfer requires the CPU to execute two instructions, estimate the maximum I/O data transfer rate, in words per second, possible through *D*.

**b.** Estimate the same rate if DMA transfer is used.

**1.10.** Consider the following code:

```
for (i = 0; i < 20; i++)
     for (j = 0; j < 10; j++)
          a[i] = a[i] * j
```

**a.** Give one example of the spatial locality in the code.

**b.** Give one example of the temporal locality in the code.

**1.11.** Extend Equations (1.1) and (1.2) in Appendix 1A to 3-level memory hierarchies.

**1.12.** Consider a memory system with cache having the following parameters:

$$Sc = 32\text{ KB} \quad Cc = 0.1\text{ cents/bytes} \quad T_C = 10\text{ ns}$$
$$Sm = 256\text{ MB} \quad Cm = 0.0001\text{ cents/bytes} \quad Tm = 100\text{ ns}$$

**a.** What was the total cost prior to addition of cache?
**b.** What is the total cost after addition of cache?
**c.** What is the percentage decrease in time due to inclusion of cache with respect to a system without cache memory considering a cache hit ratio of 0.85?

**1.13.** Suppose that a large file is being accessed by a computer memory system comprising of a cache and a main memory. The cache access time is 60 ns. Time to access main memory (including cache access) is 300 ns. The file can be opened either in read or in write mode. A write operation involves accessing both main memory and the cache (write-through cache). A read operation accesses either only the cache or both the cache and main memory depending upon whether the access word is found in the cache or not. It is estimated that read operations comprise of 80% of all operations. If the cache hit ratio for read operations is 0.9, what is the average access time of this system?

**1.14.** Suppose a stack is to be used by the processor to manage procedure calls and returns. Can the program counter be eliminated by using the top of the stack as a program counter?

## APPENDIX 1A PERFORMANCE CHARACTERISTICS OF TWO-LEVEL MEMORIES

In this chapter, reference is made to a cache that acts as a buffer between main memory and processor, creating a two-level internal memory. This two-level architecture exploits a property known as locality to provide improved performance over a comparable one-level memory.

The main memory cache mechanism is part of the computer architecture, implemented in hardware and typically invisible to the OS. Accordingly, this mechanism is not pursued in this book. However, there are two other instances of a two-level memory approach that also exploit the property of locality and that are, at least partially, implemented in the OS: virtual memory and the disk cache (Table 1.2). These two topics are explored in Chapters 8 and 11, respectively. In this appendix, we will look at some of the performance characteristics of two-level memories that are common to all three approaches.

**Table 1.2** Characteristics of Two-Level Memories

| | Main Memory Cache | Virtual Memory (Paging) | Disk Cache |
|---|---|---|---|
| **Typical access time ratios** | 5 : 1 | $10^6$: 1 | $10^6$: 1 |
| **Memory management system** | Implemented by special hardware | Combination of hardware and system software | System software |
| **Typical block size** | 4 to 128 bytes | 64 to 4096 bytes | 64 to 4096 bytes |
| **Access of processor to second level** | Direct access | Indirect access | Indirect access |

## Locality

The basis for the performance advantage of a two-level memory is the principle of locality, referred to in Section 1.5. This principle states that memory references tend to cluster. Over a long period of time, the clusters in use change; but over a short period of time, the processor is primarily working with fixed clusters of memory references.

Intuitively, the principle of locality makes sense. Consider the following line of reasoning:

1. Except for branch and call instructions, which constitute only a small fraction of all program instructions, program execution is sequential. Hence, in most cases, the next instruction to be fetched immediately follows the last instruction fetched.
2. It is rare to have a long uninterrupted sequence of procedure calls followed by the corresponding sequence of returns. Rather, a program remains confined to a rather narrow window of procedure-invocation depth. Thus, over a short period of time, references to instructions tend to be localized to a few procedures.
3. Most iterative constructs consist of a relatively small number of instructions repeated many times. For the duration of the iteration, computation is therefore confined to a small contiguous portion of a program.
4. In many programs, much of the computation involves processing data structures, such as arrays or sequences of records. In many cases, successive references to these data structures will be to closely located data items.

This line of reasoning has been confirmed in many studies. With reference to point (1), a variety of studies have analyzed the behavior of high-level language programs. Table 1.3 includes key results, measuring the appearance of various statement types during execution, from the following studies. The earliest study of programming language behavior, performed by Knuth [KNUT71], examined a collection of FORTRAN programs used as student exercises. Tanenbaum [TANE78] published measurements collected from over 300 procedures used in OS programs and written in a language that supports structured programming (SAL). Patterson and Sequin [PATT82] analyzed a set of measurements taken from compilers and

**Table 1.3** Relative Dynamic Frequency of High-Level Language Operations

| Study | [HUCK83] | [KNUT71] | [PATT82] | | [TANE78] |
|---|---|---|---|---|---|
| Language | Pascal | FORTRAN | Pascal | C | SAL |
| Workload | Scientific | Student | System | System | System |
| Assign | 74 | 67 | 45 | 38 | 42 |
| Loop | 4 | 3 | 5 | 3 | 4 |
| Call | 1 | 3 | 15 | 12 | 12 |
| IF | 20 | 11 | 29 | 43 | 36 |
| GOTO | 2 | 9 | – | 3 | – |
| Other | – | 7 | 6 | 1 | 6 |

programs for typesetting, computer-aided design (CAD), sorting, and file comparison. The programming languages C and Pascal were studied. Huck [HUCK83] analyzed four programs intended to represent a mix of general-purpose scientific computing, including fast Fourier transform and the integration of systems of differential equations. There is good agreement in the results of this mixture of languages and applications that branching and call instructions represent only a fraction of statements executed during the lifetime of a program. Thus, these studies confirm assertion (1), from the preceding list.

With respect to assertion (2), studies reported in [PATT85] provide confirmation. This is illustrated in Figure 1.21, which shows call-return behavior. Each call is represented by the line moving down and to the right, and each return by the line moving up and to the right. In the figure, a *window* with depth equal to 5 is defined. Only a sequence of calls and returns with a net movement of 6 in either direction causes the window to move. As can be seen, the executing program can remain within a stationary window for long periods of time. A study by the same analysts of C and Pascal programs showed that a window of depth 8 would only need to shift on less than 1% of the calls or returns [TAMI83].

A distinction is made in the literature between spatial locality and temporal locality. **Spatial locality** refers to the tendency of execution to involve a number of memory locations that are clustered. This reflects the tendency of a processor to access instructions sequentially. Spatial location also reflects the tendency of a program to access data locations sequentially, such as when processing a table of data. **Temporal locality** refers to the tendency for a processor to access memory locations that have been used recently. For example, when an iteration loop is executed, the processor executes the same set of instructions repeatedly.

Traditionally, temporal locality is exploited by keeping recently used instruction and data values in cache memory, and by exploiting a cache hierarchy. Spatial locality is generally exploited by using larger cache blocks, and by incorporating

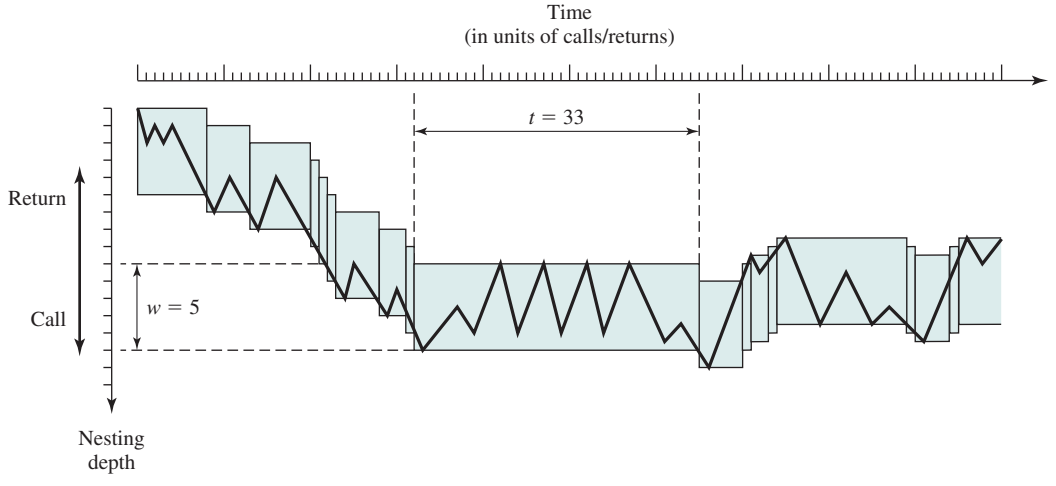



**Figure 1.21 Example Call-Return Behavior of a Program**

prefetching mechanisms (fetching items whose use is expected) into the cache control logic. Recently, there has been considerable research on refining these techniques to achieve greater performance, but the basic strategies remain the same.

## Operation of Two-Level Memory

The locality property can be exploited in the formation of a two-level memory. The upper-level memory (M1) is smaller, faster, and more expensive (per bit) than the lower-level memory (M2). M1 is used as temporary storage for part of the contents of the larger M2. When a memory reference is made, an attempt is made to access the item in M1. If this succeeds, then a quick access is made. If not, then a block of memory locations is copied from M2 to M1, and the access then takes place via M1. Because of locality, once a block is brought into M1, there should be a number of accesses to locations in that block, resulting in fast overall service.

To express the average time to access an item, we must consider not only the speeds of the two levels of memory but also the probability that a given reference can be found in M1. We have

$$T_s = H \times T_1 + (1 - H) \times (T_1 + T_2)$$
$$T_1 + (1 - H) \times T_2 \qquad \textbf{(1.1)}$$

where

$T_s$ = average (system) access time
$T_1$ = access time of M1 (e.g., cache, disk cache)
$T_2$ = access time of M2 (e.g., main memory, disk)
$H$ = hit ratio (fraction of time reference is found in M1)

Figure 1.15 shows average access time as a function of hit ratio. As can be seen, for a high percentage of hits, the average total access time is much closer to that of M1 than M2.

## Performance

Let us look at some of the parameters relevant to an assessment of a two-level memory mechanism. First, consider cost. We have

$$C_S = \frac{C_1 S_1 + C_2 S_2}{S_1 + S_2} \qquad \textbf{(1.2)}$$

where

$C_s$ = average cost per bit for the combined two-level memory
$C_1$ = average cost per bit of upper-level memory M1
$C_2$ = average cost per bit of lower-level memory M2
$S_1$ = size of M1
$S_2$ = size of M2

We would like $C_s \approx C_2$. Given that $C_1 >> C_2$, this requires $S_1 << S_2$. Figure 1.22 shows the relationship.[7]

[7]Note both axes use a log scale. A basic review of log scales is in the math refresher document on the Computer Science Student Resource Site at ComputerScienceStudent.com.

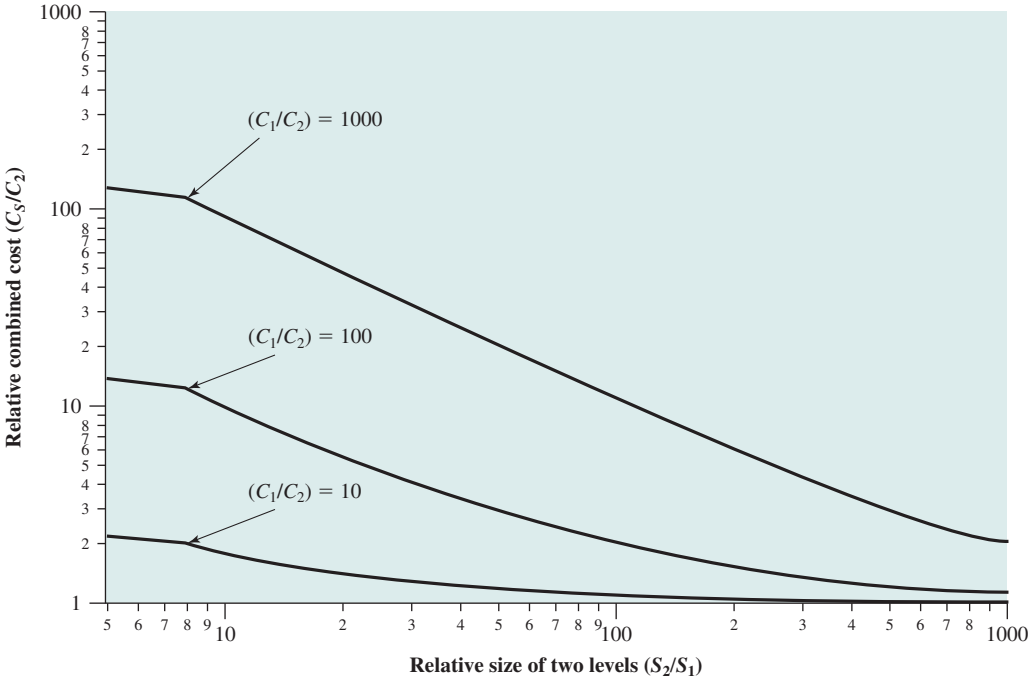



**Figure 1.22** **Relationship of Average Memory Cost to Relative Memory Size for a Two-Level Memory**

Next, consider access time. For a two-level memory to provide a significant performance improvement, we need to have $T_s$ approximately equal to $T_1$ $T_s \approx T_1$. Given that $T_1$ is much less than $T_2$ $T_s >> T_1$, a hit ratio of close to 1 is needed.

So, we would like M1 to be small to hold down cost, and large to improve the hit ratio and therefore the performance. Is there a size of M1 that satisfies both requirements to a reasonable extent? We can answer this question with a series of subquestions:

- What value of hit ratio is needed to satisfy the performance requirement?
- What size of M1 will assure the needed hit ratio?
- Does this size satisfy the cost requirement?

To get at this, consider the quantity $T_1/T_s$, which is referred to as the *access efficiency*. It is a measure of how close average access time ($T_s$) is to M1 access time ($T_1$). From Equation (1.1),

$$\frac{T_1}{T_S} = \frac{1}{1 + (1 - H)\dfrac{T_2}{T_1}} \tag{1.3}$$

In Figure 1.23, we plot $T_1/T_s$ as a function of the hit ratio $H$, with the quantity $T_2/T_1$ as a parameter. A hit ratio in the range of 0.8 to 0.9 would seem to be needed to satisfy the performance requirement.

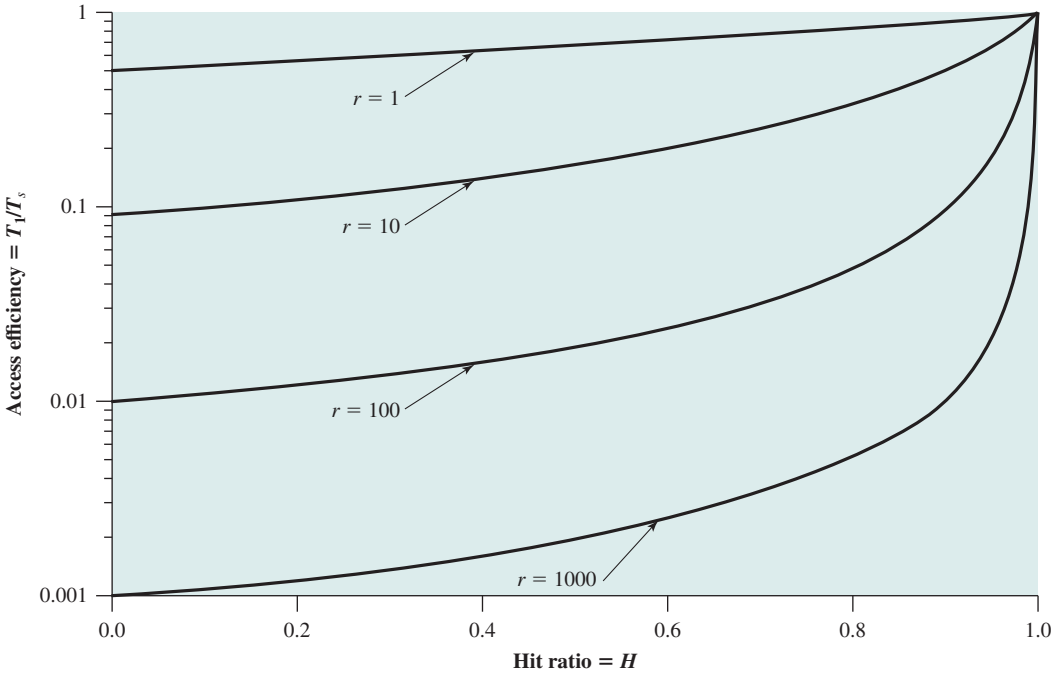



**Figure 1.23** **Access Efficiency as a Function of Hit Ratio (r = T2/T1)**

We can now phrase the question about relative memory size more exactly. Is a hit ratio of 0.8 or higher reasonable for $S_1 << S_2$? This will depend on a number of factors, including the nature of the software being executed and the details of the design of the two-level memory. The main determinant is, of course, the degree of locality. Figure 1.24 suggests the effect of locality on the hit ratio. Clearly, if M1 is the same size as M2, then the hit ratio will be 1.0: All of the items in M2 are also stored in M1. Now suppose there is no locality; that is, references are completely random. In that case, the hit ratio should be a strictly linear function of the relative memory size. For example, if M1 is half the size of M2, then at any time half of the items from M2 are also in M1, and the hit ratio will be 0.5. In practice, however, there is some degree of locality in the references. The effects of moderate and strong locality are indicated in the figure.

So, if there is strong locality, it is possible to achieve high values of hit ratio even with relatively small upper-level memory size. For example, numerous studies have shown that rather small cache sizes will yield a hit ratio above 0.75 *regardless of the size of main memory* ([AGAR89], [PRZY88], [STRE83], and [SMIT82]). A cache in the range of 1K to 128K words is generally adequate, whereas main memory is now typically in the gigabyte range. When we consider virtual memory and disk cache, we will cite other studies that confirm the same phenomenon, namely that a relatively small M1 yields a high value of hit ratio because of locality.

This brings us to the last question listed earlier: Does the relative size of the two memories satisfy the cost requirement? The answer is clearly yes. If we need only a

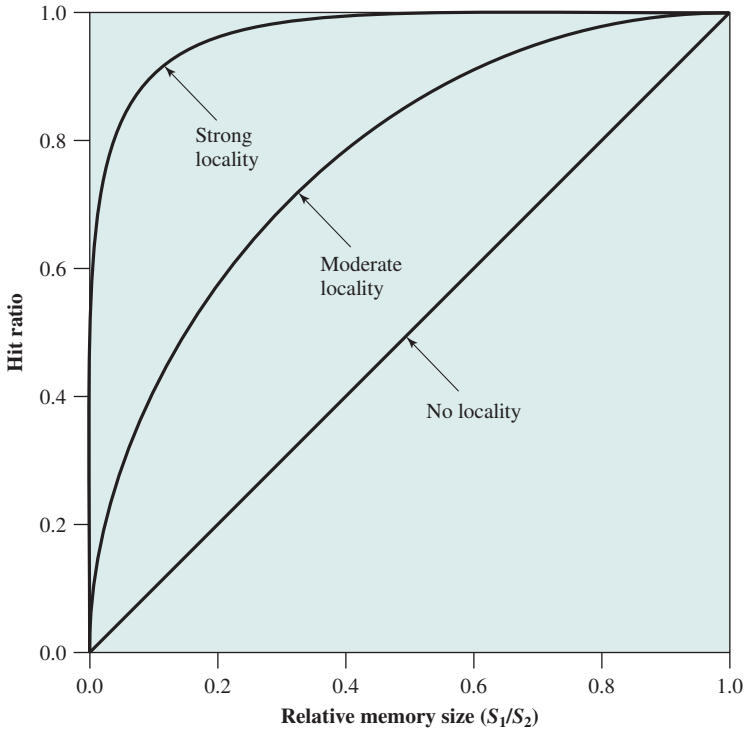



**Figure 1.24** **Hit Ratio as a Function of Relative Memory Size**

relatively small upper-level memory to achieve good performance, then the average cost per bit of the two levels of memory will approach that of the cheaper lower-level memory. Please note that with L2 cache (or even L2 and L3 caches) involved, analysis is much more complex. See [PEIR99] and [HAND98] for discussions.

CHAPTER 2

# Operating System Overview


2.1 **Operating System Objectives and Functions**
- The Operating System as a User/Computer Interface
- The Operating System as Resource Manager
- Ease of Evolution of an Operating System

2.2 **The Evolution of Operating Systems**
- Serial Processing
- Simple Batch Systems
- Multiprogrammed Batch Systems
- Time-Sharing Systems

2.3 **Major Achievements**
- The Process
- Memory Management
- Information Protection and Security
- Scheduling and Resource Management

2.4 **Developments Leading to Modern Operating Systems**

2.5 **Fault Tolerance**
- Fundamental Concepts
- Faults
- Operating System Mechanisms

2.6 **OS Design Considerations for Multiprocessor and Multicore**
- Symmetric Multiprocessor OS Considerations
- Multicore OS Considerations

2.7 **Microsoft Windows Overview**
- Background
- Architecture
- Client/Server Model
- Threads and SMP
- Windows Objects

2.8 **Traditional Unix Systems**
- History
- Description

2.9 **Modern Unix Systems**
- System V Release 4 (SVR4)
- BSD
- Solaris 11

2.10 **Linux**
- History
- Modular Structure
- Kernel Components

2.11 **Android**
- Android Software Architecture
- Android Runtime
- Android System Architecture
- Activities
- Power Management

2.12 **Key Terms, Review Questions, and Problems**

### Learning Objectives

After studying this chapter, you should be able to:

- Summarize, at a top level, the key functions of an operating system (OS).
- Discuss the evolution of operating systems for early simple batch systems to modern complex systems.
- Give a brief explanation of each of the major achievements in OS research, as defined in Section 2.3.
- Discuss the key design areas that have been instrumental in the development of modern operating systems.
- Define and discuss virtual machines and virtualization.
- Understand the OS design issues raised by the introduction of multiprocessor and multicore organization.
- Understand the basic structure of Windows.
- Describe the essential elements of a traditional UNIX system.
- Explain the new features found in modern UNIX systems.
- Discuss Linux and its relationship to UNIX.

We begin our study of operating systems (OSs) with a brief history. This history is itself interesting, and also serves the purpose of providing an overview of OS principles. The first section examines the objectives and functions of operating systems. Then, we will look at how operating systems have evolved from primitive batch systems to sophisticated multitasking, multiuser systems. The remainder of the chapter will look at the history and general characteristics of the two operating systems that serve as examples throughout this book.

## 2.1 OPERATING SYSTEM OBJECTIVES AND FUNCTIONS

An OS is a program that controls the execution of application programs, and acts as an interface between applications and the computer hardware. It can be thought of as having three objectives:

- **Convenience:** An OS makes a computer more convenient to use.
- **Efficiency:** An OS allows the computer system resources to be used in an efficient manner.
- **Ability to evolve:** An OS should be constructed in such a way as to permit the effective development, testing, and introduction of new system functions without interfering with service.

Let us examine these three aspects of an OS in turn.

## The Operating System as a User/Computer Interface

The hardware and software used in providing applications to a user can be viewed in a layered fashion, as depicted in Figure 2.1. The user of those applications (the end user) generally is not concerned with the details of computer hardware. Thus, the end user views a computer system in terms of a set of applications. An application can be expressed in a programming language, and is developed by an application programmer. If one were to develop an application program as a set of machine instructions that is completely responsible for controlling the computer hardware, one would be faced with an overwhelmingly complex undertaking. To ease this chore, a set of system programs is provided. Some of these programs are referred to as utilities, or library programs. These implement frequently used functions that assist in program creation, the management of files, and the control of I/O devices. A programmer will make use of these facilities in developing an application, and the application, while it is running, will invoke the utilities to perform certain functions. The most important collection of system programs comprises the OS. The OS masks the details of the hardware from the programmer, and provides the programmer with a convenient interface for using the system. It acts as a mediator, making it easier for the programmer and for application programs to access and use those facilities and services.

Briefly, the OS typically provides services in the following areas:

- **Program development:** The OS provides a variety of facilities and services, such as editors and debuggers, to assist the programmer in creating programs. Typically, these services are in the form of utility programs that, while not strictly part of the core of the OS, are supplied with the OS, and are referred to as application program development tools.
- **Program execution:** A number of steps need to be performed to execute a program. Instructions and data must be loaded into main memory, I/O devices and

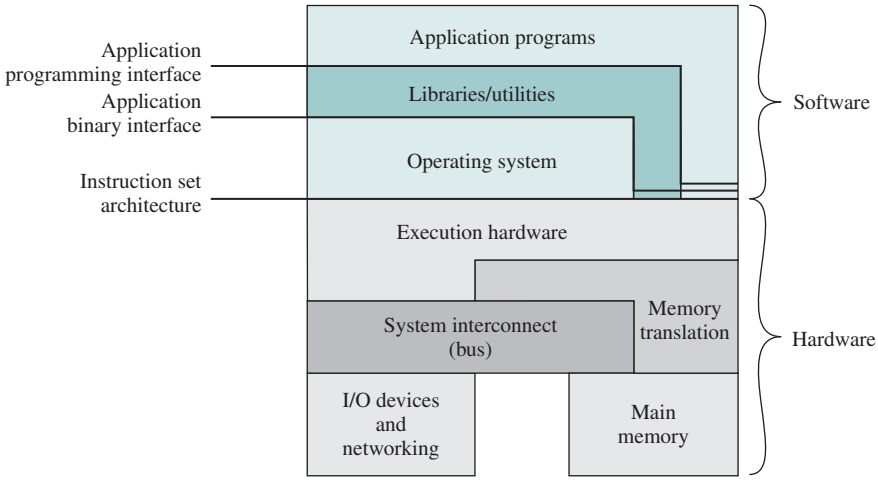



**Figure 2.1 Computer Hardware and Software Structure**

files must be initialized, and other resources must be prepared. The OS handles these scheduling duties for the user.

- **Access to I/O devices:** Each I/O device requires its own peculiar set of instructions or control signals for operation. The OS provides a uniform interface that hides these details so programmers can access such devices using simple reads and writes.
- **Controlled access to files:** For file access, the OS must reflect a detailed understanding of not only the nature of the I/O device (disk drive, tape drive), but also the structure of the data contained in the files on the storage medium. In the case of a system with multiple users, the OS may provide protection mechanisms to control access to the files.
- **System access:** For shared or public systems, the OS controls access to the system as a whole and to specific system resources. The access function must provide protection of resources and data from unauthorized users, and must resolve conflicts for resource contention.
- **Error detection and response:** A variety of errors can occur while a computer system is running. These include internal and external hardware errors (such as a memory error, or a device failure or malfunction), and various software errors, (such as division by zero, attempt to access forbidden memory location, and inability of the OS to grant the request of an application). In each case, the OS must provide a response that clears the error condition with the least impact on running applications. The response may range from ending the program that caused the error, to retrying the operation, or simply reporting the error to the application.
- **Accounting:** A good OS will collect usage statistics for various resources and monitor performance parameters such as response time. On any system, this information is useful in anticipating the need for future enhancements and in tuning the system to improve performance. On a multiuser system, the information can be used for billing purposes.

Figure 2.1 also indicates three key interfaces in a typical computer system:

- **Instruction set architecture (ISA):** The ISA defines the repertoire of machine language instructions that a computer can follow. This interface is the boundary between hardware and software. Note both application programs and utilities may access the ISA directly. For these programs, a subset of the instruction repertoire is available (user ISA). The OS has access to additional machine language instructions that deal with managing system resources (system ISA).
- **Application binary interface (ABI):** The ABI defines a standard for binary portability across programs. The ABI defines the system call interface to the operating system, and the hardware resources and services available in a system through the user ISA.
- **Application programming interface (API):** The API gives a program access to the hardware resources and services available in a system through the user ISA supplemented with high-level language (HLL) library calls. Any system calls are usually performed through libraries. Using an API enables application software to be ported easily, through recompilation, to other systems that support the same API.

## The Operating System as Resource Manager

The OS is responsible for controlling the use of a computer's resources, such as I/O, main and secondary memory, and processor execution time. But this control is exercised in a curious way. Normally, we think of a control mechanism as something external to that which is controlled, or at least as something that is a distinct and separate part of that which is controlled. (For example, a residential heating system is controlled by a thermostat, which is separate from the heat-generation and heat-distribution apparatus.) This is not the case with the OS, which as a control mechanism is unusual in two respects:

- The OS functions in the same way as ordinary computer software; that is, it is a program or suite of programs executed by the processor.
- The OS frequently relinquishes control, and must depend on the processor to allow it to regain control.

Like other computer programs, the OS consists of instructions executed by the processor. While executing, the OS decides how processor time is to be allocated and which computer resources are available for use. But in order for the processor to act on these decisions, it must cease executing the OS program and execute other programs. Thus, the OS relinquishes control for the processor to do some "useful" work, then resumes control long enough to prepare the processor to do the next piece of work. The mechanisms involved in all this should become clear as the chapter proceeds.

Figure 2.2 suggests the main resources that are managed by the OS. A portion of the OS is in main memory. This includes the **kernel**, or **nucleus**, which contains the most frequently used functions in the OS and, at a given time, other portions of the OS currently in use. The remainder of main memory contains user and utility programs and data. The OS and the memory management hardware in the processor jointly control the allocation of main memory, as we shall see. The OS decides when an I/O device can be used by a program in execution, and controls access to and use of files. The processor itself is a resource, and the OS must determine how much processor time is to be devoted to the execution of a particular user program.

## Ease of Evolution of an Operating System

A major OS will evolve over time for a number of reasons:

- **Hardware upgrades plus new types of hardware:** For example, early versions of UNIX and the Macintosh OS did not employ a paging mechanism because they were run on processors without paging hardware.[1] Subsequent versions of these operating systems were modified to exploit paging capabilities. Also, the use of graphics terminals and page-mode terminals instead of line-at-a-time scroll mode terminals affects OS design. For example, a graphics terminal typically allows the user to view several applications at the same time through "windows" on the screen. This requires more sophisticated support in the OS.

[1]Paging will be introduced briefly later in this chapter, and will be discussed in detail in Chapter 7.

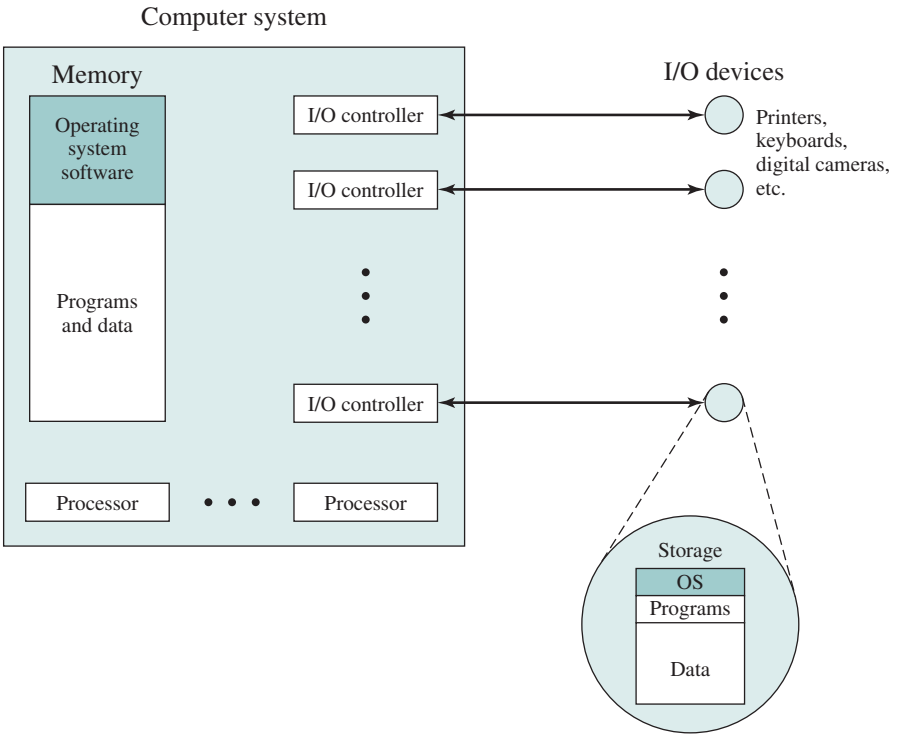



**Figure 2.2** **The Operating System as Resource Manager**

- **New services:** In response to user demand or in response to the needs of system managers, the OS expands to offer new services. For example, if it is found to be difficult to maintain good performance for users with existing tools, new measurement and control tools may be added to the OS.
- **Fixes:** Any OS has faults. These are discovered over the course of time and fixes are made. Of course, the fix may introduce new faults.

The need to regularly update an OS places certain requirements on its design. An obvious statement is that the system should be modular in construction, with clearly defined interfaces between the modules, and that it should be well documented. For large programs, such as the typical contemporary OS, what might be referred to as straightforward modularization is inadequate [DENN80a]. That is, much more must be done than simply partitioning a program into modules. We will return to this topic later in this chapter.

## 2.2 THE EVOLUTION OF OPERATING SYSTEMS

In attempting to understand the key requirements for an OS and the significance of the major features of a contemporary OS, it is useful to consider how operating systems have evolved over the years.

## Serial Processing

With the earliest computers, from the late 1940s to the mid-1950s, the programmer interacted directly with the computer hardware; there was no OS. These computers were run from a console consisting of display lights, toggle switches, some form of input device, and a printer. Programs in machine code were loaded via the input device (e.g., a card reader). If an error halted the program, the error condition was indicated by the lights. If the program proceeded to a normal completion, the output appeared on the printer. These early systems presented two main problems:

- **Scheduling:** Most installations used a hardcopy sign-up sheet to reserve computer time. Typically, a user could sign up for a block of time in multiples of a half hour or so. A user might sign up for an hour and finish in 45 minutes; this would result in wasted computer processing time. On the other hand, the user might run into problems, not finish in the allotted time, and be forced to stop before resolving the problem.
- **Setup time:** A single program, called a **job**, could involve loading the compiler plus the high-level language program (source program) into memory, saving the compiled program (object program), then loading and linking together the object program and common functions. Each of these steps could involve mounting or dismounting tapes or setting up card decks. If an error occurred, the hapless user typically had to go back to the beginning of the setup sequence. Thus, a considerable amount of time was spent just in setting up the program to run.

This mode of operation could be termed *serial processing,* reflecting the fact that users have access to the computer in series. Over time, various system software tools were developed to attempt to make serial processing more efficient. These include libraries of common functions, linkers, loaders, debuggers, and I/O driver routines that were available as common software for all users.

## Simple Batch Systems

Early computers were very expensive, and therefore it was important to maximize processor utilization. The wasted time due to scheduling and setup time was unacceptable.

To improve utilization, the concept of a batch OS was developed. It appears that the first batch OS (and the first OS of any kind) was developed in the mid-1950s by General Motors for use on an IBM 701 [WEIZ81]. The concept was subsequently refined and implemented on the IBM 704 by a number of IBM customers. By the early 1960s, a number of vendors had developed batch operating systems for their computer systems. IBSYS, the IBM OS for the 7090/7094 computers, is particularly notable because of its widespread influence on other systems.

The central idea behind the simple batch-processing scheme is the use of a piece of software known as the **monitor**. With this type of OS, the user no longer has direct access to the processor. Instead, the user submits the job on cards or tape to a

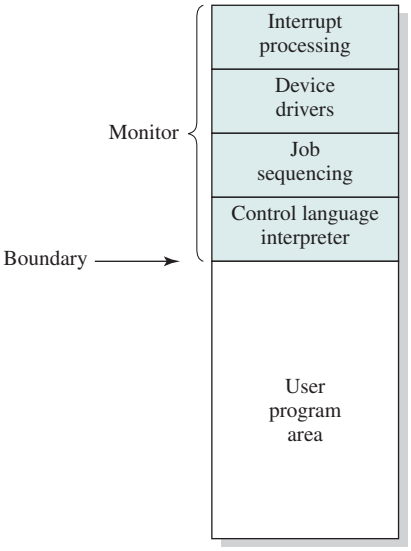



**Figure 2.3 Memory Layout for a Resident Monitor**

computer operator, who batches the jobs together sequentially and places the entire batch on an input device, for use by the monitor. Each program is constructed to branch back to the monitor when it completes processing, at which point the monitor automatically begins loading the next program.

To understand how this scheme works, let us look at it from two points of view: that of the monitor, and that of the processor.

- **Monitor point of view:** The monitor controls the sequence of events. For this to be so, much of the monitor must always be in main memory and available for execution (see Figure 2.3). That portion is referred to as the **resident monitor**. The rest of the monitor consists of utilities and common functions that are loaded as subroutines to the user program at the beginning of any job that requires them. The monitor reads in jobs one at a time from the input device (typically a card reader or magnetic tape drive). As it is read in, the current job is placed in the user program area, and control is passed to this job. When the job is completed, it returns control to the monitor, which immediately reads in the next job. The results of each job are sent to an output device, such as a printer, for delivery to the user.
- **Processor point of view:** At a certain point, the processor is executing instructions from the portion of main memory containing the monitor. These instructions cause the next job to be read into another portion of main memory. Once a job has been read in, the processor will encounter a branch instruction in the monitor that instructs the processor to continue execution at the start of

the user program. The processor will then execute the instructions in the user program until it encounters an ending or error condition. Either event causes the processor to fetch its next instruction from the monitor program. Thus the phrase "control is passed to a job" simply means the processor is now fetching and executing instructions in a user program, and "control is returned to the monitor" means the processor is now fetching and executing instructions from the monitor program.

The monitor performs a scheduling function: a batch of jobs is queued up, and jobs are executed as rapidly as possible, with no intervening idle time. The monitor improves job setup time as well. With each job, instructions are included in a primitive form of **job control language (JCL)**. This is a special type of programming language used to provide instructions to the monitor. A simple example is that of a user submitting a program written in the programming language FORTRAN plus some data to be used by the program. All FORTRAN instructions and data are on a separate punched card or a separate record on tape. In addition to FORTRAN and data lines, the job includes job control instructions, which are denoted by the beginning $. The overall format of the job looks like this:

```
$JOB
$FTN
.
.          } FORTRAN instructions
.
$LOAD
$RUN
.
.          } Data
.
$END
```

To execute this job, the monitor reads the `$FTN` line and loads the appropriate language compiler from its mass storage (usually tape). The compiler translates the user's program into object code, which is stored in memory or mass storage. If it is stored in memory, the operation is referred to as "compile, load, and go." If it is stored on tape, then the `$LOAD` instruction is required. This instruction is read by the monitor, which regains control after the compile operation. The monitor invokes the loader, which loads the object program into memory (in place of the compiler) and transfers control to it. In this manner, a large segment of main memory can be shared among different subsystems, although only one such subsystem could be executing at a time.

During the execution of the user program, any input instruction causes one line of data to be read. The input instruction in the user program causes an input routine that is part of the OS to be invoked. The input routine checks to make sure that the program does not accidentally read in a JCL line. If this happens, an error occurs and control transfers to the monitor. At the completion of the user job, the monitor will

scan the input lines until it encounters the next JCL instruction. Thus, the system is protected against a program with too many or too few data lines.

The monitor, or batch OS, is simply a computer program. It relies on the ability of the processor to fetch instructions from various portions of main memory to alternately seize and relinquish control. Certain other hardware features are also desirable:

- **Memory protection:** While the user program is executing, it must not alter the memory area containing the monitor. If such an attempt is made, the processor hardware should detect an error and transfer control to the monitor. The monitor would then abort the job, print out an error message, and load in the next job.
- **Timer:** A timer is used to prevent a single job from monopolizing the system. The timer is set at the beginning of each job. If the timer expires, the user program is stopped, and control returns to the monitor.
- **Privileged instructions:** Certain machine level instructions are designated as privileged and can be executed only by the monitor. If the processor encounters such an instruction while executing a user program, an error occurs causing control to be transferred to the monitor. Among the privileged instructions are I/O instructions, so that the monitor retains control of all I/O devices. This prevents, for example, a user program from accidentally reading job control instructions from the next job. If a user program wishes to perform I/O, it must request that the monitor perform the operation for it.
- **Interrupts:** Early computer models did not have this capability. This feature gives the OS more flexibility in relinquishing control to, and regaining control from, user programs.

Considerations of memory protection and privileged instructions lead to the concept of modes of operation. A user program executes in a **user mode**, in which certain areas of memory are protected from the user's use, and in which certain instructions may not be executed. The monitor executes in a system mode, or what has come to be called **kernel mode**, in which privileged instructions may be executed, and in which protected areas of memory may be accessed.

Of course, an OS can be built without these features. But computer vendors quickly learned that the results were chaos, and so even relatively primitive batch operating systems were provided with these hardware features.

With a batch OS, processor time alternates between execution of user programs and execution of the monitor. There have been two sacrifices: Some main memory is now given over to the monitors and some processor time is consumed by the monitor. Both of these are forms of overhead. Despite this overhead, the simple batch system improves utilization of the computer.

## Multiprogrammed Batch Systems

Even with the automatic job sequencing provided by a simple batch OS, the processor is often idle. The problem is I/O devices are slow compared to the processor.

| | |
|---|---|
| Read one record from file | 15 $\mu s$ |
| Execute 100 instructions | 1 $\mu s$ |
| Write one record to file | 15 $\mu s$ |
| Total | 31 $\mu s$ |

$$\text{Percent CPU utilization} = \frac{1}{31} = 0.032 = 3.2\%$$

**Figure 2.4 System Utilization Example**

Figure 2.4 details a representative calculation. The calculation concerns a program that processes a file of records and performs, on average, 100 machine instructions per record. In this example, the computer spends over 96% of its time waiting for I/O devices to finish transferring data to and from the file. Figure 2.5a illustrates this situation, where we have a single program, referred to as uniprogramming. The processor

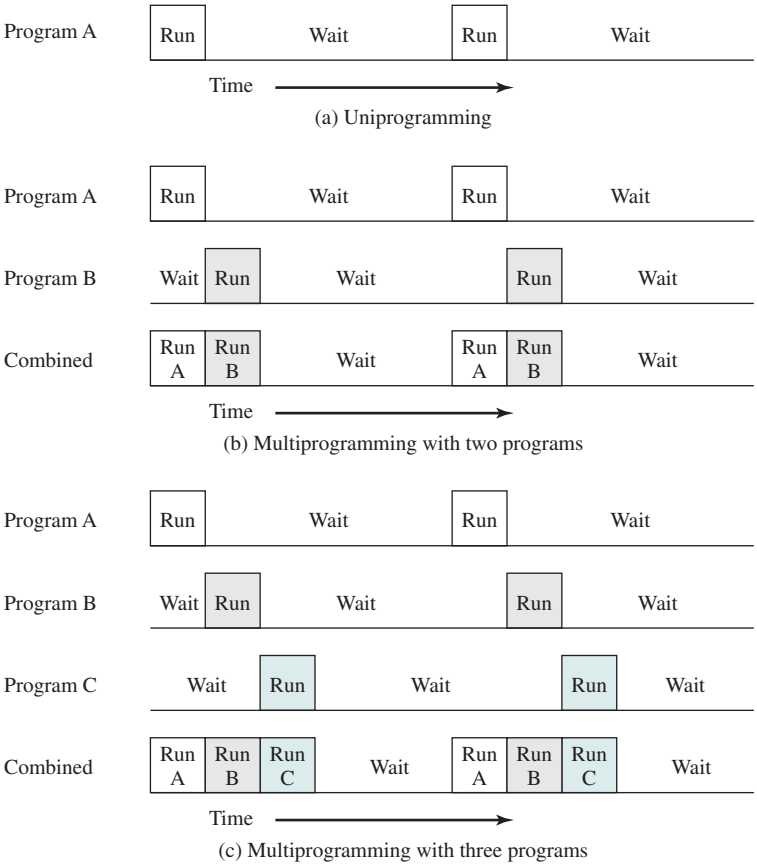



**Figure 2.5 Multiprogramming Example**

**Table 2.1** Sample Program Execution Attributes

| | JOB1 | JOB2 | JOB3 |
|---|---|---|---|
| **Type of job** | Heavy compute | Heavy I/O | Heavy I/O |
| **Duration** | 5 min | 15 min | 10 min |
| **Memory required** | 50 M | 100 M | 75 M |
| **Need disk?** | No | No | Yes |
| **Need terminal?** | No | Yes | No |
| **Need printer?** | No | No | Yes |

spends a certain amount of time executing, until it reaches an I/O instruction. It must then wait until that I/O instruction concludes before proceeding.

This inefficiency is not necessary. We know there must be enough memory to hold the OS (resident monitor) and one user program. Suppose there is room for the OS and two user programs. When one job needs to wait for I/O, the processor can switch to the other job, which is likely not waiting for I/O (see Figure 2.5b). Furthermore, we might expand memory to hold three, four, or more programs and switch among all of them (see Figure 2.5c). The approach is known as **multiprogramming**, or **multitasking**. It is the central theme of modern operating systems.

To illustrate the benefit of multiprogramming, we give a simple example. Consider a computer with 250 Mbytes of available memory (not used by the OS), a disk, a terminal, and a printer. Three programs, JOB1, JOB2, and JOB3, are submitted for execution at the same time, with the attributes listed in Table 2.1. We assume minimal processor requirements for JOB2 and JOB3, and continuous disk and printer use by JOB3. For a simple batch environment, these jobs will be executed in sequence. Thus, JOB1 completes in 5 minutes. JOB2 must wait until the 5 minutes are over, then completes 15 minutes after that. JOB3 begins after 20 minutes and completes at 30 minutes from the time it was initially submitted. The average resource utilization, throughput, and response times are shown in the uniprogramming column of Table 2.2. Device-by-device utilization is illustrated in Figure 2.6a. It is evident that there is gross underutilization for all resources when averaged over the required 30-minute time period.

Now suppose the jobs are run concurrently under a multiprogramming OS. Because there is little resource contention between the jobs, all three can run in

**Table 2.2** Effects of Multiprogramming on Resource Utilization

| | Uniprogramming | Multiprogramming |
|---|---|---|
| **Processor use** | 20% | 40% |
| **Memory use** | 33% | 67% |
| **Disk use** | 33% | 67% |
| **Printer use** | 33% | 67% |
| **Elapsed time** | 30 min | 15 min |
| **Throughput** | 6 jobs/hr | 12 jobs/hr |
| **Mean response time** | 18 min | 10 min |

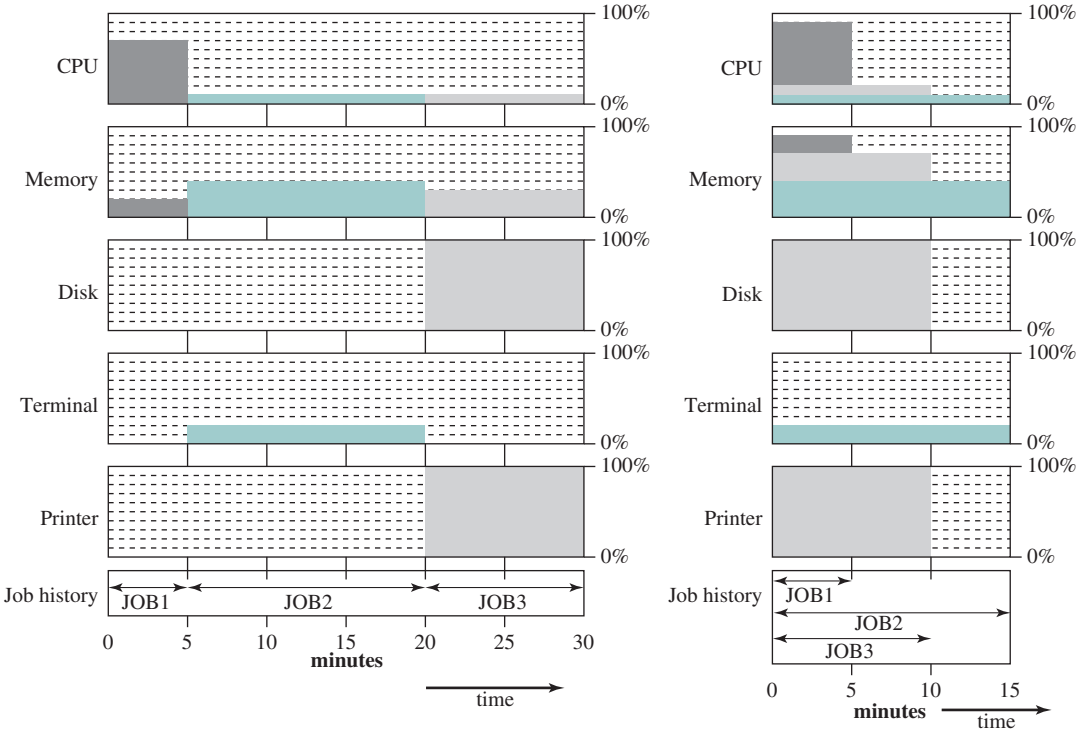



**Figure 2.6 Utilization Histograms**

nearly minimum time while coexisting with the others in the computer (assuming JOB2 and JOB3 are allotted enough processor time to keep their input and output operations active). JOB1 will still require 5 minutes to complete, but at the end of that time, JOB2 will be one-third finished and JOB3 half-finished. All three jobs will have finished within 15 minutes. The improvement is evident when examining the multiprogramming column of Table 2.2, obtained from the histogram shown in Figure 2.6b.

As with a simple batch system, a multiprogramming batch system must rely on certain computer hardware features. The most notable additional feature that is useful for multiprogramming is the hardware that supports I/O interrupts and DMA (direct memory access). With interrupt-driven I/O or DMA, the processor can issue an I/O command for one job and proceed with the execution of another job while the I/O is carried out by the device controller. When the I/O operation is complete, the processor is interrupted and control is passed to an interrupt-handling program in the OS. The OS will then pass control to another job after the interrupt is handled.

Multiprogramming operating systems are fairly sophisticated compared to single-program, or **uniprogramming**, systems. To have several jobs ready to run, they must be kept in main memory, requiring some form of **memory management**. In addition, if several jobs are ready to run, the processor must decide which one to run, and this decision requires an algorithm for scheduling. These concepts will be discussed later in this chapter.

## Time-Sharing Systems

With the use of multiprogramming, **batch processing** can be quite efficient. However, for many jobs, it is desirable to provide a mode in which the user interacts directly with the computer. Indeed, for some jobs, such as transaction processing, an interactive mode is essential.

Today, the requirement for an interactive computing facility can be, and often is, met by the use of a dedicated personal computer or workstation. That option was not available in the 1960s, when most computers were big and costly. Instead, time sharing was developed.

Just as multiprogramming allows the processor to handle multiple batch jobs at a time, multiprogramming can also be used to handle multiple interactive jobs. In this latter case, the technique is referred to as **time sharing**, because processor time is shared among multiple users. In a time-sharing system, multiple users simultaneously access the system through terminals, with the OS interleaving the execution of each user program in a short burst or quantum of computation. Thus, if there are $n$ users actively requesting service at one time, each user will only see on the average $1/n$ of the effective computer capacity, not counting OS overhead. However, given the relatively slow human reaction time, the response time on a properly designed system should be similar to that on a dedicated computer.

Both batch processing and time sharing use multiprogramming. The key differences are listed in Table 2.3.

One of the first time-sharing operating systems to be developed was the Compatible Time-Sharing System (CTSS) [CORB62], developed at MIT by a group known as Project MAC (Machine-Aided Cognition, or Multiple-Access Computers). The system was first developed for the IBM 709 in 1961 and later ported to IBM 7094.

Compared to later systems, CTSS is primitive. The system ran on a computer with 32,000 36-bit words of main memory, with the resident monitor consuming 5,000 of those. When control was to be assigned to an interactive user, the user's program and data were loaded into the remaining 27,000 words of main memory. A program was always loaded to start at the location of the 5,000th word; this simplified both the monitor and memory management. A system clock generated interrupts at a rate of approximately one every 0.2 seconds. At each clock interrupt, the OS regained control and could assign the processor to another user. This technique is known as **time slicing**. Thus, at regular time intervals, the current user would be preempted and another user loaded in. To preserve the old user program status for later resumption, the old user programs and data were written out to disk before the new user programs and data were read in. Subsequently, the old user program code and data were restored in main memory when that program was next given a turn.

**Table 2.3** Batch Multiprogramming versus Time Sharing

| | Batch Multiprogramming | Time Sharing |
|---|---|---|
| Principal objective | Maximize processor use | Minimize response time |
| Source of directives to operating system | Job control language commands provided with the job | Commands entered at the terminal |

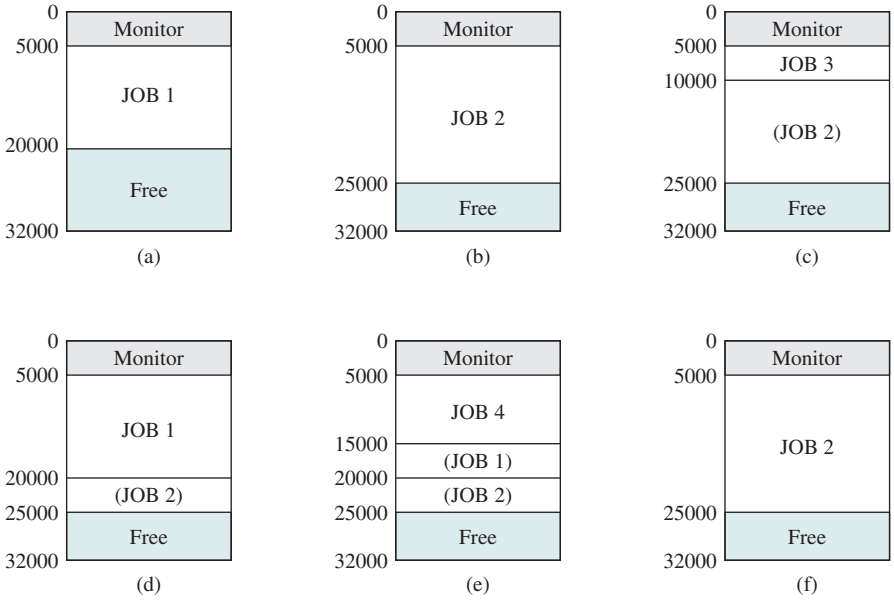



**Figure 2.7 CTSS Operation**

To minimize disk traffic, user memory was only written out when the incoming program would overwrite it. This principle is illustrated in Figure 2.7. Assume there are four interactive users with the following memory requirements, in words:

- JOB1: 15,000
- JOB2: 20,000
- JOB3: 5,000
- JOB4: 10,000

Initially, the monitor loads JOB1 and transfers control to it (Figure 2.7a). Later, the monitor decides to transfer control to JOB2. Because JOB2 requires more memory than JOB1, JOB1 must be written out first, and then JOB2 can be loaded (Figure 2.7b). Next, JOB3 is loaded in to be run. However, because JOB3 is smaller than JOB2, a portion of JOB2 can remain in memory, reducing disk write time (Figure 2.7c). Later, the monitor decides to transfer control back to JOB1. An additional portion of JOB2 must be written out when JOB1 is loaded back into memory (Figure 2.7d). When JOB4 is loaded, part of JOB1 and the portion of JOB2 remaining in memory are retained (Figure 2.7e). At this point, if either JOB1 or JOB2 is activated, only a partial load will be required. In this example, it is JOB2 that runs next. This requires that JOB4 and the remaining resident portion of JOB1 be written out, and the missing portion of JOB2 be read in (Figure 2.7f).

The CTSS approach is primitive compared to present-day time sharing, but it was effective. It was extremely simple, which minimized the size of the monitor. Because a job was always loaded into the same locations in memory, there was no need for relocation techniques at load time (discussed subsequently). The technique

of only writing out what was necessary minimized disk activity. Running on the 7094, CTSS supported a maximum of 32 users.

Time sharing and multiprogramming raise a host of new problems for the OS. If multiple jobs are in memory, then they must be protected from interfering with each other by, for example, modifying each other's data. With multiple interactive users, the file system must be protected so only authorized users have access to a particular file. The contention for resources, such as printers and mass storage devices, must be handled. These and other problems, with possible solutions, will be encountered throughout this text.

## 2.3 MAJOR ACHIEVEMENTS

Operating systems are among the most complex pieces of software ever developed. This reflects the challenge of trying to meet the difficult and in some cases competing objectives of convenience, efficiency, and ability to evolve. [DENN80a] proposes that there have been four major theoretical advances in the development of operating systems:

- Processes
- Memory management
- Information protection and security
- Scheduling and resource management

Each advance is characterized by principles, or abstractions, developed to meet difficult practical problems. Taken together, these four areas span many of the key design and implementation issues of modern operating systems. The brief review of these four areas in this section serves as an overview of much of the rest of the text.

### The Process

Central to the design of operating systems is the concept of *process*. This term was first used by the designers of Multics in the 1960s [DALE68]. It is a somewhat more general term than *job*. Many definitions have been given for the term *process*, including:

- A program in execution.
- An instance of a program running on a computer.
- The entity that can be assigned to and executed on a processor.
- A unit of activity characterized by a single sequential thread of execution, a current state, and an associated set of system resources.

This concept should become clearer as we proceed.

Three major lines of computer system development created problems in timing and synchronization that contributed to the development of the concept of the process: multiprogramming batch operation, time-sharing, and real-time transaction systems. As we have seen, multiprogramming was designed to keep the processor

and I/O devices, including storage devices, simultaneously busy to achieve maximum efficiency. The key mechanism is this: In response to signals indicating the completion of I/O transactions, the processor is switched among the various programs residing in main memory.

A second line of development was general-purpose time sharing. Here, the key design objective is to be responsive to the needs of the individual user and yet, for cost reasons, be able to support many users simultaneously. These goals are compatible because of the relatively slow reaction time of the user. For example, if a typical user needs an average of 2 seconds of processing time per minute, then close to 30 such users should be able to share the same system without noticeable interference. Of course, OS overhead must be factored into such calculations.

A third important line of development has been real-time transaction processing systems. In this case, a number of users are entering queries or updates against a database. An example is an airline reservation system. The key difference between the transaction processing system and the time-sharing system is that the former is limited to one or a few applications, whereas users of a time-sharing system can engage in program development, job execution, and the use of various applications. In both cases, system response time is paramount.

The principal tool available to system programmers in developing the early multiprogramming and multiuser interactive systems was the interrupt. The activity of any job could be suspended by the occurrence of a defined event, such as an I/O completion. The processor would save some sort of context (e.g., program counter and other registers) and branch to an interrupt-handling routine which would determine the nature of the interrupt, process the interrupt, then resume user processing with the interrupted job or some other job.

The design of the system software to coordinate these various activities turned out to be remarkably difficult. With many jobs in progress at any one time, each of which involved numerous steps to be performed in sequence, it became impossible to analyze all of the possible combinations of sequences of events. In the absence of some systematic means of coordination and cooperation among activities, programmers resorted to ad hoc methods based on their understanding of the environment that the OS had to control. These efforts were vulnerable to subtle programming errors whose effects could be observed only when certain relatively rare sequences of actions occurred. These errors were difficult to diagnose, because they needed to be distinguished from application software errors and hardware errors. Even when the error was detected, it was difficult to determine the cause, because the precise conditions under which the errors appeared were very hard to reproduce. In general terms, there are four main causes of such errors [DENN80a]:

- **Improper synchronization:** It is often the case that a routine must be suspended awaiting an event elsewhere in the system. For example, a program that initiates an I/O read must wait until the data are available in a buffer before proceeding. In such cases, a signal from some other routine is required. Improper design of the signaling mechanism can result in signals being lost or duplicate signals being received.
- **Failed mutual exclusion:** It is often the case that more than one user or program will attempt to make use of a shared resource at the same time. For example,

two users may attempt to edit the same file at the same time. If these accesses are not controlled, an error can occur. There must be some sort of mutual exclusion mechanism that permits only one routine at a time to perform an update against the file. The implementation of such mutual exclusion is difficult to verify as being correct under all possible sequences of events.

- **Nondeterminate program operation:** The results of a particular program normally should depend only on the input to that program, and not on the activities of other programs in a shared system. But when programs share memory, and their execution is interleaved by the processor, they may interfere with each other by overwriting common memory areas in unpredictable ways. Thus, the order in which various programs are scheduled may affect the outcome of any particular program.
- **Deadlocks:** It is possible for two or more programs to be hung up waiting for each other. For example, two programs may each require two I/O devices to perform some operation (e.g., disk to tape copy). One of the programs has seized control of one of the devices, and the other program has control of the other device. Each is waiting for the other program to release the desired resource. Such a deadlock may depend on the chance timing of resource allocation and release.

What is needed to tackle these problems is a systematic way to monitor and control the various programs executing on the processor. The concept of the process provides the foundation. We can think of a process as consisting of three components:

1. An executable program
2. The associated data needed by the program (variables, work space, buffers, etc.)
3. The execution context of the program

This last element is essential. The **execution context**, or **process state**, is the internal data by which the OS is able to supervise and control the process. This internal information is separated from the process, because the OS has information not permitted to the process. The context includes all of the information the OS needs to manage the process, and the processor needs to execute the process properly. The context includes the contents of the various processor registers, such as the program counter and data registers. It also includes information of use to the OS, such as the priority of the process and whether the process is waiting for the completion of a particular I/O event.

Figure 2.8 indicates a way in which processes may be managed. Two processes, A and B, exist in portions of main memory. That is, a block of memory is allocated to each process that contains the program, data, and context information. Each process is recorded in a process list built and maintained by the OS. The process list contains one entry for each process, which includes a pointer to the location of the block of memory that contains the process. The entry may also include part or all of the execution context of the process. The remainder of the execution context is stored elsewhere, perhaps with the process itself (as indicated in Figure 2.8) or frequently in a separate region of memory. The process index register contains the index into the process list of the process currently controlling the processor. The program counter

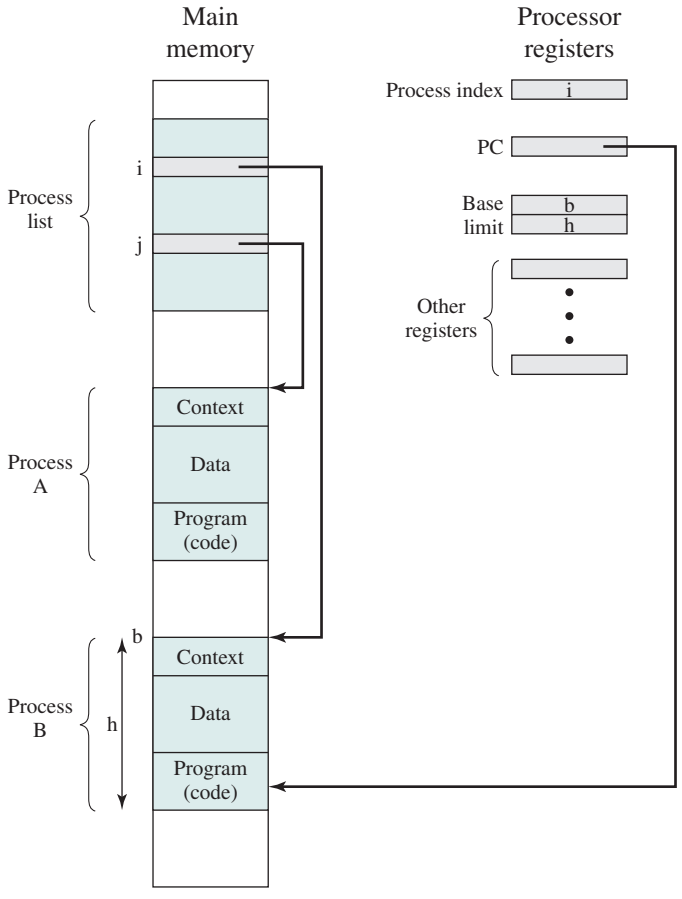



**Figure 2.8** **Typical Process Implementation**

points to the next instruction in that process to be executed. The base and limit registers define the region in memory occupied by the process: The base register is the starting address of the region of memory, and the limit is the size of the region (in bytes or words). The program counter and all data references are interpreted relative to the base register and must not exceed the value in the limit register. This prevents interprocess interference.

In Figure 2.8, the process index register indicates that process B is executing. Process A was previously executing but has been temporarily interrupted. The contents of all the registers at the moment of A's interruption were recorded in its execution context. Later, the OS can perform a process switch and resume the execution of process A. The process switch consists of saving the context of B and restoring the context of A. When the program counter is loaded with a value pointing into A's program area, process A will automatically resume execution.

Thus, the process is realized as a data structure. A process can either be executing or awaiting execution. The entire **state** of the process at any instant is contained in its context. This structure allows the development of powerful techniques for ensuring

coordination and cooperation among processes. New features can be designed and incorporated into the OS (e.g., priority) by expanding the context to include any new information needed to support the feature. Throughout this book, we will see a number of examples where this process structure is employed to solve the problems raised by multiprogramming and resource sharing.

A final point, which we introduce briefly here, is the concept of **thread**. In essence, a single process, which is assigned certain resources, can be broken up into multiple, concurrent threads that execute cooperatively to perform the work of the process. This introduces a new level of parallel activity to be managed by the hardware and software.

## Memory Management

The needs of users can be met best by a computing environment that supports modular programming and the flexible use of data. System managers need efficient and orderly control of storage allocation. The OS, to satisfy these requirements, has five principal storage management responsibilities:

1. **Process isolation:** The OS must prevent independent processes from interfering with each other's memory, both data and instructions.
2. **Automatic allocation and management:** Programs should be dynamically allocated across the memory hierarchy as required. Allocation should be transparent to the programmer. Thus, the programmer is relieved of concerns relating to memory limitations, and the OS can achieve efficiency by assigning memory to jobs only as needed.
3. **Support of modular programming:** Programmers should be able to define program modules, and to dynamically create, destroy, and alter the size of modules.
4. **Protection and access control:** Sharing of memory, at any level of the memory hierarchy, creates the potential for one program to address the memory space of another. This is desirable when sharing is needed by particular applications. At other times, it threatens the integrity of programs and even of the OS itself. The OS must allow portions of memory to be accessible in various ways by various users.
5. **Long-term storage:** Many application programs require means for storing information for extended periods of time, after the computer has been powered down.

Typically, operating systems meet these requirements with virtual memory and file system facilities. The file system implements a long-term store, with information stored in named objects called files. The file is a convenient concept for the programmer, and is a useful unit of access control and protection for the OS.

**Virtual memory** is a facility that allows programs to address memory from a logical point of view, without regard to the amount of main memory physically available. Virtual memory was conceived to meet the requirement of having multiple user jobs concurrently reside in main memory, so there would not be a hiatus between the execution of successive processes while one process was written out to secondary store and the successor process was read in. Because processes vary in size, if the processor switches among a number of processes, it is difficult to pack them compactly

into main memory. Paging systems were introduced, which allow processes to be comprised of a number of fixed-size blocks, called pages. A program references a word by means of a **virtual address** consisting of a page number and an offset within the page. Each page of a process may be located anywhere in main memory. The paging system provides for a dynamic mapping between the virtual address used in the program and a **real address**, or physical address, in main memory.

With dynamic mapping hardware available, the next logical step was to eliminate the requirement that all pages of a process simultaneously reside in main memory. All the pages of a process are maintained on disk. When a process is executing, some of its pages are in main memory. If reference is made to a page that is not in main memory, the memory management hardware detects this and, in coordination with the OS, arranges for the missing page to be loaded. Such a scheme is referred to as **virtual memory** and is depicted in Figure 2.9.

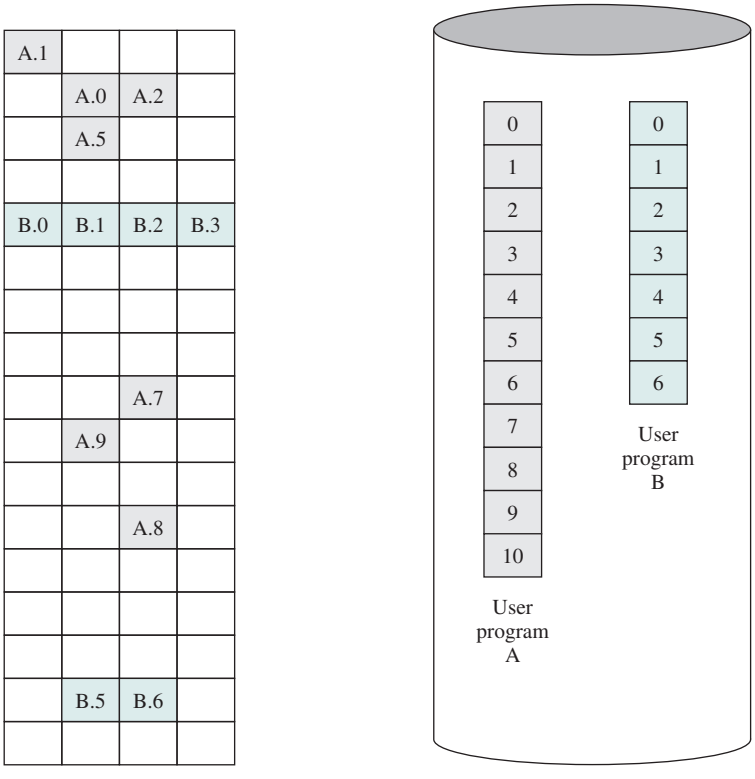



**Figure 2.9 Virtual Memory Concepts**

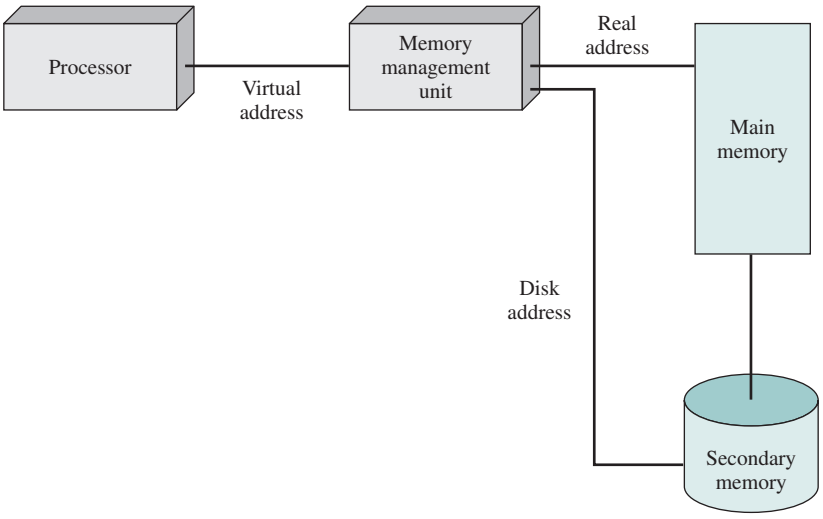



**Figure 2.10 Virtual Memory Addressing**

The processor hardware, together with the OS, provides the user with a "virtual processor" that has access to a virtual memory. This memory may be a linear address space or a collection of segments, which are variable-length blocks of contiguous addresses. In either case, programming language instructions can reference program and data locations in the virtual memory area. Process isolation can be achieved by giving each process a unique, nonoverlapping virtual memory. Memory sharing can be achieved by overlapping portions of two virtual memory spaces. Files are maintained in a long-term store. Files and portions of files may be copied into the virtual memory for manipulation by programs.

Figure 2.10 highlights the addressing concerns in a virtual memory scheme. Storage consists of directly addressable (by machine instructions) main memory, and lower-speed auxiliary memory that is accessed indirectly by loading blocks into main memory. Address translation hardware (a memory management unit) is interposed between the processor and memory. Programs reference locations using virtual addresses, which are mapped into real main memory addresses. If a reference is made to a virtual address not in real memory, then a portion of the contents of real memory is swapped out to auxiliary memory and the desired block of data is swapped in. During this activity, the process that generated the address reference must be suspended. The OS designer needs to develop an address translation mechanism that generates little overhead, and a storage allocation policy that minimizes the traffic between memory levels.

## Information Protection and Security

The growth in the use of time-sharing systems and, more recently, computer networks has brought with it a growth in concern for the protection of information. The nature of the threat that concerns an organization will vary greatly depending on the circumstances. However, there are some general-purpose tools that can be built into

computers and operating systems that support a variety of protection and security mechanisms. In general, we are concerned with the problem of controlling access to computer systems and the information stored in them.

Much of the work in security and protection as it relates to operating systems can be roughly grouped into four categories:

1. **Availability:** Concerned with protecting the system against interruption.
2. **Confidentiality:** Assures that users cannot read data for which access is unauthorized.
3. **Data integrity:** Protection of data from unauthorized modification.
4. **Authenticity:** Concerned with the proper verification of the identity of users and the validity of messages or data.

## Scheduling and Resource Management

A key responsibility of the OS is to manage the various resources available to it (main memory space, I/O devices, processors) and to schedule their use by the various active processes. Any resource allocation and scheduling policy must consider three factors:

1. **Fairness:** Typically, we would like all processes that are competing for the use of a particular resource to be given approximately equal and fair access to that resource. This is especially so for jobs of the same class, that is, jobs of similar demands.
2. **Differential responsiveness:** On the other hand, the OS may need to discriminate among different classes of jobs with different service requirements. The OS should attempt to make allocation and scheduling decisions to meet the total set of requirements. The OS should also make these decisions dynamically. For example, if a process is waiting for the use of an I/O device, the OS may wish to schedule that process for execution as soon as possible; the process can then immediately use the device, then release it for later demands from other processes.
3. **Efficiency:** The OS should attempt to maximize throughput, minimize response time, and, in the case of time sharing, accommodate as many users as possible. These criteria conflict; finding the right balance for a particular situation is an ongoing problem for OS research.

Scheduling and resource management are essentially operations-research problems and the mathematical results of that discipline can be applied. In addition, measurement of system activity is important to be able to monitor performance and make adjustments.

Figure 2.11 suggests the major elements of the OS involved in the scheduling of processes and the allocation of resources in a multiprogramming environment. The OS maintains a number of queues, each of which is simply a list of processes waiting for some resource. The short-term queue consists of processes that are in main memory (or at least an essential minimum portion of each is in main memory) and are ready to run as soon as the processor is made available. Any one of these processes could use the processor next. It is up to the short-term scheduler,

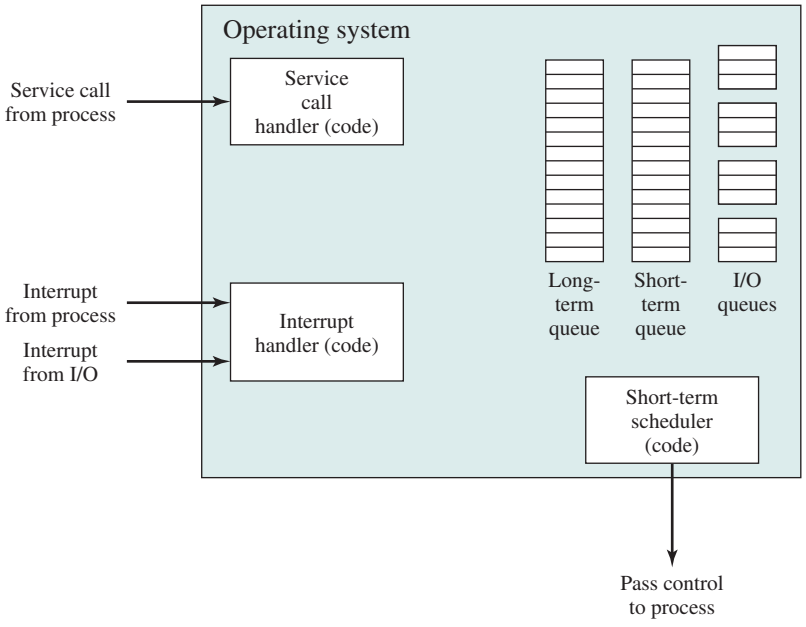



**Figure 2.11** **Key Elements of an Operating System for Multiprogramming**

or dispatcher, to pick one. A common strategy is to give each process in the queue some time in turn; this is referred to as a **round-robin** technique. In effect, the round-robin technique employs a circular queue. Another strategy is to assign priority levels to the various processes, with the scheduler selecting processes in priority order.

The long-term queue is a list of new jobs waiting to use the processor. The OS adds jobs to the system by transferring a process from the long-term queue to the short-term queue. At that time, a portion of main memory must be allocated to the incoming process. Thus, the OS must be sure that it does not overcommit memory or processing time by admitting too many processes to the system. There is an I/O queue for each I/O device. More than one process may request the use of the same I/O device. All processes waiting to use each device are lined up in that device's queue. Again, the OS must determine which process to assign to an available I/O device.

The OS receives control of the processor at the interrupt handler if an interrupt occurs. A process may specifically invoke some OS service, such as an I/O device handler, by means of a service call. In this case, a service call handler is the entry point into the OS. In any case, once the interrupt or service call is handled, the short-term scheduler is invoked to pick a process for execution.

The foregoing is a functional description; details and modular design of this portion of the OS will differ in various systems. Much of the research and development effort in operating systems has been directed at picking algorithms and data structures for this function that provide fairness, differential responsiveness, and efficiency.

## 2.4 DEVELOPMENTS LEADING TO MODERN OPERATING SYSTEMS

Over the years, there has been a gradual evolution of OS structure and capabilities. However, in recent years, a number of new design elements have been introduced into both new operating systems and new releases of existing operating systems that create a major change in the nature of operating systems. These modern operating systems respond to new developments in hardware, new applications, and new security threats. Among the key hardware drivers are multiprocessor systems, greatly increased processor speed, high-speed network attachments, and increasing size and variety of memory storage devices. In the application arena, multimedia applications, Internet and Web access, and client/server computing have influenced OS design. With respect to security, Internet access to computers has greatly increased the potential threat, and increasingly sophisticated attacks (such as viruses, worms, and hacking techniques) have had a profound impact on OS design.

The rate of change in the demands on operating systems requires not just modifications and enhancements to existing architectures, but new ways of organizing the OS. A wide range of different approaches and design elements has been tried in both experimental and commercial operating systems, but much of the work fits into the following categories:

- Microkernel architecture
- Multithreading
- Symmetric multiprocessing
- Distributed operating systems
- Object-oriented design

Until recently, most operating systems featured a large **monolithic kernel**. Most of what is thought of as OS functionality is provided in these large kernels, including scheduling, file system, networking, device drivers, memory management, and more. Typically, a monolithic kernel is implemented as a single process, with all elements sharing the same address space. A **microkernel** architecture assigns only a few essential functions to the kernel, including address space management, interprocess communication (IPC), and basic scheduling. Other OS services are provided by processes, sometimes called servers, that run in user mode and are treated like any other application by the microkernel. This approach decouples kernel and server development. Servers may be customized to specific application or environment requirements. The microkernel approach simplifies implementation, provides flexibility, and is well suited to a distributed environment. In essence, a microkernel interacts with local and remote server processes in the same way, facilitating construction of distributed systems.

**Multithreading** is a technique in which a process, executing an application, is divided into threads that can run concurrently. We can make the following distinction:

- **Thread:** A dispatchable unit of work. It includes a processor context (which includes the program counter and stack pointer) and its own data area for a

stack (to enable subroutine branching). A thread executes sequentially and is interruptible so the processor can turn to another thread.

- **Process:** A collection of one or more threads and associated system resources (such as memory containing both code and data, open files, and devices). This corresponds closely to the concept of a program in execution. By breaking a single application into multiple threads, the programmer has great control over the modularity of the application and the timing of application-related events.

Multithreading is useful for applications that perform a number of essentially independent **tasks** that do not need to be serialized. An example is a database server that listens for and processes numerous client requests. With multiple threads running within the same process, switching back and forth among threads involves less processor overhead than a major process switch between different processes. Threads are also useful for structuring processes that are part of the OS kernel, as will be described in subsequent chapters.

**Symmetric multiprocessing (SMP)** is a term that refers to a computer hardware architecture (described in Chapter 1) and also to the OS behavior that exploits that architecture. The OS of an SMP schedules processes or threads across all of the processors. SMP has a number of potential advantages over uniprocessor architecture, including the following:

- **Performance:** If the work to be done by a computer can be organized so some portions of the work can be done in parallel, then a system with multiple processors will yield greater performance than one with a single processor of the same type. This is illustrated in Figure 2.12. With multiprogramming, only one process can execute at a time; meanwhile, all other processes are waiting for the processor. With multiprocessing, more than one process can be running simultaneously, each on a different processor.
- **Availability:** In a symmetric multiprocessor, because all processors can perform the same functions, the failure of a single processor does not halt the system. Instead, the system can continue to function at reduced performance.
- **Incremental growth:** A user can enhance the performance of a system by adding an additional processor.
- **Scaling:** Vendors can offer a range of products with different price and performance characteristics based on the number of processors configured in the system.

It is important to note that these are potential, rather than guaranteed, benefits. The OS must provide tools and functions to exploit the parallelism in an SMP system.

Multithreading and SMP are often discussed together, but the two are independent facilities. Even on a uniprocessor system, multithreading is useful for structuring applications and kernel processes. An SMP system is useful even for nonthreaded processes, because several processes can run in parallel. However, the two facilities complement each other, and can be used effectively together.

An attractive feature of an SMP is that the existence of multiple processors is transparent to the user. The OS takes care of scheduling of threads or processes

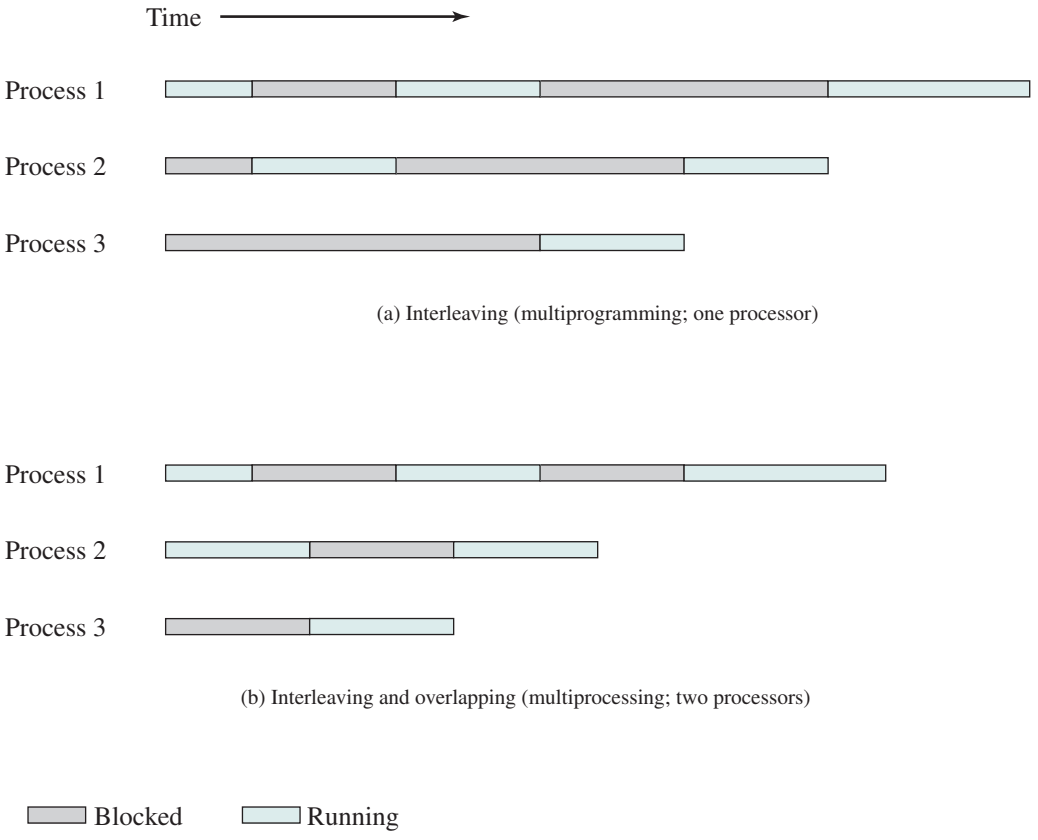



**Figure 2.12 Multiprogramming and Multiprocessing**

on individual processors and of synchronization among processors. This book discusses the scheduling and synchronization mechanisms used to provide the single-system appearance to the user. A different problem is to provide the appearance of a single system for a cluster of separate computers—a multicomputer system. In this case, we are dealing with a collection of computers, each with its own main memory, secondary memory, and other I/O modules. A **distributed operating system** provides the illusion of a single main memory space and a single secondary memory space, plus other unified access facilities, such as a distributed file system. Although clusters are becoming increasingly popular, and there are many cluster products on the market, the state of the art for distributed operating systems lags behind that of uniprocessor and SMP operating systems. We will examine such systems in Part Eight.

Another innovation in OS design is the use of object-oriented technologies. **Object-oriented design** lends discipline to the process of adding modular extensions to a small kernel. At the OS level, an object-based structure enables programmers to customize an OS without disrupting system integrity. Object orientation also eases the development of distributed tools and full-blown distributed operating systems.

## 2.5 FAULT TOLERANCE

Fault tolerance refers to the ability of a system or component to continue normal operation despite the presence of hardware or software faults. This typically involves some degree of redundancy. Fault tolerance is intended to increase the reliability of a system. Typically, increased fault tolerance (and therefore increased reliability) comes with a cost, either in financial terms or performance, or both. Thus, the extent adoption of fault tolerance measures must be determined by how critical the resource is.

### Fundamental Concepts

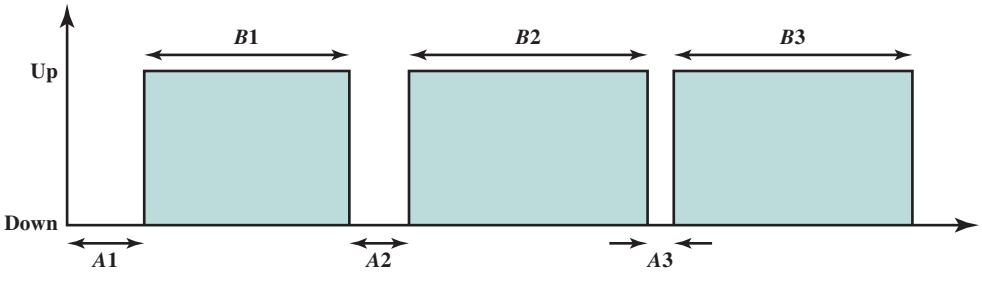
The three basic measures of the quality of the operation of a system that relate to fault tolerance are reliability, mean time to failure (MTTF), and availability. These concepts were developed with specific reference to hardware faults, but apply more generally to hardware and software faults.

The **reliability** $R(t)$ of a system is defined as the probability of its correct operation up to time $t$ given that the system was operating correctly at time $t = 0$. For computer systems and operating systems, the term *correct operation* means the correct execution of a set of programs, and the protection of data from unintended modification. The **mean time to failure (MTTF)** is defined as

$$\text{MTTF} = \int_0^\infty R(t)$$

The **mean time to repair (MTTR)** is the average time it takes to repair or replace a faulty element. Figure 2.13 illustrates the relationship between MTTF and MTTR.

The **availability** of a system or service is defined as the fraction of time the system is available to service users' requests. Equivalently, availability is the probability that an entity is operating correctly under given conditions at a given instant of time. The time during which the system is not available is called **downtime**; the time during




$$MTTF = \frac{B1 + B2 + B3}{3} \qquad MTTR = \frac{A1 + A2 + A3}{3}$$

**Figure 2.13** System Operational States

**Table 2.4** Availability Classes

| Class | Availability | Annual Downtime |
|---|---|---|
| Continuous | 1.0 | 0 |
| Fault tolerant | 0.99999 | 5 minutes |
| Fault resilient | 0.9999 | 53 minutes |
| High availability | 0.999 | 8.3 hours |
| Normal availability | 0.99–0.995 | 44–87 hours |

which the system is available is called **uptime**. The availability $A$ of a system can be expressed as follows:

$$A = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}}$$

Table 2.4 shows some commonly identified availability levels and the corresponding annual downtime.

Often, the mean uptime, which is MTTF, is a better indicator than availability. A small downtime and a small uptime combination may result in a high availability measure, but the users may not be able to get any service if the uptime is less than the time required to complete a service.

## Faults

The IEEE Standards Dictionary defines a **fault** as an erroneous hardware or software state resulting from component failure, operator error, physical interference from the environment, design error, program error, or data structure error. The standard also states that a fault manifests itself as (1) a defect in a hardware device or component; for example, a short circuit or broken wire, or (2) an incorrect step, process, or data definition in a computer program.

We can group faults into the following categories:

- **Permanent:** A fault that, after it occurs, is always present. The fault persists until the faulty component is replaced or repaired. Examples include disk head crashes, software bugs, and a burnt-out communications component.
- **Temporary:** A fault that is not present all the time for all operating conditions. Temporary faults can be further classified as follows:

  —**Transient:** A fault that occurs only once. Examples include bit transmission errors due to an impulse noise, power supply disturbances, and radiation that alters a memory bit.

  —**Intermittent:** A fault that occurs at multiple, unpredictable times. An example of an intermittent fault is one caused by a loose connection.

In general, fault tolerance is built into a system by adding redundancy. Methods of redundancy include the following:

- **Spatial (physical) redundancy:** Physical redundancy involves the use of multiple components that either perform the same function simultaneously, or are

configured so one component is available as a backup in case of the failure of another component. An example of the former is the use of multiple parallel circuitry with the majority result produced as output. An example of the latter is a backup name server on the Internet.

- **Temporal redundancy:** Temporal redundancy involves repeating a function or operation when an error is detected. This approach is effective with temporary faults, but not useful for permanent faults. An example is the retransmission of a block of data when an error is detected, such as is done with data link control protocols.
- **Information redundancy:** Information redundancy provides fault tolerance by replicating or coding data in such a way that bit errors can be both detected and corrected. An example is the error-control coding circuitry used with memory systems, and error-correction techniques used with RAID disks, as will be described in subsequent chapters.

## Operating System Mechanisms

A number of techniques can be incorporated into OS software to support fault tolerance. A number of examples will be evident throughout the book. The following list provides examples:

- **Process isolation:** As was mentioned earlier in this chapter, processes are generally isolated from one another in terms of main memory, file access, and flow of execution. The structure provided by the OS for managing processes provides a certain level of protection for other processes from a process that produces a fault.
- **Concurrency controls:** Chapters 5 and 6 will discuss some of the difficulties and faults that can occur when processes communicate or cooperate. These chapters will also discuss techniques used to ensure correct operation and to recover from fault conditions, such as deadlock.
- **Virtual machines:** Virtual machines, as will be discussed in Chapter 14, provide a greater degree of application isolation and hence fault isolation. Virtual machines can also be used to provide redundancy, with one virtual machine serving as a backup for another.
- **Checkpoints and rollbacks:** A checkpoint is a copy of an application's state saved in some storage that is immune to the failures under consideration. A rollback restarts the execution from a previously saved checkpoint. When a failure occurs, the application's state is rolled back to the previous checkpoint and restarted from there. This technique can be used to recover from transient as well as permanent hardware failures, and certain types of software failures. Database and transaction processing systems typically have such capabilities built in.

A much wider array of techniques could be discussed, but a full treatment of OS fault tolerance is beyond our current scope.

## 2.6 OS DESIGN CONSIDERATIONS FOR MULTIPROCESSOR AND MULTICORE

### Symmetric Multiprocessor OS Considerations

In an SMP system, the kernel can execute on any processor, and typically each processor does self-scheduling from the pool of available processes or threads. The kernel can be constructed as multiple processes or multiple threads, allowing portions of the kernel to execute in parallel. The SMP approach complicates the OS. The OS designer must deal with the complexity due to sharing resources (such as data structures) and coordinating actions (such as accessing devices) from multiple parts of the OS executing at the same time. Techniques must be employed to resolve and synchronize claims to resources.

An SMP operating system manages processor and other computer resources so the user may view the system in the same fashion as a multiprogramming uniprocessor system. A user may construct applications that use multiple processes or multiple threads within processes without regard to whether a single processor or multiple processors will be available. Thus, a multiprocessor OS must provide all the functionality of a multiprogramming system, plus additional features to accommodate multiple processors. The key design issues include the following:

- **Simultaneous concurrent processes or threads:** Kernel routines need to be reentrant to allow several processors to execute the same kernel code simultaneously. With multiple processors executing the same or different parts of the kernel, kernel tables and management structures must be managed properly to avoid data corruption or invalid operations.
- **Scheduling:** Any processor may perform scheduling, which complicates the task of enforcing a scheduling policy and assuring that corruption of the scheduler data structures is avoided. If kernel-level multithreading is used, then the opportunity exists to schedule multiple threads from the same process simultaneously on multiple processors. Multiprocessor scheduling will be examined in Chapter 10.
- **Synchronization:** With multiple active processes having potential access to shared address spaces or shared I/O resources, care must be taken to provide effective synchronization. Synchronization is a facility that enforces mutual exclusion and event ordering. A common synchronization mechanism used in multiprocessor operating systems is locks, and will be described in Chapter 5.
- **Memory management:** Memory management on a multiprocessor must deal with all of the issues found on uniprocessor computers, and will be discussed in Part Three. In addition, the OS needs to exploit the available hardware parallelism to achieve the best performance. The paging mechanisms on different processors must be coordinated to enforce consistency when several processors share a page or segment and to decide on page replacement. The reuse of physical pages is the biggest problem of concern; that is, it must be guaranteed that a physical page can no longer be accessed with its old contents before the page is put to a new use.

- **Reliability and fault tolerance:** The OS should provide graceful degradation in the face of processor failure. The scheduler and other portions of the OS must recognize the loss of a processor and restructure management tables accordingly.

Because multiprocessor OS design issues generally involve extensions to solutions to multiprogramming uniprocessor design problems, we do not treat multiprocessor operating systems separately. Rather, specific multiprocessor issues are addressed in the proper context throughout this book.

## Multicore OS Considerations

The considerations for multicore systems include all the design issues discussed so far in this section for SMP systems. But additional concerns arise. The issue is one of the scale of the potential parallelism. Current multicore vendors offer systems with ten or more cores on a single chip. With each succeeding processor technology generation, the number of cores and the amount of shared and dedicated cache memory increases, so we are now entering the era of "many-core" systems.

The design challenge for a many-core multicore system is to efficiently harness the multicore processing power and intelligently manage the substantial on-chip resources. A central concern is how to match the inherent parallelism of a many-core system with the performance requirements of applications. The potential for parallelism in fact exists at three levels in contemporary multicore system. First, there is hardware parallelism within each core processor, known as instruction level parallelism, which may or may not be exploited by application programmers and compilers. Second, there is the potential for multiprogramming and multithreaded execution within each processor. Finally, there is the potential for a single application to execute in concurrent processes or threads across multiple cores. Without strong and effective OS support for the last two types of parallelism just mentioned, hardware resources will not be efficiently used.

In essence, since the advent of multicore technology, OS designers have been struggling with the problem of how best to extract parallelism from computing workloads. A variety of approaches are being explored for next-generation operating systems. We will introduce two general strategies in this section, and will consider some details in later chapters.

***Parallelism within Applications*** Most applications can, in principle, be subdivided into multiple tasks that can execute in parallel, with these tasks then being implemented as multiple processes, perhaps each with multiple threads. The difficulty is that the developer must decide how to split up the application work into independently executable tasks. That is, the developer must decide what pieces can or should be executed asynchronously or in parallel. It is primarily the compiler and the programming language features that support the parallel programming design process. But the OS can support this design process, at minimum, by efficiently allocating resources among parallel tasks as defined by the developer.

One of the most effective initiatives to support developers is Grand Central Dispatch (GCD), implemented in the latest release of the UNIX-based Mac OS X and the iOS operating systems. GCD is a multicore support capability. It does not

help the developer decide how to break up a task or application into separate concurrent parts. But once a developer has identified something that can be split off into a separate task, GCD makes it as easy and noninvasive as possible to actually do so.

In essence, GCD is a thread pool mechanism, in which the OS maps tasks onto threads representing an available degree of concurrency (plus threads for blocking on I/O). Windows also has a thread pool mechanism (since 2000), and thread pools have been heavily used in server applications for years. What is new in GCD is the extension to programming languages to allow anonymous functions (called blocks) as a way of specifying tasks. GCD is hence not a major evolutionary step. Nevertheless, it is a new and valuable tool for exploiting the available parallelism of a multicore system.

One of Apple's slogans for GCD is "islands of serialization in a sea of concurrency." That captures the practical reality of adding more concurrency to run-of-the-mill desktop applications. Those islands are what isolate developers from the thorny problems of simultaneous data access, deadlock, and other pitfalls of multithreading. Developers are encouraged to identify functions of their applications that would be better executed off the main thread, even if they are made up of several sequential or otherwise partially interdependent tasks. GCD makes it easy to break off the entire unit of work while maintaining the existing order and dependencies between subtasks. In later chapters, we will look at some of the details of GCD.

***Virtual Machine Approach*** An alternative approach is to recognize that with the ever-increasing number of cores on a chip, the attempt to multiprogram individual cores to support multiple applications may be a misplaced use of resources [JACK10]. If instead, we allow one or more cores to be dedicated to a particular process, then leave the processor alone to devote its efforts to that process, we avoid much of the overhead of task switching and scheduling decisions. The multicore OS could then act as a hypervisor that makes a high-level decision to allocate cores to applications, but does little in the way of resource allocation beyond that.

The reasoning behind this approach is as follows. In the early days of computing, one program was run on a single processor. With multiprogramming, each application is given the illusion that it is running on a dedicated processor. Multiprogramming is based on the concept of a process, which is an abstraction of an execution environment. To manage processes, the OS requires protected space, free from user and program interference. For this purpose, the distinction between kernel mode and user mode was developed. In effect, kernel mode and user mode abstracted the processor into two processors. With all these virtual processors, however, come struggles over who gets the attention of the real processor. The overhead of switching between all these processors starts to grow to the point where responsiveness suffers, especially when multiple cores are introduced. But with many-core systems, we can consider dropping the distinction between kernel and user mode. In this approach, the OS acts more like a hypervisor. The programs themselves take on many of the duties of resource management. The OS assigns an application, a processor and some memory, and the program itself, using metadata generated by the compiler, would best know how to use these resources.

## 2.7 MICROSOFT WINDOWS OVERVIEW

### Background

Microsoft initially used the name Windows in 1985, for an operating environment extension to the primitive MS-DOS operating system, which was a successful OS used on early personal computers. This Windows/MS-DOS combination was ultimately replaced by a new version of Windows, known as Windows NT, first released in 1993, and intended for laptop and desktop systems. Although the basic internal architecture has remained roughly the same since Windows NT, the OS has continued to evolve with new functions and features. The latest release at the time of this writing is Windows 10. Windows 10 incorporates features from the preceding desktop/laptop release, Windows 8.1, as well as from versions of Windows intended for mobile devices for the Internet of Things (IoT). Windows 10 also incorporates software from the Xbox One system. The resulting unified Windows 10 supports desktops, laptops, smart phones, tablets, and Xbox One.

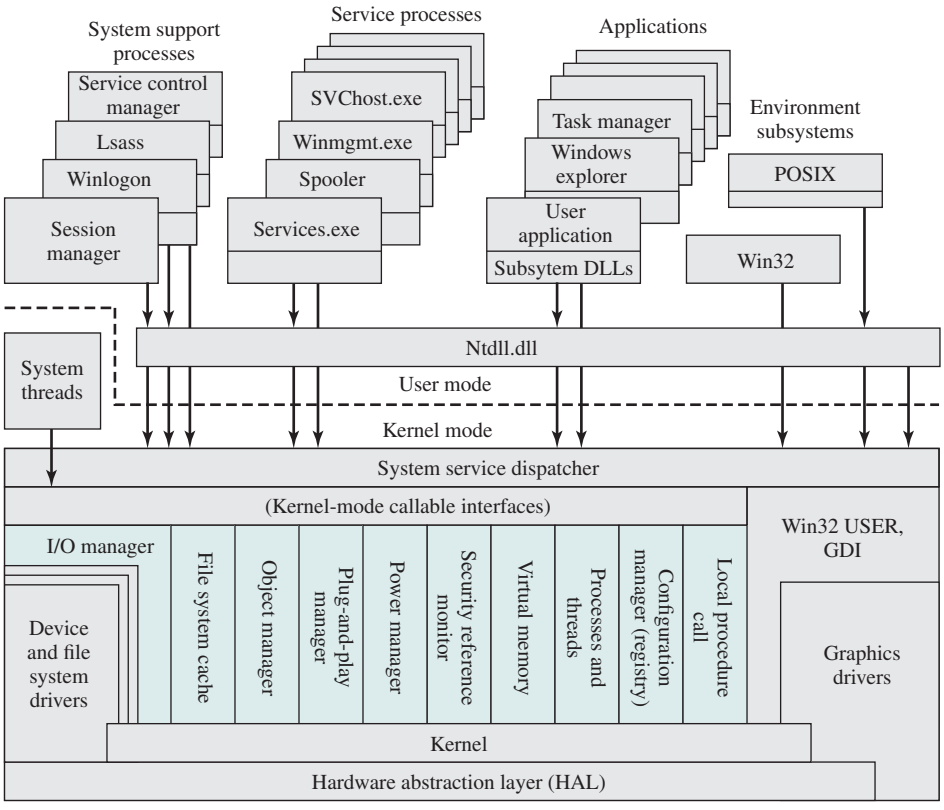
### Architecture

Figure 2.14 illustrates the overall structure of Windows. As with virtually all operating systems, Windows separates application-oriented software from the core OS software. The latter, which includes the Executive, the Kernel, device drivers, and the hardware abstraction layer, runs in kernel mode. Kernel-mode software has access to system data and to the hardware. The remaining software, running in user mode, has limited access to system data.

***Operating System Organization*** Windows has a highly modular architecture. Each system function is managed by just one component of the OS. The rest of the OS and all applications access that function through the responsible component using standard interfaces. Key system data can only be accessed through the appropriate function. In principle, any module can be removed, upgraded, or replaced without rewriting the entire system or its standard application program interfaces (APIs).

The kernel-mode components of Windows are the following:

- **Executive:** Contains the core OS services, such as memory management, process and thread management, security, I/O, and interprocess communication.
- **Kernel:** Controls execution of the processors. The Kernel manages thread scheduling, process switching, exception and interrupt handling, and multiprocessor synchronization. Unlike the rest of the Executive and the user levels, the Kernel's own code does not run in threads.
- **Hardware abstraction layer (HAL):** Maps between generic hardware commands and responses and those unique to a specific platform. It isolates the OS from platform-specific hardware differences. The HAL makes each computer's system bus, direct memory access (DMA) controller, interrupt controller,




Lsass = local security authentication server
POSIX = portable operating system interface
GDI = graphics device interface
DLL = dynamic link library

Colored area indicates Executive

**Figure 2.14 Windows Internals Architecture [RUSS11]**

system timers, and memory controller look the same to the Executive and kernel components. It also delivers the support needed for SMP, explained subsequently.

- **Device drivers:** Dynamic libraries that extend the functionality of the Executive. These include hardware device drivers that translate user I/O function calls into specific hardware device I/O requests, and software components for implementing file systems, network protocols, and any other system extensions that need to run in kernel mode.
- **Windowing and graphics system:** Implements the GUI functions, such as dealing with windows, user interface controls, and drawing.

The Windows Executive includes components for specific system functions and provides an API for user-mode software. Following is a brief description of each of the Executive modules:

- **I/O manager:** Provides a framework through which I/O devices are accessible to applications, and is responsible for dispatching to the appropriate device drivers for further processing. The I/O manager implements all the Windows I/O APIs and enforces security and naming for devices, network protocols, and file systems (using the object manager). Windows I/O will be discussed in Chapter 11.
- **Cache manager:** Improves the performance of file-based I/O by causing recently referenced file data to reside in main memory for quick access, and by deferring disk writes by holding the updates in memory for a short time before sending them to the disk in more efficient batches.
- **Object manager:** Creates, manages, and deletes Windows Executive objects that are used to represent resources such as processes, threads, and synchronization objects. It enforces uniform rules for retaining, naming, and setting the security of objects. The object manager also creates the entries in each process's handle table, which consist of access control information and a pointer to the object. Windows objects will be discussed later in this section.
- **Plug-and-play manager:** Determines which drivers are required to support a particular device and loads those drivers.
- **Power manager:** Coordinates power management among various devices and can be configured to reduce power consumption by shutting down idle devices, putting the processor to sleep, and even writing all of memory to disk and shutting off power to the entire system.
- **Security reference monitor:** Enforces access-validation and audit-generation rules. The Windows object-oriented model allows for a consistent and uniform view of security, right down to the fundamental entities that make up the Executive. Thus, Windows uses the same routines for access validation and for audit checks for all protected objects, including files, processes, address spaces, and I/O devices. Windows security will be discussed in Chapter 15.
- **Virtual memory manager:** Manages virtual addresses, physical memory, and the paging files on disk. Controls the memory management hardware and data structures which map virtual addresses in the process's address space to physical pages in the computer's memory. Windows virtual memory management will be described in Chapter 8.
- **Process/thread manager:** Creates, manages, and deletes process and thread objects. Windows process and thread management will be described in Chapter 4.
- **Configuration manager:** Responsible for implementing and managing the system registry, which is the repository for both system-wide and per-user settings of various parameters.

- **Advanced local procedure call (ALPC) facility:** Implements an efficient cross-process procedure call mechanism for communication between local processes implementing services and subsystems. Similar to the remote procedure call (RPC) facility used for distributed processing.

***User-Mode Processes*** Windows supports four basic types of user-mode processes:

1. **Special system processes:** User-mode services needed to manage the system, such as the session manager, the authentication subsystem, the service manager, and the logon process.
2. **Service processes:** The printer spooler, the event logger, user-mode components that cooperate with device drivers, various network services, and many others. Services are used by both Microsoft and external software developers to extend system functionality, as they are the only way to run background user-mode activity on a Windows system.
3. **Environment subsystems:** Provide different OS personalities (environments). The supported subsystems are Win32 and POSIX. Each environment subsystem includes a subsystem process shared among all applications using the subsystem and dynamic link libraries (DLLs) that convert the user application calls to ALPC calls on the subsystem process, and/or native Windows calls.
4. **User applications:** Executables (EXEs) and DLLs that provide the functionality users run to make use of the system. EXEs and DLLs are generally targeted at a specific environment subsystem; although some of the programs that are provided as part of the OS use the native system interfaces (NT API). There is also support for running 32-bit programs on 64-bit systems.

Windows is structured to support applications written for multiple OS personalities. Windows provides this support using a common set of kernel-mode components that underlie the OS environment subsystems. The implementation of each environment subsystem includes a separate process, which contains the shared data structures, privileges, and Executive object handles needed to implement a particular personality. The process is started by the Windows Session Manager when the first application of that type is started. The subsystem process runs as a system user, so the Executive will protect its address space from processes run by ordinary users.

An environment subsystem provides a graphical or command-line user interface that defines the look and feel of the OS for a user. In addition, each subsystem provides the API for that particular environment. This means that applications created for a particular operating environment need only be recompiled to run on Windows. Because the OS interface that applications see is the same as that for which they were written, the source code does not need to be modified.

## Client/Server Model

The Windows OS services, the environment subsystems, and the applications are structured using the client/server computing model, which is a common model for

distributed computing and will be discussed in Part Six. This same architecture can be adopted for use internally to a single system, as is the case with Windows.

The native NT API is a set of kernel-based services which provide the core abstractions used by the system, such as processes, threads, virtual memory, I/O, and communication. Windows provides a far richer set of services by using the client/server model to implement functionality in user-mode processes. Both the environment subsystems and the Windows user-mode services are implemented as processes that communicate with clients via RPC. Each server process waits for a request from a client for one of its services (e.g., memory services, process creation services, or networking services). A client, which can be an application program or another server program, requests a service by sending a message. The message is routed through the Executive to the appropriate server. The server performs the requested operation and returns the results or status information by means of another message, which is routed through the Executive back to the client.

Advantages of a client/server architecture include the following:

- **It simplifies the Executive.** It is possible to construct a variety of APIs implemented in user-mode servers without any conflicts or duplications in the Executive. New APIs can be added easily.
- **It improves reliability.** Each new server runs outside of the kernel, with its own partition of memory, protected from other servers. A single server can fail without crashing or corrupting the rest of the OS.
- **It provides a uniform means for applications to communicate with services via RPCs without restricting flexibility.** The message-passing process is hidden from the client applications by function stubs, which are small pieces of code which wrap the RPC call. When an application makes an API call to an environment subsystem or a service, the stub in the client application packages the parameters for the call and sends them as a message to the server process that implements the call.
- **It provides a suitable base for distributed computing.** Typically, distributed computing makes use of a client/server model, with remote procedure calls implemented using distributed client and server modules and the exchange of messages between clients and servers. With Windows, a local server can pass a message on to a remote server for processing on behalf of local client applications. Clients need not know whether a request is being serviced locally or remotely. Indeed, whether a request is serviced locally or remotely can change dynamically, based on current load conditions and on dynamic configuration changes.

### Threads and SMP

Two important characteristics of Windows are its support for threads and for symmetric multiprocessing (SMP), both of which were introduced in Section 2.4. [RUSS11] lists the following features of Windows that support threads and SMP:

- OS routines can run on any available processor, and different routines can execute simultaneously on different processors.

- Windows supports the use of multiple threads of execution within a single process. Multiple threads within the same process may execute on different processors simultaneously.
- Server processes may use multiple threads to process requests from more than one client simultaneously.
- Windows provides mechanisms for sharing data and resources between processes and flexible interprocess communication capabilities.

### Windows Objects

Though the core of Windows is written in C, the design principles followed draw heavily on the concepts of object-oriented design. This approach facilitates the sharing of resources and data among processes, and the protection of resources from unauthorized access. Among the key object-oriented concepts used by Windows are the following:

- **Encapsulation:** An object consists of one or more items of data, called *attributes*, and one or more procedures that may be performed on those data, called *services*. The only way to access the data in an object is by invoking one of the object's services. Thus, the data in the object can easily be protected from unauthorized use and from incorrect use (e.g., trying to execute a nonexecutable piece of data).
- **Object class and instance:** An object class is a template that lists the attributes and services of an object, and defines certain object characteristics. The OS can create specific instances of an object class as needed. For example, there is a single process object class and one process object for every currently active process. This approach simplifies object creation and management.
- **Inheritance:** Although the implementation is hand coded, the Executive uses inheritance to extend object classes by adding new features. Every Executive class is based on a base class which specifies virtual methods that support creating, naming, securing, and deleting objects. Dispatcher objects are Executive objects that inherit the properties of an event object, so they can use common synchronization methods. Other specific object types, such as the device class, allow classes for specific devices to inherit from the base class, and add additional data and methods.
- **Polymorphism:** Internally, Windows uses a common set of API functions to manipulate objects of any type; this is a feature of polymorphism, as defined in Appendix D. However, Windows is not completely polymorphic because there are many APIs that are specific to a single object type.

The reader unfamiliar with object-oriented concepts should review Appendix D.

Not all entities in Windows are objects. Objects are used in cases where data are intended for user-mode access, or when data access is shared or restricted. Among the entities represented by objects are files, processes, threads, semaphores, timers, and graphical windows. Windows creates and manages all types of objects in a uniform way, via the object manager. The object manager is responsible for creating and destroying objects on behalf of applications, and for granting access to an object's services and data.

Each object within the Executive, sometimes referred to as a kernel object (to distinguish from user-level objects not of concern to the Executive), exists as a memory block allocated by the kernel and is directly accessible only by kernel-mode components. Some elements of the data structure are common to all object types (e.g., object name, security parameters, usage count), while other elements are specific to a particular object type (e.g., a thread object's priority). Because these object data structures are in the part of each process's address space accessible only by the kernel, it is impossible for an application to reference these data structures and read or write them directly. Instead, applications manipulate objects indirectly through the set of object manipulation functions supported by the Executive. When an object is created, the application that requested the creation receives back a handle for the object. In essence, a handle is an index into a per-process Executive table containing a pointer to the referenced object. This handle can then be used by any thread within the same process to invoke Win32 functions that work with objects, or can be duplicated into other processes.

Objects may have security information associated with them, in the form of a Security Descriptor (SD). This security information can be used to restrict access to the object based on contents of a token object which describes a particular user. For example, a process may create a named semaphore object with the intent that only certain users should be able to open and use that semaphore. The SD for the semaphore object can list those users that are allowed (or denied) access to the semaphore object along with the sort of access permitted (read, write, change, etc.).

In Windows, objects may be either named or unnamed. When a process creates an unnamed object, the object manager returns a handle to that object, and the handle is the only way to refer to it. Handles can be inherited by child processes or duplicated between processes. Named objects are also given a name that other unrelated processes can use to obtain a handle to the object. For example, if process A wishes to synchronize with process B, it could create a named event object and pass the name of the event to B. Process B could then open and use that event object. However, if process A simply wished to use the event to synchronize two threads within itself, it would create an unnamed event object, because there is no need for other processes to be able to use that event.

There are two categories of objects used by Windows for synchronizing the use of the processor:

- **Dispatcher objects:** The subset of Executive objects which threads can wait on to control the dispatching and synchronization of thread-based system operations. These will be described in Chapter 6.
- **Control objects:** Used by the Kernel component to manage the operation of the processor in areas not managed by normal thread scheduling. Table 2.5 lists the Kernel control objects.

Windows is not a full-blown object-oriented OS. It is not implemented in an object-oriented language. Data structures that reside completely within one Executive component are not represented as objects. Nevertheless, Windows illustrates the power of object-oriented technology and represents the increasing trend toward the use of this technology in OS design.

**Table 2.5** Windows Kernel Control Objects

| | |
|---|---|
| Asynchronous procedure call | Used to break into the execution of a specified thread and to cause a procedure to be called in a specified processor mode. |
| Deferred procedure call | Used to postpone interrupt processing to avoid delaying hardware interrupts. Also used to implement timers and interprocessor communication. |
| Interrupt | Used to connect an interrupt source to an interrupt service routine by means of an entry in an Interrupt Dispatch Table (IDT). Each processor has an IDT that is used to dispatch interrupts that occur on that processor. |
| Process | Represents the virtual address space and control information necessary for the execution of a set of thread objects. A process contains a pointer to an address map, a list of ready threads containing thread objects, a list of threads belonging to the process, the total accumulated time for all threads executing within the process, and a base priority. |
| Thread | Represents thread objects, including scheduling priority and quantum, and which processors the thread may run on. |
| Profile | Used to measure the distribution of run time within a block of code. Both user and system codes can be profiled. |

## 2.8 TRADITIONAL UNIX SYSTEMS

### History

UNIX was initially developed at Bell Labs and became operational on a PDP-7 in 1970. Work on UNIX at Bell Labs, and later elsewhere, produced a series of versions of UNIX. The first notable milestone was porting the UNIX system from the PDP-7 to the PDP-11. This was the first hint that UNIX would be an OS for all computers. The next important milestone was the rewriting of UNIX in the programming language C. This was an unheard-of strategy at the time. It was generally felt that something as complex as an OS, which must deal with time-critical events, had to be written exclusively in assembly language. Reasons for this attitude include the following:

- Memory (both RAM and secondary store) was small and expensive by today's standards, so effective use was important. This included various techniques for overlaying memory with different code and data segments, and self-modifying code.
- Even though compilers had been available since the 1950s, the computer industry was generally skeptical of the quality of automatically generated code. With resource capacity small, efficient code, both in terms of time and space, was essential.
- Processor and bus speeds were relatively slow, so saving clock cycles could make a substantial difference in execution time.

The C implementation demonstrated the advantages of using a high-level language for most if not all of the system code. Today, virtually all UNIX implementations are written in C.

These early versions of UNIX were popular within Bell Labs. In 1974, the UNIX system was described in a technical journal for the first time [RITC74]. This spurred great interest in the system. Licenses for UNIX were provided to commercial institutions as well as universities. The first widely available version outside Bell Labs was Version 6, in 1976. The follow-on Version 7, released in 1978, is the ancestor of most modern UNIX systems. The most important of the non-AT&T systems to be developed was done at the University of California at Berkeley, called UNIX BSD (Berkeley Software Distribution), running first on PDP and then on VAX computers. AT&T continued to develop and refine the system. By 1982, Bell Labs had combined several AT&T variants of UNIX into a single system, marketed commercially as UNIX System III. A number of features was later added to the OS to produce UNIX System V.

## Description

The classic UNIX architecture can be pictured as in three levels: hardware, kernel, and user. The OS is often called the system kernel, or simply the kernel, to emphasize its isolation from the user and applications. It interacts directly with the hardware. It is the UNIX kernel that we will be concerned with in our use of UNIX as an example in this book. UNIX also comes equipped with a number of user services and interfaces that are considered part of the system. These can be grouped into the shell, which supports system calls from applications, other interface software, and the components of the C compiler (compiler, assembler, loader). The level above this consists of user applications and the user interface to the C compiler.

A look at the kernel is provided in Figure 2.15. User programs can invoke OS services either directly, or through library programs. The system call interface is the boundary with the user and allows higher-level software to gain access to specific kernel functions. At the other end, the OS contains primitive routines that interact directly with the hardware. Between these two interfaces, the system is divided into two main parts: one concerned with process control, and the other concerned with file management and I/O. The process control subsystem is responsible for memory management, the scheduling and dispatching of processes, and the synchronization and interprocess communication of processes. The file system exchanges data between memory and external devices either as a stream of characters or in blocks. To achieve this, a variety of device drivers are used. For block-oriented transfers, a disk cache approach is used: A system buffer in main memory is interposed between the user address space and the external device.

The description in this subsection has dealt with what might be termed *traditional UNIX systems*; [VAHA96] uses this term to refer to System V Release 3 (SVR3), 4.3BSD, and earlier versions. The following general statements may be made about a traditional UNIX system. It is designed to run on a single processor, and lacks the ability to protect its data structures from concurrent access by multiple processors. Its kernel is not very versatile, supporting a single type of file system, process scheduling policy, and executable file format. The traditional UNIX kernel is not designed to be extensible and has few facilities for code reuse. The result is that, as new features were added to the various UNIX versions, much new code had to be added, yielding a bloated and unmodular kernel.

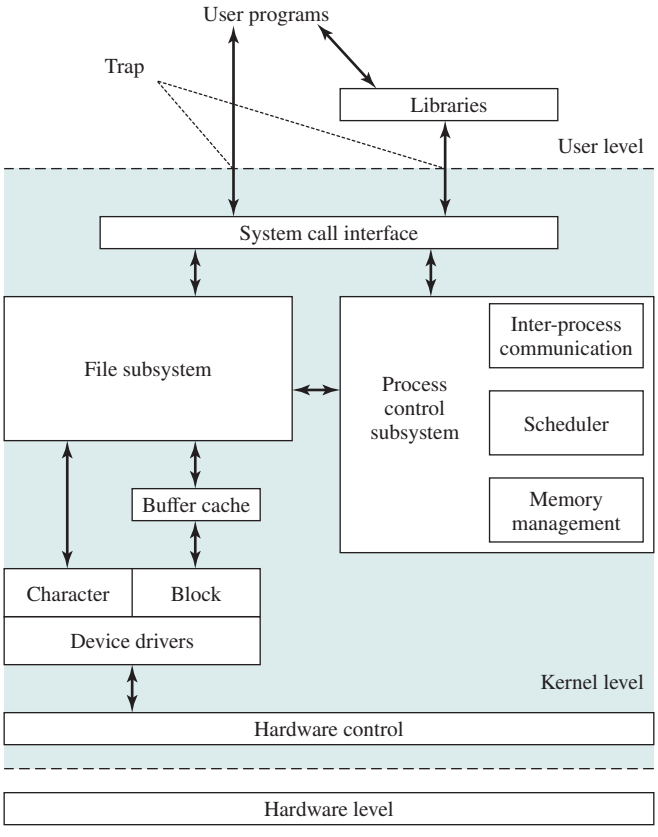



**Figure 2.15** **Traditional UNIX Architecture**

## 2.9 MODERN UNIX SYSTEMS

As UNIX evolved, the number of different implementations proliferated, each providing some useful features. There was a need to produce a new implementation that unified many of the important innovations, added other modern OS design features, and produced a more modular architecture. Typical of the modern UNIX kernel is the architecture depicted in Figure 2.16. There is a small core of facilities, written in a modular fashion, that provide functions and services needed by a number of OS processes. Each of the outer circles represents functions and an interface that may be implemented in a variety of ways.

We now turn to some examples of modern UNIX systems (see Figure 2.17).

### System V Release 4 (SVR4)

SVR4, developed jointly by AT&T and Sun Microsystems, combines features from SVR3, 4.3BSD, Microsoft Xenix System V, and SunOS. It was almost a total rewrite of the System V kernel and produced a clean, if complex, implementation. New features in the release include real-time processing support, process scheduling classes,

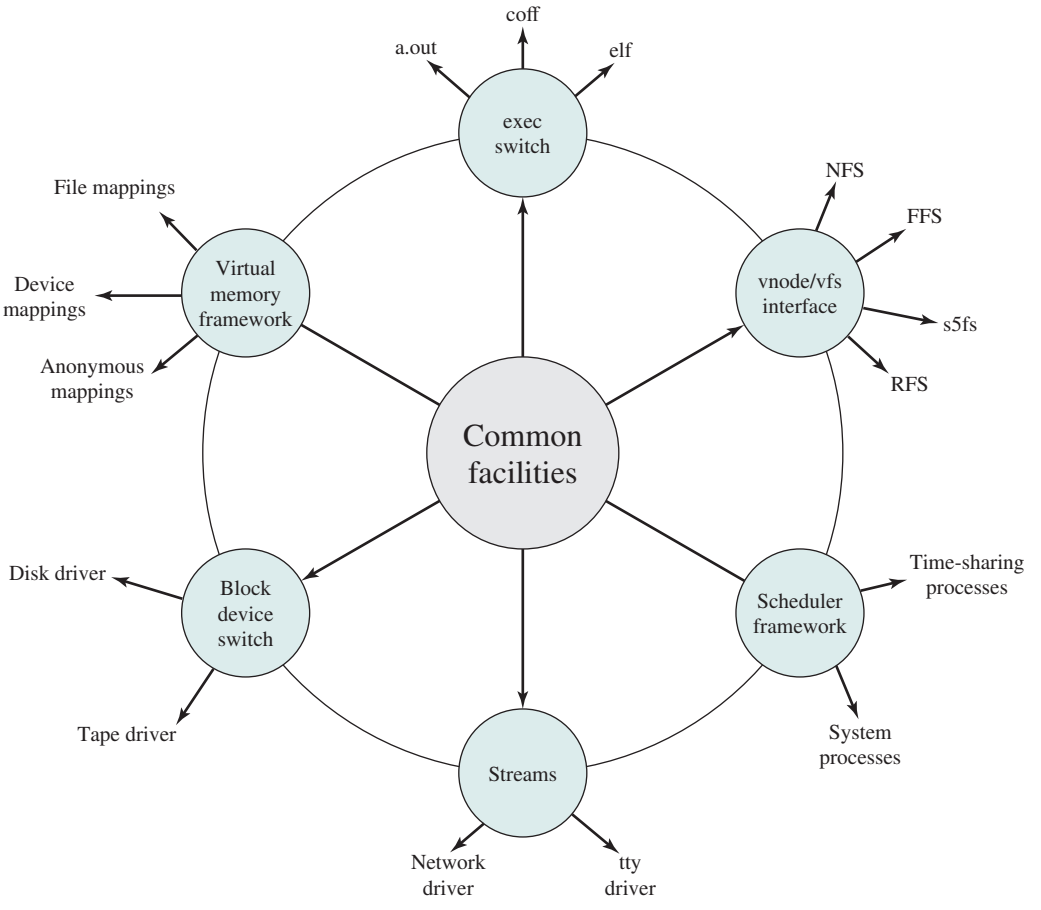



**Figure 2.16 Modern UNIX Kernel**

dynamically allocated data structures, virtual memory management, virtual file system, and a preemptive kernel.

SVR4 draws on the efforts of both commercial and academic designers, and was developed to provide a uniform platform for commercial UNIX deployment. It has succeeded in this objective and is perhaps the most important UNIX variant. It incorporates most of the important features ever developed on any UNIX system and does so in an integrated, commercially viable fashion. SVR4 runs on processors ranging from 32-bit microprocessors up to supercomputers.

## BSD

The Berkeley Software Distribution (BSD) series of UNIX releases have played a key role in the development of OS design theory. 4.xBSD is widely used in academic installations and has served as the basis of a number of commercial UNIX products. It is probably safe to say that BSD is responsible for much of the popularity of UNIX, and that most enhancements to UNIX first appeared in BSD versions.

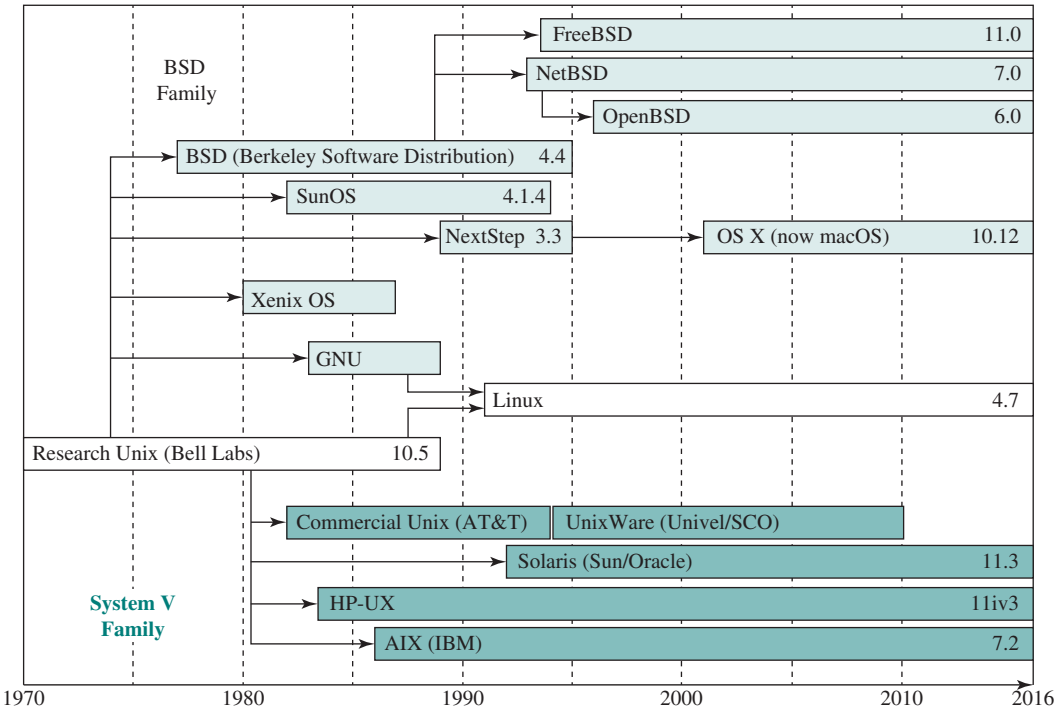



**Figure 2.17 UNIX Family Tree**

4.4BSD was the final version of BSD to be released by Berkeley, with the design and implementation organization subsequently dissolved. It is a major upgrade to 4.3BSD and includes a new virtual memory system, changes in the kernel structure, and a long list of other feature enhancements.

There are several widely used, open-source versions of BSD. FreeBSD is popular for Internet-based servers and firewalls and is used in a number of embedded systems. NetBSD is available for many platforms, including large-scale server systems, desktop systems, and handheld devices, and is often used in embedded systems. OpenBSD is an open-source OS that places special emphasis on security.

The latest version of the Macintosh OS, originally known as OS X and now called MacOS, is based on FreeBSD 5.0 and the Mach 3.0 microkernel.

## Solaris 11

Solaris is Oracle's SVR4-based UNIX release, with the latest version being 11. Solaris provides all of the features of SVR4 plus a number of more advanced features, such as a fully preemptable, multithreaded kernel, full support for SMP, and an object-oriented interface to file systems. Solaris is one the most widely used and most successful commercial UNIX implementations.

## 2.10 LINUX

### History

Linux started out as a UNIX variant for the IBM PC (Intel 80386) architecture. Linus Torvalds, a Finnish student of computer science, wrote the initial version. Torvalds posted an early version of Linux on the Internet in 1991. Since then, a number of people, collaborating over the Internet, have contributed to the development of Linux, all under the control of Torvalds. Because Linux is free and the source code is available, it became an early alternative to other UNIX workstations, such as those offered by Sun Microsystems and IBM. Today, Linux is a full-featured UNIX system that runs on virtually all platforms.

Key to the success of Linux has been the availability of free software packages under the auspices of the Free Software Foundation (FSF). FSF's goal is stable, platform-independent software that is free, high quality, and embraced by the user community. FSF's GNU project[2] provides tools for software developers, and the GNU Public License (GPL) is the FSF seal of approval. Torvalds used GNU tools in developing his kernel, which he then released under the GPL. Thus, the Linux distributions that you see today are the product of FSF's GNU project, Torvald's individual effort, and the efforts of many collaborators all over the world.

In addition to its use by many individual developers, Linux has now made significant penetration into the corporate world. This is not only because of the free software, but also because of the quality of the Linux kernel. Many talented developers have contributed to the current version, resulting in a technically impressive product. Moreover, Linux is highly modular and easily configured. This makes it easy to squeeze optimal performance from a variety of hardware platforms. Plus, with the source code available, vendors can tweak applications and utilities to meet specific requirements. There are also commercial companies such as Red Hat and Canonical, which provide highly professional and reliable support for their Linux-based distributions for long periods of time. Throughout this book, we will provide details of Linux kernel internals based on Linux kernel 4.7, released in 2016.

A large part of the success of the Linux Operating System is due to its development model. Code contributions are handled by one main mailing list, called LKML (Linux Kernel Mailing List). Apart from it, there are many other mailing lists, each dedicated to a Linux kernel subsystem (like the netdev mailing list for networking, the linux-pci for the PCI subsystem, the linux-acpi for the ACPI subsystem, and a great many more). The patches which are sent to these mailing lists should adhere to strict rules (primarily the Linux Kernel coding style conventions), and are reviewed by developers all over the world who are subscribed to these mailing lists. Anyone can send patches to these mailing lists; statistics (for example, those published in the lwn.net site from time to time) show that many patches are sent by developers from famous commercial companies like Intel, Red Hat, Google, Samsung, and others. Also, many maintainers are employees of commercial companies (like David

[2] GNU is a recursive acronym for *GNU's Not Unix*. The GNU project is a free software set of packages and tools for developing a UNIX-like operating system; it is often used with the Linux kernel.

Miller, the network maintainer, who works for Red Hat). Many times such patches are fixed according to feedback and discussions over the mailing list, and are resent and reviewed again. Eventually, the maintainer decides whether to accept or reject patches; and each subsystem maintainer from time to time sends a pull request of his tree to the main kernel tree, which is handled by Linus Torvalds. Linus himself releases a new kernel version in about every 7–10 weeks, and each such release has about 5–8 release candidates (RC) versions.

We should mention that it is interesting to try to understand why other open-source operating systems, such as various flavors of BSD or OpenSolaris, did not have the success and popularity which Linux has; there can be many reasons for that, and for sure, the openness of the development model of Linux contributed to its popularity and success. But this topic is out of the scope of this book.

### Modular Structure

Most UNIX kernels are monolithic. Recall from earlier in this chapter, a monolithic kernel is one that includes virtually all of the OS functionality in one large block of code that runs as a single process with a single address space. All the functional components of the kernel have access to all of its internal data structures and routines. If changes are made to any portion of a typical monolithic OS, all the modules and routines must be relinked and reinstalled, and the system rebooted, before the changes can take effect. As a result, any modification, such as adding a new device driver or file system function, is difficult. This problem is especially acute for Linux, for which development is global and done by a loosely associated group of independent developers.

Although Linux does not use a microkernel approach, it achieves many of the potential advantages of this approach by means of its particular modular architecture. Linux is structured as a collection of modules, a number of which can be automatically loaded and unloaded on demand. These relatively independent blocks are referred to as **loadable modules** [GOYE99]. In essence, a module is an object file whose code can be linked to and unlinked from the kernel at runtime. Typically, a module implements some specific function, such as a file system, a device driver, or some other feature of the kernel's upper layer. A module does not execute as its own process or thread, although it can create kernel threads for various purposes as necessary. Rather, a module is executed in kernel mode on behalf of the current process.

Thus, although Linux may be considered monolithic, its modular structure overcomes some of the difficulties in developing and evolving the kernel. The Linux loadable modules have two important characteristics:

1. **Dynamic linking:** A kernel module can be loaded and linked into the kernel while the kernel is already in memory and executing. A module can also be unlinked and removed from memory at any time.
2. **Stackable modules:** The modules are arranged in a hierarchy. Individual modules serve as libraries when they are referenced by client modules higher up in the hierarchy, and as clients when they reference modules further down.

Dynamic linking facilitates configuration and saves kernel memory [FRAN97]. In Linux, a user program or user can explicitly load and unload kernel modules using the `insmod` or `modprobe` and `rmmod` commands. The kernel itself monitors the need

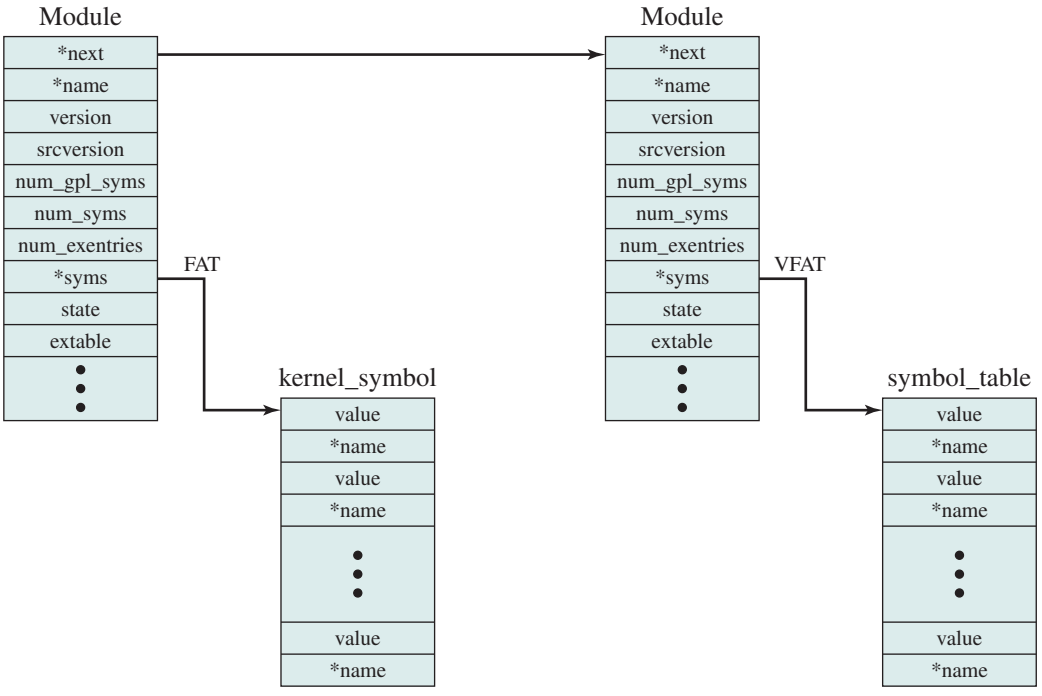



**Figure 2.18 Example List of Linux Kernel Modules**

for particular functions, and can load and unload modules as needed. With stackable modules, dependencies between modules can be defined. This has two benefits:

1. Code common to a set of similar modules (e.g., drivers for similar hardware) can be moved into a single module, reducing replication.
2. The kernel can make sure that needed modules are present, refraining from unloading a module on which other running modules depend, and loading any additional required modules when a new module is loaded.

Figure 2.18 is an example that illustrates the structures used by Linux to manage modules. The figure shows the list of kernel modules after only two modules have been loaded: FAT and VFAT. Each module is defined by two tables: the module table and the symbol table (kernel_symbol). The module table includes the following elements:

- ***name:** The module name
- **refcnt:** Module counter. The counter is incremented when an operation involving the module's functions is started and decremented when the operation terminates.
- **num_syms:** Number of exported symbols.
- ***syms:** Pointer to this module's symbol table.

The symbol table lists symbols that are defined in this module and used elsewhere.

## Kernel Components

Figure 2.19, taken from [MOSB02], shows the main components of a typical Linux kernel implementation. The figure shows several processes running on top of the kernel. Each box indicates a separate process, while each squiggly line with an arrowhead represents a thread of execution. The kernel itself consists of an interacting collection of components, with arrows indicating the main interactions. The underlying hardware is also depicted as a set of components with arrows indicating which kernel components use or control which hardware components. All of the kernel components, of course, execute on the processor. For simplicity, these relationships are not shown.

Briefly, the principal kernel components are the following:

- **Signals:** The kernel uses signals to call into a process. For example, signals are used to notify a process of certain faults, such as division by zero. Table 2.6 gives a few examples of signals.
- **System calls:** The system call is the means by which a process requests a specific kernel service. There are several hundred system calls, which can be roughly grouped into six categories: file system, process, scheduling, interprocess communication, socket (networking), and miscellaneous. Table 2.7 defines a few examples in each category.
- **Processes and scheduler:** Creates, manages, and schedules processes.
- **Virtual memory:** Allocates and manages virtual memory for processes.

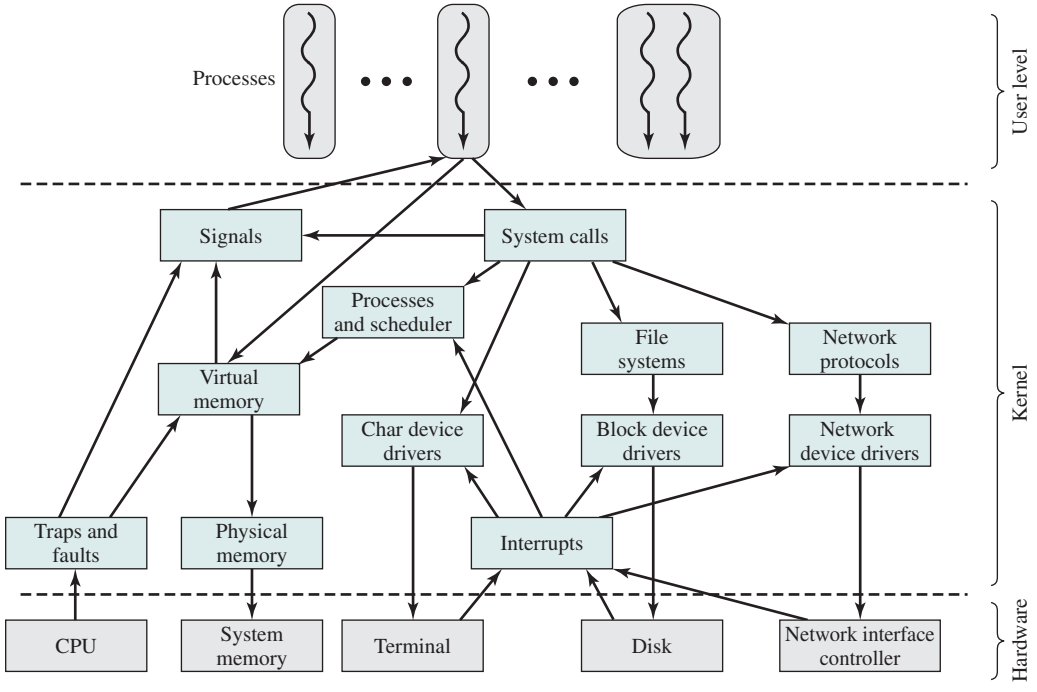



**Figure 2.19** **Linux Kernel Components**

**Table 2.6** Some Linux Signals

| | | | |
|---|---|---|---|
| SIGHUP | Terminal hangup | SIGCONT | Continue |
| SIGQUIT | Keyboard quit | SIGTSTP | Keyboard stop |
| SIGTRAP | Trace trap | SIGTTOU | Terminal write |
| SIGBUS | Bus error | SIGXCPU | CPU limit exceeded |
| SIGKILL | Kill signal | SIGVTALRM | Virtual alarm clock |
| SIGSEGV | Segmentation violation | SIGWINCH | Window size unchanged |
| SIGPIPT | Broken pipe | SIGPWR | Power failure |
| SIGTERM | Termination | SIGRTMIN | First real-time signal |
| SIGCHLD | Child status unchanged | SIGRTMAX | Last real-time signal |

- **File systems:** Provide a global, hierarchical namespace for files, directories, and other file-related objects and provide file system functions.
- **Network protocols:** Support the Sockets interface to users for the TCP/IP protocol suite.

**Table 2.7** Some Linux System Calls

| | |
|---|---|
| **File System Related** | |
| **close** | Close a file descriptor. |
| **link** | Make a new name for a file. |
| **open** | Open and possibly create a file or device. |
| **read** | Read from file descriptor. |
| **write** | Write to file descriptor. |
| **Process Related** | |
| **execve** | Execute program. |
| **exit** | Terminate the calling process. |
| **getpid** | Get process identification. |
| **setuid** | Set user identity of the current process. |
| **ptrace** | Provide a means by which a parent process may observe and control the execution of another process, and examine and change its core image and registers. |
| **Scheduling Related** | |
| **sched_getparam** | Set the scheduling parameters associated with the scheduling policy for the process identified by `pid`. |
| **sched_get_priority_max** | Return the maximum priority value that can be used with the scheduling algorithm identified by `policy`. |
| **sched_setscheduler** | Set both the scheduling policy (e.g., FIFO) and the associated parameters for the process `pid`. |
| **sched_rr_get_interval** | Write into the timespec structure pointed to by the parameter to the round-robin time quantum for the process `pid`. |
| **sched_yield** | A process can relinquish the processor voluntarily without blocking via this system call. The process will then be moved to the end of the queue for its static priority and a new process gets to run. |

**Table 2.7** (*Continued*)

| | |
|---|---|
| **Interprocess Communication (IPC) Related** | |
| **msgrcv** | A message buffer structure is allocated to receive a message. The system call then reads a message from the message queue specified by `msqid` into the newly created message buffer. |
| **semctl** | Perform the control operation specified by `cmd` on the semaphore set `semid`. |
| **semop** | Perform operations on selected members of the semaphore set `semid`. |
| **shmat** | Attach the shared memory segment identified by `semid` to the data segment of the calling process. |
| **shmctl** | Allow the user to receive information on a shared memory segment; set the owner, group, and permissions of a shared memory segment; or destroy a segment. |
| **Socket (networking) Related** | |
| **bind** | Assign the local IP address and port for a socket. Return 0 for success or −1 for error. |
| **connect** | Establish a connection between the given socket and the remote socket associated with sockaddr. |
| **gethostname** | Return local host name. |
| **send** | Send the bytes contained in buffer pointed to by *msg over the given socket. |
| **setsockopt** | Set the options on a socket. |
| **Miscellaneous** | |
| **fsync** | Copy all in-core parts of a file to disk, and wait until the device reports that all parts are on stable storage. |
| **time** | Return the time in seconds since January 1, 1970. |
| **vhangup** | Simulate a hangup on the current terminal. This call arranges for other users to have a "clean" tty at login time. |

- **Character device drivers:** Manage devices that require the kernel to send or receive data one byte at a time, such as terminals, modems, and printers.
- **Block device drivers:** Manage devices that read and write data in blocks, such as various forms of secondary memory (magnetic disks, CD-ROMs, etc.).
- **Network device drivers:** Manage network interface cards and communications ports that connect to network devices, such as bridges and routers.
- **Traps and faults:** Handle traps and faults generated by the processor, such as a memory fault.
- **Physical memory:** Manages the pool of page frames in real memory and allocates pages for virtual memory.
- **Interrupts** Handle interrupts from peripheral devices.

## 2.11 ANDROID

The Android operating system is a Linux-based system originally designed for mobile phones. It is the most popular mobile OS by a wide margin: Android handsets outsell Apple's iPhones globally by about 4 to 1 [MORR16]. But, this is just one element in

the increasing dominance of Android. Increasingly, it is the OS behind virtually any device with a computer chip other than servers and PCs. Android is a widely used OS for the Internet of things.

Initial Android OS development was done by Android, Inc., which was bought by Google in 2005. The first commercial version, Android 1.0, was released in 2008. As of this writing, the most recent version is Android 7.0 (Nougat). Android has an active community of developers and enthusiasts who use the Android Open Source Project (AOSP) source code to develop and distribute their own modified versions of the operating system. The open-source nature of Android has been the key to its success.

## Android Software Architecture

Android is defined as a software stack that includes a modified version of the Linux kernel, middleware, and key applications. Figure 2.20 shows the Android software architecture in some detail. Thus, Android should be viewed as a complete software stack, not just an OS.

***APPLICATIONS*** All the applications with which the user interacts directly are part of the application layer. This includes a core set of general-purpose applications, such as an e-mail client, SMS program, calendar, maps, browser, contacts, and other applications commonly standard with any mobile device. Applications are typically implemented in Java. A key goal of the open-source Android architecture is to make it easy for developers to implement new applications for specific devices and specific end-user requirements. Using Java enables developers to be relieved of hardware-specific considerations and idiosyncrasies, as well as tap into Java's higher-level language features, such as predefined classes. Figure 2.20 shows examples of the types of base applications found on the Android platform.

***APPLICATION FRAMEWORK*** The Application Framework layer provides high-level building blocks, accessible through standardized APIs, that programmers use to create new apps. The architecture is designed to simplify the reuse of components. Some of the key Application Framework components are:

- **Activity Manager:** Manages lifecycle of applications. It is responsible for starting, pausing, and resuming the various applications.
- **Window Manager:** Java abstraction of the underlying Surface Manager. The Surface Manager handles the frame buffer interaction and low-level drawing, whereas the Window Manager provides a layer on top of it, to allow applications to declare their client area and use features like the status bar.
- **Package Manager:** Installs and removes applications.
- **Telephony Manager:** Allows interaction with phone, SMS, and MMS services.
- **Content Providers:** These functions encapsulate application data that need to be shared between applications, such as contacts.
- **Resource Manager:** Manages application resources, such as localized strings and bitmaps.

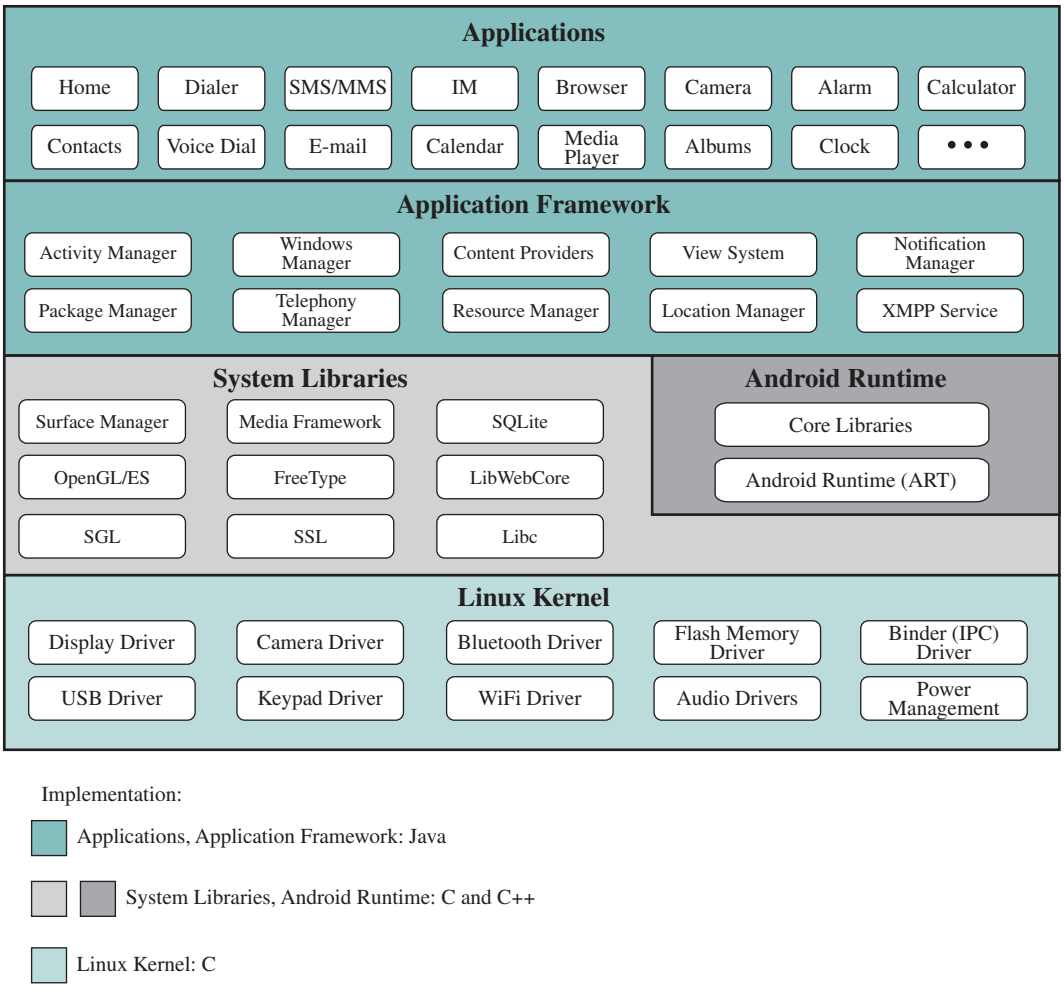



**Figure 2.20 Android Software Architecture**

- **View System:** Provides the user interface (UI) primitives, such as buttons, list-boxes, date pickers, and other controls, as well as UI Events (such as touch and gestures).
- **Location Manager:** Allows developers to tap into location-based services, whether by GPS, cell tower IDs, or local Wi-Fi databases. (recognized Wi-Fi hotspots and their status)
- **Notification Manager:** Manages events, such as arriving messages and appointments.
- **XMPP:** Provides standardized messaging (also, Chat) functions between applications.

***System Libraries*** The layer below the Application Framework consists of two parts: System Libraries, and Android Runtime. The System Libraries component is

a collection of useful system functions, written in C or C++ and used by various components of the Android system. They are called from the application framework and applications through a Java interface. These features are exposed to developers through the Android application framework. Some of the key system libraries include the following:

- **Surface Manager:** Android uses a compositing window manager similar to Vista or Compiz, but it is much simpler. Instead of drawing directly to the screen buffer, your drawing commands go into off-screen bitmaps that are then combined with other bitmaps to form the screen content the user sees. This lets the system create all sorts of interesting effects, such as see-through windows and fancy transitions.
- **OpenGL:** OpenGL (Open Graphics Library) is a cross-language, multiplatform API for rendering 2D and 3D computer graphics. OpenGL/ES (OpenGL for embedded systems) is a subset of OpenGL designed for embedded systems.
- **Media Framework:** The Media Framework supports video recording and playing in many formats, including AAC, AVC (H.264), H.263, MP3, and MPEG-4.
- **SQL Database:** Android includes a lightweight SQLite database engine for storing persistent data. SQLite is discussed in a subsequent section.
- **Browser Engine:** For fast display of HTML content, Android uses the WebKit library, which is essentially the same library used in Safari and iPhone. It was also the library used for the Google Chrome browser until Google switched to Blink.
- **Bionic LibC:** This is a stripped-down version of the standard C system library, tuned for embedded Linux-based devices. The interface is the standard Java Native Interface (JNI).

***LINUX KERNEL*** The OS kernel for Android is similar to, but not identical with, the standard Linux kernel distribution. One noteworthy change is the Android kernel lacks drivers not applicable in mobile environments, making the kernel smaller. In addition, Android enhances the Linux kernel with features that are tailored to the mobile environment, and generally not as useful or applicable on a desktop or laptop platform.

Android relies on its Linux kernel for core system services such as security, memory management, process management, network stack, and driver model. The kernel also acts as an abstraction layer between the hardware and the rest of the software stack, and enables Android to use the wide range of hardware drivers that Linux supports.

## Android Runtime

Most operating systems used on mobile devices, such as iOS and Windows, use software that is compiled directly to the specific hardware platform. In contrast, most Android software is mapped into a bytecode format, which is then transformed into

native instructions on the device itself. Earlier releases of Android used a scheme known as Dalvik. However, Dalvik has a number of limitations in terms of scaling up to larger memories and multicore architectures, so more recent releases of Android rely on a scheme known as Android runtime (ART). ART is fully compatible with Dalvik's existing bytecode format, dex (Dalvik Executable), so application developers do not need to change their coding to be executable under ART. We will first look at Dalvik, then examine ART.

***The Dalvik Virtual Machine*** The Dalvik VM (DVM) executes files in the .dex format, a format that is optimized for efficient storage and memory-mappable execution. The VM can run classes compiled by a Java language compiler that have been transformed into its native format using the included "dx" tool. The VM runs on top of Linux kernel, which it relies on for underlying functionality (such as threading and low-level memory management). The Dalvik core class library is intended to provide a familiar development base for those used to programming with Java Standard Edition, but it is geared specifically to the needs of a small mobile device.

Each Android application runs in its own process, with its own instance of the Dalvik VM. Dalvik has been written so a device can efficiently run multiple VMs efficiently.

***The Dex File Format*** The DVM runs applications and code written in Java. A standard Java compiler turns source code (written as text files) into bytecode. The bytecode is then compiled into a .dex file that the DVM can read and use. In essence, class files are converted into .dex files (much like a .jar file if one were using the standard Java VM) and then read and executed by the DVM. Duplicate data used in class files are included only once in the .dex file, which saves space and uses less overhead. The executable files can be modified again when an application is installed to make things even more optimized for mobile.

***Android Runtime Concepts*** ART is the current application runtime used by Android, introduced with Android version 4.4 (KitKat). When Android was designed initially, it was designed for single core (with minimal multithreading support in hardware) and low-memory devices, for which Dalvik seemed a suitable runtime. However, in recent times, the devices that run Android have multicore processors and more memory (at a relatively cheaper cost), which made Google to re-think the runtime design to provide developers and users a richer experience by making use of the available high-end hardware.

For both Dalvik and ART, all Android applications written in Java are compiled to dex bytecode. While Dalvik uses dex bytecode format for portability, it has to be converted (compiled) to machine code to be actually run by a processor. The Dalvik runtime did this conversion from dex bytecode to native machine code when the application ran, and this process was called JIT (just-in-time) compilation. Because JIT compiles only a part of the code, it has a smaller memory footprint and uses less physical space on the device. (Only the dex files are stored in the permanent storage as opposed to the actual machine code.) Dalvik identifies the

section of code that runs frequently and caches the compiled code for this once, so the subsequent executions of this section of code are faster. The pages of physical memory that store the cached code are not swappable/pageable, so this also adds a bit to the memory pressure if the system is already in such a state. Even with these optimizations, Dalvik has to do JIT-compilation every time the app is run, which consumes a considerable amount of processor resources. Note the processor is not only being used for actually running the app, but also for converting the dex bytecode to native code, thereby draining more power. This processor usage was also the reason for poor user interface experience in some heavy usage applications when they start.

To overcome some of these issues, and to make more effective use of the available high-end hardware, Android introduced ART. ART also executes dex bytecode but instead of compiling the bytecode at runtime, ART compiles the bytecode to native machine code during install time of the app. This is called ahead-of-time (AOT) compilation. ART uses the "`dex2oat`" tool to do this compilation at install time. The output of the tool is a file that is then executed when the application runs.

Figure 2.21 shows the life cycle of an APK, an application package that comes from the developer to the user. The cycle begins with source code being compiled into .dex format and combined with any appropriate support code to form an APK. On the user side, the received APK is unpacked. The resources and native code are generally installed directly into the application directory. The .dex code, however, requires further processing, both in the case of Dalvik and of ART. In Dalvik, a function called `dexopt` is applied to the dex file to produce an optimized version of dex (odex) referred to as quickened dex; the objective is to make the `dex` code execute more quickly on the dex interpreter. In ART, the `dex2oat` function does the same sort of optimization as `dexopt`; it also compiles the dex code to produce native code on the target device. The output of the dex2oat function is an Executable and Linkable Format (ELF) file, which runs directly without an interpreter.

***ADVANTAGES AND DISADVANTAGES*** The benefits of using ART include the following:

- Reduces startup time of applications as native code is directly executed.
- Improves battery life because processor usage for JIT is avoided.
- Lesser RAM footprint is required for the application to run (as there is no storage required for JIT cache). Moreover, because there is no JIT code cache, which is non-pageable, this provides flexibility of RAM usage when there is a low-memory scenario.
- There are a number of Garbage Collection optimizations and debug enhancements that went into ART.

Some potential disadvantages of ART:

- Because the conversion from bytecode to native code is done at install time, application installation takes more time. For Android developers who load an app a number of times during testing, this time may be noticeable.

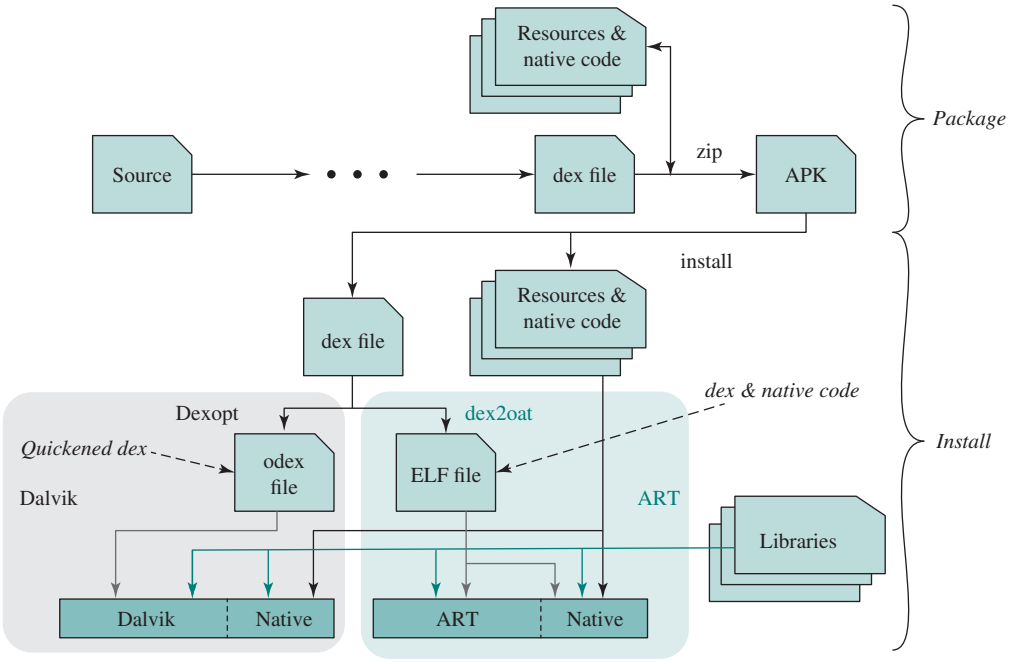



**Figure 2.21** **The Life Cycle of an APK**

- On the first fresh boot or first boot after factory reset, all applications installed on a device are compiled to native code using dex2opt. Therefore, the first boot can take significantly longer (in the order of 3–5 seconds) to reach Home Screen compared to Dalvik.
- The native code thus generated is stored on internal storage that requires a significant amount of additional internal storage space.

## Android System Architecture

It is useful to illustrate Android from the perspective of an application developer, as shown in Figure 2.22. This system architecture is a simplified abstraction of the software architecture shown in Figure 2.20. Viewed in this fashion, Android consists of the following layers:

- **Applications and Framework:** Application developers are primarily concerned with this layer and the APIs that allow access to lower-layer services.
- **Binder IPC:** The Binder Inter-Process Communication mechanism allows the application framework to cross process boundaries and call into the Android system services code. This basically allows high-level framework APIs to interact with Android's system services.

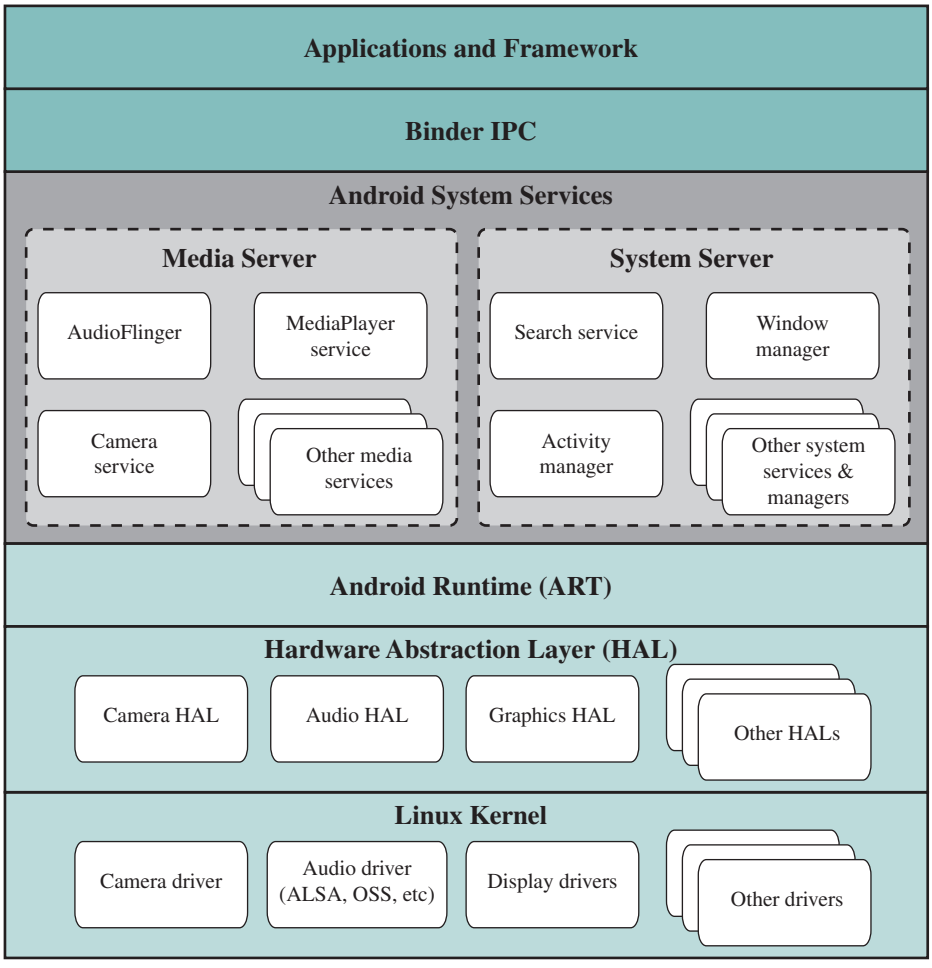



**Figure 2.22** **Android System Architecture**

- **Android System Services:** Most of the functionality exposed through the application framework APIs invokes system services that in turn access the underlying hardware and kernel functions. Services can be seen as being organized in two groups: Media services deal with playing and recording media and system services handle system-level functionalities such as power management, location management, and notification management.
- **Hardware Abstraction Layer (HAL):** The HAL provides a standard interface to kernel-layer device drivers, so upper-layer code need not be concerned with the details of the implementation of specific drivers and hardware. The HAL is virtually unchanged from that in a standard Linux distribution. This

layer is used to abstract the device-specific capabilities (which are supported by hardware and exposed by the Kernel) from the user space. The user space could either be Android's Services or Applications. The purpose of HAL is to keep the user space consistent with respect to various devices. Also, vendors can make their own enhancements and put it in their HAL layer without impacting the user space. An example for this is the HwC (Hardware Composer), which is a vendor-specific HAL implementation that understands the rendering capabilities of the underlying hardware. Surface manager seamlessly works with various implementations of the HwC from different vendors.

- **Linux Kernel:** Linux kernel is tailored to meet the demands of a mobile environment.

## Activities

An activity is a single visual user interface component, including objects such as menu selections, icons, and checkboxes. Every screen in an application is an extension of the Activity class. Activities use Views to form graphical user interfaces that display information and respond to user actions. We will discuss Activities in Chapter 4.

## Power Management

Android adds two features to the Linux kernel to enhance the ability to perform power management: alarms, and wakelocks.

The Alarms capability is implemented in the Linux kernel, and is visible to the app developer through the AlarmManager in the RunTime core libraries. Through the AlarmManager, an app can request a timed wake-up service. The Alarms facility is implemented in the kernel so an alarm can trigger even if the system is in sleep mode. This allows the system to go into sleep mode, saving power, even though there is a process that requires a wake up.

The wakelock facility prevents an Android system from entering into sleep mode. An application can hold one of the following wakelocks:

- **Full_Wake_Lock:** Processor on, full screen brightness, keyboard bright
- **Partial_Wake_Lock:** Processor on, screen off, keyboard off
- **Screen_Dim_Wake_Lock:** Processor on, screen dim, keyboard off
- **Screen_Bright_Wake_Lock:** Processor on, screen bright, keyboard off

These locks are requested through the API whenever an application requires one of the managed peripherals to remain powered on. If no wakelock exists, which locks the device, then it is powered off to conserve battery life.

These kernel objects are made visible to apps in user space by means of /sys/power/wavelock files. The wake_lock and wake_unlock files can be used to define and toggle a lock by writing to the corresponding file.

## 2.12 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

### Key Terms

| | | |
|---|---|---|
| batch processing | monolithic kernel | scheduling |
| batch system | multiprogrammed batch system | serial processing |
| execution context | multiprogramming | state |
| distributed operating system | multitasking | symmetric multiprocessing (SMP) |
| downtime | multithreading | task |
| fault | nucleus | thread |
| interrupt | object-oriented design | time sharing |
| job | operating system | time-sharing system |
| job control language (JCL) | physical address | time slicing |
| kernel | privileged instruction | uniprogramming |
| kernel mode | process | uptime |
| loadable modules | process state | user mode |
| mean time to failure (MTTF) | real address | virtual address |
| mean time to repair (MTTR) | reliability | virtual machine |
| memory management | resident monitor | virtual memory |
| microkernel | round-robin | |
| monitor | | |

### Review Questions

**2.1.** What are three objectives of an OS design?
**2.2.** What is the kernel of an OS?
**2.3.** What is multiprogramming?
**2.4.** What is a process?
**2.5.** How is the execution context of a process used by the OS?
**2.6.** List and briefly explain five storage management responsibilities of a typical OS.
**2.7.** What is time slicing?
**2.8.** Describe the round-robin scheduling technique.
**2.9.** Explain the difference between a monolithic kernel and a microkernel.
**2.10.** What is multithreading?
**2.11.** What do you understand by a distributed operating system?

### Problems

**2.1.** Suppose we have four jobs in a computer system, in the order JOB1, JOB2, JOB3 and JOB4. JOB1 requires 8 s of CPU time and 8 s of I/O time; JOB2 requires 4 s of CPU time and 14 s of disk time; JOB3 requires 6 s of CPU time; and, JOB4 requires 4 s of CPU time and 16 s of printer time. Define the following quantities for system utilization:

- Turnaround time = actual time to complete a job
- Throughput = average number of jobs completed per time period $T$
- Processor utilization = percentage of time that the processor is active (not waiting)

Compute these quantities (with illustrations if needed) in each of the following systems:

**a.** A uniprogramming system, whereby each job executes to completion before the next job can start its execution.

**b.** A multiprogramming system that follows a simple round-robin scheduling. Each process gets 2 s of CPU time turn-wise in a circular manner

**2.2.** In a batch operating system, three jobs are submitted for execution. Each job involves an I/O activity, CPU time and another I/O activity of the same time span as the first. Job JOB1 requires a total of 23 ms, with 3 ms CPU time; JOB2 requires a total time of 29 ms with 5 ms CPU time; JOB3 requires a total time of 14 ms with 4 ms CPU time. Illustrate their execution and find CPU utilization for uniprogramming and multiprogramming systems.

**2.3.** Contrast the scheduling policies you might use when trying to optimize a time-sharing system with those you would use to optimize a **multiprogrammed batch system**.

**2.4.** A computer system boots and starts a user application when an interrupt occurs. In which modes does the operating system work in this scenario?

**2.5.** In IBM's mainframe OS, OS/390, one of the major modules in the kernel is the System Resource Manager. This module is responsible for the allocation of resources among address spaces (processes). The SRM gives OS/390 a degree of sophistication unique among operating systems. No other mainframe OS, and certainly no other type of OS, can match the functions performed by SRM. The concept of resource includes processor, real memory, and I/O channels. SRM accumulates statistics pertaining to utilization of processor, channel, and various key data structures. Its purpose is to provide optimum performance based on performance monitoring and analysis. The installation sets forth various performance objectives, and these serve as guidance to the SRM, which dynamically modifies installation and job performance characteristics based on system utilization. In turn, the SRM provides reports that enable the trained operator to refine the configuration and parameter settings to improve user service.

This problem concerns one example of SRM activity. Real memory is divided into equal-sized blocks called frames, of which there may be many thousands. Each frame can hold a block of virtual memory referred to as a page. SRM receives control approximately 20 times per second, and inspects each and every page frame. If the page has not been referenced or changed, a counter is incremented by 1. Over time, SRM averages these numbers to determine the average number of seconds that a page frame in the system goes untouched. What might be the purpose of this, and what action might SRM take?

**2.6.** A multiprocessor with ten processors has 24 attached tape drives. There are a large number of jobs submitted to the system that each require a maximum of six tape drives to complete execution. Assume that each job starts running with only four tape drives for a long period before requiring the other two tape drives for a short period toward the end of its operation. Also assume an endless supply of such jobs.

**a.** Assume the scheduler in the OS will not start a job unless there are six tape drives available. When a job is started, six drives are assigned immediately and are not released until the job finishes. What is the maximum number of jobs that can be in progress at once? What are the maximum and minimum number of tape drives that may be left idle as a result of this policy?

**b.** Suggest an alternative policy to improve tape drive utilization and at the same time avoid system deadlock. What is the maximum number of jobs that can be in progress at once? What are the bounds on the number of idling tape drives?

# PART 2 Processes

## CHAPTER 3

# PROCESS DESCRIPTION AND CONTROL


**3.1 What Is a Process?**
- Background
- Processes and Process Control Blocks

**3.2 Process States**
- A Two-State Process Model
- The Creation and Termination of Processes
- A Five-State Model
- Suspended Processes

**3.3 Process Description**
- Operating System Control Structures
- Process Control Structures

**3.4 Process Control**
- Modes of Execution
- Process Creation
- Process Switching

**3.5 Execution of the Operating System**
- Nonprocess Kernel
- Execution within User Processes
- Process-Based Operating System

**3.6 UNIX SVR4 Process Management**
- Process States
- Process Description
- Process Control

**3.7 Summary**

**3.8 Key Terms, Review Questions, and Problems**

### Learning Objectives

After studying this chapter, you should be able to:

- Define the term *process* and explain the relationship between processes and process control blocks.
- Explain the concept of a process state and discuss the state transitions the processes undergo.
- List and describe the purpose of the data structures and data structure elements used by an OS to manage processes.
- Assess the requirements for process control by the OS.
- Understand the issues involved in the execution of OS code.
- Describe the process management scheme for UNIX SVR4.

All multiprogramming operating systems, from single-user systems such as Windows for end users to mainframe systems such as IBM's mainframe operating system z/OS which can support thousands of users, are built around the concept of the process. Most requirements that the OS must meet can be expressed with reference to processes:

- The OS must interleave the execution of multiple processes, to maximize processor utilization while providing reasonable response time.
- The OS must allocate resources to processes in conformance with a specific policy (e.g., certain functions or applications are of higher priority) while at the same time avoiding deadlock.[1]
- The OS may be required to support interprocess communication and user creation of processes, both of which may aid in the structuring of applications.

We begin with an examination of the way in which the OS represents and controls processes. Then, the chapter discusses process states, which characterize the behavior of processes. We will then look at the data structures that the OS uses to manage processes. These include data structures to represent the state of each process and data structures that record other characteristics of processes that the OS needs to achieve its objectives. Next, we will look at the ways in which the OS uses these data structures to control process execution. Finally, we will discuss process management in UNIX SVR4. Chapter 4 will provide more modern examples of process management.

This chapter occasionally refers to virtual memory. Much of the time, we can ignore this concept in dealing with processes, but at certain points in the discussion, virtual memory considerations are pertinent. Virtual memory was previewed in Chapter 2 and will be discussed in detail in Chapter 8.

[1]Deadlock will be examined in Chapter 6. As a simple example, deadlock occurs if two processes need the same two resources to continue and each has ownership of one. Unless some action is taken, each process will wait indefinitely for the missing resource.

## 3.1 WHAT IS A PROCESS?

### Background

Before defining the term *process*, it is useful to summarize some of the concepts introduced in Chapters 1 and 2:

1. A computer platform consists of a collection of hardware resources, such as the processor, main memory, I/O modules, timers, disk drives, and so on.
2. Computer applications are developed to perform some task. Typically, they accept input from the outside world, perform some processing, and generate output.
3. It is inefficient for applications to be written directly for a given hardware platform. The principal reasons for this are as follows:
   a. Numerous applications can be developed for the same platform. Thus, it makes sense to develop common routines for accessing the computer's resources.
   b. The processor itself provides only limited support for multiprogramming. Software is needed to manage the sharing of the processor and other resources by multiple applications at the same time.
   c. When multiple applications are active at the same time, it is necessary to protect the data, I/O use, and other resource use of each application from the others.
4. The OS was developed to provide a convenient, feature-rich, secure, and consistent interface for applications to use. The OS is a layer of software between the applications and the computer hardware (see Figure 2.1) that supports applications and utilities.
5. We can think of the OS as providing a uniform, abstract representation of resources that can be requested and accessed by applications. Resources include main memory, network interfaces, file systems, and so on. Once the OS has created these resource abstractions for applications to use, it must also manage their use. For example, an OS may permit resource sharing and resource protection.

Now that we have the concepts of applications, system software, and resources, we are in a position to discuss how the OS can, in an orderly fashion, manage the execution of applications such that:

- Resources are made available to multiple applications.
- The physical processor is switched among multiple applications so all will appear to be progressing.
- The processor and I/O devices can be used efficiently.

The approach taken by all modern operating systems is to rely on a model in which the execution of an application corresponds to the existence of one or more processes.

## Processes and Process Control Blocks

Recall from Chapter 2 that we suggested several definitions of the term *process*, including:

- A program in execution.
- An instance of a program running on a computer.
- The entity that can be assigned to and executed on a processor.
- A unit of activity characterized by the execution of a sequence of instructions, a current state, and an associated set of system resources.

We can also think of a process as an entity that consists of a number of elements. Two essential elements of a process are **program code** (which may be shared with other processes that are executing the same program) and a **set of data** associated with that code. Let us suppose the processor begins to execute this program code, and we refer to this executing entity as a process. At any given point in time, *while the program is executing*, this process can be uniquely characterized by a number of elements, including the following:

- **Identifier:** A unique identifier associated with this process, to distinguish it from all other processes.
- **State:** If the process is currently executing, it is in the **running state**.
- **Priority:** Priority level relative to other processes.
- **Program counter:** The address of the next instruction in the program to be executed.
- **Memory pointers:** Include pointers to the program code and data associated with this process, plus any memory blocks shared with other processes.
- **Context data:** These are data that are present in registers in the processor while the process is executing.
- **I/O status information:** Includes outstanding I/O requests, I/O devices assigned to this process, a list of files in use by the process, and so on.
- **Accounting information:** May include the amount of processor time and clock time used, time limits, account numbers, and so on.

The information in the preceding list is stored in a data structure, typically called a **process control block** (see Figure 3.1), that is created and managed by the OS. The significant point about the process control block is that it contains sufficient information so it is possible to interrupt a running process and later resume execution as if the interruption had not occurred. The process control block is the key tool that enables the OS to support multiple processes and to provide for multiprocessing. When a process is interrupted, the current values of the program counter and the processor registers (context data) are saved in the appropriate fields of the corresponding process control block, and the state of the process is changed to some other value, such as *blocked* or *ready* (described subsequently). The OS is now free to put some other process in the running state. The program counter and context data for this process are loaded into the processor registers, and this process now begins to execute.

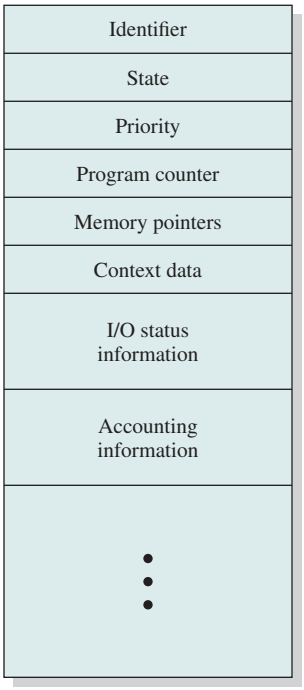



**Figure 3.1** **Simplified Process Control Block**

Thus, we can say that a process consists of program code and associated data plus a process control block. For a single-processor computer, at any given time, at most one process is executing and that process is in the *running* state.

## 3.2 PROCESS STATES

As just discussed, for a program to be executed, a process, or task, is created for that program. From the processor's point of view, it executes instructions from its repertoire in some sequence dictated by the changing values in the program counter register. Over time, the program counter may refer to code in different programs that are part of different processes. From the point of view of an individual program, its execution involves a sequence of instructions within that program.

We can characterize the behavior of an individual process by listing the sequence of instructions that execute for that process. Such a listing is referred to as a **trace** of the process. We can characterize behavior of the processor by showing how the traces of the various processes are interleaved.

Let us consider a very simple example. Figure 3.2 shows a memory layout of three processes. To simplify the discussion, we assume no use of virtual memory; thus all three processes are represented by programs that are fully loaded in main memory. In addition, there is a small **dispatcher** program that switches the processor from one process

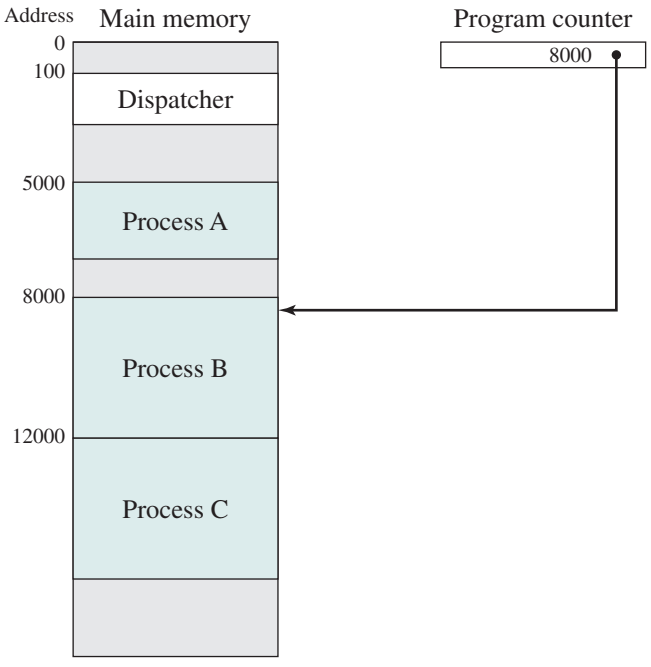



**Figure 3.2** **Snapshot of Example Execution (Figure 3.4) at Instruction Cycle 13**

| | | |
|---|---|---|
| 5000 | 8000 | 12000 |
| 5001 | 8001 | 12001 |
| 5002 | 8002 | 12002 |
| 5003 | 8003 | 12003 |
| 5004 | | 12004 |
| 5005 | | 12005 |
| 5006 | | 12006 |
| 5007 | | 12007 |
| 5008 | | 12008 |
| 5009 | | 12009 |
| 5010 | | 12010 |
| 5011 | | 12011 |

(a) Trace of process A (b) Trace of process B (c) Trace of process C

5000 = Starting address of program of process A
8000 = Starting address of program of process B
12000 = Starting address of program of process C

**Figure 3.3** **Traces of Processes of Figure 3.2**

to another. Figure 3.3 shows the traces of each of the processes during the early part of their execution. The first 12 instructions executed in processes A and C are shown. Process B executes four instructions, and we assume the fourth instruction invokes an I/O operation for which the process must wait.

Now let us view these traces from the processor's point of view. Figure 3.4 shows the interleaved traces resulting from the first 52 instruction cycles (for convenience, the instruction cycles are numbered). In this figure, the shaded areas represent code executed by the dispatcher. The same sequence of instructions is executed by the dispatcher in each instance because the same functionality of the dispatcher is being executed. We assume the OS only allows a process to continue execution for a maximum of six instruction cycles, after which it is interrupted; this prevents any single process from monopolizing processor time. As Figure 3.4 shows, the first six instructions of process A are executed, followed by a time-out and the execution of some

| | | | |
|---|---|---|---|
| 1 | 5000 | 27 | 12004 |
| 2 | 5001 | 28 | 12005 |
| 3 | 5002 | ----------Time-out | |
| 4 | 5003 | 29 | 100 |
| 5 | 5004 | 30 | 101 |
| 6 | 5005 | 31 | 102 |
| ----------Time-out | | 32 | 103 |
| 7 | 100 | 33 | 104 |
| 8 | 101 | 34 | 105 |
| 9 | 102 | 35 | 5006 |
| 10 | 103 | 36 | 5007 |
| 11 | 104 | 37 | 5008 |
| 12 | 105 | 38 | 5009 |
| 13 | 8000 | 39 | 5010 |
| 14 | 8001 | 40 | 5011 |
| 15 | 8002 | ----------Time-out | |
| 16 | 8003 | 41 | 100 |
| ----------I/O request | | 42 | 101 |
| 17 | 100 | 43 | 102 |
| 18 | 101 | 44 | 103 |
| 19 | 102 | 45 | 104 |
| 20 | 103 | 46 | 105 |
| 21 | 104 | 47 | 12006 |
| 22 | 105 | 48 | 12007 |
| 23 | 12000 | 49 | 12008 |
| 24 | 12001 | 50 | 12009 |
| 25 | 12002 | 51 | 12010 |
| 26 | 12003 | 52 | 12011 |
| | | ----------Time-out | |

100 = Starting address of dispatcher program
Shaded areas indicate execution of dispatcher process;
first and third columns count instruction cycles;
second and fourth columns show address of instruction being executed.

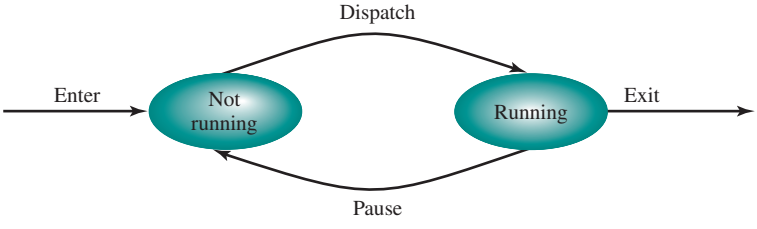
**Figure 3.4 Combined Trace of Processes of Figure 3.2**

code in the dispatcher, which executes six instructions before turning control to process B.[2] After four instructions are executed, process B requests an I/O action for which it must wait. Therefore, the processor stops executing process B and moves on, via the dispatcher, to process C. After a time-out, the processor moves back to process A. When this process times out, process B is still waiting for the I/O operation to complete, so the dispatcher moves on to process C again.

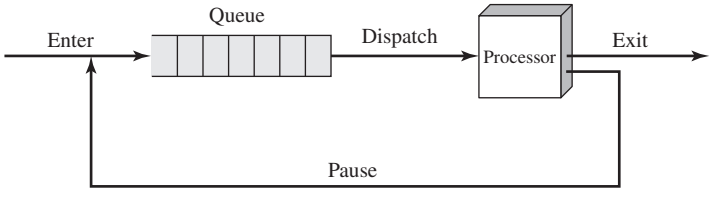
## A Two-State Process Model

The operating system's principal responsibility is controlling the execution of processes; this includes determining the interleaving pattern for execution and allocating resources to processes. The first step in designing an OS to control processes is to describe the behavior that we would like the processes to exhibit.

We can construct the simplest possible model by observing that, at any time, a process is either being executed by a processor, or it isn't. In this model, a process may be in one of the two states: Running or Not Running, as shown in Figure 3.5a. When the OS creates a new process, it creates a process control block for the process and enters that process into the system in the Not Running state. The process exists, is known to the OS, and is waiting for an opportunity to execute. From time to time, the currently running process will be interrupted, and the dispatcher portion of the OS will select some other process to run. The former process moves from the Running state to the Not Running state, and one of the other processes moves to the Running state.




(a) State transition diagram




(b) Queueing diagram

**Figure 3.5 Two-State Process Model**

[2]The small number of instructions executed for the processes and the dispatcher are unrealistically low; they are used in this simplified example to clarify the discussion.

From this simple model, we can already begin to appreciate some of the design elements of the OS. Each process must be represented in some way so the OS can keep track of it. That is, there must be some information relating to each process, including current state and location in memory; this is the process control block. Processes that are not running must be kept in some sort of queue, waiting their turn to execute. Figure 3.5b suggests a structure. There is a single queue in which each entry is a pointer to the process control block of a particular process. Alternatively, the queue may consist of a linked list of data blocks, in which each block represents one process. We will explore this latter implementation subsequently.

We can describe the behavior of the dispatcher in terms of this queueing diagram. A process that is interrupted is transferred to the queue of waiting processes. Alternatively, if the process has completed or aborted, it is discarded (exits the system). In either case, the dispatcher takes another process from the queue to execute.

## The Creation and Termination of Processes

Before refining our simple two-state model, it will be useful to discuss the creation and termination of processes; ultimately, and regardless of the model of process behavior that is used, the life of a process is bounded by its creation and termination.

***Process Creation*** When a new process is to be added to those currently being managed, the OS builds the data structures used to manage the process, and allocates address space in main memory to the process. We will describe these data structures in Section 3.3. These actions constitute the creation of a new process.

Four common events lead to the creation of a process, as indicated in Table 3.1. In a batch environment, a process is created in response to the submission of a job. In an interactive environment, a process is created when a new user attempts to log on. In both cases, the OS is responsible for the creation of the new process. An OS may also create a process on behalf of an application. For example, if a user requests that a file be printed, the OS can create a process that will manage the printing. The requesting process can thus proceed independently of the time required to complete the printing task.

Traditionally, the OS created all processes in a way that was transparent to the user or application program, and this is still commonly found with many contemporary

**Table 3.1** Reasons for Process Creation

| | |
|---|---|
| New batch job | The OS is provided with a batch job control stream, usually on tape or disk. When the OS is prepared to take on new work, it will read the next sequence of job control commands. |
| Interactive log-on | A user at a terminal logs on to the system. |
| Created by OS to provide a service | The OS can create a process to perform a function on behalf of a user program, without the user having to wait (e.g., a process to control printing). |
| Spawned by existing process | For purposes of modularity or to exploit parallelism, a user program can dictate the creation of a number of processes. |

operating systems. However, it can be useful to allow one process to cause the creation of another. For example, an application process may generate another process to receive data that the application is generating, and to organize those data into a form suitable for later analysis. The new process runs in parallel to the original process and is activated from time to time when new data are available. This arrangement can be very useful in structuring the application. As another example, a server process (e.g., print server, file server) may generate a new process for each request that it handles. When the OS creates a process at the explicit request of another process, the action is referred to as **process spawning**.

When one process spawns another, the former is referred to as the **parent process**, and the spawned process is referred to as the **child process**. Typically, the "related" processes need to communicate and cooperate with each other. Achieving this cooperation is a difficult task for the programmer; this topic will be discussed in Chapter 5.

***Process Termination*** Table 3.2 summarizes typical reasons for process termination. Any computer system must provide a means for a process to indicate its completion. A batch job should include a Halt instruction or an explicit OS service call for termination. In the former case, the Halt instruction will generate an interrupt to alert the OS that a process has completed. For an interactive application, the action of the user will indicate when the process is completed. For example, in a time-sharing system, the process for a particular user is to be terminated when the user logs off or turns off his or her terminal. On a personal computer or workstation, a user may quit an application (e.g., word processing or spreadsheet). All of these actions ultimately result in a service request to the OS to terminate the requesting process.

Additionally, a number of error and fault conditions can lead to the termination of a process. Table 3.2 lists some of the more commonly recognized conditions.[3]

Finally, in some operating systems, a process may be terminated by the process that created it, or when the parent process is itself terminated.

## A Five-State Model

If all processes were always ready to execute, then the queueing discipline suggested by Figure 3.5b would be effective. The queue is a first-in-first-out list and the processor operates in **round-robin** fashion on the available processes (each process in the queue is given a certain amount of time, in turn, to execute and then returned to the queue, unless blocked). However, even with the simple example that we have described, this implementation is inadequate: Some processes in the Not Running state are ready to execute, while others are blocked, waiting for an I/O operation to complete. Thus, using a single queue, the dispatcher could not just select the process at the oldest end of the queue. Rather, the dispatcher would have to scan the list looking for the process that is not blocked and that has been in the queue the longest.

[3]A forgiving operating system might, in some cases, allow the user to recover from a fault without terminating the process. For example, if a user requests access to a file and that access is denied, the operating system might simply inform the user that access is denied and allow the process to proceed.

**Table 3.2** Reasons for Process Termination

| | |
|---|---|
| Normal completion | The process executes an OS service call to indicate that it has completed running. |
| Time limit exceeded | The process has run longer than the specified total time limit. There are a number of possibilities for the type of time that is measured. These include total elapsed time ("wall clock time"), amount of time spent executing, and, in the case of an interactive process, the amount of time since the user last provided any input. |
| Memory unavailable | The process requires more memory than the system can provide. |
| Bounds violation | The process tries to access a memory location that it is not allowed to access. |
| Protection error | The process attempts to use a resource such as a file that it is not allowed to use, or it tries to use it in an improper fashion, such as writing to a read-only file. |
| Arithmetic error | The process tries a prohibited computation (such as division by zero) or tries to store numbers larger than the hardware can accommodate. |
| Time overrun | The process has waited longer than a specified maximum for a certain event to occur. |
| I/O failure | An error occurs during input or output, such as inability to find a file, failure to read or write after a specified maximum number of tries (when, for example, a defective area is encountered on a tape), or invalid operation (such as reading from the line printer). |
| Invalid instruction | The process attempts to execute a nonexistent instruction (often a result of branching into a data area and attempting to execute the data). |
| Privileged instruction | The process attempts to use an instruction reserved for the operating system. |
| Data misuse | A piece of data is of the wrong type or is not initialized. |
| Operator or OS intervention | For some reason, the operator or the operating system has terminated the process (e.g., if a deadlock exists). |
| Parent termination | When a parent terminates, the operating system may automatically terminate all of the offspring of that parent. |
| Parent request | A parent process typically has the authority to terminate any of its offspring. |

A more natural way to handle this situation is to split the Not Running state into two states: Ready and Blocked. This is shown in Figure 3.6. For good measure, we have added two additional states that will prove useful. The five states in this new diagram are as follows:

1. **Running:** The process that is currently being executed. For this chapter, we will assume a computer with a single processor, so at most, one process at a time can be in this state.
2. **Ready:** A process that is prepared to execute when given the opportunity.
3. **Blocked/Waiting:**[4] A process that cannot execute until some event occurs, such as the completion of an I/O operation.

[4] *Waiting* is a frequently used alternative term for *Blocked* as a process state. Generally, we will use *Blocked*, but the terms are interchangeable.

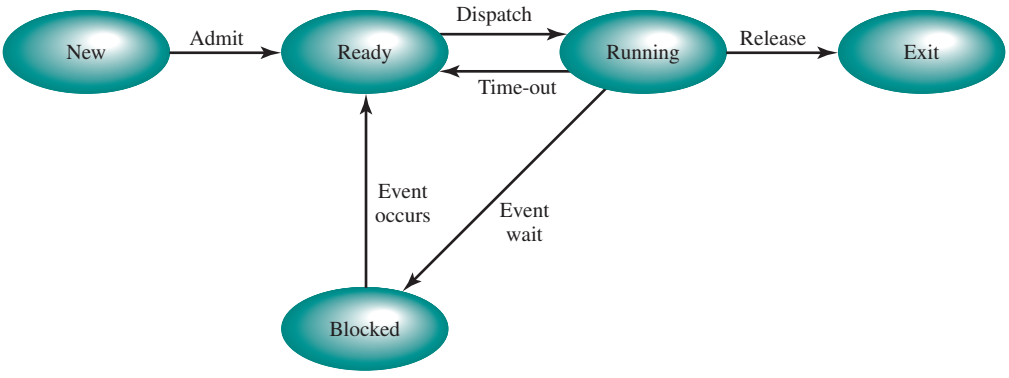



**Figure 3.6** **Five-State Process Model**

4. **New:** A process that has just been created but has not yet been admitted to the pool of executable processes by the OS. Typically, a new process has not yet been loaded into main memory, although its process control block has been created.
5. **Exit:** A process that has been released from the pool of executable processes by the OS, either because it halted or because it aborted for some reason.

The New and **Exit states** are useful constructs for process management. The **New state** corresponds to a process that has just been defined. For example, if a new user attempts to log on to a time-sharing system, or a new batch job is submitted for execution, the OS can define a new process in two stages. First, the OS performs the necessary housekeeping chores. An identifier is associated with the process. Any tables that will be needed to manage the process are allocated and built. At this point, the process is in the New state. This means that the OS has performed the necessary actions to create the process, but has not committed itself to the execution of the process. For example, the OS may limit the number of processes that may be in the system for reasons of performance or main memory limitation. While a process is in the new state, information concerning the process that is needed by the OS is maintained in control tables in main memory. However, the process itself is not in main memory. That is, the code of the program to be executed is not in main memory, and no space has been allocated for the data associated with that program. While the process is in the New state, the program remains in secondary storage, typically disk storage.[5]

Similarly, a process exits a system in two stages. First, a process is terminated when it reaches a natural completion point, when it aborts due to an unrecoverable error, or when another process with the appropriate authority causes the process to abort. Termination moves the process to the Exit state. At this point, the process is

[5]In the discussion in this paragraph, we ignore the concept of virtual memory. In systems that support virtual memory, when a process moves from New to Ready, its program code and data are loaded into virtual memory. Virtual memory was briefly discussed in Chapter 2 and will be examined in detail in Chapter 8.

no longer eligible for execution. The tables and other information associated with the job are temporarily preserved by the OS, which provides time for auxiliary or support programs to extract any needed information. For example, an accounting program may need to record the processor time and other resources utilized by the process for billing purposes. A utility program may need to extract information about the history of the process for purposes related to performance or utilization analysis. Once these programs have extracted the needed information, the OS no longer needs to maintain any data relating to the process, and the process is deleted from the system.

Figure 3.6 indicates the types of events that lead to each state transition for a process; the possible transitions are as follows:

- **Null → New:** A new process is created to execute a program. This event occurs for any of the reasons listed in Table 3.1.
- **New → Ready:** The OS will move a process from the New state to the **Ready state** when it is prepared to take on an additional process. Most systems set some limit based on the number of existing processes or the amount of virtual memory committed to existing processes. This limit assures there are not so many active processes as to degrade performance.
- **Ready → Running:** When it is time to select a process to run, the OS chooses one of the processes in the Ready state. This is the job of the scheduler or dispatcher. Scheduling is explored in Part Four.
- **Running → Exit:** The currently running process is terminated by the OS if the process indicates that it has completed or if it aborts. See Table 3.2.
- **Running → Ready:** The most common reason for this transition is that the running process has reached the maximum allowable time for uninterrupted execution; virtually all multiprogramming operating systems impose this type of time discipline. There are several other alternative causes for this transition, which are not implemented in all operating systems. Of particular importance is the case in which the OS assigns different levels of priority to different processes. Suppose, for example, process A is running at a given priority level, and process B, at a higher priority level, is blocked. If the OS learns that the event upon which process B has been waiting has occurred, this moving B to a ready state, then it can interrupt process A and dispatch process B. We say that the OS has **preempted** process A.[6] Finally, a process may voluntarily release control of the processor. An example is a background process that periodically performs some accounting or maintenance function.
- **Running → Blocked:** A process is put in the **Blocked state** if it requests something for which it must wait. A request to the OS is usually in the form of a system service call; that is, a call from the running program to a procedure that is part of the operating system code. For example, a process may request a service from the OS that the OS is not prepared to perform immediately. It can request

[6] In general, the term *preemption* is defined to be the reclaiming of a resource from a process before the process has finished using it. In this case, the resource is the processor itself. The process is executing and could continue to execute, but is preempted so another process can be executed.

a resource, such as a file or a shared section of virtual memory, that is not immediately available. Or the process may initiate an action, such as an I/O operation, that must be completed before the process can continue. When processes communicate with each other, a process may be blocked when it is waiting for another process to provide data, or waiting for a message from another process.

- **Blocked → Ready:** A process in the Blocked state is moved to the Ready state when the event for which it has been waiting occurs.
- **Ready → Exit:** For clarity, this transition is not shown on the state diagram. In some systems, a parent may terminate a child process at any time. Also, if a parent terminates, all child processes associated with that parent may be terminated.
- **Blocked → Exit:** The comments under the preceding item apply.

Returning to our simple example, Figure 3.7 shows the transition of each process among the states. Figure 3.8a suggests the way in which a queueing discipline might be implemented with two queues: a Ready queue and a Blocked queue. As each process is admitted to the system, it is placed in the Ready queue. When it is time for the OS to choose another process to run, it selects one from the Ready queue. In the absence of any priority scheme, this can be a simple first-in-first-out queue. When a running process is removed from execution, it is either terminated or placed in the Ready or Blocked queue, depending on the circumstances. Finally, when an event occurs, any process in the Blocked queue that has been waiting on that event only is moved to the Ready queue.

This latter arrangement means that, when an event occurs, the OS must scan the entire blocked queue, searching for those processes waiting on that event. In a large OS, there could be hundreds or even thousands of processes in that queue. Therefore, it would be more efficient to have a number of queues, one for each event. Then, when the event occurs, the entire list of processes in the appropriate queue can be moved to the Ready state (see Figure 3.8b).

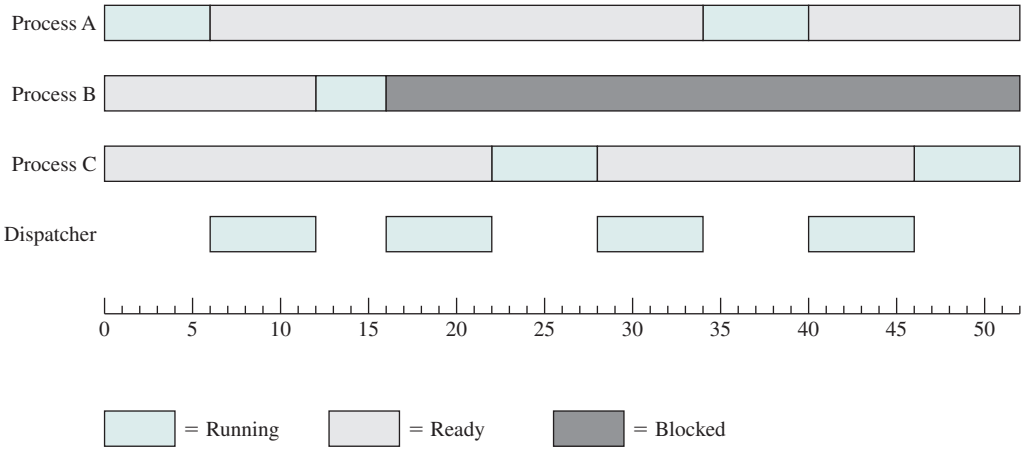
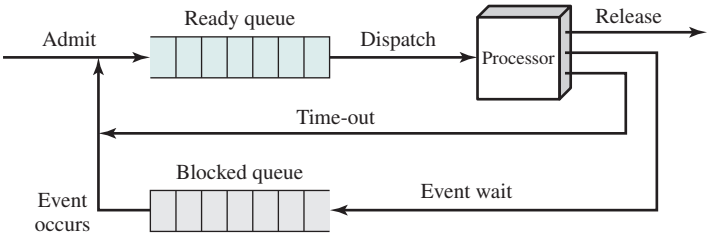
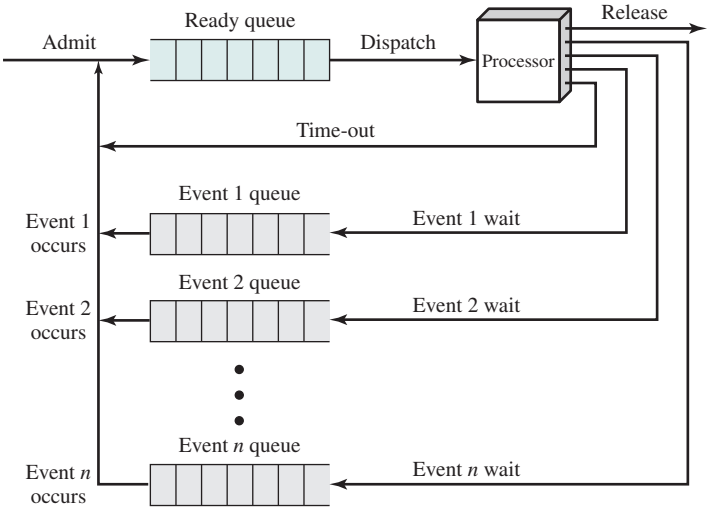



**Figure 3.7 Process States for the Trace of Figure 3.4**




(a) Single blocked queue




(b) Multiple blocked queues

**Figure 3.8 Queueing Model for Figure 3.6**

One final refinement: If the dispatching of processes is dictated by a priority scheme, then it would be convenient to have a number of Ready queues, one for each priority level. The OS could then readily determine which is the highest-priority ready process that has been waiting the longest.

## Suspended Processes

***THE NEED FOR SWAPPING*** The three principal states just described (Ready, Running, and Blocked) provide a systematic way of modeling the behavior of processes and guide the implementation of the OS. Some operating systems are constructed using just these three states.

However, there is good justification for adding other states to the model. To see the benefit of these new states, consider a system that does not employ virtual memory. Each process to be executed must be loaded fully into main memory. Thus, in Figure 3.8b, all of the processes in all of the queues must be resident in main memory.

Recall that the reason for all of this elaborate machinery is that I/O activities are much slower than computation, and therefore the processor in a uniprogramming system is idle most of the time. But the arrangement of Figure 3.8b does not entirely solve the problem. It is true that, in this case, memory holds multiple processes and the processor can move to another process when one process is blocked. But the processor is so much faster than I/O that it will be common for all of the processes in memory to be waiting for I/O. Thus, even with multiprogramming, a processor could be idle most of the time.

What to do? Main memory could be expanded to accommodate more processes. But there are two flaws in this approach. First, there is a cost associated with main memory, which, though small on a per-byte basis, begins to add up as we get into the gigabytes of storage. Second, the appetite of programs for memory has grown as fast as the cost of memory has dropped. So larger memory results in larger processes, not more processes.

Another solution is swapping, which involves moving part or all of a process from main memory to disk. When none of the processes in main memory is in the Ready state, the OS swaps one of the blocked processes out on to disk into a suspend queue. This is a queue of existing processes that have been temporarily kicked out of main memory, or suspended. The OS then brings in another process from the suspend queue or it honors a new-process request. Execution then continues with the newly arrived process.

Swapping, however, is an I/O operation, and therefore there is the potential for making the problem worse, not better. But because disk I/O is generally the fastest I/O on a system (e.g., compared to tape or printer I/O), swapping will usually enhance performance.

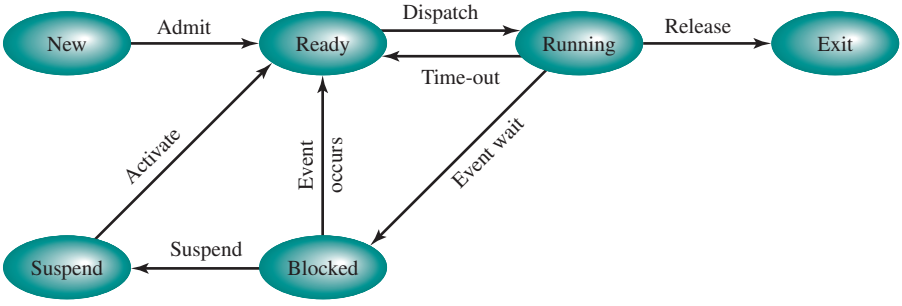
With the use of swapping as just described, one other state must be added to our process behavior model (see Figure 3.9a): the Suspend state. When all of the processes in main memory are in the Blocked state, the OS can suspend one process by putting it in the Suspend state and transferring it to disk. The space that is freed in main memory can then be used to bring in another process.

When the OS has performed a swapping-out operation, it has two choices for selecting a process to bring into main memory: It can admit a newly created process, or it can bring in a previously suspended process. It would appear that the preference should be to bring in a previously suspended process, to provide it with service rather than increasing the total load on the system.

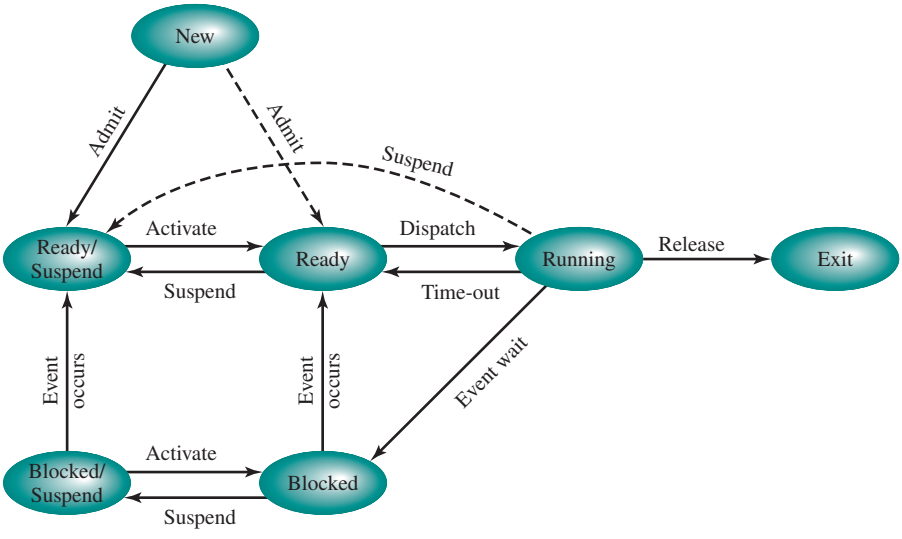
But this line of reasoning presents a difficulty. All of the processes that have been suspended were in the Blocked state at the time of suspension. It clearly would not do any good to bring a blocked process back into main memory, because it is still not ready for execution. Recognize, however, that each process in the Suspend state was originally blocked on a particular event. When that event occurs, the process is not blocked and is potentially available for execution.

Therefore, we need to rethink this aspect of the design. There are two independent concepts here: whether a process is waiting on an event (blocked or not), and whether a process has been swapped out of main memory (suspended or not). To accommodate this $2 \times 2$ combination, we need four states:

1. **Ready:** The process is in main memory and available for execution.
2. **Blocked:** The process is in main memory and awaiting an event.




(a) With one Suspend state




(b) With two Suspend states

**Figure 3.9** **Process State Transition Diagram with Suspend States**

3. **Blocked/Suspend:** The process is in secondary memory and awaiting an event.
4. **Ready/Suspend:** The process is in secondary memory but is available for execution as soon as it is loaded into main memory.

Before looking at a state transition diagram that encompasses the two new suspend states, one other point should be mentioned. The discussion so far has assumed that virtual memory is not in use, and that a process is either all in main memory or all out of main memory. With a virtual memory scheme, it is possible to execute a process that is only partially in main memory. If reference is made to a process address that is not in main memory, then the appropriate portion of the process can be brought in. The use of virtual memory would appear to eliminate the need for explicit swapping, because any desired address in any desired process can be moved

in or out of main memory by the memory management hardware of the processor. However, as we shall see in Chapter 8, the performance of a virtual memory system can collapse if there is a sufficiently large number of active processes, all of which are partially in main memory. Therefore, even in a virtual memory system, the OS will need to swap out processes explicitly and completely from time to time in the interests of performance.

Let us look now, in Figure 3.9b, at the state transition model that we have developed. (The dashed lines in the figure indicate possible but not necessary transitions.) Important new transitions are the following:

- **Blocked → Blocked/Suspend:** If there are no ready processes, then at least one blocked process is swapped out to make room for another process that is not blocked. This transition can be made even if there are ready processes available. In particular, if the OS determines that the currently running process, or a ready process that it would like to dispatch, requires more main memory to maintain adequate performance, a blocked process will be suspended.
- **Blocked/Suspend → Ready/Suspend:** A process in the Blocked/Suspend state is moved to the Ready/Suspend state when the event for which it has been waiting occurs. Note this requires that the state information concerning suspended processes must be accessible to the OS.
- **Ready/Suspend → Ready:** When there are no ready processes in main memory, the OS will need to bring one in to continue execution. In addition, it might be the case that a process in the Ready/Suspend state has higher priority than any of the processes in the Ready state. In that case, the OS designer may dictate that it is more important to get at the higher-priority process than to minimize swapping.
- **Ready → Ready/Suspend:** Normally, the OS would prefer to suspend a blocked process rather than a ready one, because the ready process can now be executed, whereas the blocked process is taking up main memory space and cannot be executed. However, it may be necessary to suspend a ready process if that is the only way to free up a sufficiently large block of main memory. Also, the OS may choose to suspend a lower–priority ready process rather than a higher–priority blocked process if it believes that the blocked process will be ready soon.

Several other transitions that are worth considering are the following:

- **New → Ready/Suspend and New → Ready:** When a new process is created, it can either be added to the Ready queue or the Ready/Suspend queue. In either case, the OS must create a process control block and allocate an address space to the process. It might be preferable for the OS to perform these housekeeping duties at an early time, so it can maintain a large pool of processes that are not blocked. With this strategy, there would often be insufficient room in main memory for a new process; hence the use of the (New → Ready/Suspend) transition. On the other hand, we could argue that a just-in-time philosophy of creating processes as late as possible reduces OS overhead, and allows that OS to perform the process creation duties at a time when the system is clogged with blocked processes anyway.

- **Blocked/Suspend → Blocked:** Inclusion of this transition may seem to be poor design. After all, if a process is not ready to execute and is not already in main memory, what is the point of bringing it in? But consider the following scenario: A process terminates, freeing up some main memory. There is a process in the (Blocked/Suspend) queue with a higher priority than any of the processes in the (Ready/Suspend) queue and the OS has reason to believe that the blocking event for that process will occur soon. Under these circumstances, it would seem reasonable to bring a blocked process into main memory in preference to a ready process.
- **Running → Ready/Suspend:** Normally, a running process is moved to the Ready state when its time allocation expires. If, however, the OS is preempting the process because a higher-priority process on the Blocked/Suspend queue has just become unblocked, the OS could move the running process directly to the (Ready/Suspend) queue and free some main memory.
- **Any State → Exit:** Typically, a process terminates while it is running, either because it has completed or because of some fatal fault condition. However, in some operating systems, a process may be terminated by the process that created it or when the parent process is itself terminated. If this is allowed, then a process in any state can be moved to the Exit state.

***Other Uses of Suspension*** So far, we have equated the concept of a suspended process with that of a process that is not in main memory. A process that is not in main memory is not immediately available for execution, whether or not it is awaiting an event.

We can generalize the concept of a suspended process. Let us define a suspended process as having the following characteristics:

1. The process is not immediately available for execution.
2. The process may or may not be waiting on an event. If it is, this blocked condition is independent of the suspend condition, and occurrence of the blocking event does not enable the process to be executed immediately.
3. The process was placed in a suspended state by an agent: either itself, a parent process, or the OS, for the purpose of preventing its execution.
4. The process may not be removed from this state until the agent explicitly orders the removal.

Table 3.3 lists some reasons for the suspension of a process. One reason we have discussed is to provide memory space either to bring in a Ready/Suspended process or to increase the memory allocated to other Ready processes. The OS may have other motivations for suspending a process. For example, an auditing or tracing process may be employed to monitor activity on the system; the process may be used to record the level of utilization of various resources (processor, memory, channels) and the rate of progress of the user processes in the system. The OS, under operator control, may turn this process on and off from time to time. If the OS detects or suspects a problem, it may suspend a process. One example of this is deadlock, which will be discussed in Chapter 6. As another example, a problem

**Table 3.3** Reasons for Process Suspension

| | |
|---|---|
| Swapping | The OS needs to release sufficient main memory to bring in a process that is ready to execute. |
| Other OS reason | The OS may suspend a background or utility process or a process that is suspected of causing a problem. |
| Interactive user request | A user may wish to suspend execution of a program for purposes of debugging or in connection with the use of a resource. |
| Timing | A process may be executed periodically (e.g., an accounting or system monitoring process) and may be suspended while waiting for the next time interval. |
| Parent process request | A parent process may wish to suspend execution of a descendent to examine or modify the suspended process, or to coordinate the activity of various descendants. |

is detected on a communications line, and the operator has the OS suspend the process that is using the line while some tests are run.

Another set of reasons concerns the actions of an interactive user. For example, if a user suspects a bug in the program, he or she may debug the program by suspending its execution, examining and modifying the program or data, and resuming execution. Or there may be a background process that is collecting trace or accounting statistics, which the user may wish to be able to turn on and off.

Timing considerations may also lead to a swapping decision. For example, if a process is to be activated periodically but is idle most of the time, then it should be swapped out between uses. A program that monitors utilization or user activity is an example.

Finally, a parent process may wish to suspend a descendant process. For example, process A may spawn process B to perform a file read. Subsequently, process B encounters an error in the file read procedure and reports this to process A. Process A suspends process B to investigate the cause.

In all of these cases, the activation of a suspended process is requested by the agent that initially requested the suspension.

## 3.3 PROCESS DESCRIPTION

The OS controls events within the computer system. It schedules and dispatches processes for execution by the processor, allocates resources to processes, and responds to requests by user processes for basic services. Fundamentally, we can think of the OS as that entity that manages the use of system resources by processes.

This concept is illustrated in Figure 3.10. In a multiprogramming environment, there are a number of processes ($P_1, \ldots, P_n$) that have been created and exist in virtual memory. Each process, during the course of its execution, needs access to certain system resources, including the processor, I/O devices, and main memory. In the figure, process $P_1$ is running; at least part of the process is in main memory, and it has control of two I/O devices. Process $P_2$ is also in main memory, but is blocked waiting for an I/O device allocated to $P_1$. Process $P_n$ has been swapped out and is therefore suspended.

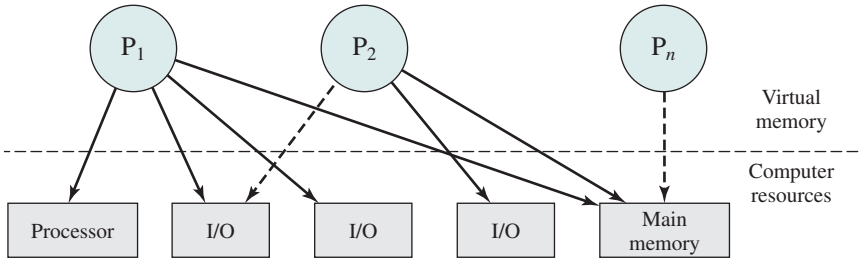



**Figure 3.10** **Processes and Resources (resource allocation at one snapshot in time)**

We will explore the details of the management of these resources by the OS on behalf of the processes in later chapters. Here we are concerned with a more fundamental question: What information does the OS need to control processes and manage resources for them?

## Operating System Control Structures

If the OS is to manage processes and resources, it must have information about the current status of each process and resource. The universal approach to providing this information is straightforward: The OS constructs and maintains tables of information about each entity that it is managing. A general idea of the scope of this effort is indicated in Figure 3.11, which shows four different types of tables maintained by the OS: memory, I/O, file, and process. Although the details will differ from one OS to another, fundamentally, all operating systems maintain information in these four categories.

**Memory tables** are used to keep track of both main (real) and secondary (virtual) memory. Some of main memory is reserved for use by the OS; the remainder is available for use by processes. Processes are maintained on secondary memory using some sort of virtual memory or simple swapping mechanism. The memory tables must include the following information:

- The allocation of main memory to processes
- The allocation of secondary memory to processes
- Any protection attributes of blocks of main or virtual memory, such as which processes may access certain shared memory regions
- Any information needed to manage virtual memory

We will examine the information structures for memory management in detail in Part Three.

**I/O tables** are used by the OS to manage the I/O devices and channels of the computer system. At any given time, an I/O device may be available or assigned to a particular process. If an I/O operation is in progress, the OS needs to know the status of the I/O operation and the location in main memory being used as the source or destination of the I/O transfer. I/O management will be examined in Chapter 11.

The OS may also maintain **file tables**. These tables provide information about the existence of files, their location on secondary memory, their current status, and

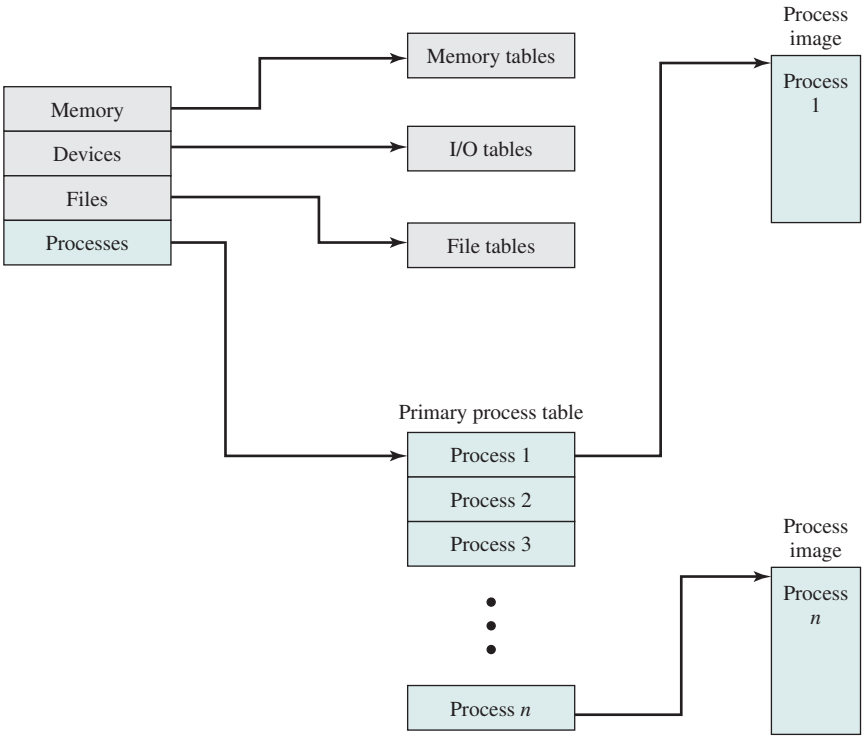



**Figure 3.11 General Structure of Operating System Control Tables**

other attributes. Much, if not all, of this information may be maintained and used by a file management system, in which case the OS has little or no knowledge of files. In other operating systems, much of the detail of file management is managed by the OS itself. This topic will be explored in Chapter 12.

Finally, the OS must maintain **process tables** to manage processes. The remainder of this section is devoted to an examination of the required process tables. Before proceeding to this discussion, two additional points should be made. First, although Figure 3.11 shows four distinct sets of tables, it should be clear that these tables must be linked or cross-referenced in some fashion. Memory, I/O, and files are managed on behalf of processes, so there must be some reference to these resources, directly or indirectly, in the process tables. The files referred to in the file tables are accessible via an I/O device and will, at some times, be in main or virtual memory. The tables themselves must be accessible by the OS, and therefore are subject to memory management.

Second, how does the OS know to create the tables in the first place? Clearly, the OS must have some knowledge of the basic environment, such as how much main memory exists, what are the I/O devices and what are their identifiers, and so on. This is an issue of configuration. That is, when the OS is initialized, it must have access to some configuration data that define the basic environment, and these data must be created outside the OS, with human assistance or by some autoconfiguration software.

## Process Control Structures

Consider what the OS must know if it is to manage and control a process. First, it must know where the process is located; second, it must know the attributes of the process that are necessary for its management (e.g., process ID and process state).

***Process Location*** Before we can deal with the questions of where a process is located or what its attributes are, we need to address an even more fundamental question: What is the physical manifestation of a process? At a minimum, a process must include a program or set of programs to be executed. Associated with these programs is a set of data locations for local and global variables and any defined constants. Thus, a process will consist of at least sufficient memory to hold the programs and data of that process. In addition, the execution of a program typically involves a stack (see Appendix P) that is used to keep track of procedure calls and parameter passing between procedures. Finally, each process has associated with it a number of attributes that are used by the OS for process control. Typically, the collection of attributes is referred to as a *process control block*.[7] We can refer to this collection of program, data, stack, and attributes as the **process image** (see Table 3.4).

The location of a process image will depend on the memory management scheme being used. In the simplest case, the process image is maintained as a contiguous, or continuous, block of memory. This block is maintained in secondary memory, usually disk. So that the OS can manage the process, at least a small portion of its image must be maintained in main memory. To execute the process, the entire process image must be loaded into main memory, or at least virtual memory. Thus, the OS needs to know the location of each process on disk and, for each such process that is in main memory, the location of that process in main memory. We saw a slightly more complex variation on this scheme with the CTSS OS in Chapter 2. With CTSS, when a process is swapped out, part of the process image may remain in main memory. Thus, the OS must keep track of which portions of the image of each process are still in main memory.

**Table 3.4** Typical Elements of a Process Image

| |
|---|
| **User Data**<br>The modifiable part of the user space. May include program data, a user stack area, and programs that may be modified. |
| **User Program**<br>The program to be executed. |
| **Stack**<br>Each process has one or more last-in-first-out (LIFO) stacks associated with it. A stack is used to store parameters and calling addresses for procedure and system calls. |
| **Process Control Block**<br>Data needed by the OS to control the process (see Table 3.5). |

[7]Other commonly used names for this data structure are *task control block*, *process descriptor*, and *task descriptor.*

Modern operating systems presume paging hardware that allows noncontiguous physical memory to support partially resident processes.[8] At any given time, a portion of a process image may be in main memory, with the remainder in secondary memory.[9] Therefore, process tables maintained by the OS must show the location of each page of each process image.

Figure 3.11 depicts the structure of the location information in the following way. There is a primary process table with one entry for each process. Each entry contains, at least, a pointer to a process image. If the process image contains multiple blocks, this information is contained directly in the primary process table or is available by cross-reference to entries in memory tables. Of course, this depiction is generic; a particular OS will have its own way of organizing the location information.

***Process Attributes*** A sophisticated multiprogramming system requires a great deal of information about each process. As was explained, this information can be considered to reside in a process control block. Different systems will organize this information in different ways, and several examples of this appear at the end of this chapter and the next. For now, let us simply explore the type of information that might be of use to an OS without considering in any detail how that information is organized.

Table 3.5 lists the typical categories of information required by the OS for each process. You may be somewhat surprised at the quantity of information required. As you gain a greater appreciation of the responsibilities of the OS, this list should appear more reasonable.

We can group the process control block information into three general categories:

1. Process identification
2. Processor state information
3. Process control information

With respect to **process identification**, in virtually all operating systems, each process is assigned a unique numeric identifier, which may simply be an index into the primary process table (see Figure 3.11); otherwise there must be a mapping that allows the OS to locate the appropriate tables based on the process identifier. This identifier is useful in several ways. Many of the other tables controlled by the OS may use process identifiers to cross-reference process tables. For example, the memory tables may be organized so as to provide a map of main memory with an indication of which process is assigned to each region. Similar references will appear in I/O and file tables. When processes communicate with one another, the

[8]A brief overview of the concepts of pages, segments, and virtual memory is provided in the subsection on memory management in Section 2.3.

[9]This brief discussion slides over some details. In particular, in a system that uses virtual memory, all of the process image for an active process is always in secondary memory. When a portion of the image is loaded into main memory, it is copied rather than moved. Thus, the secondary memory retains a copy of all segments and/or pages. However, if the main memory portion of the image is modified, the secondary copy will be out of date until the main memory portion is copied back onto disk.

**Table 3.5** Typical Elements of a Process Control Block

**Process Identification**

**Identifiers**

Numeric identifiers that may be stored with the process control block include

- Identifier of this process.
- Identifier of the process that created this process (parent process).
- User identifier.

**Processor State Information**

**User-Visible Registers**

A user-visible register is one that may be referenced by means of the machine language that the processor executes while in user mode. Typically, there are from 8 to 32 of these registers, although some RISC implementations have over 100.

**Control and Status Registers**

These are a variety of processor registers that are employed to control the operation of the processor. These include:

- **Program counter:** Contains the address of the next instruction to be fetched.
- **Condition codes:** Result of the most recent arithmetic or logical operation (e.g., sign, zero, carry, equal, overflow).
- **Status information:** Includes interrupt enabled/disabled flags, execution mode.

**Stack Pointers**

Each process has one or more last-in-first-out (LIFO) system stacks associated with it. A stack is used to store parameters and calling addresses for procedure and system calls. The stack pointer points to the top of the stack.

**Process Control Information**

**Scheduling and State Information**

This is information that is needed by the operating system to perform its scheduling function. Typical items of information include:

- **Process state:** Defines the readiness of the process to be scheduled for execution (e.g., running, ready, waiting, halted).
- **Priority:** One or more fields may be used to describe the scheduling priority of the process. In some systems, several values are required (e.g., default, current, highest allowable).
- **Scheduling-related information:** This will depend on the scheduling algorithm used. Examples are the amount of time that the process has been waiting and the amount of time that the process executed the last time it was running.
- **Event:** Identity of event the process is awaiting before it can be resumed.

**Data Structuring**

A process may be linked to other process in a queue, ring, or some other structure. For example, all processes in a waiting state for a particular priority level may be linked in a queue. A process may exhibit a parent–child (creator–created) relationship with another process. The process control block may contain pointers to other processes to support these structures.

**Interprocess Communication**

Various flags, signals, and messages may be associated with communication between two independent processes. Some or all of this information may be maintained in the process control block.

**Process Privileges**

Processes are granted privileges in terms of the memory that may be accessed and the types of instructions that may be executed. In addition, privileges may apply to the use of system utilities and services.

**Memory Management**

This section may include pointers to segment and/or page tables that describe the virtual memory assigned to this process.

**Resource Ownership and Utilization**

Resources controlled by the process may be indicated, such as opened files. A history of utilization of the processor or other resources may also be included; this information may be needed by the scheduler.

process identifier informs the OS of the destination of a particular communication. When processes are allowed to create other processes, identifiers indicate the parent and descendants of each process.

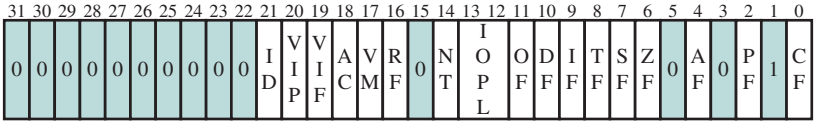
In addition to these process identifiers, a process may be assigned a user identifier that indicates the user responsible for the job.

**Processor state information** consists of the contents of processor registers. While a process is running, of course, the information is in the registers. When a process is interrupted, all of this register information must be saved so it can be restored when the process resumes execution. The nature and number of registers involved depend on the design of the processor. Typically, the register set will include user-visible registers, control and status registers, and stack pointers. These are described in Chapter 1.

Of particular note, all processor designs include a register or set of registers, often known as the **program status word** (PSW), that contains status information. The PSW typically contains condition codes plus other status information. A good example of a processor status word is that on Intel x86 processors, referred to as the EFLAGS register (shown in Figure 3.12 and Table 3.6). This structure is used by any OS (including UNIX and Windows) running on an x86 processor.

The third major category of information in the process control block can be called, for want of a better name, **process control information**. This is the additional information needed by the OS to control and coordinate the various active processes. The last part of Table 3.5 indicates the scope of this information. As we examine the details of operating system functionality in succeeding chapters, the need for the various items on this list should become clear.

Figure 3.13 suggests the structure of process images in virtual memory. Each process image consists of a process control block, a user stack, the private address space of the process, and any other address space that the process shares with other processes. In




X ID = Identification flag
X VIP = Virtual interrupt pending
X VIF = Virtual interrupt flag
X AC = Alignment check
X VM = Virtual 8086 mode
X RF = Resume flag
X NT = Nested task flag
X IOPL = I/O privilege level
S OF = Overflow flag

C DF = Direction flag
X IF = Interrupt enable flag
X TF = Trap flag
S SF = Sign flag
S ZF = Zero flag
S AF = Auxiliary carry flag
S PF = Parity flag
S CF = Carry flag

S Indicates a Status Flag
C Indicates a Control Flag
X Indicates a System Flag
Shaded bits are reserved

**Figure 3.12 x86 EFLAGS Register**

**Table 3.6** x86 EFLAGS Register Bits

**Status Flags (condition codes)**

**AF (Auxiliary carry flag)**
Represents carrying or borrowing between half-bytes of an 8-bit arithmetic or logic operation using the AL register.

**CF (Carry flag)**
Indicates carrying out or borrowing into the leftmost bit position following an arithmetic operation; also modified by some of the shift and rotate operations.

**OF (Overflow flag)**
Indicates an arithmetic overflow after an addition or subtraction.

**PF (Parity flag)**
Parity of the result of an arithmetic or logic operation. 1 indicates even parity; 0 indicates odd parity.

**SF (Sign flag)**
Indicates the sign of the result of an arithmetic or logic operation.

**ZF (Zero flag)**
Indicates that the result of an arithmetic or logic operation is 0.

**Control Flag**

**DF (Direction flag)**
Determines whether string processing instructions increment or decrement the 16-bit half-registers SI and DI (for 16-bit operations) or the 32-bit registers ESI and EDI (for 32-bit operations).

**System Flags (should not be modified by application programs)**

**AC (Alignment check)**
Set if a word or doubleword is addressed on a nonword or nondoubleword boundary.

**ID (Identification flag)**
If this bit can be set and cleared, this processor supports the CPUID instruction. This instruction provides information about the vendor, family, and model.

**RF (Resume flag)**
Allows the programmer to disable debug exceptions so the instruction can be restarted after a debug exception without immediately causing another debug exception.

**IOPL (I/O privilege level)**
When set, it causes the processor to generate an exception on all accesses to I/O devices during protected mode operation.

**IF (Interrupt enable flag)**
When set, the processor will recognize external interrupts.

**TF (Trap flag)**
When set, it causes an interrupt after the execution of each instruction. This is used for debugging.

**NT (Nested task flag)**
Indicates that the current task is nested within another task in protected mode operation.

**VM (Virtual 8086 mode)**
Allows the programmer to enable or disable virtual 8086 mode, which determines whether the processor runs as an 8086 machine.

**VIP (Virtual interrupt pending)**
Used in virtual 8086 mode to indicate that one or more interrupts are awaiting service.

**VIF (Virtual interrupt flag)**
Used in virtual 8086 mode instead of IF.

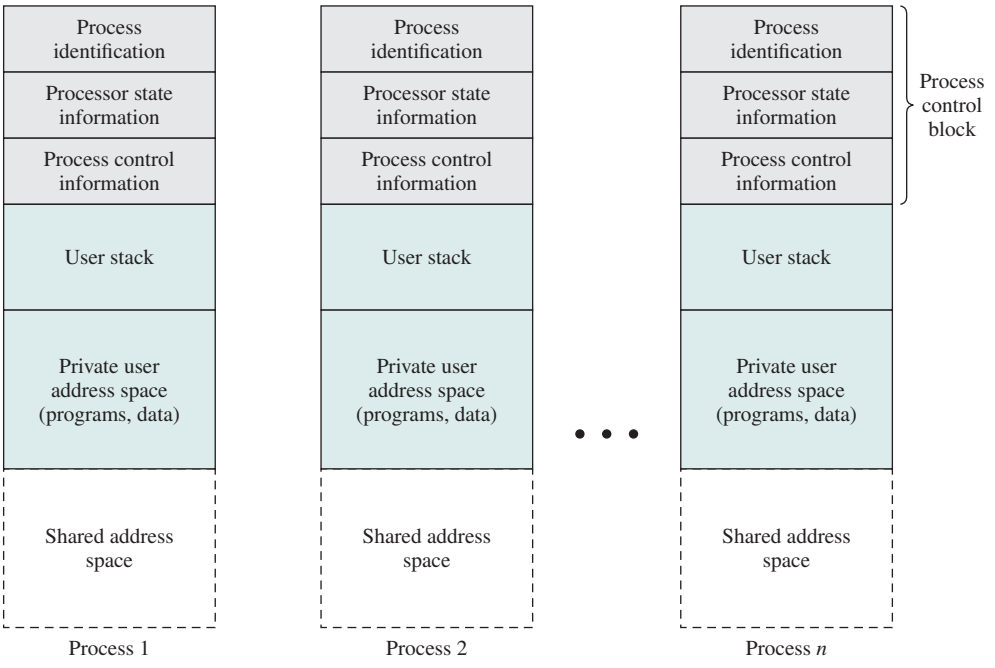



**Figure 3.13 User Processes in Virtual Memory**

the figure, each process image appears as a contiguous range of addresses. In an actual implementation, this may not be the case; it will depend on the memory management scheme and the way in which control structures are organized by the OS.

As indicated in Table 3.5, the process control block may contain structuring information, including pointers that allow the linking of process control blocks. Thus, the queues that were described in the preceding section could be implemented as linked lists of process control blocks. For example, the queueing structure of Figure 3.8a could be implemented as suggested in Figure 3.14.

***The Role of the Process Control Block*** The process control block is the most important data structure in an OS. Each process control block contains all of the information about a process that is needed by the OS. The blocks are read and/or modified by virtually every module in the OS, including those involved with scheduling, resource allocation, interrupt processing, and performance monitoring and analysis. One can say that the set of process control blocks defines the state of the OS.

This brings up an important design issue. A number of routines within the OS will need access to information in process control blocks. The provision of direct access to these tables is not difficult. Each process is equipped with a unique ID, and this can be used as an index into a table of pointers to the process control blocks. The difficulty is not access but rather protection. Two problems present themselves:

- A bug in a single routine, such as an interrupt handler, could damage process control blocks, which could destroy the system's ability to manage the affected processes.

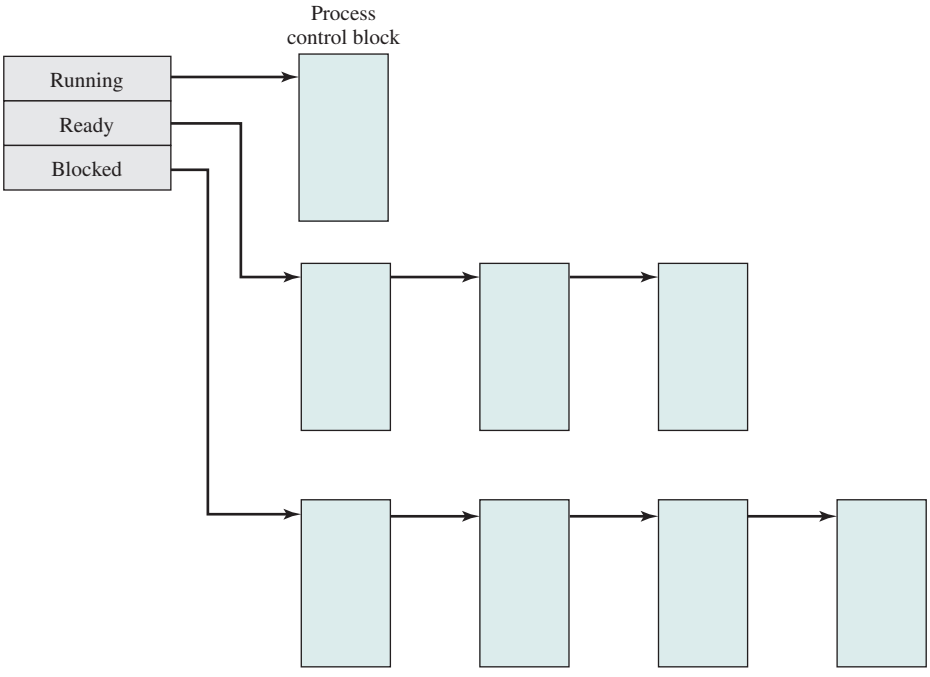



**Figure 3.14** **Process List Structures**

- A design change in the structure or semantics of the process control block could affect a number of modules in the OS.

These problems can be addressed by requiring all routines in the OS to go through a handler routine, the only job of which is to protect process control blocks, and which is the sole arbiter for reading and writing these blocks. The trade-off in the use of such a routine involves performance issues and the degree to which the remainder of the system software can be trusted to be correct.

## 3.4 PROCESS CONTROL

### Modes of Execution

Before continuing with our discussion of the way in which the OS manages processes, we need to distinguish between the mode of processor execution normally associated with the OS and that normally associated with user programs. Most processors support at least two modes of execution. Certain instructions can only be executed in the more-privileged mode. These would include reading or altering a control register, such as the PSW, primitive I/O instructions, and instructions that relate to memory management. In addition, certain regions of memory can only be accessed in the more-privileged mode.

**Table 3.7** Typical Functions of an Operating System Kernel

| Process Management |
|---|
| • Process creation and termination |
| • Process scheduling and dispatching |
| • Process switching |
| • Process synchronization and support for interprocess communication |
| • Management of process control blocks |
| **Memory Management** |
| • Allocation of address space to processes |
| • Swapping |
| • Page and segment management |
| **I/O Management** |
| • Buffer management |
| • Allocation of I/O channels and devices to processes |
| **Support Functions** |
| • Interrupt handling |
| • Accounting |
| • Monitoring |

The less-privileged mode is often referred to as the **user mode**, because user programs typically would execute in this mode. The more-privileged mode is referred to as the **system mode**, **control mode**, or **kernel mode**. This last term refers to the kernel of the OS, which is that portion of the OS that encompasses the important system functions. Table 3.7 lists the functions typically found in the kernel of an OS.

The reason for using two modes should be clear. It is necessary to protect the OS and key operating system tables, such as process control blocks, from interference by user programs. In the kernel mode, the software has complete control of the processor and all its instructions, registers, and memory. This level of control is not necessary, and for safety is not desirable for user programs.

Two questions arise: How does the processor know in which mode it is to be executing, and how is the mode changed? Regarding the first question, typically there is a bit in the PSW that indicates the mode of execution. This bit is changed in response to certain events. Typically, when a user makes a call to an operating system service or when an interrupt triggers execution of an operating system routine, the mode is set to the kernel mode and, upon return from the service to the user process, the mode is set to user mode. As an example, consider the Intel Itanium processor, which implements the 64-bit IA-64 architecture. The processor has a processor status register (PSR) that includes a 2-bit CPL (current privilege level) field. Level 0 is the most privileged level, while level 3 is the least privileged level. Most operating systems, such as Linux, use level 0 for the kernel and one other level for user mode. When an interrupt occurs, the processor clears most of the bits in the psr, including the CPL field. This automatically sets the CPL to

level 0. At the end of the interrupt-handling routine, the final instruction that is executed is IRT (interrupt return). This instruction causes the processor to restore the PSR of the interrupted program, which restores the privilege level of that program. A similar sequence occurs when an application places a system call. For the Itanium, an application places a system call by placing the system call identifier and the system call arguments in a predefined area, then executing a special instruction that has the effect of interrupting execution at the user level and transferring control to the kernel.

### Process Creation

In Section 3.2, we discussed the events that lead to the creation of a new process. Having discussed the data structures associated with a process, we are now in a position to describe briefly the steps involved in actually creating the process.

Once the OS decides, for whatever reason (see Table 3.1), to create a new process, it can proceed as follows:

1. **Assign a unique process identifier to the new process.** At this time, a new entry is added to the primary process table, which contains one entry per process.
2. **Allocate space for the process.** This includes all elements of the process image. Thus, the OS must know how much space is needed for the private user address space (programs and data) and the user stack. These values can be assigned by default based on the type of process, or they can be set based on user request at job creation time. If a process is spawned by another process, the parent process can pass the needed values to the OS as part of the process creation request. If any existing address space is to be shared by this new process, the appropriate linkages must be set up. Finally, space for a process control block must be allocated.
3. **Initialize the process control block.** The process identification portion contains the ID of this process plus other appropriate IDs, such as that of the parent process. The processor state information portion will typically be initialized with most entries zero, except for the program counter (set to the program entry point) and system stack pointers (set to define the process stack boundaries). The process control information portion is initialized based on standard default values plus attributes that have been requested for this process. For example, the process state would typically be initialized to Ready or Ready/Suspend. The priority may be set by default to the lowest priority unless an explicit request is made for a higher priority. Initially, the process may own no resources (I/O devices, files) unless there is an explicit request for these, or unless they are inherited from the parent.
4. **Set the appropriate linkages.** For example, if the OS maintains each scheduling queue as a linked list, then the new process must be put in the Ready or Ready/Suspend list.
5. **Create or expand other data structures.** For example, the OS may maintain an accounting file on each process to be used subsequently for billing and/or performance assessment purposes.

## Process Switching

On the face of it, the function of process switching would seem to be straightforward. At some time, a running process is interrupted, and the OS assigns another process to the Running state and turns control over to that process. However, several design issues are raised. First, what events trigger a process switch? Another issue is that we must recognize the distinction between mode switching and process switching. Finally, what must the OS do to the various data structures under its control to achieve a process switch?

***When to Switch Processes*** A process switch may occur any time that the OS has gained control from the currently running process. Table 3.8 suggests the possible events that may give control to the OS.

First, let us consider system interrupts. Actually, we can distinguish, as many systems do, two kinds of system interrupts, one of which is simply referred to as an interrupt, and the other as a trap. The former is due to some sort of event that is external to and independent of the currently running process, such as the completion of an I/O operation. The latter relates to an error or exception condition generated within the currently running process, such as an illegal file access attempt. With an ordinary **interrupt**, control is first transferred to an interrupt handler, which does some basic housekeeping and then branches to an OS routine that is concerned with the particular type of interrupt that has occurred. Examples include the following:

- **Clock interrupt:** The OS determines whether the currently running process has been executing for the maximum allowable unit of time, referred to as a **time slice**. That is, a time slice is the maximum amount of time that a process can execute before being interrupted. If so, this process must be switched to a Ready state and another process dispatched.
- **I/O interrupt:** The OS determines what I/O action has occurred. If the I/O action constitutes an event for which one or more processes are waiting, then the OS moves all of the corresponding blocked processes to the Ready state (and Blocked/Suspend processes to the Ready/Suspend state). The OS must then decide whether to resume execution of the process currently in the Running state, or to preempt that process for a higher-priority Ready process.
- **Memory fault:** The processor encounters a virtual memory address reference for a word that is not in main memory. The OS must bring in the block (page or segment) of memory containing the reference from secondary memory

**Table 3.8** Mechanisms for Interrupting the Execution of a Process

| Mechanism | Cause | Use |
|---|---|---|
| Interrupt | External to the execution of the current instruction | Reaction to an asynchronous external event |
| Trap | Associated with the execution of the current instruction | Handling of an error or an exception condition |
| Supervisor call | Explicit request | Call to an operating system function |

to main memory. After the I/O request is issued to bring in the block of memory, the process with the memory fault is placed in a blocked state; the OS then performs a process switch to resume execution of another process. After the desired block is brought into memory, that process is placed in the Ready state.

With a **trap**, the OS determines if the error or exception condition is fatal. If so, then the currently running process is moved to the Exit state and a process switch occurs. If not, then the action of the OS will depend on the nature of the error and the design of the OS. It may attempt some recovery procedure or simply notify the user. It may perform a process switch or resume the currently running process.

Finally, the OS may be activated by a **supervisor call** from the program being executed. For example, a user process is running and an instruction is executed that requests an I/O operation, such as a file open. This call results in a transfer to a routine that is part of the operating system code. The use of a system call may place the user process in the Blocked state.

***Mode Switching*** In Chapter 1, we discussed the inclusion of an interrupt stage as part of the instruction cycle. Recall that, in the interrupt stage, the processor checks to see if any interrupts are pending, indicated by the presence of an interrupt signal. If no interrupts are pending, the processor proceeds to the fetch stage and fetches the next instruction of the current program in the current process. If an interrupt is pending, the processor does the following:

1. It sets the program counter to the starting address of an interrupt-handler program.
2. It switches from user mode to kernel mode so the interrupt processing code may include privileged instructions.

The processor now proceeds to the fetch stage and fetches the first instruction of the interrupt-handler program, which will service the interrupt. At this point, typically, the context of the process that has been interrupted is saved into that process control block of the interrupted program.

One question that may now occur to you is, What constitutes the context that is saved? The answer is that it must include any information that may be altered by the execution of the interrupt handler, and that will be needed to resume the program that was interrupted. Thus, the portion of the process control block that was referred to as processor state information must be saved. This includes the program counter, other processor registers, and stack information.

Does anything else need to be done? That depends on what happens next. The interrupt handler is typically a short program that performs a few basic tasks related to an interrupt. For example, it resets the flag or indicator that signals the presence of an interrupt. It may send an acknowledgment to the entity that issued the interrupt, such as an I/O module. And it may do some basic housekeeping relating to the effects of the event that caused the interrupt. For example, if the interrupt relates to an I/O event, the interrupt handler will check for an error condition. If an error has occurred, the interrupt handler may send a signal to the process that originally requested the I/O operation. If the interrupt is by the clock, then the handler will hand control over

to the dispatcher, which will want to pass control to another process because the time slice allotted to the currently running process has expired.

What about the other information in the process control block? If this interrupt is to be followed by a switch to another process, then some work will need to be done. However, in most operating systems, the occurrence of an interrupt does not necessarily mean a process switch. It is possible that, after the interrupt handler has executed, the currently running process will resume execution. In that case, all that is necessary is to save the processor state information when the interrupt occurs and restore that information when control is returned to the program that was running. Typically, the saving and restoring functions are performed in hardware.

***Change of Process State*** It is clear, then, that the mode switch is a concept distinct from that of the process switch.[10] A mode switch may occur without changing the state of the process that is currently in the Running state. In that case, the context saving and subsequent restoral involve little overhead. However, if the currently running process is to be moved to another state (Ready, Blocked, etc.), then the OS must make substantial changes in its environment. The steps involved in a full process switch are as follows:

1. Save the context of the processor, including program counter and other registers.
2. Update the process control block of the process that is currently in the Running state. This includes changing the state of the process to one of the other states (Ready; Blocked; Ready/Suspend; or Exit). Other relevant fields must also be updated, including the reason for leaving the Running state and accounting information.
3. Move the process control block of this process to the appropriate queue (Ready; Blocked on Event *i*; Ready/Suspend).
4. Select another process for execution; this topic will be explored in Part Four.
5. Update the process control block of the process selected. This includes changing the state of this process to Running.
6. Update memory management data structures. This may be required, depending on how address translation is managed; this topic will be explored in Part Three.
7. Restore the context of the processor to that which existed at the time the selected process was last switched out of the Running state, by loading in the previous values of the program counter and other registers.

Thus, the process switch, which involves a state change, requires more effort than a mode switch.

[10]The term *context switch* is often found in OS literature and textbooks. Unfortunately, although most of the literature uses this term to mean what is here called a process switch, other sources use it to mean a mode switch or even a thread switch (defined in the next chapter). To avoid ambiguity, the term is not used in this book.

## 3.5 EXECUTION OF THE OPERATING SYSTEM

In Chapter 2, we pointed out two intriguing facts about operating systems:

- The OS functions in the same way as ordinary computer software, in the sense that the OS is a set of programs executed by the processor.
- The OS frequently relinquishes control and depends on the processor to restore control to the OS.

If the OS is just a collection of programs, and if it is executed by the processor just like any other program, is the OS a process? If so, how is it controlled? These interesting questions have inspired a number of design approaches. Figure 3.15 illustrates a range of approaches that are found in various contemporary operating systems.

### Nonprocess Kernel

One traditional approach, common on many older operating systems, is to execute the kernel of the OS outside of any process (see Figure 3.15a). With this approach, when the currently running process is interrupted or issues a supervisor call, the mode context of this process is saved and control is passed to the kernel. The OS has its own region of memory to use and its own system stack for controlling procedure calls and returns. The OS can perform any desired functions and restore the context

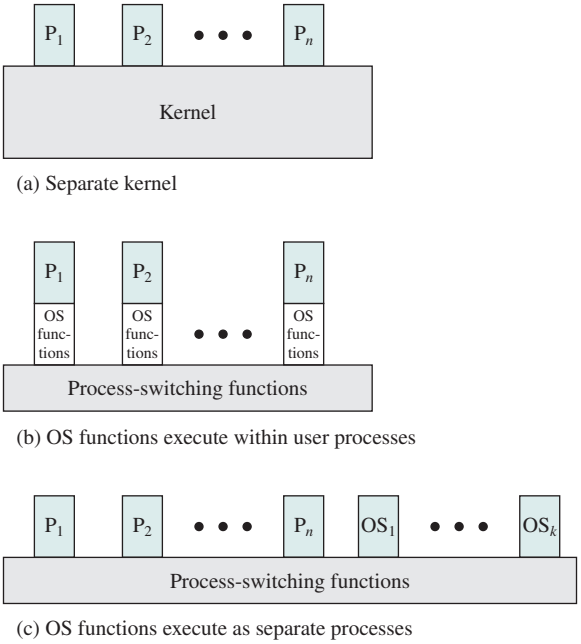



**Figure 3.15 Relationship between Operating System and User Processes**

of the interrupted process, which causes execution to resume in the interrupted user process. Alternatively, the OS can complete the function of saving the environment of the process and proceed to schedule and dispatch another process. Whether this happens depends on the reason for the interruption and the circumstances at the time.

In any case, the key point here is that the concept of process is considered to apply only to user programs. The operating system code is executed as a separate entity that operates in privileged mode.

## Execution within User Processes

An alternative that is common with operating systems on smaller computers (PCs, workstations) is to execute virtually all OS software in the context of a user process. The view is that the OS is primarily a collection of routines the user calls to perform various functions, executed within the environment of the user's process. This is illustrated in Figure 3.15b. At any given point, the OS is managing *n* process images. Each image includes not only the regions illustrated in Figure 3.13 but also program, data, and stack areas for kernel programs.

Figure 3.16 suggests a typical process image structure for this strategy. A separate kernel stack is used to manage calls/returns while the process is in kernel mode.

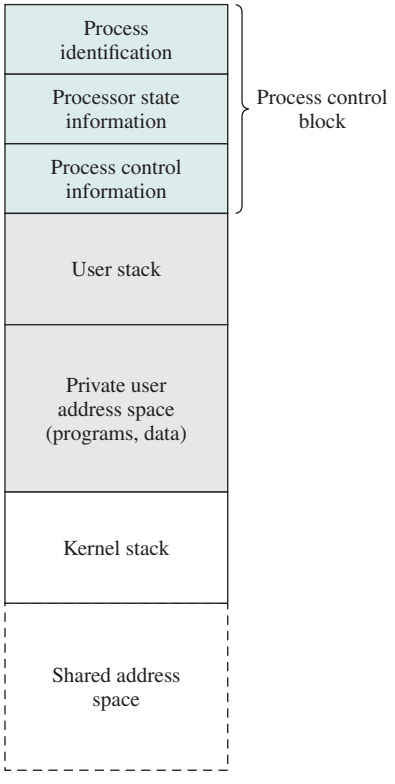



**Figure 3.16** **Process Image: Operating System Executes within User Space**

Operating system code and data are in the shared address space and are shared by all user processes.

When an interrupt, trap, or supervisor call occurs, the processor is placed in kernel mode and control is passed to the OS. To pass control from a user program to the OS, the mode context is saved and a mode switch takes place to an operating system routine. However, execution continues within the current user process. Thus, a process switch is not performed, just a mode switch within the same process.

If the OS, upon completion of its work, determines that the current process should continue to run, then a mode switch resumes the interrupted program within the current process. This is one of the key advantages of this approach: A user program has been interrupted to employ some operating system routine, and then resumed, and all of this has occurred without incurring the penalty of two process switches. If, however, it is determined that a process switch is to occur rather than returning to the previously executing program, then control is passed to a process-switching routine. This routine may or may not execute in the current process, depending on system design. At some point, however, the current process has to be placed in a nonrunning state, and another process designated as the running process. During this phase, it is logically most convenient to view execution as taking place outside of all processes.

In a way, this view of the OS is remarkable. Simply put, at certain points in time, a process will save its state information, choose another process to run from among those that are ready, and relinquish control to that process. The reason this is not an arbitrary and indeed chaotic situation is that during the critical time, the code that is executed in the user process is shared operating system code and not user code. Because of the concept of user mode and kernel mode, the user cannot tamper with or interfere with the operating system routines, even though they are executing in the user's process environment. This further reminds us that there is a distinction between the concepts of process and program, and that the relationship between the two is not one-to-one. Within a process, both a user program and operating system programs may execute, and the operating system programs that execute in the various user processes are identical.

### Process-Based Operating System

Another alternative, illustrated in Figure 3.15c, is to implement the OS as a collection of system processes. As in the other options, the software that is part of the kernel executes in a kernel mode. In this case, however, major kernel functions are organized as separate processes. Again, there may be a small amount of process-switching code that is executed outside of any process.

This approach has several advantages. It imposes a program design discipline that encourages the use of a modular OS with minimal, clean interfaces between the modules. In addition, some noncritical operating system functions are conveniently implemented as separate processes. For example, we mentioned earlier a monitor program that records the level of utilization of various resources (processor, memory, channels) and the rate of progress of the user processes in the system. Because this program does not provide a particular service to any active process, it can only be invoked by the OS. As a process, the function can run at an assigned priority level and

be interleaved with other processes under dispatcher control. Finally, implementing the OS as a set of processes is useful in a multiprocessor or multicomputer environment, in which some of the operating system services can be shipped out to dedicated processors, improving performance.

## 3.6 UNIX SVR4 PROCESS MANAGEMENT

UNIX System V makes use of a simple but powerful process facility that is highly visible to the user. UNIX follows the model of Figure 3.15b, in which most of the OS executes within the environment of a user process. UNIX uses two categories of processes: system processes, and user processes. System processes run in kernel mode and execute operating system code to perform administrative and housekeeping functions, such as allocation of memory and process swapping. User processes operate in user mode to execute user programs and utilities, and in kernel mode to execute instructions that belong to the kernel. A user process enters kernel mode by issuing a system call, when an exception (fault) is generated, or when an interrupt occurs.

### Process States

A total of nine process states are recognized by the UNIX SVR4 operating system; these are listed in Table 3.9, and a state transition diagram is shown in Figure 3.17 (based on the figure in [BACH86]). This figure is similar to Figure 3.9b, with the two UNIX sleeping states corresponding to the two blocked states. The differences are as follows:

- UNIX employs two Running states to indicate whether the process is executing in user mode or kernel mode.
- A distinction is made between the two states: (Ready to Run, in Memory) and (Preempted). These are essentially the same state, as indicated by the dotted

**Table 3.9** UNIX Process States

| | |
|---|---|
| **User Running** | Executing in user mode. |
| **Kernel Running** | Executing in kernel mode. |
| **Ready to Run, in Memory** | Ready to run as soon as the kernel schedules it. |
| **Asleep in Memory** | Unable to execute until an event occurs; process is in main memory (a blocked state). |
| **Ready to Run, Swapped** | Process is ready to run, but the swapper must swap the process into main memory before the kernel can schedule it to execute. |
| **Sleeping, Swapped** | The process is awaiting an event and has been swapped to secondary storage (a blocked state). |
| **Preempted** | Process is returning from kernel to user mode, but the kernel preempts it and does a process switch to schedule another process. |
| **Created** | Process is newly created and not yet ready to run. |
| **Zombie** | Process no longer exists, but it leaves a record for its parent process to collect. |

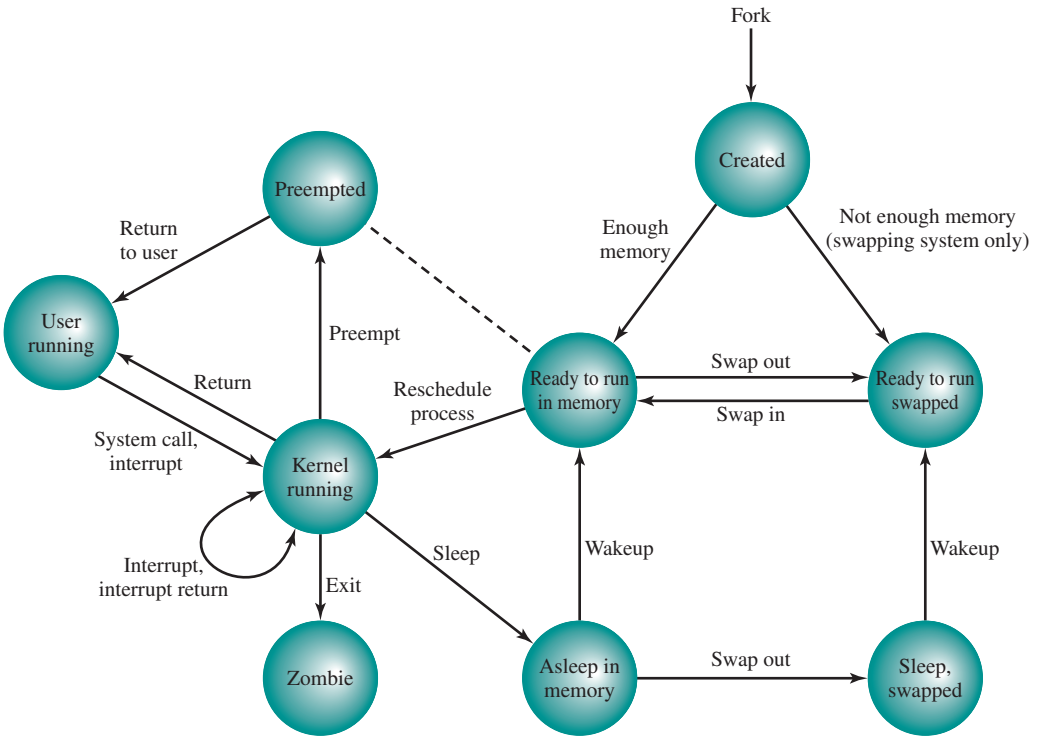



**Figure 3.17** **UNIX Process State Transition Diagram**

line joining them. The distinction is made to emphasize the way in which the Preempted state is entered. When a process is running in kernel mode (as a result of a supervisor call, clock interrupt, or I/O interrupt), there will come a time when the kernel has completed its work and is ready to return control to the user program. At this point, the kernel may decide to preempt the current process in favor of one that is ready and of higher priority. In that case, the current process moves to the Preempted state. However, for purposes of dispatching, those processes in the Preempted state and those in the (Ready to Run, in Memory) state form one queue.

Preemption can only occur when a process is about to move from kernel mode to user mode. While a process is running in kernel mode, it may not be preempted. This makes UNIX unsuitable for real-time processing. Chapter 10 will discuss the requirements for real-time processing.

Two processes are unique in UNIX. Process 0 is a special process that is created when the system boots; in effect, it is predefined as a data structure loaded at boot time. It is the swapper process. In addition, process 0 spawns process 1, referred to as the init process; all other processes in the system have process 1 as an ancestor. When a new interactive user logs on to the system, it is process 1 that creates a user process

for that user. Subsequently, the user process can create child processes in a branching tree, so any particular application can consist of a number of related processes.

## Process Description

A process in UNIX is a rather complex set of data structures that provide the OS with all of the information necessary to manage and dispatch processes. Table 3.10 summarizes the elements of the process image, which are organized into three parts: user-level context, register context, and system-level context.

The **user-level context** contains the basic elements of a user's program and can be generated directly from a compiled object file. The user's program is separated into text and data areas; the text area is read-only and is intended to hold the program's instructions. While the process is executing, the processor uses the user stack area for procedure calls and returns and parameter passing. The shared memory area is a data area that is shared with other processes. There is only one physical copy of a shared memory area, but, by the use of virtual memory, it appears to each sharing process that the shared memory region is in its address space. When a process is not running, the processor status information is stored in the **register context** area.

The **system-level context** contains the remaining information that the OS needs to manage the process. It consists of a static part, which is fixed in size and

**Table 3.10** UNIX Process Image

| | |
|---|---|
| **User-Level Context** | |
| Process text | Executable machine instructions of the program |
| Process data | Data accessible by the program of this process |
| User stack | Contains the arguments, local variables, and pointers for functions executing in user mode |
| Shared memory | Memory shared with other processes, used for interprocess communication |
| **Register Context** | |
| Program counter | Address of next instruction to be executed; may be in kernel or user memory space of this process |
| Processor status register | Contains the hardware status at the time of preemption; contents and format are hardware dependent |
| Stack pointer | Points to the top of the kernel or user stack, depending on the mode of operation at the time or preemption |
| General-purpose registers | Hardware dependent |
| **System-Level Context** | |
| Process table entry | Defines state of a process; this information is always accessible to the operating system |
| U (user) area | Process control information that needs to be accessed only in the context of the process |
| Per process region table | Defines the mapping from virtual to physical addresses; also contains a permission field that indicates the type of access allowed the process: read-only, read-write, or read-execute |
| Kernel stack | Contains the stack frame of kernel procedures as the process executes in kernel mode |

**Table 3.11** UNIX Process Table Entry

| | |
|---|---|
| Process status | Current state of process. |
| Pointers | To U area and process memory area (text, data, stack). |
| Process size | Enables the operating system to know how much space to allocate the process. |
| User identifiers | The **real user ID** identifies the user who is responsible for the running process. The **effective user ID** may be used by a process to gain temporary privileges associated with a particular program; while that program is being executed as part of the process, the process operates with the effective user ID. |
| Process identifiers | ID of this process; ID of parent process. These are set up when the process enters the Created state during the fork system call. |
| Event descriptor | Valid when a process is in a sleeping state; when the event occurs, the process is transferred to a ready-to-run state. |
| Priority | Used for process scheduling. |
| Signal | Enumerates signals sent to a process but not yet handled. |
| Timers | Include process execution time, kernel resource utilization, and user-set timer used to send alarm signal to a process. |
| P_link | Pointer to the next link in the ready queue (valid if process is ready to execute). |
| Memory status | Indicates whether process image is in main memory or swapped out. If it is in memory, this field also indicates whether it may be swapped out or is temporarily locked into main memory. |

stays with a process throughout its lifetime, and a dynamic part, which varies in size through the life of the process. One element of the static part is the process table entry. This is actually part of the process table maintained by the OS, with one entry per process. The process table entry contains process control information that is accessible to the kernel at all times; hence, in a virtual memory system, all process table entries are maintained in main memory. Table 3.11 lists the contents of a process table entry. The user area, or U area, contains additional process control information that is needed by the kernel when it is executing in the context of this process; it is also used when paging processes to and from memory. Table 3.12 shows the contents of this table.

The distinction between the process table entry and the U area reflects the fact that the UNIX kernel always executes in the context of some process. Much of the time, the kernel will be dealing with the concerns of that process. However, some of the time, such as when the kernel is performing a scheduling algorithm preparatory to dispatching another process, it will need access to information about other processes. The information in a process table can be accessed when the given process is not the current one.

The third static portion of the system-level context is the per process region table, which is used by the memory management system. Finally, the kernel stack is the dynamic portion of the system-level context. This stack is used when the process is executing in kernel mode, and contains the information that must be saved and restored as procedure calls and interrupts occur.

**Table 3.12** UNIX U Area

| | |
|---|---|
| Process table pointer | Indicates entry that corresponds to the U area. |
| User identifiers | Real and effective user IDs used to determine user privileges. |
| Timers | Record time that the process (and its descendants) spent executing in user mode and in kernel mode. |
| Signal-handler array | For each type of signal defined in the system, indicates how the process will react to receipt of that signal (exit, ignore, execute specified user function). |
| Control terminal | Indicates login terminal for this process, if one exists. |
| Error field | Records errors encountered during a system call. |
| Return value | Contains the result of system calls. |
| I/O parameters | Describe the amount of data to transfer, the address of the source (or target) data array in user space, and file offsets for I/O. |
| File parameters | Current directory and current root describe the file system environment of the process. |
| User file descriptor table | Records the files the process has opened. |
| Limit fields | Restrict the size of the process and the size of a file it can write. |
| Permission modes fields | Mask mode settings on files the process creates. |

## Process Control

Process creation in UNIX is made by means of the kernel system call, `fork()`. When a process issues a fork request, the OS performs the following functions [BACH86]:

1. It allocates a slot in the process table for the new process.
2. It assigns a unique process ID to the child process.
3. It makes a copy of the process image of the parent, with the exception of any shared memory.
4. It increments counters for any files owned by the parent, to reflect that an additional process now also owns those files.
5. It assigns the child process to the Ready to Run state.
6. It returns the ID number of the child to the parent process, and a 0 value to the child process.

All of this work is accomplished in kernel mode in the parent process. When the kernel has completed these functions, it can do one of the following, as part of the dispatcher routine:

- Stay in the parent process. Control returns to user mode at the point of the fork call of the parent.
- Transfer control to the child process. The child process begins executing at the same point in the code as the parent, namely at the return from the fork call.
- Transfer control to another process. Both parent and child are left in the Ready to Run state.

It is perhaps difficult to visualize this method of process creation because both parent and child are executing the same passage of code. The difference is this: When the return from the fork occurs, the return parameter is tested. If the value is zero, then this is the child process, and a branch can be executed to the appropriate user program to continue execution. If the value is nonzero, then this is the parent process, and the main line of execution can continue.

## 3.7 SUMMARY

The most fundamental concept in a modern OS is the process. The principal function of the OS is to create, manage, and terminate processes. While processes are active, the OS must see that each is allocated time for execution by the processor, coordinate their activities, manage conflicting demands, and allocate system resources to processes.

To perform its process management functions, the OS maintains a description of each process, or process image, which includes the address space within which the process executes, and a process control block. The latter contains all of the information that is required by the OS to manage the process, including its current state, resources allocated to it, priority, and other relevant data.

During its lifetime, a process moves among a number of states. The most important of these are Ready, Running, and Blocked. A ready process is one that is not currently executing, but that is ready to be executed as soon as the OS dispatches it. The running process is that process that is currently being executed by the processor. In a multiprocessor system, more than one process can be in this state. A blocked process is waiting for the completion of some event, such as an I/O operation.

A running process is interrupted either by an interrupt, which is an event that occurs outside the process and that is recognized by the processor, or by executing a supervisor call to the OS. In either case, the processor performs a mode switch, transferring control to an operating system routine. The OS, after it has completed necessary work, may resume the interrupted process or switch to some other process.

## 3.8 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

### Key Terms

| | | |
|---|---|---|
| blocked state | privileged mode | running state |
| child process | process | suspend state |
| dispatcher | process control block | swapping |
| exit state | process control information | system mode |
| interrupt | process image | task |
| kernel mode | process spawning | time slice |
| mode switch | process switch | trace |
| new state | program status word | trap |
| parent process | ready state | user mode |
| preempt | round-robin | |

## Review Questions

**3.1.** What is an instruction trace?
**3.2.** Explain the concept of a process and mark its differences from a program.
**3.3.** For the processing model of Figure 3.6, briefly define each state.
**3.4.** What does it mean to preempt a process?
**3.5.** What is process spawning?
**3.6.** Why does Figure 3.9b have two blocked states?
**3.7.** List four characteristics of a suspended process.
**3.8.** For what types of entities does the OS maintain tables of information for management purposes?
**3.9.** What are the elements of a process image?
**3.10.** Why are two modes (user and kernel) needed?
**3.11.** What are the steps performed by an OS to create a new process?
**3.12.** What is the difference between an interrupt and a trap?
**3.13.** Give three examples of an interrupt.
**3.14.** What is the difference between a mode switch and a process switch?

## Problems

**3.1.** A system adopts a priority-based preemptive scheduling where the initial priority of a process increases by 1 after every 5 ms. In a recorded time span, the system has four processes, P1, P2, P3 and P4, as shown in the following table:

| PROCESS ID | INITIAL PRIORITY | ARRIVAL TIME IN MS | TOTAL CPU TIME IN MS |
|---|---|---|---|
| P1 | 1 | 0 | 15 |
| P2 | 3 | 5 | 7.5 |
| P3 | 2 | 10 | 5 |
| P4 | 2 | 15 | 10 |

Draw a timing diagram similar to Figure 3.7 and find the turnaround time for each process. Assume that the dispatcher takes 2.5 milliseconds for a process switch.

**3.2.** Suppose that four interleaved processes are running in a system having start addresses 4050, 3200, 5000 and 6700. The traces of the individual processes are as follows:

| Process P1 | Process P2 | Process P3 | Process P4 |
|---|---|---|---|
| 4050 | 3200 | 5000 | 6700 |
| 4051 | 3201 | 5001 | 6701 |
| 4052 | 3202 | 5002 | 6702 |
| 4053 | 3203 | 5003 | <I/O> |
| 4054 | 3204 | 5004 | |
| 4055 | 3205 | 5005 | |
| 4056 | 3206 | 5006 | |
| 4057 | <I/O> | 5007 | |
| 4058 | | 5008 | |
| 4059 | | 5009 | |
| 4060 | | 5010 | |

Find the interleaved traces of the processes. Assume that the dispatcher is invoked after 5 instructions or for interrupts and the dispatcher cycle has 4 instructions.

**3.3.** Figure 3.9b contains seven states. In principle, one could draw a transition between any two states, for a total of 42 different transitions.

**a.** List all of the possible transitions and give an example of what could cause each transition.

**b.** List all of the impossible transitions and explain why.

**3.4.** For the seven-state process model of Figure 3.9b, draw a queueing diagram similar to that of Figure 3.8b.

**3.5.** Consider the state transition diagram of Figure 3.9b. Suppose it is time for the OS to dispatch a process and there are processes in both the Ready state and the Ready/Suspend state, and at least one process in the Ready/Suspend state has higher scheduling priority than any of the processes in the Ready state. Two extreme policies are as follows: (1) Always dispatch from a process in the Ready state, to minimize swapping, and (2) always give preference to the highest-priority process, even though that may mean swapping when swapping is not necessary. Suggest an intermediate policy that tries to balance the concerns of priority and performance.

**3.6.** Table 3.13 shows the process states for the VAX/VMS operating system.

**a.** Can you provide a justification for the existence of so many distinct wait states?

**b.** Why do the following states not have resident and swapped-out versions: Page Fault Wait, Collided Page Wait, Common Event Wait, Free Page Wait, and Resource Wait?

**Table 3.13** VAX/VMS Process States

| Process State | Process Condition |
|---|---|
| Currently Executing | Running process. |
| Computable (resident) | Ready and resident in main memory. |
| Computable (outswapped) | Ready, but swapped out of main memory. |
| Page Fault Wait | Process has referenced a page not in main memory and must wait for the page to be read in. |
| Collided Page Wait | Process has referenced a shared page that is the cause of an existing page fault wait in another process, or a private page that is in the process of being read in or written out. |
| Common Event Wait | Waiting for shared event flag (event flags are single-bit interprocess signaling mechanisms). |
| Free Page Wait | Waiting for a free page in main memory to be added to the collection of pages in main memory devoted to this process (the working set of the process). |
| Hibernate Wait (resident) | Process puts itself in a wait state. |
| Hibernate Wait (outswapped) | Hibernating process is swapped out of main memory. |
| Local Event Wait (resident) | Process in main memory and waiting for local event flag (usually I/O completion). |
| Local Event Wait (outswapped) | Process in local event wait is swapped out of main memory. |
| Suspended Wait (resident) | Process is put into a wait state by another process. |
| Suspended Wait (outswapped) | Suspended process is swapped out of main memory. |
| Resource Wait | Process waiting for miscellaneous system resource. |

**c.** Draw the state transition diagram and indicate the action or occurrence that causes each transition.

**3.7.** The VAX/VMS operating system makes use of four processor access modes to facilitate the protection and sharing of system resources among processes. The access mode determines:

- **Instruction execution privileges:** What instructions the processor may execute
- **Memory access privileges:** Which locations in virtual memory the current instruction may access

The four modes are as follows:

- **Kernel:** Executes the kernel of the VMS operating system, which includes memory management, interrupt handling, and I/O operations.
- **Executive:** Executes many of the OS service calls, including file and record (disk and tape) management routines.
- **Supervisor:** Executes other OS services, such as responses to user commands.
- **User:** Executes user programs, plus utilities such as compilers, editors, linkers, and debuggers.

A process executing in a less-privileged mode often needs to call a procedure that executes in a more-privileged mode; for example, a user program requires an operating system service. This call is achieved by using a change-mode (CHM) instruction, which causes an interrupt that transfers control to a routine at the new access mode. A return is made by executing the REI (return from exception or interrupt) instruction.

**a.** A number of operating systems have two modes: kernel and user. What are the advantages and disadvantages of providing four modes instead of two?

**b.** Can you make a case for even more than four modes?

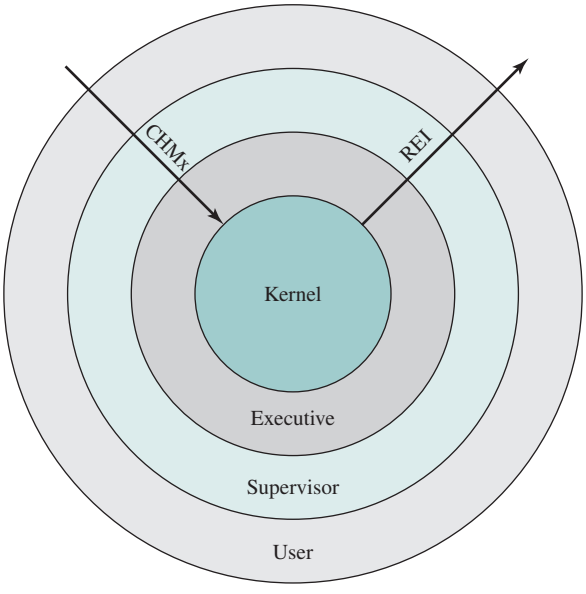



**Figure 3.18 VAX/VMS Access Modes**

**3.8.** The VMS scheme discussed in the preceding problem is often referred to as a ring protection structure, as illustrated in Figure 3.18. Indeed, the simple kernel/user scheme, as described in Section 3.3, is a two-ring structure. A potential disadvantage of this protection structure is that it cannot readily be used to enforce a "need-to-know" principle. [SILB04] gives this example: If an object is accessible in domain D$j$ but not in domain D$i$, then $j < i$. But this means that every object accessible in D$i$ is also accessible in D$j$.

Explain clearly what the problem is that is referred to in the preceding paragraph.

**3.9.** Figure 3.8b suggests that a process can only be in one event queue at a time.

**a.** Is it possible that you would want to allow a process to wait on more than one event at the same time? Provide an example.

**b.** In that case, how would you modify the queueing structure of the figure to support this new feature?

**3.10.** What is the purpose of the system call `fork()` in the UNIX operating system? Write a C routine to create a child process using the `fork()` system call. Incorporate an error check in your routine in case the creation of the child process fails.

**3.11.** What are the specialities of Process 0 and Process 1 in UNIX? Which command will you use to get information about the running processes in the system?

**3.12.** You have executed the following C program:

```
main ()
{ int pid;
pid = fork ();
printf ("%d \n", pid);
}
```

What are the possible outputs, assuming the fork succeeded?

CHAPTER 4

# THREADS


**4.1 Processes and Threads**
- Multithreading
- Thread Functionality

**4.2 Types of Threads**
- User-Level and Kernel-Level Threads
- Other Arrangements

**4.3 Multicore and Multithreading**
- Performance of Software on Multicore
- Application Example: Valve Game Software

**4.4 Windows Process and Thread Management**
- Management of Background Tasks and Application Lifecycles
- The Windows Process
- Process and Thread Objects
- Multithreading
- Thread States
- Support for OS Subsystems

**4.5 Solaris Thread and SMP Management**
- Multithreaded Architecture
- Motivation
- Process Structure
- Thread Execution
- Interrupts as Threads

**4.6 Linux Process and Thread Management**
- Linux Tasks
- Linux Threads
- Linux Namespaces

**4.7 Android Process and Thread Management**
- Android Applications
- Activities
- Processes and Threads

**4.8 Mac OS X Grand Central Dispatch**

**4.9 Summary**

**4.10 Key Terms, Review Questions, and Problems**

**Learning Objectives**

After studying this chapter, you should be able to:

- Understand the distinction between process and thread.
- Describe the basic design issues for threads.
- Explain the difference between user-level threads and kernel-level threads.
- Describe the thread management facility in Windows.
- Describe the thread management facility in Solaris.
- Describe the thread management facility in Linux.

This chapter examines some more advanced concepts related to process management, which are found in a number of contemporary operating systems. We show that the concept of process is more complex and subtle than presented so far and in fact embodies two separate and potentially independent concepts: one relating to resource ownership, and another relating to execution. This distinction has led to the development, in many operating systems, of a construct known as the **thread**.

## 4.1 PROCESSES AND THREADS

The discussion so far has presented the concept of a process as embodying two characteristics:

1. **Resource ownership:** A process includes a virtual address space to hold the process image; recall from Chapter 3 that the process image is the collection of program, data, stack, and attributes defined in the process control block. From time to time, a process may be allocated control or ownership of resources, such as main memory, I/O channels, I/O devices, and files. The OS performs a protection function to prevent unwanted interference between processes with respect to resources.
2. **Scheduling/execution:** The execution of a process follows an execution path (trace) through one or more programs (e.g., Figure 1.5). This execution may be interleaved with that of other processes. Thus, a process has an execution state (Running, Ready, etc.) and a dispatching priority, and is the entity that is scheduled and dispatched by the OS.

Some thought should convince the reader that these two characteristics are independent and could be treated independently by the OS. This is done in a number of operating systems, particularly recently developed systems. To distinguish the two characteristics, the unit of dispatching is usually referred to as a thread or

**lightweight process**, while the unit of resource ownership is usually referred to as a **process** or **task**.[1]

## Multithreading

Multithreading refers to the ability of an OS to support multiple, concurrent paths of execution within a single process. The traditional approach of a single thread of execution per process, in which the concept of a thread is not recognized, is referred to as a single-threaded approach. The two arrangements shown in the left half of Figure 4.1 are single-threaded approaches. MS-DOS is an example of an OS that supports a single-user process and a single thread. Other operating systems, such as some variants of UNIX, support multiple user processes, but only support one thread per process. The right half of Figure 4.1 depicts multithreaded approaches. A Java runtime environment is an example of a system of one process with multiple threads. Of interest in this section is the use of multiple processes, each of which supports multiple threads. This approach is taken in Windows, Solaris, and many

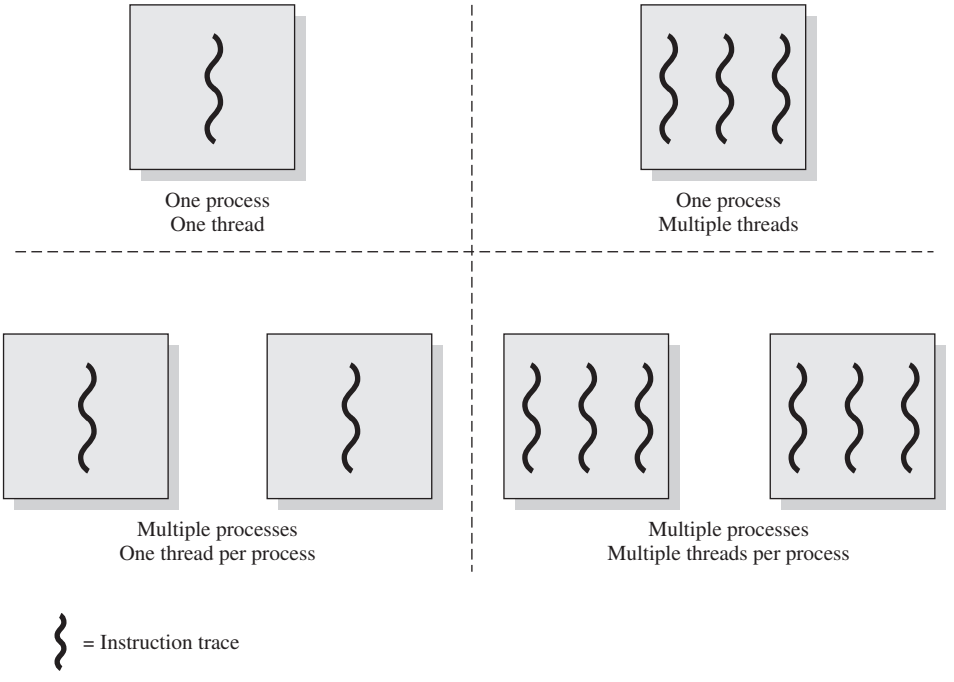



**Figure 4.1 Threads and Processes**

[1]Alas, even this degree of consistency is not maintained. In IBM's mainframe operating systems, the concepts of address space and task, respectively, correspond roughly to the concepts of process and thread that we describe in this section. Also, in the literature, the term *lightweight process* is used as either (1) equivalent to the term *thread*, (2) a particular type of thread known as a kernel-level thread, or (3) in the case of Solaris, an entity that maps user-level threads to kernel-level threads.

modern versions of UNIX, among others. In this section, we give a general description of multithreading; the details of the Windows, Solaris, and Linux approaches will be discussed later in this chapter.

In a multithreaded environment, a process is defined as the unit of resource allocation and a unit of protection. The following are associated with processes:

- A virtual address space that holds the process image
- Protected access to processors, other processes (for interprocess communication), files, and I/O resources (devices and channels)

Within a process, there may be one or more threads, each with the following:

- A thread execution state (Running, Ready, etc.)
- A saved thread context when not running; one way to view a thread is as an independent program counter operating within a process
- An execution stack
- Some per-thread static storage for local variables
- Access to the memory and resources of its process, shared with all other threads in that process

Figure 4.2 illustrates the distinction between threads and processes from the point of view of process management. In a single-threaded process model (i.e., there is no distinct concept of thread), the representation of a process includes its process control block and user address space, as well as user and kernel stacks to manage the call/return behavior of the execution of the process. While the process is running, it controls the processor registers. The contents of these registers are saved when the process is not running. In a multithreaded environment, there is still a single process

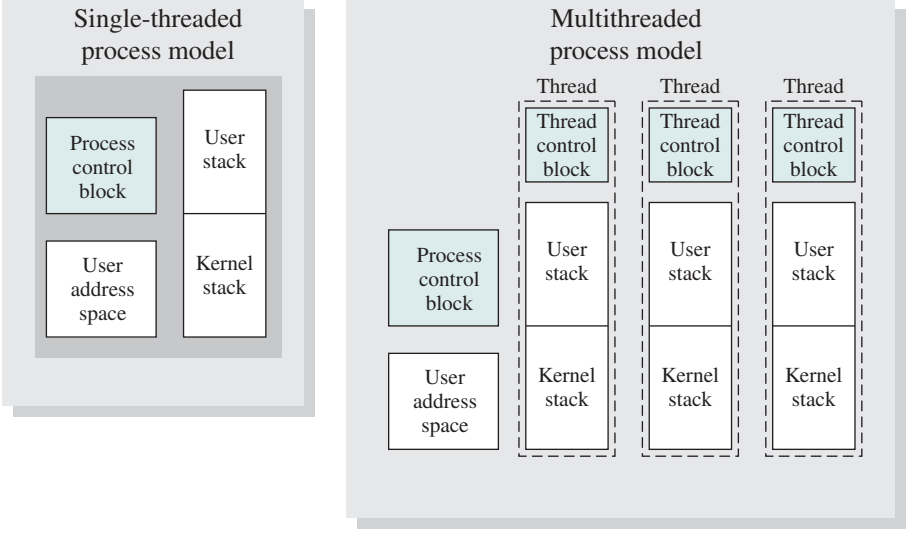



**Figure 4.2 Single-Threaded and Multithreaded Process Models**

control block and user address space associated with the process, but now there are separate stacks for each thread, as well as a separate control block for each thread containing register values, priority, and other thread-related state information.

Thus, all of the threads of a process share the state and resources of that process. They reside in the same address space and have access to the same data. When one thread alters an item of data in memory, other threads see the results if and when they access that item. If one thread opens a file with read privileges, other threads in the same process can also read from that file.

The key benefits of threads derive from the performance implications:

1. It takes far less time to create a new thread in an existing process, than to create a brand-new process. Studies done by the Mach developers show that thread creation is ten times faster than process creation in UNIX [TEVA87].
2. It takes less time to terminate a thread than a process.
3. It takes less time to switch between two threads within the same process than to switch between processes.
4. Threads enhance efficiency in communication between different executing programs. In most operating systems, communication between independent processes requires the intervention of the kernel to provide protection and the mechanisms needed for communication. However, because threads within the same process share memory and files, they can communicate with each other without invoking the kernel.

Thus, if there is an application or function that should be implemented as a set of related units of execution, it is far more efficient to do so as a collection of threads, rather than a collection of separate processes.

An example of an application that could make use of threads is a file server. As each new file request comes in, a new thread can be spawned for the file management program. Because a server will handle many requests, many threads will be created and destroyed in a short period. If the server runs on a multiprocessor computer, then multiple threads within the same process can be executing simultaneously on different processors. Further, because processes or threads in a file server must share file data and therefore coordinate their actions, it is faster to use threads and shared memory than processes and message passing for this coordination.

The thread construct is also useful on a single processor to simplify the structure of a program that is logically doing several different functions.

[LETW88] gives four examples of the uses of threads in a single-user multiprocessing system:

1. **Foreground and background work:** For example, in a spreadsheet program, one thread could display menus and read user input, while another thread executes user commands and updates the spreadsheet. This arrangement often increases the perceived speed of the application by allowing the program to prompt for the next command before the previous command is complete.
2. **Asynchronous processing:** Asynchronous elements in the program can be implemented as threads. For example, as a protection against power failure, one can design a word processor to write its random access memory (RAM)

buffer to disk once every minute. A thread can be created whose sole job is periodic backup and that schedules itself directly with the OS; there is no need for fancy code in the main program to provide for time checks or to coordinate input and output.

3. **Speed of execution:** A multithreaded process can compute one batch of data while reading the next batch from a device. On a multiprocessor system, multiple threads from the same process may be able to execute simultaneously. Thus, even though one thread may be blocked for an I/O operation to read in a batch of data, another thread may be executing.
4. **Modular program structure:** Programs that involve a variety of activities or a variety of sources and destinations of input and output may be easier to design and implement using threads.

In an OS that supports threads, scheduling and dispatching is done on a thread basis; hence, most of the state information dealing with execution is maintained in thread-level data structures. There are, however, several actions that affect all of the threads in a process, and that the OS must manage at the process level. For example, suspension involves swapping the address space of one process out of main memory to make room for the address space of another process. Because all threads in a process share the same address space, all threads are suspended at the same time. Similarly, termination of a process terminates all threads within that process.

## Thread Functionality

Like processes, threads have execution states and may synchronize with one another. We look at these two aspects of thread functionality in turn.

***Thread States*** As with processes, the key states for a thread are Running, Ready, and Blocked. Generally, it does not make sense to associate suspend states with threads because such states are process-level concepts. In particular, if a process is swapped out, all of its threads are necessarily swapped out because they all share the address space of the process.

There are four basic thread operations associated with a change in thread state [ANDE04]:

1. **Spawn:** Typically, when a new process is spawned, a thread for that process is also spawned. Subsequently, a thread within a process may spawn another thread within the same process, providing an instruction pointer and arguments for the new thread. The new thread is provided with its own register context and stack space and placed on the Ready queue.
2. **Block:** When a thread needs to wait for an event, it will block (saving its user registers, program counter, and stack pointers). The processor may then turn to the execution of another ready thread in the same or a different process.
3. **Unblock:** When the event for which a thread is blocked occurs, the thread is moved to the Ready queue.
4. **Finish:** When a thread completes, its register context and stacks are deallocated.

A significant issue is whether the blocking of a thread results in the blocking of the entire process. In other words, if one thread in a process is blocked, does this prevent the running of any other thread in the same process, even if that other thread is in a ready state? Clearly, some of the flexibility and power of threads is lost if the one blocked thread blocks an entire process.

We will return to this issue subsequently in our discussion of user-level versus kernel-level threads, but for now, let us consider the performance benefits of threads that do not block an entire process. Figure 4.3 (based on one in [KLEI96]) shows a program that performs two remote procedure calls (RPCs)[2] to two different hosts to obtain a combined result. In a single-threaded program, the results are obtained in sequence, so the program has to wait for a response from each server in turn. Rewriting the program to use a separate thread for each RPC results in a substantial speedup. Note if this program operates on a uniprocessor, the requests must be generated sequentially and the results processed in sequence; however, the program waits concurrently for the two replies.

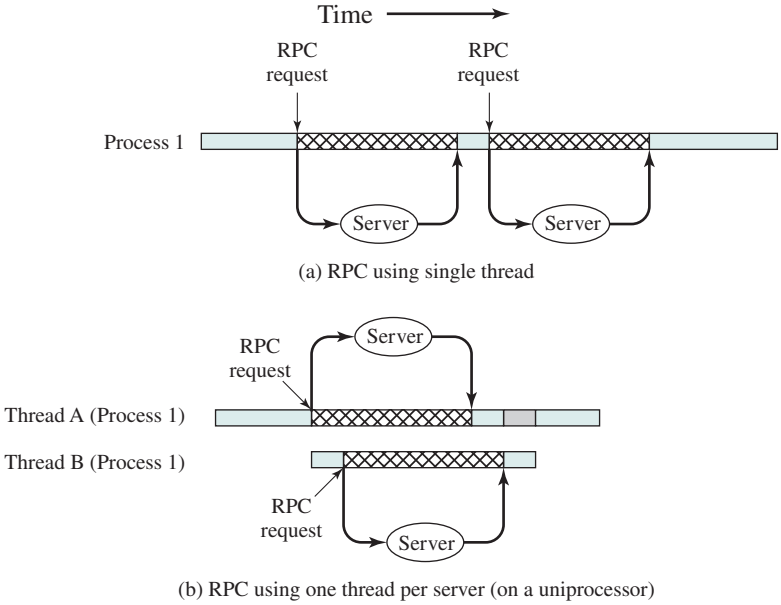



**Figure 4.3 Remote Procedure Call (RPC) Using Threads**

[2]An RPC is a technique by which two programs, which may execute on different machines, interact using procedure call/return syntax and semantics. Both the called and calling programs behave as if the partner program were running on the same machine. RPCs are often used for client/server applications and will be discussed in Chapter 16.

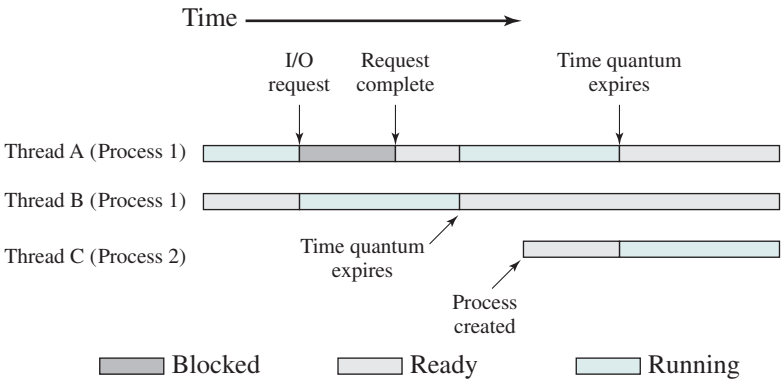



**Figure 4.4** **Multithreading Example on a Uniprocessor**

On a uniprocessor, multiprogramming enables the interleaving of multiple threads within multiple processes. In the example of Figure 4.4, three threads in two processes are interleaved on the processor. Execution passes from one thread to another either when the currently running thread is blocked or when its time slice is exhausted.[3]

***THREAD SYNCHRONIZATION*** All of the threads of a process share the same address space and other resources, such as open files. Any alteration of a resource by one thread affects the environment of the other threads in the same process. It is therefore necessary to synchronize the activities of the various threads so that they do not interfere with each other or corrupt data structures. For example, if two threads each try to add an element to a doubly linked list at the same time, one element may be lost or the list may end up malformed.

The issues raised and the techniques used in the synchronization of threads are, in general, the same as for the synchronization of processes. These issues and techniques will be the subject of Chapters 5 and 6.

## 4.2 TYPES OF THREADS

### User-Level and Kernel-Level Threads

There are two broad categories of thread implementation: user-level threads (ULTs) and kernel-level threads (KLTs).[4] The latter are also referred to in the literature as *kernel-supported threads* or *lightweight processes*.

***USER-LEVEL THREADS*** In a pure ULT facility, all of the work of thread management is done by the application and the kernel is not aware of the existence of threads.

[3]In this example, thread C begins to run after thread A exhausts its time quantum, even though thread B is also ready to run. The choice between B and C is a scheduling decision, a topic covered in Part Four.

[4]The acronyms ULT and KLT are not widely used, but are introduced for conciseness.

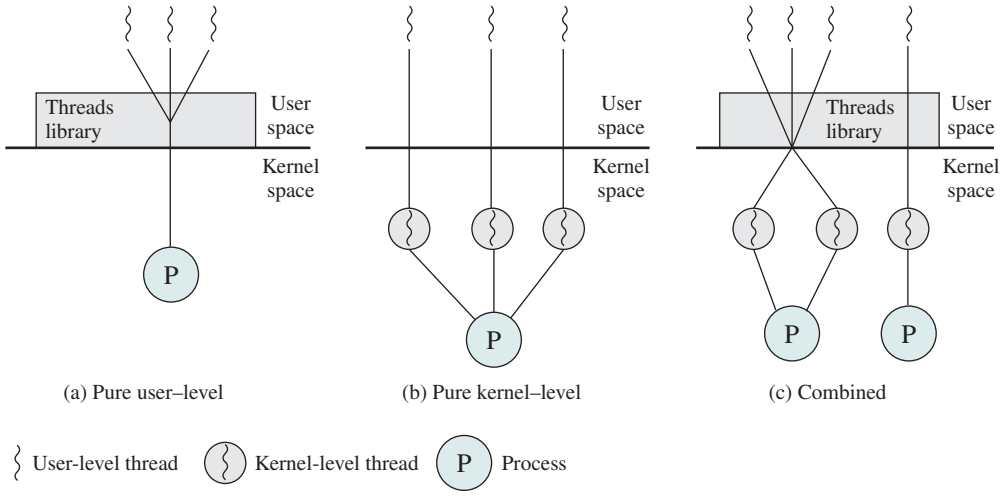



**Figure 4.5 User-Level and Kernel-Level Threads**

Figure 4.5a illustrates the pure ULT approach. Any application can be programmed to be multithreaded by using a threads library, which is a package of routines for ULT management. The threads library contains code for creating and destroying threads, for passing messages and data between threads, for scheduling thread execution, and for saving and restoring thread contexts.

By default, an application begins with a single thread and begins running in that thread. This application and its thread are allocated to a single process managed by the kernel. At any time that the application is running (the process is in the Running state), the application may spawn a new thread to run within the same process. Spawning is done by invoking the spawn utility in the threads library. Control is passed to that utility by a procedure call. The threads library creates a data structure for the new thread and then passes control to one of the threads within this process that is in the Ready state, using some scheduling algorithm. When control is passed to the library, the context of the current thread is saved, and when control is passed from the library to a thread, the context of that thread is restored. The context essentially consists of the contents of user registers, the program counter, and stack pointers.

All of the activity described in the preceding paragraph takes place in user space and within a single process. The kernel is unaware of this activity. The kernel continues to schedule the process as a unit and assigns a single execution state (Ready, Running, Blocked, etc.) to that process. The following examples should clarify the relationship between thread scheduling and process scheduling. Suppose process B is executing in its thread 2; the states of the process and two ULTs that are part of the process are shown in Figure 4.6a. Each of the following is a possible occurrence:

1. The application executing in thread 2 makes a system call that blocks B. For example, an I/O call is made. This causes control to transfer to the kernel. The kernel invokes the I/O action, places process B in the Blocked state, and

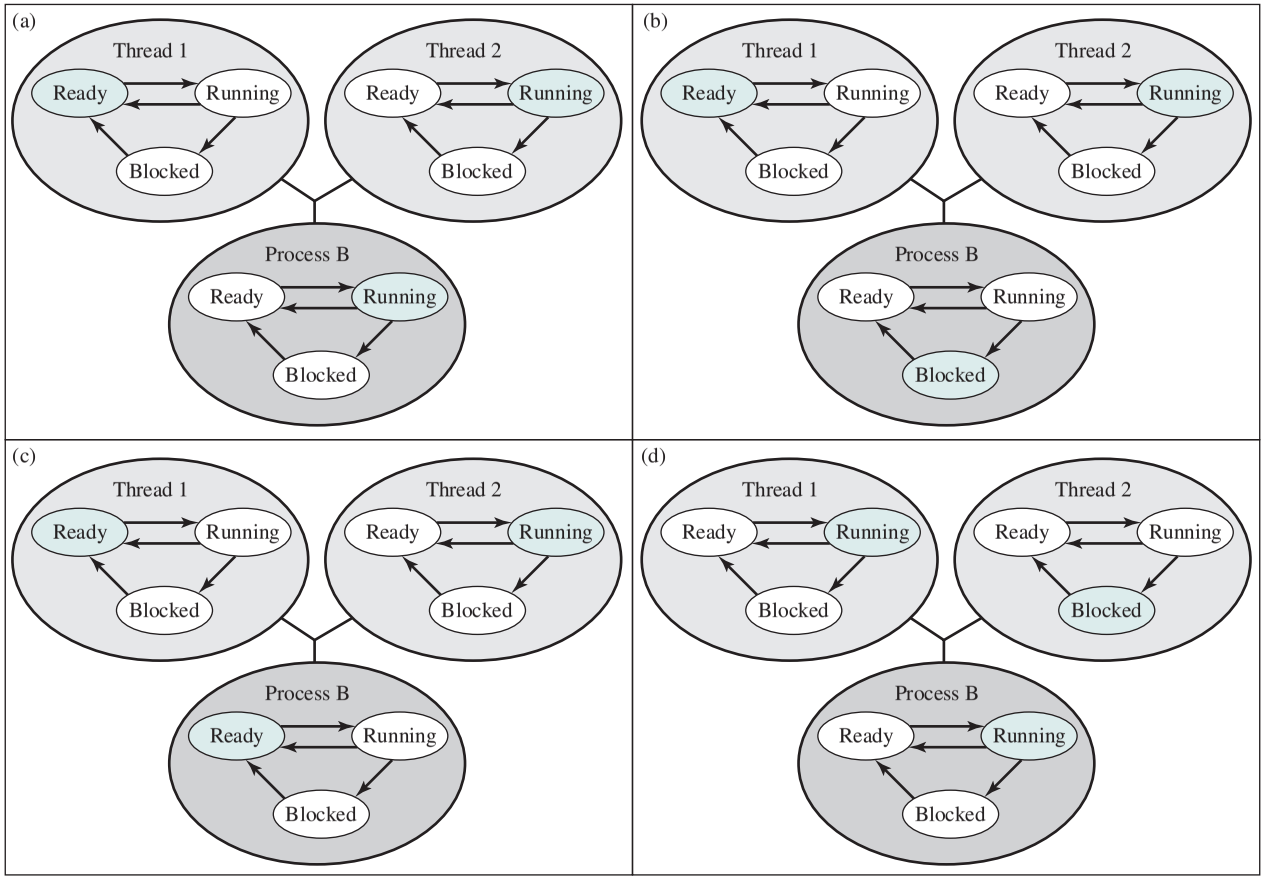



**Figure 4.6** **Examples of the Relationships between User-Level Thread States and Process States**

switches to another process. Meanwhile, according to the data structure maintained by the threads library, thread 2 of process B is still in the Running state. It is important to note that thread 2 is not actually running in the sense of being executed on a processor; but it is perceived as being in the Running state by the threads library. The corresponding state diagrams are shown in Figure 4.6b.

2. A clock interrupt passes control to the kernel, and the kernel determines that the currently running process (B) has exhausted its time slice. The kernel places process B in the Ready state and switches to another process. Meanwhile, according to the data structure maintained by the threads library, thread 2 of process B is still in the Running state. The corresponding state diagrams are shown in Figure 4.6c.
3. Thread 2 has reached a point where it needs some action performed by thread 1 of process B. Thread 2 enters a Blocked state and thread 1 transitions from Ready to Running. The process itself remains in the Running state. The corresponding state diagrams are shown in Figure 4.6d.

Note that each of the three preceding items suggests an alternative event starting from diagram (a) of Figure 4.6. So each of the three other diagrams (b, c, d) shows a transition from the situation in (a). In cases 1 and 2 (Figures 4.6b and 4.6c), when the kernel switches control back to process B, execution resumes in thread 2. Also note that a process can be interrupted, either by exhausting its time slice or by being preempted by a higher-priority process, while it is executing code in the threads library. Thus, a process may be in the midst of a thread switch from one thread to another when interrupted. When that process is resumed, execution continues within the threads library, which completes the thread switch and transfers control to another thread within that process.

There are a number of advantages to the use of ULTs instead of KLTs, including the following:

1. Thread switching does not require kernel-mode privileges because all of the thread management data structures are within the user address space of a single process. Therefore, the process does not switch to the kernel mode to do thread management. This saves the overhead of two mode switches (user to kernel; kernel back to user).
2. Scheduling can be application specific. One application may benefit most from a simple round-robin scheduling algorithm, while another might benefit from a priority-based scheduling algorithm. The scheduling algorithm can be tailored to the application without disturbing the underlying OS scheduler.
3. ULTs can run on any OS. No changes are required to the underlying kernel to support ULTs. The threads library is a set of application-level functions shared by all applications.

There are two distinct disadvantages of ULTs compared to KLTs:

1. In a typical OS, many system calls are blocking. As a result, when a ULT executes a system call, not only is that thread blocked, but all of the threads within the process are blocked as well.

2. In a pure ULT strategy, a multithreaded application cannot take advantage of multiprocessing. A kernel assigns one process to only one processor at a time. Therefore, only a single thread within a process can execute at a time. In effect, we have application-level multiprogramming within a single process. While this multiprogramming can result in a significant speedup of the application, there are applications that would benefit from the ability to execute portions of code simultaneously.

There are ways to work around these two problems. For example, both problems can be overcome by writing an application as multiple processes rather than multiple threads. But this approach eliminates the main advantage of threads: Each switch becomes a process switch rather than a thread switch, resulting in much greater overhead.

Another way to overcome the problem of blocking threads is to use a technique referred to as **jacketing**. The purpose of jacketing is to convert a blocking system call into a nonblocking system call. For example, instead of directly calling a system I/O routine, a thread calls an application-level I/O jacket routine. Within this jacket routine is code that checks to determine if the I/O device is busy. If it is, the thread enters the Blocked state and passes control (through the threads library) to another thread. When this thread is later given control again, the jacket routine checks the I/O device again.

***Kernel-Level Threads*** In a pure KLT facility, all of the work of thread management is done by the kernel. There is no thread management code in the application level, simply an application programming interface (API) to the kernel thread facility. Windows is an example of this approach.

Figure 4.5b depicts the pure KLT approach. The kernel maintains context information for the process as a whole and for individual threads within the process. Scheduling by the kernel is done on a thread basis. This approach overcomes the two principal drawbacks of the ULT approach. First, the kernel can simultaneously schedule multiple threads from the same process on multiple processors. Second, if one thread in a process is blocked, the kernel can schedule another thread of the same process. Another advantage of the KLT approach is that kernel routines themselves can be multithreaded.

The principal disadvantage of the KLT approach compared to the ULT approach is that the transfer of control from one thread to another within the same process requires a mode switch to the kernel. To illustrate the differences, Table 4.1 shows the results of measurements taken on a uniprocessor VAX computer running a UNIX-like OS. The two benchmarks are as follows: Null Fork, the time to create, schedule, execute, and complete a process/thread that invokes

**Table 4.1** Thread and Process Operation Latencies ($\mu s$)

| Operation | User-Level Threads | Kernel-Level Threads | Processes |
|---|---|---|---|
| **Null Fork** | 34 | 948 | 11,300 |
| **Signal Wait** | 37 | 441 | 1,840 |

the null procedure (i.e., the overhead of forking a process/thread); and Signal-Wait, the time for a process/thread to signal a waiting process/thread and then wait on a condition (i.e., the overhead of synchronizing two processes/threads together). We see there is an order of magnitude or more of difference between ULTs and KLTs, and similarly between KLTs and processes.

Thus, on the face of it, while there is a significant speedup by using KLT multithreading compared to single-threaded processes, there is an additional significant speedup by using ULTs. However, whether or not the additional speedup is realized depends on the nature of the applications involved. If most of the thread switches in an application require kernel-mode access, then a ULT-based scheme may not perform much better than a KLT-based scheme.

***Combined Approaches*** Some operating systems provide a combined ULT/KLT facility (see Figure 4.5c). In a combined system, thread creation is done completely in user space, as is the bulk of the scheduling and synchronization of threads within an application. The multiple ULTs from a single application are mapped onto some (smaller or equal) number of KLTs. The programmer may adjust the number of KLTs for a particular application and processor to achieve the best overall results.

In a combined approach, multiple threads within the same application can run in parallel on multiple processors, and a blocking system call need not block the entire process. If properly designed, this approach should combine the advantages of the pure ULT and KLT approaches while minimizing the disadvantages.

Solaris is a good example of an OS using this combined approach. The current Solaris version limits the ULT/KLT relationship to be one-to-one.

## Other Arrangements

As we have said, the concepts of resource allocation and dispatching unit have traditionally been embodied in the single concept of the process—that is, as a 1 : 1 relationship between threads and processes. Recently, there has been much interest in providing for multiple threads within a single process, which is a many-to-one relationship. However, as Table 4.2 shows, the other two combinations have also been investigated, namely, a many-to-many relationship and a one-to-many relationship.

**Table 4.2** Relationship between Threads and Processes

| Threads: Processes | Description | Example Systems |
|---|---|---|
| 1:1 | Each thread of execution is a unique process with its own address space and resources. | Traditional UNIX implementations |
| M:1 | A process defines an address space and dynamic resource ownership. Multiple threads may be created and executed within that process. | Windows NT, Solaris, Linux, OS/2, OS/390, MACH |
| 1:M | A thread may migrate from one process environment to another. This allows a thread to be easily moved among distinct systems. | Ra (Clouds), Emerald |
| M:N | It combines attributes of M:1 and 1:M cases. | TRIX |

***MANY-TO-MANY RELATIONSHIP*** The idea of having a many-to-many relationship between threads and processes has been explored in the experimental operating system TRIX [PAZZ92, WARD80]. In TRIX, there are the concepts of domain and thread. A domain is a static entity, consisting of an address space and "ports" through which messages may be sent and received. A thread is a single execution path, with an execution stack, processor state, and scheduling information.

As with the multithreading approaches discussed so far, multiple threads may execute in a single domain, providing the efficiency gains discussed earlier. However, it is also possible for a single-user activity, or application, to be performed in multiple domains. In this case, a thread exists that can move from one domain to another.

The use of a single thread in multiple domains seems primarily motivated by a desire to provide structuring tools for the programmer. For example, consider a program that makes use of an I/O subprogram. In a multiprogramming environment that allows user-spawned processes, the main program could generate a new process to handle I/O, then continue to execute. However, if the future progress of the main program depends on the outcome of the I/O operation, then the main program will have to wait for the other I/O program to finish. There are several ways to implement this application:

1. The entire program can be implemented as a single process. This is a reasonable and straightforward solution. There are drawbacks related to memory management. The process as a whole may require considerable main memory to execute efficiently, whereas the I/O subprogram requires a relatively small address space to buffer I/O and to handle the relatively small amount of program code. Because the I/O program executes in the address space of the larger program, either the entire process must remain in main memory during the I/O operation, or the I/O operation is subject to swapping. This memory management effect would also exist if the main program and the I/O subprogram were implemented as two threads in the same address space.
2. The main program and I/O subprogram can be implemented as two separate processes. This incurs the overhead of creating the subordinate process. If the I/O activity is frequent, one must either leave the subordinate process alive, which consumes management resources, or frequently create and destroy the subprogram, which is inefficient.
3. Treat the main program and the I/O subprogram as a single activity that is to be implemented as a single thread. However, one address space (domain) could be created for the main program and one for the I/O subprogram. Thus, the thread can be moved between the two address spaces as execution proceeds. The OS can manage the two address spaces independently, and no process creation overhead is incurred. Furthermore, the address space used by the I/O subprogram could also be shared by other simple I/O programs.

The experiences of the TRIX developers indicate that the third option has merit, and may be the most effective solution for some applications.

***ONE-TO-MANY RELATIONSHIP*** In the field of distributed operating systems (designed to control distributed computer systems), there has been interest in the

concept of a thread as primarily an entity that can move among address spaces.[5] A notable example of this research is the Clouds operating system, and especially its kernel, known as Ra [DASG92]. Another example is the Emerald system [STEE95].

A thread in Clouds is a unit of activity from the user's perspective. A process is a virtual address space with an associated process control block. Upon creation, a thread starts executing in a process by invoking an entry point to a program in that process. Threads may move from one address space to another, and actually span computer boundaries (i.e., move from one computer to another). As a thread moves, it must carry with it certain information, such as the controlling terminal, global parameters, and scheduling guidance (e.g., priority).

The Clouds approach provides an effective way of insulating both users and programmers from the details of the distributed environment. A user's activity may be represented as a single thread, and the movement of that thread among computers may be dictated by the OS for a variety of system-related reasons, such as the need to access a remote resource, and load balancing.

## 4.3 MULTICORE AND MULTITHREADING

The use of a multicore system to support a single application with multiple threads (such as might occur on a workstation, a video game console, or a personal computer running a processor-intense application) raises issues of performance and application design. In this section, we first look at some of the performance implications of a multithreaded application on a multicore system, then describe a specific example of an application designed to exploit multicore capabilities.

### Performance of Software on Multicore

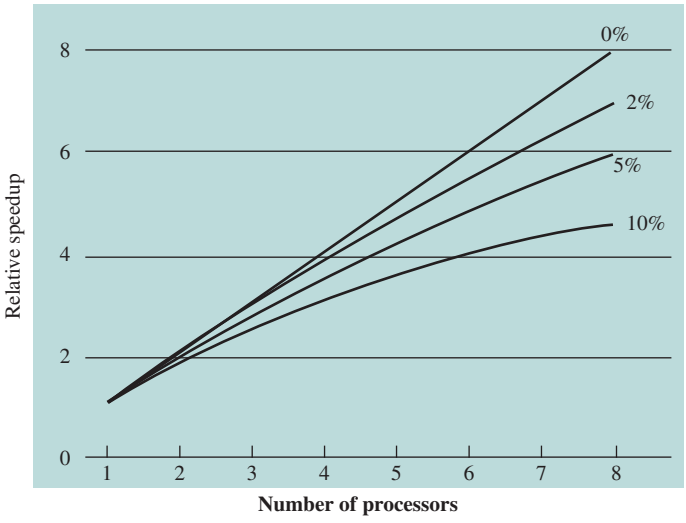
The potential performance benefits of a multicore organization depend on the ability to effectively exploit the parallel resources available to the application. Let us focus first on a single application running on a multicore system. Amdahl's law (see Appendix E) states that:

$$\text{Speedup} = \frac{\text{time to execute program on a single processor}}{\text{time to execute program on } N \text{ parallel processors}} = \frac{1}{(1 - f) + \frac{f}{N}}$$

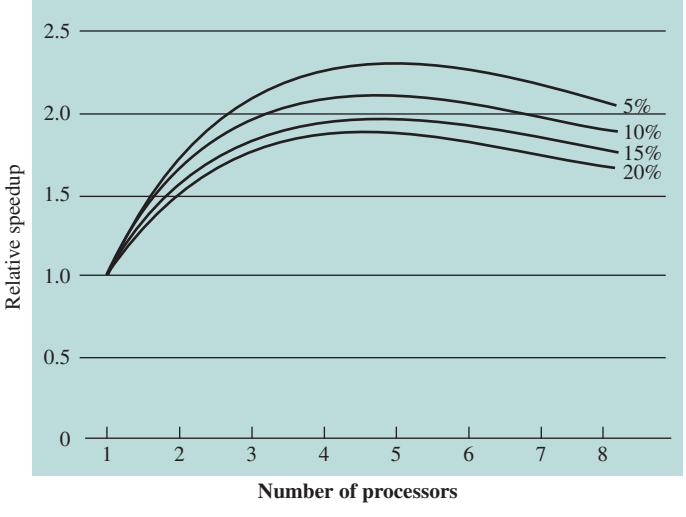
The law assumes a program in which a fraction $(1 - f)$ of the execution time involves code that is inherently serial, and a fraction $f$ that involves code that is infinitely parallelizable with no scheduling overhead.

This law appears to make the prospect of a multicore organization attractive. But as Figure 4.7a shows, even a small amount of serial code has a noticeable impact. If only 10% of the code is inherently serial ($f = 0.9$), running the program on a multicore system with eight processors yields a performance gain of a factor of only 4.7. In

[5]The movement of processes or threads among address spaces, or thread migration, on different machines has become a hot topic in recent years. Chapter 18 will explore this topic.




(a) Speedup with 0%, 2%, 5%, and 10% sequential portions




(b) Speedup with overheads

**Figure 4.7 Performance Effect of Multiple Cores**

addition, software typically incurs overhead as a result of communication and distribution of work to multiple processors and cache coherence overhead. This results in a curve where performance peaks and then begins to degrade because of the increased burden of the overhead of using multiple processors. Figure 4.7b, from [MCDO07], is a representative example.

However, software engineers have been addressing this problem, and there are numerous applications in which it is possible to effectively exploit a multicore

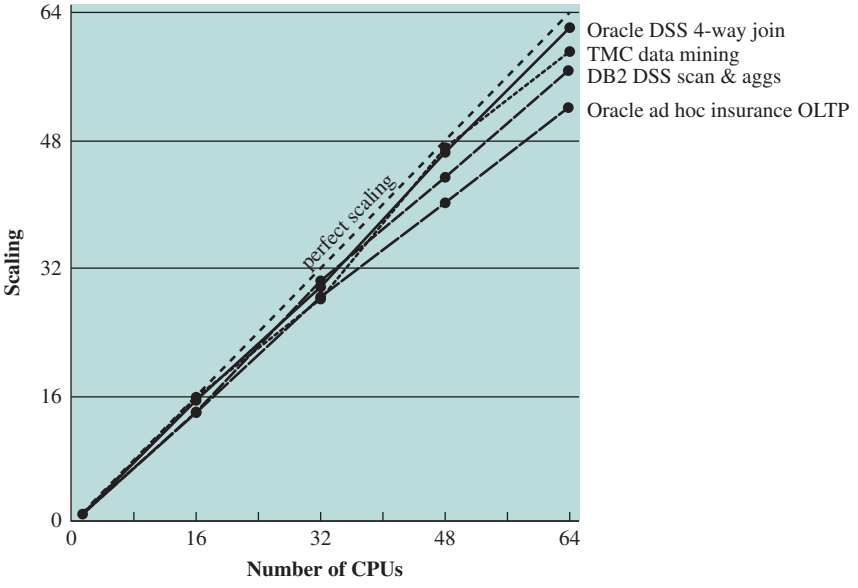



**Figure 4.8 Scaling of Database Workloads on Multiprocessor Hardware**

system. [MCDO07] reports on a set of database applications, in which great attention was paid to reducing the serial fraction within hardware architectures, operating systems, middleware, and the database application software. Figure 4.8 shows the result. As this example shows, database management systems and database applications are one area in which multicore systems can be used effectively. Many kinds of servers can also effectively use the parallel multicore organization, because servers typically handle numerous relatively independent transactions in parallel.

In addition to general-purpose server software, a number of classes of applications benefit directly from the ability to scale throughput with the number of cores. [MCDO06] lists the following examples:

- **Multithreaded native applications:** Multithreaded applications are characterized by having a small number of highly threaded processes. Examples of threaded applications include Lotus Domino or Siebel CRM (Customer Relationship Manager).
- **Multiprocess applications:** Multiprocess applications are characterized by the presence of many single-threaded processes. Examples of multiprocess applications include the Oracle database, SAP, and PeopleSoft.
- **Java applications:** Java applications embrace threading in a fundamental way. Not only does the Java language greatly facilitate multithreaded applications, but the Java Virtual Machine is a multithreaded process that provides scheduling and memory management for Java applications. Java applications that can benefit directly from multicore resources include application servers such as Oracle's Java Application Server, BEA's Weblogic, IBM's Websphere, and the open-source Tomcat application server. All applications that use a Java 2

Platform, Enterprise Edition (J2EE platform) application server can immediately benefit from multicore technology.

- **Multi-instance applications:** Even if an individual application does not scale to take advantage of a large number of threads, it is still possible to gain from multicore architecture by running multiple instances of the application in parallel. If multiple application instances require some degree of isolation, virtualization technology (for the hardware of the operating system) can be used to provide each of them with its own separate and secure environment.

## Application Example: Valve Game Software

Valve is an entertainment and technology company that has developed a number of popular games, as well as the Source engine, one of the most widely played game engines available. Source is an animation engine used by Valve for its games and licensed for other game developers.

In recent years, Valve has reprogrammed the Source engine software to use multithreading to exploit the power of multicore processor chips from Intel and AMD [REIM06]. The revised Source engine code provides more powerful support for Valve games such as *Half Life 2*.

From Valve's perspective, threading granularity options are defined as follows [HARR06]:

- **Coarse threading:** Individual modules, called systems, are assigned to individual processors. In the Source engine case, this would mean putting rendering on one processor, AI (artificial intelligence) on another, physics on another, and so on. This is straightforward. In essence, each major module is single-threaded and the principal coordination involves synchronizing all the threads with a timeline thread.
- **Fine-grained threading:** Many similar or identical tasks are spread across multiple processors. For example, a loop that iterates over an array of data can be split up into a number of smaller parallel loops in individual threads that can be scheduled in parallel.
- **Hybrid threading:** This involves the selective use of fine-grained threading for some systems, and single-threaded for other systems.

Valve found that through coarse threading, it could achieve up to twice the performance across two processors compared to executing on a single processor. But this performance gain could only be achieved with contrived cases. For real-world gameplay, the improvement was on the order of a factor of 1.2. Valve also found that effective use of fine-grained threading was difficult. The time-per-work unit can be variable, and managing the timeline of outcomes and consequences involved complex programming.

Valve found that a hybrid threading approach was the most promising and would scale the best, as multicore systems with 8 or 16 processors became available. Valve identified systems that operate very effectively being permanently assigned to a single processor. An example is sound mixing, which has little user interaction, is not constrained by the frame configuration of windows, and works on its own set

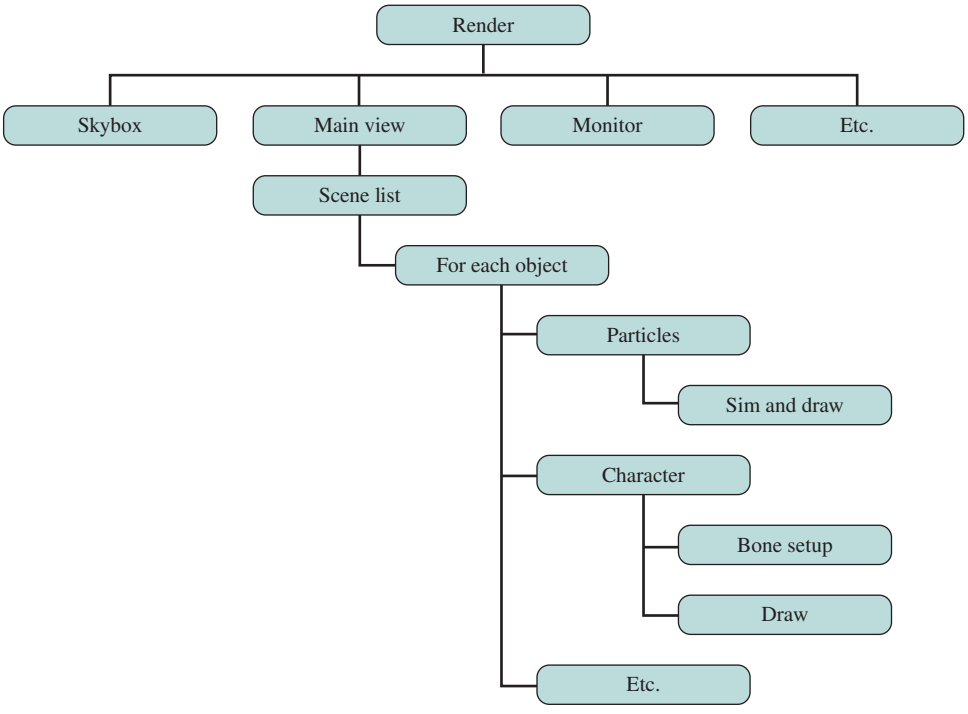



**Figure 4.9 Hybrid Threading for Rendering Module**

of data. Other modules, such as scene rendering, can be organized into a number of threads so that the module can execute on a single processor but achieve greater performance as it is spread out over more and more processors.

Figure 4.9 illustrates the thread structure for the rendering module. In this hierarchical structure, higher-level threads spawn lower-level threads as needed. The rendering module relies on a critical part of the Source engine, the world list, which is a database representation of the visual elements in the game's world. The first task is to determine what are the areas of the world that need to be rendered. The next task is to determine what objects are in the scene as viewed from multiple angles. Then comes the processor-intensive work. The rendering module has to work out the rendering of each object from multiple points of view, such as the player's view, the view of monitors, and the point of view of reflections in water.

Some of the key elements of the threading strategy for the rendering module are listed in [LEON07] and include the following:

- Construct scene rendering lists for multiple scenes in parallel (e.g., the world and its reflection in water).
- Overlap graphics simulation.
- Compute character bone transformations for all characters in all scenes in parallel.
- Allow multiple threads to draw in parallel.

The designers found that simply locking key databases, such as the world list, for a thread was too inefficient. Over 95% of the time, a thread is trying to read from a data set, and only 5% of the time at most is spent in writing to a data set. Thus, a concurrency mechanism known as the single-writer-multiple-readers model works effectively.

## 4.4 WINDOWS PROCESS AND THREAD MANAGEMENT

This section begins with an overview of the key objects and mechanisms that support application execution in Windows. The remainder of the section looks in more detail at how processes and threads are managed.

An **application** consists of one or more processes. Each **process** provides the resources needed to execute a program. A process has a virtual address space, executable code, open handles to system objects, a security context, a unique process identifier, environment variables, a priority class, minimum and maximum working set sizes, and at least one thread of execution. Each process is started with a single thread, often called the primary thread, but can create additional threads from any of its threads.

A **thread** is the entity within a process that can be scheduled for execution. All threads of a process share its virtual address space and system resources. In addition, each thread maintains exception handlers, a scheduling priority, thread local storage, a unique thread identifier, and a set of structures the system will use to save the thread context until it is scheduled. On a multiprocessor computer, the system can simultaneously execute as many threads as there are processors on the computer.

A **job object** allows groups of processes to be managed as a unit. Job objects are namable, securable, sharable objects that control attributes of the processes associated with them. Operations performed on the job object affect all processes associated with the job object. Examples include enforcing limits such as working set size and process priority or terminating all processes associated with a job.

A **thread pool** is a collection of worker threads that efficiently execute asynchronous callbacks on behalf of the application. The thread pool is primarily used to reduce the number of application threads and provide management of the worker threads.

A **fiber** is a unit of execution that must be manually scheduled by the application. Fibers run in the context of the threads that schedule them. Each thread can schedule multiple fibers. In general, fibers do not provide advantages over a well-designed multithreaded application. However, using fibers can make it easier to port applications that were designed to schedule their own threads. From a system standpoint, a fiber assumes the identity of the thread that runs it. For example if a fiber accesses thread local storage, it is accessing the thread local storage of the thread that is running it. In addition, if a fiber calls the `ExitThread` function, the thread that is running it exits. However, a fiber does not have all the same state information associated with it as that associated with a thread. The only state information maintained for a fiber is its stack, a subset of its registers, and the fiber data provided during fiber creation. The saved registers are the set of registers typically preserved across

a function call. Fibers are not preemptively scheduled. A thread schedules a fiber by switching to it from another fiber. The system still schedules threads to run. When a thread that is running fibers is preempted, its currently running fiber is preempted but remains selected.

**User-mode scheduling (UMS)** is a lightweight mechanism that applications can use to schedule their own threads. An application can switch between UMS threads in user mode without involving the system scheduler, and regain control of the processor if a UMS thread blocks in the kernel. Each UMS thread has its own thread context instead of sharing the thread context of a single thread. The ability to switch between threads in user mode makes UMS more efficient than thread pools for short-duration work items that require few system calls. UMS is useful for applications with high performance requirements that need to efficiently run many threads concurrently on multiprocessor or multicore systems. To take advantage of UMS, an application must implement a scheduler component that manages the application's UMS threads and determines when they should run.

## Management of Background Tasks and Application Lifecycles

Beginning with Windows 8, and carrying through to Windows 10, developers are responsible for managing the state of their individual applications. Previous versions of Windows always give the user full control of the lifetime of a process. In the classic desktop environment, a user is responsible for closing an application. A dialog box might prompt them to save their work. In the new Metro interface, Windows takes over the process lifecycle of an application. Although a limited number of applications can run alongside the main app in the Metro UI using the SnapView functionality, only one Store application can run at one time. This is a direct consequence of the new design. Windows Live Tiles give the appearance of applications constantly running on the system. In reality, they receive push notifications and do not use system resources to display the dynamic content offered.

The foreground application in the Metro interface has access to all of the processor, network, and disk resources available to the user. All other apps are suspended and have no access to these resources. When an app enters a suspended mode, an event should be triggered to store the state of the user's information. This is the responsibility of the application developer. For a variety of reasons, whether it needs resources or because an application timed out, Windows may terminate a background app. This is a significant departure from the Windows operating systems that precede it. The app needs to retain any data the user entered, settings they changed, and so on. That means you need to save your app's state when it's suspended, in case Windows terminates it, so you can restore its state later. When the app returns to the foreground, another event is triggered to obtain the user state from memory. No event fires to indicate termination of a background app. Rather, the application data will remain resident on the system, as though it is suspended, until the app is launched again. Users expect to find the app as they left it, whether it was suspended or terminated by Windows, or closed by the user. Application developers can use code to determine whether it should restore a saved state.

Some applications, such as news feeds, may look at the date stamp associated with the previous execution of the app and elect to discard the data in favor of newly obtained information. This is a determination made by the developer, not by the operating system. If the user closes an app, unsaved data is not saved. With foreground tasks occupying all of the system resources, starvation of background apps is a reality in Windows. This makes the application development relating to the state changes critical to the success of a Windows app.

To process the needs of background tasks, a background task API is built to allow apps to perform small tasks while not in the foreground. In this restricted environment, apps may receive push notifications from a server or a user may receive a phone call. Push notifications are template XML strings. They are managed through a cloud service known as the Windows Notification Service (WNS). The service will occasionally push updates to the user's background apps. The API will queue those requests and process them when it receives enough processor resources. Background tasks are severely limited in the usage of processor, receiving only one processor second per processor hour. This ensures that critical tasks receive guaranteed application resource quotas. It does not, however, guarantee a background app will ever run.

## The Windows Process

Important characteristics of Windows processes are the following:

- Windows processes are implemented as objects.
- A process can be created as a new process or as a copy of an existing process.
- An executable process may contain one or more threads.
- Both process and thread objects have built-in synchronization capabilities.

Figure 4.10, based on one in [RUSS11], illustrates the way in which a process relates to the resources it controls or uses. Each process is assigned a security access token, called the primary token of the process. When a user first logs on, Windows creates an access token that includes the security ID for the user. Every process that is created by or runs on behalf of this user has a copy of this access token. Windows uses the token to validate the user's ability to access secured objects, or to perform restricted functions on the system and on secured objects. The access token controls whether the process can change its own attributes. In this case, the process does not have a handle opened to its access token. If the process attempts to open such a handle, the security system determines whether this is permitted, and therefore whether the process may change its own attributes.

Also related to the process is a series of blocks that define the virtual address space currently assigned to this process. The process cannot directly modify these structures, but must rely on the virtual memory manager, which provides a memory-allocation service for the process.

Finally, the process includes an object table, with handles to other objects known to this process. Figure 4.10 shows a single thread. In addition, the process has access to a file object and to a section object that defines a section of shared memory.

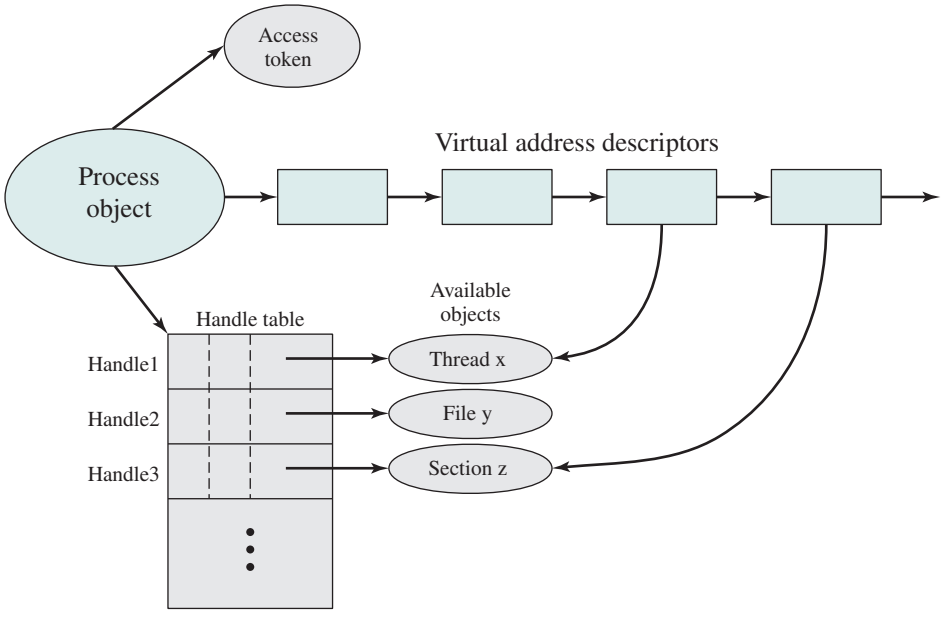



**Figure 4.10** **A Windows Process and Its Resources**

## Process and Thread Objects

The object-oriented structure of Windows facilitates the development of a general-purpose process facility. Windows makes use of two types of process-related objects: processes and threads. A process is an entity corresponding to a user job or application that owns resources, such as memory and open files. A thread is a dispatchable unit of work that executes sequentially and is interruptible, so the processor can turn to another thread.

Each Windows process is represented by an object. Each process object includes a number of attributes and encapsulates a number of actions, or services, that it may perform. A process will perform a service when called upon through a set of published interface methods. When Windows creates a new process, it uses the object class, or type, defined for the Windows process as a template to generate a new object instance. At the time of creation, attribute values are assigned. Table 4.3 gives a brief definition of each of the object attributes for a process object.

A Windows process must contain at least one thread to execute. That thread may then create other threads. In a multiprocessor system, multiple threads from the same process may execute in parallel. Table 4.4 defines the thread object attributes. Note some of the attributes of a thread resemble those of a process. In those cases, the thread attribute value is derived from the process attribute value. For example, the *thread processor affinity* is the set of processors in a multiprocessor system that may execute this thread; this set is equal to or a subset of the *process processor affinity.*

**Table 4.3** Windows Process Object Attributes

| | |
|---|---|
| **Process ID** | A unique value that identifies the process to the operating system. |
| **Security descriptor** | Describes who created an object, who can gain access to or use the object, and who is denied access to the object. |
| **Base priority** | A baseline execution priority for the process's threads. |
| **Default processor affinity** | The default set of processors on which the process's threads can run. |
| **Quota limits** | The maximum amount of paged and nonpaged system memory, paging file space, and processor time a user's processes can use. |
| **Execution time** | The total amount of time all threads in the process have executed. |
| **I/O counters** | Variables that record the number and type of I/O operations that the process's threads have performed. |
| **VM operation counters** | Variables that record the number and types of virtual memory operations that the process's threads have performed. |
| **Exception/debugging ports** | Interprocess communication channels to which the process manager sends a message when one of the process's threads causes an exception. Normally, these are connected to environment subsystem and debugger processes, respectively. |
| **Exit status** | The reason for a process's termination. |

Note one of the attributes of a thread object is context, which contains the values of the processor registers when the thread last ran. This information enables threads to be suspended and resumed. Furthermore, it is possible to alter the behavior of a thread by altering its context while it is suspended.

**Table 4.4** Windows Thread Object Attributes

| | |
|---|---|
| **Thread ID** | A unique value that identifies a thread when it calls a server. |
| **Thread context** | The set of register values and other volatile data that defines the execution state of a thread. |
| **Dynamic priority** | The thread's execution priority at any given moment. |
| **Base priority** | The lower limit of the thread's dynamic priority. |
| **Thread processor affinity** | The set of processors on which the thread can run, which is a subset or all of the processor affinity of the thread's process. |
| **Thread execution time** | The cumulative amount of time a thread has executed in user mode and in kernel mode. |
| **Alert status** | A flag that indicates whether a waiting thread may execute an asynchronous procedure call. |
| **Suspension count** | The number of times the thread's execution has been suspended without being resumed. |
| **Impersonation token** | A temporary access token allowing a thread to perform operations on behalf of another process (used by subsystems). |
| **Termination port** | An interprocess communication channel to which the process manager sends a message when the thread terminates (used by subsystems). |
| **Thread exit status** | The reason for a thread's termination. |

## Multithreading

Windows supports concurrency among processes because threads in different processes may execute concurrently (appear to run at the same time). Moreover, multiple threads within the same process may be allocated to separate processors and execute simultaneously (actually run at the same time). A multithreaded process achieves concurrency without the overhead of using multiple processes. Threads within the same process can exchange information through their common address space and have access to the shared resources of the process. Threads in different processes can exchange information through shared memory that has been set up between the two processes.

An object-oriented multithreaded process is an efficient means of implementing a server application. For example, one server process can service a number of clients concurrently.

## Thread States

An existing Windows thread is in one of six states (see Figure 4.11):

1. **Ready:** A ready thread may be scheduled for execution. The Kernel dispatcher keeps track of all ready threads and schedules them in priority order.
2. **Standby:** A standby thread has been selected to run next on a particular processor. The thread waits in this state until that processor is made available. If the standby thread's priority is high enough, the running thread on that processor may be preempted in favor of the standby thread. Otherwise, the standby thread waits until the running thread blocks or exhausts its time slice.

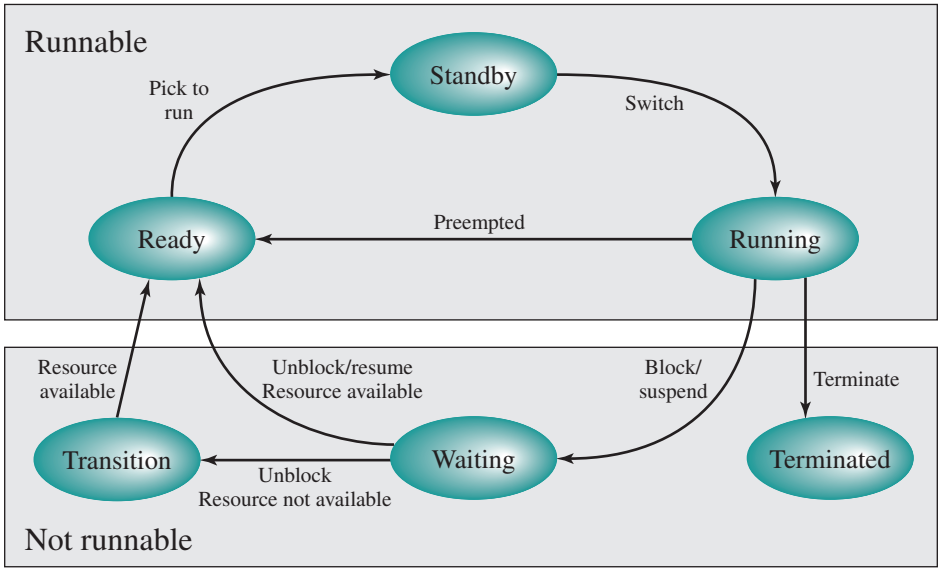



**Figure 4.11 Windows Thread States**

3. **Running:** Once the Kernel dispatcher performs a thread switch, the standby thread enters the Running state and begins execution and continues execution until it is preempted by a higher-priority thread, exhausts its time slice, blocks, or terminates. In the first two cases, it goes back to the Ready state.
4. **Waiting:** A thread enters the Waiting state when (1) it is blocked on an event (e.g., I/O), (2) it voluntarily waits for synchronization purposes, or (3) an environment subsystem directs the thread to suspend itself. When the waiting condition is satisfied, the thread moves to the Ready state if all of its resources are available.
5. **Transition:** A thread enters this state after waiting if it is ready to run, but the resources are not available. For example, the thread's stack may be paged out of memory. When the resources are available, the thread goes to the Ready state.
6. **Terminated:** A thread can be terminated by itself, by another thread, or when its parent process terminates. Once housekeeping chores are completed, the thread is removed from the system, or it may be retained by the Executive[6] for future reinitialization.

### Support for OS Subsystems

The general-purpose process and thread facility must support the particular process and thread structures of the various OS environments. It is the responsibility of each OS subsystem to exploit the Windows process and thread features to emulate the process and thread facilities of its corresponding OS. This area of process/thread management is complicated, and we give only a brief overview here.

Process creation begins with a request for a new process from an application. The application issues a create-process request to the corresponding protected subsystem, which passes the request to the Executive. The Executive creates a process object and returns a handle for that object to the subsystem. When Windows creates a process, it does not automatically create a thread. In the case of Win32, a new process must always be created with an initial thread. Therefore, the Win32 subsystem calls the Windows process manager again to create a thread for the new process, receiving a thread handle back from Windows. The appropriate thread and process information are then returned to the application. In the case of POSIX, threads are not supported. Therefore, the POSIX subsystem obtains a thread for the new process from Windows so that the process may be activated but returns only process information to the application. The fact that the POSIX process is implemented using both a process and a thread from the Windows Executive is not visible to the application.

When a new process is created by the Executive, the new process inherits many of its attributes from the creating process. However, in the Win32 environment, this process creation is done indirectly. An application client process issues its process creation request to the Win32 subsystem; then the subsystem in turn issues a process request to the Windows executive. Because the desired effect is that the new process inherits characteristics of the client process and not of the server process, Windows enables the

[6]The Windows Executive is described in Chapter 2. It contains the base operating system services, such as memory management, process and thread management, security, I/O, and interprocess communication.

subsystem to specify the parent of the new process. The new process then inherits the parent's access token, quota limits, base priority, and default processor affinity.

## 4.5 SOLARIS THREAD AND SMP MANAGEMENT

Solaris implements multilevel thread support designed to provide considerable flexibility in exploiting processor resources.

### Multithreaded Architecture

Solaris makes use of four separate thread-related concepts:

1. **Process:** This is the normal UNIX process and includes the user's address space, stack, and process control block.
2. **User-level threads:** Implemented through a threads library in the address space of a process, these are invisible to the OS. A user-level thread (ULT)[7] is a user-created unit of execution within a process.
3. **Lightweight processes:** A lightweight process (LWP) can be viewed as a mapping between ULTs and kernel threads. Each LWP supports ULT and maps to one kernel thread. LWPs are scheduled by the kernel independently, and may execute in parallel on multiprocessors.
4. **Kernel threads:** These are the fundamental entities that can be scheduled and dispatched to run on one of the system processors.

Figure 4.12 illustrates the relationship among these four entities. Note there is always exactly one kernel thread for each LWP. An LWP is visible within a process to the application. Thus, LWP data structures exist within their respective process address space. At the same time, each LWP is bound to a single dispatchable kernel thread, and the data structure for that kernel thread is maintained within the kernel's address space.

A process may consist of a single ULT bound to a single LWP. In this case, there is a single thread of execution, corresponding to a traditional UNIX process. When concurrency is not required within a single process, an application uses this process structure. If an application requires concurrency, its process contains multiple threads, each bound to a single LWP, which in turn are each bound to a single kernel thread.

In addition, there are kernel threads that are not associated with LWPs. The kernel creates, runs, and destroys these kernel threads to execute specific system functions. The use of kernel threads rather than kernel processes to implement system functions reduces the overhead of switching within the kernel (from a process switch to a thread switch).

### Motivation

The three-level thread structure (ULT, LWP, kernel thread) in Solaris is intended to facilitate thread management by the OS and to provide a clean interface to applications. The ULT interface can be a standard thread library. A defined ULT maps onto a LWP, which is managed by the OS and which has defined states of execution,

[7]Again, the acronym ULT is unique to this book and is not found in the Solaris literature.

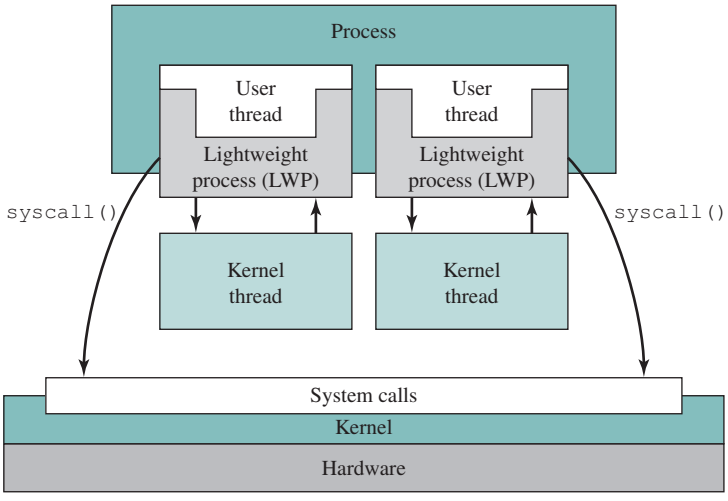



**Figure 4.12 Processes and Threads in Solaris [MCDO07]**

defined subsequently. An LWP is bound to a kernel thread with a one-to-one correspondence in execution states. Thus, concurrency and execution are managed at the level of the kernel thread.

In addition, an application has access to hardware through an application programming interface consisting of system calls. The API allows the user to invoke kernel services to perform privileged tasks on behalf of the calling process, such as read or write a file, issue a control command to a device, create a new process or thread, allocate memory for the process to use, and so on.

## Process Structure

Figure 4.13 compares, in general terms, the process structure of a traditional UNIX system with that of Solaris. On a typical UNIX implementation, the process structure includes:

- Process ID.
- User IDs.
- Signal dispatch table, which the kernel uses to decide what to do when sending a signal to a process.
- File descriptors, which describe the state of files in use by this process.
- Memory map, which defines the address space for this process.
- Processor state structure, which includes the kernel stack for this process.

Solaris retains this basic structure but replaces the processor state block with a list of structures containing one data block for each LWP.

The LWP data structure includes the following elements:

- An LWP identifier
- The priority of this LWP and hence the kernel thread that supports it

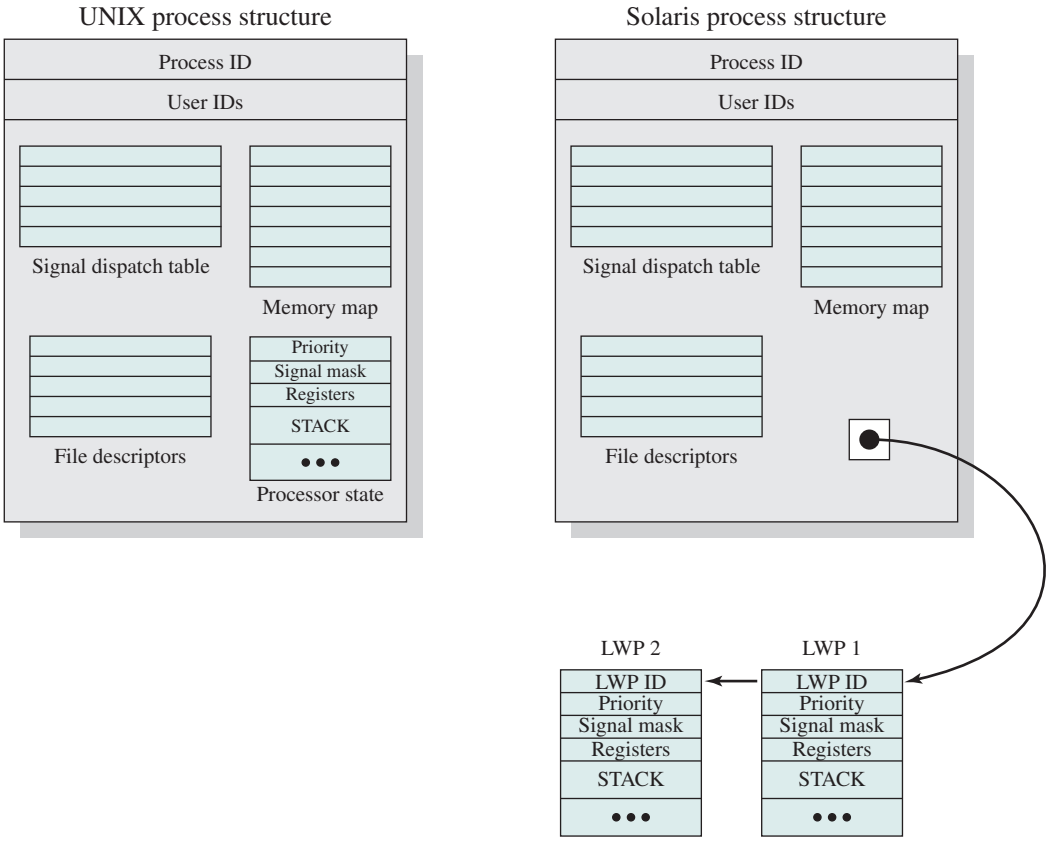



**Figure 4.13 Process Structure in Traditional UNIX and Solaris [LEWI96]**

- A signal mask that tells the kernel which signals will be accepted
- Saved values of user-level registers (when the LWP is not running)
- The kernel stack for this LWP, which includes system call arguments, results, and error codes for each call level
- Resource usage and profiling data
- Pointer to the corresponding kernel thread
- Pointer to the process structure

## Thread Execution

Figure 4.14 shows a simplified view of both thread execution states. These states reflect the execution status of both a kernel thread and the LWP bound to it. As mentioned, some kernel threads are not associated with an LWP; the same execution diagram applies. The states are as follows:

- **RUN:** The thread is runnable; that is, the thread is ready to execute.
- **ONPROC:** The thread is executing on a processor.

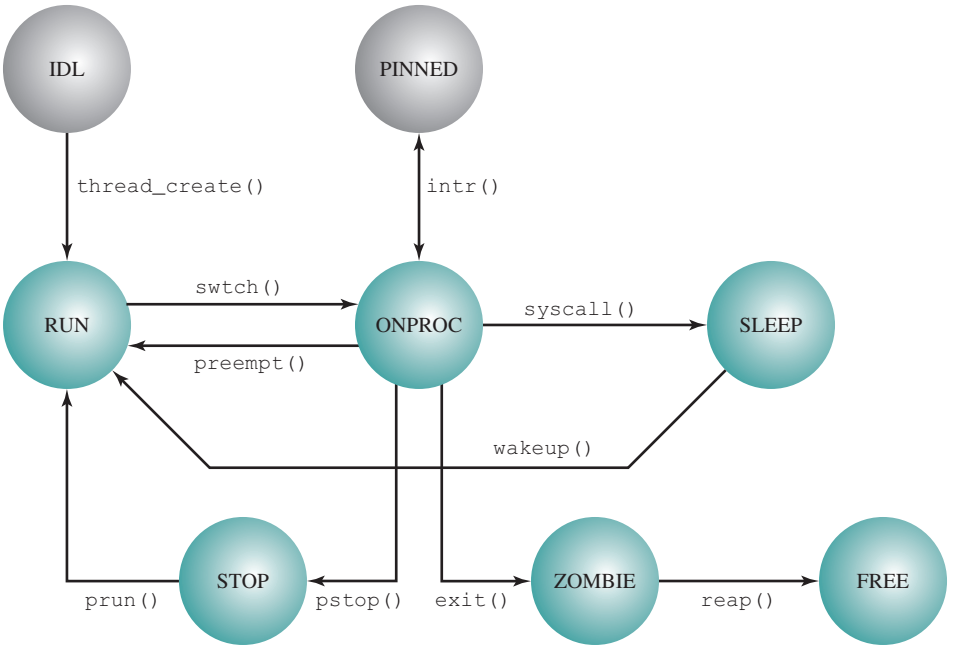



**Figure 4.14 Solaris Thread States**

- **SLEEP:** The thread is blocked.
- **STOP:** The thread is stopped.
- **ZOMBIE:** The thread has terminated.
- **FREE:** Thread resources have been released and the thread is awaiting removal from the OS thread data structure.

A thread moves from ONPROC to RUN if it is preempted by a higher-priority thread or because of time slicing. A thread moves from ONPROC to SLEEP if it is blocked and must await an event to return the RUN state. Blocking occurs if the thread invokes a system call and must wait for the system service to be performed. A thread enters the STOP state if its process is stopped; this might be done for debugging purposes.

## Interrupts as Threads

Most operating systems contain two fundamental forms of concurrent activity: processes and interrupts. Processes (or threads) cooperate with each other and manage the use of shared data structures by means of a variety of primitives that enforce mutual exclusion (only one process at a time can execute certain code or access certain data) and that synchronize their execution. Interrupts are synchronized by preventing their handling for a period of time. Solaris unifies these two concepts into a single model, namely kernel threads, and the mechanisms for scheduling and executing kernel threads. To do this, interrupts are converted to kernel threads.

The motivation for converting interrupts to threads is to reduce overhead. Interrupt handlers often manipulate data shared by the rest of the kernel. Therefore, while a kernel routine that accesses such data is executing, interrupts must be blocked, even though most interrupts will not affect that data. Typically, the way this is done is for the routine to set the interrupt priority level higher to block interrupts, then lower the priority level after access is completed. These operations take time. The problem is magnified on a multiprocessor system. The kernel must protect more objects and may need to block interrupts on all processors.

The solution in Solaris can be summarized as follows:

1. Solaris employs a set of kernel threads to handle interrupts. As with any kernel thread, an interrupt thread has its own identifier, priority, context, and stack.
2. The kernel controls access to data structures and synchronizes among interrupt threads using mutual exclusion primitives, of the type to be discussed in Chapter 5. That is, the normal synchronization techniques for threads are used in handling interrupts.
3. Interrupt threads are assigned higher priorities than all other types of kernel threads.

When an interrupt occurs, it is delivered to a particular processor and the thread that was executing on that processor is pinned. A pinned thread cannot move to another processor and its context is preserved; it is simply suspended until the interrupt is processed. The processor then begins executing an interrupt thread. There is a pool of deactivated interrupt threads available, so a new thread creation is not required. The interrupt thread then executes to handle the interrupt. If the handler routine needs access to a data structure that is currently locked in some fashion for use by another executing thread, the interrupt thread must wait for access to that data structure. An interrupt thread can only be preempted by another interrupt thread of higher priority.

Experience with Solaris interrupt threads indicates that this approach provides superior performance to the traditional interrupt-handling strategy [KLEI95].

## 4.6 LINUX PROCESS AND THREAD MANAGEMENT

### Linux Tasks

A process, or task, in Linux is represented by a `task_struct` data structure. The `task_struct` data structure contains information in a number of categories:

- **State:** The execution state of the process (executing, ready, suspended, stopped, zombie). This is described subsequently.
- **Scheduling information:** Information needed by Linux to schedule processes. A process can be normal or real time and has a priority. Real-time processes are scheduled before normal processes, and within each category, relative priorities can be used. A reference counter keeps track of the amount of time a process is allowed to execute.
- **Identifiers:** Each process has a unique process identifier (PID) and also has user and group identifiers. A group identifier is used to assign resource access privileges to a group of processes.

- **Interprocess communication:** Linux supports the IPC mechanisms found in UNIX SVR4, described later in Chapter 6.
- **Links:** Each process includes a link to its parent process, links to its siblings (processes with the same parent), and links to all of its children.
- **Times and timers:** Includes process creation time and the amount of processor time so far consumed by the process. A process may also have associated one or more interval timers. A process defines an interval timer by means of a system call; as a result, a signal is sent to the process when the timer expires. A timer may be single use or periodic.
- **File system:** Includes pointers to any files opened by this process, as well as pointers to the current and the root directories for this process
- **Address space:** Defines the virtual address space assigned to this process
- **Processor-specific context:** The registers and stack information that constitute the context of this process

Figure 4.15 shows the execution states of a process. These are as follows:

- **Running:** This state value corresponds to two states. A Running process is either executing, or it is ready to execute.
- **Interruptible:** This is a blocked state, in which the process is waiting for an event, such as the end of an I/O operation, the availability of a resource, or a signal from another process.

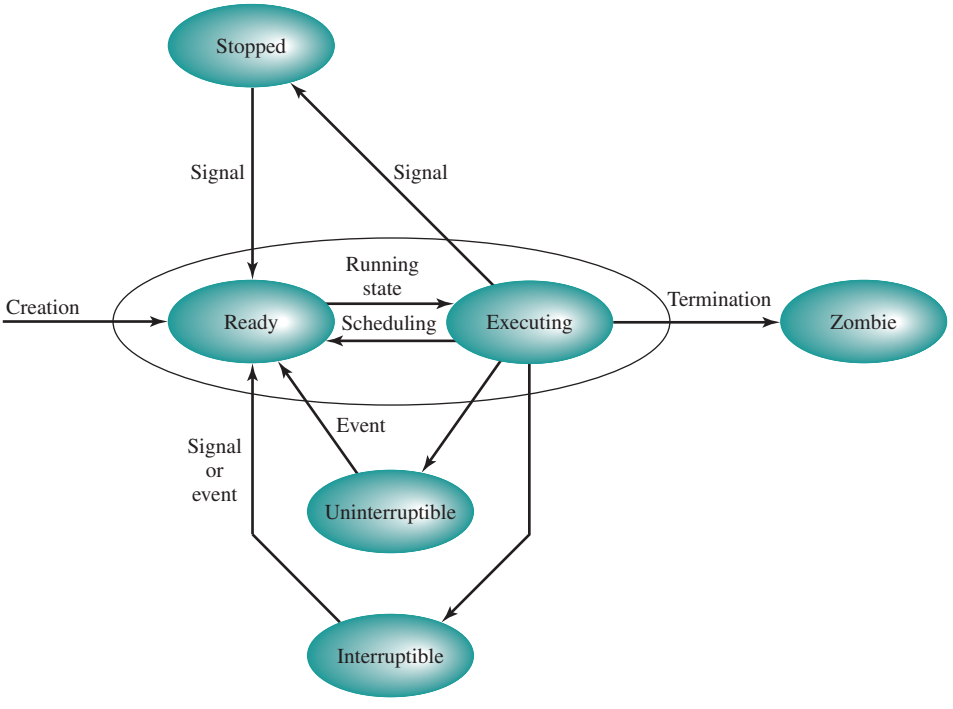



**Figure 4.15 Linux Process/Thread Model**

- **Uninterruptible:** This is another blocked state. The difference between this and the Interruptible state is that in an Uninterruptible state, a process is waiting directly on hardware conditions and therefore will not handle any signals.
- **Stopped:** The process has been halted and can only resume by positive action from another process. For example, a process that is being debugged can be put into the Stopped state.
- **Zombie:** The process has been terminated but, for some reason, still must have its task structure in the process table.

## Linux Threads

Traditional UNIX systems support a single thread of execution per process, while modern UNIX systems typically provide support for multiple kernel-level threads per process. As with traditional UNIX systems, older versions of the Linux kernel offered no support for multithreading. Instead, applications would need to be written with a set of user-level library functions, the most popular of which is known as *pthread (POSIX thread) libraries*, with all of the threads mapping into a single kernel-level process.[8] We have seen that modern versions of UNIX offer kernel-level threads. Linux provides a unique solution in that it does not recognize a distinction between threads and processes. Using a mechanism similar to the lightweight processes of Solaris, user-level threads are mapped into kernel-level processes. Multiple user-level threads that constitute a single user-level process are mapped into Linux kernel-level processes that share the same group ID. This enables these processes to share resources such as files and memory, and to avoid the need for a context switch when the scheduler switches among processes in the same group.

A new process is created in Linux by copying the attributes of the current process. A new process can be *cloned* so it shares resources such as files, signal handlers, and virtual memory. When the two processes share the same virtual memory, they function as threads within a single process. However, no separate type of data structure is defined for a thread. In place of the usual fork() command, processes are created in Linux using the clone() command. This command includes a set of flags as arguments. The traditional fork() system call is implemented by Linux as a clone() system call with all of the clone flags cleared.

Examples of clone flags include the following:

- CLONE_NEWPID: Creates new process ID namespace.
- CLONE_PARENT: Caller and new task share the same parent process.
- CLONE_SYSVSEM: Shares System V SEM_UNDO semantics.
- CLONE_THREAD: Inserts this process into the same thread group of the parent. If this flag is true, it implicitly enforces CLONE_PARENT.
- CLONE_VM: Shares the address space (memory descriptor and all page tables).

[8]POSIX (Portable Operating Systems based on UNIX) is an IEEE API standard that includes a standard for a thread API. Libraries implementing the POSIX Threads standard are often named *Pthreads*. Pthreads are most commonly used on UNIX-like POSIX systems such as Linux and Solaris, but Microsoft Windows implementations also exist.

- CLONE_FS: Shares the same filesystem information (including current working directory, the root of the filesystem, and the umask).
- CLONE_FILES: Shares the same file descriptor table. Creating a file descriptor or closing a file descriptor is propagated to the another process, as well as changing the associated flags of a file descriptor using the fcntl() system call.

When the Linux kernel performs a context switch from one process to another, it checks whether the address of the page directory of the current process is the same as that of the to-be-scheduled process. If they are, then they are sharing the same address space, so a context switch is basically just a jump from one location of code to another location of code.

Although cloned processes that are part of the same process group can share the same memory space, they cannot share the same user stacks. Thus the clone() call creates separate stack spaces for each process.

## Linux Namespaces

Associated with each process in Linux are a set of **namespaces**. A namespace enables a process (or multiple processes that share the same namespace) to have a different view of the system than other processes that have other associated namespaces. Namespaces and cgroups (which will be described in the following section) are the basis of Linux lightweight virtualization, which is a feature that provides a process or group of processes with the illusion that they are the only processes on the system. This feature is used widely by Linux Containers projects. There are currently six namespaces in Linux: mnt, pid, net, ipc, uts, and user.

Namespaces are created by the clone() system call, which gets as a parameter one of the six namespaces clone flags (CLONE_NEWNS, CLONE_NEWPID, CLONE_NEWNET, CLONE_NEWIPC, CLONE_NEWUTS, and CLONE_NEWUSER). A process can also create a namespace with the unshare() system call with one of these flags; as opposed to clone(), a new process is not created in such a case; only a new namespace is created, which is attached to the calling process.

***Mount Namespace*** A mount namespace provides the process with a specific view of the filesystem hierarchy, such that two processes with different mount namespaces see different filesystem hierarchies. All of the file operations that a process employs apply only to the filesystem visible to the process.

***UTS Namespace*** The UTS (UNIX timesharing) namespace is related to the uname Linux system call. The uname call returns the name and information about the current kernel, including nodename, which is the system name within some implementation-defined network; and domainname, which is the NIS domain name. NIS (Network Information Service) is a standard scheme used on all major UNIX and UNIX-like systems. It allows a group of machines within an NIS domain to share a common set of configuration files. This permits a system administrator to set up NIS client systems with only minimal configuration data and add, remove, or modify configuration data from a single location. With the UTS namespace, initialization and configuration parameters can vary for different processes on the same system.

***IPC Namespace*** An IPC namespace isolates certain interprocess communication (IPC) resources, such as semaphores, POSIX message queues, and more. Thus, concurrency mechanisms can be employed by the programmer that enable IPC among processes that share the same IPC namespace.

***PID Namespace*** PID namespaces isolate the process ID space, so processes in different PID namespaces can have the same PID. This feature is used for Checkpoint/Restore In Userspace (CRIU), a Linux software tool. Using this tool, you can freeze a running application (or part of it) and checkpoint it to a hard drive as a collection of files. You can then use the files to restore and run the application from the freeze point on that machine or on a different host. A distinctive feature of the CRIU project is that it is mainly implemented in user space, after attempts to implement it mainly in kernel failed.

***Network Namespace*** Network namespaces provide isolation of the system resources associated with networking. Thus, each network namespace has its own network devices, IP addresses, IP routing tables, port numbers, and so on. These namespaces virtualize all access to network resources. This allows each process or a group of processes that belong to this network namespace to have the network access it needs (but no more). At any given time, a network device belongs to only one network namespace. Also, a socket can belong to only one namespace.

***User Namespace*** User namespaces provide a container with its own set of UIDs, completely separate from those in the parent. So when a process clones a new process it can assign it a new user namespace, as well as a new PID namespace, and all the other namespaces. The cloned process can have access to and privileges for all of the resources of the parent process, or a subset of the resources and privileges of the parent. The user namespaces are considered sensitive in terms of security, as they enable creating non-privileged containers (processes which are created by a non-root user).

***The Linux cgroup Subsystem*** The Linux cgroup subsystem, together with the namespace subsystem, are the basis of lightweight process virtualization, and as such they form the basis of Linux containers; almost every Linux containers project nowadays (such as Docker, LXC, Kubernetes, and others) is based on both of them. The Linux cgroups subsystem provides resource management and accounting. It handles resources such as CPU, network, memory, and more; and it is mostly needed in both ends of the spectrum (embedded devices and servers), and much less in desktops. Development of cgroups was started in 2006 by engineers at Google under the name "process containers," which was later changed to "cgroups" to avoid confusion with Linux Containers. In order to implement cgroups, no new system call was added. A new virtual file system (VFS), "cgroups" (also referred to sometimes as cgroupfs) was added, as all the cgroup filesystem operations are filesystem based. A new version of cgroups, called cgroups v2, was released in kernel 4.5 (March 2016). The cgroup v2 subsystem addressed many of the inconsistencies across cgroup v1 controllers, and made cgroup v2 better organized, by establishing strict and consistent interfaces.

Currently, there are 12 cgroup v1 controllers and 3 cgroup v2 controllers (memory, I/O, and PIDs) and there are other v2 controllers that are a work in progress.

In order to use the cgroups filesystem (i.e., browse it, attach tasks to cgroups, and so on), it first must be mounted, like when working with any other filesystem. The cgroup filesystem can be mounted on any path on the filesystem, and many userspace applications and container projects use `/sys/fs/cgroup` as a mounting point. After mounting the cgroups filesystem, you can create subgroups, attach processes and tasks to these groups, set limitations on various system resources, and more. The cgroup v1 implementation will probably coexist with the cgroup v2 implementation as long as there are userspace projects that use it; we have a parallel phenomenon in other kernel subsystems, when a new implementation of existing subsystem replaces the current one; for example, currently both iptables and the new nftables coexist, and in the past, iptables coexisted with ipchains.

## 4.7 ANDROID PROCESS AND THREAD MANAGEMENT

Before discussing the details of the Android approach to process and thread management, we need to describe the Android concepts of applications and activities.

### Android Applications

An Android application is the software that implements an app. Each Android application consists of one or more instance of one or more of four types of application components. Each component performs a distinct role in the overall application behavior, and each component can be activated independently within the application and even by other applications. The following are the four types of components:

1. **Activities:** An activity corresponds to a single screen visible as a user interface. For example, an e-mail application might have one activity that shows a list of new e-mails, another activity to compose an e-mail, and another activity for reading e-mails. Although the activities work together to form a cohesive user experience in the e-mail application, each one is independent of the others. Android makes a distinction between internal and exported activities. Other apps may start exported activities, which generally include the main screen of the app. However, other apps cannot start the internal activities. For example, a camera application can start the activity in the e-mail application that composes new mail, in order for the user to share a picture.
2. **Services:** Services are typically used to perform background operations that take a considerable amount of time to finish. This ensures faster responsiveness, for the main thread (a.k.a. UI thread) of an application, with which the user is directly interacting. For example, a service might create a thread to play music in the background while the user is in a different application, or it might create a thread to fetch data over the network without blocking user interaction with an activity. A service may be invoked by an application. Additionally, there are system services that run for the entire lifetime of the Android system, such as Power Manager, Battery, and Vibrator services. These system services create threads that are part of the System Server process.
3. **Content providers:** A content provider acts as an interface to application data that can be used by the application. One category of managed data is private

data, which is used only by the application containing the content provider. For example the NotePad application uses a content provider to save notes. The other category is shared data, accessible by multiple applications. This category includes data stored in file systems, an SQLite database, on the Web, or any other persistent storage location your application can access.

4. **Broadcast receivers:** A broadcast receiver responds to system-wide broadcast announcements. A broadcast can originate from another application, such as to let other applications know that some data has been downloaded to the device and is available for them to use, or from the system (for example, a low-battery warning).

Each application runs on its own dedicated virtual machine and its own single process that encompasses the application and its virtual machine (see Figure 4.16). This approach, referred to as the sandboxing model, isolates each application. Thus, one application cannot access the resources of the other without permission being granted. Each application is treated as a separate Linux user with its own unique user ID, which is used to set file permissions.

## Activities

An Activity is an application component that provides a screen with which users can interact in order to do something, such as make a phone call, take a photo, send an e-mail, or view a map. Each activity is given a window in which to draw its user interface. The window typically fills the screen, but may be smaller than the screen and float on top of other windows.

As was mentioned, an application may include multiple activities. When an application is running, one activity is in the foreground, and it is this activity that

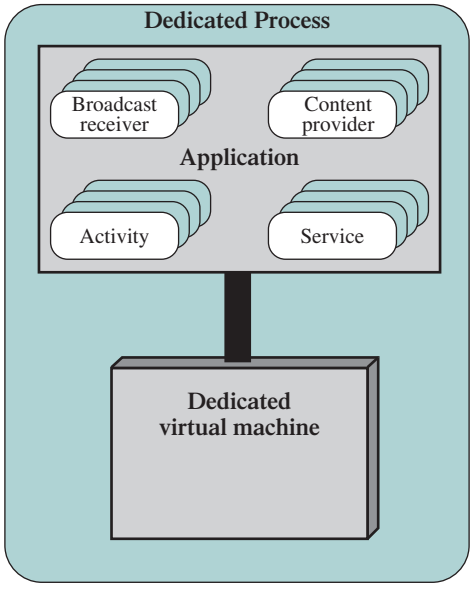



**Figure 4.16 Android Application**

interacts with the user. The activities are arranged in a last-in-first-out stack (the *back stack*), in the order in which each activity is opened. If the user switches to some other activity within the application, the new activity is created and pushed on to the top of the back stack, while the preceding foreground activity becomes the second item on the stack for this application. This process can be repeated multiple times, adding to the stack. The user can back up to the most recent foreground activity by pressing a Back button or similar interface feature.

***Activity States*** Figure 4.17 provides a simplified view of the state transition diagram of an activity. Keep in mind there may be multiple activities in the application, each one at its own particular point on the state transition diagram. When a new activity is launched, the application software performs a series of API calls to the

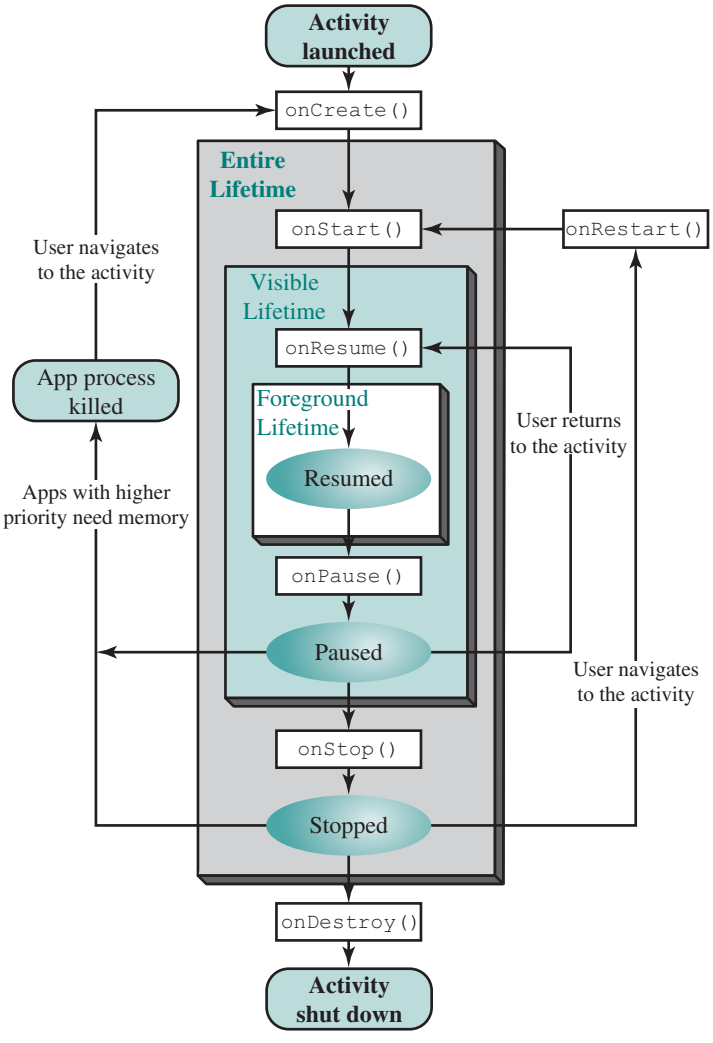



**Figure 4.17 Activity State Transition Diagram**

Activity Manager (Figure 2.20): `onCreate()` does the static setup of the activity, including any data structure initialization; `onStart()` makes the activity visible to the user on the screen; `onResume()` passes control to the activity so user input goes to the activity. At this point the activity is in the Resumed state. This is referred to as the *foreground lifetime* of the activity. During this time, the activity is in front of all other activities on screen and has user input focus.

A user action may invoke another activity within the application. For example, during the execution of the e-mail application, when the user selects an e-mail, a new activity opens to view that e-mail. The system responds to such an activity with the onPause() system call, which places the currently running activity on the stack, putting it in the Paused state. The application then creates a new activity, which will enter the Resumed state.

At any time, a user may terminate the currently running activity by means of the Back button, closing a window, or some other action relevant to this activity. The application then invokes `onStop(0)` to stop the activity. The application then pops the activity that is on the top of the stack and resumes it. The Resumed and Paused states together constitute the *visible lifetime* of the activity. During this time, the user can see the activity on-screen and interact with it.

If the user leaves one application to go to another, for example, by going to the Home screen, the currently running activity is paused and then stopped. When the user resumes this application, the stopped activity, which is on top of the back stack, is restarted and becomes the foreground activity for the application.

***KILLING AN APPLICATION*** If too many things are going on, the system may need to recover some of main memory to maintain responsiveness. In that case, the system will reclaim memory by killing one or more activities within an application and also terminating the process for that application. This frees up memory used to manage the process as well as memory to manage the activities that were killed. However, the application itself still exists. The user is unaware of its altered status. If the user returns to that application, it is necessary for the system to recreate any killed activities as they are invoked.

The system kills applications in a stack-oriented style: So it will kill least recently used apps first. Apps with foregrounded services are extremely unlikely to be killed.

## Processes and Threads

The default allocation of processes and threads to an application is a single process and a single thread. All of the components of the application run on the single thread of the single process for that application. To avoid slowing down the user interface when slow and/or blocking operations occur in a component, the developer can create multiple threads within a process and/or multiple processes within an application. In any case, all processes and their threads for a given application execute within the same virtual machine.

In order to reclaim memory in a system that is becoming heavily loaded, the system may kill one or more processes. As was discussed in the preceding section, when a process is killed, one or more of the activities supported by that process are also killed. A precedence hierarchy is used to determine which process or processes to kill in order

to reclaim needed resources. Every process exists at a particular level of the hierarchy at any given time, and processes are killed beginning with the lowest precedence first. The levels of the hierarchy, in descending order of precedence, are as follows:

- **Foreground process:** A process that is required for what the user is currently doing. More than one process at a time can be a foreground process. For example, both the process that hosts the activity with which the user is interacting (activity in Resumed state), and the process that hosts a service that is bound to the activity with which the user is interacting, are foreground processes.
- **Visible process:** A process that hosts a component that is not in the foreground, but still visible to the user.
- **Service process:** A process running a service that does not fall into either of the higher categories. Examples include playing music in the background or downloading data on the network.
- **Background process:** A process hosting an activity in the Stopped state.
- **Empty process:** A process that doesn't hold any active application components. The only reason to keep this kind of process alive is for caching purposes, to improve startup time the next time a component needs to run in it.

## 4.8 MAC OS X GRAND CENTRAL DISPATCH

As was mentioned in Chapter 2, Mac OS X Grand Central Dispatch (GCD) provides a pool of available threads. Designers can designate portions of applications, called blocks, that can be dispatched independently and run concurrently. The OS will provide as much concurrency as possible based on the number of cores available and the thread capacity of the system. Although other operating systems have implemented thread pools, GCD provides a qualitative improvement in ease of use and efficiency [LEVI16].

A block is a simple extension to C or other languages, such as C++. The purpose of defining a block is to define a self-contained unit of work, including code plus data. Here is a simple example of a block definition:

```
x = ^{printf("hello world\n");}
```

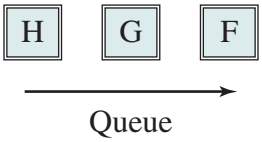
A block is denoted by a caret at the start of the function, which is enclosed in curly brackets. The above block definition defines `x` as a way of calling the function, so that invoking the function `x()` would print the words *hello world.*

Blocks enable the programmer to encapsulate complex functions, together with their arguments and data, so that they can easily be referenced and passed around in a program, much like a variable. Symbolically:

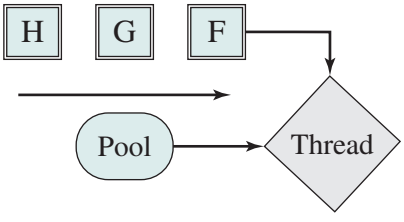
$$\boxed{F} = F + \text{data}$$

Blocks are scheduled and dispatched by means of queues. The application makes use of system queues provided by GCD and may also set up private queues. Blocks are put onto a queue as they are encountered during program execution. GCD then uses those queues to describe concurrency, serialization, and callbacks. Queues are lightweight user-space data structures, which generally makes them far

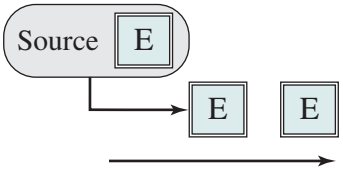
more efficient than manually managing threads and locks. For example, this queue has three blocks:




Depending on the queue and how it is defined, GCD treats these blocks either as potentially concurrent activities, or as serial activities. In either case, blocks are dispatched on a first-in-first-out basis. If this is a concurrent queue, then the dispatcher assigns F to a thread as soon as one is available, then G, then H. If this is a serial queue, the dispatcher assigns F to a thread, then only assigns G to a thread after F has completed. The use of predefined threads saves the cost of creating a new thread for each request, reducing the latency associated with processing a block. Thread pools are automatically sized by the system to maximize the performance of the applications using GCD while minimizing the number of idle or competing threads.




In addition to scheduling blocks directly, the application can associate a single block and queue with an event source, such as a timer, network socket, or file descriptor. Every time the source issues an event, the block is scheduled if it is not already running. This allows rapid response without the expense of polling or "parking a thread" on the event source.




An example from [SIRA09] indicates the ease of using GCD. Consider a document-based application with a button that, when clicked, will analyze the current document and display some interesting statistics about it. In the common case, this analysis should execute in under a second, so the following code is used to connect the button with an action:

```
- (Inaction)analyzeDocument:(NSButton *)sender
{
   NSDictionary *stats = [myDoc analyze];
   [myModel setDict:stats];
```

```
    [myStatsView setNeedsDisplay:YES];
    [stats release];
}
```

The first line of the function body analyzes the document, the second line updates the application's internal state, and the third line tells the application that the statistics view needs to be updated to reflect this new state. This code, which follows a common pattern, is executed in the main thread. The design is acceptable so long as the analysis does not take too long, because after the user clicks the button, the main thread of the application needs to handle that user input as fast as possible so it can get back to the main event loop to process the next user action. But if the user opens a very large or complex document, the analyze step may take an unacceptably long amount of time. A developer may be reluctant to alter the code to meet this unlikely event, which may involve application-global objects, thread management, callbacks, argument marshalling, context objects, new variables, and so on. But with GCD, a modest addition to the code produces the desired result:

```
- (IBAction)analyzeDocument:(NSButton *)sender
    {dispatch_async(dispatch_get_global_queue(0, 0), ^{
       NSDictionary *stats = [myDoc analyze];
       dispatch_async(dispatch_get_main_queue(), ^{
           [myModel setDict:stats];
           [myStatsView setNeedsDisplay:YES];
           [stats release];
         });
     });
}
```

All functions in GCD begin with `dispatch_`. The outer `dispatch_async()` call puts a task on a global concurrent queue. This tells the OS that the block can be assigned to a separate concurrent queue, off the main queue, and executed in parallel. Therefore, the main thread of execution is not delayed. When the analyze function is complete, the inner `dispatch_async()` call is encountered. This directs the OS to put the following block of code at the end of the main queue, to be executed when it reaches the head of the queue. So, with very little work on the part of the programmer, the desired requirement is met.

## 4.9 SUMMARY

Some operating systems distinguish the concepts of process and thread, the former related to resource ownership, and the latter related to program execution. This approach may lead to improved efficiency and coding convenience. In a multithreaded system, multiple concurrent threads may be defined within a single process. This may be done using either user-level threads or kernel-level threads. User-level

threads are unknown to the OS and are created and managed by a threads library that runs in the user space of a process. User-level threads are very efficient because a mode switch is not required to switch from one thread to another. However, only a single user-level thread within a process can execute at a time, and if one thread blocks, the entire process is blocked. Kernel-level threads are threads within a process that are maintained by the kernel. Because they are recognized by the kernel, multiple threads within the same process can execute in parallel on a multiprocessor and the blocking of a thread does not block the entire process. However, a mode switch is required to switch from one thread to another.

## 4.10 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

### Key Terms

| | | |
|---|---|---|
| application | message | thread |
| fiber | multithreading | thread pool |
| jacketing | namespaces | user-level thread |
| job object | port | user-mode scheduling (UMS) |
| kernel-level thread | process | |
| lightweight process | task | |

### Review Questions

**4.1.** Table 3.5 lists typical elements found in a process control block for an unthreaded OS. Of these, which should belong to a thread control block, and which should belong to a process control block for a multithreaded system?

**4.2.** List reasons why a mode switch between threads may be cheaper than a mode switch between processes.

**4.3.** What are the two separate and potentially independent characteristics embodied in the concept of process?

**4.4.** Give four general examples of the use of threads in a single-user multiprocessing system.

**4.5.** How is a thread different from a process?

**4.6.** What are the advantages of using multithreading instead of multiple processes?

**4.7.** List some advantages and disadvantages of using kernel-level threads.

**4.8.** Explain the concept of threads in the case of the Clouds operating system.

### Problems

**4.1.** The use of multithreading improves the overall efficiency and performance of the execution of an application or program. However, not all programs are suitable for multithreading. Can you give some examples of programs where a multithreaded solution fails to improve on the performance of a single-threaded solution? Also give some examples where the performance improves when multiple threads are used in place of single threads.

**4.2.** Suppose a program has a main routine that calls two sub-routines. The sub-routines can be executed in parallel. Give two possible approaches to implement this program, one using threads and the other without.

**4.3.** OS/2 from IBM is an obsolete OS for PCs. In OS/2, what is commonly embodied in the concept of process in other operating systems is split into three separate types of entities: session, processes, and threads. A session is a collection of one or more processes associated with a user interface (keyboard, display, and mouse). The session represents an interactive user application, such as a word processing program or a spreadsheet. This concept allows the personal computer user to open more than one application, giving each one or more windows on the screen. The OS must keep track of which window, and therefore which session, is active, so that keyboard and mouse input are routed to the appropriate session. At any time, one session is in foreground mode, with other sessions in background mode. All keyboard and mouse input is directed to one of the processes of the foreground session, as dictated by the applications. When a session is in foreground mode, a process performing video output sends it directly to the hardware video buffer and then to the user's display. When the session is moved to the background, the hardware video buffer is saved to a logical video buffer for that session. While a session is in background, if any of the threads of any of the processes of that session executes and produces screen output, that output is directed to the logical video buffer. When the session returns to foreground, the screen is updated to reflect the current contents of the logical video buffer for the new foreground session.

There is a way to reduce the number of process-related concepts in OS/2 from three to two. Eliminate sessions, and associate the user interface (keyboard, mouse, and display) with processes. Thus, one process at a time is in foreground mode. For further structuring, processes can be broken up into threads.

**a.** What benefits are lost with this approach?

**b.** If you go ahead with this modification, where do you assign resources (memory, files, etc.): at the process or thread level?

**4.4.** Consider an environment in which there is a one-to-one mapping between user-level threads and kernel-level threads that allows one or more threads within a process to issue blocking system calls while other threads continue to run. Explain why this model can make multithreaded programs run faster than their single-threaded counterparts on a uniprocessor computer.

**4.5.** An application has 20% of code that is inherently serial. Theoretically, what will its maximum speedup be if it is run on a multicore system with four processors?

**4.6.** The OS/390 mainframe operating system is structured around the concepts of address space and task. Roughly speaking, a single address space corresponds to a single application and corresponds more or less to a process in other operating systems. Within an address space, a number of tasks may be generated and executed concurrently; this corresponds roughly to the concept of multithreading. Two data structures are key to managing this task structure. An address space control block (ASCB) contains information about an address space needed by OS/390 whether or not that address space is executing. Information in the ASCB includes dispatching priority, real and virtual memory allocated to this address space, the number of ready tasks in this address space, and whether each is swapped out. A task control block (TCB) represents a user program in execution. It contains information needed for managing a task within an address space, including processor status information, pointers to programs that are part of this task, and task execution state. ASCBs are global structures maintained in system memory, while TCBs are local structures maintained within their address space. What is the advantage of splitting the control information into global and local portions?

**4.7.** Many current language specifications, such as for C and C++, are inadequate for multithreaded programs. This can have an impact on compilers and the correctness

of code, as this problem illustrates. Consider the following declarations and function definition:

```
int global_positives = 0;
typedef struct list {
   struct list *next;
   double val;
} * list;
void count_positives(list l)
{
   list p;
   for (p = l; p; p = p -> next)
      if (p -> val > 0.0)
         ++global_positives;
}
```

Now consider the case in which thread A performs

```
count_positives(<list containing only negative values>);
```

while thread B performs

```
++global_positives;
```

**a.** What does the function do?
**b.** The C language only addresses single-threaded execution. Does the use of two parallel threads create any problems or potential problems?

**4.8.** But some existing optimizing compilers (including gcc, which tends to be relatively conservative) will "optimize" `count_positives` to something similar to

```
void count_positives(list l)
{
     list p;
     register int r;
r = global_positives;
   for (p = l; p; p = p -> next)
      if (p -> val > 0.0) ++r;
   global_positives = r;
}
```

What problem or potential problem occurs with this compiled version of the program if threads A and B are executed concurrently?

**4.9.** Consider the following code using the POSIX Pthreads API:

**thread2.c**

```
#include <pthread.h>
#include <stdlib.h>
#include <unistd.h>
#include <stdio.h>
int myglobal;
    void *thread_function(void *arg) {
       int i,j;
```

```
        for ( i=0; i<20; i++ ) {
           j=myglobal;
           j=j+1;
           printf(".");
           fflush(stdout);
           sleep(1);
           myglobal=j;
        }
        return NULL;
}

int main(void) {
   pthread_t mythread;
   int i;
   if ( pthread_create( &mythread, NULL, thread_function,
       NULL) ) {
      printf(ldquo;error creating thread.");
      abort();
   }
for ( i=0; i<20; i++) {
   myglobal=myglobal+1;
   printf("o");
   fflush(stdout);
   sleep(1);
}
if ( pthread_join ( mythread, NULL ) ) {
   printf("error joining thread.");
abort();
}
printf("\nmyglobal equals %d\n",myglobal);
exit(0);
}
```

In `main()` we first declare a variable called mythread, which has a type of `pthread_t`. This is essentially an ID for a thread. Next, the `if` statement creates a thread associated with `mythread`. The call `pthread_create()` returns zero on success and a nonzero value on failure. The third argument of `pthread_create()` is the name of a function that the new thread will execute when it starts. When this `thread_function()` returns, the thread terminates. Meanwhile, the main program itself defines a thread, so there are two threads executing. The `pthread_join` function enables the main thread to wait until the new thread completes.

**a.** What does this program accomplish?

**b.** Here is the output from the executed program:

```
$ ./thread2
..o.o.o.o.oo.o.o.o.o.o.o.o.o.o..o.o.o.o.o
myglobal equals 21
```

Is this the output you would expect? If not, what has gone wrong?

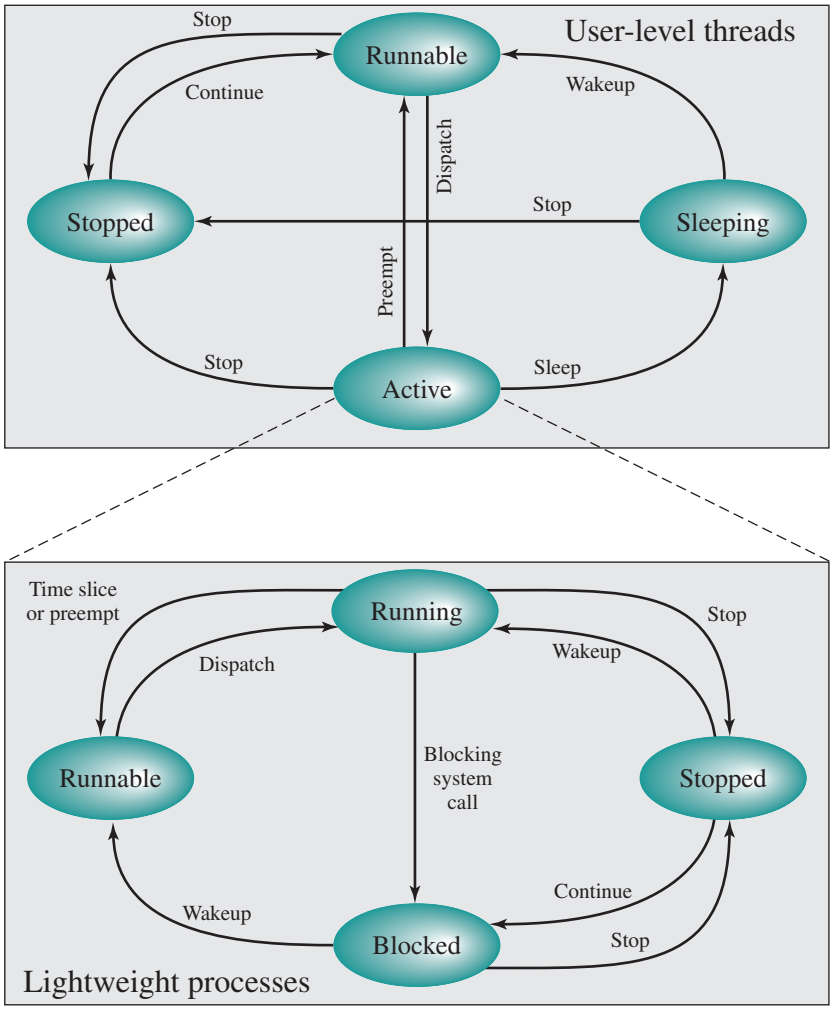



**Figure 4.18** **Solaris User-Level Thread and LWP States**

**4.10.** It is sometimes required that when two threads are running, one thread should automatically preempt the other. The preempted thread can execute only when the other has run to completion. Implement the stated situation by setting priorities for the threads; use any programming language of your choice.

**4.11.** In Solaris 9 and Solaris 10, there is a one-to-one mapping between ULTs and LWPs. In Solaris 8, a single LWP supports one or more ULTs.

- **a.** What is the possible benefit of allowing a many-to-one mapping of ULTs to LWPs?
- **b.** In Solaris 8, the thread execution state of a ULT is distinct from that of its LWP. Explain why.
- **c.** Figure 4.18 shows the state transition diagrams for a ULT and its associated LWP in Solaris 8 and 9. Explain the operation of the two diagrams and their relationships.

**4.12.** Explain the rationale for the Uninterruptible state in Linux.

CHAPTER 5

# Concurrency: Mutual Exclusion and Synchronization


5.1 **Mutual Exclusion: Software Approaches**
- Dekker's Algorithm
- Peterson's Algorithm

5.2 **Principles of Concurrency**
- A Simple Example
- Race Condition
- Operating System Concerns
- Process Interaction
- Requirements for Mutual Exclusion

5.3 **Mutual Exclusion: Hardware Support**
- Interrupt Disabling
- Special Machine Instructions

5.4 **Semaphores**
- Mutual Exclusion
- The Producer/Consumer Problem
- Implementation of Semaphores

5.5 **Monitors**
- Monitor with Signal
- Alternate Model of Monitors with Notify and Broadcast

5.6 **Message Passing**
- Synchronization
- Addressing
- Message Format
- Queueing Discipline
- Mutual Exclusion

5.7 **Readers/Writers Problem**
- Readers Have Priority
- Writers Have Priority

5.8 **Summary**

5.9 **Key Terms, Review Questions, and Problems**

### Learning Objectives

After studying this chapter, you should be able to:

- Discuss basic concepts related to concurrency, such as race conditions, OS concerns, and mutual exclusion requirements.
- Understand hardware approaches to supporting mutual exclusion.
- Define and explain semaphores.
- Define and explain monitors.
- Explain the readers/writers problem.

The central themes of operating system design are all concerned with the management of processes and threads:

- **Multiprogramming:** The management of multiple processes within a uniprocessor system
- **Multiprocessing:** The management of multiple processes within a multiprocessor
- **Distributed processing:** The management of multiple processes executing on multiple, distributed computer systems. The recent proliferation of clusters is a prime example of this type of system.

Fundamental to all of these areas, and fundamental to OS design, is **concurrency**. Concurrency encompasses a host of design issues, including communication among processes, sharing of and competing for resources (such as memory, files, and I/O access), synchronization of the activities of multiple processes, and allocation of processor time to processes. We shall see that these issues arise not just in multiprocessing and distributed processing environments, but also in single-processor multiprogramming systems.

Concurrency arises in three different contexts:

1. **Multiple applications:** Multiprogramming was invented to allow processing time to be dynamically shared among a number of active applications.
2. **Structured applications:** As an extension of the principles of modular design and structured programming, some applications can be effectively programmed as a set of **concurrent processes**.
3. **Operating system structure:** The same structuring advantages apply to systems programs, and we have seen that operating systems are themselves often implemented as a set of processes or threads.

Because of the importance of this topic, four chapters and an appendix focus on concurrency-related issues. Chapters 5 and 6 will deal with concurrency in multiprogramming and multiprocessing systems. Chapters 16 and 18 will examine concurrency issues related to distributed processing.

This chapter begins with an introduction to the concept of concurrency and the implications of the execution of multiple concurrent processes.[1] We find that the basic requirement for support of concurrent processes is the ability to enforce mutual exclusion; that is, the ability to exclude all other processes from a course of action while one process is granted that ability. Section 5.2 covers various approaches to achieving mutual exclusion. All of these are software solutions that require the use of a technique known as busy waiting. Next, we will examine some hardware mechanisms that can support mutual exclusion. Then, we will look at solutions that do not involve busy waiting and that can be either supported by the OS or enforced by language compilers. We will examine three approaches: semaphores, monitors, and **message passing**.

Two classic problems in concurrency are used to illustrate the concepts and compare the approaches presented in this chapter. The producer/consumer problem will be introduced in Section 5.4 and used as a running example. The chapter closes with the readers/writers problem.

Our discussion of concurrency will continue in Chapter 6, and we defer a discussion of the concurrency mechanisms of our example systems until the end of that chapter. Appendix A covers additional topics on concurrency. Table 5.1 lists some key terms related to concurrency. A set of animations that illustrate concepts in this chapter is available at the Companion website for this book.

**Table 5.1** Some Key Terms Related to Concurrency

| | |
|---|---|
| **Atomic operation** | A function or action implemented as a sequence of one or more instructions that appears to be indivisible; that is, no other process can see an intermediate state or interrupt the operation. The sequence of instruction is guaranteed to execute as a group, or not execute at all, having no visible effect on system state. Atomicity guarantees isolation from concurrent processes. |
| **Critical section** | A section of code within a process that requires access to shared resources, and that must not be executed while another process is in a corresponding section of code. |
| **Deadlock** | A situation in which two or more processes are unable to proceed because each is waiting for one of the others to do something. |
| **Livelock** | A situation in which two or more processes continuously change their states in response to changes in the other process(es) without doing any useful work. |
| **Mutual exclusion** | The requirement that when one process is in a critical section that accesses shared resources, no other process may be in a critical section that accesses any of those shared resources. |
| **Race condition** | A situation in which multiple threads or processes read and write a shared data item, and the final result depends on the relative timing of their execution. |
| **Starvation** | A situation in which a runnable process is overlooked indefinitely by the scheduler; although it is able to proceed, it is never chosen. |

[1] For simplicity, we generally refer to the concurrent execution of *processes*. In fact, as we have seen in the preceding chapter, in some systems the fundamental unit of concurrency is a thread rather than a process.

## 5.1 MUTUAL EXCLUSION: SOFTWARE APPROACHES

Software approaches can be implemented for concurrent processes that execute on a single-processor or a multiprocessor machine with shared main memory. These approaches usually assume elementary mutual exclusion at the memory access level ([LAMP91], but see Problem 5.3). That is, simultaneous accesses (reading and/or writing) to the same location in main memory are serialized by some sort of memory arbiter, although the order of access granting is not specified ahead of time. Beyond this, no support in the hardware, operating system, or programming language is assumed.

### Dekker's Algorithm

Dijkstra [DIJK65] reported an algorithm for mutual exclusion for two processes, designed by the Dutch mathematician Dekker. Following Dijkstra, we develop the solution in stages. This approach has the advantage of illustrating many of the common bugs encountered in developing concurrent programs.

***First Attempt*** As mentioned earlier, any attempt at mutual exclusion must rely on some fundamental exclusion mechanism in the hardware. The most common of these is the constraint that only one access to a memory location can be made at a time. Using this constraint, we reserve a global memory location labeled `turn`. A process (P0 or P1) wishing to execute its critical section first examines the contents of `turn`. If the value of `turn` is equal to the number of the process, then the process may proceed to its critical section. Otherwise, it is forced to wait. Our waiting process repeatedly reads the value of `turn` until it is allowed to enter its critical section. This procedure is known as **busy waiting**, or **spin waiting**, because the thwarted process can do nothing productive until it gets permission to enter its critical section. Instead, it must linger and periodically check the variable; thus it consumes processor time (busy) while waiting for its chance.

After a process has gained access to its critical section, and after it has completed that section, it must update the value of `turn` to that of the other process.

In formal terms, there is a shared global variable:

```
int  turn = 0;
```

Figure 5.1a shows the program for the two processes. This solution guarantees the mutual exclusion property but has two drawbacks. First, processes must strictly alternate in their use of their critical section; therefore, the pace of execution is dictated by the slower of the two processes. If P0 uses its critical section only once per hour, but P1 would like to use its critical section at a rate of 1,000 times per hour, P1 is forced to adopt the pace of P0. A much more serious problem is that if one process fails, the other process is permanently blocked. This is true whether a process fails in its critical section or outside of it.

The foregoing construction is that of a **coroutine**. Coroutines are designed to be able to pass execution control back and forth between themselves (see Problem

```
   /* PROCESS 0 /*                  /* PROCESS 1 *
.                                  .
.                                  .
while (turn != 0)                  while (turn != 1)
   /* do nothing */ ;                 /* do nothing */;
/* critical section*/;              /* critical section*/;
turn = 1;                          turn = 0;
.                                  .
```

(a) First attempt

```
      /* PROCESS 0 *                /* PROCESS 1 *
   .                                  .
   .                                  .
   while (flag[1])                    while (flag[0])
      /* do nothing */;                  /* do nothing */;
   flag[0] = true;                    flag[1] = true;
   /*critical section*/;              /* critical section*/;
   flag[0] = false;                   flag[1] = false;
   .                                  .
```

(b) Second attempt

```
      /* PROCESS 0 *                /* PROCESS 1 *
   .                                  .
   .                                  .
   flag[0] = true;                    flag[1] = true;
   while (flag[1])                    while (flag[0])
      /* do nothing */;                  /* do nothing */;
   /* critical section*/;              /* critical section*/;
   flag[0] = false;                   flag[1] = false;
   .                                  .
```

(c) Third attempt

```
      /* PROCESS 0 *                /* PROCESS 1 *
   .                                  .
   .                                  .
   flag[0] = true;                    flag[1] = true;
   while (flag[1]) {                  while (flag[0]) {
      flag[0] = false;                   flag[1] = false;
      /*delay */;                        /*delay */;
      flag[0] = true;                    flag[1] = true;
   }                                  }
   /*critical section*/;              /* critical section*/;
   flag[0] = false;                   flag[1] = false;
   .
```

(d) Fourth attempt



VideoNote

**Figure 5.1 Mutual Exclusion Attempts**

5.5). While this is a useful structuring technique for a single process, it is inadequate to support concurrent processing.

***SECOND ATTEMPT*** The flaw in the first attempt is that it stores the name of the process that may enter its critical section, when in fact we need state information about both processes. In effect, each process should have its own key to the critical section so that if one fails, the other can still access its critical section. To meet this requirement a Boolean vector `flag` is defined, with `flag[0]` corresponding to P0 and `flag[1]` corresponding to P1. Each process may examine the other's flag but may not alter it. When a process wishes to enter its critical section, it periodically checks the other's flag until that flag has the value `false`, indicating that the other process is not in its critical section. The checking process immediately sets its own flag to `true` and proceeds to its critical section. When it leaves its critical section, it sets its flag to `false`.

The shared global variable[2] now is

```
enum     boolean (false = 0; true = 1);
boolean  flag[2] = 0, 0
```

Figure 5.1b shows the algorithm. If one process fails outside the critical section, including the flag-setting code, then the other process is not blocked. In fact, the other process can enter its critical section as often as it likes, because the flag of the other process is always `false`. However, if a process fails inside its critical section or after setting its flag to `true` just before entering its critical section, then the other process is permanently blocked.

This solution is, if anything, worse than the first attempt because it does not even guarantee mutual exclusion. Consider the following sequence:

P0 executes the **while** statement and finds `flag[1]` set to `false`

Pl executes the **while** statement and finds `flag[0]` set to `false`

P0 sets `flag[0]` to `true` and enters its critical section

Pl sets `flag[1]` to `true` and enters its critical section

Because both processes are now in their critical sections, the program is incorrect. The problem is that the proposed solution is not independent of relative process execution speeds.

***THIRD ATTEMPT*** Because a process can change its state after the other process has checked it but before the other process can enter its critical section, the second attempt failed. Perhaps we can fix this problem with a simple interchange of two statements, as shown in Figure 5.1c.

As before, if one process fails inside its critical section, including the flag-setting code controlling the critical section, then the other process is blocked, and if a process fails outside its critical section, then the other process is not blocked.

---

[2]The **enum** declaration is used here to declare a data type (`boolean`) and to assign its values.

Next, let us check that mutual exclusion is guaranteed, using the point of view of process P0. Once P0 has set `flag[0]` to `true`, Pl cannot enter its critical section until after P0 has entered and left its critical section. It could be that Pl is already in its critical section when P0 sets its flag. In that case, P0 will be blocked by the **while** statement until Pl has left its critical section. The same reasoning applies from the point of view of Pl.

This guarantees mutual exclusion, but creates yet another problem. If both processes set their flags to `true` before either has executed the **while** statement, then each will think that the other has entered its critical section, causing deadlock.

***FOURTH ATTEMPT*** In the third attempt, a process sets its state without knowing the state of the other process. Deadlock occurs because each process can insist on its right to enter its critical section; there is no opportunity to back off from this position. We can try to fix this in a way that makes each process more deferential: Each process sets its flag to indicate its desire to enter its critical section, but is prepared to reset the flag to defer to the other process, as shown in Figure 5.1d.

This is close to a correct solution, but is still flawed. Mutual exclusion is still guaranteed, using similar reasoning to that followed in the discussion of the third attempt. However, consider the following sequence of events:

P0 sets `flag[0]` to `true`.
Pl sets `flag[1]` to `true`.
P0 checks `flag[1]`.
Pl checks `flag[0]`.
P0 sets `flag[0]` to `false`.
Pl sets `flag[1]` to `false`.
P0 sets `flag[0]` to `true`.
Pl sets `flag[1]` to `true`.

This sequence could be extended indefinitely, and neither process could enter its critical section. Strictly speaking, this is not deadlock, because any alteration in the relative speed of the two processes will break this cycle and allow one to enter the critical section. This condition is referred to as **livelock**. Recall that deadlock occurs when a set of processes wishes to enter their critical sections, but no process can succeed. With livelock, there are possible sequences of executions that succeed, but it is also possible to describe one or more execution sequences in which no process ever enters its critical section.

Although the scenario just described is not likely to be sustained for very long, it is nevertheless a possible scenario. Thus, we reject the fourth attempt.

***A CORRECT SOLUTION*** We need to be able to observe the state of both processes, which is provided by the array variable `flag`. But, as the fourth attempt shows, this is not enough. We must impose an order on the activities of the two processes to avoid the problem of ”mutual courtesy” that we have just observed. The variable `turn`

from the first attempt can be used for this purpose; in this case the variable indicates which process has the right to insist on entering its critical region.

We can describe this solution, referred to as Dekker's algorithm, as follows. When P0 wants to enter its critical section, it sets its flag to `true`. It then checks the flag of Pl. If that is `false`, P0 may immediately enter its critical section. Otherwise, P0 consults `turn`. If P0 finds that `turn = 0`, then it knows that it is its turn to insist and periodically checks Pl's flag. Pl will at some point note that it is its turn to defer and set its to flag `false`, allowing P0 to proceed. After P0 has used its critical section, it sets its flag to `false` to free the critical section, and sets `turn` to l to transfer the right to insist to Pl.

```
boolean flag [2];
int turn;
void P0()
{
      while (true) {
            flag [0] = true;
            while (flag [1]) {
                  if (turn == 1)
                        flag [0] = false;
                        while (turn == 1) /* do nothing */;
                        flag [0] = true;
                  }
            }
            /* critical section */;
            turn = 1;
            flag [0] = false;
            /* remainder */;
      }
}
void P1( )
{
      while (true) {
            flag [1] = true;
            while (flag [0]) {
                  if (turn == 0) {
                        flag [1] = false;
                        while (turn == 0) /* do nothing */;
                        flag [1] = true;
                  }
            }
            /* critical section */;
            turn = 0;
            flag [1] = false;
            /* remainder */;
      }
}
void main ()
{
      flag [0] = false;
      flag [1] = false;
      turn = 1;
      parbegin  (P0, P1);
}
```


VideoNote

**Figure 5.2 Dekker's Algorithm**

Figure 5.2 provides a specification of Dekker's algorithm. The construct **parbegin** (Pl, P2, . . . , P*n*) means the following: suspend the execution of the main program; initiate concurrent execution of procedures Pl, P2, . . . , P*n*; when all of Pl, P2, . . . , P*n* have terminated, resume the main program. A verification of Dekker's algorithm is left as an exercise (see Problem 5.l).

## Peterson's Algorithm

Dekker's algorithm solves the mutual exclusion problem, but with a rather complex program that is difficult to follow and whose correctness is tricky to prove. Peterson [PETE8l] has provided a simple, elegant solution. As before, the global array variable `flag` indicates the position of each process with respect to mutual exclusion, and the global variable `turn` resolves simultaneity conflicts. The algorithm is presented in Figure 5.3.

That mutual exclusion is preserved is easily shown. Consider process P0. Once it has set `flag[0]` to `true`, Pl cannot enter its critical section. If Pl already is in its critical section, then `flag[1] = true` and P0 is blocked from entering its critical section. On the other hand, mutual blocking is prevented. Suppose that P0 is blocked in its **while** loop. This means that `flag[1]` is true and `turn = 1`. P0 can

```
boolean flag [2];
int turn;
void P0()
{
      while (true) {
            flag [0] = true;
            turn = 1;
            while (flag [1] && turn == 1) /* do nothing */;
            /* critical section */;
            flag [0] = false;
            /* remainder */;
      }
}
 void P1()
{
      while (true) {
            flag [1] = true;
            turn = 0;
            while (flag [0] && turn == 0) /* do nothing */;
            /* critical section */;
            flag [1] = false;
            /* remainder */
      }
}
void main()
{
      flag [0] = false;
      flag [1] = false;
      parbegin  (P0, P1);
}
```




**Figure 5.3 Peterson's Algorithm for Two Processes**

enter its critical section when either `flag[1]` becomes `false` or `turn` becomes 0. Now consider three exhaustive cases:

1. Pl has no interest in its critical section. This case is impossible, because it implies `flag[1] = false`.
2. Pl is waiting for its critical section. This case is also impossible, because if `turn = 1`, Pl is able to enter its critical section.
3. Pl is using its critical section repeatedly and therefore monopolizing access to it. This cannot happen, because Pl is obliged to give P0 an opportunity by setting `turn` to 0 before each attempt to enter its critical section.

Thus, we have a simple solution to the mutual exclusion problem for two processes. Furthermore, Peterson's algorithm is easily generalized to the case of processes [HOFR90].

## 5.2 PRINCIPLES OF CONCURRENCY

In a single-processor multiprogramming system, processes are interleaved in time to yield the appearance of simultaneous execution (see Figure 2.12a). Even though actual parallel processing is not achieved, and even though there is a certain amount of overhead involved in switching back and forth between processes, interleaved execution provides major benefits in processing efficiency and in program structuring. In a multiprocessor system, it is possible not only to interleave the execution of multiple processes, but also to overlap them (see Figure 2.12b).

At first glance, it may seem that interleaving and overlapping represent fundamentally different modes of execution and present different problems. In fact, both techniques can be viewed as examples of concurrent processing, and both present the same problems. In the case of a uniprocessor, the problems stem from a basic characteristic of multiprogramming systems: The relative speed of execution of processes cannot be predicted. It depends on the activities of other processes, the way in which the OS handles interrupts, and the scheduling policies of the OS. The following difficulties arise:

1. The sharing of global resources is fraught with peril. For example, if two processes both make use of the same global variable and both perform reads and writes on that variable, then the order in which the various reads and writes are executed is critical. An example of this problem is shown in the following subsection.
2. It is difficult for the OS to optimally manage the allocation of resources. For example, process A may request use of, and be granted control of, a particular I/O channel, then be suspended before using that channel. It may be undesirable for the OS simply to lock the channel and prevent its use by other processes; indeed this may lead to a deadlock condition, as will be described in Chapter 6.
3. It becomes very difficult to locate a programming error because results are typically not deterministic and reproducible (e.g., see [LEBL87, CARR89, SHEN02] for a discussion of this point).

All of the foregoing difficulties present themselves in a multiprocessor system as well, because here too the relative speed of execution of processes is unpredictable. A multiprocessor system must also deal with problems arising from the simultaneous execution of multiple processes. Fundamentally, however, the problems are the same as those for uniprocessor systems. This should become clear as the discussion proceeds.

## A Simple Example

Consider the following procedure:

```
void echo()
{
  chin = getchar();
  chout = chin;
  putchar(chout);
}
```

This procedure shows the essential elements of a program that will provide a character `echo` procedure; input is obtained from a keyboard one keystroke at a time. Each input character is stored in variable `chin`. The character is then transferred to variable `chout` and sent to the display. Any program can call this procedure repeatedly to accept user input and display it on the user's screen.

Now consider that we have a single-processor multiprogramming system supporting a single user. The user can jump from one application to another, and each application uses the same keyboard for input and the same screen for output. Because each application needs to use the procedure `echo`, it makes sense for it to be a shared procedure that is loaded into a portion of memory global to all applications. Thus, only a single copy of the `echo` procedure is used, saving space.

The sharing of main memory among processes is useful to permit efficient and close interaction among processes. However, such sharing can lead to problems. Consider the following sequence:

1. Process P1 invokes the `echo` procedure and is interrupted immediately after `getchar` returns its value and stores it in `chin`. At this point, the most recently entered character, x, is stored in variable `chin`.
2. Process P2 is activated and invokes the `echo` procedure, which runs to conclusion, inputting and then displaying a single character, `y`, on the screen.
3. Process P1 is resumed. By this time, the value `x` has been overwritten in `chin` and therefore lost. Instead, `chin` contains `y`, which is transferred to `chout` and displayed.

Thus, the first character is lost and the second character is displayed twice. The essence of this problem is the shared global variable, `chin`. Multiple processes have access to this variable. If one process updates the global variable and is then interrupted, another process may alter the variable before the first process can use its value. Suppose, however, that we permit only one process at a time to be in that procedure. Then the foregoing sequence would result in the following:

1. Process P1 invokes the `echo` procedure and is interrupted immediately after the conclusion of the input function. At this point, the most recently entered character, `x`, is stored in variable `chin`.
2. Process P2 is activated and invokes the `echo` procedure. However, because P1 is still inside the `echo` procedure, although currently suspended, P2 is blocked from entering the procedure. Therefore, P2 is suspended awaiting the availability of the `echo` procedure.
3. At some later time, process P1 is resumed and completes execution of `echo`. The proper character, `x`, is displayed.
4. When P1 exits `echo`, this removes the block on P2. When P2 is later resumed, the `echo` procedure is successfully invoked.

This example shows that it is necessary to protect shared global variables (and other shared global resources) and the only way to do that is to control the code that accesses the variable. If we impose the discipline that only one process at a time may enter `echo`, and that once in `echo` the procedure must run to completion before it is available for another process, then the type of error just discussed will not occur. How that discipline may be imposed is a major topic of this chapter.

This problem was stated with the assumption that there was a single-processor, multiprogramming OS. The example demonstrates that the problems of concurrency occur even when there is a single processor. In a multiprocessor system, the same problems of protected shared resources arise, and the same solution works. First, suppose there is no mechanism for controlling access to the shared global variable:

1. Processes P1 and P2 are both executing, each on a separate processor. Both processes invoke the `echo` procedure.
2. The following events occur; events on the same line take place in parallel:

| **Process P1** | **Process P2** |
|---|---|
| • | • |
| `chin = getchar();` | • |
| • | `chin = getchar();` |
| `chout = chin;` | `chout = chin;` |
| `putchar(chout);` | • |
| • | `putchar(chout);` |
| • | • |

The result is that the character input to P1 is lost before being displayed, and the character input to P2 is displayed by both P1 and P2. Again, let us add the capability of enforcing the discipline that only one process at a time may be in `echo`. Then the following sequence occurs:

1. Processes P1 and P2 are both executing, each on a separate processor. P1 invokes the `echo` procedure.
2. While P1 is inside the `echo` procedure, P2 invokes `echo`. Because P1 is still inside the `echo` procedure (whether P1 is suspended or executing), P2 is

blocked from entering the procedure. Therefore, P2 is suspended awaiting the availability of the `echo` procedure.

3. At a later time, process P1 completes execution of `echo`, exits that procedure, and continues executing. Immediately upon the exit of P1 from `echo`, P2 is resumed and begins executing `echo`.

In the case of a uniprocessor system, the reason we have a problem is that an interrupt can stop instruction execution anywhere in a process. In the case of a multiprocessor system, we have that same condition and, in addition, a problem can be caused because two processes may be executing simultaneously and both trying to access the same global variable. However, the solution to both types of problem is the same: control access to the shared resource.

## Race Condition

A race condition occurs when multiple processes or threads read and write data items so that the final result depends on the order of execution of instructions in the multiple processes. Let us consider two simple examples.

As a first example, suppose two processes, P1 and P2, share the global variable `a`. At some point in its execution, P1 updates `a` to the value 1, and at some point in its execution, P2 updates `a` to the value 2. Thus, the two tasks are in a race to write variable `a`. In this example, the "loser" of the race (the process that updates last) determines the final value of `a`.

For our second example, consider two processes, P3 and P4, that share global variables `b` and `c`, with initial values `b = 1` and `c = 2`. At some point in its execution, P3 executes the assignment `b = b + c`, and at some point in its execution, P4 executes the assignment `c = b + c`. Note the two processes update different variables. However, the final values of the two variables depend on the order in which the two processes execute these two assignments. If P3 executes its assignment statement first, then the final values are `b = 3` and `c = 5`. If P4 executes its assignment statement first, then the final values are `b = 4` and `c = 3`.

Appendix A includes a discussion of race conditions using semaphores as an example.

## Operating System Concerns

What design and management issues are raised by the existence of concurrency? We can list the following concerns:

1. The OS must be able to keep track of the various processes. This is done with the use of process control blocks and was described in Chapter 4.
2. The OS must allocate and deallocate various resources for each active process. At times, multiple processes want access to the same resource. These resources include
   - **Processor time:** This is the scheduling function, to be discussed in Part Four.
   - **Memory:** Most operating systems use a virtual memory scheme. The topic will be addressed in Part Three.

- **Files:** To be discussed in Chapter 12
- **I/O devices:** To be discussed in Chapter 11

3. The OS must protect the data and physical resources of each process against unintended interference by other processes. This involves techniques that relate to memory, files, and I/O devices. A general treatment of protection found in Part Seven.
4. The functioning of a process, and the output it produces, must be independent of the speed at which its execution is carried out relative to the speed of other concurrent processes. This is the subject of this chapter.

To understand how the issue of speed independence can be addressed, we need to look at the ways in which processes can interact.

## Process Interaction

We can classify the ways in which processes interact on the basis of the degree to which they are aware of each other's existence. Table 5.2 lists three possible degrees of awareness and the consequences of each:

- **Processes unaware of each other:** These are independent processes that are not intended to work together. The best example of this situation is the multiprogramming of multiple independent processes. These can either be batch jobs or interactive sessions or a mixture. Although the processes are not working together, the OS needs to be concerned about **competition** for resources. For example, two independent applications may both want to access the same disk or file or printer. The OS must regulate these accesses.

**Table 5.2** Process Interaction

| Degree of Awareness | Relationship | Influence that One Process Has on the Other | Potential Control Problems |
|---|---|---|---|
| Processes unaware of each other | Competition | • Results of one process independent of the action of others<br>• Timing of process may be affected | • Mutual exclusion<br>• Deadlock (renewable resource)<br>• Starvation |
| Processes indirectly aware of each other (e.g., shared object) | Cooperation by sharing | • Results of one process may depend on information obtained from others<br>• Timing of process may be affected | • Mutual exclusion<br>• Deadlock (renewable resource)<br>• Starvation<br>• Data coherence |
| Processes directly aware of each other (have communication primitives available to them) | Cooperation by communication | • Results of one process may depend on information obtained from others<br>• Timing of process may be affected | • Deadlock (consumable resource)<br>• Starvation |

- **Processes indirectly aware of each other:** These are processes that are not necessarily aware of each other by their respective process IDs but that share access to some object, such as an I/O buffer. Such processes exhibit **cooperation** in sharing the common object.
- **Processes directly aware of each other:** These are processes that are able to communicate with each other by process ID and that are designed to work jointly on some activity. Again, such processes exhibit **cooperation**.

Conditions will not always be as clear-cut as suggested in Table 5.2. Rather, several processes may exhibit aspects of both competition and cooperation. Nevertheless, it is productive to examine each of the three items in the preceding list separately and determine their implications for the OS.

***Competition among Processes for Resources*** Concurrent processes come into conflict with each other when they are competing for the use of the same resource. In its pure form, we can describe the situation as follows. Two or more processes need to access a resource during the course of their execution. Each process is unaware of the existence of other processes, and each is to be unaffected by the execution of the other processes. It follows from this each process should leave the state of any resource that it uses unaffected. Examples of resources include I/O devices, memory, processor time, and the clock.

There is no exchange of information between the competing processes. However, the execution of one process may affect the behavior of competing processes. In particular, if two processes both wish access to a single resource, then one process will be allocated that resource by the OS, and the other will have to wait. Therefore, the process that is denied access will be slowed down. In an extreme case, the blocked process may never get access to the resource, and hence will never terminate successfully.

In the case of competing processes three control problems must be faced. First is the need for **mutual exclusion**. Suppose two or more processes require access to a single nonsharable resource, such as a printer. During the course of execution, each process will be sending commands to the I/O device, receiving status information, sending data, and/or receiving data. We will refer to such a resource as a **critical resource**, and the portion of the program that uses it as a **critical section** of the program. It is important that only one program at a time be allowed in its critical section. We cannot simply rely on the OS to understand and enforce this restriction because the detailed requirements may not be obvious. In the case of the printer, for example, we want any individual process to have control of the printer while it prints an entire file. Otherwise, lines from competing processes will be interleaved.

The enforcement of mutual exclusion creates two additional control problems. One is that of **deadlock**. For example, consider two processes, P1 and P2, and two resources, R1 and R2. Suppose that each process needs access to both resources to perform part of its function. Then it is possible to have the following situation: the OS assigns R1 to P2, and R2 to P1. Each process is waiting for one of the two resources. Neither will release the resource that it already owns until it has acquired the other resource and performed the function requiring both resources. The two processes are deadlocked.

```
/* PROCESS 1 * void  P1
{
  while  (true) {
   /* preceding code */;
   entercritical (Ra);
   /* critical section */;
   exitcritical (Ra);
   /* following code */;
  }
}
```

```
/* PROCESS 2 * void  P2
{
  while  (true) {
   /* preceding code */;
   entercritical (Ra);
   /* critical section */;
   exitcritical (Ra);
   /* following code */;
  }
}
```

• • •

```
/* PROCESS n * void  Pn
{
  while  (true) {
   /* preceding code */;
   entercritical (Ra);
   /* critical section */;
   exitcritical (Ra);
   /* following code */;
  }
}
```

VideoNote

**Figure 5.4 Illustration of Mutual Exclusion**

A final control problem is **starvation**. Suppose three processes (P1, P2, P3) each require periodic access to resource R. Consider the situation in which P1 is in possession of the resource, and both P2 and P3 are delayed, waiting for that resource. When P1 exits its critical section, either P2 or P3 should be allowed access to R. Assume the OS grants access to P3, and P1 again requires access before P3 completes its critical section. If the OS grants access to P1 after P3 has finished, and subsequently alternately grants access to P1 and P3, then P2 may indefinitely be denied access to the resource, even though there is no deadlock situation.

Control of competition inevitably involves the OS because the OS allocates resources. In addition, the processes themselves will need to be able to express the requirement for mutual exclusion in some fashion, such as locking a resource prior to its use. Any solution will involve some support from the OS, such as the provision of the locking facility. Figure 5.4 illustrates the mutual exclusion mechanism in abstract terms. There are *n* processes to be executed concurrently. Each process includes (1) a critical section that operates on some resource Ra, and (2) additional code preceding and following the critical section that does not involve access to Ra. Because all processes access the same resource Ra, it is desired that only one process at a time be in its critical section. To enforce mutual exclusion, two functions are provided: `entercritical` and `exitcritical`. Each function takes as an argument the name of the resource that is the subject of competition. Any process that attempts to enter its critical section while another process is in its critical section, for the same resource, is made to wait.

It remains to examine specific mechanisms for providing the functions `entercritical` and `exitcritical`. For the moment, we defer this issue while we consider the other cases of process interaction.

***Cooperation among Processes by Sharing*** The case of cooperation by sharing covers processes that interact with other processes without being explicitly aware of them. For example, multiple processes may have access to shared variables or to shared files or databases. Processes may use and update the shared data without reference to other processes, but know that other processes may have access to the same data. Thus the processes must cooperate to ensure that the data they share are properly managed. The control mechanisms must ensure the integrity of the shared data.

Because data are held on resources (devices, memory), the control problems of mutual exclusion, deadlock, and starvation are again present. The only difference is that data items may be accessed in two different modes, reading and writing, and only writing operations must be mutually exclusive.

However, over and above these problems, a new requirement is introduced: that of data coherence. As a simple example, consider a bookkeeping application in which various data items may be updated. Suppose two items of data $a$ and $b$ are to be maintained in the relationship $a = b$. That is, any program that updates one value must also update the other to maintain the relationship. Now consider the following two processes:

P1:
$a = a + 1;$
$b = b + 1;$

P2:
$b = 2 * b;$
$a = 2 * a;$

If the state is initially consistent, each process taken separately will leave the shared data in a consistent state. Now consider the following concurrent execution sequence, in which the two processes respect mutual exclusion on each individual data item ($a$ and $b$):

$a = a + 1;$
$b = 2 * b;$
$b = b + 1;$
$a = 2 * a;$

At the end of this execution sequence, the condition $a = b$ no longer holds. For example, if we start with $a = b = 1$, at the end of this execution sequence we have $a = 4$ and $b = 3$. The problem can be avoided by declaring the entire sequence in each process to be a critical section.

Thus, we see that the concept of critical section is important in the case of cooperation by sharing. The same abstract functions of `entercritical` and `exitcritical` discussed earlier (see Figure 5.4) can be used here. In this case, the argument for the functions could be a variable, a file, or any other shared object. Furthermore, if critical sections are used to provide data integrity, then there may be no specific resource or variable that can be identified as an argument. In that case, we can think of the argument as being an identifier that is shared among concurrent processes to identify critical sections that must be mutually exclusive.

***Cooperation among Processes by Communication*** In the first two cases that we have discussed, each process has its own isolated environment that does not include the other processes. The interactions among processes are indirect. In both cases, there is a sharing. In the case of competition, they are sharing resources without being aware of the other processes. In the second case, they are sharing values, and although each process is not explicitly aware of the other processes, it is aware of the need to maintain data integrity. When processes cooperate by communication, however, the various processes participate in a common effort that links all of the processes. The communication provides a way to synchronize, or coordinate, the various activities.

Typically, communication can be characterized as consisting of messages of some sort. Primitives for sending and receiving messages may be provided as part of the programming language or provided by the OS kernel.

Because nothing is shared between processes in the act of passing messages, mutual exclusion is not a control requirement for this sort of cooperation. However, the problems of deadlock and starvation are still present. As an example of deadlock, two processes may be blocked, each waiting for a communication from the other. As an example of starvation, consider three processes, P1, P2, and P3, that exhibit the following behavior. P1 is repeatedly attempting to communicate with either P2 or P3, and P2 and P3 are both attempting to communicate with P1. A sequence could arise in which P1 and P2 exchange information repeatedly, while P3 is blocked waiting for a communication from P1. There is no deadlock, because P1 remains active, but P3 is starved.

## Requirements for Mutual Exclusion

Any facility or capability that is to provide support for mutual exclusion should meet the following requirements:

1. Mutual exclusion must be enforced: Only one process at a time is allowed into its critical section, among all processes that have critical sections for the same resource or shared object.
2. A process that halts in its noncritical section must do so without interfering with other processes.
3. It must not be possible for a process requiring access to a critical section to be delayed indefinitely: no deadlock or starvation.
4. When no process is in a critical section, any process that requests entry to its critical section must be permitted to enter without delay.
5. No assumptions are made about relative process speeds or number of processors.
6. A process remains inside its critical section for a finite time only.

There are a number of ways in which the requirements for mutual exclusion can be satisfied. One approach is to leave the responsibility with the processes that wish to execute concurrently. Processes, whether they are system programs or application programs, would be required to coordinate with one another to enforce mutual exclusion, with no support from the programming language or the OS. We can refer to these as software approaches. Although this approach is prone to high processing overhead and bugs, it is nevertheless useful to examine such approaches to gain a better understanding of the complexity of concurrent processing. This topic was covered in the preceding section. A second approach involves the use of special-purpose machine instructions. These have the advantage of reducing overhead but nevertheless will be shown to be unattractive as a general-purpose solution; they will be covered in Section 5.3. A third approach is to provide some level of support within the OS or a programming language. Three of the most important such approaches will be examined in Sections 5.4 through 5.6.

## 5.3 MUTUAL EXCLUSION: HARDWARE SUPPORT

In this section, we look at several interesting hardware approaches to mutual exclusion.

### Interrupt Disabling

In a uniprocessor system, concurrent processes cannot have overlapped execution; they can only be interleaved. Furthermore, a process will continue to run until it invokes an OS service or until it is interrupted. Therefore, to guarantee mutual exclusion, it is sufficient to prevent a process from being interrupted. This capability can be provided in the form of primitives defined by the OS kernel for disabling and enabling interrupts. A process can then enforce mutual exclusion in the following way (compare to Figure 5.4):

```
while (true) {
    /* disable interrupts */;
    /* critical section */;
    /* enable interrupts */;
    /* remainder */;
}
```

Because the critical section cannot be interrupted, mutual exclusion is guaranteed. The price of this approach, however, is high. The efficiency of execution could be noticeably degraded because the processor is limited in its ability to interleave processes. Another problem is that this approach will not work in a multiprocessor architecture. When the computer includes more than one processor, it is possible (and typical) for more than one process to be executing at a time. In this case, disabled interrupts do not guarantee mutual exclusion.

### Special Machine Instructions

In a multiprocessor configuration, several processors share access to a common main memory. In this case, there is not a master/slave relationship; rather the processors behave independently in a peer relationship. There is no interrupt mechanism between processors on which mutual exclusion can be based.

At the hardware level, as was mentioned, access to a memory location excludes any other access to that same location. With this as a foundation, processor designers have proposed several machine instructions that carry out two actions atomically,[3] such as reading and writing or reading and testing, of a single memory location with one instruction fetch cycle. During execution of the instruction, access to the memory location is blocked for any other instruction referencing that location.

In this section, we look at two of the most commonly implemented instructions. Others are described in [RAYN86] and [STON93].

[3]The term *atomic* means the instruction is treated as a single step that cannot be interrupted.

***COMPARE&SWAP INSTRUCTION*** The `compare&swap` instruction, also called a compare and exchange instruction, can be defined as follows [HERL90]:

```
int compare_and_swap (int *word, int testval, int newval)
{
    int oldval;
    oldval = *word
    if (oldval == testval) *word = newval;
    return oldval;
}
```

This version of the instruction checks a memory location (`*word`) against a test value (`testval`). If the memory location's current value is `testval`, it is replaced with `newval`; otherwise, it is left unchanged. The old memory value is always returned; thus, the memory location has been updated if the returned value is the same as the test value. This atomic instruction therefore has two parts: A **compare** is made between a memory value and a test value; if the values are the same, a **swap** occurs. The entire `compare&swap` function is carried out atomically—that is, it is not subject to interruption.

Another version of this instruction returns a Boolean value: true if the swap occurred; false otherwise. Some version of this instruction is available on nearly all processor families (x86, IA64, sparc, IBM z series, etc.), and most operating systems use this instruction for support of concurrency.

Figure 5.5a shows a mutual exclusion protocol based on the use of this instruction.[4] A shared variable `bolt` is initialized to 0. The only process that may enter its

```
/* program  mutualexclusion */
const int n = /* number of processes */;
int bolt;
void P(int  i)
{
   while (true)  {
     while (compare_and_swap(bolt, 0, 1) == 1)
          /* do nothing */;
     /* critical section */;
     bolt = 0;
     /* remainder */;
   }
}
void main() {
   bolt = 0;
   parbegin (P(1), P(2), ... ,P(n));
}
```

(a) Compare and swap instruction

```
/* program mutualexclusion */
int const n = /* number of processes */;
int bolt;
void P(int  i)
{
   while (true)
      int  keyi = 1;
      do  exchange (&keyi, &bolt)
      while  (keyi != 0);
      /* critical section */;
      bolt = 0;
      /* remainder */;
   }
}
void main() {
   bolt = 0;
   parbegin  (P(1), P(2), ..., P(n));
}
```

(b) Exchange instruction

VideoNote **Figure 5.5 Hardware Support for Mutual Exclusion**

[4] The construct **parbegin** `(P1, P2, . . . , Pn)` means the following: suspend the execution of the main program; initiate concurrent execution of procedures P1, P2, . . . , P*n*; when all of P1, P2, . . . , P*n* have terminated, resume the main program.

critical section is one that finds `bolt` equal to 0. All other processes attempting to enter their critical section go into a busy waiting mode. The term **busy waiting,** or **spin waiting,** refers to a technique in which a process can do nothing until it gets permission to enter its critical section, but continues to execute an instruction or set of instructions that tests the appropriate variable to gain entrance. When a process leaves its critical section, it resets *bolt* to 0; at this point one and only one of the waiting processes is granted access to its critical section. The choice of process depends on which process happens to execute the `compare&swap` instruction next.

***EXCHANGE INSTRUCTION*** The exchange instruction can be defined as follows:

```
void exchange (int *register,  int *memory)
{
    int temp;
    temp = *memory;
    *memory = *register;
    *register = temp;
}
```

The instruction exchanges the contents of a register with that of a memory location. Both the Intel IA-32 architecture (Pentium) and the IA-64 architecture (Itanium) contain an XCHG instruction.

Figure 5.5b shows a mutual exclusion protocol based on the use of an exchange instruction. A shared variable *bolt* is initialized to 0. Each process uses a local variable *key* that is initialized to 1. The only process that may enter its critical section is one that finds *bolt* equal to 0. It excludes all other processes from the critical section by setting *bolt* to 1. When a process leaves its critical section, it resets *bolt* to 0, allowing another process to gain access to its critical section.

Note the following expression always holds because of the way in which the variables are initialized and because of the nature of the exchange algorithm:

$$bolt + \sum_i key_i = n$$

If $bolt = 0$, then no process is in its critical section. If $bolt = 1$, then exactly one process is in its critical section, namely the process whose *key* value equals 0.

***PROPERTIES OF THE MACHINE-INSTRUCTION APPROACH*** The use of a special machine instruction to enforce mutual exclusion has a number of advantages:

- It is applicable to any number of processes on either a single processor or multiple processors sharing main memory.
- It is simple and therefore easy to verify.
- It can be used to support multiple critical sections; each critical section can be defined by its own variable.

However, there are some serious disadvantages:

- **Busy waiting is employed:** Thus, while a process is waiting for access to a critical section, it continues to consume processor time.

- **Starvation is possible:** When a process leaves a critical section and more than one process is waiting, the selection of a waiting process is arbitrary. Thus, some process could indefinitely be denied access.
- **Deadlock is possible:** Consider the following scenario on a single-processor system. Process P1 executes the special instruction (e.g., `compare&swap`, `exchange`) and enters its critical section. P1 is then interrupted to give the processor to P2, which has higher priority. If P2 now attempts to use the same resource as P1, it will be denied access because of the mutual exclusion mechanism. Thus, it will go into a busy waiting loop. However, P1 will never be dispatched because it is of lower priority than another ready process, P2.

Because of the drawbacks of both the software and hardware solutions, we need to look for other mechanisms.

## 5.4 SEMAPHORES

We now turn to OS and programming language mechanisms that are used to provide concurrency. Table 5.3 summarizes mechanisms in common use. We begin, in this section, with semaphores. The next two sections will discuss monitors and message passing. The other mechanisms in Table 5.3 will be discussed when treating specific OS examples, in Chapters 6 and 13.

**Table 5.3** Common Concurrency Mechanisms

| | |
|---|---|
| **Semaphore** | An integer value used for signaling among processes. Only three operations may be performed on a semaphore, all of which are atomic: initialize, decrement, and increment. The decrement operation may result in the blocking of a process, and the increment operation may result in the unblocking of a process. Also known as a **counting semaphore** or a **general semaphore**. |
| **Binary semaphore** | A semaphore that takes on only the values 0 and 1. |
| **Mutex** | Similar to a binary semaphore. A key difference between the two is that the process that locks the mutex (sets the value to 0) must be the one to unlock it (sets the value to 1). |
| **Condition variable** | A data type that is used to block a process or thread until a particular condition is true. |
| **Monitor** | A programming language construct that encapsulates variables, access procedures, and initialization code within an abstract data type. The monitor's variable may only be accessed via its access procedures and only one process may be actively accessing the monitor at any one time. The access procedures are *critical sections*. A monitor may have a queue of processes that are waiting to access it. |
| **Event flags** | A memory word used as a synchronization mechanism. Application code may associate a different event with each bit in a flag. A thread can wait for either a single event or a combination of events by checking one or multiple bits in the corresponding flag. The thread is blocked until all of the required bits are set (AND) or until at least one of the bits is set (OR). |
| **Mailboxes/messages** | A means for two processes to exchange information and that may be used for synchronization. |
| **Spinlocks** | Mutual exclusion mechanism in which a process executes in an infinite loop waiting for the value of a lock variable to indicate availability. |

The first major advance in dealing with the problems of concurrent processes came in 1965 with Dijkstra's treatise [DIJK65]. Dijkstra was concerned with the design of an OS as a collection of cooperating sequential processes, and with the development of efficient and reliable mechanisms for supporting cooperation. These mechanisms can just as readily be used by user processes if the processor and OS make the mechanisms available.

The fundamental principle is this: Two or more processes can cooperate by means of simple signals, such that a process can be forced to stop at a specified place until it has received a specific signal. Any complex coordination requirement can be satisfied by the appropriate structure of signals. For signaling, special variables called semaphores are used. To transmit a signal via semaphore `s`, a process executes the primitive `semSignal (s)`. To receive a signal via semaphore `s`, a process executes the primitive `semWait (s)`; if the corresponding signal has not yet been transmitted, the process is suspended until the transmission takes place.[5]

To achieve the desired effect, we can view the semaphore as a variable that has an integer value upon which only three operations are defined:

1. A semaphore may be initialized to a nonnegative integer value.
2. The `semWait` operation decrements the semaphore value. If the value becomes negative, then the process executing the `semWait` is blocked. Otherwise, the process continues execution.
3. The `semSignal` operation increments the semaphore value. If the resulting value is less than or equal to zero, then a process blocked by a `semWait` operation, if any, is unblocked.

Other than these three operations, there is no way to inspect or manipulate semaphores.

We explain these operations as follows. To begin, the semaphore has a zero or positive value. If the value is positive, that value equals the number of processes that can issue a wait and immediately continue to execute. If the value is zero, either by initialization or because a number of processes equal to the initial semaphore value have issued a wait, the next process to issue a wait is blocked, and the semaphore value goes negative. Each subsequent wait drives the semaphore value further into minus territory. The negative value equals the number of processes waiting to be unblocked. Each signal unblocks one of the waiting processes when the semaphore value is negative.

[Subject] points out three interesting consequences of the semaphore definition:

1. In general, there is no way to know before a process decrements a semaphore whether it will block or not.

[5] In Dijkstra's original paper and in much of the literature, the letter P is used for `semWait` and the letter V for `semSignal`; these are the initials of the Dutch words for test (*proberen*) and increment (*verhogen*). In some of the literature, the terms `wait` and `signal` are used. This book uses `semWait` and `semSignal` for clarity, and to avoid confusion with similar wait and signal operations in monitors, discussed subsequently.

```
struct semaphore {
        int count;
        queueType queue;
};
void semWait(semaphore s)
{
        s.count--;
        if (s.count < 0) {
          /* place this process in s.queue */;
          /* block this process */;
        }
}
void semSignal(semaphore s)
{
        s.count++;
        if (s.count<= 0) {
          /* remove a process P from s.queue */;
          /* place process P on ready list */;
        }
}
```



VideoNote **Figure 5.6 A Definition of Semaphore Primitives**

2. After a process increments a semaphore and another process gets woken up, both processes continue running concurrently. There is no way to know which process, if either, will continue immediately on a uniprocessor system.
3. When you signal a semaphore, you don't necessarily know whether another process is waiting, so the number of unblocked processes may be zero or one.

Figure 5.6 suggests a more formal definition of the primitives for semaphores. The `semWait` and `semSignal` primitives are assumed to be atomic. A more restricted version, known as the **binary semaphore**, is defined in Figure 5.7. A binary semaphore may only take on the values 0 and 1, and can be defined by the following three operations:

1. A binary semaphore may be initialized to 0 or 1.
2. The `semWaitB` operation checks the semaphore value. If the value is zero, then the process executing the `semWaitB` is blocked. If the value is one, then the value is changed to zero and the process continues execution.
3. The `semSignalB` operation checks to see if any processes are blocked on this semaphore (semaphore value equals 0). If so, then a process blocked by a `semWaitB` operation is unblocked. If no processes are blocked, then the value of the semaphore is set to one.

In principle, it should be easier to implement the binary semaphore, and it can be shown that it has the same expressive power as the general semaphore (see Problem 5.19). To contrast the two types of semaphores, the nonbinary semaphore is often referred to as either a **counting semaphore** or a **general semaphore**.

```
struct binary_semaphore {
       enum {zero, one} value;
       queueType queue;
};
 void semWaitB(binary_semaphore s)
{
       if (s.value == one)
           s.value = zero;
       else {
                   /* place this process in s.queue */;
                   /* block this process */;
       }
}
 void semSignalB(semaphore s)
{
       if (s.queue is empty())
           s.value = one;
       else {
                   /* remove a process P from s.queue */;
                   /* place process P on ready list */;
       }
}
```



VideoNote **Figure 5.7 A Definition of Binary Semaphore Primitives**

A concept related to the binary semaphore is the **mutual exclusion lock (mutex)**. A mutex is a programming flag used to grab and release an object. When data are acquired that cannot be shared, or processing is started that cannot be performed simultaneously elsewhere in the system, the mutex is set to lock (typically zero), which blocks other attempts to use it. The mutex is set to unlock when the data are no longer needed or the routine is finished. A key difference between the a mutex and a binary semaphore is that the process that locks the mutex (sets the value to zero) must be the one to unlock it (sets the value to 1). In contrast, it is possible for one process to lock a binary semaphore and for another to unlock it.[6]

For both counting semaphores and binary semaphores, a queue is used to hold processes waiting on the semaphore. The question arises of the order in which processes are removed from such a queue. The fairest removal policy is first-in-first-out (FIFO): The process that has been blocked the longest is released from the queue first; a semaphore whose definition includes this policy is called a **strong semaphore**. A semaphore that does not specify the order in which processes are removed from the queue is a **weak semaphore**. Figure 5.8 is an example of the operation of a strong semaphore. Here processes A, B, and C depend on a result from process D. Initially (1), A is running; B, C, and D are ready; and the semaphore count is 1, indicating that one of D's results is available. When A issues a `semWait` instruction on semaphore *s*, the semaphore decrements to 0, and A can continue to execute; subsequently

[6]In some of the literature, and in some textbooks, no distinction is made between a mutex and a binary semaphore. However, in practice, a number of operating systems, such as Linux, Windows, and Solaris, offer a mutex facility which conforms to the definition in this book.

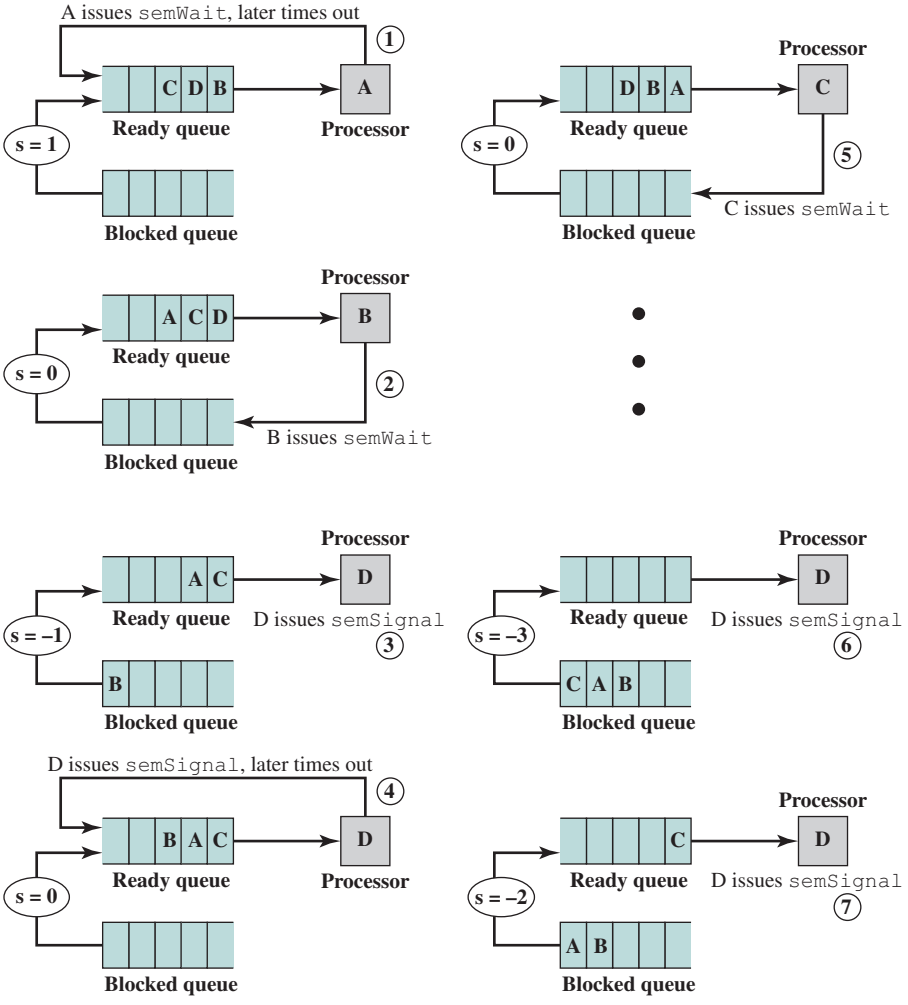



**Figure 5.8** **Example of Semaphore Mechanism**

it rejoins the ready queue. Then B runs (2), eventually issues a `semWait` instruction, and is blocked, allowing D to run (3). When D completes a new result, it issues a `semSignal` instruction, which allows B to move to the ready queue (4). D rejoins the ready queue and C begins to run (5) but is blocked when it issues a `semWait` instruction. Similarly, A and B run and are blocked on the semaphore, allowing D to resume execution (6). When D has a result, it issues a `semSignal`, which transfers C to the ready queue. Later cycles of D will release A and B from the Blocked state.

For the mutual exclusion algorithm discussed in the next subsection and illustrated in Figure 5.9, strong semaphores guarantee freedom from starvation, while weak semaphores do not. We will assume strong semaphores because they are more convenient, and because this is the form of semaphore typically provided by operating systems.

```
/*  program mutualexclusion */
const int n = /* number of processes */;
semaphore s = 1;
void P(int  i)
{
     while (true) {
          semWait(s);
          /* critical section */;
          semSignal(s);
          /* remainder */;
     }
}
void main()
{
     parbegin (P(1), P(2), . . . , P(n));
}
```



VideoNote **Figure 5.9 Mutual Exclusion Using Semaphores**

## Mutual Exclusion

Figure 5.9 shows a straightforward solution to the mutual exclusion problem using a semaphore *s* (compare to Figure 5.4). Consider *n* processes, identified in the array *P*(*i*), all of which need access to the same resource. Each process has a critical section used to access the resource. In each process, a `semWait (s)` is executed just before its critical section. If the value of `s` becomes negative, the process is blocked. If the value is 1, then it is decremented to 0 and the process immediately enters its critical section; because `s` is no longer positive, no other process will be able to enter its critical section.

The semaphore is initialized to 1. Thus, the first process that executes a `semWait` will be able to enter the critical section immediately, setting the value of `s` to 0. Any other process attempting to enter the critical section will find it busy and will be blocked, setting the value of `s` to −1. Any number of processes may attempt entry; each such unsuccessful attempt results in a further decrement of the value of `s`. When the process that initially entered its critical section departs, `s` is incremented and one of the blocked processes (if any) is removed from the queue of blocked processes associated with the semaphore and put in a Ready state. When it is next scheduled by the OS, it may enter the critical section.

Figure 5.10, based on one in [BACO03], shows a possible sequence for three processes using the mutual exclusion discipline of Figure 5.9. In this example three processes (A, B, C) access a shared resource protected by the semaphore *lock*. Process A executes `semWait (`*lock*`)`; because the semaphore has a value of 1 at the time of the `semWait` operation, A can immediately enter its critical section and the semaphore takes on the value 0. While A is in its critical section, both B and C perform a `semWait` operation and are blocked pending the availability of the semaphore. When A exits its critical section and performs `semSignal (`*lock*`)`, B, which was the first process in the queue, can now enter its critical section.

The program of Figure 5.9 can equally well handle a requirement that more than one process be allowed in its critical section at a time. This requirement is met

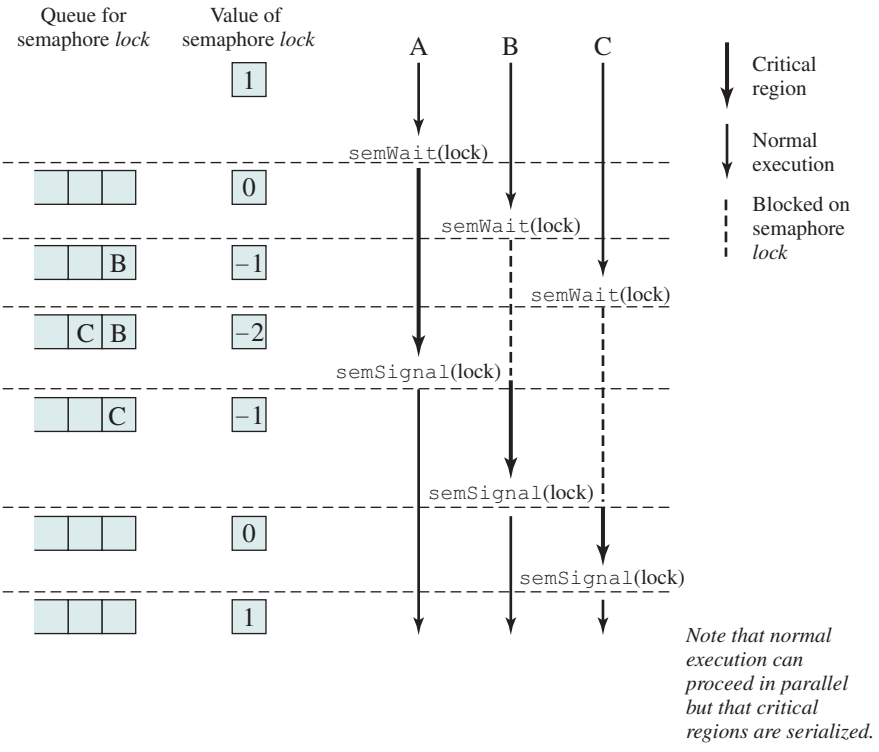



**Figure 5.10** **Processes Accessing Shared Data Protected by a Semaphore**

simply by initializing the semaphore to the specified value. Thus, at any time, the value of *s.count* can be interpreted as follows:

- *s.count* $\geq$ 0: *s.count* is the number of processes that can execute `semWait (s)` without suspension (if no `semSignal (s)` is executed in the meantime). Such situations will allow semaphores to support synchronization as well as mutual exclusion.
- *s.count* $<$ 0: The magnitude of *s.count* is the number of processes suspended in *s.queue*.

## The Producer/Consumer Problem

We now examine one of the most common problems faced in concurrent processing: the producer/consumer problem. The general statement is this: There are one or more producers generating some type of data (records, characters) and placing these in a buffer. There is a single consumer that is taking items out of the buffer one at a time. The system is to be constrained to prevent the overlap of buffer operations. That is, only one agent (producer or consumer) may access the buffer at any one time. The problem is to make sure that the producer won't try to add data into the buffer if it's full, and that the consumer won't try to remove data from an empty buffer. We will

look at a number of solutions to this problem to illustrate both the power and the pitfalls of semaphores.

To begin, let us assume that the buffer is infinite and consists of a linear array of elements. In abstract terms, we can define the producer and consumer functions as follows:

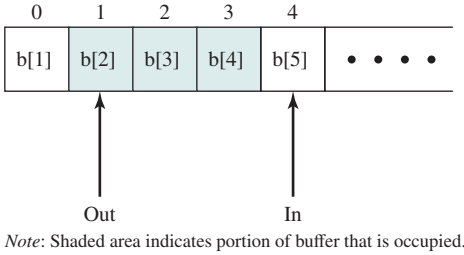
```
producer:
while (true) {
    /* produce item v */;
    b[in] = v;
    in++;
}
```

```
consumer:
while (true) {
    while (in <= out)
        /* do nothing */;
    w = b[out];
    out++;
    /* consume item w */;
}
```

Figure 5.11 illustrates the structure of buffer `b`. The producer can generate items and store them in the buffer at its own pace. Each time, an index (*in*) into the buffer is incremented. The consumer proceeds in a similar fashion but must make sure that it does not attempt to read from an empty buffer. Hence, the consumer makes sure that the producer has advanced beyond it (*in* > *out*) before proceeding.

Let us try to implement this system using binary semaphores. Figure 5.12 is a first attempt. Rather than deal with the indices *in* and *out*, we can simply keep track of the number of items in the buffer, using the integer variable *n* (= *in* − *out*). The semaphore `s` is used to enforce mutual exclusion; the semaphore `delay` is used to force the consumer to `semWait` if the buffer is empty.

This solution seems rather straightforward. The producer is free to add to the buffer at any time. It performs `semWaitB (s)` before appending and `semSignalB (s)` afterward to prevent the consumer (or any other producer) from accessing the buffer during the append operation. Also, while in the critical section, the producer increments the value of *n*. If $n = 1$, then the buffer was empty just prior to this append, so the producer performs `semSignalB (delay)` to alert the consumer of this fact. The consumer begins by waiting for the first item to be produced, using `semWaitB (delay)`. It then takes an item and decrements *n* in its critical section. If the producer is able to stay ahead of the consumer (a common situation), then the




*Note*: Shaded area indicates portion of buffer that is occupied.

**Figure 5.11 Infinite Buffer for the Producer/Consumer Problem**

```
/*  program producerconsumer */
      int n;
      binary_semaphore s = 1, delay = 0;
      void producer()
      {
           while (true) {
                 produce();
                 semWaitB(s);
                 append();
                 n++;
                 if (n==1) semSignalB(delay);
                 semSignalB(s);
           }
      }
      void consumer()
      {
           semWaitB(delay);
           while (true) {
                 semWaitB(s);
                 take();
                 n--;
                 semSignalB(s);
                 consume();
                 if (n==0) semWaitB(delay);
           }
      }
      void main()
      {
           n = 0;
           parbegin (producer, consumer);
      }
```



VideoNote **Figure 5.12 An Incorrect Solution to the Infinite-Buffer Producer/Consumer Problem Using Binary Semaphores**

consumer will rarely block on the semaphore `delay` because *n* will usually be positive. Hence, both producer and consumer run smoothly.

There is, however, a flaw in this program. When the consumer has exhausted the buffer, it needs to reset the delay semaphore so it will be forced to wait until the producer has placed more items in the buffer. This is the purpose of the statement: **`if`** `n == 0 semWaitB (delay)`. Consider the scenario outlined in Table 5.4. In line 14, the consumer fails to execute the `semWaitB` operation. The consumer did indeed exhaust the buffer and set *n* to 0 (line 8), but the producer has incremented *n* before the consumer can test it in line 14. The result is a `semSignalB` not matched by a prior `semWaitB`. The value of −1 for *n* in line 20 means the consumer has consumed an item from the buffer that does not exist. It would not do simply to move the conditional statement inside the critical section of the consumer, because this could lead to deadlock (e.g., after line 8 of Table 5.4).

A fix for the problem is to introduce an auxiliary variable that can be set in the consumer's critical section for use later on. This is shown in Figure 5.13. A careful trace of the logic should convince you that deadlock can no longer occur.

A somewhat cleaner solution can be obtained if general semaphores (also called counting semaphores) are used, as shown in Figure 5.14. The variable *n* is now

**Table 5.4** Possible Scenario for the Program of Figure 5.12

| | Producer | Consumer | s | *n* | Delay |
|---|---|---|---|---|---|
| 1 | | | 1 | 0 | 0 |
| 2 | semWaitB(s) | | 0 | 0 | 0 |
| 3 | n++ | | 0 | 1 | 0 |
| 4 | **if** (n==1) (semSignalB(delay)) | | 0 | 1 | 1 |
| 5 | semSignalB(s) | | 1 | 1 | 1 |
| 6 | | semWaitB(delay) | 1 | 1 | 0 |
| 7 | | semWaitB(s) | 0 | 1 | 0 |
| 8 | | n-- | 0 | 0 | 0 |
| 9 | | semSignalB(s) | 1 | 0 | 0 |
| 10 | semWaitB(s) | | 0 | 0 | 0 |
| 11 | n++ | | 0 | 1 | 0 |
| 12 | **if** (n==1) (semSignalB(delay)) | | 0 | 1 | 1 |
| 13 | semSignalB(s) | | 1 | 1 | 1 |
| 14 | | **if** (n==0) (semWaitB(delay)) | 1 | 1 | 1 |
| 15 | | semWaitB(s) | 0 | 1 | 1 |
| 16 | | n-- | 0 | 0 | 1 |
| 17 | | semSignalB(s) | 1 | 0 | 1 |
| 18 | | **if** (n==0) (semWaitB(delay)) | 1 | 0 | 0 |
| 19 | | semWaitB(s) | 0 | 0 | 0 |
| 20 | | n-- | 0 | −1 | 0 |
| 21 | | semSignalB(s) | 1 | −1 | 0 |

*Note:* White areas represent the critical section controlled by semaphore s.

a semaphore. Its value still is equal to the number of items in the buffer. Suppose now that in transcribing this program, a mistake is made and the operations `semSignal (s)` and `semSignal (n)` are interchanged. This would require that the `semSignal (n)` operation be performed in the producer's critical section without interruption by the consumer or another producer. Would this affect the program? No, because the consumer must wait on both semaphores before proceeding in any case.

Now suppose the `semWait (n)` and `semWait (s)` operations are accidentally reversed. This produces a serious, indeed a fatal, flaw. If the consumer ever enters its critical section when the buffer is empty ($n.count = 0$), then no producer can ever append to the buffer and the system is deadlocked. This is a good example of the subtlety of semaphores and the difficulty of producing correct designs.

Finally, let us add a new and realistic restriction to the producer/consumer problem: namely, that the buffer is finite. The buffer is treated as a circular storage

```
/* program producerconsumer */
      int n;
      binary_semaphore s = 1, delay = 0;
      void producer()
      {
           while (true) {
                 produce();
                 semWaitB(s);
                 append();
                 n++;
                 if (n==1) semSignalB(delay);
                 semSignalB(s);
           }
      }
      void  consumer()
      {
           int  m; /* a local variable */
           semWaitB(delay);
           while  (true) {
                 semWaitB(s);
                 take();
                 n--;
                 m = n;
                 semSignalB(s);
                 consume();
                 if  (m==0) semWaitB(delay);
           }
      }
      void  main()
      {
           n = 0;
           parbegin  (producer, consumer);
      }
```



VideoNote **Figure 5.13 A Correct Solution to the Infinite-Buffer Producer/Consumer Problem Using Binary Semaphores**

(see Figure 5.15), and pointer values must be expressed modulo the size of the buffer. The following relationships hold:

| Block on: | Unblock on: |
|---|---|
| Producer: insert in full buffer | Consumer: item inserted |
| Consumer: remove from empty buffer | Producer: item removed |

The producer and consumer functions can be expressed as follows (variable *in* and *out* are initialized to 0 and *n* is the size of the buffer):

```
producer:
while (true) {
    /* produce item v */
    while ((in + 1) % n == out)
       /* do nothing */;
    b[in] = v;
    in = (in + 1)  %  n;
}
```

```
consumer:
while (true) {
    while (in == out)
       /* do nothing */;
    w = b[out];
    out = (out + 1) % n;
    /* consume item w */;
}
```

```
/* program producerconsumer */
      semaphore n = 0, s = 1;
      void producer()
      {
           while (true) {
                 produce();
                 semWait(s);
                 append();
                 semSignal(s);
                 semSignal(n);
           }
      }
      void consumer()
      {
           while (true) {
                 semWait(n);
                 semWait(s);
                 take();
                 semSignal(s);
                 consume();
           }
      }
      void main()
      {
                parbegin  (producer, consumer);
      }
```

VideoNote **Figure 5.14 A Solution to the Infinite-Buffer Producer/Consumer Problem Using Semaphores**

Figure 5.16 shows a solution using general semaphores. The semaphore *e* has been added to keep track of the number of empty spaces.

Another instructive example in the use of semaphores is the barbershop problem described in Appendix A. Appendix A also includes additional examples of the problem of race conditions when using semaphores.

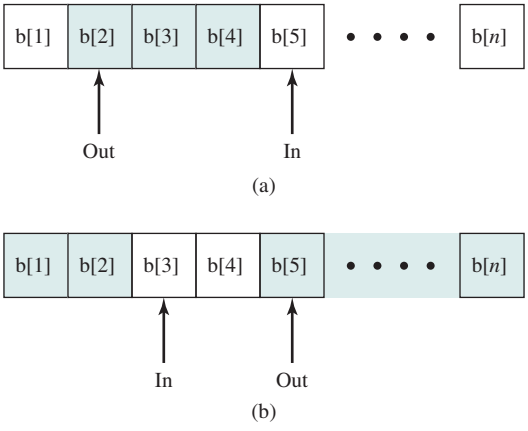



**Figure 5.15 Finite Circular Buffer for the Producer/Consumer Problem**

```
/* program boundedbuffer */
      const int sizeofbuffer = /* buffer size */;
      semaphore s = 1, n = 0, e = sizeofbuffer;
      void producer()
      {
           while (true) {
                 produce();
                 semWait(e);
                 semWait(s);
                 append();
                 semSignal(s);
                 semSignal(n);
           }
      }
      void consumer()
      {
           while  (true) {
                 semWait(n);
                 semWait(s);
                 take();
                 semSignal(s);
                 semSignal(e);
                 consume();
           }
      }
      void main()
      {
           parbegin  (producer, consumer);
      }
```




**Figure 5.16 A Solution to the Bounded-Buffer Producer/Consumer Problem Using Semaphores**

## Implementation of Semaphores

As was mentioned earlier, it is imperative that the `semWait` and `semSignal` operations be implemented as atomic primitives. One obvious way is to implement them in hardware or firmware. Failing this, a variety of schemes have been suggested. The essence of the problem is one of mutual exclusion: Only one process at a time may manipulate a semaphore with either a `semWait` or `semSignal` operation. Thus, any of the software schemes, such as Dekker's algorithm or Peterson's algorithm (see Section 5.1), could be used; this would entail a substantial processing overhead.

Another alternative is to use one of the hardware-supported schemes for mutual exclusion. For example, Figure 5.17 shows the use of a `compare&swap` instruction. In this implementation, the semaphore is again a structure, as in Figure 5.6, but now includes a new integer component, *s.flag*. Admittedly, this involves a form of busy waiting. However, the `semWait` and `semSignal` operations are relatively short, so the amount of busy waiting involved should be minor.

For a single-processor system, it is possible to inhibit interrupts for the duration of a `semWait` or `semSignal` operation, as suggested in Figure 5.17b. Once again, the relatively short duration of these operations means that this approach is reasonable.

```
semWait(s)
{
   while (compare_and_swap(s.flag, 0 , 1) == 1)
      /* do nothing */;
   s.count--;
   if (s.count < 0) {
      /* place this process in s.queue*/;
      /* block this process (must also set
s.flag to 0) */;
   }
   s.flag = 0;
}
semSignal(s)
{
   while (compare_and_swap(s.flag, 0 , 1) == 1)
      /* do nothing */;
   s.count++;
   if  (s.count<= 0) {
      /* remove a process P from s.queue */;
      /* place process P on ready list */;
   }
   s.flag = 0;
}
```

(a) Compare and Swap Instruction

```
semWait(s)
{
   inhibit interrupts;
   s.count--;
   if (s.count < 0) {
      /* place this process in s.queue */;
      /* block this process and allow inter-
rupts*/;
   }
   else
      allow interrupts;
}

semSignal(s)
{
   inhibit interrupts;
   s.count++;
   if (s.count<= 0) {
      /* remove a process P from s.queue */;
      /* place process P on ready list */;
   }
   allow interrupts;
}
```

(b) Interrupts

VideoNote **Figure 5.17 Two Possible Implementations of Semaphores**

## 5.5 MONITORS

Semaphores provide a primitive yet powerful and flexible tool for enforcing mutual exclusion and for coordinating processes. However, as Figure 5.12 suggests, it may be difficult to produce a correct program using semaphores. The difficulty is that `semWait` and `semSignal` operations may be scattered throughout a program, and it is not easy to see the overall effect of these operations on the semaphores they affect.

The monitor is a programming language construct that provides equivalent functionality to that of semaphores and that is easier to control. The concept was first formally defined in [HOAR74]. The monitor construct has been implemented in a number of programming languages, including Concurrent Pascal, Pascal-Plus, Modula-2, Modula-3, and Java. It has also been implemented as a program library. This allows programmers to put a monitor lock on any object. In particular, for something like a linked list, you may want to lock all linked lists with one lock, or have one lock for each list, or have one lock for each element of each list.

We begin with a look at Hoare's version, and then examine a refinement.

### Monitor with Signal

A monitor is a software module consisting of one or more procedures, an initialization sequence, and local data. The chief characteristics of a monitor are the following:

1. The local data variables are accessible only by the monitor's procedures and not by any external procedure.

2. A process enters the monitor by invoking one of its procedures.
3. Only one process may be executing in the monitor at a time; any other processes that have invoked the monitor are blocked, waiting for the monitor to become available.

The first two characteristics are reminiscent of those for objects in object-oriented software. Indeed, an object-oriented OS or programming language can readily implement a monitor as an object with special characteristics.

By enforcing the discipline of one process at a time, the monitor is able to provide a mutual exclusion facility. The data variables in the monitor can be accessed by only one process at a time. Thus, a shared data structure can be protected by placing it in a monitor. If the data in a monitor represent some resource, then the monitor provides a mutual exclusion facility for accessing the resource.

To be useful for concurrent processing, the monitor must include synchronization tools. For example, suppose a process invokes the monitor and, while in the monitor, must be blocked until some condition is satisfied. A facility is needed by which the process is not only blocked, but releases the monitor so some other process may enter it. Later, when the condition is satisfied and the monitor is again available, the process needs to be resumed and allowed to reenter the monitor at the point of its suspension.

A monitor supports synchronization by the use of **condition variables** that are contained within the monitor and accessible only within the monitor. Condition variables are a special data type in monitors, which are operated on by two functions:

- `cwait(c)`: Suspend execution of the calling process on condition *c*. The monitor is now available for use by another process.
- `csignal(c)`: Resume execution of some process blocked after a `cwait` on the same condition. If there are several such processes, choose one of them; if there is no such process, do nothing.

Note that monitor *wait* and *signal* operations are different from those for the semaphore. If a process in a monitor signals and no task is waiting on the condition variable, the signal is lost.

Figure 5.18 illustrates the structure of a monitor. Although a process can enter the monitor by invoking any of its procedures, we can think of the monitor as having a single entry point that is guarded so only one process may be in the monitor at a time. Other processes that attempt to enter the monitor join a queue of processes blocked waiting for monitor availability. Once a process is in the monitor, it may temporarily block itself on condition `x` by issuing `cwait(x)`; it is then placed in a queue of processes waiting to reenter the monitor when the condition changes, and resume execution at the point in its program following the `cwait(x)` call.

If a process that is executing in the monitor detects a change in the condition variable `x`, it issues `csignal(x)`, which alerts the corresponding condition queue that the condition has changed.

As an example of the use of a monitor, let us return to the bounded-buffer producer/consumer problem. Figure 5.19 shows a solution using a monitor. The monitor module, `boundedbuffer`, controls the buffer used to store and retrieve characters. The monitor includes two condition variables (declared with the

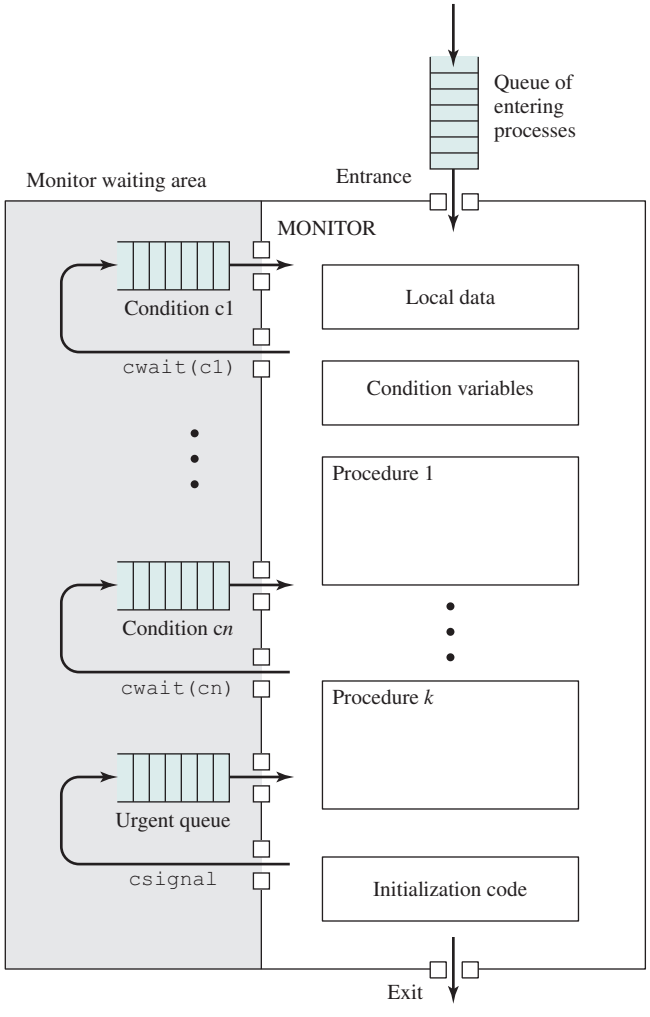



**Figure 5.18 Structure of a Monitor**

construct **cond**): *notfull* is true when there is room to add at least one character to the buffer, and *notempty* is true when there is at least one character in the buffer.

A producer can add characters to the buffer only by means of the procedure append inside the monitor; the producer does not have direct access to *buffer*. The procedure first checks the condition *notfull* to determine if there is space available in the buffer. If not, the process executing the monitor is blocked on that condition. Some other process (producer or consumer) may now enter the monitor. Later, when the buffer is no longer full, the blocked process may be removed from the queue, reactivated, and resume processing. After placing a character in the buffer, the process signals the *notempty* condition. A similar description can be made of the consumer function.

This example points out the division of responsibility with monitors compared to semaphores. In the case of monitors, the monitor construct itself enforces mutual

```
/* program producerconsumer */
monitor boundedbuffer;
char buffer [N];                                          /* space for N items */
int nextin, nextout;                                        /* buffer pointers */
int count;                                             /* number of items in buffer */
cond notfull, notempty;              /* condition variables for synchronization */
void append (char x)

{
      if (count == N) cwait(notfull);          /* buffer is full; avoid overflow */
      buffer[nextin] = x;
      nextin = (nextin + 1) % N;
      count++;
      /* one more item in buffer */
      csignal (notempty);                           /*resume any waiting consumer */
}
void take (char x)
{
      if (count == 0) cwait(notempty);       /* buffer is empty; avoid underflow */
      x = buffer[nextout];
      nextout = (nextout + 1) % N);
      count--;                                          /* one fewer item in buffer */
      csignal (notfull);                           /* resume any waiting producer */
}
{                                                                  /* monitor body */
      nextin = 0; nextout = 0; count = 0;              /* buffer initially empty */
}
```

```
void producer()
{
      char x;
      while (true) {
      produce(x);
      append(x);
      }
}
void consumer()
{
      char x;
      while (true) {
      take(x);
      consume(x);
      }
}
void main()
{
      parbegin (producer, consumer);
}
```

VideoNote **Figure 5.19 A Solution to the Bounded-Buffer Producer/Consumer Problem Using a Monitor**

exclusion: It is not possible for both a producer and a consumer to simultaneously access the buffer. However, the programmer must place the appropriate `cwait` and `csignal` primitives inside the monitor to prevent processes from depositing items in a full buffer or removing them from an empty one. In the case of semaphores, both mutual exclusion and synchronization are the responsibility of the programmer.

Note in Figure 5.19, a process exits the monitor immediately after executing the `csignal` function. If the `csignal` does not occur at the end of the procedure, then, in Hoare's proposal, the process issuing the signal is blocked to make the monitor available and placed in a queue until the monitor is free. One possibility at this point would be to place the blocked process in the entrance queue, so it would have to compete for access with other processes that had not yet entered the monitor. However, because a process blocked on a `csignal` function has already partially performed its task in the monitor, it makes sense to give this process precedence over newly entering processes by setting up a separate urgent queue (see Figure 5.18). One language that uses monitors, Concurrent Pascal, requires that `csignal` only appear as the last operation executed by a monitor procedure.

If there are no processes waiting on condition *x*, then the execution of `csignal (x)` has no effect.

As with semaphores, it is possible to make mistakes in the synchronization function of monitors. For example, if either of the `csignal` functions in the `bounded-buffer` monitor are omitted, then processes entering the corresponding condition queue are permanently hung up. The advantage that monitors have over semaphores is that all of the synchronization functions are confined to the monitor. Therefore, it is easier to verify that the synchronization has been done correctly and to detect bugs. Furthermore, once a monitor is correctly programmed, access to the protected resource is correct for access from all processes. In contrast, with semaphores, resource access is correct only if all of the processes that access the resource are programmed correctly.

## Alternate Model of Monitors with Notify and Broadcast

Hoare's definition of monitors [HOAR74] requires that if there is at least one process in a condition queue, a process from that queue runs immediately when another process issues a `csignal` for that condition. Thus, the process issuing the `csignal` must either immediately exit the monitor or be blocked on the monitor.

There are two drawbacks to this approach:

1. If the process issuing the `csignal` has not finished with the monitor, then two additional process switches are required: one to block this process, and another to resume it when the monitor becomes available.
2. Process scheduling associated with a signal must be perfectly reliable. When a `csignal` is issued, a process from the corresponding condition queue must be activated immediately, and the scheduler must ensure that no other process enters the monitor before activation. Otherwise, the condition under which the process was activated could change. For example, in Figure 5.19, when a `csignal(notempty)` is issued, a process from the `notempty` queue must be activated before a new consumer enters the monitor. Another example: A producer process may append a character to an empty buffer then fail before signaling; any processes in the `notempty` queue would be permanently hung up.

Lampson and Redell developed a different definition of monitors for the language Mesa [LAMP80]. Their approach overcomes the problems just listed and supports several useful extensions. The Mesa monitor structure is also used in the Modula-3 systems programming language [NELS91]. In Mesa, the `csignal` primitive

```
void append (char x)
{
      while (count == N) cwait(notfull);       /* buffer is full; avoid overflow */
      buffer[nextin] = x;
      nextin = (nextin + 1) % N;
      count++;                                        /* one more item in buffer */
      cnotify(notempty);                           /* notify any waiting consumer */
}
void take (char x)
{
      while (count == 0) cwait(notempty);    /* buffer is empty; avoid underflow */
      x = buffer[nextout];
      nextout = (nextout + 1) % N);
      count--;                                          /* one fewer item in buffer */
      cnotify(notfull);                            /* notify any waiting producer */
}
```


VideoNote

**Figure 5.20 Bounded-Buffer Monitor Code for Mesa Monitor**

is replaced by `cnotify`, with the following interpretation: When a process executing in a monitor executes `cnotify(x)`, it causes the *x* condition queue to be notified, but the signaling process continues to execute. The result of the notification is that the process at the head of the condition queue will be resumed at some convenient future time when the monitor is available. However, because there is no guarantee that some other process will not enter the monitor before the waiting process, the waiting process must recheck the condition. For example, the procedures in the `boundedbuffer` monitor would now have the code of Figure 5.20.

The **if** statements are replaced by **while** loops. Thus, this arrangement results in at least one extra evaluation of the condition variable. In return, however, there are no extra process switches, and no constraints on when the waiting process must run after a `cnotify`.

One useful refinement that can be associated with the `cnotify` primitive is a watchdog timer associated with each condition primitive. A process that has been waiting for the maximum timeout interval will be placed in a ready state regardless of whether the condition has been notified. When activated, the process checks the condition and continues if the condition is satisfied. The timeout prevents the indefinite starvation of a process in the event that some other process fails before signaling a condition.

With the rule that a process is notified rather than forcibly reactivated, it is possible to add a `cbroadcast` primitive to the repertoire. The broadcast causes all processes waiting on a condition to be placed in a ready state. This is convenient in situations where a process does not know how many other processes should be reactivated. For example, in the producer/consumer program, suppose both the append and the `take` functions can apply to variable-length blocks of characters. In that case, if a producer adds a block of characters to the buffer, it need not know how many characters each waiting consumer is prepared to consume. It simply issues a `cbroadcast`, and all waiting processes are alerted to try again.

In addition, a broadcast can be used when a process would have difficulty figuring out precisely which other process to reactivate. A good example is a memory

manager. The manager has $j$ bytes free; a process frees up an additional $k$ bytes, but it does not know which waiting process can proceed with a total of $k + j$ bytes. Hence it uses broadcast, and all processes check for themselves if there is enough memory free.

An advantage of Lampson/Redell monitors over Hoare monitors is that the Lampson/Redell approach is less prone to error. In the Lampson/Redell approach, because each procedure checks the monitor variable after being signaled, with the use of the **while** construct, a process can signal or broadcast incorrectly without causing an error in the signaled program. The signaled program will check the relevant variable and, if the desired condition is not met, continue to wait.

Another advantage of the Lampson/Redell monitor is that it lends itself to a more modular approach to program construction. For example, consider the implementation of a buffer allocator. There are two levels of conditions to be satisfied for cooperating sequential processes:

1. Consistent data structures. Thus, the monitor enforces mutual exclusion and completes an input or output operation before allowing another operation on the buffer.
2. Level 1, plus enough memory for this process to complete its allocation request.

In the Hoare monitor, each signal conveys the level 1 condition but also carries the implicit message, "I have freed enough bytes for your particular allocate call to work now." Thus, the signal implicitly carries the level 2 condition. If the programmer later changes the definition of the level 2 condition, it will be necessary to reprogram all signaling processes. If the programmer changes the assumptions made by any particular waiting process (i.e., waiting for a slightly different level 2 invariant), it may be necessary to reprogram all signaling processes. This is unmodular and likely to cause synchronization errors (e.g., wake up by mistake) when the code is modified. The programmer has to remember to modify all procedures in the monitor every time a small change is made to the level 2 condition. With a Lampson/Redell monitor, a broadcast ensures the level 1 condition and carries a hint that level 2 might hold; each process should check the level 2 condition itself. If a change is made in the level 2 condition in either a waiter or a signaler, there is no possibility of erroneous wakeup because each procedure checks its own level 2 condition. Therefore, the level 2 condition can be hidden within each procedure. With the Hoare monitor, the level 2 condition must be carried from the waiter into the code of every signaling process, which violates data abstraction and interprocedural modularity principles.

## 5.6 MESSAGE PASSING

When processes interact with one another, two fundamental requirements must be satisfied: synchronization and communication. Processes need to be synchronized to enforce mutual exclusion; cooperating processes may need to exchange information. One approach to providing both of these functions is message passing. Message passing has the further advantage that it lends itself to implementation in distributed systems as well as in shared-memory multiprocessor and uniprocessor systems.

**Table 5.5** Design Characteristics of Message Systems for Interprocess Communication and Synchronization

| | |
|---|---|
| **Synchronization** | **Format** |
| Send | Content |
| blocking | Length |
| nonblocking | fixed |
| Receive | variable |
| blocking | |
| nonblocking | **Queueing Discipline** |
| test for arrival | FIFO |
| | Priority |
| **Addressing** | |
| Direct | |
| send | |
| receive | |
| explicit | |
| implicit | |
| Indirect | |
| static | |
| dynamic | |
| ownership | |

Message-passing systems come in many forms. In this section, we will provide a general introduction that discusses features typically found in such systems. The actual function of message passing is normally provided in the form of a pair of primitives:

```
send (destination, message)
receive (source, message)
```

This is the minimum set of operations needed for processes to engage in message passing. A process sends information in the form of a *message* to another process designated by a *destination*. A process receives information by executing the `receive` primitive, indicating the *source* and the *message*.

A number of design issues relating to message-passing systems are listed in Table 5.5, and examined in the remainder of this section.

## Synchronization

The communication of a message between two processes implies some level of synchronization between the two: The receiver cannot receive a message until it has been sent by another process. In addition, we need to specify what happens to a process after it issues a `send` or `receive` primitive.

Consider the `send` primitive first. When a send primitive is executed in a process, there are two possibilities: Either the sending process is blocked until the

message is received, or it is not. Similarly, when a process issues a `receive` primitive, there are two possibilities:

1. If a message has previously been sent, the message is received and execution continues.
2. If there is no waiting message, then either (a) the process is blocked until a message arrives, or (b) the process continues to execute, abandoning the attempt to receive.

Thus, both the sender and receiver can be blocking or nonblocking. Three combinations are common, although any particular system will usually have only one or two combinations implemented:

1. **Blocking send, blocking receive:** Both the sender and receiver are blocked until the message is delivered; this is sometimes referred to as a *rendezvous*. This combination allows for tight synchronization between processes.
2. **Nonblocking send, blocking receive:** Although the sender may continue on, the receiver is blocked until the requested message arrives. This is probably the most useful combination. It allows a process to send one or more messages to a variety of destinations as quickly as possible. A process that must receive a message before it can do useful work needs to be blocked until such a message arrives. An example is a server process that exists to provide a service or resource to other processes.
3. **Nonblocking send, nonblocking receive:** Neither party is required to wait.

The nonblocking `send` is more natural for many concurrent programming tasks. For example, if it is used to request an output operation such as printing, it allows the requesting process to issue the request in the form of a message, then carry on. One potential danger of the nonblocking `send` is that an error could lead to a situation in which a process repeatedly generates messages. Because there is no blocking to discipline the process, these messages could consume system resources, including processor time and buffer space, to the detriment of other processes and the OS. Also, the nonblocking `send` places the burden on the programmer to determine that a message has been received: Processes must employ reply messages to acknowledge receipt of a message.

For the `receive` primitive, the blocking version appears to be more natural for many concurrent programming tasks. Generally, a process that requests a message will need the expected information before proceeding. However, if a message is lost, which can happen in a distributed system, or if a process fails before it sends an anticipated message, a receiving process could be blocked indefinitely. This problem can be solved by the use of the nonblocking `receive`. However, the danger of this approach is that if a message is sent after a process has already executed a matching `receive`, the message will be lost. Other possible approaches are to allow a process to test whether a message is waiting before issuing a `receive` and allow a process to specify more than one source in a `receive` primitive. The latter approach is useful if a process is waiting for messages from more than one source, and can proceed if any of these messages arrive.

## Addressing

Clearly, it is necessary to have a way of specifying in the `send` primitive which process is to receive the message. Similarly, most implementations allow a receiving process to indicate the source of a message to be received.

The various schemes for specifying processes in `send` and `receive` primitives fall into two categories: direct addressing and indirect addressing. With **direct addressing**, the send primitive includes a specific identifier of the destination process. The `receive` primitive can be handled in one of two ways. One possibility is to require that the process explicitly designate a sending process. Thus, the process must know ahead of time from which process a message is expected. This will often be effective for cooperating concurrent processes. In other cases, however, it is impossible to specify the anticipated source process. An example is a printer-server process, which will accept a print request message from any other process. For such applications, a more effective approach is the use of implicit addressing. In this case, the *source* parameter of the `receive` primitive possesses a value returned when the receive operation has been performed.

The other general approach is **indirect addressing**. In this case, messages are not sent directly from sender to receiver, but rather are sent to a shared data structure consisting of queues that can temporarily hold messages. Such queues are generally referred to as *mailboxes*. Thus, for two processes to communicate, one process sends a message to the appropriate mailbox, and the other process picks up the message from the mailbox.

A strength of the use of indirect addressing is that, by decoupling the sender and receiver, it allows for greater flexibility in the use of messages. The relationship between senders and receivers can be one-to-one, many-to-one, one-to-many, or many-to-many (see Figure 5.21). A **one-to-one** relationship allows a private communications link to be set up between two processes. This insulates their interaction from erroneous interference from other processes. A **many-to-one** relationship is useful for client/server interaction; one process provides service to a number of other processes. In this case, the mailbox is often referred to as a *port*. A **one-to-many** relationship allows for one sender and multiple receivers; it is useful for applications where a message or some information is to be broadcast to a set of processes. A **many-to-many** relationship allows multiple server processes to provide concurrent service to multiple clients.

The association of processes to mailboxes can be either static or dynamic. Ports are often statically associated with a particular process; that is, the port is created and permanently assigned to the process. Similarly, a one-to-one relationship is typically defined statically and permanently. When there are many senders, the association of a sender to a mailbox may occur dynamically. Primitives such as `connect` and `disconnect` may be used for this purpose.

A related issue has to do with the ownership of a mailbox. In the case of a port, it is typically owned and created by the receiving process. Thus, when the process is destroyed, the port is also destroyed. For the general mailbox case, the OS may offer a create mailbox service. Such mailboxes can be viewed either as being owned by the creating process, in which case they terminate with the process, or as being owned by the OS, in which case an explicit command will be required to destroy the mailbox.

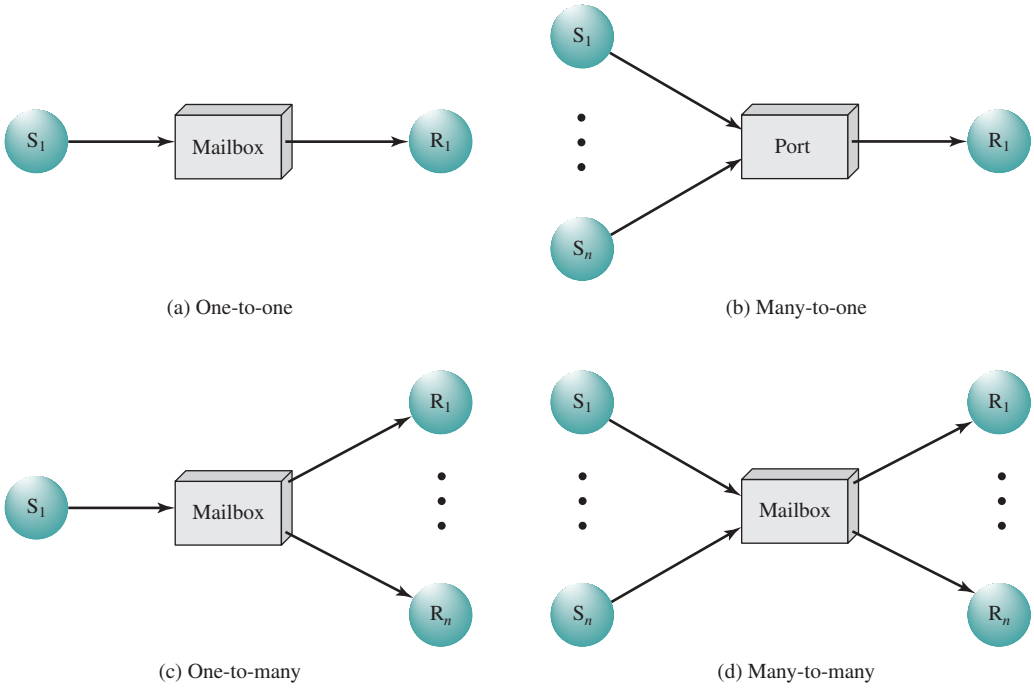



**Figure 5.21** **Indirect Process Communication**

## Message Format

The format of the message depends on the objectives of the messaging facility and whether the facility runs on a single computer or on a distributed system. For some operating systems, designers have preferred short, fixed-length messages to minimize processing and storage overhead. If a large amount of data is to be passed, the data can be placed in a file and the message then simply references that file. A more flexible approach is to allow variable-length messages.

Figure 5.22 shows a typical message format for operating systems that support variable-length messages. The message is divided into two parts: a header, which

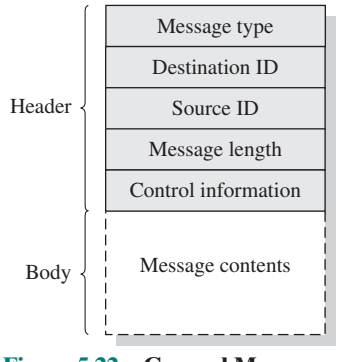



**Figure 5.22** **General Message Format**

contains information about the message, and a body, which contains the actual contents of the message. The header may contain an identification of the source and intended destination of the message, a length field, and a type field to discriminate among various types of messages. There may also be additional control information, such as a pointer field so that a linked list of messages can be created; a sequence number, to keep track of the number and order of messages passed between source and destination; and a priority field.

## Queueing Discipline

The simplest queueing discipline is first-in-first-out, but this may not be sufficient if some messages are more urgent than others. An alternative is to allow the specifying of message priority, on the basis of message type or by designation by the sender. Another alternative is to allow the receiver to inspect the message queue and select which message to receive next.

## Mutual Exclusion

Figure 5.23 shows one way in which message passing can be used to enforce mutual exclusion (compare to Figures 5.4, 5.5, and 5.9). We assume the use of the blocking receive primitive and the nonblocking send primitive. A set of concurrent processes share a mailbox, box, which can be used by all processes to send and receive. The mailbox is initialized to contain a single message with null content. A process wishing to enter its critical section first attempts to receive a message. If the mailbox is empty, then the process is blocked. Once a process has acquired the message, it performs its critical section then places the message back into the mailbox. Thus, the message functions as a token that is passed from process to process.

```
/* program mutualexclusion */
const int n = /* number of process */
void P(int i)
{
      message msg;
      while (true) {
         receive (box, msg);
         /* critical section */;
         send (box, msg);
         /* remainder */;
      }
}
void main()
{
      create mailbox (box);
      send (box, null);
      parbegin (P(1), P(2), . . . , P(n));
}
```


VideoNote **Figure 5.23 Mutual Exclusion Using Messages**

The preceding solution assumes that if more than one process performs the receive operation concurrently, then:

- If there is a message, it is delivered to only one process and the others are blocked, or
- If the message queue is empty, all processes are blocked; when a message is available, only one blocked process is activated and given the message.

These assumptions are true of virtually all message-passing facilities.

As an example of the use of message passing, Figure 5.24 is a solution to the bounded-buffer producer/consumer problem. Using the basic mutual exclusion power of message passing, the problem could have been solved with an algorithmic structure similar to that of Figure 5.16. Instead, the program of Figure 5.24 takes advantage of the ability of message passing to be used to pass data in addition to signals. Two mailboxes are used. As the producer generates data, it is sent as messages to the mailbox `mayconsume`. As long as there is at least one message in that mailbox, the consumer can consume. Hence `mayconsume` serves as the buffer; the data in the buffer are organized as a queue of messages. The "size" of the buffer is determined by the global variable `capacity`. Initially, the mailbox `mayproduce`

```
const int
    capacity = /* buffering capacity */ ;
    null = /* empty message */ ;
int i;
void producer()
{   message pmsg;
    while (true) {
     receive (mayproduce,pmsg);
     pmsg = produce();
     send (mayconsume,pmsg);
    }
}
void consumer()
{   message cmsg;
    while (true) {
     receive (mayconsume,cmsg);
     consume (cmsg);
     send (mayproduce,null);
    }
}
void main()
{
    create_mailbox (mayproduce);
    create_mailbox (mayconsume);
    for (int i = 1;i<= capacity;i++) send (mayproduce,null);
    parbegin (producer,consumer);
}
```




**Figure 5.24 A Solution to the Bounded-Buffer Producer/Consumer Problem Using Messages**

is filled with a number of null messages equal to the capacity of the buffer. The number of messages in `mayproduce` shrinks with each production and grows with each consumption.

This approach is quite flexible. There may be multiple producers and consumers, as long as all have access to both mailboxes. The system may even be distributed, with all producer processes and the `mayproduce` mailbox at one site and all the consumer processes and the `mayconsume` mailbox at another.

## 5.7 READERS/WRITERS PROBLEM

In dealing with the design of synchronization and concurrency mechanisms, it is useful to be able to relate the problem at hand to known problems, and to be able to test any solution in terms of its ability to solve these known problems. In the literature, several problems have assumed importance and appear frequently, both because they are examples of common design problems and because of their educational value. One such problem is the producer/consumer problem, which has already been explored. In this section, we will look at another classic problem: the readers/writers problem.

The readers/writers problem is defined as follows: There is a data area shared among a number of processes. The data area could be a file, a block of main memory, or even a bank of processor registers. There are a number of processes that only read the data area (readers) and a number that only write to the data area (writers). The conditions that must be satisfied are as follows:

1. Any number of readers may simultaneously read the file.
2. Only one writer at a time may write to the file.
3. If a writer is writing to the file, no reader may read it.

Thus, readers are processes that are not required to exclude one another, and writers are processes that are required to exclude all other processes, readers and writers alike.

Before proceeding, let us distinguish this problem from two others: the general mutual exclusion problem, and the producer/consumer problem. In the readers/writers problem, readers do not also write to the data area, nor do writers read the data area while writing. A more general case, which includes this case, is to allow any of the processes to read or write the data area. In that case, we can declare any portion of a process that accesses the data area to be a critical section and impose the general mutual exclusion solution. The reason for being concerned with the more restricted case is that more efficient solutions are possible for this case, and the less efficient solutions to the general problem are unacceptably slow. For example, suppose that the shared area is a library catalog. Ordinary users of the library read the catalog to locate a book. One or more librarians are able to update the catalog. In the general solution, every access to the catalog would be treated as a critical section, and users would be forced to read the catalog one at a time. This would clearly impose intolerable delays. At the same time,

it is important to prevent writers from interfering with each other, and it is also required to prevent reading while writing is in progress to prevent the access of inconsistent information.

Can the producer/consumer problem be considered simply a special case of the readers/writers problem with a single writer (the producer) and a single reader (the consumer)? The answer is no. The producer is not just a writer. It must read queue pointers to determine where to write the next item, and it must determine if the buffer is full. Similarly, the consumer is not just a reader, because it must adjust the queue pointers to show that it has removed a unit from the buffer.

We now examine two solutions to the problem.

## Readers Have Priority

Figure 5.25 is a solution using semaphores, showing one instance each of a reader and a writer; the solution does not change for multiple readers and writers. The writer process is simple. The semaphore `wsem` is used to enforce mutual exclusion.

```
/* program readersandwriters */
int readcount;
semaphore x = 1,wsem = 1;
void reader()
{
    while (true){
     semWait (x);
     readcount++;
     if(readcount == 1)
         semWait (wsem);
     semSignal (x);
     READUNIT();
     semWait (x);
     readcount--;
     if(readcount == 0)
         semSignal (wsem);
     semSignal (x);
     }
}
void writer()
{
      while (true){
       semWait (wsem);
       WRITEUNIT();
       semSignal (wsem);
      }
}
void  main()
{
     readcount = 0;
     parbegin (reader,writer);
}
```



VideoNote **Figure 5.25 A Solution to the Readers/Writers Problem Using Semaphore: Readers Have Priority**

As long as one writer is accessing the shared data area, no other writers and no readers may access it. The reader process also makes use of `wsem` to enforce mutual exclusion. However, to allow multiple readers, we require that, when there are no readers reading, the first reader that attempts to read should wait on `wsem`. When there is already at least one reader reading, subsequent readers need not wait before entering. The global variable `readcount` is used to keep track of the number of readers, and the semaphore `x` is used to assure that `readcount` is updated properly.

### Writers Have Priority

In the previous solution, readers have priority. Once a single reader has begun to access the data area, it is possible for readers to retain control of the data area as long as there is at least one reader in the act of reading. Therefore, writers are subject to starvation.

Figure 5.26 shows a solution that guarantees no new readers are allowed access to the data area once at least one writer has declared a desire to write. For writers, the following semaphores and variables are added to the ones already defined:

- A semaphore `rsem` that inhibits all readers while there is at least one writer desiring access to the data area
- A variable `writecount` that controls the setting of `rsem`
- A semaphore `y` that controls the updating of `writecount`

For readers, one additional semaphore is needed. A long queue must not be allowed to build up on `rsem`; otherwise writers will not be able to jump the queue. Therefore, only one reader is allowed to queue on `rsem`, with any additional readers queueing on semaphore `z`, immediately before waiting on `rsem`. Table 5.6 summarizes the possibilities.

**Table 5.6** State of the Process Queues for Program of Figure 5.26

| | |
|---|---|
| Readers only in the system | • *wsem* set<br>• no queues |
| Writers only in the system | • *wsem* and *rsem* set<br>• writers queue on *wsem* |
| Both readers and writers with read first | • *wsem* set by reader<br>• *rsem* set by writer<br>• all writers queue on *wsem*<br>• one reader queues on *rsem*<br>• other readers queue on *z* |
| Both readers and writers with write first | • *wsem* set by writer<br>• *rsem* set by writer<br>• writers queue on *wsem*<br>• one reader queues on *rsem*<br>• other readers queue on *z* |

```
/* program readersandwriters */
int readcount,writecount; semaphore x = 1, y = 1, z = 1, wsem = 1, rsem = 1;
void reader()
{
   while (true){
     semWait (z);
          semWait (rsem);
               semWait (x);
                    readcount++;
                    if (readcount == 1)
                         semWait (wsem);
                    semSignal (x);
               semSignal (rsem);
          semSignal (z);
          READUNIT();
          semWait (x);
               readcount--;
               if (readcount == 0) semSignal (wsem);
          semSignal (x);
   }
}
void writer ()
{
   while (true){
     semWait (y);
          writecount++;
          if (writecount == 1)
               semWait (rsem);
     semSignal (y);
     semWait (wsem);
     WRITEUNIT();
     semSignal (wsem);
     semWait (y);
          writecount--;
          if (writecount == 0) semSignal (rsem);
     semSignal (y);
   }
}
void main()
{
   readcount = writecount = 0;
   parbegin (reader, writer);
}
```

VideoNote

**Figure 5.26 A Solution to the Readers/Writers Problem Using Semaphore: Writers Have Priority**

An alternative solution, which gives writers priority and which is implemented using message passing, is shown in Figure 5.27. In this case, there is a controller process that has access to the shared data area. Other processes wishing to access the data area send a request message to the controller, are granted access with an "OK" reply message, and indicate completion of access with a "finished" message. The controller is equipped with three mailboxes, one for each type of message that it may receive.

The controller process services write request messages before read request messages to give writers priority. In addition, mutual exclusion must be enforced. To do

```
void reader(int i)
{
    message rmsg;
        while (true) {
                rmsg = i;
            send (readrequest, rmsg);
            receive (mbox[i], rmsg);
            READUNIT ();
            rmsg = i;
            send (finished, rmsg);
        }
}
void writer(int j)
{
    message rmsg;
    while (true){
        rmsg = j;
        send (writerequest, rmsg);
        receive (mbox[j], rmsg);
        WRITEUNIT ();
        rmsg = j;
        send (finished, rmsg);
    }
}
```

```
void controller()
{
    while (true)
    {
        if (count > 0) {
            if (!empty (finished)) {
                receive (finished, msg);
                count++;
            }
            else if (!empty (writerequest)) {
                receive (writerequest, msg);
                writer_id = msg.id;
                count = count - 100;
            }
            else if (!empty (readrequest)) {
                receive (readrequest, msg);
                count--;
                send (msg.id, "OK");
            }
        }
        if (count == 0) {
            send (writer_id, "OK");
            receive (finished, msg);
            count = 100;
        }
        while (count < 0) {
            receive (finished, msg);
            count++;
        }
    }
}
```

VideoNote **Figure 5.27 A Solution to the Readers/Writers Problem Using Message Passing**

this the variable *count* is used, which is initialized to some number greater than the maximum possible number of readers. In this example, we use a value of 100. The action of the controller can be summarized as follows:

- If *count* $>$ 0, then no writer is waiting and there may or may not be readers active. Service all "finished" messages first to clear active readers. Then service write requests, and then read requests.
- If *count* $=$ 0, then the only request outstanding is a write request. Allow the writer to proceed and wait for a "finished" message.
- If *count* $<$ 0, then a writer has made a request and is being made to wait to clear all active readers. Therefore, only "finished" messages should be serviced.

## 5.8 SUMMARY

The central themes of modern operating systems are multiprogramming, multiprocessing, and distributed processing. Fundamental to these themes, and fundamental to the technology of OS design, is concurrency. When multiple processes

are executing concurrently, either actually in the case of a multiprocessor system or virtually in the case of a single-processor multiprogramming system, issues of conflict resolution and cooperation arise.

Concurrent processes may interact in a number of ways. Processes that are unaware of each other may nevertheless compete for resources, such as processor time or access to I/O devices. Processes may be indirectly aware of one another because they share access to a common object, such as a block of main memory or a file. Finally, processes may be directly aware of each other and cooperate by the exchange of information. The key issues that arise in these interactions are mutual exclusion and deadlock.

Mutual exclusion is a condition in which there is a set of concurrent processes, only one of which is able to access a given resource or perform a given function at any time. Mutual exclusion techniques can be used to resolve conflicts, such as competition for resources, and to synchronize processes so they can cooperate. An example of the latter is the producer/consumer model, in which one process is putting data into a buffer, and one or more processes are extracting data from that buffer.

One approach to supporting mutual exclusion involves the use of special-purpose machine instructions. This approach reduces overhead, but is still inefficient because it uses busy waiting.

Another approach to supporting mutual exclusion is to provide features within the OS. Two of the most common techniques are semaphores and message facilities. Semaphores are used for signaling among processes and can be readily used to enforce a mutual exclusion discipline. Messages are useful for the enforcement of mutual exclusion and also provide an effective means of interprocess communication.

## 5.9 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

### Key Terms

| | | |
|---|---|---|
| atomic | critical resource | mutual exclusion |
| binary semaphore | critical section | mutual exclusion lock (mutex) |
| blocking | deadlock | nonblocking |
| busy waiting | direct addressing | race condition |
| concurrency | general semaphore | semaphore |
| concurrent processes | indirect addressing | spin waiting |
| condition variable | livelock | starvation |
| coroutine | message passing | strong semaphore |
| counting semaphore | monitor | weak semaphore |

### Review Questions

**5.1.** List four design issues for which the concept of concurrency is relevant.
**5.2.** What are three contexts in which concurrency arises?
**5.3.** What is a race condition?
**5.4.** List three degrees of awareness between processes and briefly define each.

**5.5.** What is the distinction between competing processes and cooperating processes?
**5.6.** List the three control problems associated with competing processes, and briefly define each.
**5.7.** What is starvation with respect to concurrency control by mutual exclusion?
**5.8.** What operations can be performed on a semaphore?
**5.9.** What is the difference between binary and general semaphores?
**5.10.** What is the key difference between a mutex and a binary semaphore?
**5.11.** Which characteristics of monitors mark them as high-level synchronization tools?
**5.12.** Compare direct and indirect addressing with respect to message passing.
**5.13.** What conditions are generally associated with the readers/writers problem?

## Problems

**5.1.** Demonstrate the correctness of Dekker's algorithm.

**a.** Show that mutual exclusion is enforced. *Hint:* Show that when Pi enters its critical section, the following expression is true:

```
flag[i] and ( not flag[1 - i] )
```

**b.** Show that a process requiring access to its critical section will not be delayed indefinitely. *Hint:* Consider the following cases: (1) a single process is attempting to enter the critical section; (2) both processes are attempting to enter the critical section, and (2a) `turn = 0` and `flag[0] = false`, and (2b) `turn = 0` and `flag[0] = true`.

**5.2.** Consider Dekker's algorithm written for an arbitrary number of processes by changing the statement executed when leaving the critical section from

```
turn = 1 - i        /* i.e. P0 sets turn to 1 and P1 sets turn
to 0 */
to
turn = (turn + 1) % n    /* n = number of processes */
```

Evaluate the algorithm when the number of concurrently executing processes is greater than two.

**5.3.** Demonstrate that the following software approaches to mutual exclusion do not depend on elementary mutual exclusion at the memory access level:

**a.** The bakery algorithm.
**b.** Peterson's algorithm.

**5.4.** With respect to mutual exclusion using interrupt disabling

**a.** Mention the requirements for this exclusion and state which of them are met when interrupts are disabled.
**b.** Identify the problems associated with this mechanism.

**5.5.** Processes and threads provide a powerful structuring tool for implementing programs that would be much more complex as simple sequential programs. An earlier construct that is instructive to examine is the coroutine. The purpose of this problem is to introduce coroutines and compare them to processes. Consider this simple problem from [CONW63]:

Read 80-column cards and print them on 125-character lines, with the following changes. After every card image an extra blank is inserted, and every adjacent pair of asterisks (**) on a card is replaced by the character(↑).

**a.** Develop a solution to this problem as an ordinary sequential program. You will find that the program is tricky to write. The interactions among the various elements

```
char rs, sp;
char inbuf[80], outbuf[125] ;
void read()
{
  while (true) {
     READCARD (inbuf);
     for (int  i=0; i < 80; i++){
          rs = inbuf [i];
          RESUME squash
     }
     rs = " ";
     RESUME squash;
  }
}
void print()
{
  while (true) {
     for (int  j = 0; j < 125; j++)
          outbuf [j] = sp;
          RESUME squash
     }
     OUTPUT (outbuf);
  }
}
```

```
void squash()
{
   while (true)  {
      if (rs != "*") {
           sp = rs;
           RESUME print;
      }
      else {
       RESUME read;
        if  (rs == "*") {
           sp = " ";
           RESUME print;
      }
        else {
           sp = "*";
           RESUME print;
           sp = rs;
           RESUME print;
      }
    }
      RESUME read;
  }
}
```

VideoNote **Figure 5.28 An Application of Coroutines**

of the program are uneven because of the conversion from a length of 80 to 125; furthermore, the length of the card image, after conversion, will vary depending on the number of double asterisk occurrences. One way to improve clarity, and to minimize the potential for bugs, is to write the application as three separate procedures. The first procedure reads in card images, pads each image with a blank, and writes a stream of characters to a temporary file. After all of the cards have been read, the second procedure reads the temporary file, does the character substitution, and writes out a second temporary file. The third procedure reads the stream of characters from the second temporary file and prints lines of 125 characters each.

**b.** The sequential solution is unattractive because of the overhead of I/O and temporary files. Conway proposed a new form of program structure, the coroutine, that allows the application to be written as three programs connected by one-character buffers (see Figure 5.28). In a traditional **procedure**, there is a master/slave relationship between the called and calling procedures. The calling procedure may execute a call from any point in the procedure; the called procedure is begun at its entry point and returns to the calling procedure at the point of call. The **coroutine** exhibits a more symmetric relationship. As each call is made, execution takes up from the last active point in the called procedure. Because there is no sense in which a calling procedure is "higher" than the called, there is no return. Rather, any coroutine can pass control to any other coroutine with a resume command. The first time a coroutine is invoked, it is "resumed" at its entry point. Subsequently, the coroutine is reactivated at the point of its own last resume command. Note only one coroutine in a program can be in execution at one time, and the transition points are explicitly defined in the code, so this is not an example of concurrent processing. Explain the operation of the program in Figure 5.28.

**c.** The program does not address the termination condition. Assume that the I/O routine READCARD returns the value true if it has placed an 80-character image in *inbuf*; otherwise it returns false. Modify the program to include this contingency. Note the last printed line may therefore contain less than 125 characters.

**d.** Rewrite the solution as a set of three processes using semaphores.

**5.6.** Consider the following processes P1 and P2 that update the value of the shared variables, x and y, as follows:

```
Process P1 :                    Process P2 :
(  performs the operations:     (  performs the operations:
      x := x * y                      x ++
      y ++                            y := x * y
)                               )
LOAD R1, X                      LOAD R3, X
LOAD R2, Y                      INC R3
MUL R1, R2                      LOAD R4, Y
STORE X, R1                     MUL R4, R3
INC R2                          STORE X, R3
STORE Y, R2                     STORE Y, R4
```

Assume that the initial values of x and y are 2 and 3 respectively. P1 enters the system first and so it is required that the output is equivalent to a serial execution of P1 followed by P2. The scheduler in the uniprocessor system implements a pseudo-parallel execution of these two concurrent processes by interleaving their instructions without restricting the order of the interleaving.

**a.** If the processes P1 and P2 had executed serially, what would the values of x and y have been after the execution of both processes?

**b.** Write an interleaved concurrent schedule that gives the same output as a serial schedule.

**c.** Write an interleaved concurrent schedule that gives an output that is different from that of a serial schedule.

**5.7.** Consider the following program:

```
const int n = 50;
int tally;
void total()
{
    int count;
    for (count = 1; count<= n; count++) {
        tally++;
    }
}
 void main()
{
    tally = 0;
    parbegin (total (), total ());
    write (tally);
}
```

**a.** Determine the proper lower bound and upper bound on the final value of the shared variable *tally* output by this concurrent program. Assume processes can execute at any relative speed, and a value can only be incremented after it has been loaded into a register by a separate machine instruction.

**b.** Suppose that an arbitrary number of these processes are permitted to execute in parallel under the assumptions of part (a). What effect will this modification have on the range of final values of *tally*?

**5.8.** In Table 5.3, the use of spinlocks for concurrency control has been identified. Give an implementation for mutual exclusion using spinlocks.

**5.9.** Consider the following program:

```
boolean blocked [2];
int turn;
void P (int id)
{
  while (true) {
      blocked[id] = true;
      while (turn != id) {
          while (blocked[1-id])
              /* do nothing */;
          turn = id;
      }
      /* critical section */
      blocked[id] = false;
      /* remainder */
  }
}
void main()
{
  blocked[0] = false;
  blocked[1] = false;
  turn = 0;
  parbegin (P(0), P(1));
}
```

This software solution to the mutual exclusion problem for two processes is proposed in [HYMA66]. Find a counterexample that demonstrates that this solution is incorrect. It is interesting to note that even the *Communications of the ACM* was fooled on this one.

**5.10.** A software approach to mutual exclusion is Lamport's **bakery algorithm** [LAMP74], so called because it is based on the practice in bakeries and other shops in which every customer receives a numbered ticket on arrival, allowing each to be served in turn. The algorithm is as follows:

```
boolean choosing[n];
int number[n];
while (true) {
    choosing[i] = true;
    number[i] = 1 + getmax(number[], n);
    choosing[i] = false;
    for (int j = 0; j < n; j++){
      while (choosing[j]) { };
      while ((number[j] != 0) && (number[j],j) < (number[i],i))
              { } ;
      }
    /* critical section */;
    number [i] = 0;
    /* remainder */;
}
```

The arrays *choosing* and *number* are initialized to false and 0, respectively. The *i*th element of each array may be read and written by process *i* but only read by other processes. The notation $(a, b) < (c, d)$ is defined as:

$$(a < c) \text{ or } (a = c \text{ and } b < d)$$

**a.** Describe the algorithm in words.

**b.** Show that this algorithm avoids deadlock.

**c.** Show that it enforces mutual exclusion.

**5.11.** Now consider a version of the bakery algorithm without the variable choosing. Then we have

```
1 int number[n];
2 while (true) {
3     number[i] = 1 + getmax(number[], n);
4     for (int  j = 0; j < n; j++){
5       while ((number[j] != 0) && (number[j],j) <
                (number[i],i)) { };
6       }
7     /* critical section */;
8     number [i] = 0;
9     /* remainder */;
10 }
```

Does this version violate mutual exclusion? Explain why or why not.

**5.12.** Consider the following program which provides a software approach to mutual exclusion:

**integer array** control [1 :N]; **integer** k

where 1 ≤ k ≤ N, and each element of "control" is either 0, 1, or 2. All elements of "control" are initially zero; the initial value of k is immaterial.

The program of the *i*th process (1 ≤ *i* ≤ N) is

```
begin integer  j;
L0: control [i] := 1;
LI:  for  j:=k  step  1  until  N, 1  step  1  until  k  do
       begin
         if  j = i  then goto  L2;
         if  control [j] ≠ 0  then goto  L1
       end;
L2: control [i] := 2;
     for  j := 1  step  1  until  N do
      if  j ≠ i  and  control [j] = 2  then goto  L0;
L3:  if  control [k] ≠ 0 and k ≠ i  then goto  L0;
L4: k := i;
    critical section;
L5:  for  j := k  step  1  until  N, 1  step  1  until  k  do
     if  j ≠ k  and  control [j] ≠ 0  then
      begin
         k := j;
            goto  L6
      end;
```

```
L6: control [i] := 0;
L7:  remainder of cycle;
     goto  L0;
end
```

This is referred to as the Eisenberg-McGuire algorithm. Explain its operation and its key features.

**5.13.** Consider the mutual exclusion protocol in Figure 5.5b. How can you modify it so that it satisfies the bounded waiting requirement as well?

**5.14.** When a special machine instruction is used to provide mutual exclusion in the fashion of Figure 5.5, there is no control over how long a process must wait before being granted access to its critical section. Devise an algorithm that uses the `compare&swap` instruction, but that guarantees that any process waiting to enter its critical section will do so within $n - 1$ turns, where $n$ is the number of processes that may require access to the critical section, and a "turn" is an event consisting of one process leaving the critical section and another process being granted access.

**5.15.** Consider the following definition of semaphores:

```
void semWait(s)
{
    if  (s.count > 0){
       s.count--;
}
    else  {
      place this process in s.queue;
      block;
    }
}
 void  semSignal (s)
{
    if  (there is at least one process blocked on
        semaphore s) {
        remove a process P from s.queue;
        place process P on ready list;
    }
    else         s.count++;
}
```

Compare this set of definitions with that of Figure 5.6. Note one difference: With the preceding definition, a semaphore can never take on a negative value. Is there any difference in the effect of the two sets of definitions when used in programs? That is, could you substitute one set for the other without altering the meaning of the program?

**5.16.** Consider a sharable resource with the following characteristics: (1) As long as there are fewer than three processes using the resource, new processes can start using it right away. (2) Once there are three process using the resource, all three must leave before any new processes can begin using it. We realize that counters are needed to keep track of how many processes are waiting and active, and that these counters are themselves

shared resources that must be protected with mutual exclusion. So we might create the following solution:

```
1   semaphore mutex = 1, block = 0;          /* share variables: semaphores, */
2   int active = 0, waiting = 0;                            /* counters, and */
3   boolean  must_wait = false;                           /* state information */
4
5   semWait(mutex);                              /* Enter the mutual exclusion */
6   if(must_wait)  {                          /* If there are (or were) 3, then */
7       ++waiting;                           /* we must wait, but we must leave */
8       semSignal(mutex);                       /* the mutual exclusion first */
9       semWait(block);               /* Wait for all current users to depart */
10      SemWait(mutex);                       /* Reenter the mutual exclusion */
11      --waiting;                            /* and update the waiting count */
12  }
13  ++active;                              /* Update active count, and remember */
14  must_wait = active == 3;                          /* if the count reached 3 */
15  semSignal(mutex);                          /* Leave the mutual exclusion */
16
17  /* critical section */
18
19  semWait(mutex);                                   /* Enter mutual exclusion */
20  --active;                                  /* and update the active count */
21  if(active == 0) {                                    /* Last one to leave? */
22      int  n;
23      if  (waiting < 3) n = waiting;
24      else  n = 3;                                  /* If so, unblock up to 3 */
25      while( n > 0 ) {                                  /* waiting processes */
26          semSignal(block);
27          --n;
28      }
29  must_wait = false;                      /* All active processes have left */
30  }
31  semSignal(mutex);                          /* Leave the mutual exclusion */
```

The solution appears to do everything right: All accesses to the shared variables are protected by mutual exclusion, processes do not block themselves while in the mutual exclusion, new processes are prevented from using the resource if there are (or were) three active users, and the last process to depart unblocks up to three waiting processes.

**a.** The program is nevertheless incorrect. Explain why.

**b.** Suppose we change the **if** in line 6 to a **while**. Does this solve any problem in the program? Do any difficulties remain?

**5.17.** Now consider this correct solution to the preceding problem:

```
1   semaphore  mutex = 1, block = 0;          /* share variables: semaphores, */
2   int  active = 0, waiting = 0;                           /* counters, and */
3   boolean  must_wait = false;                           /* state information */
4
5   semWait(mutex);                              /* Enter the mutual exclusion */
6   if(must_wait) {                           /* If there are (or were) 3, then */
7       ++waiting;                           /* we must wait, but we must leave */
8       semSignal(mutex);                       /* the mutual exclusion first */
9       semWait(block);               /* Wait for all current users to depart */
10  }    else  {
11      ++active;                                  /* Update active count, and */
11  }
12      must_wait = active == 3;           /* remember if the count reached 3 */
13      semSignal(mutex);                          /* Leave mutual exclusion */
```

```
}

/* critical section */

semWait(mutex);                                  /* Enter mutual exclusion */
--active;                                      /* and update the active count */
if(active == 0) {                                      /* Last one to leave? */
    int  n;
    if  (waiting < 3) n = waiting;
    else  n = 3;              /* If so, see how many processes to unblock */
    waiting -= n;               /* Deduct this number from waiting count */
    active = n;                           /* and set active to this number */
    while( n > 0 ) {                        /* Now unblock the processes */
       semSignal(block);                                    /* one by one */
       --n;
    }
    must_wait = active == 3;               /* Remember if the count is 3 */
}
semSignal(mutex);                          /* Leave the mutual exclusion */
```

**a.** Explain how this program works and why it is correct.

**b.** This solution does not completely prevent newly arriving processes from cutting in line but it does make it less likely. Give an example of cutting in line.

**c.** This program is an example of a general design pattern that is a uniform way to implement solutions to many concurrency problems using semaphores. It has been referred to as the **I'll Do It For You** pattern. Describe the pattern.

**5.18.** Now consider another correct solution to the preceding problem:

```
semaphore  mutex = 1, block = 0;         /* share variables: semaphores, */
int  active = 0, waiting = 0;                          /* counters, and */
boolean  must_wait = false;                          /* state information */

semWait(mutex);                              /* Enter the mutual exclusion */
if(must_wait)  {                          /* If there are (or were) 3, then */
    ++waiting;                           /* we must wait, but we must leave */
    semSignal(mutex);                        /* the mutual exclusion first */
    semWait(block);                /* Wait for all current users to depart */
    --waiting;           /* We've got the mutual exclusion; update count */
}
++active;                              /* Update active count, and remember */
must_wait = active == 3;                          /* if the count reached 3 */
if(waiting > 0 && !must_wait)               /* If there are others waiting */
    semSignal(block);;                   /* and we don't yet have 3 active, */
                                                /* unblock a waiting process */
else  semSignal(mutex);            /* otherwise open the mutual exclusion */

/* critical section */

semWait(mutex);                                  /* Enter mutual exclusion */
--active;                                      /* and update the active count */
if(active == 0)                                    /* If last one to leave? */
    must_wait = false;                /* set up to let new processes enter */
if(waiting == 0 && !must_wait)               /* If there are others waiting */
    semSignal(block);;                       /* and we don't have 3 active, */
                                              /* unblock a waiting process */
else  semSignal(mutex);            /* otherwise open the mutual exclusion */
```

**a.** Explain how this program works and why it is correct.

**b.** Does this solution differ from the preceding one in terms of the number of processes that can be unblocked at a time? Explain.

**c.** This program is an example of a general design pattern that is a uniform way to implement solutions to many concurrency problems using semaphores. It has been referred to as the **Pass The Baton** pattern. Describe the pattern.

**5.19.** It should be possible to implement general semaphores using binary semaphores. We can use the operations `semWaitB` and `semSignalB` and two binary semaphores, `delay` and `mutex`. Consider the following:

```
void semWait(semaphore s)
{
      semWaitB(mutex);
      s--;
      if (s < 0) {
            semSignalB(mutex);
            semWaitB(delay);

      }
      else SemsignalB(mutex);
}
 void semSignal(semaphore s);
{
      semWaitB(mutex);
      s++;
      if (s <= 0)
            semSignalB(delay);
      semSignalB(mutex);
}
```

Initially, `s` is set to the desired semaphore value. Each `semWait` operation decrements `s`, and each `semSignal` operation increments `s`. The binary semaphore mutex, which is initialized to 1, assures that there is mutual exclusion for the updating of `s`. The binary semaphore delay, which is initialized to 0, is used to block processes.

There is a flaw in the preceding program. Demonstrate the flaw and propose a change that will fix it. *Hint:* Suppose two processes each call `semWait(s)` when `s` is initially 0, and after the first has just performed `semSignalB(mutex)` but not performed `semWaitB(delay)`, the second call to `semWait(s)` proceeds to the same point. All that you need to do is move a single line of the program.

**5.20.** In 1978, Dijkstra put forward the conjecture that there was no solution to the mutual exclusion problem avoiding starvation, applicable to an unknown but finite number of processes, using a finite number of weak semaphores. In 1979, J. M. Morris refuted this conjecture by publishing an algorithm using three weak semaphores. The behavior of the algorithm can be described as follows: If one or several process are waiting in a `semWait(S)` operation and another process is executing `semSignal(S)`, the value of the semaphore `S` is not modified and one of the waiting processes is unblocked independently of `semWait(S)`. Apart from the three semaphores, the algorithm uses two nonnegative integer variables as counters of the number of processes in certain sections of the algorithm. Thus, semaphores A and B are initialized to 1, while semaphore M and counters NA and NM are initialized to 0. The mutual exclusion semaphore B protects access to the shared variable NA. A process attempting to enter its critical section must cross two barriers represented by semaphores A and M. Counters NA and NM, respectively, contain the number of processes ready to cross barrier A, and

those having already crossed barrier A but not yet barrier M. In the second part of the protocol, the NM processes blocked at M will enter their critical sections one by one, using a cascade technique similar to that used in the first part. Define an algorithm that conforms to this description.

**5.21.** The following problem was once used on an exam:

> Jurassic Park consists of a dinosaur museum and a park for safari riding. There are *m* passengers and *n* single-passenger cars. Passengers wander around the museum for a while, then line up to take a ride in a safari car. When a car is available, it loads the one passenger it can hold and rides around the park for a random amount of time. If the *n* cars are all out riding passengers around, then a passenger who wants to ride waits; if a car is ready to load but there are no waiting passengers, then the car waits. Use semaphores to synchronize the *m* passenger processes and the *n* car processes.

The following skeleton code was found on a scrap of paper on the floor of the exam room. Grade it for correctness. Ignore syntax and missing variable declarations. Remember that `P` and `V` correspond to `semWait` and `semSignal`.

```
resource Jurassic_Park()
  sem car_avail := 0, car_taken := 0, car_filled := 0,
       passenger_released := 0
  process passenger(i := 1 to num_passengers)
  do true -> nap(int(random(1000*wander_time)))
   P(car_avail); V(car_taken); P(car_filled)
   P(passenger_released)
  od
end passenger
process car(j := 1 to num_cars)
 do true -> V(car_avail); P(car_taken); V(car_filled)
  nap(int(random(1000*ride_time)))
  V(passenger_released)
  od
 end car
end Jurassic_Park
```

**5.22.** In the commentary on Figure 5.12 and Table 5.4, it was stated that "it would not do simply to move the conditional statement inside the critical section (controlled by s) of the consumer because this could lead to deadlock." Demonstrate this with a table similar to Table 5.4.

**5.23.** Consider the solution to the infinite-buffer producer/consumer problem defined in Figure 5.13. Suppose we have the (common) case in which the producer and consumer are running at roughly the same speed. The scenario could be:

> Producer: append; `semSignal`; produce; . . . ; append; `semSignal`; produce; . . .
> Consumer: consume; . . . ; take; `semWait`; consume; . . . ; take; `semWait`; . . .
>
> The producer always manages to append a new element to the buffer and signal during the consumption of the previous element by the consumer. The producer is always appending to an empty buffer and the consumer is always taking the sole item in the buffer. Although the consumer never blocks on the semaphore, a large number of calls to the semaphore mechanism is made, creating considerable overhead.

Construct a new program that will be more efficient under these circumstances. *Hints:* Allow *n* to have the value −1, which is to mean that not only is the buffer empty but

that the consumer has detected this fact and is going to block until the producer supplies fresh data. The solution does not require the use of the local variable *m* found in Figure 5.13.

**5.24.** Consider Figure 5.16. Would the meaning of the program change if the following were interchanged?

**a.** `semWait(e);semWait(s)`

**b.** `semSignal(s);semSignal(n)`

**c.** `semWait(n);semWait(s)`

**d.** `semSignal(s);semSignal(e)`

**5.25.** The following pseudocode is a correct implementation of the producer/consumer problem with a bounded buffer:

```
item[3] buffer; // initially empty
semaphore empty; // initialized to +3
semaphore full; // initialized to 0
binary_semaphore mutex; // initialized to 1
```

```
void producer()
{
     ...
     while (true) {
        item = produce();
p1:     wait(empty);
  /     wait(mutex);
p2:     append(item);
  \     signal(mutex);
p3:     signal(full);
     }
}
```

```
void consumer()
{
     ...
     while (true) {
c1:     wait(full);
  /     wait(mutex);
c2:     item = take();
  \     signal(mutex);
c3:     signal(empty);
        consume(item);
     }
}
```

Labels p1, p2, p3 and c1, c2, c3 refer to the lines of code shown above (p2 and c2 each cover three lines of code). Semaphores empty and full are linear semaphores that can take unbounded negative and positive values. There are multiple producer processes, referred to as Pa, Pb, Pc, etc., and multiple consumer processes, referred to as Ca, Cb, Cc, etc. Each semaphore maintains a FIFO (first-in-first-out) queue of blocked processes. In the scheduling chart below, each line represents the state of the buffer and semaphores after the scheduled execution has occurred. To simplify, we assume that scheduling is such that processes are never interrupted while executing a given portion of code p1, or p2, . . . , or c3. Your task is to complete the following chart.

| Scheduled Step of Execution | full's State and Queue | Buffer | empty's State and Queue |
|---|---|---|---|
| Initialization | full = 0 | OOO | empty = +3 |
| Ca executes c1 | full = −1 (Ca) | OOO | empty = +3 |
| Cb executes c1 | full = −2 (Ca, Cb) | OOO | empty = +3 |
| Pa executes p1 | full = −2 (Ca, Cb) | OOO | empty = +2 |
| Pa executes p2 | full = −2 (Ca, Cb) | X OO | empty = +2 |
| Pa executes p3 | full = −1 (Cb) Ca | X OO | empty = +2 |
| Ca executes c2 | full = −1 (Cb) | OOO | empty = +2 |
| Ca executes c3 | full = −1 (Cb) | OOO | empty = +3 |

| Scheduled Step of Execution | full's State and Queue | Buffer | empty's State and Queue |
|---|---|---|---|
| Pb executes p1 | full = | | empty = |
| Pa executes p1 | full = | | empty = |
| Pa executes __ | full = | | empty = |
| Pb executes __ | full = | | empty = |
| Pb executes __ | full = | | empty = |
| Pc executes p1 | full = | | empty = |
| Cb executes __ | full = | | empty = |
| Pc executes __ | full = | | empty = |
| Cb executes __ | full = | | empty = |
| Pa executes __ | full = | | empty = |
| Pb executes p1-p3 | full = | | empty = |
| Pc executes __ | full = | | empty = |
| Pa executes p1 | full = | | empty = |
| Pd executes p1 | full = | | empty = |
| Ca executes c1-c3 | full = | | empty = |
| Pa executes __ | full = | | empty = |
| Cc executes c1-c2 | full = | | empty = |
| Pa executes __ | full = | | empty = |
| Cc executes c3 | full = | | empty = |
| Pd executes p2-p3 | full = | | empty = |

**5.26.** This problem demonstrates the use of semaphores to coordinate three types of processes.[7] Santa Claus sleeps in his shop at the North Pole and can only be awakened by either (1) all nine reindeer being back from their vacation in the South Pacific, or (2) some of the elves having difficulties making toys; to allow Santa to get some sleep, the elves can only wake him when three of them have problems. When three elves are having their problems solved, any other elves wishing to visit Santa must wait for those elves to return. If Santa wakes up to find three elves waiting at his shop's door, along with the last reindeer having come back from the tropics, Santa has decided that the elves can wait until after Christmas, because it is more important to get his sleigh ready. (It is assumed the reindeer do not want to leave the tropics, and therefore they stay there until the last possible moment.) The last reindeer to arrive must get Santa while the others wait in a warming hut before being harnessed to the sleigh. Solve this problem using semaphores.

**5.27.** Show that message passing and semaphores have equivalent functionality by

**a.** Implementing message passing using semaphores. *Hint:* Make use of a shared buffer area to hold mailboxes, each one consisting of an array of message slots.

**b.** Implementing a semaphore using message passing. *Hint:* Introduce a separate synchronization process.

[7] I am grateful to John Trono of St. Michael's College in Vermont for supplying this problem.

**5.28.** Explain what is the problem with this implementation of the one-writer many-readers problem?

```
int readcount;                          // shared and initialized to 0
Semaphore mutex, wrt;                   // shared and initialized to 1;

// Writer :                             // Readers :
                                        semWait(mutex);
                                        readcount := readcount + 1;
semWait(wrt);                           if readcount == 1 then semWait(wrt);
/* Writing performed*/                  semSignal(mutex);
semSignal(wrt);                         /*reading performed*/
                                        semWait(mutex);
                                        readcount := readcount - 1;
                                        if readcount == 0 then Up(wrt);
                                        semSignal(mutex);
```

CHAPTER 6

# Concurrency: Deadlock and Starvation


6.1 **Principles of Deadlock**
- Reusable Resources
- Consumable Resources
- Resource Allocation Graphs
- The Conditions for Deadlock

6.2 **Deadlock Prevention**
- Mutual Exclusion
- Hold and Wait
- No Preemption
- Circular Wait

6.3 **Deadlock Avoidance**
- Process Initiation Denial
- Resource Allocation Denial

6.4 **Deadlock Detection**
- Deadlock Detection Algorithm
- Recovery

6.5 **An Integrated Deadlock Strategy**

6.6 **Dining Philosophers Problem**
- Solution Using Semaphores
- Solution Using a Monitor

6.7 **UNIX Concurrency Mechanisms**
- Pipes
- Messages
- Shared Memory
- Semaphores
- Signals

6.8 **Linux Kernel Concurrency Mechanisms**
- Atomic Operations
- Spinlocks
- Semaphores
- Barriers

6.9 **Solaris Thread Synchronization Primitives**
- Mutual Exclusion Lock
- Semaphores
- Readers/Writer Lock
- Condition Variables

6.10 **Windows Concurrency Mechanisms**
- Wait Functions
- Dispatcher Objects
- Critical Sections
- Slim Reader–Writer Locks and Condition Variables
- Lock-free Synchronization

6.11 **Android Interprocess Communication**

6.12 **Summary**

6.13 **Key Terms, Review Questions, and Problems**

**Learning Objectives**

After studying this chapter, you should be able to:

- List and explain the conditions for deadlock.
- Define deadlock prevention and describe deadlock prevention strategies related to each of the conditions for deadlock.
- Explain the difference between deadlock prevention and deadlock avoidance.
- Understand two approaches to deadlock avoidance.
- Explain the fundamental difference in approach between deadlock detection and deadlock prevention or avoidance.
- Understand how an integrated deadlock strategy can be designed.
- Analyze the dining philosophers problem.
- Explain the concurrency and synchronization methods used in UNIX, Linux, Solaris, Windows, and Android.

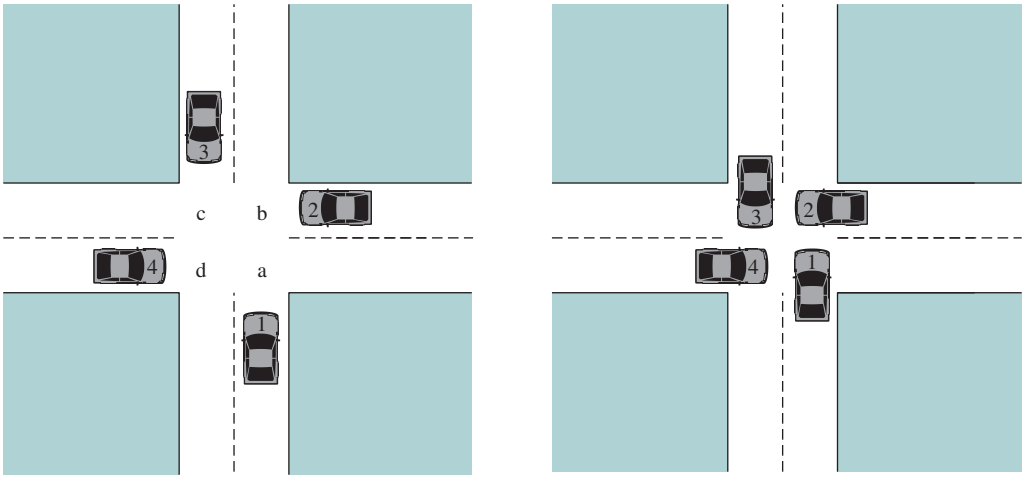
This chapter examines two problems that plague all efforts to support concurrent processing: deadlock and starvation. We begin with a discussion of the underlying principles of deadlock and the related problem of starvation. Then we will examine the three common approaches to dealing with deadlock: prevention, detection, and avoidance. We will then look at one of the classic problems used to illustrate both synchronization and deadlock issues: the dining philosophers problem.

As with Chapter 5, the discussion in this chapter is limited to a consideration of concurrency and deadlock on a single system. Measures to deal with distributed deadlock problems will be assessed in Chapter 18. An animation illustrating deadlock is available at the Companion website for this book.

## 6.1 PRINCIPLES OF DEADLOCK

Deadlock can be defined as the *permanent* blocking of a set of processes that either compete for system resources or communicate with each other. A set of processes is deadlocked when each process in the set is blocked awaiting an event (typically the freeing up of some requested resource) that can only be triggered by another blocked process in the set. Deadlock is permanent because none of the events is ever triggered. Unlike other problems in concurrent process management, there is no efficient solution in the general case.

All deadlocks involve conflicting needs for resources by two or more processes. A common example is the traffic deadlock. Figure 6.1a shows a situation in which four cars have arrived at a four-way stop intersection at approximately the same time.




(a) Deadlock possible (b) Deadlock

**Figure 6.1 Illustration of Deadlock**

The four quadrants of the intersection are the resources over which control is needed. In particular, if all four cars wish to go straight through the intersection, the resource requirements are as follows:

- Car 1, traveling north, needs quadrants a and b.
- Car 2, traveling west, needs quadrants b and c.
- Car 3, traveling south, needs quadrants c and d.
- Car 4, traveling east, needs quadrants d and a.

The rule of the road in the United States is that a car at a four-way stop should defer to a car immediately to its right. This rule works if there are only two or three cars at the intersection. For example, if only the northbound and westbound cars arrive at the intersection, the northbound car will wait and the westbound car proceeds. However, if all four cars arrive at about the same time and all four follow the rule, each will refrain from entering the intersection. This causes a potential deadlock. It is only a potential deadlock, because the necessary resources are available for any of the cars to proceed. If one car eventually chooses to proceed, it can do so.

However, if all four cars ignore the rules and proceed (cautiously) into the intersection at the same time, then each car seizes one resource (one quadrant) but cannot proceed because the required second resource has already been seized by another car. This is an actual deadlock.

Let us now look at a depiction of deadlock involving processes and computer resources. Figure 6.2, which we refer to as a **joint progress diagram**, illustrates the progress of two processes competing for two resources. Each process needs exclusive

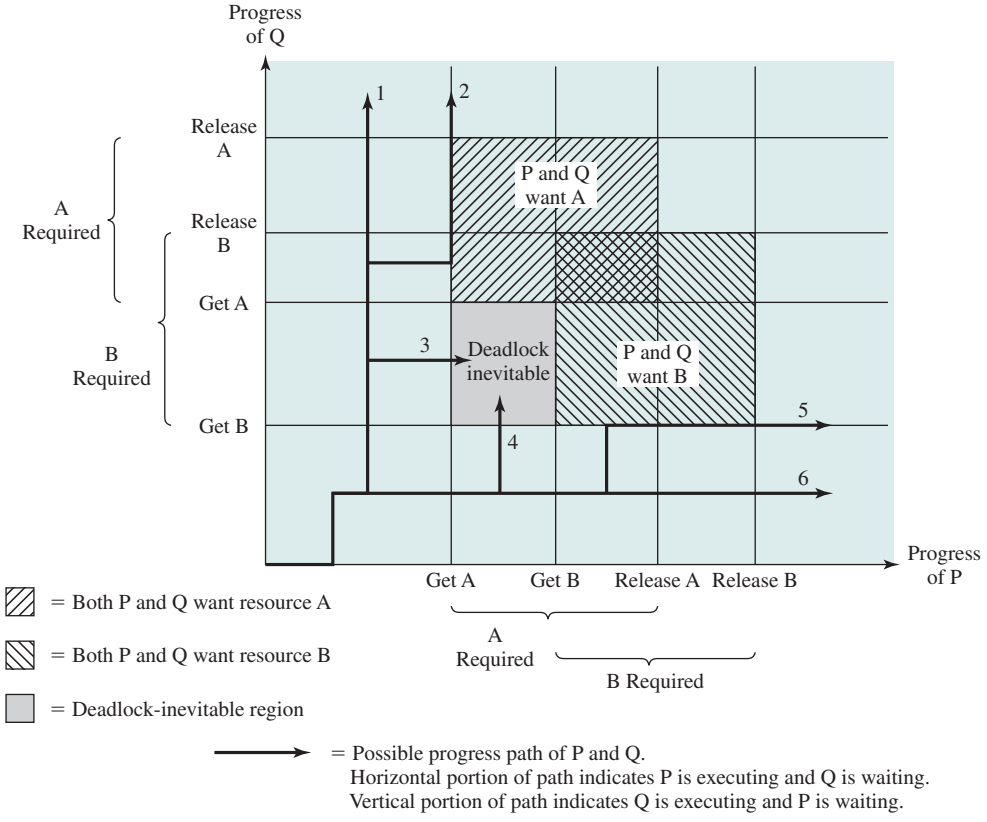



**Figure 6.2 Example of Deadlock**

use of both resources for a certain period of time. Two processes, P and Q, have the following general form:

| **Process P** | **Process Q** |
|---|---|
| • • • | • • • |
| Get A | Get B |
| • • • | • • • |
| Get B | Get A |
| • • • | • • • |
| Release A | Release B |
| • • • | • • • |
| Release B | Release A |
| • • • | • • • |

In Figure 6.2, the *x*-axis represents progress in the execution of P and the *y*-axis represents progress in the execution of Q. The joint progress of the two processes is therefore represented by a path that progresses from the origin in a northeasterly direction. For a uniprocessor system, only one process at a time may execute, and the path consists of alternating horizontal and vertical segments, with a horizontal

segment representing a period when P executes, and Q waits, and a vertical segment representing a period when Q executes and P waits. The figure indicates areas in which both P and Q require resource A (upward slanted lines); both P and Q require resource B (downward slanted lines); and both P and Q require both resources. Because we assume that each process requires exclusive control of any resource, these are all forbidden regions; that is, it is impossible for any path representing the joint execution progress of P and Q to enter these regions.

The figure shows six different execution paths. These can be summarized as follows:

1. Q acquires B then A, then releases B and A. When P resumes execution, it will be able to acquire both resources.
2. Q acquires B then A. P executes and blocks on a request for A. Q releases B and A. When P resumes execution, it will be able to acquire both resources.
3. Q acquires B then P acquires A. Deadlock is inevitable, because as execution proceeds, Q will block on A and P will block on B.
4. P acquires A then Q acquires B. Deadlock is inevitable, because as execution proceeds, Q will block on A and P will block on B.
5. P acquires A then B. Q executes and blocks on a request for B. P releases A and B. When Q resumes execution, it will be able to acquire both resources.
6. P acquires A then B, then releases A and B. When Q resumes execution, it will be able to acquire both resources.

The gray-shaded area of Figure 6.2, which can be referred to as a **fatal region**, applies to the commentary on paths 3 and 4. If an execution path enters this fatal region, then deadlock is inevitable. Note the existence of a fatal region depends on the logic of the two processes. However, deadlock is only inevitable if the joint progress of the two processes creates a path that enters the fatal region.

Whether or not deadlock occurs depends on both the dynamics of the execution and on the details of the application. For example, suppose P does not need both resources at the same time so the two processes have the following form:

| **Process P** | **Process Q** |
|---|---|
| • • • | • • • |
| Get A | Get B |
| • • • | • • • |
| Release A | Get A |
| • • • | • • • |
| Get B | Release B |
| • • • | • • • |
| Release B | Release A |
| • • • | • • • |

This situation is reflected in Figure 6.3. Some thought should convince you that regardless of the relative timing of the two processes, deadlock cannot occur.

As shown, the joint progress diagram can be used to record the execution history of two processes that share resources. In cases where more than two

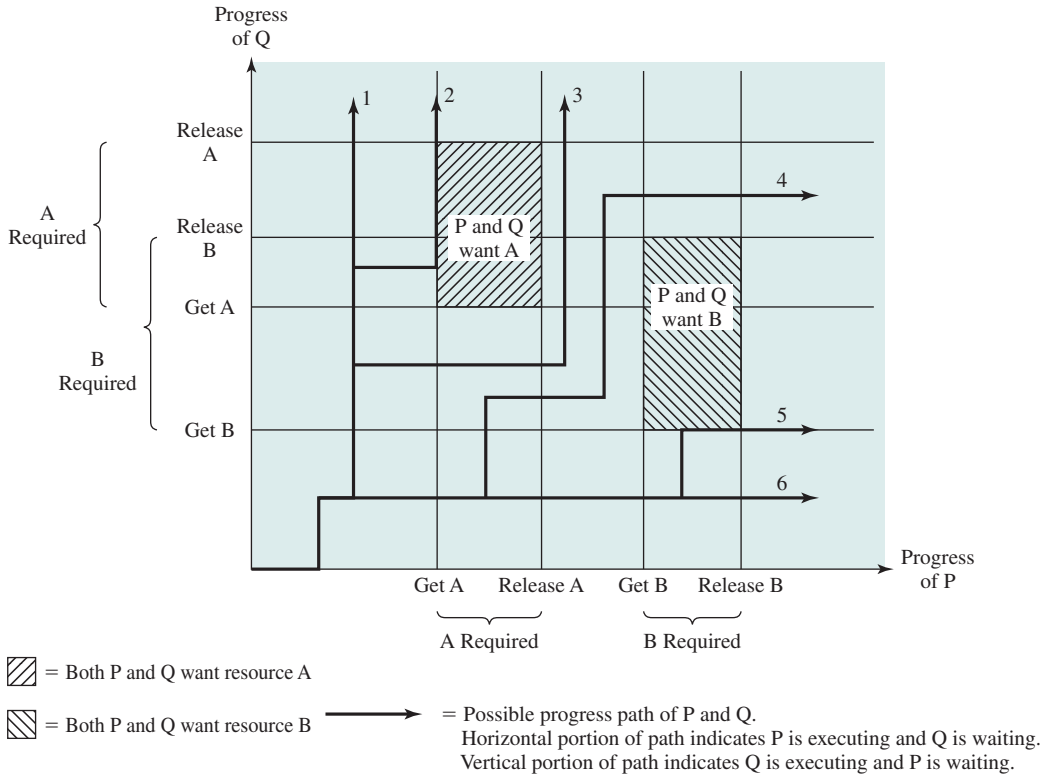



**Figure 6.3 Example of No Deadlock [BACO03]**

processes may compete for the same resource, a higher-dimensional diagram would be required. The principles concerning fatal regions and deadlock would remain the same.

## Reusable Resources

Two general categories of resources can be distinguished: reusable and consumable. A reusable resource is one that can be safely used by only one process at a time and is not depleted by that use. Processes obtain resource units that they later release for reuse by other processes. Examples of reusable resources include processors, I/O channels, main and secondary memory, devices, and data structures (such as files, databases, and semaphores).

As an example of deadlock involving reusable resources, consider two processes that compete for exclusive access to a disk file D and a tape drive T. The programs engage in the operations depicted in Figure 6.4. Deadlock occurs if each process holds one resource and requests the other. For example, deadlock occurs if the multiprogramming system interleaves the execution of the two processes as follows:

$$p_0\ p_1\ q_0\ q_1\ p_2\ q_2$$

| Step | Process P Action |
|---|---|
| $p_0$ | Request (D) |
| $p_1$ | Lock (D) |
| $p_2$ | Request (T) |
| $p_3$ | Lock (T) |
| $p_4$ | Perform function |
| $p_5$ | Unlock (D) |
| $p_6$ | Unlock (T) |

| Step | Process Q Action |
|---|---|
| $q_0$ | Request (T) |
| $q_1$ | Lock (T) |
| $q_2$ | Request (D) |
| $q_3$ | Lock (D) |
| $q_4$ | Perform function |
| $q_5$ | Unlock (T) |
| $q_6$ | Unlock (D) |

**Figure 6.4** **Example of Two Processes Competing for Reusable Resources**

It may appear that this is a programming error rather than a problem for the OS designer. However, we have seen that concurrent program design is challenging. Such deadlocks do occur, and the cause is often embedded in complex program logic, making detection difficult. One strategy for dealing with such a deadlock is to impose system design constraints concerning the order in which resources can be requested.

Another example of deadlock with a reusable resource has to do with requests for main memory. Suppose the space available for allocation is 200 Kbytes, and the following sequence of requests occurs:

| P1 | P2 |
|---|---|
| . . . | . . . |
| Request 80 Kbytes; | Request 70 Kbytes; |
| . . . | . . . |
| Request 60 Kbytes; | Request 80 Kbytes; |

Deadlock occurs if both processes progress to their second request. If the amount of memory to be requested is not known ahead of time, it is difficult to deal with this type of deadlock by means of system design constraints. The best way to deal with this particular problem is, in effect, to eliminate the possibility by using virtual memory, which will be discussed in Chapter 8.

## Consumable Resources

A consumable resource is one that can be created (produced) and destroyed (consumed). Typically, there is no limit on the number of consumable resources of a particular type. An unblocked producing process may create any number of such resources. When a resource is acquired by a consuming process, the resource ceases to exist. Examples of consumable resources are interrupts, signals, messages, and information in I/O buffers.

As an example of deadlock involving consumable resources, consider the following pair of processes, in which each process attempts to receive a message from the other process then send a message to the other process:

| P1 | P2 |
|---|---|
| . . . | . . . |
| Receive (P2); | Receive (P1); |
| . . . | . . . |
| Send (P2, M1); | Send (P1, M2); |

Deadlock occurs if the Receive is blocking (i.e., the receiving process is blocked until the message is received). Once again, a design error is the cause of the deadlock. Such errors may be quite subtle and difficult to detect. Furthermore, it may take a rare combination of events to cause the deadlock; thus a program could be in use for a considerable period of time, even years, before the deadlock actually occurs.

There is no single effective strategy that can deal with all types of deadlock. Three approaches are common:

- **Deadlock prevention:** Disallow one of the three necessary conditions for deadlock occurrence, or prevent circular wait condition from happening.
- **Deadlock avoidance:** Do not grant a resource request if this allocation might lead to deadlock.
- **Deadlock detection:** Grant resource requests when possible, but periodically check for the presence of deadlock and take action to recover.

We examine each of these in turn, after first introducing resource allocation graphs and then discussing the conditions for deadlock.

## Resource Allocation Graphs

A useful tool in characterizing the allocation of resources to processes is the **resource allocation graph**, introduced by Holt [HOLT72]. The resource allocation graph is a directed graph that depicts a state of the system of resources and processes, with each process and each resource represented by a node. A graph edge directed from a process to a resource indicates a resource that has been requested by the process but not yet granted (see Figure 6.5a). Within a resource node, a dot is shown for each instance of that resource. Examples of resource types that may have multiple instances are I/O devices that are allocated by a resource management module in the OS. A graph edge directed from a reusable resource node dot to a process indicates a request that has been granted (see Figure 6.5b); that is, the process has been assigned one unit of that resource. A graph edge directed from a consumable resource node dot to a process indicates the process is the producer of that resource.

Figure 6.5c shows an example deadlock. There is only one unit each of resources Ra and Rb. Process P1 holds Rb and requests Ra, while P2 holds Ra but requests Rb. Figure 6.5d has the same topology as Figure 6.5c, but there is no deadlock because multiple units of each resource are available.

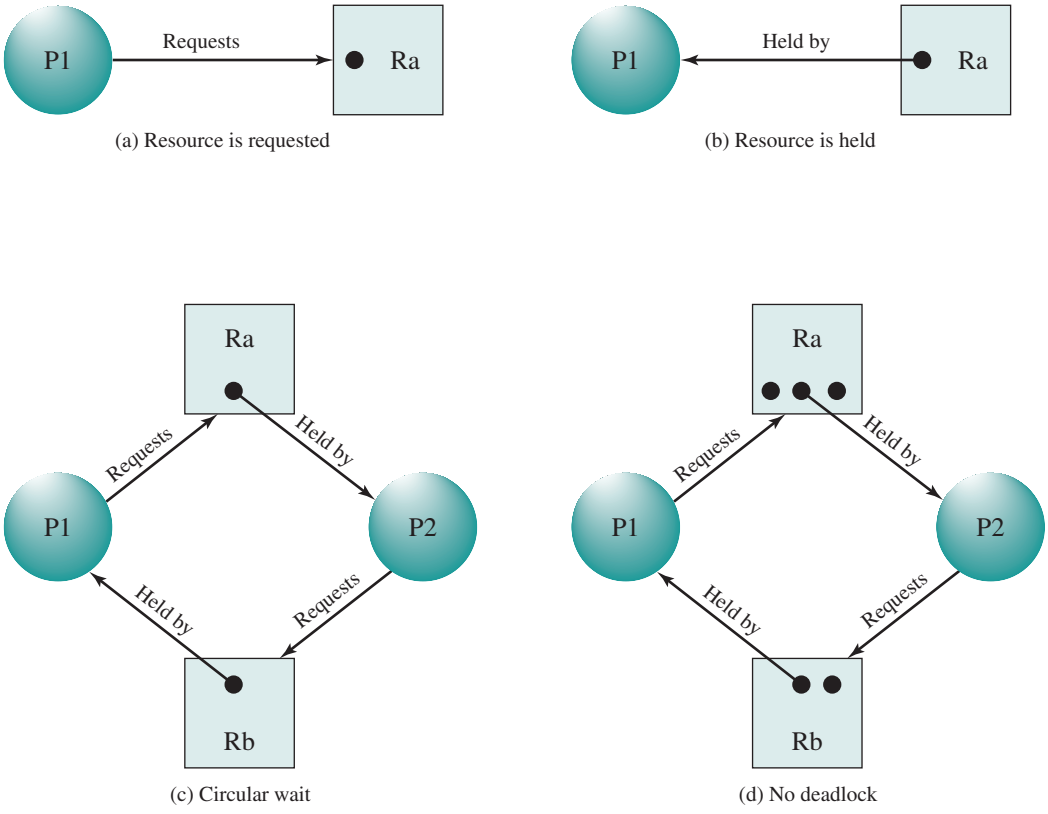



**Figure 6.5 Examples of Resource Allocation Graphs**

The resource allocation graph of Figure 6.6 corresponds to the deadlock situation in Figure 6.1b. Note in this case, we do not have a simple situation in which two processes each have one resource the other needs. Rather, in this case, there is a circular chain of processes and resources that results in deadlock.

## The Conditions for Deadlock

Three conditions of policy must be present for a deadlock to be possible:

1. **Mutual exclusion.** Only one process may use a resource at a time. No process may access a resource unit that has been allocated to another process.
2. **Hold and wait.** A process may hold allocated resources while awaiting assignment of other resources.
3. **No preemption.** No resource can be forcibly removed from a process holding it.

In many ways these conditions are quite desirable. For example, mutual exclusion is needed to ensure consistency of results and the integrity of a database. Similarly,

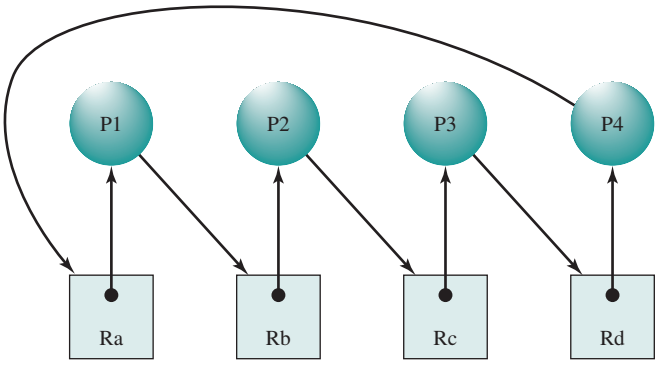



**Figure 6.6** **Resource Allocation Graph for Figure 6.1b**

preemption should not be done arbitrarily. For example, when data resources are involved, preemption must be supported by a rollback recovery mechanism, which restores a process and its resources to a suitable previous state from which the process can eventually repeat its actions.

The first three conditions are necessary, but not sufficient, for a deadlock to exist. For deadlock to actually take place, a fourth condition is required:

4. **Circular wait.** A closed chain of processes exists, such that each process holds at least one resource needed by the next process in the chain (e.g., Figure 6.5c and Figure 6.6).

The fourth condition is, actually, a potential consequence of the first three. That is, given that the first three conditions exist, a sequence of events may occur that lead to an unresolvable circular wait. The unresolvable circular wait is in fact the definition of deadlock. The circular wait listed as condition 4 is unresolvable because the first three conditions hold. Thus, the four conditions, taken together, constitute necessary and sufficient conditions for deadlock.[1]

To clarify this discussion, it is useful to return to the concept of the joint progress diagram, such as the one shown in Figure 6.2. Recall that we defined a fatal region as one such that once the processes have progressed into that region, those processes will deadlock. A fatal region exists only if all of the first three conditions listed above are met. If one or more of these conditions are not met, there is no fatal region and deadlock cannot occur. Thus, these are necessary conditions for deadlock. For deadlock to occur, there must be not only a, fatal region but also a sequence of resource requests that has led into the fatal region. If a circular wait condition occurs,

[1]Virtually all textbooks simply list these four conditions as the conditions needed for deadlock, but such a presentation obscures some of the subtler issues. Item 4, the circular wait condition, is fundamentally different from the other three conditions. Items 1 through 3 are policy decisions, while item 4 is a circumstance that might occur depending on the sequencing of requests and releases by the involved processes. Linking circular wait with the three necessary conditions leads to inadequate distinction between prevention and avoidance. See [SHUB90] and [SHUB03] for a discussion.

then in fact the fatal region has been entered. Thus, all four conditions listed above are sufficient for deadlock. To summarize,

| Possibility of Deadlock | Existence of Deadlock |
|---|---|
| **1.** Mutual exclusion | **1.** Mutual exclusion |
| **2.** No preemption | **2.** No preemption |
| **3.** Hold and wait | **3.** Hold and wait |
| | **4.** Circular wait |

Three general approaches exist for dealing with deadlock. First, one can **prevent** deadlock by adopting a policy that eliminates one of the conditions (conditions 1 through 4). Second, one can **avoid** deadlock by making the appropriate dynamic choices based on the current state of resource allocation. Third, one can attempt to **detect** the presence of deadlock (conditions 1 through 4 hold) and take action to recover. We will discuss each of these approaches in turn.

## 6.2 DEADLOCK PREVENTION

The strategy of deadlock prevention is, simply put, to design a system in such a way that the possibility of deadlock is excluded. We can view deadlock prevention methods as falling into two classes. An indirect method of deadlock prevention is to prevent the occurrence of one of the three necessary conditions previously listed (items 1 through 3). A direct method of deadlock prevention is to prevent the occurrence of a circular wait (item 4). We now examine techniques related to each of the four conditions.

### Mutual Exclusion

In general, the first of the four listed conditions cannot be disallowed. If access to a resource requires mutual exclusion, then mutual exclusion must be supported by the OS. Some resources, such as files, may allow multiple accesses for reads but only exclusive access for writes. Even in this case, deadlock can occur if more than one process requires write permission.

### Hold and Wait

The hold-and-wait condition can be prevented by requiring that a process request all of its required resources at one time and blocking the process until all requests can be granted simultaneously. This approach is inefficient in two ways. First, a process may be held up for a long time waiting for all of its resource requests to be filled, when in fact it could have proceeded with only some of the resources. Second, resources allocated to a process may remain unused for a considerable period, during which time they are denied to other processes. Another problem is that a process may not know in advance all of the resources that it will require.

There is also the practical problem created by the use of modular programming or a multithreaded structure for an application. An application would need to be aware of all resources that will be requested at all levels or in all modules to make the simultaneous request.

### No Preemption

This condition can be prevented in several ways. First, if a process holding certain resources is denied a further request, that process must release its original resources and, if necessary, request them again together with the additional resource. Alternatively, if a process requests a resource that is currently held by another process, the OS may preempt the second process and require it to release its resources. This latter scheme would prevent deadlock only if no two processes possessed the same priority.

This approach is practical only when applied to resources whose state can be easily saved and restored later, as is the case with a processor.

### Circular Wait

The circular wait condition can be prevented by defining a linear ordering of resource types. If a process has been allocated resources of type $R$, then it may subsequently request only those resources of types following $R$ in the ordering.

To see that this strategy works, let us associate an index with each resource type. Then resource $R_i$ precedes $R_j$ in the ordering if $i < j$. Now suppose two processes, A and B, are deadlocked because A has acquired $R_i$ and requested $R_j$, and B has acquired $R_j$ and requested $R_i$. This condition is impossible because it implies $i < j$ and $j < i$.

As with hold-and-wait prevention, circular wait prevention may be inefficient, unnecessarily slowing down processes and denying resource access.

## 6.3 DEADLOCK AVOIDANCE

An approach to solving the deadlock problem that differs subtly from deadlock prevention is deadlock avoidance.[2] In **deadlock prevention**, we constrain resource requests to prevent at least one of the four conditions of deadlock. This is either done indirectly by preventing one of the three necessary policy conditions (mutual exclusion, hold and wait, no preemption), or directly by preventing circular wait. This leads to inefficient use of resources and inefficient execution of processes. **Deadlock avoidance**, on the other hand, allows the three necessary conditions but makes judicious choices to assure that the deadlock point is never reached. As such, avoidance allows more concurrency than prevention. With deadlock avoidance, a decision is made dynamically whether the current resource allocation request will, if granted,

[2]The term *avoidance* is a bit confusing. In fact, one could consider the strategies discussed in this section to be examples of deadlock prevention because they indeed prevent the occurrence of a deadlock.

potentially lead to a deadlock. Deadlock avoidance thus requires knowledge of future process resource requests.

In this section, we describe two approaches to deadlock avoidance:

1. Do not start a process if its demands might lead to deadlock.
2. Do not grant an incremental resource request to a process if this allocation might lead to deadlock.

## Process Initiation Denial

Consider a system of $n$ processes and $m$ different types of resources. Let us define the following vectors and matrices:

| | |
|---|---|
| Resource = $\mathbf{R} = (R_1, R_2, \ldots, R_m)$ | Total amount of each resource in the system |
| Available = $\mathbf{V} = (V_1, V_2, \ldots, V_m)$ | Total amount of each resource not allocated to any process |
| Claim = $\mathbf{C} = \begin{pmatrix} C_{11} & C_{12} & \ldots & C_{1m} \\ C_{21} & C_{22} & \ldots & C_{2m} \\ \vdots & \vdots & \vdots & \vdots \\ C_{n1} & C_{n2} & \ldots & C_{nm} \end{pmatrix}$ | $C_{ij}$ = requirement of process $i$ for resource $j$ |
| Allocation = $\mathbf{A} = \begin{pmatrix} A_{11} & A_{12} & \ldots & A_{1m} \\ A_{21} & A_{22} & \ldots & A_{2m} \\ \vdots & \vdots & \vdots & \vdots \\ A_{n1} & A_{n2} & \ldots & A_{nm} \end{pmatrix}$ | $A_{ij}$ = current allocation to process $i$ of resource $j$ |

The matrix Claim gives the maximum requirement of each process for each resource, with one row dedicated to each process. This information must be declared in advance by a process for deadlock avoidance to work. Similarly, the matrix Allocation gives the current allocation to each process. The following relationships hold:

1. $R_j = V_j + \sum_{i=1}^{n} A_{ij}$, for all $j$ — All resources are either available or allocated.
2. $C_{ij} \leq R_j$, for all $i, j$ — No process can claim more than the total amount of resources in the system.
3. $A_{ij} \leq C_{ij}$, for all $i, j$ — No process is allocated more resources of any type than the process originally claimed to need.

With these quantities defined, we can define a deadlock avoidance policy that refuses to start a new process if its resource requirements might lead to deadlock. Start a new process $P_{n+1}$ only if

$$R_j \geq C_{(n+1)j} + \sum_{i=1}^{n} C_{ij}, \quad \text{for all } j$$

That is, a process is only started if the maximum claim of all current processes plus those of the new process can be met. This strategy is hardly optimal, because it assumes the worst: that all processes will make their maximum claims together.

## Resource Allocation Denial

The strategy of resource allocation denial, referred to as the **banker's algorithm**,[3] was first proposed in [DIJK65]. Let us begin by defining the concepts of state and safe state. Consider a system with a fixed number of processes and a fixed number of resources. At any time a process may have zero or more resources allocated to it. The **state** of the system reflects the current allocation of resources to processes. Thus, the state consists of the two vectors, Resource and Available, and the two matrices, Claim and Allocation, defined earlier. A **safe state** is one in which there is at least one sequence of resource allocations to processes that does not result in a deadlock (i.e., all of the processes can be run to completion). An **unsafe state** is, of course, a state that is not safe.

The following example illustrates these concepts. Figure 6.7a shows the state of a system consisting of four processes and three resources. The total amount of resources R1, R2, and R3 are 9, 3, and 6 units, respectively. In the current state allocations have been made to the four processes, leaving 1 unit of R2 and 1 unit of R3 available. Is this a safe state? To answer this question, we ask an intermediate question: Can any of the four processes be run to completion with the resources available? That is, can the difference between the maximum requirement and current allocation for any process be met with the available resources? In terms of the matrices and vectors introduced earlier, the condition to be met for process $i$ is:

$$C_{ij} - A_{ij} \leq V_j, \quad \text{for all } j$$

Clearly, this is not possible for P1, which has only 1 unit of R1 and requires 2 more units of R1, 2 units of R2, and 2 units of R3. However, by assigning one unit of R3 to process P2, P2 has its maximum required resources allocated and can run to completion. Let us assume this is accomplished. When P2 completes, its resources can be returned to the pool of available resources. The resulting state is shown in Figure 6.7b. Now we can ask again if any of the remaining processes can be completed. In this case, each of the remaining processes could be completed. Suppose we choose P1, allocate the required resources, complete P1, and return all of P1's resources to the available pool. We are left in the state shown in Figure 6.7c. Next, we can complete P3, resulting in the state of Figure 6.7d. Finally, we can complete P4. At this point, all of the processes have been run to completion. Thus, the state defined by Figure 6.7a is a safe state.

[3]Dijkstra used this name because of the analogy of this problem to one in banking, with customers who wish to borrow money corresponding to processes, and the money to be borrowed corresponding to resources. Stated as a banking problem, the bank has a limited reserve of money to lend and a list of customers, each with a line of credit. A customer may choose to borrow against the line of credit a portion at a time, and there is no guarantee that the customer will make any repayment until after having taken out the maximum amount of loan. The banker can refuse a loan to a customer if there is a risk that the bank will have insufficient funds to make further loans that will permit the customers to repay eventually.

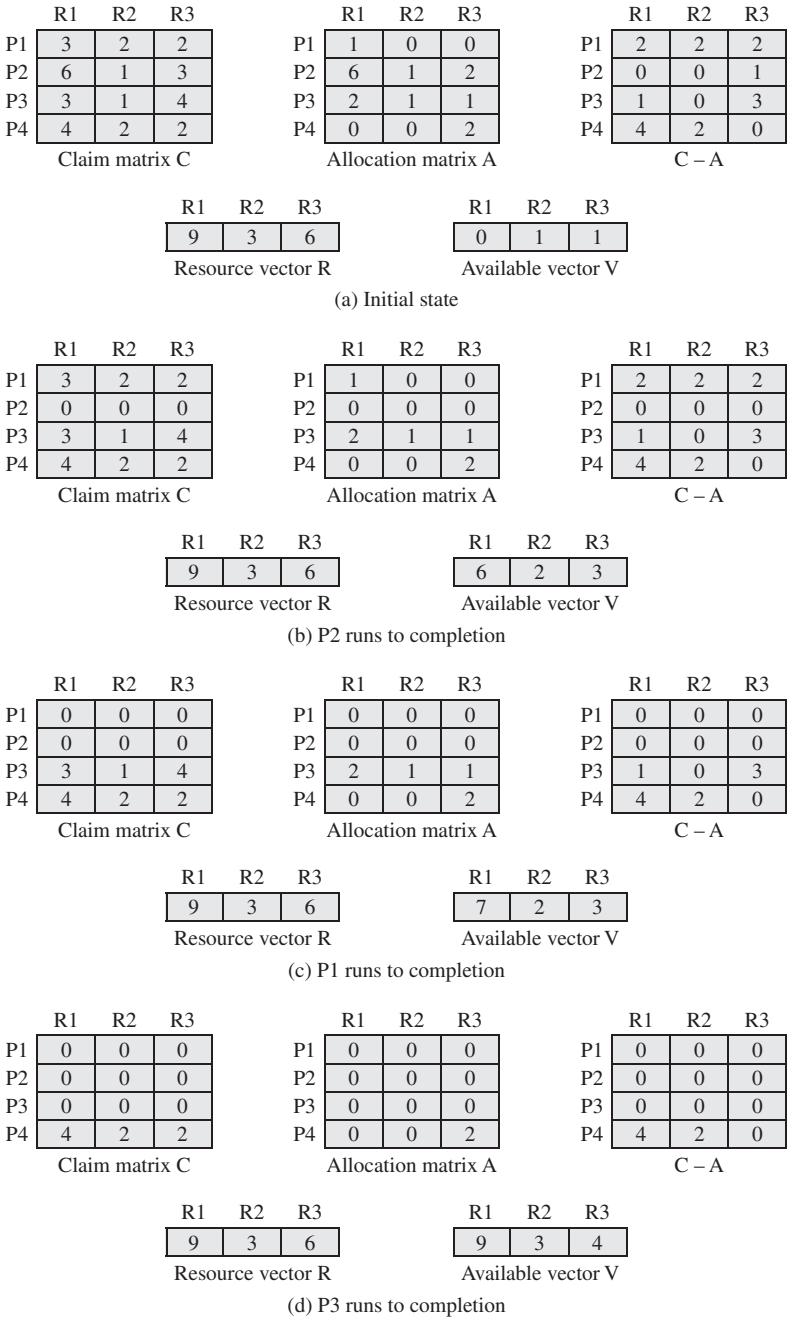



**Figure 6.7 Determination of a Safe State**

These concepts suggest the following deadlock avoidance strategy, which ensures that the system of processes and resources is always in a safe state. When a process makes a request for a set of resources, assume the request is granted, update the system state accordingly, then determine if the result is a safe state. If so, grant the request and, if not, block the process until it is safe to grant the request.

Consider the state defined in Figure 6.8a. Suppose P2 makes a request for one additional unit of R1 and one additional unit of R3. If we assume the request is granted, then the resulting state is that of Figure 6.7a. We have already seen that this is a safe state; therefore, it is safe to grant the request. Now let us return to the state of Figure 6.8a, and suppose P1 makes the request for one additional unit each of R1 and R3; if we assume the request is granted, we are left in the state of Figure 6.8b. Is this a safe state? The answer is no, because each process will need at least one additional unit of R1, and there are none available. Thus, on the basis of deadlock avoidance, the request by P1 should be denied and P1 should be blocked.

It is important to point out that Figure 6.8b is not a deadlocked state. It merely has the potential for deadlock. It is possible, for example, that if P1 were run from this state, it would subsequently release one unit of R1 and one unit of R3 prior to needing these resources again. If that happened, the system would return to a safe state. Thus, the deadlock avoidance strategy does not predict deadlock with certainty; it merely anticipates the possibility of deadlock and assures that there is never such a possibility.

Figure 6.9 gives an abstract version of the deadlock avoidance logic. The main algorithm is shown in part (b). With the state of the system defined by the data

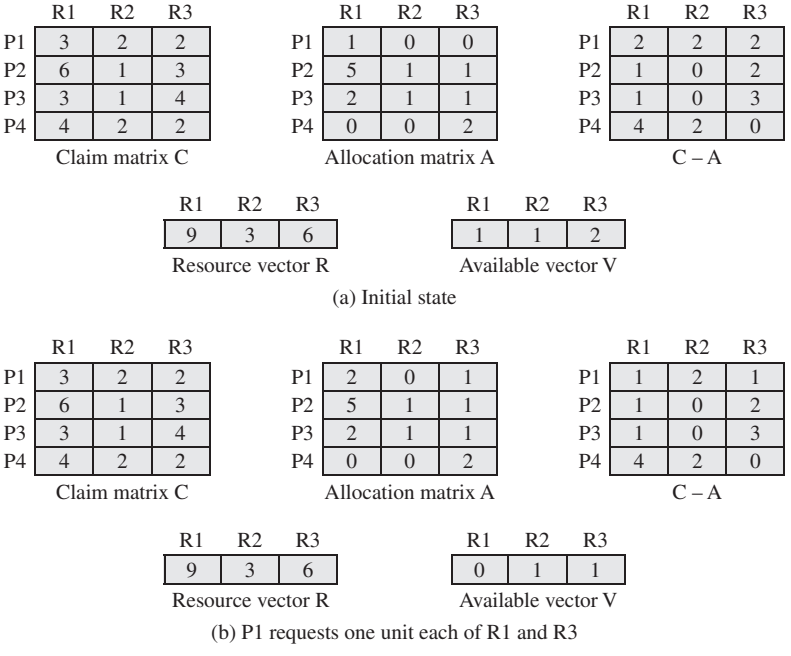



**Figure 6.8 Determination of an Unsafe State**

```
struct  state {
           int  resource[m];
           int  available[m];
           int  claim[n][m];
           int  alloc[n][m];
}
```

(a) Global data structures

```
if (alloc [i,*] + request [*] > claim [i,*])
     <error>;                    /* total request > claim*/
else if (request [*] > available [*])
     <suspend process>;
else  {                                  /* simulate alloc */
     <define newstate by:
     alloc [i,*] = alloc [i,*] + request [*];
     available [*] = available [*] - request [*]>;
}
if  (safe (newstate))
      <carry out allocation>;
else  {
      <restore original state>;
      <suspend process>;
}
```

(b) Resource allocation algorithm

```
boolean safe (state S) {
    int currentavail[m];
    process rest[<number of processes>];
    currentavail = available;
    rest = {all processes};
    possible = true;
    while (possible) {
        <find a process Pk in rest such that
        claim [k,*] - alloc [k,*]<= currentavail;
        if (found) {         /* simulate execution of Pk */
            currentavail = currentavail + alloc [k,*];
            rest = rest - {Pk};
        }
        else  possible = false;
    }
    return (rest == null);
}
```

(c) Test for safety algorithm (banker's algorithm)



VideoNote **Figure 6.9 Deadlock Avoidance Logic**

structure `state, request [*]` is a vector defining the resources requested by process *i*. First, a check is made to assure that the request does not exceed the original claim of the process. If the request is valid, the next step is to determine if it is possible to fulfill the request (i.e., there are sufficient resources available). If it is not possible, then the process is suspended. If it is possible, the final step is to determine if it is safe to fulfill the request. To do this, the resources are tentatively assigned to process *i* to form `newstate`. Then a test for safety is made using the algorithm in Figure 6.9c.

Deadlock avoidance has the advantage that it is not necessary to preempt and rollback processes, as in deadlock detection, and is less restrictive than deadlock prevention. However, it does have a number of restrictions on its use:

- The maximum resource requirement for each process must be stated in advance.
- The processes under consideration must be independent; that is, the order in which they execute must be unconstrained by any synchronization requirements.
- There must be a fixed number of resources to allocate.
- No process may exit while holding resources.

## 6.4 DEADLOCK DETECTION

Deadlock prevention strategies are very conservative; they solve the problem of deadlock by limiting access to resources and by imposing restrictions on processes. At the opposite extreme, deadlock detection strategies do not limit resource access or restrict process actions. With deadlock detection, requested resources are granted to processes whenever possible. Periodically, the OS performs an algorithm that allows it to detect the circular wait condition described earlier in condition (4) and illustrated in Figure 6.6.

### Deadlock Detection Algorithm

A check for deadlock can be made as frequently as each resource request, or less frequently, depending on how likely it is for a deadlock to occur. Checking at each resource request has two advantages: It leads to early detection, and the algorithm is relatively simple because it is based on incremental changes to the state of the system. On the other hand, such frequent checks consume considerable processor time.

A common algorithm for deadlock detection is one described in [COFF71], which is designed to detect a deadlock by accounting for all possibilities of sequencing of the tasks that remain to be completed. The Allocation matrix and Available vector described in the previous section are used. In addition, a request matrix **Q** is defined such that *Qij* represents the amount of resources of type *j* requested by process *i*. The algorithm proceeds by marking processes that are not part of a deadlocked set. Initially, all processes are unmarked. Then the following steps are performed:

1. Mark each process that has a row in the Allocation matrix of all zeros. A process that has no allocated resources cannot participate in a deadlock.
2. Initialize a temporary vector **W** to equal the Available vector.

3. Find an index $i$ such that process $i$ is currently unmarked and the $i$th row of **Q** is less than or equal to **W**. That is, $Q_{ik} \leq W_k$, for $1 \leq k \leq m$. If no such row is found, terminate the algorithm.
4. If such a row is found, mark process $i$ and add the corresponding row of the allocation matrix to **W**. That is, set $W_k = W_k + A_{ik}$, for $1 \leq k \leq m$. Return to step 3.

A deadlock exists if and only if there are unmarked processes at the end of the algorithm. The set of unmarked rows corresponds precisely to the set of deadlocked processes. The strategy in this algorithm is to find a process whose resource requests can be satisfied with the available resources, then assume those resources are granted and the process runs to completion and releases all of its resources. The algorithm then looks for another process to satisfy. Note this algorithm does not guarantee to prevent deadlock; that will depend on the order in which future requests are granted. All that it does is determine if deadlock currently exists.

We can use Figure 6.10 to illustrate the deadlock detection algorithm. The algorithm proceeds as follows:

1. Mark P4, because P4 has no allocated resources.
2. Set $\mathbf{W} = (0\,0\,0\,0\,1)$.
3. The request of process P3 is less than or equal to **W**, so mark P3 and set
$$\mathbf{W} = \mathbf{W} + (0\,0\,0\,1\,0) = (0\,0\,0\,1\,1).$$
4. No other unmarked process has a row in **Q** that is less than or equal to **W**. Therefore, terminate the algorithm.

The algorithm concludes with P1 and P2 unmarked, indicating these processes are deadlocked.

## Recovery

Once deadlock has been detected, some strategy is needed for recovery. The following are possible approaches, listed in the order of increasing sophistication:

1. Abort all deadlocked processes. This is, believe it or not, one of the most common, if not the most common, solutions adopted in operating systems.

| | R1 | R2 | R3 | R4 | R5 |
|---|---|---|---|---|---|
| P1 | 0 | 1 | 0 | 0 | 1 |
| P2 | 0 | 0 | 1 | 0 | 1 |
| P3 | 0 | 0 | 0 | 0 | 1 |
| P4 | 1 | 0 | 1 | 0 | 1 |

Request matrix Q

| | R1 | R2 | R3 | R4 | R5 |
|---|---|---|---|---|---|
| P1 | 1 | 0 | 1 | 1 | 0 |
| P2 | 1 | 1 | 0 | 0 | 0 |
| P3 | 0 | 0 | 0 | 1 | 0 |
| P4 | 0 | 0 | 0 | 0 | 0 |

Allocation matrix A

| R1 | R2 | R3 | R4 | R5 |
|---|---|---|---|---|
| 2 | 1 | 1 | 2 | 1 |

Resource vector

| R1 | R2 | R3 | R4 | R5 |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 |

Available vector

**Figure 6.10 Example for Deadlock Detection**

2. Back up each deadlocked process to some previously defined checkpoint, and restart all processes. This requires that rollback and restart mechanisms be built into the system. The risk in this approach is that the original deadlock may recur. However, the nondeterminancy of concurrent processing may ensure that this does not happen.
3. Successively abort deadlocked processes until deadlock no longer exists. The order in which processes are selected for abortion should be on the basis of some criterion of minimum cost. After each abortion, the detection algorithm must be reinvoked to see whether deadlock still exists.
4. Successively preempt resources until deadlock no longer exists. As in (3), a cost-based selection should be used, and reinvocation of the detection algorithm is required after each preemption. A process that has a resource preempted from it must be rolled back to a point prior to its acquisition of that resource.

For (3) and (4), the selection criteria could be one of the following. Choose the process with the:

- least amount of processor time consumed so far.
- least amount of output produced so far.
- most estimated time remaining.
- least total resources allocated so far.
- lowest priority.

Some of these quantities are easier to measure than others. Estimated time remaining is particularly suspect. Also, other than by means of the priority measure, there is no indication of the "cost" to the user, as opposed to the cost to the system as a whole.

## 6.5 AN INTEGRATED DEADLOCK STRATEGY

There are strengths and weaknesses to all of the strategies for dealing with deadlock. Rather than attempting to design an OS facility that employs only one of these strategies, it might be more efficient to use different strategies in different situations. [HOWA73] suggests one approach:

- Group resources into a number of different resource classes.
- Use the linear ordering strategy defined previously for the prevention of circular wait to prevent deadlocks between resource classes.
- Within a resource class, use the algorithm that is most appropriate for that class.

As an example of this technique, consider the following classes of resources:

- **Swappable space:** Blocks of memory on secondary storage for use in swapping processes
- **Process resources:** Assignable devices, such as tape drives, and files

- **Main memory:** Assignable to processes in pages or segments
- **Internal resources:** Such as I/O channels

The order of the preceding list represents the order in which resources are assigned. The order is a reasonable one, considering the sequence of steps that a process may follow during its lifetime. Within each class, the following strategies could be used:

- **Swappable space:** Prevention of deadlocks by requiring that all of the required resources that may be used be allocated at one time, as in the hold-and-wait prevention strategy. This strategy is reasonable if the maximum storage requirements are known, which is often the case. Deadlock avoidance is also a possibility.
- **Process resources:** Avoidance will often be effective in this category, because it is reasonable to expect processes to declare ahead of time the resources that they will require in this class. Prevention by means of resource ordering within this class is also possible.
- **Main memory:** Prevention by preemption appears to be the most appropriate strategy for main memory. When a process is preempted, it is simply swapped to secondary memory, freeing space to resolve the deadlock.
- **Internal resources:** Prevention by means of resource ordering can be used.

## 6.6 DINING PHILOSOPHERS PROBLEM

We now turn to the dining philosophers problem, introduced by Dijkstra [DIJK71]. Five philosophers live in a house, where a table is set for them. The life of each philosopher consists principally of thinking and eating, and through years of thought, all of the philosophers had agreed that the only food that contributed to their thinking efforts was spaghetti. Due to a lack of manual skill, each philosopher requires two forks to eat spaghetti.

The eating arrangements are simple (see Figure 6.11): a round table on which is set a large serving bowl of spaghetti, five plates, one for each philosopher, and five forks. A philosopher wishing to eat goes to his or her assigned place at the table and, using the two forks on either side of the plate, takes and eats some spaghetti. The problem: Devise a ritual (algorithm) that will allow the philosophers to eat. The algorithm must satisfy mutual exclusion (no two philosophers can use the same fork at the same time) while avoiding deadlock and starvation (in this case, the term has literal as well as algorithmic meaning!).

This problem may not seem important or relevant in itself. However, it does illustrate basic problems in deadlock and starvation. Furthermore, attempts to develop solutions reveal many of the difficulties in concurrent programming (e.g., see [GING90]). In addition, the dining philosophers problem can be seen as representative of problems dealing with the coordination of shared resources, which may occur when an application includes concurrent threads of execution. Accordingly, this problem is a standard test case for evaluating approaches to synchronization.

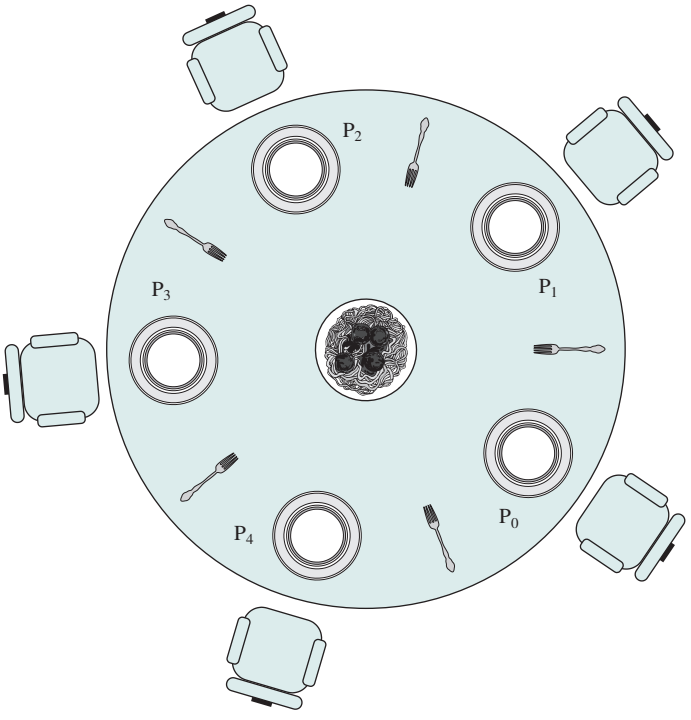



**Figure 6.11** **Dining Arrangement for Philosophers**

## Solution Using Semaphores

Figure 6.12 suggests a solution using semaphores. Each philosopher first picks up the fork on the left then the fork on the right. After the philosopher is finished eating, the two forks are replaced on the table. This solution, alas, leads to deadlock: If all of the philosophers are hungry at the same time, they all sit down, they all pick up the fork on their left, and they all reach out for the other fork, which is not there. In this undignified position, all philosophers starve.

To overcome the risk of deadlock, we could buy five additional forks (a more sanitary solution!) or teach the philosophers to eat spaghetti with just one fork. As another approach, we could consider adding an attendant who only allows four philosophers at a time into the dining room. With at most four seated philosophers, at least one philosopher will have access to two forks. Figure 6.13 shows such a solution, again using semaphores. This solution is free of deadlock and starvation.

## Solution Using a Monitor

Figure 6.14 shows a solution to the dining philosophers problem using a monitor. A vector of five condition variables is defined, one condition variable per fork. These condition variables are used to enable a philosopher to wait for the availability of a fork. In addition, there is a Boolean vector that records the availability

```
/* program diningphilosophers */
semaphore fork [5] = {1};
int i;
void philosopher (int i)
{
     while (true) {
          think();
          wait (fork[i]);
          wait (fork [(i+1) mod 5]);
          eat();
          signal(fork [(i+1) mod 5]);
          signal(fork[i]);
     }
}
void main()
{
     parbegin (philosopher (0), philosopher (1),
          philosopher (2), philosopher (3),
          philosopher (4));
     }
```



VideoNote **Figure 6.12 A First Solution to the Dining Philosophers Problem**

```
/* program diningphilosophers */
semaphore fork[5] = {1};
semaphore room = {4};
int i;
void philosopher (int  i)
{
     while (true) {
          think();
          wait (room);
          wait (fork[i]);
          wait (fork [(i+1) mod 5]);
          eat();
          signal (fork [(i+1) mod 5]);
          signal (fork[i]);
          signal (room);
     }
}
void main()
{
     parbegin  (philosopher (0), philosopher (1),
          philosopher (2), philosopher (3),
          philosopher (4));
 }
```



VideoNote **Figure 6.13 A Second Solution to the Dining Philosophers Problem**

```
monitor dining_controller;
cond ForkReady[5];    /* condition variable for synchronization */
boolean fork[5] = {true}; /* availability status of each fork */

void get_forks(int pid)    /* pid is the philosopher id number */
{
    int left = pid;
    int right = (++pid) % 5;
    /*grant the left fork*/
    if (!fork[left])
       cwait(ForkReady[left]); /* queue on condition variable */
    fork(left) = false;
    /*grant the right fork*/
    if (!fork[right])
       cwait(ForkReady[right]);/* queue on condition variable */
    fork[right] = false:
    }
void release_forks(int  pid)
{
    int left = pid;
    int right = (++pid) % 5;
    /*release the left fork*/
    if (empty(ForkReady[left]) /*no one is waiting for this fork */
       fork[left] = true;
    else              /* awaken a process waiting on this fork */
       csignal(ForkReady[left]);
    /*release the right fork*/
    if (empty(ForkReady[right])/*no one is waiting for this fork */
       fork[right] = true;
    else              /* awaken a process waiting on this fork */
       csignal(ForkReady[right]);
}
```

```
void philosopher[k=0 to 4]    /* the five philosopher clients */
{
    while  (true) {
      <think>;
     get_forks(k);    /* client requests two forks via monitor */
      <eat spaghetti>;
      release_forks(k); /* client releases forks via the monitor */
    }
}
```

VideoNote **Figure 6.14** **A Solution to the Dining Philosophers Problem Using a Monitor**

status of each fork (`true` means the fork is available). The monitor consists of two procedures. The `get_forks` procedure is used by a philosopher to seize his or her left and right forks. If either fork is unavailable, the philosopher process is queued on the appropriate condition variable. This enables another philosopher process to enter the monitor. The `release-forks` procedure is used to make two forks available. Note the structure of this solution is similar to that of the semaphore solution proposed in Figure 6.12. In both cases, a philosopher seizes first the left fork then the right fork. Unlike the semaphore solution, this monitor solution does not suffer from deadlock, because only one process at a time may be in the monitor. For example, the first philosopher process to enter the monitor is guaranteed that it can pick up the right fork after it picks up the left fork before the next philosopher to the right has a chance to seize his or her left fork, which is this philosopher's right fork.

## 6.7 UNIX CONCURRENCY MECHANISMS

UNIX provides a variety of mechanisms for interprocessor communication and synchronization. Here, we look at the most important of these:

- Pipes
- Messages
- Shared memory
- Semaphores
- Signals

Pipes, messages, and shared memory can be used to communicate data between processes, whereas semaphores and signals are used to trigger actions by other processes.

### Pipes

One of the most significant contributions of UNIX to the development of operating systems is the pipe. Inspired by the concept of coroutines [RITC84], a pipe is a circular buffer allowing two processes to communicate on the producer–consumer model. Thus, it is a first-in-first-out queue, written by one process and read by another.

When a pipe is created, it is given a fixed size in bytes. When a process attempts to write into the pipe, the write request is immediately executed if there is sufficient room; otherwise the process is blocked. Similarly, a reading process is blocked if it attempts to read more bytes than are currently in the pipe; otherwise the read request is immediately executed. The OS enforces mutual exclusion: that is, only one process can access a pipe at a time.

There are two types of pipes: named and unnamed. Only related processes can share unnamed pipes, while either related or unrelated processes can share named pipes.

### Messages

A message is a block of bytes with an accompanying type. UNIX provides `msgsnd` and `msgrcv` system calls for processes to engage in message passing. Associated with each process is a message queue, which functions like a mailbox.

The message sender specifies the type of message with each message sent, and this can be used as a selection criterion by the receiver. The receiver can either retrieve messages in first-in-first-out order or by type. A process will block when trying to send a message to a full queue. A process will also block when trying to read from an empty queue. If a process attempts to read a message of a certain type and fails because no message of that type is present, the process is not blocked.

### Shared Memory

The fastest form of interprocess communication provided in UNIX is shared memory. This is a common block of virtual memory shared by multiple processes. Processes read and write shared memory using the same machine instructions they use to read and write other portions of their virtual memory space. Permission is read-only or read-write for a process, determined on a per-process basis. Mutual exclusion constraints are not part of the shared-memory facility, but must be provided by the processes using the shared memory.

### Semaphores

The semaphore system calls in UNIX System V are a generalization of the `semWait` and `semSignal` primitives defined in Chapter 5; several operations can be performed simultaneously, and the increment and decrement operations can be values greater than 1. The kernel does all of the requested operations atomically; no other process may access the semaphore until all operations have completed.

A semaphore consists of the following elements:

- Current value of the semaphore
- Process ID of the last process to operate on the semaphore
- Number of processes waiting for the semaphore value to be greater than its current value
- Number of processes waiting for the semaphore value to be zero

Associated with the semaphore are queues of processes blocked on that semaphore.

Semaphores are actually created in sets, with a semaphore set consisting of one or more semaphores. There is a `semctl` system call that allows all of the semaphore values in the set to be set at the same time. In addition, there is a `sem_op` system call that takes as an argument a list of semaphore operations, each defined on one of the semaphores in a set. When this call is made, the kernel performs the indicated operations one at a time. For each operation, the actual function is specified by the value `sem_op`. The following are the possibilities:

- If `sem_op` is positive, the kernel increments the value of the semaphore and awakens all processes waiting for the value of the semaphore to increase.

- If `sem_op` is 0, the kernel checks the semaphore value. If the semaphore value equals 0, the kernel continues with the other operations on the list. Otherwise, the kernel increments the number of processes waiting for this semaphore to be 0 and suspends the process to wait for the event that the value of the semaphore equals 0.
- If `sem_op` is negative and its absolute value is less than or equal to the semaphore value, the kernel adds `sem_op` (a negative number) to the semaphore value. If the result is 0, the kernel awakens all processes waiting for the value of the semaphore to equal 0.
- If `sem_op` is negative and its absolute value is greater than the semaphore value, the kernel suspends the process on the event that the value of the semaphore increases.

This generalization of the semaphore provides considerable flexibility in performing process synchronization and coordination.

### Signals

A signal is a software mechanism that informs a process of the occurrence of asynchronous events. A signal is similar to a hardware interrupt but does not employ priorities. That is, all signals are treated equally; signals that occur at the same time are presented to a process one at a time, with no particular ordering.

Processes may send each other signals, or the kernel may send signals internally. A signal is delivered by updating a field in the process table for the process to which the signal is being sent. Because each signal is maintained as a single bit, signals of a given type cannot be queued. A signal is processed just after a process wakes up to run or whenever the process is preparing to return from a system call. A process may respond to a signal by performing some default action (e.g., termination), executing a signal-handler function, or ignoring the signal.

Table 6.1 lists signals defined for UNIX SVR4.

## 6.8 LINUX KERNEL CONCURRENCY MECHANISMS

Linux includes all of the concurrency mechanisms found in other UNIX systems, such as SVR4, including pipes, messages, shared memory, and signals. Linux also supports a special type of signaling known as real-time (RT) signals. These are part of the POSIX.1b Real-time Extensions feature. RT signals differ from standard UNIX (or POSIX.1) signals in three primary ways:

- Signal delivery in priority order is supported.
- Multiple signals can be queued.
- With standard signals, no value or message can be sent to the target process; it is only a notification. With RT signals, it is possible to send a value (an integer or a pointer) along with the signal.

Linux also includes a rich set of concurrency mechanisms specifically intended for use when a thread is executing in kernel mode. That is, these are mechanisms used

**Table 6.1** UNIX Signals

| Value | Name | Description |
|---|---|---|
| 01 | SIGHUP | Hang up; sent to process when kernel assumes that the user of that process is doing no useful work |
| 02 | SIGINT | Interrupt |
| 03 | SIGQUIT | Quit; sent by user to induce halting of process and production of core dump |
| 04 | SIGILL | Illegal instruction |
| 05 | SIGTRAP | Trace trap; triggers the execution of code for process tracing |
| 06 | SIGIOT | IOT instruction |
| 07 | SIGEMT | EMT instruction |
| 08 | SIGFPE | Floating-point exception |
| 09 | SIGKILL | Kill; terminate process |
| 10 | SIGBUS | Bus error |
| 11 | SIGSEGV | Segmentation violation; process attempts to access location outside its virtual address space |
| 12 | SIGSYS | Bad argument to system call |
| 13 | SIGPIPE | Write on a pipe that has no readers attached to it |
| 14 | SIGALRM | Alarm clock; issued when a process wishes to receive a signal after a period of time |
| 15 | SIGTERM | Software termination |
| 16 | SIGUSR1 | User-defined signal 1 |
| 17 | SIGUSR2 | User-defined signal 2 |
| 18 | SIGCHLD | Death of a child |
| 19 | SIGPWR | Power failure |

within the kernel to provide concurrency in the execution of kernel code. This section examines the Linux kernel concurrency mechanisms.

## Atomic Operations

Linux provides a set of operations that guarantee atomic operations on a variable. These operations can be used to avoid simple race conditions. An atomic operation executes without interruption and without interference. On a uniprocessor system, a thread performing an atomic operation cannot be interrupted once the operation has started until the operation is finished. In addition, on a multiprocessor system, the variable being operated on is locked from access by other threads until this operation is completed.

Two types of atomic operations are defined in Linux: integer operations, which operate on an integer variable, and bitmap operations, which operate on one bit in a bitmap (see Table 6.2). These operations must be implemented on any architecture that implements Linux. For some architectures, there are corresponding assembly language instructions for the atomic operations. On other architectures, an operation that locks the memory bus is used to guarantee that the operation is atomic.

**Table 6.2** Linux Atomic Operations

| Atomic Integer Operations | |
|---|---|
| `ATOMIC_INIT (int i)` | At declaration: initialize an atomic_t to i |
| `int atomic_read(atomic_t *v)` | Read integer value of v |
| `void atomic_set(atomic_t *v, int i)` | Set the value of v to integer i |
| `void atomic_add(int i, atomic_t *v)` | Add i to v |
| `void atomic_sub(int i, atomic_t *v)` | Subtract i from v |
| `void atomic_inc(atomic_t *v)` | Add 1 to v |
| `void atomic_dec(atomic_t *v)` | Subtract 1 from v |
| `int atomic_sub_and_test(int i, atomic_t *v)` | Subtract i from v; return 1 if the result is 0; return 0 otherwise |
| `int atomic_add_negative(int i, atomic_t *v)` | Add i to v; return 1 if the result is negative; return 0 otherwise (used for implementing semaphores) |
| `int atomic_dec_and_test(atomic_t *v)` | Subtract 1 from v; return 1 if the result is 0; return 0 otherwise |
| `int atomic_inc_and_test(atomic_t *v)` | Add 1 to v; return 1 if the result is 0; return 0 otherwise |
| **Atomic Bitmap Operations** | |
| `void set_bit(int nr, void *addr)` | Set bit nr in the bitmap pointed to by addr |
| `void clear_bit(int nr, void *addr)` | Clear bit nr in the bitmap pointed to by addr |
| `void change_bit(int nr, void *addr)` | Invert bit nr in the bitmap pointed to by addr |
| `int test_and_set_bit(int nr, void *addr)` | Set bit nr in the bitmap pointed to by addr; return the old bit value |
| `int test_and_clear_bit(int nr, void *addr)` | Clear bit nr in the bitmap pointed to by addr; return the old bit value |
| `int test_and_change_bit(int nr, void *addr)` | Invert bit nr in the bitmap pointed to by addr; return the old bit value |
| `int test_bit(int nr, void *addr)` | Return the value of bit nr in the bitmap pointed to by addr |

For **atomic integer operations**, a special data type is used, `atomic_t`. The atomic integer operations can be used only on this data type, and no other operations are allowed on this data type. [LOVE04] lists the following advantages for these restrictions:

1. The atomic operations are never used on variables that might in some circumstances be unprotected from race conditions.
2. Variables of this data type are protected from improper use by nonatomic operations.
3. The compiler cannot erroneously optimize access to the value (e.g., by using an alias rather than the correct memory address).
4. This data type serves to hide architecture-specific differences in its implementation.

A typical use of the atomic integer data type is to implement counters.

The **atomic bitmap operations** operate on one of a sequence of bits at an arbitrary memory location indicated by a pointer variable. Thus, there is no equivalent to the `atomic_t` data type needed for atomic integer operations.

Atomic operations are the simplest of the approaches to kernel synchronization. More complex locking mechanisms can be built on top of them.

## Spinlocks

The most common technique used for protecting a critical section in Linux is the spinlock. Only one thread at a time can acquire a spinlock. Any other thread attempting to acquire the same lock will keep trying (spinning) until it can acquire the lock. In essence, a spinlock is built on an integer location in memory that is checked by each thread before it enters its critical section. If the value is 0, the thread sets the value to 1 and enters its critical section. If the value is nonzero, the thread continually checks the value until it is 0. The spinlock is easy to implement, but has the disadvantage that locked-out threads continue to execute in a busy waiting mode. Thus, spinlocks are most effective in situations where the wait time for acquiring a lock is expected to be very short, say on the order of less than two context switches.

The basic form of use of a spinlock is the following:

```
spin_lock(&lock)
/* critical section */
spin_unlock(&lock)
```

***Basic Spinlocks*** The basic spinlock (as opposed to the reader–writer spinlock explained subsequently) comes in four flavors (see Table 6.3):

- **Plain:** If the critical section of code is not executed by interrupt handlers, or if the interrupts are disabled during the execution of the critical section, then the plain spinlock can be used. It does not affect the interrupt state on the processor on which it is run.
- **_irq:** If interrupts are always enabled, then this spinlock should be used.
- **_irqsave:** If it is not known which, if any, interrupts will be enabled or disabled at the time of execution, then this version should be used. When a lock is acquired, the current state of interrupts on the local processor is saved, to be restored when the lock is released.
- **_bh:** When an interrupt occurs, the minimum amount of work necessary is performed by the corresponding interrupt handler. A piece of code, called the *bottom half*, performs the remainder of the interrupt-related work, allowing the current interrupt to be enabled as soon as possible. The `_bh` spinlock is used to disable and then enable bottom halves to avoid conflict with the protected critical section.

The plain spinlock is used if the programmer knows that the protected data is not accessed by an interrupt handler or bottom half. Otherwise, the appropriate nonplain spinlock is used.

**Table 6.3** Linux Spinlocks

| | |
|---|---|
| `void spin_lock(spinlock_t *lock)` | Acquires the specified lock, spinning if needed until it is available |
| `void spin_lock_irq(spinlock_t *lock)` | Like spin_lock, but also disables interrupts on the local processor |
| `void spin_lock_irqsave(spinlock_t *lock, unsigned long flags)` | Like spin_lock_irq, but also saves the current interrupt state in flags |
| `void spin_lock_bh(spinlock_t *lock)` | Like spin_lock, but also disables the execution of all bottom halves |
| `void spin_unlock(spinlock_t *lock)` | Releases given lock |
| `void spin_unlock_irq(spinlock_t *lock)` | Releases given lock and enables local interrupts |
| `void spin_unlock_irqrestore(spinlock_t *lock, unsigned long flags)` | Releases given lock and restores local interrupts to given previous state |
| `void spin_unlock_bh(spinlock_t *lock)` | Releases given lock and enables bottom halves |
| `void spin_lock_init(spinlock_t *lock)` | Initializes given spinlock |
| `int spin_trylock(spinlock_t *lock)` | Tries to acquire specified lock; returns nonzero if lock is currently held and zero otherwise |
| `int spin_is_locked(spinlock_t *lock)` | Returns nonzero if lock is currently held and zero otherwise |

Spinlocks are implemented differently on a uniprocessor system versus a multiprocessor system. For a uniprocessor system, the following considerations apply. If kernel preemption is turned off, so a thread executing in kernel mode cannot be interrupted, then the locks are deleted at compile time; they are not needed. If kernel preemption is enabled, which does permit interrupts, then the spinlocks again compile away (i.e., no test of a spinlock memory location occurs) but are simply implemented as code that enables/disables interrupts. On a multiprocessor system, the spinlock is compiled into code that does in fact test the spinlock location. The use of the spinlock mechanism in a program allows it to be independent of whether it is executed on a uniprocessor or multiprocessor system.

***Reader–Writer Spinlock*** The reader–writer spinlock is a mechanism that allows a greater degree of concurrency within the kernel than the basic spinlock. The reader–writer spinlock allows multiple threads to have simultaneous access to the same data structure for reading only, but gives exclusive access to the spinlock for a thread that intends to update the data structure. Each reader–writer spinlock consists of a 24-bit reader counter and an unlock flag, with the following interpretation:

| Counter | Flag | Interpretation |
|---|---|---|
| 0 | 1 | The spinlock is released and available for use. |
| 0 | 0 | Spinlock has been acquired for writing by one thread. |
| $n$ ($n > 0$) | 0 | Spinlock has been acquired for reading by $n$ threads. |
| $n$ ($n > 0$) | 1 | Not valid. |

As with the basic spinlock, there are plain, _irq, and _irqsave versions of the reader–writer spinlock.

Note that the reader–writer spinlock favors readers over writers. If the spinlock is held for readers, then so long as there is at least one reader, the spinlock cannot be preempted by a writer. Furthermore, new readers may be added to the spinlock even while a writer is waiting.

## Semaphores

At the user level, Linux provides a semaphore interface corresponding to that in UNIX SVR4. Internally, Linux provides an implementation of semaphores for its own use. That is, code that is part of the kernel can invoke kernel semaphores. These kernel semaphores cannot be accessed directly by the user program via system calls. They are implemented as functions within the kernel, and are thus more efficient than user-visible semaphores.

Linux provides three types of semaphore facilities in the kernel: binary semaphores, counting semaphores, and reader–writer semaphores.

***Binary and Counting Semaphores*** The binary and counting semaphores defined in Linux 2.6 (see Table 6.4) have the same functionality as described for such semaphores in Chapter 5. The function names down and up are used for the functions referred to in Chapter 5 as semWait and semSignal, respectively.

A counting semaphore is initialized using the sema_init function, which gives the semaphore a name and assigns an initial value to the semaphore. Binary semaphores, called MUTEXes in Linux, are initialized using the init_MUTEX and init_MUTEX_LOCKED functions, which initialize the semaphore to 1 or 0, respectively.

Linux provides three versions of the down (semWait) operation.

1. The down function corresponds to the traditional semWait operation. That is, the thread tests the semaphore and blocks if the semaphore is not available. The thread will awaken when a corresponding up operation on this semaphore occurs. Note this function name is used for an operation on either a counting semaphore or a binary semaphore.
2. The down_interruptible function allows the thread to receive and respond to a kernel signal while being blocked on the down operation. If the thread is woken up by a signal, the down_interruptible function increments the count value of the semaphore and returns an error code known in Linux as -EINTR. This alerts the thread that the invoked semaphore function has aborted. In effect, the thread has been forced to "give up" the semaphore. This feature is useful for device drivers and other services in which it is convenient to override a semaphore operation.
3. The down_trylock function makes it possible to try to acquire a semaphore without being blocked. If the semaphore is available, it is acquired. Otherwise, this function returns a nonzero value without blocking the thread.

**Table 6.4** Linux Semaphores

| Traditional Semaphores | |
|---|---|
| `void sema_init(struct semaphore *sem, int count)` | Initializes the dynamically created semaphore to the given count |
| `void init_MUTEX(struct semaphore *sem)` | Initializes the dynamically created semaphore with a count of 1 (initially unlocked) |
| `void init_MUTEX_LOCKED(struct semaphore *sem)` | Initializes the dynamically created semaphore with a count of 0 (initially locked) |
| `void down(struct semaphore *sem)` | Attempts to acquire the given semaphore, entering uninterruptible sleep if semaphore is unavailable |
| `int down_interruptible(struct semaphore *sem)` | Attempts to acquire the given semaphore, entering interruptible sleep if semaphore is unavailable; returns EINTR value if a signal other than the result of an up operation is received |
| `int down_trylock(struct semaphore *sem)` | Attempts to acquire the given semaphore, and returns a nonzero value if semaphore is unavailable |
| `void up(struct semaphore *sem)` | Releases the given semaphore |
| **Reader–Writer Semaphores** | |
| `void init_rwsem(struct rw_semaphore, *rwsem)` | Initializes the dynamically created semaphore with a count of 1 |
| `void down_read(struct rw_semaphore, *rwsem)` | Down operation for readers |
| `void up_read(struct rw_semaphore, *rwsem)` | Up operation for readers |
| `void down_write(struct rw_semaphore, *rwsem)` | Down operation for writers |
| `void up_write(struct rw_semaphore, *rwsem)` | Up operation for writers |

***Reader–Writer Semaphores*** The reader–writer semaphore divides users into readers and writers; it allows multiple concurrent readers (with no writers) but only a single writer (with no concurrent readers). In effect, the semaphore functions as a counting semaphore for readers but a binary semaphore (MUTEX) for writers. Table 6.4 shows the basic reader–writer semaphore operations. The reader–writer semaphore uses uninterruptible sleep, so there is only one version of each of the `down` operations.

## Barriers

In some architectures, compilers and/or the processor hardware may reorder memory accesses in source code to optimize performance. These reorderings are done to optimize the use of the instruction pipeline in the processor. The reordering algorithms

contain checks to ensure that data dependencies are not violated. For example, the code:

```
a = 1;
b = 1;
```

may be reordered so that memory location `b` is updated before memory location `a` is updated. However, the code:

```
a = 1;
b = a;
```

will not be reordered. Even so, there are occasions when it is important that reads or writes are executed in the order specified because of use of the information that is made by another thread or a hardware device.

To enforce the order in which instructions are executed, Linux provides the memory barrier facility. Table 6.5 lists the most important functions that are defined for this facility. The `rmb()` operation insures that no reads occur across the barrier defined by the place of the `rmb()` in the code. Similarly, the `wmb()` operation insures that no writes occur across the barrier defined by the place of the `wmb()` in the code. The `mb()` operation provides both a load and store barrier.

Two important points to note about the barrier operations:

1. The barriers relate to machine instructions, namely loads and stores. Thus, the higher-level language instruction `a = b` involves both a load (read) from location `b` and a store (write) to location `a`.
2. The `rmb`, `wmb`, and `mb` operations dictate the behavior of both the compiler and the processor. In the case of the compiler, the barrier operation dictates that the compiler not reorder instructions during the compile process. In the case of the processor, the barrier operation dictates that any instructions pending in the pipeline before the barrier must be committed for execution before any instructions encountered after the barrier.

The `barrier()` operation is a lighter-weight version of the `mb()` operation, in that it only controls the compiler's behavior. This would be useful if it is known that the processor will not perform undesirable reorderings. For example, the Intel x86 processors do not reorder writes.

**Table 6.5** Linux Memory Barrier Operations

| | |
|---|---|
| `rmb()` | Prevents loads from being reordered across the barrier |
| `wmb()` | Prevents stores from being reordered across the barrier |
| `mb()` | Prevents loads and stores from being reordered across the barrier |
| `barrier()` | Prevents the compiler from reordering loads or stores across the barrier |
| `smp_rmb()` | On SMP, provides a `rmb()` and on UP provides a `barrier()` |
| `smp_wmb()` | On SMP, provides a `wmb()` and on UP provides a `barrier()` |
| `smp_mb()` | On SMP, provides a `mb()` and on UP provides a `barrier()` |

*Note*: SMP = symmetric multiprocessor;
UP = uniprocessor

The smp_rmb, smp_wmb, and smp_mb operations provide an optimization for code that may be compiled on either a uniprocessor (UP) or a symmetric multiprocessor (SMP). These instructions are defined as the usual memory barriers for an SMP, but for a UP, they are all treated only as compiler barriers. The smp_ operations are useful in situations in which the data dependencies of concern will only arise in an SMP context.

***RCU (READ-COPY-UPDATE)*** The RCU (read-copy update) mechanism is an advanced lightweight synchronization mechanism which was integrated into the Linux kernel in 2002. The RCU is used widely in the Linux kernel, for example, in the Networking subsystem, the Memory subsystem, the virtual file system, and more. RCU is also used by other operating systems; and DragonFly BSD uses a mechanism that resembles Linux Sleepable RCU (SRCU). There is also a userspace RCU library called liburcu.

As opposed to common Linux synchronization mechanisms, RCU readers are not locked. The shared resources that the RCU mechanism protects must be accessed via a pointer. The RCU core API is quite small, and consists only of the six following methods:

- rcu_read_lock()
- rcu_read_unlock()
- call_rcu()
- synchronize_rcu()
- rcu_assign_pointer()
- rcu_dereference()

Apart from these methods, there are about 20 RCU application programming interface (API) minor methods.

The RCU mechanism provides access for multiple readers and writers to a shared resource; when a writer wants to update that resource, it creates a copy of it, updates it, and assigns the pointer to point to the new copy. Afterwards, the old version of the resource is freed, when it is no longer needed. Updating a pointer is an atomic operation. Hence, the reader can access that resource before or after the update is completed, but not during the update operation itself. In terms of performance, the RCU synchronization mechanism best suites scenarios when reads are frequent and writes are rare.

Access to a shared resource by readers must be encapsulated within rcu_read_lock()/rcu_read_unlock() block; moreover, access to the pointer (ptr) of the shared resource within that block must be done by rcu_dereference(ptr) and not by a direct access to it, and one should not invoke the rcu_dereference() method outside of such a block.

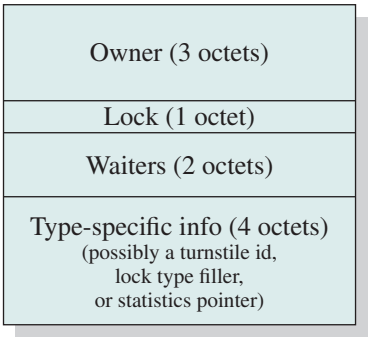
After a writer has created a copy and changed its value, the writer cannot free the old version until it is sure that all readers do not need it anymore. This can be done by calling synchronize_rcu(), or by calling the nonblocking method call_rcu(). The second parameter of the call_rcu() method references a callback, which will be invoked when the RCU mechanism is assured that the resource can be freed.

## 6.9 SOLARIS THREAD SYNCHRONIZATION PRIMITIVES

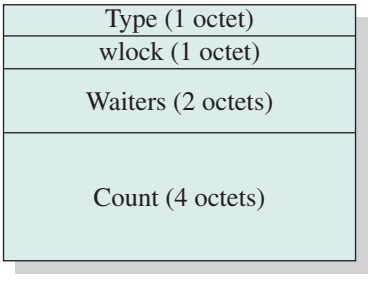
In addition to the concurrency mechanisms of UNIX SVR4, Solaris supports four thread synchronization primitives:

1. Mutual exclusion (mutex) locks
2. Semaphores
3. Multiple readers, single writer (readers/writer) locks
4. Condition variables

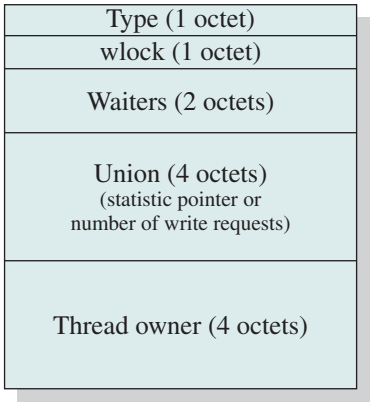
Solaris implements these primitives within the kernel for kernel threads; they are also provided in the threads library for user-level threads. Figure 6.15 shows the data structures for these primitives. The initialization functions for the primitives fill in some of the data members. Once a synchronization object is created, there are essentially only two operations that can be performed: enter (acquire lock) and release (unlock). There are no mechanisms in the kernel or the threads library to enforce mutual exclusion or to prevent deadlock. If a thread attempts to access a piece of data or code that is supposed to be protected but does not use the appropriate synchronization primitive, then such access occurs. If a thread locks an object and then fails to unlock it, no kernel action is taken.




(a) MUTEX lock




(b) Semaphore




(c) Reader/writer lock




(d) Condition variable

**Figure 6.15 Solaris Synchronization Data Structures**

All of the synchronization primitives require the existence of a hardware instruction that allows an object to be tested and set in one atomic operation.

## Mutual Exclusion Lock

A mutex is used to ensure that only one thread at a time can access the resource protected by the mutex. The thread that locks the mutex must be the one that unlocks it. A thread attempts to acquire a mutex lock by executing the `mutex_enter` primitive. If `mutex_enter` cannot set the lock (because it is already set by another thread), the blocking action depends on type-specific information stored in the mutex object. The default blocking policy is a spinlock: A blocked thread polls the status of the lock while executing in a busy waiting loop. An interrupt-based blocking mechanism is optional. In this latter case, the mutex includes a `turnstile id` that identifies a queue of threads sleeping on this lock.

The operations on a mutex lock are:

| | |
|---|---|
| `mutex_enter()` | Acquires the lock, potentially blocking if it is already held |
| `mutex_exit()` | Releases the lock, potentially unblocking a waiter |
| `mutex_tryenter()` | Acquires the lock if it is not already held |

The `mutex_tryenter()` primitive provides a nonblocking way of performing the mutual exclusion function. This enables the programmer to use a busy-wait approach for user-level threads, which avoids blocking the entire process because one thread is blocked.

## Semaphores

Solaris provides classic counting semaphores, with the following primitives:

| | |
|---|---|
| `sema_p()` | Decrements the semaphore, potentially blocking the thread |
| `sema_v()` | Increments the semaphore, potentially unblocking a waiting thread |
| `sema_tryp()` | Decrements the semaphore if blocking is not required |

Again, the `sema_tryp()` primitive permits busy waiting.

## Readers/Writer Lock

The readers/writer lock allows multiple threads to have simultaneous read-only access to an object protected by the lock. It also allows a single thread to access the object for writing at one time, while excluding all readers. When the lock is acquired for writing, it takes on the status of `write lock`: All threads attempting access for reading or writing must wait. If one or more readers have acquired the lock, its status is `read lock`. The primitives are as follows:

| | |
|---|---|
| `rw_enter()` | Attempts to acquire a lock as reader or writer |
| `rw_exit()` | Releases a lock as reader or writer |
| `rw_tryenter()` | Acquires the lock if blocking is not required |

| | |
|---|---|
| `rw_downgrade()` | A thread that has acquired a write lock converts it to a read lock. Any waiting writer remains waiting until this thread releases the lock. If there are no waiting writers, the primitive wakes up any pending readers. |
| `rw_tryupgrade()` | Attempts to convert a reader lock into a writer lock |

### Condition Variables

A condition variable is used to wait until a particular condition is true. Condition variables must be used in conjunction with a mutex lock. This implements a monitor of the type illustrated in Figure 6.14. The primitives are as follows:

| | |
|---|---|
| `cv_wait()` | Blocks until the condition is signaled |
| `cv_signal()` | Wakes up one of the threads blocked in `cv_wait()` |
| `cv_broadcast()` | Wakes up all of the threads blocked in `cv_wait()` |

`cv_wait()` releases the associated mutex before blocking and reacquires it before returning. Because reacquisition of the mutex may be blocked by other threads waiting for the mutex, the condition that caused the wait must be retested. Thus, typical usage is as follows:

```
mutex_enter(&m)
* *
while (some_condition) {
  cv_wait(&cv, &m);
}
* *
mutex_exit(&m);
```

This allows the condition to be a complex expression, because it is protected by the mutex.

## 6.10 WINDOWS CONCURRENCY MECHANISMS

Windows provides synchronization among threads as part of the object architecture. The most important methods of synchronization are Executive dispatcher objects, user-mode critical sections, slim reader–writer locks, condition variables, and lock-free operations. Dispatcher objects make use of wait functions. We will first describe wait functions, then look at the synchronization methods.

### Wait Functions

The wait functions allow a thread to block its own execution. The wait functions do not return until the specified criteria have been met. The type of wait function determines the set of criteria used. When a wait function is called, it checks

whether the wait criteria have been met. If the criteria have not been met, the calling thread enters the wait state. It uses no processor time while waiting for the criteria to be met.

The most straightforward type of wait function is one that waits on a single object. The `WaitForSingleObject` function requires a handle to one synchronization object. The function returns when one of the following occurs:

- The specified object is in the signaled state.
- The time-out interval elapses. The time-out interval can be set to `INFINITE` to specify that the wait will not time out.

## Dispatcher Objects

The mechanism used by the Windows Executive to implement synchronization facilities is the family of dispatcher objects, which are listed with brief descriptions in Table 6.6.

The first five object types in the table are specifically designed to support synchronization. The remaining object types have other uses, but also may be used for synchronization.

Each dispatcher object instance can be in either a signaled or an unsignaled state. A thread can be blocked on an object in an unsignaled state; the thread is released

**Table 6.6** Windows Synchronization Objects

| Object Type | Definition | Set to Signaled State When | Effect on Waiting Threads |
|---|---|---|---|
| Notification event | An announcement that a system event has occurred | Thread sets the event | All released |
| Synchronization event | An announcement that a system event has occurred | Thread sets the event | One thread released |
| Mutex | A mechanism that provides mutual exclusion capabilities; equivalent to a binary semaphore | Owning thread or other thread releases the mutex | One thread released |
| Semaphore | A counter that regulates the number of threads that can use a resource | Semaphore count drops to zero | All released |
| Waitable timer | A counter that records the passage of time | Set time arrives or time interval expires | All released |
| File | An instance of an opened file or I/O device | I/O operation completes | All released |
| Process | A program invocation, including the address space and resources required to run the program | Last thread terminates | All released |
| Thread | An executable entity within a process | Thread terminates | All released |

*Note:* Shaded rows correspond to objects that exist for the sole purpose of synchronization.

when the object enters the signaled state. The mechanism is straightforward: A thread issues a wait request to the Windows Executive, using the handle of the synchronization object. When an object enters the signaled state, the Windows Executive releases one or all of the thread objects that are waiting on that dispatcher object.

The **event object** is useful in sending a signal to a thread indicating that a particular event has occurred. For example, in overlapped input and output, the system sets a specified event object to the signaled state when the overlapped operation has been completed. The **mutex object** is used to enforce mutually exclusive access to a resource, allowing only one thread object at a time to gain access. It therefore functions as a binary semaphore. When the mutex object enters the signaled state, only one of the threads waiting on the mutex is released. Mutexes can be used to synchronize threads running in different processes. Like mutexes, **semaphore objects** may be shared by threads in multiple processes. The Windows semaphore is a counting semaphore. In essence, the **waitable timer object** signals at a certain time and/or at regular intervals.

## Critical Sections

Critical sections provide a synchronization mechanism similar to that provided by mutex objects, except that critical sections can be used only by the threads of a single process. Event, mutex, and semaphore objects can also be used in a single-process application, but critical sections provide a much faster, more efficient mechanism for mutual–exclusion synchronization.

The process is responsible for allocating the memory used by a critical section. Typically, this is done by simply declaring a variable of type `CRITICAL_SECTION`. Before the threads of the process can use it, initialize the critical section by using the `InitializeCriticalSection` function.

A thread uses the `EnterCriticalSection` or `TryEnterCriticalSection` function to request ownership of a critical section. It uses the `LeaveCriticalSection` function to release ownership of a critical section. If the critical section is currently owned by another thread, `EnterCriticalSection` waits indefinitely for ownership. In contrast, when a mutex object is used for mutual exclusion, the wait functions accept a specified time-out interval. The `TryEnterCriticalSection` function attempts to enter a critical section without blocking the calling thread.

Critical sections use a sophisticated algorithm when trying to acquire the mutex. If the system is a multiprocessor, the code will attempt to acquire a spinlock. This works well in situations where the critical section is acquired for only a short time. Effectively the spinlock optimizes for the case where the thread that currently owns the critical section is executing on another processor. If the spinlock cannot be acquired within a reasonable number of iterations, a dispatcher object is used to block the thread so that the Kernel can dispatch another thread onto the processor.

The dispatcher object is only allocated as a last resort. Most critical sections are needed for correctness, but in practice are rarely contended. By lazily

allocating the dispatcher object, the system saves significant amounts of kernel virtual memory.

### Slim Reader–Writer Locks and Condition Variables

Windows Vista added a user-mode reader–writer. Like critical sections, the reader–writer lock enters the kernel to block only after attempting to use a spinlock. It is *slim* in the sense that it normally only requires allocation of a single pointer-sized piece of memory.

To use an SRW lock, a process declares a variable of type SRWLOCK and a calls `InitializeSRWLock` to initialize it. Threads call `AcquireSRWLockExclusive` or `AcquireSRWLockShared` to acquire the lock and `ReleaseSRWLockExclusive` or `ReleaseSRWLockShared` to release it.

Windows also has condition variables. The process must declare a CONDITION_VARIABLE and initialize it in some thread by calling `InitializeConditionVariable`. Condition variables can be used with either critical sections or SRW locks, so there are two methods, `SleepConditionVariableCS` and `SleepConditionVariableSRW`, which sleep on the specified condition and release the specified lock as an atomic operation.

There are two wake methods, `WakeConditionVariable` and `WakeAllConditionVariable`, which wake one or all of the sleeping threads, respectively. Condition variables are used as follows:

1. Acquire exclusive lock
2. While (predicate() == FALSE) SleepConditionVariable()
3. Perform the protected operation
4. Release the lock

### Lock-free Synchronization

Windows also relies heavily on interlocked operations for synchronization. Interlocked operations use hardware facilities to guarantee that memory locations can be read, modified, and written in a single atomic operation. Examples include `InterlockedIncrement` and `InterlockedCompareExchange`; the latter allows a memory location to be updated only if it hasn't changed values since being read.

Many of the synchronization primitives use interlocked operations within their implementation, but these operations are also available to programmers for situations where they want to synchronize without taking a software lock. These so-called *lock-free* synchronization primitives have the advantage that a thread can never be switched away from a processor (say at the end of its timeslice) while still holding a lock. Thus, they cannot block another thread from running.

More complex lock-free primitives can be built out of the interlocked operations, most notably Windows SLists, which provide a lock-free LIFO queue. SLists are managed using functions like `InterlockedPushEntrySList` and `InterlockedPopEntrySList`.

## 6.11 ANDROID INTERPROCESS COMMUNICATION

The Linux kernel includes a number of features that can be used for interprocess communication (IPC), including pipes, shared memory, messages, sockets, semaphores, and signals. Android does not use these features for IPC, but rather adds to the kernel a new capability known as Binder. Binder provides a lightweight remote procedure call (RPC) capability that is efficient in terms of both memory and processing requirements, and is well suited to the requirements of an embedded system.

The Binder is used to mediate all interaction between two processes. A component in one process (the client) issues a call. This call is directed to the Binder in the kernel, which passes the call on to the destination component in the destination process (the service). The return from the destination goes through the Binder and is delivered to the calling component in the calling process.

Traditionally, the term *RPC* referred to a call/return interaction between a client process on one machine and a server process on another machine. In the Android case, the RPC mechanism works between two processes on the same system, but running on different virtual machines.

The method used for communicating with the Binder is the ioctl system call. The ioctl call is a general-purpose system call for device-specific I/O operations. It can be used to access device drivers and also what are called pseudo-device drivers, of which Binder is an example. A pseudo-device driver uses the same general interface as a device driver, but is used to control some kernel function. The ioctl call includes as parameters the command to be performed and the appropriate arguments.

Figure 6.16 illustrates a typical use of the Binder. The dashed vertical lines represent threads in a process. Before a process can make use of a service, that service must be known. A process that hosts a service will spawn multiple threads so it can handle multiple requests concurrently. Each thread makes itself known to the Binder by a blocking ioctl.

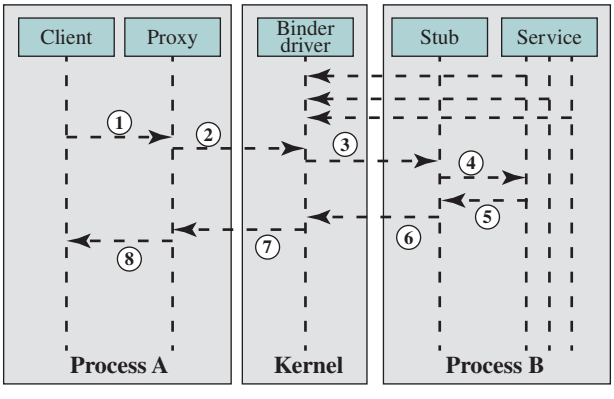



**Figure 6.16 Binder Operation**

The interaction proceeds as follows:

1. A client component, such as an activity, invokes a service in the form of a call with argument data.
2. The call invokes a proxy, which is able to translate the call into a transaction with the Binder driver. The proxy performs a procedure called **marshalling**, which converts higher-level applications data structures (i.e., request/response parameters) into a **parcel**. The parcel is a container for a message (data and object references) that can be sent through the Binder driver. The proxy then submits the transaction to the binder by a blocking ioctl call.
3. The Binder sends a signal to the target thread that wakes the thread up from its blocking ioctl call. The parcel is delivered to a stub component in the target process.
4. The stub performs a procedure called **unmarshalling**, which reconstructs higher-level application data structures from parcels received through binder transactions. The proxy then calls the service component with a call that is identical to the call issued by the client component.
5. The called service component returns the appropriate result to the stub.
6. The stub marshals the return data into a reply parcel then submits the reply parcel to the Binder via an ioctl.
7. The Binder wakes up the calling ioctl in the client proxy, which gets the transaction reply data.
8. The proxy unmarshals the result from the reply parcel and returns the result to the client component that issued the service call.

## 6.12 SUMMARY

Deadlock is the blocking of a set of processes that either compete for system resources or communicate with each other. The blockage is permanent unless the OS takes some extraordinary action, such as killing one or more processes, or forcing one or more processes to backtrack. Deadlock may involve reusable resources or consumable resources. A reusable resource is one that is not depleted or destroyed by use, such as an I/O channel or a region of memory. A consumable resource is one that is destroyed when it is acquired by a process, such as messages and information in I/O buffers.

There are three general approaches to dealing with deadlock: prevention, detection, and avoidance. Deadlock prevention guarantees that deadlock will not occur, by assuring that one of the necessary conditions for deadlock is not met. Deadlock detection is needed if the OS is always willing to grant resource requests; periodically, the OS must check for deadlock and take action to break the deadlock. Deadlock avoidance involves the analysis of each new resource request to determine if it could lead to deadlock, and granting it only if deadlock is not possible.

## 6.13 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

### Key Terms

| | | |
|---|---|---|
| banker's algorithm | fatal region | preemption |
| circular wait | hold and wait | resource allocation graph |
| consumable resource | joint progress diagram | reusable resource |
| deadlock | memory barrier | safe state |
| deadlock avoidance | message | spinlock |
| deadlock detection | mutual exclusion | starvation |
| deadlock prevention | pipe | unsafe state |

### Review Questions

**6.1.** Give examples of reusable and consumable resources.
**6.2.** What are the three conditions that must be present for deadlock to be possible?
**6.3.** What are the four conditions that create deadlock?
**6.4.** How can the hold-and-wait condition be prevented?
**6.5.** Why can't you disallow mutual exclusion in order to prevent deadlocks?
**6.6.** How can the circular wait condition be prevented?
**6.7.** List some of the methods that may be adopted to recover from deadlocks.

### Problems

**6.1.** Show that the four conditions of deadlock apply to Figure 6.1a.
**6.2.** Show how each of the techniques of prevention, avoidance, and detection can be applied to Figure 6.1.
**6.3.** For Figure 6.3, provide a narrative description of each of the six depicted paths, similar to the description of the paths of Figure 6.2 provided in Section 6.1.
**6.4.** Give two alternative execution sequences for the situation depicted in Figure 6.3, showing that deadlock does not occur.
**6.5.** Given the following state of a system:
The system comprises of five processes and four resources.
P1–P5 denotes the set of processes.
R1–R4 denotes the set of resources.
Total Existing Resources:

| R1 | R2 | R3 | R4 |
|---|---|---|---|
| 6 | 3 | 4 | 3 |

Snapshot at the initial time stage:

| | Allocation | | | | Claim | | | |
|---|---|---|---|---|---|---|---|---|
| | R1 | R2 | R3 | R4 | R1 | R2 | R3 | R4 |
| P1 | 3 | 0 | 1 | 1 | 6 | 2 | 1 | 1 |
| P2 | 0 | 1 | 0 | 0 | 0 | 2 | 1 | 2 |
| P3 | 1 | 1 | 1 | 0 | 3 | 2 | 1 | 0 |
| P4 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 1 |
| P5 | 0 | 0 | 0 | 0 | 2 | 1 | 1 | 1 |

**a.** Compute the Available vector.

**b.** Compute the Need Matrix.

**c.** Is the current allocation state safe? If so, give a safe sequence of the process. In addition, show how the Available (working array) changes as each process terminates.

**d.** If the request (1, 1, 0, 0) from P1 arrives, will it be correct to grant the request? Justify your decision.

**6.6.** In the code below, three processes are competing for six resources labeled A to F.

**a.** Using a resource allocation graph (see Figures 6.5 and 6.6), show the possibility of a deadlock in this implementation.

**b.** Modify the order of some of the get requests to prevent the possibility of any deadlock. You cannot move requests across procedures, only change the order inside each procedure. Use a resource allocation graph to justify your answer.

```
void P0()
{
 while (true) {
 get(A);
 get(B);
 get(C);
 // critical region:
 // use A, B, C
 release(A);
 release(B);
 release(C);
 }
}
```

```
void P1()
{
 while (true) {
 get(D);
 get(E);
 get(B);
 // critical region:
 // use D, E, B
 release(D);
 release(E);
 release(B);
 }
}
```

```
void P2()
{
 while (true) {
 get(C);
 get(F);
 get(D);
 // critical region:
 // use C, F, D
 release(C);
 release(F);
 release(D);
 }
}
```

**6.7.** A spooling system (see Figure 6.17) consists of an input process I, a user process P, and an output process O connected by two buffers. The processes exchange data in blocks of equal size. These blocks are buffered on a disk using a floating boundary between the input and the output buffers, depending on the speed of the processes.

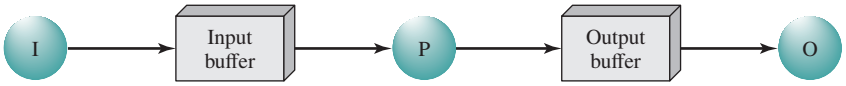



**Figure 6.17 A Spooling System**

The communication primitives used ensure that the following resource constraint is satisfied:

$$i + o \leq \max$$

where

max = maximum number of blocks on disk
$i$ = number of input blocks on disk
$o$ = number of output blocks on disk

The following is known about the processes:

1. As long as the environment supplies data, process I will eventually input it to the disk (provided disk space becomes available).
2. As long as input is available on the disk, process P will eventually consume it and output a finite amount of data on the disk for each block input (provided disk space becomes available).
3. As long as output is available on the disk, process O will eventually consume it.

Show that this system can become deadlocked.

**6.8.** Suggest an additional resource constraint that will prevent the deadlock in Problem 6.7, but still permit the boundary between input and output buffers to vary in accordance with the present needs of the processes.

**6.9.** In the THE multiprogramming system [DIJK68], a drum (precursor to the disk for secondary storage) is divided into input buffers, processing areas, and output buffers, with floating boundaries, depending on the speed of the processes involved. The current state of the drum can be characterized by the following parameters:

$max$ = maximum number of pages on drum
$i$ = number of input pages on drum
$p$ = number of processing pages on drum
$o$ = number of output pages on drum
$reso$ = minimum number of pages reserved for output
$resp$ = minimum number of pages reserved for processing

Formulate the necessary resource constraints that guarantee that the drum capacity is not exceeded, and that a minimum number of pages is reserved permanently for output and processing.

**6.10.** In the THE multiprogramming system, a page can make the following state transitions:

| | |
|---|---|
| **1.** empty → input buffer | (input production) |
| **2.** input buffer → processing area | (input consumption) |
| **3.** processing area → output buffer | (output production) |
| **4.** output buffer → empty | (output consumption) |
| **5.** empty → processing area | (procedure call) |
| **6.** processing area → empty | (procedure return) |

**a.** Define the effect of these transitions in terms of the quantities $i$, $o$, and $p$.

**b.** Can any of them lead to a deadlock if the assumptions made in Problem 6.6 about input processes, user processes, and output processes hold?

**6.11.** At an instant, the resource allocation state in a system is as follows:

4 processes P1–P4

4 resource types: R1–R4

R1 (5 instances), R2 (3 instances), R3 (3 instances), R4 (3 instance)

Snapshot at time $T_0$:

| | Allocation | | | | Request | | | | Available | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | R1 | R2 | R3 | R4 | R1 | R2 | R3 | R4 | R1 | R2 | R3 | R4 |
| P1 | 0 | 0 | 1 | 0 | 2 | 0 | 0 | 2 | 2 | 1 | 1 | 2 |
| P2 | 2 | 0 | 0 | 1 | 1 | 3 | 0 | 1 | | | | |
| P3 | 0 | 1 | 1 | 0 | 2 | 1 | 1 | 0 | | | | |
| P4 | 1 | 1 | 0 | 0 | 4 | 0 | 3 | 1 | | | | |

Run the deadlock detection algorithm and test whether the system is deadlocked or not. If it is, identify the processes that are deadlocked.

**6.12.** Suggest a deadlock recovery strategy for the situation depicted in Figure 6.10.

**6.13.** A pipeline algorithm is implemented so a stream of data elements of type T produced by a process $P_0$ passes through a sequence of processes $P_1, P_2, \ldots, P_{n-1}$, which operates on the elements in that order.

**a.** Define a generalized message buffer that contains all the partially consumed data elements, and write an algorithm for process $Pi$ ($0 \leq i \leq n - 1$), of the form

**repeat**
receive from predecessor;
consume element;
send to successor:
**forever**

Assume $P_0$ receives input elements sent by $P_{n-1}$. The algorithm should enable the processes to operate directly on messages stored in the buffer so copying is unnecessary.

**b.** Show that the processes cannot be deadlocked with respect to the common buffer.

**6.14.** Suppose the following two processes, foo and bar, are executed concurrently and share the semaphore variables S and R (each initialized to 1) and the integer variable x (initialized to 0).

```
void foo( ) {
   do {
     semWait(S);
     semWait(R);
     x++;
     semSignal(S);
     SemSignal(R);
  } while (1);
}
```

```
void bar( ) {
do {
    semWait(R);
    semWait(S);
    x--;
    semSignal(S;
    SemSignal(R);
} while (1);
}
```

**a.** Can the concurrent execution of these two processes result in one or both being blocked forever? If yes, give an execution sequence in which one or both are blocked forever.

**b.** Can the concurrent execution of these two processes result in the indefinite postponement of one of them? If yes, give an execution sequence in which one is indefinitely postponed.

**6.15.** Consider a system consisting of four processes and 9 instances of a single resource. The current state of the claim (C) and allocation (A) matrices is:

$$C = \begin{pmatrix} 2 \\ 6 \\ 9 \\ 5 \end{pmatrix} \quad A = \begin{pmatrix} 1 \\ 2 \\ 2 \\ 3 \end{pmatrix}$$

Is the system in a safe state? If so, will it remain in a safe state if the available resources are allocated to the last process in sequence?

**6.16.** Consider the following ways of handling deadlock: (1) banker's algorithm, (2) detect deadlock and kill thread, releasing all resources, (3) reserve all resources in advance, (4) restart thread and release all resources if thread needs to wait, (5) resource ordering, and (6) detect deadlock and roll back thread's actions.

**a.** One criterion to use in evaluating different approaches to deadlock is which approach permits the greatest concurrency. In other words, which approach allows the most threads to make progress without waiting when there is no deadlock? Give a rank order from 1 to 6 for each of the ways of handling deadlock just listed, where 1 allows the greatest degree of concurrency. Comment on your ordering.

**b.** Another criterion is efficiency; in other words, which requires the least processor overhead. Rank order the approaches from 1 to 6, with 1 being the most efficient, assuming deadlock is a very rare event. Comment on your ordering. Does your ordering change if deadlocks occur frequently?

**6.17.** Consider a variation of the dining philosophers problem where the number of philosophers is even. Can you devise a deadlock-free solution to the problem? Assume that all other requirements are like those in the original problem.

**6.18.** Suppose there are two types of philosophers. One type always picks up his left fork first (a "lefty"), and the other type always picks up his right fork first (a "righty"). The behavior of a lefty is defined in Figure 6.12. The behavior of a righty is as follows:

```
begin
  repeat
       think;
       wait ( fork[ (i+1)  mod  5] );
       wait ( fork[i] );
       eat;
       signal ( fork[i] );
       signal ( fork[ (i+1)  mod  5] );
  forever
end;
```

Prove the following:

**a.** Any seating arrangement of lefties and righties with at least one of each avoids deadlock.

**b.** Any seating arrangement of lefties and righties with at least one of each prevents starvation.

**6.19.** Figure 6.18 shows another solution to the dining philosophers problem using monitors. Compare to Figure 6.14 and report your conclusions.

```
monitor dining_controller;
enum states {thinking, hungry, eating} state[5];
cond needFork[5]                                /* condition variable */

void get_forks(int pid)          /* pid is the philosopher id number */
{
    state[pid] = hungry;                 /* announce that I'm hungry */
    if (state[(pid+1) % 5] == eating || (state[(pid-1) % 5] == eating)
    cwait(needFork[pid]);       /* wait if either neighbor is eating */
    state[pid] = eating;    /* proceed if neither neighbor is eating */
}
void  release_forks(int  pid)
{
    state[pid] = thinking;
    /* give right (higher) neighbor a chance to eat */
    if  (state[(pid+1) % 5] == hungry) && (state[(pid+2)
    % 5]) != eating)
    csignal(needFork[pid+1]);
    /* give left (lower) neighbor a chance to eat */
    else if  (state[(pid-1) % 5] == hungry) && (state[(pid-2)
    % 5]) != eating)
    csignal(needFork[pid-1]);
}
void  philosopher[k=0 to 4]          /* the five philosopher clients */
{
    while  (true) {
      <think>;
      get_forks(k);          /* client requests two forks via monitor */
      <eat spaghetti>;
      release_forks(k);      /* client releases forks via the monitor */
    }
}
```

VideoNote

**Figure 6.18 Another Solution to the Dining Philosophers Problem Using a Monitor**

**6.20.** Some of the Linux atomic operations are listed in Table 6.2. Can you identify some benefits of implementing these operations in uniprocessor and multiprocessor systems? Write a simple program depicting the use of an atomic integer data type in implementing counters.

**6.21.** Consider the following fragment of code on a Linux system.

```
read_lock(&mr_rwlock);
write_lock(&mr_rwlock);
```

Where `mr_rwlock` is a reader–writer lock. What is the effect of this code?

**6.22.** The two variables `a` and `b` have initial values of 1 and 2, respectively. The following code is for a Linux system:

| Thread 1 | Thread 2 |
|---|---|
| `a = 3;` | — |
| `mb();` | — |
| `b = 4;` | `c = b;` |
| — | `rmb();` |
| — | `d = a;` |

What possible errors are avoided by the use of the memory barriers?

# PART 3 Memory

## CHAPTER 7

# MEMORY MANAGEMENT


**7.1 Memory Management Requirements**
- Relocation
- Protection
- Sharing
- Logical Organization
- Physical Organization

**7.2 Memory Partitioning**
- Fixed Partitioning
- Dynamic Partitioning
- Buddy System
- Relocation

**7.3 Paging**

**7.4 Segmentation**

**7.5 Summary**

**7.6 Key Terms, Review Questions, and Problems**

**APPENDIX 7A Loading and Linking**

### Learning Objectives

After studying this chapter, you should be able to:

- Discuss the principal requirements for memory management.
- Understand the reason for memory partitioning and explain the various techniques that are used.
- Understand and explain the concept of paging.
- Understand and explain the concept of segmentation.
- Assess the relative advantages of paging and segmentation.
- Describe the concepts of loading and linking.

In a uniprogramming system, main memory is divided into two parts: one part for the operating system (resident monitor, kernel) and other part for the program currently being executed. In a multiprogramming system, the "user" part of memory must be further subdivided to accommodate multiple processes. The task of subdivision is carried out dynamically by the operating system and is known as **memory management**.

Effective memory management is vital in a multiprogramming system. If only a few processes are in memory, then for much of the time all of the processes will be waiting for I/O (input/output), and the processor will be idle. Thus, memory needs to be allocated to ensure a reasonable supply of ready processes to consume available processor time.

We begin with the requirements that memory management is intended to satisfy. Next, we will discuss a variety of simple schemes that have been used for memory management.

Table 7.1 introduces some key terms for our discussion.

## 7.1 MEMORY MANAGEMENT REQUIREMENTS

While surveying the various mechanisms and policies associated with memory management, it is helpful to keep in mind the requirements that memory management is intended to satisfy. These requirements include the following:

- Relocation
- Protection

**Table 7.1** Memory Management Terms

| | |
|---|---|
| **Frame** | A fixed-length block of main memory. |
| **Page** | A fixed-length block of data that resides in secondary memory (such as a disk). A page of data may temporarily be copied into a frame of main memory. |
| **Segment** | A variable-length block of data that resides in secondary memory. An entire segment may temporarily be copied into an available region of main memory (segmentation) or the segment may be divided into pages, which can be individually copied into main memory (combined segmentation and paging). |

- Sharing
- Logical organization
- Physical organization

## Relocation

In a multiprogramming system, the available main memory is generally shared among a number of processes. Typically, it is not possible for the programmer to know in advance which other programs will be resident in main memory at the time of execution of his or her program. In addition, we would like to be able to swap active processes in and out of main memory to maximize processor utilization by providing a large pool of ready processes to execute. Once a program is swapped out to disk, it would be quite limiting to specify that when it is next swapped back in, it must be placed in the same main memory region as before. Instead, we may need to **relocate** the process to a different area of memory.

Thus, we cannot know ahead of time where a program will be placed, and we must allow for the possibility that the program may be moved about in main memory due to swapping. These facts raise some technical concerns related to addressing, as illustrated in Figure 7.1. The figure depicts a process image. For simplicity, let us assume that the process image occupies a contiguous region of main memory. Clearly, the operating system will need to know the location of process control information and of the execution stack, as well as the entry point to begin execution of the program for this process. Because the operating system is managing memory and is responsible for bringing this process into main memory, these addresses are easy to come by. In addition, however, the processor must deal with memory references

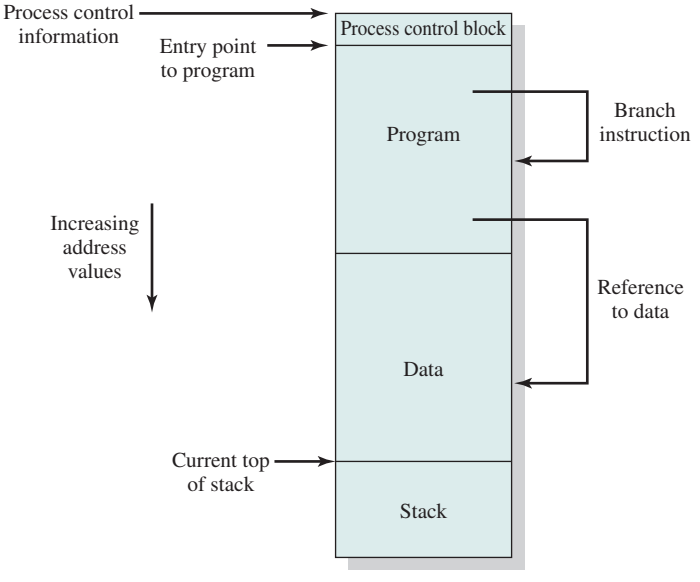



**Figure 7.1 Addressing Requirements for a Process**

within the program. Branch instructions contain an address to reference the instruction to be executed next. Data reference instructions contain the address of the byte or word of data referenced. Somehow, the processor hardware and operating system software must be able to translate the memory references found in the code of the program into actual physical memory addresses, reflecting the current location of the program in main memory.

## Protection

Each process should be protected against unwanted interference by other processes, whether accidental or intentional. Thus, programs in other processes should not be able to reference memory locations in a process for reading or writing purposes without permission. In one sense, satisfaction of the relocation requirement increases the difficulty of satisfying the protection requirement. Because the location of a program in main memory is unpredictable, it is impossible to check absolute addresses at compile time to assure protection. Furthermore, most programming languages allow the dynamic calculation of addresses at run time (e.g., by computing an array subscript or a pointer into a data structure). Hence, all memory references generated by a process must be checked at run time to ensure they refer only to the memory space allocated to that process. Fortunately, we shall see that mechanisms that support relocation also support the protection requirement.

Normally, a user process cannot access any portion of the operating system, neither program nor data. Again, usually a program in one process cannot branch to an instruction in another process. Without special arrangement, a program in one process cannot access the data area of another process. The processor must be able to abort such instructions at the point of execution.

Note the memory protection requirement must be satisfied by the processor (hardware) rather than the operating system (software). This is because the OS cannot anticipate all of the memory references that a program will make. Even if such anticipation were possible, it would be prohibitively time consuming to screen each program in advance for possible memory-reference violations. Thus, it is only possible to assess the permissibility of a memory reference (data access or branch) at the time of execution of the instruction making the reference. To accomplish this, the processor hardware must have that capability.

## Sharing

Any protection mechanism must have the flexibility to allow several processes to access the same portion of main memory. For example, if a number of processes are executing the same program, it is advantageous to allow each process to access the same copy of the program, rather than have its own separate copy. Processes that are cooperating on some task may need to share access to the same data structure. The memory management system must therefore allow controlled access to shared areas of memory without compromising essential protection. Again, we will see that the mechanisms used to support relocation also support sharing capabilities.

## Logical Organization

Almost invariably, main memory in a computer system is organized as a linear or one-dimensional address space, consisting of a sequence of bytes or words. Secondary memory, at its physical level, is similarly organized. While this organization closely mirrors the actual machine hardware, it does not correspond to the way in which programs are typically constructed. Most programs are organized into modules, some of which are unmodifiable (read only, execute only) and some of which contain data that may be modified. If the operating system and computer hardware can effectively deal with user programs and data in the form of modules of some sort, then a number of advantages can be realized:

1. Modules can be written and compiled independently, with all references from one module to another resolved by the system at run time.
2. With modest additional overhead, different degrees of protection (read only, execute only) can be given to different modules.
3. It is possible to introduce mechanisms by which modules can be shared among processes. The advantage of providing sharing on a module level is that this corresponds to the user's way of viewing the problem, hence it is easy for the user to specify the sharing that is desired.

The tool that most readily satisfies these requirements is segmentation, which is one of the memory management techniques explored in this chapter.

## Physical Organization

As we discussed in Section 1.5, computer memory is organized into at least two levels, referred to as main memory and secondary memory. Main memory provides fast access at relatively high cost. In addition, main memory is volatile; that is, it does not provide permanent storage. Secondary memory is slower and cheaper than main memory, but is usually not volatile. Thus, secondary memory of large capacity can be provided for long-term storage of programs and data, while a smaller main memory holds programs and data currently in use.

In this two-level scheme, the organization of the flow of information between main and secondary memory is a major system concern. The responsibility for this flow could be assigned to the individual programmer, but this is impractical and undesirable for two reasons:

1. The main memory available for a program and its data may be insufficient. In that case, the programmer must engage in a practice known as **overlaying**, in which the program and data are organized in such a way that various modules can be assigned the same region of memory, with a main program responsible for switching the modules in and out as needed. Even with the aid of compiler tools, overlay programming wastes programmer time.
2. In a multiprogramming environment, the programmer does not know at the time of coding how much space will be available or where that space will be.

It is clear, then, that the task of moving information between the two levels of memory should be a system responsibility. This task is the essence of memory management.

## 7.2 MEMORY PARTITIONING

The principal operation of memory management is to bring processes into main memory for execution by the processor. In almost all modern multiprogramming systems, this involves a sophisticated scheme known as virtual memory. Virtual memory is, in turn, based on the use of one or both of two basic techniques: segmentation and paging. Before we can look at these virtual memory techniques, we must prepare the ground by looking at simpler techniques that do not involve virtual memory (Table 7.2 summarizes all the techniques examined in this chapter and the next). One of these techniques, partitioning, has been used in several variations in some now-obsolete operating systems. The other two techniques, simple paging and simple segmentation, are not used by themselves. However, it will clarify the discussion of virtual memory if we look first at these two techniques in the absence of virtual memory considerations.

### Fixed Partitioning

In most schemes for memory management, we can assume the OS occupies some fixed portion of main memory, and the rest of main memory is available for use by multiple processes. The simplest scheme for managing this available memory is to partition it into regions with fixed boundaries.

***Partition Sizes*** Figure 7.2 shows examples of two alternatives for fixed partitioning. One possibility is to make use of equal-size partitions. In this case, any process whose size is less than or equal to the partition size can be loaded into any available partition. If all partitions are full, and no process is in the Ready or Running state, the operating system can swap a process out of any of the partitions and load in another process, so there is some work for the processor.

There are two difficulties with the use of equal-size fixed partitions:

- A program may be too big to fit into a partition. In this case, the programmer must design the program with the use of overlays so only a portion of the program need be in main memory at any one time. When a module is needed that is not present, the user's program must load that module into the program's partition, overlaying whatever programs or data are there.
- Main memory utilization is extremely inefficient. Any program, no matter how small, occupies an entire partition. In our example, there may be a program whose length is less than 2 Mbytes; yet it occupies an 8-Mbyte partition whenever it is swapped in. This phenomenon, in which there is wasted space internal to a partition due to the fact that the block of data loaded is smaller than the partition, is referred to as **internal fragmentation**.

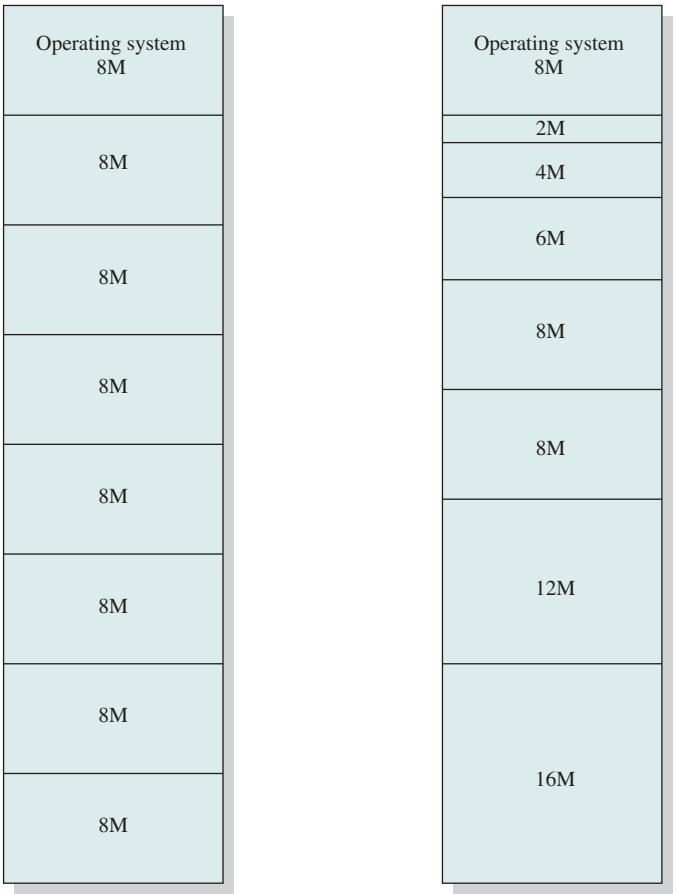
Both of these problems can be lessened, though not solved, by using unequal-size partitions (see Figure 7.2b). In this example, programs as large as 16 Mbytes can

**Table 7.2** Memory Management Techniques

| Technique | Description | Strengths | Weaknesses |
|---|---|---|---|
| **Fixed Partitioning** | Main memory is divided into a number of static partitions at system generation time. A process may be loaded into a partition of equal or greater size. | Simple to implement; little operating system overhead. | Inefficient use of memory due to internal fragmentation; maximum number of active processes is fixed. |
| **Dynamic Partitioning** | Partitions are created dynamically, so each process is loaded into a partition of exactly the same size as that process. | No internal fragmentation; more efficient use of main memory. | Inefficient use of processor due to the need for compaction to counter external fragmentation. |
| **Simple Paging** | Main memory is divided into a number of equal-size frames. Each process is divided into a number of equal-size pages of the same length as frames. A process is loaded by loading all of its pages into available, not necessarily contiguous, frames. | No external fragmentation. | A small amount of internal fragmentation. |
| **Simple Segmentation** | Each process is divided into a number of segments. A process is loaded by loading all of its segments into dynamic partitions that need not be contiguous. | No internal fragmentation; improved memory utilization and reduced overhead compared to dynamic partitioning. | External fragmentation. |
| **Virtual Memory Paging** | As with simple paging, except that it is not necessary to load all of the pages of a process. Nonresident pages that are needed are automatically brought in later. | No external fragmentation; higher degree of multiprogramming; large virtual address space. | Overhead of complex memory management. |
| **Virtual Memory Segmentation** | As with simple segmentation, except that it is not necessary to load all of the segments of a process. Nonresident segments that are needed are automatically brought in later. | No internal fragmentation, higher degree of multiprogramming; large virtual address space; protection and sharing support. | Overhead of complex memory management. |

be accommodated without overlays. Partitions smaller than 8 Mbytes allow smaller programs to be accommodated with less internal fragmentation.

***Placement Algorithm*** With equal-size partitions, the placement of processes in memory is trivial. As long as there is any available partition, a process can be loaded into that partition. Because all partitions are of equal size, it does not matter which partition is used. If all partitions are occupied with processes that are not ready




(a) Equal-size partitions (b) Unequal-size partitions

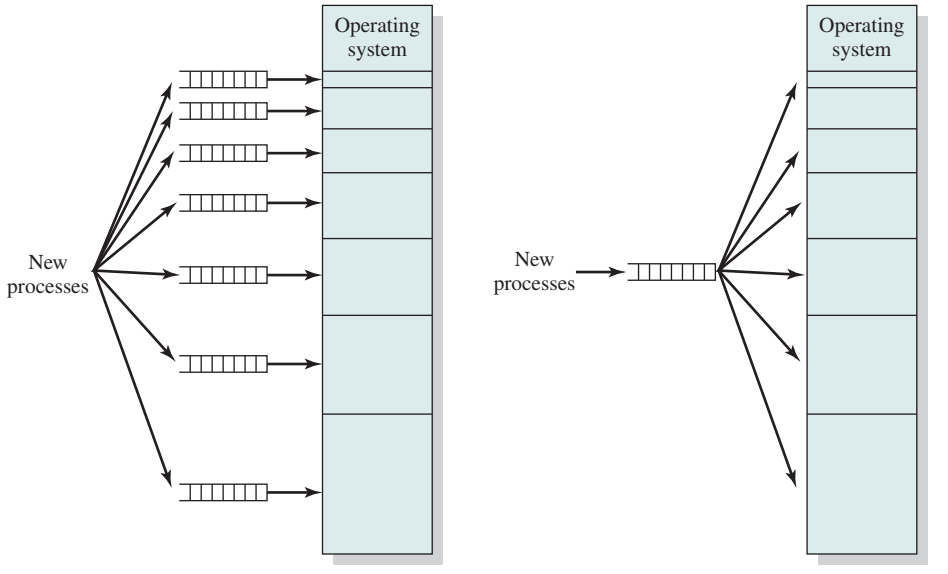
**Figure 7.2 Example of Fixed Partitioning of a 64-Mbyte Memory**

to run, then one of these processes must be swapped out to make room for a new process. Which one to swap out is a scheduling decision; this topic will be explored in Part Four.

With unequal-size partitions, there are two possible ways to assign processes to partitions. The simplest way is to assign each process to the smallest partition within which it will fit.[1] In this case, a scheduling queue is needed for each partition to hold swapped-out processes destined for that partition (see Figure 7.3a). The advantage of this approach is that processes are always assigned in such a way as to minimize wasted memory within a partition (internal fragmentation).

Although this technique seems optimum from the point of view of an individual partition, it is not optimum from the point of view of the system as a whole.

[1]This assumes one knows the maximum amount of memory that a process will require. This is not always the case. If it is not known how large a process may become, the only alternatives are an overlay scheme or the use of virtual memory.




(a) One process queue per partition (b) Single queue

**Figure 7.3 Memory Assignment for Fixed Partitioning**

In Figure 7.2b, for example, consider a case in which there are no processes with a size between 12 and 16M at a certain point in time. In that case, the 16M partition will remain unused, even though some smaller process could have been assigned to it. Thus, a preferable approach would be to employ a single queue for all processes (see Figure 7.3b). When it is time to load a process into main memory, the smallest available partition that will hold the process is selected. If all partitions are occupied, then a swapping decision must be made. Preference might be given to swapping out of the smallest partition that will hold the incoming process. It is also possible to consider other factors, such as priority, and a preference for swapping out blocked processes versus ready processes.

The use of unequal-size partitions provides a degree of flexibility to fixed partitioning. In addition, it can be said that fixed partitioning schemes are relatively simple and require minimal OS software and processing overhead. However, there are disadvantages:

- The number of partitions specified at system generation time limits the number of active (not suspended) processes in the system.
- Because partition sizes are preset at system generation time, small jobs will not utilize partition space efficiently. In an environment where the main storage requirement of all jobs is known beforehand, this may be reasonable, but in most cases, it is an inefficient technique.

The use of fixed partitioning is almost unknown today. One example of a successful operating system that did use this technique was an early IBM mainframe operating system, OS/MFT (Multiprogramming with a Fixed Number of Tasks).

## Dynamic Partitioning

To overcome some of the difficulties with fixed partitioning, an approach known as dynamic partitioning was developed. Again, this approach has been supplanted by more sophisticated memory management techniques. An important operating system that used this technique was IBM's mainframe operating system, OS/MVT (Multiprogramming with a Variable Number of Tasks).

With dynamic partitioning, the partitions are of variable length and number. When a process is brought into main memory, it is allocated exactly as much memory as it requires and no more. An example, using 64 Mbytes of main memory, is shown in Figure 7.4. Initially, main memory is empty, except for the OS (see Figure 7.4a). The first three processes are loaded in, starting where the operating system ends and occupying just enough space for each process (see Figure 7.4b, c, d). This leaves a "hole" at the end of memory that is too small for a fourth process. At some point, none of the processes in memory is ready. The operating system swaps out process 2 (see Figure 7.4e), which leaves sufficient room to load a new process, process 4 (see Figure 7.4f). Because process 4 is smaller than process 2, another small hole is created.

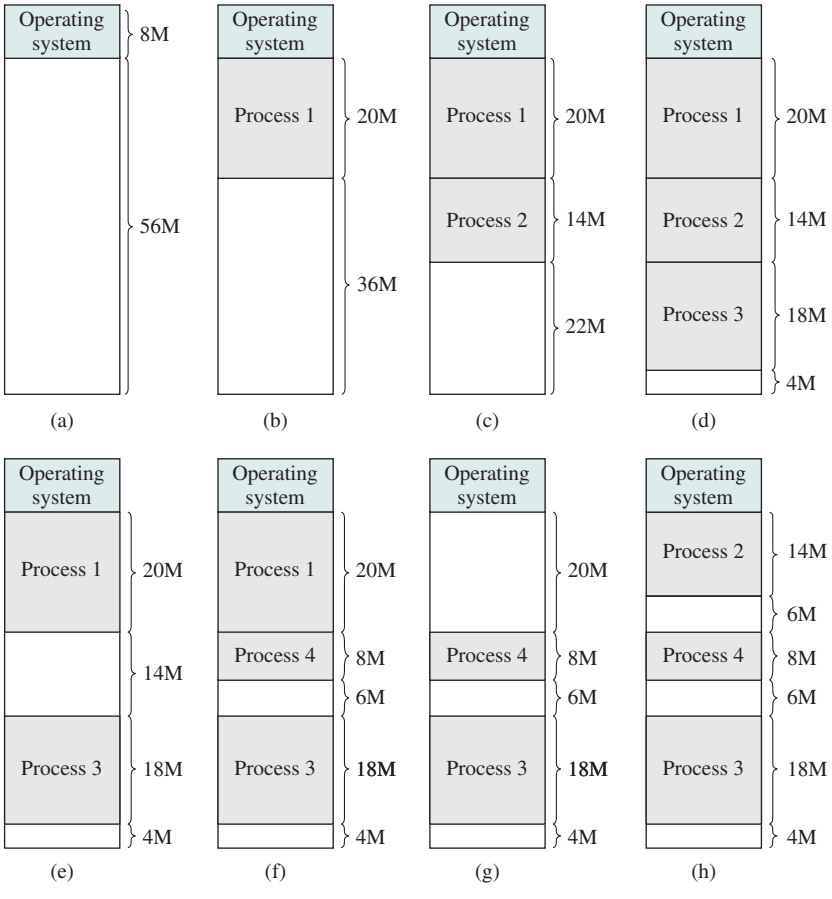



**Figure 7.4 The Effect of Dynamic Partitioning**

Later, a point is reached at which none of the processes in main memory is ready, but process 2, in the Ready-Suspend state, is available. Because there is insufficient room in memory for process 2, the operating system swaps process 1 out (see Figure 7.4g) and swaps process 2 back in (see Figure 7.4h).

As this example shows, this method starts out well, but eventually it leads to a situation in which there are a lot of small holes in memory. As time goes on, memory becomes more and more fragmented, and memory utilization declines. This phenomenon is referred to as **external fragmentation**, indicating the memory that is external to all partitions becomes increasingly fragmented. This is in contrast to internal fragmentation, referred to earlier.

One technique for overcoming external fragmentation is **compaction**: From time to time, the OS shifts the processes so they are contiguous and all of the free memory is together in one block. For example, in Figure 7.4h, compaction will result in a block of free memory of length 16M. This may well be sufficient to load in an additional process. The difficulty with compaction is that it is a time-consuming procedure and wasteful of processor time. Note that compaction implies the need for a dynamic relocation capability. That is, it must be possible to move a program from one region to another in main memory, without invalidating the memory references in the program (see Appendix 7A).

***Placement Algorithm*** Because memory compaction is time consuming, the OS designer must be clever in deciding how to assign processes to memory (how to plug the holes). When it is time to load or swap a process into main memory, and if there is more than one free block of memory of sufficient size, then the operating system must decide which free block to allocate.

Three placement algorithms that might be considered are best-fit, first-fit, and next-fit. All, of course, are limited to choosing among free blocks of main memory that are equal to or larger than the process to be brought in. **Best-fit** chooses the block that is closest in size to the request. **First-fit** begins to scan memory from the beginning and chooses the first available block that is large enough. **Next-fit** begins to scan memory from the location of the last placement and chooses the next available block that is large enough.

Figure 7.5a shows an example memory configuration after a number of placement and swapping-out operations. The last block that was used was a 22-Mbyte block from which a 14-Mbyte partition was created. Figure 7.5b shows the difference between the best-, first-, and next-fit placement algorithms in satisfying a 16-Mbyte allocation request. Best-fit will search the entire list of available blocks and make use of the 18-Mbyte block, leaving a 2-Mbyte fragment. First-fit results in a 6-Mbyte fragment, and next-fit results in a 20-Mbyte fragment.

Which of these approaches is best will depend on the exact sequence of process swappings that occurs and the size of those processes. However, some general comments can be made (see also [BREN89], [SHOR75], and [BAYS77]). The first-fit algorithm is not only the simplest but usually the best and fastest as well. The next-fit algorithm tends to produce slightly worse results than the first-fit. The next-fit algorithm will more frequently lead to an allocation from a free block at the end of memory. The result is that the largest block of free memory, which usually appears at the end of the memory space, is quickly broken up into small fragments. Thus,

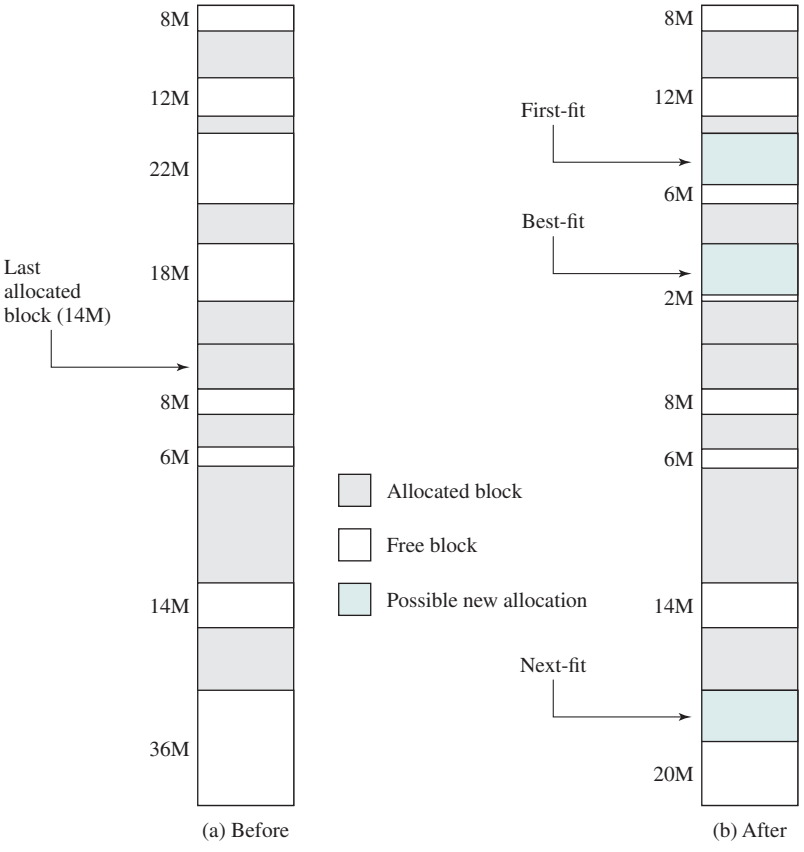



**Figure 7.5** **Example of Memory Configuration before and after Allocation of 16-Mbyte Block**

compaction may be required more frequently with next-fit. On the other hand, the first-fit algorithm may litter the front end with small free partitions that need to be searched over on each subsequent first-fit pass. The best-fit algorithm, despite its name, is usually the worst performer. Because this algorithm looks for the smallest block that will satisfy the requirement, it guarantees that the fragment left behind is as small as possible. Although each memory request always wastes the smallest amount of memory, the result is that main memory is quickly littered by blocks too small to satisfy memory allocation requests. Thus, memory compaction must be done more frequently than with the other algorithms.

***Replacement Algorithm*** In a multiprogramming system using dynamic partitioning, there will come a time when all of the processes in main memory are in a blocked state and there is insufficient memory, even after compaction, for an additional process. To avoid wasting processor time waiting for an active process to become unblocked, the OS will swap one of the processes out of main memory to make room for a new process or for a process in a Ready-Suspend state. Therefore,

the operating system must choose which process to replace. Because the topic of replacement algorithms will be covered in some detail with respect to various virtual memory schemes, we defer a discussion of replacement algorithms until then.

## Buddy System

Both fixed and dynamic partitioning schemes have drawbacks. A fixed partitioning scheme limits the number of active processes and may use space inefficiently if there is a poor match between available partition sizes and process sizes. A dynamic partitioning scheme is more complex to maintain and includes the overhead of compaction. An interesting compromise is the buddy system ([KNUT97], [PETE77]).

In a buddy system, memory blocks are available of size $2^K$ words, $L \leq K \leq U$, where

$2^L$ = smallest size block that is allocated

$2^U$ = largest size block that is allocated; generally $2^U$ is the size of the entire memory available for allocation

To begin, the entire space available for allocation is treated as a single block of size $2^U$. If a request of size $s$ such that $2^{U-1} < s \leq 2^U$ is made, then the entire block is allocated. Otherwise, the block is split into two equal buddies of size $2^{U-1}$. If $2^{U-2} < s \leq 2^{U-1}$, then the request is allocated to one of the two buddies. Otherwise, one of the buddies is split in half again. This process continues until the smallest block greater than or equal to $s$ is generated and allocated to the request. At any time, the buddy system maintains a list of holes (unallocated blocks) of each size $2^i$. A hole may be removed from the $(i + 1)$ list by splitting it in half to create two buddies of size $2^i$ in the $i$ list. Whenever a pair of buddies on the $i$ list both become unallocated, they are removed from that list and coalesced into a single block on the $(i + 1)$ list. Presented with a request for an allocation of size $k$ such that $2^{i-1} < k \leq 2^i$, the following recursive algorithm is used to find a hole of size $2^i$:

```
void  get_hole(int  i)
{
  if  (i == (U + 1)) <failure>;
  if  (<i_list empty>) {
       get_hole(i + 1);
       <split hole into buddies>;
       <put buddies on i_list>;
  }
  <take first hole on i_list>;
}
```

Figure 7.6 gives an example using a 1-Mbyte initial block. The first request, A, is for 100 Kbytes, for which a 128K block is needed. The initial block is divided into two 512K buddies. The first of these is divided into two 256K buddies, and the first of these is divided into two 128K buddies, one of which is allocated to A. The next request, B, requires a 256K block. Such a block is already available and is allocated. The process continues with splitting and coalescing occurring as needed. Note that

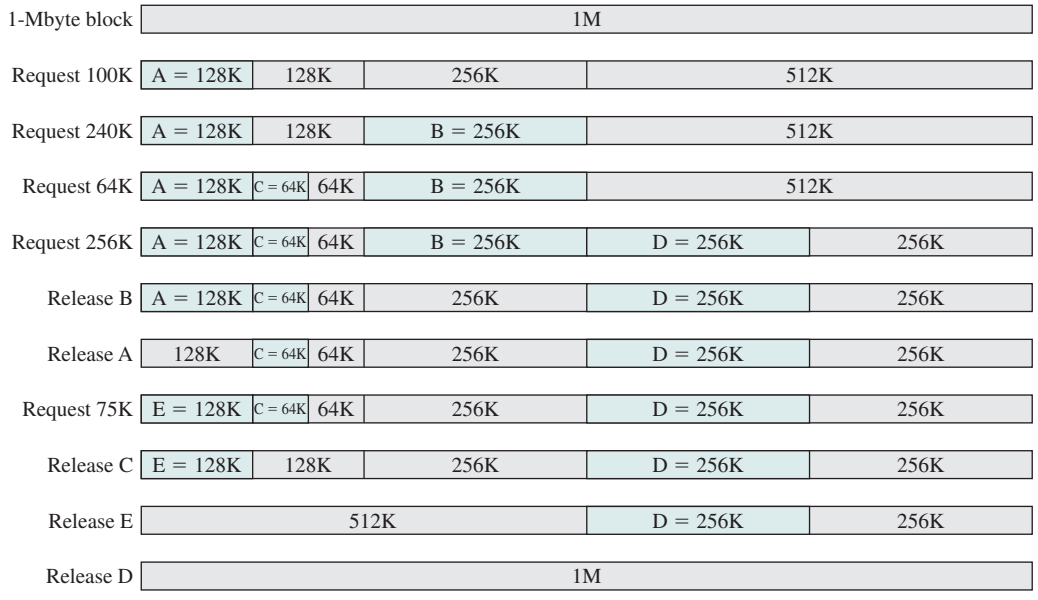



**Figure 7.6 Example of the Buddy System**

when E is released, two 128K buddies are coalesced into a 256K block, which is immediately coalesced with its buddy.

Figure 7.7 shows a binary tree representation of the buddy allocation immediately after the Release B request. The leaf nodes represent the current partitioning of the memory. If two buddies are leaf nodes, then at least one must be allocated; otherwise, they would be coalesced into a larger block.

The buddy system is a reasonable compromise to overcome the disadvantages of both the fixed and variable partitioning schemes, but in contemporary operating systems, virtual memory based on paging and segmentation is superior. However, the buddy system has found application in parallel systems as an efficient means of allocation and release for parallel programs (e.g., see [JOHN92]). A modified form of the buddy system is used for UNIX kernel memory allocation (described in Chapter 8).

## Relocation

Before we consider ways of dealing with the shortcomings of partitioning, we must clear up one loose end, which relates to the placement of processes in memory. When the fixed partition scheme of Figure 7.3a is used, we can expect a process will always be assigned to the same partition. That is, whichever partition is selected when a new process is loaded will always be used to swap that process back into memory after it has been swapped out. In that case, a simple relocating loader, such as is described in Appendix 7A, can be used: When the process is first loaded, all relative memory references in the code are replaced by absolute main memory addresses, determined by the base address of the loaded process.

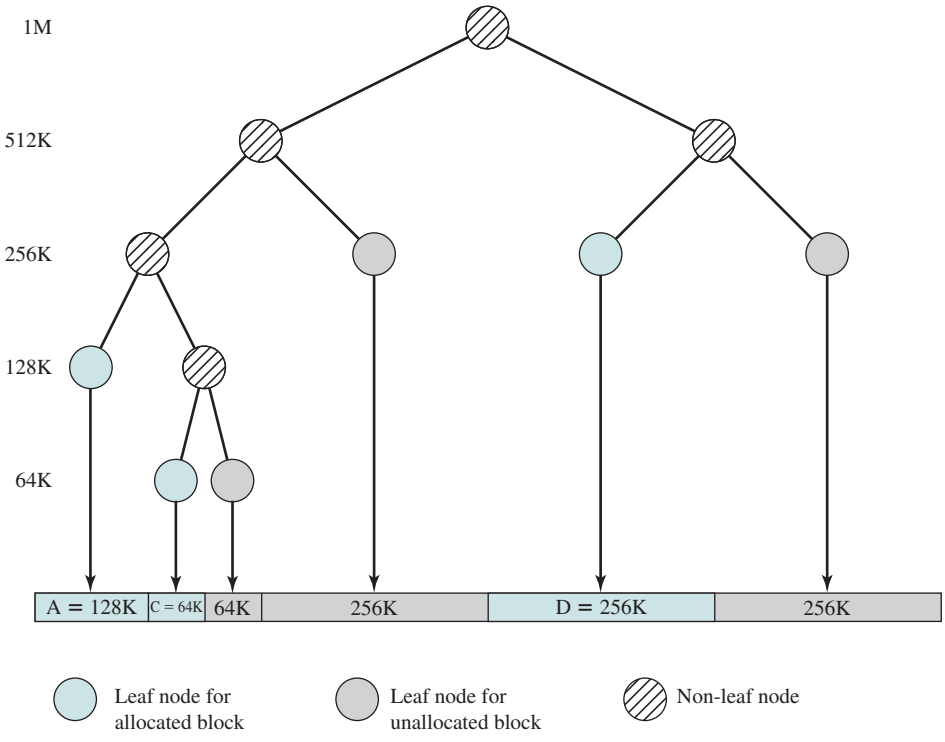



**Figure 7.7** **Tree Representation of the Buddy System**

In the case of equal-size partitions (see Figure 7.2a) and in the case of a single process queue for unequal-size partitions (see Figure 7.3b), a process may occupy different partitions during the course of its life. When a process image is first created, it is loaded into some partition in main memory. Later, the process may be swapped out; when it is subsequently swapped back in, it may be assigned to a different partition than the last time. The same is true for dynamic partitioning. Observe in Figure 7.4c and Figure 7.4h that process 2 occupies two different regions of memory on the two occasions when it is brought in. Furthermore, when compaction is used, processes are shifted while they are in main memory. Thus, the locations (of instructions and data) referenced by a process are not fixed. They will change each time a process is swapped in or shifted. To solve this problem, a distinction is made among several types of addresses. A **logical address** is a reference to a memory location independent of the current assignment of data to memory; a translation must be made to a physical address before the memory access can be achieved. A **relative address** is a particular example of logical address, in which the address is expressed as a location relative to some known point, usually a value in a processor register. A **physical address**, or absolute address, is an actual location in main memory.

Programs that employ relative addresses in memory are loaded using dynamic run-time loading (see Appendix 7A for a discussion). Typically, all of the memory references in the loaded process are relative to the origin of the program. Thus, a

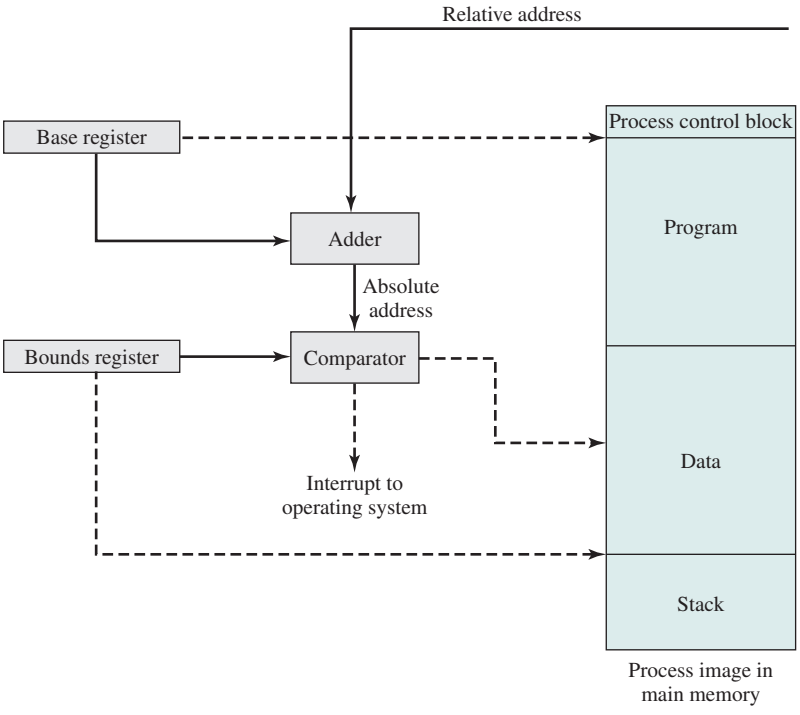



**Figure 7.8 Hardware Support for Relocation**

hardware mechanism is needed for translating relative addresses to physical main memory addresses at the time of execution of the instruction that contains the reference.

Figure 7.8 shows the way in which this address translation is typically accomplished. When a process is assigned to the Running state, a special processor register, sometimes called the base register, is loaded with the starting address in main memory of the program. There is also a "bounds" register that indicates the ending location of the program; these values must be set when the program is loaded into memory or when the process image is swapped in. During the course of execution of the process, relative addresses are encountered. These include the contents of the instruction register, instruction addresses that occur in branch and call instructions, and data addresses that occur in load and store instructions. Each such relative address goes through two steps of manipulation by the processor. First, the value in the base register is added to the relative address to produce an absolute address. Second, the resulting address is compared to the value in the bounds register. If the address is within bounds, then the instruction execution may proceed. Otherwise, an interrupt is generated to the operating system, which must respond to the error in some fashion.

The scheme of Figure 7.8 allows programs to be swapped in and out of memory during the course of execution. It also provides a measure of protection: Each process image is isolated by the contents of the base and bounds registers, and is safe from unwanted accesses by other processes.

## 7.3 PAGING

Both unequal fixed-size and variable-size partitions are inefficient in the use of memory; the former results in internal fragmentation, the latter in external fragmentation. Suppose, however, main memory is partitioned into equal fixed-size chunks that are relatively small, and each process is also divided into small fixed-size chunks of the same size. Then the chunks of a process, known as **pages**, could be assigned to available chunks of memory, known as **frames**, or page frames. We show in this section that the wasted space in memory for each process is due to internal fragmentation consisting of only a fraction of the last page of a process. There is no external fragmentation.

Figure 7.9 illustrates the use of pages and frames. At a given point in time, some of the frames in memory are in use, and some are free. A list of free frames is maintained by the OS. Process A, stored on disk, consists of four pages. When it is time to load this process, the OS finds four free frames and loads the four pages of process A into the four frames (see Figure 7.9b). Process B, consisting of three pages, and process C, consisting of four pages, are subsequently loaded. Then process B is suspended and is swapped out of main memory. Later, all of the processes in main memory are blocked, and the OS needs to bring in a new process, process D, which consists of five pages.

Now suppose, as in this example, there are not sufficient unused contiguous frames to hold the process. Does this prevent the operating system from loading D? The answer is no, because we can once again use the concept of logical address. A simple base address register will no longer suffice. Rather, the operating system maintains a **page table** for each process. The page table shows the frame location for each page of the process. Within the program, each logical address consists of a page number and an offset within the page. Recall that in the case of simple partition, a logical address is the location of a word relative to the beginning of the program; the processor translates that into a physical address. With paging, the logical-to-physical address translation is still done by processor hardware. Now the processor must know how to access the page table of the current process. Presented with a logical address (page number, offset), the processor uses the page table to produce a physical address (frame number, offset).

Continuing our example, the five pages of process D are loaded into frames 4, 5, 6, 11, and 12. Figure 7.10 shows the various page tables at this time. A page table contains one entry for each page of the process, so the table is easily indexed by the page number (starting at page 0). Each page table entry contains the number of the frame in main memory, if any, that holds the corresponding page. In addition, the OS maintains a single free-frame list of all the frames in main memory that are currently unoccupied and available for pages.

Thus, we see that simple paging, as described here, is similar to fixed partitioning. The differences are that, with paging, the partitions are rather small; a program may occupy more than one partition; and these partitions need not be contiguous.

To make this paging scheme convenient, let us dictate that the page size, hence the frame size, must be a power of 2. With the use of a page size that is a power of 2, it is easy to demonstrate that the relative address (which is defined with reference to the origin of the program) and the logical address (expressed as a page number

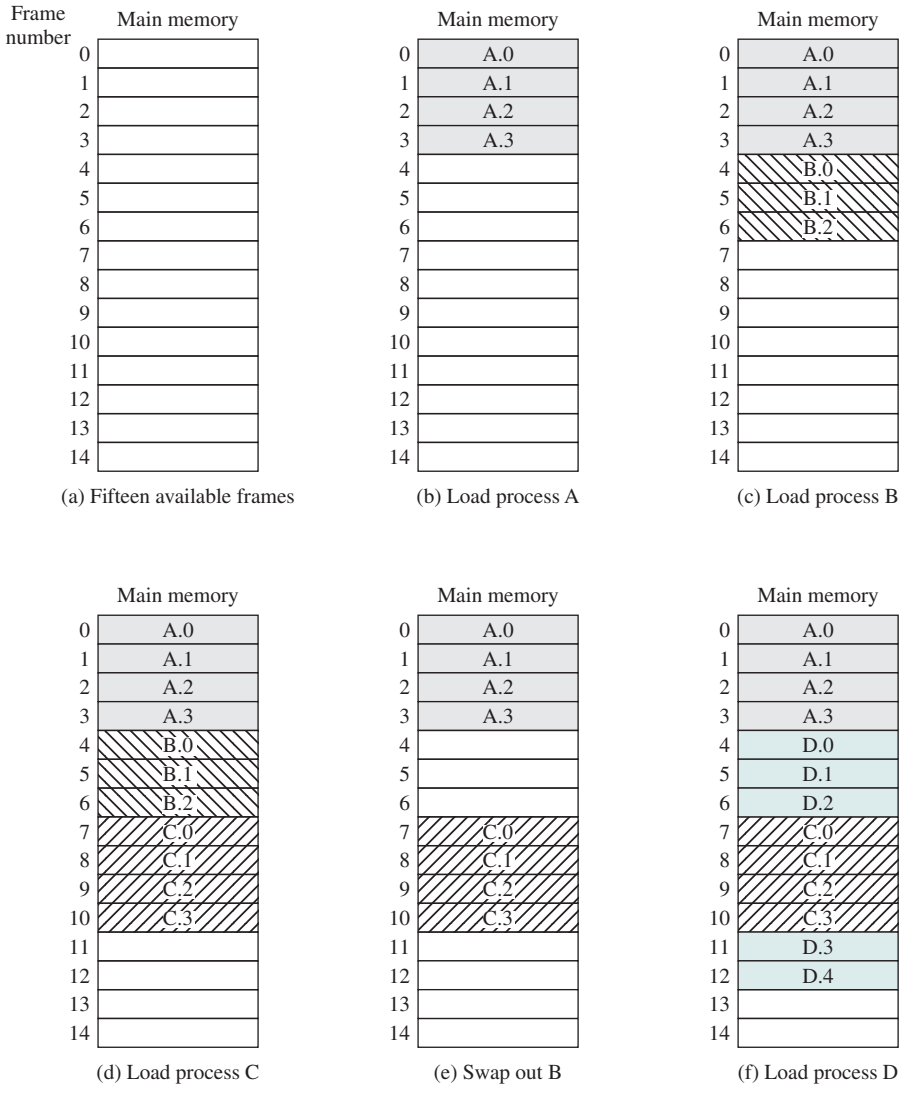



**Figure 7.9 Assignment of Process to Free Frames**

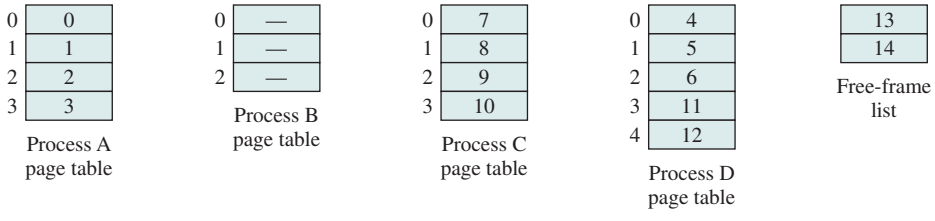



**Figure 7.10 Data Structures for the Example of Figure 7.9 at Time Epoch (f)**

and offset) are the same. An example is shown in Figure 7.11. In this example, 16-bit addresses are used, and the page size is 1K = 1024 bytes. The relative address 1502 in binary form is 0000010111011110. With a page size of 1K, an offset field of 10 bits is needed, leaving 6 bits for the page number. Thus, a program can consist of a maximum of $2^6 = 64$ pages of size 1 Kbytes each. As Figure 7.11b shows, relative address 1502 corresponds to an offset of 478 (0111011110) on page 1 (000001), which yields the same 16-bit number, 0000010111011110.

The consequences of using a page size that is a power of 2 are twofold. First, the logical addressing scheme is transparent to the programmer, the assembler, and the linker. Each logical address (page number, offset) of a program is identical to its relative address. Second, it is a relatively easy matter to implement a function in hardware to perform dynamic address translation at run time. Consider an address of $n + m$ bits, where the leftmost $n$ bits are the page number, and the rightmost $m$ bits are the offset. In our example (see Figure 7.11b), $n = 6$ and $m = 10$. The following steps are needed for address translation:

1. Extract the page number as the leftmost $n$ bits of the logical address.
2. Use the page number as an index into the process page table to find the frame number, $k$.
3. The starting physical address of the frame is $k \times 2_m$, and the physical address of the referenced byte is that number plus the offset. This physical address need not be calculated; it is easily constructed by appending the frame number to the offset.

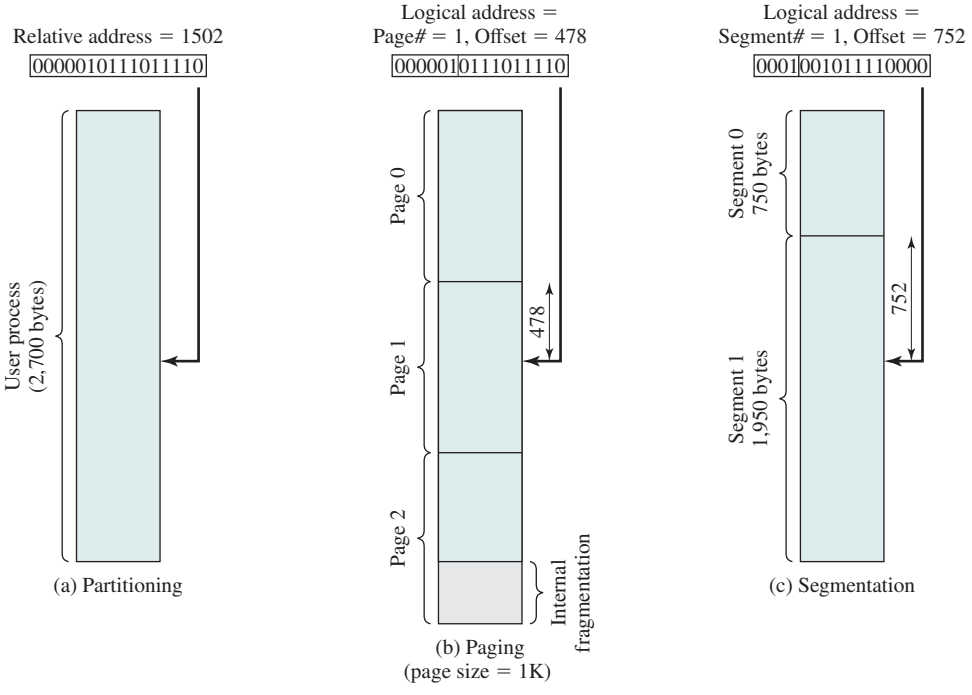



**Figure 7.11 Logical Addresses**

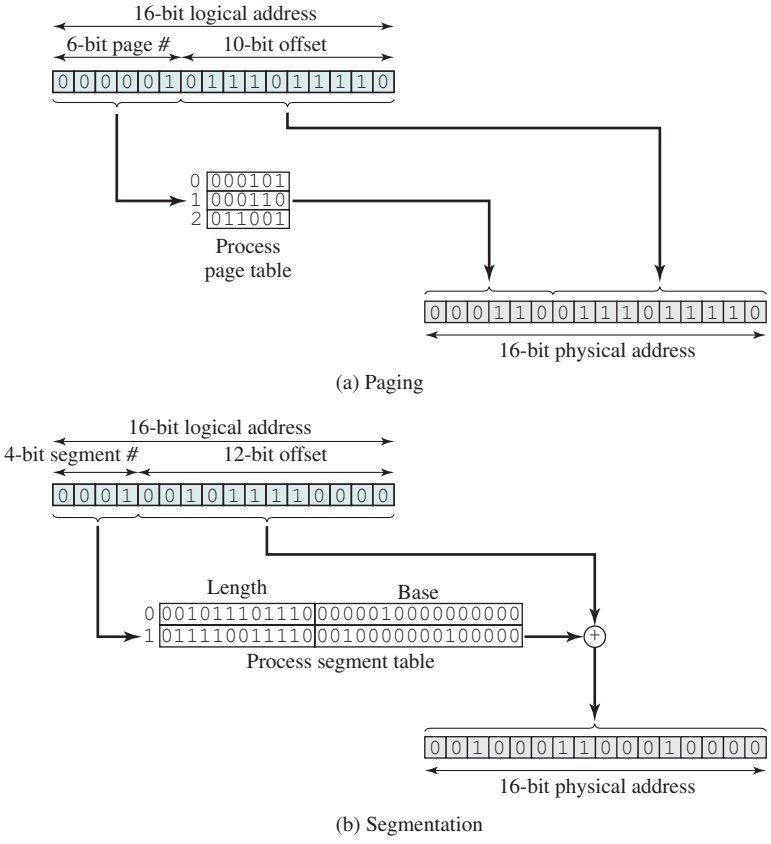



**Figure 7.12 Examples of Logical-to-Physical Address Translation**

In our example, we have the logical address 0000010111011110, which is page number 1, offset 478. Suppose this page is residing in main memory frame 6 = binary 000110. Then the physical address is frame number 6, offset 478 = 0001100111011110 (see Figure 7.12a).

To summarize, with simple paging, main memory is divided into many small equal-size frames. Each process is divided into frame-size pages. Smaller processes require fewer pages; larger processes require more. When a process is brought in, all of its pages are loaded into available frames, and a page table is set up. This approach solves many of the problems inherent in partitioning.

## 7.4 SEGMENTATION

A user program can be subdivided using segmentation, in which the program and its associated data are divided into a number of **segments**. It is not required that all segments of all programs be of the same length, although there is a maximum segment

length. As with paging, a logical address using segmentation consists of two parts, in this case, a segment number and an offset.

Because of the use of unequal-size segments, segmentation is similar to dynamic partitioning. In the absence of an overlay scheme or the use of virtual memory, it would be required that all of a program's segments be loaded into memory for execution. The difference, compared to dynamic partitioning, is that with segmentation a program may occupy more than one partition, and these partitions need not be contiguous. Segmentation eliminates internal fragmentation but, like dynamic partitioning, it suffers from external fragmentation. However, because a process is broken up into a number of smaller pieces, the external fragmentation should be less.

Whereas paging is invisible to the programmer, segmentation is usually visible and is provided as a convenience for organizing programs and data. Typically, the programmer or compiler will assign programs and data to different segments. For purposes of modular programming, the program or data may be further broken down into multiple segments. The principal inconvenience of this service is that the programmer must be aware of the maximum segment size limitation.

Another consequence of unequal-size segments is that there is no simple relationship between logical addresses and physical addresses. Analogous to paging, a simple segmentation scheme would make use of a segment table for each process, and a list of free blocks of main memory. Each segment table entry would have to give the starting address in main memory of the corresponding segment. The entry should also provide the length of the segment to assure that invalid addresses are not used. When a process enters the Running state, the address of its segment table is loaded into a special register used by the memory management hardware. Consider an address of $n + m$ bits, where the leftmost $n$ bits are the segment number and the rightmost $m$ bits are the offset. In our example (see Figure 7.11c), $n = 4$ and $m = 12$. Thus, the maximum segment size is $2^{12} = 4096$. The following steps are needed for address translation:

1. Extract the segment number as the leftmost $n$ bits of the logical address.
2. Use the segment number as an index into the process segment table to find the starting physical address of the segment.
3. Compare the offset, expressed in the rightmost $m$ bits, to the length of the segment. If the offset is greater than or equal to the length, the address is invalid.
4. The desired physical address is the sum of the starting physical address of the segment plus the offset.

In our example, we have the logical address 0001001011110000, which is segment number 1, offset 752. Suppose this segment is residing in main memory starting at physical address 0010000000100000. Then the physical address is 0010000000100000 + 001011110000 = 0010001100010000 (see Figure 7.12b).

To summarize, with simple segmentation, a process is divided into a number of segments that need not be of equal size. When a process is brought in, all of its segments are loaded into available regions of memory, and a segment table is set up.

## 7.5 SUMMARY

One of the most important and complex tasks of an operating system is memory management. Memory management involves treating main memory as a resource to be allocated to and shared among a number of active processes. To use the processor and the I/O facilities efficiently, it is desirable to maintain as many processes in main memory as possible. In addition, it is desirable to free programmers from size restrictions in program development.

The basic tools of memory management are paging and segmentation. With paging, each process is divided into relatively small, fixed-size pages. Segmentation provides for the use of pieces of varying size. It is also possible to combine segmentation and paging in a single memory management scheme.

## 7.6 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

### Key Terms

| | | |
|---|---|---|
| absolute loading | linkage editor | physical address |
| buddy system | linking | physical organization |
| compaction | loading | protection |
| dynamic linking | logical address | relative address |
| dynamic partitioning | logical organization | relocatable loading |
| dynamic run-time loading | memory management | relocation |
| external fragmentation | page | segment |
| fixed partitioning | page table | segmentation |
| frame | paging | sharing |
| internal fragmentation | partitioning | |

### Review Questions

**7.1.** What requirements is memory management intended to satisfy?
**7.2.** What is relocation of a program?
**7.3.** What are the advantages of organizing programs and data into modules?
**7.4.** What are some reasons to allow two or more processes to all have access to a particular region of memory?
**7.5.** In a fixed partitioning scheme, what are the advantages of using unequal-size partitions?
**7.6.** What is the difference between internal and external fragmentation?
**7.7.** What is address binding? State the different timings when address binding may occur.
**7.8.** What is the difference between a page and a frame?
**7.9.** What is the difference between a page and a segment?

## Problems

**7.1.** In Section 2.3, we listed five objectives of memory management, and in Section 7.1, we listed five requirements. Argue that each list encompasses all of the concerns addressed in the other.

**7.2.** Consider a fixed partitioning scheme with equal-size partitions of $2^{16}$ bytes and a total main memory size of $2^{24}$ bytes. A process table is maintained that includes a pointer to a partition for each resident process. How many bits are required for the pointer?

**7.3.** Consider a dynamic partitioning scheme. Show that, on average, the memory contains half as many holes as segments.

**7.4.** To implement the various placement algorithms discussed for dynamic partitioning (see Section 7.2), a list of the free blocks of memory must be kept. For each of the three methods discussed (best-fit, first-fit, next-fit), what is the average length of the search?

**7.5.** Another placement algorithm for dynamic partitioning is referred to as worst-fit. In this case, the largest free block of memory is used for bringing in a process.

**a.** Discuss the pros and cons of this method compared to first-, next-, and best-fit.

**b.** What is the average length of the search for worst-fit?

**7.6.** This diagram shows an example of memory configuration under dynamic partitioning, after a number of placement and swapping-out operations have been carried out. Addresses go from left to right; gray areas indicate blocks occupied by processes; white areas indicate free memory blocks. The last process placed is 2 Mbytes and is marked with an X. Only one process was swapped out after that.




**a.** What was the maximum size of the swapped-out process?

**b.** What was the size of the free block just before it was partitioned by X?

**c.** A new 3-Mbyte allocation request must be satisfied next. Indicate the intervals of memory where a partition will be created for the new process under the following four placement algorithms: best-fit, first-fit, next-fit, and worst-fit. For each algorithm, draw a horizontal segment under the memory strip and label it clearly.

**7.7.** A 512 KB block of memory is allocated using the buddy system. Show the results of the following sequence of requests and returns in a figure that is similar to Figure 7.6: Request A: 100; Request B: 40; Request C: 190; Return A; Request D: 60; Return B; Return D; Return C. Also, find the internal fragmentation at each stage of allocation/de-allocation.

**7.8.** Consider a memory-management system that uses simple paging strategy and employs registers to speed up page lookups. The associative registers have a lookup performance of 120 ns and a hit ratio of 80%. The main-memory page-table lookup takes 600 ns. Compute the average page-lookup time.

**7.9.** Let $\text{buddy}_k(x)$ = address of the buddy of the block of size $2^k$ whose address is $x$. Write a general expression for $\text{buddy}_k(x)$.

**7.10.** The Fibonacci sequence is defined as follows:

$$F_0 = 0, F_1 = 1, F_{n+2} = F_{n+1} + F_n, n \geq 0$$

**a.** Could this sequence be used to establish a buddy system?

**b.** What would be the advantage of this system over the binary buddy system described in this chapter?

**7.11.** During the course of execution of a program, the processor will increment the contents of the instruction register (program counter) by one word after each instruction fetch, but will alter the contents of that register if it encounters a branch or call instruction that causes execution to continue elsewhere in the program. Now consider Figure 7.8. There are two alternatives with respect to instruction addresses:

1. Maintain a relative address in the instruction register and do the dynamic address translation using the instruction register as input. When a successful branch or call is encountered, the relative address generated by that branch or call is loaded into the instruction register.
2. Maintain an absolute address in the instruction register. When a successful branch or call is encountered, dynamic address translation is employed, with the results stored in the instruction register.

Which approach is preferable?

**7.12.** Consider a memory-management system based on paging. The total size of the physical memory is 2 GB, laid out over pages of size 8 KB. The logical address space of each process has been limited to 256 MB.

**a.** Determine the total number of bits in the physical address.

**b.** Determine the number of bits specifying page replacement and the number of bits for page frame number.

**c.** Determine the number of page frames.

**d.** Determine the logical address layout.

**7.13.** Write the binary translation of the logical address 0011000000110011 under the following hypothetical memory management schemes, and explain your answer:

**a.** A paging system with a 512-address page size, using a page table in which the frame number happens to be half of the page number.

**b.** A segmentation system with a 2K-address maximum segment size, using a segment table in which bases happen to be regularly placed at real addresses: segment # + 20 + offset + 4,096.

**7.14.** Consider a simple segmentation system that has the following segment table:

| Starting Address | Length (bytes) |
|---|---|
| 830 | 346 |
| 648 | 110 |
| 1,508 | 408 |
| 770 | 812 |

For each of the following logical addresses, determine the physical address or indicate if a segment fault occurs:

**a.** 0, 228

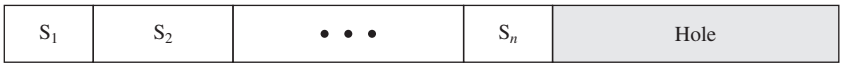
**b.** 2, 648

**c.** 3, 776

**d.** 1, 98

**e.** 1, 240

**7.15.** Consider a memory in which contiguous segments $S_1, S_2, \ldots, S_n$ are placed in their order of creation from one end of the store to the other, as suggested by the following figure:



When segment $S_{n+1}$ is being created, it is placed immediately after segment $S_n$ even though some of the segments $S_1, S_2, \ldots, S_n$ may already have been deleted. When the boundary between segments (in use or deleted) and the hole reaches the other end of the memory, the segments in use are compacted.

**a.** Show that the fraction of time $F$ spent on compacting obeys the following inequality:

$$F \geq \frac{1 - f}{1 + kf} \text{ where } k = \frac{t}{2s} - 1$$

where

$s$ = average length of a segment, in words
$t$ = average lifetime of a segment, in memory references
$f$ = fraction of the memory that is unused under equilibrium conditions

*Hint:* Find the average speed at which the boundary crosses the memory and assume that the copying of a single word requires at least two memory references.

**b.** Find $F$ for $f = 0.25$, $t = 1{,}200$, and $s = 75$.

## APPENDIX 7A LOADING AND LINKING

The first step in the creation of an active process is to load a program into main memory and create a process image (see Figure 7.13). Figure 7.14 depicts a scenario typical for most systems. The application consists of a number of compiled or assembled modules in object-code form. These are linked to resolve any references between modules. At the same time, references to library routines are resolved. The library routines themselves may be incorporated into the program or referenced as shared

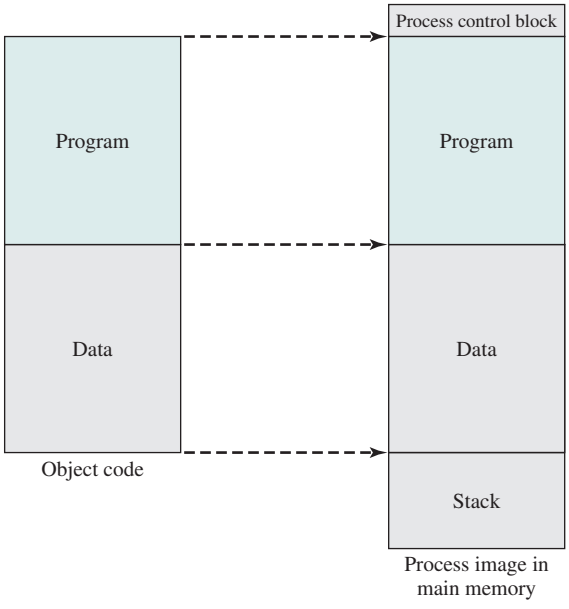



**Figure 7.13 The Loading Function**

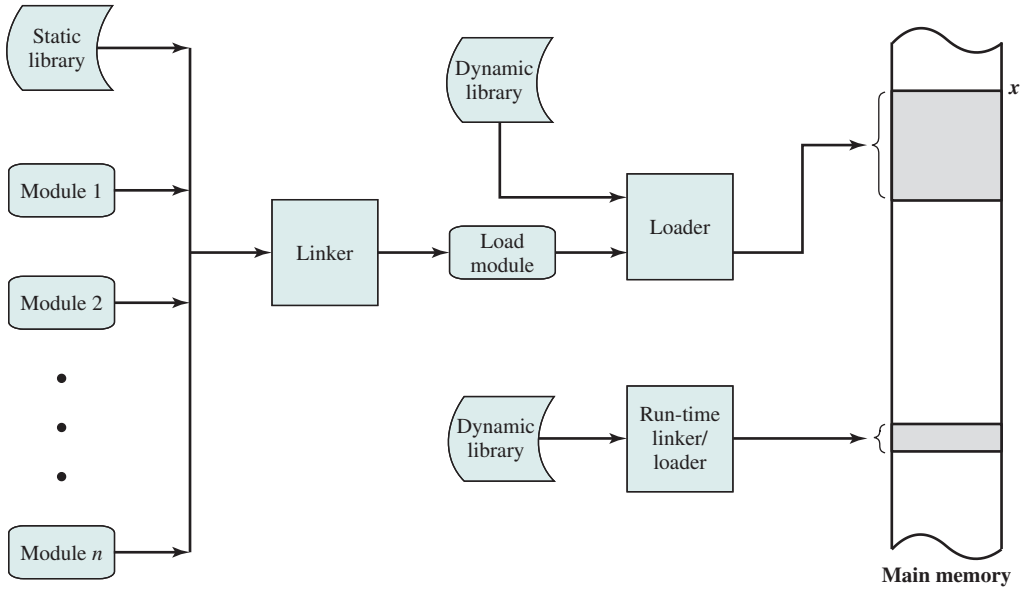



**Figure 7.14** **A Linking and Loading Scenario**

code that must be supplied by the operating system at run time. In this appendix, we summarize the key features of linkers and loaders. For clarity in the presentation, we begin with a description of the loading task when a single program module is involved; no linking is required.

## Loading

In Figure 7.14, the loader places the load module in main memory starting at location $x$. In loading the program, the addressing requirement illustrated in Figure 7.1 must be satisfied. In general, three approaches can be taken:

1. Absolute loading
2. Relocatable loading
3. Dynamic run-time loading

***Absolute Loading*** An absolute loader requires that a given load module always be loaded into the same location in main memory. Thus, in the load module presented to the loader, all address references must be to specific, or absolute, main memory addresses. For example, if $x$ in Figure 7.14 is location 1024, then the first word in a load module destined for that region of memory has address 1024.

The assignment of specific address values to memory references within a program can be done either by the programmer or at compile or assembly time (see Table 7.3a). There are several disadvantages to the former approach. First, every programmer would have to know the intended assignment strategy for placing modules into main memory. Second, if any modifications are made to the program that involve insertions or deletions in the body of the module, then all of the addresses will have

**Table 7.3** Address Binding

**(a) Loader**

| Binding Time | Function |
|---|---|
| Programming time | All actual physical addresses are directly specified by the programmer in the program itself. |
| Compile or assembly time | The program contains symbolic address references, and these are converted to actual physical addresses by the compiler or assembler. |
| Load time | The compiler or assembler produces relative addresses. The loader translates these to absolute addresses at the time of program loading. |
| Run time | The loaded program retains relative addresses. These are converted dynamically to absolute addresses by processor hardware. |

**(b) Linker**

| Linkage Time | Function |
|---|---|
| Programming time | No external program or data references are allowed. The programmer must place into the program the source code for all subprograms that are referenced. |
| Compile or assembly time | The assembler must fetch the source code of every subroutine that is referenced and assemble them as a unit. |
| Load module creation | All object modules have been assembled using relative addresses. These modules are linked together and all references are restated relative to the origin of the final load module. |
| Load time | External references are not resolved until the load module is to be loaded into main memory. At that time, referenced dynamic link modules are appended to the load module, and the entire package is loaded into main or virtual memory. |
| Run time | External references are not resolved until the external call is executed by the processor. At that time, the process is interrupted and the desired module is linked to the calling program. |

to be altered. Accordingly, it is preferable to allow memory references within programs to be expressed symbolically, then resolve those symbolic references at the time of compilation or assembly. This is illustrated in Figure 7.15. Every reference to an instruction or item of data is initially represented by a symbol. In preparing the module for input to an absolute loader, the assembler or compiler will convert all of these references to specific addresses (in this example, for a module to be loaded starting at location 1024), as shown in Figure 7.15b.

***Relocatable Loading*** The disadvantage of binding memory references to specific addresses prior to loading is that the resulting load module can only be placed in one region of main memory. However, when many programs share main memory, it may not be desirable to decide ahead of time into which region of memory a particular module should be loaded. It is better to make that decision at load time. Thus, we need a load module that can be located anywhere in main memory.

To satisfy this new requirement, the assembler or compiler produces not actual main memory addresses (absolute addresses) but addresses that are relative to some known point, such as the start of the program. This technique is illustrated in

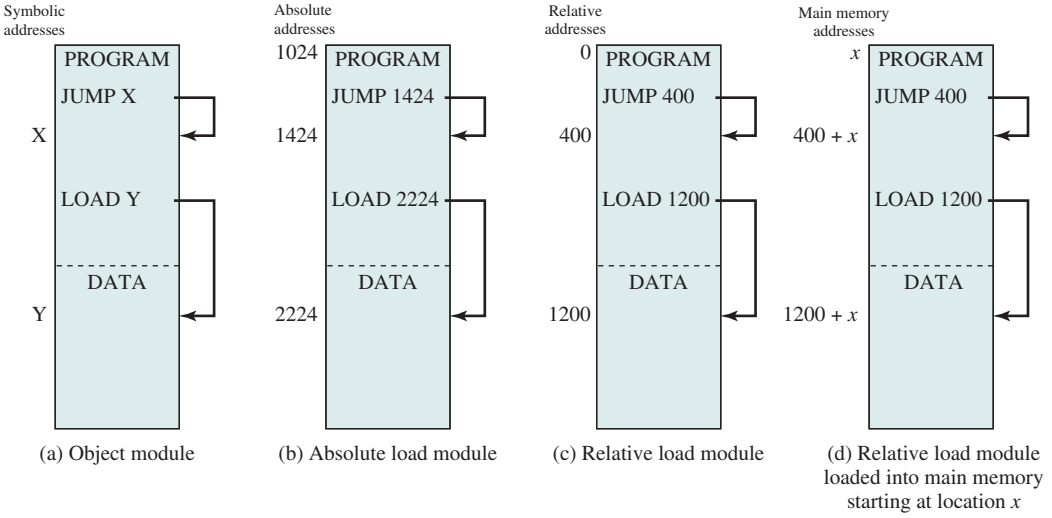



**Figure 7.15 Absolute and Relocatable Load Modules**

Figure 7.15c. The start of the load module is assigned the relative address 0, and all other memory references within the module are expressed relative to the beginning of the module.

With all memory references expressed in relative format, it becomes a simple task for the loader to place the module in the desired location. If the module is to be loaded beginning at location $x$, then the loader must simply add $x$ to each memory reference as it loads the module into memory. To assist in this task, the load module must include information that tells the loader where the address references are and how they are to be interpreted (usually relative to the program origin, but also possibly relative to some other point in the program, such as the current location). This set of information is prepared by the compiler or assembler, and is usually referred to as the relocation dictionary.

***Dynamic Run-Time Loading*** Relocatable loaders are common and provide obvious benefits relative to absolute loaders. However, in a multiprogramming environment, even one that does not depend on virtual memory, the relocatable loading scheme is inadequate. We have referred to the need to swap process images in and out of main memory to maximize the utilization of the processor. To maximize main memory utilization, we would like to be able to swap the process image back into different locations at different times. Thus, a program, once loaded, may be swapped out to disk then swapped back in at a different location. This would be impossible if memory references had been bound to absolute addresses at the initial load time.

The alternative is to defer the calculation of an absolute address until it is actually needed at run time. For this purpose, the load module is loaded into main memory with all memory references in relative form (see Figure 7.15d). It is not until an instruction is actually executed that the absolute address is calculated. To assure

that this function does not degrade performance, it must be done by special processor hardware rather than software. This hardware is described in Section 7.2.

Dynamic address calculation provides complete flexibility. A program can be loaded into any region of main memory. Subsequently, the execution of the program can be interrupted and the program can be swapped out of main memory, to be later swapped back in at a different location.

## Linking

The function of a linker is to take as input a collection of object modules and produce a load module, consisting of an integrated set of program and data modules, to be passed to the loader. In each object module, there may be address references to locations in other modules. Each such reference can only be expressed symbolically in an unlinked object module. The linker creates a single load module that is the contiguous joining of all of the object modules. Each intramodule reference must be changed from a symbolic address to a reference to a location within the overall load module. For example, module A in Figure 7.16a contains a procedure invocation of module B. When these modules are combined in the load module, this symbolic reference to module B is changed to a specific reference to the location of the entry point of B within the load module.

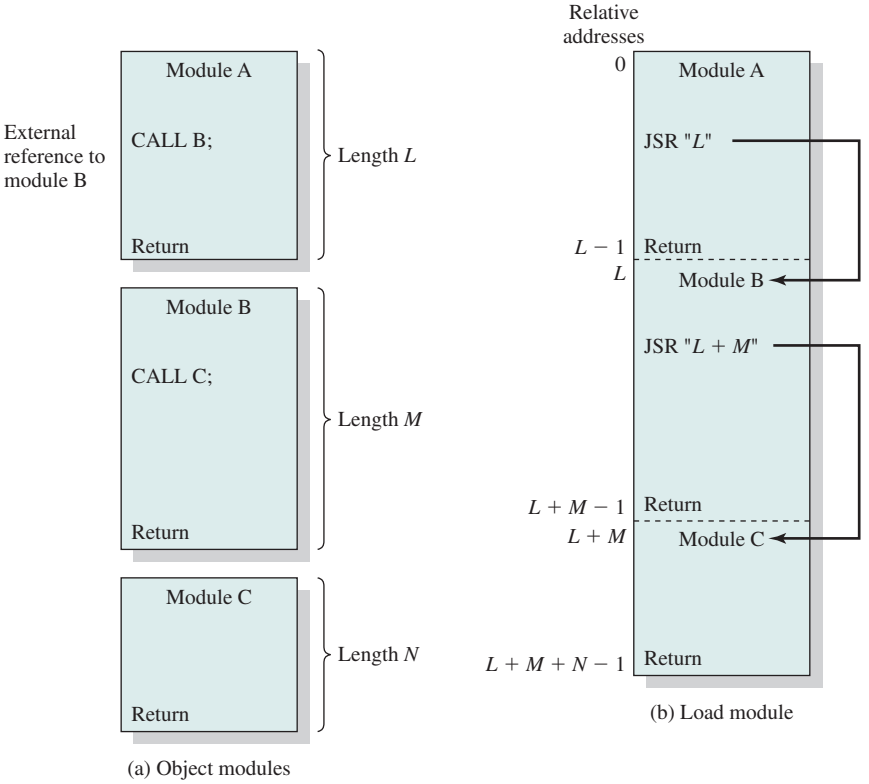



**Figure 7.16 The Linking Function**

***Linkage Editor*** The nature of this address linkage will depend on the type of load module to be created and when the linkage occurs (see Table 7.3b). If, as is usually the case, a relocatable load module is desired, then linkage is usually done in the following fashion. Each compiled or assembled object module is created with references relative to the beginning of the object module. All of these modules are put together into a single relocatable load module with all references relative to the origin of the load module. This module can be used as input for relocatable loading or dynamic run-time loading.

A linker that produces a relocatable load module is often referred to as a linkage editor. Figure 7.16 illustrates the linkage editor function.

***Dynamic Linker*** As with loading, it is possible to defer some linkage functions. The term *dynamic linking* is used to refer to the practice of deferring the linkage of some external modules until after the load module has been created. Thus, the load module contains unresolved references to other programs. These references can be resolved either at load time or run time.

For *load-time dynamic linking* (involving the upper dynamic library in Figure 7.14), the following steps occur. The load module (application module) to be loaded is read into memory. Any reference to an external module (target module) causes the loader to find the target module, load it, and alter the reference to a relative address in memory from the beginning of the application module. There are several advantages to this approach over what might be called static linking:

- It becomes easier to incorporate changed or upgraded versions of the target module, which may be an operating system utility or some other general-purpose routine. With static linking, a change to such a supporting module would require the relinking of the entire application module. Not only is this inefficient, but it may be impossible in some circumstances. For example, in the personal computer field, most commercial software is released in load module form; source and object versions are not released.
- Having target code in a dynamic link file paves the way for automatic code sharing. The operating system can recognize that more than one application is using the same target code, because it loaded and linked that code. It can use that information to load a single copy of the target code and link it to both applications, rather than having to load one copy for each application.
- It becomes easier for independent software developers to extend the functionality of a widely used operating system such as Linux. A developer can come up with a new function that may be useful to a variety of applications, and package it as a dynamic link module.

With **run-time dynamic linking** (involving the lower dynamic library in Figure 7.14), some of the linking is postponed until execution time. External references to target modules remain in the loaded program. When a call is made to the absent module, the operating system locates the module, loads it, and links it to the calling module. Such modules are typically shareable. In the Windows environment, these are called dynamic link libraries (DLLs). Thus, if one process is already making use

of a dynamically linked shared module, then that module is in main memory and a new process can simply link to the already-loaded module.

The use of DLLs can lead to a problem commonly referred to as **DLL hell**. DLL hell occurs if two or more processes are sharing a DLL module but expect different versions of the module. For example, an application or system function might be reinstalled and bring in with it an older version of a DLL file.

We have seen that dynamic loading allows an entire load module to be moved around; however, the structure of the module is static, being unchanged throughout the execution of the process and from one execution to the next. However, in some cases, it is not possible to determine prior to execution which object modules will be required. This situation is typified by transaction-processing applications, such as an airline reservation system or a banking application. The nature of the transaction dictates which program modules are required, and they are loaded as appropriate and linked with the main program. The advantage of the use of such a dynamic linker is that it is not necessary to allocate memory for program units unless those units are referenced. This capability is used in support of segmentation systems.

One additional refinement is possible: An application need not know the names of all the modules or entry points that may be called. For example, a charting program may be written to work with a variety of plotters, each of which is driven by a different driver package. The application can learn the name of the plotter that is currently installed on the system from another process or by looking it up in a configuration file. This allows the user of the application to install a new plotter that did not exist at the time the application was written.

CHAPTER 8

# VIRTUAL MEMORY


**8.1 Hardware and Control Structures**
- Locality and Virtual Memory
- Paging
- Segmentation
- Combined Paging and Segmentation
- Protection and Sharing

**8.2 Operating System Software**
- Fetch Policy
- Placement Policy
- Replacement Policy
- Resident Set Management
- Cleaning Policy
- Load Control

**8.3 UNIX and Solaris Memory Management**
- Paging System
- Kernel Memory Allocator

**8.4 Linux Memory Management**
- Linux Virtual Memory
- Kernel Memory Allocation

**8.5 Windows Memory Management**
- Windows Virtual Address Map
- Windows Paging
- Windows Swapping

**8.6 Android Memory Management**

**8.7 Summary**

**8.8 Key Terms, Review Questions, and Problems**

**Learning Objectives**

After studying this chapter, you should be able to:

- Define virtual memory.
- Describe the hardware and control structures that support virtual memory.
- Describe the various OS mechanisms used to implement virtual memory.
- Describe the virtual memory management mechanisms in UNIX, Linux, and Windows.

Chapter 7 introduced the concepts of paging and segmentation and analyzed their shortcomings. We now move to a discussion of virtual memory. An analysis of this topic is complicated by the fact that memory management is a complex interrelationship between processor hardware and operating system software. We will focus first on the hardware aspect of virtual memory, looking at the use of paging, segmentation, and combined paging and segmentation. Then we will look at the issues involved in the design of a virtual memory facility in operating systems.

Table 8.1 defines some key terms related to virtual memory.

## 8.1 HARDWARE AND CONTROL STRUCTURES

Comparing simple paging and simple segmentation, on the one hand, with fixed and dynamic partitioning, on the other, we see the foundation for a fundamental breakthrough in memory management. Two characteristics of paging and segmentation are the keys to this breakthrough:

1. All memory references within a process are logical addresses that are dynamically translated into physical addresses at run time. This means that a process may be swapped in and out of main memory such that it occupies different regions of main memory at different times during the course of execution.

**Table 8.1** Virtual Memory Terminology

| | |
|---|---|
| **Virtual memory** | A storage allocation scheme in which secondary memory can be addressed as though it were part of main memory. The addresses a program may use to reference memory are distinguished from the addresses the memory system uses to identify physical storage sites, and program-generated addresses are translated automatically to the corresponding machine addresses. The size of virtual storage is limited by the addressing scheme of the computer system, and by the amount of secondary memory available and not by the actual number of main storage locations. |
| **Virtual address** | The address assigned to a location in virtual memory to allow that location to be accessed as though it were part of main memory. |
| **Virtual address space** | The virtual storage assigned to a process. |
| **Address space** | The range of memory addresses available to a process. |
| **Real address** | The address of a storage location in main memory. |

2. A process may be broken up into a number of pieces (pages or segments) and these pieces need not be contiguously located in main memory during execution. The combination of dynamic run-time address translation and the use of a page or segment table permits this.

Now we come to the breakthrough. *If the preceding two characteristics are present, then it is not necessary that all of the pages or all of the segments of a process be in main memory during execution.* If the piece (segment or page) that holds the next instruction to be fetched and the piece that holds the next data location to be accessed are in main memory, then at least for a time execution may proceed.

Let us consider how this may be accomplished. For now, we can talk in general terms, and we will use the term *piece* to refer to either page or segment, depending on whether paging or segmentation is employed. Suppose it is time to bring a new process into memory. The OS begins by bringing in only one or a few pieces, to include the initial program piece and the initial data piece to which those instructions refer. The portion of a process that is actually in main memory at any time is called the **resident set** of the process. As the process executes, things proceed smoothly as long as all memory references are to locations that are in the resident set. Using the segment or page table, the processor always is able to determine whether this is so. If the processor encounters a logical address that is not in main memory, it generates an interrupt indicating a memory access fault. The OS puts the interrupted process in a blocking state. For the execution of this process to proceed later, the OS must bring into main memory the piece of the process that contains the logical address that caused the access fault. For this purpose, the OS issues a disk I/O (input/output) read request. After the I/O request has been issued, the OS can dispatch another process to run while the disk I/O is performed. Once the desired piece has been brought into main memory, an I/O interrupt is issued, giving control back to the OS, which places the affected process back into a Ready state.

It may immediately occur to you to question the efficiency of this maneuver, in which a process may be executing and have to be interrupted for no other reason than that you have failed to load in all of the needed pieces of the process. For now, let us defer consideration of this question with the assurance that efficiency is possible. Instead, let us ponder the implications of our new strategy. There are two implications, the second more startling than the first, and both lead to improved system utilization:

1. **More processes may be maintained in main memory.** Because we are only going to load some of the pieces of any particular process, there is room for more processes. This leads to more efficient utilization of the processor, because it is more likely that at least one of the more numerous processes will be in a Ready state at any particular time.
2. **A process may be larger than all of main memory.** One of the most fundamental restrictions in programming is lifted. Without the scheme we have been discussing, a programmer must be acutely aware of how much memory is available. If the program being written is too large, the programmer must devise ways to structure the program into pieces that can be loaded separately in some sort of overlay strategy. With virtual memory based on paging or segmentation, that job is left to the OS and the hardware. As far as the programmer is concerned,

he or she is dealing with a huge memory, the size associated with disk storage. The OS automatically loads pieces of a process into main memory as required.

Because a process executes only in main memory, that memory is referred to as **real memory**. But a programmer or user perceives a potentially much larger memory–that which is allocated on disk. This latter is referred to as **virtual memory**. Virtual memory allows for very effective multiprogramming and relieves the user of the unnecessarily tight constraints of main memory. Table 8.2 summarizes characteristics of paging and segmentation with and without the use of virtual memory.

## Locality and Virtual Memory

The benefits of virtual memory are attractive, but is the scheme practical? At one time, there was considerable debate on this point, but experience with numerous operating systems has demonstrated beyond doubt that virtual memory does work. Accordingly, virtual memory, based on either paging or paging plus segmentation, has become an essential component of contemporary operating systems.

**Table 8.2** Characteristics of Paging and Segmentation

| Simple Paging | Virtual Memory Paging | Simple Segmentation | Virtual Memory Segmentation |
|---|---|---|---|
| Main memory partitioned into small fixed-size chunks called frames. | | Main memory not partitioned. | |
| Program broken into pages by the compiler or memory management system. | | Program segments specified by the programmer to the compiler (i.e., the decision is made by the programmer). | |
| Internal fragmentation within frames. | | No internal fragmentation. | |
| No external fragmentation. | | External fragmentation. | |
| Operating system must maintain a page table for each process showing which frame each page occupies. | | Operating system must maintain a segment table for each process showing the load address and length of each segment. | |
| Operating system must maintain a free-frame list. | | Operating system must maintain a list of free holes in main memory. | |
| Processor uses page number, offset to calculate absolute address. | | Processor uses segment number, offset to calculate absolute address. | |
| All the pages of a process must be in main memory for process to run, unless overlays are used. | Not all pages of a process need be in main memory frames for the process to run. Pages may be read in as needed. | All the segments of a process must be in main memory for process to run, unless overlays are used. | Not all segments of a process need be in main memory for the process to run. Segments may be read in as needed. |
| | Reading a page into main memory may require writing a page out to disk. | | Reading a segment into main memory may require writing one or more segments out to disk. |

To understand the key issue and why virtual memory was a matter of much debate, let us examine again the task of the OS with respect to virtual memory. Consider a large process, consisting of a long program plus a number of arrays of data. Over any short period of time, execution may be confined to a small section of the program (e.g., a subroutine) and access to perhaps only one or two arrays of data. If this is so, then it would clearly be wasteful to load in dozens of pieces for that process when only a few pieces will be used before the program is suspended and swapped out. We can make better use of memory by loading in just a few pieces. Then, if the program branches to an instruction or references a data item on a piece not in main memory, a fault is triggered. This tells the OS to bring in the desired piece.

Thus, at any one time, only a few pieces of any given process are in memory, and therefore more processes can be maintained in memory. Furthermore, time is saved because unused pieces are not swapped in and out of memory. However, the OS must be clever about how it manages this scheme. In the steady state, practically all of main memory will be occupied with process pieces, so the processor and OS have direct access to as many processes as possible. Thus, when the OS brings one piece in, it must throw another out. If it throws out a piece just before it is used, then it will just have to go get that piece again almost immediately. Too much of this leads to a condition known as **thrashing**: The system spends most of its time swapping pieces rather than executing instructions. The avoidance of thrashing was a major research area in the 1970s and led to a variety of complex but effective algorithms. In essence, the OS tries to guess, based on recent history, which pieces are least likely to be used in the near future.

This reasoning is based on belief in the **principle of locality**, which was introduced in Chapter 1 (see especially Appendix 1A). To summarize, the principle of locality states that program and data references within a process tend to cluster. Hence, the assumption that only a few pieces of a process will be needed over a short period of time is valid. Also, it should be possible to make intelligent guesses about which pieces of a process will be needed in the near future, which avoids thrashing.

The principle of locality suggests that a virtual memory scheme may be effective. For virtual memory to be practical and effective, two ingredients are needed. First, there must be hardware support for the paging and/or segmentation scheme to be employed. Second, the OS must include software for managing the movement of pages and/or segments between secondary memory and main memory. In this section, we will examine the hardware aspect and look at the necessary control structures, which are created and maintained by the OS but are used by the memory management hardware. An examination of the OS issues will be provided in the next section.

## Paging

The term *virtual memory* is usually associated with systems that employ paging, although virtual memory based on segmentation is also used and will be discussed next. The use of paging to achieve virtual memory was first reported for the Atlas computer [KILB62] and soon came into widespread commercial use. Recall from Chapter 7 that with simple paging, main memory is divided into a number of equal-size frames. Each process is divided into a number of equal-size pages of the same length as frames. A process is loaded by loading all of its pages into available, not necessarily contiguous, frames. With virtual memory paging, we again have equal-size

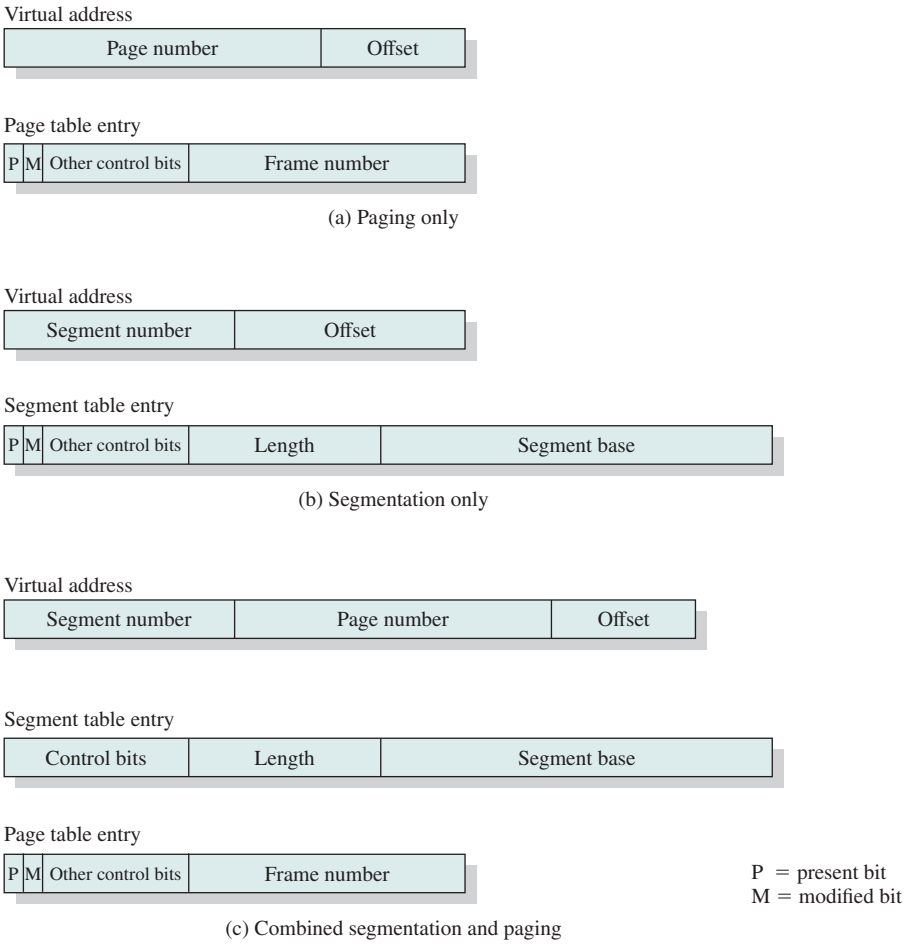



**Figure 8.1 Typical Memory Management Formats**

pages of the same length as frames; however, not all pages need to be loaded into main memory frames for execution.

In the discussion of simple paging, we indicated that each process has its own page table, and when all of its pages are loaded into main memory, the page table for a process is created and loaded into main memory. Each page table entry (PTE) contains the frame number of the corresponding page in main memory. A page table is also needed for a virtual memory scheme based on paging. Again, it is typical to associate a unique page table with each process. In this case, however, the page table entries become more complex (see Figure 8.1a). Because only some of the pages of a process may be in main memory, a bit is needed in each page table entry to indicate whether the corresponding page is present (P) in main memory or not. If the bit indicates that the page is in memory, then the entry also includes the frame number of that page.

The page table entry includes a modify (M) bit, indicating whether the contents of the corresponding page have been altered since the page was last loaded into main

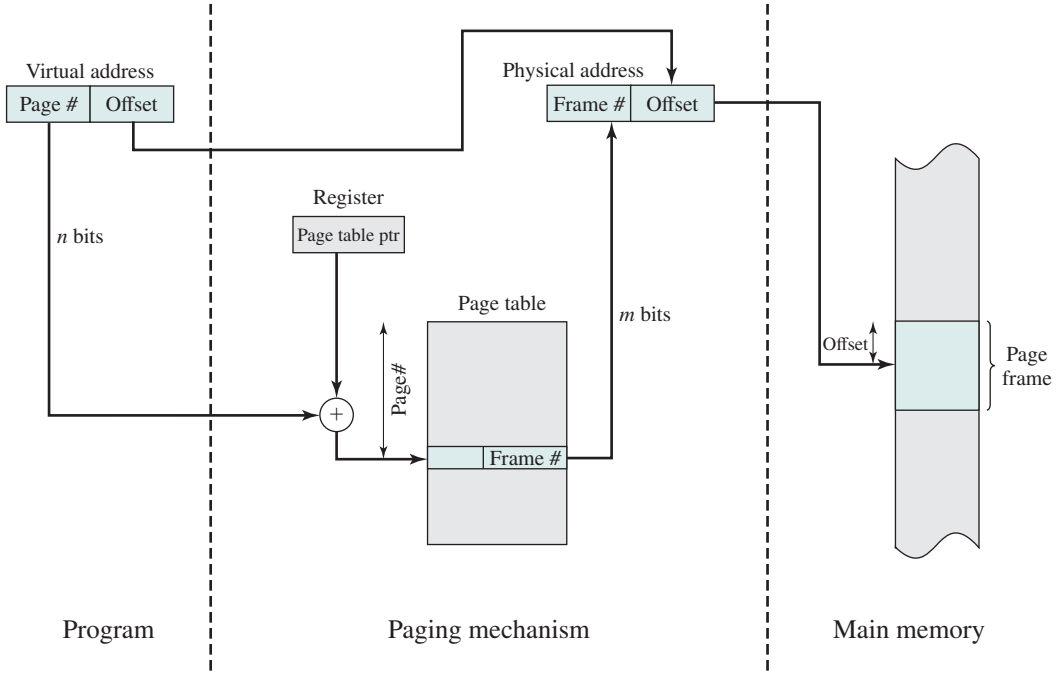



**Figure 8.2 Address Translation in a Paging System**

memory. If there has been no change, then it is not necessary to write the page out when it comes time to replace the page in the frame that it currently occupies. Other control bits may also be present. For example, if protection or sharing is managed at the page level, then bits for that purpose will be required.

***PAGE TABLE STRUCTURE*** The basic mechanism for reading a word from memory involves the translation of a virtual, or logical, address, consisting of page number and offset, into a physical address, consisting of frame number and offset, using a page table. Because the page table is of variable length, depending on the size of the process, we cannot expect to hold it in registers. Instead, it must be in main memory to be accessed. Figure 8.2 suggests a hardware implementation. When a particular process is running, a register holds the starting address of the page table for that process. The page number of a virtual address is used to index that table and look up the corresponding frame number. This is combined with the offset portion of the virtual address to produce the desired real address. Typically, the page number field is longer than the frame number field ($n > m$). This inequality results from the fact that the number of pages in a process may exceed the number of frames in main memory.

In most systems, there is one page table per process. But each process can occupy huge amounts of virtual memory. For example, in the VAX (Virtual Address Extension) architecture, each process can have up to $2^{31} = 2$ GB of virtual memory. Using $2^9 = 512$-byte pages means that as many as $2^{22}$ page table entries are required *per process*. Clearly, the amount of memory devoted to page tables alone could be unacceptably high. To overcome this problem, most virtual memory schemes store

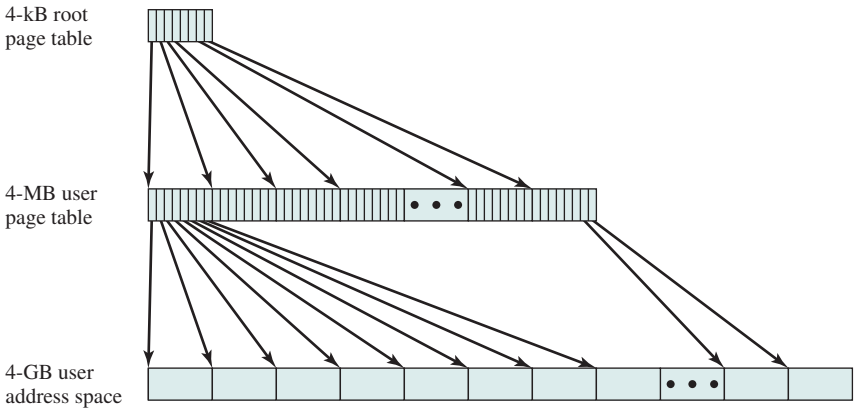



**Figure 8.3 A Two-Level Hierarchical Page Table**

page tables in virtual memory rather than real memory. This means page tables are subject to paging just as other pages are. When a process is running, at least a part of its page table must be in main memory, including the page table entry of the currently executing page. Some processors make use of a two-level scheme to organize large page tables. In this scheme, there is a page directory, in which each entry points to a page table. Thus, if the number of entries in the page directory is $X$, and if the maximum number of entries in a page table is $Y$, then a process can consist of up to $X \times Y$ pages. Typically, the maximum length of a page table is restricted to be equal to one page. For example, the Pentium processor uses this approach.

Figure 8.3 shows an example of a two-level scheme typical for use with a 32-bit address. If we assume byte-level addressing and 4-kB ($2^{12}$) pages, then the 4-GB ($2^{32}$) virtual address space is composed of $2^{20}$ pages. If each of these pages is mapped by a 4-byte page table entry, we can create a user page table composed of $2^{20}$ PTEs requiring 4 MB ($2^{22}$). This huge user page table, occupying $2^{10}$ pages, can be kept in virtual memory and mapped by a root page table with $2^{10}$ PTEs occupying 4 kB ($2^{12}$) of main memory. Figure 8.4 shows the steps involved in address translation for this scheme. The root page always remains in main memory. The first 10 bits of a virtual address are used to index into the root page to find a PTE for a page of the user page table. If that page is not in main memory, a page fault occurs. If that page is in main memory, then the next 10 bits of the virtual address index into the user PTE page to find the PTE for the page that is referenced by the virtual address.

***INVERTED PAGE TABLE*** A drawback of the type of page tables that we have been discussing is that their size is proportional to that of the virtual address space.

An alternative approach to the use of one or multiple-level page tables is the use of an **inverted page table** structure. Variations on this approach are used on the PowerPC, UltraSPARC, and the IA-64 architecture. An implementation of the Mach operating system on the RT-PC also uses this technique.

In this approach, the page number portion of a virtual address is mapped into a hash value using a simple hashing function.[1] The hash value is a pointer to the

[1]See Appendix F for a discussion of hashing.

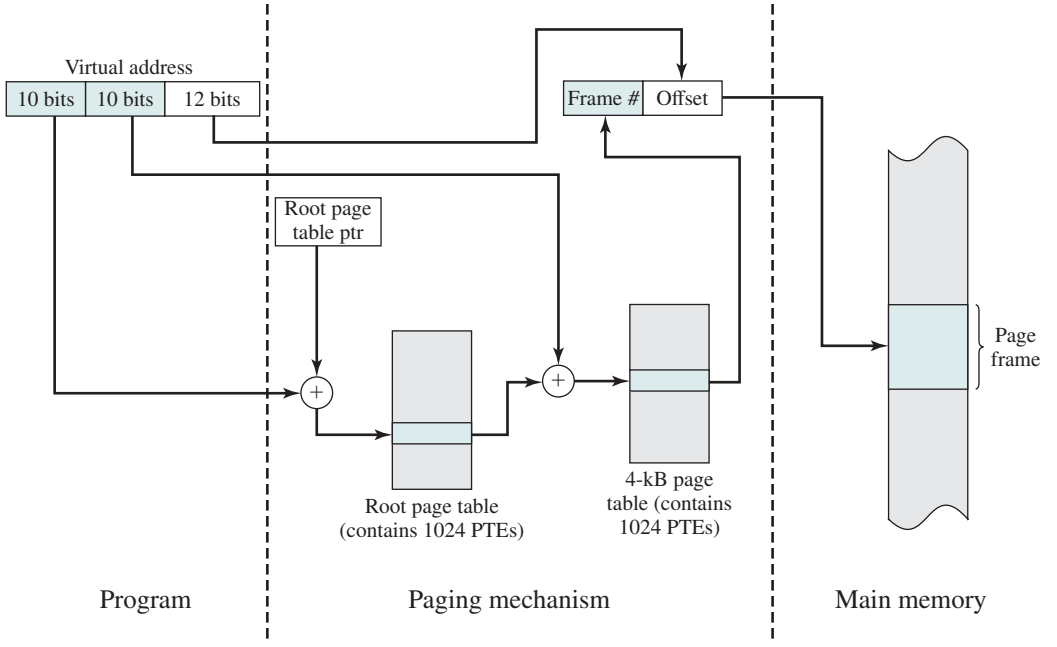



**Figure 8.4 Address Translation in a Two-Level Paging System**

inverted page table, which contains the page table entries. There is one entry in the inverted page table for each real memory page frame, rather than one per virtual page. Thus, a fixed proportion of real memory is required for the tables regardless of the number of processes or virtual pages supported. Because more than one virtual address may map into the same hash table entry, a chaining technique is used for managing the overflow. The hashing technique results in chains that are typically short—between one and two entries. The page table's structure is called *inverted* because it indexes page table entries by frame number rather than by virtual page number.

Figure 8.5 shows a typical implementation of the inverted page table approach. For a physical memory size of $2^m$ frames, the inverted page table contains $2^m$ entries, so that the $i$th entry refers to frame $i$. Each entry in the page table includes the following:

- **Page number:** This is the page number portion of the virtual address.
- **Process identifier:** The process that owns this page. The combination of page number and process identifier identifies a page within the virtual address space of a particular process.
- **Control bits:** This field includes flags, such as valid, referenced, and modified; and protection and locking information.
- **Chain pointer:** This field is null (perhaps indicated by a separate bit) if there are no chained entries for this entry. Otherwise, the field contains the index value (number between 0 and $2^m - 1$) of the next entry in the chain.

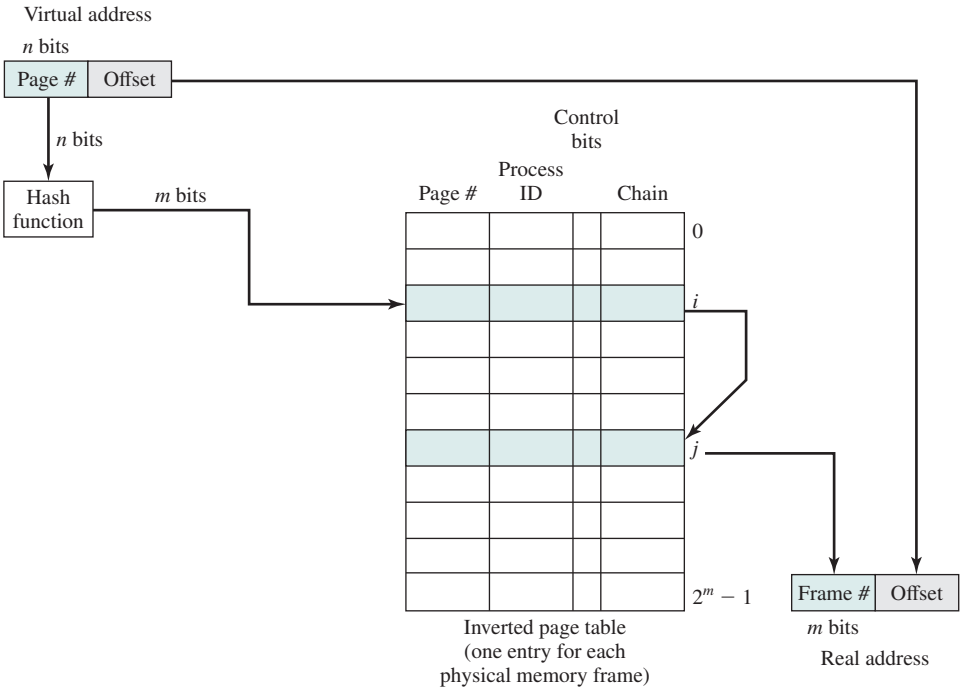



**Figure 8.5** **Inverted Page Table Structure**

In this example, the virtual address includes an $n$-bit page number, with $n > m$. The hash function maps the $n$-bit page number into an $m$-bit quantity, which is used to index into the inverted page table.

***TRANSLATION LOOKASIDE BUFFER*** In principle, every virtual memory reference can cause two physical memory accesses: one to fetch the appropriate page table entry, and another to fetch the desired data. Thus, a straightforward virtual memory scheme would have the effect of doubling the memory access time. To overcome this problem, most virtual memory schemes make use of a special high-speed cache for page table entries, usually called a **translation lookaside buffer (TLB)**. This cache functions in the same way as a memory cache (see Chapter 1) and contains those page table entries that have been most recently used. The organization of the resulting paging hardware is illustrated in Figure 8.6. Given a virtual address, the processor will first examine the TLB. If the desired page table entry is present (*TLB hit*), then the frame number is retrieved and the real address is formed. If the desired page table entry is not found (*TLB miss*), then the processor uses the page number to index the process page table and examine the corresponding page table entry. If the "present bit" is set, then the page is in main memory, and the processor can retrieve the frame number from the page table entry to form the real address. The processor also updates the TLB to include this new page table entry. Finally, if the present bit is not set, then the desired page is not in main memory and a memory access fault, called a **page fault**, is

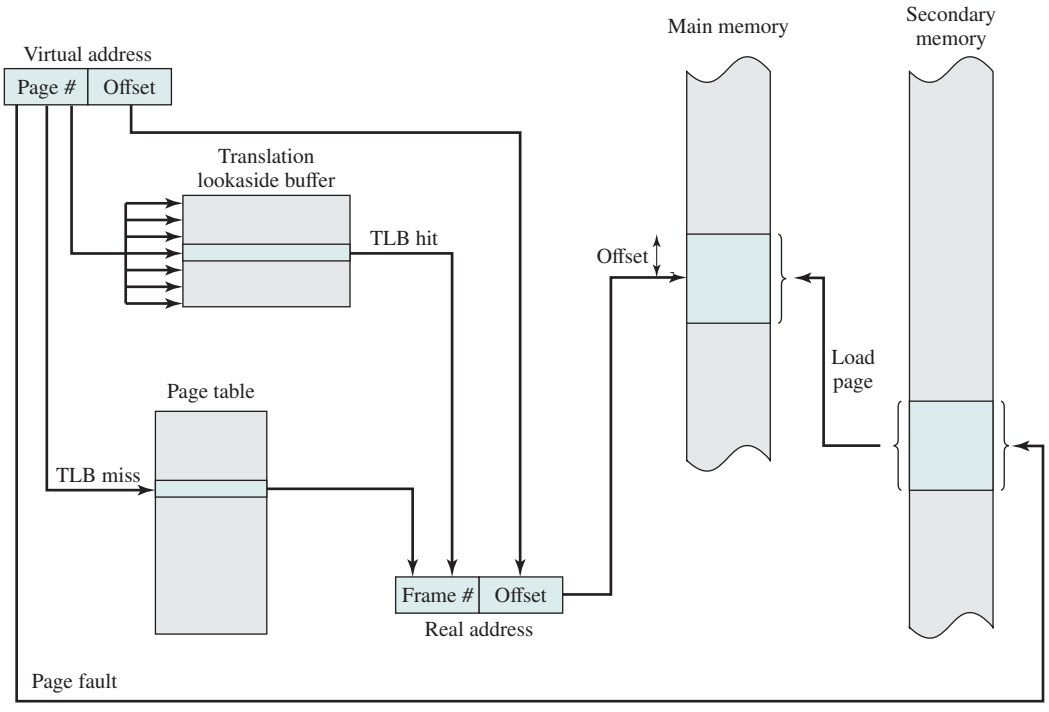



**Figure 8.6** **Use of a Translation Lookaside Buffer**

issued. At this point, we leave the realm of hardware and invoke the OS, which loads the needed page and updates the page table.

Figure 8.7 is a flowchart that shows the use of the TLB. The flowchart shows that if the desired page is not in main memory, a page fault interrupt causes the page fault handling routine to be invoked. To keep the flowchart simple, the fact that the OS may dispatch another process while disk I/O is underway is not shown. By the principle of locality, most virtual memory references will be to locations in recently used pages. Therefore, most references will involve page table entries in the cache. Studies of the VAX TLB have shown this scheme can significantly improve performance [CLAR85, SATY81].

There are a number of additional details concerning the actual organization of the TLB. Because the TLB contains only some of the entries in a full page table, we cannot simply index into the TLB based on page number. Instead, each entry in the TLB must include the page number as well as the complete page table entry. The processor is equipped with hardware that allows it to interrogate simultaneously a number of TLB entries to determine if there is a match on page number. This technique is referred to as **associative mapping** and is contrasted with the direct mapping, or indexing, used for lookup in the page table in Figure 8.8. The design of the TLB also must consider the way in which entries are organized in the TLB and which entry to replace when a new entry is brought in. These issues must be considered in any hardware cache design. This topic is not pursued here; the reader may consult a treatment of cache design for further details (e.g., [STAL16a]).

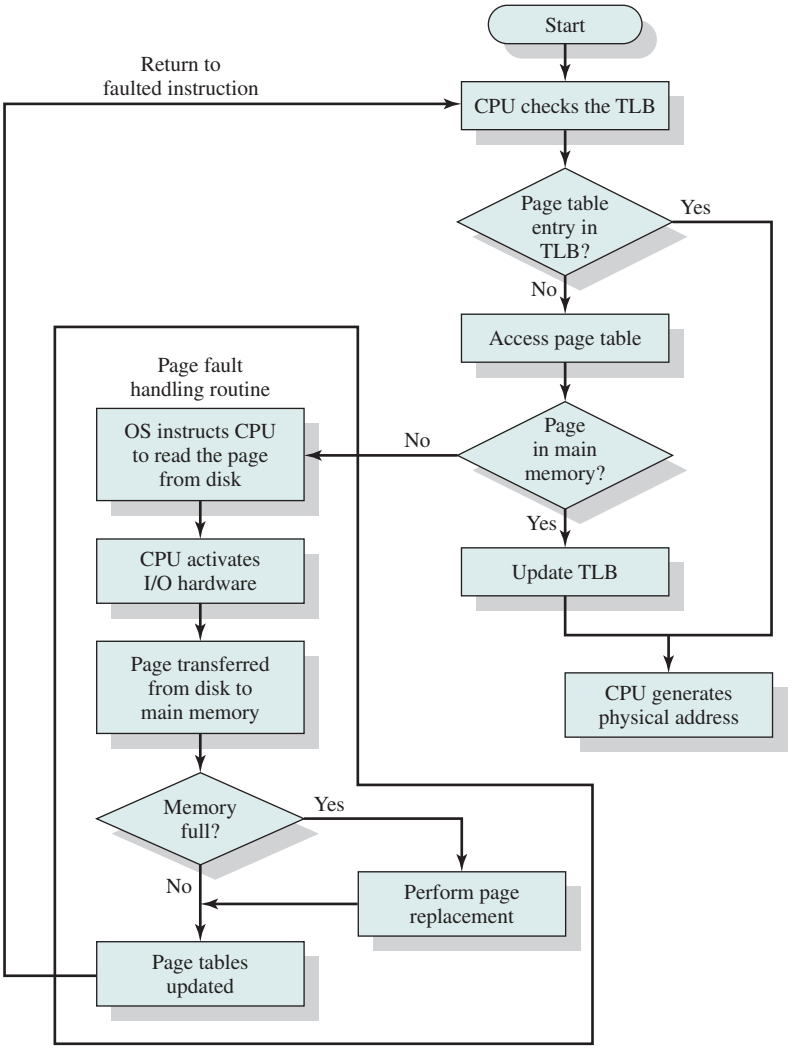



**Figure 8.7 Operation of Paging and Translation Lookaside Buffer (TLB)**

Finally, the virtual memory mechanism must interact with the cache system (not the TLB cache, but the main memory cache). This is illustrated in Figure 8.9. A virtual address will generally be in the form of a page number, offset. First, the memory system consults the TLB to see if the matching page table entry is present. If it is, the real (physical) address is generated by combining the frame number with the offset. If not, the entry is accessed from a page table. Once the real address is generated, which is in the form of a tag[2] and a remainder, the cache is consulted to see if the

[2]See Figure 1.17. Typically, a tag is just the leftmost bits of the real address. Again, for a more detailed discussion of caches, see [STAL16a].

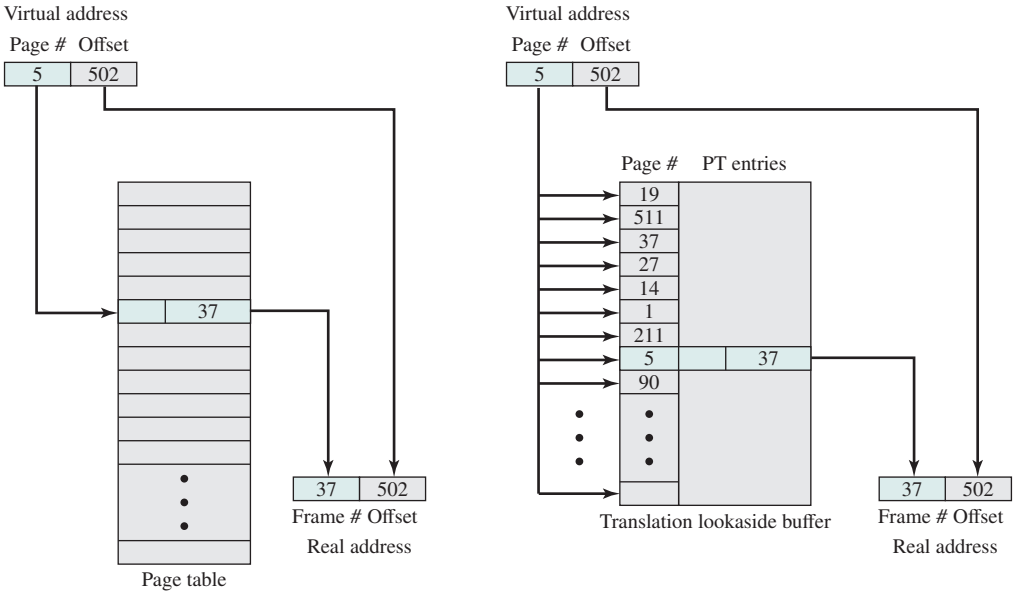



**Figure 8.8 Direct versus Associative Lookup for Page Table Entries**

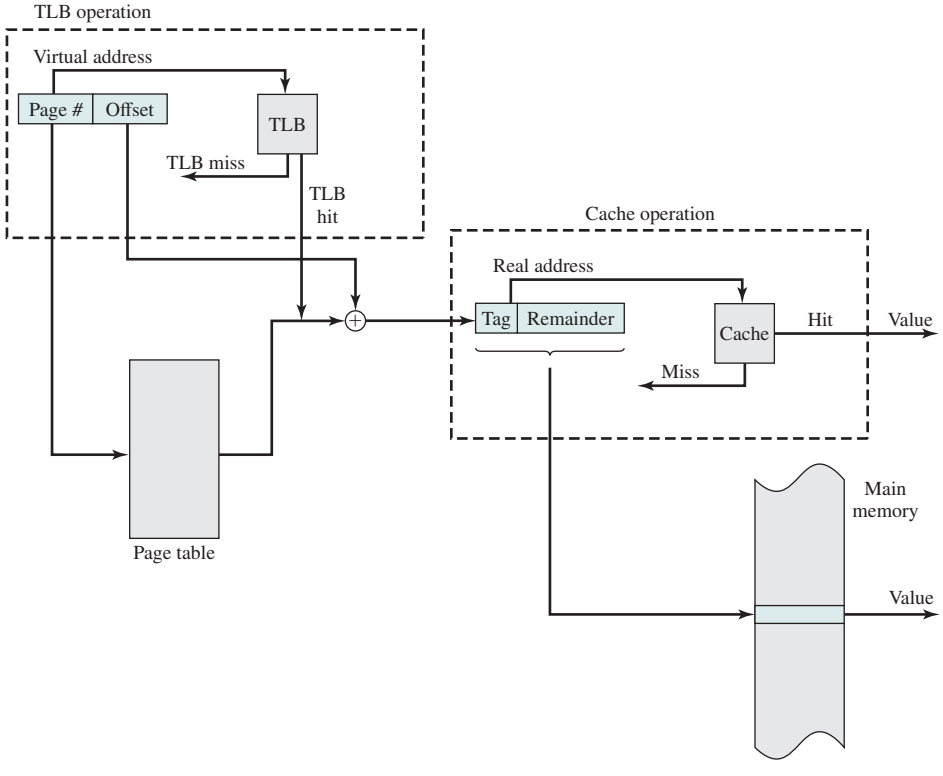


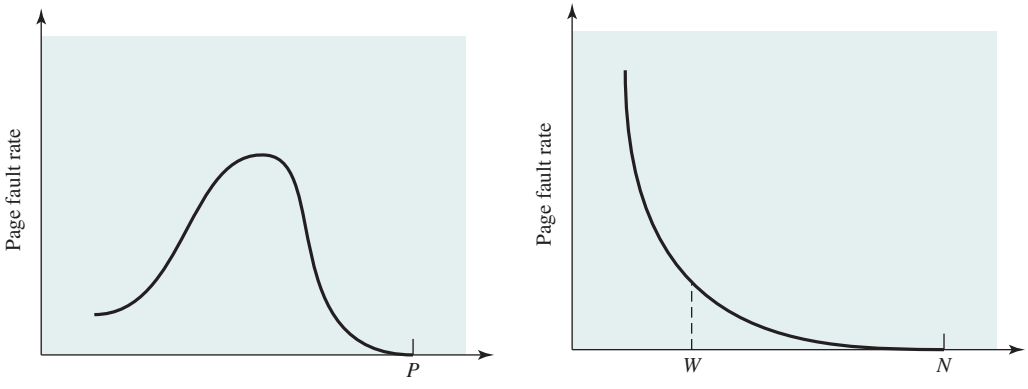

**Figure 8.9 Translation Lookaside Buffer and Cache Operation**

block containing that word is present. If so, it is returned to the CPU. If not, the word is retrieved from main memory.

The reader should be able to appreciate the complexity of the CPU hardware involved in a single memory reference. The virtual address is translated into a real address. This involves reference to a page table entry, which may be in the TLB, in main memory, or on disk. The referenced word may be in cache, main memory, or on disk. If the referenced word is only on disk, the page containing the word must be loaded into main memory and its block loaded into the cache. In addition, the page table entry for that page must be updated.

***PAGE SIZE*** An important hardware design decision is the size of page to be used. There are several factors to consider. One is internal fragmentation. Clearly, the smaller the page size, the lesser is the amount of internal fragmentation. To optimize the use of main memory, we would like to reduce internal fragmentation. On the other hand, the smaller the page, the greater is the number of pages required per process. More pages per process means larger page tables. For large programs in a heavily multiprogrammed environment, this may mean that some portion of the page tables of active processes must be in virtual memory, not in main memory. Thus, there may be a double page fault for a single reference to memory: first to bring in the needed portion of the page table, and second to bring in the process page. Another factor is that the physical characteristics of most secondary-memory devices, which are rotational, favor a larger page size for more efficient block transfer of data.

Complicating these matters is the effect of page size on the rate at which page faults occur. This behavior, in general terms, is depicted in Figure 8.10a and is based on the principle of locality. If the page size is very small, then ordinarily a relatively large number of pages will be available in main memory for a process. After a time,




(a) Page size

(b) Number of page frames allocated

$P$ = size of entire process
$W$ = working set size
$N$ = total number of pages in process

**Figure 8.10 Typical Paging Behavior of a Program**

**Table 8.3** Example of Page Sizes

| Computer | Page Size |
|---|---|
| Atlas | 512 48-bit words |
| Honeywell-Multics | 1,024 36-bit words |
| IBM 370/XA and 370/ESA | 4 kB |
| VAX family | 512 bytes |
| IBM AS/400 | 512 bytes |
| DEC Alpha | 8 kB |
| MIPS | 4 kB to 16 MB |
| UltraSPARC | 8 kB to 4 MB |
| Pentium | 4 kB or 4 MB |
| Intel Itanium | 4 kB to 256 MB |
| Intel core i7 | 4 kB to 1 GB |

the pages in memory will all contain portions of the process near recent references. Thus, the page fault rate should be low. As the size of the page is increased, each individual page will contain locations further and further from any particular recent reference. Thus, the effect of the principle of locality is weakened and the page fault rate begins to rise. Eventually, however, the page fault rate will begin to fall as the size of a page approaches the size of the entire process (point *P* in the diagram). When a single page encompasses the entire process, there will be no page faults.

A further complication is that the page fault rate is also determined by the number of frames allocated to a process. Figure 8.10b shows that for a fixed page size, the fault rate drops as the number of pages maintained in main memory grows.[3] Thus, a software policy (the amount of memory to allocate to each process) interacts with a hardware design decision (page size).

Table 8.3 lists the page sizes used on some machines.

Finally, the design issue of page size is related to the size of physical main memory and program size. At the same time that main memory is getting larger, the address space used by applications is also growing. The trend is most obvious on personal computers and workstations, where applications are becoming increasingly complex. Furthermore, contemporary programming techniques used in large programs tend to decrease the locality of references within a process [HUCK93]. For example,

- Object-oriented techniques encourage the use of many small program and data modules with references scattered over a relatively large number of objects over a relatively short period of time.
- Multithreaded applications may result in abrupt changes in the instruction stream and in scattered memory references.

[3]The parameter *W* represents working set size, a concept discussed in Section 8.2.

For a given size of TLB, as the memory size of processes grows and as locality decreases, the hit ratio on TLB accesses declines. Under these circumstances, the TLB can become a performance bottleneck (e.g., see [CHEN92]).

One way to improve TLB performance is to use a larger TLB with more entries. However, TLB size interacts with other aspects of the hardware design, such as the main memory cache and the number of memory accesses per instruction cycle [TALL92]. The upshot is that TLB size is unlikely to grow as rapidly as main memory size. An alternative is to use larger page sizes so each page table entry in the TLB refers to a larger block of memory. But we have just seen that the use of large page sizes can lead to performance degradation.

Accordingly, a number of designers have investigated the use of multiple page sizes [TALL92, KHAL93], and several microprocessor architectures support multiple pages sizes, including MIPS R4000, Alpha, UltraSPARC, x86, and IA-64. Multiple page sizes provide the flexibility needed to use a TLB effectively. For example, large contiguous regions in the address space of a process, such as program instructions, may be mapped using a small number of large pages rather than a large number of small pages, while thread stacks may be mapped using the small page size. However, most commercial operating systems still support only one page size, regardless of the capability of the underlying hardware. The reason for this is that page size affects many aspects of the OS; thus, a change to multiple page sizes is a complex undertaking (see [GANA98] for a discussion).

## Segmentation

***Virtual Memory Implications*** Segmentation allows the programmer to view memory as consisting of multiple address spaces or segments. Segments may be of unequal, indeed dynamic, size. Memory references consist of a (segment number, offset) form of address.

This organization has a number of advantages to the programmer over a non-segmented address space:

1. It simplifies the handling of growing data structures. If the programmer does not know ahead of time how large a particular data structure will become, it is necessary to guess unless dynamic segment sizes are allowed. With segmented virtual memory, the data structure can be assigned its own segment, and the OS will expand or shrink the segment as needed. If a segment that needs to be expanded is in main memory and there is insufficient room, the OS may move the segment to a larger area of main memory, if available, or swap it out. In the latter case, the enlarged segment would be swapped back in at the next opportunity.
2. It allows programs to be altered and recompiled independently, without requiring the entire set of programs to be relinked and reloaded. Again, this is accomplished using multiple segments.
3. It lends itself to sharing among processes. A programmer can place a utility program or a useful table of data in a segment that can be referenced by other processes.
4. It lends itself to protection. Because a segment can be constructed to contain a well-defined set of programs or data, the programmer or system administrator can assign access privileges in a convenient fashion.

***ORGANIZATION*** In the discussion of simple segmentation, we indicated that each process has its own segment table, and when all of its segments are loaded into main memory, the segment table for a process is created and loaded into main memory. Each segment table entry contains the starting address of the corresponding segment in main memory, as well as the length of the segment. The same device, a segment table, is needed when we consider a virtual memory scheme based on segmentation. Again, it is typical to associate a unique segment table with each process. In this case, however, the segment table entries become more complex (see Figure 8.1b). Because only some of the segments of a process may be in main memory, a bit is needed in each segment table entry to indicate whether the corresponding segment is present in main memory or not. If the bit indicates that the segment is in memory, then the entry also includes the starting address and length of that segment.

Another control bit in the segmentation table entry is a modify bit, indicating whether the contents of the corresponding segment have been altered since the segment was last loaded into main memory. If there has been no change, then it is not necessary to write the segment out when it comes time to replace the segment in the frame that it currently occupies. Other control bits may also be present. For example, if protection or sharing is managed at the segment level, then bits for that purpose will be required.

The basic mechanism for reading a word from memory involves the translation of a virtual, or logical, address, consisting of segment number and offset, into a physical address, using a segment table. Because the segment table is of variable length, depending on the size of the process, we cannot expect to hold it in registers. Instead, it must be in main memory to be accessed. Figure 8.11 suggests a hardware implementation of this scheme (note similarity to Figure 8.2). When a particular process is running, a register holds the starting address of the segment table for that process.

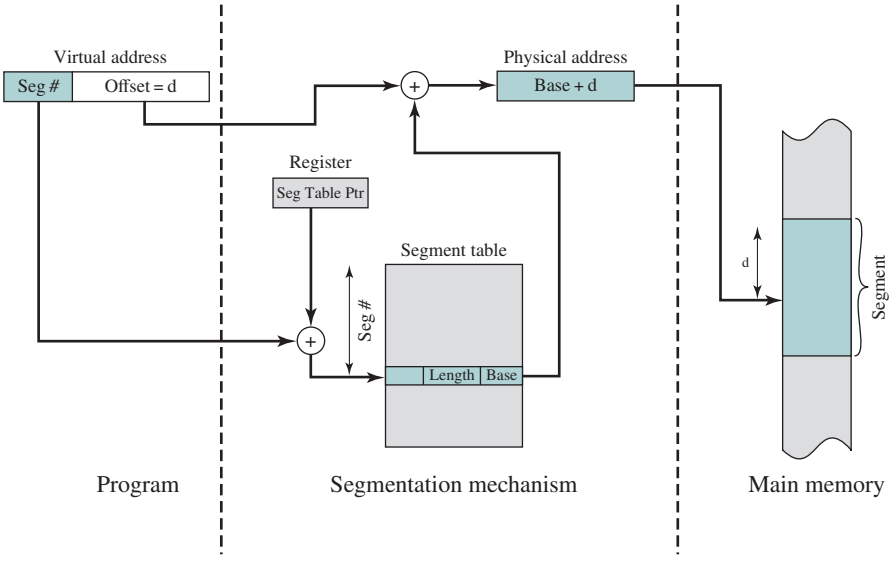



**Figure 8.11 Address Translation in a Segmentation System**

The segment number of a virtual address is used to index that table and look up the corresponding main memory address for the start of the segment. This is added to the offset portion of the virtual address to produce the desired real address.

## Combined Paging and Segmentation

Both paging and segmentation have their strengths. Paging, which is transparent to the programmer, eliminates external fragmentation and thus provides efficient use of main memory. In addition, because the pieces that are moved in and out of main memory are of fixed, equal size, it is possible to develop sophisticated memory management algorithms that exploit the behavior of programs, as we shall see. Segmentation, which is visible to the programmer, has the strengths listed earlier, including the ability to handle growing data structures, modularity, and support for sharing and protection. To combine the advantages of both, some systems are equipped with processor hardware and OS software to provide both.

In a combined paging/segmentation system, a user's address space is broken up into a number of segments, at the discretion of the programmer. Each segment is, in turn, broken up into a number of fixed-size pages, which are equal in length to a main memory frame. If a segment has length less than that of a page, the segment occupies just one page. From the programmer's point of view, a logical address still consists of a segment number and a segment offset. From the system's point of view, the segment offset is viewed as a page number and page offset for a page within the specified segment.

Figure 8.12 suggests a structure to support combined paging/segmentation (note the similarity to Figure 8.4). Associated with each process is a segment table and a number of page tables, one per process segment. When a particular process is

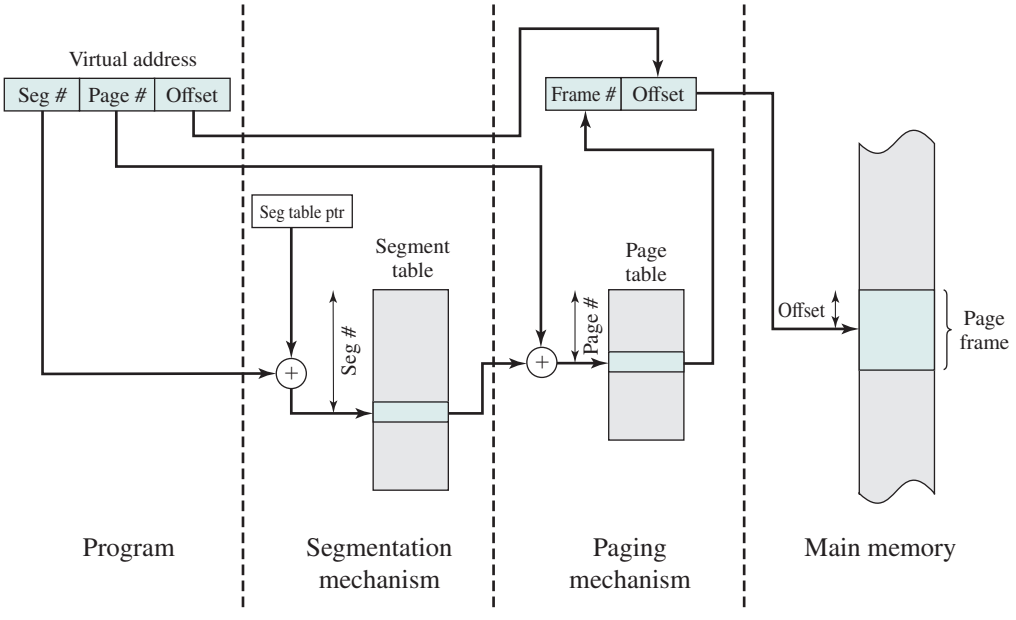



**Figure 8.12 Address Translation in a Segmentation/Paging System**

running, a register holds the starting address of the segment table for that process. Presented with a virtual address, the processor uses the segment number portion to index into the process segment table to find the page table for that segment. Then the page number portion of the virtual address is used to index the page table and look up the corresponding frame number. This is combined with the offset portion of the virtual address to produce the desired real address.

Figure 8.1c suggests the segment table entry and page table entry formats. As before, the segment table entry contains the length of the segment. It also contains a base field, which now refers to a page table. The present and modified bits are not needed because these matters are handled at the page level. Other control bits may be used, for purposes of sharing and protection. The page table entry is essentially the same as is used in a pure paging system. Each page number is mapped into a corresponding frame number if the page is present in main memory. The modified bit indicates whether this page needs to be written back out when the frame is allocated to another page. There may be other control bits dealing with protection or other aspects of memory management.

### Protection and Sharing

Segmentation lends itself to the implementation of protection and sharing policies. Because each segment table entry includes a length as well as a base address, a program cannot inadvertently access a main memory location beyond the limits of a segment. To achieve sharing, it is possible for a segment to be referenced in the segment tables of more than one process. The same mechanisms are, of course, available in a paging system. However, in this case, the page structure of programs and data is not visible to the programmer, making the specification of protection and sharing requirements more awkward. Figure 8.13 illustrates the types of protection relationships that can be enforced in such a system.

More sophisticated mechanisms can also be provided. A common scheme is to use a ring-protection structure, of the type we referred to in Chapter 3 (see Figure 3.18). In this scheme, lower-numbered, or inner, rings enjoy greater privilege than higher-numbered, or outer, rings. Typically, ring 0 is reserved for kernel functions of the OS, with applications at a higher level. Some utilities or OS services may occupy an intermediate ring. Basic principles of the ring system are as follows:

- A program may access only data that reside on the same ring or a less-privileged ring.
- A program may call services residing on the same or a more-privileged ring.

## 8.2 OPERATING SYSTEM SOFTWARE

The design of the memory management portion of an OS depends on three fundamental areas of choice:

1. Whether or not to use virtual memory techniques
2. The use of paging or segmentation or both
3. The algorithms employed for various aspects of memory management

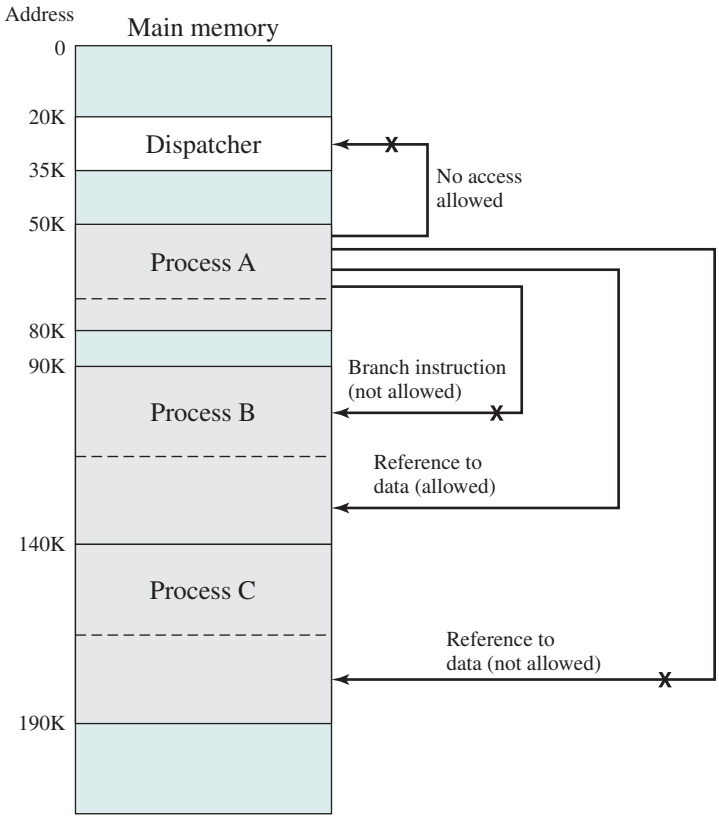



**Figure 8.13** **Protection Relationships between Segments**

The choices made in the first two areas depend on the hardware platform available. Thus, earlier UNIX implementations did not provide virtual memory because the processors on which the system ran did not support paging or segmentation. Neither of these techniques is practical without hardware support for address translation and other basic functions.

Two additional comments about the first two items in the preceding list: First, with the exception of operating systems for some of the older personal computers, such as MS-DOS, and specialized systems, all important operating systems provide virtual memory. Second, pure segmentation systems are becoming increasingly rare. When segmentation is combined with paging, most of the memory management issues confronting the OS designer are in the area of paging.[4] Thus, we can concentrate in this section on the issues associated with paging.

The choices related to the third item are the domain of operating system software and are the subject of this section. Table 8.4 lists the key design elements that

[4]Protection and sharing are usually dealt with at the segment level in a combined segmentation/paging system. We will deal with these issues in later chapters.

**Table 8.4** Operating System Policies for Virtual Memory

| | |
|---|---|
| **Fetch Policy**<br>Demand paging<br>Prepaging<br><br>**Placement Policy**<br><br>**Replacement Policy**<br>Basic Algorithms<br>Optimal<br>Least recently used (LRU)<br>First-in-first-out (FIFO)<br>Clock<br>Page Buffering | **Resident Set Management**<br>Resident set size<br>Fixed<br>Variable<br>Replacement Scope<br>Global<br>Local<br><br>**Cleaning Policy**<br>Demand<br>Precleaning<br><br>**Load Control**<br>Degree of multiprogramming |

we examine. In each case, the key issue is one of performance: We would like to minimize the rate at which page faults occur, because page faults cause considerable software overhead. At a minimum, the overhead includes deciding which resident page or pages to replace, and the I/O of exchanging pages. Also, the OS must schedule another process to run during the page I/O, causing a process switch. Accordingly, we would like to arrange matters so during the time that a process is executing, the probability of referencing a word on a missing page is minimized. In all of the areas referred to in Table 8.4, there is no definitive policy that works best.

As we shall see, the task of memory management in a paging environment is fiendishly complex. Furthermore, the performance of any particular set of policies depends on main memory size, the relative speed of main and secondary memory, the size and number of processes competing for resources, and the execution behavior of individual programs. This latter characteristic depends on the nature of the application, the programming language and compiler employed, the style of the programmer who wrote it, and, for an interactive program, the dynamic behavior of the user. Thus, the reader must expect no final answers here or anywhere. For smaller systems, the OS designer should attempt to choose a set of policies that seems "good" over a wide range of conditions, based on the current state of knowledge. For larger systems, particularly mainframes, the operating system should be equipped with monitoring and control tools that allow the site manager to tune the operating system to get "good" results based on site conditions.

## Fetch Policy

The fetch policy determines when a page should be brought into main memory. The two common alternatives are demand paging and prepaging. With **demand paging**, a page is brought into main memory only when a reference is made to a location on that page. If the other elements of memory management policy are good, the following should happen. When a process is first started, there will be a flurry of page faults. As more and more pages are brought in, the principle of locality suggests that most future references will be to pages that have recently been brought in. Thus, after

a time, matters should settle down and the number of page faults should drop to a very low level.

With **prepaging**, pages other than the one demanded by a page fault are brought in. Prepaging exploits the characteristics of most secondary memory devices, such as disks, which have seek times and rotational latency. If the pages of a process are stored contiguously in secondary memory, then it is more efficient to bring in a number of contiguous pages at one time rather than bringing them in one at a time over an extended period. Of course, this policy is ineffective if most of the extra pages that are brought in are not referenced.

The prepaging policy could be employed either when a process first starts up, in which case the programmer would somehow have to designate desired pages, or every time a page fault occurs. This latter course would seem preferable because it is invisible to the programmer.

Prepaging should not be confused with swapping. When a process is swapped out of memory and put in a suspended state, all of its resident pages are moved out. When the process is resumed, all of the pages that were previously in main memory are returned to main memory.

## Placement Policy

The placement policy determines where in real memory a process piece is to reside. In a pure segmentation system, the placement policy is an important design issue; policies such as best-fit, first-fit, and so on, which were discussed in Chapter 7, are possible alternatives. However, for a system that uses either pure paging or paging combined with segmentation, placement is usually irrelevant because the address translation hardware and the main memory access hardware can perform their functions for any page-frame combination with equal efficiency.

There is one area in which placement does become a concern, and this is a subject of research and development. On a so-called nonuniform memory access (NUMA) multiprocessor, the distributed, shared memory of the machine can be referenced by any processor on the machine, but the time for accessing a particular physical location varies with the distance between the processor and the memory module. Thus, performance depends heavily on the extent to which data reside close to the processors that use them [LARO92, BOLO89, COX89]. For NUMA systems, an automatic placement strategy is desirable to assign pages to the memory module that provides the best performance.

## Replacement Policy

In most operating system texts, the treatment of memory management includes a section entitled "replacement policy," which deals with the selection of a page in main memory to be replaced when a new page must be brought in. This topic is sometimes difficult to explain because several interrelated concepts are involved:

- How many page frames are to be allocated to each active process

- Whether the set of pages to be considered for replacement should be limited to those of the process that caused the page fault or encompass all the page frames in main memory
- Among the set of pages considered, which particular page should be selected for replacement

We shall refer to the first two concepts as *resident set management*, which will be dealt with in the next subsection, and reserve the term *replacement policy* for the third concept, which is discussed in this subsection.

The area of replacement policy is probably the most studied of any area of memory management. When all of the frames in main memory are occupied and it is necessary to bring in a new page to satisfy a page fault, the replacement policy determines which page currently in memory is to be replaced. All of the policies have as their objective that the page to be removed should be the page least likely to be referenced in the near future. Because of the principle of locality, there is often a high correlation between recent referencing history and near-future referencing patterns. Thus, most policies try to predict future behavior on the basis of past behavior. One trade-off that must be considered is that the more elaborate and sophisticated the replacement policy, the greater will be the hardware and software overhead to implement it.

***Frame Locking*** One restriction on replacement policy needs to be mentioned before looking at various algorithms: Some of the frames in main memory may be locked. When a frame is locked, the page currently stored in that frame may not be replaced. Much of the kernel of the OS, as well as key control structures, are held in locked frames. In addition, I/O buffers and other time-critical areas may be locked into main memory frames. Locking is achieved by associating a lock bit with each frame. This bit may be kept in a frame table as well as being included in the current page table.

***Basic Algorithms*** Regardless of the resident set management strategy (discussed in the next subsection), there are certain basic algorithms that are used for the selection of a page to replace. Replacement algorithms that have been discussed in the literature include:

- Optimal
- Least recently used (LRU)
- First-in-first-out (FIFO)
- Clock

The **optimal** policy selects for replacement that page for which the time to the next reference is the longest. It can be shown that this policy results in the fewest number of page faults [BELA66]. Clearly, this policy is impossible to implement, because it would require the OS to have perfect knowledge of future events. However, it does serve as a standard against which to judge real-world algorithms.

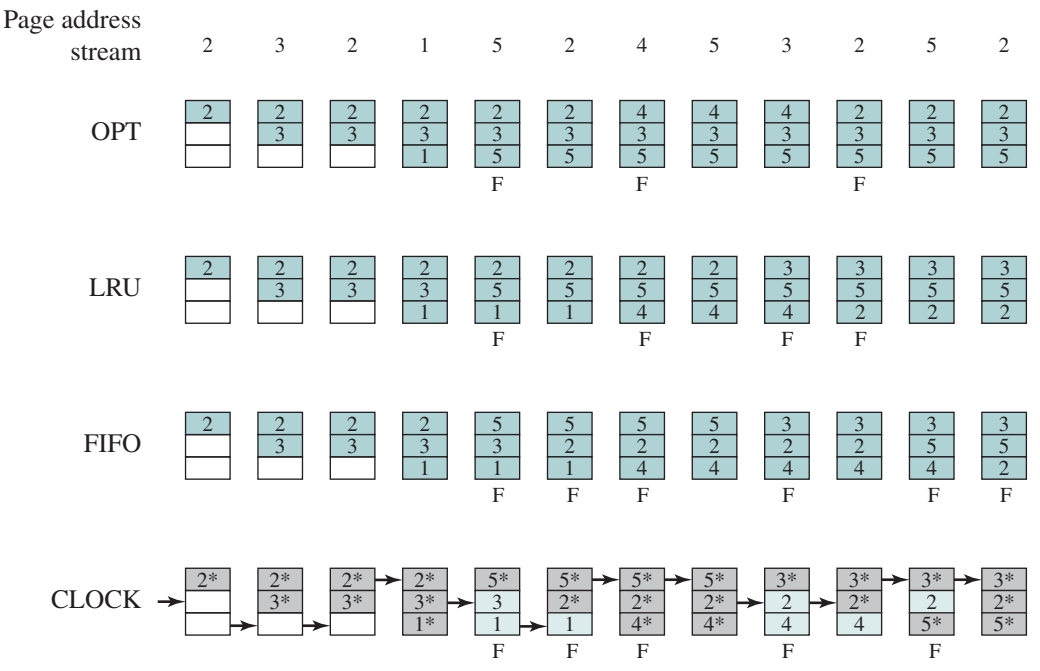



**Figure 8.14 Behavior of Four Page Replacement Algorithms**

Figure 8.14 gives an example of the optimal policy. The example assumes a fixed frame allocation (fixed resident set size) for this process of three frames. The execution of the process requires reference to five distinct pages. The page address stream formed by executing the program is

2 3 2 1 5 2 4 5 3 2 5 2

which means that the first page referenced is 2, the second page referenced is 3, and so on. The optimal policy produces three page faults after the frame allocation has been filled.

The **least recently used (LRU)** policy replaces the page in memory that has not been referenced for the longest time. By the principle of locality, this should be the page least likely to be referenced in the near future. And, in fact, the LRU policy does nearly as well as the optimal policy. The problem with this approach is the difficulty in implementation. One approach would be to tag each page with the time of its last reference; this would have to be done at each memory reference, both instruction and data. Even if the hardware would support such a scheme, the overhead would be tremendous. Alternatively, one could maintain a stack of page references, again an expensive prospect.

Figure 8.14 shows an example of the behavior of LRU, using the same page address stream as for the optimal policy example. In this example, there are four page faults.

The **first-in-first-out (FIFO)** policy treats the page frames allocated to a process as a circular buffer, and pages are removed in round-robin style. All that is required is a pointer that circles through the page frames of the process. This is therefore one of the simplest page replacement policies to implement. The logic behind this choice, other than its simplicity, is that one is replacing the page that has been in memory the longest: A page fetched into memory a long time ago may have now fallen out of use. This reasoning will often be wrong, because there will often be regions of program or data that are heavily used throughout the life of a program. Those pages will be repeatedly paged in and out by the FIFO algorithm.

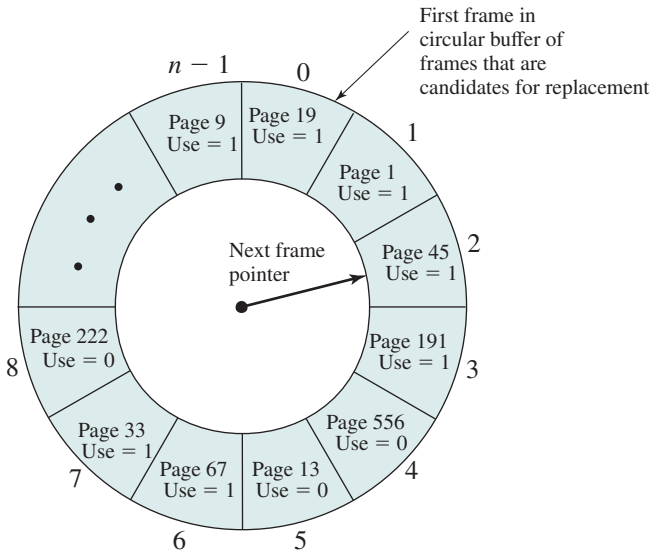
Continuing our example in Figure 8.14, the FIFO policy results in six page faults. Note that LRU recognizes that pages 2 and 5 are referenced more frequently than other pages, whereas FIFO does not.

Although the LRU policy does nearly as well as an optimal policy, it is difficult to implement and imposes significant overhead. On the other hand, the FIFO policy is very simple to implement but performs relatively poorly. Over the years, OS designers have tried a number of other algorithms to approximate the performance of LRU while imposing little overhead. Many of these algorithms are variants of a scheme referred to as the **clock policy**.

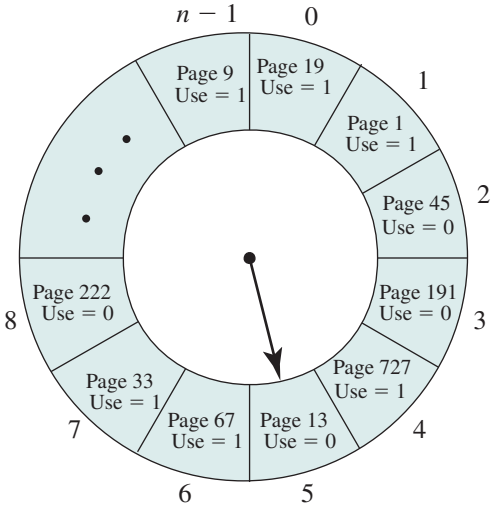
The simplest form of clock policy requires the association of an additional bit with each frame, referred to as the use bit. When a page is first loaded into a frame in memory, the use bit for that frame is set to 1. Whenever the page is subsequently referenced (after the reference that generated the page fault), its use bit is set to 1. For the page replacement algorithm, the set of frames that are candidates for replacement (this process: local scope; all of main memory: global scope[5]) is considered to be a circular buffer, with which a pointer is associated. When a page is replaced, the pointer is set to indicate the next frame in the buffer after the one just updated. When it comes time to replace a page, the OS scans the buffer to find a frame with a use bit set to 0. Each time it encounters a frame with a use bit of 1, it resets that bit to 0 and continues on. If any of the frames in the buffer have a use bit of 0 at the beginning of this process, the first such frame encountered is chosen for replacement. If all of the frames have a use bit of 1, then the pointer will make one complete cycle through the buffer, setting all the use bits to 0, and stop at its original position, replacing the page in that frame. We can see that this policy is similar to FIFO, except that, in the clock policy, any frame with a use bit of 1 is passed over by the algorithm. The policy is referred to as a clock policy because we can visualize the page frames as laid out in a circle. A number of operating systems have employed some variation of this simple clock policy (e.g., Multics [CORB68]).

Figure 8.15 provides an example of the simple clock policy mechanism. A circular buffer of $n$ main memory frames is available for page replacement. Just prior to the replacement of a page from the buffer with incoming page 727, the next frame pointer points at frame 2, which contains page 45. The clock policy is now executed. Because the use bit for page 45 in frame 2 is equal to 1, this page is not replaced. Instead, the use bit is set to 0 and the pointer advances. Similarly, page 191 in frame 3 is not replaced; its use bit is set to 0 and the pointer advances. In the next frame,

[5]The concept of scope will be discussed in the subsection "Replacement Scope."




(a) State of buffer just prior to a page replacement




(b) State of buffer just after the next page replacement

**Figure 8.15 Example of Clock Policy Operation**

frame 4, the use bit is set to 0. Therefore, page 556 is replaced with page 727. The use bit is set to 1 for this frame and the pointer advances to frame 5, completing the page replacement procedure.

The behavior of the clock policy is illustrated in Figure 8.14. The presence of an asterisk indicates that the corresponding use bit is equal to 1, and the arrow indicates the current position of the pointer. Note the clock policy is adept at protecting frames 2 and 5 from replacement.

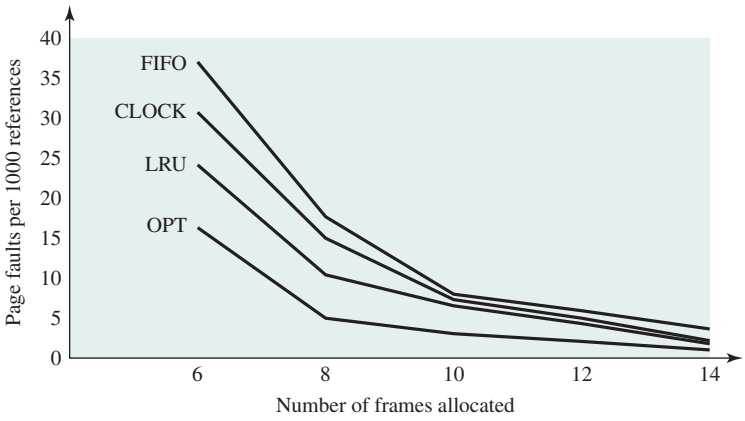



**Figure 8.16** **Comparison of Fixed-Allocation, Local Page Replacement Algorithms**

Figure 8.16 shows the results of an experiment reported in [BAER80], which compares the four algorithms that we have been discussing; it is assumed the number of page frames assigned to a process is fixed. The results are based on the execution of $0.25 \times 10^6$ references in a FORTRAN program, using a page size of 256 words. Baer ran the experiment with frame allocations of 6, 8, 10, 12, and 14 frames. The differences among the four policies are most striking at small allocations, with FIFO being over a factor of 2 worse than optimal. All four curves have the same shape as the idealized behavior shown in Figure 8.10b. In order to run efficiently, we would like to be to the right of the knee of the curve (with a small page fault rate) while keeping a small frame allocation (to the left of the knee of the curve). These two constraints indicate that a desirable mode of operation would be at the knee of the curve.

Almost identical results have been reported in [FINK88], again showing a maximum spread of about a factor of 2. Finkel's approach was to simulate the effects of various policies on a synthesized page-reference string of 10,000 references selected from a virtual space of 100 pages. To approximate the effects of the principle of locality, an exponential distribution for the probability of referencing a particular page was imposed. Finkel observes that some might be led to conclude that there is little point in elaborate page replacement algorithms when only a factor of 2 is at stake. But he notes that this difference will have a noticeable effect either on main memory requirements (to avoid degrading operating system performance) or operating system performance (to avoid enlarging main memory).

The clock algorithm has also been compared to these other algorithms when a variable allocation and either global or local replacement scope (see the following discussion of replacement policy) is used [CARR84]. The clock algorithm was found to approximate closely the performance of LRU.

The clock algorithm can be made more powerful by increasing the number of bits that it employs.[6] In all processors that support paging, a modify bit is associated with every page in main memory, and hence with every frame of main memory. This

[6]On the other hand, if we reduce the number of bits employed to zero, the clock algorithm degenerates to FIFO.

bit is needed so that when a page has been modified, it is not replaced until it has been written back into secondary memory. We can exploit this bit in the clock algorithm in the following way. If we take the use and modify bits into account, each frame falls into one of four categories:

1. Not accessed recently, not modified ($u = 0; m = 0$)
2. Accessed recently, not modified ($u = 1; m = 0$)
3. Not accessed recently, modified ($u = 0; m = 1$)
4. Accessed recently, modified ($u = 1; m = 1$)

With this classification, the clock algorithm performs as follows:

1. Beginning at the current position of the pointer, scan the frame buffer. During this scan, make no changes to the use bit. The first frame encountered with ($u = 0; m = 0$) is selected for replacement.
2. If step 1 fails, scan again, looking for the frame with ($u = 0; m = 1$). The first such frame encountered is selected for replacement. During this scan, set the use bit to 0 on each frame that is bypassed.
3. If step 2 fails, the pointer should have returned to its original position and all of the frames in the set will have a use bit of 0. Repeat step 1 and, if necessary, step 2. This time, a frame will be found for the replacement.

In summary, the page replacement algorithm cycles through all of the pages in the buffer, looking for one that has not been modified since being brought in and has not been accessed recently. Such a page is a good bet for replacement and has the advantage that, because it is unmodified, it does not need to be written back out to secondary memory. If no candidate page is found in the first sweep, the algorithm cycles through the buffer again, looking for a modified page that has not been accessed recently. Even though such a page must be written out to be replaced, because of the principle of locality, it may not be needed again anytime soon. If this second pass fails, all of the frames in the buffer are marked as having not been accessed recently and a third sweep is performed.

This strategy was used on an earlier version of the Macintosh virtual memory scheme [GOLD89]. The advantage of this algorithm over the simple clock algorithm is that pages that are unchanged are given preference for replacement. Because a page that has been modified must be written out before being replaced, there is an immediate saving of time.

***PAGE BUFFERING*** Although LRU and the clock policies are superior to FIFO, they both involve complexity and overhead not suffered with FIFO. In addition, there is the related issue that the cost of replacing a page that has been modified is greater than for one that has not, because the former must be written back out to secondary memory.

An interesting strategy that can improve paging performance and allow the use of a simpler page replacement policy is page buffering. The VAX VMS approach is representative. The page replacement algorithm is simple FIFO. To improve performance, a replaced page is not lost but rather is assigned to one of two lists: the free page list if the page has not been modified, or the modified

page list if it has. Note the page is not physically moved about in main memory; instead, the entry in the page table for this page is removed and placed in either the free or modified page list.

The free page list is a list of page frames available for reading in pages. VMS tries to keep some small number of frames free at all times. When a page is to be read in, the page frame at the head of the free page list is used, destroying the page that was there. When an unmodified page is to be replaced, it remains in memory and its page frame is added to the tail of the free page list. Similarly, when a modified page is to be written out and replaced, its page frame is added to the tail of the modified page list.

The important aspect of these maneuvers is that the page to be replaced remains in memory. Thus if the process references that page, it is returned to the resident set of that process at little cost. In effect, the free and modified page lists act as a cache of pages. The modified page list serves another useful function: Modified pages are written out in clusters rather than one at a time. This significantly reduces the number of I/O operations and therefore the amount of disk access time.

A simpler version of page buffering is implemented in the Mach operating system [RASH88]. In this case, no distinction is made between modified and unmodified pages.

***Replacement Policy and Cache Size*** As discussed earlier, main memory size is getting larger and the locality of applications is decreasing. In compensation, cache sizes have been increasing. Large cache sizes, even multimegabyte ones, are now feasible design alternatives [BORG90]. With a large cache, the replacement of virtual memory pages can have a performance impact. If the page frame selected for replacement is in the cache, then that cache block is lost as well as the page that it holds.

In systems that use some form of page buffering, it is possible to improve cache performance by supplementing the page replacement policy with a policy for page placement in the page buffer. Most operating systems place pages by selecting an arbitrary page frame from the page buffer; typically a first-in-first-out discipline is used. A study reported in [KESS92] shows that a careful page placement strategy can result in 10–20% fewer cache misses than naive placement.

Several page placement algorithms are examined in [KESS92]. The details are beyond the scope of this book, as they depend on the details of cache structure and policies. The essence of these strategies is to bring consecutive pages into main memory in such a way as to minimize the number of page frames that are mapped into the same cache slots.

## Resident Set Management

As was stated earlier in this chapter, the portion of a process that is actually in main memory at any time is defined to be the resident set of the process.

***Resident Set Size*** With paged virtual memory, it is not necessary (and indeed may not be possible) to bring all of the pages of a process into main memory to prepare it for execution. Thus, the OS must decide how many pages to bring in, that is, how much main memory to allocate to a particular process. Several factors come into play:

- The smaller the amount of memory allocated to a process, the more processes that can reside in main memory at any one time. This increases the probability

that the OS will find at least one ready process at any given time, and hence reduces the time lost due to swapping.

- If a relatively small number of pages of a process are in main memory, then, despite the principle of locality, the rate of page faults will be rather high (see Figure 8.10b).
- Beyond a certain size, additional allocation of main memory to a particular process will have no noticeable effect on the page fault rate for that process because of the principle of locality.

With these factors in mind, two sorts of policies are to be found in contemporary operating systems. A **fixed-allocation** policy gives a process a fixed number of frames in main memory within which to execute. That number is decided at initial load time (process creation time) and may be determined based on the type of process (interactive, batch, type of application) or may be based on guidance from the programmer or system manager. With a fixed-allocation policy, whenever a page fault occurs in the execution of a process, one of the pages of that process must be replaced by the needed page.

A **variable-allocation** policy allows the number of page frames allocated to a process to be varied over the lifetime of the process. Ideally, a process that is suffering persistently high levels of page faults (indicating that the principle of locality only holds in a weak form for that process) will be given additional page frames to reduce the page fault rate; whereas a process with an exceptionally low page fault rate (indicating that the process is quite well behaved from a locality point of view) will be given a reduced allocation, with the hope that this will not noticeably increase the page fault rate. The use of a variable-allocation policy relates to the concept of replacement scope, as explained in the next subsection.

The variable-allocation policy would appear to be the more powerful one. However, the difficulty with this approach is that it requires the OS to assess the behavior of active processes. This inevitably requires software overhead in the OS, and is dependent on hardware mechanisms provided by the processor platform.

***Replacement Scope*** The scope of a replacement strategy can be categorized as global or local. Both types of policies are activated by a page fault when there are no free page frames. A **local replacement policy** chooses only among the resident pages of the process that generated the page fault in selecting a page to replace. A **global replacement policy** considers all unlocked pages in main memory as candidates for replacement, regardless of which process owns a particular page. As mentioned earlier, when a frame is locked, the page currently stored in that frame may not be replaced. An unlocked page is simply a page in a frame of main memory that is not locked. While it happens that local policies are easier to analyze, there is no convincing evidence that they perform better than global policies, which are attractive because of their simplicity of implementation and minimal overhead [CARR84, MAEK87].

There is a correlation between replacement scope and resident set size (see Table 8.5). A fixed resident set implies a local replacement policy: To hold the size of a resident set fixed, a page that is removed from main memory must be replaced by another page from the same process. A variable-allocation policy can clearly employ a global replacement policy: The replacement of a page from one process in main memory

**Table 8.5** Resident Set Management

| | Local Replacement | Global Replacement |
|---|---|---|
| **Fixed Allocation** | • Number of frames allocated to a process is fixed.<br>• Page to be replaced is chosen from among the frames allocated to that process. | • Not possible. |
| **Variable Allocation** | • The number of frames allocated to a process may be changed from time to time to maintain the working set of the process.<br>• Page to be replaced is chosen from among the frames allocated to that process. | • Page to be replaced is chosen from all available frames in main memory; this causes the size of the resident set of processes to vary. |

with that of another causes the allocation of one process to grow by one page, and that of the other to shrink by one page. We shall also see that variable allocation and local replacement is a valid combination. We will now examine these three combinations.

***Fixed Allocation, Local Scope*** For this case, we have a process that is running in main memory with a fixed number of frames. When a page fault occurs, the OS must choose which page is to be replaced from among the currently resident pages for this process. Replacement algorithms such as those discussed in the preceding subsection can be used.

With a fixed-allocation policy, it is necessary to decide ahead of time the amount of allocation to give to a process. This could be decided on the basis of the type of application and the amount requested by the program. The drawback to this approach is twofold: If allocations tend to be too small, then there will be a high page fault rate, causing the entire multiprogramming system to run slowly. If allocations tend to be unnecessarily large, then there will be too few programs in main memory, and there will be either considerable processor idle time or considerable time spent in swapping.

***Variable Allocation, Global Scope*** This combination is perhaps the easiest to implement and has been adopted in a number of operating systems. At any given time, there are a number of processes in main memory, each with a certain number of frames allocated to it. Typically, the OS also maintains a list of free frames. When a page fault occurs, a free frame is added to the resident set of a process, and the page is brought in. Thus, a process experiencing page faults will gradually grow in size, which should help reduce overall page faults in the system.

The difficulty with this approach is in the replacement choice. When there are no free frames available, the OS must choose a page currently in memory to replace. The selection is made from among all of the frames in memory, except for locked frames such as those of the kernel. Using any of the policies discussed in the preceding subsection, the page selected for replacement can belong to any of the resident processes; there is no discipline to determine which process should lose a page from its resident set. Therefore, the process that suffers the reduction in resident set size may not be optimum.

One way to counter the potential performance problems of a variable-allocation, global-scope policy is to use page buffering. In this way, the choice of which page to replace becomes less significant, because the page may be reclaimed if it is referenced before the next time that a block of pages are overwritten.

***Variable Allocation, Local Scope*** The variable-allocation, local-scope strategy attempts to overcome the problems with a global-scope strategy. It can be summarized as follows:

1. When a new process is loaded into main memory, allocate to it a certain number of page frames as its resident set, based on application type, program request, or other criteria. Use either prepaging or demand paging to fill up the allocation.
2. When a page fault occurs, select the page to replace from among the resident set of the process that suffers the fault.
3. From time to time, reevaluate the allocation provided to the process, and increase or decrease it to improve overall performance.

With this strategy, the decision to increase or decrease a resident set size is a deliberate one, and is based on an assessment of the likely future demands of active processes. Because of this evaluation, such a strategy is more complex than a simple global replacement policy. However, it may yield better performance.

The key elements of the variable-allocation, local-scope strategy are the criteria used to determine resident set size and the timing of changes. One specific strategy that has received much attention in the literature is known as the **working set strategy**. Although a true working set strategy would be difficult to implement, it is useful to examine it as a baseline for comparison.

The working set is a concept introduced and popularized by Denning [DENN68, DENN70, DENN80b]; it has had a profound impact on virtual memory management design. The working set with parameter $\Delta$ for a process at virtual time $t$, which we designate as $W(t, \Delta)$, is the set of pages of that process that have been referenced in the last $\Delta$ virtual time units.

Virtual time is defined as follows. Consider a sequence of memory references, $r(1), r(2), \ldots$, in which $r(i)$ is the page that contains the $i$th virtual address generated by a given process. Time is measured in memory references; thus $t = 1, 2, 3, \ldots$ measures the process's internal virtual time.

Let us consider each of the two variables of W. The variable $\Delta$ is a window of virtual time over which the process is observed. The working set size will be a nondecreasing function of the window size. The result is illustrated in Figure 8.17 (based on [BACH86]), which shows a sequence of page references for a process. The dots indicate time units in which the working set does not change. Note that the larger the window size, the larger is the working set. This can be expressed in the following relationship:

$$W(t, \Delta + 1) \supseteq W(t, \Delta)$$

The working set is also a function of time. If a process executes over $\Delta$ time units and uses only a single page, then $|W(t, \Delta)| = 1$. A working set can also grow

| Sequence of Page References W | Window Size, Δ | | | |
|---|---|---|---|---|
| | 2 | 3 | 4 | 5 |
| 24 | 24 | 24 | 24 | 24 |
| 15 | 24 15 | 24 15 | 24 15 | 24 15 |
| 18 | 15 18 | 24 15 18 | 24 15 18 | 24 15 18 |
| 23 | 18 23 | 15 18 23 | 24 15 18 23 | 24 15 18 23 |
| 24 | 23 24 | 18 23 24 | • | • |
| 17 | 24 17 | 23 24 17 | 18 23 24 17 | 15 18 23 24 17 |
| 18 | 17 18 | 24 17 18 | • | 18 23 24 17 |
| 24 | 18 24 | • | 24 17 18 | • |
| 18 | • | 18 24 | • | 24 17 18 |
| 17 | 18 17 | 24 18 17 | • | • |
| 17 | 17 | 18 17 | • | • |
| 15 | 17 15 | 17 15 | 18 17 15 | 24 18 17 15 |
| 24 | 15 24 | 17 15 24 | 17 15 24 | • |
| 17 | 24 17 | • | • | 17 15 24 |
| 24 | • | 24 17 | • | • |
| 18 | 24 18 | 17 24 18 | 17 24 18 | 15 17 24 18 |

**Figure 8.17 Working Set of Process as Defined by Window Size**

as large as the number of pages $N$ of the process, if many different pages are rapidly addressed and if the window size allows. Thus,

$$1 \leq |W(t, \Delta)| \leq \min(\Delta, N)$$

Figure 8.18 indicates the way in which the working set size can vary over time for a fixed value of Δ. For many programs, periods of relatively stable working set sizes alternate with periods of rapid change. When a process first begins executing, it gradually builds up to a working set as it references new pages. Eventually, by the principle of locality, the process should stabilize on a certain set of pages. Subsequent transient periods reflect a shift of the program to a new locality. During the transition phase, some of the pages from the old locality remain within the window, Δ, causing a surge in the size of the working set as new pages are referenced. As the window slides past these page references, the working set size declines until it contains only those pages from the new locality.

This concept of a working set can be used to guide a strategy for resident set size:

1. Monitor the working set of each process.
2. Periodically remove from the resident set of a process those pages that are not in its working set. This is essentially an LRU policy.

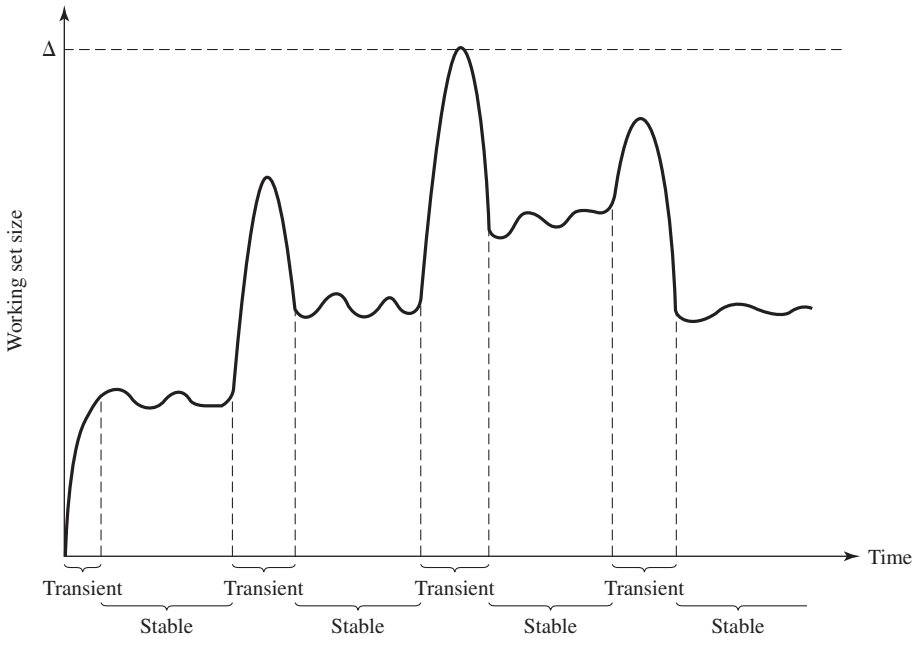



**Figure 8.18 Typical Graph of Working Set Size [MAEK87]**

3. A process may execute only if its working set is in main memory (i.e., if its resident set includes its working set).

This strategy is appealing because it takes an accepted principle, the principle of locality, and exploits it to achieve a memory management strategy that should minimize page faults. Unfortunately, there are a number of problems with the working set strategy:

1. The past does not always predict the future. Both the size and the membership of the working set will change over time (see Figure 8.18).
2. A true measurement of working set for each process is impractical. It would be necessary to time-stamp every page reference for every process using the virtual time of that process then maintain a time-ordered queue of pages for each process.
3. The optimal value of $\Delta$ is unknown and in any case would vary.

Nevertheless, the spirit of this strategy is valid, and a number of operating systems attempt to approximate a working set strategy. One way to do this is to focus not on the exact page references, but on the page fault rate of a process. As Figure 8.10b illustrates, the page fault rate falls as we increase the resident set size of a process. The working set size should fall at a point on this curve such as indicated by W in the figure. Therefore, rather than monitor the working set size directly, we can achieve comparable results by monitoring the page fault rate. The line of reasoning is as follows: If the page fault rate for a process is below some minimum threshold, the system

as a whole can benefit by assigning a smaller resident set size to this process (because more page frames are available for other processes) without harming the process (by causing it to incur increased page faults). If the page fault rate for a process is above some maximum threshold, the process can benefit from an increased resident set size (by incurring fewer faults) without degrading the system.

An algorithm that follows this strategy is the **page fault frequency (PFF)** algorithm [CHU72, GUPT78]. It requires a use bit to be associated with each page in memory. The bit is set to 1 when that page is accessed. When a page fault occurs, the OS notes the virtual time since the last page fault for that process; this could be done by maintaining a counter of page references. A threshold $F$ is defined. If the amount of time since the last page fault is less than $F$, then a page is added to the resident set of the process. Otherwise, discard all pages with a use bit of 0, and shrink the resident set accordingly. At the same time, reset the use bit on the remaining pages of the process to 0. The strategy can be refined by using two thresholds: an upper threshold that is used to trigger a growth in the resident set size, and a lower threshold that is used to trigger a contraction in the resident set size.

The time between page faults is the reciprocal of the page fault rate. Although it would seem to be better to maintain a running average of the page fault rate, the use of a single time measurement is a reasonable compromise that allows decisions about resident set size to be based on the page fault rate. If such a strategy is supplemented with page buffering, the resulting performance should be quite good.

Nevertheless, there is a major flaw in the PFF approach, which is that it does not perform well during the transient periods when there is a shift to a new locality. With PFF, no page ever drops out of the resident set before $F$ virtual time units have elapsed since it was last referenced. During interlocality transitions, the rapid succession of page faults causes the resident set of a process to swell before the pages of the old locality are expelled; the sudden peaks of memory demand may produce unnecessary process deactivations and reactivations, with the corresponding undesirable switching and swapping overheads.

An approach that attempts to deal with the phenomenon of interlocality transition, with a similar relatively low overhead to that of PFF, is the **variable-interval sampled working set (VSWS)** policy [FERR83]. The VSWS policy evaluates the working set of a process at sampling instances based on elapsed virtual time. At the beginning of a sampling interval, the use bits of all the resident pages for the process are reset; at the end, only the pages that have been referenced during the interval will have their use bit set; these pages are retained in the resident set of the process throughout the next interval, while the others are discarded. Thus the resident set size can only decrease at the end of an interval. During each interval, any faulted pages are added to the resident set; thus the resident set remains fixed or grows during the interval.

The VSWS policy is driven by three parameters:

$M$: The minimum duration of the sampling interval

$L$: The maximum duration of the sampling interval

$Q$: The number of page faults that are allowed to occur between sampling instances

The VSWS policy is as follows:

1. If the virtual time since the last sampling instance reaches $L$, then suspend the process and scan the use bits.
2. If, prior to an elapsed virtual time of $L$, $Q$ page faults occur,
   a. If the virtual time since the last sampling instance is less than $M$, then wait until the elapsed virtual time reaches $M$ to suspend the process and scan the use bits.
   b. If the virtual time since the last sampling instance is greater than or equal to $M$, suspend the process and scan the use bits.

The parameter values are to be selected so the sampling will normally be triggered by the occurrence of the $Q$th page fault after the last scan (case 2b). The other two parameters ($M$ and $L$) provide boundary protection for exceptional conditions. The VSWS policy tries to reduce the peak memory demands caused by abrupt interlocality transitions by increasing the sampling frequency, and hence the rate at which unused pages drop out of the resident set, when the page fault rate increases. Experience with this technique in the Bull mainframe operating system, GCOS 8, indicates that this approach is as simple to implement as PFF and more effective [PIZZ89].

## Cleaning Policy

A cleaning policy is the opposite of a fetch policy; it is concerned with determining when a modified page should be written out to secondary memory. Two common alternatives are demand cleaning and precleaning. With **demand cleaning**, a page is written out to secondary memory only when it has been selected for replacement. A **precleaning** policy writes modified pages before their page frames are needed so pages can be written out in batches.

Both precleaning and demand cleaning have drawbacks. With precleaning, a page is written out but remains in main memory until the page replacement algorithm dictates that it be removed. Precleaning allows the writing of pages in batches, but it makes little sense to write out hundreds or thousands of pages only to find that the majority of them have been modified again before they are replaced. The transfer capacity of secondary memory is limited, and should not be wasted with unnecessary cleaning operations.

On the other hand, with demand cleaning, the writing of a dirty page is coupled to, and precedes, the reading in of a new page. This technique may minimize page writes, but it means that a process that suffers a page fault may have to wait for two page transfers before it can be unblocked. This may decrease processor utilization.

A better approach incorporates page buffering. This allows the adoption of the following policy: Clean only pages that are replaceable, but decouple the cleaning and replacement operations. With page buffering, replaced pages can be placed on two lists: modified and unmodified. The pages on the modified list can periodically be written out in batches and moved to the unmodified list. A page on the unmodified list is either reclaimed if it is referenced or lost when its frame is assigned to another page.

## Load Control

Load control is concerned with determining the number of processes that will be resident in main memory, which has been referred to as the multiprogramming level. The load control policy is critical in effective memory management. If too few processes are resident at any one time, then there will be many occasions when all processes are blocked, and much time will be spent in swapping. On the other hand, if too many processes are resident, then, on average, the size of the resident set of each process will be inadequate and frequent faulting will occur. The result is thrashing.

***Multiprogramming Level*** Thrashing is illustrated in Figure 8.19. As the multiprogramming level increases from a small value, one would expect to see processor utilization rise, because there is less chance that all resident processes are blocked. However, a point is reached at which the average resident set is inadequate. At this point, the number of page faults rises dramatically, and processor utilization collapses.

There are a number of ways to approach this problem. A working set or PFF algorithm implicitly incorporates load control. Only those processes whose resident set is sufficiently large are allowed to execute. In providing the required resident set size for each active process, the policy automatically and dynamically determines the number of active programs.

Another approach, suggested by Denning and his colleagues [DENN80b], is known as the $L = S$ *criterion*, which adjusts the multiprogramming level so the mean time between faults equals the mean time required to process a page fault. Performance studies indicate this is the point at which processor utilization attained a maximum. A policy with a similar effect, proposed in [LERO76], is the *50% criterion*, which attempts to keep utilization of the paging device at approximately 50%. Again, performance studies indicate this is a point of maximum processor utilization.

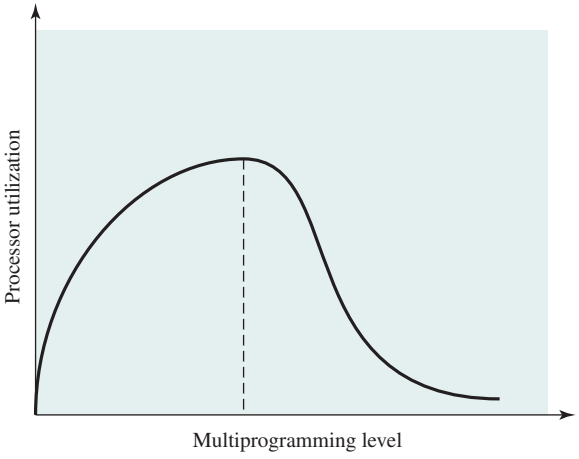



**Figure 8.19 Multiprogramming Effects**

Another approach is to adapt the clock page replacement algorithm described earlier (see Figure 8.15). [CARR84] describes a technique, using a global scope, that involves monitoring the rate at which the pointer scans the circular buffer of frames. If the rate is below a given lower threshold, this indicates one or both of two circumstances:

1. Few page faults are occurring, resulting in few requests to advance the pointer.
2. For each request, the average number of frames scanned by the pointer is small, indicating there are many resident pages not being referenced and are readily replaceable.

In both cases, the multiprogramming level can safely be increased. On the other hand, if the pointer scan rate exceeds an upper threshold, this indicates either a high fault rate or difficulty in locating replaceable pages, which implies that the multiprogramming level is too high.

***Process Suspension*** If the degree of multiprogramming is to be reduced, one or more of the currently resident processes must be suspended (swapped out). [CARR84] lists six possibilities:

- **Lowest-priority process:** This implements a scheduling policy decision, and is unrelated to performance issues.
- **Faulting process:** The reasoning is there is a greater probability that the faulting task does not have its working set resident, and performance would suffer least by suspending it. In addition, this choice has an immediate payoff because it blocks a process that is about to be blocked anyway, and it eliminates the overhead of a page replacement and I/O operation.
- **Last process activated:** This is the process least likely to have its working set resident.
- **Process with the smallest resident set:** This will require the least future effort to reload. However, it penalizes programs with strong locality.
- **Largest process:** This obtains the most free frames in an overcommitted memory, making additional deactivations unlikely soon.
- **Process with the largest remaining execution window:** In most process scheduling schemes, a process may only run for a certain quantum of time before being interrupted and placed at the end of the Ready queue. This approximates a shortest-processing-time-first scheduling discipline.

As in so many other areas of OS design, which policy to choose is a matter of judgment and depends on many other design factors in the OS, as well as the characteristics of the programs being executed.

## 8.3 UNIX AND SOLARIS MEMORY MANAGEMENT

Because UNIX is intended to be machine independent, its memory management scheme will vary from one system to the next. Earlier versions of UNIX simply used variable partitioning with no virtual memory scheme. Current implementations of UNIX and Solaris make use of paged virtual memory.

In SVR4 and Solaris, there are actually two separate memory management schemes. The **paging system** provides a virtual memory capability that allocates page frames in main memory to processes and also allocates page frames to disk block buffers. Although this is an effective memory management scheme for user processes and disk I/O, a paged virtual memory scheme is less suited to managing the memory allocation for the kernel. For this latter purpose, a **kernel memory allocator** is used. We will examine these two mechanisms in turn.

## Paging System

***DATA STRUCTURES*** For paged virtual memory, UNIX makes use of a number of data structures that, with minor adjustment, are machine independent (see Figure 8.20 and Table 8.6):

- **Page table:** Typically, there will be one page table per process, with one entry for each page in virtual memory for that process.
- **Disk block descriptor:** Associated with each page of a process is an entry in this table that describes the disk copy of the virtual page.
- **Page frame data table:** Describes each frame of real memory and is indexed by frame number. This table is used by the replacement algorithm.
- **Swap-use table:** There is one swap-use table for each swap device, with one entry for each page on the device.

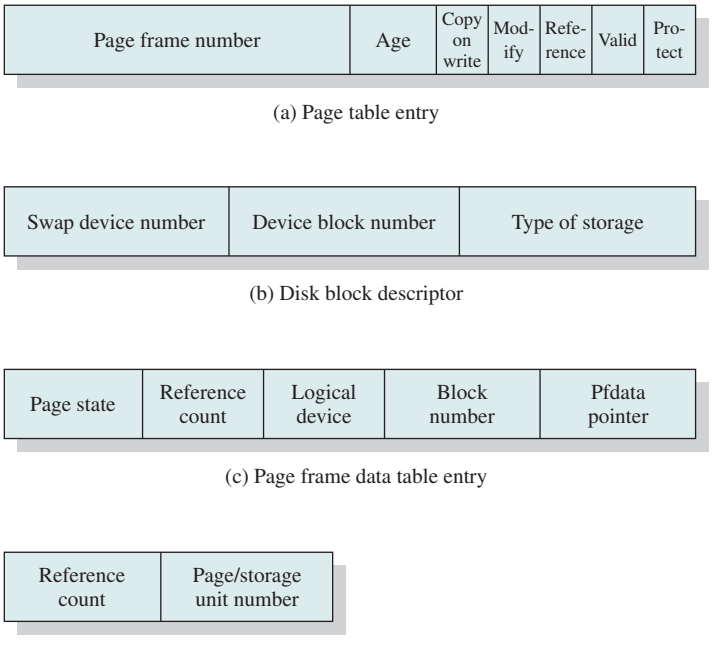



**Figure 8.20 UNIX SVR4 Memory Management Formats**

**Table 8.6** UNIX SVR4 Memory Management Parameters

| |
|---|
| **Page Table Entry** |
| **Page frame number**<br>Refers to frame in real memory. |
| **Age**<br>Indicates how long the page has been in memory without being referenced. The length and contents of this field are processor dependent. |
| **Copy on write**<br>Set when more than one process shares a page. If one of the processes writes into the page, a separate copy of the page must first be made for all other processes that share the page. This feature allows the copy operation to be deferred until necessary and avoided in cases where it turns out not to be necessary. |
| **Modify**<br>Indicates page has been modified. |
| **Reference**<br>Indicates page has been referenced. This bit is set to 0 when the page is first loaded, and may be periodically reset by the page replacement algorithm. |
| **Valid**<br>Indicates page is in main memory. |
| **Protect**<br>Indicates whether write operation is allowed. |
| **Disk Block Descriptor** |
| **Swap device number**<br>Logical device number of the secondary device that holds the corresponding page. This allows more than one device to be used for swapping. |
| **Device block number**<br>Block location of page on swap device. |
| **Type of storage**<br>Storage may be swap unit or executable file. In the latter case, there is an indication as to whether or not the virtual memory to be allocated should be cleared first. |
| **Page Frame Data Table Entry** |
| **Page state**<br>Indicates whether this frame is available or has an associated page. In the latter case, the status of the page is specified: on swap device, in executable file, or DMA in progress. |
| **Reference count**<br>Number of processes that reference the page. |
| **Logical device**<br>Logical device that contains a copy of the page. |
| **Block number**<br>Block location of the page copy on the logical device. |
| **Pfdata pointer**<br>Pointer to other pfdata table entries on a list of free pages and on a hash queue of pages. |
| **Swap-Use Table Entry** |
| **Reference count**<br>Number of page table entries that point to a page on the swap device. |
| **Page/storage unit number**<br>Page identifier on storage unit. |

Most of the fields defined in Table 8.6 are self-explanatory. A few warrant further comment. The Age field in the page table entry is an indication of how long it has been since a program referenced this frame. However, the number of bits and the frequency of update of this field are implementation dependent. Therefore, there is no universal UNIX use of this field for page replacement policy.

The Type of Storage field in the disk block descriptor is needed for the following reason: When an executable file is first used to create a new process, only a portion of the program and data for that file may be loaded into real memory. Later, as page faults occur, new portions of the program and data are loaded. It is only at the time of first loading that virtual memory pages are created and assigned to locations on one of the devices to be used for swapping. At that time, the OS is told whether it needs to clear (set to 0) the locations in the page frame before the first loading of a block of the program or data.

***PAGE REPLACEMENT*** The page frame data table is used for page replacement. Several pointers are used to create lists within this table. All of the available frames are linked together in a list of free frames available for bringing in pages. When the number of available frames drops below a certain threshold, the kernel will steal a number of frames to compensate.

The page replacement algorithm used in SVR4 is a refinement of the clock policy algorithm (see Figure 8.15) known as the two-handed clock algorithm (see Figure 8.21). The algorithm uses the reference bit in the page table entry for each page

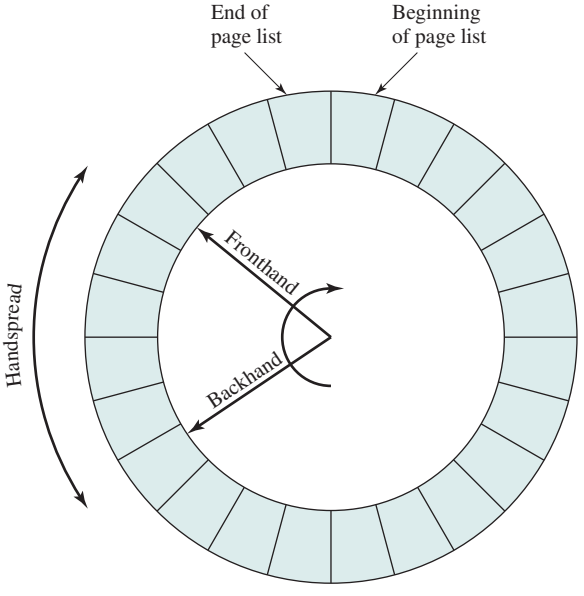



**Figure 8.21 Two-Handed Clock Page Replacement Algorithm**

in memory that is eligible (not locked) to be swapped out. This bit is set to 0 when the page is first brought in, and set to 1 when the page is referenced for a read or write. One hand in the clock algorithm, the fronthand, sweeps through the pages on the list of eligible pages and sets the reference bit to 0 on each page. Sometime later, the backhand sweeps through the same list and checks the reference bit. If the bit is set to 1, then that page has been referenced since the fronthand swept by; these frames are ignored. If the bit is still set to 0, then the page has not been referenced in the time interval between the visit by fronthand and backhand; these pages are placed on a list to be paged out.

Two parameters determine the operation of the algorithm:

1. **Scanrate:** The rate at which the two hands scan through the page list, in pages per second
2. **Handspread:** The gap between fronthand and backhand

These two parameters have default values set at boot time based on the amount of physical memory. The scanrate parameter can be altered to meet changing conditions. The parameter varies linearly between the values slowscan and fastscan (set at configuration time) as the amount of free memory varies between the values *lotsfree* and *minfree*. In other words, as the amount of free memory shrinks, the clock hands move more rapidly to free up more pages. The handspread parameter determines the gap between the fronthand and the backhand and therefore, together with scanrate, determines the window of opportunity to use a page before it is swapped out due to lack of use.

## Kernel Memory Allocator

The kernel generates and destroys small tables and buffers frequently during the course of execution, each of which requires dynamic memory allocation. [VAHA96] lists the following examples:

- The pathname translation routing may allocate a buffer to copy a pathname from user space.
- The `allocb()` routine allocates STREAMS buffers of arbitrary size.
- Many UNIX implementations allocate zombie structures to retain exit status and resource usage information about deceased processes.
- In SVR4 and Solaris, the kernel allocates many objects (such as proc structures, vnodes, and file descriptor blocks) dynamically when needed.

Most of these blocks are significantly smaller than the typical machine page size, and therefore the paging mechanism would be inefficient for dynamic kernel memory allocation. For SVR4, a modification of the buddy system, described in Section 7.2, is used.

In buddy systems, the cost to allocate and free a block of memory is low compared to that of best-fit or first-fit policies [KNUT97]. However, in the case of kernel memory management, the allocation and free operations must be made as fast as

possible. The drawback of the buddy system is the time required to fragment and coalesce blocks.

Barkley and Lee at AT&T proposed a variation known as a lazy buddy system [BARK89], and this is the technique adopted for SVR4. The authors observed that UNIX often exhibits steady-state behavior in kernel memory demand; that is, the amount of demand for blocks of a particular size varies slowly in time. Therefore, if a block of size $2^i$ is released and is immediately coalesced with its buddy into a block of size $2^{i+1}$, the kernel may next request a block of size $2^i$, which may necessitate splitting the larger block again. To avoid this unnecessary coalescing and splitting, the lazy buddy system defers coalescing until it seems likely that it is needed, then coalesces as many blocks as possible.

The lazy buddy system uses the following parameters:

$N_i$ = current number of blocks of size $2^i$.

$A_i$ = current number of blocks of size $2^i$ that are allocated (occupied).

$G_i$ = current number of blocks of size $2^i$ that are globally free; these are blocks that are eligible for coalescing; if the buddy of such a block becomes globally free, then the two blocks will be coalesced into a globally free block of size $2^{i+1}$. All free blocks (holes) in the standard buddy system could be considered globally free.

$L_i$ = current number of blocks of size $2^i$ that are locally free; these are blocks that are not eligible for coalescing. Even if the buddy of such a block becomes free, the two blocks are not coalesced. Rather, the locally free blocks are retained in anticipation of future demand for a block of that size.

The following relationship holds:

$$N_i = A_i + G_i + L_i$$

In general, the lazy buddy system tries to maintain a pool of locally free blocks and only invokes coalescing if the number of locally free blocks exceeds a threshold. If there are too many locally free blocks, then there is a chance that there will be a lack of free blocks at the next level to satisfy demand. Most of the time, when a block is freed, coalescing does not occur, so there is minimal bookkeeping and operational costs. When a block is to be allocated, no distinction is made between locally and globally free blocks; again, this minimizes bookkeeping.

The criterion used for coalescing is that the number of locally free blocks of a given size should not exceed the number of allocated blocks of that size (i.e., we must have $L_i \leq A_i$). This is a reasonable guideline for restricting the growth of locally free blocks, and experiments in [BARK89] confirm that this scheme results in noticeable savings.

To implement the scheme, the authors define a delay variable as follows:

$$D_i = A_i - L_i = N_i - 2L_i - G_i$$

Figure 8.22 shows the algorithm.

Initial value of $D_i$ is 0.
After an operation, the value of $D_i$ is updated as follows:

**(I)** if the next operation is a block allocate request:
if there is any free block, select one to allocate
if the selected block is locally free
then $D_i := D_i + 2$
else $D_i := D_i + 1$

otherwise
first get two blocks by splitting a larger one into two (recursive operation) allocate one and mark the other locally free
$D_i$ remains unchanged (but $D$ may change for other block sizes because of the recursive call)

**(II)** if the next operation is a block free request
Case $D_i > 2$
mark it locally free and free it locally
$D_i = 2$
Case $D_i = 1$
mark it globally free and free it globally; coalesce if possible
$D_i = 0$
Case $D_i = 0$
mark it globally free and free it globally; coalesce if possible
select one locally free block of size $2_i$ and free it globally; coalesce if possible
$D_i := 0$

**Figure 8.22** **Lazy Buddy System Algorithm**

## 8.4 LINUX MEMORY MANAGEMENT

Linux shares many of the characteristics of the memory management schemes of other UNIX implementations but has its own unique features. Overall, the Linux memory management scheme is quite complex [DUBE98]. In this section, we will give a brief overview of the two main aspects of Linux memory management: process virtual memory and kernel memory allocation. The basic unit of memory is a physical page, which is represented in the Linux kernel by struct page. The size of this page depends on the architecture; typically it is 4kB . Linux also supports Hugepages, which enables one to set larger sizes for pages (for example, 2MB). There are several projects which use Hugepages in order to improve performance. For example, Data Plane Development Kit (http://dpdk.org/) uses Hugepages for packet buffers, and this decreases the number of Translation Lookaside Buffers accesses on the system, comparing to when using the 4kB page size.

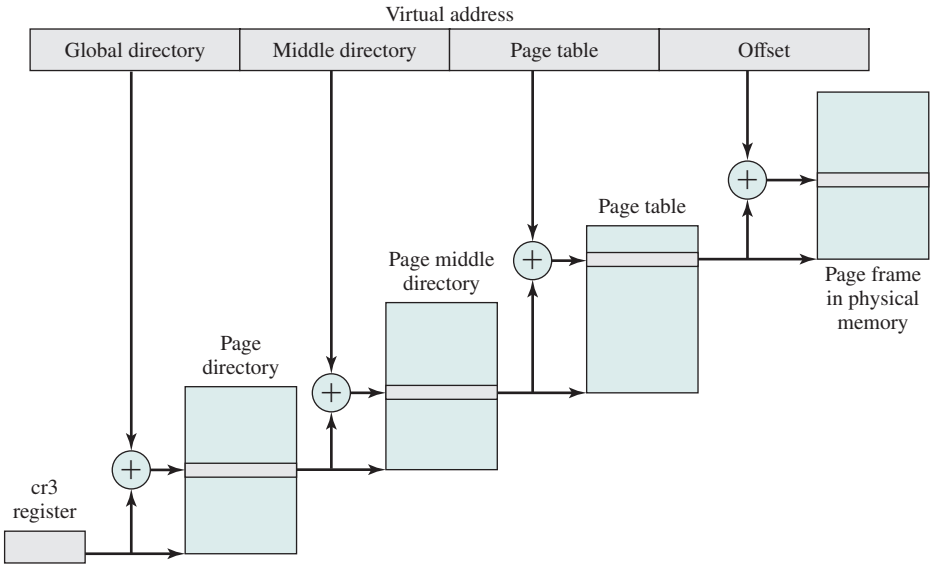



**Figure 8.23 Address Translation in Linux Virtual Memory Scheme**

## Linux Virtual Memory

***VIRTUAL MEMORY ADDRESSING*** Linux makes use of a three-level page table structure, consisting of the following types of tables (each individual table is the size of one page):

- **Page directory:** An active process has a single page directory that is the size of one page. Each entry in the page directory points to one page of the page middle directory. The page directory must be in main memory for an active process.
- **Page middle directory:** The page middle directory may span multiple pages. Each entry in the page middle directory points to one page in the page table.
- **Page table:** The page table may also span multiple pages. Each page table entry refers to one virtual page of the process.

To use this three-level page table structure, a virtual address in Linux is viewed as consisting of four fields (see Figure 8.23). The leftmost (most significant) field is used as an index into the page directory. The next field serves as an index into the page middle directory. The third field serves as an index into the page table. The fourth field gives the offset within the selected page of memory.

The Linux page table structure is platform independent and was designed to accommodate the 64-bit Alpha processor, which provides hardware support for three levels of paging. With 64-bit addresses, the use of only two levels of pages on the Alpha would result in very large page tables and directories. The 32-bit x86 architecture has a two-level hardware paging mechanism. The Linux software accommodates the two-level scheme by defining the size of the page middle directory as one. Note all references to an extra level of indirection are optimized away at compile time, not at run time. Therefore, there is no performance overhead for using generic three-level design on platforms which support only two levels in hardware.

***Page Allocation*** To enhance the efficiency of reading in and writing out pages to and from main memory, Linux defines a mechanism for dealing with contiguous blocks of pages mapped into contiguous blocks of page frames. For this purpose, the buddy system is used. The kernel maintains a list of contiguous page frame groups of fixed size; a group may consist of 1, 2, 4, 8, 16, or 32 page frames. As pages are allocated and deallocated in main memory, the available groups are split and merged using the buddy algorithm.

***Page Replacement Algorithm*** Prior to Linux release 2.6.28, the Linux page replacement algorithm was based on the clock algorithm described in Section 8.2 (see Figure 8.15). In the simple clock algorithm, a use bit and a modify bit are associated with each page in main memory. In the Linux scheme, the use bit was replaced with an 8-bit age variable. Each time that a page is accessed, the age variable is incremented. In the background, Linux periodically sweeps through the global page pool and decrements the age variable for each page as it rotates through all the pages in main memory. A page with an age of 0 is an "old" page that has not been referenced in some time and is the best candidate for replacement. The larger the value of age, the more frequently a page has been used in recent times and the less eligible it is for replacement. Thus, the Linux algorithm was a form of least frequently used policy.

Beginning with Linux release 2.6.28, the page replacement algorithm described in the preceding paragraph was scrapped and a new algorithm, referred to as a split LRU algorithm, was merged into the kernel. One problem with the older algorithm is that the periodic sweeps through the page pool consumes increasing amounts of processor time for increasingly large memories.

The new algorithm makes use of two flags added to each page table entry: PG_active and PG_referenced. The entire physical memory is divided into different "zones" in Linux based on their address. Two linked lists, namely the active and inactive lists, are used in each zone for page reclamation by the memory manager. A kernel daemon `kswapd` runs in the background periodically to perform periodic page reclamation in each zone. This daemon sweeps through the page table entries to which the system page frames are mapped. For all page table entries marked as accessed, PG_referenced bit is set. This bit is set by the processor the first time a page is accessed. For each iteration of kswapd, it checks whether the page accessed bit is set in the page table entry. Every time it reads the page accessed bit, `kswapd` clears the bit. We can summarize the steps involved in page management as follows (see Figure 8.24):

1. The first time a page on the inactive list is accessed, the PG_referenced flag is set.
2. The next time that page is accessed, it is moved to the active list. That is, it takes two accesses for a page to be declared active. More precisely, it takes two accesses in different scans for a page to become active.
3. If the second access doesn't happen soon enough, PG_referenced is reset.
4. Similarly, for active pages, two timeouts are required to move the page to the inactive list.

Pages on the inactive list are then available for page replacement, using an LRU type of algorithm.

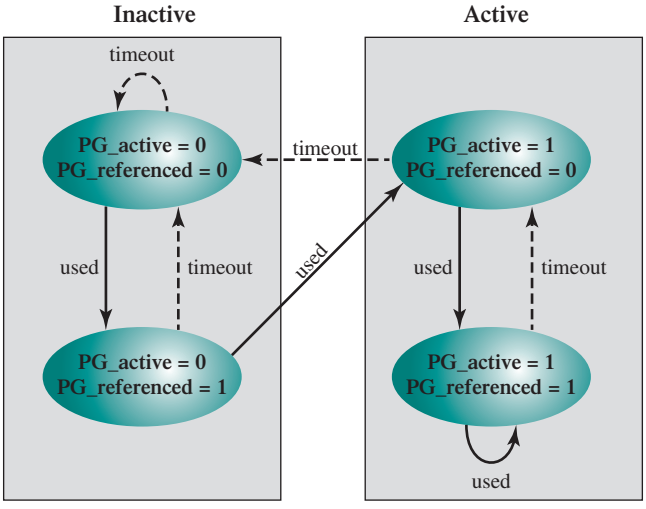



**Figure 8.24 Linux Page Reclaiming**

## Kernel Memory Allocation

The Linux kernel memory capability manages physical main memory page frames. Its primary function is to allocate and deallocate frames for particular uses. Possible owners of a frame include user-space processes (i.e., the frame is part of the virtual memory of a process that is currently resident in real memory), dynamically allocated kernel data, static kernel code, and the page cache.[7]

The foundation of kernel memory allocation for Linux is the page allocation mechanism used for user virtual memory management. As in the virtual memory scheme, a buddy algorithm is used so memory for the kernel can be allocated and deallocated in units of one or more pages. Because the minimum amount of memory that can be allocated in this fashion is one page, the page allocator alone would be inefficient because the kernel requires small short-term memory chunks in odd sizes. To accommodate these small chunks, Linux uses a scheme known as *slab allocation* [BONW94] within an allocated page. On a x86 machine, the page size is 4 kB, and chunks within a page may be allocated of sizes 32, 64, 128, 252, 508, 2,040, and 4,080 bytes.

The SLAB allocator is relatively complex and is not examined in detail here; a good description can be found in [VAHA96]. In essence, Linux maintains a set of linked lists, one for each size of chunk. Chunks may be split and aggregated in a manner similar to the buddy algorithm and moved between lists accordingly.

While SLAB is the most commonly used, there are three memory allocators in Linux for allocating small chunks of memory:

1. SLAB: Designed to be as cache-friendly as possible, minimizing cache misses.
2. SLUB (unqueued slab allocator): Designed to be simple and minimize instruction count [CORB07].

[7]The page cache has properties similar to a disk buffer, described in this chapter, as well as a disk cache, to be described in Chapter 11. We defer a discussion of the Linux page cache to Chapter 11.

3. SLOB (simple list of blocks): Designed to be as compact as possible; intended for systems with memory limitations [MACK05].

## 8.5 WINDOWS MEMORY MANAGEMENT

The Windows virtual memory manager controls how memory is allocated and how paging is performed. The memory manager is designed to operate over a variety of platforms and to use page sizes ranging from 4 kB to 64 kB. Intel and AMD64 platforms have 4 kB per page, and Intel Itanium platforms have 8 kB per page.

### Windows Virtual Address Map

On 32-bit platforms, each Windows user process sees a separate 32-bit address space, allowing 4 GB of virtual memory per process. By default, half of this memory is reserved for the OS, so each user actually has 2 GB of available virtual address space and all processes share most of the upper 2 GB of system space when running in kernel mode. Large memory intensive applications, on both clients and servers, can run more effectively using 64-bit Windows. Other than netbooks, most modern PCs use the AMD64 processor architecture which is capable of running as either a 32-bit or 64-bit system.

Figure 8.25 shows the default virtual address space seen by a normal 32-bit user process. It consists of four regions:

1. **0x00000000 to 0x0000FFFF:** Set aside to help programmers catch NULL-pointer assignments.
2. **0x00010000 to 0x7FFEFFFF:** Available user address space. This space is divided into pages that may be loaded into main memory.
3. **0x7FFF0000 to 0x7FFFFFFF:** A guard page inaccessible to the user. This page makes it easier for the OS to check on out-of-bounds pointer references.
4. **0x80000000 to 0xFFFFFFFF:** System address space. This 2-GB process is used for the Windows Executive, Kernel, HAL, and device drivers.

   On 64-bit platforms, 8 TB of user address space is available in Windows.

### Windows Paging

When a process is created, it can in principle make use of the entire user space of almost 2 GB (or 8 TB on 64-bit Windows). This space is divided into fixed-size pages, any of which can be brought into main memory, but the OS manages the addresses in contiguous regions allocated on 64-kB boundaries. A region can be in one of three states:

1. **Available:** addresses not currently used by this process.
2. **Reserved:** addresses that the virtual memory manager has set aside for a process so they cannot be allocated to another use (e.g., saving contiguous space for a stack to grow).

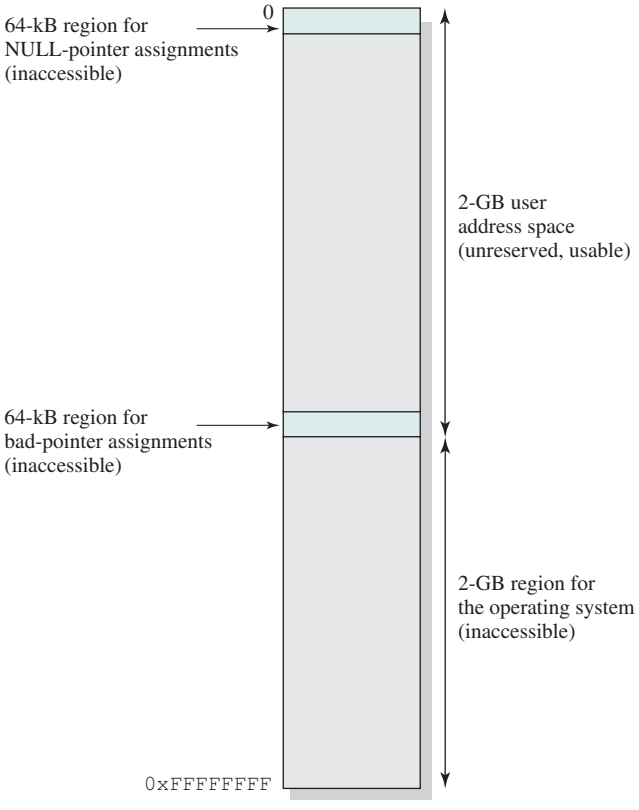



**Figure 8.25 Windows Default 32-Bit Virtual Address Space**

3. **Committed:** addresses that the virtual memory manager has initialized for use by the process to access virtual memory pages. These pages can reside either on disk or in physical memory. When on disk, they can be either kept in files (mapped pages) or occupy space in the paging file (i.e., the disk file to which it writes pages when removing them from main memory).

The distinction between reserved and committed memory is useful because it (1) reduces the amount of total virtual memory space needed by the system, allowing the page file to be smaller; and (2) allows programs to reserve addresses without making them accessible to the program or having them charged against their resource quotas.

The resident set management scheme used by Windows is variable allocation, local scope (see Table 8.5). When a process is first activated, it is assigned data structures to manage its working set. As the pages needed by the process are brought into physical memory, the memory manager uses the data structures to keep track of the pages assigned to the process. Working sets of active processes are adjusted using the following general conventions:

- When main memory is plentiful, the virtual memory manager allows the resident sets of active processes to grow. To do this, when a page fault occurs, a new

physical page is added to the process but no older page is swapped out, resulting in an increase of the resident set of that process by one page.

- When memory becomes scarce, the virtual memory manager recovers memory for the system by removing less recently used pages out of the working sets of active processes, reducing the size of those resident sets.
- Even when memory is plentiful, Windows watches for large processes that are rapidly increasing their memory usage. The system begins to remove pages that have not been recently used from the process. This policy makes the system more responsive because a new program will not suddenly cause a scarcity of memory and make the user wait while the system tries to reduce the resident sets of the processes that are already running.

### Windows Swapping

With the Metro UI comes a new virtual memory system to handle the interrupt requests from Windows Store apps. Swapfile.sys joins its familiar Windows counterpart pagefile.sys to provide access to temporary memory storage on the hard drive. Paging will hold items that haven't been accessed in a long time, whereas swapping holds items that were recently taken out of memory. The items in pagingfile may not be accessed again for a long time, whereas the items in swapfile might be accessed much sooner. Only Store apps use the swapfile.sys file, and because of the relatively small size of Store apps, the fixed size is only 256MB. The pagefile.sys file will be roughly one to two times the size of the amount of physical RAM found in the system. Swapfile.sys operates by swapping the entire process from system memory into the swapfile. This immediately frees up memory for other applications to use. By contrast, paging files function by moving "pages" of a program from system memory into the paging file. These pages are 4kB in size. The entire program does not get swapped wholesale into the paging file.

## 8.6 ANDROID MEMORY MANAGEMENT

Android includes a number of extensions to the normal Linux kernel memory management facility. These include the following:

- **ASHMem:** This feature provides anonymous shared memory, which abstracts memory as file descriptors. A file descriptor can be passed to another process to share memory.
- **ION:** ION is a memory pool manager and also enables its clients to share buffers. ION manages one or more memory pools, some of which are set aside at boot time to combat fragmentation or to serve special hardware needs. GPUs, display controllers, and cameras are some of the hardware blocks that may have special memory requirements. ION presents its memory pools as ION heaps. Each type of Android device can be provisioned with a different set of ION heaps according to the memory requirements of the device.
- **Low Memory Killer:** Most mobile devices do not have a swap capability (because of flash memory lifetime considerations). When main memory is

exhausted, the application or applications using the most memory must either back off their use of memory or be terminated. This feature enables the system to notify an app or apps that they need to free up memory. If an app does not cooperate, it is terminated.

## 8.7 SUMMARY

To use the processor and the I/O facilities efficiently, it is desirable to maintain as many processes in main memory as possible. In addition, it is desirable to free programmers from size restrictions in program development.

The way to address both of these concerns is virtual memory. With virtual memory, all address references are logical references that are translated at run time to real addresses. This allows a process to be located anywhere in main memory and for that location to change over time. Virtual memory also allows a process to be broken up into pieces. These pieces need not be contiguously located in main memory during execution and, indeed, it is not even necessary for all of the pieces of the process to be in main memory during execution.

Two basic approaches to providing virtual memory are paging and segmentation. With paging, each process is divided into relatively small, fixed-size pages. Segmentation provides for the use of pieces of varying size. It is also possible to combine segmentation and paging in a single memory management scheme.

A virtual memory management scheme requires both hardware and software support. The hardware support is provided by the processor. The support includes dynamic translation of virtual addresses to physical addresses and the generation of an interrupt when a referenced page or segment is not in main memory. Such an interrupt triggers the memory management software in the OS.

A number of design issues relate to OS support for memory management:

- **Fetch policy:** Process pages can be brought in on demand, or a prepaging policy can be used, which clusters the input activity by bringing in a number of pages at once.
- **Placement policy:** With a pure segmentation system, an incoming segment must be fit into an available space in memory.
- **Replacement policy:** When memory is full, a decision must be made as to which page or pages are to be replaced.
- **Resident set management:** The OS must decide how much main memory to allocate to a particular process when that process is swapped in. This can be a static allocation made at process creation time, or it can change dynamically.
- **Cleaning policy:** Modified process pages can be written out at the time of replacement, or a precleaning policy can be used, which clusters the output activity by writing out a number of pages at once.
- **Load control:** Load control is concerned with determining the number of processes that will be resident in main memory at any given time.

## 8.8 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

### Key Terms

| | | |
|---|---|---|
| associative mapping | page fault | segment table |
| demand paging | page placement policy | segmentation |
| external fragmentation | page replacement policy | slab allocation |
| fetch policy | page table | thrashing |
| frame | paging | translation lookaside buffer (TLB) |
| hash table | prepaging | virtual memory |
| hashing | real memory | working set |
| internal fragmentation | resident set | |
| locality | resident set management | |
| page | segment | |

### Review Questions

**8.1.** How does the use of virtual memory improve system utilization?
**8.2.** Explain thrashing.
**8.3.** Why is the principle of locality crucial to the use of virtual memory?
**8.4.** Which considerations determine the size of a page?
**8.5.** What is the purpose of a translation lookaside buffer?
**8.6.** What is demand paging?
**8.7.** What are the drawbacks of using either only a precleaning policy or only a demand cleaning policy?
**8.8.** What is the relationship between FIFO and clock page replacement algorithms?
**8.9.** How is a page fault trap dealt with?
**8.10.** Why is it not possible to combine a global replacement policy and a fixed allocation policy?
**8.11.** What is the difference between a resident set and a working set?
**8.12.** What is the difference between demand cleaning and precleaning?

### Problems

**8.1.** Suppose the page table for the process currently executing on the processor looks like the following. All numbers are decimal, everything is numbered starting from zero, and all addresses are memory byte addresses. The page size is 2,048 bytes.

| Virtual page number | Valid bit | Reference bit | Modify bit | Page frame number |
|---|---|---|---|---|
| 0 | 1 | 1 | 1 | 7 |
| 1 | 0 | 0 | 0 | – |
| 2 | 0 | 0 | 0 | – |
| 3 | 1 | 0 | 0 | 6 |
| 4 | 1 | 1 | 0 | 0 |
| 5 | 1 | 0 | 1 | 3 |

**a.** Describe exactly how, in general, a virtual address generated by the CPU is translated into a physical main memory address.
**b.** What physical address, if any, would each of the following virtual addresses correspond to? (Do not try to handle any page faults, if any.)
**(i)** 6,204
**(ii)** 3,021
**(iii)** 9,000

**8.2.** Consider the following program.

```
#define Size 64
 int  A[Size; Size], B[Size; Size], C[Size; Size];
 int register  i, j;
 for  (j = 0; j< Size; j ++)
   for  (i = 0; i< Size; i++)
C[i; j] = A[i; j] + B[i; j];
```

Assume the program is running on a system using demand paging, and the page size is 1 kB. Each integer is 4 bytes long. It is clear that each array requires a 16-page space. As an example, A[0, 0]-A[0, 63], A[1, 0]-A[1, 63], A[2, 0]-A[2, 63], and A[3, 0]-A[3, 63] will be stored in the first data page. A similar storage pattern can be derived for the rest of array A and for arrays B and C. Assume the system allocates a 4-page working set for this process. One of the pages will be used by the program, and three pages can be used for the data. Also, two index registers are assigned for i and j (so no memory accesses are needed for references to these two variables).

**a.** Discuss how frequently the page fault would occur (in terms of number of times C[i, j] = A[i, j] + B[i, j] are executed).
**b.** Can you modify the program to minimize the page fault frequency?
**c.** What will be the frequency of page faults after your modification?

**8.3.** **a.** How much memory space is needed for the user page table of Figure 8.3?
**b.** Assume you want to implement a hashed inverted page table for the same addressing scheme as depicted in Figure 8.3, using a hash function that maps the 24-bit page number into an 8-bit hash value. The table entry contains the page number, the frame number, and a chain pointer. If the page table allocates space for up to 4 overflow entries per hashed entry, how much memory space does the hashed inverted page table take?

**8.4.** Consider the following page-reference string: *a, b, d, c, b, e, d, b, d, b, a, c, b, c, a, c, f, a, f, d*. Assume that there are 3 frames available and that they are all initially empty. Complete a figure, similar to Figure 8.14, showing the frame allocation for each of the following page replacement policies:

**a.** First-in-first-out
**b.** Optimal
**c.** Least recently used

Then, find the relative performance of each policy with respect to page faults.

**8.5.** A process references five pages, A, B, C, D, and E, in the following order:

A; B; C; D; A; B; E; A; B; C; D; E

Assume the replacement algorithm is first-in-first-out and find the number of page transfers during this sequence of references starting with an empty main memory with three page frames. Repeat for four page frames.

**8.6.** A process contains eight virtual pages on disk and is assigned a fixed allocation of four page frames in main memory. The following page trace occurs:

1, 0, 2, 2, 1, 7, 6, 7, 0, 1, 2, 0, 3, 0, 4, 5, 1, 5, 2, 4, 5, 6, 7, 6, 7, 2, 4, 2, 7, 3, 3, 2, 3

**a.** Show the successive pages residing in the four frames using the LRU replacement policy. Compute the hit ratio in main memory. Assume the frames are initially empty.
**b.** Repeat part (a) for the FIFO replacement policy.
**c.** Compare the two hit ratios and comment on the effectiveness of using FIFO to approximate LRU with respect to this particular trace.

**8.7.** In the VAX, user page tables are located at virtual addresses in the system space. What is the advantage of having user page tables in virtual rather than main memory? What is the disadvantage?

**8.8.** A system has a total of 128 frames. There are 4 processes in the system with the following memory requirements:

$$p_1 : 45 \qquad p_2 : 75 \qquad p_3 : 33 \qquad p_4 : 135$$

Using the following allocation methods, compute the number of frames allocated to each of the processes stated above:

**a.** Equal Allocation Algorithm
**b.** Proportional Allocation Algorithm

**8.9.** The IBM System/370 architecture uses a two-level memory structure and refers to the two levels as segments and pages, although the segmentation approach lacks many of the features described earlier in this chapter. For the basic 370 architecture, the page size may be either 2 kB or 4 kB, and the segment size is fixed at either 64 kB or 1 MB. For the 370/XA and 370/ESA architectures, the page size is 4 kB and the segment size is 1 MB. Which advantages of segmentation does this scheme lack? What is the benefit of segmentation for the 370?

**8.10.** Suppose the virtual space accessed by memory is 6 GB, the page size is 8 KB, and each page table entry is 6 bytes. Compute the number of virtual pages that is implied. Also, compute the space required for the whole page table.

**8.11.** Consider a system with memory mapping done on a page basis and using a single level page table. Assume that the necessary page table is always in memory.

**a.** If a memory reference takes 250 ns, how long does a paged memory reference take?
**b.** Now we add an MMU that imposes an overhead of 30 ns on a hit or a miss. If we assume that 85% of all memory references hit in the MMU TLB, what is the Effective Memory Access Time (EMAT)?
**c.** Explain how the TLB hit rate affects the EMAT.

**8.12.** Consider a page reference string for a process with a working set of four frames, initially all empty. The page reference string is of length 20 with six distinct page numbers in it. For any page replacement algorithm,

**a.** What is the lower bound on the number of page faults? Justify your answer.
**b.** What is the upper bound on the number of page faults? Justify your answer.

**8.13.** In discussing a page replacement algorithm, one author makes an analogy with a snowplow moving around a circular track. Snow is falling uniformly on the track, and a lone snowplow continually circles the track at constant speed. The snow that is plowed off the track disappears from the system.

**a.** For which of the page replacement algorithms discussed in Section 8.2 is this a useful analogy?
**b.** What does this analogy suggest about the behavior of the page replacement algorithm in question?

**8.14.** In the S/370 architecture, a storage key is a control field associated with each page-sized frame of real memory. Two bits of that key that are relevant for page replacement are the reference bit and the change bit. The reference bit is set to 1 when any address within the frame is accessed for read or write, and is set to 0 when a new page is loaded into the frame. The change bit is set to 1 when a write operation is performed on any

location within the frame. Suggest an approach for determining which page frames are least recently used, making use of only the reference bit.

**8.15.** Consider the following sequence of page references (each element in the sequence represents a page number):

1 2 3 4 5 2 1 3 3 2 3 4 5 4 5 1 1 3 2 5

Define the *mean working set size* after the $k$th reference as $s_k(\Delta) = \frac{1}{k}\sum_{t=1}^{k} |W(t, \Delta)|$ and define the *missing page probability* after the $k$th reference as $m_k(\Delta) = \frac{1}{k}\sum_{t=1}^{k} |F(t, \Delta)|$ where $F(t, \Delta) = 1$ if a page fault occurs at virtual time $t$ and 0 otherwise.

**a.** Draw a diagram similar to that of Figure 8.17 for the reference sequence just defined for the values $\Delta = 1, 2, 3, 4, 5, 6$.
**b.** Plot $s_{20}(\Delta)$ as a function of $\Delta$.
**c.** Plot $m_{20}(\Delta)$ as a function of $\Delta$.

**8.16.** A key to the performance of the VSWS resident set management policy is the value of $Q$. Experience has shown that with a fixed value of $Q$ for a process, there are considerable differences in page fault frequencies at various stages of execution. Furthermore, if a single value of $Q$ is used for different processes, dramatically different frequencies of page faults occur. These differences strongly indicate that a mechanism that would dynamically adjust the value of $Q$ during the lifetime of a process would improve the behavior of the algorithm. Suggest a simple mechanism for this purpose.

**8.17.** Assume a task is divided into four equal-sized segments, and the system builds an eight-entry page descriptor table for each segment. Thus, the system has a combination of segmentation and paging. Assume also the page size is 2 kB.

**a.** What is the maximum size of each segment?
**b.** What is the maximum logical address space for the task?
**c.** Assume an element in physical location 00021ABC is accessed by this task. What is the format of the logical address that the task generates for it? What is the maximum physical address space for the system?

**8.18.** Consider the following sequence of page references:

A, B, B, C, A, E, D, B, D, E, A, C, E, B, A, C, A, F, D, F

and consider that a working set strategy is used for page replacement. What will the contents of the working set at each stage be for the following?

**a.** Window Size = 2
**b.** Window Size = 3
**c.** Window Size = 4

**8.19.** The UNIX kernel will dynamically grow a process's stack in virtual memory as needed, but it will never try to shrink it. Consider the case in which a program calls a C subroutine that allocates a local array on the stack that consumes 10 K. The kernel will expand the stack segment to accommodate it. When the subroutine returns, the stack pointer is adjusted and this space could be released by the kernel, but it is not released. Explain why it would be possible to shrink the stack at this point, and why the UNIX kernel does not shrink it.

# PART 4 Scheduling

## CHAPTER 9

# UNIPROCESSOR SCHEDULING


**9.1 Types of Processor Scheduling**
- Long-Term Scheduling
- Medium-Term Scheduling
- Short-Term Scheduling

**9.2 Scheduling Algorithms**
- Short-Term Scheduling Criteria
- The Use of Priorities
- Alternative Scheduling Policies
- Performance Comparison
- Fair-Share Scheduling

**9.3 Traditional UNIX Scheduling**

**9.4 Summary**

**9.5 Key Terms, Review Questions, and Problems**

**Learning Objectives**

After studying this chapter, you should be able to:

- Explain the differences among long-, medium-, and short-term scheduling.
- Assess the performance of different scheduling policies.
- Understand the scheduling technique used in traditional UNIX.

In a multiprogramming system, multiple processes exist concurrently in main memory. Each process alternates between using a processor and waiting for some event to occur, such as the completion of an I/O operation. The processor or processors are kept busy by executing one process while the others processes wait.

The key to multiprogramming is scheduling. In fact, four types of scheduling are typically involved (see Table 9.1). One of these, I/O scheduling, will be more conveniently addressed in Chapter 11, where I/O is discussed. The remaining three types of scheduling, which are types of processor scheduling, will be addressed in this chapter and the next.

This chapter begins with an examination of the three types of processor scheduling, showing how they are related. We see that long-term scheduling and medium-term scheduling are driven primarily by performance concerns related to the degree of multiprogramming. These issues are dealt with to some extent in Chapter 3, and in more detail in Chapters 7 and 8. Thus, the remainder of this chapter concentrates on short-term scheduling and is limited to a consideration of scheduling on a uniprocessor system. Because the use of multiple processors adds additional complexity, it is best to focus on the uniprocessor case first, so the differences among scheduling algorithms can be clearly seen.

Section 9.2 looks at the various algorithms that may be used to make short-term scheduling decisions.

## 9.1 TYPES OF PROCESSOR SCHEDULING

The aim of processor scheduling is to assign processes to be executed by the processor or processors over time, in a way that meets system objectives, such as response time, throughput, and processor efficiency. In many systems, this scheduling activity is broken down into three separate functions: long-, medium-, and short-term scheduling. The names suggest the relative time scales with which these functions are performed.

**Table 9.1** Types of Scheduling

| | |
|---|---|
| **Long-term scheduling** | The decision to add to the pool of processes to be executed. |
| **Medium-term scheduling** | The decision to add to the number of processes that are partially or fully in main memory. |
| **Short-term scheduling** | The decision as to which available process will be executed by the processor. |
| **I/O scheduling** | The decision as to which process's pending I/O request shall be handled by an available I/O device. |

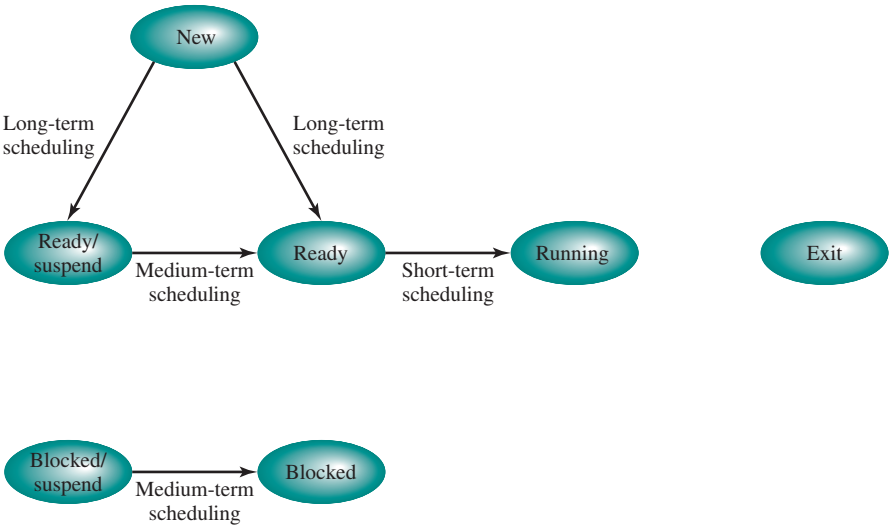



**Figure 9.1 Scheduling and Process State Transitions**

Figure 9.1 relates the scheduling functions to the process state transition diagram (first shown in Figure 3.9b). Long-term scheduling is performed when a new process is created. This is a decision whether to add a new process to the set of processes that are currently active. Medium-term scheduling is a part of the swapping function. This is a decision whether to add a process to those that are at least partially in main memory and therefore available for execution. Short-term scheduling is the actual decision of which ready process to execute next. Figure 9.2 reorganizes the state transition diagram of Figure 3.9b to suggest the nesting of scheduling functions.

Scheduling affects the performance of the system because it determines which processes will wait, and which will progress. This point of view is presented in Figure 9.3, which shows the queues involved in the state transitions of a process.[1] Fundamentally, scheduling is a matter of managing queues to minimize queueing delay and to optimize performance in a queueing environment.

## Long-Term Scheduling

The long-term scheduler determines which programs are admitted to the system for processing. Thus, it controls the degree of multiprogramming. Once admitted, a job or user program becomes a process and is added to the queue for the short-term scheduler. In some systems, a newly created process begins in a swapped-out condition, in which case it is added to a queue for the medium-term scheduler.

In a batch system, or for the batch portion of an OS, newly submitted jobs are routed to disk and held in a batch queue. The long-term scheduler creates processes from the queue when it can. There are two decisions involved. The scheduler must

[1]For simplicity, Figure 9.3 shows new processes going directly to the Ready state, whereas Figures 9.1 and 9.2 show the option of either the Ready state or the Ready/Suspend state.

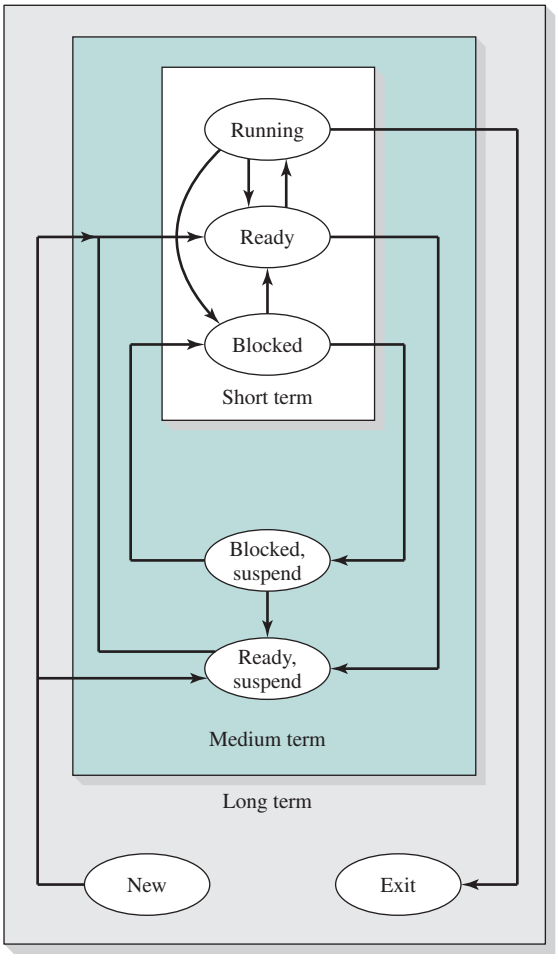



**Figure 9.2 Levels of Scheduling**

decide when the OS can take on one or more additional processes. And the scheduler must decide which job or jobs to accept and turn into processes. We briefly consider these two decisions.

The decision as to when to create a new process is generally driven by the desired degree of multiprogramming. The more processes that are created, the smaller is the percentage of time that each process can be executed (i.e., more processes are competing for the same amount of processor time). Thus, the long-term scheduler may limit the degree of multiprogramming to provide satisfactory service to the current set of processes. Each time a job terminates, the scheduler may decide to add one or more new jobs. Additionally, if the fraction of time that the processor is idle exceeds a certain threshold, the long-term scheduler may be invoked.

The decision as to which job to admit next can be on a simple first-come-first-served (FCFS) basis, or it can be a tool to manage system performance. The criteria used may include priority, expected execution time, and I/O requirements.

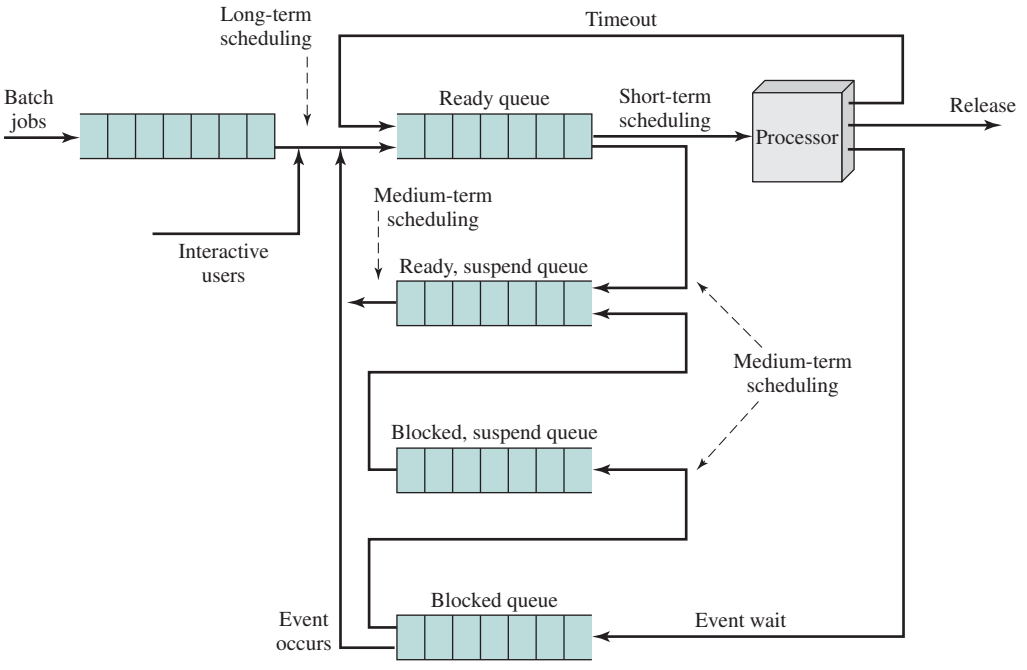



**Figure 9.3 Queueing Diagram for Scheduling**

For example, if the information is available, the scheduler may attempt to keep a mix of processor-bound and I/O-bound processes.[2] Also, the decision can depend on which I/O resources are to be requested, in an attempt to balance I/O usage.

For interactive programs in a time-sharing system, a process creation request can be generated by the act of a user attempting to connect to the system. Time-sharing users are not simply queued up and kept waiting until the system can accept them. Rather, the OS will accept all authorized users until the system is saturated, using some predefined measure of saturation. At that point, a connection request is met with a message indicating that the system is full and the user should try again later.

## Medium-Term Scheduling

Medium-term scheduling is part of the swapping function. The issues involved are discussed in Chapters 3, 7, and 8. Typically, the swapping-in decision is based on the need to manage the degree of multiprogramming. On a system that does not use virtual memory, memory management is also an issue. Thus, the swapping-in decision will consider the memory requirements of the swapped-out processes.

[2]A process is regarded as *processor bound* if it mainly performs computational work and occasionally uses I/O devices. A process is regarded as *I/O bound* if the time it takes to execute the process depends primarily on the time spent waiting for I/O operations.

### Short-Term Scheduling

In terms of frequency of execution, the long-term scheduler executes relatively infrequently and makes the coarse-grained decision of whether or not to take on a new process, and which one to take. The medium-term scheduler is executed somewhat more frequently to make a swapping decision. The short-term scheduler, also known as the dispatcher, executes most frequently and makes the fine-grained decision of which process to execute next.

The short-term scheduler is invoked whenever an event occurs that may lead to the blocking of the current process, or that may provide an opportunity to preempt a currently running process in favor of another. Examples of such events include:

- Clock interrupts
- I/O interrupts
- Operating system calls
- Signals (e.g., semaphores)

## 9.2 SCHEDULING ALGORITHMS

### Short-Term Scheduling Criteria

The main objective of short-term scheduling is to allocate processor time in such a way as to optimize one or more aspects of system behavior. Generally, a set of criteria is established against which various scheduling policies may be evaluated.

The commonly used criteria can be categorized along two dimensions. First, we can make a distinction between user-oriented and system-oriented criteria. User-oriented criteria relate to the behavior of the system as perceived by the individual user or process. An example is response time in an interactive system. Response time is the elapsed time between the submission of a request and when the response begins to appear as output. This quantity is visible to the user and is naturally of interest to the user. We would like a scheduling policy that provides "good" service to various users. In the case of response time, a threshold may be defined as, say, two seconds. Then a goal of the scheduling mechanism should be to maximize the number of users who experience an average response time of two seconds or less.

Other criteria are system oriented. That is, the focus is on effective and efficient utilization of the processor. An example is throughput, which is the rate at which processes are completed. This is certainly a worthwhile measure of system performance and one that we would like to maximize. However, it focuses on system performance rather than service provided to the user. Thus, throughput is of concern to a system administrator but not to the user population.

Whereas user-oriented criteria are important on virtually all systems, system-oriented criteria are generally of minor importance on single-user systems. On a single-user system, it probably is not important to achieve high processor utilization or high throughput as long as the responsiveness of the system to user applications is acceptable.

Another dimension along which criteria can be classified is those that are performance related, and those that are not directly performance related. Performance-related criteria are quantitative and generally can be readily measured. Examples include response time and throughput. Criteria that are not performance-related are either qualitative in nature or do not lend themselves readily to measurement and analysis. An example of such a criterion is predictability. We would like for the service provided to users to exhibit the same characteristics over time, independent of other work being performed by the system. To some extent, this criterion can be measured by calculating variances as a function of workload. However, this is not nearly as straightforward as measuring throughput or response time as a function of workload.

Table 9.2 summarizes key scheduling criteria. These are interdependent, and it is impossible to optimize all of them simultaneously. For example, providing good response time may require a scheduling algorithm that switches between processes

**Table 9.2** Scheduling Criteria

| |
|---|
| **User Oriented, Performance Related** |
| **Turnaround time** This is the interval of time between the submission of a process and its completion. Includes actual execution time plus time spent waiting for resources, including the processor. This is an appropriate measure for a batch job. |
| **Response time** For an interactive process, this is the time from the submission of a request until the response begins to be received. Often a process can begin producing some output to the user while continuing to process the request. Thus, this is a better measure than turnaround time from the user's point of view. The scheduling discipline should attempt to achieve low response time and to maximize the number of interactive users receiving acceptable response time. |
| **Deadlines** When process completion deadlines can be specified, the scheduling discipline should subordinate other goals to that of maximizing the percentage of deadlines met. |
| **User Oriented, Other** |
| **Predictability** A given job should run in about the same amount of time and at about the same cost regardless of the load on the system. A wide variation in response time or turnaround time is distracting to users. It may signal a wide swing in system workloads or the need for system tuning to cure instabilities. |
| **System Oriented, Performance Related** |
| **Throughput** The scheduling policy should attempt to maximize the number of processes completed per unit of time. This is a measure of how much work is being performed. This clearly depends on the average length of a process, but is also influenced by the scheduling policy, which may affect utilization. |
| **Processor utilization** This is the percentage of time that the processor is busy. For an expensive shared system, this is a significant criterion. In single-user systems and in some other systems, such as real-time systems, this criterion is less important than some of the others. |
| **System Oriented, Other** |
| **Fairness** In the absence of guidance from the user or other system-supplied guidance, processes should be treated the same, and no process should suffer starvation. |
| **Enforcing priorities** When processes are assigned priorities, the scheduling policy should favor higher-priority processes. |
| **Balancing resources** The scheduling policy should keep the resources of the system busy. Processes that will underutilize stressed resources should be favored. This criterion also involves medium-term and long-term scheduling. |

frequently. This increases the overhead of the system, reducing throughput. Thus, the design of a scheduling policy involves compromising among competing requirements; the relative weights given the various requirements will depend on the nature and intended use of the system.

In most interactive operating systems, whether single user or time shared, adequate response time is the critical requirement. Because of the importance of this requirement, and because the definition of adequacy will vary from one application to another, the topic is explored further in Appendix G.

## The Use of Priorities

In many systems, each process is assigned a priority, and the scheduler will always choose a process of higher priority over one of lower priority. Figure 9.4 illustrates the use of priorities. For clarity, the queueing diagram is simplified, ignoring the existence of multiple blocked queues and of suspended states (compare to Figure 3.8a). Instead of a single ready queue, we provide a set of queues, in descending order of priority: RQ0, RQ1, . . . , RQ$n$, with priority[RQ$i$] > priority[RQ$j$] for $i > j$.[3] When a scheduling selection is to be made, the scheduler will start at the highest-priority ready queue (RQ0). If there are one or more processes in the queue, a process is selected using some scheduling policy. If RQ0 is empty, then RQ1 is examined, and so on.

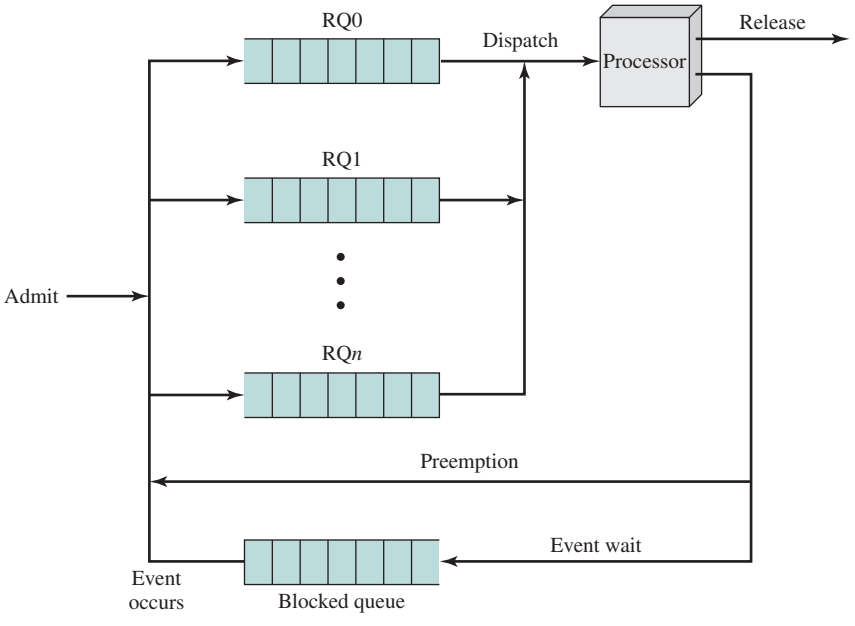



**Figure 9.4 Priority Queueing**

[3]In UNIX and many other systems, larger-priority values represent lower-priority processes; unless otherwise stated we follow that convention. Some systems, such as Windows, use the opposite convention: a higher number means a higher priority.

One problem with a pure priority scheduling scheme is that lower-priority processes may suffer starvation. This will happen if there is always a steady supply of higher-priority ready processes. If this behavior is not desirable, the priority of a process can change with its age or execution history. We will give one example of this subsequently.

## Alternative Scheduling Policies

Table 9.3 presents some summary information about the various scheduling policies that are examined in this subsection. The **selection function** determines which process, among ready processes, is selected next for execution. The function may be based on priority, resource requirements, or the execution characteristics of the process. In the latter case, three quantities are significant:

$w$ = time spent in system so far, waiting

$e$ = time spent in execution so far

$s$ = total service time required by the process, including $e$; generally, this quantity must be estimated or supplied by the user

For example, the selection function max[$w$] indicates an FCFS discipline.

**Table 9.3** Characteristics of Various Scheduling Policies

| | FCFS | Round Robin | SPN | SRT | HRRN | Feedback |
|---|---|---|---|---|---|---|
| **Selection Function** | max[$w$] | constant | min[$s$] | min[$s - e$] | $\max\left(\frac{w+s}{s}\right)$ | (see text) |
| **Decision Mode** | Non-preemptive | Preemptive (at time quantum) | Non-preemptive | Preemptive (at arrival) | Non-preemptive | Preemptive (at time quantum) |
| **Throughput** | Not emphasized | May be low if quantum is too small | High | High | High | Not emphasized |
| **Response Time** | May be high, especially if there is a large variance in process execution times | Provides good response time for short processes | Provides good response time for short processes | Provides good response time | Provides good response time | Not emphasized |
| **Overhead** | Minimum | Minimum | Can be high | Can be high | Can be high | Can be high |
| **Effect on Processes** | Penalizes short processes; penalizes I/O-bound processes | Fair treatment | Penalizes long processes | Penalizes long processes | Good balance | May favor I/O-bound processes |
| **Starvation** | No | No | Possible | Possible | No | Possible |

The **decision mode** specifies the instants in time at which the selection function is exercised. There are two general categories:

- **Nonpreemptive:** In this case, once a process is in the Running state, it continues to execute until (a) it terminates or (b) it blocks itself to wait for I/O or to request some OS service.
- **Preemptive:** The currently running process may be interrupted and moved to the Ready state by the OS. The decision to preempt may be performed when a new process arrives, when an interrupt occurs that places a blocked process in the Ready state, or periodically, based on a clock interrupt.

Preemptive policies incur greater overhead than nonpreemptive ones, but may provide better service to the total population of processes because they prevent any one process from monopolizing the processor for very long. In addition, the cost of preemption may be kept relatively low by using efficient process-switching mechanisms (as much help from hardware as possible) and by providing a large main memory to keep a high percentage of programs in main memory.

As we describe the various scheduling policies, we will use the set of processes in Table 9.4 as a running example. We can think of these as batch jobs, with the service time being the total execution time required. Alternatively, we can consider these to be ongoing processes that require alternate use of the processor and I/O in a repetitive fashion. In this latter case, the service times represent the processor time required in one cycle. In either case, in terms of a queueing model, this quantity corresponds to the service time.[4]

For the example of Table 9.4, Figure 9.5 shows the execution pattern for each policy for one cycle, and Table 9.5 summarizes some key results. First, the finish time of each process is determined. From this, we can determine the turnaround time. In terms of the queueing model, **turnaround time (TAT)** is the residence time $T_r$, or total time that the item spends in the system (waiting time plus service time). A more useful figure is the normalized turnaround time, which is the ratio of turnaround time to service time. This value indicates the relative delay experienced by a process. Typically, the longer the process execution time, the greater is the absolute amount of delay that can be tolerated. The minimum possible value for this ratio is 1.0; increasing values correspond to a decreasing level of service.

**Table 9.4** Process Scheduling Example

| Process | Arrival Time | Service Time |
|---|---|---|
| A | 0 | 3 |
| B | 2 | 6 |
| C | 4 | 4 |
| D | 6 | 5 |
| E | 8 | 2 |

[4]See Appendix H for a summary of queueing model terminology, and Chapter 20 for a more detailed discussion of queueing analysis.

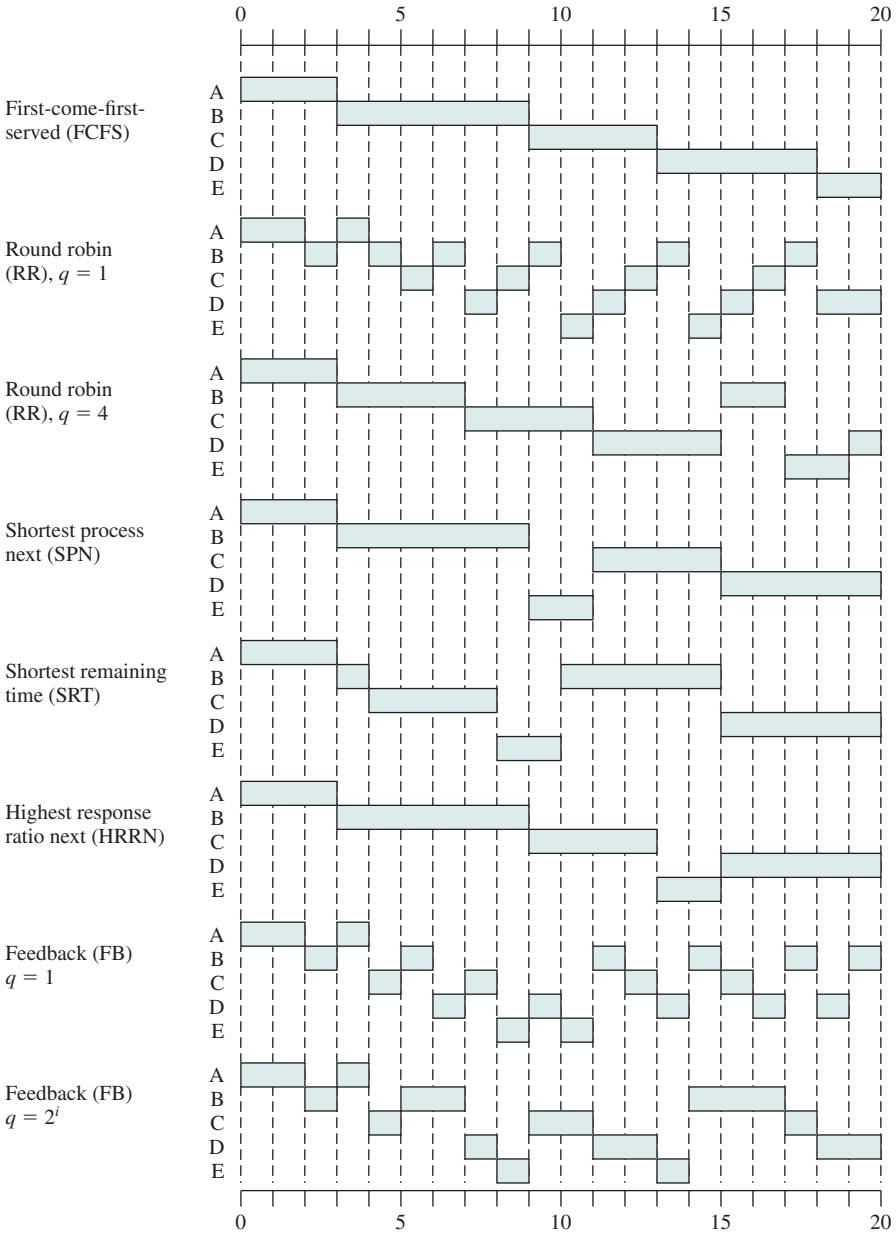



**Figure 9.5 A Comparison of Scheduling Policies**

***FIRST-COME-FIRST-SERVED*** The simplest scheduling policy is first-come-first-served (FCFS), also known as first-in, first-out (FIFO) or a strict queueing scheme. As each process becomes ready, it joins the ready queue. When the currently running process ceases to execute, the process that has been in the ready queue the longest is selected for running.

**Table 9.5** A Comparison of Scheduling Policies

| Process | A | B | C | D | E | |
|---|---|---|---|---|---|---|
| Arrival Time | 0 | 2 | 4 | 6 | 8 | |
| Service Time ($T_s$) | 3 | 6 | 4 | 5 | 2 | Mean |
| **FCFS** | | | | | | |
| Finish Time | 3 | 9 | 13 | 18 | 20 | |
| Turnaround Time ($T_r$) | 3 | 7 | 9 | 12 | 12 | 8.60 |
| $T_r/T_s$ | 1.00 | 1.17 | 2.25 | 2.40 | 6.00 | 2.56 |
| **RR $q = 1$** | | | | | | |
| Finish Time | 4 | 18 | 17 | 20 | 15 | |
| Turnaround Time ($T_r$) | 4 | 16 | 13 | 14 | 7 | 10.80 |
| $T_r/T_s$ | 1.33 | 2.67 | 3.25 | 2.80 | 3.50 | 2.71 |
| **RR $q = 4$** | | | | | | |
| Finish Time | 3 | 17 | 11 | 20 | 19 | |
| Turnaround Time ($T_r$) | 3 | 15 | 7 | 14 | 11 | 10.00 |
| $T_r/T_s$ | 1.00 | 2.5 | 1.75 | 2.80 | 5.50 | 2.71 |
| **SPN** | | | | | | |
| Finish Time | 3 | 9 | 15 | 20 | 11 | |
| Turnaround Time ($T_r$) | 3 | 7 | 11 | 14 | 3 | 7.60 |
| $T_r/T_s$ | 1.00 | 1.17 | 2.75 | 2.80 | 1.50 | 1.84 |
| **SRT** | | | | | | |
| Finish Time | 3 | 15 | 8 | 20 | 10 | |
| Turnaround Time ($T_r$) | 3 | 13 | 4 | 14 | 2 | 7.20 |
| $T_r/T_s$ | 1.00 | 2.17 | 1.00 | 2.80 | 1.00 | 1.59 |
| **HRRN** | | | | | | |
| Finish Time | 3 | 9 | 13 | 20 | 15 | |
| Turnaround Time ($T_r$) | 3 | 7 | 9 | 14 | 7 | 8.00 |
| $T_r/T_s$ | 1.00 | 1.17 | 2.25 | 2.80 | 3.5 | 2.14 |
| **FB $q = 1$** | | | | | | |
| Finish Time | 4 | 20 | 16 | 19 | 11 | |
| Turnaround Time ($T_r$) | 4 | 18 | 12 | 13 | 3 | 10.00 |
| $T_r/T_s$ | 1.33 | 3.00 | 3.00 | 2.60 | 1.5 | 2.29 |
| **FB $q = 2^i$** | | | | | | |
| Finish Time | 4 | 17 | 18 | 20 | 14 | |
| Turnaround Time ($T_r$) | 4 | 15 | 14 | 14 | 6 | 10.60 |
| $T_r/T_s$ | 1.33 | 2.50 | 3.50 | 2.80 | 3.00 | 2.63 |

FCFS performs much better for long processes than short ones. Consider the following example, based on one in [FINK88]:

| Process | Arrival Time | Service Time ($T_s$) | Start Time | Finish Time | Turnaround Time ($T_r$) | $T_r/T_s$ |
|---|---|---|---|---|---|---|
| **W** | 0 | 1 | 0 | 1 | 1 | 1 |
| **X** | 1 | 100 | 1 | 101 | 100 | 1 |
| **Y** | 2 | 1 | 101 | 102 | 100 | 100 |
| **Z** | 3 | 100 | 102 | 202 | 199 | 1.99 |
| **Mean** | | | | | 100 | 26 |

The normalized turnaround time for process Y is way out of line compared to the other processes: the total time that it is in the system is 100 times the required processing time. This will happen whenever a short process arrives just after a long process. On the other hand, even in this extreme example, long processes do not fare poorly. Process Z has a turnaround time that is almost double that of Y, but its normalized residence time is under 2.0.

Another difficulty with FCFS is that it tends to favor processor-bound processes over I/O-bound processes. Consider that there is a collection of processes, one of which mostly uses the processor (processor bound) and a number of which favor I/O (I/O bound). When a processor-bound process is running, all of the I/O-bound processes must wait. Some of these may be in I/O queues (blocked state) but may move back to the ready queue while the processor-bound process is executing. At this point, most or all of the I/O devices may be idle, even though there is potentially work for them to do. When the currently running process leaves the Running state, the ready I/O-bound processes quickly move through the Running state and become blocked on I/O events. If the processor-bound process is also blocked, the processor becomes idle. Thus, FCFS may result in inefficient use of both the processor and the I/O devices.

FCFS is not an attractive alternative on its own for a uniprocessor system. However, it is often combined with a priority scheme to provide an effective scheduler. Thus, the scheduler may maintain a number of queues, one for each priority level, and dispatch within each queue on a first-come-first-served basis. We see one example of such a system later, in our discussion of feedback scheduling.

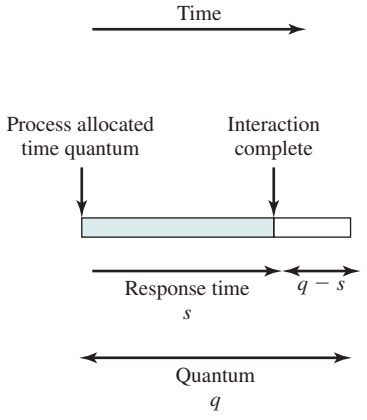
***Round Robin*** A straightforward way to reduce the penalty that short jobs suffer with FCFS is to use preemption based on a clock. The simplest such policy is round robin. A clock interrupt is generated at periodic intervals. When the interrupt occurs, the currently running process is placed in the ready queue, and the next ready job is selected on a FCFS basis. This technique is also known as **time slicing**, because each process is given a slice of time before being preempted.

With round robin, the principal design issue is the length of the time quantum, or slice, to be used. If the quantum is very short, then short processes will move through the system relatively quickly. On the other hand, there is processing overhead involved in handling the clock interrupt and performing the scheduling and

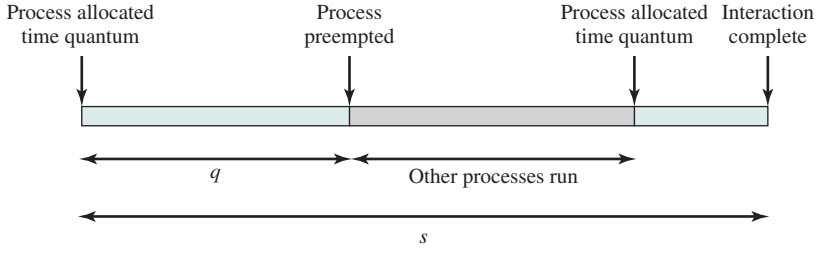
dispatching function. Thus, very short time quanta should be avoided. One useful guide is that the time quantum should be slightly greater than the time required for a typical interaction or process function. If it is less, then most processes will require at least two time quanta. Figure 9.6 illustrates the effect this has on response time. Note in the limiting case of a time quantum that is longer than the longest-running process, round robin degenerates to FCFS.

Figure 9.5 and Table 9.5 show the results for our example using time quanta $q$ of 1 and 4 time units. Note process E, which is the shortest job, enjoys significant improvement for a time quantum of 1.

Round robin is particularly effective in a general-purpose time-sharing system or transaction processing system. One drawback to round robin is its relative treatment of processor-bound and I/O-bound processes. Generally, an I/O-bound process has a shorter processor burst (amount of time spent executing between I/O operations) than a processor-bound process. If there is a mix of processor-bound and




(a) Time quantum greater than typical interaction




(b) Time quantum less than typical interaction

**Figure 9.6 Effect of Size of Preemption Time Quantum**

I/O-bound processes, then the following will happen: An I/O-bound process uses a processor for a short period, then is blocked for I/O; it waits for the I/O operation to complete then joins the ready queue. On the other hand, a processor-bound process generally uses a complete time quantum while executing and immediately returns to the ready queue. Thus, processor-bound processes tend to receive an unfair portion of processor time, which results in poor performance for I/O-bound processes, inefficient use of I/O devices, and an increase in the variance of response time.

[HALD91] suggests a refinement to round robin to which he refers as a virtual round robin (VRR) and that avoids this unfairness. Figure 9.7 illustrates the scheme. New processes arrive and join the ready queue, which is managed on an FCFS basis. When a running process times out, it is returned to the ready queue. When a process is blocked for I/O, it joins an I/O queue. So far, this is as usual. The new feature is an FCFS auxiliary queue to which processes are moved after being released from an I/O block. When a dispatching decision is to be made, processes in the auxiliary queue get preference over those in the main ready queue. When a process is dispatched from the auxiliary queue, it runs no longer than a time equal to the basic time quantum minus the total time spent running since it was last selected from the main ready queue. Performance studies by the authors indicate that this approach is indeed superior to round robin in terms of fairness.

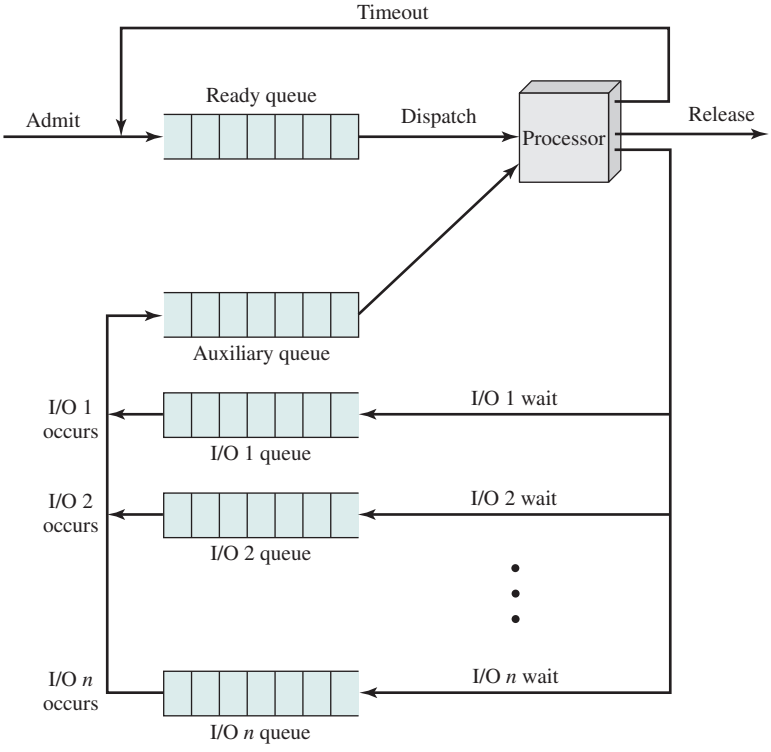



**Figure 9.7 Queueing Diagram for Virtual Round-Robin Scheduler**

***Shortest Process Next*** Another approach to reducing the bias in favor of long processes inherent in FCFS is the shortest process next (SPN) policy. This is a nonpreemptive policy in which the process with the shortest expected processing time is selected next. Thus, a short process will jump to the head of the queue past longer jobs.

Figure 9.5 and Table 9.5 show the results for our example. Note process E receives service much earlier than under FCFS. Overall performance is also significantly improved in terms of response time. However, the variability of response times is increased, especially for longer processes, and thus predictability is reduced.

One difficulty with the SPN policy is the need to know (or at least estimate) the required processing time of each process. For batch jobs, the system may require the programmer to estimate the value and supply it to the OS. If the programmer's estimate is substantially under the actual running time, the system may abort the job. In a production environment, the same jobs run frequently, and statistics may be gathered. For interactive processes, the OS may keep a running average of each "burst" for each process. The simplest calculation would be the following:

$$S_{n+1} = \frac{1}{n}\sum_{i=1}^{n} T_i \tag{9.1}$$

where

$T_i$ = processor execution time for the $i$th instance of this process (total execution time for batch job; processor burst time for interactive job),

$S_i$ = predicted value for the $i$th instance, and

$S_1$ = predicted value for first instance; not calculated.

To avoid recalculating the entire summation each time, we can rewrite Equation (9.1) as

$$S_{n+1} = \frac{1}{n}T_n + \frac{n-1}{n}S_n \tag{9.2}$$

Note each term in this summation is given equal weight; that is, each term is multiplied by the same constant $1/(n)$. Typically, we would like to give greater weight to more recent instances, because these are more likely to reflect future behavior. A common technique for predicting a future value on the basis of a time series of past values is **exponential averaging**:

$$S_{n+1} = \alpha T_n + (1 - \alpha)S_n \tag{9.3}$$

where $\alpha$ is a constant weighting factor ($0 < \alpha < 1$) that determines the relative weight given to more recent observations relative to older observations. Compare with Equation (9.2). By using a constant value of $\alpha$, independent of the number of past observations, Equation (9.3) considers all past values, but the less recent ones have less weight. To see this more clearly, consider the following expansion of Equation (9.3):

$$S_{n+1} = \alpha T_n + (1 - \alpha)\alpha T_{n-1} + \ \ldots \ + (1 - \alpha)^i \alpha T_{n-i} + \ \ldots \ + (1 - \alpha)^n S_1 \tag{9.4}$$

Because both $\alpha$ and $(1 - \alpha)$ are less than 1, each successive term in the preceding equation is smaller. For example, for $\alpha = 0.8$, Equation (9.4) becomes

$$S_{n+1} = 0.8T_n + 0.16T_{n-1} + 0.032T_{n-2} + 0.0064T_{n-3} + \ldots + (0.2)^n S_1$$

The older the observation, the less it is counted in to the average.

The size of the coefficient as a function of its position in the expansion is shown in Figure 9.8. The larger the value of $\alpha$, the greater is the weight given to the more recent observations. For $\alpha = 0.8$, virtually all of the weight is given to the four most recent observations, whereas for $\alpha = 0.2$, the averaging is effectively spread out over the eight or so most recent observations. The advantage of using a value of $\alpha$ close to 1 is that the average will quickly reflect a rapid change in the observed quantity. The disadvantage is that if there is a brief surge in the value of the observed quantity and it then settles back to some average value, the use of a large value of $\alpha$ will result in jerky changes in the average.

Figure 9.9 compares simple averaging with exponential averaging (for two different values of $\alpha$). In Figure 9.9a, the observed value begins at 1, grows gradually to a value of 10, then stays there. In Figure 9.9b, the observed value begins at 20, declines gradually to 10, then stays there. In both cases, we start out with an estimate of $S_1 = 0$. This gives greater priority to new processes. Note exponential averaging tracks changes in process behavior faster than does simple averaging and the larger value of $\alpha$ results in a more rapid reaction to the change in the observed value.

A risk with SPN is the possibility of starvation for longer processes, as long as there is a steady supply of shorter processes. On the other hand, although SPN reduces the bias in favor of longer jobs, it still is not desirable for a time-sharing or transaction-processing environment because of the lack of preemption. Looking back at our worst-case analysis described under FCFS, processes W, X, Y, and Z will still execute in the same order, heavily penalizing the short process Y.

***Shortest Remaining Time*** The shortest remaining time (SRT) policy is a preemptive version of SPN. In this case, the scheduler always chooses the process

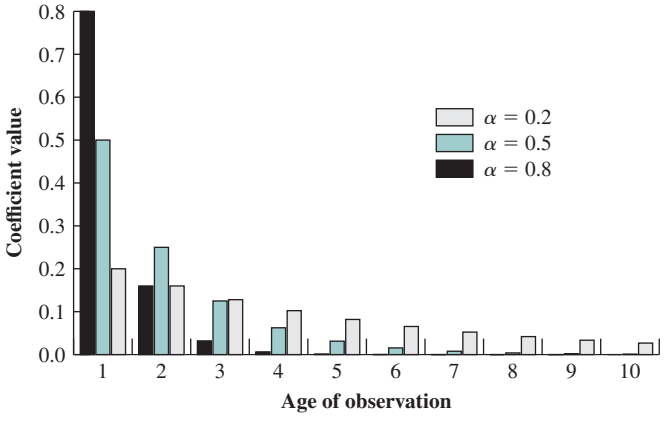
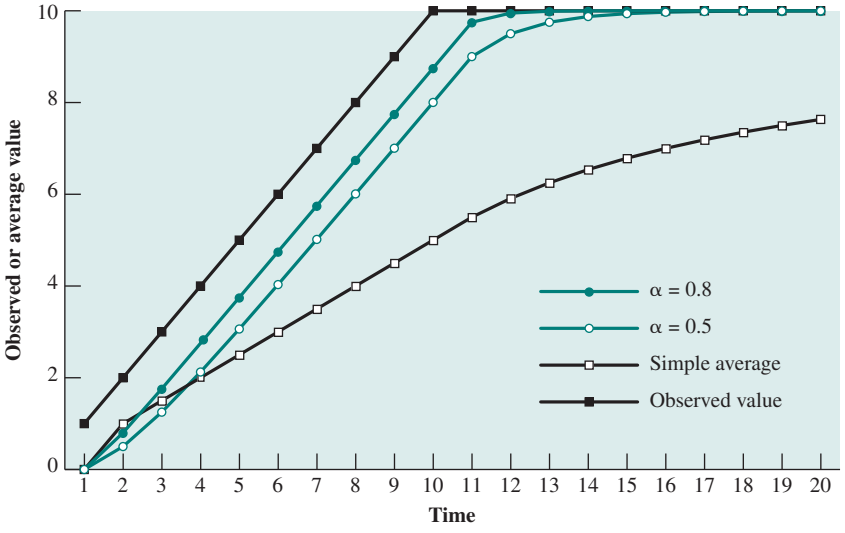
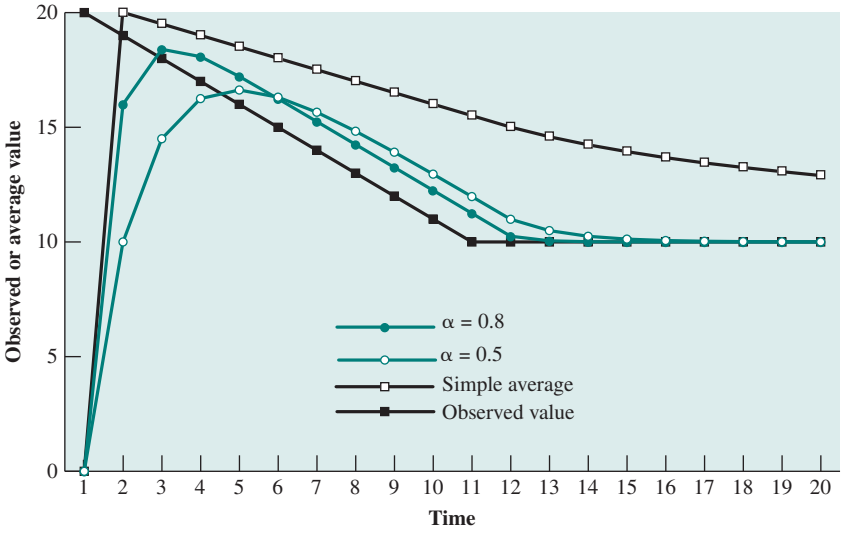



**Figure 9.8 Exponential Smoothing Coefficients**




(a) Increasing function




(b) Decreasing function

**Figure 9.9 Use of Exponential Averaging**

that has the shortest expected remaining processing time. When a new process joins the ready queue, it may in fact have a shorter remaining time than the currently running process. Accordingly, the scheduler may preempt the current process when a new process becomes ready. As with SPN, the scheduler must have an estimate of processing time to perform the selection function, and there is a risk of starvation of longer processes.

SRT does not have the bias in favor of long processes found in FCFS. Unlike round robin, no additional interrupts are generated, reducing overhead. On the other

hand, elapsed service times must be recorded, contributing to overhead. SRT should also give superior turnaround time performance to SPN, because a short job is given immediate preference to a running longer job.

Note in our example (see Table 9.5), the three shortest processes all receive immediate service, yielding a normalized turnaround time for each of 1.0.

***HIGHEST RESPONSE RATIO NEXT*** In Table 9.5, we have used the normalized turnaround time, which is the ratio of turnaround time to actual service time, as a figure of merit. For each individual process, we would like to minimize this ratio, and we would like to minimize the average value over all processes. In general, we cannot know ahead of time what the service time is going to be, but we can approximate it, either based on past history or some input from the user or a configuration manager. Consider the following ratio:

$$R = \frac{w + s}{s}$$

where

$R$ = response ratio,
$w$ = time spent waiting for the processor, and
$s$ = expected service time.

If the process with this value is dispatched immediately, $R$ is equal to the normalized turnaround time. Note the minimum value of $R$ is 1.0, which occurs when a process first enters the system.

Thus, our scheduling rule becomes the following: When the current process completes or is blocked, choose the ready process with the greatest value of $R$. This approach is attractive because it accounts for the age of the process. While shorter jobs are favored (a smaller denominator yields a larger ratio), aging without service increases the ratio so a longer process will eventually get past competing shorter jobs.

As with SRT and SPN, the expected service time must be estimated to use highest response ratio next (HRRN).

***FEEDBACK*** If we have no indication of the relative length of various processes, then none of SPN, SRT, and HRRN can be used. Another way of establishing a preference for shorter jobs is to penalize jobs that have been running longer. In other words, if we cannot focus on the time remaining to execute, let us focus on the time spent in execution so far.

The way to do this is as follows. Scheduling is done on a preemptive (at time quantum) basis, and a dynamic priority mechanism is used. When a process first enters the system, it is placed in RQ0 (see Figure 9.4). After its first preemption, when it returns to the Ready state, it is placed in RQ1. Each subsequent time that it is preempted, it is demoted to the next lower-priority queue. A short process will complete quickly, without migrating very far down the hierarchy of ready queues. A longer process will gradually drift downward. Thus, newer, shorter processes are favored over older, longer processes. Within each queue, except the lowest-priority queue, a simple FCFS mechanism is used. Once in the lowest-priority queue, a process

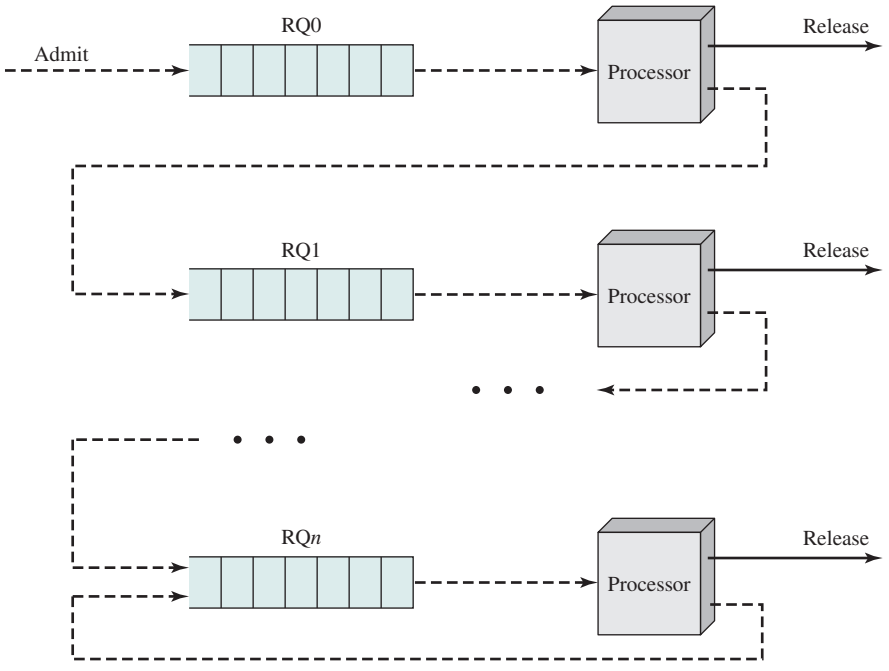



**Figure 9.10** **Feedback Scheduling**

cannot go lower, but is returned to this queue repeatedly until it completes execution. Thus, this queue is treated in round-robin fashion.

Figure 9.10 illustrates the feedback scheduling mechanism by showing the path that a process will follow through the various queues.[5] This approach is known as **multilevel feedback**, meaning the OS allocates the processor to a process and, when the process blocks or is preempted, feeds it back into one of several priority queues.

There are a number of variations on this scheme. A simple version is to perform preemption in the same fashion as for round robin: at periodic intervals. Our example shows this (see Figure 9.5 and Table 9.5) for a quantum of one time unit. Note that in this case, the behavior is similar to round robin with a time quantum of $q = 1$.

One problem with the simple scheme just outlined is that the turnaround time of longer processes can stretch out alarmingly. Indeed, it is possible for starvation to occur if new jobs are entering the system frequently. To compensate for this, we can vary the preemption times according to the queue: A process scheduled from RQ0 is allowed to execute for one time unit and is then preempted; a process scheduled from RQ1 is allowed to execute two time units, and so on. In general, a process scheduled from RQ$i$ is allowed to execute $q = 2^i$ time units before preemption. This scheme is illustrated for our example in Figure 9.5 and Table 9.5.

[5]Dotted lines are used to emphasize that this is a time sequence diagram rather than a static depiction of possible transitions, such as Figure 9.4.

Even with the allowance for greater time allocation at lower priority, a longer process may still suffer starvation. A possible remedy is to promote a process to a higher-priority queue after it spends a certain amount of time waiting for service in its current queue.

## Performance Comparison

Clearly, the performance of various scheduling policies is a critical factor in the choice of a scheduling policy. However, it is impossible to make definitive comparisons because relative performance will depend on a variety of factors, including the probability distribution of service times of the various processes, the efficiency of the scheduling and context switching mechanisms, and the nature of the I/O demand and the performance of the I/O subsystem. Nevertheless, we attempt in what follows to draw some general conclusions.

***Queueing Analysis*** In this section, we make use of basic queueing formulas, with the common assumptions of Poisson arrivals and exponential service times.[6]

First, we make the observation that any such scheduling discipline that chooses the next item to be served independent of service time obeys the following relationship:

$$\frac{T_r}{T_s} = \frac{1}{1 - \rho}$$

where

$T_r$ = turnaround time or residence time; total time in system, waiting plus execution,

$T_s$ = average service time; average time spent in Running state, and

$\rho$ = processor utilization.

In particular, a priority-based scheduler, in which the priority of each process is assigned independent of expected service time, provides the same average turnaround time and average normalized turnaround time as a simple FCFS discipline. Furthermore, the presence or absence of preemption makes no differences in these averages.

With the exception of round robin and FCFS, the various scheduling disciplines considered so far do make selections on the basis of expected service time. Unfortunately, it turns out to be quite difficult to develop closed analytic models of these disciplines. However, we can get an idea of the relative performance of such scheduling algorithms, compared to FCFS, by considering priority scheduling in which priority is based on service time.

If scheduling is done on the basis of priority, and if processes are assigned to a priority class on the basis of service time, then differences do emerge. Table 9.6 shows the formulas that result when we assume two priority classes, with different service times for each class. In the table, $\lambda$ refers to the arrival rate. These results

[6]The queueing terminology used in this chapter is summarized in Appendix H. Poisson arrivals essentially mean random arrivals, as explained in Appendix H.

**Table 9.6** Formulas for Single-Server Queues with Two Priority Categories

Assumptions:
1. Poisson arrival rate.
2. Priority 1 items are serviced before priority 2 items.
3. First-come-first-served dispatching for items of equal priority.
4. No item is interrupted while being served.
5. No items leave the queue (lost calls delayed).

**(a) General formulas**

$$\lambda = \lambda_1 + \lambda_2$$

$$\rho_1 = \lambda_1 T_{s1}; \rho_2 = \lambda_2 T_{s2}$$

$$\rho = \rho_1 + \rho_2$$

$$T_s = \frac{\lambda_1}{\lambda} T_{s1} + \frac{\lambda_2}{\lambda} T_{s2}$$

$$T_r = \frac{\lambda_1}{\lambda} T_{r1} + \frac{\lambda_2}{\lambda} T_{r2}$$

**(b) No interrupts; exponential service times**

$$T_{r1} = T_{s1} + \frac{\rho_1 T_{s1} + \rho_2 T_{s2}}{1 + \rho_1}$$

$$T_{r2} = T_{s2} + \frac{T_{r1} - T_{s1}}{1 - \rho}$$

**(c) Preemptive-resume queueing discipline; exponential service times**

$$T_{r1} = T_{s1} + \frac{\rho_1 T_{s1}}{1 - \rho_1}$$

$$T_{r2} = T_{s2} + \frac{1}{1 - \rho_1}\left(\rho_1 T_{s2} + \frac{\rho T_s}{1 - \rho}\right)$$

can be generalized to any number of priority classes. Note the formulas differ for nonpreemptive versus preemptive scheduling. In the latter case, it is assumed a lower-priority process is immediately interrupted when a higher-priority process becomes ready.

As an example, let us consider the case of two priority classes, with an equal number of process arrivals in each class and with the average service time for the lower-priority class being five times that of the upper-priority class. Thus, we wish to give preference to shorter processes. Figure 9.11 shows the overall result. By giving preference to shorter jobs, the average normalized turnaround time is improved at higher levels of utilization. As might be expected, the improvement is greatest with the use of preemption. Notice, however, overall performance is not much affected.

However, significant differences emerge when we consider the two priority classes separately. Figure 9.12 shows the results for the higher-priority, shorter processes. For comparison, the upper line on the graph assumes priorities are not used, but that we are simply looking at the relative performance of that half of all processes that have the shorter processing time. The other two lines assume these processes are assigned a higher priority. When the system is run using priority scheduling without preemption, the improvements are significant. They are even more significant when preemption is used.

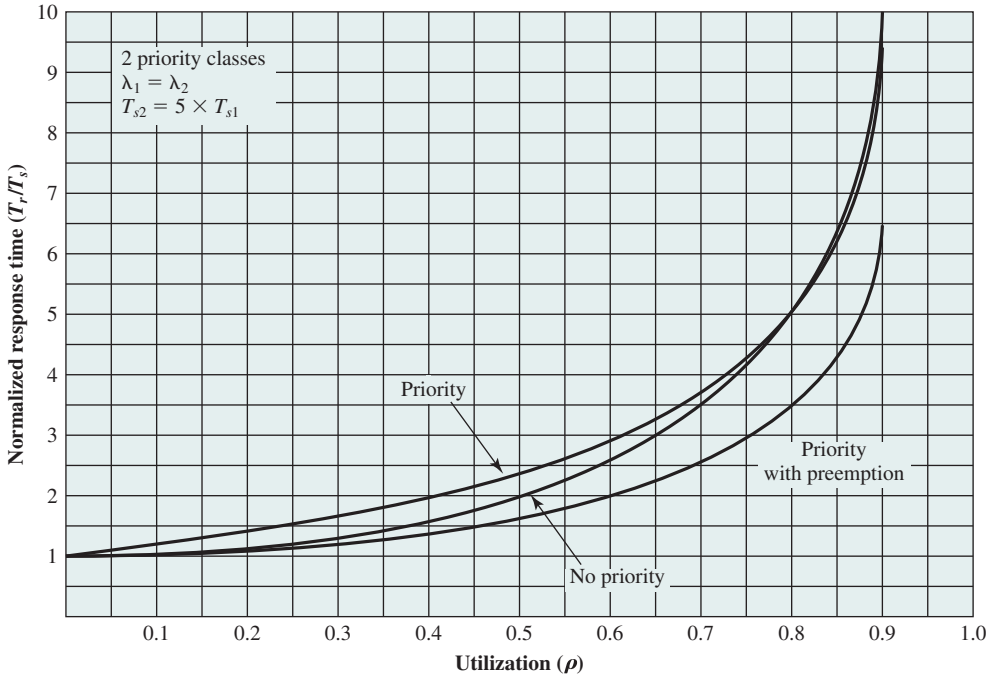



**Figure 9.11** Overall Normalized Response Time

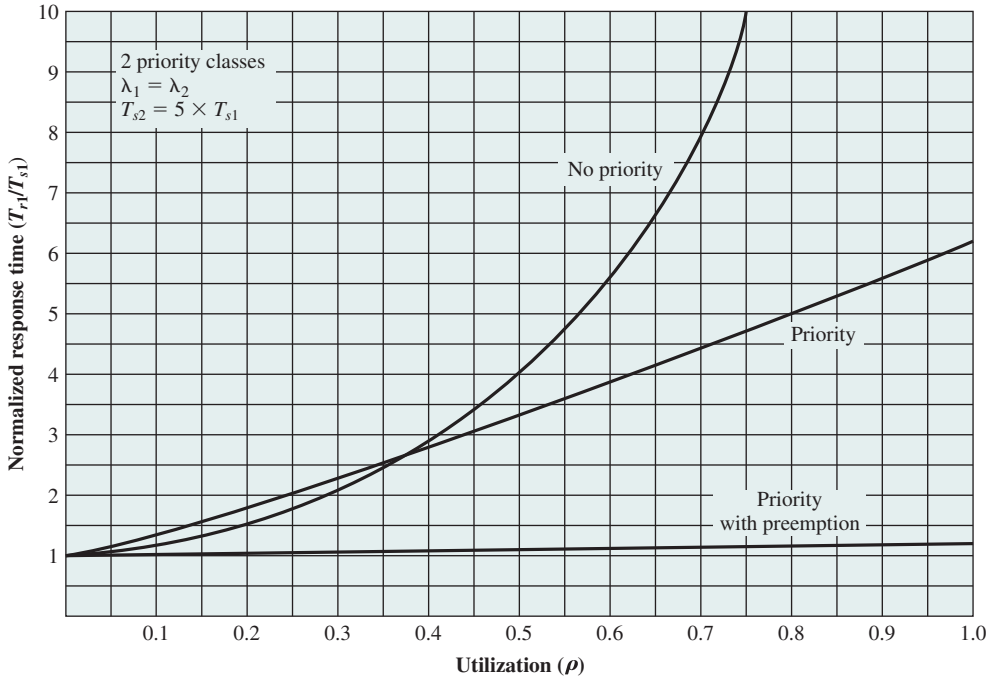



**Figure 9.12** Normalized Response Time for Shorter Processes

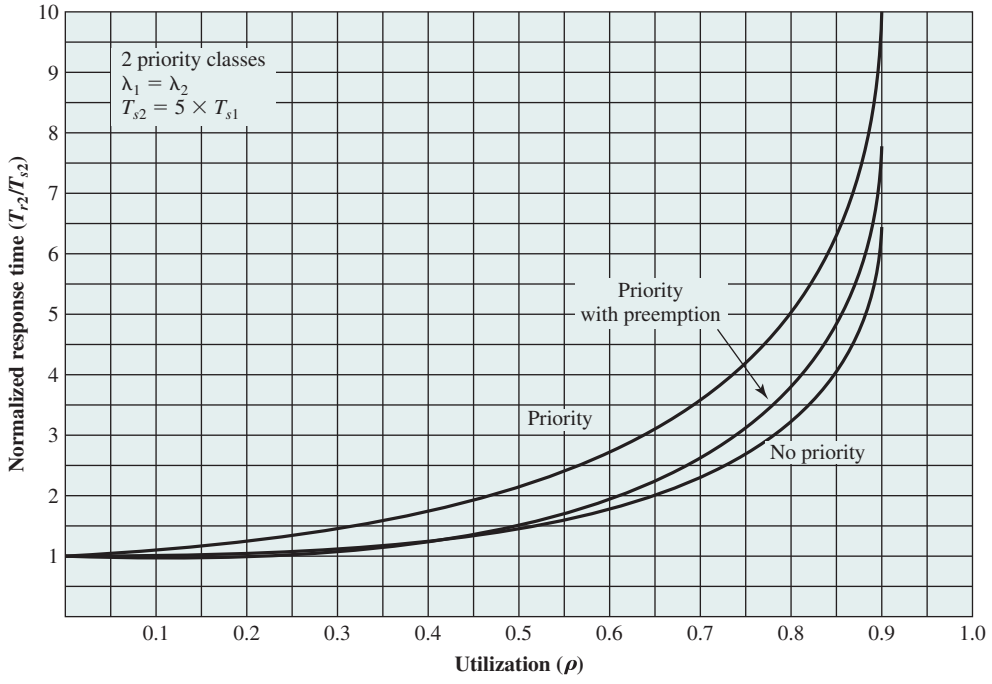



**Figure 9.13** **Normalized Response Time for Longer Processes**

Figure 9.13 shows the same analysis for the lower-priority, longer processes. As expected, such processes suffer a performance degradation under priority scheduling.

***Simulation Modeling*** Some of the difficulties of analytic modeling are overcome by using discrete-event simulation, which allows a wide range of policies to be modeled. The disadvantage of simulation is that the results for a given "run" only apply to that particular collection of processes under that particular set of assumptions. Nevertheless, useful insights can be gained.

The results of one such study are reported in [FINK88]. The simulation involved 50,000 processes with an arrival rate of $\lambda = 0.8$ and an average service time of $T_s = 1$. Thus, the assumption is the processor utilization is $\rho = \lambda T_s = 0.8$. Note, therefore, we are only measuring one utilization point.

To present the results, processes are grouped into service-time percentiles, each of which has 500 processes. Thus, the 500 processes with the shortest service time are in the first percentile; with these eliminated, the 500 remaining processes with the shortest service time are in the second percentile; and so on. This allows us to view the effect of various policies on processes as a function of the length of the process.

Figure 9.14 shows the normalized turnaround time, and Figure 9.15 shows the average waiting time. Looking at the turnaround time, we can see that the performance of FCFS is very unfavorable, with one-third of the processes having a normalized turnaround time greater than 10 times the service time; furthermore,

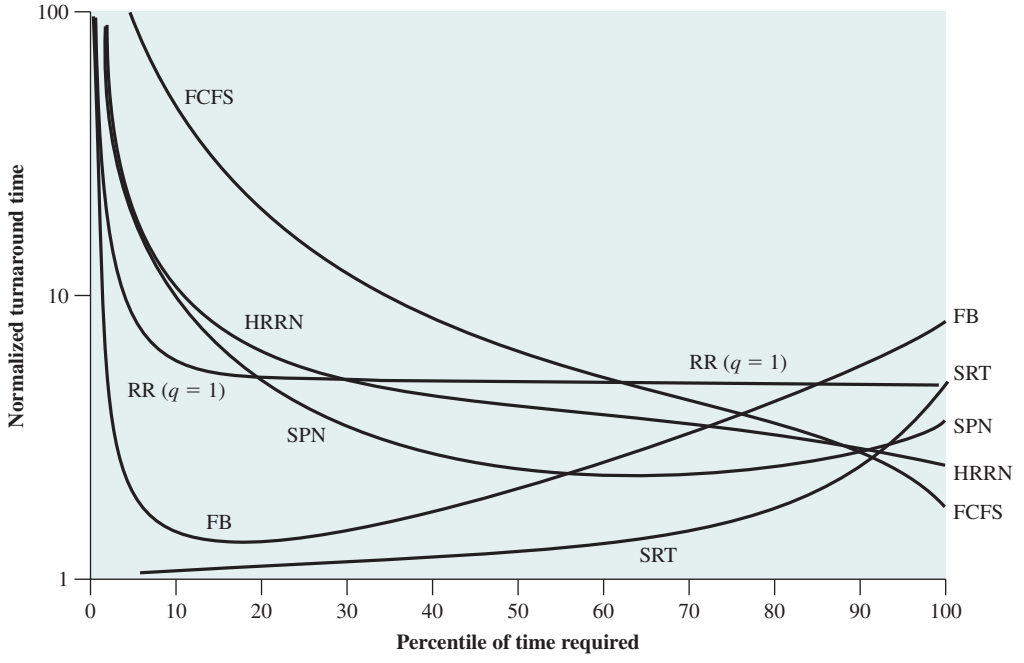



**Figure 9.14** Simulation Result for Normalized Turnaround Time

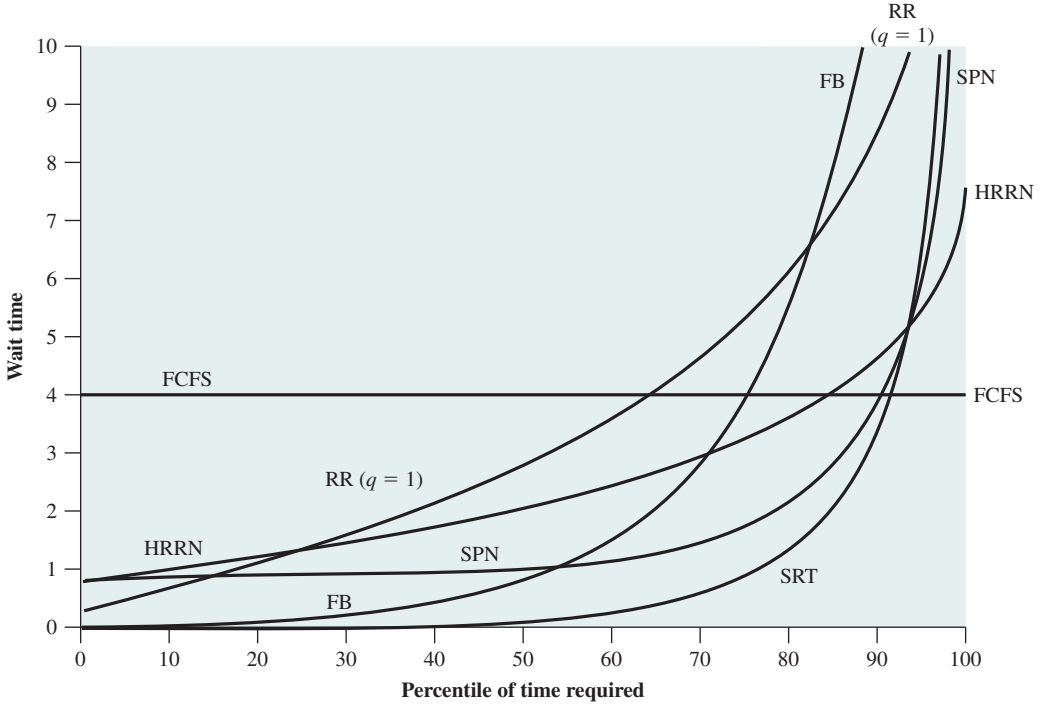



**Figure 9.15** Simulation Result for Waiting Time

these are the shortest processes. On the other hand, the absolute waiting time is uniform, as is to be expected because scheduling is independent of service time. The figures show round robin using a quantum of one time unit. Except for the shortest processes, which execute in less than one quantum, round robin yields a normalized turnaround time of about five for all processes, treating all fairly. Shortest process next performs better than round robin, except for the shortest processes. Shortest remaining time, the preemptive version of SPN, performs better than SPN except for the longest 7% of all processes. We have seen that, among nonpreemptive policies, FCFS favors long processes, and SPN favors short ones. Highest response ratio next is intended to be a compromise between these two effects, and this is indeed confirmed in the figures. Finally, the figure shows feedback scheduling with fixed, uniform quanta in each priority queue. As expected, FB performs quite well for short processes.

## Fair-Share Scheduling

All of the scheduling algorithms discussed so far treat the collection of ready processes as a single pool of processes from which to select the next running process. This pool may be broken down by priority, but is otherwise homogeneous.

However, in a multiuser system, if individual user applications or jobs may be organized as multiple processes (or threads), then there is a structure to the collection of processes that is not recognized by a traditional scheduler. From the user's point of view, the concern is not how a particular process performs but rather how his or her set of processes (which constitute a single application) performs. Thus, it would be attractive to make scheduling decisions on the basis of these process sets. This approach is generally known as fair-share scheduling. Further, the concept can be extended to groups of users, even if each user is represented by a single process. For example, in a time-sharing system, we might wish to consider all of the users from a given department to be members of the same group. Scheduling decisions could then be made that attempt to give each group similar service. Thus, if a large number of people from one department log onto the system, we would like to see response time degradation primarily affect members of that department, rather than users from other departments.

The term *fair share* indicates the philosophy behind such a scheduler. Each user is assigned a weighting of some sort that defines that user's share of system resources as a fraction of the total usage of those resources. In particular, each user is assigned a share of the processor. Such a scheme should operate in a more or less linear fashion, so if user A has twice the weighting of user B, then in the long run, user A should be able to do twice as much work as user B. The objective of a fair-share scheduler is to monitor usage to give fewer resources to users who have had more than their fair share, and more to those who have had less than their fair share.

A number of proposals have been made for fair-share schedulers [HENR84, KAY88, WOOD86]. In this section, we describe the scheme proposed in [HENR84] and implemented on a number of UNIX systems. The scheme is simply referred to as the fair-share scheduler (FSS). FSS considers the execution history of a related group of processes, along with the individual execution history of each process in

making scheduling decisions. The system divides the user community into a set of fair-share groups and allocates a fraction of the processor resource to each group. Thus, there might be four groups, each with 25% of the processor usage. In effect, each fair-share group is provided with a virtual system that runs proportionally slower than a full system.

Scheduling is done on the basis of priority, which takes into account the underlying priority of the process, its recent processor usage, and the recent processor usage of the group to which the process belongs. The higher the numerical value of the priority, the lower is the priority. The following formulas apply for process $j$ in group $k$:

$$CPU_j(i) = \frac{CPU_j(i-1)}{2}$$

$$GCPU_k(i) = \frac{GCPU_k(i-1)}{2}$$

$$P_j(i) = Base_j + \frac{CPU_j(i)}{2} + \frac{GCPU_k(i)}{4 \times W_k}$$

where

$CPU_j(i)$ = measure of processor utilization by process $j$ through interval $i$,
$GCPU_k(i)$ = measure of processor utilization of group $k$ through interval $i$,
$P_j(i)$ = priority of process $j$ at beginning of interval $i$; lower values equal higher priorities,
$Base_j$ = base priority of process $j$, and
$W_k$ = weighting assigned to group $k$, with the constraint that and $0 < W_k \leq 1$ and $\sum_k W_k = 1$.

Each process is assigned a base priority. The priority of a process drops as the process uses the processor and as the group to which the process belongs uses the processor. In the case of the group utilization, the average is normalized by dividing by the weight of that group. The greater the weight assigned to the group, the less its utilization will affect its priority.

Figure 9.16 is an example in which process A is in one group, and processes B and C are in a second group, with each group having a weighting of 0.5. Assume all processes are processor bound and are usually ready to run. All processes have a base priority of 60. Processor utilization is measured as follows: The processor is interrupted 60 times per second; during each interrupt, the processor usage field of the currently running process is incremented, as is the corresponding group processor field. Once per second, priorities are recalculated.

In the figure, process A is scheduled first. At the end of one second, it is preempted. Processes B and C now have the higher priority, and process B is scheduled. At the end of the second time unit, process A has the highest priority. Note the pattern repeats: The kernel schedules the processes in order: A, B, A, C, A, B, and so on. Thus, 50% of the processor is allocated to process A, which constitutes one group, and 50% to processes B and C, which constitute another group.




**Figure 9.16** **Example of Fair-Share Scheduler—Three Processes, Two Groups**

## 9.3 TRADITIONAL UNIX SCHEDULING

In this section, we examine traditional UNIX scheduling, which is used in both SVR3 and 4.3 BSD UNIX. These systems are primarily targeted at the time-sharing interactive environment. The scheduling algorithm is designed to provide good response time for interactive users while ensuring that low-priority background jobs do not starve. Although this algorithm has been replaced in modern

UNIX systems, it is worthwhile to examine the approach because it is representative of practical time-sharing scheduling algorithms. The scheduling scheme for SVR4 includes an accommodation for real-time requirements, and so its discussion is deferred to Chapter 10.

The traditional UNIX scheduler employs multilevel feedback using round robin within each of the priority queues. The system makes use of one-second preemption. That is, if a running process does not block or complete within one second, it is preempted. Priority is based on process type and execution history. The following formulas apply:

$$CPU_j(i) = \frac{CPU_j(i-1)}{2}$$

$$P_j(i) = Base_j + \frac{CPU_j(i)}{2} + nice_j$$

where

$CPU_j(i)$ = measure of processor utilization by process $j$ through interval $i$,
$P_j(i)$ = priority of process $j$ at beginning of interval $i$; lower values equal higher priorities,
$Base_j$ = base priority of process $j$, and
$nice_j$ = user-controllable adjustment factor.

The priority of each process is recomputed once per second, at which time a new scheduling decision is made. The purpose of the base priority is to divide all processes into fixed bands of priority levels. The *CPU* and *nice* components are restricted to prevent a process from migrating out of its assigned band (assigned by the base priority level). These bands are used to optimize access to block devices (e.g., disk) and to allow the OS to respond quickly to system calls. In decreasing order of priority, the bands are:

- Swapper.
- Block I/O device control.
- File manipulation.
- Character I/O device control.
- User processes.

This hierarchy should provide the most efficient use of the I/O devices. Within the user process band, the use of execution history tends to penalize processor-bound processes at the expense of I/O-bound processes. Again, this should improve efficiency. Coupled with the round-robin preemption scheme, the scheduling strategy is well equipped to satisfy the requirements for general-purpose time sharing.

An example of process scheduling is shown in Figure 9.17. Processes A, B, and C are created at the same time with base priorities of 60 (we will ignore the *nice* value). The clock interrupts the system 60 times per second and increments a counter for the running process. The example assumes none of the processes block themselves, and no other processes are ready to run. Compare this with Figure 9.16.

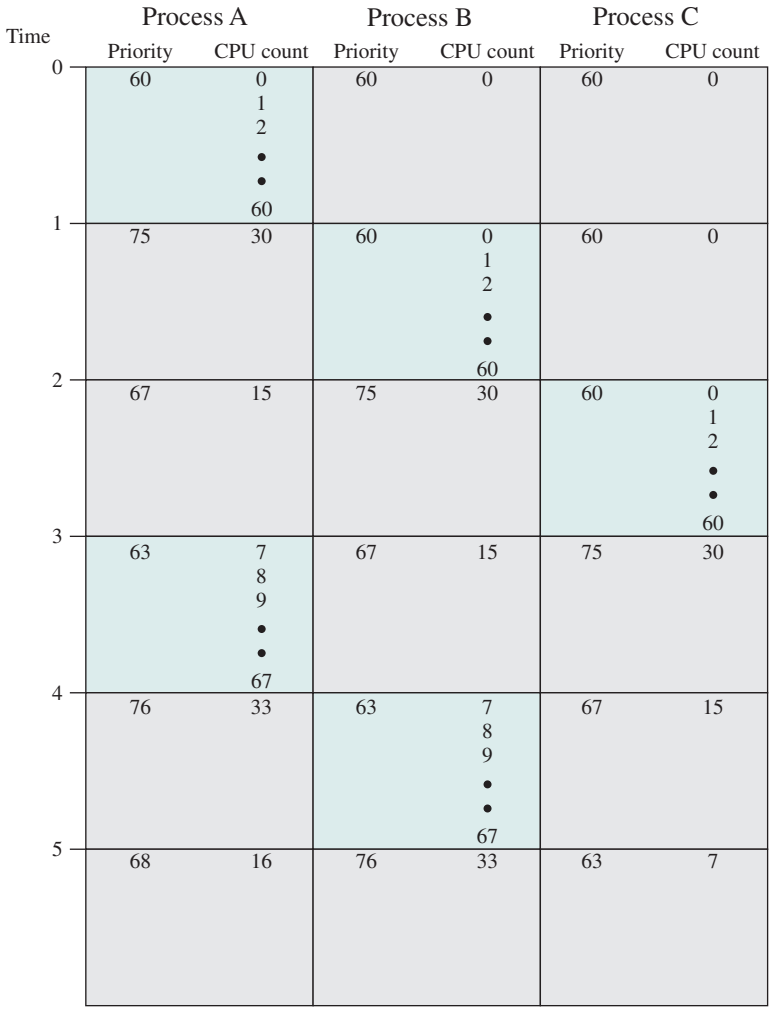



**Figure 9.17** **Example of a Traditional UNIX Process Scheduling**

## 9.4 SUMMARY

The OS must make three types of scheduling decisions with respect to the execution of processes. Long-term scheduling determines when new processes are admitted to the system. Medium-term scheduling is part of the swapping function and determines when a program is brought partially or fully into main memory so it may be executed. Short-term scheduling determines which ready process will be executed

next by the processor. This chapter focuses on the issues relating to short-term scheduling.

A variety of criteria are used in designing the short-term scheduler. Some of these criteria relate to the behavior of the system as perceived by the individual user (user oriented), while others view the total effectiveness of the system in meeting the needs of all users (system oriented). Some of the criteria relate specifically to quantitative measures of performance, while others are more qualitative in nature. From a user's point of view, response time is generally the most important characteristic of a system, while from a system point of view, throughput or processor utilization is important.

A variety of algorithms have been developed for making the short-term scheduling decision among all ready processes:

- **First-come-first-served:** Select the process that has been waiting the longest for service.
- **Round robin:** Use time slicing to limit any running process to a short burst of processor time, and rotate among all ready processes.
- **Shortest process next:** Select the process with the shortest expected processing time, and do not preempt the process.
- **Shortest remaining time:** Select the process with the shortest expected remaining process time. A process may be preempted when another process becomes ready.
- **Highest response ratio next:** Base the scheduling decision on an estimate of normalized turnaround time.
- **Feedback:** Establish a set of scheduling queues and allocate processes to queues based on execution history and other criteria.

The choice of scheduling algorithm will depend on expected performance and on implementation complexity.

## 9.5 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

### Key Terms

| | | |
|---|---|---|
| arrival rate | medium-term scheduler | short-term scheduler |
| dispatcher | multilevel feedback | throughput |
| exponential averaging | predictability | time slicing |
| fair-share scheduling | residence time | turnaround time (TAT) |
| fairness | response time | utilization |
| first-come-first-served | round robin | waiting time |
| first-in-first-out | scheduling priority | |
| long-term scheduler | service time | |

## Review Questions

**9.1.** How is processor scheduling done in the batch portion of an OS?
**9.2.** What is the main function of a dispatcher? Give some examples of events when it is invoked.
**9.3.** What scheduling criteria affect the performance of a system?
**9.4.** If purely priority-based scheduling is used in a system, what are the problems that the system will face?
**9.5.** Identify the advantages and disadvantages of preemptive scheduling.
**9.6.** Briefly define FCFS scheduling.
**9.7.** Briefly define round-robin scheduling.
**9.8.** Briefly define shortest-process-next scheduling.
**9.9.** Briefly define shortest-remaining-time scheduling.
**9.10.** Briefly define highest-response-ratio-next scheduling.
**9.11.** Briefly define feedback scheduling.

## Problems

**9.1.** Consider the following workload:

| Process | Burst Time | Priority | Arrival Time |
|---|---|---|---|
| P1 | 50 ms | 4 | 0 ms |
| P2 | 20 ms | 1 | 20 ms |
| P3 | 100 ms | 3 | 40 ms |
| P4 | 40 ms | 2 | 60 ms |

**a.** Show the schedule using shortest remaining time, nonpreemptive priority (a smaller priority number implies higher priority) and round robin with quantum 30 ms. Use time scale diagram as shown below for the FCFS example to show the schedule for each requested scheduling policy.
Example for FCFS (1 unit = 10 ms):

| P1 | P1 | P1 | P1 | P1 | P2 | P2 | P3 | P3 | P3 | P3 | P3 | P3 | P3 | P3 | P3 | P3 | P4 | P4 | P4 | P4 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |

**b.** What is the average waiting time of the above scheduling policies?

**9.2.** What factors determine the time quantum in round robin scheduling? Consider the system:

| Process | Arrival Time | Processing Time |
|---|---|---|
| P1 | 0 | 12 |
| P2 | 2 | 6 |
| P3 | 8 | 18 |
| P4 | 10 | 4 |

Context switch takes a time of 1 unit. Compute the average turnaround time of the processes for the time quanta $q = 2, q = 4$, and $q = 8$ respectively.

**9.3.** Consider that a uniprocessor system has $n$ processes to be scheduled. If only nonpreemptive scheduling algorithms are allowed, can you determine the maximum number of possible schedules in terms of $n$?

**9.4.** Assume the following burst-time pattern for a process: 6, 4, 6, 4, 13, 13, 13, and assume the initial guess is 10. Produce a plot similar to those of Figure 9.9.

**9.5.** Consider the following pair of equations as an alternative to Equation (9.3):

$$S_{n+1} = \alpha T_n + (1 - \alpha)S_n$$
$$X_{n+1} = \min [Ubound, \max[Lbound, (\beta S_{n+1})]]$$

where *Ubound* and *Lbound* are prechosen upper and lower bounds on the estimated value of $T$. The value of $X_{n+1}$ is used in the shortest-process-next algorithm, instead of the value of $S_{n+1}$. What functions do $\alpha$ and $\beta$ perform, and what is the effect of higher and lower values on each?

**9.6.** In the bottom example in Figure 9.5, process A runs for two time units before control is passed to process B. Another plausible scenario would be that A runs for three time units before control is passed to process B. What policy differences in the feedback-scheduling algorithm would account for the two different scenarios?

**9.7.** In a nonpreemptive uniprocessor system, the ready queue contains three jobs at time $t$ immediately after the completion of a job. These jobs arrived at times $t_1$, $t_2$, and $t_3$ with estimated execution times of $r_1$, $r_2$, and $r_3$, respectively. Figure 9.18 shows the linear increase of their response ratios over time. Use this example to find a variant of response ratio scheduling, known as minimax response ratio scheduling, that minimizes the maximum response ratio for a given batch of jobs ignoring further arrivals. (*Hint:* Decide, first, which job to schedule as the last one.)

**9.8.** Prove that the minimax response ratio algorithm of the preceding problem minimizes the maximum response ratio for a given batch of jobs. (*Hint*: Focus attention on the job that will achieve the highest response ratio and all jobs executed before it. Consider the same subset of jobs scheduled in any other order and observe the response ratio of the job that is executed as the last one among them. Notice this subset may now be mixed with other jobs from the total set.)

**9.9.** Define residence time $T_r$ as the average total time a process spends waiting and being served. Show that for FIFO, with mean service time $T_s$, we have $T_r = T_s/(1 - \rho)$, where $\rho$ is utilization.

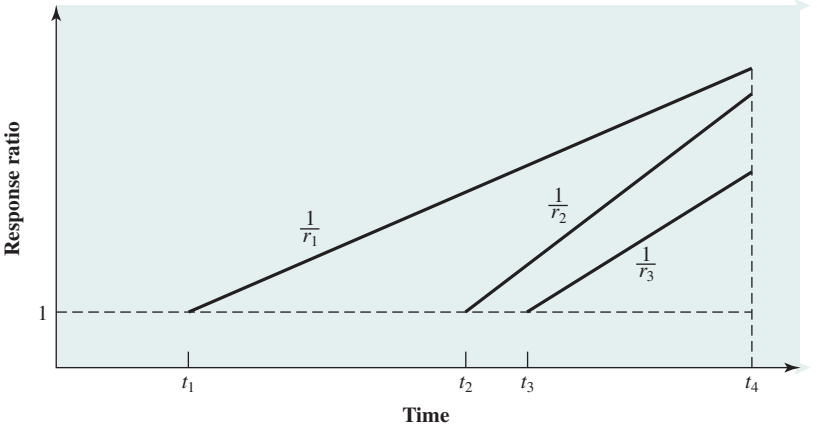



**Figure 9.18** **Response Ratio as a Function of Time**

**9.10.** A processor is multiplexed at infinite speed among all processes present in a ready queue with no overhead. (This is an idealized model of round-robin scheduling among ready processes using time slices that are very small compared to the mean service time.) Show that for Poisson input from an infinite source with exponential service times, the mean response time $R_x$ of a process with service time $x$ is given by $R_x = x/(1 - \rho)$. (*Hint:* Review the basic queueing equations in Appendix H or Chapter 20. Then consider the number of items waiting, $w$, in the system upon arrival of the given process.)

**9.11.** Consider a variation of round robin scheduling, say NRR scheduling. In NRR scheduling, each process can have its own time quantum, $q$. The value of $q$ starts out at 40 ms and decreases by 10 ms each time it goes through the round robin queue, until it reaches a minimum of 10 ms. Thus, long jobs get decreasingly shorter time slices. Analyze this scheduling algorithm for three jobs A, B, and C that arrive in the system having estimated burst times of 100 ms, 120 ms, and 60 ms respectively. Also identify some advantages and disadvantages that are associated with this algorithm.

**9.12.** In a queueing system, new jobs must wait for a while before being served. While a job waits, its priority increases linearly with time from zero at a rate $\alpha$. A job waits until its priority reaches the priority of the jobs in service; then, it begins to share the processor equally with other jobs in service using round robin while its priority continues to increase at a slower rate $\beta$. The algorithm is referred to as selfish round robin, because the jobs in service try (in vain) to monopolize the processor by increasing their priority continuously. Use Figure 9.19 to show that the mean response time $R_x$ for a job of service time $x$ is given by:

$$R_x = \frac{s}{1 - \rho} + \frac{x - s}{1 - \rho'}$$

where

$$\rho = \lambda s \quad \rho' = \rho\left(1 - \frac{\beta}{\alpha}\right) \quad 0 \leq \beta < \alpha$$

assuming arrival and service times are exponentially distributed with means $1/\lambda$ and $s$, respectively. (*Hint:* Consider the total system and the two subsystems separately.)

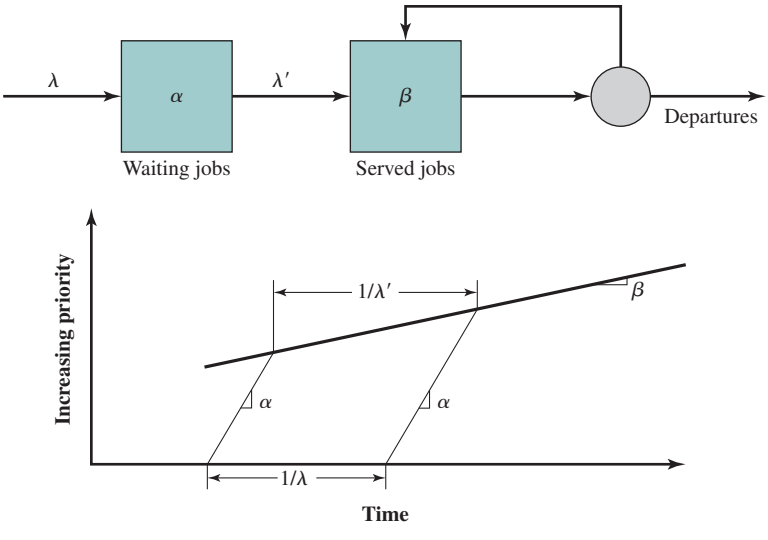



**Figure 9.19** **Selfish Round Robin**

**9.13.** An interactive system using round-robin scheduling and swapping tries to give guaranteed response to trivial requests as follows. After completing a round-robin cycle among all ready processes, the system determines the time slice to allocate to each ready process for the next cycle by dividing a maximum response time by the number of processes requiring service. Is this a reasonable policy?

**9.14.** Which type of process is generally favored by a multilevel feedback queueing scheduler—a processor-bound process, or an I/O-bound process? Briefly explain why.

**9.15.** A variation of preemptive priority scheduling has dynamically changing priorities. A new process is assigned a priority 0. While a process is in the ready queue, its priority changes at the rate of $\alpha$; and while it is executing, its priority changes at the rate of $\beta$. The different values of $\alpha$ and $\beta$ give different algorithms. If larger values imply higher priorities, state with reasons the type of algorithm that will result if:

**a.** $\alpha < \beta < 0$

**b.** $\beta > \alpha > 0$

**9.16.** Five batch jobs, A through E, arrive at a computer center at essentially the same time. They have an estimated running time of 15, 9, 3, 6, and 12 minutes, respectively. Their (externally defined) priorities are 6, 3, 7, 9, and 4, respectively, with a lower value corresponding to a higher priority. For each of the following scheduling algorithms, determine the turnaround time for each process and the average turnaround for all jobs. Ignore process switching overhead. Explain how you arrived at your answers. In the last three cases, assume only one job at a time runs until it finishes, and all jobs are completely processor bound.

**a.** round robin with a time quantum of 1 minute

**b.** priority scheduling

**c.** FCFS (run in order 15, 9, 3, 6, and 12)

**d.** shortest job first

CHAPTER 10

# Multiprocessor, Multicore, and Real-Time Scheduling


**10.1 Multiprocessor and Multicore Scheduling**
- Granularity
- Design Issues
- Process Scheduling
- Thread Scheduling
- Multicore Thread Scheduling

**10.2 Real-Time Scheduling**
- Background
- Characteristics of Real-Time Operating Systems
- Real-Time Scheduling
- Deadline Scheduling
- Rate Monotonic Scheduling
- Priority Inversion

**10.3 Linux Scheduling**
- Real-Time Scheduling
- Non-Real-Time Scheduling

**10.4 UNIX SVR4 Scheduling**

**10.5 UNIX FreeBSD Scheduling**
- Priority Classes
- SMP and Multicore Support

**10.6 Windows Scheduling**
- Process and Thread Priorities
- Multiprocessor Scheduling

**10.7 Summary**

**10.8 Key Terms, Review Questions, and Problems**

**Learning Objectives**

After studying this chapter, you should be able to:

- Understand the concept of thread granularity.
- Discuss the key design issues in multiprocessor thread scheduling and some of the key approaches to scheduling.
- Understand the requirements imposed by real-time scheduling.
- Explain the scheduling methods used in Linux, UNIX SVR4, and Windows 10.

This chapter continues our survey of process and thread scheduling. We begin with an examination of issues raised by the availability of more than one processor. A number of design issues are explored. This is followed by a look at the scheduling of processes on a multiprocessor system. Then the somewhat different design considerations for multiprocessor thread scheduling are examined. The second section of this chapter covers real-time scheduling. The section begins with a discussion of the characteristics of real-time processes, then looks at the nature of the scheduling process. Two approaches to real-time scheduling, deadline scheduling and rate monotonic scheduling, are examined.

## 10.1 MULTIPROCESSOR AND MULTICORE SCHEDULING

When a computer system contains more than a single processor, several new issues are introduced into the design of the scheduling function. We begin with a brief overview of multiprocessors then look at the rather different considerations when scheduling is done at the process level and at the thread level.

We can classify multiprocessor systems as follows:

- **Loosely coupled or distributed multiprocessor, or cluster:** Consists of a collection of relatively autonomous systems, each processor having its own main memory and I/O channels. We will address this type of configuration in Chapter 16.
- **Functionally specialized processors:** An example is an I/O processor. In this case, there is a master, general-purpose processor; specialized processors are controlled by the master processor and provide services to it. Issues relating to I/O processors are addressed in Chapter 11.
- **Tightly coupled multiprocessor:** Consists of a set of processors that share a common main memory and are under the integrated control of an operating system.

Our concern in this section is with the last category, specifically with issues relating to scheduling.

### Granularity

A good way of characterizing multiprocessors and placing them in context with other architectures is to consider the synchronization granularity, or frequency of

**Table 10.1** Synchronization Granularity and Processes

| Grain Size | Description | Synchronization Interval (Instructions) |
|---|---|---|
| Fine | Parallelism inherent in a single instruction stream | < 20 |
| Medium | Parallel processing or multitasking within a single application | 20–200 |
| Coarse | Multiprocessing of concurrent processes in a multiprogramming environment | 200–2,000 |
| Very Coarse | Distributed processing across network nodes to form a single computing environment | 2,000–1M |
| Independent | Multiple unrelated processes | Not applicable |

synchronization, between processes in a system. We can distinguish five categories of parallelism that differ in the degree of granularity. These are summarized in Table 10.1.

***Independent Parallelism*** With independent parallelism, there is no explicit synchronization among processes. Each represents a separate, independent application or job. A typical use of this type of parallelism is in a time-sharing system. Each user is performing a particular application, such as word processing or using a spreadsheet. The multiprocessor provides the same service as a multiprogrammed uniprocessor. Because more than one processor is available, average response time to the users will be shorter.

It is possible to achieve a similar performance gain by providing each user with a personal computer or workstation. If any files or information are to be shared, then the individual systems must be hooked together into a distributed system supported by a network. This approach will be examined in Chapter 16. On the other hand, a single, multiprocessor shared system in many instances is more cost-effective than a distributed system, allowing economies of scale in disks and other peripherals.

***Coarse and Very Coarse-Grained Parallelism*** With coarse and very coarse-grained parallelism, there is synchronization among processes, but at a very gross level. This kind of situation is easily handled as a set of concurrent processes running on a multiprogrammed uniprocessor, and can be supported on a multiprocessor with little or no change to user software.

A simple example of an application that can exploit the existence of a multiprocessor is given in [WOOD89]. The authors have developed a program that takes a specification of files needing recompilation to rebuild a piece of software and determines which of these compiles (usually all of them) can be run simultaneously. The program then spawns one process for each parallel compile. The authors report that the speedup on a multiprocessor actually exceeds what would be expected by simply adding up the number of processors in use, due to synergies in the disk buffer caches (a topic explored in Chapter 11) and sharing of compiler code, which is loaded into memory only once.

In general, any collection of concurrent processes that need to communicate or synchronize can benefit from the use of a multiprocessor architecture. In the case of very infrequent interaction among processes, a distributed system can provide good support. However, if the interaction is somewhat more frequent, then the overhead of communication across the network may negate some of the potential speedup. In that case, the multiprocessor organization provides the most effective support.

***Medium-Grained Parallelism*** We saw in Chapter 4 that a single application can be effectively implemented as a collection of threads within a single process. In this case, the programmer must explicitly specify the potential parallelism of an application. Typically, there will need to be rather a high degree of coordination and interaction among the threads of an application, leading to a medium-grain level of synchronization.

Whereas independent, very coarse, and coarse-grained parallelism can be supported on either a multiprogrammed uniprocessor or a multiprocessor with little or no impact on the scheduling function, we need to reexamine scheduling when dealing with the scheduling of threads. Because the various threads of an application interact so frequently, scheduling decisions concerning one thread may affect the performance of the entire application. We will return to this issue later in this section.

***Fine-Grained Parallelism*** Fine-grained parallelism represents a much more complex use of parallelism than is found in the use of threads. Although much work has been done on highly parallel applications, this is so far a specialized and fragmented area, with many different approaches.

Chapter 4 provides an example of the use of granularity for the Valve game software.

## Design Issues

Scheduling on a multiprocessor involves three interrelated issues:

1. The assignment of processes to processors
2. The use of multiprogramming on individual processors
3. The actual dispatching of a process

In looking at these three issues, it is important to keep in mind that the approach taken will depend, in general, on the degree of granularity of the applications, and on the number of processors available.

***Assignment of Processes to Processors*** If we assume the architecture of the multiprocessor is uniform, in the sense that no processor has a particular physical advantage with respect to access to main memory or to I/O devices, then the simplest scheduling approach is to treat the processors as a pooled resource, and assign processes to processors on demand. The question then arises as to whether the assignment should be static or dynamic.

If a process is permanently assigned to one processor from activation until its completion, then a dedicated short-term queue is maintained for each processor. An advantage of this approach is that there may be less overhead in the scheduling

function, because the processor assignment is made once and for all. Also, the use of dedicated processors allows a strategy known as group or gang scheduling, as discussed later.

A disadvantage of static assignment is that one processor can be idle, with an empty queue, while another processor has a backlog. To prevent this situation, a common queue can be used. All processes go into one global queue and are scheduled to any available processor. Thus, over the life of a process, the process may be executed on different processors at different times. In a tightly coupled shared-memory architecture, the context information for all processes will be available to all processors, and therefore the cost of scheduling a process will be independent of the identity of the processor on which it is scheduled. Yet another option is dynamic load balancing, in which threads are moved from a queue for one processor to a queue for another processor; Linux uses this approach.

Regardless of whether processes are dedicated to processors, some means is needed to assign processes to processors. Two approaches have been used: master/slave and peer. With a master/slave architecture, key kernel functions of the operating system always run on a particular processor. The other processors may only execute user programs. The master is responsible for scheduling jobs. Once a process is active, if the slave needs service (e.g., an I/O call), it must send a request to the master and wait for the service to be performed. This approach is quite simple and requires little enhancement to a uniprocessor multiprogramming operating system. Conflict resolution is simplified because one processor has control of all memory and I/O resources. There are two disadvantages to this approach: (1) A failure of the master brings down the whole system, and (2) the master can become a performance bottleneck.

In a peer architecture, the kernel can execute on any processor, and each processor does self-scheduling from the pool of available processes. This approach complicates the operating system. The operating system must ensure that two processors do not choose the same process and that the processes are not somehow lost from the queue. Techniques must be employed to resolve and synchronize competing claims to resources.

There is, of course, a spectrum of approaches between these two extremes. One approach is to provide a subset of processors dedicated to kernel processing instead of just one. Another approach is simply to manage the difference between the needs of kernel processes and other processes on the basis of priority and execution history.

***The Use of Multiprogramming on Individual Processors*** When each process is statically assigned to a processor for the duration of its lifetime, a new question arises: Should that processor be multiprogrammed? The reader's first reaction may be to wonder why the question needs to be asked; it would appear particularly wasteful to tie up a processor with a single process when that process may frequently be blocked waiting for I/O or because of concurrency/synchronization considerations.

In the traditional multiprocessor, which is dealing with coarse-grained or independent synchronization granularity (see Table 10.1), it is clear that each individual processor should be able to switch among a number of processes to achieve high

utilization and therefore better performance. However, for medium-grained applications running on a multiprocessor with many processors, the situation is less clear. When many processors are available, it is no longer paramount that every single processor be busy as much as possible. Rather, we are concerned to provide the best performance, on average, for the applications. An application that consists of a number of threads may run poorly unless all of its threads are available to run simultaneously.

***Process Dispatching*** The final design issue related to multiprocessor scheduling is the actual selection of a process to run. We have seen that, on a multiprogrammed uniprocessor, the use of priorities or of sophisticated scheduling algorithms based on past usage may improve performance over a simple-minded first-come-first-served strategy. When we consider multiprocessors, these complexities may be unnecessary or even counterproductive, and a simpler approach may be more effective with less overhead. In the case of thread scheduling, new issues come into play that may be more important than priorities or execution histories. We address each of these topics in turn.

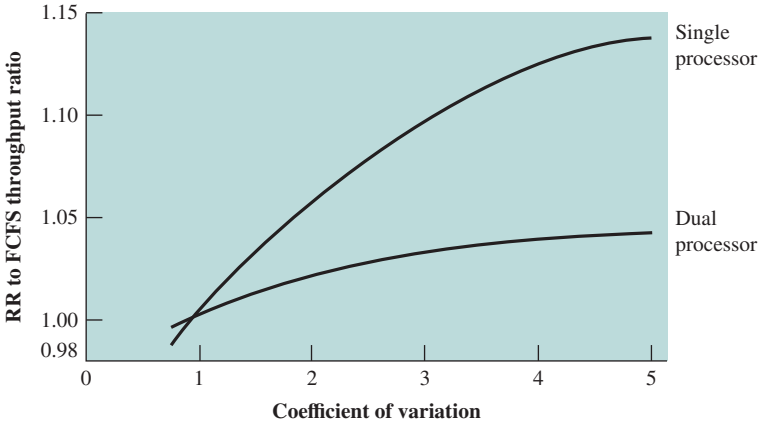
## Process Scheduling

In most traditional multiprocessor systems, processes are not dedicated to processors. Rather, there is a single queue for all processors, or if some sort of priority scheme is used, there are multiple queues based on priority, all feeding into the common pool of processors. In any case, we can view the system as being a multiserver queueing architecture.

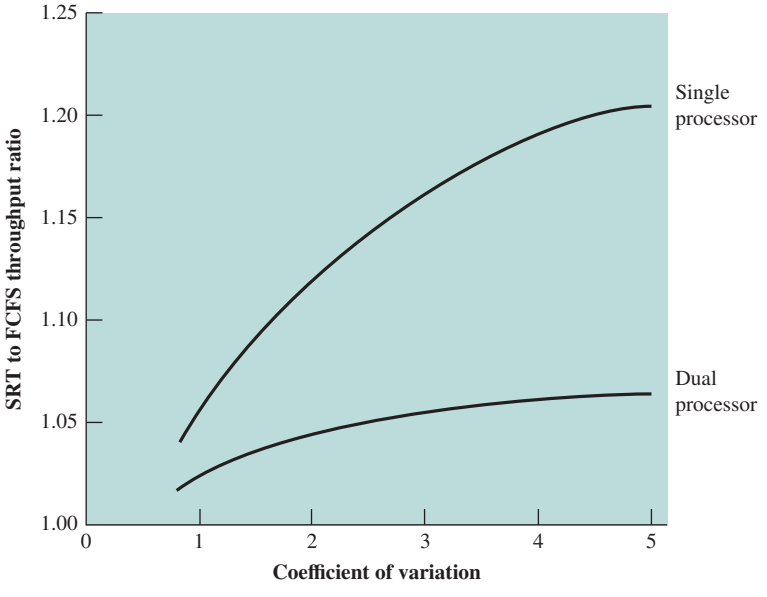
Consider the case of a dual-processor system in which each processor of the dual-processor system has half the processing rate of a processor in the single-processor system. [SAUE81] reports a queueing analysis that compares FCFS scheduling to round robin and to shortest remaining time. The study is concerned with process service time, which measures the amount of processor time a process needs, either for a total job or the amount of time needed each time the process is ready to use the processor. In the case of round robin, it is assumed that the time quantum is large compared to context-switching overhead and small compared to mean service time. The results depend on the variability that is seen in service times. A common measure of variability is the coefficient of variation, $C_s$.[1] A value of $C_s$ corresponds to the case where there is no variability: the service times of all processes are equal. Increasing values of $C_s = 0$ correspond to increasing variability among the service times. That is, the larger the value of $C_s$, the more widely do the values of the service times vary. Values of $C_s$ of 5 or more are not unusual for processor service time distributions.

Figure 10.1a compares round-robin throughput to FCFS throughput as a function of $C_s$s. Note the difference in scheduling algorithms is much smaller in the dual-processor case. With two processors, a single process with long service time is much

[1]The value of $C_s$ is calculated as $\sigma_s/T_s$, where $\sigma_s$ is the standard deviation of service time and $T_s$ is the mean service time. For a further explanation of $C_s$, see the discussion in Chapter 20.




(a) Comparison of RR and FCFS




(b) Comparison of SRT and FCFS

**Figure 10.1 Comparison of Scheduling Performance for One and Two Processors**

less disruptive in the FCFS case; other processes can use the other processor. Similar results are shown in Figure 10.1b.

The study in [SAUE81] repeated this analysis under a number of assumptions about degree of multiprogramming, mix of I/O-bound versus CPU-bound processes, and the use of priorities. The general conclusion is that the specific scheduling discipline is much less important with two processors than with one. It should be evident that this conclusion becomes even stronger as the number of processors increases. Thus, a simple FCFS discipline, or the use of FCFS within a static priority scheme, may suffice for a multiprocessor system.

## Thread Scheduling

As we have seen, with threads, the concept of execution is separated from the rest of the definition of a process. An application can be implemented as a set of threads that cooperate and execute concurrently in the same address space.

On a uniprocessor, threads can be used as a program structuring aid and to overlap I/O with processing. Because of the minimal penalty in doing a thread switch compared to a process switch, these benefits are realized with little cost. However, the full power of threads becomes evident in a multiprocessor system. In this environment, threads can be used to exploit true parallelism in an application. If the various threads of an application are simultaneously run on separate processors, dramatic gains in performance are possible. However, it can be shown for applications that require significant interaction among threads (medium-grained parallelism), small differences in thread management and scheduling can have a significant performance impact [ANDE89].

Among the many proposals for multiprocessor thread scheduling and processor assignment, four general approaches stand out:

1. **Load sharing:** Processes are not assigned to a particular processor. A global queue of ready threads is maintained, and each processor, when idle, selects a thread from the queue. The term **load sharing** is used to distinguish this strategy from load-balancing schemes in which work is allocated on a more permanent basis (e.g., see [FEIT90a]).[2]
2. **Gang scheduling:** A set of related threads is scheduled to run on a set of processors at the same time, on a one-to-one basis.
3. **Dedicated processor assignment:** This is the opposite of the load-sharing approach and provides implicit scheduling defined by the assignment of threads to processors. Each program, for the duration of its execution, is allocated a number of processors equal to the number of threads in the program. When the program terminates, the processors return to the general pool for possible allocation to another program.
4. **Dynamic scheduling:** The number of threads in a process can be altered during the course of execution.

***Load Sharing*** Load sharing is perhaps the simplest approach and the one that carries over most directly from a uniprocessor environment. It has several advantages:

- The load is distributed evenly across the processors, assuring that no processor is idle while work is available to do.
- No centralized scheduler is required; when a processor is available, the scheduling routine of the operating system is run on that processor to select the next thread.

[2]Some of the literature on this topic refers to this approach as *self-scheduling*, because each processor schedules itself without regard to other processors. However, this term is also used in the literature to refer to programs written in a language that allows the programmer to specify the scheduling (e.g., see [FOST91]).

- The global queue can be organized and accessed using any of the schemes discussed in Chapter 9, including priority-based schemes and schemes that consider execution history or anticipated processing demands.

[LEUT90] analyzes three different versions of load sharing:

1. **First-come-first-served (FCFS):** When a job arrives, each of its threads is placed consecutively at the end of the shared queue. When a processor becomes idle, it picks the next ready thread, which it executes until completion or blocking.
2. **Smallest number of threads first:** The shared ready queue is organized as a priority queue, with highest priority given to threads from jobs with the smallest number of unscheduled threads. Jobs of equal priority are ordered according to which job arrives first. As with FCFS, a scheduled thread is run to completion or blocking.
3. **Preemptive smallest number of threads first:** Highest priority is given to jobs with the smallest number of unscheduled threads. An arriving job with a smaller number of threads than an executing job will preempt threads belonging to the scheduled job.

Using simulation models, the authors report that, over a wide range of job characteristics, FCFS is superior to the other two policies in the preceding list. Further, the authors find that some form of gang scheduling, discussed in the next subsection, is generally superior to load sharing.

There are several disadvantages of load sharing:

- The central queue occupies a region of memory that must be accessed in a manner that enforces mutual exclusion. Thus, it may become a bottleneck if many processors look for work at the same time. When there is only a small number of processors, this is unlikely to be a noticeable problem. However, when the multiprocessor consists of dozens or even hundreds of processors, the potential for bottleneck is real.
- Preempted threads are unlikely to resume execution on the same processor. If each processor is equipped with a local cache, caching becomes less efficient.
- If all threads are treated as a common pool of threads, it is unlikely that all of the threads of a program will gain access to processors at the same time. If a high degree of coordination is required between the threads of a program, the process switches involved may seriously compromise performance.

Despite the potential disadvantages, load sharing is one of the most commonly used schemes in current multiprocessors.

A refinement of the load-sharing technique is used in the Mach operating system [BLAC90, WEND89]. The operating system maintains a local run queue for each processor and a shared global run queue. The local run queue is used by threads that have been temporarily bound to a specific processor. A processor examines the local run queue first to give bound threads absolute preference over unbound threads. As an example of the use of bound threads, one or more processors could be dedicated to running processes that are part of the operating system. Another example is that the threads of a particular application could be distributed among a number

of processors; with the proper additional software, this provides support for gang scheduling, discussed next.

***GANG SCHEDULING*** The concept of scheduling a set of processes simultaneously on a set of processors predates the use of threads. [JONE80] refers to the concept as *group scheduling*. This approach has the following performance benefits:

- If processes in the group are related or coordinated in some fashion, synchronization blocking may be reduced, less process switching may be necessary, and performance will increase.
- A single scheduling decision affects a number of processors and processes at one time, reducing scheduling overhead.

On the Cm* multiprocessor, the term *coscheduling* is used [GEHR87]. Coscheduling is based on the concept of scheduling a related set of tasks, called a task force. The individual elements of a task force tend to be quite small and are hence close to the idea of a thread.

The term *gang scheduling* has been applied to the simultaneous scheduling of the threads that make up a single process [FEIT90b]. Gang scheduling is useful for medium-grained to fine-grained parallel applications whose performance severely degrades when any part of the application is not running while other parts are ready to run. It is also beneficial for any parallel application, even one that is not quite so performance sensitive. The need for gang scheduling is widely recognized, and implementations exist on a variety of multiprocessor operating systems.

One obvious way in which gang scheduling improves the performance of a single application is that process switches are minimized. Suppose one thread of a process is executing and reaches a point at which it must synchronize with another thread of the same process. If that other thread is not running, but is in a ready queue, the first thread is hung up until a process switch can be done on some other processor to bring in the needed thread. In an application with tight coordination among threads, such switches will dramatically reduce performance. The simultaneous scheduling of cooperating threads can also save time in resource allocation. For example, multiple gang-scheduled threads can access a file without the additional overhead of locking during a seek, read/write operation.

The use of gang scheduling creates a requirement for processor allocation. One possibility is the following. Suppose we have $N$ processors and $M$ applications, each of which has $N$ or fewer threads. Then each application could be given $1/M$ of the available time on the $N$ processors, using time slicing. [FEIT90a] notes that this strategy can be inefficient. Consider an example in which there are two applications, one with four threads, and one with one thread. Using uniform time allocation wastes 37.5% of the processing resource, because when the single-thread application runs, three processors are left idle (see Figure 10.2). If there are several one-thread applications, these could all be fit together to increase processor utilization. If that option is not available, an alternative to uniform scheduling is scheduling that is weighted by the number of threads. Thus, the four-thread application could be given four-fifths of the time and the one-thread application given only one-fifth of the time, reducing the processor waste to 15%.

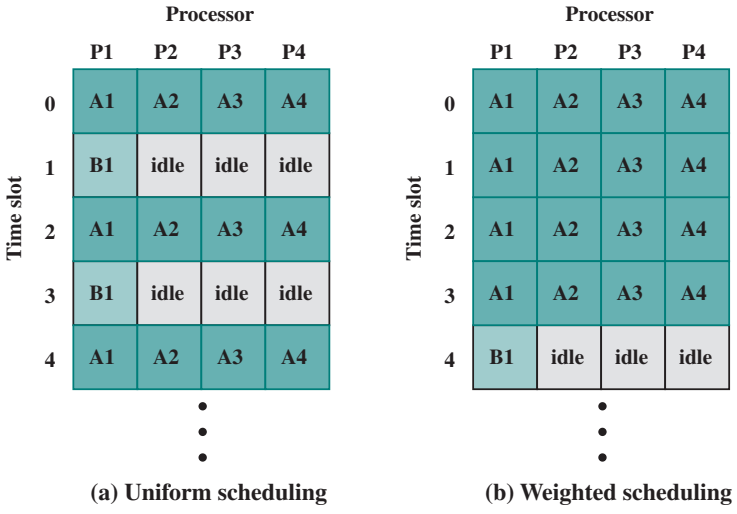



**Figure 10.2** **Gang Scheduling**

***Dedicated Processor Assignment*** An extreme form of gang scheduling, suggested in [TUCK89], is to dedicate a group of processors to an application for the duration of the application. That is, when an application is scheduled, each of its threads is assigned a processor that remains dedicated to that thread until the application runs to completion.

This approach would appear to be extremely wasteful of processor time. If a thread of an application is blocked waiting for I/O or for synchronization with another thread, then that thread's processor remains idle: There is no multiprogramming of processors. Two observations can be made in defense of this strategy:

1. In a highly parallel system, with tens or hundreds of processors, each of which represents a small fraction of the cost of the system, processor utilization is no longer so important as a metric for effectiveness or performance.
2. The total avoidance of process switching during the lifetime of a program should result in a substantial speedup of that program.

Both [TUCK89] and [ZAHO90] report analyses that support statement 2. Table 10.2 shows the results of one experiment [TUCK89]. The authors ran two applications simultaneously (executing concurrently), a matrix multiplication and a fast Fourier transform (FFT) calculation, on a system with 16 processors. Each application breaks its problem into a number of tasks, which are mapped onto the threads executing that application. The programs are written in such a way as to allow the number of threads to be used to vary. In essence, a number of tasks are defined and queued by an application. Tasks are taken from the queue and mapped onto the available threads by the application. If there are fewer threads than tasks, then leftover tasks remain queued and are picked up by threads as they complete their assigned tasks. Clearly, not all applications can be structured in this way, but many numerical problems and some other applications can be dealt with in this fashion.

**Table 10.2** Application Speedup as a Function of Number of Threads

| Number of Threads per Application | Matrix Multiplication | FFT |
|---|---|---|
| 1 | 1 | 1 |
| 2 | 1.8 | 1.8 |
| 4 | 3.8 | 3.8 |
| 8 | 6.5 | 6.1 |
| 12 | 5.2 | 5.1 |
| 16 | 3.9 | 3.8 |
| 20 | 3.3 | 3 |
| 24 | 2.8 | 2.4 |

Table 10.2 shows the speedup for the applications as the number of threads executing the tasks in each application is varied from 1 to 24. For example, we see that when both applications are started simultaneously with 24 threads each, the speedup obtained (compared to using a single thread for each application) is 2.8 for matrix multiplication and 2.4 for FFT. Table 10.2 also shows that the performance of both applications worsens considerably when the number of threads in each application exceeds eight, and thus the total number of processes in the system exceeds the number of processors. Furthermore, the larger the number of threads, the worse the performance gets, because there is a greater frequency of thread preemption and rescheduling. This excessive preemption results in inefficiency from many sources, including time spent waiting for a suspended thread to leave a critical section, time wasted in process switching, and inefficient cache behavior.

The authors conclude that an effective strategy is to limit the number of active threads to the number of processors in the system. If most of the applications are either single thread or can use the task-queue structure, this will provide an effective and reasonably efficient use of the processor resources.

Both dedicated processor assignment and gang scheduling attack the scheduling problem by addressing the issue of processor allocation. One can observe that the processor allocation problem on a multiprocessor more closely resembles the memory allocation problem on a uniprocessor than the scheduling problem on a uniprocessor. The issue is how many processors to assign to a program at any given time, which is analogous to how many page frames to assign to a given process at any time. [GEHR87] proposes the term *activity working set*, analogous to a virtual memory working set, as the minimum number of activities (threads) that must be scheduled simultaneously on processors for the application to make acceptable progress. As with memory management schemes, the failure to schedule all of the elements of an activity working set can lead to processor thrashing. This occurs when the scheduling of threads whose services are required induces the descheduling of other threads whose services will soon be needed. Similarly, processor fragmentation refers to a situation in which some processors are left over when others are allocated, and the leftover processors are either insufficient in number or unsuitably organized to support the requirements of waiting applications. Gang scheduling and dedicated processor allocation are meant to avoid these problems.

***Dynamic Scheduling*** For some applications, it is possible to provide language and system tools that permit the number of threads in the process to be altered dynamically. This would allow the operating system to adjust the load to improve utilization.

[ZAHO90] proposes an approach in which both the operating system and the application are involved in making scheduling decisions. The operating system is responsible for partitioning the processors among the jobs. Each job uses the processors currently in its partition to execute some subset of its runnable tasks by mapping these tasks to threads. An appropriate decision about which subset to run, as well as which thread to suspend when a process is preempted, is left to the individual applications (perhaps through a set of run-time library routines). This approach may not be suitable for all applications. However, some applications could default to a single thread while others could be programmed to take advantage of this particular feature of the operating system.

In this approach, the scheduling responsibility of the operating system is primarily limited to processor allocation and proceeds according to the following policy. When a job requests one or more processors (either when the job arrives for the first time or because its requirements change),

1. If there are idle processors, use them to satisfy the request.
2. Otherwise, if the job making the request is a new arrival, allocate it a single processor by taking one away from any job currently allocated more than one processor.
3. If any portion of the request cannot be satisfied, it remains outstanding until either a processor becomes available for it or the job rescinds the request (e.g., if there is no longer a need for the extra processors).

Upon release of one or more processors (including job departure),

4. Scan the current queue of unsatisfied requests for processors. Assign a single processor to each job in the list that currently has no processors (i.e., to all waiting new arrivals). Then scan the list again, allocating the rest of the processors on an FCFS basis.

Analyses reported in [ZAHO90] and [MAJU88] suggest that for applications that can take advantage of dynamic scheduling, this approach is superior to gang scheduling or dedicated processor assignment. However, the overhead of this approach may negate this apparent performance advantage. Experience with actual systems is needed to prove the worth of dynamic scheduling.

## Multicore Thread Scheduling

The most widely used contemporary OSs, such as Windows and Linux, essentially treat scheduling in multicore systems in the same fashion as a multiprocessor system. Such schedulers tend to focus on keeping processors busy by load balancing so threads ready to run are evenly distributed among the processors. However, this strategy is unlikely to produce the desired performance benefits of the multicore architecture.

As the number of cores per chip increases, a need to minimize access to off-chip memory takes precedence over a desire to maximize processor utilization. The traditional (and still principal) means of minimizing off-chip memory access is the use of caches to take advantage of locality. This approach is complicated by some of the cache architectures used on multicore chips, specifically when a cache is shared by some but not all of the cores. A good example is the AMD Bulldozer chip used in the Operton FX-8000 system, illustrated in Figure 10.3. In this architecture, each core has a dedicated L1 cache; each pair of cores share an L2 cache; and all cores share an L3 cache. Compare this with the Intel Core i7-5960X (see Figure 1.20), in which both L1 and L2 caches are dedicated to a single core.

When some but not all cores share a cache, the way in which threads are allocated to cores during scheduling has a significant effect on performance. Let us define two cores that share the same L2 cache as adjacent, and otherwise nonadjacent. Thus, cores 0 and 1 in Figure 10.3 are adjacent, but cores 1 and 2 are nonadjacent. Ideally, if two threads are going to share memory resources, they should be assigned to adjacent cores to improve the effects of locality, and if they do not share memory resources, they may be assigned to nonadjacent cores to achieve load balance.

There are in fact two different aspects of cache sharing to take into account: cooperative resource sharing and resource contention. With cooperative resource sharing, multiple threads access the same set of main memory locations. Examples are applications that are multithreaded and producer–consumer thread interaction. In both these cases, data brought into a cache by one thread need to be accessed by a cooperating thread. For this case, it is desirable to schedule cooperating threads on adjacent cores.

The other case is when threads, if operating on adjacent cores, compete for cache memory locations. Whatever technique is used for cache replacement, such as least-recently-used (LRU), if more of the cache is dynamically allocated to one

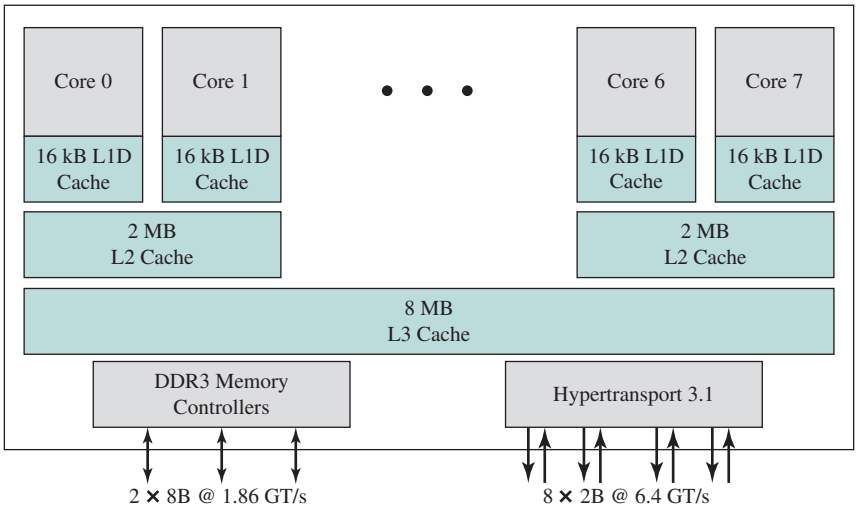



**Figure 10.3** **AMD Bulldozer Architecture**

thread, the competing thread necessarily has less cache space available and thus suffers performance degradation. The objective of contention-aware scheduling is to allocate threads to cores in such a way as to maximize the effectiveness of the shared cache memory, and therefore to minimize the need for off-chip memory accesses. The design of algorithms for this purpose is an area of ongoing research and a subject of some complexity. Accordingly, this area is beyond our scope; see [ZHUR12] for a recent survey.

## 10.2 REAL-TIME SCHEDULING

### Background

Real-time computing is becoming an increasingly important discipline. The operating system, and in particular the scheduler, is perhaps the most important component of a real-time system. Examples of current applications of real-time systems include control of laboratory experiments, process control in industrial plants, robotics, air traffic control, telecommunications, and military command and control systems. Next-generation systems will include the autonomous land rover, controllers of robots with elastic joints, systems found in intelligent manufacturing, the space station, and undersea exploration.

Real-time computing may be defined as that type of computing in which the correctness of the system depends not only on the logical result of the computation, but also on the time at which the results are produced. We can define a real-time system by defining what is meant by a real-time process, or task.[3] In general, in a real-time system, some of the tasks are real-time tasks, and these have a certain degree of urgency to them. Such tasks are attempting to control or react to events that take place in the outside world. Because these events occur in "real time," a real-time task must be able to keep up with the events with which it is concerned. Thus, it is usually possible to associate a deadline with a particular task, where the deadline specifies either a start time or a completion time. Such a task may be classified as hard or soft. A **hard real-time task** is one that must meet its deadline; otherwise it will cause unacceptable damage or a fatal error to the system. A **soft real-time task** has an associated deadline that is desirable but not mandatory; it still makes sense to schedule and complete the task even if it has passed its deadline.

Another characteristic of real-time tasks is whether they are periodic or aperiodic. An **aperiodic task** has a deadline by which it must finish or start, or it may have a constraint on both start and finish time. In the case of a **periodic task**, the requirement may be stated as "once per period $T$" or "exactly $T$ units apart."

[3]As usual, terminology poses a problem, because various words are used in the literature with varying meanings. It is common for a particular process to operate under real-time constraints of a repetitive nature. That is, the process lasts for a long time and, during that time, performs some repetitive function in response to real-time events. Let us, for this section, refer to an individual function as a task. Thus, the process can be viewed as progressing through a sequence of tasks. At any given time, the process is engaged in a single task, and it is the process/task that must be scheduled.

## Characteristics of Real-Time Operating Systems

Real-time operating systems can be characterized as having unique requirements in five general areas [MORG92]:

1. Determinism
2. Responsiveness
3. User control
4. Reliability
5. Fail-soft operation

An operating system is **deterministic** to the extent that it performs operations at fixed, predetermined times or within predetermined time intervals. When multiple processes are competing for resources and processor time, no system will be fully deterministic. In a real-time operating system, process requests for service are dictated by external events and timings. The extent to which an operating system can deterministically satisfy requests depends first on the speed with which it can respond to interrupts and, second, on whether the system has sufficient capacity to handle all requests within the required time.

One useful measure of the ability of an operating system to function deterministically is the maximum delay from the arrival of a high-priority device interrupt to when servicing begins. In non-real-time operating systems, this delay may be in the range of tens to hundreds of milliseconds, while in real-time operating systems that delay may have an upper bound of anywhere from a few microseconds to a millisecond.

A related but distinct characteristic is **responsiveness**. Determinism is concerned with how long an operating system delays before acknowledging an interrupt. Responsiveness is concerned with how long, after acknowledgment, it takes an operating system to service the interrupt. Aspects of responsiveness include the following:

1. The amount of time required to initially handle the interrupt and begin execution of the interrupt service routine (ISR). If execution of the ISR requires a process switch, then the delay will be longer than if the ISR can be executed within the context of the current process.
2. The amount of time required to perform the ISR. This generally is dependent on the hardware platform.
3. The effect of interrupt nesting. If an ISR can be interrupted by the arrival of another interrupt, then the service will be delayed.

Determinism and responsiveness together make up the response time to external events. Response time requirements are critical for real-time systems, because such systems must meet timing requirements imposed by individuals, devices, and data flows external to the system.

**User control** is generally much broader in a real-time operating system than in ordinary operating systems. In a typical non-real-time operating system, the user either has no control over the scheduling function of the operating system, or can only provide broad guidance, such as grouping users into more than one priority

class. In a real-time system, however, it is essential to allow the user fine-grained control over task priority. The user should be able to distinguish between hard and soft tasks and to specify relative priorities within each class. A real-time system may also allow the user to specify such characteristics as the use of paging or process swapping, what processes must always be resident in main memory, what disk transfer algorithms are to be used, what rights the processes in various priority bands have, and so on.

**Reliability** is typically far more important for real-time systems than non-real-time systems. A transient failure in a non-real-time system may be solved by simply rebooting the system. A processor failure in a multiprocessor non-real-time system may result in a reduced level of service until the failed processor is repaired or replaced. But a real-time system is responding to and controlling events in real time. Loss or degradation of performance may have catastrophic consequences, ranging from financial loss to major equipment damage and even loss of life.

As in other areas, the difference between a real-time and a non-real-time operating system is one of degree. Even a real-time system must be designed to respond to various failure modes. **Fail-soft operation** is a characteristic that refers to the ability of a system to fail in such a way as to preserve as much capability and data as possible. For example, a typical traditional UNIX system, when it detects a corruption of data within the kernel, issues a failure message on the system console, dumps the memory contents to disk for later failure analysis, and terminates execution of the system. In contrast, a real-time system will attempt either to correct the problem or minimize its effects while continuing to run. Typically, the system notifies a user or user process that it should attempt corrective action then continues operation perhaps at a reduced level of service. In the event a shutdown is necessary, an attempt is made to maintain file and data consistency.

An important aspect of fail-soft operation is referred to as stability. A real-time system is stable if, in cases where it is impossible to meet all task deadlines, the system will meet the deadlines of its most critical, highest-priority tasks, even if some less critical task deadlines are not always met.

Although there is a wide variety of real-time OS designs to meet the wide variety of real-time applications, the following features are common to most real-time OSs:

- A stricter use of priorities than in an ordinary OS, with preemptive scheduling that is designed to meet real-time requirements
- Interrupt latency (the amount of time between when a device generates an interrupt and when that device is serviced) is bounded and relatively short
- More precise and predictable timing characteristics than general purpose OSs

The heart of a real-time system is the short-term task scheduler. In designing such a scheduler, fairness and minimizing average response time are not paramount. What is important is that all hard real-time tasks complete (or start) by their deadline and that as many as possible soft real-time tasks also complete (or start) by their deadline.

Most contemporary real-time operating systems are unable to deal directly with deadlines. Instead, they are designed to be as responsive as possible to real-time

tasks so when a deadline approaches, a task can be quickly scheduled. From this point of view, real-time applications typically require deterministic response times in the several-millisecond to submillisecond span under a broad set of conditions; leading-edge applications (in simulators for military aircraft, for example) often have constraints in the range of 10–100 $\mu$s [ATLA89].

Figure 10.4 illustrates a spectrum of possibilities. In a preemptive scheduler that uses simple round-robin scheduling, a real-time task would be added to the ready queue to await its next timeslice, as illustrated in Figure 10.4a. In this case, the scheduling time will generally be unacceptable for real-time applications. Alternatively, in a nonpreemptive scheduler, we could use a priority scheduling mechanism, giving real-time tasks higher priority. In this case, a real-time task that is ready would be scheduled as soon as the current process blocks or runs to completion (see Figure 10.4b). This could lead to a delay of several seconds if a slow, low-priority task were executing at a critical time. Again, this approach is not acceptable. A more promising approach is to combine priorities with clock-based interrupts. Preemption points occur at regular intervals. When a preemption point occurs, the currently running task is preempted if a higher-priority task is waiting. This would include the preemption of tasks that are part of the operating system kernel. Such a delay may be on the order of several milliseconds (see Figure 10.4c). While this last approach may be adequate for some real-time applications, it will not suffice for more demanding applications. In those cases, the approach that has been taken is sometimes referred to as immediate preemption. In this case, the operating system responds to an interrupt almost immediately, unless the system is in a critical-code lockout section. Scheduling delays for a real-time task can then be reduced to 100 $\mu$s or less.

## Real-Time Scheduling

Real-time scheduling is one of the most active areas of research in computer science. In this subsection, we provide an overview of the various approaches to real-time scheduling and look at two popular classes of scheduling algorithms.

In a survey of real-time scheduling algorithms, [RAMA94] observes that the various scheduling approaches depend on (1) whether a system performs schedulability analysis, (2) if it does, whether it is done statically or dynamically, and (3) whether the result of the analysis itself produces a schedule or plan according to which tasks are dispatched at run time. Based on these considerations, the authors identify the following classes of algorithms:

- **Static table-driven approaches:** These perform a static analysis of feasible schedules of dispatching. The result of the analysis is a schedule that determines, at run time, when a task must begin execution.
- **Static priority-driven preemptive approaches:** Again, a static analysis is performed, but no schedule is drawn up. Rather, the analysis is used to assign priorities to tasks, so a traditional priority-driven preemptive scheduler can be used.
- **Dynamic planning-based approaches:** Feasibility is determined at run time (dynamically) rather than offline prior to the start of execution (statically).

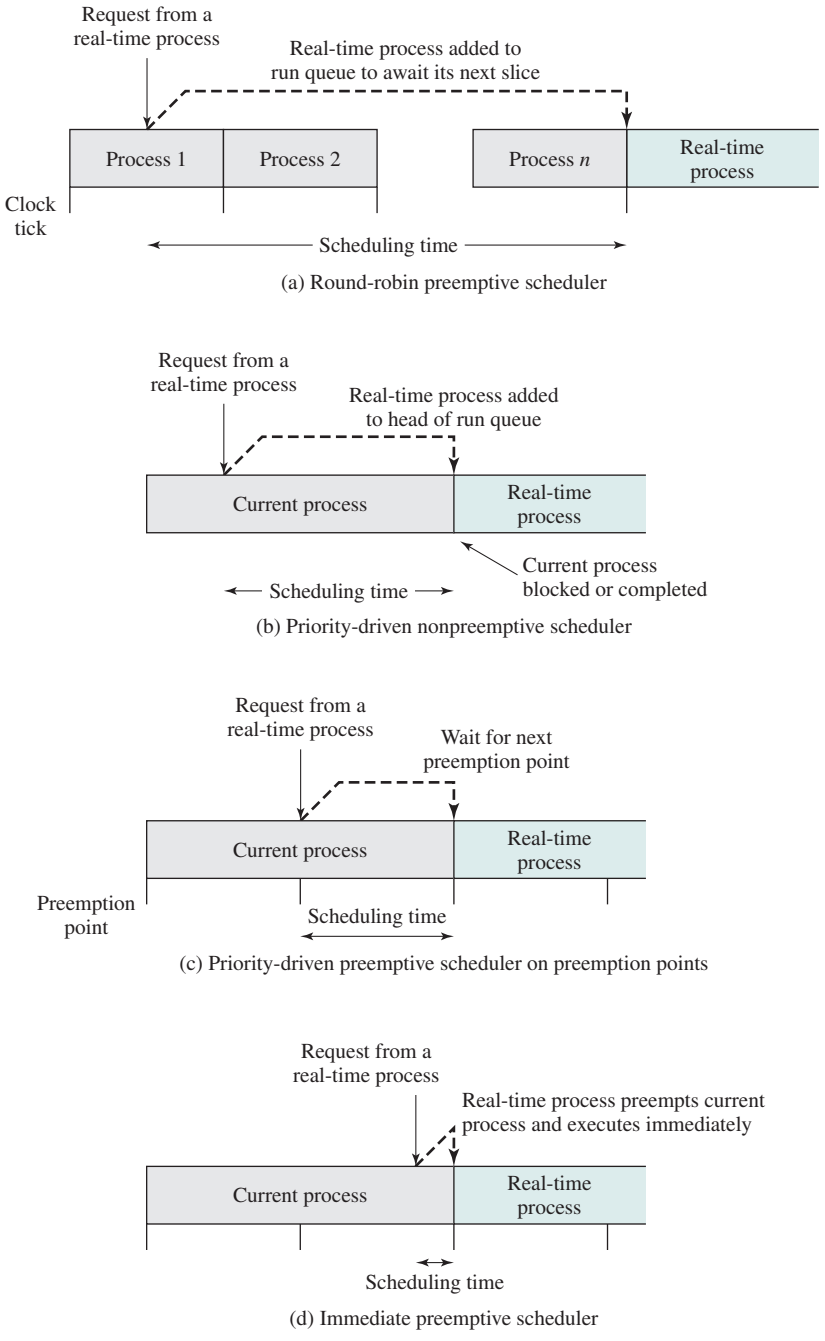



**Figure 10.4 Scheduling of Real-Time Process**

An arriving task is accepted for execution only if it is feasible to meet its time constraints. One of the results of the feasibility analysis is a schedule or plan that is used to decide when to dispatch this task.

- **Dynamic best effort approaches:** No feasibility analysis is performed. The system tries to meet all deadlines and aborts any started process whose deadline is missed.

**Static table-driven scheduling** is applicable to tasks that are periodic. Input to the analysis consists of the periodic arrival time, execution time, periodic ending deadline, and relative priority of each task. The scheduler attempts to develop a schedule that enables it to meet the requirements of all periodic tasks. This is a predictable approach but one that is inflexible, because any change to any task requirements requires that the schedule be redone. Earliest-deadline-first or other periodic deadline techniques (discussed subsequently) are typical of this category of scheduling algorithms.

**Static priority-driven preemptive scheduling** makes use of the priority-driven preemptive scheduling mechanism common to most non-real-time multiprogramming systems. In a non-real-time system, a variety of factors might be used to determine priority. For example, in a time-sharing system, the priority of a process changes depending on whether it is processor bound or I/O bound. In a real-time system, priority assignment is related to the time constraints associated with each task. One example of this approach is the rate monotonic algorithm (discussed subsequently), which assigns static priorities to tasks based on the length of their periods.

With **dynamic planning-based scheduling**, after a task arrives, but before its execution begins, an attempt is made to create a schedule that contains the previously scheduled tasks as well as the new arrival. If the new arrival can be scheduled in such a way that its deadlines are satisfied and that no currently scheduled task misses a deadline, then the schedule is revised to accommodate the new task.

**Dynamic best effort scheduling** is the approach used by many real-time systems that are currently commercially available. When a task arrives, the system assigns a priority based on the characteristics of the task. Some form of deadline scheduling, such as earliest-deadline scheduling, is typically used. Typically, the tasks are aperiodic, so no static scheduling analysis is possible. With this type of scheduling, until a deadline arrives or until the task completes, we do not know whether a timing constraint will be met. This is the major disadvantage of this form of scheduling. Its advantage is that it is easy to implement.

## Deadline Scheduling

Most contemporary real-time operating systems are designed with the objective of starting real-time tasks as rapidly as possible, and hence emphasize rapid interrupt handling and task dispatching. In fact, this is not a particularly useful metric in evaluating real-time operating systems. Real-time applications are generally not concerned with sheer speed but rather with completing (or starting) tasks at the most valuable times, neither too early nor too late, despite dynamic resource demands and conflicts, processing overloads, and hardware or software faults. It follows that priorities provide a crude tool and do not capture the requirement of completion (or initiation) at the most valuable time.

There have been a number of proposals for more powerful and appropriate approaches to real-time task scheduling. All of these are based on having additional information about each task. In its most general form, the following information about each task might be used:

- **Ready time:** Time at which task becomes ready for execution. In the case of a repetitive or periodic task, this is actually a sequence of times that is known in advance. In the case of an aperiodic task, this time may be known in advance, or the operating system may only be aware when the task is actually ready.
- **Starting deadline:** Time by which a task must begin
- **Completion deadline:** Time by which a task must be completed. The typical real-time application will either have starting deadlines or completion deadlines, but not both.
- **Processing time:** Time required to execute the task to completion. In some cases, this is supplied. In others, the operating system measures an exponential average (as defined in Chapter 9). For still other scheduling systems, this information is not used.
- **Resource requirements:** Set of resources (other than the processor) required by the task while it is executing
- **Priority:** Measures relative importance of the task. Hard real-time tasks may have an "absolute" priority, with the system failing if a deadline is missed. If the system is to continue to run no matter what, then both hard and soft real-time tasks may be assigned relative priorities as a guide to the scheduler.
- **Subtask structure:** A task may be decomposed into a mandatory subtask and an optional subtask. Only the mandatory subtask possesses a hard deadline.

There are several dimensions to the real-time scheduling function when deadlines are taken into account: which task to schedule next and what sort of preemption is allowed. It can be shown, for a given preemption strategy and using either starting or completion deadlines, that a policy of scheduling the task with the earliest deadline minimizes the fraction of tasks that miss their deadlines [BUTT99, HONG89, PANW88]. This conclusion holds for both single-processor and multiprocessor configurations.

The other critical design issue is that of preemption. When starting deadlines are specified, then a nonpreemptive scheduler makes sense. In this case, it would be the responsibility of the real-time task to block itself after completing the mandatory or critical portion of its execution, allowing other real-time starting deadlines to be satisfied. This fits the pattern of Figure 10.4b. For a system with completion deadlines, a preemptive strategy (see Figure 10.4c or 10.4d) is most appropriate. For example, if task X is running and task Y is ready, there may be circumstances in which the only way to allow both X and Y to meet their completion deadlines is to preempt X, execute Y to completion, then resume X to completion.

As an example of scheduling periodic tasks with completion deadlines, consider a system that collects and processes data from two sensors, A and B. The deadline for collecting data from sensor A must be met every 20 ms, and that for B every 50 ms. It takes 10 ms, including operating system overhead, to process each sample of data from A and 25 ms to process each sample of data from B. Table 10.3 summarizes

**Table 10.3** Execution Profile of Two Periodic Tasks

| Process | Arrival Time | Execution Time | Ending Deadline |
|---|---|---|---|
| A(1) | 0 | 10 | 20 |
| A(2) | 20 | 10 | 40 |
| A(3) | 40 | 10 | 60 |
| A(4) | 60 | 10 | 80 |
| A(5) | 80 | 10 | 100 |
| • | • | • | • |
| • | • | • | • |
| • | • | • | • |
| B(1) | 0 | 25 | 50 |
| B(2) | 50 | 25 | 100 |
| • | • | • | • |
| • | • | • | • |
| • | • | • | • |

the execution profile of the two tasks. Figure 10.5 compares three scheduling techniques using the execution profile of Table 10.3. The first row of Figure 10.6 repeats the information in Table 10.3; the remaining three rows illustrate three scheduling techniques.

The computer is capable of making a scheduling decision every 10 ms.[4] Suppose under these circumstances, we attempted to use a priority scheduling scheme. The first two timing diagrams in Figure 10.5 show the result. If A has higher priority, the first instance of task B is given only 20 ms of processing time, in two 10-ms chunks, by the time its deadline is reached, and thus fails. If B is given higher priority, then A will miss its first deadline. The final timing diagram shows the use of earliest-deadline scheduling. At time $t = 0$, both A1 and B1 arrive. Because A1 has the earliest deadline, it is scheduled first. When A1 completes, B1 is given the processor. At $t = 20$, A2 arrives. Because A2 has an earlier deadline than B1, B1 is interrupted so A2 can execute to completion. Then B1 is resumed at $t = 30$. At $t = 40$, A3 arrives. However, B1 has an earlier ending deadline and is allowed to execute to completion at $t = 45$. A3 is then given the processor and finishes at $t = 55$.

In this example, by scheduling to give priority at any preemption point to the task with the nearest deadline, all system requirements can be met. Because the tasks are periodic and predictable, a static table-driven scheduling approach is used.

Now consider a scheme for dealing with aperiodic tasks with starting deadlines. The top part of Figure 10.6 shows the arrival times and starting deadlines for an example consisting of five tasks, each of which has an execution time of 20 ms. Table 10.4 summarizes the execution profile of the five tasks.

[4]This need not be on a 10-ms boundary if more than 10 ms has elapsed since the last scheduling decision.

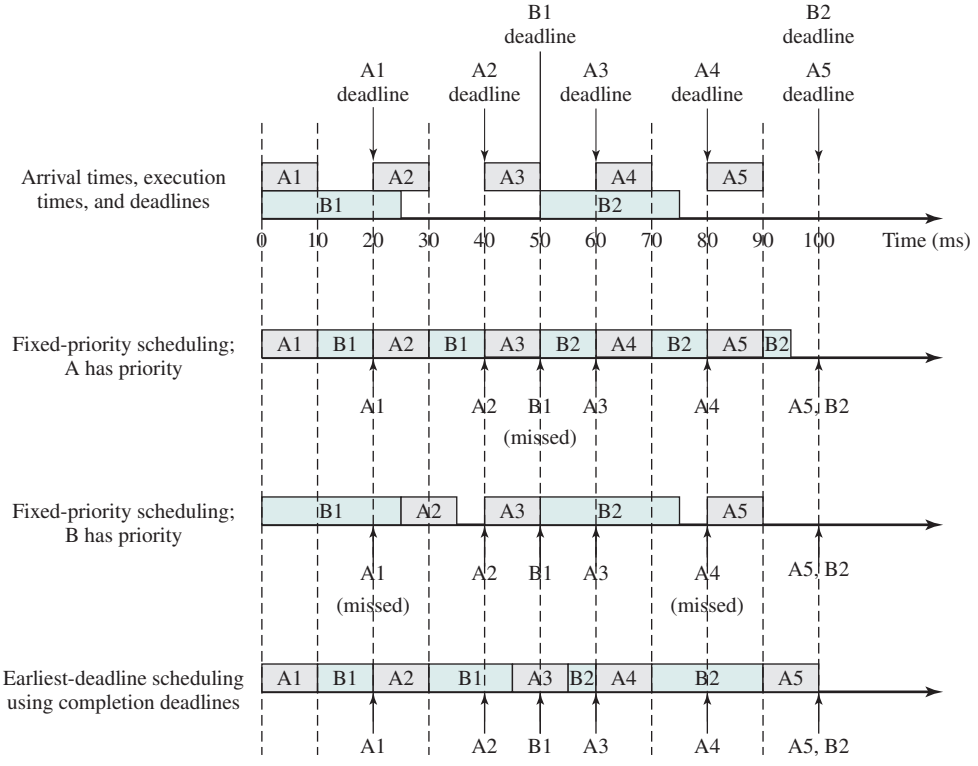



**Figure 10.5** **Scheduling of Periodic Real-Time Tasks with Completion Deadlines (Based on Table 10.3)**

A straightforward scheme is to always schedule the ready task with the earliest deadline and let that task run to completion. When this approach is used in the example of Figure 10.6, note although task B requires immediate service, the service is denied. This is the risk in dealing with aperiodic tasks, especially with starting deadlines. A refinement of the policy will improve performance if deadlines can be known in advance of the time that a task is ready. This policy, referred to as earliest deadline with unforced idle times, operates as follows: Always schedule the eligible task with the earliest deadline and let that task run to completion. An eligible task may not be ready, and this may result in the processor remaining idle even though there are ready tasks. Note in our example the system refrains from scheduling task A even though that is the only ready task. The result is, even though the processor is not used to maximum efficiency, all scheduling requirements are met. Finally, for comparison, the FCFS policy is shown. In this case, tasks B and E do not meet their deadlines.

## Rate Monotonic Scheduling

One of the more promising methods of resolving multitask scheduling conflicts for periodic tasks is rate monotonic scheduling (RMS) [LIU73, BRIA99, SHA94]. RMS assigns priorities to tasks on the basis of their periods.

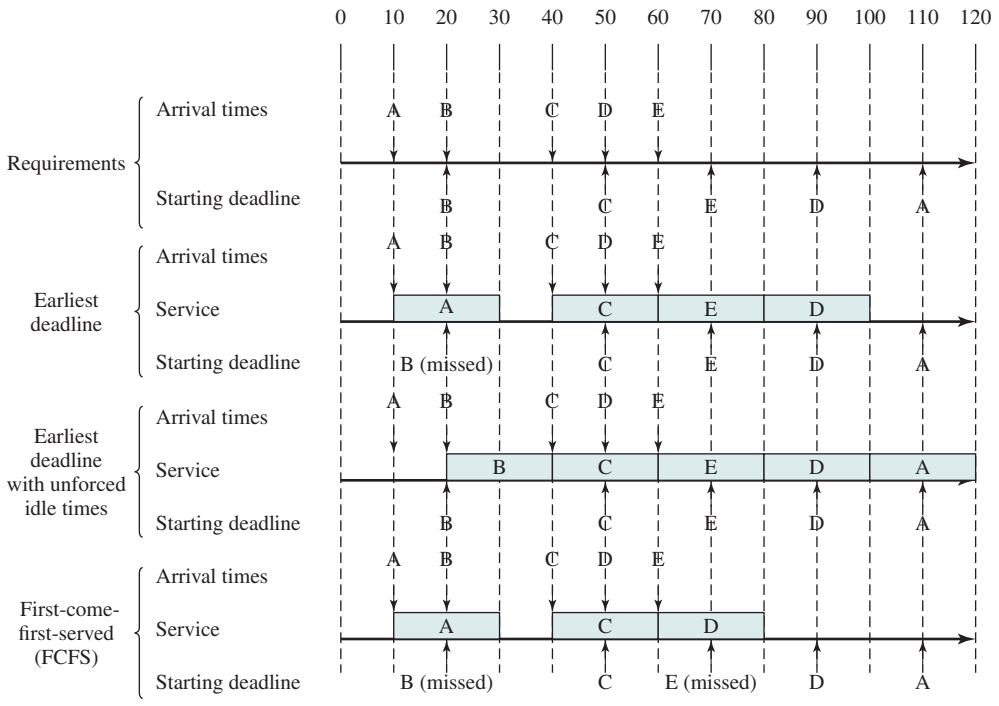



**Figure 10.6** **Scheduling of Aperiodic Real-Time Tasks with Starting Deadlines**

For RMS, the highest-priority task is the one with the shortest period, the second highest-priority task is the one with the second shortest period, and so on. When more than one task is available for execution, the one with the shortest period is serviced first. If we plot the priority of tasks as a function of their rate, the result is a monotonically increasing function, hence the name "rate monotonic scheduling."

Figure 10.7 illustrates the relevant parameters for periodic tasks. The task's period, $T$, is the amount of time between the arrival of one instance of the task and the arrival of the next instance of the task. A task's rate (in hertz) is simply the inverse of its period (in seconds). For example, a task with a period of 50 ms occurs at a rate of 20 Hz. Typically, the end of a task's period is also the task's hard deadline, although some tasks may have earlier deadlines. The execution (or computation) time, $C$, is the amount of processing time required for each occurrence of the task. It

**Table 10.4** Execution Profile of Five Aperiodic Tasks

| Process | Arrival Time | Execution Time | Starting Deadline |
|---|---|---|---|
| A | 10 | 20 | 110 |
| B | 20 | 20 | 20 |
| C | 40 | 20 | 50 |
| D | 50 | 20 | 90 |
| E | 60 | 20 | 70 |

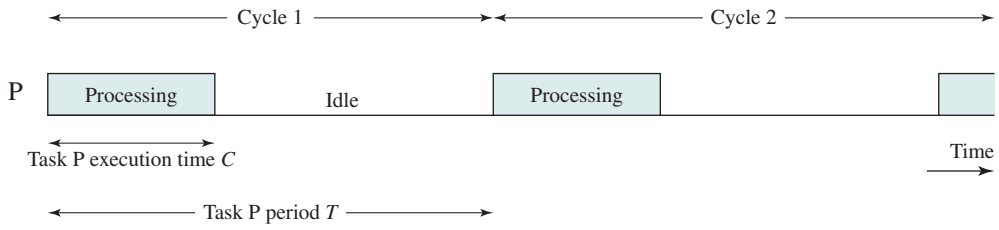
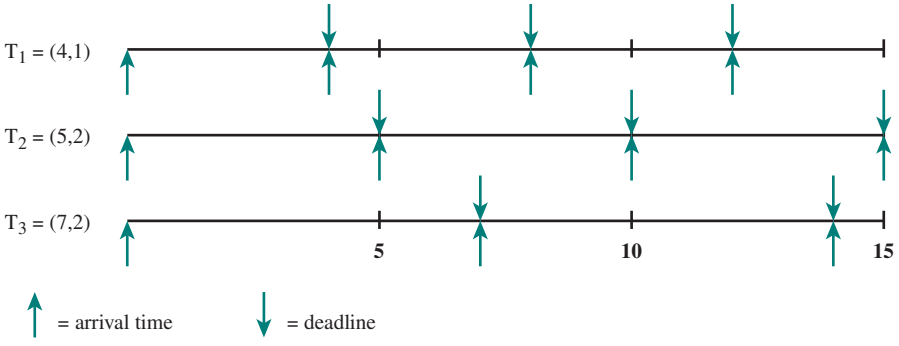
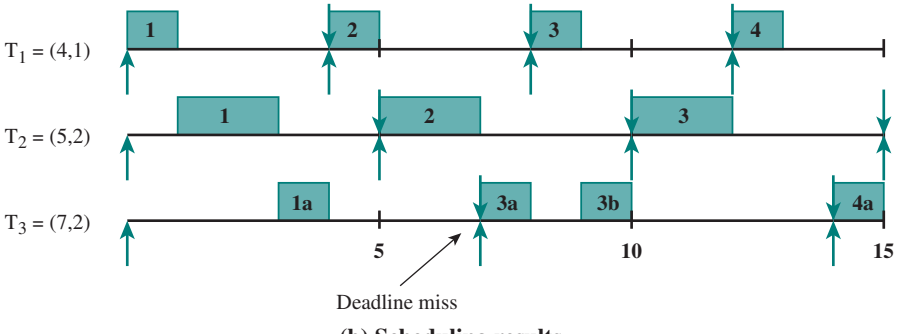



**Figure 10.7 Periodic Task Timing Diagram**

should be clear that in a uniprocessor system, the execution time must be no greater than the period (must have $C \leq T$). If a periodic task is always run to completion, that is, if no instance of the task is ever denied service because of insufficient resources, then the utilization of the processor by this task is $U = C/T$. For example, if a task has a period of 80 ms and an execution time of 55 ms, its processor utilization is 55/80 = 0.6875. Figure 10.8 is a simple example of RMS. Task instances are numbered sequentially within tasks. As can be seen, for task 3, the second instance is not executed because the deadline is missed. The third instance experiences a preemption but is still able to complete before the deadline.




**(a) Arrival times and deadines for task $T_i = (P_i, C_i)$; $P_i$ = period, $C_i$ = processing time**




**(b) Scheduling results**

**Figure 10.8 Rate Monotonic Scheduling Example**

One measure of the effectiveness of a periodic scheduling algorithm is whether or not it guarantees that all hard deadlines are met. Suppose we have $n$ tasks, each with a fixed period and execution time. Then for it to be possible to meet all deadlines, the following inequality must hold:

$$\frac{C_1}{T_1} + \frac{C_2}{T_2} + \ \ldots \ + \frac{C_n}{T_n} \leq 1 \tag{10.1}$$

The sum of the processor utilizations of the individual tasks cannot exceed a value of 1, which corresponds to total utilization of the processor. Equation (10.1) provides a bound on the number of tasks that a perfect scheduling algorithm can successfully schedule. For any particular algorithm, the bound may be lower. For RMS, it can be shown that the following inequality holds:

$$\frac{C_1}{T_1} + \frac{C_2}{T_2} + \ \ldots \ + \frac{C_n}{T_n} \leq n(2^{1/n} - 1) \tag{10.2}$$

Table 10.5 gives some values for this upper bound. As the number of tasks increases, the scheduling bound converges to $\ln 2 \approx 0.693$.

As an example, consider the case of three periodic tasks, where $U_i = C_i/T_i$:

- **Task $P_1$:** $C_1 = 20$; $T_1 = 100$; $U_1 = 0.2$
- **Task $P_2$:** $C_2 = 40$; $T_2 = 150$; $U_2 = 0.267$
- **Task $P_3$:** $C_3 = 100$; $T_3 = 350$; $U_3 = 0.286$

The total utilization of these three tasks is $0.2 + 0.267 + 0.286 = 0.753$. The upper bound for the schedulability of these three tasks using RMS is

$$\frac{C_1}{T_1} + \frac{C_2}{T_2} + \frac{C_3}{T_3} \leq n(2^{1/3} - 1) = 0.779$$

Because the total utilization required for the three tasks is less than the upper bound for RMS ($0.753 < 0.779$), we know if RMS is used, all tasks will be successfully scheduled.

**Table 10.5** Value of the RMS Upper Bound

| n | $n(2^{1/n} - 1)$ |
|---|---|
| 1 | 1.0 |
| 2 | 0.828 |
| 3 | 0.779 |
| 4 | 0.756 |
| 5 | 0.743 |
| 6 | 0.734 |
| • | • |
| • | • |
| • | • |
| $\infty$ | $\ln 2 \approx 0.693$ |

It can also be shown that the upper bound of Equation (10.1) holds for earliest-deadline scheduling. Thus, it is possible to achieve greater overall processor utilization and therefore accommodate more periodic tasks with earliest-deadline scheduling. Nevertheless, RMS has been widely adopted for use in industrial applications. [SHA91] offers the following explanation:

1. The performance difference is small in practice. The upper bound of Equation (10.2) is a conservative one and, in practice, utilization as high as 90% is often achieved.
2. Most hard real-time systems also have soft real-time components, such as certain noncritical displays and built-in self tests that can execute at lower-priority levels to absorb the processor time that is not used with RMS scheduling of hard real-time tasks.
3. Stability is easier to achieve with RMS. When a system cannot meet all deadlines because of overload or transient errors, the deadlines of essential tasks need to be guaranteed provided that this subset of tasks is schedulable. In a static priority assignment approach, one only needs to ensure that essential tasks have relatively high priorities. This can be done in RMS by structuring essential tasks to have short periods or by modifying the RMS priorities to account for essential tasks. With earliest-deadline scheduling, a periodic task's priority changes from one period to another. This makes it more difficult to ensure that essential tasks meet their deadlines.

## Priority Inversion

Priority inversion is a phenomenon that can occur in any priority-based preemptive scheduling scheme, but is particularly relevant in the context of real-time scheduling. The best-known instance of priority inversion involved the Mars Pathfinder mission. This rover robot landed on Mars on July 4, 1997, and began gathering and transmitting voluminous data back to Earth. But a few days into the mission, the lander software began experiencing total system resets, each resulting in losses of data. After much effort by the Jet Propulsion Laboratory (JPL) team that built the Pathfinder, the problem was traced to priority inversion [JONE97].

In any priority scheduling scheme, the system should always be executing the task with the highest priority. **Priority inversion** occurs when circumstances within the system force a higher-priority task to wait for a lower-priority task. A simple example of priority inversion occurs if a lower-priority task has locked a resource (such as a device or a binary semaphore) and a higher-priority task attempts to lock that same resource. The higher-priority task will be put in a blocked state until the resource is available. If the lower-priority task soon finishes with the resource and releases it, the higher-priority task may quickly resume and it is possible that no real-time constraints are violated.

A more serious condition is referred to as an **unbounded priority inversion**, in which the duration of a priority inversion depends not only on the time required to handle a shared resource but also on the unpredictable actions of other unrelated tasks. The priority inversion experienced in the Pathfinder software was unbounded and serves as a good example of the phenomenon. Our discussion follows that of

[TIME02]. The Pathfinder software included the following three tasks, in decreasing order of priority:

$T_1$: Periodically checks the health of the spacecraft systems and software

$T_2$: Processes image data

$T_3$: Performs an occasional test on equipment status

After $T_1$ executes, it reinitializes a timer to its maximum value. If this timer ever expires, it is assumed the integrity of the lander software has somehow been compromised. The processor is halted, all devices are reset, the software is completely reloaded, the spacecraft systems are tested, and the system starts over. This recovery sequence does not complete until the next day. $T_1$ and $T_3$ share a common data structure, protected by a binary semaphore `s`. Figure 10.9a shows the sequence that caused the priority inversion:

$t_1$: $T_3$ begins executing.

$t_2$: $T_3$ locks semaphore `s` and enters its critical section.

$t_3$: $T_1$, which has a higher priority than $T_3$, preempts $T_3$ and begins executing.

$t_4$: $T_1$ attempts to enter its critical section but is blocked because the semaphore is locked by $T_3$; $T_3$ resumes execution in its critical section.

$t_5$: $T_2$, which has a higher priority than $T_3$, preempts $T_3$ and begins executing.

$t_6$: $T_2$ is suspended for some reason unrelated to $T_1$ and $T_3$; $T_3$ resumes.

$t_7$: $T_3$ leaves its critical section and unlocks the semaphore. $T_1$ preempts $T_3$, locks the semaphore, and enters its critical section.

In this set of circumstances, $T_1$ must wait for both $T_3$ and $T_2$ to complete and fails to reset the timer before it expires.

In practical systems, two alternative approaches are used to avoid unbounded priority inversion: priority inheritance protocol and priority ceiling protocol.

The basic idea of **priority inheritance** is that a lower-priority task inherits the priority of any higher-priority task pending on a resource they share. This priority change takes place as soon as the higher-priority task blocks on the resource; it should end when the resource is released by the lower-priority task. Figure 10.9b shows that priority inheritance resolves the problem of unbounded priority inversion illustrated in Figure 10.9a. The relevant sequence of events is as follows:

$t_1$: $T_3$ begins executing.

$t_2$: $T_3$ locks semaphore `s` and enters its critical section.

$t_3$: $T_1$, which has a higher priority than $T_3$, preempts $T_3$ and begins executing.

$t_4$: $T_1$ attempts to enter its critical section but is blocked because the semaphore is locked by $T_3$. $T_3$ is immediately and temporarily assigned the same priority as $T_1$. $T_3$ resumes execution in its critical section.

$t_5$: $T_2$ is ready to execute, but because $T_3$ now has a higher priority, $T_2$ is unable to preempt $T_3$.

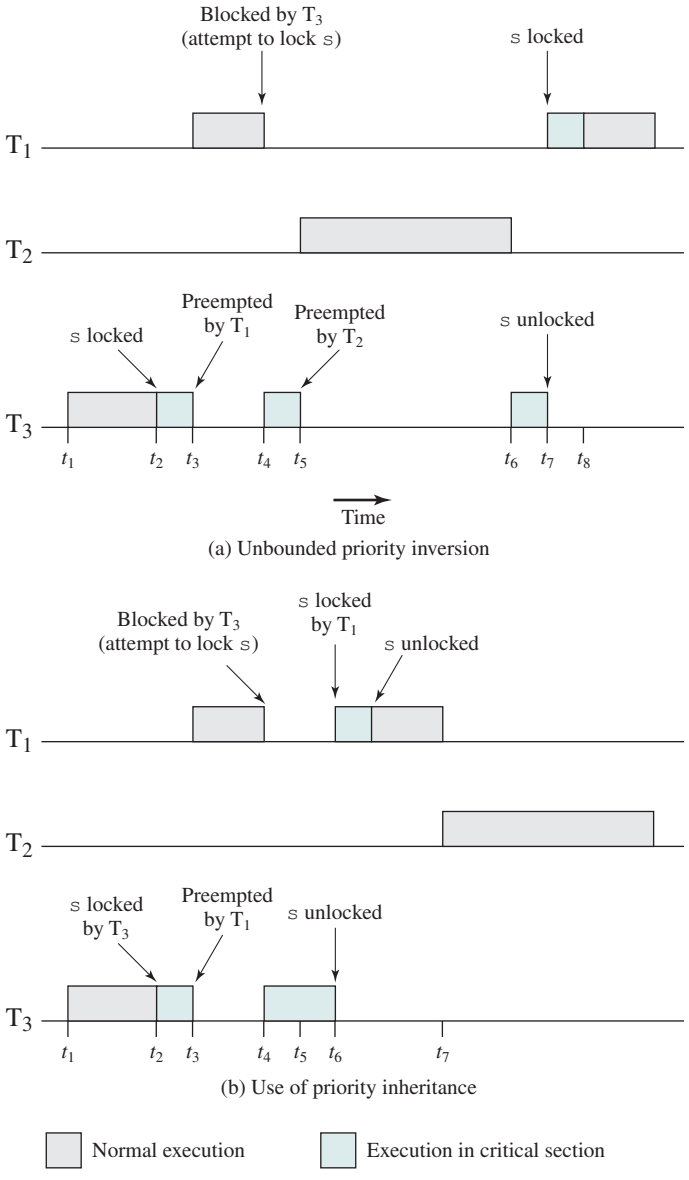



**Figure 10.9 Priority Inversion**

$t_6$: $T_3$ leaves its critical section and unlocks the semaphore: Its priority level is downgraded to its previous default level. $T_1$ preempts $T_3$, locks the semaphore, and enters its critical section.

$t_7$: $T_1$ is suspended for some reason unrelated to $T_2$, and $T_2$ begins executing.

This was the approach taken to solving the Pathfinder problem.

In the **priority ceiling** approach, a priority is associated with each resource. The priority assigned to a resource is one level higher than the priority of its highest-priority

user. The scheduler then dynamically assigns this priority to any task that accesses the resource. Once the task finishes with the resource, its priority returns to normal.

## 10.3 LINUX SCHEDULING

For Linux 2.4 and earlier, Linux provided a real-time scheduling capability coupled with a scheduler for non-real-time processes that made use of the traditional UNIX scheduling algorithm described in Section 9.3. Linux 2.6 includes essentially the same real-time scheduling capability as previous releases, and a substantially revised scheduler for non-real-time processes. We examine these two areas in turn.

### Real-Time Scheduling

The three primary Linux scheduling classes are as follows:

1. **`SCHED_FIFO`:** First-in-first-out real-time threads
2. **`SCHED_RR`:** Round-robin real-time threads
3. **`SCHED_NORMAL`:** Other, non-real-time threads (was called `SCHED_OTHER` in older kernels).

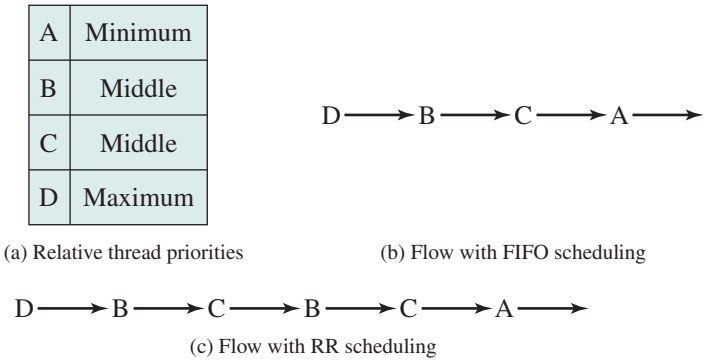
Within each class, multiple priorities may be used, with priorities in the real-time classes higher than the priorities for the `SCHED_NORMAL` class. The default values are as follows: Real-time priority classes range from 0 to 99 inclusively, and `SCHED_NORMAL` classes range from 100 to 139. A lower number equals a higher priority.

For FIFO threads, the following rules apply:

1. The system will not interrupt an executing FIFO thread except in the following cases:
   a. Another FIFO thread of higher priority becomes ready.
   b. The executing FIFO thread becomes blocked waiting for an event, such as I/O.
   c. The executing FIFO thread voluntarily gives up the processor following a call to the primitive `sched_yield`.
2. When an executing FIFO thread is interrupted, it is placed in the queue associated with its priority.
3. When a FIFO thread becomes ready, and if that thread has a higher priority than the currently executing thread, then the currently executing thread is preempted and the highest-priority ready FIFO thread is executed. If more than one thread has that highest priority, the thread that has been waiting the longest is chosen.

The `SCHED_RR` policy is similar to the `SCHED_FIFO` policy, except for the addition of a timeslice associated with each thread. When a `SCHED_RR` thread has executed for its timeslice, it is suspended and a real-time thread of equal or higher priority is selected for running.

Figure 10.10 is an example that illustrates the distinction between FIFO and RR scheduling. Assume a process has four threads with three relative priorities assigned




(a) Relative thread priorities

(b) Flow with FIFO scheduling

(c) Flow with RR scheduling

**Figure 10.10 Example of Linux Real-Time Scheduling**

as shown in Figure 10.10a. Assume all waiting threads are ready to execute when the current thread waits or terminates, and no higher-priority thread is awakened while a thread is executing. Figure 10.10b shows a flow in which all of the threads are in the `SCHED_FIFO` class. Thread D executes until it waits or terminates. Next, although threads B and C have the same priority, thread B starts because it has been waiting longer than thread C. Thread B executes until it waits or terminates, then thread C executes until it waits or terminates. Finally, thread A executes.

Figure 10.10c shows a sample flow if all of the threads are in the `SCHED_RR` class. Thread D executes until it waits or terminates. Next, threads B and C are time sliced, because they both have the same priority. Finally, thread A executes.

The final scheduling class is `SCHED_NORMAL`. A thread in this class can only execute if there are no real-time threads ready to execute.

## Non-Real-Time Scheduling

The Linux 2.4 scheduler for the `SCHED_OTHER` (now called `SCHED_NORMAL`) class did not scale well with increasing number of processors and increasing number of processes. The drawbacks of this scheduler include the following:

- The Linux 2.4 scheduler uses a single runqueue for all processors in a symmetric multiprocessing system (SMP). This means a task can be scheduled on any processor, which can be good for load balancing but bad for memory caches. For example, suppose a task executed on CPU-1, and its data were in that processor's cache. If the task got rescheduled to CPU-2, its data would need to be invalidated in CPU-1 and brought into CPU-2.
- The Linux 2.4 scheduler uses a single runqueue lock. Thus, in an SMP system, the act of choosing a task to execute locks out any other processor from manipulating the runqueues. The result is idle processors awaiting release of the runqueue lock and decreased efficiency.
- Preemption is not possible in the Linux 2.4 scheduler; this means that a lower-priority task can execute while a higher-priority task waited for it to complete.

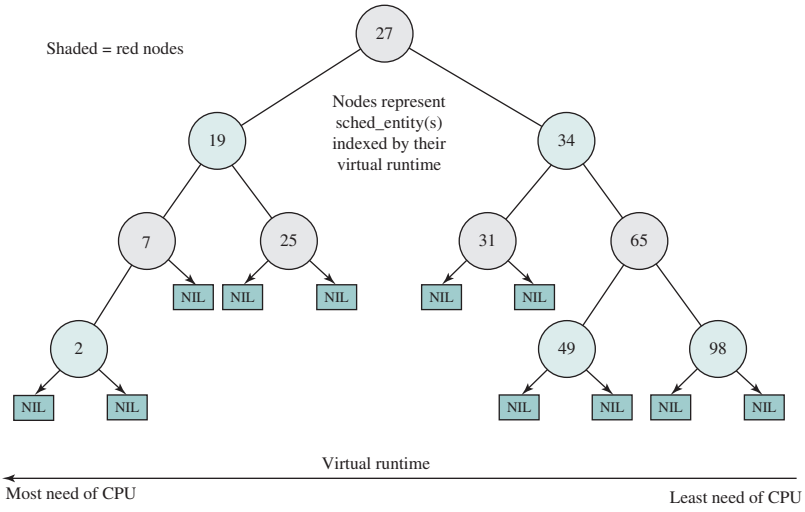



**Figure 10.11 Example of Red Black Tree for CFS**

To correct these problems, Linux 2.6 uses a completely new priority scheduler known as the O(1) scheduler.[5] The scheduler is designed so the time to select the appropriate process and assign it to a processor is constant, regardless of the load on the system or the number of processors. However, the O(1) scheduler proved to be unwieldy in the kernel. The amount of code is large and the algorithms are complex.

As a result of the drawbacks of the O(1) scheduler, from Linux 2.6.23, a new scheduler, called the Completely Fair Scheduler (CFS), is being used [PABL04]. The CFS models an ideal multitasking CPU on real hardware that provides fair access to all tasks. In order to achieve this goal, the CFS maintains a *virtual runtime* value for each task. The virtual runtime is the amount of time spent executing so far, normalized by the number of runnable processes. The smaller a task's virtual runtime is (i.e., the smaller the amount of time a task has been permitted access to the processor), the higher is its need for the processor. The CFS also includes the concept of sleeper fairness to ensure that tasks that are not currently runnable (e.g., waiting for I/O) receive a comparable share of the processor when they eventually need it.

The CFS scheduler is implemented by the *fair_sched_class* scheduler class. It is based on using a Red Black tree, as opposed to other schedulers, which are typically based on run queues. A Red Black tree is a type of self-balancing binary search tree that obeys the following rules:

1. A node is either red or black.
2. The root is black.

[5]The term *O(1)* is an example of the "big-O" notation, used for characterizing the time complexity of algorithms. Appendix I explains this notation.

3. All leaves (NIL) are black.
4. If a node is red, then both its children are black.
5. Every path from a given node to any of its descendant NIL nodes contains the same number of black nodes.

This scheme provides high efficiency in inserting, deleting, and searching tasks, due to its O(log N) complexity.

Figure 10.11 illustrates a Red Black Tree. The Linux RB tree contains information about runnable processes. The *rb_node* elements of the tree are embedded in *sched_entity* object. The Red Black tree is ordered by *vruntime*, where the leftmost node of the tree represents the process that has the lowest *vruntime*, and that has the highest need for CPU; this node is the first one to be picked by the CPU, and when it runs, its *vruntime* is updated according to the time it consumed; so when it is inserted back into the tree, very likely it will no longer be the one with the lowest *vruntime*, and the tree will be rebalanced to reflect the current state. And this process of rebalancing will continue with the next process to be picked by the CFS scheduler, and so on. In this way, the CFS manages a fair scheduling policy.

While an insert operation is being performed in RB tree, the leftmost child value is cached in sched_entity for fast lookup.

In Kernel 2.6.24, a new feature called *CFS group scheduling* was introduced. This feature allows the kernel to schedule a group of tasks as if they were a single task. Group scheduling is designed to provide fairness when a task spawns many other tasks.

## 10.4 UNIX SVR4 SCHEDULING

The scheduling algorithm used in UNIX SVR4 is a complete overhaul of the scheduling algorithm used in earlier UNIX systems (described in Section 9.3). The new algorithm is designed to give highest preference to real-time processes, next-highest preference to kernel-mode processes, and lowest preference to other user-mode processes, referred to as time-shared processes.[6]

The two major modifications implemented in SVR4 are as follows:

1. The addition of a preemptable static priority scheduler and the introduction of a set of 160 priority levels divided into three priority classes.
2. The insertion of preemption points. Because the basic kernel is not preemptive, it can only be split into processing steps that must run to completion without interruption. In between the processing steps, safe places known as preemption points have been identified where the kernel can safely interrupt processing and schedule a new process. A safe place is defined as a region of code where all kernel data structures are either updated and consistent, or locked via a semaphore.

[6]Time-shared processes are the processes that correspond to users in a traditional time-sharing system.

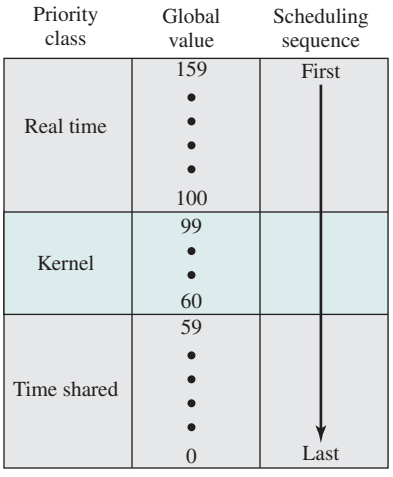



**Figure 10.12 SVR4 Dispatch Queues**

Figure 10.12 illustrates the 160 priority levels defined in SVR4. Each process is defined to belong to one of three priority classes and is assigned a priority level within that class. The classes are as follows:

- **Real time (159-100):** Processes at these priority levels are guaranteed to be selected to run before any kernel or time-sharing process. In addition, real-time processes can make use of preemption points to preempt kernel processes and user processes.
- **Kernel (99-60):** Processes at these priority levels are guaranteed to be selected to run before any time-sharing process, but must defer to real-time processes.
- **Time-shared (59-0):** The lowest-priority processes, intended for user applications other than real-time applications.

Figure 10.13 indicates how scheduling is implemented in SVR4. A dispatch queue is associated with each priority level, and processes at a given priority level are executed in round-robin fashion. A bit-map vector, `dqactmap`, contains one bit for each priority level; the bit is set to one for any priority level with a nonempty




**Figure 10.13 SVR4 Priority Classes**

queue. Whenever a running process leaves the Running state, due to a block, timeslice expiration, or preemption, the dispatcher checks `dqactmap` and dispatches a ready process from the highest-priority nonempty queue. In addition, whenever a defined preemption point is reached, the kernel checks a flag called `kprunrun`. If set, this indicates at least one real-time process is in the Ready state, and the kernel preempts the current process if it is of lower priority than the highest-priority real-time ready process.

Within the time-sharing class, the priority of a process is variable. The scheduler reduces the priority of a process each time it uses up a time quantum, and it raises its priority if it blocks on an event or resource. The time quantum allocated to a time-sharing process depends on its priority, ranging from 100 ms for priority 0 to 10 ms for priority 59. Each real-time process has a fixed priority and a fixed-time quantum.

## 10.5 UNIX FREEBSD SCHEDULING

The UNIX FreeBSD scheduler is designed to provide a more efficient operation than previous UNIX schedulers under heavy load and when used on a multiprocessor or multicore platform. The scheduler is quite complex, and here we present an overview of the most significant design features; for more detail, see [MCKU15] and [ROBE03].

### Priority Classes

The underlying priority mechanism in the FreeBSD scheduler is similar to that of UNIX SVR4. For FreeBSD, five priority classes are defined (see Table 10.6); the first two classes are for kernel-mode threads and the remaining classes for user-mode threads. Kernel threads execute code that is compiled into the kernel's load image and operate with the kernel's privileged execution code.

The highest-priority threads are referred to as *bottom-half kernel.* Threads in this class run in the kernel are scheduled based on interrupt priorities. These priorities are set when the corresponding devices are configured and do not change. *Top-half kernel* threads also run in the kernel and execute various kernel functions. These priorities are set based on predefined priorities and never change.

**Table 10.6** FreeBSD Thread Scheduling Classes

| Priority Class | Thread Type | Description |
|---|---|---|
| 0–63 | Bottom-half kernel | Scheduled by interrupts. Can block to await a resource |
| 64–127 | Top-half kernel | Runs until blocked or done. Can block to await a resource |
| 128–159 | Real-time user | Allowed to run until blocked or until a higher-priority thread becomes available. Preemptive scheduling |
| 160–223 | Time-sharing user | Adjusts priorities based on processor usage |
| 224–255 | Idle user | Only run when there are no time sharing or real-time threads to run |
| *Note*: Lower number corresponds to higher priority. | | |

The next lower-priority class is referred to as *real-time user.* A thread with a real-time priority is not subject to priority degradation. That is, a real-time thread maintains the priority it began with and does not drop to a lower priority as a result of using resources. Next comes the *time-sharing user* priority class. For threads in this class, priority is periodically recalculated based on a number of parameters, including the amount of processor time used, the amount of memory resources held, and other resource consumption parameters. The lowest range of priorities is referred to as the *idle user* class. This class is intended for applications that will only consume processor time when no other threads are ready to execute.

## SMP and Multicore Support

The latest version of the FreeBSD scheduler, introduced with FreeBSD 5.0, was designed to provide effective scheduling for an SMP or multicore system. The new scheduler meets three design goals:

1. Address the need for processor affinity in SMP and multicore systems. The term *processor affinity* refers to a scheduler that only migrates a thread (moves thread from one processor to another) when necessary to avoid having an idle processor.
2. Provide better support for multithreading on multicore systems.
3. Improve the performance of the scheduling algorithm, so it is no longer a function of the number of threads in the system.

In this subsection, we look at three key features of the new scheduler: queue structure, interactivity scoring, and thread migration.

***Queue Structure*** The previous version of the FreeBSD schedule used a single global scheduling queue for all processors that it traverses once per second to recalculate their priorities. The use of a single list for all threads means the performance of the scheduler is dependent on the number of tasks in the system, and as the number of tasks grows, more processor time must be spent in the scheduler maintaining the list.

The new scheduler performs scheduling independently for each processor. For each processor, three queues are maintained. Each of the queues has the structure shown in Figure 10.14 for SVR4. Two runqueues implement the kernel, real-time, and time-sharing scheduling classes (priorities 0 through 223). The third queue is only for the idle class (priorities 224 through 255).

The two runqueues are designated *current* and *next.* Every thread that is granted a timeslice (place in the Ready state) is placed in either the current queue or the next queue (as explained subsequently) at the appropriate priority for that thread. The scheduler for a processor selects threads from the current queue in priority order until the current queue is empty. When the current queue is empty, the scheduler swaps the current and next queue, and begins to schedule threads from the new current queue. The use of two runqueues guarantees that each thread will be granted processor time at least once every two queue switches regardless of priority, avoiding starvation.

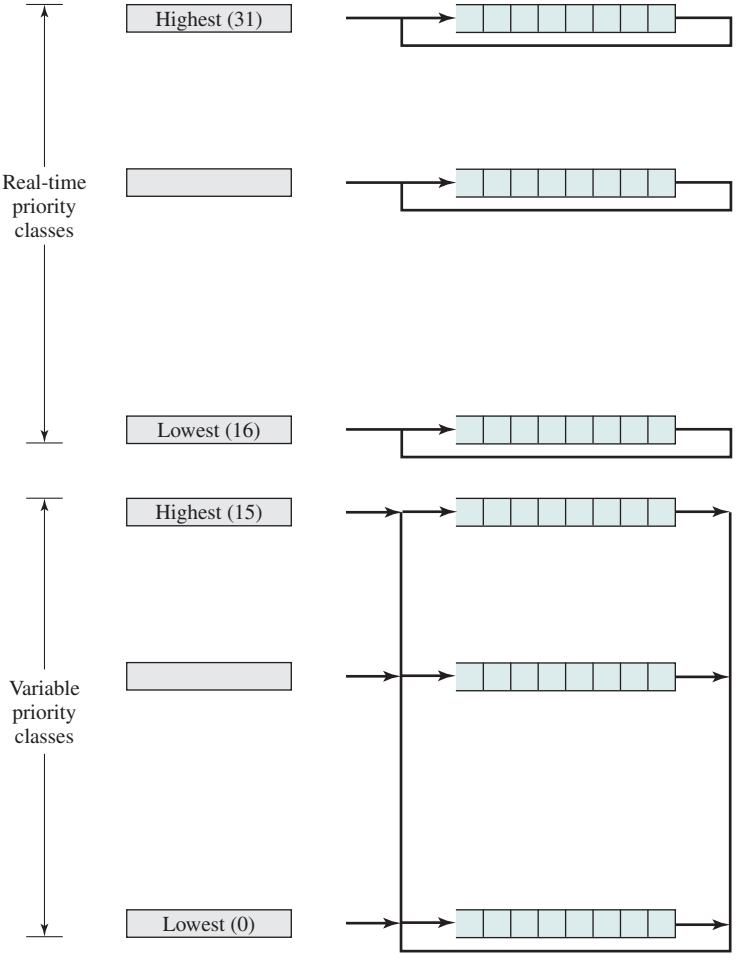



**Figure 10.14 Windows Thread Dispatching Priorities**

Several rules determine the assignment of a thread to either the current queue or the next queue:

1. Kernel and real-time threads are always inserted onto the current queue.
2. A time-sharing thread is assigned to the current queue if it is interactive (explained in the next subsection) or to the next queue otherwise. Inserting interactive threads onto the current queue results in a low interactive response time for such threads, compared to other time-sharing threads that do not exhibit a high degree of interactivity.

***INTERACTIVITY SCORING*** A thread is considered to be **interactive** if the ratio of its voluntary sleep time versus its run time is below a certain threshold. Interactive threads typically have high sleep times as they wait for user input. These sleep

intervals are followed by bursts of processor activity as the thread processes the user's request.

The interactivity threshold is defined in the scheduler code and is not configurable. The scheduler uses two equations to compute the interactivity score of a thread. First, we define a scaling factor:

$$\text{Scaling factor} = \frac{\text{Maximum interactivity score}}{2}$$

For threads whose sleep time exceeds their run time, the following equation is used:

$$\text{Interactivity score} = \text{Scaling factor}\left(\frac{\text{run}}{\text{sleep}}\right)$$

When a thread's run time exceeds its sleep time, the following equation is used instead:

$$\text{Interactivity score} = \text{Scaling factor}\left(1 + \frac{\text{sleep}}{\text{run}}\right)$$

The result is that threads whose sleep time exceeds their run time score in the lower half of the range of interactivity scores, and threads whose run time exceeds their sleep time score in the upper half of the range.

***Thread Migration*** In general, it is desirable to schedule a Ready thread onto the last processor that it ran on; this is called **processor affinity**. The alternative is to allow a thread to migrate to another processor for its next execution time slice. Processor affinity is significant because of local caches dedicated to a single processor. When a thread is run, it may still have data in the cache of its last processor. Changing to another processor means the necessary data must be loaded into caches in the new processor and cache lines from the preceding processor must be invalidated. On the other hand, processor migration may allow a better load balancing, and may prevent idle periods on some processors while other processors have more work than they can handle in a timely fashion.

The FreeBSD scheduler supports two mechanisms for thread migration to balance load: pull and push. With the **pull mechanism**, an idle processor steals a thread from a nonidle processor. When a processor has no work to do, it sets a bit in a global bit-mask indicating that it is idle. When an active processor is about to add work to its own run queue, it first checks for such idle bits and if a set idle bit is found, passes the thread to the idle processor. It is primarily useful when there is a light or sporadic load, or in situations where processes are starting and exiting very frequently.

The pull mechanism is effective in preventing the waste of a processor due to idleness. But it is not effective, or indeed relevant, in a situation in which every processor has work to do but the load has developed in an uneven fashion. With the **push mechanism**, a periodic scheduler task evaluates the current load situation and evens it out. Twice per second, this task picks the most-loaded and least-loaded processors in the system and equalizes their run queues. Push migration ensures fairness among the runnable threads.

## 10.6 WINDOWS SCHEDULING

Windows is designed to be as responsive as possible to the needs of a single user in a highly interactive environment or in the role of a server. Windows implements a preemptive scheduler with a flexible system of priority levels that includes round-robin scheduling within each level and, for some levels, dynamic priority variation on the basis of their current thread activity. Threads are the unit of scheduling in Windows rather than processes.

### Process and Thread Priorities

Priorities in Windows are organized into two bands, or classes: real time and variable. Each of these bands consists of 16 priority levels. Threads requiring immediate attention are in the real-time class, which includes functions such as communications and real-time tasks.

Overall, because Windows makes use of a priority-driven preemptive scheduler, threads with real-time priorities have precedence over other threads. When a thread becomes ready whose priority is higher than the currently executing thread, the lower-priority thread is preempted and the processor is given to the higher-priority thread.

Priorities are handled somewhat differently in the two classes (see Figure 10.14). In the **real-time priority class**, all threads have a fixed priority that never changes. All of the active threads at a given priority level are in a round-robin queue. In the **variable priority class**, a thread's priority begins an initial priority value and then may be temporarily boosted (raised) during the thread's lifetime. There is a FIFO queue at each priority level; a thread will change queues among the variable priority classes as its own priority changes. However, a thread at priority level 15 or below is never boosted to level 16 (or any other level in the real-time class).

The initial priority of a thread in the variable priority class is determined by two quantities: process base priority and thread base priority. The process base priority is an attribute of the process object and can take on any value from 1 through 15 (priority 0 is reserved for the Executive's per-processor idle threads). Each thread object associated with a process object has a thread base priority attribute that indicates the thread's base priority relative to that of the process. The thread's base priority can be equal to that of its process or within two levels above or below that of the process. So, for example, if a process has a base priority of 4 and one of its threads has a base priority of −1, then the initial priority of that thread is 3.

Once a thread in the variable priority class has been created, its actual priority, referred to as the thread's current priority, may fluctuate within given boundaries. The current priority may never fall below the thread's base priority, and it may never exceed 15. Figure 10.15 gives an example. The process object has a base priority attribute of 4. Each thread object associated with this process object must have an initial priority of between 2 and 6. Suppose the base priority for thread is 4. Then the current priority for that thread may fluctuate in the range from 4 through 15 depending on what boosts it has been given. If a thread is interrupted to wait on an I/O event, the kernel boosts its priority. If a boosted thread is interrupted because it has used up its

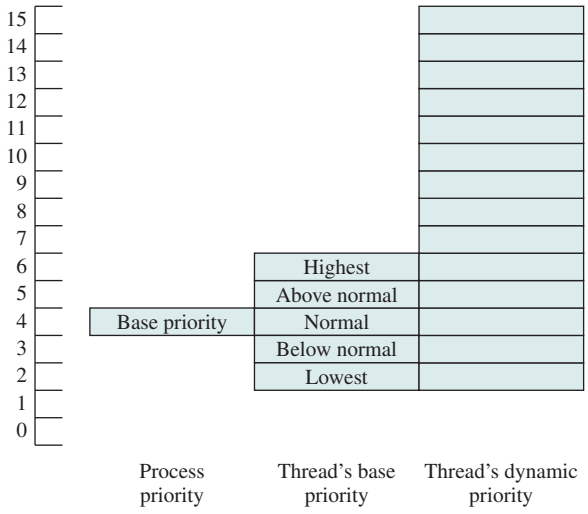



**Figure 10.15 Example of Windows Priority Relationship**

current time quantum, the kernel lowers its priority. Thus, processor-bound threads tend toward lower priorities, and I/O-bound threads tend toward higher priorities. In the case of I/O-bound threads, the kernel boosts the priority more for interactive waits (e.g., wait on keyboard or mouse) than for other types of I/O (e.g., disk I/O). Thus, interactive threads tend to have the highest priorities within the variable priority class.

## Multiprocessor Scheduling

Windows supports multiprocessor and multicore hardware configurations. The threads of any process, including those of the Executive, can run on any processor. In the absence of affinity restrictions, explained in the next paragraph, the kernel dispatcher assigns a ready thread to the next available processor. This assures that no processor is idle or is executing a lower-priority thread when a higher-priority thread is ready. Multiple threads from the same process can be executing simultaneously on multiple processors.

As a default, the kernel dispatcher uses the policy of **soft affinity** in assigning threads to processors: The dispatcher tries to assign a ready thread to the same processor it last ran on. This helps reuse data still in that processor's memory caches from the previous execution of the thread. It is possible for an application to restrict its thread execution only to certain processors (**hard affinity**).

When Windows is run on a single processor, the highest-priority thread is always active unless it is waiting on an event. If there is more than one thread that has the same highest priority, then the processor is shared, round robin, among all the threads at that priority level. In a multiprocessor system with $N$ processors, the kernel tries to give the $N$ processors to the $N$ highest-priority threads that are ready to run. The remaining, lower-priority threads must wait until the other threads block or have

their priority decay. Lower-priority threads may also have their priority boosted to 15 for a very short time if they are being starved, solely to correct instances of priority inversion.

The foregoing scheduling discipline is affected by the processor affinity attribute of a thread. If a thread is ready to execute, but the only available processors are not in its processor affinity set, then that thread is forced to wait, and the kernel schedules the next available thread.

## 10.7 SUMMARY

With a tightly coupled multiprocessor, multiple processors have access to the same main memory. In this configuration, the scheduling structure is somewhat more complex. For example, a given process may be assigned to the same processor for its entire life or dispatched to any processor each time it enters the Running state. Performance studies suggest that the differences among various scheduling algorithms are less significant in a multiprocessor system.

A real-time process or task is one that is executed in connection with some process or function or set of events external to the computer system and that must meet one or more deadlines to interact effectively and correctly with the external environment. A real-time operating system is one that is capable of managing real-time processes. In this context, the traditional criteria for a scheduling algorithm do not apply. Rather, the key factor is the meeting of deadlines. Algorithms that rely heavily on preemption, and on reacting to relative deadlines, are appropriate in this context.

## 10.8 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

### Key Terms

| | | |
|---|---|---|
| aperiodic task | load sharing | real-time operating system |
| deadline scheduling | periodic task | real-time scheduling |
| deterministic | priority ceiling | responsiveness |
| deterministic operating system | priority inheritance | soft affinity |
| fail-soft operation | priority inversion | soft real-time task |
| gang scheduling | processor affinity | thread scheduling |
| granularity | pull mechanism | unbounded priority inversion |
| hard affinity | push mechanism | |
| hard real-time task | rate monotonic scheduling | |

### Review Questions

**10.1.** List and briefly define five different categories of synchronization granularity.
**10.2.** What grain size of parallelism is appropriate for a multiprogrammed uniprocessor?
**10.3.** For which kinds of applications is gang scheduling of threads most useful?
**10.4.** What is the difference between hard and soft real-time tasks?

**10.5.** Discuss the concept of dynamic scheduling.

**10.6.** List and briefly define five general areas of requirements for a real-time operating system.

**10.7.** List and briefly define four classes of real-time scheduling algorithms.

**10.8.** What is priority inversion? What is unbounded priority inversion?

## Problems

**10.1.** Consider a set of three periodic tasks with the execution profiles of Table 10.7. Develop scheduling diagrams similar to those of Figure 10.5 for this set of tasks.

**10.2.** Consider a set of five aperiodic tasks with the execution profiles of Table 10.8. Develop scheduling diagrams similar to those of Figure 10.6 for this set of tasks.

**10.3.** Least-laxity-first (LLF) is a real-time scheduling algorithm for periodic tasks. Slack time, or laxity, is the amount of time between when a task would complete if it started now and its next deadline. This is the size of the available scheduling window. Laxity can be expressed as

$$\text{Laxity} = (\text{deadline time}) - (\text{current time}) - (\text{processor time needed})$$

LLF selects the task with the minimum laxity to execute next. If two or more tasks have the same minimum laxity value, they are serviced on a FCFS basis.

**a.** Suppose a task currently has a laxity of $t$. By how long may the scheduler delay starting this task and still meet its deadline?

**b.** Suppose a task currently has a laxity of 0. What does this mean?

**c.** What does it mean if a task has negative laxity?

**d.** Consider a set of three periodic tasks with the execution profiles of Table 10.9a. Develop scheduling diagrams similar to those of Figure 10.5 for this set of tasks

**Table 10.7** Execution Profile for Problem 10.1

| Process | Arrival Time | Execution Time | Ending Deadline |
|---|---|---|---|
| A(1) | 0 | 10 | 20 |
| A(2) | 20 | 10 | 40 |
| • | • | • | • |
| • | • | • | • |
| • | • | • | • |
| B(1) | 0 | 10 | 50 |
| B(2) | 50 | 10 | 100 |
| • | • | • | • |
| • | • | • | • |
| • | • | • | • |
| C(1) | 0 | 15 | 50 |
| C(2) | 50 | 15 | 100 |
| • | • | • | • |
| • | • | • | • |
| • | • | • | • |

**Table 10.8** Execution Profile for Problem 10.2

| Process | Arrival Time | Execution Time | Starting Deadline |
|---|---|---|---|
| A | 10 | 20 | 100 |
| B | 20 | 20 | 30 |
| C | 40 | 20 | 60 |
| D | 50 | 20 | 80 |
| E | 60 | 20 | 70 |

that compare rate monotonic, earliest-deadline first, and LLF. Assume preemption may occur at 5-ms intervals. Comment on the results.

**10.4.** Repeat Problem 10.3d for the execution profiles of Table 10.9b. Comment on the results.

**10.5.** Maximum-urgency-first (MUF) is a real-time scheduling algorithm for periodic tasks. Each task is assigned an urgency that is defined as a combination of two fixed priorities and one dynamic priority. One of the fixed priorities, the criticality, has precedence over the dynamic priority. Meanwhile, the dynamic priority has precedence over the other fixed priority, called the user priority. The dynamic priority is inversely proportional to the laxity of a task. MUF can be explained as follows. First, tasks are ordered from shortest to longest period. Define the critical task set as the first $N$ tasks such that worst-case processor utilization does not exceed 100%. Among critical set tasks that are ready, the scheduler selects the task with the least laxity. If no critical set tasks are ready, the schedule chooses among the noncritical tasks the one with the least laxity. Ties are broken through an optional user priority then by FCFS. Repeat Problem 10.3d, adding MUF to the diagrams. Assume user-defined priorities are A highest, B next, C lowest. Comment on the results.

**10.6.** Repeat Problem 10.4, adding MUF to the diagrams. Comment on the results.

**10.7.** A system is predominated by periodic tasks and so rate monotonic scheduling (RMS) is proposed as a way to resolve multitask scheduling conflicts. Assume that in a given time span the system has five tasks with parameters as listed below:

- **Task $P_1$**: Processing Time $C_1 = 20$; Period $T_1 = 90$
- **Task $P_2$**: Processing Time $C_2 = 30$; Period $T_2 = 250$
- **Task $P_3$**: Processing Time $C_3 = 70$; Period $T_3 = 370$

**Table 10.9** Execution Profiles for Problems 10.3 through 10.6

| (a) Light load | | |
|---|---|---|
| **Task** | **Period** | **Execution Time** |
| A | 6 | 2 |
| B | 8 | 2 |
| C | 12 | 3 |
| **(b) Heavy load** | | |
| **Task** | **Period** | **Execution Time** |
| A | 6 | 2 |
| B | 8 | 5 |
| C | 12 | 3 |

- **Task $P_4$**: Processing Time $C_4 = 50$; Period $T_4 = 330$
- **Task $P_5$**: Processing Time $C_5 = 125$; Period $T_5 = 2000$

We have seen that Equation (10.2) provides an upper bound on the number of tasks that a perfect scheduling algorithm can successfully schedule. If RMS is used, analyze whether the tasks can be successfully scheduled as per Equation (10.2).

**10.8.** Suppose that an application has three threads $T_1$, $T_2$ and $T_3$ having decreasing priority. Give a scenario that may cause unbounded priority inversion.

*This page intentionally left blank*

# PART 5 Input/Output and Files

## CHAPTER 11

# I/O MANAGEMENT AND DISK SCHEDULING


11.1 **I/O Devices**
11.2 **Organization of the I/O Function**
- The Evolution of the I/O Function
- Direct Memory Access

11.3 **Operating System Design Issues**
- Design Objectives
- Logical Structure of the I/O Function

11.4 **I/O Buffering**
- Single Buffer
- Double Buffer
- Circular Buffer
- The Utility of Buffering

11.5 **Disk Scheduling**
- Disk Performance Parameters
- Disk Scheduling Policies

11.6 **RAID**
- RAID Level 0
- RAID Level 1
- RAID Level 2
- RAID Level 3
- RAID Level 4
- RAID Level 5
- RAID Level 6

11.7 **Disk Cache**
- Design Considerations
- Performance Considerations

11.8 **UNIX SVR4 I/O**
- Buffer Cache
- Character Queue
- Unbuffered I/O
- UNIX Devices

11.9 **Linux I/O**
- Disk Scheduling
- Linux Page Cache

11.10 **Windows I/O**
- Basic I/O Facilities
- Asynchronous and Synchronous I/O
- Software RAID
- Volume Shadow Copies
- Volume Encryption

11.11 **Summary**
11.12 **Key Terms, Review Questions, and Problems**

### Learning Objectives

After studying this chapter, you should be able to:

- Summarize key categories of I/O devices on computers.
- Discuss the organization of the I/O function.
- Explain some of the key issues in the design of OS support for I/O.
- Analyze the performance implications of various I/O buffering alternatives.
- Understand the performance issues involved in magnetic disk access.
- Explain the concept of RAID and describe the various levels.
- Understand the performance implications of disk cache.
- Describe the I/O mechanisms in UNIX, Linux, and Windows.

Perhaps the messiest aspect of operating system design is input/output. Because there is such a wide variety of devices and applications of those devices, it is difficult to develop a general, consistent solution.

We begin with a brief discussion of I/O devices and the organization of the I/O function. These topics, which generally come within the scope of computer architecture, set the stage for an examination of I/O from the point of view of the OS.

The next section examines operating system design issues, including design objectives, and the way in which the I/O function can be structured. Then I/O buffering is examined; one of the basic I/O services provided by the operating system is a buffering function, which improves overall performance.

The next sections of the chapter are devoted to magnetic disk I/O. In contemporary systems, this form of I/O is the most important and is key to the performance as perceived by the user. We begin by developing a model of disk I/O performance then examine several techniques that can be used to enhance performance.

Appendix J summarizes characteristics of secondary storage devices, including magnetic disk and optical memory.

## 11.1 I/O DEVICES

As was mentioned in Chapter 1, external devices that engage in I/O with computer systems can be roughly grouped into three categories:

1. **Human readable:** Suitable for communicating with the computer user. Examples include printers and terminals, the latter consisting of video display, keyboard, and perhaps other devices such as a mouse.
2. **Machine readable:** Suitable for communicating with electronic equipment. Examples are disk drives, USB keys, sensors, controllers, and actuators.
3. **Communication:** Suitable for communicating with remote devices. Examples are digital line drivers and modems.

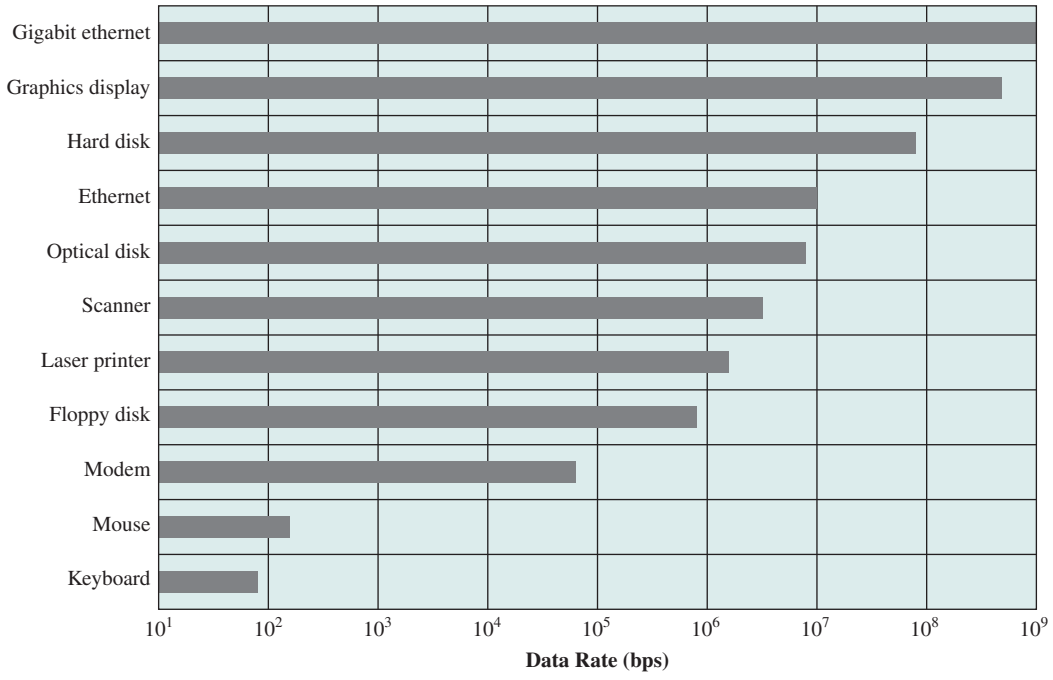



**Figure 11.1** **Typical I/O Device Data Rates**

There are great differences across classes and even substantial differences within each class. Among the key differences are the following:

- **Data rate:** There may be differences of several orders of magnitude between the data transfer rates. Figure 11.1 gives some examples.
- **Application:** The use to which a device is put has an influence on the software and policies in the OS and supporting utilities. For example, a disk used for files requires the support of file management software. A disk used as a backing store for pages in a virtual memory scheme depends on the use of virtual memory hardware and software. Furthermore, these applications have an impact on disk scheduling algorithms (discussed later in this chapter). As another example, a terminal may be used by an ordinary user or a system administrator. These uses imply different privilege levels and perhaps different priorities in the OS.
- **Complexity of control:** A printer requires a relatively simple control interface. A disk is much more complex. The effect of these differences on the OS is filtered to some extent by the complexity of the I/O module that controls the device, as discussed in the next section.
- **Unit of transfer:** Data may be transferred as a stream of bytes or characters (e.g., terminal I/O) or in larger blocks (e.g., disk I/O).
- **Data representation:** Different data encoding schemes are used by different devices, including differences in character code and parity conventions.

- **Error conditions:** The nature of errors, the way in which they are reported, their consequences, and the available range of responses differ widely from one device to another.

This diversity makes a uniform and consistent approach to I/O, both from the point of view of the operating system and from the point of view of user processes, difficult to achieve.

## 11.2 ORGANIZATION OF THE I/O FUNCTION

Appendix C summarizes three techniques for performing I/O:

1. **Programmed I/O:** The processor issues an I/O command, on behalf of a process, to an I/O module; that process then busy waits for the operation to be completed before proceeding.
2. **Interrupt-driven I/O:** The processor issues an I/O command on behalf of a process. There are then two possibilities. If the I/O instruction from the process is nonblocking, then the processor continues to execute instructions from the process that issued the I/O command. If the I/O instruction is blocking, then the next instruction that the processor executes is from the OS, which will put the current process in a blocked state and schedule another process.
3. **Direct memory access (DMA):** A DMA module controls the exchange of data between main memory and an I/O module. The processor sends a request for the transfer of a block of data to the DMA module, and is interrupted only after the entire block has been transferred.

Table 11.1 indicates the relationship among these three techniques. In most computer systems, DMA is the dominant form of transfer that must be supported by the operating system.

### The Evolution of the I/O Function

As computer systems have evolved, there has been a pattern of increasing complexity and sophistication of individual components. Nowhere is this more evident than in the I/O function. The evolutionary steps can be summarized as follows:

1. The processor directly controls a peripheral device. This is seen in simple microprocessor-controlled devices.
2. A controller or I/O module is added. The processor uses programmed I/O without interrupts. With this step, the processor becomes somewhat divorced from the specific details of external device interfaces.

**Table 11.1** I/O Techniques

| | No Interrupts | Use of Interrupts |
|---|---|---|
| **I/O-to-Memory Transfer through Processor** | Programmed I/O | Interrupt-driven I/O |
| **Direct I/O-to-Memory Transfer** | | Direct memory access (DMA) |

3. The same configuration as step 2 is used, but now interrupts are employed. The processor need not spend time waiting for an I/O operation to be performed, thus increasing efficiency.
4. The I/O module is given direct control of memory via DMA. It can now move a block of data to or from memory without involving the processor, except at the beginning and end of the transfer.
5. The I/O module is enhanced to become a separate processor, with a specialized instruction set tailored for I/O. The central processing unit (CPU) directs the I/O processor to execute an I/O program in main memory. The I/O processor fetches and executes these instructions without processor intervention. This allows the processor to specify a sequence of I/O activities and to be interrupted only when the entire sequence has been performed.
6. The I/O module has a local memory of its own and is, in fact, a computer in its own right. With this architecture, a large set of I/O devices can be controlled, with minimal processor involvement. A common use for such an architecture has been to control communications with interactive terminals. The I/O processor takes care of most of the tasks involved in controlling the terminals.

As one proceeds along this evolutionary path, more and more of the I/O function is performed without processor involvement. The central processor is increasingly relieved of I/O-related tasks, improving performance. With the last two steps (5 and 6), a major change occurs with the introduction of the concept of an I/O module capable of executing a program.

A note about terminology: For all of the modules described in steps 4 through 6, the term *direct memory access* is appropriate, because all of these types involve direct control of main memory by the I/O module. Also, the I/O module in step 5 is often referred to as an **I/O channel**, and that in step 6 as an **I/O processor**; however, each term is, on occasion, applied to both situations. In the latter part of this section, we will use the term *I/O channel* to refer to both types of I/O modules.

## Direct Memory Access

Figure 11.2 indicates, in general terms, the DMA logic. The DMA unit is capable of mimicking the processor and, indeed, of taking over control of the system bus just like a processor. It needs to do this to transfer data to and from memory over the system bus.

The DMA technique works as follows. When the processor wishes to read or write a block of data, it issues a command to the DMA module by sending to the DMA module the following information:

- Whether a read or write is requested, using the read or write control line between the processor and the DMA module
- The address of the I/O device involved, communicated on the data lines
- The starting location in memory to read from or write to, communicated on the data lines and stored by the DMA module in its address register
- The number of words to be read or written, again communicated via the data lines and stored in the data count register

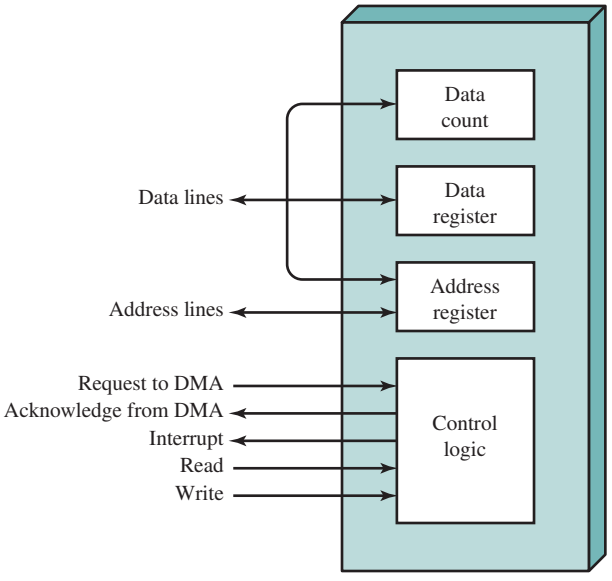



**Figure 11.2 Typical DMA Block Diagram**

The processor then continues with other work. It has delegated this I/O operation to the DMA module. The DMA module transfers the entire block of data, one word at a time, directly to or from memory, without going through the processor. When the transfer is complete, the DMA module sends an interrupt signal to the processor. Thus, the processor is involved only at the beginning and end of the transfer (see Figure C.4c).

The DMA mechanism can be configured in a variety of ways. Some possibilities are shown in Figure 11.3. In the first example, all modules share the same system bus. The DMA module, acting as a surrogate processor, uses programmed I/O to exchange data between memory and an I/O module through the DMA module. This configuration, while it may be inexpensive, is clearly inefficient: As with processor-controlled programmed I/O, each transfer of a word consumes two bus cycles (transfer request followed by transfer).

The number of required bus cycles can be cut substantially by integrating the DMA and I/O functions. As Figure 11.3b indicates, this means there is a path between the DMA module and one or more I/O modules that does not include the system bus. The DMA logic may actually be a part of an I/O module, or it may be a separate module that controls one or more I/O modules. This concept can be taken one step further by connecting I/O modules to the DMA module using an I/O bus (see Figure 11.3c). This reduces the number of I/O interfaces in the DMA module to one and provides for an easily expandable configuration. In all of these cases (see Figures 11.3b and 11.3c), the system bus that the DMA module shares with the processor and main memory is used by the DMA module only to exchange data with memory and to exchange control signals with the processor. The exchange of data between the DMA and I/O modules takes place off the system bus.

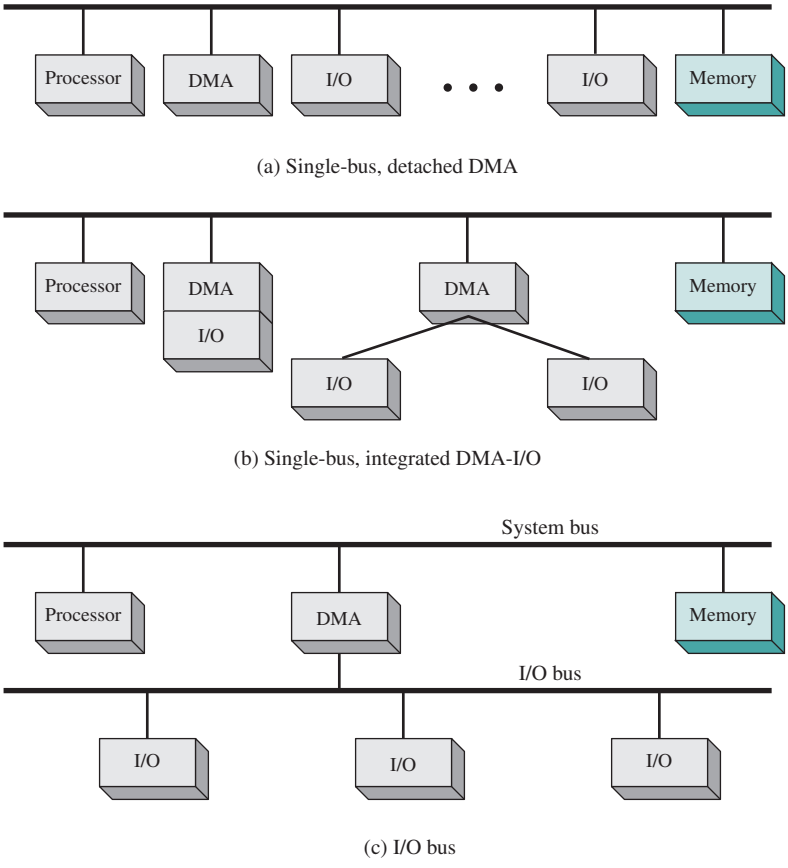



**Figure 11.3 Alternative DMA Configurations**

## 11.3 OPERATING SYSTEM DESIGN ISSUES

### Design Objectives

Two objectives are paramount in designing the I/O facility: efficiency and generality. **Efficiency** is important because I/O operations often form a bottleneck in a computing system. Looking again at Figure 11.1, we see that most I/O devices are extremely slow compared with main memory and the processor. One way to tackle this problem is multiprogramming, which, as we have seen, allows some processes to be waiting on I/O operations while another process is executing. However, even with the vast size of main memory in today's machines, it will still often be the case that I/O is not keeping up with the activities of the processor. Swapping is used to bring in additional ready processes to keep the processor busy, but this in itself is an I/O operation. Thus, a major effort in I/O design has been schemes for improving the efficiency of the I/O. The area that has received the most attention, because of its importance, is disk I/O, and much of this chapter will be devoted to a study of disk I/O efficiency.

The other major objective is **generality**. In the interests of simplicity and freedom from error, it is desirable to handle all devices in a uniform manner. This applies both to the way in which processes view I/O devices, and to the way in which the OS manages I/O devices and operations. Because of the diversity of device characteristics, it is difficult in practice to achieve true generality. What can be done is to use a hierarchical, modular approach to the design of the I/O function. This approach hides most of the details of device I/O in lower-level routines so user processes and upper levels of the OS see devices in terms of general functions such as read, write, open, close, lock, and unlock. We turn now to a discussion of this approach.

## Logical Structure of the I/O Function

In Chapter 2, in the discussion of system structure, we emphasized the hierarchical nature of modern operating systems. The hierarchical philosophy is that the functions of the OS should be separated according to their complexity, their characteristic time scale, and their level of abstraction. Applying this philosophy specifically to the I/O facility leads to the type of organization suggested by Figure 11.4. The details of the organization will depend on the type of device and the application. The three most important logical structures are presented in the figure. Of course, a particular operating system may not conform exactly to these structures. However, the general principles are valid, and most operating systems approach I/O in approximately this way.

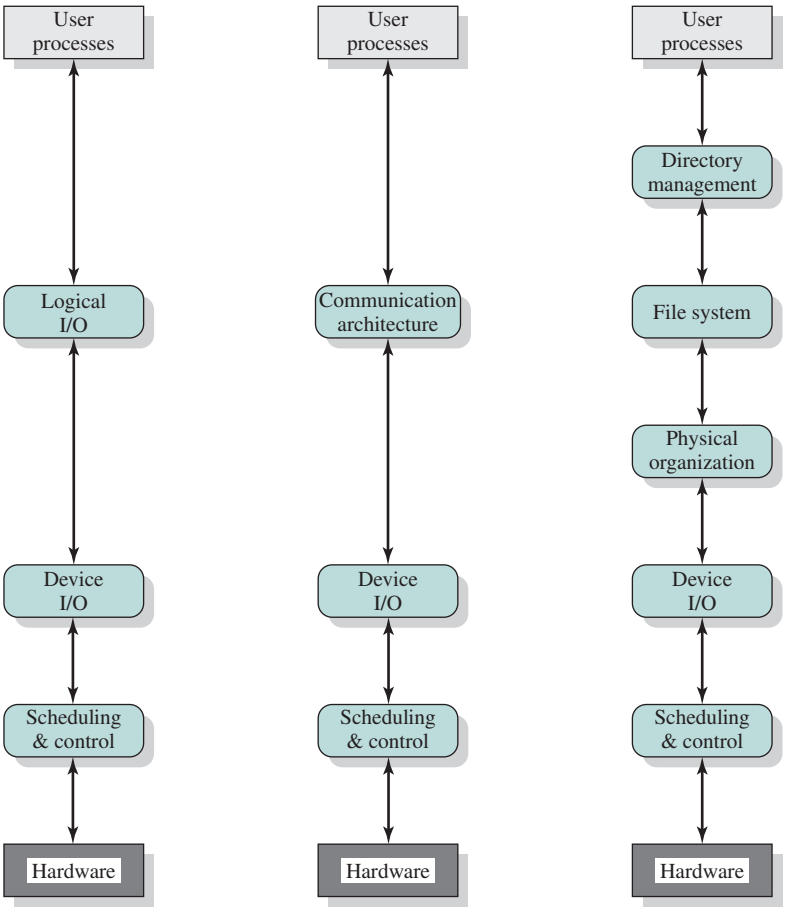
Let us consider the simplest case first, that of a local peripheral device that communicates in a simple fashion, such as a stream of bytes or records (see Figure 11.4a). The following layers are involved:

- **Logical I/O:** The logical I/O module deals with the device as a logical resource and is not concerned with the details of actually controlling the device. The logical I/O module is concerned with managing general I/O functions on behalf of user processes, allowing them to deal with the device in terms of a device identifier and simple commands such as open, close, read, and write.
- **Device I/O:** The requested operations and data (buffered characters, records, etc.) are converted into appropriate sequences of I/O instructions, channel commands, and controller orders. Buffering techniques may be used to improve utilization.
- **Scheduling and control:** The actual queueing and scheduling of I/O operations occurs at this layer, as well as the control of the operations. Thus, interrupts are handled at this layer and I/O status is collected and reported. This is the layer of software that actually interacts with the I/O module and hence the device hardware.

For a communications device, the I/O structure (see Figure 11.4b) looks much the same as that just described. The principal difference is that the logical I/O module is replaced by a communications architecture, which may itself consist of a number of layers. An example is TCP/IP, which will be discussed in Chapter 17.

Figure 11.4c shows a representative structure for managing I/O on a secondary storage device that supports a file system. The three layers not previously discussed are as follows:

1. **Directory management:** At this layer, symbolic file names are converted to identifiers that either reference the file directly or indirectly through a file




(a) Local peripheral device (b) Communications port (c) File system

**Figure 11.4 A Model of I/O Organization**

descriptor or index table. This layer is also concerned with user operations that affect the directory of files, such as add, delete, and reorganize.

2. **File system:** This layer deals with the logical structure of files and with the operations that can be specified by users, such as open, close, read, and write. Access rights are also managed at this layer.
3. **Physical organization:** Just as virtual memory addresses must be converted into physical main memory addresses, taking into account the segmentation and paging structure, logical references to files and records must be converted to physical secondary storage addresses, taking into account the physical track and sector structure of the secondary storage device. Allocation of secondary storage space and main storage buffers is generally treated at this layer as well.

Because of the importance of the file system, we will spend some time, in this chapter and the next, looking at its various components. The discussion in this chapter focuses on the lower three layers, while the upper two layers will be examined in Chapter 12.

## 11.4 I/O BUFFERING

Suppose a user process wishes to read blocks of data from a disk one at a time, with each block having a length of 512 bytes. The data are to be read into a data area within the address space of the user process at virtual location 1000 to 1511. The simplest way would be to execute an I/O command (something like `Read_Block[1000, disk]`) to the disk unit then wait for the data to become available. The waiting could either be busy waiting (continuously test the device status) or, more practically, process suspension on an interrupt.

There are two problems with this approach. First, the program is hung up waiting for the relatively slow I/O to complete. The second problem is that this approach to I/O interferes with swapping decisions by the OS. Virtual locations 1000 to 1511 must remain in main memory during the course of the block transfer. Otherwise, some of the data may be lost. If paging is being used, at least the page containing the target locations must be locked into main memory. Thus, although portions of the process may be paged out to disk, it is impossible to swap the process out completely, even if this is desired by the operating system. Notice also there is a risk of single-process deadlock. If a process issues an I/O command, is suspended awaiting the result, and then is swapped out prior to the beginning of the operation, the process is blocked waiting on the I/O event, and the I/O operation is blocked waiting for the process to be swapped in. To avoid this deadlock, the user memory involved in the I/O operation must be locked in main memory immediately before the I/O request is issued, even though the I/O operation is queued and may not be executed for some time.

The same considerations apply to an output operation. If a block is being transferred from a user process area directly to an I/O module, then the process is blocked during the transfer and the process may not be swapped out.

To avoid these overheads and inefficiencies, it is sometimes convenient to perform input transfers in advance of requests being made, and to perform output transfers some time after the request is made. This technique is known as buffering. In this section, we look at some of the buffering schemes that are supported by operating systems to improve the performance of the system.

In discussing the various approaches to buffering, it is sometimes important to make a distinction between two types of I/O devices: block-oriented and stream-oriented. A **block-oriented device** stores information in blocks that are usually of fixed size, and transfers are made one block at a time. Generally, it is possible to reference data by its block number. Disks and USB keys are examples of block-oriented devices. A **stream-oriented device** transfers data in and out as a stream of bytes, with no block structure. Terminals, printers, communications ports, mouse and other pointing devices, and most other devices that are not secondary storage are stream-oriented.

### Single Buffer

The simplest type of support that the OS can provide is single buffering (see Figure 11.5b). When a user process issues an I/O request, the OS assigns a buffer in the system portion of main memory to the operation.

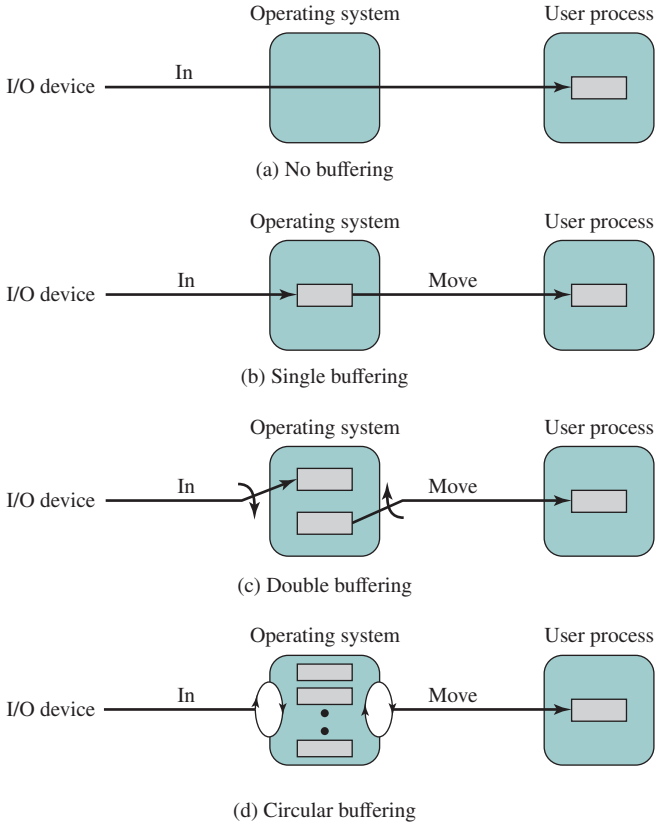



**Figure 11.5 I/O Buffering Schemes (Input)**

For block-oriented devices, the single buffering scheme can be described as follows: Input transfers are made to the system buffer. When the transfer is complete, the process moves the block into user space and immediately requests another block. This is called reading ahead, or anticipated input; it is done in the expectation that the block will eventually be needed. For many types of computation, this is a reasonable assumption most of the time because data are usually accessed sequentially. Only at the end of a sequence of processing will a block be read in unnecessarily.

This approach will generally provide a speedup compared to the lack of system buffering. The user process can be processing one block of data while the next block is being read in. The OS is able to swap the process out because the input operation is taking place in system memory rather than user process memory. This technique does, however, complicate the logic in the operating system. The OS must keep track of the assignment of system buffers to user processes. The swapping logic is also affected: If the I/O operation involves the same disk that is used for swapping, it hardly makes sense to queue disk writes to the same device for swapping the process out. This attempt to swap the process and release main memory will itself not begin until after the I/O operation finishes, at which time swapping the process to disk may no longer be appropriate.

Similar considerations apply to block-oriented output. When data are being transmitted to a device, they are first copied from the user space into the system buffer, from which they will ultimately be written. The requesting process is now free to continue or to be swapped as necessary.

[KNUT97] suggests a crude but informative performance comparison between single buffering and no buffering. Suppose $T$ is the time required to input one block, and $C$ is the computation time that intervenes between input requests. Without buffering, the execution time per block is essentially $T + C$. With a single buffer, the time is max $[C, T] + M$, where $M$ is the time required to move the data from the system buffer to user memory. In most cases, execution time per block is substantially less with a single buffer compared to no buffer.

For stream-oriented I/O, the single buffering scheme can be used in a line-at-a-time fashion or a byte-at-a-time fashion. Line-at-a-time operation is appropriate for scroll-mode terminals (sometimes called dumb terminals). With this form of terminal, user input is one line at a time, with a carriage return signaling the end of a line, and output to the terminal is similarly one line at a time. A line printer is another example of such a device. Byte-at-a-time operation is used on forms-mode terminals, when each keystroke is significant, and for many other peripherals, such as sensors and controllers.

In the case of line-at-a-time I/O, the buffer can be used to hold a single line. The user process is suspended during input, awaiting the arrival of the entire line. For output, the user process can place a line of output in the buffer and continue processing. It need not be suspended unless it has a second line of output to send before the buffer is emptied from the first output operation. In the case of byte-at-a-time I/O, the interaction between the OS and the user process follows the producer/consumer model discussed in Chapter 5.

### Double Buffer

An improvement over single buffering can be had by assigning two system buffers to the operation (see Figure 11.5c). A process now transfers data to (or from) one buffer while the operating system empties (or fills) the other. This technique is known as **double buffering** or **buffer swapping**.

For block-oriented transfer, we can roughly estimate the execution time as max $[C, T]$. It is therefore possible to keep the block-oriented device going at full speed if $C \leq T$. On the other hand, if $C > T$, double buffering ensures that the process will not have to wait on I/O. In either case, an improvement over single buffering is achieved. Again, this improvement comes at the cost of increased complexity.

For stream-oriented input, we again are faced with the two alternative modes of operation. For line-at-a-time I/O, the user process need not be suspended for input or output, unless the process runs ahead of the double buffers. For byte-at-a-time operation, the double buffer offers no particular advantage over a single buffer of twice the length. In both cases, the producer/consumer model is followed.

### Circular Buffer

A double-buffer scheme should smooth out the flow of data between an I/O device and a process. If the performance of a particular process is the focus of our concern,

then we would like for the I/O operation to be able to keep up with the process. Double buffering may be inadequate if the process performs rapid bursts of I/O. In this case, the problem can often be alleviated by using more than two buffers.

When more than two buffers are used, the collection of buffers is itself referred to as a circular buffer (see Figure 11.5d), with each individual buffer being one unit in the circular buffer. This is simply the bounded-buffer producer/consumer model studied in Chapter 5.

### The Utility of Buffering

Buffering is a technique that smoothes out peaks in I/O demand. However, no amount of buffering will allow an I/O device to keep pace with a process indefinitely when the average demand of the process is greater than the I/O device can service. Even with multiple buffers, all of the buffers will eventually fill up, and the process will have to wait after processing each chunk of data. However, in a multiprogramming environment, when there is a variety of I/O activity and a variety of process activity to service, buffering is one tool that can increase the efficiency of the OS and the performance of individual processes.

## 11.5 DISK SCHEDULING

Over the last 40 years, the increase in the speed of processors and main memory has far outpaced that for disk access, with processor and main memory speeds increasing by about two orders of magnitude compared to one order of magnitude for disk. The result is disks are currently at least four orders of magnitude slower than main memory. This gap is expected to continue into the foreseeable future. Thus, the performance of disk storage subsystem is of vital concern, and much research has gone into schemes for improving that performance. In this section, we highlight some of the key issues and look at the most important approaches. Because the performance of the disk system is tied closely to file system design issues, the discussion will continue in Chapter 12.

### Disk Performance Parameters

The actual details of disk I/O operation depend on the computer system, the operating system, and the nature of the I/O channel and disk controller hardware. A general timing diagram of disk I/O transfer is shown in Figure 11.6.

When the disk drive is operating, the disk is rotating at constant speed. To read or write, the head must be positioned at the desired track and at the beginning of the

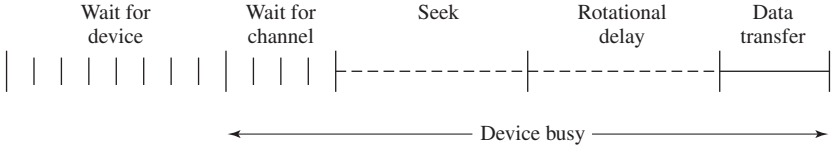



**Figure 11.6 Timing of a Disk I/O Transfer**

desired sector on that track.[1] Track selection involves moving the head in a movable-head system or electronically selecting one head on a fixed-head system. On a movable-head system, the time it takes to position the head at the track is known as **seek time**. In either case, once the track is selected, the disk controller waits until the appropriate sector rotates to line up with the head. The time it takes for the beginning of the sector to reach the head is known as **rotational delay**, or rotational latency. The sum of the seek time, if any, and the rotational delay equals the **access time**, which is the time it takes to get into position to read or write. Once the head is in position, the read or write operation is then performed as the sector moves under the head; this is the data transfer portion of the operation. The time required for the transfer is the **transfer time**.

In addition to the access time and transfer time, there are several queueing delays normally associated with a disk I/O operation. When a process issues an I/O request, it must first wait in a queue for the device to be available. At that time, the device is assigned to the process. If the device shares a single I/O channel or a set of I/O channels with other disk drives, then there may be an additional wait for the channel to be available. At that point, the seek is performed to begin disk access.

In some high-end systems for servers, a technique known as rotational positional sensing (RPS) is used. This works as follows: When the seek command has been issued, the channel is released to handle other I/O operations. When the seek is completed, the device determines when the data will rotate under the head. As that sector approaches the head, the device tries to reestablish the communication path back to the host. If either the control unit or the channel is busy with another I/O, then the reconnection attempt fails and the device must rotate one whole revolution before it can attempt to reconnect, which is called an RPS miss. This is an extra delay element that must be added to the time line of Figure 11.6.

***Seek Time*** Seek time is the time required to move the disk arm to the required track. It turns out this is a difficult quantity to pin down. The seek time consists of two key components: the initial startup time, and the time taken to traverse the tracks that have to be crossed once the access arm is up to speed. Unfortunately, the traversal time is not a linear function of the number of tracks but includes a settling time (time after positioning the head over the target track until track identification is confirmed).

Much improvement comes from smaller and lighter disk components. Some years ago, a typical disk was 14 inches (36 cm) in diameter, whereas the most common size today is 3.5 inches (8.9 cm), reducing the distance that the arm has to travel. A typical average seek time on contemporary hard disks is under 10 ms.

***Rotational Delay*** Rotational delay is the time required for the addressed area of the disk to rotate into a position where it is accessible by the read/write head. Disks rotate at speeds ranging from 3,600 rpm (for handheld devices such as digital cameras) up to, as of this writing, 15,000 rpm; at this latter speed, there is one revolution per 4 ms. Thus, on average, the rotational delay will be 2 ms.

[1]See Appendix J for a discussion of disk organization and formatting.

***Transfer Time*** The transfer time to or from the disk depends on the rotation speed of the disk in the following fashion:

$$T = \frac{b}{rN}$$

where

$T$ = transfer time,

$b$ = number of bytes to be transferred,

$N$ = number of bytes on a track, and

$r$ = rotation speed, in revolutions per second.

Thus, the total average access time can be expressed as

$$T_a = T_s + \frac{1}{2r} + \frac{b}{rN}$$

where $T_s$ is the average seek time.

***A Timing Comparison*** With the foregoing parameters defined, let us look at two different I/O operations that illustrate the danger of relying on average values. Consider a disk with an advertised average seek time of 4 ms, rotation speed of 7,500 rpm, and 512-byte sectors with 500 sectors per track. Suppose we wish to read a file consisting of 2,500 sectors for a total of 1.28 Mbytes. We would like to estimate the total time for the transfer.

First, let us assume the file is stored as compactly as possible on the disk. That is, the file occupies all of the sectors on 5 adjacent tracks (5 tracks × 500 sectors/track = 2,500 sectors). This is known as *sequential organization*. The time to read the first track is as follows:

| | |
|---|---|
| Average seek | 4 ms |
| Rotational delay | 4 ms |
| Read 500 sectors | 8 ms |
| | 16 ms |

Suppose the remaining tracks can now be read with essentially no seek time. That is, the I/O operation can keep up with the flow from the disk. Then, at most, we need to deal with rotational delay for each succeeding track. Thus, each successive track is read in 4 + 8 = 12 ms. To read the entire file,

$$\text{Total time} = 16 + (4 \times 12) = 64 \text{ ms} = 0.064 \text{ seconds}$$

Now, let us calculate the time required to read the same data using random access rather than sequential access; that is, accesses to the sectors are distributed randomly over the disk. For each sector, we have:

| | |
|---|---|
| Average seek | 4 ms |
| Rotational delay | 4 ms |
| Read 1 sector | 0.016 ms |
| | 8.016 ms |

$$\text{Total time} = 2{,}500 \times 8.016 = 20{,}040 \text{ ms} = 20.04 \text{ seconds}$$

It is clear the order in which sectors are read from the disk has a tremendous effect on I/O performance. In the case of file access in which multiple sectors are read or written, we have some control over the way in which sectors of data are deployed, and we shall have something to say on this subject in the next chapter. However, even in the case of a file access, in a multiprogramming environment, there will be I/O requests competing for the same disk. Thus, it is worthwhile to examine ways in which the performance of disk I/O can be improved over that achieved with purely random access to the disk.

## Disk Scheduling Policies

In the example just described, the reason for the difference in performance can be traced to seek time. If sector access requests involve selection of tracks at random, then the performance of the disk I/O system will be as poor as possible. To improve matters, we need to reduce the average time spent on seeks.

Consider the typical situation in a multiprogramming environment, in which the OS maintains a queue of requests for each I/O device. So, for a single disk, there will be a number of I/O requests (reads and writes) from various processes in the queue. If we selected items from the queue in random order, then we can expect that the tracks to be visited will occur randomly, giving poor performance. This **random scheduling** is useful as a benchmark against which to evaluate other techniques.

Figure 11.7 compares the performance of various scheduling algorithms for an example sequence of I/O requests. The vertical axis corresponds to the tracks on the disk. The horizontal axis corresponds to time or, equivalently, the number of tracks traversed. For this figure, we assume the disk head is initially located at track 100. In this example, we assume a disk with 200 tracks, and the disk request queue has random requests in it. The requested tracks, in the order received by the disk scheduler, are 55, 58, 39, 18, 90, 160, 150, 38, 184. Table 11.2a tabulates the results.

***First-In-First-Out*** The simplest form of scheduling is first-in-first-out (FIFO) scheduling, which processes items from the queue in sequential order. This strategy has the advantage of being fair, because every request is honored, and the requests are honored in the order received. Figure 11.7a illustrates the disk arm movement with FIFO. This graph is generated directly from the data in Table 11.2a. As can be seen, the disk accesses are in the same order as the requests were originally received.

With FIFO, if there are only a few processes that require access and if many of the requests are to clustered file sectors, then we can hope for good performance. However, this technique will often approximate random scheduling in performance, if there are many processes competing for the disk. Thus, it may be profitable to consider a more sophisticated scheduling policy. A number of these are listed in Table 11.3 and will now be considered.

***Priority*** With a system based on priority (PRI), the control of the scheduling is outside the control of disk management software. Such an approach is not intended to optimize disk utilization, but to meet other objectives within the OS. Often, short batch jobs and interactive jobs are given higher priority than jobs that require longer

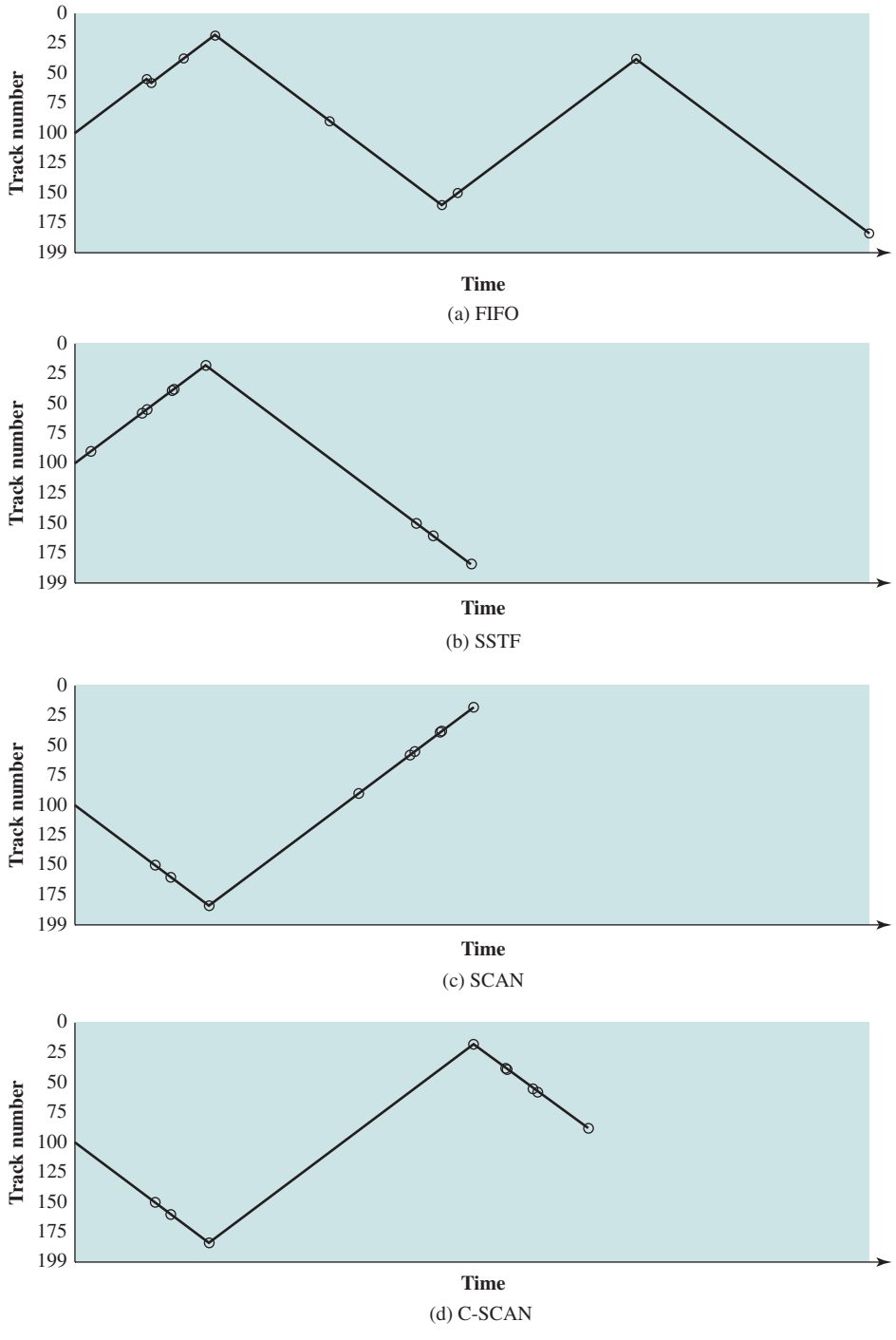



**Figure 11.7** **Comparison of Disk Scheduling Algorithms (see Table 11.2)**

**Table 11.2** Comparison of Disk Scheduling Algorithms

| **(a) FIFO** (starting at track 100) | | **(b) SSTF** (starting at track 100) | | **(c) SCAN** (starting at track 100, in the direction of increasing track number) | | **(d) C-SCAN** (starting at track 100, in the direction of increasing track number) | |
|---|---|---|---|---|---|---|---|
| **Next track accessed** | **Number of tracks traversed** | **Next track accessed** | **Number of tracks traversed** | **Next track accessed** | **Number of tracks traversed** | **Next track accessed** | **Number of tracks traversed** |
| 55 | 45 | 90 | 10 | 150 | 50 | 150 | 50 |
| 58 | 3 | 58 | 32 | 160 | 10 | 160 | 10 |
| 39 | 19 | 55 | 3 | 184 | 24 | 184 | 24 |
| 18 | 21 | 39 | 16 | 90 | 94 | 18 | 166 |
| 90 | 72 | 38 | 1 | 58 | 32 | 38 | 20 |
| 160 | 70 | 18 | 20 | 55 | 3 | 39 | 1 |
| 150 | 10 | 150 | 132 | 39 | 16 | 55 | 16 |
| 38 | 112 | 160 | 10 | 38 | 1 | 58 | 3 |
| 184 | 146 | 184 | 24 | 18 | 20 | 90 | 32 |
| **Average seek length** | 55.3 | **Average seek length** | 27.5 | **Average seek length** | 27.8 | **Average seek length** | 35.8 |

computation. This allows a lot of short jobs to be flushed through the system quickly and may provide good interactive response time. However, longer jobs may have to wait excessively long times. Furthermore, such a policy could lead to countermeasures on the part of users, who split their jobs into smaller pieces to beat the system. This type of policy tends to be poor for database systems.

**Table 11.3** Disk Scheduling Algorithms

| **Name** | **Description** | **Remarks** |
|---|---|---|
| **Selection according to requestor** | | |
| Random | Random scheduling | For analysis and simulation |
| FIFO | First-in-first-out | Fairest of them all |
| PRI | Priority by process | Control outside of disk queue management |
| LIFO | Last-in-first-out | Maximize locality and resource utilization |
| **Selection according to requested item** | | |
| SSTF | Shortest-service-time first | High utilization, small queues |
| SCAN | Back and forth over disk | Better service distribution |
| C-SCAN | One way with fast return | Lower service variability |
| *N*-step-SCAN | SCAN of *N* records at a time | Service guarantee |
| FSCAN | *N*-step-SCAN with $N$ = queue size at beginning of SCAN cycle | Load sensitive |

***LAST-IN-FIRST-OUT*** Surprisingly, a policy of always taking the most recent request has some merit. In transaction-processing systems, giving the device to the most recent user should result in little or no arm movement for moving through a sequential file. Taking advantage of this locality improves throughput and reduces queue lengths. As long as a job can actively use the file system, it is processed as fast as possible. However, if the disk is kept busy because of a large workload, there is the distinct possibility of starvation. Once a job has entered an I/O request in the queue and fallen back from the head of the line, the job can never regain the head of the line unless the queue in front of it empties.

FIFO, priority, and LIFO (last-in-first-out) scheduling are based solely on attributes of the queue or the requester. If the current track position is known to the scheduler, then scheduling based on the requested item can be employed. We will examine these policies next.

***SHORTEST-SERVICE-TIME-FIRST*** The shortest-service-time-first (SSTF) policy is to select the disk I/O request that requires the least movement of the disk arm from its current position. Thus, we always choose to incur the minimum seek time. Of course, always choosing the minimum seek time does not guarantee the average seek time over a number of arm movements will be minimum. However, this should provide better performance than FIFO. Because the arm can move in two directions, a random tie-breaking algorithm may be used to resolve cases of equal distances.

Figure 11.7b and Table 11.2b show the performance of SSTF on the same example as was used for FIFO. The first track accessed is 90, because this is the closest requested track to the starting position. The next track accessed is 58 because this is the closest of the remaining requested tracks to the current position of 90. Subsequent tracks are selected accordingly.

***SCAN*** With the exception of FIFO, all of the policies described so far can leave some request unfulfilled until the entire queue is emptied. That is, there may always be new requests arriving that will be chosen before an existing request. A simple alternative that prevents this sort of starvation is the SCAN algorithm, also known as the elevator algorithm because it operates much the way an elevator does.

With SCAN, the arm is required to move in one direction only, satisfying all outstanding requests en route, until it reaches the last track in that direction or until there are no more requests in that direction. This latter refinement is sometimes referred to as the LOOK policy. The service direction is then reversed and the scan proceeds in the opposite direction, again picking up all requests in order.

Figure 11.7c and Table 11.2c illustrate the SCAN policy. Assuming the initial direction is of increasing track number, then the first track selected is 150, since this is the closest track to the starting track of 100 in the increasing direction.

As can be seen, the SCAN policy behaves almost identically with the SSTF policy. Indeed, if we had assumed the arm was moving in the direction of lower track numbers at the beginning of the example, then the scheduling pattern would have been identical for SSTF and SCAN. However, this is a static example in which no new items are added to the queue. Even when the queue is dynamically changing, SCAN will be similar to SSTF unless the request pattern is unusual.

Note the SCAN policy is biased against the area most recently traversed. Thus, it does not exploit locality as well as SSTF.

It is not difficult to see that the SCAN policy favors jobs whose requests are for tracks nearest to both innermost and outermost tracks and favors the latest-arriving jobs. The first problem can be avoided via the C-SCAN policy, while the second problem is addressed by the *N*-step-SCAN policy.

***C-SCAN*** The C-SCAN (circular SCAN) policy restricts scanning to one direction only. Thus, when the last track has been visited in one direction, the arm is returned to the opposite end of the disk and the scan begins again. This reduces the maximum delay experienced by new requests. With SCAN, if the expected time for a scan from inner track to outer track is $t$, then the expected service interval for sectors at the periphery is $2t$. With C-SCAN, the interval is on the order of $t + s_{\max}$, where $s_{\max}$ is the maximum seek time.

Figure 11.7d and Table 11.2d illustrate C-SCAN behavior. In this case, the first three requested tracks encountered are 150, 160, and 184. Then the scan begins starting at the lowest track number, and the next requested track encountered is 18.

***N*-STEP-*SCAN* AND *FSCAN*** With SSTF, SCAN, and C-SCAN, it is possible the arm may not move for a considerable period of time. For example, if one or a few processes have high access rates to one track, they can monopolize the entire device by repeated requests to that track. High-density multisurface disks are more likely to be affected by this characteristic than lower-density disks and/or disks with only one or two surfaces. To avoid this "arm stickiness," the disk request queue can be segmented, with one segment at a time being processed completely. Two examples of this approach are *N*-step-SCAN and FSCAN.

The *N*-step-SCAN policy segments the disk request queue into subqueues of length *N*. Subqueues are processed one at a time, using SCAN. While a queue is being processed, new requests must be added to some other queue. If fewer than *N* requests are available at the end of a scan, then all of them are processed with the next scan. With large values of *N*, the performance of *N*-step-SCAN approaches that of SCAN; with a value of $N = 1$, the FIFO policy is adopted.

FSCAN is a policy that uses two subqueues. When a scan begins, all of the requests are in one of the queues, with the other empty. During the scan, all new requests are put into the other queue. Thus, service of new requests is deferred until all of the old requests have been processed.

## 11.6 RAID

As discussed earlier, the rate in improvement in secondary storage performance has been considerably less than the rate for processors and main memory. This mismatch has made the disk storage system perhaps the main focus of concern in improving overall computer system performance.

As in other areas of computer performance, disk storage designers recognize that if one component can only be pushed so far, additional gains in performance are to be had by using multiple parallel components. In the case of disk storage, this leads

to the development of arrays of disks that operate independently and in parallel. With multiple disks, separate I/O requests can be handled in parallel, as long as the data required reside on separate disks. Further, a single I/O request can be executed in parallel if the block of data to be accessed is distributed across multiple disks.

With the use of multiple disks, there is a wide variety of ways in which the data can be organized and in which redundancy can be added to improve reliability. This could make it difficult to develop database schemes that are usable on a number of platforms and operating systems. Fortunately, the industry has agreed on a standardized scheme for multiple-disk database design, known as RAID (redundant array of independent disks). The RAID scheme consists of seven levels,[2] zero through six. These levels do not imply a hierarchical relationship but designate different design architectures that share three common characteristics:

1. RAID is a set of physical disk drives viewed by the OS as a single logical drive.
2. Data are distributed across the physical drives of an array in a scheme known as striping, described subsequently.
3. Redundant disk capacity is used to store parity information, which guarantees data recoverability in case of a disk failure.

The details of the second and third characteristics differ for the different RAID levels. RAID 0 and RAID 1 do not support the third characteristic.

The term *RAID* was originally coined in a paper by a group of researchers at the University of California at Berkeley [PATT88].[3] The paper outlined various RAID configurations and applications, and introduced the definitions of the RAID levels that are still used. The RAID strategy employs multiple disk drives and distributes data in such a way as to enable simultaneous access to data from multiple drives, thereby improving I/O performance and allowing easier incremental increases in capacity.

The unique contribution of the RAID proposal is to effectively address the need for redundancy. Although allowing multiple heads and actuators to operate simultaneously achieves higher I/O and transfer rates, the use of multiple devices increases the probability of failure. To compensate for this decreased reliability, RAID makes use of stored parity information that enables the recovery of data lost due to a disk failure.

We now examine each of the RAID levels. Table 11.4 provides a rough guide to the seven levels. In the table, I/O performance is shown both in terms of data transfer capacity, or ability to move data, and I/O request rate, or ability to satisfy I/O requests, since these RAID levels inherently perform differently relative to these two metrics. Each RAID level's strong point is highlighted in color. Figure 11.8 is an example that illustrates the use of the seven RAID schemes to support a data

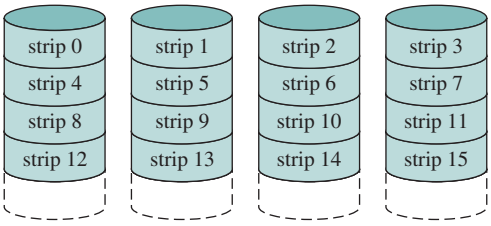
[2]Additional levels have been defined by some researchers and some companies, but the seven levels described in this section are the ones universally agreed on.

[3]In that paper, the acronym RAID stood for Redundant Array of Inexpensive Disks. The term *inexpensive* was used to contrast the small relatively inexpensive disks in the RAID array to the alternative, a single large expensive disk (SLED). The SLED is essentially a thing of the past, with similar disk technology being used for both RAID and non-RAID configurations. Accordingly, the industry has adopted the term *independent* to emphasize that the RAID array creates significant performance and reliability gains.

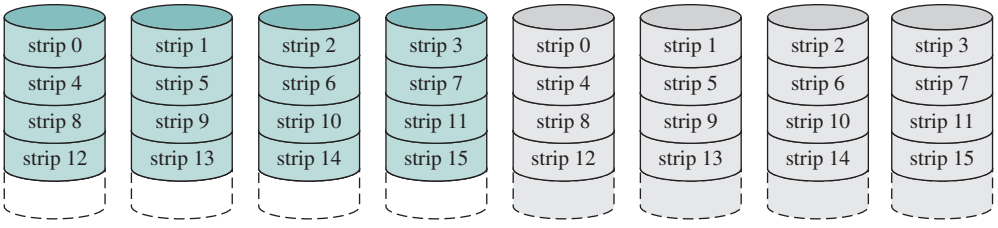
**Table 11.4** RAID Levels

| Category | Level | Description | Disks Required | Data Availability | Large I/O Data Transfer Capacity | Small I/O Request Rate |
|---|---|---|---|---|---|---|
| Striping | 0 | Nonredundant | $N$ | Lower than single disk | Very high | Very high for both read and write |
| Mirroring | 1 | Mirrored | $2N$ | Higher than RAID 2, 3, 4, or 5; lower than RAID 6 | Higher than single disk for read; similar to single disk for write | Up to twice that of a single disk for read; similar to single disk for write |
| Parallel access | 2 | Redundant via Hamming code | $N + m$ | Much higher than single disk; comparable to RAID 3, 4, or 5 | Highest of all listed alternatives | Approximately twice that of a single disk |
| | 3 | Bit-interleaved parity | $N + 1$ | Much higher than single disk; comparable to RAID 2, 4, or 5 | Highest of all listed alternatives | Approximately twice that of a single disk |
| Independent access | 4 | Block-interleaved parity | $N + 1$ | Much higher than single disk; comparable to RAID 2, 3, or 5 | Similar to RAID 0 for read; significantly lower than single disk for write | Similar to RAID 0 for read; significantly lower than single disk for write |
| | 5 | Block-interleaved distributed parity | $N + 1$ | Much higher than single disk; comparable to RAID 2, 3, or 4 | Similar to RAID 0 for read; lower than single disk for write | Similar to RAID 0 for read; generally lower than single disk for write |
| | 6 | Block-interleaved dual distributed parity | $N + 2$ | Highest of all listed alternatives | Similar to RAID 0 for read; lower than RAID 5 for write | Similar to RAID 0 for read; significantly lower than RAID 5 for write |

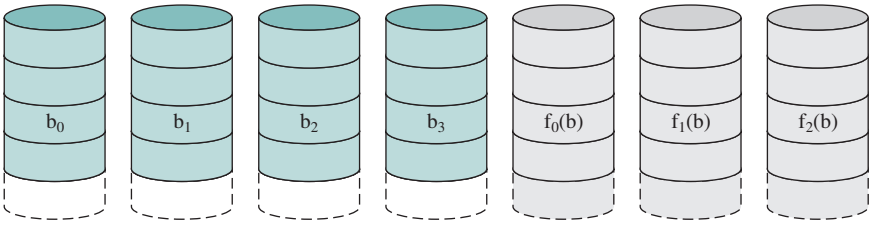
*Note*: $N$, number of data disks; $m$, proportional to log $N$.



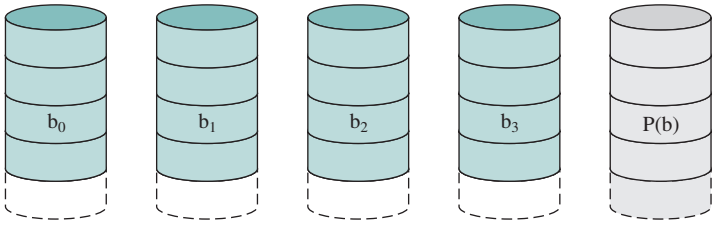

(a) RAID 0 (nonredundant)



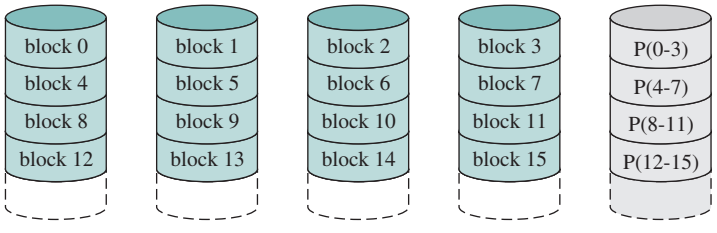

(b) RAID 1 (mirrored)




(c) RAID 2 (redundancy through Hamming code)

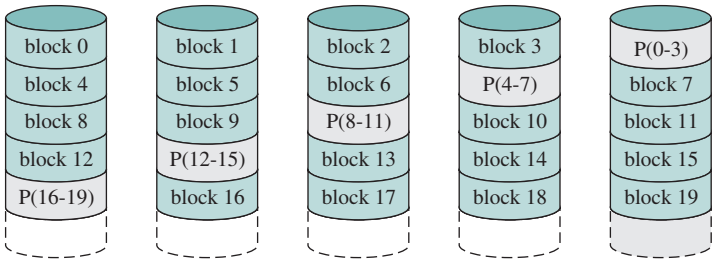



(d) RAID 3 (bit-interleaved parity)

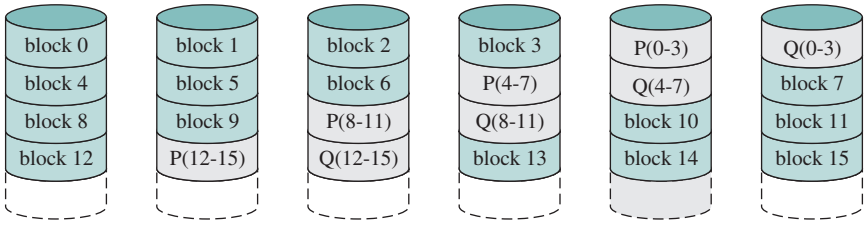



(e) RAID 4 (block-interleaved parity)

**Figure 11.8 RAID Levels**




(f) RAID 5 (block-interleaved distributed parity)




(g) RAID 6 (block-interleaved dual distributed parity)

**Figure 11.8 RAID Levels (continued)**

capacity requiring four disks with no redundancy. The figure highlights the layout of user data and redundant data and indicates the relative storage requirements of the various levels. We refer to this figure throughout the following discussion.

Of the seven RAID levels described, only four are commonly used: RAID levels 0, 1, 5, and 6.

## RAID Level 0

RAID level 0 is not a true member of the RAID family, because it does not include redundancy to improve performance or provide data protection. However, there are a few applications, such as some on supercomputers, in which performance and capacity are primary concerns and low cost is more important than improved reliability.

For RAID 0, the user and system data are distributed across all of the disks in the array. This has a notable advantage over the use of a single large disk: If two different I/O requests are pending for two different blocks of data, then there is a good chance the requested blocks are on different disks. Thus, the two requests can be issued in parallel, reducing the I/O queueing time.

But RAID 0, as with all of the RAID levels, goes further than simply distributing the data across a disk array: The data are *striped* across the available disks. This is best understood by considering Figure 11.8. All user and system data are viewed as being stored on a logical disk. The logical disk is divided into strips; these strips may be physical blocks, sectors, or some other unit. The strips are mapped round robin to consecutive physical disks in the RAID array. A set of logically consecutive strips that maps exactly one strip to each array member is referred to as a **stripe**. In an $n$-disk array, the first $n$ logical strips are physically stored as the first strip on each of the $n$

disks, forming the first stripe; the second $n$ strips are distributed as the second strips on each disk; and so on. The advantage of this layout is that if a single I/O request consists of multiple logically contiguous strips, then up to $n$ strips for that request can be handled in parallel, greatly reducing the I/O transfer time.

***RAID 0 for High Data Transfer Capacity*** The performance of any of the RAID levels depends critically on the request patterns of the host system and on the layout of the data. These issues can be most clearly addressed in RAID 0, where the impact of redundancy does not interfere with the analysis. First, let us consider the use of RAID 0 to achieve a high data transfer rate. For applications to experience a high transfer rate, two requirements must be met. First, a high transfer capacity must exist along the entire path between host memory and the individual disk drives. This includes internal controller buses, host system I/O buses, I/O adapters, and host memory buses.

The second requirement is the application must make I/O requests that drive the disk array efficiently. This requirement is met if the typical request is for large amounts of logically contiguous data, compared to the size of a strip. In this case, a single I/O request involves the parallel transfer of data from multiple disks, increasing the effective transfer rate compared to a single-disk transfer.

***RAID 0 for High I/O Request Rate*** In a transaction-oriented environment, the user is typically more concerned with response time than with transfer rate. For an individual I/O request for a small amount of data, the I/O time is dominated by the motion of the disk heads (seek time) and the movement of the disk (rotational latency).

In a transaction environment, there may be hundreds of I/O requests per second. A disk array can provide high I/O execution rates by balancing the I/O load across multiple disks. Effective load balancing is achieved only if there are typically multiple I/O requests outstanding. This, in turn, implies there are multiple independent applications or a single transaction-oriented application that is capable of multiple asynchronous I/O requests. The performance will also be influenced by the strip size. If the strip size is relatively large, so that a single I/O request only involves a single disk access, then multiple waiting I/O requests can be handled in parallel, reducing the queueing time for each request.

## RAID Level 1

RAID 1 differs from RAID levels 2 through 6 in the way in which redundancy is achieved. In these other RAID schemes, some form of parity calculation is used to introduce redundancy, whereas in RAID 1, redundancy is achieved by the simple expedient of duplicating all the data. Figure 11.8b shows data striping being used, as in RAID 0. But in this case, each logical strip is mapped to two separate physical disks so every disk in the array has a mirror disk that contains the same data. RAID 1 can also be implemented without data striping, though this is less common.

There are a number of positive aspects to the RAID 1 organization:

1. A read request can be serviced by either of the two disks that contains the requested data, whichever one involves the minimum seek time plus rotational latency.

2. A write request requires both corresponding strips be updated, but this can be done in parallel. Thus, the write performance is dictated by the slower of the two writes (i.e., the one that involves the larger seek time plus rotational latency). However, there is no "write penalty" with RAID 1. RAID levels 2 through 6 involve the use of parity bits. Therefore, when a single strip is updated, the array management software must first compute and update the parity bits as well as update the actual strip in question.
3. Recovery from a failure is simple. When a drive fails, the data may still be accessed from the second drive.

The principal disadvantage of RAID 1 is the cost; it requires twice the disk space of the logical disk that it supports. Because of that, a RAID 1 configuration is likely to be limited to drives that store system software and data and other highly critical files. In these cases, RAID 1 provides real-time backup of all data so in the event of a disk failure, all of the critical data is still immediately available.

In a transaction-oriented environment, RAID 1 can achieve high I/O request rates if the bulk of the requests are reads. In this situation, the performance of RAID 1 can approach double of that of RAID 0. However, if a substantial fraction of the I/O requests are write requests, then there may be no significant performance gain over RAID 0. RAID 1 may also provide improved performance over RAID 0 for data transfer-intensive applications with a high percentage of reads. Improvement occurs if the application can split each read request so both disk members participate.

### RAID Level 2

RAID levels 2 and 3 make use of a parallel access technique. In a parallel access array, all member disks participate in the execution of every I/O request. Typically, the spindles of the individual drives are synchronized so each disk head is in the same position on each disk at any given time.

As in the other RAID schemes, data striping is used. In the case of RAID 2 and 3, the strips are very small, often as small as a single byte or word. With RAID 2, an error-correcting code is calculated across corresponding bits on each data disk, and the bits of the code are stored in the corresponding bit positions on multiple parity disks. Typically, a Hamming code is used, which is able to correct single-bit errors and detect double-bit errors.

Although RAID 2 requires fewer disks than RAID 1, it is still rather costly. The number of redundant disks is proportional to the log of the number of data disks. On a single read, all disks are simultaneously accessed. The requested data and the associated error-correcting code are delivered to the array controller. If there is a single-bit error, the controller can recognize and correct the error instantly, so the read access time is not slowed. On a single write, all data disks and parity disks must be accessed for the write operation.

RAID 2 would only be an effective choice in an environment in which many disk errors occur. Given the high reliability of individual disks and disk drives, RAID 2 is overkill and is not implemented.

## RAID Level 3

RAID 3 is organized in a similar fashion to RAID 2. The difference is RAID 3 requires only a single redundant disk, no matter how large the disk array. RAID 3 employs parallel access, with data distributed in small strips. Instead of an error-correcting code, a simple parity bit is computed for the set of individual bits in the same position on all of the data disks.

***Redundancy*** In the event of a drive failure, the parity drive is accessed and data is reconstructed from the remaining devices. Once the failed drive is replaced, the missing data can be restored on the new drive and operation resumed.

Data reconstruction is simple. Consider an array of five drives in which X0 through X3 contain data and X4 is the parity disk. The parity for the $i$th bit is calculated as follows:

$$X4(i) = X3(i) \oplus X2(i) \oplus X1(i) \oplus X0(i)$$

where $\oplus$ is exclusive-OR function.

Suppose drive X1 has failed. If we add $X4(i) \oplus X1(i)$ to both sides of the preceding equation, we get

$$X1(i) = X4(i) \oplus X3(i) \oplus X2(i) \oplus X0(i)$$

Thus, the contents of each strip of data on X1 can be regenerated from the contents of the corresponding strips on the remaining disks in the array. This principle is true for RAID levels 3 through 6.

In the event of a disk failure, all of the data are still available in what is referred to as reduced mode. In this mode, for reads, the missing data are regenerated on the fly using the exclusive-OR calculation. When data are written to a reduced RAID 3 array, consistency of the parity must be maintained for later regeneration. Return to full operation requires the failed disk be replaced and the entire contents of the failed disk be regenerated on the new disk.

***Performance*** Because data are striped in very small strips, RAID 3 can achieve very high data transfer rates. Any I/O request will involve the parallel transfer of data from all of the data disks. For large transfers, the performance improvement is especially noticeable. On the other hand, only one I/O request can be executed at a time. Thus, in a transaction-oriented environment, performance suffers.

## RAID Level 4

RAID levels 4 through 6 make use of an independent access technique. In an independent access array, each member disk operates independently, so separate I/O requests can be satisfied in parallel. Because of this, independent access arrays are more suitable for applications that require high I/O request rates and are relatively less suitable for applications that require high data transfer rates.

As in the other RAID schemes, data striping is used. In the case of RAID 4 through 6, the strips are relatively large. With RAID 4, a bit-by-bit parity strip is calculated across corresponding strips on each data disk, and the parity bits are stored in the corresponding strip on the parity disk.

RAID 4 involves a write penalty when an I/O write request of small size is performed. Each time that a write occurs, the array management software must update not only the user data but also the corresponding parity bits. Consider an array of five drives in which X0 through X3 contain data and X4 is the parity disk. Suppose a write is performed that only involves a strip on disk X1. Initially, for each bit $i$, we have the following relationship:

$$X4(i) = X3(i) \oplus X2(i) \oplus X1(i) \oplus X0(i) \quad \textbf{(11.1)}$$

After the update, with potentially altered bits indicated by a prime symbol:

$$\begin{aligned} X4'(i) &= X3(i) \oplus X2(i) \oplus X1'(i) \oplus X0(i) \\ &= X3(i) \oplus X2(i) \oplus X1'(i) \oplus X0(i) \oplus X1(i) \oplus X1(i) \\ &= X3(i) \oplus X2(i) \oplus X1(i) \oplus X0(i) \oplus X1(i) \oplus X1'(i) \\ &= X4(i) \oplus X1(i) \oplus X1'(i) \end{aligned}$$

The preceding set of equations is derived as follows. The first line shows a change in X1 will also affect the parity disk X4. In the second line, we add the terms $[\oplus X1(i) \oplus X1(i)]$. Because the exclusive-OR of any quantity with itself is 0, this does not affect the equation. However, it is a convenience that is used to create the third line, by reordering. Finally, Equation (11.1) is used to replace the first four terms by $X4(i)$.

To calculate the new parity, the array management software must read the old user strip and the old parity strip. Then it can update these two strips with the new data and the newly calculated parity. Thus, each strip write involves two reads and two writes.

In the case of a larger size I/O write that involves strips on all disk drives, parity is easily computed by calculation using only the new data bits. Thus, the parity drive can be updated in parallel with the data drives and there are no extra reads or writes.

In any case, every write operation must involve the parity disk, which therefore can become a bottleneck.

## RAID Level 5

RAID 5 is organized in a similar fashion to RAID 4. The difference is RAID 5 distributes the parity strips across all disks. A typical allocation is a round-robin scheme, as illustrated in Figure 11.8f. For an $n$-disk array, the parity strip is on a different disk for the first $n$ stripes, and the pattern then repeats.

The distribution of parity strips across all drives avoids the potential I/O bottleneck of the single parity disk found in RAID 4. Further, RAID 5 has the characteristic that the loss of any one disk does not result in data loss.

## RAID Level 6

RAID 6 was introduced in a subsequent paper by the Berkeley researchers [KATZ89]. In the RAID 6 scheme, two different parity calculations are carried out and stored in separate blocks on different disks. Thus, a RAID 6 array whose user data require $N$ disks consists of $N + 2$ disks.

Figure 11.8g illustrates the scheme. P and Q are two different data check algorithms. One of the two is the exclusive-OR calculation used in RAID 4 and 5. But the other is an independent data check algorithm. This makes it possible to regenerate data even if two disks containing user data fail.

The advantage of RAID 6 is that it provides extremely high data availability. Three disks would have to fail within the MTTR (mean time to repair) interval to cause data to be lost. On the other hand, RAID 6 incurs a substantial write penalty, because each write affects two parity blocks. Performance benchmarks [EISC07] show a RAID 6 controller can suffer more than a 30% drop in overall write performance compared with a RAID 5 implementation. RAID 5 and RAID 6 read performance is comparable.

## 11.7 DISK CACHE

In Section 1.6 and Appendix 1A, we summarized the principles of cache memory. The term *cache memory* is usually used to apply to a memory that is smaller and faster than main memory, and that is interposed between main memory and the processor. Such a cache memory reduces average memory access time by exploiting the principle of locality.

The same principle can be applied to disk memory. Specifically, a disk cache is a buffer in main memory for disk sectors. The cache contains a copy of some of the sectors on the disk. When an I/O request is made for a particular sector, a check is made to determine if the sector is in the disk cache. If so, the request is satisfied via the cache. If not, the requested sector is read into the disk cache from the disk. Because of the phenomenon of locality of reference, when a block of data is fetched into the cache to satisfy a single I/O request, it is likely that there will be future references to that same block.

### Design Considerations

Several design issues are of interest. First, when an I/O request is satisfied from the disk cache, the data in the disk cache must be delivered to the requesting process. This can be done either by transferring the block of data within main memory from the disk cache to memory assigned to the user process, or simply by using a shared memory capability and passing a pointer to the appropriate slot in the disk cache. The latter approach saves the time of a memory-to-memory transfer and also allows shared access by other processes using the readers/writers model described in Chapter 5.

A second design issue has to do with the replacement strategy. When a new sector is brought into the disk cache, one of the existing blocks must be replaced. This is the identical problem presented in Chapter 8; there, the requirement was for a page replacement algorithm. A number of algorithms have been tried. The most commonly used algorithm is least recently used (LRU): Replace the block that has been in the cache longest with no reference to it. Logically, the cache consists of a stack of blocks, with the most recently referenced block on the top of the stack. When a block in the cache is referenced, it is moved from its existing position on the stack to the top of the

stack. When a block is brought in from secondary memory, remove the block on the bottom of the stack and push the incoming block onto the top of the stack. Naturally, it is not necessary actually to move these blocks around in main memory; a stack of pointers can be associated with the cache.

Another possibility is **least frequently used (LFU)**: Replace the block in the set that has experienced the fewest references. LFU could be implemented by associating a counter with each block. When a block is brought in, it is assigned a count of 1; with each reference to the block, its count is incremented by 1. When replacement is required, the block with the smallest count is selected. Intuitively, it might seem that LFU is more appropriate than LRU because LFU makes use of more pertinent information about each block in the selection process.

A simple LFU algorithm has the following problem. It may be that certain blocks are referenced relatively infrequently overall, but when they are referenced, there are short intervals of repeated references due to locality, thus building up high reference counts. After such an interval is over, the reference count may be misleading and not reflect the probability that the block will soon be referenced again. Thus, the effect of locality may actually cause the LFU algorithm to make poor replacement choices.

To overcome this difficulty with LFU, a technique known as frequency-based replacement is proposed in [ROBI90]. For clarity, let us first consider a simplified version, illustrated in Figure 11.9a. The blocks are logically organized in a stack, as with the LRU algorithm. A certain portion of the top part of the stack is designated the new section. When there is a cache hit, the referenced block is moved to the top of the stack. If the block was already in the new section, its reference count is not incremented; otherwise, it is incremented by 1. Given a sufficiently large new section, this results in the reference counts for blocks that are repeatedly re-referenced within a short interval remaining unchanged. On a miss, the block with the smallest reference

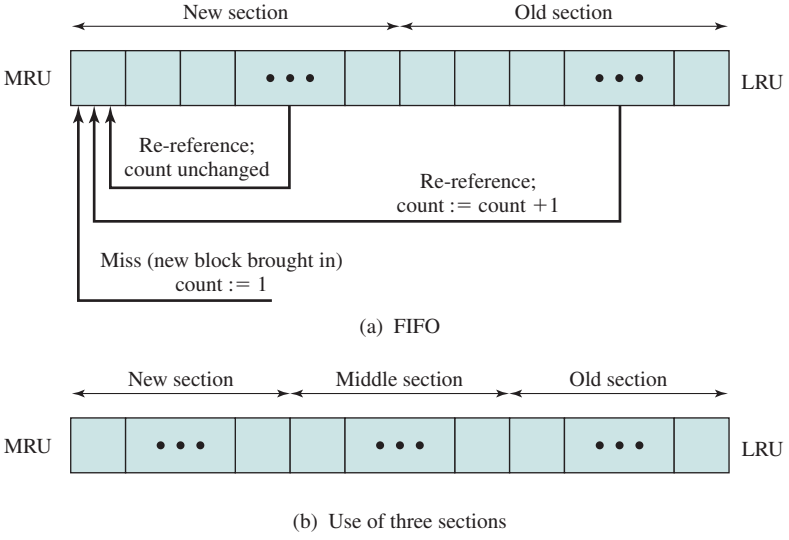



**Figure 11.9 Frequency-Based Replacement**

count that is not in the new section is chosen for replacement; the least recently used such block is chosen in the event of a tie.

The authors report this strategy achieved only slight improvement over LRU. The problem is the following:

1. On a cache miss, a new block is brought into the new section, with a count of 1.
2. The count remains at 1 as long as the block remains in the new section.
3. Eventually the block ages out of the new section, with its count still at 1.
4. If the block is not now re-referenced fairly quickly, it is very likely to be replaced because it necessarily has the smallest reference count of those blocks that are not in the new section. In other words, there does not seem to be a sufficiently long interval for blocks aging out of the new section to build up their reference counts, even if they were relatively frequently referenced.

A further refinement addresses this problem: Divide the stack into three sections: new, middle, and old (see Figure 11.9b). As before, reference counts are not incremented on blocks in the new section. However, only blocks in the old section are eligible for replacement. Assuming a sufficiently large middle section, this allows relatively frequently referenced blocks a chance to build up their reference counts before becoming eligible for replacement. Simulation studies by the authors indicate this refined policy is significantly better than simple LRU or LFU.

Regardless of the particular replacement strategy, the replacement can take place on demand or preplanned. In the former case, a sector is replaced only when the slot is needed. In the latter case, a number of slots are released at a time. The reason for this latter approach is related to the need to write back sectors. If a sector is brought into the cache and only read, then when it is replaced, it is not necessary to write it back out to the disk. However, if the sector has been updated, then it is necessary to write it back out before replacing it. In this latter case, it makes sense to cluster the writing and to order the writing to minimize seek time.

## Performance Considerations

The same performance considerations discussed in Appendix 1A apply here. The issue of cache performance reduces itself to a question of whether a given miss ratio can be achieved. This will depend on the locality behavior of the disk references, the replacement algorithm, and other design factors. Principally, however, the miss ratio is a function of the size of the disk cache. Figure 11.10 summarizes results from several studies using LRU, one for a UNIX system running on a VAX [OUST85] and one for IBM mainframe operating systems [SMIT85]. Figure 11.11 shows results for simulation studies of the frequency-based replacement algorithm. A comparison of the two figures points out one of the risks of this sort of performance assessment.

The figures appear to show LRU outperforms the frequency-based replacement algorithm. However, when identical reference patterns using the same cache structure are compared, the frequency-based replacement algorithm is superior. Thus, the exact sequence of reference patterns, plus related design issues such as block size, will have a profound influence on the performance achieved.

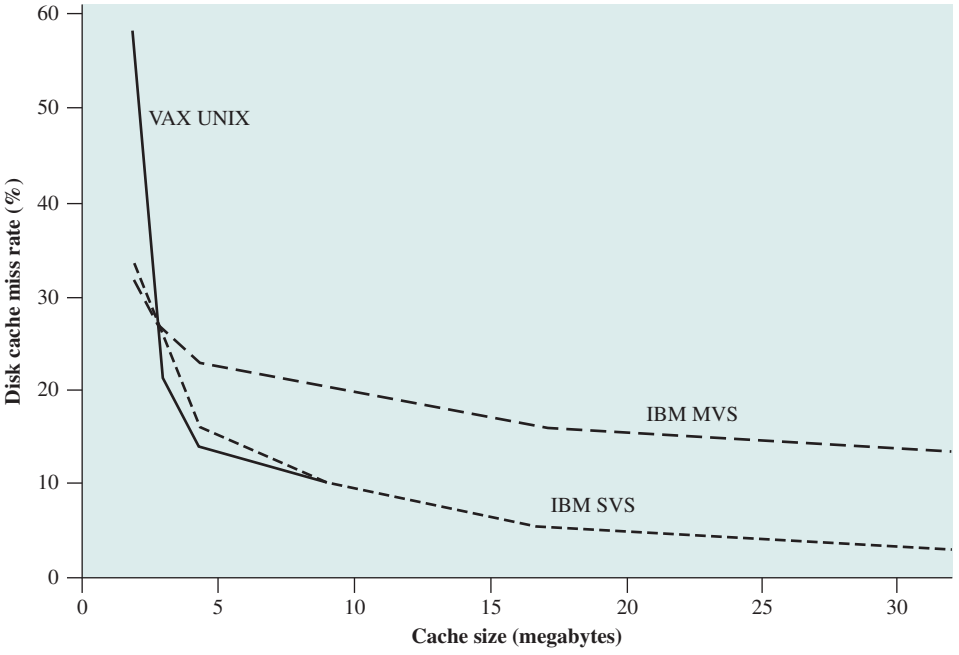



**Figure 11.10** Some Disk Cache Performance Results Using LRU

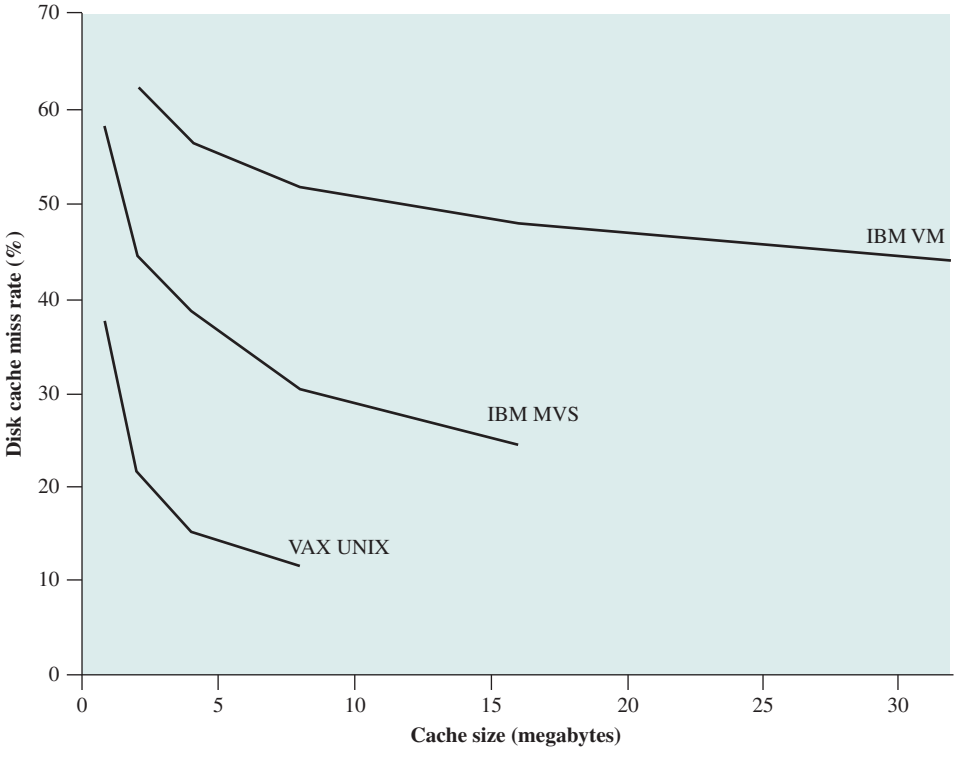



**Figure 11.11** Disk Cache Performance Using Frequency-Based Replacement

## 11.8 UNIX SVR4 I/O

In UNIX, each individual I/O device is associated with a special file. These are managed by the file system and are read and written in the same manner as user data files. This provides a clean, uniform interface to users and processes. To read from or write to a device, read and write requests are made for the special file associated with the device.

Figure 11.12 illustrates the logical structure of the I/O facility. The file subsystem manages files on secondary storage devices. In addition, it serves as the process interface to devices, because these are treated as files.

There are two types of I/O in UNIX: buffered and unbuffered. Buffered I/O passes through system buffers, whereas unbuffered I/O typically involves the DMA facility, with the transfer taking place directly between the I/O module and the process I/O area. For buffered I/O, two types of buffers are used: system buffer caches and character queues.

### Buffer Cache

The buffer cache in UNIX is essentially a disk cache. I/O operations with disk are handled through the buffer cache. The data transfer between the buffer cache and the user process space always occurs using DMA. Because both the buffer cache and the process I/O area are in main memory, the DMA facility is used in this case to perform a memory-to-memory copy. This does not use up any processor cycles, but it does consume bus cycles.

To manage the buffer cache, three lists are maintained:

1. **Free list:** List of all slots in the cache (a slot is referred to as a buffer in UNIX; each slot holds one disk sector) that are available for allocation
2. **Device list:** List of all buffers currently associated with each disk
3. **Driver I/O queue:** List of buffers that are actually undergoing or waiting for I/O on a particular device

All buffers should be on the free list or on the driver I/O queue list. A buffer, once associated with a device, remains associated with the device even if it is on the

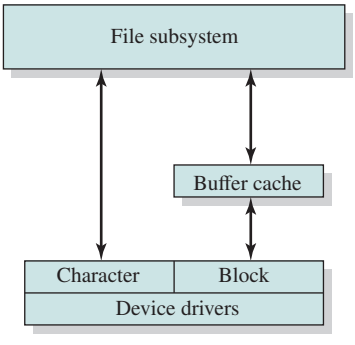



**Figure 11.12 UNIX I/O Structure**

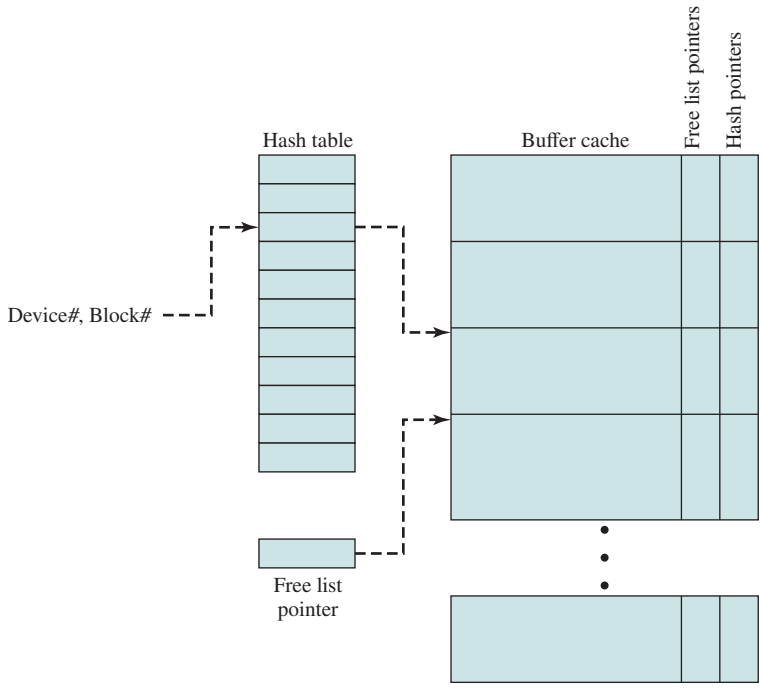



**Figure 11.13 UNIX Buffer Cache Organization**

free list, until is actually reused and becomes associated with another device. These lists are maintained as pointers associated with each buffer, rather than physically separate lists.

When a reference is made to a physical block number on a particular device, the OS first checks to see if the block is in the buffer cache. To minimize the search time, the device list is organized as a hash table, using a technique similar to the overflow with chaining technique discussed in Appendix F (see Figure F.1b). Figure 11.13 depicts the general organization of the buffer cache. There is a hash table of fixed length that contains pointers into the buffer cache. Each reference to a (device#, block#) maps into a particular entry in the hash table. The pointer in that entry points to the first buffer in the chain. A hash pointer associated with each buffer points to the next buffer in the chain for that hash table entry. Thus, for all (device#, block#) references that map into the same hash table entry, if the corresponding block is in the buffer cache, then that buffer will be in the chain for that hash table entry. Thus, the length of the search of the buffer cache is reduced by a factor on the order of $N$, where $N$ is the length of the hash table.

For block replacement, a least-recently-used algorithm is used: After a buffer has been allocated to a disk block, it cannot be used for another block until all other buffers have been used more recently. The free list preserves this least-recently-used order.

### Character Queue

Block-oriented devices, such as disk and USB keys, can be effectively served by the buffer cache. A different form of buffering is appropriate for character-oriented devices, such as terminals and printers. A character queue is either written by the I/O device and read by the process, or written by the process and read by the device. In both cases, the producer/consumer model introduced in Chapter 5 is used. Thus, character queues may only be read once; as each character is read, it is effectively destroyed. This is in contrast to the buffer cache, which may be read multiple times and hence follows the readers/writers model (also discussed in Chapter 5).

### Unbuffered I/O

Unbuffered I/O, which is simply DMA between device and process space, is always the fastest method for a process to perform I/O. A process that is performing unbuffered I/O is locked in main memory and cannot be swapped out. This reduces the opportunities for swapping by tying up part of main memory, thus reducing the overall system performance. Also, the I/O device is tied up with the process for the duration of the transfer, making it unavailable for other processes.

### UNIX Devices

Among the categories of devices recognized by UNIX are the following:

- Disk drives
- Tape drives
- Terminals
- Communication lines
- Printers

Table 11.5 shows the types of I/O suited to each type of device. Disk drives are heavily used in UNIX, are block oriented, and have the potential for reasonable high throughput. Thus, I/O for these devices tends to be unbuffered or via buffer cache. Tape drives are functionally similar to disk drives and use similar I/O schemes.

**Table 11.5** Device I/O in UNIX

| | Unbuffered I/O | Buffer Cache | Character Queue |
|---|---|---|---|
| Disk Drive | X | X | |
| Tape Drive | X | X | |
| Terminals | | | X |
| Communication Lines | | | X |
| Printers | X | | X |

Because terminals involve relatively slow exchange of characters, terminal I/O typically makes use of the character queue. Similarly, communication lines require serial processing of bytes of data for input or output and are best handled by character queues. Finally, the type of I/O used for a printer will generally depend on its speed. Slow printers will normally use the character queue, while a fast printer might employ unbuffered I/O. A buffer cache could be used for a fast printer. However, because data going to a printer are never reused, the overhead of the buffer cache is unnecessary.

## 11.9 LINUX I/O

In general terms, the Linux I/O kernel facility is very similar to that of other UNIX implementation, such as SVR4. Block and character devices are recognized. In this section, we look at several features of the Linux I/O facility.

### Disk Scheduling

The default disk scheduler in Linux 2.4 is known as the Linux Elevator, which is a variation on the LOOK algorithm discussed in Section 11.5. For Linux 2.6, the Elevator algorithm has been augmented by two additional algorithms: the deadline I/O scheduler and the anticipatory I/O scheduler [LOVE04]. We examine each of these in turn.

***The Elevator Scheduler*** The elevator scheduler maintains a single queue for disk read and write requests and performs both sorting and merging functions on the queue. In general terms, the elevator scheduler keeps the list of requests sorted by block number. Thus, as the disk requests are handled, the drive moves in a single direction, satisfying each request as it is encountered. This general strategy is refined in the following manner. When a new request is added to the queue, four operations are considered in order:

1. If the request is to the same on-disk sector or an immediately adjacent sector to a pending request in the queue, then the existing request and the new request are merged into one request.
2. If a request in the queue is sufficiently old, the new request is inserted at the tail of the queue.
3. If there is a suitable location, the new request is inserted in sorted order.
4. If there is no suitable location, the new request is placed at the tail of the queue.

***Deadline Scheduler*** Operation 2 in the preceding list is intended to prevent starvation of a request, but is not very effective [LOVE04]. It does not attempt to service requests in a given time frame, but merely stops insertion-sorting requests after a suitable delay. Two problems manifest themselves with the elevator scheme.

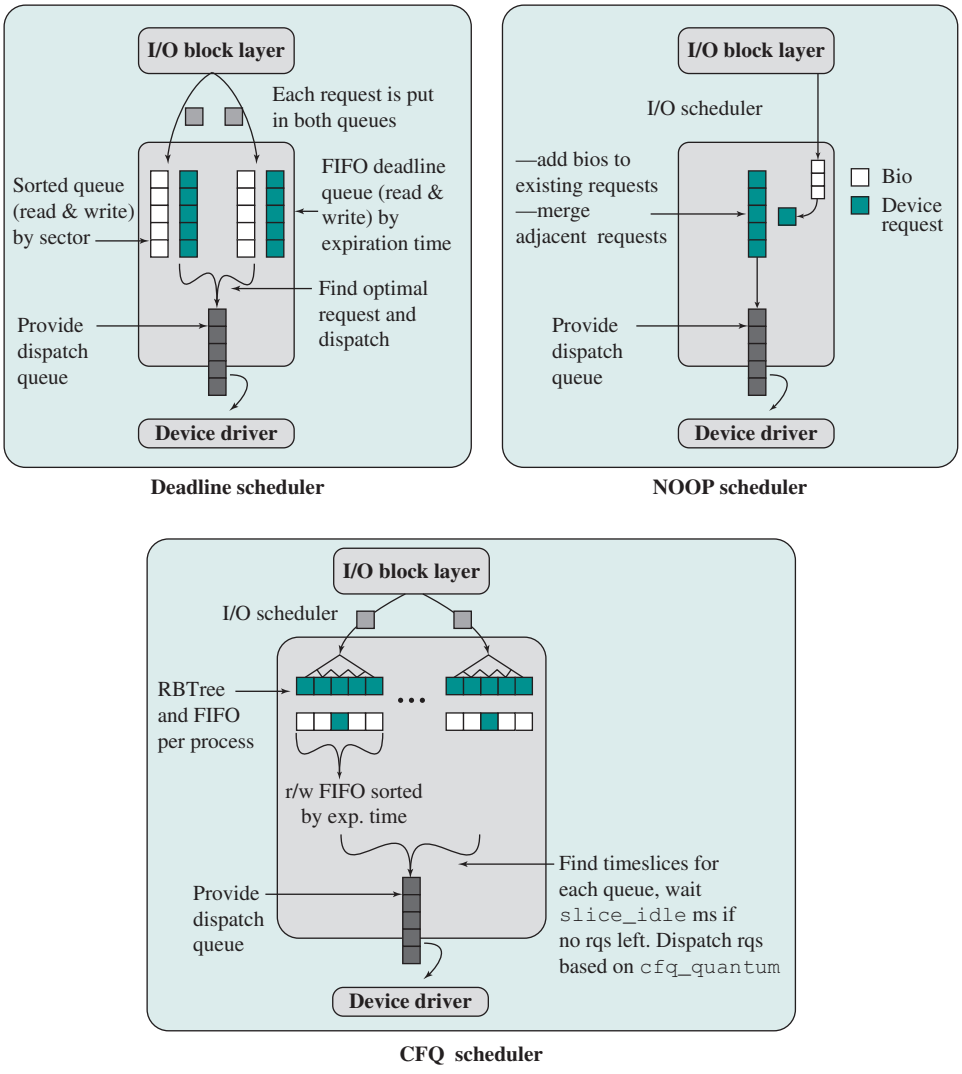



**Figure 11.14** **Linux I/O Schedulers**

The first problem is a distant block request can be delayed for a substantial time because the queue is dynamically updated. For example, consider the following stream of requests for disk blocks: 20, 30, 700, 25. The elevator scheduler reorders these so the requests are placed in the queue as 20, 25, 30, 700, with 20 being the head of the queue. If a continuous sequence of low-numbered block requests arrive, then the request for 700 continues to be delayed.

An even more serious problem concerns the distinction between read and write requests. Typically, a write request is issued asynchronously. That is, once a process issues the write request, it need not wait for the request to actually

be satisfied. When an application issues a write, the kernel copies the data into an appropriate buffer, to be written out as time permits. Once the data are captured in the kernel's buffer, the application can proceed. However, for many read operations, the process must wait until the requested data are delivered to the application before proceeding. Thus, a stream of write requests (e.g., to place a large file on the disk) can block a read request for a considerable time, and thus block a process.

To overcome these problems, a new deadline I/O scheduler was developed in 2002. This scheduler makes use of two pairs of queues (see Figure 11.14). Each incoming request is placed in a sorted elevator queue (read or write), as before. In addition, the same request is placed at the tail of a read FIFO queue for a read request or a write FIFO queue for a write request. Thus, the read and write queues maintain a list of requests in the sequence in which the requests were made. Associated with each request is an expiration time, with a default value of 0.5 seconds for a read request and of 5 seconds for a write request. Ordinarily, the scheduler dispatches from the sorted queue. When a request is satisfied, it is removed from the head of the sorted queue and of also from the appropriate FIFO queue. However, when the item at the head of one of the FIFO queues becomes older than its expiration time, then the scheduler next dispatches from that FIFO queue, taking the expired request, plus the next few requests from the queue. As each request is dispatched, it is also removed from the sorted queue.

The deadline I/O scheduler scheme overcomes the starvation problem and also the read versus write problem.

***ANTICIPATORY I/O SCHEDULER*** The original elevator scheduler and the deadline scheduler both are designed to dispatch a new request as soon as the existing request is satisfied, thus keeping the disk as busy as possible. This same policy applies to all of the scheduling algorithms discussed in Section 11.5. However, such a policy can be counterproductive if there are numerous synchronous read requests. Typically, an application will wait until a read request is satisfied and the data is available before issuing the next request. The small delay between receiving the data for the last read and issuing the next read enables the scheduler to turn elsewhere for a pending request and dispatch that request.

Because of the principle of locality, it is likely that successive reads from the same process will be to disk blocks that are near one another. If the scheduler were to delay a short period of time after satisfying a read request, to see if a new nearby read request is made, the overall performance of the system could be enhanced. This is the philosophy behind the anticipatory scheduler, proposed in [IYER01], and implemented in Linux 2.6.

In Linux, the anticipatory scheduler is superimposed on the deadline scheduler. When a read request is dispatched, the anticipatory scheduler causes the scheduling system to delay for up to 6 ms, depending on the configuration. During this small delay, there is a good chance that the application that issued the last read request will issue another read request to the same region of the disk. If so, that request will be serviced immediately. If no such read request occurs, the scheduler resumes using the deadline scheduling algorithm.

[LOVE04] reports on two tests of the Linux scheduling algorithms. The first test involved the reading of a 200-MB file while doing a long streaming write in the background. The second test involved doing a read of a large file in the background while reading every file in the kernel source tree. The results are listed in the following table:

| I/O Scheduler and Kernel | Test 1 | Test 2 |
|---|---|---|
| Linux elevator on 2.4 | 45 seconds | 30 minutes, 28 seconds |
| Deadline I/O scheduler on 2.6 | 40 seconds | 3 minutes, 30 seconds |
| Anticipatory I/O scheduler on 2.6 | 4.6 seconds | 15 seconds |

As can be seen, the performance improvement depends on the nature of the workload. But in both cases, the anticipatory scheduler provides a dramatic improvement. In Kernel 2.6.33, the anticipatory scheduler was removed from the kernel, due to adopting the CFQ scheduler (described subsequently).

***The NOOP Scheduler*** This is the simplest among Linux I/O schedulers. It is a minimal scheduler that inserts I/O requests into a FIFO queue and uses merging. Its main uses include nondisk-based block devices such as memory devices, and specialized software or hardware environments that do their own scheduling and need only minimal support in the kernel.

***Completely Fair Queuing I/O Scheduler*** The Completely Fair Queuing (CFQ) I/O scheduler was developed in 2003, and is the default I/O scheduler in Linux. The CFQ scheduler guarantees a fair allocation of the disk I/O bandwidth among all processes. It maintains per process I/O queues; each process is assigned a single queue. Each queue has an allocated timeslice. Requests are submitted into these queues and are processed in round robin.

When the scheduler services a specific queue, and there are no more requests in that queue, it waits in idle mode for a predefined time interval for new requests, and if there are no requests, it continues to the next queue. This optimization improves performance in the case that there are more requests in that time interval.

We should note the I/O scheduler can be set as a boot parameter in grub or in run time, for example, by echoing "noop","deadline",or "cfq" into */sys/class/block/sda/queue/scheduler*. There are also several optimization sysfs scheduler settings, which are described in the Linux kernel documentation.

## Linux Page Cache

In Linux 2.2 and earlier releases, the kernel maintained a page cache for reads and writes from regular file system files and for virtual memory pages, and a separate buffer cache for block I/O. For Linux 2.4 and later, there is a single unified page cache that is involved in all traffic between disk and main memory.

The page cache confers two benefits. First, when it is time to write back dirty pages to disk, a collection of them can be ordered properly and written out efficiently.

Second, because of the principle of temporal locality, pages in the page cache are likely to be referenced again before they are flushed from the cache, thus saving a disk I/O operation.

Dirty pages are written back to disk in two situations:

1. When free memory falls below a specified threshold, the kernel reduces the size of the page cache to release memory to be added to the free memory pool.
2. When dirty pages grow older than a specified threshold, a number of dirty pages are written back to disk.

## 11.10 WINDOWS I/O

Figure 11.15 shows the key kernel-mode components related to the Windows I/O manager. The I/O manager is responsible for all I/O for the operating system and provides a uniform interface that all types of drivers can call.

### Basic I/O Facilities

The I/O manager works closely with four types of kernel components:

1. **Cache manager:** The cache manager handles file caching for all file systems. It can dynamically increase and decrease the size of the cache devoted to a particular file as the amount of available physical memory varies. The system records updates in the cache only and not on disk. A kernel thread, the lazy writer, periodically batches the updates together to write to disk. Writing the updates in batches allows the I/O to be more efficient. The cache manager works by mapping regions of files into kernel virtual memory then relying on the virtual memory manager to do most of the work to copy pages to and from the files on disk.
2. **File system drivers:** The I/O manager treats a file system driver as just another device driver and routes I/O requests for file system volumes to the appropriate software driver for that volume. The file system, in turn, sends I/O requests to the software drivers that manage the hardware device adapter.
3. **Network drivers:** Windows includes integrated networking capabilities and support for remote file systems. The facilities are implemented as software drivers rather than part of the Windows Executive.

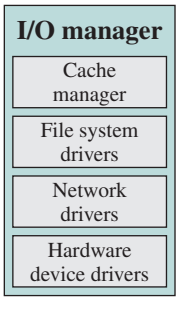



**Figure 11.15 Windows I/O Manager**

4. **Hardware device drivers:** These software drivers access the hardware registers of the peripheral devices using entry points in the Hardware Abstraction Layer. A set of these routines exists for every platform that Windows supports; because the routine names are the same for all platforms, the source code of Windows device drivers is portable across different processor types.

## Asynchronous and Synchronous I/O

Windows offers two modes of I/O operation: asynchronous and synchronous. The asynchronous mode is used whenever possible to optimize application performance. With asynchronous I/O, an application initiates an I/O operation then can continue processing while the I/O request is fulfilled. With synchronous I/O, the application is blocked until the I/O operation completes.

Asynchronous I/O is more efficient, from the point of view of the calling thread, because it allows the thread to continue execution while the I/O operation is queued by the I/O manager and subsequently performed. However, the application that invoked the asynchronous I/O operation needs some way to determine when the operation is complete. Windows provides five different techniques for signaling I/O completion:

1. **Signaling the file object:** With this approach, the event associated with a file object is set when an operation on that object is complete. The thread that invoked the I/O operation can continue to execute until it reaches a point where it must stop until the I/O operation is complete. At that point, the thread can wait until the operation is complete then continue. This technique is simple and easy to use but is not appropriate for handling multiple I/O requests. For example, if a thread needs to perform multiple simultaneous actions on a single file (such as reading from one portion and writing to another portion of the file) with this technique the thread could not distinguish between the completion of the read and the completion of the write. It would simply know that one of the requested I/O operations on this file had finished.
2. **Signaling an event object:** This technique allows multiple simultaneous I/O requests against a single device or file. The thread creates an event for each request. Later, the thread can wait on a single one of these requests or on an entire collection of requests.
3. **Asynchronous procedure call:** This technique makes use of a queue associated with a thread, known as the asynchronous procedure call (APC) queue. In this case, the thread makes I/O requests, specifying a user-mode routine to call when the I/O completes. The I/O manager places the results of each request in the calling thread's APC queue. The next time the thread blocks in the kernel, the APCs will be delivered, each causing the thread to return to user mode and execute the specified routine.
4. **I/O completion ports:** This technique is used on a Windows server to optimize the use of threads. The application creates a pool of threads for handling the completion of I/O requests. Each thread waits on the completion port, and the kernel wakes threads to handle each I/O completion. One of the advantages of this approach is that the application can specify a limit for how many of these threads will run at the same time.

5. **Polling:** Asynchronous I/O requests write a status and transfer count into the process's user virtual memory when the operation completes. A thread can just check these values to see if the operation has completed.

### Software RAID

Windows supports two sorts of RAID configurations, defined in [MS96] as follows:

1. **Hardware RAID:** Separate physical disks combined into one or more logical disks by the disk controller or disk storage cabinet hardware
2. **Software RAID:** Noncontiguous disk space combined into one or more logical partitions by the fault-tolerant software disk driver, FTDISK

In hardware RAID, the controller interface handles the creation and regeneration of redundant information. The software RAID, available on Windows Server, implements the RAID functionality as part of the operating system and can be used with any set of multiple disks. The software RAID facility implements RAID 1 and RAID 5. In the case of RAID 1 (disk mirroring), the two disks containing the primary and mirrored partitions may be on the same disk controller or different disk controllers. The latter configuration is referred to as *disk duplexing*.

### Volume Shadow Copies

Shadow copies are an efficient way of making consistent snapshots of volumes so they can be backed up. They are also useful for archiving files on a per-volume basis. If a user deletes a file, he or she can retrieve an earlier copy from any available shadow copy made by the system administrator. Shadow copies are implemented by a software driver that makes copies of data on the volume before it is overwritten.

### Volume Encryption

Windows supports the encryption of entire volumes, using a feature called BitLocker. This is more secure than encrypting individual files, as the entire system works to be sure the data is safe. Up to three different methods of supplying the cryptographic key can be provided, allowing multiple interlocking layers of security.

## 11.11 SUMMARY

The computer system's interface to the outside world is its I/O architecture. This architecture is designed to provide a systematic means of controlling interaction with the outside world, and to provide the operating system with the information it needs to manage I/O activity effectively.

The I/O function is generally broken up into a number of layers, with lower layers dealing with details that are closer to the physical functions to be performed, and higher layers dealing with I/O in a logical and generic fashion. The result is changes in hardware parameters need not affect most of the I/O software.

A key aspect of I/O is the use of buffers that are controlled by I/O utilities rather than by application processes. Buffering smoothes out the differences between the internal speeds of the computer system and the speeds of I/O devices. The use of buffers also decouples the actual I/O transfer from the address space of the application process. This allows the operating system more flexibility in performing its memory management function.

The aspect of I/O that has the greatest impact on overall system performance is disk I/O. Accordingly, there has been greater research and design effort in this area than in any other kind of I/O. Two of the most widely used approaches to improve disk I/O performance are disk scheduling and the disk cache.

At any time, there may be a queue of requests for I/O on the same disk. It is the object of disk scheduling to satisfy these requests in a way that minimizes the mechanical seek time of the disk and hence improves performance. The physical layout of pending requests plus considerations of locality come into play.

A disk cache is a buffer, usually kept in main memory, that functions as a cache of disk blocks between disk memory and the rest of main memory. Because of the principle of locality, the use of a disk cache should substantially reduce the number of block I/O transfers between main memory and disk.

## 11.12 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

### Key Terms

| | | |
|---|---|---|
| block | interrupt-driven I/O | redundant array of independent disks |
| block-oriented device | input/output (I/O) | rotational delay |
| buffer swapping | least frequently used (LFU) | sector |
| circular buffer | I/O buffer | seek time |
| device I/O | I/O channel | stream-oriented device |
| direct memory access | I/O processor | stripe |
| disk access time | logical I/O | track |
| disk cache | magnetic disk | transfer time |
| double buffering | programmed I/O | |
| gap | read/write head | |

### Review Questions

**11.1.** List and briefly define three techniques for performing I/O.

**11.2.** What are the differences between a blocking I/O and a nonblocking I/O?

**11.3.** What is the difference between block-oriented devices and stream-oriented devices? Give a few examples of each.

**11.4.** Why would you expect improved performance using a double buffer rather than a single buffer for I/O?

**11.5.** State some utilities of buffering.

**11.6.** Briefly define the disk scheduling policies illustrated in Figure 11.7.

**11.7.** Cite the differences between the implementation of hardware RAID and software RAID.

**11.8.** What is a Linux Elevator? Point out some problems associated with it.

## Problems

**11.1.** Consider a program that accesses a single I/O device and compare unbuffered I/O to the use of a buffer. Show that the use of the buffer can reduce the running time by at most a factor of two.

**11.2.** Generalize the result of Problem 11.1 to the case in which a program refers to $n$ devices.

**11.3.** Consider a disk drive with 4,000 cylinders, numbered from 0 to 3,999. The request queue has the following composition:

1045 750 932 878 1365 1787 1245 664 1678 1897

If the current position is 1167 and the previous request was served at 1250, compute the total distance (in cylinders) that the disk arm would move for each of the following algorithms: FIFO, SSTF, SCAN, and C-SCAN scheduling.

**11.4.** Consider a disk with $N$ tracks numbered from 0 to $(N - 1)$ and assume requested sectors are distributed randomly and evenly over the disk. We want to calculate the average number of tracks traversed by a seek.

**a.** Calculate the probability of a seek of length $j$ when the head is currently positioned over track $t$. (*Hint:* This is a matter of determining the total number of combinations, recognizing that all track positions for the destination of the seek are equally likely.)

**b.** Calculate the probability of a seek of length $K$, for an arbitrary current position of the head. (*Hint:* This involves the summing over all possible combinations of movements of $K$ tracks.)

**c.** Calculate the average number of tracks traversed by a seek, using the formula for expected value

$$\mathrm{E}[x] = \sum_{i=0}^{N-1} i \times \Pr[x = i]$$

*Hint*: Use the equalities $\sum_{i=1}^{n} = \frac{n(n+1)}{2}$; $\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$.

**d.** Show that for large values of $N$, the average number of tracks traversed by a seek approaches $N/3$.

**11.5.** It has been found that in a certain disk drive about 80% of all requests are for a small, fixed number of cylinders. Which of the scheduling algorithms will you recommend for this situation? Justify your choice.

**11.6.** In a single-user system, determine whether buffering, spooling, caching, or a combination of these should be used for each of the following I/O scenarios:

**a.** A mouse used with a graphical user interface.

**b.** A tape drive on a multitasking OS.

**c.** A disk drive containing user files.

**11.7.** Calculate how much disk space (in sectors, tracks, and surfaces) will be required to store 250,000 200-byte logical records if the disk is fixed sector with 1024 bytes/sector, with 108 sectors/track, 140 tracks per surface, and 12 usable surfaces. Ignore any file header record(s) and track indexes, and assume that records cannot span two sectors.

**11.8.** Consider the disk system described in Problem 11.7, and assume that the disk rotates at 1,200 rpm. A processor reads one sector from the disk using interrupt-driven I/O, with one interrupt every 4 bytes. If it takes 1.5 μs to process each interrupt, what percentage of the time will the processor spend handling I/O (disregard seek time)?

**11.9.** Repeat the preceding problem using DMA, and assume one interrupt per sector.

**11.10.** A 32-bit computer has two selector channels and one multiplexor channel. Each selector channel supports two magnetic disk and three magnetic tape units. The multiplexor channel has three line printers, two card readers, and twelve VDT terminals connected to it. Assume the following transfer rates:

| | |
|---|---|
| Disk drive | 1100 Kbytes/s |
| Magnetic tape drive | 400 Kbytes/s |
| Line printer | 7.8 Kbytes/s |
| Card reader | 1.6 Kbytes/s |
| VDT | 1.2 Kbyte/s |

Estimate the maximum aggregate I/O transfer rate in this system.

**11.11.** A disk pack has the following specifications: it comprises 25 double sided disks; each surface of a disk has 480 tracks and a track has 20 blocks in it. Each block is of 2048 bytes, with an inter-block gap of 64 bytes.

Compute the total capacity of a track, the useful capacity of a track (excluding inter-block gap), the total capacity and useful capacity of a cylinder, the total capacity and useful capacity of the disk pack, and the percentage of space wasted.

**11.12.** In a certain device, the disk rotates at 7,500 rpm. What is the average rotational delay of this disk drive?

CHAPTER 12

# File Management


12.1 **Overview**
- Files and File Systems
- File Structure
- File Management Systems

12.2 **File Organization and Access**
- The Pile
- The Sequential File
- The Indexed Sequential File
- The Indexed File
- The Direct or Hashed File

12.3 **B-Trees**

12.4 **File Directories**
- Contents
- Structure
- Naming

12.5 **File Sharing**
- Access Rights
- Simultaneous Access

12.6 **Record Blocking**

12.7 **Secondary Storage Management**
- File Allocation
- Free Space Management
- Volumes
- Reliability

12.8 **UNIX File Management**
- Inodes
- File Allocation
- Directories
- Volume Structure

12.9 **Linux Virtual File System**
- The Superblock Object
- The Inode Object
- The Dentry Object
- The File Object
- Caches

12.10 **Windows File System**
- Key Features of NTFS
- NTFS Volume and File Structure
- Recoverability

12.11 **Android File Management**
- File System
- SQLite

12.12 **Summary**

12.13 **Key Terms, Review Questions, and Problems**

**Learning Objectives**

After studying this chapter, you should be able to:

- Describe the basic concepts of files and file systems.
- Understand the principal techniques for file organization and access.
- Define B-trees.
- Explain file directories.
- Understand the requirements for file sharing.
- Understand the concept of record blocking.
- Describe the principal design issues for secondary storage management.
- Understand the design issues for file system security.
- Explain the OS file systems used in Linux, UNIX, and Windows.

In most applications, the file is the central element. With the exception of real-time applications and some other specialized applications, the input to the application is by means of a file. And in virtually all applications, output is saved in a file for long-term storage, and for later access by the user and by other programs.

Files have a life outside of any individual application that uses them for input and/or output. Users wish to be able to access files, save them, and maintain the integrity of their contents. To aid in these objectives, virtually all operating systems provide file management systems. Typically, a file management system consists of system utility programs that run as privileged applications. However, at the very least, a file management system needs special services from the operating system; at the most, the entire file management system is considered part of the operating system. Thus, it is appropriate to consider the basic elements of file management in this book.

We begin with an overview, followed by a look at various file organization schemes. Although file organization is generally beyond the scope of the operating system, it is essential to have a general understanding of the common alternatives to appreciate some of the design trade-offs involved in file management. The remainder of this chapter looks at other topics in file management.

## 12.1 OVERVIEW

### Files and File Systems

From the user's point of view, one of the most important parts of an operating system is the file system. The file system permits users to create data collections, called files, with desirable properties, such as:

- **Long-term existence:** Files are stored on disk or other secondary storage and do not disappear when a user logs off.
- **Sharable between processes:** Files have names and can have associated access permissions that permit controlled sharing.

- **Structure:** Depending on the file system, a file can have an internal structure that is convenient for particular applications. In addition, files can be organized into a hierarchical or more complex structure to reflect the relationships among files.

Any file system provides not only a means to store data organized as files, but a collection of functions that can be performed on files. Typical operations include the following:

- **Create:** A new file is defined and positioned within the structure of files.
- **Delete:** A file is removed from the file structure and subsequently destroyed.
- **Open:** An existing file is declared to be "opened" by a process, allowing the process to perform functions on the file.
- **Close:** The file is closed with respect to a process, so the process no longer may perform functions on the file, until the process opens the file again.
- **Read:** A process reads all or a portion of the data in a file.
- **Write:** A process updates a file, either by adding new data that expands the size of the file, or by changing the values of existing data items in the file.

Typically, a file system maintains a set of attributes associated with the file. These include owner, creation time, time last modified, and access privileges.

## File Structure

Four terms are in common use when discussing files:

- Field
- Record
- File
- Database

A **field** is the basic element of data. An individual field contains a single value, such as an employee's last name, a date, or the value of a sensor reading. It is characterized by its length and data type (e.g., ASCII string, decimal). Depending on the file design, fields may be fixed length or variable length. In the latter case, the field often consists of two or three subfields: the actual value to be stored, the name of the field, and, in some cases, the length of the field. In other cases of variable-length fields, the length of the field is indicated by the use of special demarcation symbols between fields.

A **record** is a collection of related fields that can be treated as a unit by some application program. For example, an employee record would contain such fields as name, social security number, job classification, date of hire, and so on. Again, depending on design, records may be of fixed length or variable length. A record will be of variable length if some of its fields are of variable length or if the number of fields may vary. In the latter case, each field is usually accompanied by a field name. In either case, the entire record usually includes a length field.

A **file** is a collection of similar records. The file is treated as a single entity by users and applications and may be referenced by name. Files have file names and

may be created and deleted. Access control restrictions usually apply at the file level. That is, in a shared system, users and programs are granted or denied access to entire files. In some more sophisticated systems, such controls are enforced at the record or even the field level.

Some file systems are structured only in terms of fields, not records. In that case, a file is a collection of fields.

A **database** is a collection of related data. The essential aspects of a database are that the relationships that exist among elements of data are explicit, and that the database is designed for use by a number of different applications. A database may contain all of the information related to an organization or a project, such as a business or a scientific study. The database itself consists of one or more types of files. Usually, there is a separate database management system that is independent of the operating system, although that system may make use of some file management programs.

Users and applications wish to make use of files. Typical operations that must be supported include the following:

- `Retrieve_All`: Retrieve all the records of a file. This will be required for an application that must process all of the information in the file at one time. For example, an application that produces a summary of the information in the file would need to retrieve all records. This operation is often equated with the term *sequential processing*, because all of the records are accessed in sequence.
- `Retrieve_One`: This requires the retrieval of just a single record. Interactive, transaction-oriented applications need this operation.
- `Retrieve_Next`: This requires the retrieval of the record that is "next" in some logical sequence to the most recently retrieved record. Some interactive applications, such as filling in forms, may require such an operation. A program that is performing a search may also use this operation.
- `Retrieve_Previous`: Similar to `Retrieve_Next`, but in this case the record that is "previous" to the currently accessed record is retrieved.
- `Insert_One`: Insert a new record into the file. It may be necessary that the new record fit into a particular position to preserve a sequencing of the file.
- `Delete_One`: Delete an existing record. Certain linkages or other data structures may need to be updated to preserve the sequencing of the file.
- `Update_One`: Retrieve a record, update one or more of its fields, and rewrite the updated record back into the file. Again, it may be necessary to preserve sequencing with this operation. If the length of the record has changed, the update operation is generally more difficult than if the length is preserved.
- `Retrieve_Few`: Retrieve a number of records. For example, an application or user may wish to retrieve all records that satisfy a certain set of criteria.

The nature of the operations most commonly performed on a file will influence the way the file is organized, as discussed in Section 12.2.

It should be noted that not all file systems exhibit the sort of structure discussed in this subsection. On UNIX and UNIX-like systems, the basic file structure is just a stream of bytes. For example, a C program is stored as a file but does not have physical fields, records, and so on.

## File Management Systems

A file management system is that set of system software that provides services to users and applications in the use of files. Typically, the only way a user or application may access files is through the file management system. This relieves the user or programmer of the necessity of developing special-purpose software for each application and provides the system with a consistent, well-defined means of controlling its most important asset. [GROS86] suggests the following objectives for a file management system:

- To meet the data management needs and requirements of the user, which include storage of data and the ability to perform the aforementioned operations
- To guarantee, to the extent possible, that the data in the file are valid
- To optimize performance, both from the system point of view in terms of overall throughput, and from the user's point of view in terms of response time
- To provide I/O support for a variety of storage device types
- To minimize or eliminate the potential for lost or destroyed data
- To provide a standardized set of I/O interface routines to user processes
- To provide I/O support for multiple users, in the case of multiple-user systems

With respect to the first point, meeting user requirements, the extent of such requirements depends on the variety of applications and the environment in which the computer system will be used. For an interactive, general-purpose system, the following constitute a minimal set of requirements:

1. Each user should be able to create, delete, read, write, and modify files.
2. Each user may have controlled access to other users' files.
3. Each user may control what types of accesses are allowed to the user's files.
4. Each user should be able to move data between files.
5. Each user should be able to back up and recover the user's files in case of damage.
6. Each user should be able to access his or her files by name rather than by numeric identifier.

These objectives and requirements should be kept in mind throughout our discussion of file management systems.

***File System Architecture*** One way of getting a feel for the scope of file management is to look at a depiction of a typical software organization, as suggested in Figure 12.1. Of course, different systems will be organized differently, but this organization is reasonably representative. At the lowest level, **device drivers** communicate directly with peripheral devices or their controllers or channels. A device driver is responsible for starting I/O operations on a device and processing the completion of an I/O request. For file operations, the typical devices controlled are disk and tape drives. Device drivers are usually considered to be part of the operating system.

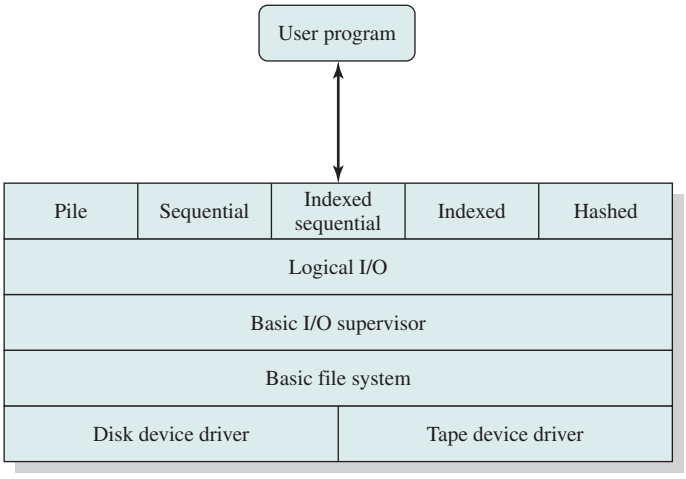



**Figure 12.1** **File System Software Architecture**

The next level is referred to as the **basic file system**, or the **physical I/O** level. This is the primary interface with the environment outside of the computer system. It deals with blocks of data that are exchanged with disk or tape systems. Thus, it is concerned with the placement of those blocks on the secondary storage device and on the buffering of those blocks in main memory. It does not understand the content of the data or the structure of the files involved. The basic file system is often considered part of the operating system.

The **basic I/O supervisor** is responsible for all file I/O initiation and termination. At this level, control structures are maintained that deal with device I/O, scheduling, and file status. The basic I/O supervisor selects the device on which file I/O is to be performed, based on the particular file selected. It is also concerned with scheduling disk and tape accesses to optimize performance. I/O buffers are assigned and secondary memory is allocated at this level. The basic I/O supervisor is part of the operating system.

**Logical I/O** enables users and applications to access records. Thus, whereas the basic file system deals with blocks of data, the logical I/O module deals with file records. Logical I/O provides a general-purpose record I/O capability and maintains basic data about files.

The level of the file system closest to the user is often termed the **access method**. It provides a standard interface between applications and the file systems and devices that hold the data. Different access methods reflect different file structures and different ways of accessing and processing the data. Some of the most common access methods are shown in Figure 12.1, and these are briefly described in Section 12.2.

***File Management Functions*** Another way of viewing the functions of a file system is shown in Figure 12.2. Let us follow this diagram from left to right. Users and application programs interact with the file system by means of commands for creating and deleting files and for performing operations on files. Before performing any

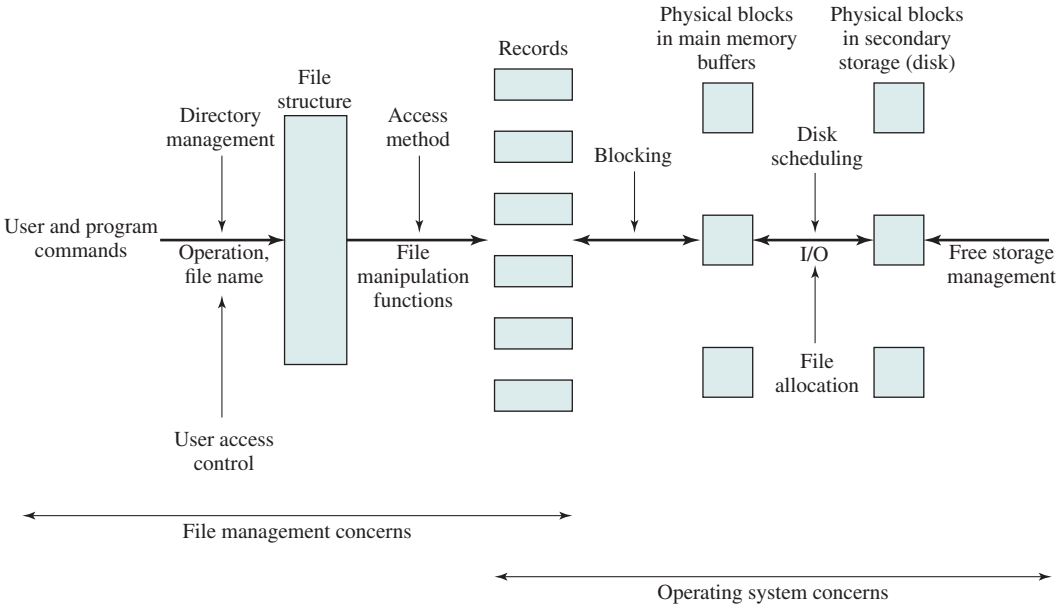



**Figure 12.2 Elements of File Management**

operation, the file system must identify and locate the selected file. This requires the use of some sort of directory that serves to describe the location of all files, plus their attributes. In addition, most shared systems enforce user access control: Only authorized users are allowed to access particular files in particular ways. The basic operations that a user or an application may perform on a file are performed at the record level. The user or application views the file as having some structure that organizes the records, such as a sequential structure (e.g., personnel records are stored alphabetically by last name). Thus, to translate user commands into specific file manipulation commands, the access method appropriate to this file structure must be employed.

Whereas users and applications are concerned with records or fields, I/O is done on a block basis. Thus, the records or fields of a file must be organized as a sequence of blocks for output and unblocked after input. To support block I/O of files, several functions are needed. The secondary storage must be managed. This involves allocating files to free blocks on secondary storage and managing free storage so as to know what blocks are available for new files and growth in existing files. In addition, individual block I/O requests must be scheduled; this issue was dealt with in Chapter 11. Both disk scheduling and file allocation are concerned with optimizing performance. As might be expected, these functions therefore need to be considered together. Furthermore, the optimization will depend on the structure of the files and the access patterns. Accordingly, developing an optimum file management system from the point of view of performance is an exceedingly complicated task.

Figure 12.2 suggests a division between what might be considered the concerns of the file management system as a separate system utility and the concerns of the operating system, with the point of intersection being record processing. This division is arbitrary; various approaches are taken in various systems.

In the remainder of this chapter, we look at some of the design issues suggested in Figure 12.2. We begin with a discussion of file organizations and access methods. Although this topic is beyond the scope of what is usually considered the concerns of the operating system, it is impossible to assess the other file-related design issues without an appreciation of file organization and access. Next, we look at the concept of file directories. These are often managed by the operating system on behalf of the file management system. The remaining topics deal with the physical I/O aspects of file management and are properly treated as aspects of OS design. One such issue is the way in which logical records are organized into physical blocks. Finally, there are the related issues of file allocation on secondary storage and the management of free secondary storage.

## 12.2 FILE ORGANIZATION AND ACCESS

In this section, we use the term *file organization* to refer to the logical structuring of the records as determined by the way in which they are accessed. The physical organization of the file on secondary storage depends on the blocking strategy and the file allocation strategy, issues dealt with later in this chapter.

In choosing a file organization, several criteria are important:

- Short access time
- Ease of update
- Economy of storage
- Simple maintenance
- Reliability

The relative priority of these criteria will depend on the applications that will use the file. For example, if a file is only to be processed in batch mode, with all of the records accessed every time, then rapid access for retrieval of a single record is of minimal concern. A file stored on CD-ROM will never be updated, and so ease of update is not an issue.

These criteria may conflict. For example, for economy of storage, there should be minimum redundancy in the data. On the other hand, redundancy is a primary means of increasing the speed of access to data. An example of this is the use of indexes.

The number of alternative file organizations that have been implemented or just proposed is unmanageably large, even for a book devoted to file systems. In this brief survey, we will outline five fundamental organizations. Most structures used in actual systems either fall into one of these categories, or can be implemented with a combination of these organizations. The five organizations, the first four of which are depicted in Figure 12.3, are as follows:

1. The pile
2. The sequential file
3. The indexed sequential file
4. The indexed file
5. The direct, or hashed, file

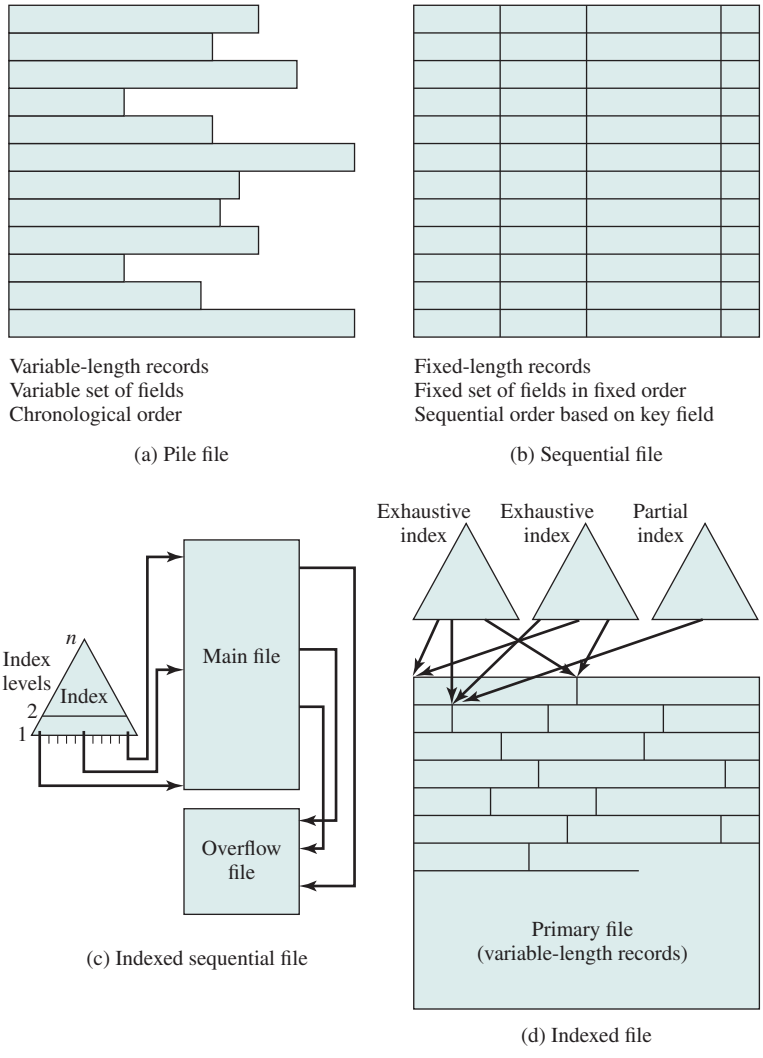



**Figure 12.3 Common File Organizations**

## The Pile

The least complicated form of file organization may be termed the *pile*. Data are collected in the order in which they arrive. Each record consists of one burst of data. The purpose of the pile is simply to accumulate the mass of data and save it. Records may have different fields, or similar fields in different orders. Thus, each field should be self-describing, including a field name as well as a value. The length of each field must be implicitly indicated by delimiters, explicitly included as a subfield, or known as default for that field type.

Because there is no structure to the pile file, record access is by exhaustive search. That is, if we wish to find a record that contains a particular field with a particular value, it is necessary to examine each record in the pile until the desired record

is found or the entire file has been searched. If we wish to find all records that contain a particular field or contain that field with a particular value, then the entire file must be searched.

Pile files are encountered when data are collected and stored prior to processing or when data are not easy to organize. This type of file uses space well when the stored data vary in size and structure, is perfectly adequate for exhaustive searches, and is easy to update. However, beyond these limited uses, this type of file is unsuitable for most applications.

### The Sequential File

The most common form of file structure is the sequential file. In this type of file, a fixed format is used for records. All records are of the same length, consisting of the same number of fixed-length fields in a particular order. Because the length and position of each field are known, only the values of fields need to be stored; the field name and length for each field are attributes of the file structure.

One particular field, usually the first field in each record, is referred to as the **key field**. The key field uniquely identifies the record; thus key values for different records are always different. Further, the records are stored in key sequence: alphabetical order for a text key, and numerical order for a numerical key.

Sequential files are typically used in batch applications and are generally optimum for such applications if they involve the processing of all the records (e.g., a billing or payroll application). The sequential file organization is the only one that is easily stored on tape as well as disk.

For interactive applications that involve queries and/or updates of individual records, the sequential file provides poor performance. Access requires the sequential search of the file for a key match. If the entire file, or a large portion of the file, can be brought into main memory at one time, more efficient search techniques are possible. Nevertheless, considerable processing and delay are encountered to access a record in a large sequential file. Additions to the file also present problems. Typically, a sequential file is stored in simple sequential ordering of the records within blocks. That is, the physical organization of the file on tape or disk directly matches the logical organization of the file. In this case, the usual procedure is to place new records in a separate pile file, called a log file or transaction file. Periodically, a batch update is performed that merges the log file with the master file to produce a new file in correct key sequence.

An alternative is to organize the sequential file physically as a linked list. One or more records are stored in each physical block. Each block on disk contains a pointer to the next block. The insertion of new records involves pointer manipulation but does not require that the new records occupy a particular physical block position. Thus, some added convenience is obtained at the cost of additional processing and overhead.

### The Indexed Sequential File

A popular approach to overcoming the disadvantages of the sequential file is the indexed sequential file. The indexed sequential file maintains the key characteristic of the sequential file: Records are organized in sequence based on a key field. Two

features are added: an index to the file to support random access, and an overflow file. The index provides a lookup capability to reach quickly the vicinity of a desired record. The overflow file is similar to the log file used with a sequential file but is integrated so a record in the overflow file is located by following a pointer from its predecessor record.

In the simplest indexed sequential structure, a single level of indexing is used. The index in this case is a simple sequential file. Each record in the index file consists of two fields: a key field, which is the same as the key field in the main file, and a pointer into the main file. To find a specific field, the index is searched to find the highest key value that is equal to or precedes the desired key value. The search continues in the main file at the location indicated by the pointer.

To see the effectiveness of this approach, consider a sequential file with 1 million records. To search for a particular key value will require on average one-half million record accesses. Now suppose an index containing 1,000 entries is constructed, with the keys in the index more or less evenly distributed over the main file. Now it will take on average 500 accesses to the index file followed by 500 accesses to the main file to find the record. The average search length is reduced from 500,000 to 1,000.

Additions to the file are handled in the following manner: Each record in the main file contains an additional field not visible to the application, which is a pointer to the overflow file. When a new record is to be inserted into the file, it is added to the overflow file. The record in the main file that immediately precedes the new record in logical sequence is updated to contain a pointer to the new record in the overflow file. If the immediately preceding record is itself in the overflow file, then the pointer in that record is updated. As with the sequential file, the indexed sequential file is occasionally merged with the overflow file in batch mode.

The indexed sequential file greatly reduces the time required to access a single record, without sacrificing the sequential nature of the file. To process the entire file sequentially, the records of the main file are processed in sequence until a pointer to the overflow file is found, then accessing continues in the overflow file until a null pointer is encountered, at which time accessing of the main file is resumed where it left off.

To provide even greater efficiency in access, multiple levels of indexing can be used. Thus the lowest level of index file is treated as a sequential file and a higher-level index file is created for that file. Consider again a file with 1 million records. A lower-level index with 10,000 entries is constructed. A higher-level index into the lower-level index of 100 entries can then be constructed. The search begins at the higher-level index (average length $= 50$ accesses) to find an entry point into the lower-level index. This index is then searched (average length $= 50$) to find an entry point into the main file, which is then searched (average length $= 50$). Thus the average length of search has been reduced from 500,000 to 1,000 to 150.

## The Indexed File

The indexed sequential file retains one limitation of the sequential file: Effective processing is limited to that which is based on a single field of the file. For example, when it is necessary to search for a record on the basis of some other attribute than the key field, both forms of sequential file are inadequate. In some applications, the flexibility of efficiently searching by various attributes is desirable.

To achieve this flexibility, a structure is needed that employs multiple indexes, one for each type of field that may be the subject of a search. In the general indexed file, the concept of sequentiality and a single key are abandoned. Records are accessed only through their indexes. The result is there is now no restriction on the placement of records as long as a pointer in at least one index refers to that record. Furthermore, variable-length records can be employed.

Two types of indexes are used. An exhaustive index contains one entry for every record in the main file. The index itself is organized as a sequential file for ease of searching. A partial index contains entries to records where the field of interest exists. With variable-length records, some records will not contain all fields. When a new record is added to the main file, all of the index files must be updated.

Indexed files are used mostly in applications where timeliness of information is critical and where data are rarely processed exhaustively. Examples are airline reservation systems and inventory control systems.

### The Direct or Hashed File

The direct, or hashed, file exploits the capability found on disks to access directly any block of a known address. As with sequential and indexed sequential files, a key field is required in each record. However, there is no concept of sequential ordering here.

The direct file makes use of hashing on the key value. This function is explained in Appendix F. Figure F.1b shows the type of hashing organization with an overflow file that is typically used in a hash file.

Direct files are often used where very rapid access is required, where fixed-length records are used, and where records are always accessed one at a time. Examples are directories, pricing tables, schedules, and name lists.

## 12.3 B-TREES

The preceding section referred to the use of an index file to access individual records in a file or database. For a large file or database, a single sequential file of indexes on the primary key does not provide for rapid access. To provide more efficient access, a structured index file is typically used. The simplest such structure is a two-level organization in which the original file is broken into sections, and the upper level consists of a sequenced set of pointers to the lower-level sections. This structure can then be extended to more than two levels, resulting in a tree structure. Unless some discipline is imposed on the construction of the tree index, it is likely to end up with an uneven structure, with some short branches and some long branches, so the time to search the index is uneven. Therefore, a balanced tree structure, with all branches of equal length, would appear to give the best average performance. Such a structure is the B-tree, which has become the standard method of organizing indexes for databases and is commonly used in OS file systems, including those supported by Mac OS X, Windows, and several Linux file systems. The B-tree structure provides for efficient searching, adding, and deleting of items.

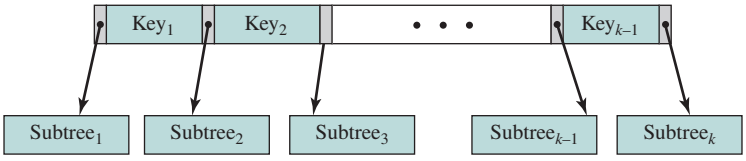



**Figure 12.4 A B-tree Node with *k* Children**

Before illustrating the concept of B-tree, let us define a B-tree and its characteristics more precisely. A B-tree is a tree structure (no closed loops) with the following characteristics (see Figure 12.4):

1. The tree consists of a number of nodes and leaves.
2. Each node contains at least one key which uniquely identifies a file record, and more than one pointer to child nodes or leaves. The number of keys and pointers contained in a node may vary, within limits explained below.
3. Each node is limited to the same number of maximum keys.
4. The keys in a node are stored in nondecreasing order. Each key has an associated child that is the root of a subtree containing all nodes with keys less than or equal to the key but greater than the preceding key. A node also has an additional rightmost child that is the root for a subtree containing all keys greater than any keys in the node. Thus, each node has one more pointer than keys.

A B-tree is characterized by its minimum degree $d$ and satisfies the following properties:

1. Every node has at most $2d - 1$ keys and $2d$ children or, equivalently, $2d$ pointers.[1]
2. Every node, except for the root, has at least $d - 1$ keys and $d$ pointers. As a result, each internal node, except the root, is at least half full and has at least $d$ children.
3. The root has at least 1 key and 2 children.
4. All leaves appear on the same level and contain no information. This is a logical construct to terminate the tree; the actual implementation may differ. For example, each bottom-level node may contain keys alternating with null pointers.
5. A nonleaf node with $k$ pointers contains $k - 1$ keys.

Typically, a B-tree has a relatively large branching factor (large number of children) resulting in a tree of low height.

Figure 12.4 illustrates two levels of a B-tree. The upper level has $(k - 1)$ keys and $k$ pointers and satisfies the following relationship:

$$\text{Key}_1 < \text{Key}_3 < \ldots < \text{Key}_{k-1}$$

[1]Some treatments require, as stated here, that the maximum number of keys in a node is odd (e.g., [CORM09]); others specify even [COME79]; still others allow odd or even [KNUT98]. The choice does not fundamentally affect the performance of B-trees.

Each pointer points to a node that is the top level of a subtree of this upper-level node. Each of these subtree nodes contains some number of keys and pointers, unless it is a leaf node. The following relationships hold:

All the keys in $\text{Subtree}_1$ are less than $\text{Key}_1$
All the keys in $\text{Subtree}_2$ are greater than $\text{Key}_1$ and are less than $\text{Key}_2$
All the keys in $\text{Subtree}_3$ are greater than $\text{Key}_2$ and are less than $\text{Key}_3$

•
•
•

All the keys in $\text{Subtree}_{k-1}$ are greater than $\text{Key}_{k-2}$ and are less than $\text{Key}_{k-1}$
All the keys in $\text{Subtree}_k$ are greater than $\text{Key}_{k-1}$

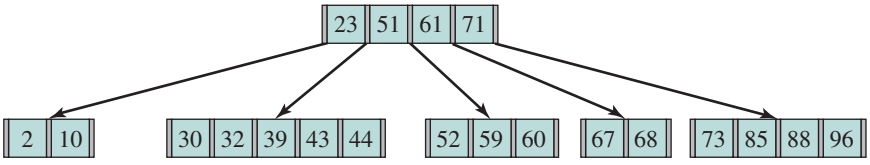
To search for a key, you start at the root node. If the key you want is in the node, you're done. If not, you go down one level. There are three cases:

1. The key you want is less then the smallest key in this node. Take the leftmost pointer down to the next level.
2. The key you want is greater than the largest key in this node. Take the rightmost pointer down to the next level.
3. The value of the key is between the values of two adjacent keys in this node. Take the pointer between these keys down to the next level.

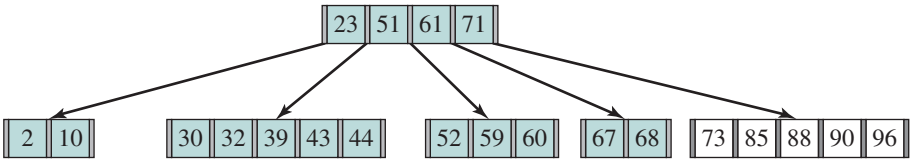
For example, consider the tree in Figure 12.5d and the desired key is 84. At the root level, $84 > 51$, so you take the rightmost branch down to the next level. Here, we have $71 < 84 < 88$, so take the pointer between 71 and 88 down to the next level, where the key 84 is found. Associated with this key is a pointer to the desired record. An advantage of this tree structure over other tree structures is that it is broad and shallow, so the search terminates quickly. Furthermore, because it is balanced (all branches from root to leaf are of equal length), there are no long searches compared to other searches.

The rules for inserting a new key into the B-tree must maintain a balanced tree. This is done as follows:

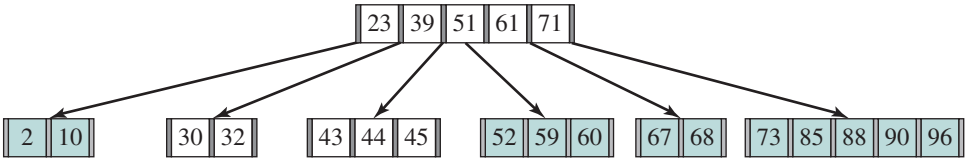
1. Search the tree for the key. If the key is not in the tree, then you have reached a node at the lowest level.
2. If this node has fewer than $2d - 1$ keys, then insert the key into this node in the proper sequence.
3. If the node is full (having $2d - 1$ keys), then split this node around its median key into two new nodes with $d - 1$ keys each and promote the median key to the next higher level, as described in step 4. If the new key has a value less than the median key, insert it into the left–hand new node; otherwise, insert it into the right–hand new node. The result is that the original node has been split into two nodes: one with $d - 1$ keys, and one with $d$ keys.
4. The promoted node is inserted into the parent node following the rules of step 3. Therefore, if the parent node is already full, it must be split and its median key promoted to the next highest layer.
5. If the process of promotion reaches the root node and the root node is already full, then insertion again follows the rules of step 3. However, in this case, the median key becomes a new root node and the height of the tree increases by 1.



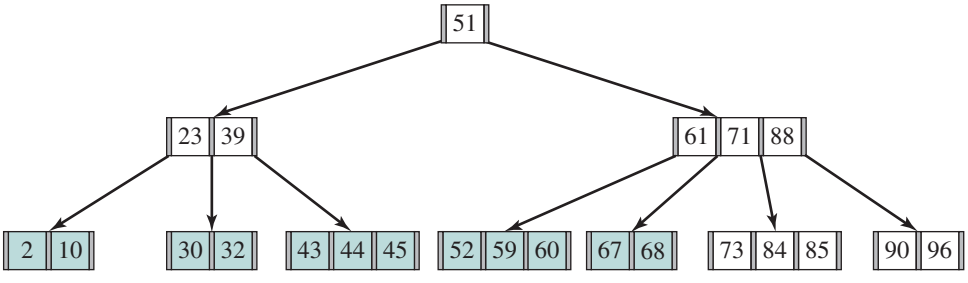

(a) B-tree of minimum degree $d = 3$.




(b) Key = 90 inserted. This is a simple insertion into a node.




(c) Key = 45 inserted. This requires splitting a node into two parts and promoting one key to the root node.




(d) Key = 84 inserted. This requires splitting a node into two parts and promoting one key to the root node. This then requires the root node to be split and a new root created.

**Figure 12.5 Inserting Nodes into a B-tree**

Figure 12.5 illustrates the insertion process on a B-tree of degree $d = 3$. In each part of the figure, the nodes affected by the insertion process are unshaded.

## 12.4 FILE DIRECTORIES

### Contents

Associated with any file management system and collection of files is a file directory. The directory contains information about the files, including attributes, location, and ownership. Much of this information, especially that concerned with storage,

**Table 12.1** Information Elements of a File Directory

| | |
|---|---|
| | **Basic Information** |
| **File Name** | Name as chosen by creator (user or program). Must be unique within a specific directory |
| **File Type** | For example: text, binary, load module, etc. |
| **File Organization** | For systems that support different organizations |
| | **Address Information** |
| **Volume** | Indicates device on which file is stored |
| **Starting Address** | Starting physical address on secondary storage (e.g., cylinder, track, and block number on disk) |
| **Size Used** | Current size of the file in bytes, words, or blocks |
| **Size Allocated** | The maximum size of the file |
| | **Access Control Information** |
| **Owner** | User who is assigned control of this file. The owner may be able to grant/deny access to other users and to change these privileges. |
| **Access Information** | A simple version of this element would include the user's name and password for each authorized user. |
| **Permitted Actions** | Controls reading, writing, executing, and transmitting over a network |
| | **Usage Information** |
| **Date Created** | When file was first placed in directory |
| **Identity of Creator** | Usually but not necessarily the current owner |
| **Date Last Read Access** | Date of the last time a record was read |
| **Identity of Last Reader** | User who did the reading |
| **Date Last Modified** | Date of the last update, insertion, or deletion |
| **Identity of Last Modifier** | User who did the modifying |
| **Date of Last Backup** | Date of the last time the file was backed up on another storage medium |
| **Current Usage** | Information about current activity on the file, such as process or processes that have the file open, whether it is locked by a process, and whether the file has been updated in main memory but not yet on disk |

is managed by the operating system. The directory is itself a file, accessible by various file management routines. Although some of the information in directories is available to users and applications, this is generally provided indirectly by system routines.

Table 12.1 suggests the information typically stored in the directory for each file in the system. From the user's point of view, the directory provides a mapping between file names, known to users and applications, and the files themselves. Thus, each file entry includes the name of the file. Virtually all systems deal with different types of files and different file organizations, and this information is also provided. An important category of information about each file concerns its storage, including its location and size. In shared systems, it is also important to provide information that is used to control access to the file. Typically, one user is the owner of the file and may grant certain access privileges to other users. Finally, usage information is needed to manage the current use of the file and to record the history of its usage.

### Structure

The way in which the information of Table 12.1 is stored differs widely among various systems. Some of the information may be stored in a header record associated with the file; this reduces the amount of storage required for the directory, making it easier to keep all or much of the directory in main memory to improve speed.

The simplest form of structure for a directory is that of a list of entries, one for each file. This structure could be represented by a simple sequential file, with the name of the file serving as the key. In some earlier single-user systems, this technique has been used. However, it is inadequate when multiple users share a system and even for single users with many files.

To understand the requirements for a file structure, it is helpful to consider the types of operations that may be performed on the directory:

- **Search:** When a user or application references a file, the directory must be searched to find the entry corresponding to that file.
- **Create file:** When a new file is created, an entry must be added to the directory.
- **Delete file:** When a file is deleted, an entry must be removed from the directory.
- **List directory:** All or a portion of the directory may be requested. Generally, this request is made by a user and results in a listing of all files owned by that user, plus some of the attributes of each file (e.g., type, access control information, usage information).
- **Update directory:** Because some file attributes are stored in the directory, a change in one of these attributes requires a change in the corresponding directory entry.

The simple list is not suited to supporting these operations. Consider the needs of a single user. The user may have many types of files, including word-processing text files, graphic files, and spreadsheets. The user may like to have these organized by project, by type, or in some other convenient way. If the directory is a simple sequential list, it provides no help in organizing the files and forces the user to be careful not to use the same name for two different types of files. The problem is much worse in a shared system. Unique naming becomes a serious problem. Furthermore, it is difficult to conceal portions of the overall directory from users when there is no inherent structure in the directory.

A start in solving these problems would be to go to a two-level scheme. In this case, there is one directory for each user, and a master directory. The master directory has an entry for each user directory, providing address and access control information. Each user directory is a simple list of the files of that user. This arrangement means names must be unique only within the collection of files of a single user, and the file system can easily enforce access restriction on directories. However, it still provides users with no help in structuring collections of files.

A more powerful and flexible approach, and one that is almost universally adopted, is the hierarchical, or tree-structure, approach (see Figure 12.6). As before, there is a master directory, which has under it a number of user directories. Each of

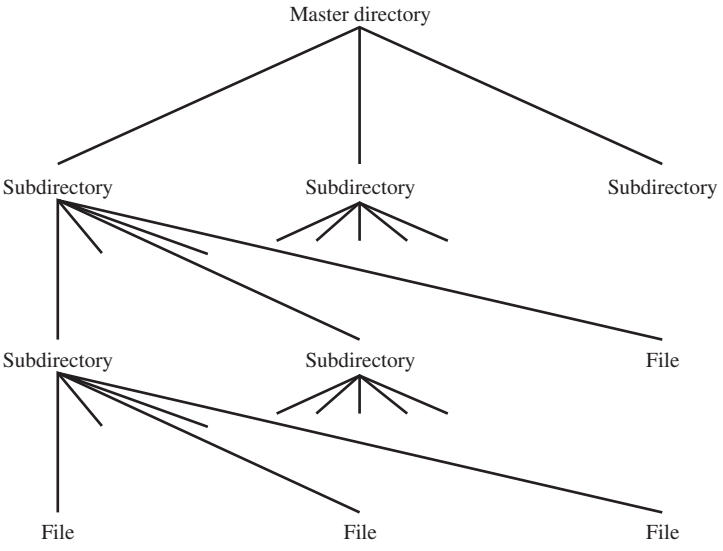



**Figure 12.6 Tree-Structured Directory**

these user directories, in turn, may have subdirectories and files as entries. This is true at any level: That is, at any level, a directory may consist of entries for subdirectories and/or entries for files.

It remains to say how each directory and subdirectory is organized. The simplest approach, of course, is to store each directory as a sequential file. When directories may contain a very large number of entries, such an organization may lead to unnecessarily long search times. In that case, a hashed structure is to be preferred.

## Naming

Users need to be able to refer to a file by a symbolic name. Clearly, each file in the system must have a unique name in order that file references be unambiguous. On the other hand, it is an unacceptable burden on users to require they provide unique names, especially in a shared system.

The use of a tree-structured directory minimizes the difficulty in assigning unique names. Any file in the system can be located by following a path from the root or master directory down various branches until the file is reached. The series of directory names, culminating in the file name itself, constitutes a **pathname** for the file. As an example, the file in the lower left-hand corner of Figure 12.7 has the pathname `User_B/Word/Unit_A/ABC`. The slash is used to delimit names in the sequence. The name of the master directory is implicit, because all paths start at that directory. Note it is perfectly acceptable to have several files with the same file name, as long as they have unique pathnames, which is equivalent to saying that the same file name may be used in different directories. In our example, there is another file in the system with the file name ABC, but that file has the pathname `/User_B/Draw/ABC`.

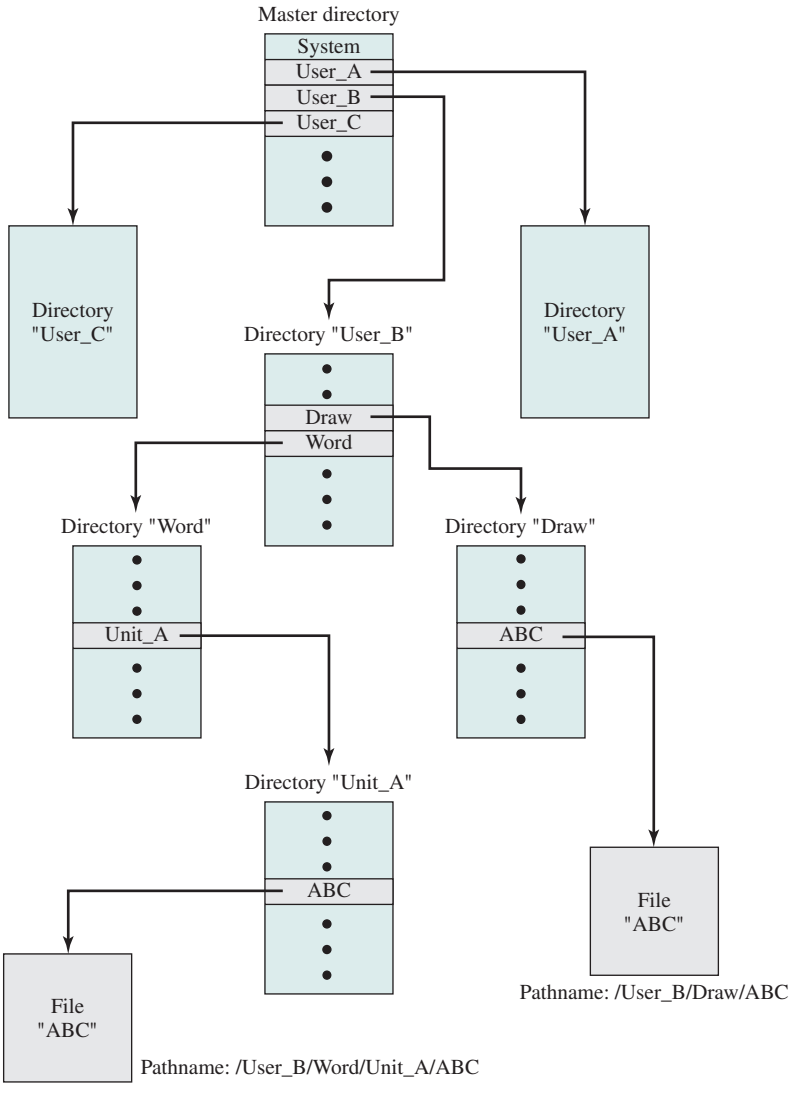



**Figure 12.7 Example of Tree-Structured Directory**

Although the pathname facilitates the selection of file names, it would be awkward for a user to have to spell out the entire pathname every time a reference is made to a file. Typically, an interactive user or a process has associated with it a current directory, often referred to as the **working directory**. Files are then referenced relative to the working directory. For example, if the working directory for user B is "Word," then the pathname `Unit_A/ABC` is sufficient to identify the file in the lower left-hand corner of Figure 12.7. When an interactive user logs on, or when a process is created, the default for the working directory is the user home directory. During execution, the user can navigate up or down in the tree to change to a different working directory.

## 12.5 FILE SHARING

In a multiuser system, there is almost always a requirement for allowing files to be shared among a number of users. Two issues arise: access rights and the management of simultaneous access.

### Access Rights

The file system should provide a flexible tool for allowing extensive file sharing among users. The file system should provide a number of options so the way in which a particular file is accessed can be controlled. Typically, users or groups of users are granted certain access rights to a file. A wide range of access rights has been used. The following list is representative of access rights that can be assigned to a particular user for a particular file:

- **None:** The user may not even learn of the existence of the file, much less access it. To enforce this restriction, the user would not be allowed to read the user directory that includes this file.
- **Knowledge:** The user can determine that the file exists and who its owner is. The user is then able to petition the owner for additional access rights.
- **Execution:** The user can load and execute a program but cannot copy it. Proprietary programs are often made accessible with this restriction.
- **Reading:** The user can read the file for any purpose, including copying and execution. Some systems are able to enforce a distinction between viewing and copying. In the former case, the contents of the file can be displayed to the user, but the user has no means for making a copy.
- **Appending:** The user can add data to the file, often only at the end, but cannot modify or delete any of the file's contents. This right is useful in collecting data from a number of sources.
- **Updating:** The user can modify, delete, and add to the file's data. This normally includes writing the file initially, rewriting it completely or in part, and removing all or a portion of the data. Some systems distinguish among different degrees of updating.
- **Changing protection:** The user can change the access rights granted to other users. Typically, this right is held only by the owner of the file. In some systems, the owner can extend this right to others. To prevent abuse of this mechanism, the file owner will typically be able to specify which rights can be changed by the holder of this right.
- **Deletion:** The user can delete the file from the file system.

These rights can be considered to constitute a hierarchy, with each right implying those that precede it. Thus, if a particular user is granted the updating right for a particular file, then that user is also granted the following rights: knowledge, execution, reading, and appending.

One user is designated as the owner of a given file, usually the person who initially created the file. The owner has all of the access rights listed previously and may grant rights to others. Access can be provided to different classes of users:

- **Specific user:** Individual users who are designated by user ID
- **User groups:** A set of users who are not individually defined. The system must have some way of keeping track of the membership of user groups.
- **All:** All users who have access to this system. These are public files.

### Simultaneous Access

When access is granted to append or update a file to more than one user, the operating system or file management system must enforce discipline. A brute-force approach is to allow a user to lock the entire file when it is to be updated. A finer grain of control is to lock individual records during update. Essentially, this is the readers/writers problem discussed in Chapter 5. Issues of mutual exclusion and deadlock must be addressed in designing the shared access capability.

## 12.6 RECORD BLOCKING

As indicated in Figure 12.2, records are the logical unit of access of a structured file,[2] whereas blocks are the unit of I/O with secondary storage. For I/O to be performed, records must be organized as blocks.

There are several issues to consider. First, should blocks be of fixed or variable length? On most systems, blocks are of fixed length. This simplifies I/O, buffer allocation in main memory, and the organization of blocks on secondary storage. Second, what should the relative size of a block be compared to the average record size? The trade-off is this: The larger the block, the more records that are passed in one I/O operation. If a file is being processed or searched sequentially, this is an advantage, because the number of I/O operations is reduced by using larger blocks, thus speeding up processing. On the other hand, if records are being accessed randomly and no particular locality of reference is observed, then larger blocks result in the unnecessary transfer of unused records. However, combining the frequency of sequential operations with the potential for locality of reference, we can say the I/O transfer time is reduced by using larger blocks. The competing concern is that larger blocks require larger I/O buffers, making buffer management more difficult.

Given the size of a block, there are three methods of blocking that can be used:

1. **Fixed blocking:** Fixed-length records are used, and an integral number of records are stored in a block. There may be unused space at the end of each block. This is referred to as internal fragmentation.
2. **Variable-length spanned blocking:** Variable-length records are used and are packed into blocks with no unused space. Thus, some records must span two blocks, with the continuation indicated by a pointer to the successor block.

[2] As opposed to a file that is treated only as a stream of bytes, such as in the UNIX file system.

3. **Variable-length unspanned blocking:** Variable-length records are used, but spanning is not employed. There is wasted space in most blocks because of the inability to use the remainder of a block if the next record is larger than the remaining unused space.

Figure 12.8 illustrates these methods assuming a file is stored in sequential blocks on a disk. The figure assumes the file is large enough to span two tracks.[3] The effect would not be changed if some other file allocation scheme were used (see Section 12.6).

Fixed blocking is the common mode for sequential files with fixed-length records. Variable-length spanned blocking is efficient of storage and does not limit the size of records. However, this technique is difficult to implement. Records that span two blocks require two I/O operations, and files are difficult to update, regardless of the organization. Variable-length unspanned blocking results in wasted space and limits record size to the size of a block.

The record-blocking technique may interact with the virtual memory hardware, if such is employed. In a virtual memory environment, it is desirable to make the page the basic unit of transfer. Pages are generally quite small, so it is impractical to treat a

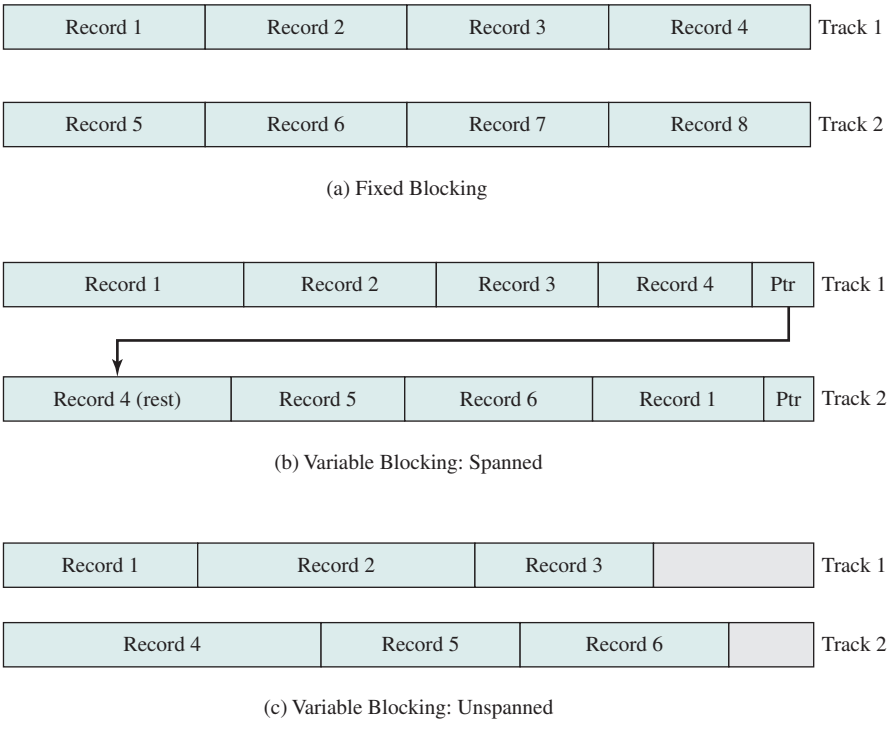



**Figure 12.8 Record-Blocking Methods**

[3]The organization of data on a disk is in a concentric set of rings, called *tracks*. Each track is the same width as the read/write head. See Appendix J.

page as a block for unspanned blocking. Accordingly, some systems combine multiple pages to create a larger block for file I/O purposes. This approach is used for VSAM files on IBM mainframes.

## 12.7 SECONDARY STORAGE MANAGEMENT

On secondary storage, a file consists of a collection of blocks. The operating system or file management system is responsible for allocating blocks to files. This raises two management issues. First, space on secondary storage must be allocated to files, and second, it is necessary to keep track of the space available for allocation. We will see that these two tasks are related; that is, the approach taken for file allocation may influence the approach taken for free space management. Further, we will see that there is an interaction between file structure and allocation policy.

We begin this section by looking at alternatives for file allocation on a single disk. Then we will look at the issue of free space management, and finally we will discuss reliability.

### File Allocation

Several issues are involved in file allocation:

1. When a new file is created, is the maximum space required for the file allocated at once?
2. Space is allocated to a file as one or more contiguous units, which we shall refer to as portions. A **portion** is a contiguous set of allocated blocks. The size of a portion can range from a single block to the entire file. What size of portion should be used for file allocation?
3. What sort of data structure or table is used to keep track of the portions assigned to a file? An example of such a structure is a **file allocation table (FAT)**, found on DOS and some other systems.

Let us examine these issues in turn.

***Preallocation versus Dynamic Allocation*** A preallocation policy requires the maximum size of a file be declared at the time of the file creation request. In a number of cases, such as program compilations, the production of summary data files, or the transfer of a file from another system over a communications network, this value can be reliably estimated. However, for many applications, it is difficult if not impossible to estimate reliably the maximum potential size of the file. In those cases, users and application programmers would tend to overestimate file size so as not to run out of space. This clearly is wasteful from the point of view of secondary storage allocation. Thus, there are advantages to the use of dynamic allocation, which allocates space to a file in portions as needed.

***Portion Size*** The second issue listed is that of the size of the portion allocated to a file. At one extreme, a portion large enough to hold the entire file is allocated. At the other extreme, space on the disk is allocated one block at a time. In choosing a portion size, there is a trade-off between efficiency from the point of view of a single file versus overall system efficiency. [WIED87] lists four items to be considered in the trade-off:

1. Contiguity of space increases performance, especially for `Retrieve_Next` operations, and greatly for transactions running in a transaction-oriented operating system.
2. Having a large number of small portions increases the size of tables needed to manage the allocation information.
3. Having fixed-size portions (e.g., blocks) simplifies the reallocation of space.
4. Having variable-size or small fixed-size portions minimizes waste of unused storage due to overallocation.

Of course, these items interact and must be considered together. The result is that there are two major alternatives:

1. **Variable, large contiguous portions:** This will provide better performance. The variable size avoids waste, and the file allocation tables are small. However, space is hard to reuse.
2. **Blocks:** Small fixed portions provide greater flexibility. They may require large tables or complex structures for their allocation. Contiguity has been abandoned as a primary goal; blocks are allocated as needed.

Either option is compatible with preallocation or dynamic allocation. In the case of variable, large contiguous portions, a file is preallocated one contiguous group of blocks. This eliminates the need for a file allocation table; all that is required is a pointer to the first block and the number of blocks allocated. In the case of blocks, all of the portions required are allocated at one time. This means the file allocation table for the file will remain of fixed size, because the number of blocks allocated is fixed.

With variable-size portions, we need to be concerned with the fragmentation of free space. This issue was faced when we considered partitioned main memory in Chapter 7. The following are possible alternative strategies:

- **First fit:** Choose the first unused contiguous group of blocks of sufficient size from a free block list.
- **Best fit:** Choose the smallest unused group that is of sufficient size.
- **Nearest fit:** Choose the unused group of sufficient size that is closest to the previous allocation for the file to increase locality.

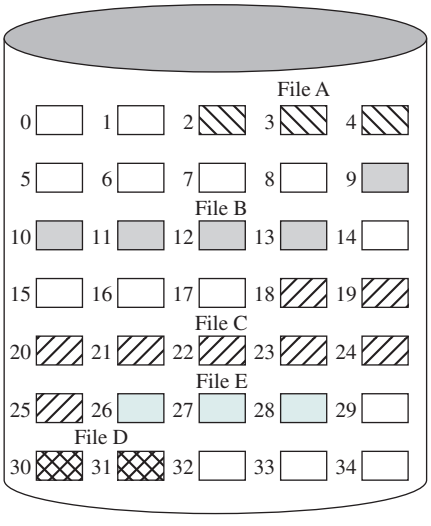
It is not clear which strategy is best. The difficulty in modeling alternative strategies is that so many factors interact, including types of files, pattern of file access, degree of multiprogramming, other performance factors in the system, disk caching, and disk scheduling.

**Table 12.2** File Allocation Methods

| | Contiguous | Chained | Indexed | |
|---|---|---|---|---|
| **Preallocation?** | Necessary | Possible | Possible | |
| **Fixed or Variable Size Portions?** | Variable | Fixed blocks | Fixed blocks | Variable |
| **Portion Size** | Large | Small | Small | Medium |
| **Allocation Frequency** | Once | Low to high | High | Low |
| **Time to Allocate** | Medium | Long | Short | Medium |
| **File Allocation Table Size** | One entry | One entry | Large | Medium |

***File Allocation Methods*** Having looked at the issues of preallocation versus dynamic allocation and portion size, we are in a position to consider specific file allocation methods. Three methods are in common use: contiguous, chained, and indexed. Table 12.2 summarizes some of the characteristics of each method.

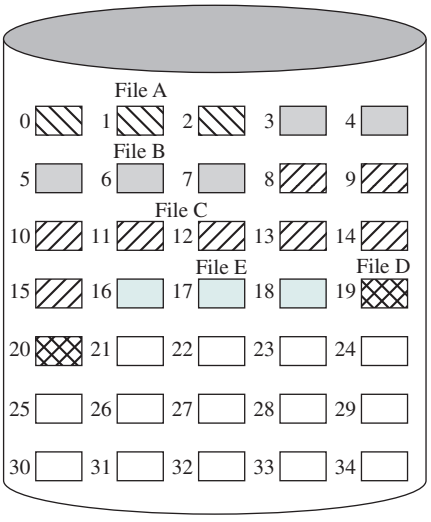
With **contiguous allocation**, a single contiguous set of blocks is allocated to a file at the time of file creation (see Figure 12.9). Thus, this is a preallocation strategy, using variable-size portions. The file allocation table needs just a single entry for each file, showing the starting block and the length of the file. Contiguous allocation is the best from the point of view of the individual sequential file. Multiple blocks can be read in at a time to improve I/O performance for sequential processing. It is also easy to retrieve a single block. For example, if a file starts at block $b$, and the $i$th block of the file is wanted, its location on secondary storage is simply $b + i - 1$. However, contiguous allocation presents some problems. External fragmentation will occur, making it difficult to find contiguous blocks of space of sufficient length. From time to time, it will be necessary to perform a compaction algorithm to free up additional space on the disk (see Figure 12.10). Also, with preallocation, it is necessary to declare the size of the file at the time of creation, with the problems mentioned earlier.




File allocation table

| File name | Start block | Length |
|---|---|---|
| File A | 2 | 3 |
| File B | 9 | 5 |
| File C | 18 | 8 |
| File D | 30 | 2 |
| File E | 26 | 3 |

**Figure 12.9** **Contiguous File Allocation**

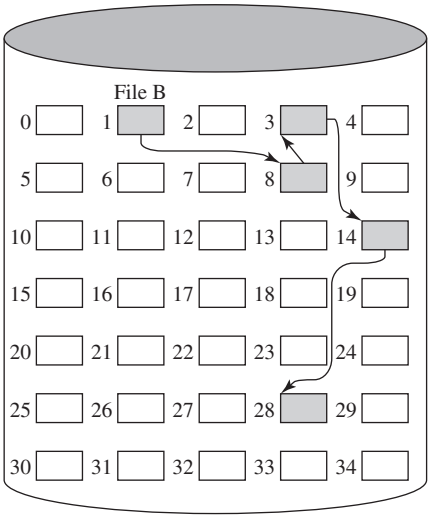



File allocation table

| File name | Start block | Length |
|---|---|---|
| File A | 0 | 3 |
| File B | 3 | 5 |
| File C | 8 | 8 |
| File D | 19 | 2 |
| File E | 16 | 3 |

**Figure 12.10 Contiguous File Allocation (After Compaction)**

At the opposite extreme from contiguous allocation is **chained allocation** (see Figure 12.11). Typically, allocation is on an individual block basis. Each block contains a pointer to the next block in the chain. Again, the file allocation table needs just a single entry for each file, showing the starting block and the length of the file. Although preallocation is possible, it is more common simply to allocate blocks as needed. The selection of blocks is now a simple matter: Any free block can be added to a chain. There is no external fragmentation to worry about, because only one block at a time is needed. This type of physical organization is best suited to sequential files




File allocation table

| File name | Start block | Length |
|---|---|---|
| • • • | • • • | • • • |
| File B | 1 | 5 |
| • • • | • • • | • • • |

**Figure 12.11 Chained Allocation**

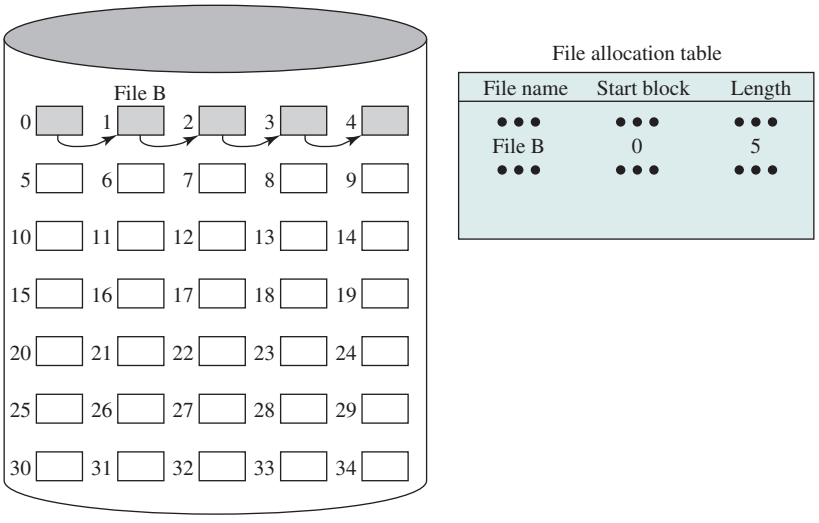



**Figure 12.12 Chained Allocation (After Consolidation)**

that are to be processed sequentially. To select an individual block of a file requires tracing through the chain to the desired block.

One consequence of chaining, as described so far, is there is no accommodation of the principle of locality. Thus, if it is necessary to bring in several blocks of a file at a time (as in sequential processing) then a series of accesses to different parts of the disk are required. This is perhaps a more significant effect on a single-user system, but may also be of concern on a shared system. To overcome this problem, some systems periodically consolidate files (see Figure 12.12).

**Indexed allocation** addresses many of the problems of contiguous and chained allocation. In this case, the file allocation table contains a separate one-level index for each file; the index has one entry for each portion allocated to the file. Typically, the file indexes are not physically stored as part of the file allocation table. Rather, the file index for a file is kept in a separate block, and the entry for the file in the file allocation table points to that block. Allocation may be on the basis of either fixed-size blocks (see Figure 12.13) or variable-size portions (see Figure 12.14). Allocation by blocks eliminates external fragmentation, whereas allocation by variable-size portions improves locality. In either case, file consolidation may be done from time to time. File consolidation reduces the size of the index in the case of variable-size portions, but not in the case of block allocation. Indexed allocation supports both sequential and direct access to the file and thus is the most popular form of file allocation.

## Free Space Management

Just as the space allocated to files must be managed, so the space that is not currently allocated to any file must be managed. To perform any of the file allocation techniques described previously, it is necessary to know what blocks on the disk are available. Thus, we need a **disk allocation table** in addition to a file allocation table. We discuss here a number of techniques that have been implemented.

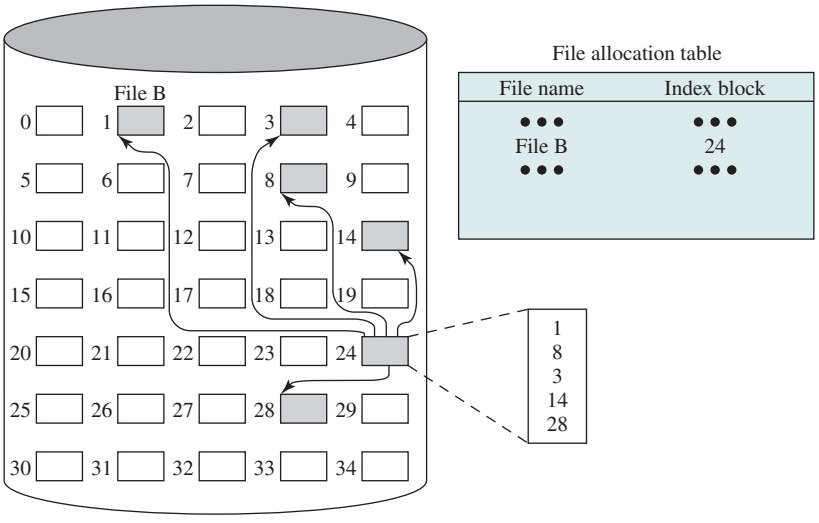



**Figure 12.13 Indexed Allocation with Block Portions**

***BIT TABLES*** This method uses a vector containing one bit for each block on the disk. Each entry of a 0 corresponds to a free block, and each 1 corresponds to a block in use. For example, for the disk layout of Figure 12.9, a vector of length 35 is needed and would have the following value:

00111000011111000011111111111011000

A bit table has the advantage that it is relatively easy to find one or a contiguous group of free blocks. Thus, a bit table works well with any of the file allocation methods outlined. Another advantage is that a bit table is as small as possible.

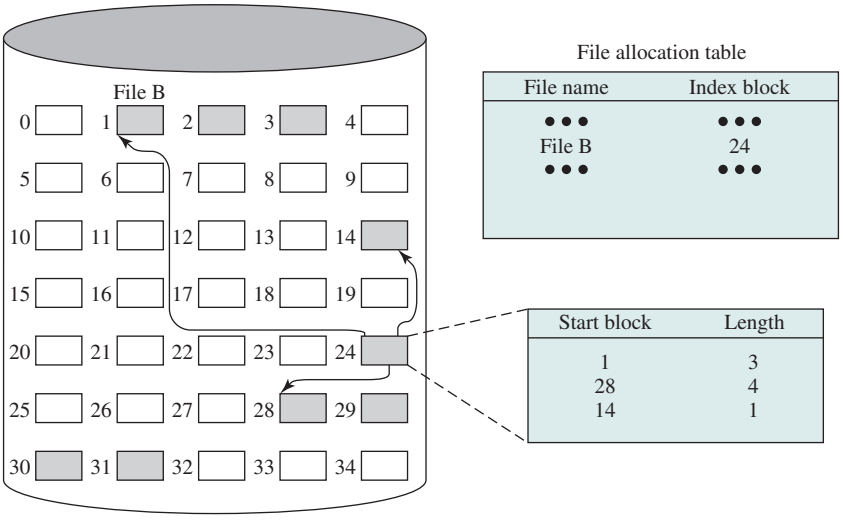



**Figure 12.14 Indexed Allocation with Variable-Length Portions**

However, it can still be sizable. The amount of memory (in bytes) required for a block bitmap is

$$\frac{\text{disk size in bytes}}{8 \times \text{file system block size}}$$

Thus, for a 16-Gbyte disk with 512-byte blocks, the bit table occupies about 4 Mbytes. Can we spare 4 Mbytes of main memory for the bit table? If so, then the bit table can be searched without the need for disk access. But even with today's memory sizes, 4 Mbytes is a hefty chunk of main memory to devote to a single function. The alternative is to put the bit table on disk. But a 4-Mbyte bit table would require about 8,000 disk blocks. We can't afford to search that amount of disk space every time a block is needed, so a bit table resident in memory is indicated.

Even when the bit table is in main memory, an exhaustive search of the table can slow file system performance to an unacceptable degree. This is especially true when the disk is nearly full and there are few free blocks remaining. Accordingly, most file systems that use bit tables maintain auxiliary data structures that summarize the contents of subranges of the bit table. For example, the table could be divided logically into a number of equal-size subranges. A summary table could include, for each subrange, the number of free blocks and the maximum-sized contiguous number of free blocks. When the file system needs a number of contiguous blocks, it can scan the summary table to find an appropriate subrange and then search that subrange.

***Chained Free Portions*** The free portions may be chained together by using a pointer and length value in each free portion. This method has negligible space overhead because there is no need for a disk allocation table, merely for a pointer to the beginning of the chain and the length of the first portion. This method is suited to all of the file allocation methods. If allocation is a block at a time, simply choose the free block at the head of the chain and adjust the first pointer or length value. If allocation is by variable-length portion, a first-fit algorithm may be used: The headers from the portions are fetched one at a time to determine the next suitable free portion in the chain. Again, pointer and length values are adjusted.

This method has its own problems. After some use, the disk will become quite fragmented and many portions will be a single block long. Also note every time you allocate a block, you need to read the block first to recover the pointer to the new first free block before writing data to that block. If many individual blocks need to be allocated at one time for a file operation, this greatly slows file creation. Similarly, deleting highly fragmented files is very time consuming.

***Indexing*** The indexing approach treats free space as a file and uses an index table as described under file allocation. For efficiency, the index should be on the basis of variable-size portions rather than blocks. Thus, there is one entry in the table for every free portion on the disk. This approach provides efficient support for all of the file allocation methods.

***Free Block List*** In this method, each block is assigned a number sequentially and the list of the numbers of all free blocks is maintained in a reserved portion of the

disk. Depending on the size of the disk, either 24 or 32 bits will be needed to store a single block number, so the size of the free block list is 24 or 32 times the size of the corresponding bit table and thus must be stored on disk rather than in main memory. However, this is a satisfactory method. Consider the following points:

1. The space on disk devoted to the free block list is less than 1% of the total disk space. If a 32-bit block number is used, then the space penalty is 4 bytes for every 512-byte block.
2. Although the free block list is too large to store in main memory, there are two effective techniques for storing a small part of the list in main memory.

   a. The list can be treated as a push-down stack (see Appendix P) with the first few thousand elements of the stack kept in main memory. When a new block is allocated, it is popped from the top of the stack, which is in main memory. Similarly, when a block is deallocated, it is pushed onto the stack. There only has to be a transfer between disk and main memory when the in-memory portion of the stack becomes either full or empty. Thus, this technique gives almost zero-time access most of the time.

   b. The list can be treated as a FIFO queue, with a few thousand entries from both the head and the tail of the queue in main memory. A block is allocated by taking the first entry from the head of the queue, and deallocated by adding it to the end of the tail of the queue. There only has to be a transfer between disk and main memory when either the in-memory portion of the head of the queue becomes empty or the in-memory portion of the tail of the queue becomes full.

In either of the strategies listed in the preceding point (stack or FIFO queue), a background thread can slowly sort the in-memory list or lists to facilitate contiguous allocation.

## Volumes

The term *volume* is used somewhat differently by different operating systems and file management systems, but in essence a volume is a logical disk. [CARR05] defines a volume as follows:

> **Volume:** A collection of addressable sectors in secondary memory that an OS or application can use for data storage. The sectors in a volume need not be consecutive on a physical storage device; instead, they need only appear that way to the OS or application. A volume may be the result of assembling and merging smaller volumes.

In the simplest case, a single disk equals one volume. Frequently, a disk is divided into partitions, with each partition functioning as a separate volume. It is also common to treat multiple disks as a single volume or partitions on multiple disks as a single volume.

### Reliability

Consider the following scenario:

1. User A requests a file allocation to add to an existing file.
2. The request is granted and the disk and file allocation tables are updated in main memory but not yet on disk.
3. The system crashes and subsequently restarts.
4. User B requests a file allocation and is allocated space on disk that overlaps the last allocation to user A.
5. User A accesses the overlapped portion via a reference that is stored inside A's file.

This difficulty arose because the system maintained a copy of the disk allocation table and file allocation table in main memory for efficiency. To prevent this type of error, the following steps could be performed when a file allocation is requested:

1. Lock the disk allocation table on disk. This prevents another user from causing alterations to the table until this allocation is completed.
2. Search the disk allocation table for available space. This assumes a copy of the disk allocation table is always kept in main memory. If not, it must first be read in.
3. Allocate space, update the disk allocation table, and update the disk. Updating the disk involves writing the disk allocation table back onto disk. For chained disk allocation, it also involves updating some pointers on disk.
4. Update the file allocation table and update the disk.
5. Unlock the disk allocation table.

This technique will prevent errors. However, when small portions are allocated frequently, the impact on performance will be substantial. To reduce this overhead, a batch storage allocation scheme could be used. In this case, a batch of free portions on the disk is obtained for allocation. The corresponding portions on disk are marked "in use." Allocation using this batch may proceed in main memory. When the batch is exhausted, the disk allocation table is updated on disk and a new batch may be acquired. If a system crash occurs, portions on the disk marked "in use" must be cleaned up in some fashion before they can be reallocated. The technique for cleanup will depend on the file system's particular characteristics.

## 12.8 UNIX FILE MANAGEMENT

In the UNIX file system, six types of files are distinguished:

1. **Regular, or ordinary:** Contains arbitrary data in zero or more data blocks. Regular files contain information entered in them by a user, an application program, or a system utility program. The file system does not impose any internal structure to a regular file, but treats it as a stream of bytes.

2. **Directory:** Contains a list of file names plus pointers to associated inodes (index nodes), described later. Directories are hierarchically organized (see Figure 12.6). Directory files are actually ordinary files with special write protection privileges so only the file system can write into them, while read access is available to user programs.
3. **Special:** Contains no data but provides a mechanism to map physical devices to file names. The file names are used to access peripheral devices, such as terminals and printers. Each I/O device is associated with a special file, as discussed in Section 11.8.
4. **Named pipes:** As discussed in Section 6.7, a pipe is an interprocess communications facility. A pipe file buffers data received in its input so a process that reads from the pipe's output receives the data on a first-in-first-out basis.
5. **Links:** In essence, a link is an alternative file name for an existing file.
6. **Symbolic links:** This is a data file that contains the name of the file to which it is linked.

In this section, we are concerned with the handling of ordinary files, which correspond to what most systems treat as files.

## Inodes

Modern UNIX operating systems support multiple file systems but map all of these into a uniform underlying system for supporting file systems and allocating disk space to files. All types of UNIX files are administered by the OS by means of inodes. An inode (index node) is a control structure that contains the key information needed by the operating system for a particular file. Several file names may be associated with a single inode, but an active inode is associated with exactly one file, and each file is controlled by exactly one inode.

The attributes of the file as well as its permissions and other control information are stored in the inode. The exact inode structure varies from one UNIX implementation to another. The FreeBSD inode structure, shown in Figure 12.15, includes the following data elements:

- The type and access mode of the file
- The file's owner and group-access identifiers
- The time when the file was created, when it was most recently read and written, and when its inode was most recently updated by the system
- The size of the file in bytes
- A sequence of block pointers, explained in the next subsection
- The number of physical blocks used by the file, including blocks used to hold indirect pointers and attributes
- The number of directory entries that reference the file
- The kernel and user-settable flags that describe the characteristics of the file
- The generation number of the file (a randomly selected number assigned to the inode each time that the latter is allocated to a new file; the generation number is used to detect references to deleted files)

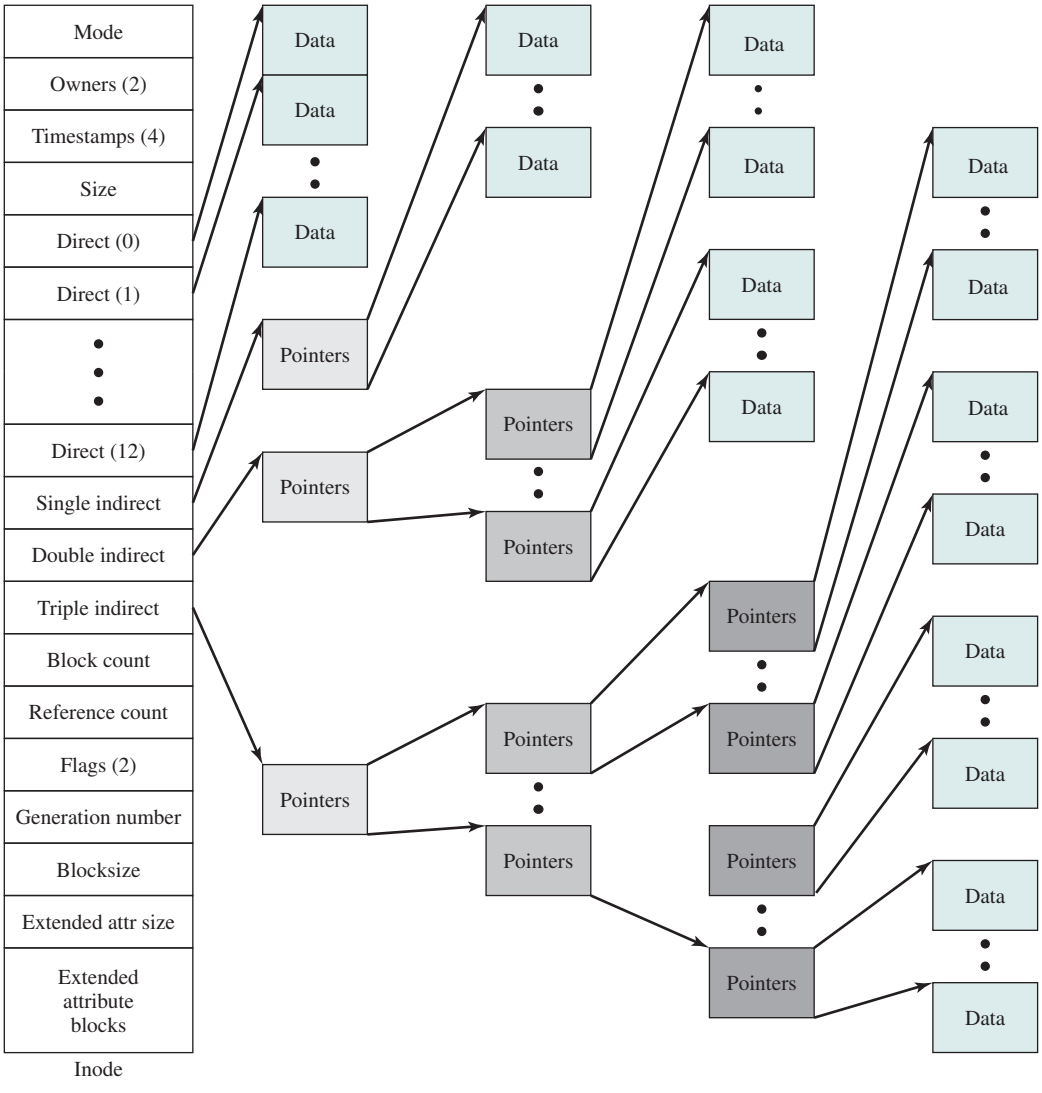



**Figure 12.15** **Structure of FreeBSD Inode and File**

- The blocksize of the data blocks referenced by the inode (typically the same as, but sometimes larger than, the file system blocksize)
- The size of the extended attribute information
- Zero or more extended attribute entries

The blocksize value is typically the same as, but sometimes larger than, the file system blocksize. On traditional UNIX systems, a fixed blocksize of 512 bytes was used. FreeBSD has a minimum blocksize of 4,096 bytes (4 Kbytes); the blocksize can be any power of 2 greater than or equal to 4,096. For typical file systems, the blocksize is 8 Kbytes or 16 Kbytes. The default FreeBSD blocksize is 16 Kbytes.

Extended attribute entries are variable-length entries used to store auxiliary data that are separate from the contents of the file. The first two extended attributes defined for FreeBSD deal with security. The first of these support access control lists; this will be described in Chapter 15. The second defined extended attribute supports the use of security labels, which are part of what is known as a mandatory access control scheme, also defined in Chapter 15.

On the disk, there is an inode table, or inode list, that contains the inodes of all the files in the file system. When a file is opened, its inode is brought into main memory and stored in a memory-resident inode table.

## File Allocation

File allocation is done on a block basis. Allocation is dynamic, as needed, rather than using preallocation. Hence, the blocks of a file on disk are not necessarily contiguous. An indexed method is used to keep track of each file, with part of the index stored in the inode for the file. In all UNIX implementations, the inode includes a number of direct pointers and three indirect pointers (single, double, triple).

The FreeBSD inode includes 120 bytes of address information organized as fifteen 64-bit addresses, or pointers. The first 12 addresses point to the first 12 data blocks of the file. If the file requires more than 12 data blocks, one or more levels of indirection is used as follows:

- The thirteenth address in the inode points to a block on disk that contains the next portion of the index. This is referred to as the single indirect block. This block contains the pointers to succeeding blocks in the file.
- If the file contains more blocks, the fourteenth address in the inode points to a double indirect block. This block contains a list of addresses of additional single indirect blocks. Each of single indirect blocks, in turn, contains pointers to file blocks.
- If the file contains still more blocks, the fifteenth address in the inode points to a triple indirect block that is a third level of indexing. This block points to additional double indirect blocks.

All of this is illustrated in Figure 12.15. The total number of data blocks in a file depends on the capacity of the fixed-size blocks in the system. In FreeBSD, the minimum block size is 4 Kbytes, and each block can hold a total of 512 block addresses. Thus, the maximum size of a file with this block size is over 500 GB (see Table 12.3).

**Table 12.3** Capacity of a FreeBSD File with 4-Kbyte Block Size

| Level | Number of Blocks | Number of Bytes |
|---|---|---|
| **Direct** | 12 | 48K |
| **Single Indirect** | 512 | 2M |
| **Double Indirect** | $512 \times 512 = 256K$ | 1G |
| **Triple Indirect** | $512 \times 256K = 128M$ | 512G |

This scheme has several advantages:

1. The inode is of fixed size and relatively small, and hence may be kept in main memory for long periods.
2. Smaller files may be accessed with little or no indirection, reducing processing and disk access time.
3. The theoretical maximum size of a file is large enough to satisfy virtually all applications.

## Directories

Directories are structured in a hierarchical tree. Each directory can contain files and/or other directories. A directory inside another directory is referred to as a subdirectory. As was mentioned, a directory is simply a file that contains a list of file names plus pointers to associated inodes. Figure 12.16 shows the overall structure. Each directory entry (dentry) contains a name for the associated file or subdirectory plus an integer called the i-number (index number). When the file or directory is accessed, its i-number is used as an index into the inode table.

## Volume Structure

A UNIX file system resides on a single logical disk or disk partition and is laid out with the following elements:

- **Boot block:** Contains code required to boot the operating system

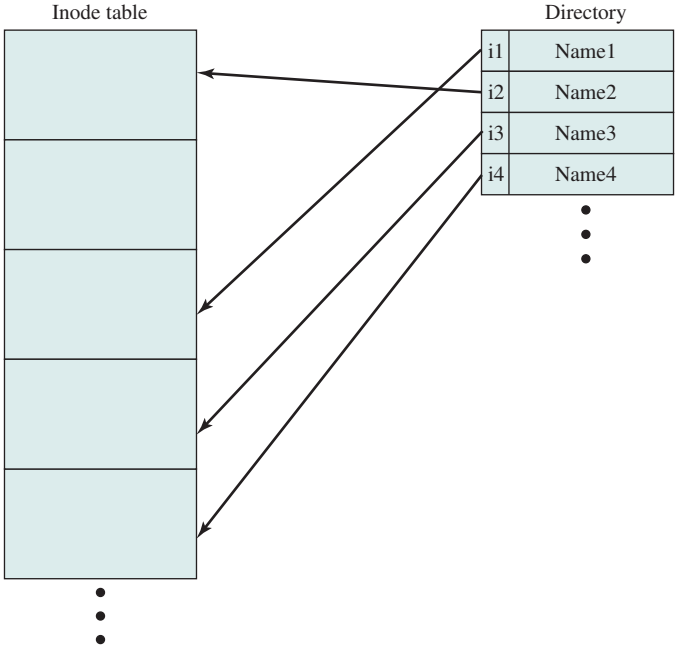



**Figure 12.16 UNIX Directories and Inodes**

- **Superblock:** Contains attributes and information about the file system, such as partition size, and inode table size
- **Inode table:** The collection of inodes for each file
- **Data block:** Storage space available for data files and subdirectories

## 12.9 LINUX VIRTUAL FILE SYSTEM

Linux includes a versatile and powerful file-handling facility, designed to support a wide variety of file management systems and file structures. The approach taken in Linux is to make use of a **virtual file system (VFS)**, which presents a single, uniform file system interface to user processes. The VFS defines a common file model that is capable of representing any conceivable file system's general feature and behavior. The VFS assumes files are objects in a computer's mass storage memory that share basic properties regardless of the target file system or the underlying processor hardware. Files have symbolic names that allow them to be uniquely identified within a specific directory within the file system. A file has an owner, protection against unauthorized access or modification, and a variety of other properties. A file may be created, read from, written to, or deleted. For any specific file system, a mapping module is needed to transform the characteristics of the real file system to the characteristics expected by the virtual file system.

Figure 12.17 indicates the key ingredients of the Linux file system strategy. A user process issues a file system call (e.g., read) using the VFS file scheme. The VFS converts this into an internal (to the kernel) file system call that is passed to a mapping function for a specific file system [e.g., ext2 FS (second extended filesystem)]. In most cases, the mapping function is simply a mapping of file system functional calls from one scheme to another. In some cases, the mapping function is more complex.

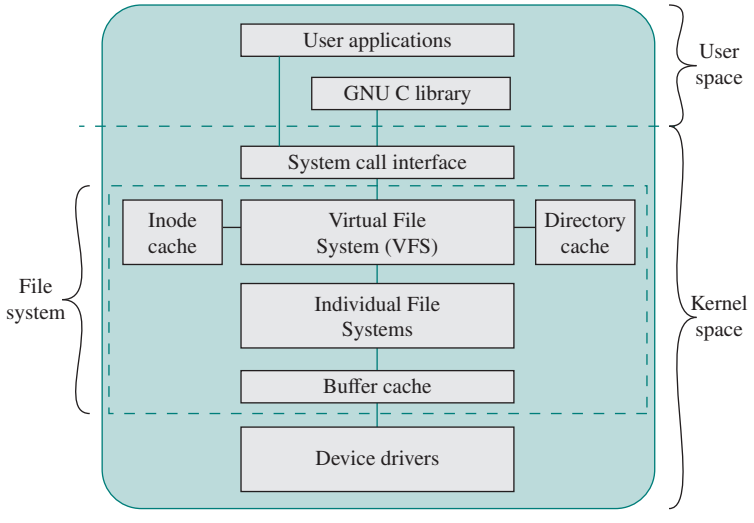



**Figure 12.17** **Linux Virtual File System Context**

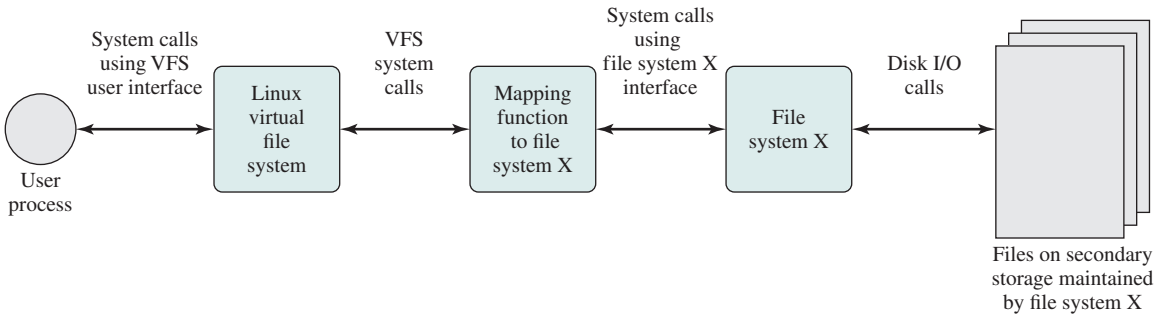



**Figure 12.18 Linux Virtual File System Concept**

For example, some file systems use a file allocation table (FAT), which stores the position of each file in the directory tree. In these file systems, directories are not files. For such file systems, the mapping function must be able to construct dynamically, and when needed, the files corresponding to the directories. In any case, the original user file system call is translated into a call that is native to the target file system. The target file system software is then invoked to perform the requested function on a file or directory under its control and secondary storage. The results of the operation are then communicated back to the user in a similar fashion.

Figure 12.18 indicates the role that VFS plays within the Linux kernel. When a process initiates a file-oriented system call (e.g., read), the kernel calls a function in the VFS. This function handles the file-system-independent manipulations and initiates a call to a function in the target file system code. This call passes through a mapping function that converts the call from the VFS into a call to the target file system. The VFS is independent of any file system, so the implementation of a mapping function must be part of the implementation of a file system on Linux. The target file system converts the file system request into device-oriented instructions that are passed to a device driver by means of page cache functions.

VFS is an object-oriented scheme. Because it is written in C, rather than a language that supports object programming (such as C++ or Java), VFS objects are implemented simply as C data structures. Each object contains both data and pointers to file-system-implemented functions that operate on data. The four primary object types in VFS are as follows:

- **Superblock object:** Represents a specific mounted file system
- **Inode object:** Represents a specific file
- **Dentry object:** Represents a specific directory entry
- **File object:** Represents an open file associated with a process

This scheme is based on the concepts used in UNIX file systems, as described in Section 12.7. The key concepts of UNIX file system to remember are the following: A file system consists of a hierarchal organization of directories. A directory is the same as what is known as a folder on many non-UNIX platforms and may contain files and/or other directories. Because a directory may contain other directories, a

tree structure is formed. A path through the tree structure from the root consists of a sequence of directory entries, ending in either a directory entry (dentry) or a file name. In UNIX, a directory is implemented as a file that lists the files and directories contained within it. Thus, file operations can be performed on either files or directories.

### The Superblock Object

The superblock object stores information describing a specific file system. Typically, the superblock corresponds to the file system superblock or file system control block, which is stored in a special sector on disk.

The superblock object consists of a number of data items. Examples include the following:

- The device this file system is mounted on
- The basic block size of the file system
- Dirty flag, to indicate that the superblock has been changed but not written back to disk
- File system type
- Flags, such as a read-only flag
- Pointer to the root of the file system directory
- List of open files
- Semaphore for controlling access to the file system
- List of superblock operations

The last item on the preceding list refers to an operations object contained within the superblock object. The operations object (`super_operations`) defines the object methods (functions) that the kernel can invoke against the superblock object. The methods defined for the superblock object include the following:

- `alloc_inode`: Allocate an inode.
- `write_inode`: Write given inode to disk.
- `put_super`: Called by the VFS on unmount to release the given superblock.
- `statfs`: Obtain file system statistics.
- `remount_fs`: Called by the VFS when the file system is remounted with new mount options.

### The Inode Object

An inode is associated with each file. The inode object holds all the information about a named file except its name and the actual data contents of the file. Items contained in an inode object include owner, group, permissions, access times for a file, size of data it holds, and number of links.

The inode object also includes an inode operations object that describes the file system's implemented functions that the VFS can invoke on an inode. The methods defined for the inode object include the following:

- `create`: Creates a new inode for a regular file associated with a dentry object in some directory
- `lookup`: Searches a directory for an inode corresponding to a file name
- `mkdir`: Creates a new inode for a directory associated with a dentry object in some directory

### The Dentry Object

A dentry (directory entry) is a specific component in a path. The component may be either a directory name or a file name. Dentry objects facilitate quick lookups to files and directories, and are used in a dentry cache for that purpose. The dentry object includes a pointer to the inode and superblock. It also includes a pointer to the parent dentry and pointers to any subordinate dentrys.

### The File Object

The file object is used to represent a file opened by a process. The object is created in response to the `open()` system call, and destroyed in response to the `close()` system call. The file object consists of a number of items, including the following:

- Dentry object associated with the file
- File system containing the file
- File objects usage counter
- User's user ID
- User's group ID
- File pointer, which is the current position in the file from which the next operation will take place

The file object also includes an inode operations object that describes the file system's implemented functions that the VFS can invoke on a file object. The methods defined for the file object include read, write, open, release, and lock.

### Caches

The VFS employs three caches to improve performance:

1. **Inode cache:** Because every file and directory is represented by a VFS inode, a directory listing command or a file access command causes a number of inodes to be accessed. The inode cache stores recently visited inodes to make access quicker.
2. **Directory cache:** The directory cache stores the mapping between the full directory names and their inode numbers. This speeds up the process of listing a directory.
3. **Buffer cache:** The buffer cache is independent of the file systems and is integrated into the mechanisms that the Linux kernel uses to allocate and read

and write data buffers. As the real file systems read data from the underlying physical disks, this results in requests to the block device drivers to read physical blocks from the device that they control. So, if the same data is needed often, it will be retrieved from the buffer cache rather than read from the disk.

## 12.10 WINDOWS FILE SYSTEM

The developers of Windows NT designed a new file system, the New Technology File System (NTFS), which is intended to meet high-end requirements for workstations and servers. Examples of high-end applications include the following:

- Client/server applications such as file servers, compute servers, and database servers
- Resource-intensive engineering and scientific applications
- Network applications for large corporate systems

This section provides an overview of NTFS.

### Key Features of NTFS

NTFS is a flexible and powerful file system built, as we shall see, on an elegantly simple file system model. The most noteworthy features of NTFS include the following:

- **Recoverability:** High on the list of requirements for the new Windows file system was the ability to recover from system crashes and disk failures. In the event of such failures, NTFS is able to reconstruct disk volumes and return them to a consistent state. It does this by using a transaction-processing model for changes to the file system; each significant change is treated as an atomic action that is either entirely performed or not performed at all. Each transaction that was in process at the time of a failure is subsequently backed out or brought to completion. In addition, NTFS uses redundant storage for critical file system data, so failure of a disk sector does not cause the loss of data describing the structure and status of the file system.
- **Security:** NTFS uses the Windows object model to enforce security. An open file is implemented as a file object with a security descriptor that defines its security attributes. The security descriptor is persisted as an attribute of each file on disk.
- **Large disks and large files:** NTFS supports very large disks and very large files more efficiently than other file systems, such as FAT.
- **Multiple data streams:** The actual contents of a file are treated as a stream of bytes. In NTFS, it is possible to define multiple data streams for a single file. An example of the utility of this feature is that it allows Windows to be used by remote Macintosh systems to store and retrieve files. On Macintosh, each file has two components: the file data and a resource fork that contains information

about the file. NTFS treats these two components as two data streams within a single file.

- **Journaling:** NTFS keeps a log of all changes made to files on the volumes. Programs, such as desktop search, can read the journal to identify what files have changed.
- **Compression and encryption:** Entire directories and individual files can be transparently compressed and/or encrypted.
- **Hard and symbolic links:** In order to support POSIX, Windows has always supported "hard links," which allow a single file to be accessible by multiple path names on the same volume. Starting with Windows Vista, "symbolic links" are supported which allow a file or directory to be accessible by multiple path names, even if the names are on different volumes. Windows also supports "mount points" which allow volumes to appear at junction points on other volumes, rather than be named by driver letters, such as "D:".

## NTFS Volume and File Structure

NTFS makes use of the following disk storage concepts:

- **Sector:** The smallest physical storage unit on the disk. The data size in bytes is a power of 2 and is almost always 512 bytes.
- **Cluster:** One or more contiguous (next to each other on the disk) sectors. The cluster size in sectors is a power of 2.
- **Volume:** A logical partition on a disk, consisting of one or more clusters and used by a file system to allocate space. At any time, a volume consists of file system information, a collection of files, and any additional unallocated space remaining on the volume that can be allocated to files. A volume can be all or a portion of a single disk, or it can extend across multiple disks. If hardware or software RAID 5 is employed, a volume consists of stripes spanning multiple disks. The maximum volume size for NTFS is $2^{64}$ clusters.

The cluster is the fundamental unit of allocation in NTFS, which does not recognize sectors. For example, suppose each sector is 512 bytes and the system is configured with two sectors per cluster (one cluster = 1K bytes). If a user creates a file of 1,600 bytes, two clusters are allocated to the file. Later, if the user updates the file to 3,200 bytes, another two clusters are allocated. The clusters allocated to a file need not be contiguous; it is permissible to fragment a file on the disk. Currently, the maximum file size supported by NTFS is $2^{32}$ clusters, which is equivalent to a maximum of $2^{48}$ bytes. A cluster can have at most $2^{16}$ bytes.

The use of clusters for allocation makes NTFS independent of physical sector size. This enables NTFS to support easily nonstandard disks that do not have a 512-byte sector size, and to support efficiently very large disks and very large files by using a larger cluster size. The efficiency comes from the fact that the file system must keep track of each cluster allocated to each file; with larger clusters, there are fewer items to manage.

**Table 12.4** Windows NTFS Partition and Cluster Sizes

| Volume Size | Sectors per Cluster | Cluster Size |
|---|---|---|
| ≤512 Mbyte | 1 | 512 bytes |
| 512 Mbyte–1 Gbyte | 2 | 1K |
| 1 Gbyte–2 Gbyte | 4 | 2K |
| 2 Gbyte–4 Gbyte | 8 | 4K |
| 4 Gbyte–8 Gbyte | 16 | 8K |
| 8 Gbyte–16 Gbyte | 32 | 16K |
| 16 Gbyte–32 Gbyte | 64 | 32K |
| >32 Gbyte | 128 | 64K |

Table 12.4 shows the default cluster sizes for NTFS. The defaults depend on the size of the volume. The cluster size that is used for a particular volume is established by NTFS when the user requests that a volume be formatted.

***NTFS Volume Layout*** NTFS uses a remarkably simple but powerful approach to organizing information on a disk volume. Every element on a volume is a file, and every file consists of a collection of attributes. Even the data contents of a file is treated as an attribute. With this simple structure, a few general-purpose functions suffice to organize and manage a file system.

Figure 12.19 shows the layout of an NTFS volume, which consists of four regions. The first few sectors on any volume are occupied by the **partition boot sector** (although it is called a sector, it can be up to 16 sectors long), which contains information about the volume layout and the file system structures as well as boot startup information and code. This is followed by the **master file table (MFT)**, which contains information about all of the files and folders (directories) on this NTFS volume. In essence, the MFT is a list of all files and their attributes on this NTFS volume, organized as a set of rows in a table structure.

Following the MFT is a region containing **system files**. Among the files in this region are the following:

- **MFT2:** A mirror of the first few rows of the MFT, used to guarantee access to the volume in the case of a single-sector failure in the sectors storing the MFT
- **Log file:** A list of transaction steps used for NTFS recoverability
- **Cluster bit map:** A representation of the space on the volume, showing which clusters are in use

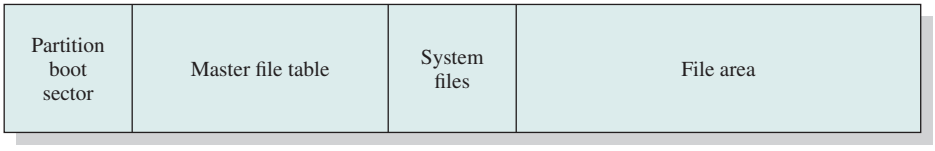



**Figure 12.19** **NTFS Volume Layout**

**Table 12.5** Windows NTFS File and Directory Attribute Types

| Attribute Type | Description |
|---|---|
| Standard information | Includes access attributes (read-only, read/write, etc.); time stamps, including when the file was created or last modified; and how many directories point to the file (link count) |
| Attribute list | A list of attributes that make up the file and the file reference of the MFT file record in which each attribute is located. Used when all attributes do not fit into a single MFT file record |
| File name | A file or directory must have one or more names. |
| Security descriptor | Specifies who owns the file and who can access it |
| Data | The contents of the file. A file has one default unnamed data attribute and may have one or more named data attributes. |
| Index root | Used to implement folders |
| Index allocation | Used to implement folders |
| Volume information | Includes volume-related information, such as the version and name of the volume |
| Bitmap | Provides a map representing records in use on the MFT or folder |

*Note:* Green-colored rows refer to required file attributes; the other attributes are optional.

- **Attribute definition table:** Defines the attribute types supported on this volume and indicates whether they can be indexed, and whether they can be recovered during a system recovery operation

***Master File Table*** The heart of the Windows file system is the MFT. The MFT is organized as a table of 1,024-byte rows, called records. Each row describes a file on this volume, including the MFT itself, which is treated as a file. If the contents of a file are small enough, then the entire file is located in a row of the MFT. Otherwise, the row for that file contains partial information and the remainder of the file spills over into other available clusters on the volume, with pointers to those clusters in the MFT row of that file.

Each record in the MFT consists of a set of attributes that serve to define the file (or folder) characteristics and the file contents. Table 12.5 lists the attributes that may be found in a row, with the required attributes indicated by shading.

## Recoverability

NTFS makes it possible to recover the file system to a consistent state following a system crash or disk failure. The key elements that support recoverability are as follows (see Figure 12.20):

- **I/O manager:** Includes the NTFS driver, which handles the basic open, close, read, and write functions of NTFS. In addition, the software RAID module FTDISK can be configured for use.
- **Log file service:** Maintains a log of file system metadata changes on disk. The log file is used to recover an NTFS-formatted volume in the case of a system failure (i.e., without having to run the file system check utility).

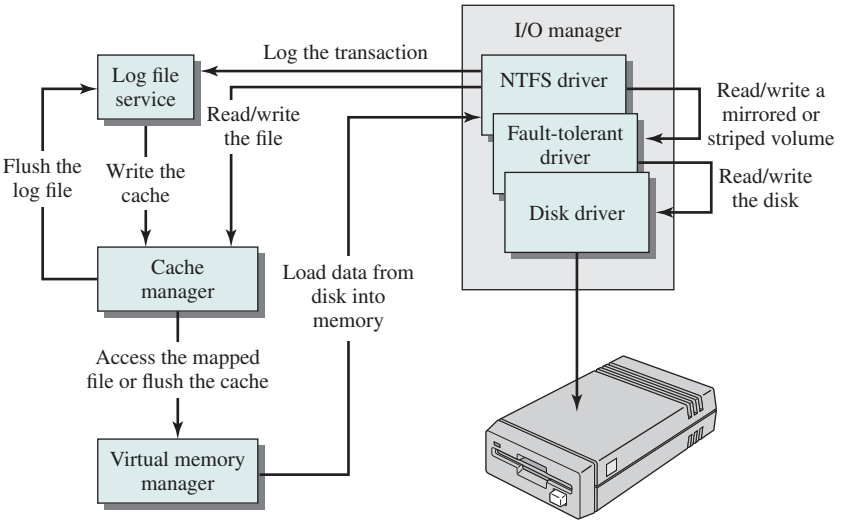



**Figure 12.20 Windows NTFS Components**

- **Cache manager:** Responsible for caching file reads and writes to enhance performance. The cache manager optimizes disk I/O.
- **Virtual memory manager:** The NTFS accesses cached files by mapping file references to virtual memory references, and reading and writing virtual memory.

It is important to note the recovery procedures used by NTFS are designed to recover file system metadata, not file contents. Thus, the user should never lose a volume or the directory/file structure of an application because of a crash. However, user data are not guaranteed by the file system. Providing full recoverability, including user data, would make for a much more elaborate and resource-consuming recovery facility.

The essence of the NTFS recovery capability is logging. Each operation that alters a file system is treated as a transaction. Each suboperation of a transaction that alters important file system data structures is recorded in a log file before being recorded on the disk volume. Using the log, a partially completed transaction at the time of a crash can later be redone or undone when the system recovers.

In general terms, these are the steps taken to ensure recoverability, as described in [RUSS11]:

1. NTFS first calls the log file system to record in the log file (in the cache) any transactions that will modify the volume structure.
2. NTFS modifies the volume (in the cache).
3. The cache manager calls the log file system to prompt it to flush the log file to disk.
4. Once the log file updates are safely on disk, the cache manager flushes the volume changes to disk.

## 12.11 ANDROID FILE MANAGEMENT

### File System

Android makes use of the file management capabilities built into Linux. The Android file system directory is similar to what is seen on a typical Linux installation, with some Android-specific features.

Figure 12.21 shows the top levels of a typical Android file system directory. The **system directory** contains the core parts of the operating system, including system binaries, system libraries, and configuration files. It also includes a basic set of Android applications, such as Alarmclock, Calculator, and Camera. The system image is locked, with only read-only access granted to file system users. The remaining directories shown in Figure 12.21 are read-write.

The **data directory** provides the principal location used by applications to store their private data. This partition contains the user's data, such as contacts, SMS, settings, and all Android applications that you have installed. While the user performs a factory reset on the device, this partition is wiped out. Then, the device will be in the state when used for the first time, or the way it was after the last official or custom ROM installation. When a new application is installed in the system, the following actions, among others, are taken with respect to the data directory:

- The .apk (Android package) is placed into /data/app.
- Application-centric libraries are installed into /data/data/<application name>. This is an application-specific sandbox area, accessible by the application but not accessible to other applications.
- Application-relevant files databases are set up.

The **cache directory** is used for temporary storage by the OS. This is the partition where Android stores frequently accessed data and app components. Wiping the

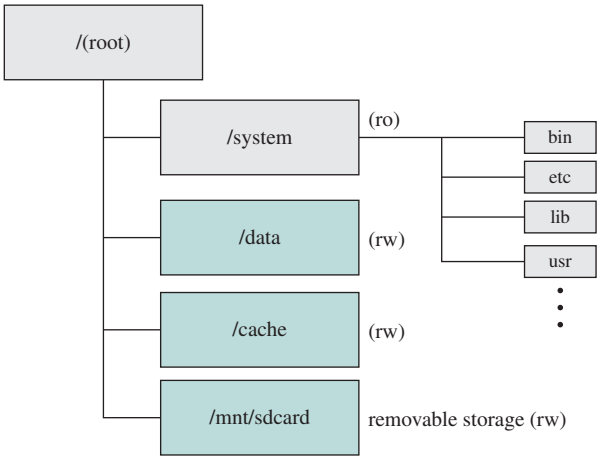



**Figure 12.21 Typical Directory Tree of Android**

cache doesn't affect the user's personal data but simply gets rid of the existing data there, which gets automatically rebuilt as the user continues using the device.

The **mnt/sdcard** directory is not a partition on the internal memory of the device but rather the SD card, which is a nonvolatile memory card that can be incorporated with the Android devices. The SD card is a removable memory card the user can remove and plug into his or her computer. In terms of usage, this is storage space for the user to read/write data, audio and video files. On devices with both an internal and external SD card, the /sdcard partition is always used to refer to the internal SD card. For the external SD card, if present, an alternative partition is used, which differs from device to device.

### SQLite

SQLite, which is based on SQL, is worth special mention. The Structured Query Language (SQL) provides as standardized means for definition of, and access to, a relational database by either a local or remote user or application. Structured Query Language (SQL), originally developed by IBM in the mid-1970s, is a standardized language that can be used to define schema, manipulate, and query data in a relational database. There are several versions of the ANSI/ISO standard and a variety of different implementations, but all follow the same basic syntax and semantics.

SQLite is the most widely deployed SQL database engine in the world. It is designed to provide a streamlined SQL-based database management system suitable for embedded systems and other limited-memory systems. The full SQLite library can be implemented in under 400 kilobytes (KB). Unnecessary features can be disabled at compile time to further reduce the size of the library to under 190 KB if desired.

In contrast to other database management systems, SQLite is not a separate process that is accessed from the client application. Instead, the SQLite library is linked in, and thus becomes an integral part of the application program.

## 12.12 SUMMARY

A file management system is a set of system software that provides services to users and applications in the use of files, including file access, directory maintenance, and access control. The file management system is typically viewed as a system service that itself is served by the operating system, rather than being part of the operating system itself. However, in any system, at least part of the file management function is performed by the operating system.

A file consists of a collection of records. The way in which these records may be accessed determines its logical organization, and to some extent, its physical organization on disk. If a file is primarily to be processed as a whole, then a sequential file organization is the simplest and most appropriate. If sequential access is needed but random access to individual file is also desired, then an indexed sequential file may give the best performance. If access to the file is principally at random, then an indexed file or hashed file may be the most appropriate.

Whatever file structure is chosen, a directory service is also needed. This allows files to be organized in a hierarchical fashion. This organization is useful to the user in keeping track of files, and is useful to the file management system in providing access control and other services to users.

File records, even when of fixed size, generally do not conform to the size of a physical disk block. Accordingly, some sort of blocking strategy is needed. A trade-off among complexity, performance, and space utilization determines the blocking strategy to be used.

A key function of any file management scheme is the management of disk space. Part of this function is the strategy for allocating disk blocks to a file. A variety of methods have been employed, and a variety of data structures have been used to keep track of the allocation for each file. In addition, the space on disk that has not been allocated must be managed. This latter function primarily consists of maintaining a disk allocation table indicating which blocks are free.

## 12.13 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

### Key Terms

| | | |
|---|---|---|
| access method | file | master file table (MFT) |
| basic file system | file allocation | mnt/sdcard |
| bit table | file allocation table (FAT) | partition boot sector |
| block | file directory | pathname |
| cache directory | file management system | physical I/O |
| chained file allocation | file name | pile |
| contiguous file allocation | hashed file | portion |
| database | indexed file | record |
| data directory | indexed file allocation | sequential file |
| device driver | indexed sequential file | system directory |
| disk allocation table | inode | system files |
| field | key field | virtual file system (VFS) |
| | logical I/O | working directory |

### Review Questions

**12.1.** What are the desirable properties of a file system?

**12.2.** What is the difference between a file and a database?

**12.3.** What is a file management system?

**12.4.** What criteria are important in choosing a file organization?

**12.5.** What are some advantages and disadvantages of sequential file organization?

**12.6.** Why is the average search time to find a record in a file less for an indexed sequential file than for a sequential file?

**12.7.** What is a pathname? State the two alternate ways to assign pathnames.

**12.8.** What is the relationship between a pathname and a working directory?

**12.9.** What are typical access rights that may be granted or denied to a particular user for a particular file?

**12.10.** What is an inode in UNIX?

**12.11.** List and briefly define three file allocation methods.

## Problems

**12.1.** A file contains 20,000 records, each of a fixed size of 140 bytes. The file is to be stored in a disk drive having blocks of 3096 bytes with 512 bytes of inter-block gaps. If unspanned blocking is used, compute the following:

- **a.** Blocking factor (i.e., the average number of blocks per record).
- **b.** Number of blocks needed to store the 20,000 records.
- **c.** Total size of the file.

**12.2.** One scheme to avoid the problem of preallocation versus waste or lack of contiguity is to allocate portions of increasing size as the file grows. For example, begin with a portion size of one block, and double the portion size for each allocation. Consider a file of $n$ records with a blocking factor of $F$, and suppose a simple one-level index is used as a file allocation table.

- **a.** Give an upper limit on the number of entries in the file allocation table as a function of $F$ and $n$.
- **b.** What is the maximum amount of the allocated file space that is unused at any time?

**12.3.** In a hashed file organization, the division method is used to compute the hash address of a record. This method can be stated as follows:

> Choose a large prime number $m$ which is close to the number of keys $n$. Define the hash function $h(k) = k \pmod{m} + c$, where $c$ is the lower limit of addresses.

If a set of records needs to be stored in 100 locations, starting from the address 7865, compute the address for the records having IDs 1234, 2345, 3333, and 4433.

**12.4.** For the B-tree in Figure 12.4c, show the result of inserting the key 97.

**12.5.** An alternative algorithm for insertion into a B-tree is the following: As the insertion algorithm travels down the tree, each full node that is encountered is immediately split, even though it may turn out that the split was unnecessary.

- **a.** What is the advantage of this technique?
- **b.** What are the disadvantages?

**12.6.** Both the search and the insertion time for a B-tree are a function of the height of the tree. We would like to develop a measure of the worst-case search or insertion time. Consider a B-tree of degree $d$ that contains a total of $n$ keys. Develop an inequality that shows an upper bound on the height $h$ of the tree as a function of $d$ and $n$.

**12.7.** A sequential file is stored in a disk occupying 100 contiguous disk blocks. The disk has an average rotational delay of 2.5 ms. The time taken to seek the head of the drive to the required cylinder is 25 ms and the time taken to read a block is 0.25 ms. Find the minimum, maximum, and average time to search for a record using a linear search process.

**12.8.** What will be the size of the bit table for a 160-GB disk with 1024-byte blocks?

**12.9.** Fragmentation of a disk can be removed by the process of compaction. Compaction involves a relocation of the files. But disks do not have relocation registers or base registers. How, then, can files be relocated in a disk?

**12.10.** Some operating systems have a tree–structured file system but limit the depth of the tree to some small number of levels. What effect does this limit have on users? How does this simplify file system design (if it does)?

**12.11.** Consider a hierarchical file system in which free disk space is kept in a free space list.

**a.** Suppose the pointer to free space is lost. Can the system reconstruct the free space list?

**b.** Suggest a scheme to ensure that the pointer is never lost as a result of a single memory failure.

**12.12.** A sequential file has 10 million records. How does efficiency in access improve by using a two-level index? Assume 100 entries in a higher-level index and 10,000 entries in a lower-level index.

**12.13.** Consider the organization of a UNIX file as represented by the inode (see Figure 12.15). Assume there are 12 direct block pointers, and a singly, doubly, and triply indirect pointer in each inode. Further, assume the system block size and the disk sector size are both 8K. If the disk block pointer is 32 bits, with 8 bits to identify the physical disk and 24 bits to identify the physical block, then:

**a.** What is the maximum file size supported by this system?

**b.** What is the maximum file system partition supported by this system?

**c.** Assuming no information other than that the file inode is already in main memory, how many disk accesses are required to access the byte in position 13,423,956?

# PART 6 Embedded Systems

## CHAPTER 13

# EMBEDDED OPERATING SYSTEMS


**13.1 Embedded Systems**
- Embedded System Concepts
- Application Processors versus Dedicated Processors
- Microprocessors
- Microcontrollers
- Deeply Embedded Systems

**13.2 Characteristics of Embedded Operating Systems**
- Host and Target Environments
- Development Approaches
- Adapting an Existing Commercial Operating System
- Purpose-Built Embedded Operating System

**13.3 Embedded Linux**
- Characteristics of an Embedded Linux System
- Embedded Linux File Systems
- Advantages of Embedded Linux
- $\mu$Clinux
- Android

**13.4 TinyOS**
- Wireless Sensor Networks
- TinyOS Goals
- TinyOS Components
- TinyOS Scheduler
- Example Configuration
- TinyOS Resource Interface

**13.5 Key Terms, Review Questions, and Problems**

**Learning Objectives**

After studying this chapter, you should be able to:

- Explain the concept of embedded system.
- Understand the characteristics of embedded operating systems.
- Explain the distinction between Linux and embedded Linux.
- Describe the architecture and key features of TinyOS.

In this chapter, we examine one of the most important and widely used categories of operating systems: embedded operating systems. The embedded system environment places unique and demanding requirements on the OS and calls for design strategies quite different than those found in ordinary operating systems.

We begin with an overview of the concept of embedded systems then turn to an examination of the principles of embedded operating systems. Finally, this chapter surveys two very different approaches to embedded OS design: embedded Linux and TinyOS. Appendix Q discusses eCos, another important embedded OS.

## 13.1 EMBEDDED SYSTEMS

This section introduces the concept of an embedded system. In doing so, we need to also explain the difference between a microprocessor and a microcontroller.

### Embedded System Concepts

The term *embedded system* refers to the use of electronics and software within a product that has a specific function or set of functions, as opposed to a general-purpose computer, such as a laptop or desktop system. We can also define an embedded system as any device that includes a computer chip, but that is not a general-purpose workstation, or desktop or laptop computer. Hundreds of millions of computers are sold every year, including laptops, personal computers, workstations, servers, mainframes, and supercomputers. In contrast, tens of billions of microcontrollers are produced each year that are embedded within larger devices. Today, many, perhaps most devices that use electric power have an embedded computing system. It is likely in the near future, virtually all such devices will have embedded computing systems.

Types of devices with embedded systems are almost too numerous to list. Examples include cell phones, digital cameras, video cameras, calculators, microwave ovens, home security systems, washing machines, lighting systems, thermostats, printers, various automotive systems (e.g., transmission control, cruise control, fuel injection, anti-lock brakes, and suspension systems), tennis rackets, toothbrushes, and numerous types of sensors and actuators in automated systems.

Often, embedded systems are tightly coupled to their environment. This can give rise to real-time constraints imposed by the need to interact with the environment. Constraints, such as required speeds of motion, required precision of measurement,

and required time durations, dictate the timing of software operations. If multiple activities must be managed simultaneously, this imposes more complex real-time constraints.

Figure 13.1 shows in general terms an embedded system organization. In addition to the processor and memory, there are a number of elements that differ from the typical desktop or laptop computer:

- There may be a variety of interfaces that enable the system to measure, manipulate, and otherwise interact with the external environment. Embedded systems often interact (sense, manipulate, and communicate) with the external world through sensors and actuators, and hence are typically reactive systems; a reactive system is in continual interaction with the environment and executes at a pace determined by that environment.
- The human interface may be as simple as a flashing light or as complicated as real-time robotic vision. In many cases, there is no human interface.
- The diagnostic port may be used for diagnosing the system that is being controlled—not just for diagnosing the computer.
- Special-purpose field programmable (FPGA), application-specific (ASIC), or even nondigital hardware may be used to increase performance or reliability.
- Software often has a fixed function and is specific to the application.
- Efficiency is of paramount importance for embedded systems. These systems are optimized for energy, code size, execution time, weight and dimensions, and cost.

There are several noteworthy areas of similarity to general-purpose computer systems as well:

- Even with nominally fixed function software, the ability to field upgrade to fix bugs, to improve security, and to add functionality have become very important for embedded systems, and not just in consumer devices.

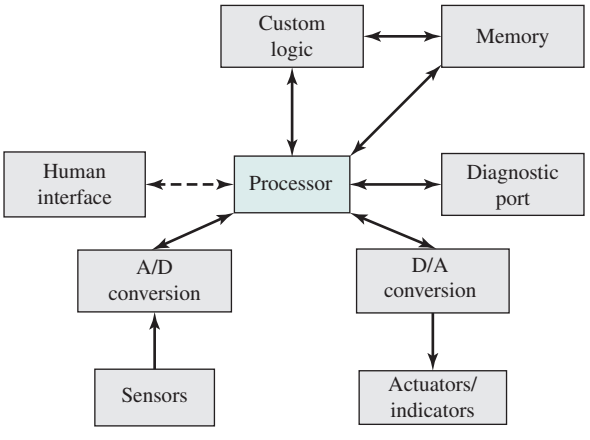



**Figure 13.1** **Possible Organization of an Embedded System**

- One comparatively recent development has been of embedded system platforms that support a wide variety of apps. Good examples of this are smartphones and audio/visual devices, such as smart TVs.

## Application Processors versus Dedicated Processors

**Application processors** are defined by the processor's ability to execute complex operating systems, such as Linux, Android, and Chrome. Thus, the application processor is general purpose in nature. A good example of the use of an embedded application processor is the smartphone. The embedded system is designed to support numerous apps and perform a wide variety of functions.

Most embedded systems employ a **dedicated processor**, which, as the name implies, is dedicated to one or a small number of specific tasks required by the host device. Because such an embedded system is dedicated to a specific task or tasks, the processor and associated components can be engineered to reduce size and cost.

## Microprocessors

A microprocessor is a processor whose elements have been miniaturized into one or a few integrated circuits. Early microprocessor chips included registers, an arithmetic logic unit (ALU), and some sort of control unit or instruction processing logic. As transistor density increased, it became possible to increase the complexity of the instruction set architecture, and ultimately to add memory and more than one processor. Contemporary microprocessor chips include multiple processors, called cores, and a substantial amount of cache memory. However, as shown in Figure 13.2, a microprocessor chip includes only some of the elements that make up a computer system.

Most computers, including embedded computers in smartphones and tablets, as well as personal computers, laptops, and workstations, are housed on a motherboard. Before describing this arrangement, we need to define some terms. A **printed circuit board** (PCB) is a rigid, flat board that holds and interconnects chips and other electronic components. The board is made of layers, typically two to ten, that interconnect components via copper pathways that are etched into the board. The main PCB in a computer is called a system board or **motherboard**, while smaller ones that plug into the slots in the main board are called expansion boards.

The most prominent elements on the motherboard are the chips. A **chip** is a single piece of semiconducting material, typically silicon, upon which electronic circuits and logic gates are fabricated. The resulting product is referred to as an **integrated circuit**.

The motherboard contains a slot or socket for the processor chip, which typically contains multiple individual cores, in what is known as a *multicore processor*. There are also slots for memory chips, I/O controller chips, and other key computer components. For desktop computers, expansion slots enable the inclusion of more components on expansion boards. Thus, a modern motherboard connects only a few individual chip components, with each chip containing from a few thousand up to hundreds of millions of transistors.

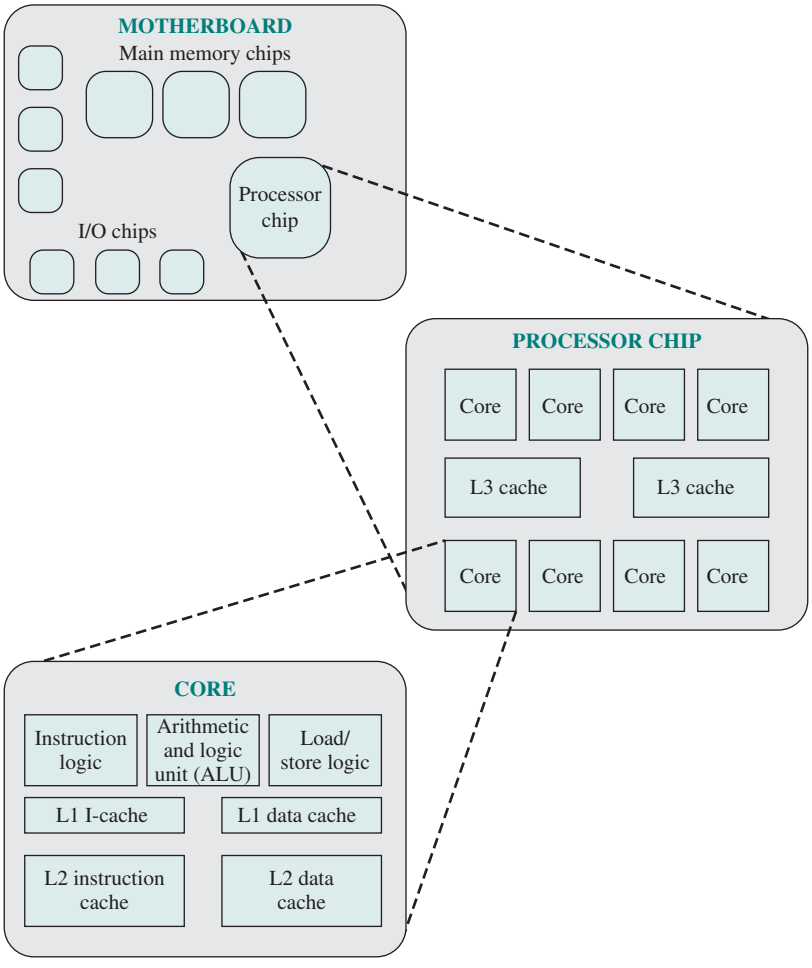



**Figure 13.2** **Simplified View of Major Elements of a Multicore computer**

## Microcontrollers

A **microcontroller** is a single chip that contains the processor, nonvolatile memory for the program (ROM or flash), volatile memory for input and output (RAM), a clock, and an I/O control unit. It is also called a "computer on a chip." A microcontroller chip makes a substantially different use of the logic space available. Figure 13.3 shows in general terms the elements typically found on a microcontroller chip. The processor portion of the microcontroller has a much lower silicon area than other microprocessors and much higher energy efficiency.

Billions of microcontroller units are embedded each year in myriad products from toys to appliances to automobiles. For example, a single vehicle can use 70 or more microcontrollers. Typically, especially for the smaller, less-expensive microcontrollers, they are used as dedicated processors for specific tasks. For example,

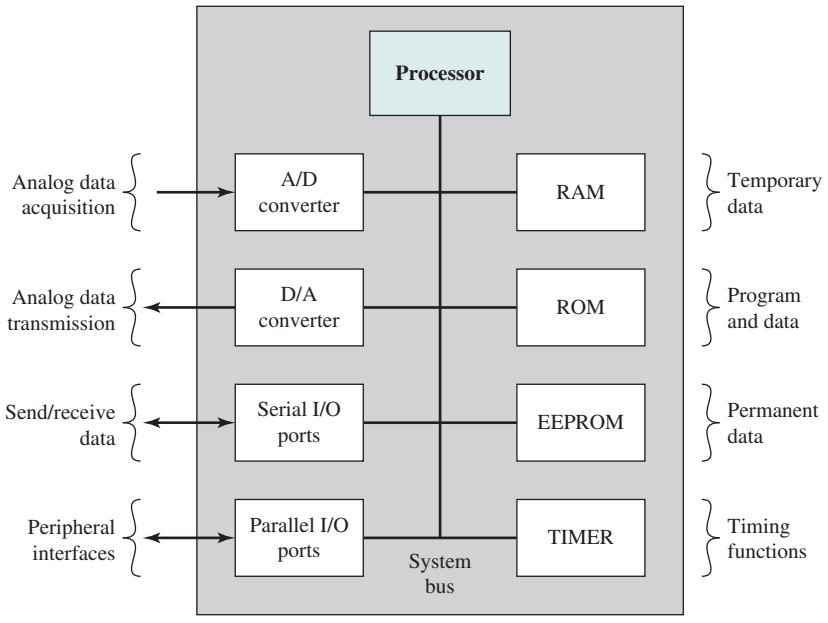



**Figure 13.3 Typical Microcontroller Chip Elements**

microcontrollers are heavily utilized in automation processes. By providing simple reactions to input, they can control machinery, turn fans on and off, open and close valves, and so forth. They are integral parts of modern industrial technology and are among the most inexpensive ways to produce machinery that can handle extremely complex functionalities.

Microcontrollers come in a range of physical sizes and processing power. Processors range from 4-bit to 32-bit architectures. Microcontrollers tend to be much slower than microprocessors, typically operating in the MHz range rather than the GHz speeds of microprocessors. Another typical feature of a microcontroller is that it does not provide for human interaction. The microcontroller is programmed for a specific task, embedded in its device, and executes as and when required.

## Deeply Embedded Systems

A large percentage of the total number of embedded systems are referred to as **deeply embedded systems**. Although this term is widely used in the technical and commercial literature, you will search the Internet in vain (at least the writer did) for a straightforward definition. Generally, we can say a deeply embedded system has a processor whose behavior is difficult to observe both by the programmer and the user. A deeply embedded system uses a microcontroller rather than a microprocessor, is not programmable once the program logic for the device has been burned into ROM (read-only memory), and has no interaction with a user.

Deeply embedded systems are dedicated, single-purpose devices that detect something in the environment, perform a basic level of processing, then do something with the results. Deeply embedded systems often have wireless capability

and appear in networked configurations, such as networks of sensors deployed over a large area (e.g., factory, agricultural field). The Internet of Things depends heavily on deeply embedded systems. Typically, deeply embedded systems have extreme resource constraints in terms of memory, processor size, time, and power consumption.

## 13.2 CHARACTERISTICS OF EMBEDDED OPERATING SYSTEMS

A simple embedded system, with simple functionality, may be controlled by a special-purpose program or set of programs with no other software. Typically, more complex embedded systems include an OS. Although it is possible in principle to use a general-purpose OS (such as Linux) for an embedded system, constraints of memory space, power consumption, and real-time requirements typically dictate the use of a special-purpose OS designed for the embedded system environment.

The following are some of the unique characteristics and design requirements for embedded operating systems:

- **Real-time operation:** In many embedded systems, the correctness of a computation depends, in part, on the time at which it is delivered. Often, real-time constraints are dictated by external I/O and control stability requirements.
- **Reactive operation:** Embedded software may execute in response to external events. If these events do not occur periodically or at predictable intervals, the embedded software may need to take into account worst-case conditions and set priorities for execution of routines.
- **Configurability:** Because of the large variety of embedded systems, there is a large variation in the requirements, both qualitative and quantitative, for embedded OS functionality. Thus, an embedded OS intended for use on a variety of embedded systems must lend itself to flexible configuration so only the functionality needed for a specific application and hardware suite is provided. [MARW06] gives the following examples: The linking and loading functions can be used to select only the necessary OS modules to load. Conditional compilation can be used. If an object-oriented structure is used, proper subclasses can be defined. However, verification is a potential problem for designs with a large number of derived tailored operating systems. Takada cites this as a potential problem for eCos [TAKA01].
- **I/O device flexibility:** There is virtually no device that needs to be supported by all versions of the OS, and the range of I/O devices is large. [MARW06] suggests that it makes sense to handle relatively slow devices (such as disks and network interfaces) by using special tasks instead of integrating their drives into the OS kernel.
- **Streamlined protection mechanisms:** Embedded systems are typically designed for a limited, well-defined functionality. Untested programs are rarely added to the software. After the software has been configured and tested, it can be assumed to be reliable. Thus, apart from security measures, embedded systems have limited protection mechanisms. For example, I/O instructions need

not be privileged instructions that trap to the OS; tasks can directly perform their own I/O. Similarly, memory protection mechanisms can be minimized. [MARW06] provides the following example: Let `switch` correspond to the memory-mapped I/O address of a value that needs to be checked as part of an I/O operation. We can allow the I/O program to use an instruction such as `load register, switch` to determine the current value. This approach is preferable to the use of an OS service call, which would generate overhead for saving and restoring the task context.

- **Direct use of interrupts:** General-purpose operating systems typically do not permit any user process to use interrupts directly. [MARW06] lists three reasons why it is possible to let interrupts directly start or stop tasks (e.g., by storing the task's start address in the interrupt vector address table) rather than going through OS interrupt service routines: (1) Embedded systems can be considered to be thoroughly tested, with infrequent modifications to the OS or application code; (2) protection is not necessary, as discussed in the preceding bullet item; and (3) efficient control over a variety of devices is required.

## Host and Target Environments

A key differentiator between desktop/server and embedded Linux distributions is that desktop and server software is typically compiled or configured on the platform where it will execute, while embedded Linux distributions are usually compiled or configured on one platform, called the host platform, but are intended to be executed on another, called the target platform (see Figure 13.4). The key elements that are developed on the host system and then transferred to the target system are the boot loader, the kernel, and the root file system.

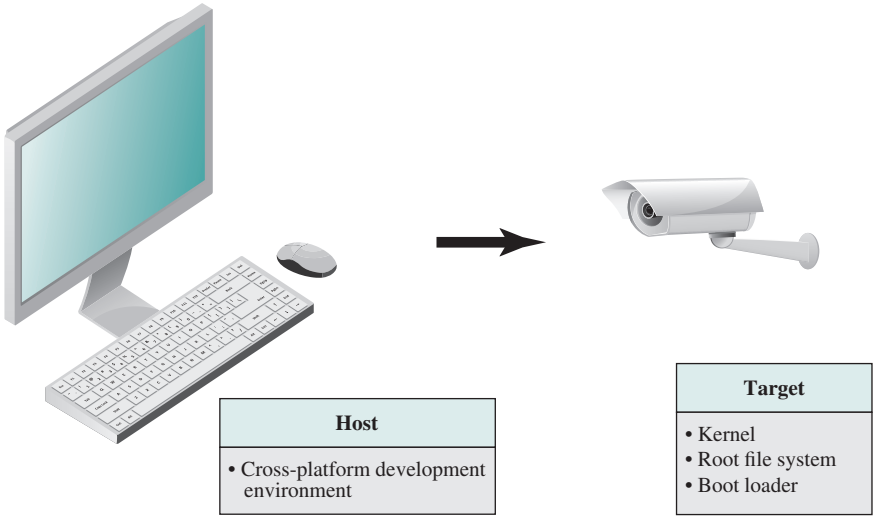



**Figure 13.4 Host-Target Environment**

***BOOT LOADER*** The boot loader is a small program that calls the OS into memory (RAM) after the power is turned on. It is responsible for the initial boot process of the system, and for loading the kernel into main memory. A typical sequence in an embedded system is the following:

1. The processor in the embedded system executes code in ROM to load a first-stage boot loader from internal flash memory, a Secure Digital (SD) card, or a serial I/O port.
2. The first-stage boot loader initializes the memory controller and a few peripherals and loads a second-stage boot loader into RAM. No interaction is possible with this boot loader, and it is typically provided by the processor vendor on ROM.
3. The second-stage boot loader loads the kernel and root file system from flash memory to main memory (RAM). The kernel and the root file system are generally stored in flash memory in compressed files, so part of the book loading process is to decompress the files into binary images of the kernel and root file system. The boot loader then passes control to the kernel. Typically, an open-source boot loader is used for the second stage.

***KERNEL*** The full kernel includes a number of separate modules, including:

- Memory management.
- Process/thread management.
- Inter process communication, timers.
- Device drivers for I/O, network, sound, storage, graphics, etc.
- File systems.
- Networking.
- Power management.

From the full kernel software for a given OS, a number of optional components will be left out for an embedded system. For example, if the embedded system hardware does not support paging, then the memory management subsystem can be eliminated. The full kernel will include multiple file systems, device drivers, and so on, and only a few of these may be needed.

A key differentiator between desktop/server and embedded Linux distributions is that desktop and server software is typically compiled on the platform where it will execute, while embedded Linux distributions are usually compiled on one platform but are intended to be executed on another. The software used for this purpose is referred to as a *cross-compiler*. Figure 13.5 illustrates its use.

***ROOT FILE SYSTEM*** In an embedded OS, or any OS, a global single hierarchy of directories and files is used to represent all the files in the system. At the top, or root of this hierarchy is the root file system, which contains all the files needed for the system to work properly. The root file system of an embedded OS is similar to that found on a workstation or server, except that it contains only the minimal set of applications, libraries, and related files needed to run the system.

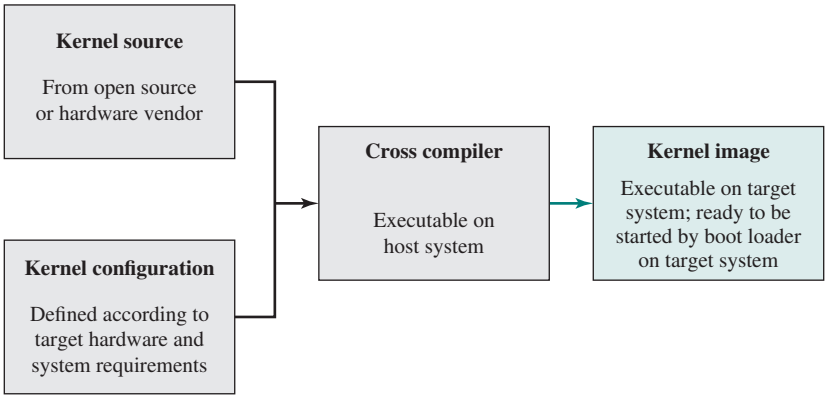



**Figure 13.5** **Kernel Compilation**

## Development Approaches

There are two general approaches to developing an embedded OS. The first approach is to take an existing OS and adapt it for the embedded application. The other approach is to design and implement an OS intended solely for embedded use.

## Adapting an Existing Commercial Operating System

An existing commercial OS can be used for an embedded system by adding real-time capability, streamlining operation, and adding necessary functionality. This approach typically makes use of Linux, but FreeBSD, Windows, and other general-purpose operating systems have also been used. Such operating systems are typically slower and less predictable than a special-purpose embedded OS. An advantage of this approach is that the embedded OS derived from a commercial general-purpose OS is based on a set of familiar interfaces, which facilitates portability.

The disadvantage of using a general-purpose OS is that it is not optimized for real-time and embedded applications. Thus, considerable modification may be required to achieve adequate performance. In particular, a typical OS optimizes for the average case rather than the worst case for scheduling, usually assigns resources on demand, and ignores most if not all semantic information about an application.

## Purpose-Built Embedded Operating System

A significant number of operating systems have been designed from the ground up for embedded applications. Two prominent examples of this latter approach are eCos and TinyOS, both of which will be discussed later in this chapter.

Typical characteristics of a specialized embedded OS include the following:

- Has a fast and lightweight process or thread switch
- Scheduling policy is real time and dispatcher module is part of scheduler instead of separate component.
- Has a small size

- Responds to external interrupts quickly; typical requirement is response time of less than 10 $\mu$s.
- Minimizes intervals during which interrupts are disabled
- Provides fixed or variable-sized partitions for memory management as well as the ability to lock code and data in memory
- Provides special sequential files that can accumulate data at a fast rate

To deal with timing constraints, the kernel:

- Provides bounded execution time for most primitives.
- Maintains a real-time clock.
- Provides for special alarms and time-outs.
- Supports real-time queuing disciplines such as earliest deadline first and primitives for jamming a message into the front of a queue.
- Provides primitives to delay processing by a fixed amount of time and to suspend/resume execution.

The characteristics just listed are common in embedded operating systems with real-time requirements. However, for complex embedded systems, the requirement may emphasize predictable operation over fast operation, necessitating different design decisions, particularly in the area of task scheduling.

## 13.3 EMBEDDED LINUX

The term *embedded Linux* simply means a version of Linux running in an embedded system. Typically, an embedded Linux system uses one of the official kernel releases, although some systems use a modified kernel tailored to a specific hardware configuration or to support a certain class of applications. Primarily, an embedded Linux kernel differs from a Linux kernel used on a workstation or server by the build configuration and development framework.

In this section, we highlight some of the key differences between embedded Linux and a version of Linux running on a desktop or server, then examine a popular software offering, $\mu$Clinux.

### Characteristics of an Embedded Linux System

***KERNEL SIZE*** Desktop and server Linux systems need to support a large number of devices because of the wide variety of configurations that use Linux. Similarly, such systems also need to support a range of communication and data exchange protocols so they can be used for a large number of different purposes. Embedded devices typically require support for a specific set of devices, peripherals, and protocols, depending on the hardware that is present in a given device and the intended purpose of that device. Fortunately, the Linux kernel is highly configurable in terms of the architecture for which it is compiled and the processors and devices that it supports.

An embedded Linux distribution is a version of Linux to be customized for the size and hardware constraints of embedded devices, and includes software packages that support a variety of services and applications on those devices. Thus, an embedded Linux kernel will be far smaller than an ordinary Linux kernel.

***MEMORY SIZE*** [ETUT16] classifies the size of an embedded Linux system by the amount of available ROM and RAM, using the three broad categories of small, medium, and large. Small systems are characterized by a low-powered processor with a minimum of 2 MB of ROM and 4 MB of RAM. Medium-sized systems are characterized by a medium-powered processor with around 32 MB of ROM and 64 MB of RAM. Large systems are characterized by a powerful processor or collection of processors combined with large amounts of RAM and permanent storage.

On a system without permanent storage, the entire Linux kernel must fit in the RAM and ROM. A full-featured modern Linux system would not do so. As an indication of this, Figure 13.6 shows the compressed size of the full Linux kernel as it has grown over time. Of course, any Linux system will be configured with only some of the components of the full release. Even so, this chart gives an indication of the fact that substantial amounts of the kernel must be left out, especially for small and medium-sized embedded systems.

***OTHER CHARACTERISTICS*** Other characteristics of embedded Linux systems include:

- **Time constraints:** Stringent time constraints require the system to respond in a specified time period. Mild time constraints are appropriate for systems in which timely response is not critical.

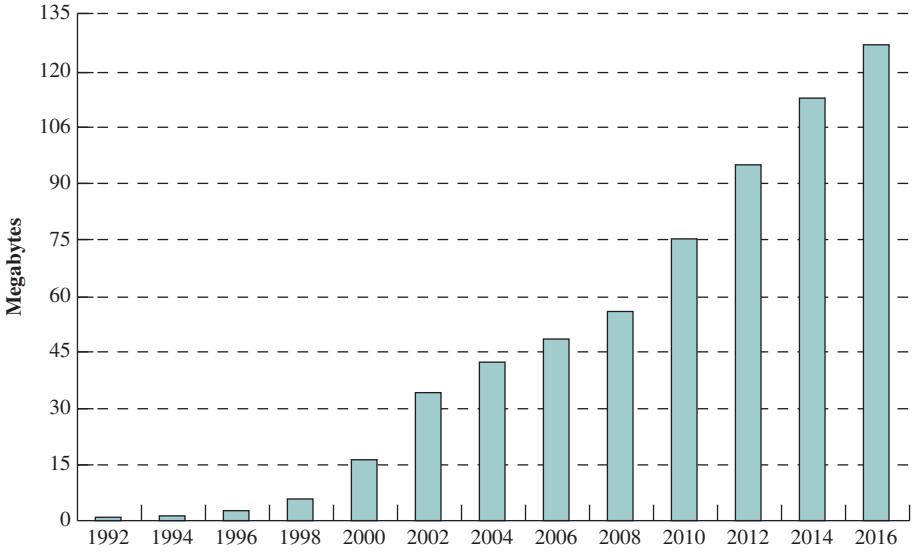



**Figure 13.6 Size of Linux Kernel (shown in GZIP-compressed file size)**

- **Networkability:** Networkability refers to whether a system can be connected to a network. Virtually all embedded devices today have this capability, typically wireless.
- **Degree of user interaction:** Some devices are centered on user interaction, such as smartphones. Other devices, such as industrial process control, might provide a very simple interface, such as LEDs and buttons for interaction. And other devices have no end user interaction, such as IoT sensors that gather information and transmit them to a cloud.

Table 13.1, from [ETUT16], gives characteristics of some commercially available embedded systems using a Linux kernel.

## Embedded Linux File Systems

Some applications may create relatively small file systems to be used only for the duration of the application and which can be stored in main memory. But in general, a file system must be stored in persistent memory, such as flash memory or traditional disk-based storage devices. For most embedded systems, an internal or external disk is not an option, and persistent storage is generally provided by flash memory.

As with other aspects of an embedded Linux system, the file system must be as small as possible. A number of such compact file systems have been designed for use in embedded systems. The following are commonly used examples:

- **cramfs:** The Compressed RAM file system is a simple read-only file system designed to minimize size by maximizing the efficient use of underlying storage. Files on cramfs file systems are compressed in units that match the Linux page size (typically 4096 bytes or 4 MB, based on kernel version and configuration) to provide efficient random access to file contents.

**Table 13.1** Characteristics of Example Embedded Linux Systems

| Description | Type | Size | Time Constraints | Networkability | Degree of User Interaction |
|---|---|---|---|---|---|
| Accelerator control devices | Industrial process control | Medium | Stringent | Yes | Low |
| Computer-aided training system | Aerospace | Large | Stringent | No | High |
| Bluetooth device for accessing local information | Networking | Small | Mild | Yes | Very low |
| System control and data acquisition protocol converter | Industrial process control | Medium | Stringent | No | Very low |
| Personal digital assistant | Consumer electronics | Medium | Mild | Yes | Very high |
| Motor control device involved with space vehicle control | Aerospace | Large | Stringent | Yes | High |

- **squashfs:** Like cramfs, squashfs is a compressed, read-only file system that was designed for use on low memory or limited storage size environments such as embedded Linux systems.
- **jffs2:** The Journaling Flash File System, version 2, is a log-based file system that, as the name suggests, is designed for use on NOR and NAND flash devices with special attention to flash-oriented issues such as wear leveling.
- **ubifs:** The Unsorted Block Image File System generally provides better performance than jffs2 on larger flash devices, and also supports write caching to provide additional performance improvements.
- **yaffs2:** Yet another Flash File System, version 2, provides a fast and robust file system for large flash devices. yaffs2 requires less RAM to hold file system state information than files systems such as jffs2, and also generally provides better performance if the file system is being written too frequently.

## Advantages of Embedded Linux

Embedded versions of Linux began to appear as early as 1999. A number of companies have developed their own versions tailored to specific platforms. Advantages of using Linux as the basis for an embedded OS include:

- **Vendor independence:** The platform provider is not dependent on a particular vendor to provide needed features and meet deadlines for deployment.
- **Varied hardware support:** Linux support for a wide range of processor architectures and peripheral devices makes it suitable for virtually any embedded system.
- **Low cost:** The use of Linux minimizes cost for development and training.
- **Open source:** The use of Linux provides all of the advantages of open-source software.

## $\mu$Clinux

$\mu$Clinux (microcontroller Linux) is a popular open-source Linux kernel variation targeted at microcontrollers and other very small embedded systems. Because of the modular nature of Linux, it is easy to slim down the operating environment by removing utility programs, tools, and other system services that are not needed in an embedded environment. This is the design philosophy for $\mu$Clinux.

To get some feel for the size of a $\mu$Clinux bootable image (kernel plus root file system), we look at the experience of EmCraft Systems, which builds board-level systems using Cortex-M microcontrollers and Cortex-A microprocessors [EMCR15]. These are by no means the smallest embedded systems that use $\mu$Clinux. A minimal configuration could be as little as 0.5 MB, but the vendor found the size of a practical bootable image, with Ethernet, TCP/IP and a reasonable set of user space tools and applications configured, would be in the range of 1.5 to 2 MB. The size of RAM required for run-time $\mu$Clinux operation would be in the range of 8 to 32 MB. These numbers are dramatically smaller than those of a typical Linux system.

***Comparison with Full Linux*** Key differences between μClinux and Linux for larger systems include the following (see [MCCU04] for a further discussion):

- Linux is a multiuser OS based on UNIX. μClinix is a version of Linux intended for embedded systems typically with no interactive user.
- Unlike Linux, μClinix does not support memory management. Thus, with μClinix there are no virtual address spaces; applications must be linked to absolute addresses.
- The Linux kernel maintains a separate virtual address space for each process. μClinix has a single shared address space for all processes.
- In Linux, address space is recovered on context switching; this is not done in μClinix.
- Unlike Linux, μClinix does not provide the fork system call; the only option is to use vfork. The fork call essentially makes a duplicate of the calling process, identical in almost every way (not everything is copied over, for example, resource limits in some implementations, but the idea is to create as close a copy as possible). The new process (child) gets a different process ID (PID) and has the PID of the old process (parent) as its parent PID (PPID). The basic difference between vfork and fork is that when a new process is created with vfork, the parent process is temporarily suspended, and the child process might borrow the parent's address space. This continues until the child process either exits, or calls execve, at which point the parent process continues.
- μClinix modifies device drivers to use the local system bus rather than ISA or PCI.

The most significant difference between full Linux and μClinux is in the area of memory management. The lack of memory management support in μClinux has a number of implications, including:

- The main memory allocated to a process must generally be contiguous. If a number of processes swap in and out of memory, this can lead to fragmentation (see Figure 7.4). However, embedded systems typically have a fixed set of processes that are loaded at boot up time and continue until the next reset, so this feature is generally not needed.
- μClinux cannot expand memory for running process, because there may be other processes contiguous to it. Thus, the brk and sbrk calls (dynamically change the amount of space allocated for the data segment of the calling process) are not available. But μClinux does provide an implementation of malloc, which is used to allocate a block of memory from a global memory pool.
- μClinux lacks a dynamic application stack. This can result in a stack overflow, which will corrupt memory. Care must be taken in application development and configuration to avoid this.
- μClinux does not provide memory protection, which presents the risk of an application corrupting part of another application or even the kernel. Some implementations do provide a fix for this. For example, the Cortex-M3/M4 architecture provides a memory protection mechanism called MPU (Memory

**Table 13.2** Size of Some Functions in GNU C Library and μClibc

| glibc name | glibc size | μClibc name | μClibc size |
|---|---|---|---|
| libc-2.3.2.so | 1.2M | libuClibc-0.9.2.7so | 284K |
| ld-2.3.2.so | 92K | libcrypt-0.9.2.7so | 20K |
| libcrypt-2.3.2.so | 20K | libdl-0.9.2.7so | 12K |
| libdl-2.3.2.so | 12K | libm-0.9.2.7so | 8K |
| libm-2.3.2.so | 136K | libnsl-0.9.2.7so | 56K |
| libnsl-2.3.2.so | 76K | libpthread-0.9.2.7so | 4K |
| libpthread-2.3.2.so | 84K | libresolv-0.9.2.7so | 84K |
| libresolv-2.3.2.so | 68K | libutil-0.9.2.7so | 4K |
| libutil-2.3.2.so | 8K | libcrypt-0.9.2.7so | 8K |

Protection Unit). Using the MPU, Emcraft Systems has added to the kernel an optional feature that implements process-to-process and process-to-kernel protection on par with the memory protection mechanisms implemented in Linux using MMU [KHUS12].

***μCLIBC*** μClibc is a C system library originally developed to support μClinux and it generally used in conjunction with μClinux. However, μClibc can also be used with other Linux kernels. The main objective for μClibc is to provide a system library that is to provide a C library suitable for developing embedded Linux system. It is much smaller than the GNU C Library, which is widely used on Linux systems, but nearly all applications supported by glibc also work perfectly with μClibc. Porting applications from glibc to μClibc typically involves just recompiling the source code. μClibc even supports shared libraries and threading.

Table 13.2, based on [ANDE05], compares the sizes of functions in the two libraries. As can be seen, the space savings are considerable. These savings are achieved by disabling some features by default and aggressively rewriting the code to eliminate redundancy.

Figure 13.7 shows the top-level software architecture of an embedded system using μClinux and μClibc.

## Android

As we have discussed throughout this book, Android is an embedded OS based on a Linux kernel. Thus, it is reasonable to consider Android an example of embedded Linux. However, many embedded Linux developers do not consider Android to be an instance of embedded Linux [CLAR13]. From the point of view of these developers, a classic embedded device has a fixed function, frozen at the factory. Android is more of a platform OS that can support a variety of applications that vary from one platform to the next. Further, Android is a vertically integrated system, including some Android-specific modifications to the Linux kernel. The focus of Android lies in

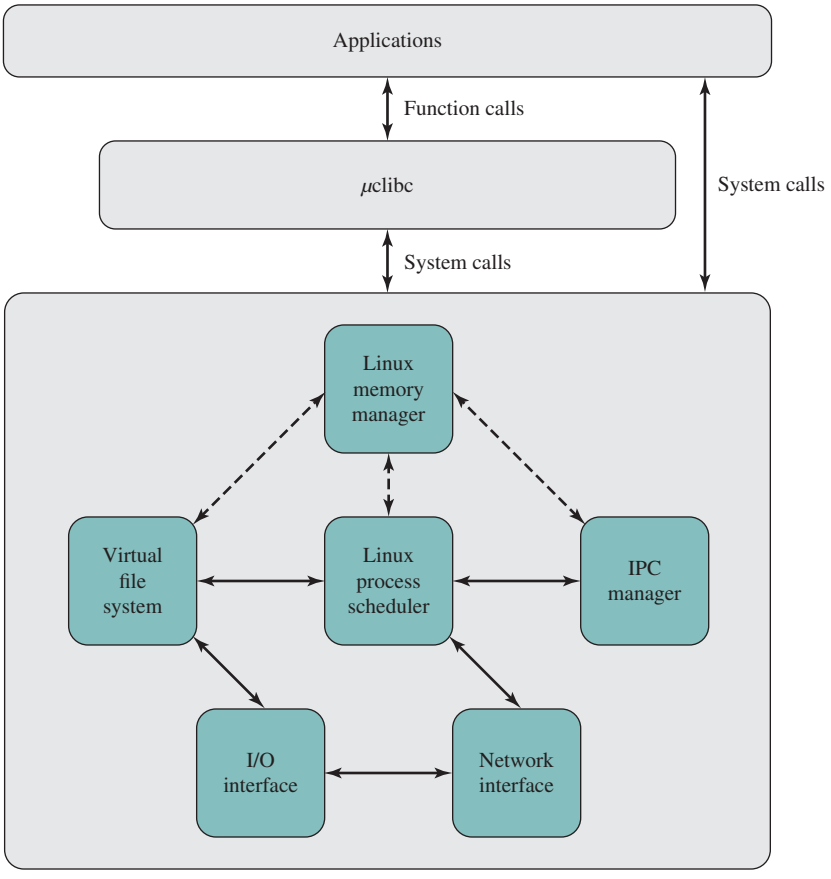



**Figure 13.7** **μClinux/μClibc Software Architecture**

the vertical integration of the Linux kernel and the Android user space components. Ultimately, it is a matter of semantics, with no "official" definition of embedded Linux on which to rely.

## 13.4 TINYOS

TinyOS provides a more streamlined approach for an embedded OS than one based on a commercial general-purpose OS, such as an embedded version of Linux. Thus, TinyOS and similar systems are better suited for small embedded systems with tight requirements on memory, processing time, real-time response, power consumption, and so on. TinyOS takes the process of streamlining quite far, resulting in a very minimal OS for embedded systems. The core OS requires 400 bytes of code and data memory, combined.

TinyOS represents a significant departure from other embedded operating systems. One striking difference is that TinyOS is not a real-time OS. The reason for

this is the expected workload, which is in the context of a wireless sensor network, as described in the next subsection. Because of power consumption, these devices are off most of the time. Applications tend to be simple, with processor contention not much of an issue.

Additionally, in TinyOS there is no kernel, as there is no memory protection and it is a component-based OS; there are no processes; the OS itself does not have a memory allocation system (although some rarely used components do introduce one); interrupt and exception handling is dependent on the peripheral; and it is completely nonblocking, so there are few explicit synchronization primitives.

TinyOS has become a popular approach to implementing wireless sensor network software. Currently, over 500 organizations are developing and contributing to an open-source standard for Tiny OS.

## Wireless Sensor Networks

TinyOS was developed primarily for use with networks of small wireless sensors. A number of trends have enabled the development of extremely compact, low-power sensors. The well-known Moore's law continues to drive down the size of memory and processing logic elements. Smaller size in turn reduces power consumption. Low power and small-size trends are also evident in wireless communications hardware, micro-electromechanical sensors (MEMS), and transducers. As a result, it is possible to develop an entire sensor complete with logic in a cubic millimeter. The application and system software must be compact enough that sensing, communication, and computation capabilities can be incorporated into a complete, but tiny, architecture.

Low–cost, small–size, low-power-consuming wireless sensors can be used in a host of applications [ROME04]. Figure 13.8 shows a typical configuration. A base station connects the sensor network to a host PC and passes on sensor data from the network to the host PC, which can do data analysis and/or transmit the data

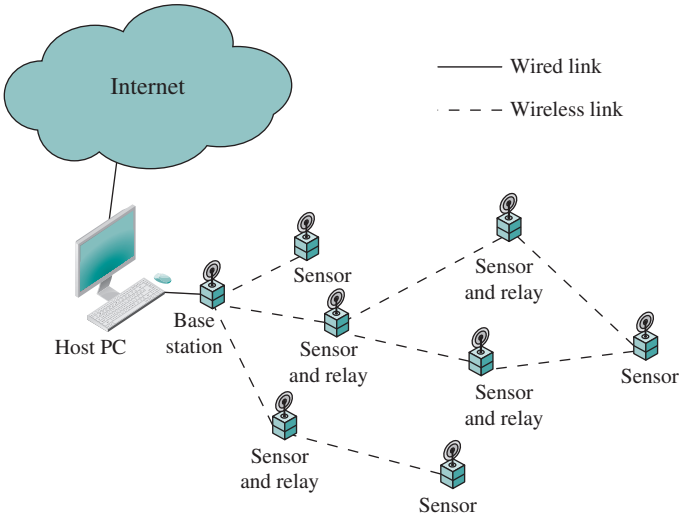



**Figure 13.8 Typical Wireless Sensor Network Topology**

over a corporate network or Internet to an analysis server. Individual sensors collect data and transmit these to the base station, either directly or through sensors that act as data relays. Routing functionality is needed to determine how to relay the data through the sensor network to the base station. [BUON01] points out that, in many applications, the user will want to be able to quickly deploy a large number of low-cost devices without having to configure or manage them. This means that they must be capable of assembling themselves into an ad hoc network. The mobility of individual sensors and the presence of RF interference means the network will have to be capable of reconfiguring itself in a matter of seconds.

## TinyOS Goals

With the tiny, distributed sensor application in mind, a group of researchers from UC Berkeley [HILL00] set the following goals for TinyOS:

- **Allow high concurrency:** In a typical wireless sensor network application, the devices are concurrency intensive. Several different flows of data must be kept moving simultaneously. While sensor data are input in a steady stream, processed results must be transmitted in a steady stream. In addition, external controls from remote sensors or base stations must be managed.
- **Operate with limited resources:** The target platform for TinyOS will have limited memory and computational resources and run on batteries or solar power. A single platform may offer only kilobytes of program memory and hundreds of bytes of RAM. The software must make efficient use of the available processor and memory resources while enabling low-power communication.
- **Adapt to hardware evolution:** Most hardware is in constant evolution; applications and most system services must be portable across hardware generations. Thus, it should be possible to upgrade the hardware with little or no software change, if the functionality is the same.
- **Support a wide range of applications:** Applications exhibit a wide range of requirements in terms of lifetime, communication, sensing, and so on. A modular, general-purpose embedded OS is desired so a standardized approach leads to economies of scale in developing applications and support software.
- **Support a diverse set of platforms:** As with the preceding point, a general-purpose embedded OS is desirable.
- **Be robust:** Once deployed, a sensor network must run unattended for months or years. Ideally, there should be redundancy both within a single system and across the network of sensors. However, both types of redundancy require additional resources. One software characteristic that can improve robustness is to use highly modular, standardized software components.

It is worth elaborating on the concurrency requirement. In a typical application, there will be dozens, hundreds, or even thousands of sensors networked together. Usually, little buffering is done, because of latency issues. For example, if you are sampling every 5 minutes and want to buffer four samples before sending, the average latency is 10 minutes. Thus, information is typically captured, processed, and streamed onto the network in a continuous flow. Further, if the sensor sampling produces a

significant amount of data, the limited memory space available limits the number of samples that could be buffered. Even so, in some applications, each of the flows may involve a large number of low-level events interleaved with higher-level processing. Some of the high-level processing will extend over multiple real-time events. Further, sensors in a network, because of the low power of transmission available, typically operate over a short physical range. Thus data from outlying sensors must be relayed to one or more base stations by intermediate nodes.

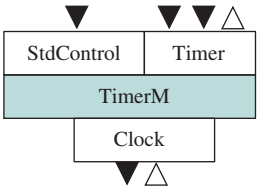
## TinyOS Components

An embedded software system built using TinyOS consists of a set of small modules, called components, each of which performs a simple task or set of tasks and which interface with each other and with hardware in limited and well-defined ways. The only other software module is the scheduler, discussed subsequently. In fact, because there is no kernel, there is no actual OS. But we can take the following view. The application area of interest is the wireless sensor network (WSN). To meet the demanding software requirements of this application, a rigid, simplified software architecture is dictated, consisting of components. The TinyOS development community has implemented a number of open-source components that provide the basic functions needed for the WSN application. Examples of such standardized components include single-hop networking, ad hoc routing, power management, timers, and nonvolatile storage control. For specific configurations and applications, users build additional special-purpose components and link and load all of the components needed for the user's application. TinyOS, then, consists of a suite of standardized components. Some, but not all, of these components are used, together with application-specific user-written components, for any given implementation. The OS for that implementation is simply the set of standardized components from the TinyOS suite.

All components in a TinyOS configuration have the same structure, an example of which is shown in Figure 13.9a. The shaded box in the diagram indicates the component, which is treated as an object that can only be accessed by defined interfaces, indicated by white boxes. A component may be hardware or software. Software components are implemented in nesC, which is an extension of C with two distinguishing features: 1) a programming model where components interact via interfaces, and 2) an event-based concurrency model with run-to-completion task and interrupt handlers, explained subsequently.

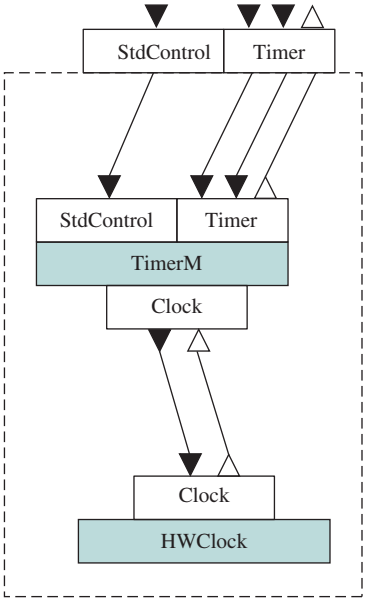
The architecture consists of a layered arrangement of components. Each component can link to only two other components, one below it in the hierarchy and one above it. A component issues commands to its lower-level component and receives event signals from it. Similarly, the component accepts commands from its upper-level component and issues event signals to it. At the bottom of the hierarchy are hardware components, and at the top of the hierarchy are application components, which may not be part of the standardized TinyOS suite but which must conform to the TinyOS component structure.

A software component implements one or more tasks. Each **task** in a component is similar to a thread in an ordinary OS, with certain limitations. Within a component, tasks are atomic: Once a task has started, it runs to completion. It cannot




(a) TimerM component

```
module TimerM {
   provides {
      interface StdControl;
      interface Timer;
   }
   uses interface Clock as Clk;
} ...
```




(b) TimerC configuration

```
configuration TimerC {
   provides {
      interface StdControl;
      interface Timer;
   }
}

implementation {
   components TimerM, HWClock;
   StdControl = TimerM.StdControl;
   Timer = TimerM.Timer;
   TimerM.Clk -> HWClock.Clock;
}
```

**Figure 13.9 Example of Component and Configuration**

be preempted by another task in the same component, and there is no time slicing. However, a task can be preempted by an event. A task cannot block or spin wait. These limitations greatly simplify the scheduling and management of tasks within a component. There is only a single stack, assigned to the currently running task. Tasks can perform computations, call lower-level components (commands) and signal higher-level events, and schedule other tasks.

**Commands** are nonblocking requests. That is, a task that issues a command does not block or spin wait for a reply from the lower-level component. A command is typically a request for the lower-level component to perform some service, such as initiating a sensor reading. The effect on the component that receives the command is specific to the command given and the task required to satisfy the command. Generally, when a command is received, a task is scheduled for later execution, because a command cannot preempt the currently running task. The command returns immediately to the calling component; at a later time, an event will signal completion to the calling component. Thus, a command does not cause a preemption in the called component, and does not cause blocking in the calling component.

**Events** in TinyOS may be tied either directly or indirectly to hardware events. The lowest-level software components interface directly to hardware interrupts, which may be external interrupts, timer events, or counter events. An event handler in a lowest-level component may handle the interrupt itself, or may propagate event messages up through the component hierarchy. A command can post a task that will signal an event in the future. In this case, there is no tie of any kind to a hardware event.

A task can be viewed as having three phases. A caller posts a command to a module. The module then runs the requested task. The module then notifies the caller, via an event, that the task is complete.

The component depicted in Figure 13.9a, TimerM, is part of the TinyOS timer service. This component **provides** the StdControl and Timer interface and **uses** a Clock interface. Providers implement commands (i.e., the logic in this component). Users implement events (i.e., external to the component). Many TinyOS components use the StdControl interface to be initialized, started, or stopped. TimerM provides the logic that maps from a hardware clock into TinyOS's timer abstraction. The timer abstraction can be used for counting down a given time interval. Figure 13.9a also shows the formal specification of the TimerM interfaces.

The interfaces associated with TimerM are specified as follows:

```
interface StdControl {
     command result_t init();
     command result_t start();
     command result_t stop();
}
interface Timer {
    command result_t start(char type, uint32_t interval);
    command result_t stop();
    event result_t fired();
}
interface Clock {
    command result_t setRate(char interval, char scale);
    event result_t fire();
}
```

Components are organized into configurations by "wiring" them together at their interfaces and equating the interfaces of the configuration with some of the interfaces of the components. A simple example is shown in Figure 13.9b. The uppercase C stands for Component. It is used to distinguish between an interface (e.g., Timer) and a component that provides the interface (e.g., TimerC). The uppercase M stands for Module. This naming convention is used when a single logical component has both a configuration and a module. The TimerC component, providing the Timer interface, is a configuration that links its implementation (TimerM) to Clock and LED providers. Otherwise, any user of TimerC would have to explicitly wire its subcomponents.

## TinyOS Scheduler

The TinyOS scheduler operates across all components. Virtually all embedded systems using TinyOS will be uniprocessor systems, so only one task among all the tasks in all the components may execute at a time. The scheduler is a separate component. It is the one portion of TinyOS that must be present in any system.

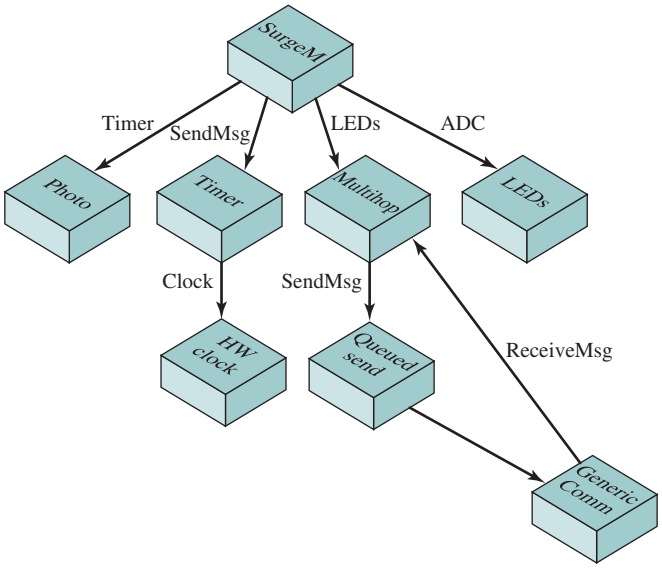
The default scheduler in TinyOS is a simple FIFO (first-in-first-out) queue. A task is posted to the scheduler (place in the queue) either as a result of an event, which triggers the posting, or as a result of a specific request by a running task to schedule another task. The scheduler is power aware. This means the scheduler puts the processor to sleep when there are no tasks in the queue. The peripherals remain operating, so one of them can wake up the system by means of a hardware event signaled to a lowest-level component. Once the queue is empty, another task can be scheduled only as a result of a direct hardware event. This behavior enables efficient battery usage.

The scheduler has gone through two generations. In TinyOS 1.x, there is a shared task queue for all tasks, and a component can post a task to the scheduler multiple times. If the task queue is full, the post operation fails. Experience with networking stacks showed this to be problematic, as the task might signal completion of a split-phase operation: If the post fails, the component above might block forever, waiting for the completion event. In TinyOS 2.x, every task has its own reserved slot in the task queue, and a task can only be posted once. A post fails if and only if the task has already been posted. If a component needs to post a task multiple times, it can set an internal state variable so that when the task executes, it reposts itself. This slight change in semantics greatly simplifies a lot of component code. Rather than test to see if a task is posted already before posting it, a component can just post the task. Components do not have to try to recover from failed posts and retry. The cost is one byte of state per task.

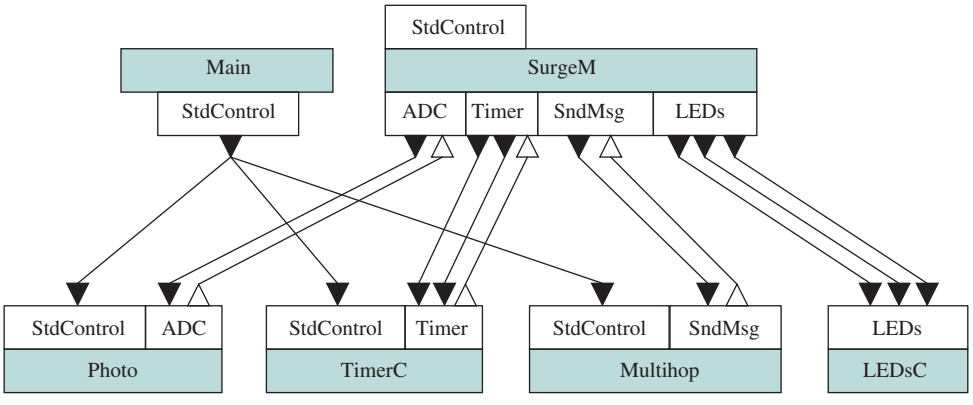
A user can replace the default scheduler with one that uses a different dispatching scheme, such as a priority-based scheme or a deadline scheme. However, preemption and time slicing should not be used because of the overhead such systems generate. More importantly, they violate the TinyOS concurrency model, which assumes tasks do not preempt each other.

## Example of Configuration

Figure 13.10 shows a configuration assembled from software and hardware components. This simplified example, called Surge and described in [GAY03], performs periodic sensor sampling and uses ad hoc multihop routing over the wireless network to deliver samples to the base station. The upper part of the figure shows the components of Surge (represented by boxes) and the interfaces by which they are wired (represented by arrowed lines). The SurgeM component is the application-level component that orchestrates the operation of the configuration.




(a) Simplified view of the Surge application




(b) Top-level Surge configuration

LED = light-emitting diode
ADC = analog-to-digital converter

**Figure 13.10 Examples of TinyOS Application**

Figure 13.10b shows a portion of the configuration for the Surge application. The following is a simplified excerpt from the SurgeM specification.

```
module SurgeM {
    provides interface StdControl;
    uses interface ADC;
    uses interface Timer;
    uses interface SendMsg;
    uses interface LEDs;
}
```

```
implementation {
     uint16_t sensorReading;
     command result_t StdControl.init() {
     return call Timer.start(TIMER_REPEAT, 1000);
     }
     event result_t Timer.fired() {
     call ADC.getData();
     return SUCCESS;
     }
     event result_t ADC.dataReady(uint16_t data) {
     sensorReading = data;
     ...send message with data in it...
     return SUCCESS;
     }
     ...
}
```

This example illustrates the strength of the TinyOS approach. The software is organized as an interconnected set of simple modules, each of which defines one or a few tasks. Components have simple, standardized interfaces to other components, be they hardware or software. Thus, components can easily be replaced. Components can be hardware or software, with a boundary change not visible to the application programmer.

## TinyOS Resource Interface

TinyOS provides a simple but powerful set of conventions for dealing with resources. Three abstractions for resources are used in TinyOS:

1. **Dedicated:** A resource that a subsystem needs exclusive access to at all times. In this class of resources, no sharing policy is needed since only a single component ever requires use of the resource. Examples of dedicated abstractions include interrupts and counters.
2. **Virtualized:** Every client of a virtualized resource interacts with it as if it were a dedicated resource, with all virtualized instances being multiplexed on top of a single underlying resource. The virtualized abstraction may be used when the underlying resource need not be protected by mutual exclusion. An example is a clock or timer.
3. **Shared:** The shared resource abstraction provides access to a dedicated resource through an arbiter component. The arbiter enforces mutual exclusion, allowing only one user (called a client) at a time to have access to a resource and enabling the client to lock the resource.

In the remainder of this subsection, we briefly define the shared resource facility of TinyOS. The arbiter determines which client has access to the resource at which time. While a client holds a resource, it has complete and unfettered control. Arbiters assume clients are cooperative, only acquiring the resource when needed and holding on to it no longer than necessary. Clients explicitly release resources: There is no way for an arbiter to forcibly reclaim it.

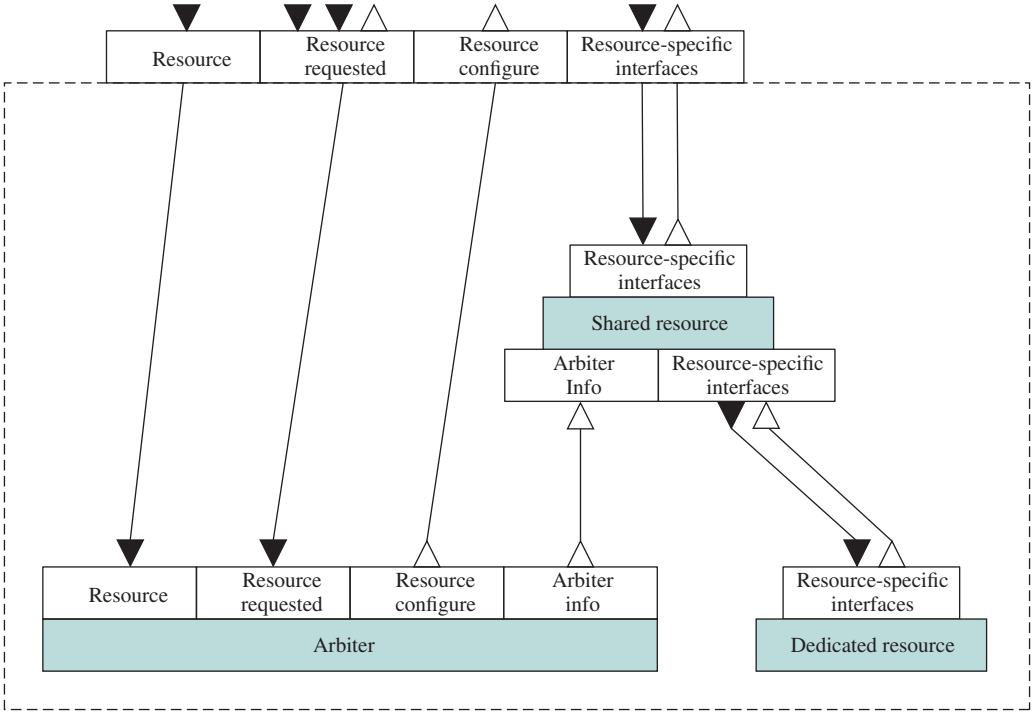



**Figure 13.11 Shared Resource Configuration**

Figure 13.11 shows a simplified view of the shared resource configuration used to provide access to an underlying resource. Associated with each resource to be shared is an arbiter component. The Arbiter enforces a policy that enables a client to lock the resource, use it, then release the resource. The shared resource configuration provides the following interfaces to a client:

- **Resource:** The client issues a request at this interface, requesting access to the resource. If the resource is currently locked, the arbiter places the request in a queue. When a client is finished with the resource, it issues a release command at this interface.
- **Resource requested:** This is similar to the Resource interface. In this case, the client is able to hold on to a resource until the client is notified that someone else needs the resource.
- **Resource Configure:** This interface allows a resource to be automatically configured just before a client is granted access to it. Components providing the Resource Configure interface use the interfaces provided by an underlying dedicated resource to configure it into one of its desired modes of operation.
- **Resource-specific interfaces:** Once a client has access to a resource, it uses resource-specific interfaces to exchange data and control information with the resource.

In addition to the dedicated resource, the shared resource configuration consists of two components. The Arbiter accepts requests for access and configuration from a

client and enforces the lock on the underlying resource. The shared resource component mediates data exchange between the client and the underlying resource. Arbiter information passed from the arbiter to the shared resource component controls the access of the client to the underlying resource.

## 13.5 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

### Key Terms

| | | |
|---|---|---|
| application processors, chip | embedded operating system | microcontroller |
| commands | embedded system | motherboard |
| dedicated processor | events | printed circuit board |
| eCos | deeply embedded system | task |
| | integrated circuit | TinyOS |

### Review Questions

**13.1.** What is an embedded system?
**13.2.** What are some typical requirements or constraints on embedded systems?
**13.3.** What is an embedded OS?
**13.4.** What are some of the key characteristics of an embedded OS?
**13.5.** Explain the relative advantages and disadvantages of an embedded OS based on an existing commercial OS compared to a purpose-built embedded OS.
**13.6.** What is the target application for TinyOS?
**13.7.** What are the design goals for TinyOS?
**13.8.** What is a TinyOS component?
**13.9.** What software comprises the TinyOS operating system?
**13.10.** What is the default scheduling discipline for TinyOS?

### Problems

**13.1.** In a particular sensor network that runs on TinyOS, the default FIFO algorithm has been replaced with a priority-based one that continually checks for high priority tasks before assigning it to schedulers. What constraints will this technique face?

**13.2.** **a.** The TinyOS Resource interface does not allow a component that already has a request in the queue for a resource to make a second request. Suggest a reason.

**b.** However, the TinyOS Resource interface allows a component holding the resource lock to re-request the lock. This request is queued for a later grant. Suggest a reason for this policy. *Hint:* What might cause there to be latency between one component releasing a lock and the next requester being granted it?

*Note*: The remaining problems concern eCos, discussed in Appendix Q.

**13.3.** With reference to the device driver interface to the eCos kernel (see Table Q.1), it is recommended that device drivers should use the `_intsave()` variants to claim and release spinlocks rather than the non-`_intsave()` variants. Explain why

**13.4.** Also in Table Q.1, it is recommended that `cyg_drv_spinlock_spin` should be used sparingly, and in situations where deadlocks/livelocks cannot occur. Explain why.

**13.5.** In Table Q.1, what should be the limitations on the use of `cyg_drv_spinlock_destroy`? Explain.

**13.6.** In Table Q.1, what limitations should be placed in the use of `cyg_drv_mutex_destroy`?

**13.7.** Why does the eCos bitmap scheduler not support time slicing?

**13.8.** The implementation of mutexes within the eCos kernel does not support recursive locks. If a thread has locked a mutex then attempts to lock the mutex again, typically as a result of some recursive call in a complicated call graph, then either an assertion failure will be reported or the thread will deadlock. Suggest a reason for this policy.

**13.9.** Figure 13.12 is a listing of code intended for use on the eCos kernel.

**a.** Explain the operation of the code. Assume thread B begins execution first, and thread A begins to execute after some event occurs.

**b.** What would happen if the mutex unlock and wait code execution in the call to `cyg_cond_wait`, on line 30, were not atomic?

**c.** Why is the while loop on line 26 needed?

**13.10.** The discussion of eCos spinlocks included an example showing why spinlocks should not be used on a uniprocessor system if two threads of different priorities can compete for the same spinlock. Explain why the problem still exists even if only threads of the same priority can claim the same spinlock.

```
1   unsigned char buffer_empty = true;
2   cyg_mutex_t mut_cond_var;
3   cyg_cond-t cond_var;
4
5   void thread_a( cyg_addrword_t index )
6   {
7      while ( 1 ) // run this thread forever
8      {
9         // acquire data into the buffer...
10
11        // there is data in the buffer now
12        buffer_empty = false;
13
14        cyg_mutex_lock( &mut_cond_var );
15
16        cyg_cond_signal( &cond_var );
17
18        cyg_mutex_unlock( &mut_cond_var );
19     }
20  }
21
22  void thread_b( cyg_addrword_t index )
23  {
24     while ( 1 ) // run this thread forever
25     {
26        cyg_mutex_lock( &mut_cond_var );
27
28        while ( buffer_empty == true )
29        {
30        cyg_cond_wait( &cond_var );
31        }
32
33
34        // get the buffer data...
35
36        // set flag to indicate the data in the buffer has been processed
37        buffer_empty = true;
38
39        cyg_mutex_unlock( &mut_cond_var );
40
41        // process the data in the buffer
42     }
43  {
```

VideoNote **Figure 13.12 Condition Variable Example Code**

CHAPTER 14

# VIRTUAL MACHINES


14.1 **Virtual Machine Concepts**

14.2 **Hypervisors**
- Hypervisors
- Paravirtualization
- Hardware-Assisted Virtualization
- Virtual Appliance

14.3 **Container Virtualization**
- Kernel Control Groups
- Container Concepts
- Container File System
- Microservices
- Docker

14.4 **Processor Issues**

14.5 **Memory Management**

14.6 **I/O Management**

14.7 **VMware ESXi**

14.8 **Microsoft Hyper-V and Xen Variants**

14.9 **Java VM**

14.10 **Linux Vserver Virtual Machine Architecture**
- Architecture
- Process Scheduling

14.11 **Summary**

14.12 **Key Terms, Review Questions, and Problems**

### Learning Objectives

After studying this chapter, you should be able to:

- Discuss Type 1 and Type 2 virtualization.
- Explain container virtualization and compare it to the hypervisor approach.
- Understand the processor issues involved in implementing a virtual machine.
- Understand the memory management issues involved in implementing a virtual machine.
- Understand the I/O management issues involved in implementing a virtual machine.
- Compare and contrast VMware ESXi, Hyper-V, Xen, and Java VM.
- Explain the operation of the Linux virtual machine.

This chapter focuses on the application of virtualization to operating system design. Virtualization encompasses a variety of technologies for managing computing resources by providing a software translation layer, known as an abstraction layer, between the software and the physical hardware. Virtualization turns physical resources into logical, or virtual, resources. Virtualization enables users, applications, and management software operating above the abstraction layer to manage and use resources without needing to be aware of the physical details of the underlying resources.

The first three sections of this chapter deal with the two main approaches to virtualization: virtual machines and containers. The remainder of the chapter looks at some specific systems.

## 14.1 VIRTUAL MACHINE CONCEPTS

Traditionally, applications have run directly on an operating system (OS) on a personal computer (PC) or on a server, with the PC or server running only one OS at a time. Thus, the application vendor had to rewrite parts of its applications for each OS/platform they would run on and support, which increased time to market for new features/functions, increased the likelihood of defects, increased quality testing efforts, and usually led to increased price. To support multiple OSs, application vendors needed to create, manage, and support multiple hardware and OS infrastructures, a costly and resource-intensive process. One effective strategy for dealing with this problem is known as **hardware virtualization**. Virtualization technology enables a single PC or server to simultaneously run multiple OSs or multiple sessions of a single OS. A machine with virtualization software can host numerous applications, including those that run on different OSs, on a single platform. In essence, the host OS can support a number of **virtual machines (VMs)**, each of that has the characteristics of a particular OS and, in some versions of virtualization, the characteristics of a particular hardware platform. A VM is also referred to as a *system virtual machine*, emphasizing that it is the hardware of the system that is being virtualized.

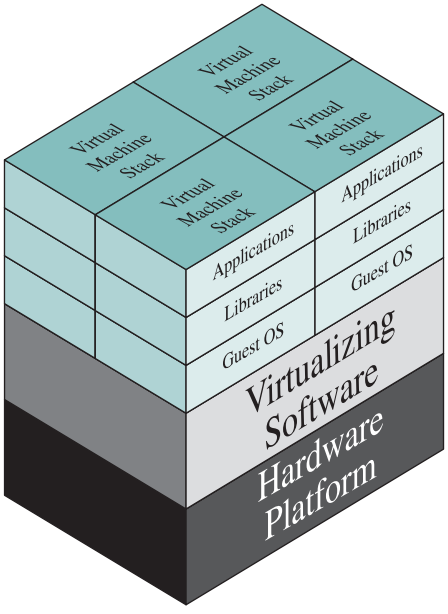



**Figure 14.1** **Virtual Machine Concept**

Virtualization is not a new technology. During the 1970s, IBM mainframe systems offered the first capabilities that would allow programs to use only a portion of a system's resources. Various forms of that ability have been available on platforms since that time. Virtualization came into mainstream computing in the early 2000s when the technology was commercially available on x86 servers. Organizations were suffering from a surfeit of servers due to a Microsoft Windows-driven "one application, one server" strategy. Moore's Law drove rapid hardware improvements outpacing software's ability, and most of these servers were vastly underutilized, often consuming less than 5% of the available resources in each server. In addition, this overabundance of servers filled datacenters and consumed vast amounts of power and cooling, straining a corporation's ability to manage and maintain their infrastructure. Virtualization helped relieve this stress.

The solution that enables virtualization is a **virtual machine monitor (VMM)**, or commonly known today as a **hypervisor**. This software sits between the hardware and the VMs acting as a resource broker. Simply put, it allows multiple VMs to safely coexist on a single physical server host and share that host's resources. Figure 14.1 illustrates this type of virtualization in general terms. On top of the hardware platform sits some sort of virtualizing software, which may consist of the host OS plus specialized virtualizing software or a simply a software package that includes host OS functions and virtualizing functions, as explained subsequently. The virtualizing software provides abstraction of all physical resources (such as processor, memory, network, and storage) and thus enables multiple computing stacks, called virtual machines, to be run on a single physical host.

Each VM includes an OS, called the guest OS. This OS may be the same as the host OS or a different one. For example, a guest Windows OS could be run in a VM

on top of a Linux host OS. The guest OS, in turn, supports a set of standard library functions and other binary files and applications. From the point of view of the applications and the user, this stack appears as an actual machine, with hardware and an OS; thus, the term *virtual machine* is appropriate. In other words, it is the hardware that is being virtualized.

The number of guests that can exist on a single host is measured a **consolidation ratio**. For example, a host that is supporting 4 VMs is said to have a consolidation ratio of 4 to 1, also written as 4:1 (see Figure 14.1). The initial commercially available hypervisors provided consolidation ratios of between 4:1 and 12:1, but even at the low end, if a company virtualized all of their servers, they could remove 75% of the servers from their datacenters. More importantly, they could remove the cost as well, which often ran into the millions or tens of millions of dollars annually. With fewer physical servers, less power and less cooling was needed. Also this leads to fewer cables, fewer network switches, and less floor space. Server consolidation became, and continues to be, a tremendously valuable way to solve a costly and wasteful problem. Today, more virtual servers are deployed in the world than physical servers, and virtual server deployment continues to accelerate.

We can summarize the key reasons the organizations use virtualization as follows:

- **Legacy hardware**: Applications built for legacy hardware can still be run by virtualizing (emulating) the legacy hardware, enabling the retirement of the old hardware.
- **Rapid deployment**: As discussed subsequently, whereas it may take weeks or longer to deploy new servers in an infrastructure, a new VM may be deployed in a matter of minutes. As explained subsequently, a VM consists of files. By duplicating those files, in a virtual environment there is a perfect copy of the server available.
- **Versatility**: Hardware usage can be optimized by maximizing the number of kinds of applications that a single computer can handle.
- **Consolidation**: A large-capacity or high-speed resource, such a server can be used more efficiently by sharing the resource among multiple applications simultaneously.
- **Aggregating**: Virtualization makes it easy to combine multiple resources in to one virtual resource, such as in the case of storage virtualization.
- **Dynamics**: With the use of virtual machines, hardware resources can be easily allocated in a dynamic fashion. This enhances load balancing and fault tolerance.
- **Ease of management**: Virtual machines facilitate deployment and testing of software.
- **Increased availability**: Virtual machine hosts are clustered together to form pools of compute resources. Multiple VMs are hosted on each of these servers and, in the case of a physical server failure, the VMs on the failed host can be quickly and automatically restarted on another host in the cluster. Compared with providing this type of availability for a physical server, virtual environments can provide higher availability at significantly less cost and with less complexity.

Commercial VM offerings by companies such as VMware and Microsoft are widely used on servers, with millions of copies having been sold. A key aspect of server virtualization is, in addition to the capability of running multiple VMs on one machine, VMs can be viewed as network resources. Server virtualization masks server resources, including the number and identity of individual physical servers, processors, and OSs, from server users. This makes it possible to partition a single host into multiple independent servers, conserving hardware resources. It also makes it possible to quickly migrate a server from one machine to another for load balancing, or for dynamic switchover in the case of machine failure. Server virtualization has become a central element in dealing with "big data" applications and in implementing cloud computing infrastructures.

In addition to their use in server environments, these VM technologies also are used in desktop environments to run multiple OSs, typically Windows and Linux.

## 14.2 HYPERVISORS

There is no definitive classification of the various approaches that have been taken to the development of virtual machines. Various methods of classification are discussed in [UHLI05], [PEAR13], [RPSE04], [ROSE05], [NAND05], and [GOLD11]. This section examines the concept of a hypervisor, which is the most common basis for classifying virtual machine approaches.

### Hypervisors

Virtualization is a form of abstraction. Much like an OS abstracts the disk I/O commands from a user through the use of program layers and interfaces, virtualization abstracts the physical hardware from the virtual machines it supports. The virtual machine monitor or hypervisor is the software that provides this abstraction. It acts as a broker, or traffic cop, acting as a proxy for the guests (VMs) as they request and consume resources of the physical host.

A virtual machine is a software construct that mimics the characteristics of a physical server. It is configured with some number of processors, some amount of RAM, storage resources, and connectivity through the network ports. Once that VM is created, it can be powered on like a physical server, loaded with an OS and software solutions, and utilized in the manner of a physical server. Unlike a physical server, this virtual server only sees the resources it has been configured with, not all of the resources of the physical host itself. This isolation allows a host machine to run many virtual machines, each of them running the same or different copies of an OS, sharing RAM, storage and network bandwidth, without problems. An OS in a virtual machine accesses the resource that is presented to it by the hypervisor. The hypervisor facilitates the translation and I/O from the virtual machine to the physical server devices, and back again to the correct virtual machine. In this way, certain privileged instructions that a "native" OS would be executing on its hosts hardware are trapped and run by the hypervisor as a proxy for the virtual machine. This creates some performance degradation in the virtualization process, though over time both hardware and software improvements have minimalized this overhead.

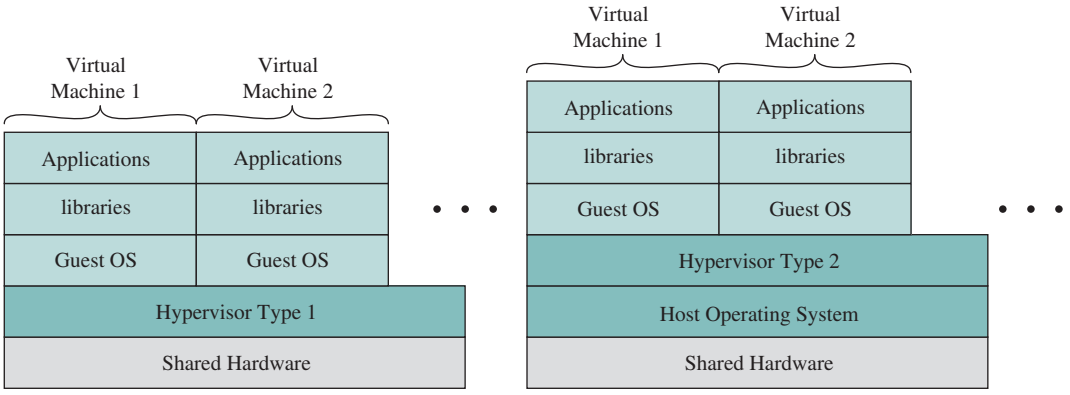
A VM instance is defined in files. A typical virtual machine can consist of just a few files. There is a configuration file that describes the attributes of the virtual machine. It contains the server definition, how many virtual processors (vCPUs) are allocated to this virtual machine, how much RAM is allocated, to which I/O devices the VM has access, how many network interface cards (NICs) are in the virtual server, and more. It also describes the storage that the VM can access. Often that storage is presented as virtual disks that exist as additional files in the physical file system. When a virtual machine is powered on, or instantiated, additional files are created for logging, for memory paging, and other functions. Because a VM essentially consists of files, certain functions in a virtual environment can be defined simpler and quicker than in a physical environment. Since the earliest days of computers, backing up data has been a critical function. Since VMs are already files, copying them produces not only a backup of the data but also a copy of the entire server, including the OS, applications, and the hardware configuration itself.

A common method to rapidly deploy new VMs is through the use of templates. A template provides a standardized group of hardware and software settings that can be used to create new VMs configured with those settings. Creating a new VM from a template consists of providing unique identifiers for the new VM, and having the provisioning software build a VM from the template and adding in the configuration changes as part of the deployment.

***Hypervisor Functions*** The principal functions performed by a hypervisor are the following:

- **Execution management of VMs**: Includes scheduling VMs for execution, virtual memory management to ensure VM isolation from other VMs, context switching between various processor states. Also includes isolation of VMs to prevent conflicts in resource usage and emulation of timer and interrupt mechanisms.
- **Devices emulation and access control**: Emulating all network and storage (block) devices that different native drivers in VMs are expecting, mediating access to physical devices by different VMs.
- **Execution of privileged operations by hypervisor for guest VMs**: Certain operations invoked by guest OSs, instead of being executed directly by the host hardware, may have to be executed on its behalf by the hypervisor, because of their privileged nature.
- **Management of VMs (also called VM lifecycle management)**: Configuring guest VMs and controlling VM states (e.g., Start, Pause, and Stop).
- **Administration of hypervisor platform and hypervisor software**: Involves setting of parameters for user interactions with the hypervisor host as well as hypervisor software.

***Type 1 Hypervisor*** There are two types of hypervisors, distinguished by whether there is an OS between the hypervisor and the host. A type 1 hypervisor (see Figure 14.2a) is loaded as a software layer directly onto a physical server, much like an OS is loaded. The type 1 hypervisor can directly control the physical resources of




(a) Type 1 Hypervisor

(b) Type 2 Hypervisor

**Figure 14.2 Type 1 and Type 2 Hypervisors**

the host. Once it is installed and configured, the server is then capable of supporting virtual machines as guests. In mature environments, where virtualization hosts are clustered together for increased availability and load balancing, a hypervisor can be staged on a new host. Then, that new host is joined to an existing cluster, and VMs can be moved to the new host without any interruption of service. Some examples of type 1 hypervisors are VMware ESXi, Microsoft Hyper-V, and the various Xen variants.

***TYPE 2 HYPERVISOR*** A type 2 hypervisor exploits the resources and functions of a host OS and runs as a software module on top of the OS (see Figure 14.2b). It relies on the OS to handle all of the hardware interactions on the hypervisor's behalf. Some examples of type 2 hypervisors are VMware Workstation and Oracle VM Virtual Box.

Key differences between the two hypervisor types are as follows:

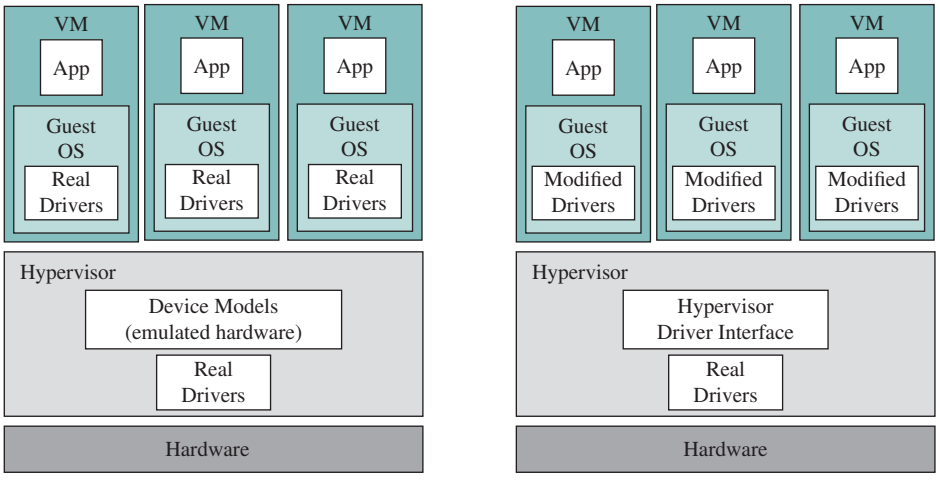
- Typically, type 1 hypervisors perform better than type 2 hypervisors. Because a type 1 hypervisor doesn't compete for resources with an OS, there are more resources available on the host, and by extension, more virtual machines can be hosted on a virtualization server using a type 1 hypervisor.
- Type 1 hypervisors are also considered to be more secure than the type 2 hypervisors. Virtual machines on a type 1 hypervisor make resource requests that are handled external to that guest, and they cannot affect other VMs or the hypervisor they are supported by. This is not necessarily true for VMs on a type 2 hypervisor, and a malicious guest could potentially affect more than itself.
- Type 2 hypervisors allow a user to take advantage of virtualization without needing to dedicate a server to only that function. Developers who need to run multiple environments as part of their process, in addition to taking advantage of the personal productive workspace that a PC OS provides, can do both with a type 2 hypervisor installed as an application on their LINUX or Windows desktop. The virtual machines that are created and used can be migrated or

copied from one hypervisor environment to another, reducing deployment time and increasing the accuracy of what is deployed, reducing the time to market of a project.

## Paravirtualization

As virtualization became more prevalent in corporations, both hardware and software vendors looked for ways to provide even more efficiencies. Unsurprisingly, these paths led to both software-assisted virtualization and hardware-assisted virtualization. **Paravirtualization** is a software-assisted virtualization technique that uses specialized APIs to link virtual machines with the hypervisor to optimize their performance. The OS in the virtual machine, Linux or Microsoft Windows, has specialized paravirtualization support as part of the kernel, as well as specific paravirtualization drivers that allow the OS and hypervisor to work together more efficiently with the overhead of the hypervisor translations. This software-assisted offers optimized virtualization support on servers with or without processors that provide virtualization extensions. Paravirtualization support has been offered as part of many of the general Linux distributions since 2008.

Although the details of this approach differ among the various offerings, a general description is as follows (see Figure 14.3). Without paravirtualization, the guest OS can run without modification if the hypervisor emulates the hardware. In this case, calls from the guest OS drivers to the hardware are intercepted by the hypervisor, which does any necessary translation for native hardware and redirects the call to real driver. With paravirtualization, the source code of an OS is modified to run as a guest OS in a specific virtual machine environment. Calls to the hardware are replaced to calls to the hypervisor, which is able to accept these calls and redirect them without




(a) Type 1 Hypervisor

(b) Paravirtualized Type 1 Hypervisor with Paravirtualized Guest OSs

**Figure 14.3 Paravirtualization**

modification to the real drivers. This arrangement is faster with less overhead than a non-paravirtualized configuration.

### Hardware-Assisted Virtualization

Similarly, processor manufacturers AMD and Intel added functionality to their processors to enhance performance with hypervisors. AMD-V and Intel's VT-x designate the hardware-assisted virtualization extensions that the hypervisors can take advantage of during processing. Intel processors offer an extra instruction set called Virtual Machine Extensions (VMX). By having some of these instructions as part of the processor, the hypervisors no longer need to maintain these functions as part of their codebase, the code itself can be smaller and more efficient, and the operations they support are much faster as they occur entirely on the processor. This hardware-assisted support does not require a modified guest OS in contrast with paravirtualization.

### Virtual Appliance

A virtual appliance is standalone software that can be distributed as a virtual machine image. Thus, it consists of a packaged set of applications and guest OS. It is independent of hypervisor or processor architecture, and can run on either a type 1 or type 2 hypervisor.

Deploying a pre-installed and pre-configured application appliance is far easier than preparing a system, installing the app, and configuring and setting it up. Virtual appliances are becoming a de-facto means of software distribution and have spawned a new type of business—the virtual appliance vendor.

In addition to many useful application-oriented virtual appliances, a relatively recent and important development is the security virtual appliance (SVA). The SVA is a security tool that performs the function of monitoring and protecting the other VMs (User VMs), and is run outside of those VMs in a specially security-hardened VM. The SVA obtains its visibility into the state of a VM (including processor state, registers, and state of memory and I/O devices) as well as the network traffic between VMs, and between VMs and the hypervisor, through the *virtual machine introspection* API of the hypervisor. NIST SP 800-125 (*Security Recommendations for Hypervisor Deployment*, October 2014) points out the advantages of this solution. Specifically, the SVA is:

- Not vulnerable to a flaw in the Guest OS
- Independent of the virtual network configuration and does not have to be reconfigured every time the virtual network configuration changes due to migration of VMs or change in connectivity among VMs resident on the hypervisor host.

## 14.3 CONTAINER VIRTUALIZATION

A relatively recent approach to virtualization is known as **container virtualization**. In this approach, software, known as a **virtualization container**, runs on top of the host OS kernel and provides an isolated execution environment for applications. Unlike

hypervisor-based VMs, containers do not aim to emulate physical servers. Instead, all containerized applications on a host share a common OS kernel. This eliminates the resources needed to run a separate OS for each application and can greatly reduce overhead.

## Kernel Control Groups

Much of the technology for containers as used today was developed for Linux and Linux-based containers are by far the most widely used. Before turning to a discussion of containers, it is useful to introduce the concept of Linux kernel control group. In 2007 [MENA07], the standard Linux process API was extended to incorporate the containerization of user environment so as to allow grouping of multiple processes, user security permission and system resource management. Initially referred to as *process containers*, in late 2007, the nomenclature changed to *control groups* (cgroups) to avoid confusion caused by multiple meanings of the term *container* in the Linux kernel context, and the control groups functionality was merged into the Linux kernel mainline in kernel version 2.6.24, released in January 2008.

Linux process namespace is hierarchical, in which all the processes are child of common boot time process called init. This forms a single process hierarchy. The kernel control group allows multiple process hierarchies to coexist in single OS. Each hierarchy is attached to system resources at configuration time.

Cgroups provide:

- **Resource limiting**: Groups can be set to not exceed a configured memory limit.
- **Prioritization**: Some groups may get a larger share of CPU utilization or disk I/O throughput.
- **Accounting**: This measures a group's resource usage, which may be used, as an example, for billing purposes.
- **Control**: Freezing groups of processes, their checkpointing and restarting.

## Container Concepts

Figure 14.4 compares container and hypervisor software stacks. For containers, only a small container engine is required as support for the containers. The container engine sets up each container as an isolated instance by requesting dedicated resources from the OS for each container. Each container app then directly uses the resources of the host OS. Although the details differ from one container product to another, the following are typical tasks performed by a container engine:

- Maintain a lightweight runtime environment and toolchain that manages containers, images and builds.
- Create a process for the container.
- Manage file system mount points.
- Request resources from kernel, such as memory, I/O devices, and IP addresses.

A typical life cycle of Linux-based containers can be understood through different phases of Linux containers:

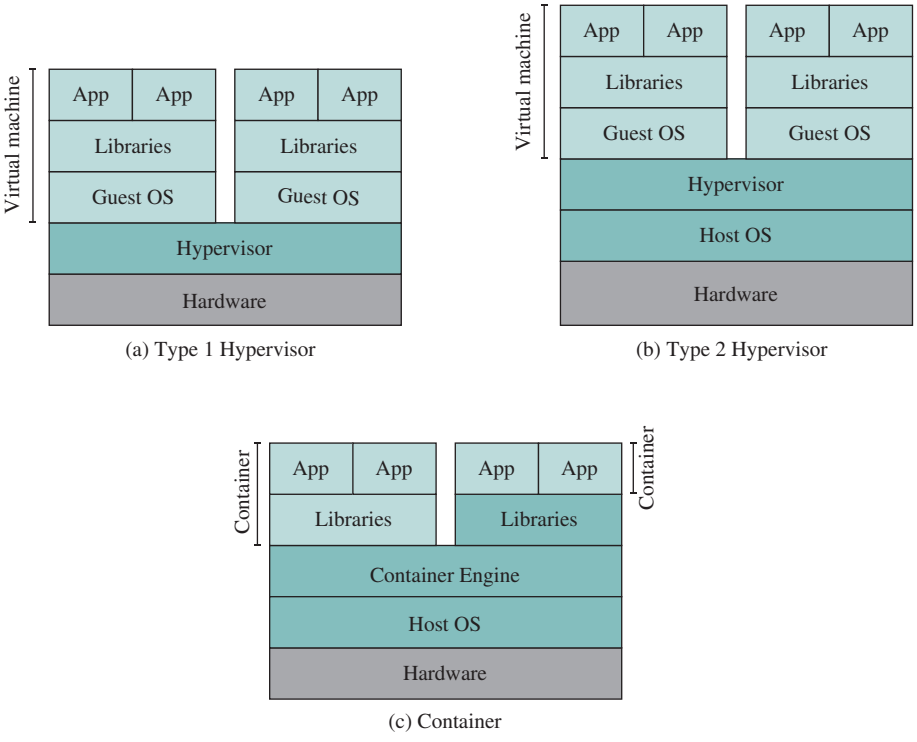



**Figure 14.4 Comparison of Virtual Machines and Containers**

- **Setup:** Setup phase includes the environment to create and start the Linux containers. A typical example of setup phase is Linux kernel enabled with flags or packages installed so as to allow userspace partition. Setup also includes installation of toolchain and utilities (e.g., lxc, bridge utils) to instantiate the container environment and networking configuration into host OS.
- **Configuration:** Containers are configured to run specific applications or commands. Linux container configuration includes networking parameters (e.g., IP address), root file systems, mount operations, and devices that are allowed access through the container environment. In general, containers are configured to allow execution of an application in controlled system resources (such as upper bound on application memory access).
- **Management:** Once a container is set up and configured, it has to be managed so as to allow seamless bootstrap (start up) and shutdown of the container. Typically, managed operations for a container-based environment include start, stop, freeze, and migrate. In addition, there are meta commands and toolchains that allows controlled and managed allocation of containers in a single node for end user access.

Because all the containers on one machine execute on the same kernel, thus sharing most of the base OS, a configuration with containers is much smaller and lighter weight compared to a hypervisor/guest OS virtual machine arrangement. Accordingly, an OS can have many containers running on top of it, compared to the limited number of hypervisors and guest OSs that can be supported.

Virtual containers are feasible due to resource control and process isolations as explained using techniques such as the kernel control group. This approach allows system resources being shared between multiple instances of isolated containers. Cgroups provides a mechanism to manage and monitor the system resources. The application performance is close to native system performance due to single kernel shared between all userspace container instances, and overhead is only to provide mechanism to isolate the containers via cgroups. Linux subsystems partitioned using control group primitives include filesystem, process namespace, network stack, hostname, IPC, and users.

To compare virtual machines with containers, consider I/O operation in during an application with process P in virtualized environment. In classical system virtualization environment (with no hardware support), process P would be executed inside a guest virtual machine. I/O operation is routed though guest OS stack to emulated guest I/O device. I/O call is further intercepted by hypervisor that forward it through host OS stack to the physical device. In comparison, the container is primarily based on indirection mechanism provided by container framework extensions that have been incorporated into main stream kernel. Here, a single kernel is shared between multiple containers (in comparison with individual OS kernel in system virtual machines). Figure 14.5 gives an overview of the dataflow of virtual machines and containers.

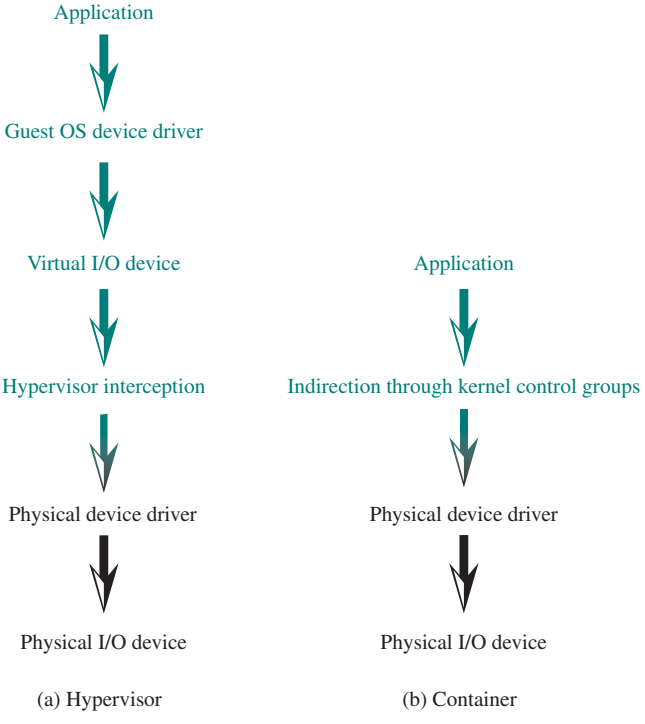



**Figure 14.5** **Data Flow for I/O Operation via Hypervisor and Container**

The two noteworthy characteristics of containers are the following:

1. There is no need for a guest OS in the container environment. Therefore, containers are lightweight and have less overhead compared to virtual machines.
2. Container management software simplifies the procedure for container creation and management.

Because they are light weight, containers are an attractive alternative to virtual machines. An additional attractive feature of containers is that they provide application portability. Containerized applications can be quickly moved from one system to another.

These container benefits do not mean containers are always a preferred alternative to virtual machines, as the following considerations show:

- Container applications are only portable across systems that support the same OS kernel with the same virtualization support features, which typically means Linux. Thus, a containerized Windows application would only run on Windows machines.
- A virtual machine may require a unique kernel setup that is not applicable to other VMs on the host; this requirement is addressed by the use of the guest OS.
- VM virtualization functions at the border of hardware and OS. It's able to provide strong performance isolation and security guarantees with the narrowed interface between VMs and hypervisors. Containerization, which sits in between the OS and applications, incurs lower overhead, but potentially introduces greater security vulnerabilities.

One potential use case, cited in [KERN16], revolves around Kubernetes, an open source container orchestration technology built by Google but now managed by the Cloud Native Computing Foundation (CNCF). The foundation itself operates as a Linux Foundation Collaborative project. As an example, if an administrator dedicates 500 Mbps to a particular application running on Kubernetes, then the networking control plane can be involved in the scheduling of this application to find the best place to guarantee that bandwidth. Or, by working with the Kubernetes API, a network control plane can start making ingress firewall rules that are aware of the container applications.

### Container File System

As part of the isolation of a container, each container must maintain its own isolated file system. The specific features vary from one container product to another, but the essential principals are the same.

As an example, we look at the container file system used in OpenVZ. This is depicted in Figure 14.6. The scheduler init is run to schedule user applications and each container has its own init process, which from the hardware nodes perspective is just another running process.

The multiple containers on a host are most likely running the same processes, but each of them doesn't have an individual copy even though the ls command shows the container's /bin directory is full of programs. Instead the containers share a template, a design feature in which all the apps that come with the OS, and many of the

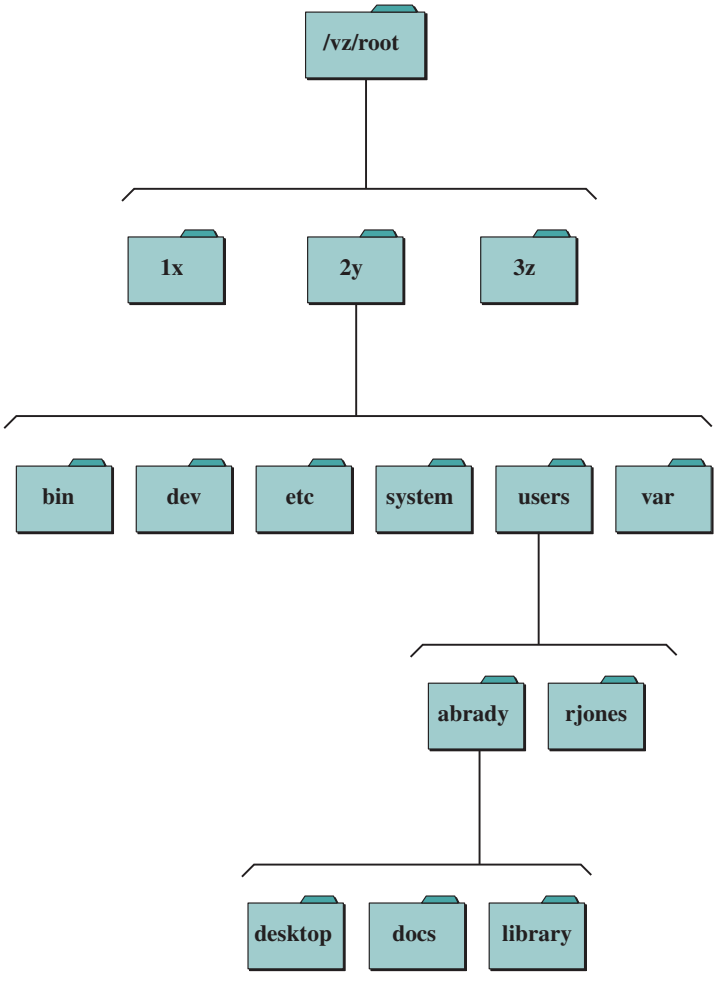



**Figure 14.6** **OpenVZ File scheme**

most common applications, are packaged together as groups of files hosted by the platform's OS and symbolically linked into each container. This includes configuration files as well, unless the container modifies them; when that happens, the OS copies the template file (called copy on write), removes the virtual sym link and puts the modified file in the container's file system. By using this virtual file sharing scheme, a considerable space saving is achieved, with only locally created files actually existing in the container's file system.

At a disk level, a container is a file, and can easily be scaled up or down. From a virus checking point of view, the container's file system is mounted under a special mount point on the hardware node so system tools at the hardware node level can safely and securely check every file if needed.

## Microservices

A concept related to containers is that of microservice. NIST SP 800-180 (*NIST Definition of Microservices, Application Containers and System Virtual Machines*, February 2016) defines a microservice as a basic element that results from the architectural decomposition of an application's components into loosely coupled patterns consisting of self-contained services that communicate with each other using a standard communications protocol 219 and a set of well-defined APIs, independent of any vendor, product, or technology.

The basic idea behind microservices is, instead of having a monolithic application stack, each specific service in an application delivery chain is broken out into individual parts. When using containers, people are making a conscious effort to break their infrastructure down into more understandable units. This opens an opportunity for networking technologies to make decisions on behalf of the user that they couldn't make before in a machine-focused world.

Two key advantages of microservices are the following:

- Microservices implement much smaller deployable units, which then enables the user to push out updates or do features and capabilities much more quickly. This coincides with continuous delivery practices, where the goal is to push out small units without having to create a monolithic system.
- Microservices also support precise scalability. Because a microservice is section of a much larger application, it can easily be replicated to create multiple instances, and spread the load for just that one small piece of the application instead of having to do so for the entire application.

## Docker

Historically, containers emerged as a way of running applications in a more flexible and agile way. Linux containers enabled running lightweight applications, within Linux OS directly. Without a need for the hypervisor and virtual machines, applications can run in isolation in the same operating system. Google has been using Linux containers in its data centers since 2006. But the container approach became more popular with the arrival of Docker containers in 2013. Docker provides a simpler and more standardized way to run containers compared to earlier version of containers. The Docker container also runs in Linux. But Docker is not the only way to run containers. Linux Containers (LXC) is another way to run containers. Both LXC and Docker have roots in Linux. One of the reasons the Docker container is more popular compared to competing containers such as LXC is its ability to load a container image on a host operating system in a simple and quick manner. Docker containers are stored in the cloud as images and called upon for execution by users when needed in a simple way.

Docker consists of the following principal components:

- **Docker image:** Docker images are read-only templates from which Docker containers are instantiated.
- **Docker client:** A Docker client request that an image be used to create a new container. The client can be on the same platform as a Docker host or a Docker machine.

- **Docker host:** A platform with it own host OS that executes containerized applications.
- **Docker engine:** This is the lightweight runtime package that builds and runs the Docker containers on a host system.
- **Docker machine:** The Docker machine can run on a separate system from the Docker hosts, used to set up Docker engines. The Docker machine installs the Docker engine on a host and configures the Docker client to talk to the Docker engine. The Docker machine can also be used locally to set up a Docker image on the same host as is running Docker machine.
- **Docker registry:** A Docker registry stores Docker images. After you build a Docker image, you can push it to a public registry such as Docker hub or to a private registry running behind your firewall. You can also search for existing images and pull them from the registry to a host.
- **Docker hub:** This is the collaboration platform, a public repository of Docker container images. Users can use images stored in a hub that are contributed by others and contribute their own custom images.

## 14.4 PROCESSOR ISSUES

In a virtual environment, there are two main strategies for providing processor resources. The first is to emulate a chip as software and provide access to that resource. Examples of this method are QEMU and the Android Emulator in the Android SDK. They have the benefit of being easily transportable since they are not platform dependent, but they are not very efficient from a performance standpoint, as the emulation process is resource intensive. The second model doesn't actually virtualize processors but provides segments of processing time on the physical processors (pCPUs) of the virtualization host to the virtual processors of the virtual machines hosted on the physical server. This is how most of the virtualization hypervisors offer processor resources to their guests. When the operating system in a virtual machine passes instructions to the processor, the hypervisor intercepts the request. It then schedules time on the host's physical processors, sends the request for execution, and returns the results to the VM's operating system. This ensures the most efficient use of the available processor resources on the physical server. To add some complexity, when multiple VMs are contending for processor, the hypervisor acts as the traffic controller, scheduling processor time for each VM's request as well as directing the requests and data to and from the virtual machines.

Along with memory, the number of processors a server has is one of the more important metrics when sizing a server. This is especially true, and in some way more critical, in a virtual environment than a physical one. In a physical server, typically the application has exclusive use of all the compute resources configured in the system. For example, in a server with four quad-core processors, the application can utilize sixteen cores of processor. Usually, the application's requirements are far less than that. This is because the physical server has been sized for some possible future state of the application that includes growth over three to five years and also incorporates some degree of high-water performance spikes. In reality, from a processor

standpoint, most servers are vastly underutilized, which is a strong driver for consolidation through virtualization as discussed earlier.

When applications are migrated to virtual environments, one of the larger topics of discussion is how many virtual processors should be allocated to their virtual machines. Since the physical server they are vacating had sixteen cores, often the request from the application team is to duplicate that in the virtual environment, regardless of what their actual usage was. In addition to ignoring the usage on the physical server, another overlooked item is the improved capabilities of the processors on the newer virtualization server. If the application was migrated at the low end of when its server's life/lease ended, it would be three to five years. Even at three years, Moore's law provides processors that would be four times faster than those on the original physical server. In order to help "right-size" the virtual machine configurations, there are tools available that will monitor resource (processor, memory, network, and storage I/O) usage on the physical servers then make recommendations for the optimum VM sizing. If that consolidation estimate utility cannot be run, there are a number of good practices in place. One basic rule during VM creation is to begin with one vCPU and monitor the application's performance. Adding additional vCPUs in a VM is simple, requiring an adjustment in the VM settings. Most modern operating systems do not even require a reboot before being able to recognize and utilize the additional vCPU. Another good practice is not to overallocate the number of vCPUs in a VM. A matching number of pCPUs need to be scheduled for the vCPUs in a VM. If you have four vCPUs in your VM, the hypervisor needs to simultaneously schedule four pCPUs on the virtualization host on behalf of the VM. On a very busy virtualization host, having too many vCPUs configured for a VM can actually negatively impact the performance of the VM's application since it is faster to schedule a single pCPU. This doesn't mean there are not applications that require multiple vCPUs. There are, and they should be configured appropriately, but most do not.

Native operating systems manage hardware by acting as the intermediary between application code requests and the hardware. As requests for data or processing are made, the operating system passes these to the correct device drivers, through the physical controllers, to the storage or I/O devices, and back again. The operating system is the central router of information and controls access to all of the physical resources of the hardware. One key function of the operating system is to help prevent malicious or accidental system calls from disrupting the applications or the operating system itself. Protection rings describe level of access or privilege inside of a computer system, and many operating systems and processor architectures take advantage of this security model. The most trusted layer is often called Ring 0 (zero) and is where the operating system kernel works and can interact directly with hardware. Rings 1 and 2 are where device drivers execute while user applications run in the least trusted area, Ring 3. In practice, though, Rings 1 and 2 are not often used, simplifying the model to trusted and untrusted execution spaces. Application code cannot directly interact with hardware since it runs in Ring 3 and needs the operating system to execute the code on its behalf in Ring 0. This separation prevents unprivileged code from causing untrusted actions such as a system shutdown or an unauthorized access of data from a disk or network connection.

Hypervisors run in Ring 0 controlling hardware access for the virtual machines they host. The operating systems in those virtual machines also believe they run

in Ring 0, and in a way they do, but only on the virtual hardware that is created as part of the virtual machine. In the case of a system shutdown, the operating system on the guest would request a shutdown command in Ring 0. The hypervisor intercepts the request; otherwise, the physical server would be shutdown, causing havoc for the hypervisor and any other virtual machines being hosted. Instead, the hypervisor replies to the guest operating system that the shutdown is proceeding as requested, which allows the guest operating system to complete the necessary software shutdown processes.

## 14.5 MEMORY MANAGEMENT

Like the number of vCPUs, the amount of memory allocated to a virtual machine is one of the more crucial configuration choices; in fact, memory resources are usually the first bottleneck that virtual infrastructures reach as they grow. Also, like the virtualization of processors, memory usage in virtual environments is more about the management of the physical resource rather than the creation of a virtual entity. As with a physical server, a virtual machine needs to be configured with enough memory to function efficiently by providing space for the operating system and applications. Again, the virtual machine is configured with fewer resources than the physical host contains. A simple example would be a physical server with 8GB of RAM. A virtual machine provisioned with 1GB of memory would only see 1GB of memory, even though the physical server on which it is hosted has more. When the virtual machine uses memory resources, the hypervisor manages the memory requests through the use of translation tables so the guest (VM) operating system addresses the memory space at the addresses that they expect. This is a good first step, but problems remain. Similar to processor, application owners ask for memory allocations that mirror the physical infrastructures they migrated from, regardless of whether the size of the allocation is warranted or not. This leads to overprovisioned virtual machines and wasted memory resources. In the case of our 8GB server, only seven 1GB VMs could be hosted, with the remaining 1GB needed for the hypervisor itself. Aside from "right-sizing" the virtual machines based on their actual performance characteristics, there are features built into hypervisors that help optimize memory usage. One of these is **page sharing** (see Figure 14.7). Page sharing is similar to data de-duplication, a storage technique that reduces the number of storage blocks being used. When a VM is instantiated, operating system and application pages are loaded into memory. If multiple VMs are loading the same version of the OS, or running the same applications, many of these memory blocks are duplicates. The hypervisor is already managing the virtual to physical memory transfers and can determine if a page is already loaded into memory. Rather than loading a duplicate page into physical memory, the hypervisor provides a link to the shared page in the virtual machine's translation table. On hosts where the guests are running the same operating system and the same applications, between 10 and 40% of the actual physical memory can be reclaimed. At 25%, an 8-GB server could host two additional 1-GB virtual machines.

Since the hypervisor manages page sharing, the virtual machine operating systems are unaware of what is happening in the physical system. Another strategy for efficient memory use is akin to thin provisioning in storage management. This allows

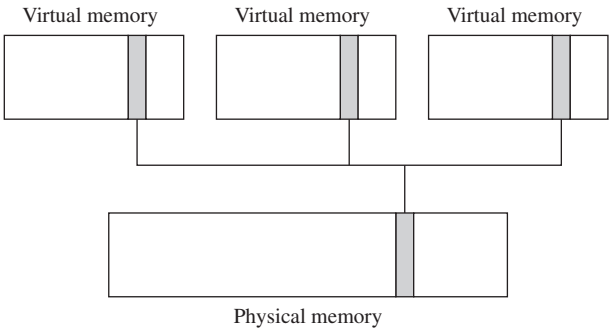



**Figure 14.7 Page Sharing**

an administrator to allocate more storage to a user than is actually present in the system. The reason is to provide a high water mark that often is never approached. The same can be done with virtual machine memory. We allocate 1GB of memory but that is what is seen by the VM operating system. The hypervisor can use some portion of that allocated memory for another VM by reclaiming older pages that are not being used. The reclamation process is done through **ballooning**. The hypervisor activates a balloon driver that (virtually) inflates and presses the guest operating system to flush pages to disk. Once the pages are cleared, the balloon driver deflates and the hypervisor can use the physical memory for other VMs. This process happens during times of memory contention. If our 1GB VMs used half of their memory on average, nine VMs would require only 4.5GB with the remainder as a shared pool managed by the hypervisor and some for the hypervisor overhead. Even if we host an additional three 1GB VMs, there is still a shared reserve. This capability to allocate more memory than physically exists on a host is called **memory overcommit**. It is not uncommon for virtualized environments to have between 1.2 and 1.5 times the memory allocated, and in extreme cases, many times more.

There are additional memory management techniques that provide better resource utilization. In all cases, the operating systems in the virtual machines see and have access to the amount of memory that has been allocated to them. The hypervisor manages that access to the physical memory to Ensure vs. Insure all requests are serviced in a timely manner without impacting the virtual machines. In cases where more physical memory is required than is available, the hypervisor will be forced to resort to paging to disk. In multiple host cluster environments, virtual machines can be automatically live migrated to other hosts when certain resources become scarce.

## 14.6 I/O MANAGEMENT

Application performance is often directly linked to the bandwidth that a server has been allocated. Whether it is storage access that has been bottlenecked, or constrained traffic to the network, either case will cause an application to be perceived as underperforming. In this way, during the virtualization of workloads, I/O virtualization is a critical item. The architecture of how I/O is managed in a virtual environment

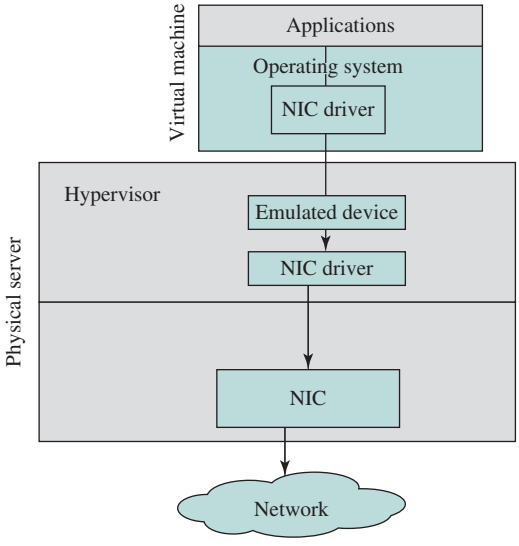



**Figure 14.8 I/O in a Virtual Environment**

is straightforward (see Figure 14.8). In the virtual machine, the operating system makes a call to the device driver as it would in a physical server. The device driver then connects with the device; though in the case of the virtual server, the device is an emulated device that is staged and managed by the hypervisor. These emulated devices are usually a common actual device, such as an Intel e1000 network interface card or simple generic SGVA or IDE controllers. This virtual device plugs into the hypervisor's I/O stack that communicates with the device driver that is mapped to a physical device in the host server, translating guest I/O addresses to the physical host I/O addresses. The hypervisor controls and monitors the requests from the virtual machine's device driver, through the I/O stack, out the physical device, and back again, routing the I/O calls to the correct devices on the correct virtual machines. There are some architectural differences between vendors, but the basic model is similar.

The advantages of virtualizing the workload's I/O path are many. It enables hardware independence by abstracting vendor-specific drivers to more generalized versions that run on the hypervisor. A virtual machine running on an IBM server as a host can be live migrated to an HP blade server host without worrying about hardware incompatibilities or versioning mismatches. This abstraction enables of one of virtualization's greatest availability strengths: live migration. Sharing of aggregate resources, network paths, for example, is also due to this abstraction. In more mature solutions, capabilities exist to granularly control the types of network traffic and the bandwidth afforded to individual VMs or groups of virtual machines to insure adequate performance in a shared environment to guarantee a chosen Quality of Service level. In addition to these, there are other features that enhance security and availability. The trade-off is that the hypervisor is managing all the traffic, for which it is designed, but it requires processor overhead. In the early days of virtualization this was an issue that could be a limiting factor, but faster multicore processors and sophisticated hypervisors have all but removed this concern.

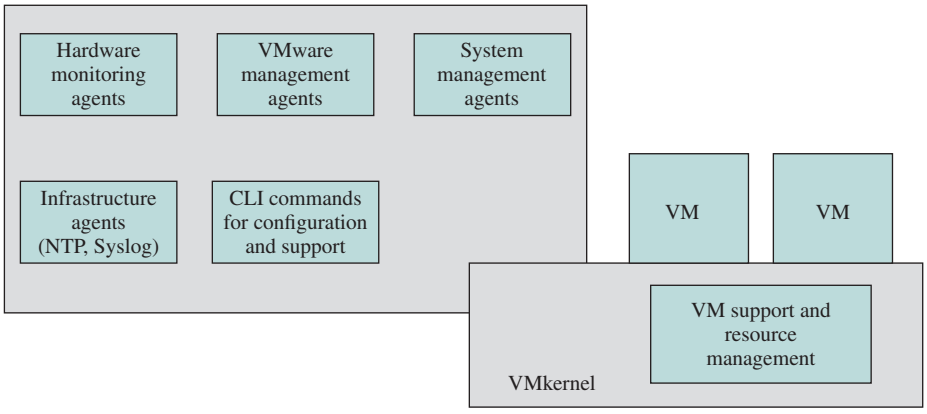
A faster processor enables the hypervisor to perform its I/O management functions more quickly, and also speeds the rate at which the guest processor processing is done. Explicit hardware changes for virtualization support also improve performance. Intel offers I/O Acceleration Technology (I/OAT), a physical subsystem that moves memory copies via direct memory access (DMA) from the main processor to this specialized portion of the motherboard. Though designed for improving network performance, remote DMA also improves live migration speeds. Offloading work from the processor to intelligent devices is another path to improved performance. Intelligent network interface cards support a number of technologies in this space. TCP Offload Engine (TOE) removes the TCP/IP processing from the server processor entirely to the NIC. Other variations on this theme are Large Receive Offload (LRO), which aggregates incoming packets into bundles for more efficient processing, and its inverse Large Segment Offload (LSO), which allows the hypervisor to aggregate multiple outgoing TCP/IP packets and has the NIC hardware segment them into separate packets.

In addition to the model described earlier, some applications or users will demand a dedicated path. In this case, there are options to bypass the hypervisor's I/O stack and oversight, and directly connect from the virtual machine's device driver to physical device on the virtualization host. This provides the virtue of having a dedicated resource without any overhead delivering the greatest throughput possible. In addition to better throughput, since the hypervisor is minimally involved, there is less impact on the host server's processor. The disadvantage to a directly connected I/O device is that the virtual machine is tied to the physical server it is running on. Without the device abstraction, live migration is not easily possible, which can potentially reduce availability. Features provided by the hypervisor, like memory overcommit or I/O control, are not available, which could waste underutilized resources and mitigate the need for virtualization. Though a dedicated device model provides better performance, today it is rarely used, as datacenters opt for the flexibility that virtualized I/O provides.

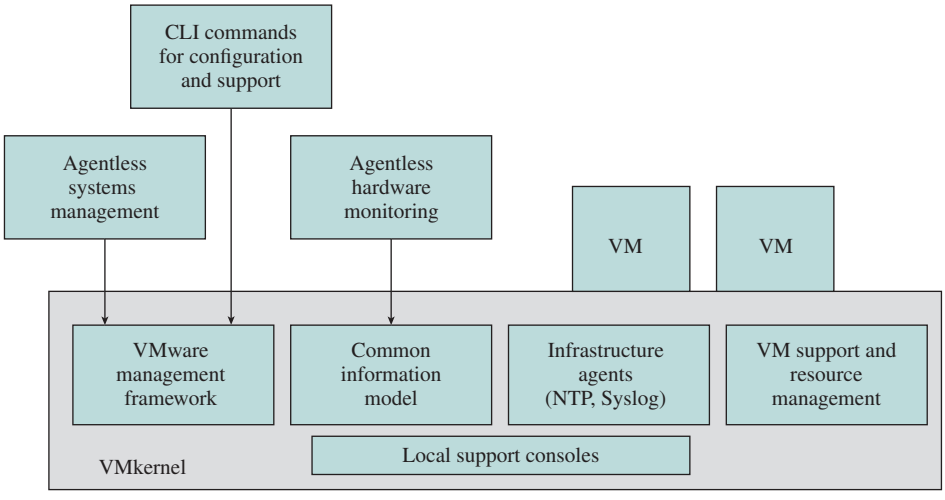
## 14.7 VMware ESXi

ESXi is a commercially available hypervisor from VMware that provides users a Type 1, or bare-metal, hypervisor to host virtual machines on their servers. VMware developed their initial x86-based solutions in the late 1990s and were the first to deliver a commercial product to the marketplace. This first-to-market timing, coupled with continuous innovations, has kept VMware firmly on top of the heap in market share, but more importantly, in the lead from a breadth of feature and maturity of solution standpoint. The growth of the virtualization market and the changes in the VMware solutions have been outlined elsewhere, but there are certain fundamental differences in the ESXi architecture compared to the other available solutions.

The virtualization kernel (VMkernel) is the core of the hypervisor and performs all of the virtualization functions. In earlier releases of ESX (see Figure 14.9a), the hypervisor was deployed alongside a Linux installation that served as a management layer. Certain management functions like logging, name services, and often




(a) ESX




(b) ESXi

**Figure 14.9 ESX and ESXi**

third-party agents for backup or hardware monitoring were installed on this service console. It also made a great place for administrators to run other scripts and programs. The service console had two issues. The first was that it was considerably larger than the hypervisor; a typical install required about 32MB for the hypervisor and about 900MB for the service console. The second was that the Linux-based service console was a well-understood interface and system, and was vulnerable to attack by malware or people. VMware then re-architected ESX to be installed and managed without the service console.

This new architecture, dubbed ESXi (the "i" for integrated) has all of the management services as part of the VMkernel (see Figure 14.9b). This provides a smaller and much more secure package than before. Current versions are in the neighborhood

of about 100MB. This small size allows server vendors to deliver hardware with ESXi already available on flash memory in the server. Configuration management, monitoring, and scripting are now all available through command line interface utilities. Third-party agents are also run in the VMkernel after being certified and digitally signed. This allows, for example, a server vendor who provides hardware monitoring, to include an agent in the VMkernel that can seamlessly return hardware metrics such as internal temperature or component statuses to either VMware management tools or other management tools.

Virtual machines are hosted via the infrastructure services in the VMkernel. When resources are requested by the virtual machines, the hypervisor fulfills those requests, working through the appropriate device drivers. As described earlier, the hypervisor coordinates all of the transactions between the multiple virtual machines and the hardware resources on the physical server.

Though the examples discussed so far are very basic, VMware ESXi provides advanced and sophisticated features for availability, scalability, security, manageability, and performance. Additional capabilities are introduced with each release, improving the capabilities of the platform. Some examples are as follows:

- **Storage VMotion:** Permits the relocation of the data files that compose a virtual machine, while that virtual machine is in use.
- **Fault Tolerance:** Creates a lockstep copy of a virtual machine on a different host. If the original host suffers a failure, the virtual machine's connections get shifted to the copy, without interrupting users or the application they are using. This differs from High Availability, which would require a virtual machine restart on another server.
- **Site Recovery Manager:** Uses various replication technologies to copy selected virtual machines to a secondary site in the case of a data center disaster. The secondary site can be stood up in a matter of minutes; virtual machines power-on in a selected and tiered manner automatically to insure a smooth and accurate transition.
- **Storage and Network I/O Control:** Allows an administrator to allocate network bandwidth in a virtual network in a very granular manner. These policies are activated when there is contention on the network and can guarantee that specific virtual machines, groups of virtual machines that comprise a particular application, or classes of data or storage traffic have the required priority and bandwidth to operate as desired.
- **Distributed Resource Scheduler (DRS):** Intelligently places virtual machines on hosts for startup and can automatically balance the workloads via VMotion based on business policies and resource usage. An aspect of this, Distributed Power Management (DPM), can power-off (and on) physical hosts as they are needed. Storage DRS can actively migrate virtual machine files based on storage capacity and I/O latency, again based on the business rules and resource utilization.

These are just a few of the features that extend VMware's ESXi solution past being merely a hypervisor that can support virtual machines, into a platform for the new data center and the foundation for cloud computing.

## 14.8 MICROSOFT HYPER-V AND XEN VARIANTS

In the early 2000s, an effort based in Cambridge University led to the development of the Xen, an open-source hypervisor. Over time, and as the need for virtualization increased, many hypervisor variants have come out of the main Xen branch. Today, in addition to the open-source hypervisor, there are a number of Xen-based commercial hypervisor offerings from Citrix, Oracle, and others. Architected differently than the VMware model, Xen requires a dedicated operating system or domain to work with the hypervisor, similar to the VMware service console (see Figure 14.10). This initial domain is known as domain zero (Dom0), runs the Xen tool stack, and as the privileged area, has direct access to the hardware. Many versions of Linux contain a Xen hypervisor that is capable of creating a virtual environment. Some of these are CentOS, Debian, Fedora, Ubuntu, OracleVM, Red Hat (RHEL), SUSE, and XenServer. Companies that use Xen-based virtualization solutions do so due to the lower (or no) cost of the software, or due to their own in-house Linux expertise.

Guests on Xen are unprivileged domains, or sometimes user domains, referred to as DomU. Dom0 provides access to network and storage resources to the guests via BackEnd drivers that communicate with the FrontEnd drivers in DomU. Unless there are pass-through devices configured (usually USB), all of the network and storage I/O is handled through Dom0. Since Dom0 is itself an instance of Linux, if something unexpected happens to it, all of the virtual machines it supports will be affected. Standard operating system maintenance like patching also can potentially affect the overall availability.

Like most open-source offerings, Xen does not contain many of the advanced capabilities offered by VMware ESXi, though with each release, additional features appear and existing features are enhanced.

Microsoft has had a number of virtualization technologies, including Virtual Server, a Type 2 hypervisor offering that was acquired in 2005 and is still available today at no cost. Microsoft Hyper-V, a Type 1 hypervisor, was first released in 2008 as part of the Windows Server 2008 Operating System release. Similar to the Xen architecture, Hyper-V has a parent partition that serves as an administrative adjunct to the Type 1 hypervisor (see Figure 14.11). Guest virtual machines are designated as child partitions. The parent partition runs the Windows Server operating system in addition to its functions, such as managing the hypervisor, the guest partitions, and

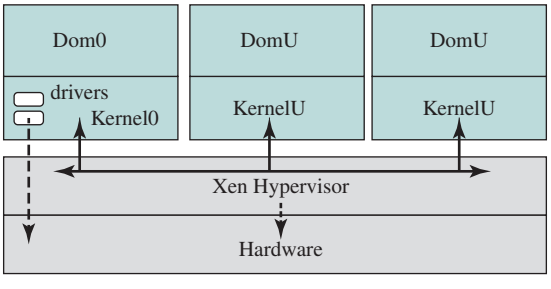



**Figure 14.10 Xen**

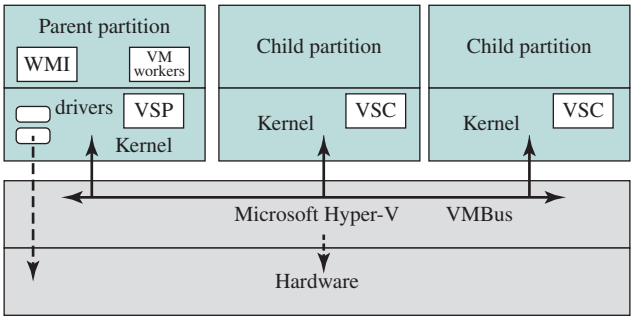



**Figure 14.11 Hyper-V**

the devices drivers. Similar to the FrontEnd and BackEnd drivers in Xen, the parent partition in Hyper-V uses a Virtualization Service Provider (VSP) to provide device services to the child partitions. The child partitions communicate with the VSPs using a Virtualization Service Client (or Consumer) (VSC) for their I/O needs.

Microsoft Hyper-V has similar availability challenges to Xen due to the operating system needs in the parent partition, the resource contention an extra copy of Windows requires on the server, and the single I/O conduit. From a feature standpoint, Hyper-V is very robust, though not as widely used as ESXi since it is still relatively new to the marketplace. As time passes and new functionality appears, adoption will probably increase.

## 14.9 JAVA VM

Though the Java Virtual Machine (JVM) has the term *virtual machine* as part of its name, its implementation and uses are different from the models we have covered. Hypervisors support one or more virtual machines on a host. These virtual machines are self-contained workloads, each supporting an operating system and applications, and from their perspective, have access to a set of hardware devices that provide compute, storage, and I/O resources. The goal of a Java Virtual Machine is to provide a runtime space for a set of Java code to run on any operating system staged on any hardware platform, without needing to make code changes to accommodate the different operating systems or hardware. Both models are aimed at being platform independent through the use of some degree of abstraction.

The JVM is described as being an abstract computing machine, consisting of an **instruction set**, a pc (program counter) **register**, a **stack** to hold variables and results, a **heap** for runtime data and garbage collection, and a **method** area for code and constants. The JVM can support multiple threads and each thread has its own register and stack areas, though the heap and method areas are shared among all of the threads. When the JVM is instantiated, the runtime environment is started, the memory structures are allocated and populated with the selected method (code) and variables, and the program begins. The code run in the JVM is interpreted in real time from the Java language into the appropriate binary code. If that code is valid, and adheres to the expected standards, it will begin processing. If it is invalid,

and the process fails, an error condition is raised and returned to the JVM and the user.

Java and JVMs are used in a very wide variety of areas including Web applications, mobile devices, and smart devices from television set-top boxes to gaming devices to Blu-ray players and other items that use smart cards. Java's promise of "Write Once, Run Anywhere" provides an agile and simple deployment model, allowing applications to be developed independent of the execution platform.

## 14.10 LINUX VSERVER VIRTUAL MACHINE ARCHITECTURE

Linux VServer is an open-source, fast, virtualized container approach to implementing virtual machines on a Linux server [SOLT07, LIGN05]. Only a single copy of the Linux kernel is involved. VServer consists of a relatively modest modification to the kernel plus a small set of OS userland[1] tools. The VServer Linux kernel supports a number of separate *virtual servers*. The kernel manages all system resources and tasks, including process scheduling, memory, disk space, and processor time.

### Architecture

Each virtual server is isolated from the others using Linux kernel capabilities. This provides security and makes it easy to set up multiple virtual machines on a single platform. The isolation involves four elements: chroot, chcontext, chbind, and capabilities.

The **chroot** command is a UNIX or Linux command to make the root directory (/) become something other than its default for the lifetime of the current process. It can only be run by privileged users and is used to give a process (commonly a network server such as FTP or HTTP) access to a restricted portion of the file system. This command provides **file system isolation**. All commands executed by the virtual server can only affect files that start with the defined root for that server.

The **chcontext** Linux utility allocates a new security context and executes commands in that context. The usual or *hosted* security context is the context 0. This context has the same privileges as the root user (UID 0): This context can see and kill other tasks in the other contexts. Context number 1 is used to view other contexts but cannot affect them. All other contexts provide complete isolation: Processes from one context can neither see nor interact with processes from another context. This provides the ability to run similar contexts on the same computer without any interaction possible at the application level. Thus, each virtual server has its own execution context that provides **process isolation**.

The **chbind** utility executes a command and locks the resulting process and its children into using a specific IP address. Once called, all packets sent out by this virtual server through the system's network interface are assigned the sending IP address derived from the argument given to chbind. This system call provides **network**

[1] The term *userland* refers to all application software that runs in user space rather than kernel space. *OS userland* usually refers to the various programs and libraries the operating system uses to interact with the kernel: software that performs input/output, manipulates file system objects, etc.

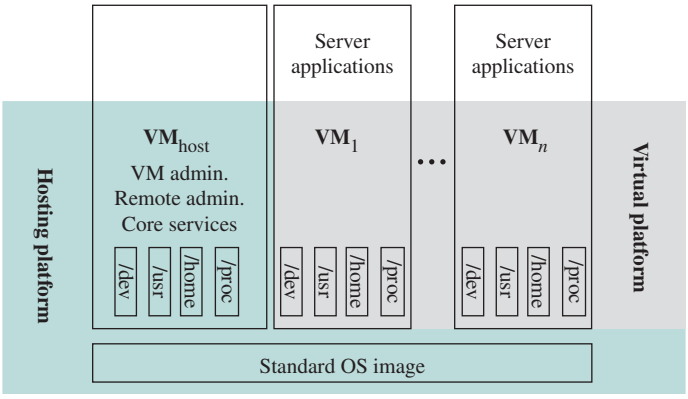



**Figure 14.12 Linux VServer Architecture**

**isolation**: Each virtual server uses a separate and distinct IP address. Incoming traffic intended for one virtual server cannot be accessed by other virtual servers.

Finally, each virtual server is assigned a set of **capabilities**. The concept of capabilities, as used in Linux, refers to a partitioning of the privileges available to a root user, such as the ability to read files or to trace processes owned by another user. Thus, each virtual server can be assigned a limited subset of the root user's privileges. This provides **root isolation**. VServer can also set resource limits, such as limits to the amount of virtual memory a process may use.

Figure 14.12 shows the general architecture of Linux VServer. VServer provides a shared, virtualized OS image, consisting of a root file system, and a shared set of system libraries and kernel services. Each VM can be booted, shut down, and rebooted independently. Figure 14.12 shows three groupings of software running on the computer system. The **hosting platform** includes the shared OS image and a privileged host VM, whose function is to monitor and manage the other VMs. The **virtual platform** creates virtual machines and is the view of the system seen by the **applications** running on the individual VMs.

## Process Scheduling

The Linux VServer virtual machine facility provides a way of controlling VM use of processor time. VServer overlays a token bucket filter (TBF) on top of the standard Linux schedule. The purpose of the TBF is to determine how much of the processor execution time (single processor, multiprocessor, or multicore) is allocated to each VM. If only the underlying Linux scheduler is used to globally schedule processes across all VMs, then resource hunger processes in one VM crowd out processes in other VMs.

Figure 14.13 illustrates the TBF concept. For each VM, a bucket is defined with a capacity of $S$ tokens. Tokens are added to the bucket at a rate of $R$ tokens during every time interval of length $T$. When the bucket is full, additional incoming tokens are simply discarded. When a process is executing on this VM, it consumes one token for each timer clock tick. If the bucket empties, the process is put in a hold and cannot be restarted until the bucket is refilled to a minimum threshold value of $M$ tokens. At that point, the process is rescheduled. A significant consequence of the TBF approach

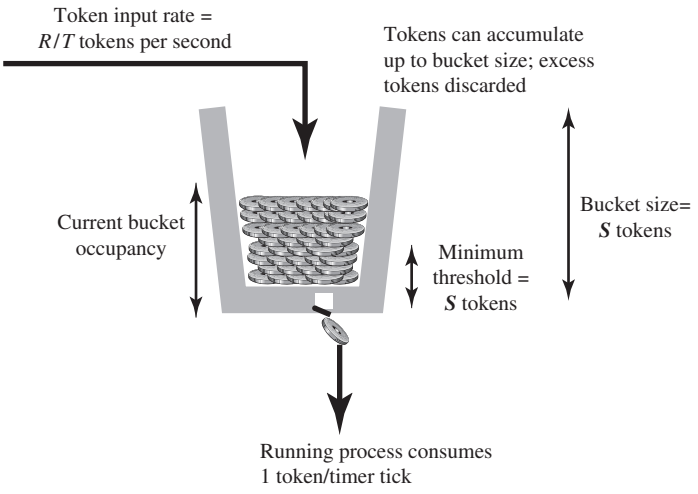



**Figure 14.13 Linux VServer Token Bucket Scheme**

is that a VM may accumulate tokens during a period of quiescence, then later use the tokens in a burst when required.

Adjusting the values of $R$ and $T$ allows for regulating the percentage of capacity that a VM can claim. For a single processor, we can define capacity allocation as follows:

$$\frac{R}{T} = \text{fraction of processor allocation}$$

This equation denotes the fraction of a single processor in a system. Thus, for example, if a system is multicore with four cores and we wish to provide one VM an average of one dedicated processor, then we set $R = 1$ and $T = 4$. The overall system is limited as follows. If there are $N$ VMs, then:

$$\sum_{i=1}^{N} \frac{R_i}{T_i} \leq 1$$

The parameters $S$ and $M$ are set so as to penalize a VM after a certain amount of burst time. The following parameters must be configured or allocated for a VM: Following a burst time of $B$, the VM suffers a hold time of $H$. With these parameters, it is possible to calculate the desired values of $S$ and $M$ as follows:

$$M = W \times H \times \frac{R}{T}$$

$$S = W \times B \times \left(1 - \frac{R}{T}\right)$$

where $W$ is the rate at which the schedule runs (makes decisions). For example, consider a VM with a limit of 1/2 of processor time, and we wish to say that after using the processor for 30 seconds, there will be a hold time of 5 seconds. The scheduler runs at 1000 Hz. This requirement is met with the following values: $M = 1{,}000 \times 5 \times 0.5 = 2500$ tokens; $S = 1000 \times 30 \times (1 - 0.5) = 15{,}000$ tokens.

## 14.11 SUMMARY

Virtualization technology enables a single PC or server to simultaneously run multiple operating systems or multiple sessions of a single OS. In essence, the host operating system can support a number of virtual machines (VM), each of which has the characteristics of a particular OS and, in some versions of virtualization, the characteristics of a particular hardware platform.

A common virtual machine technology makes use of a virtual machine monitor (VMM), or hypervisor, which is at a lower level than the VM and supports VMs. There are two types of hypervisors, distinguished by whether there is another operating system between the hypervisor and the host. A Type 1 hypervisor executes directly on the machine hardware, and a Type 2 hypervisor operates on top of the host operating system.

A very different approach to implementing a VM environment is exemplified by the Java VM. The goal of a Java VM is to provide a runtime space for a set of Java code to run on any operating system staged on any hardware platform, without needing to make code changes to accommodate the different operating systems or hardware.

## 14.12 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

### Key Terms

| | | |
|---|---|---|
| container | hypervisor | type-1 hypervisor |
| container virtualization | Java Virtual Machine (JVM) | type-2 hypervisor |
| consolidation ratio | kernel control group | virtual appliance |
| Docker | memory ballooning | virtualization container |
| guest OS | memory overcommit | virtualization |
| hardware virtualization | microservice | virtual machine (VM) |
| hardware-assisted virtualization | page sharing | virtual machine monitor (VMM) |
| host OS | paravirtualization | |

### Review Questions

**14.1.** Briefly describe Type 1 and Type 2 virtualization.
**14.2.** Briefly describe container virtualization.
**14.3.** Explain the concept of ballooning.
**14.4.** Give a brief description of Java VM.

### Problems

**14.1.** Techniques like memory overcommit and page sharing permit virtual machines to be allocated more resources than are physically in a single virtualization host. Does this allow the aggregate of the virtual machines to perform more real work than a physical workload would be capable of on the same hardware?

**14.2.** Type 1 hypervisors operate directly on physical hardware without any intervening operating system. Type 2 hypervisors run as an application installed on an existing operating system. Type 1 hypervisors perform much better than Type 2 hypervisors since there is no intervening layer to negotiate between themselves and the system hardware, nor do they need to contend for resources with another controlling layer of software. Why then are Type 2 hypervisors widely used? What are some of the use cases?

**14.3.** When virtualization first appeared in the x86 marketplace, many server vendors were skeptical of the technology and were concerned that consolidation would impact the sales of servers. Instead, server vendors found that they were selling larger, costlier servers. Why did this happen?

**14.4.** Providing additional bandwidth for virtualization servers initially involved additional network interface cards (NICs) for more network connections. With the advent of increasingly greater network backbone bandwidths (10Gbit/s, 40Gbit/s, and 100Gbit/s), fewer NICs are necessary. What issues might result from these converged network connections and how might they be resolved?

**14.5.** Virtual machines are presented with storage in manners similar to physical machines via TCP/IP, Fibre-Channel, or iSCSI connections. There are features in virtualization that optimize memory and processor usage, and advanced features that can provide more efficient use of I/O resources. What do you think might be available to provide better use of storage resources in a virtualized environment?

CHAPTER 15

# Operating System Security


15.1 **Intruders and Malicious Software**
- System Access Threats
- Countermeasures

15.2 **Buffer Overflow**
- Buffer Overflow Attacks
- Compile-Time Defenses
- Runtime Defenses

15.3 **Access Control**
- File System Access Control
- Access Control Policies

15.4 **Unix Access Control**
- Traditional UNIX File Access Control
- Access Control Lists in UNIX

15.5 **Operating Systems Hardening**
- Operating System Installation: Initial Setup and Patching
- Remove Unnecessary Services, Application, and Protocols
- Configure Users, Groups, and Authentication
- Configure Resource Controls
- Install Additional Security Controls
- Test the System Security

15.6 **Security Maintenance**
- Logging
- Data Backup and Archive

15.7 **Windows Security**
- Access Control Scheme
- Access Token
- Security Descriptors

15.8 **Summary**

15.9 **Key Terms, Review Questions, and Problems**

**LEARNING OBJECTIVES**

After studying this chapter, you should be able to:

- Assess the key security issues that relate to operating systems.
- Understand the design issues for file system security.
- Distinguish among various types of intruder behavior patterns and understand the types of intrusion techniques used to breach computer security.
- Compare and contrast two methods of access control.
- Understand how to defend against buffer overflow attacks.

## 15.1 INTRUDERS AND MALICIOUS SOFTWARE

An OS associates a set of privileges with each process. These privileges dictate what resources the process may access, including regions of memory, files, and privileged system instructions. Typically, a process that executes on behalf of a user has the privileges that the OS recognizes for that user. A system or utility process may have privileges assigned at configuration time.

On a typical system, the highest level of privilege is referred to as administrator, supervisor, or root access.[1] Root access provides access to all the functions and services of the operating system. With root access, a process has complete control of the system and can add or change programs and files, monitor other processes, send and receive network traffic, and alter privileges.

A key security issue in the design of any OS is to prevent, or at least detect, attempts by a user or a piece of malicious software (malware) from gaining unauthorized privileges on the system and, in particular, from gaining root access. In this section, we briefly summarize the threats and countermeasures related to this security issue. Subsequent sections will examine some of the issues raised in this section in more detail.

### System Access Threats

System access threats fall into two general categories: intruders and malicious software.

***INTRUDERS*** One of the most common threats to security is an intruder (the other is viruses), often referred to as a hacker or cracker. In an important early study of intrusion, Anderson [ANDE80] identifies three classes of intruders:

- **Masquerader:** An individual who is not authorized to use the computer and who penetrates a system's access controls to exploit a legitimate user's account

[1] On UNIX systems, the administrator, or *superuser*, account is called root; hence the term *root access*.

- **Misfeasor:** A legitimate user who accesses data, programs, or resources for which such access is not authorized, or who is authorized for such access but misuses his or her privileges
- **Clandestine user:** An individual who seizes supervisory control of the system and uses this control to evade auditing and access controls or to suppress audit collection

The masquerader is likely to be an outsider; the misfeasor generally is an insider; and the clandestine user can be either an outsider or an insider.

Intruder attacks range from the benign to the serious. At the benign end of the scale, there are many people who simply wish to explore the Internet and other networks and see what is out there. At the serious end are individuals who are attempting to read privileged data, perform unauthorized modifications to data, or disrupt the system.

The objective of the intruder is to gain access to a system or to increase the range of privileges accessible on a system. Most initial attacks use system or software vulnerabilities that allow a user to execute code that opens a backdoor into the system. Intruders can get access to a system by exploiting attacks such as buffer overflows on a program that runs with certain privileges. We will introduce buffer overflow attacks in Section 15.2.

Alternatively, the intruder attempts to acquire information that should have been protected. In some cases, this information is in the form of a user password. With knowledge of some other user's password, an intruder can log in to a system and exercise all the privileges accorded to the legitimate user.

***MALICIOUS SOFTWARE*** Perhaps the most sophisticated types of threats to computer systems are presented by programs that exploit vulnerabilities in computing systems. Such threats are referred to as **malicious software**, or **malware**. In this context, we are concerned with threats to application programs as well as utility programs, such as editors and compilers, and kernel-level programs.

Malicious software can be divided into two categories: those that need a host program, and those that are independent. The former, referred to as **parasitic**, are essentially fragments of programs that cannot exist independently of some actual application program, utility, or system program. Viruses, logic bombs, and backdoors are examples. The latter are self-contained programs that can be scheduled and run by the operating system. Worms and bot programs are examples.

We can also differentiate between those software threats that do not replicate and those that do. The former are programs or fragments of programs that are activated by a trigger. Examples are logic bombs, backdoors, and bot programs. The latter consists of either a program fragment or an independent program that, when executed, may produce one or more copies of itself to be activated later on the same system or some other system. Viruses and worms are examples.

Malicious software can be relatively harmless, or may perform one or more of a number of harmful actions, including destroying files and data in main memory, bypassing controls to gain privileged access, and providing a means for intruders to bypass access controls.

## Countermeasures

***Intrusion Detection*** RFC 4949 (*Internet Security Glossary*) defines intrusion detection as follows: A security service that monitors and analyzes system events for the purpose of finding, and providing real-time or near real-time warning of, attempts to access system resources in an unauthorized manner.

Intrusion detection systems (IDSs) can be classified as follows:

- **Host-based IDS:** Monitors the characteristics of a single host and the events occurring within that host for suspicious activity
- **Network-based IDS:** Monitors network traffic for particular network segments or devices and analyzes network, transport, and application protocols to identify suspicious activity

An IDS comprises three logical components:

- **Sensors:** Sensors are responsible for collecting data. The input for a sensor may be any part of a system that could contain evidence of an intrusion. Types of input to a sensor include network packets, log files, and system call traces. Sensors collect and forward this information to the analyzer.
- **Analyzers:** Analyzers receive input from one or more sensors or from other analyzers. The analyzer is responsible for determining if an intrusion has occurred. The output of this component is an indication that an intrusion has occurred. The output may include evidence supporting the conclusion that an intrusion occurred. The analyzer may provide guidance about what actions to take as a result of the intrusion.
- **User interface:** The user interface to an IDS enables a user to view output from the system or control the behavior of the system. In some systems, the user interface may equate to a manager, director, or console component.

Intrusion detection systems are typically designed to detect human intruder behavior as well as malicious software behavior.

***Authentication*** In most computer security contexts, user authentication is the fundamental building block and the primary line of defense. User authentication is the basis for most types of access control and for user accountability. RFC 4949 defines user authentication as follows:

The process of verifying an identity claimed by or for a system entity. An authentication process consists of two steps:

1. **Identification step:** Presenting an identifier to the security system (Identifiers should be assigned carefully, because authenticated identities are the basis for other security services, such as access control service.)
2. **Verification step:** Presenting or generating authentication information that corroborates the binding between the entity and the identifier

For example, user Alice Toklas could have the user identifier ABTOKLAS. This information needs to be stored on any server or computer system that Alice wishes to use, and could be known to system administrators and other users. A typical item

of authentication information associated with this user ID is a password, which is kept secret (known only to Alice and to the system). If no one is able to obtain or guess Alice's password, then the combination of Alice's user ID and password enables administrators to set up Alice's access permissions and audit her activity. Because Alice's ID is not secret, system users can send her e-mail, but because her password is secret, no one can pretend to be Alice.

In essence, identification is the means by which a user provides a claimed identity to the system; user authentication is the means of establishing the validity of the claim.

There are four general means of authenticating a user's identity, which can be used alone or in combination:

1. **Something the individual knows:** Examples include a password, a personal identification number (PIN), or answers to a prearranged set of questions.
2. **Something the individual possesses:** Examples include electronic keycards, smart cards, and physical keys. This type of authenticator is referred to as a *token*.
3. **Something the individual is (static biometrics):** Examples include recognition by fingerprint, retina, and face.
4. **Something the individual does (dynamic biometrics):** Examples include recognition by voice pattern, handwriting characteristics, and typing rhythm.

All of these methods, properly implemented and used, can provide secure user authentication. However, each method has problems. An adversary may be able to guess or steal a password. Similarly, an adversary may be able to forge or steal a token. A user may forget a password or lose a token. Further, there is a significant administrative overhead for managing password and token information on systems and securing such information on systems. With respect to biometric authenticators, there are a variety of problems, including dealing with false positives and false negatives, user acceptance, cost, and convenience.

***Access Control*** Access control implements a security policy that specifies who or what (e.g., in the case of a process) may have access to each specific system resource and the type of access that is permitted in each instance.

An access control mechanism mediates between a user (or a process executing on behalf of a user) and system resources, such as applications, operating systems, firewalls, routers, files, and databases. The system must first authenticate a user seeking access. Typically, the authentication function determines whether the user is permitted to access the system at all. Then the access control function determines if the specific requested access by this user is permitted. A security administrator maintains an authorization database that specifies what type of access to which resources is allowed for this user. The access control function consults this database to determine whether to grant access. An auditing function monitors and keeps a record of user accesses to system resources.

***Firewalls*** Firewalls can be an effective means of protecting a local system or network of systems from network-based security threats while affording access to

the outside world via wide area networks and the Internet. Traditionally, a firewall is a dedicated computer that interfaces with computers outside a network and has special security precautions built into it in order to protect sensitive files on computers within the network. It is used to service outside network, especially Internet connections and dial-in lines. Personal firewalls implemented in hardware or software, and associated with a single workstation or PC, are also common.

[BELL94] lists the following design goals for a firewall:

1. All traffic from inside to outside, and vice versa, must pass through the firewall. This is achieved by physically blocking all access to the local network except via the firewall.
2. Only authorized traffic, as defined by the local security policy, will be allowed to pass. Various types of firewalls are used, which implement various types of security policies.
3. The firewall itself is immune to penetration. This implies the use of a hardened system with a secured operating system. Trusted computer systems are suitable for hosting a firewall and often required in government applications.

## 15.2 BUFFER OVERFLOW

Main memory and virtual memory are system resources subject to security threats and for which security countermeasures need to be taken. The most obvious security requirement is the prevention of unauthorized access to the memory contents of processes. If a process has not declared a portion of its memory to be sharable, then no other process should have access to the contents of that portion of memory. If a process declares a portion of memory may be shared by other designated processes, then the security service of the OS must ensure that only the designated processes have access. The security threats and countermeasures discussed in the preceding section are relevant to this type of memory protection.

In this section, we summarize another threat, which involves memory protection.

### Buffer Overflow Attacks

**Buffer overflow**, also known as a **buffer overrun**, is defined in the *NIST* (National Institute of Standards and Technology) *Glossary of Key Information Security Terms* as follows:

> **Buffer overflow:** A condition at an interface under which more input can be placed into a buffer or data-holding area than the capacity allocated, overwriting other information. Attackers exploit such a condition to crash a system or to insert specially crafted code that allows them to gain control of the system.

A buffer overflow can occur as a result of a programming error when a process attempts to store data beyond the limits of a fixed-sized buffer and consequently

overwrites adjacent memory locations. These locations could hold other program variables or parameters or program control flow data such as return addresses and pointers to previous stack frames. The buffer could be located on the stack, in the heap, or in the data section of the process. The consequences of this error include corruption of data used by the program, unexpected transfer of control in the program, possibly memory access violations, and very likely eventual program termination. When done deliberately as part of an attack on a system, the transfer of control could be to any code of the attacker's choosing, resulting in the ability to execute arbitrary code with the privileges of the attacked process. Buffer overflow attacks are one of the most prevalent and dangerous types of security attacks.

To illustrate the basic operation of a common type of buffer overflow, known as **stack overflow**, consider the C main function given in Figure 15.1a. This contains three variables (`valid, str1,` and `str2`),[2] whose values will typically be saved in adjacent memory locations. Their order and location depends on the type of variable (local or global), the language and compiler used, and the target machine architecture. For this example, we assume they are saved in consecutive memory locations, from highest to lowest, as shown in Figure 15.2.[3] This is typically the case for local variables in a C function on common processor architectures such as the Intel Pentium family. The purpose of the code fragment is to call the function `next_tag(str1)`

```
int main(int argc, char *argv[]) {
    int valid = FALSE;
    char str1[8];
    char str2[8];
    next_tag(str1);
    gets(str2);
    if (strncmp(str1, str2, 8) == 0)
       valid = TRUE;
    printf("buffer1: str1(%s), str2(%s), valid(%d) \n", str1, str2, valid);
}
```

**(a) Basic buffer overflow C code**

```
$ cc -g -o buffer1 buffer1.c
$ ./buffer1
START
buffer1: str1(START), str2(START), valid(1)
$ ./buffer1
EVILINPUTVALUE
buffer1: str1(TVALUE), str2(EVILINPUTVALUE), valid(0)
$ ./buffer1
BADINPUTBADINPUT
buffer1: str1(BADINPUT), str2(BADINPUTBADINPUT), valid(1)
```

**(b) Basic buffer overflow example runs**

**Figure 15.1 Basic Buffer Overflow Example**

[2]In this example, the flag variable is saved as an integer rather than a Boolean. This is done because it is the classic C style and to avoid issues of word alignment in its storage. The buffers are deliberately small to accentuate the buffer overflow issue being illustrated.

[3]Address and data values are specified in hexadecimal in this and related figures. Data values are also shown in ASCII where appropriate.

| Memory Address | Before gets(str2) | After gets(str2) | Contains Value of |
|---|---|---|---|
| . . . . | . . . . | . . . . | |
| bffffbf4 | 34fcffbf<br>4 . . . | 34fcffbf<br>3 . . . | argv |
| bffffbf0 | 01000000<br>. . . . | 01000000<br>. . . . | argc |
| bffffbec | c6bd0340<br>. . . @ | c6bd0340<br>. . . @ | return addr |
| bffffbe8 | 08fcffbf<br>. . . . | 08fcffbf<br>. . . . | old base ptr |
| bffffbe4 | 00000000<br>. . . . | 01000000<br>. . . . | valid |
| bffffbe0 | 80640140<br>. d . @ | 00640140<br>. d . @ | |
| bffffbdc | 54001540<br>T . . @ | 4e505554<br>N P U T | str1[4-7] |
| bffffbd8 | 53544152<br>S T A R | 42414449<br>B A D I | str1[0-3] |
| bffffbd4 | 00850408<br>. . . . | 4e505554<br>N P U T | str2[4-7] |
| bffffbd0 | 30561540<br>0 V . @ | 42414449<br>B A D I | str2[0-3] |
| . . . . | . . . . | . . . . | |

**Figure 15.2 Basic Buffer Overflow Stack Values**

to copy into `str1` some expected tag value. Let's assume this will be the string `START`. It then reads the next line from the standard input for the program using the C library `gets()` function, then compares the string read with the expected tag. If the next line did indeed contain just the string `START`, this comparison would succeed and the variable `valid` would be set to `TRUE`.[4] This case is shown in the first of the three example program runs in Figure 15.1b. Any other input tag would leave it with the value `FALSE`. Such a code fragment might be used to parse some structured network protocol interaction or formatted text file.

The problem with this code exists because the traditional C library `gets()` function does not include any checking on the amount of data copied. It reads the next line of text from the program's standard input up until the first newline[5]

[4]In C, the logical values FALSE and TRUE are simply integers with the values 0 and 1 (or indeed any non-zero value), respectively. Symbolic defines are often used to map these symbolic names to their underlying value, as was done in this program.

[5]The newline (NL) or linefeed (LF) character is the standard end of line terminator for UNIX systems, and hence for C, and is the character with the ASCII value 0x0a.

character occurs and copies it into the supplied buffer followed by the NULL terminator used with C strings.[6] If more than seven characters are present on the input line, when read in they will (along with the terminating NULL character) require more room than is available in the `str2` buffer. Consequently, the extra characters will overwrite the values of the adjacent variable, `str1` in this case. For example, if the input line contained `EVILINPUTVALUE`, the result will be that `str1` will be overwritten with the characters `TVALUE`, and `str2` will use not only the eight characters allocated to it but seven more from `str1` as well. This can be seen in the second example run in Figure 15.1b. The overflow has resulted in corruption of a variable not directly used to save the input. Because these strings are not equal, `valid` also retains the value `FALSE`. Further, if 16 or more characters were input, additional memory locations would be overwritten.

The preceding example illustrates the basic behavior of a buffer overflow. At its simplest, any unchecked copying of data into a buffer could result in corruption of adjacent memory locations, which may be other variables, or possibly program control addresses and data. Even this simple example could be taken further. Knowing the structure of the code processing it, an attacker could arrange for the overwritten value to set the value in `str1` equal to the value placed in `str2`, resulting in the subsequent comparison succeeding. For example, the input line could be the string `BADINPUTBADINPUT`. This results in the comparison succeeding, as shown in the third of the three example program runs in Figure 15.1b and illustrated in Figure 15.2, with the values of the local variables before and after the call to `gets()`. Note also the terminating NULL for the input string was written to the memory location following `str1`. This means the flow of control in the program will continue as if the expected tag was found, when in fact the tag read was something completely different. This will almost certainly result in program behavior that was not intended. How serious this is depends very much on the logic in the attacked program. One dangerous possibility occurs if instead of being a tag, the values in these buffers were an expected and supplied password needed to access privileged features. If so, the buffer overflow provides the attacker with a means of accessing these features without actually knowing the correct password.

To exploit any type of buffer overflow, such as those we have illustrated here, the attacker needs:

1. To identify a buffer overflow vulnerability in some program that can be triggered using externally sourced data under the attackers control
2. To understand how that buffer will be stored in the processes memory, and hence the potential for corrupting adjacent memory locations and potentially altering the flow of execution of the program

Identifying vulnerable programs may be done by inspection of program source, tracing the execution of programs as they process oversized input, or using tools such as *fuzzing*, which involves the use of randomly generated input data, to automatically

[6]Strings in C are stored in an array of characters and terminated with the NULL character, which has the ASCII value 0x00. Any remaining locations in the array are undefined, and typically contain whatever value was previously saved in that area of memory. This can be clearly seen in the value in the variable str2 in the "Before" column of Figure 15.2.

identify potentially vulnerable programs. What the attacker does with the resulting corruption of memory varies considerably, depending on what values are being overwritten.

## Compile-Time Defenses

Finding and exploiting a stack buffer overflow is not that difficult. The large number of exploits over the previous couple of decades clearly illustrates this. There is consequently a need to defend systems against such attacks by either preventing them or at least detecting and aborting such attacks. Countermeasures can be broadly classified into two categories:

1. Compile-time defenses, which aim to harden programs to resist attacks
2. Runtime defenses, which aim to detect and abort attacks in executing programs programs

While suitable defenses have been known for a couple of decades, the very large existing base of vulnerable software and systems hinders their deployment. Hence the interest in runtime defenses, which can be deployed in operating systems and updates and can provide some protection for existing vulnerable programs.

In this subsection, we look at compile-time defenses, then subsequently look at runtime defenses. Compile-time defenses aim to prevent or detect buffer overflows by instrumenting programs when they are compiled. The possibilities for doing this range from choosing a high-level language that does not permit buffer overflows to encouraging safe coding standards, using safe standard libraries, or including additional code to detect corruption of the stack frame.

***Choice of Programming Language*** One possibility is to write the program using a modern high-level programming language, one that has a strong notion of variable type and what constitutes permissible operations on them. Such languages are not vulnerable to buffer overflow attacks, because their compilers include additional code to enforce range checks automatically, thus removing the need for the programmer to explicitly code them. The flexibility and safety provided by these languages does come at a cost in resource use, both at compile time and also in additional code that must execute at runtime to impose checks such as that on buffer limits. These disadvantages are much less significant than they used to be, due to the rapid increase in processor performance. Increasingly programs are being written in these languages and hence should be immune to buffer overflows in their code (though if they use existing system libraries or runtime execution environments written in less safe languages, they may still be vulnerable). The distance from the underlying machine language and architecture also means that access to some instructions and hardware resources is lost. This limits their usefulness in writing code, such as device drivers, that must interact with such resources. For these reasons, there is still likely to be at least some code written in less safe languages such as C.

***Safe Coding Techniques*** If languages such as C are being used, programmers need to be aware that their ability to manipulate pointer addresses and access memory directly comes at a cost. C was designed as a systems programming language,

running on systems that were vastly smaller and more constrained than we now use. This meant that C's designers placed much more emphasis on space efficiency and performance considerations than on type safety. They assumed programmers would exercise due care in writing code using these languages and take responsibility for ensuring the safe use of all data structures and variables.

Unfortunately, as several decades of experience have shown, this has not been the case. This may be seen in large legacy body of potentially unsafe code in the UNIX and Linux operating systems and applications, some of which are potentially vulnerable to buffer overflows.

In order to harden these systems, the programmer needs to inspect the code and rewrite any unsafe coding constructs in a safe manner. Given the rapid uptake of buffer overflow exploits, this process has begun in some cases. A good example is the OpenBSD project, which produces a free, multiplatform 4.4BSD-based UNIX-like operating system. Among other technology changes, programmers have undertaken an extensive audit of the existing code base, including the operating system, standard libraries, and common utilities. This has resulted in what is widely regarded as one of the safest operating systems in widespread use. The OpenBSD project claims as of mid-2006 that there has only been one remote hole discovered in the default install in more than eight years. This is a clearly enviable record. Microsoft has also undertaken a major project in reviewing its code base, partly in response to continuing bad publicity over the number of vulnerabilities, including many buffer overflow issues, that have been found in their operating systems and applications code.

***Language Extensions and Use of Safe Libraries*** Given the problems that can occur in C with unsafe array and pointer references, there have been a number of proposals to augment compilers to automatically insert range checks on such references. While this is fairly easy for statically allocated arrays, handling dynamically allocated memory is more problematic, because the size information is not available at compile time. Handling this requires an extension to the semantics of a pointer to include bounds information and the use of library routines to ensure that these values are set correctly. Several such approaches are listed in [LHEE03]. However, there is generally a performance penalty with the use of such techniques that may or may not be acceptable. These techniques also require all programs and libraries that require these safety features to be recompiled with the modified compiler. While this can be feasible for a new release of an operating system and its associated utilities, there will still likely be problems with third-party applications.

A common concern with C comes from the use of unsafe standard library routines, especially some of the string manipulation routines. One approach to improving the safety of systems has been to replace these with safer variants. This can include the provision of new functions, such as `strlcpy()`, in the BSD family of systems, including OpenBSD. Using these requires rewriting the source to conform to the new safer semantics. Alternatively, it involves replacement of the standard string library with a safer variant. Libsafe is a well-known example of this; it implements the standard semantics but includes additional checks to ensure that the copy operations do not extend beyond the local variable space in the stack frame. So, while Libsafe cannot prevent corruption of adjacent local variables, it can prevent any modification of the old stack frame and return address values, and thus prevent the

classic stack buffer overflow types of attack we examined previously. This library is implemented as a dynamic library, arranged to load before the existing standard libraries, and can thus provide protection for existing programs without requiring them to be recompiled, provided they dynamically access the standard library routines (as most programs do). The modified library code has been found to typically be at least as efficient as the standard libraries, and thus its use is an easy way of protecting existing programs against some forms of buffer overflow attacks.

***Stack Protection Mechanisms*** An effective method for protecting programs against classic stack overflow attacks is to instrument the function entry and exit code to set up and then check its stack frame for any evidence of corruption. If any modification is found, the program is aborted rather than allowing the attack to proceed. There are several approaches to providing this protection, which we discuss next.

Stackguard is one of the best-known protection mechanisms. It is a GCC (GNU Compiler Collection) compiler extension that inserts additional function entry and exit code. The added function entry code writes a **canary**[7] value below the old frame pointer address, before the allocation of space for local variables. The added function exit code checks that the canary value has not changed before continuing with the usual function exit operations of restoring the old frame pointer and transferring control back to the return address. Any attempt at a classic stack buffer overflow would have to alter this value in order to change the old frame pointer and return addresses and would thus be detected, resulting in the program being aborted. For this defense to function successfully, it is critical that the canary value be unpredictable and should be variable on different systems. If this were not the case, the attacker would simply ensure the shellcode included the correct canary value in the required location. Typically, a random value is chosen as the canary value on process creation and saved as part of the processes state. The code added to the function entry and exit then uses this value.

There are some issues with using this approach. First, it requires that all programs needing protection be recompiled. Second, because the structure of the stack frame has changed, it can cause problems with programs, such as debuggers, which analyze stack frames. However, the canary technique has been used to recompile an entire Linux distribution and provide it with a high level of resistance to stack overflow attacks. Similar functionality is available for Windows programs by compiling them using Microsoft's /GS Visual C++ compiler option.

## Runtime Defenses

As has been noted, most of the compile-time approaches require recompilation of existing programs. Hence there is interest in runtime defenses that can be deployed as operating systems updates to provide some protection for existing vulnerable programs. These defenses involve changes to the memory management of the virtual address space of processes. These changes act either to alter the properties of regions of memory or to make predicting the location of targeted buffers sufficiently difficult to thwart many types of attacks.

---

[7]Named after the miner's canary used to detect poisonous air in a mine and thus warn the miners in time for them to escape.

***Executable Address Space Protection*** Many of the buffer overflow attacks involve copying machine code into the targeted buffer then transferring execution to it. A possible defense is to block the execution of code on the stack, on the assumption that executable code should only be found elsewhere in the processes address space.

To support this feature efficiently requires support from the processor's memory management unit (MMU) to tag pages of virtual memory as being nonexecutable. Some processors, such as the SPARC used by Solaris, have had support for this for some time. Enabling its use in Solaris requires a simple kernel parameter change. Other processors, such as the x86 family, have not had this support until recently, with the relatively recent addition of the **no-execute** bit in its MMU. Extensions have been made available to Linux, BSD, and other UNIX-style systems to support the use of this feature. Some indeed are also capable of protecting the heap as well as the stack, which also is the target of attacks. Support for enabling no-execute protection is also included in recent Windows systems.

Making the stack (and heap) nonexecutable provides a high degree of protection against many types of buffer overflow attacks for existing programs; hence the inclusion of this practice is standard in a number of recent operating systems releases. However, one issue is support for programs that do need to place executable code on the stack. This can occur, for example, in just-in-time compilers, such as is used in the Java runtime system. Executable code on the stack is also used to implement nested functions in C (a GCC extension) and also Linux signal handlers. Special provisions are needed to support these requirements. Nonetheless, this is regarded as one of the best methods for protecting existing programs and hardening systems against some attacks.

***Address Space Randomization*** Another runtime technique that can be used to thwart attacks involves manipulation of the location of key data structures in the address space of a process. In particular, recall that in order to implement the classic stack overflow attack, the attacker needs to be able to predict the approximate location of the targeted buffer. The attacker uses this predicted address to determine a suitable return address to use in the attack to transfer control to the shellcode. One technique to greatly increase the difficulty of this prediction is to change the address at which the stack is located in a random manner for each process. The range of addresses available on modern processors is large (32 bits), and most programs only need a small fraction of that. Therefore, moving the stack memory region around by a megabyte or so has minimal impact on most programs, but makes predicting the targeted buffer's address almost impossible.

Another target of attack is the location of standard library routines. In an attempt to bypass protections such as nonexecutable stacks, some buffer overflow variants exploit existing code in standard libraries. These are typically loaded at the same address by the same program. To counter this form of attack, we can use a security extension that randomizes the order of loading standard libraries by a program and their virtual memory address locations. This makes the address of any specific function sufficiently unpredictable as to render the chance of a given attack correctly predicting its address very low.

The OpenBSD system includes versions of these extensions in its technological support for a secure system.

***GUARD PAGES*** A final runtime technique that can be used places **guard pages** between critical regions of memory in a processes address space. Again, this exploits the fact that a process has much more virtual memory available than it typically needs. Gaps are placed between the ranges of addresses used for each of the components of the address space. These gaps, or guard pages, are flagged in the MMU as illegal addresses, and any attempt to access them results in the process being aborted. This can prevent buffer overflow attacks, typically of global data, which attempt to overwrite adjacent regions in the processes address space.

A further extension places guard pages between stack frames or between different allocations on the heap. This can provide further protection against stack and heap overflow attacks, but at cost in execution time supporting the large number of page mappings necessary.

## 15.3 ACCESS CONTROL

Access control is a function exercised by the OS, by the file system, or at both levels. The principles that have been typically applied are the same at both levels. In this section, we begin by looking at access control specifically from the point of view of file access control, then generalize the discussion to access control policies that apply to a variety of system resources.

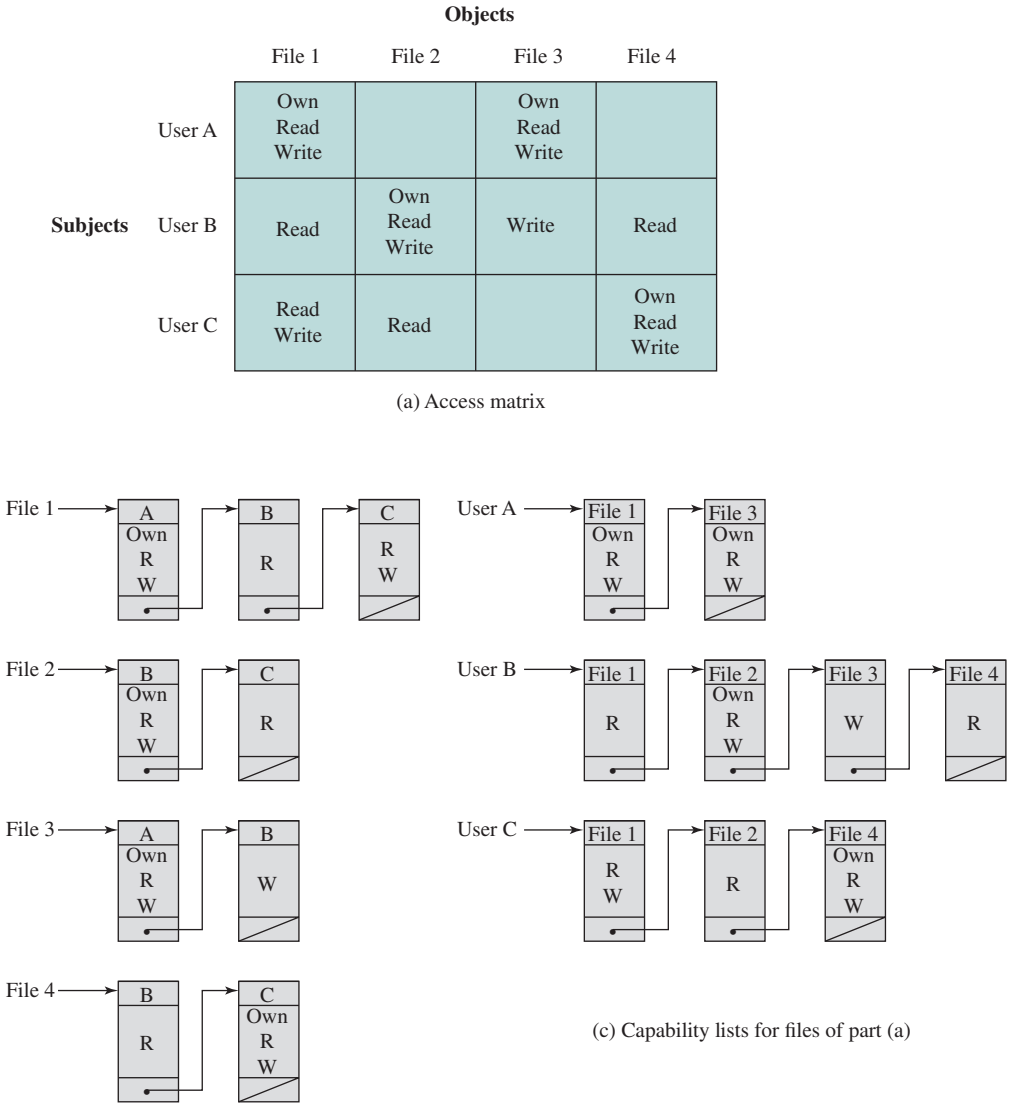
### File System Access Control

Following successful logon, the user has been granted access to one or a set of hosts and applications. This is generally not sufficient for a system that includes sensitive data in its database. Through the user-access control procedure, a user can be identified to the system. Associated with each user, there can be a profile that specifies permissible operations and file accesses. The operating system can then enforce rules based on the user profile. The database management system, however, must control access to specific records or even portions of records. For example, it may be permissible for anyone in administration to obtain a list of company personnel, but only selected individuals may have access to salary information. The issue is more than just a matter of level of detail. Whereas the operating system may grant a user permission to access a file or use an application, following which there are no further security checks, the database management system must make a decision on each individual access attempt. That decision will depend not only on the user's identity but also on the specific parts of the data being accessed and even on the information already divulged to the user.

A general model of access control as exercised by a file or database management system is that of an **access matrix** (see Figure 15.3a). The basic elements of the model are as follows:

- **Subject:** An entity capable of accessing objects. Generally, the concept of subject equates with that of process. Any user or application actually gains access to an object by means of a process that represents that user or application.

- **Object:** Anything to which access is controlled. Examples include files, portions of files, programs, segments of memory, and software objects (e.g., Java objects).
- **Access right:** The way in which an object is accessed by a subject. Examples are read, write, execute, and functions in software objects.




(a) Access matrix

(b) Access control lists for files of part (a)

(c) Capability lists for files of part (a)

**Figure 15.3 Example of Access Control Structures**

One dimension of the matrix consists of identified subjects that may attempt data access. Typically, this list will consist of individual users or user groups, although access could be controlled for terminals, hosts, or applications instead of or in addition to users. The other dimension lists the objects that may be accessed. At the greatest level of detail, objects may be individual data fields. More aggregate groupings, such as records, files, or even the entire database, may also be objects in the matrix. Each entry in the matrix indicates the access rights of that subject for that object.

In practice, an access matrix is usually sparse and is implemented by decomposition in one of two ways. The matrix may be decomposed by columns, yielding **access control lists** (see Figure 15.3b). Thus for each object, an access control list lists users and their permitted access rights. The access control list may contain a default, or public, entry. This allows users that are not explicitly listed as having special rights to have a default set of rights. Elements of the list may include individual users as well as groups of users.

Decomposition by rows yields **capability tickets** (see Figure 15.3c). A capability ticket specifies authorized objects and operations for a user. Each user has a number of tickets and may be authorized to loan or give them to others. Because tickets may be dispersed around the system, they present a greater security problem than access control lists. In particular, the ticket must be unforgeable. One way to accomplish this is to have the operating system hold all tickets on behalf of users. These tickets would have to be held in a region of memory inaccessible to users.

Network considerations for data-oriented access control parallel those for user-oriented access control. If only certain users are permitted to access certain items of data, then encryption may be needed to protect those items during transmission to authorized users. Typically, data access control is decentralized, that is, controlled by host-based database management systems. If a network database server exists on a network, then data access control becomes a network function.

## Access Control Policies

An access control policy dictates what types of access are permitted, under what circumstances, and by whom. Access control policies are generally grouped into the following categories:

- **Discretionary access control (DAC):** Controls access based on the identity of the requestor and on access rules (authorizations) stating what requestors are (or are not) allowed to do. This policy is termed *discretionary* because an entity might have access rights that permit the entity, by its own volition, to enable another entity to access some resource.
- **Mandatory access control (MAC):** Controls access based on comparing security labels (which indicate how sensitive or critical system resources are) with security clearances (which indicate system entities are eligible to access certain resources). This policy is termed *mandatory* because an entity that has clearance to access a resource may not, just by its own volition, enable another entity to access that resource.

- **Role-based access control (RBAC):** Controls access based on the roles that users have within the system, and on rules stating what accesses are allowed to users in given roles.
- **Attribute-based access control (ABAC):** Controls access based on attributes of the user, the resource to be accesses, and current environmental conditions.

DAC is the traditional method of implementing access control. This method was introduced in the preceding discussion of file access control; we provide more detail in this section. MAC is a concept that evolved out of requirements for military information security and is beyond the scope of this book. Both RBAC and ABAC have become increasingly popular. DAC and RBAC are discussed subsequently.

These four policies are not mutually exclusive. An access control mechanism can employ two or even all three of these policies to cover different classes of system resources.

***DISCRETIONARY ACCESS CONTROL*** This section introduces a general model for DAC developed by Lampson, Graham, and Denning [LAMP71, GRAH72, DENN71]. The model assumes a set of subjects, a set of objects, and a set of rules that govern the access of subjects to objects. Let us define the protection state of a system to be the set of information, at a given point in time, that specifies the access rights for each subject with respect to each object. We can identify three requirements: representing the protection state, enforcing access rights, and allowing subjects to alter the protection state in certain ways. The model addresses all three requirements, giving a general, logical description of a DAC system.

To represent the protection state, we extend the universe of objects in the access control matrix to include the following:

- **Processes:** Access rights include the ability to delete a process, stop (block), and wake up a process.
- **Devices:** Access rights include the ability to read/write the device, to control its operation (e.g., a disk seek), and to block/unblock the device for use.
- **Memory locations or regions:** Access rights include the ability to read/write certain locations of regions of memory that are protected so the default is that access is not allowed.
- **Subjects:** Access rights with respect to a subject have to do with the ability to grant or delete access rights of that subject to other objects, as explained subsequently.

Figure 15.4 is an example (compare to Figure 15.3a). For an access control matrix $A$, each entry $A[S, X]$ contains strings, called access attributes, that specify the access rights of subject $S$ to object $X$. For example, in Figure 15.4, $S_1$ may read file $F_2$, because *read* appears in $A[S_1, F_1]$.

From a logical or functional point of view, a separate access control module is associated with each type of object (see Figure 15.4). The module evaluates each

| | | Objects | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | | Subjects | | | Files | | Processes | | Disk drives | |
| | | $S_1$ | $S_2$ | $S_3$ | $F_1$ | $F_2$ | $P_1$ | $P_2$ | $D_1$ | $D_2$ |
| Subjects | $S_1$ | Control | Owner | Owner control | Read * | Read owner | Wakeup | Wakeup | Seek | Owner |
| | $S_2$ | | Control | | Write * | Execute | | | Owner | Seek * |
| | $S_3$ | | | Control | | Write | Stop | | | |

* — Copy flag set

**Figure 15.4 Extended Access Control Matrix**

request by a subject to access an object to determine if the access right exists. An access attempt triggers the following steps:

1. A subject $S_0$ issues a request of type $\alpha$ for object $X$.
2. The request causes the system (the operating system or an access control interface module of some sort) to generate a message of the form $(S_0, \alpha, X)$ to the controller for $X$.
3. The controller interrogates the access matrix $A$ to determine if $\alpha$ is in $A[S_0, X]$. If so, the access is allowed; if not, the access is denied and a protection violation occurs. The violation should trigger a warning and an appropriate action.

Figure 15.5 suggests that every access by a subject to an object is mediated by the controller for that object, and that the controller's decision is based on the current contents of the matrix. In addition, certain subjects have the authority to make specific changes to the access matrix. A request to modify the access matrix is treated as an access to the matrix, with the individual entries in the matrix treated as objects. Such accesses are mediated by an access matrix controller, which controls updates to the matrix.

The model also includes a set of rules that govern modifications to the access matrix, shown in Table 15.1. For this purpose, we introduce the access rights *owner* and *control* and the concept of a copy flag, explained in the subsequent paragraphs.

The first three rules deal with transferring, granting, and deleting access rights. Suppose the entry $\alpha^*$ exists in $A[S_0, X]$. This means $S_0$ has access right $\alpha$ to subject $X$ and, because of the presence of the copy flag, can transfer this right, with or without copy flag, to another subject. Rule R1 expresses this capability. A subject would transfer the access right without the copy flag if there were a concern that the new subject would maliciously transfer the right to another subject that should not have that access right. For example, $S_1$ may place *read* or *read** in any matrix entry in the $F_1$ column. Rule R2 states if $S_0$ is designated as the owner of object $X$, then $S_0$ can grant an access right to that object for any other subject. Rule 2 states $S_0$ can add any access right to $A[S, X]$ for any $S$, if $S_0$ has *owner* access to $X$. Rule R3 permits $S_0$

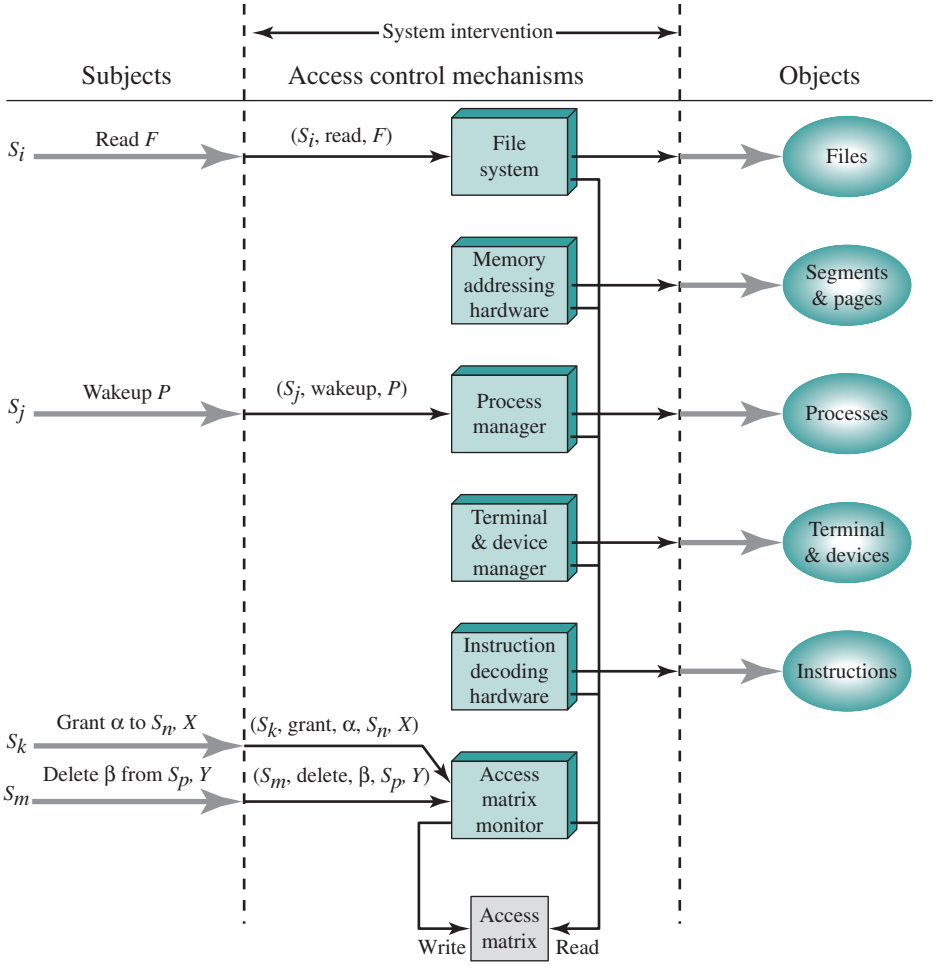



**Figure 15.5 An Organization of the Access Control Function**

to delete any access right from any matrix entry in a row for which $S_0$ controls the subject, and for any matrix entry in a column for which $S_0$ owns the object. Rule R4 permits a subject to read that portion of the matrix that it owns or controls.

The remaining rules in Table 15.1 govern the creation and deletion of subjects and objects. Rule R5 states any subject can create a new object, which it owns, and can then grant and delete access to the object. Under rule R6, the owner of an object can destroy the object, resulting in the deletion of the corresponding column of the access matrix. Rule R7 enables any subject to create a new subject; the creator owns the new subject and the new subject has control access to itself. Rule R8 permits the owner of a subject to delete the row and column (if there are subject columns) of the access matrix designated by that subject.

**Table 15.1** Access Control System Commands

| Rule | Command (by $S_0$) | Authorization | Operation |
|---|---|---|---|
| R1 | **transfer** $\left\{\begin{matrix}\alpha* \\ \alpha\end{matrix}\right\}$ **to** $S, X$ | '$\alpha$*' in $A[S_0, X]$ | store $\left\{\begin{matrix}\alpha* \\ \alpha\end{matrix}\right\}$ in $A[S, X]$ |
| R2 | **grant** $\left\{\begin{matrix}\alpha* \\ \alpha\end{matrix}\right\}$ **to** $S, X$ | 'owner' in $A[S_0, X]$ | store $\left\{\begin{matrix}\alpha* \\ \alpha\end{matrix}\right\}$ in $A[S, X]$ |
| R3 | **delete** $\alpha$ **from** $S, X$ | 'control' in $A[S_0, S]$ or 'owner' in $A[S_0, X]$ | delete $\alpha$ from $A[S, X]$ |
| R4 | $w \leftarrow$ **read** $S, X$ | 'control' in $A[S_0, S]$ or 'owner' in $A[S_0, X]$ | copy $A[S, X]$ into $w$ |
| R5 | **create object** $X$ | None | add column for $X$ to $A$; store 'owner' in $A[S_0, X]$ |
| R6 | **destroy object** $X$ | 'owner' in $A[S_0, X]$ | delete column for $X$ from $A$ |
| R7 | **create subject** $S$ | None | add row for $S$ to $A$; execute **create object** $S$; store 'control' in $A[S, S]$ |
| R8 | **destroy subject** $S$ | 'owner' in $A[S_0, S]$ | delete row for $S$ from $A$; execute **destroy object** $S$ |

The set of rules in Table 15.1 is an example of the rule set that could be defined for an access control system. The following are examples of additional or alternative rules that could be included. A transfer-only right could be defined, which results in the transferred right being added to the target subject and deleted from the transferring subject. The number of owners of an object or a subject could be limited to one by not allowing the copy flag to accompany the owner right.

The ability of one subject to create another subject and to have *owner* access right to that subject can be used to define a hierarchy of subjects. For example, in Figure 15.4, $S_1$ owns $S_2$ and $S_3$, so $S_2$ and $S_3$ are subordinate to $S_1$. By the rules of Table 15.1, $S_1$ can grant and delete to $S_2$ access rights that $S_1$ already has. Thus, a subject can create another subject with a subset of its own access rights. This might be useful, for example, if a subject is invoking an application that is not fully trusted, and does not want that application to be able to transfer access rights to other subjects.

***ROLE-BASED ACCESS CONTROL*** Traditional DAC systems define the access rights of individual users and groups of users. In contrast, RBAC is based on the roles that users assume in a system, rather than the user's identity. Typically, RBAC models define a role as a job function within an organization. RBAC systems assign access rights to roles instead of individual users. In turn, users are assigned to different roles, either statically or dynamically, according to their responsibilities.

RBAC now enjoys widespread commercial use and remains an area of active research. The National Institute of Standards and Technology (NIST) has issued a standard, *Security Requirements for Cryptographic Modules* (FIPS PUB 140-2, May 25, 2001), that requires support for access control and administration through roles.

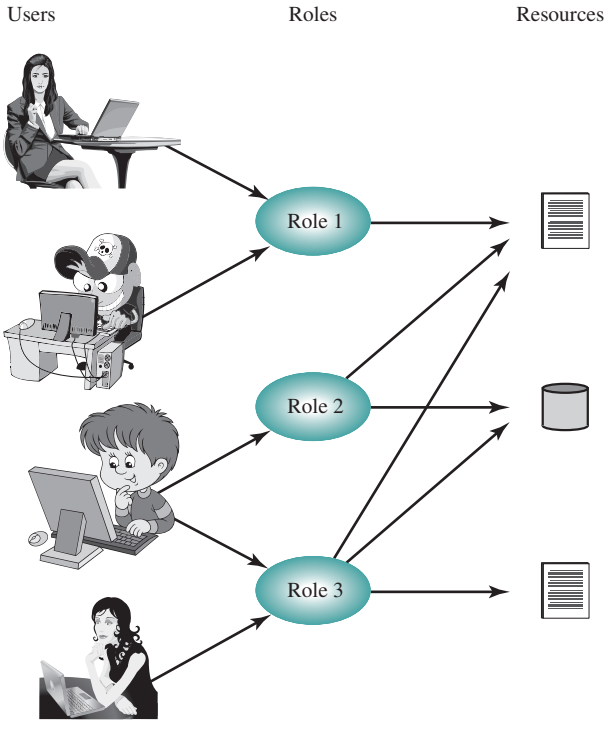



**Figure 15.6** **Users, Roles, and Resources**

The relationship of users to roles is many to many, as is the relationship of roles to resources, or system objects (see Figure 15.6). The set of users changes, in some environments frequently, and the assignment of a user to one or more roles may also be dynamic. The set of roles in the system in most environments is likely to be static, with only occasional additions or deletions. Each role will have specific access rights to one or more resources. The set of resources and the specific access rights associated with a particular role are also likely to change infrequently.

We can use the access matrix representation to depict the key elements of an RBAC system in simple terms, as shown in Figure 15.7. The upper matrix relates individual users to roles. Typically, there are many more users than roles. Each matrix entry is either blank or marked, the latter indicating that this user is assigned to this role. Note a single user may be assigned multiple roles (more than one mark in a row), and multiple users may be assigned to a single role (more than one mark in a column). The lower matrix has the same structure as the DAC matrix, with roles as subjects. Typically, there are few roles and many objects, or resources. In this matrix the entries are the specific access rights enjoyed by the roles. Note a role can be treated as an object, allowing the definition of role hierarchies.

RBAC lends itself to an effective implementation of the principle of least privilege. That is, each role should contain the minimum set of access rights needed for that role. A user is assigned to a role that enables him or her to perform only what is required for that role. Multiple users assigned to the same role enjoy the same minimal set of access rights.

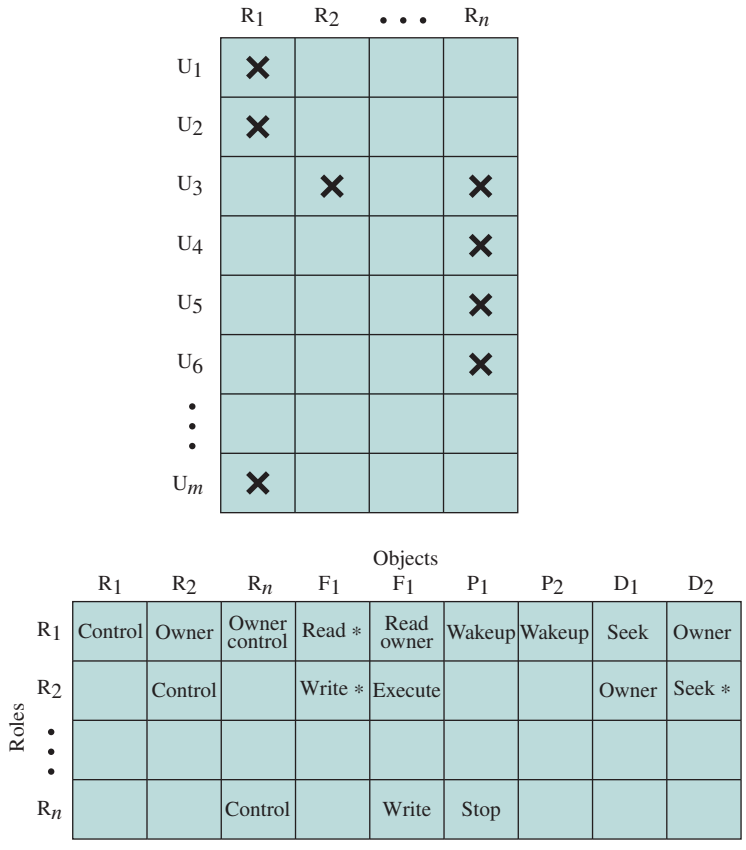


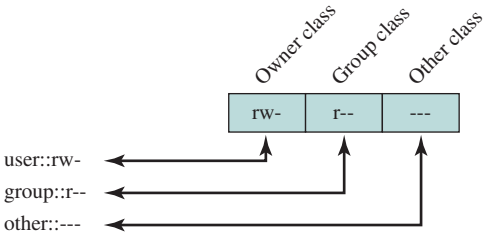

**Figure 15.7 Access Control Matrix Representation of RBAC**

## 15.4 UNIX ACCESS CONTROL

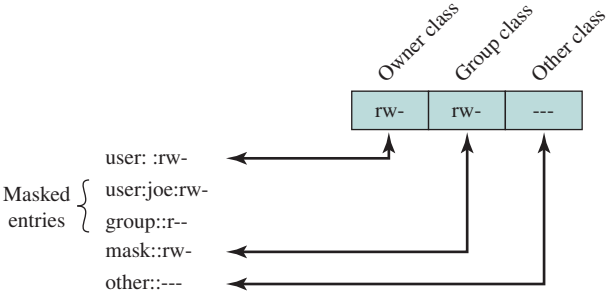
### Traditional UNIX File Access Control

Most UNIX systems depend on, or at least are based on, the file access control scheme introduced with the early versions of UNIX. Each UNIX user is assigned a unique user identification number (user ID). A user is also a member of a primary group, and possibly a number of other groups, each identified by a group ID. When a file is created, it is designated as owned by a particular user and marked with that user's ID. It also belongs to a specific group, which initially is either its creator's primary group or the group of its parent directory if that directory has SetGID permission set. Associated with each file is a set of 12 protection bits. The owner ID, group ID, and protection bits are part of the file's inode.

Nine of the protection bits specify read, write, and execute permission for the owner of the file, other members of the group to which this file belongs, and all other users. These form a hierarchy of owner, group, and all others, with the highest relevant set of permissions being used. Figure 15.8a shows an example in which the file owner




(a) Traditional UNIX approach (minimal access control list)




(b) Extended access control list

**Figure 15.8 UNIX File Access Control**

has read and write access; all other members of the file's group have read access, and users outside the group have no access rights to the file. When applied to a directory, the read and write bits grant the right to list and to create/rename/delete files in the directory.[8] The execute bit grants the right to search the directory for a component of a filename.

The remaining three bits define special additional behavior for files or directories. Two of these are the "set user ID" (SetUID) and "set group ID" (SetGID) permissions. If these are set on an executable file, the operating system functions as follows. When a user (with execute privileges for this file) executes the file, the system temporarily allocates the rights of the user's ID of the file creator or the file's group, respectively, to those of the user executing the file. These are known as the "effective user ID" and "effective group ID" and are used in addition to the "real user ID" and "real group ID" of the executing user when making access control decisions for this program. This change is only effective while the program is being executed. This feature enables the creation and use of privileged programs that may use files normally inaccessible to other users. It enables users to access certain files in a controlled fashion. Alternatively, when applied to a directory, the SetGID permission

[8]Note the permissions that apply to a directory are distinct from those that apply to any file or directory it contains. The fact that a user has the right to write to the directory does not give the user the right to write to a file in that directory. That is governed by the permissions of the specific file. The user would, however, have the right to rename the file.

indicates that newly created files will inherit the group of this directory. The SetUID permission is ignored.

The final permission bit is the "Sticky" bit. When set on a file, this originally indicated that the system should retain the file contents in memory following execution. This is no longer used. When applied to a directory, though, it specifies that only the owner of any file in the directory can rename, move, or delete that file. This is useful for managing files in shared temporary directories.

One particular user ID is designated as *superuser*. The superuser is exempt from the usual file access control constraints and has systemwide access. Any program that is owned by, and SetUID to, the "superuser" potentially grants unrestricted access to the system to any user executing that program. Hence, great care is needed when writing such programs.

This access scheme is adequate when file access requirements align with users and a modest number of groups of users. For example, suppose a user wants to give read access for file X to users A and B and read access for file Y to users B and C. We would need at least two user groups, and user B would need to belong to both groups in order to access the two files. However, if there are a large number of different groupings of users requiring a range of access rights to different files, then a very large number of groups may be needed to provide this. This rapidly becomes unwieldy and difficult to manage, even if possible at all.[9] One way to overcome this problem is to use access control lists, which are provided in most modern UNIX systems.

A final point to note is the traditional UNIX file access control scheme implements a simple protection domain structure. A domain is associated with the user, and switching the domain corresponds to changing the user ID temporarily.

### Access Control Lists in UNIX

Many modern UNIX and UNIX-based operating systems support access control lists, including FreeBSD, OpenBSD, Linux, and Solaris. In this section, we describe the FreeBSD approach, but other implementations have essentially the same features and interface. The feature is referred to as extended access control list, while the traditional UNIX approach is referred to as minimal access control list.

FreeBSD allows the administrator to assign a list of UNIX user IDs and groups to a file by using the `setfacl` command. Any number of users and groups can be associated with a file, each with three protection bits (read, write, execute), offering a flexible mechanism for assigning access rights. A file need not have an ACL but may be protected solely by the traditional UNIX file access mechanism. FreeBSD files include an additional protection bit that indicates whether the file has an extended ACL.

FreeBSD and most UNIX implementations that support extended ACLs use the following strategy (e.g., Figure 15.8b):

1. The owner class and other class entries in the 9-bit permission field have the same meaning as in the minimal ACL case.

[9]Most UNIX systems impose a limit on the maximum number of groups any user may belong to, as well as to the total number of groups possible on the system.

2. The group class entry specifies the permissions for the owner group for this file. These permissions represent the maximum permissions that can be assigned to named users or named groups, other than the owning user. In this latter role, the group class entry functions as a mask.
3. Additional named users and named groups may be associated with the file, each with a 3-bit permission field. The permissions listed for a named user or named group are compared to the mask field. Any permission for the named user or named group that is not present in the mask field is disallowed.

When a process requests access to a file system object, two steps are performed. Step 1 selects the ACL entry that most closely matches the requesting process. The ACL entries are looked at in the following order: owner, named users, owning or named groups, and others. Only a single entry determines access. Step 2 checks if the matching entry contains sufficient permissions. A process can be a member in more than one group; so more than one group entry can match. If any of these matching group entries contain the requested permissions, one that contains the requested permissions is picked (the result is the same no matter which entry is picked). If none of the matching group entries contains the requested permissions, access will be denied no matter which entry is picked.

## 15.5 OPERATING SYSTEMS HARDENING

The first critical step in securing a system is to secure the base operating system upon which all other applications and services rely. A good security foundation needs a properly installed, patched, and configured operating system. Unfortunately, the default configuration for many operating systems often maximizes ease of use and functionality, rather than security. Further, since every organization has its own security needs, the appropriate security profile, and hence configuration, will also differ. What is required for a particular system should be identified during the planning phase, as we have just discussed.

While the details of how to secure each specific operating system differ, the broad approach is similar. Appropriate security configuration guides and checklists exist for most common operating systems, and these should be consulted, though always informed by the specific needs of each organization and their systems. In some cases, automated tools may be available to further assist in securing the system configuration.

[NIST08] suggests the following basic steps should be used to secure an operating system:

- Install and patch the operating system.
- Harden and configure the operating system to adequately address the identified security needs of the system by:
  - Removing unnecessary services, applications, and protocols.
  - Configuring users, groups, and permissions.
  - Configuring resource controls.

- Install and configure additional security controls, such as antivirus, host-based firewalls, and intrusion detection systems (IDS), if needed.
- Test the security of the basic operating system to ensure that the steps taken adequately address its security needs.

## Operating System Installation: Initial Setup and Patching

System security begins with the installation of the operating system. As we have already noted, a network-connected, unpatched system is vulnerable to exploit during its installation or continued use. Hence it is important that the system not be exposed while it is in this vulnerable state. Ideally new systems should be constructed on a protected network. This may be a completely isolated network, with the operating system image and all available patches transferred to it using removable media such as DVDs or USB drives. Given the existence of malware that can propagate using removable media, care is needed to ensure the media used here is not so infected. Alternatively, a network with severely restricted access to the wider Internet may be used. Ideally it should have no inbound access, and have outbound access only to the key sites needed for the system installation and patching process. In either case, the full installation and hardening process should occur before the system is deployed to its intended, more accessible, and hence vulnerable, location.

The initial installation should comprise the minimum necessary for the desired system, with additional software packages included only if they are required for the function of the system. We explore the rationale for minimizing the number of packages on the system shortly.

The overall boot process must also be secured. This may require adjusting options on, or specifying a password required for changes to, the BIOS code used when the system initially boots. It may also require limiting which media from which the system is normally permitted to boot. This is necessary to prevent an attacker from changing the boot process to install a covert hypervisor, or to just boot a system of their choice from external media in order to bypass the normal system access controls on locally stored data. The use of a cryptographic file system may also be used to address this threat, as we note later.

Care is also required with the selection and installation of any additional device driver code, since this executes with full kernel-level privileges, but is often supplied by a third party. The integrity and source of such driver code must be carefully validated given the high level of trust it has. A malicious driver can potentially bypass many security controls to install malware. Given the continuing discovery of software and other vulnerabilities for commonly used operating systems and applications, it is critical that the system be kept as up-to-date as possible, with all critical security-related patches installed. Nearly all commonly used systems now provide utilities that can automatically download and install security updates. These tools should be configured and used to minimize the amount of time a system is vulnerable to weaknesses for which patches are available.

Note on change-controlled systems, you should not run automatic updates, because security patches can, on rare but significant occasions, introduce instability. For systems on which availability and uptime are of paramount importance, therefore,

you should stage and validate all patches on test systems before deploying them in production.

### Remove Unnecessary Services, Application, and Protocols

Because any of the software running on a system may contain software vulnerabilities, clearly if fewer software packages are available to run, then the risk is reduced. There is clearly a balance between usability, providing all software that may be required at some time, and security and a desire to limit the amount of software installed. The range of services, applications, and protocols required will vary widely between organizations, and indeed between systems within an organization. The system planning process should identify what is actually required for a given system, so a suitable level of functionality is provided, while eliminating software that is not required to improve security.

The default configuration for most distributed systems is set to maximize ease of use and functionality, rather than security. When performing the initial installation, the supplied defaults should not be used, but rather the installation should be customized so only the required packages are installed. If additional packages are needed later, they can be installed when they are required. [NIST08] and many of the security-hardening guides provide lists of services, applications, and protocols that should not be installed if not required.

[NIST08] also states a strong preference for not installing unwanted software, rather than installing then later removing or disabling it. They argue this preference because they note that many uninstall scripts fail to completely remove all components of a package. They also note that disabling a service means while it is not available as an initial point of attack, should an attacker succeed in gaining some access to a system, then disabled software could be reenabled and used to further compromise a system. It is better for security if unwanted software is not installed, and thus not available for use at all.

### Configure Users, Groups, and Authentication

Not all users with access to a system will have the same access to all data and resources on that system. All modern operating systems implement access controls to data and resources. Nearly all provide some form of discretionary access controls. Some systems may provide role-based or mandatory access control mechanisms as well.

The system planning process should consider the categories of users on the system, the privileges they have, the types of information they can access, and how and where they are defined and authenticated. Some users will have elevated privileges to administer the system; others will be normal users, sharing appropriate access to files and other data as required; and there may even be guest accounts with very limited access. The third of the four key DSD mitigation strategies is to restrict elevated privileges to only those users that require them. Further, it is highly desirable that such users only access elevated privileges when needed to perform some task that requires them, and to otherwise access the system as a normal user. This improves security by providing a smaller window of opportunity for an attacker to exploit the actions of such privileged users. Some operating systems provide special tools or

access mechanisms to assist administrative users to elevate their privileges only when necessary, and to appropriately log these actions.

One key decision is whether the users, the groups to which they belong, and their authentication methods are specified locally on the system, or will use a centralized authentication server. Whichever is chosen, the appropriate details are now configured on the system.

Also at this stage, any default accounts included as part of the system installation should be secured. Those which are not required should be either removed or at least disabled. System accounts that manage services on the system should be set so they cannot be used for interactive logins. And any passwords installed by default should be changed to new values with appropriate security.

Any policy that applies to authentication credentials, and especially to password security, is also configured. This includes details of which authentication methods are accepted for different methods of account access. And it includes details of the required length, complexity, and age allowed for passwords.

### Configure Resource Controls

Once the users and their associated groups are defined, appropriate permissions can be set on data and resources to match the specified policy. This may be to limit which users can execute some programs, especially those that modify the system state, or to limit which users can read or write data in certain directory trees. Many of the security-hardening guides provide lists of recommended changes to the default access configuration to improve security.

### Install Additional Security Controls

Further security improvement may be possible by installing and configuring additional security tools such as antivirus software, host-based firewalls, IDS or IPS software, or application white-listing. Some of these may be supplied as part of the operating systems installation, but not configured and enabled by default. Others are third-party products that are acquired and used.

Given the wide spread prevalence of malware, appropriate antivirus (which, as noted, addresses a wide range of malware types) is a critical security component on many systems. Antivirus products have traditionally been used on Windows systems, since their high use made them a preferred target for attackers. However, the growth in other platforms, particularly smart phones, has led to more malware being developed for them. Hence appropriate antivirus products should be considered for any system as part of its security profile.

Host-based firewalls, IDS, and IPS software also may improve security by limiting remote network access to services on the system. If remote access to a service is not required, though some local access is, then such restrictions help secure such services from remote exploit by an attacker. Firewalls are traditionally configured to limit access by port or protocol, from some or all external systems. Some may also be configured to allow access from or to specific programs on the systems, to further restrict the points of attack, and to prevent an attacker from installing and accessing their own malware. IDS and IPS software may include additional mechanisms such as traffic monitoring or file integrity checking to identify and even respond to some types of attack.

Another additional control is to white-list applications. This limits the programs that can execute on the system to just those in an explicit list. Such a tool can prevent an attacker installing and running their own malware, and was the last of the four key DSD mitigation strategies. While this will improve security, it functions best in an environment with a predictable set of applications that users require. Any change in software usage would require a change in the configuration, which may result in increased IT support demands. Not all organizations or all systems will be sufficiently predictable to suit this type of control.

### Test the System Security

The final step in the process of initially securing the base operating system is security testing. The goal is to ensure that the previous security configuration steps are correctly implemented and to identify any possible vulnerabilities that must be corrected or managed.

Suitable checklists are included in many security-hardening guides. There are also programs specifically designed to review a system to ensure that a system meets the basic security requirements, and to scan for known vulnerabilities and poor configuration practices. This should be done following the initial hardening of the system, then repeated periodically as part of the security maintenance process.

## 15.6 SECURITY MAINTENANCE

Once the system is appropriately built, secured, and deployed, the process of maintaining security is continuous. This results from the constantly changing environment, the discovery of new vulnerabilities, and hence exposure to new threats. [NIST08] suggests that this process of security maintenance includes the following additional steps:

- Monitoring and analyzing logging information
- Performing regular backups
- Recovering from security compromises
- Regularly testing system security
- Using appropriate software maintenance processes to patch and update all critical software, and to monitor and revise configuration as needed

We have already noted the need to configure automatic patching and update where possible, or to have a process to manually test and install patches on configuration-controlled systems, and that the system should be regularly tested using checklist or automated tools where possible.

### Logging

[NIST08] notes that "logging is a cornerstone of a sound security posture." Logging is a reactive control that can only inform you about bad things that have already happened. But effective logging helps ensure that in the event of a system breach or failure, system administrators can more quickly and accurately identify what happened, and thus most effectively focus their remediation and recovery efforts. The key is to ensure you capture the correct data in the logs then appropriately monitor

and analyze this data. Logging information can be generated by the system, network, and applications. The range of logging data acquired should be determined during the system planning stage, as it depends on the security requirements and information sensitivity of the server.

Logging can generate significant volumes of information. It is important that sufficient space is allocated for them. A suitable automatic log rotation and archive system should also be configured to assist in managing the overall size of the logging information.

Manual analysis of logs is tedious and is not a reliable means of detecting adverse events. Rather, some form of automated analysis is preferred, as it is more likely to identify abnormal activity.

### Data Backup and Archive

Performing regular backups of data on a system is another critical control that assists with maintaining the integrity of the system and user data. There are many reasons why data can be lost from a system, including hardware or software failures, or accidental or deliberate corruption. There may also be legal or operational requirements for the retention of data. **Backup** is the process of making copies of data at regular intervals, allowing the recovery of lost or corrupted data over relatively short time periods of a few hours to some weeks. **Archive** is the process of retaining copies of data over extended periods of time, being months or years, in order to meet legal and operational requirements to access past data. These processes are often linked and managed together, although they do address distinct needs.

The needs and policy relating to backup and archive should be determined during the system planning stage. Key decisions include whether the backup copies should be kept online or offline, and whether copies should be stored locally or transported to a remote site. The trade-offs include ease of implementation and cost verses greater security and robustness against different threats.

A good example of the consequences of poor choices here was seen in the attack on an Australian hosting provider in early 2011. The attackers destroyed not only the live copies of thousands of customer's sites but also all of the online backup copies. As a result, many customers who had not kept their own backup copies lost all of their site content and data, with serious consequences for many of them, and for the hosting provider as well. In other examples, many organizations who only retained onsite backups have lost all their data as a result of fire or flooding in their IT center. These risks must be appropriately evaluated.

## 15.7 WINDOWS SECURITY

A good example of the access control concepts we have been discussing is the Windows access control facility, which uses object-oriented concepts to provide a powerful and flexible access control capability.

Windows provides a uniform access control facility that applies to processes, threads, files, semaphores, windows, and other objects. Access control is governed by two entities: an access token associated with each process, and a security descriptor associated with each object for which interprocess access is possible.

## Access Control Scheme

When a user logs on to a Windows system, Windows uses a name/password scheme to authenticate the user. If the logon is accepted, a process is created for the user and an access token is associated with that process object. The access token, whose details are described later, include a security ID (SID), which is the identifier by which this user is known to the system for purposes of security. The token also contains SIDs for the security groups to which the user belongs. If the initial user process spawns a new process, the new process object inherits the same access token.

The access token serves two purposes:

1. It keeps all necessary security information together to speed up access validation. When any process associated with a user attempts access, the security subsystem can make use of the token associated with that process to determine the user's access privileges.
2. It allows each process to modify its security characteristics in limited ways without affecting other processes running on behalf of the user.

The chief significance of the second point has to do with privileges that may be associated with a user. The access token indicates which privileges a user may have. Generally, the token is initialized with each of these privileges in a disabled state. Subsequently, if one of the user's processes needs to perform a privileged operation, the process may enable the appropriate privilege and attempt access. It would be undesirable to share the same token among all of the user's processes, because in that case, enabling a privilege for one process enables it for all of them.

Associated with each object for which interprocess access is possible is a security descriptor. The chief component of the security descriptor is an access control list that specifies access rights for various users and user groups for this object. When a process attempts to access this object, the SIDs in the process token are matched against the access control list of the object to determine if access will be allowed or denied.

When an application opens a reference to a securable object, Windows verifies that the object's security descriptor grants the process the requested access. If the check succeeds, Windows caches the resulting granted access rights.

An important aspect of Windows security is the concept of impersonation, which simplifies the use of security in a client/server environment. If client and server talk through an RPC connection, the server can temporarily assume the identity of the client so that it can evaluate a request for access relative to that client's rights. After the access, the server reverts to its own identity.

## Access Token

Figure 15.9a shows the general structure of an access token, which includes the following parameters:

- **Security ID:** Identifies a user uniquely across all of the machines on the network. This generally corresponds to a user's logon name. Special user SIDs were added in Windows 7 for use by processes and services. These specially managed SIDs are designed for secure management; they do not use the ordinary password policies human accounts do.

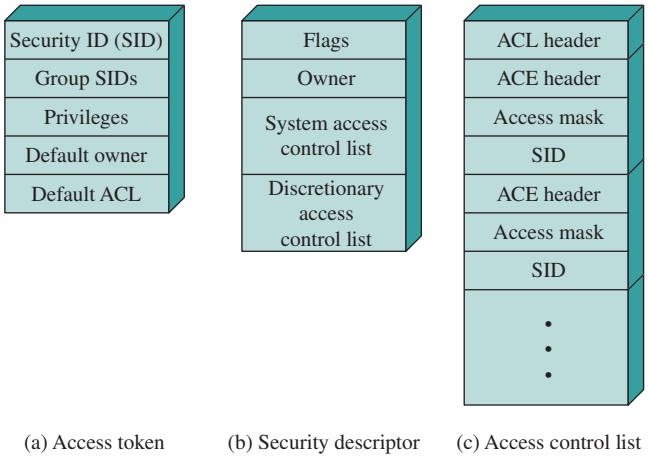



**Figure 15.9 Windows Security Structures**

- **Group SIDs:** A list of the groups to which this user belongs. A group is simply a set of user IDs that are identified as a group for purposes of access control. Each group has a unique group SID. Access to an object can be defined on the basis of group SIDs, individual SIDs, or a combination. There is also an SID which reflects the process integrity level (low, medium, high, or system).
- **Privileges:** A list of security-sensitive system services that this user may call, for example, CreateToken. Another example is the SetBackupPrivilege; users with this privilege are allowed to use a backup tool to back up files they normally would not be able to read.
- **Default owner:** If this process creates another object, this field specifies the owner of the new object. Generally, the owner of a new object is the same as the owner of the spawning process. However, a user may specify that the default owner of any processes spawned by this process is a group SID to which this user belongs.
- **Default ACL:** This is an initial list of protections applied to the objects that the user creates. The user may subsequently alter the ACL for any object that it owns or that one of its groups owns.

## Security Descriptors

Figure 15.9b shows the general structure of a security descriptor, which includes the following parameters:

- **Flags:** Define the type and contents of a security descriptor. They indicate whether or not the SACL and DACL are present, whether or not they were placed on the object by a defaulting mechanism, and whether the pointers in the descriptor use absolute or relative addressing. Relative descriptors are required for objects that are transmitted over a network, such as information transmitted in an RPC.

- **Owner:** The owner of the object can generally perform any action on the security descriptor. The owner can be an individual or a group SID. The owner has the authority to change the contents of the DACL.
- **System access control list (SACL):** Specifies what kinds of operations on the object should generate audit messages. An application must have the corresponding privilege in its access token to read or write the SACL of any object. This is to prevent unauthorized applications from reading SACLs (thereby learning what not to do to avoid generating audits) or writing them (to generate many audits to cause an illicit operation to go unnoticed). The SACL also specifies the object integrity level. Processes cannot modify an object unless the process integrity level meets or exceeds the level on the object.
- **Discretionary access control list (DACL):** Determines which users and groups can access this object for which operations. It consists of a list of access control entries (ACEs).

When an object is created, the creating process can assign as owner its own SID or any group SID in its access token. The creating process cannot assign an owner that is not in the current access token. Subsequently, any process that has been granted the right to change the owner of an object may do so, but again with the same restriction. The reason for the restriction is to prevent a user from covering his or her tracks after attempting some unauthorized action.

Let us look in more detail at the structure of access control lists, because these are at the heart of the Windows access control facility (see Figure 15.9c). Each list consists of an overall header and a variable number of access control entries. Each entry specifies an individual or a group SID and an access mask that defines the rights to be granted to this SID. When a process attempts to access an object, the object manager in the Windows Executive reads the SID and group SIDs from the access token along with the integrity level SID. If the access requested includes modifying the object, the integrity level is checked against the object integrity level in the SACL. If that test passes, the object manager then scans down the object's DACL. If a match is found—that is, if an ACE is found with an SID that matches one of the SIDs from the access token—then the process can have the access rights specified by the access mask in that ACE. This also may include denying access, in which case the access request fails. The first matching ACE determines the result of the access check.

Figure 15.10 shows the contents of the access mask. The least significant 16 bits specify access rights that apply to a particular type of object. For example, bit 0 for a file object is FILE_READ_DATA access and bit 0 for an event object is EVENT_QUERY_STATE access.

The most significant 16 bits of the mask contain bits that apply to all types of objects. Five of these are referred to as standard access types:

- **Synchronize:** Gives permission to synchronize execution with some event associated with this object. In particular, this object can be used in a wait function.
- **Write_owner:** Allows a program to modify the owner of the object. This is useful because the owner of an object can always change the protection on the object. (The owner may not be denied Write DAC access.)

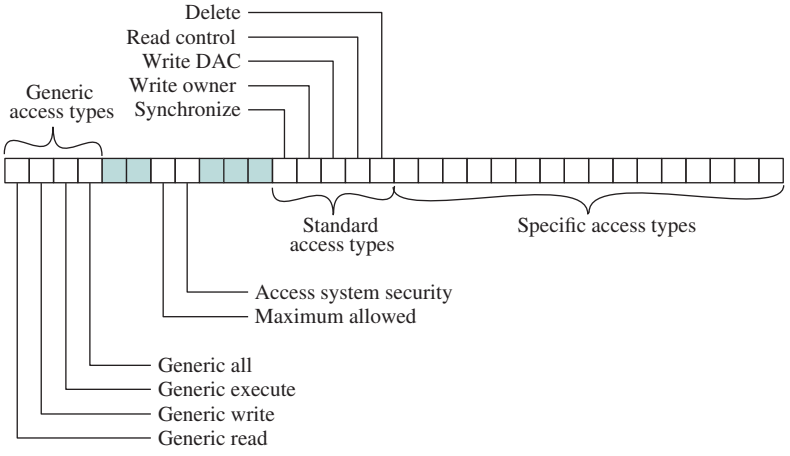



**Figure 15.10 Access Mask**

- **Write_DAC:** Allows the application to modify the DACL and hence the protection on this object
- **Read_control:** Allows the application to query the owner and DACL fields of the security descriptor of this object
- **Delete:** Allows the application to delete this object

The high-order half of the access mask also contains the four generic access types. These bits provide a convenient way to set specific access types in a number of different object types. For example, suppose an application wishes to create several types of objects and ensure that users have read access to the objects, even though read has a somewhat different meaning for each object type. To protect each object of each type without the generic access bits, the application would have to construct a different ACE for each type of object and be careful to pass the correct ACE when creating each object. It is more convenient to create a single ACE that expresses the generic concept "allow read," and simply apply this ACE to each object that is created, and have the right thing happen. That is the purpose of the generic access bits, which are as follows:

- **Generic_all:** Allows all access
- **Generic_execute:** Allows execution if executable
- **Generic_write:** Allows write access
- **Generic_read:** Allows read-only access

The generic bits also affect the standard access types. For example, for a file object, the Generic_Read bit maps to the standard bits Read_Control and Synchronize, and to the object-specific bits File_Read_Data, File_Read_Attributes, and File_Read_EA. Placing an ACE on a file object that grants some SID Generic_Read grants those five access rights as if they had been specified individually in the access mask.

The remaining two bits in the access mask have special meanings. The Access_System_Security bit allows modifying audit and alarm control for this object. However, not only must this bit be set in the ACE for an SID but the access token for the process with that SID must have the corresponding privilege enabled.

Finally, the Maximum_Allowed bit is not really an access bit, but a bit that modifies the algorithm for scanning the DACL for this SID. Normally, Windows will scan through the DACL until it reaches an ACE that specifically grants (bit set) or denies (bit not set) the access requested by the requesting process, or until it reaches the end of the DACL; in the latter case, access is denied. The Maximum_Allowed bit allows the object's owner to define a set of access rights that is the maximum that will be allowed to a given user. With this in mind, suppose an application does not know all of the operations that it is going to be asked to perform on an object during a session. There are three options for requesting access:

1. Attempt to open the object for all possible accesses. The disadvantage of this approach is that access may be denied even though the application may have all of the access rights actually required for this session.
2. Only open the object when a specific access is requested, and open a new handle to the object for each different type of request. This is generally the preferred method because it will not unnecessarily deny access, nor will it allow more access than necessary. In many cases the object itself does not need to be referenced a second time, but the DuplicateHandle function can be used to make a copy of the handle with a lower level of access.
3. Attempt to open the object for as much access as the object will allow this SID. The advantage is that the client application will not be artificially denied access, but the application may have more access than it needs. This latter situation may mask bugs in the application.

An important feature of Windows security is applications can make use of the Windows security framework for user-defined objects. For example, a database server might create its own security descriptors and attach them to portions of a database. In addition to normal read/write access constraints, the server could secure database-specific operations, such as scrolling within a result set or performing a join. It would be the server's responsibility to define the meaning of special rights and perform access checks. But the checks would occur in a standard context, using system-wide user/group accounts and audit logs. The extensible security model should also prove useful to implementers of non-Microsoft file systems.

## 15.8 SUMMARY

The scope of operating system security is broad. This chapter focuses on some of the most important topics. The most prominent issue for OS security is countering thread from intruders and malicious software. Intruders attempt to gain unauthorized access to system resources, while malicious software is designed to penetrate system defenses and become executable on target systems. Countermeasures to both types of threat include intrusion detection systems, authentication protocols, access control mechanisms, and firewalls.

One of the most common techniques for compromising OS security is the buffer overflow attack. A condition at an interface under which more input can be placed into a buffer or data-holding area than the capacity allocated, overwriting other information. Attackers exploit such a condition to crash a system or to insert specially crafted code that allows them to gain control of the system. System designers use a variety of compile-time and runtime defenses to counter this type of attack.

Another important area of security defense is access control. Access control measures include those that secure access to file system and to the OS user interface. Traditional techniques for access control are referred to as discretionary access control. A more flexible approach that has gained considerable support is role-based access control, in which access depends not only on the identity of the user, but on the specific role that user can assume for a specific task or set of tasks.

## 15.9 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

### Key Terms

| | | |
|---|---|---|
| access control | capability ticket | logging |
| access control list (ACL) | discretionary access control (DAC) | malicious software |
| access control policy | file system access control | malware |
| access matrix | firewall | role-based access control (RBAC) |
| address space randomization | guard page | stack overflow |
| authentication | intruder | |
| buffer overrun | intrusion detection | |
| buffer overflow | | |

### Review Questions

**15.1.** What are typical access rights that may be granted or denied to a particular user for a particular file?

**15.2.** List and briefly define three classes of intruders.

**15.3.** In general terms, what are four means of authenticating a user's identity?

**15.4.** Briefly describe the difference between DAC and RBAC.

**15.5.** What types of programming languages are vulnerable to buffer overflows?

**15.6.** What are the two broad categories of defenses against buffer overflows?

**15.7.** List and briefly describe some of the defenses against buffer overflows that can be used when compiling new programs.

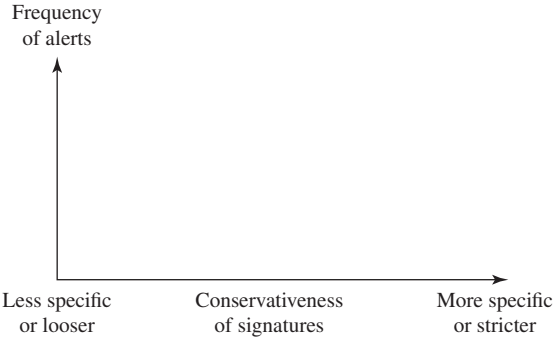
**15.8.** List and briefly describe some of the defenses against buffer overflows that can be implemented when running existing, vulnerable programs.

### Problems

**15.1.** State some threats that result from a process running with administrator or root privileges on a system.

**15.2.** In the context of an IDS, we define a false positive to be an alarm generated by an IDS in which the IDS alerts to a condition that is actually benign. A false negative occurs

when an IDS fails to generate an alarm when an alert-worthy condition is in effect. Using the following diagram, depict two curves that roughly indicate false positives and false negatives, respectively.




**15.3.** Rewrite the function shown in Figure 15.2a so it is no longer vulnerable to a stack buffer overflow.

**15.4.** For the DAC model discussed in Section 15.3, an alternative representation of the protection state is a directed graph. Each subject and each object in the protection state is represented by a node (a single node is used for an entity that is both subject and object). A directed line from a subject to an object indicates an access right, and the label on the link defines the access right.

- **a.** Draw a directed graph that corresponds to the access matrix of Figure 15.3a.
- **b.** Draw a directed graph that corresponds to the access matrix of Figure 15.4.
- **c.** Is there a one-to-one correspondence between the directed graph representation and the access matrix representation? Explain.

**15.5.** Set user (SetUID) and set group (SetGID) programs and scripts are a powerful mechanism provided by Unix to support "controlled invocation" to manage access to sensitive resources. However, precisely because of this, it is a potential security hole, and bugs in such programs have led to many compromises on Unix systems. Detail a command you could use to locate all set user or group scripts and programs on a Unix system, and how you might use this information.

**15.6.** User "abram" owns a directory, "myDir," containing an executable file called "myScript.sh" that he shares with users belonging to the group "myGroup" and others. User of "myGroup" may read and execute this file, but not delete it. They may not add other files to the directory. Others may only execute anything in "myDir" and "myScript.sh".What would appropriate ownerships and permissions for both "myDir" and "myScript.sh" look like? (Write answers in the form of "long listing" output.)

**15.7.** The UNIX command `ls` can be used to view the ownerships and permissions of a file. What will be the output of the command `ls -l myStore | cut -d' '-f1` if `myStore` is a file with protection mode 644(octal)? What command(s) will be used to give write permission to the users of the same group and to remove all permissions from others? What will be the output in this case?

**15.8.** In the traditional UNIX file access model, UNIX systems provide a default setting for newly created files and directories, which the owner may later change. The default is typically full access for the owner combined with one of the following: no access for group and other, read/execute access for group and none for other, or read/execute access for both group and other. Briefly discuss the advantages and disadvantages of each of these cases, including an example of a type of organization where each would be appropriate.

**15.9.** Consider user accounts on a system with a Web server configured to provide access to user Web areas. In general, this scheme uses a standard directory name, such as `public_html`, in a user's home directory. This acts as the user's Web area if it exists. However, to allow the Web server to access the pages in this directory, it must have at least search (execute) access to the user's home directory, read/execute access to the Web directory, and read access to any Web pages in it. Consider the interaction of this requirement with the cases you discussed for the preceding problem. What consequences does this requirement have? Note a Web server typically executes as a special user and in a group that is not shared with most users on the system. Are there some circumstances when running such a Web service is simply not appropriate? Explain.

**15.10.** Assume a system with $N$ job positions. For job position $i$, the number of individual users in that position is $U_i$ and the number of permissions required for the job position is $P_i$.

**a.** For a traditional DAC scheme, how many relationships between users and permissions must be defined?

**b.** For an RBAC scheme, how many relationships between users and permissions must be defined?

**15.11.** Why is logging important? What are its limitations as a security control? What are pros and cons of remote logging?

**15.12.** Consider an automated audit log analysis tool (e.g., swatch). Can you propose some rules which could be used to distinguish "suspicious activities" from normal user behavior on a system for some organization?

**15.13.** What are the advantages and disadvantages of using a file integrity checking tool (e.g., tripwire). This is a program which notifies the administrator of any changes to files on a regular basis? Consider issues such as which files you really only want to change rarely, which files may change more often, and which may change often. Discuss how this influences the configuration of the tool, especially as to which parts of the file system are scanned, and how much work monitoring its responses imposes on the administrator.

**15.14.** Some have argued that Unix/Linux systems reuse a small number of security features in many contexts across the system; while Windows systems provide a much larger number of more specifically targeted security features used in the appropriate contexts. This may be seen as a trade-off between simplicity verses lack of flexibility in the Unix/Linux approach against a better targeted but more complex and harder to correctly configure approach in Windows. Discuss this trade-off as it impacts on the security of these respective systems, and the load placed on administrators in managing their security.

CHAPTER 16

# Cloud and IoT Operating Systems


**16.1 Cloud Computing**
- Cloud Computing Elements
- Cloud Service Models
- Cloud Deployment Models
- Cloud Computing Reference Architecture

**16.2 Cloud Operating Systems**
- Infrastructure as a Service
- Requirements for Cloud Operating System
- General Architecture of a Cloud Operating System
- OpenStack

**16.3 The Internet of Things**
- Things on the Internet of Things
- Evolution
- Components of IoT-Enabled Devices
- IoT and Cloud Context

**16.4 IoT Operating Systems**
- Constrained Devices
- Requirements for an IoT OS
- IoT OS Architecture
- RIOT

**16.5 Key Terms and Review Questions**

### Learning Objectives

After studying this chapter, you should be able to:

- Present an overview of cloud computing concepts.
- List and define the principal cloud services.
- List and define the cloud deployment models.
- Explain the NIST cloud computing reference architecture.
- Describe the principal functions of a cloud operating system.
- Present an overview of OpenStack.
- Explain the scope of the Internet of Things.
- List and discuss the five principal components of IoT-enabled devices.
- Understand the relationship between cloud computing and IoT.
- Define constrained devices.
- Describe the principal functions of a cloud operating system.
- Present an overview of RIOT.

The two most significant developments in computing in recent years are cloud computing and the Internet of Things (IoT). In both cases, operating systems tailored to the specific requirements of these environments are evolving. This chapter begins with an overview of the concepts of cloud computing, followed by a discussion of cloud operating systems. Next the chapter examines the concepts of IoT, and closes with a discussion of IoT operating systems.

For further detail on the material on cloud computing and IoT in Sections 16.1 and 16.3, see [STAL16b].

## 16.1 CLOUD COMPUTING

There is an increasingly prominent trend in many organizations to move a substantial portion or even all information technology (IT) operations to an Internet-connected infrastructure known as enterprise cloud computing. This section provides an overview of cloud computing.

### Cloud Computing Elements

NIST defines cloud computing, in NIST SP-800-145 (*The NIST Definition of Cloud Computing*) as follows:

**Cloud computing:** A model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction. This cloud model promotes availability and is composed of five essential characteristics, three service models, and four deployment models.

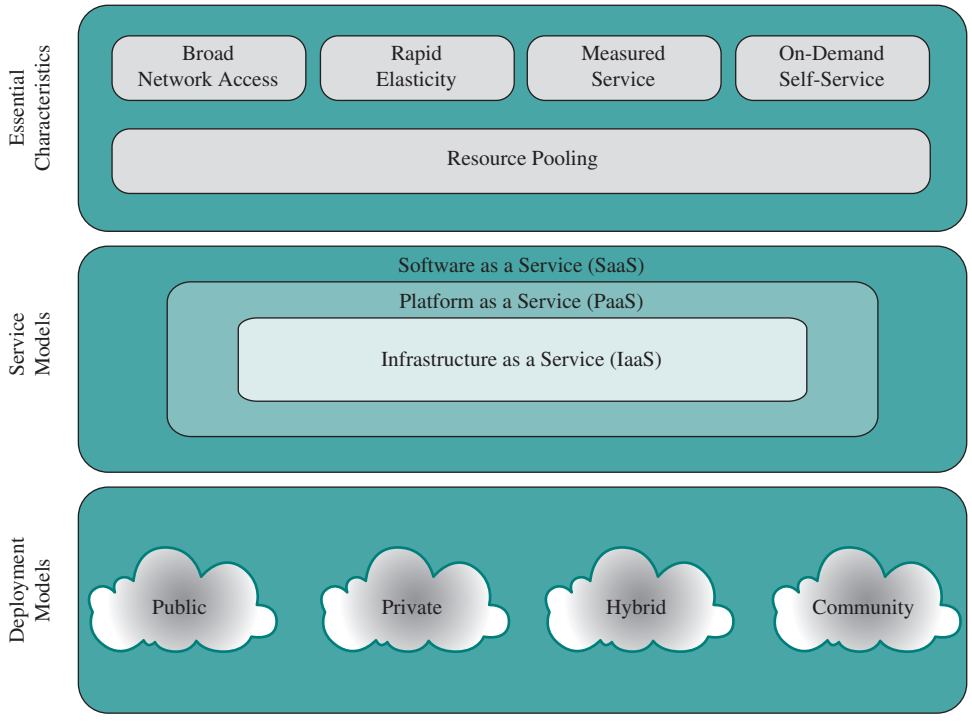



**Figure 16.1 Cloud Computing Elements**

The definition refers to various models and characteristics, whose relationship is illustrated in Figure 16.1. The essential characteristics of cloud computing include the following:

- **Broad network access:** Capabilities are available over the network and accessed through standard mechanisms that promote use by heterogeneous thin or thick client platforms (e.g., mobile phones, laptops, and tablets) as well as other traditional or cloud-based software services.
- **Rapid elasticity:** Cloud computing gives you the ability to expand and reduce resources according to your specific service requirement. For example, you may need a large number of server resources for the duration of a specific task. You can then release these resources upon completion of the task.
- **Measured service:** Cloud systems automatically control and optimize resource use by leveraging a metering capability at some level of abstraction appropriate to the type of service (e.g., storage, processing, bandwidth, and active user accounts). Resource usage can be monitored, controlled, and reported, providing transparency for both the provider and consumer of the utilized service.
- **On-demand self-service:** A cloud service consumer (CSC) can unilaterally provision computing capabilities, such as server time and network storage, as needed automatically without requiring human interaction with each service provider. Because the service is on demand, the resources are not permanent parts of your IT infrastructure.

- **Resource pooling:** The provider's computing resources are pooled to serve multiple CSCs using a multitenant model, with different physical and virtual resources dynamically assigned and reassigned according to consumer demand. There is a degree of location independence, in that the CSC generally has no control or knowledge of the exact location of the provided resources, but may be able to specify location at a higher level of abstraction (e.g., country, state, or datacenter). Examples of resources include storage, processing, memory, network bandwidth, and virtual machines (VMs). Even private clouds tend to pool resources between different parts of the same organization.

## Cloud Service Models

NIST defines three **service models**, which can be viewed as nested service alternatives: software as a service (SaaS), platform as a service (PaaS), and infrastructure as a service (IaaS).

***Software as a Service*** SaaS provides service to customers in the form of software, specifically application software, running on and accessible in the cloud. SaaS follows the familiar model of Web services, in this case applied to cloud resources. SaaS enables the customer to use the cloud provider's applications running on the provider's cloud infrastructure. The applications are accessible from various client devices through a simple interface, such as a Web browser. Instead of obtaining desktop and server licenses for software products it uses, an enterprise obtains the same functions from the cloud service. The use of SaaS avoids the complexity of software installation, maintenance, upgrades, and patches. Examples of services at this level are Google Gmail, Microsoft 365, Salesforce, Citrix GoToMeeting, and Cisco WebEx.

Common subscribers to SaaS are organizations that want to provide their employees with access to typical office productivity software, such as document management and email. Individuals also commonly use the SaaS model to acquire cloud resources. Typically, subscribers use specific applications on demand. The cloud provider also usually offers data-related features, such as automatic backup and data sharing between subscribers.

***Platform as a Service*** A PaaS cloud provides service to customers in the form of a platform on which the customer's applications can run. PaaS enables the customer to deploy onto the cloud infrastructure customer-created or acquired applications. A PaaS cloud provides useful software building blocks, plus a number of development tools, such as programming language tools, run-time environments, and other tools that assist in deploying new applications. In effect, PaaS is an operating system in the cloud. PaaS is useful for an organization that wants to develop new or tailored applications while paying for the needed computing resources only as needed, and only for as long as needed. AppEngine, Engine Yard, Heroku, Microsoft Azure, Force.com, and Apache Stratos are examples of PaaS.

***Infrastructure as a Service*** With IaaS, the customer has access to the resources of the underlying cloud infrastructure. The cloud service user does not manage or control the resources of the underlying cloud infrastructure, but has control over operating systems, deployed applications, and possibly limited control of select

networking components (e.g., host firewalls). IaaS provides virtual machines and other virtualized hardware and operating systems. IaaS offers the customer processing, storage, networks, and other fundamental computing resources so the customer is able to deploy and run arbitrary software, which can include operating systems and applications. IaaS enables customers to combine basic computing services, such as number crunching and data storage, to build highly adaptable computer systems.

Typically, customers are able to self-provision this infrastructure, using a Web-based graphical user interface that serves as an IT operations management console for the overall environment. API access to the infrastructure may also be offered as an option. Examples of IaaS are Amazon Elastic Compute Cloud (Amazon EC2), Microsoft Windows Azure, Google Compute Engine (GCE), and Rackspace.

Figure 16.2 compares the functions implemented by the cloud service provider for the three service models.

## Cloud Deployment Models

There is an increasingly prominent trend in many organizations to move a substantial portion or even all information technology (IT) operations to enterprise cloud computing. The organization is faced with a range of choices as to cloud ownership and management. Here, we look at the four most prominent deployment models for cloud computing.

***PUBLIC CLOUD*** A public cloud infrastructure is made available to the general public or a large industry group, and is owned by an organization selling cloud services.

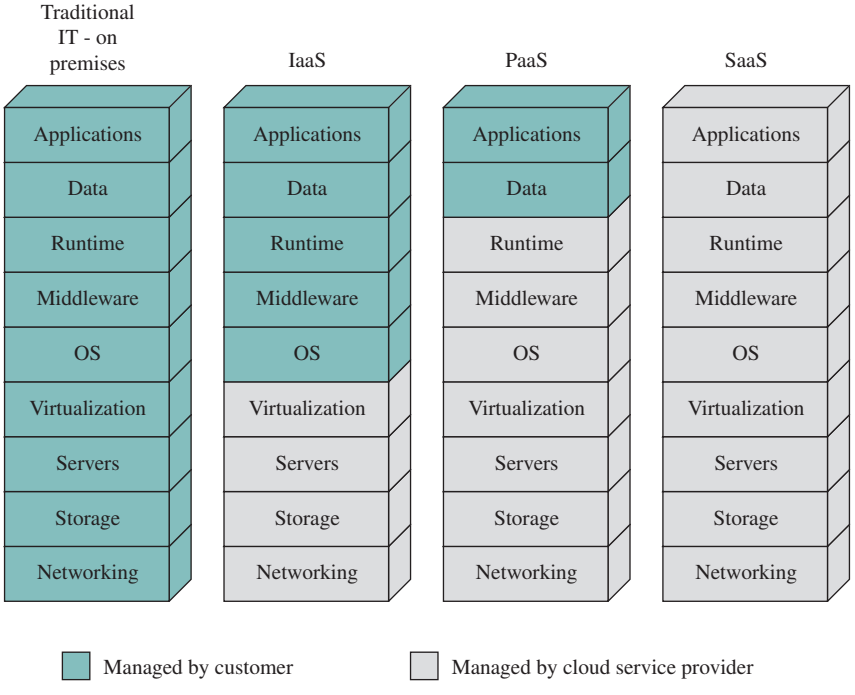



**Figure 16.2 Separation of Responsibilities in Cloud Operation**

The cloud provider is responsible both for the cloud infrastructure and for the control of data and operations within the cloud. A public cloud may be owned, managed, and operated by a business, academic, or government organization, or some combination of them. It exists on the premises of the cloud service provider.

In a public cloud model, all major components are outside the enterprise firewall, located in a multitenant infrastructure. Applications and storage are made available over the Internet via secured IP, and can be free or offered at a pay-per-usage fee. This type of cloud supplies easy-to-use consumer-type services, such as: Amazon and Google on-demand Web applications or capacity; Yahoo mail; and Facebook or LinkedIn social media providing free storage for photographs. While public clouds are inexpensive and scale to meet needs, they typically provide no or lower SLAs and may not offer the guarantees against data loss or corruption found with private or hybrid cloud offerings. The public cloud is appropriate for CSCs and entities not requiring the same levels of service that are expected within the firewall. Also, the public IaaS clouds do not necessarily provide for restrictions and compliance with privacy laws, which remain the responsibility of the subscriber or corporate end user. In many public clouds, the focus is on the CSC and small and medium sized businesses where pay-per-use pricing is available, often equating to pennies per gigabyte. Examples of services here might be picture and music sharing, laptop backup, or file sharing.

The major advantage of the public cloud is cost. A subscribing organization only pays for the services and resources it needs and can adjust these as needed. Further, the subscriber has greatly reduced management overhead. The principal concern is security. However, there are a number of public cloud providers that have demonstrated strong security controls and, in fact, such providers may have more resources and expertise to devote to security that would be available in a private cloud.

***Private Cloud*** A private cloud is implemented within the internal IT environment of the organization. The organization may choose to manage the cloud in house or contract the management function to a third party. Additionally, the cloud servers and storage devices may exist on premise or off premise.

Private clouds can deliver IaaS internally to employees or business units through an intranet or the Internet via a virtual private network (VPN), as well as software (applications) or storage as services to its branch offices. In both cases, private clouds are a way to leverage existing infrastructure, and deliver and chargeback for bundled or complete services from the privacy of the organization's network. Examples of services delivered through the private cloud include database on demand, email on demand, and storage on demand.

A key motivation for opting for a private cloud is security. A private cloud infrastructure offers tighter controls over the geographic location of data storage and other aspects of security. Other benefits include easy resource sharing and rapid deployment to organizational entities.

***Community Cloud*** A community cloud shares characteristics of private and public clouds. Like a private cloud, a community cloud has restricted access. Like a public cloud, the cloud resources are shared among a number of independent organizations. The organizations that share the community cloud have similar requirements and,

typically, a need to exchange data with each other. One example of an industry that is employing the community cloud concept is the health care industry. A community cloud can be implemented to comply with government privacy and other regulations. The community participants can exchange data in a controlled fashion.

The cloud infrastructure may be managed by the participating organizations or a third party, and may exist on premise or off premise. In this deployment model, the costs are spread over fewer users than a public cloud (but more than a private cloud), so only some of the cost savings potential of cloud computing are realized.

***HYBRID CLOUD*** The hybrid cloud infrastructure is a composition of two or more clouds (private, community, or public) that remain unique entities but are bound together by standardized or proprietary technology that enables data and application portability (e.g., cloud bursting for load balancing between clouds). With a hybrid cloud solution, sensitive information can be placed in a private area of the cloud, and less sensitive data can take advantage of the benefits of the public cloud.

A hybrid public/private cloud solution can be particularly attractive for smaller businesses. Many applications for which security concerns are less can be offloaded at considerable cost savings without committing the organization to moving more sensitive data and applications to the public cloud.

Table 16.1 lists some of the relative strengths and weaknesses of the four cloud deployment models.

## Cloud Computing Reference Architecture

NIST SP 500-292 (*NIST Cloud Computing Reference Architecture*) establishes reference architecture, described as follows:

> The NIST cloud computing reference architecture focuses on the requirements of "what" cloud services provide, not a "how to" design solution and implementation. The reference architecture is intended to facilitate the understanding of the operational intricacies in cloud computing. It does not represent the system architecture of a specific cloud computing system; instead it is a tool for describing, discussing, and developing a system-specific architecture using a common framework of reference.

**Table 16.1** Comparison of Cloud Deployment Models

| | Private | Community | Public | Hybrid |
|---|---|---|---|---|
| **Scalability** | Limited | Limited | Very high | Very high |
| **Security** | Most secure option | Very secure | Moderately secure | Very secure |
| **Performance** | Very good | Very good | Low to medium | Good |
| **Reliability** | Very high | Very high | Medium | Medium to high |
| **Cost** | High | Medium | Low | Medium |

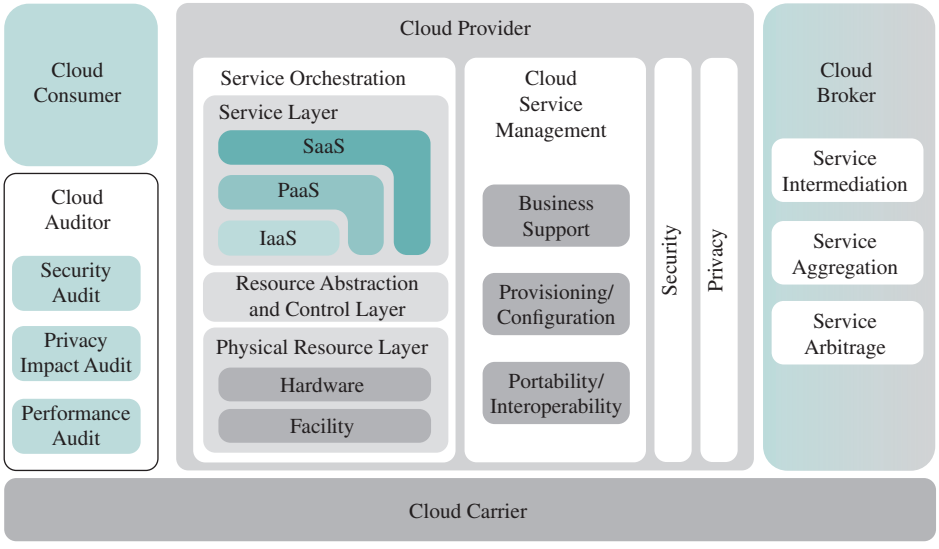



**Figure 16.3 NIST Cloud Computing Reference Architecture**

NIST developed the reference architecture with the following objectives in mind:

- To illustrate and understand the various cloud services in the context of an overall cloud computing conceptual model
- To provide a technical reference for CSCs to understand, discuss, categorize, and compare cloud services
- To facilitate the analysis of candidate standards for security, interoperability, and portability and reference implementations.

The reference architecture, depicted in Figure 16.3, defines five major actors in terms of the roles and responsibilities:

- **Cloud service consumer (CSC):** A person or organization that maintains a business relationship with, and uses service from, cloud providers.
- **Cloud service provider (CSP):** A person, organization, or entity responsible for making a service available to interested parties.
- **Cloud auditor:** A party that can conduct independent assessment of cloud services, information system operations, performance, and security of the cloud implementation.
- **Cloud broker:** An entity that manages the use, performance, and delivery of cloud services, and negotiates relationships between CSPs and cloud consumers.
- **Cloud carrier:** An intermediary that provides connectivity and transport of cloud services from CSPs to cloud consumers.

The roles of the cloud consumer and provider have already been discussed. To summarize, a **cloud service provider** can provide one or more of the cloud services

to meet IT and business requirements of **cloud service consumers**. For each of the three service models (SaaS, PaaS, IaaS), the CSP provides the storage and processing facilities needed to support that service model, together with a cloud interface for cloud service consumers. For SaaS, the CSP deploys, configures, maintains, and updates the operation of the software applications on a cloud infrastructure so that the services are provisioned at the expected service levels to cloud consumers. The consumers of SaaS can be organizations that provide their members with access to software applications, end users who directly use software applications, or software application administrators who configure applications for end users.

For PaaS, the CSP manages the computing infrastructure for the platform and runs the cloud software that provides the components of the platform, such as runtime software execution stacks, databases, and other middleware components. Cloud consumers of PaaS can employ the tools and execution resources provided by CSPs to develop, test, deploy, and manage the applications hosted in a cloud environment.

For IaaS, the CSP acquires the physical computing resources underlying the service, including the servers, networks, storage, and hosting infrastructure. The IaaS CSC in turn uses these computing resources, such as a virtual machine, for their fundamental computing needs.

The **cloud carrier** is a networking facility that provides connectivity and transport of cloud services between cloud consumers and CSPs. Typically, a CSP will set up service level agreements (SLAs) with a cloud carrier to provide services consistent with the level of SLAs offered to cloud consumers, and may require the cloud carrier to provide dedicated and secure connections between cloud consumers and CSPs.

A **cloud broker** is useful when cloud services are too complex for a cloud consumer to easily manage. A cloud broker can offer three areas of support:

- **Service intermediation:** These are value-added services, such as identity management, performance reporting, and enhanced security.
- **Service aggregation:** The broker combines multiple cloud services to meet consumer needs not specifically addressed by a single CSP, or to optimize performance or minimize cost.
- **Service arbitrage:** This is similar to service aggregation except that the services being aggregated are not fixed. Service arbitrage means a broker has the flexibility to choose services from multiple agencies. The cloud broker, for example, can use a credit-scoring service to measure and select an agency with the best score.

A **cloud auditor** can evaluate the services provided by a CSP in terms of security controls, privacy impact, performance, and so on. The auditor is an independent entity that can assure that the CSP conforms to a set of standards.

Figure 16.4 illustrates the interactions between the actors. A cloud consumer may request cloud services from a cloud provider directly or via a cloud broker. A cloud auditor conducts independent audits and may contact the others to collect necessary information. This figure shows that cloud networking issues involve three separate types of networks. For a cloud producer, the network architecture is that of a typical large datacenter, which consists of racks of high-performance servers and storage devices, interconnected with high-speed top-of-rack Ethernet switches. The

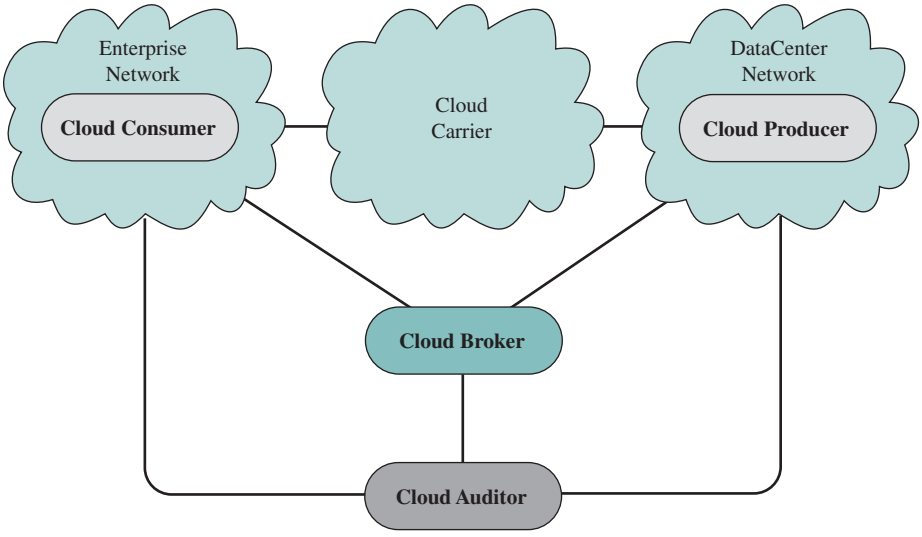



**Figure 16.4 Interactions Between Actors in Cloud Computing**

concerns in this context focus on virtual machine placement and movement, load balancing, and availability issues. The enterprise network is likely to have a quite different architecture, typically including a number of LANs, servers, workstations, PCs, and mobile devices, with a broad range of network performance, security, and management issues. The concern of both producer and consumer with respect to the cloud carrier, which is shared with many users, is the ability to create virtual networks, with appropriate SLA and security guarantees.

## 16.2 CLOUD OPERATING SYSTEMS

The term *cloud operating system* refers to a distributed operating system that is designed to run in the cloud service provider's datacenter and is used to manage high-performance servers, network, and storage resources and provide those services to cloud service users. In essence, the cloud OS is the software that implements IaaS.

It is important to note the distinction between a cloud OS and PaaS. As discussed in Section 16.1, PaaS is a platform for executing customer applications. PaaS enables the customer to deploy onto the cloud infrastructure customer-created or acquired applications. It provides useful software building blocks, plus a number of development tools, such as programming language tools, run-time environments, and other tools that assist in deploying new applications. In effect, PaaS is a user-visible operating system in the cloud. In contrast, a cloud OS is distinct from the operating system run by the cloud service user on cloud virtual machines. Because the provider provides an IaaS, the user's OS runs on the cloud infrastructure. The cloud OS manages the provision of these services and may provide some tools to the user, but is otherwise transparent to the cloud service user.

This section begins with a look in more detail at the IaaS model, then examines the characteristics of a cloud OS suitable for implementing IaaS. Finally, we look at the most important open-source cloud OS, OpenStack.

## Infrastructure as a Service

IaaS represents the infrastructure layer composed mostly of virtualized environments providing computing, storage, and network resources. Hypervisors run a collection of virtual machines on real IT resources and provide virtualized versions of these resources to cloud service users. The users are free to install any OS and application environment they want on these virtualized resources. The provider is responsible for enabling access to the virtualized resources, provisioning the quantity of resources needed, and managing the resources. The CSC does not manage or control the underlying cloud infrastructure but has control over operating systems, storage, deployed applications, and possibly limited control of select networking components (e.g., host firewalls).

Note IaaS is not simply another name for a virtualized environment. Although virtualization is a key enabling technology for cloud computing, it is only when the basic environment is extended to incorporate advanced management tools (for moving virtual machines, for monitoring and managing availability, recovery, lifecycle management, self service, chargeback, etc.) that a virtualized environment becomes capable of satisfying the essential IaaS characteristics.

Figure 16.5 illustrates the principal features of IaaS as seen by the CSC. Three interactions, which are shown as numbered lines with double arrowheads in the figure are important.

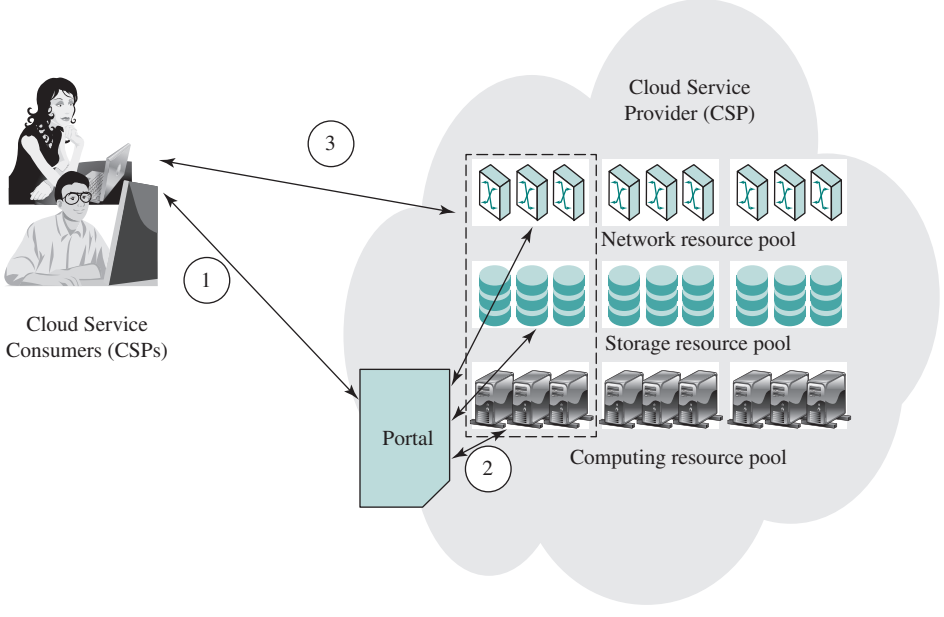



**Figure 16.5 IaaS Conceptual Framework**

The first interaction is between the CSC and a CSP portal to provide consumer access to the cloud resources with appropriate security mechanisms. It encompasses the following actions:

1. CSC accesses the IaaS service with appropriate security mechanisms and queries the CSP portal to retrieve the list of supported functions (e.g., infrastructure templates) related to the infrastructure.
2. CSC selects the appropriate infrastructure template from the query results and requests the CSP to create an infrastructure based on the selection.
3. CSC manages and monitors the created infrastructure during its lifecycle. This includes, but is not limited to:

   —**assign**: start IaaS by allocating to the service the available resources as identified by configuration (e.g., create, initiate, start, enable, and power-on)

   —**modify**: change the amount of resource being in-use according to the demand (e.g., update, add, enable, and disable)

   —**release**: close the IaaS service by making available the resource being in-use by the service (e.g., delete, shutdown, disable, and power-off).

   The second interaction includes the following actions:

1. The CSC has selected the template or configured a specific VM and/or physical host.
2. The CSC has selected the storage resources (such as block, file, and object storage) then attached them via their computing capabilities or used them directly.
3. The CSC has selected the network connectivity services, such as the IP address, VLAN, firewall and load balance, then applied them to the related computing and/or storage capabilities.
4. The CSC confirmed the service-level agreements (SLAs) and charge model with selected computing, storage and network connectivity services provided by the CSP.

Once the CSP grants access and configures the resources for the CSC, the third interaction proceeds as follows:

1. The CSC manages and monitors computing, storage, and network capabilities with arbitrary applications.
2. The CSP configures, deploys, and maintains hypervisors and storage resources.
3. The CSP establishes, configures, delivers, and maintains network connectivity to the CSC.
4. The CSP provides security infrastructure to the CSC.

## Requirements for Cloud Operating System

A cloud OS must manage and provides CSCs with access to an IaaS environment. A useful way to define the requirements for a cloud OS is to look at the functions that must be supported for IaaS. ITU-T Recommendation Y.3513 (*Cloud computing—Functional requirements of Infrastructure as a Service*, August 2014) lists the functional

**Table 16.2** CSP Functional Requirements for IaaS

| Scope | Requirements |
|---|---|
| General | —Provide IaaS functions, such as a composition of processing, storage, and networking resources with service logic, specific SLAs, and charging model.<br>—Provides status information about the infrastructure in response to CSC queries.<br>—Provide template to the CSC, related to instantiation of infrastructure, which allows to provision processing, storage, and networking resources that could be implemented based on the configuration. |
| Computing service | —Provide VMs based on template or on configuration specified by CSC.<br>—Provide CSC with operations handling mechanisms, including, create, delete, start, shutdown, suspend, restore, hibernate, and wakeup.<br>—Provide VM with following functions: migration from one host to another; scaling, including configuration changes (e.g., processor, memory, bandwidth increased or decreased) and component changes (VM added or removed); snapshot; clone; and backup. |
| Storage service | —Provide storage functions, such as block level storage, file level storage, and object-based storage.<br>—Provide with operations handling mechanisms, such as create, attach, detach, query, and delete a volume of storage at either block level or file-system level, write, read, and delete data for a given storage.<br>—Provide following functions: storage migration; snapshot; and backup. |
| Network service | —Provide network functions, such as IP address, VLAN, virtual switch, load balance, and firewall. |

requirements for the CSP to provide an IaaS, and these provide a useful summary of the scope of the cloud OS (see Table 16.2).

## General Architecture of a Cloud Operating System

A central feature of a cloud OS is that it exploits virtualization technology to provide compute, storage, and network resources via an IaaS environment. Figure 16.6 illustrates the concepts involved at a conceptual level. We examine each of the major elements of this figure in turn.

***VIRTUALIZATION*** Virtual machine technology, as discussed in Chapter 14, enables migration of dedicated application, network, and storage servers to commercial off-the-shelf (COTS) servers. In traditional networks of servers and storage devices, all devices are deployed on private platforms. All elements are enclosed boxes, and hardware cannot be shared. Each device requires additional hardware for increased capacity, but this hardware is idle when the system is running below capacity. With virtualization, however, compute, storage, and network elements are independent applications that are flexibly deployed on a unified platform comprising standard servers, storage devices, and switches. In this way, software and hardware are decoupled, and capacity for each application is increased or decreased by adding or reducing virtual resources.

In a cloud environment, the hardware resources are standard servers, network-attached storage, and switches (generally Ethernet switches). Hypervisors executing on these hardware devices provide the support for developing virtual machines that deliver virtual computing, storage, and network resources.

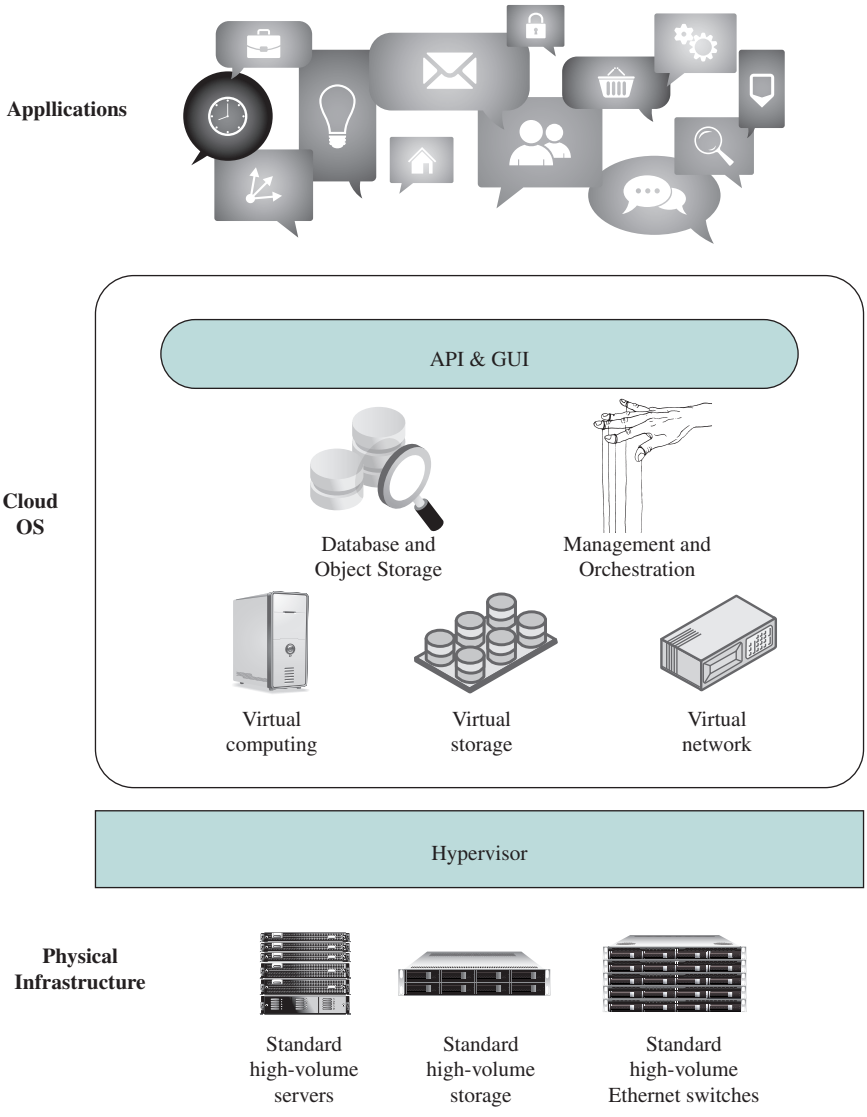



**Figure 16.6 Cloud Operating System Concept**

The CSP maintains total control over the physical hardware and administrative control over the hypervisor layer. The consumer may make requests to the cloud to create and manage new VMs, but these requests are honored only if they conform to the provider's policies over resource assignment. Through the hypervisor, the provider will typically provide interfaces to networking features (such as virtual network switches) that consumers may use to configure custom virtual networks within the provider's infrastructure. The consumer will typically maintain complete control over the operation of the guest operating system in each VM, and all software layers above it.

***VIRTUAL COMPUTING*** The virtual computing component of the cloud OS controls virtual machines within the IaaS cloud computing environment. The OS views each VM as a compute instance, whose principal elements include the following:

- **CPU/memory**: A COTS processor, with main memory, that executes the code of the VM.
- **Internal storage**: Nonvolatile storage housed in the same physical structure as the processor, such as flash memory.
- **Accelerator**: Accelerator functions for security, networking, and packet processing may also be included. These virtual accelerator functions correspond to accelerator hardware associated with the physical server.
- **External storage with storage controller**: Access to secondary memory devices. These are memory devices attached to a physical server, in contrast to network attached storage (NAS).

The virtual computing component also includes software for interacting with other components of the cloud OS and with the API and GUI interfaces to applications and users.

***VIRTUAL STORAGE*** The virtual storage component of the cloud OS provides data storage services for the cloud infrastructure. This component includes the following services:

- Stores cloud management information, including virtual machine and virtual network definitions.
- Provides working space to applications and workloads running in the cloud environment.
- Provides storage-related mechanisms, including workload migration, automated backups, integrated version control, and optimized application-specific storage mechanisms.

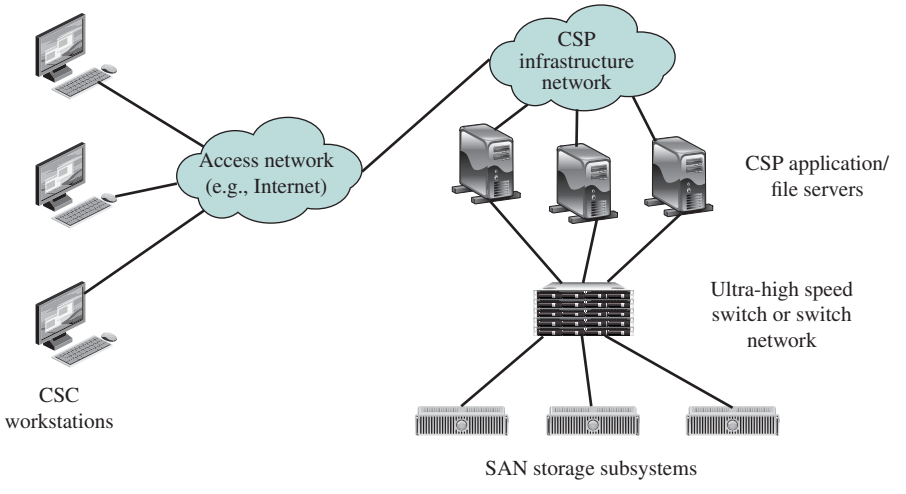
For the CSC, this component provides block storage, often with additional functionality and implemented as a collection of virtual disk drives within a hypervisor. This component must isolate the stored data of the different CSC workloads.

Storage comes in the following topologies:

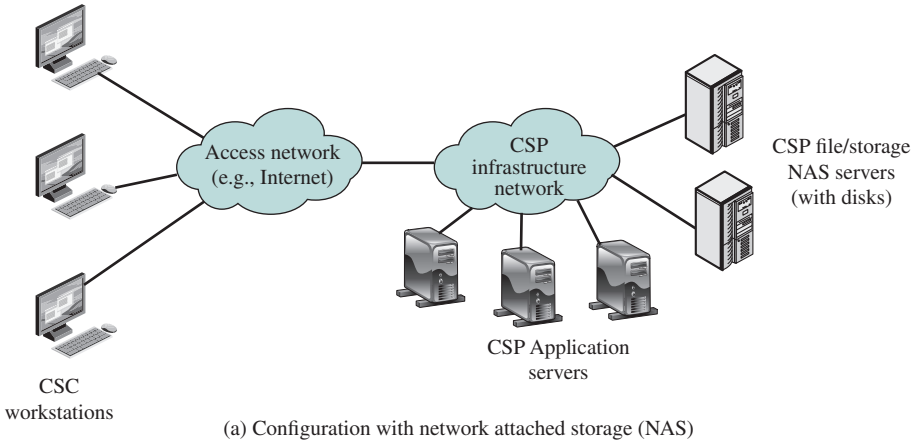
- **Direct Attached Storage (DAS)**: While typically associated with internal server hard drives, a better way of thinking about DAS is that it is captive to the server to which it is attached.
- **Storage Area Network (SAN)**: A SAN is a dedicated network that provides access to various types of storage devices including tape libraries, optical jukeboxes, and disk arrays. To servers and other devices in the network, a SAN's storage devices look like locally attached devices. A disk block-based storage technology, SAN is probably the most pervasive form of storage for very large datacenters and has been a de facto staple as it relates to database intensive applications. These applications require shareable storage, large bandwidth, and support for the distances from rack to rack within the datacenter.

- **Network Attached Storage (NAS)**: NAS systems are networked appliances that contain one or more hard drives that can be shared with multiple, heterogeneous computers. Their specialized role within networks is to store and serve files. NAS disk drives typically support built-in data protection mechanisms including redundant storage containers or redundant arrays of independent disks (RAID). NAS enables file serving responsibilities to be separated from other servers on the network and typically provide faster data access than traditional file servers.

Figure 16.7 illustrates the distinction between SAN and NAS. A CSP will typically implement a SAN within the cloud infrastructure and may also make use of NAS. The cloud OS should be able to accommodate both topologies and provide transparent access to the CSC, who need not know the internal storage topological structure of the cloud.




(b) Configuration with storage area network (SAN)




(a) Configuration with network attached storage (NAS)

**Figure 16.7 SAN and NAS in a Cloud Infrastructure**

***Virtual Network*** The virtual network component of the cloud OS provides networking services for the cloud infrastructure. It enables connectivity among the computer, storage, and other elements of the infrastructure as well as with the broader environment outside of the cloud. This component also provides CSCs with the ability to create virtual networks among VMs and network appliances.

In addition to basic connectivity services, the virtual network component may include the following services and functions:

- An infrastructure addressing scheme (there may well be more than one scheme) with address allocation and management.
- A routing process that can relate infrastructure addresses to routes through the infrastructure network topology.
- A bandwidth allocation process, including priority and quality of service (QoS) features.
- Support network functions such virtual LAN (VLAN), load balancing, and firewalls.

***Data Structure Management*** A cloud OS provides not only raw storage capability but services for accessing data in a structured fashion. The three common structures supported by the cloud OS and the IaaS are block, file, and object.

With **block storage**, data are stored on hard disk as fixed-size blocks. Each block is a contiguous sequence of bytes. SANs provide block storage access and it is also used with DAS. Block storage lends itself to snapshot capabilities and to resiliency schemes such as mirroring. Typically, SAN controllers will utilize a copy-on-write mechanism to keep the local copy and the mirror volume synchronously mirrored.

**File-based storage** systems are typically synonymous with NAS and consist of a storage array, some type of controller and operating system, and one to several networked-storage protocols. The most widely deployed protocol for large-scale virtualization environments is Network File System (NFS). The data are stored on hard disk as files in a directory structure. These devices have their own processors and OS and are accessed using a standard protocol over a TCP/IP network. Common protocols include:

- NFS (Network File System): NFS is common in Unix- and Linux-based networks.
- SMB (Server Message Block) or CIFS (Common Internet File System): SMB (or CIFS) is commonly used in Windows-based networks.
- HTTP (Hypertext Transfer Protocol): HTTP is the protocol you most commonly use when using a Web browser.

NAS appliances are relatively easy to deploy, and client access is straightforward using the common protocols. Servers and the NAS appliances are all connected over a shared TCP/IP network, and the data stored on NAS appliances can be accessed by virtually any server, regardless of the server's OS.

One of the advantages of file-based storage is the ability to treat a file as a block device or disk drive. Files can be easily appended to in order to create larger virtual drives. And files can be easily replicated to other locations. One of the disadvantages

of working with file-based storage is the difficulty of rapidly cloning an existing virtual disk image into a new image. Also, a NAS-based storage system is typically slower than DAS or SAN.

In contrast to file-based storage, **object storage** uses a flat address space instead of the hierarchical, directory-based scheme [MESN03, TAUR12]. Each object consists of a container that stores both the data and also metadata describing the data, such as date, size, and format. Each object is assigned a unique object ID and can be addressed directly using that ID. The object ID is stored in a database or application and is used to reference objects in one or more containers. Object storage is widely used in cloud systems.

The data in an object-based storage system is typically accessed using HTTP using a Web browser or directly through an API. The flat address space in an object-based storage system enables simplicity and massive scalability, but the data in these systems typically can't be modified. One of the key advantages of object storage is the ability to directly couple unique methods or security implementations with the actual data as opposed to having such capabilities come from an adjacent system or service.

Figure 16.8 contrasts block, file, and object storage.

***MANAGEMENT AND ORCHESTRATION*** The management and orchestration (MANO) component of a cloud OS has as its primary function the control of the IaaS environment. NIST defines this function as the composition of system components to support the cloud provider activities in arrangement, coordination, and management of computing resources in order to provide cloud services to cloud consumers (*US Government Cloud Computing Technology Roadmap Volume I*. SP 500-293, October 2014).

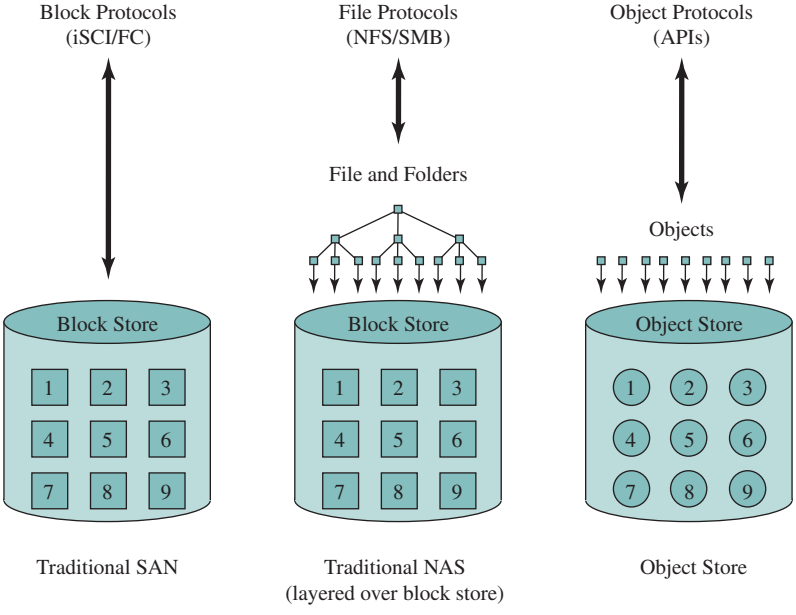



**Figure 16.8 Block, File, and Object Storage**

MANO encompasses the following functions and services:

- **Orchestration**: Responsible for installing and configuring new network services (NS); NS lifecycle management; global resource management; and validation and authorization of resource requests.
- **VM manager**: Oversees lifecycle management of VM instances.
- **Infrastructure manager**: Controls and manages the interaction of a VM with computing, storage, and network resources under its authority, as well as their virtualization.

## OpenStack

OpenStack is an open-source software project of the OpenStack Foundation that aims to produce an open-source cloud operating system [ROSA14, SEFR12]. The principal objective is to enable creating and managing huge groups of virtual private servers in a cloud computing environment. OpenStack is embedded, to one degree or another, into datacenter infrastructure and cloud computing products offered by Cisco, IBM, Hewlett-Packard, and other vendors. It provides multitenant IaaS, and aims to meets the needs of public and private clouds regardless of size, by being simple to implement and massively scalable.

The OpenStack OS consists of a number of independent modules, each of which has a project name and a functional name. The modular structure is easy to scale out and provides a commonly used set of core services. Typically the components are configured together to provide a comprehensive IaaS capability. However, the modular design is such that the components are generally capable of being used independently.

To understand OpenStack it is useful to distinguish three types of storage that are part of the OpenStack environment:

- **Network block storage**: This type of storage makes data persistent by mounting one or more network block storage devices. It represents an allocation of persistent, readable, and writable block storage that could be utilized as the root disk for a VM instance, or as secondary storage that could be attached and/or detached from a VM instance.
- **Object storage**: Object storage is the persistent storage of objects on a network. From the object storage viewpoints, the objects are arbitrary, unstructured data. The storage objects are generally write-once, read-many. This is reliable storage with redundant copies. Access control lists determine visibility for the owner and authorized users.
- **Virtual machine image storage**: VM images are disk images that can be booted on a VM by a hypervisor. It can be a single image that contains the boot loader, kernel and operating system, or the boot loader and kernel can be separated. This type of storage allows for custom kernels and resizable images.

Figure 16.9, from [CALL15], illustrates the OpenStack conceptual architecture, with the interaction among the principal software components. Table 16.3 defines the functional interaction; the leftmost column indicates the source of an action, while the

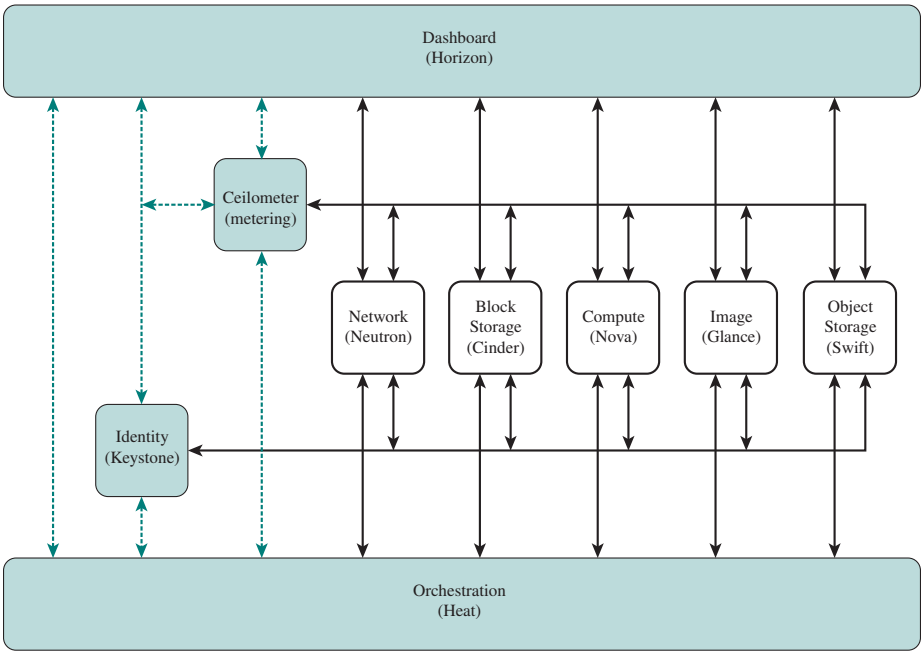



**Figure 16.9** **OpenStack High Level Architecture**

**Table 16.3** OpenStack Functional Interactions

| | **Glance (image)** | **Horizon (dashboard)** | **Nova (compute)** | **Swift (object storage)** | **Cinder (block storage)** | **Neutron (network)** |
|---|---|---|---|---|---|---|
| Glance (image) | | | sends images to | stores disk files | stores blocks on | |
| Horizon (dashboard) | provides UI | | provides UI | provides UI | provides UI | provides UI |
| Nova (compute) | receives images from | | | stores volumes on | | |
| Swift (object storage) | supplies disk files | | provides volumes for | | | |
| Cinder (block storage) | provides volumes for | | | | | |
| Neutron (network) | | | | | | |
| Keystone (identity) | authenticates with | authenticates with | authenticates with | authenticates with | authenticates with | authenticates with |
| Heat (orchestration) | orchestrates | orchestrates | orchestrates | orchestrates | orchestrates | orchestrates |
| Trove (database) | | | provides instances to | | | |
| Ceilometer | monitors | monitors | monitors | monitors | monitors | monitors |
| VM | retrieves image files | | | | | |

(*Continued*)

**Table 16.3** OpenStack Functional Interactions (*continued*)

| | **Keystone (identity)** | **Heat (orchestration)** | **Trove (database)** | **ceilometer** | **VM** |
|---|---|---|---|---|---|
| Glance (image) | authenticates with | | | | supplies image files |
| Horizon (dashboard) | authenticates with | provides UI | provides UI | provides UI | provides UI |
| Nova (compute) | authenticates with | | receives instances from | | launches volume |
| Swift (object storage) | authenticates with | | | | |
| Cinder (block storage) | authenticates with | | | | |
| Neutron (network) | authenticates with | | | | provides network connectivity |
| Keystone (identity) | | authenticates with | authenticates with | authenticates with | authenticates with |
| Heat (orchestration) | authenticates with | | orchestrates | orchestrates | |
| Trove (database) | authenticates with | | | | |
| Ceilometer | authenticates with | monitors | monitors | | |
| VM | authenticates with | | | | |

topmost row indicates the destination of an action. These components can be roughly divided into five functional groups:

- **Computing**: Compute (Nova), Image (Glance)
- **Networking**: Network (Neutron)
- **Storing**: Object Storage (Swift), Block Storage (Cinder)
- **Shared Services**: Security (Keystone), Dashboard (Keystone), Metering (Ceilometer), Orchestration (Heat)
- **Other Optional Services**: Discussed subsequently.

We now examine each of the components listed in the first four bullet items, then briefly discuss other components.

***Compute (Nova)*** Nova is the management software that controls virtual machines within the IaaS cloud computing platform. It manages the lifecycle of compute instances in an OpenStack environment. Responsibilities include spawning, scheduling, and decommissioning of machines on demand. Thus,

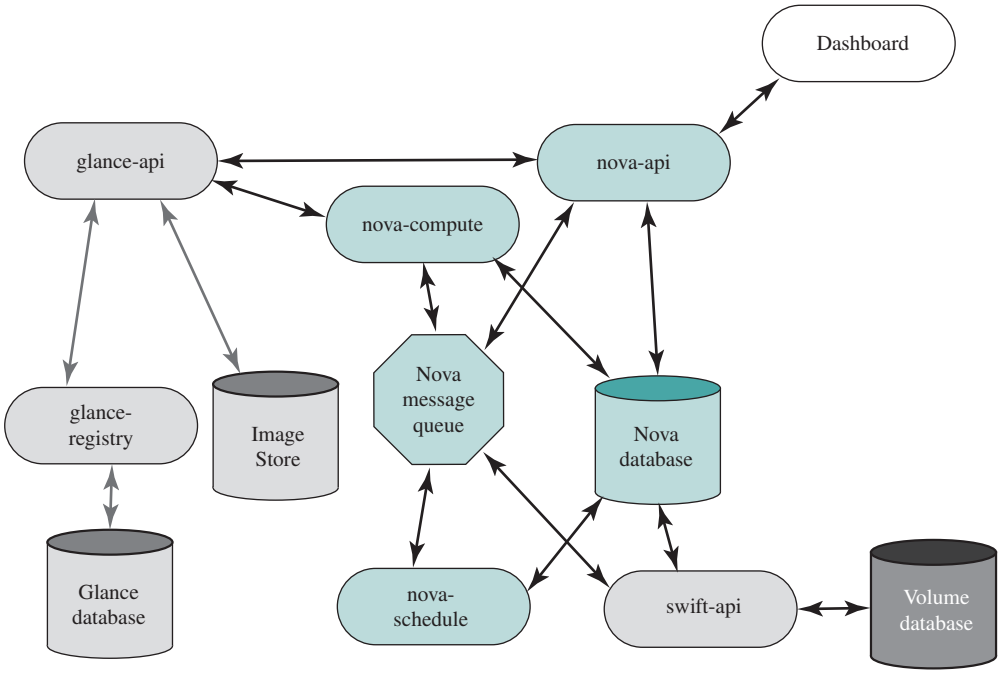



**Figure 16.10** **Nova Logical Architecture**

Nova enables enterprises and service providers to offer on-demand computing resources, by provisioning and managing large networks of virtual machines. Nova is similar in scope to Amazon Elastic Compute Cloud (EC2). Nova is capable of interacting with various open-source and commercial hypervisors. Nova does not include any virtualization software; rather, it defines drivers that interact with underlying virtualization mechanisms that run on the host operating system, and it provides functionality over a Web API. Thus, Nova enables the management of large networks of virtual machines and supports redundant and scalable architectures. It includes instances management for servers, networks, and access control. Nova requires no prerequisite hardware and is completely independent of the hypervisor.

Nova consists of five main components (see Figure 16.10):

- **API server**: This is the external interface to the Dashboard for users and applications.
- **Message queue**: Nova components exchange info through the queue (actions) and database (information) to carry out API requests. The message queue implements the mechanism for dispatching the exchanged instructions to facilitate communication.
- **Compute controller**: Handles the lifecycle of virtual machine instances, it is responsible for creating and manipulating virtual servers. It interacts with Glance.

- **Database**: Stores most of the build-time and run-time state for a cloud infrastructure. This includes the instance types that are available for use, instances in use, networks available and projects.
- **Scheduler**: Takes virtual machine instance requests and determines where (on which compute server host) they should be executed.

Note several components interact with Swift. Swift manages the creation, attaching and detaching of volumes to compute instances.

***IMAGE (GLANCE)*** Glance is a lookup and retrieval system for virtual machine (VM) disk images. It provides services for discovering, registering, and retrieving virtual images through an API. It also provides an SQL-style interface for queries for information on the images hosted on various storage systems. OpenStack Compute makes use of this during instance provisioning.

***NETWORK (NEUTRON)*** Neutron is an OpenStack project designed to provide network connectivity as a service between interface devices managed by other OpenStack services (e.g., NOVA). A Neutron server provides a Web server that exposes the Neutron API and passes all Web service calls to the Neutron plugin for processing. In essence, Neutron provides a consistent set of network services for use by other elements, such as virtual machines, systems management modules, and other networks. Users interact with networking functions via the Dashboard GUI; other management systems and networks interact with networking services using Neutron's API.

Currently Neutron implements Layer 2 virtual LANs (VLANs) and IP-based (Layer 3) routers. There are also extensions to support firewalls, load balancers, and IPSec virtual private networks (VPNs).

Three key benefits of using Neutron are the following [PARK13]:

- By using a consistent approach to networking for multiple types of virtual machines, Neutron helps providers operate efficiently in heterogeneous environments, which is frequently the requirement in service provider systems.
- By supplying a consistent set of APIs for plugging in a variety of physical network underlays, providers gain flexibility in altering the design of their underlying physical network while keeping the cloud service logically intact.
- Orchestration and system management suppliers, as well as providers' own technical teams, can use the Neutron API to integrate management of the network for the cloud with multiple higher-level service management tasks. This offers a range of opportunities, including service-level agreement monitoring, as well as integration into automation platforms like catalogs and portals for dynamic management of customer clouds.

***OBJECT STORAGE (SWIFT)*** Swift is a distributed object store that creates a redundant and scalable storage space of up to multiple petabytes of data. Object storage does not present a traditional file system, but rather a distributed storage system for static data such as virtual machine images, photo storage, email storage, backups, and archives. It can be used by Cinder components to back up VM volumes.

***Block Storage (Cinder)*** Cinder provides persistent block storage (or volumes) to guest virtual machines. Cinder can use Swift to back up the VM's volumes. Cinder also interacts with Nova, providing volumes for its instances, allowing through its API the manipulation of volumes, volume types, and volume snapshots.

***Identity (Keystone)*** Keystone provides the shared security services essential for a functioning cloud computing infrastructure. It provides main services:

- **Identity**: This is user information authentication. This information defines a user's role and permissions within a project, and is the basis for a role-based access control (RBAC) mechanism.
- **Token**: After a username/password log on, a token is assigned and used for access control. OpenStack services retain tokens and use them to query Keystone during operations.
- **Service catalog**: OpenStack service endpoints are registered with Keystone to create a service catalog. A client for a service connects to Keystone, and determines an endpoint to call based on the returned catalog.
- **Policies**: This service enforces different user access levels.

Figure 16.11 illustrates the way in which Keystone interacts with other OpenStack components to launch a new virtual machine.

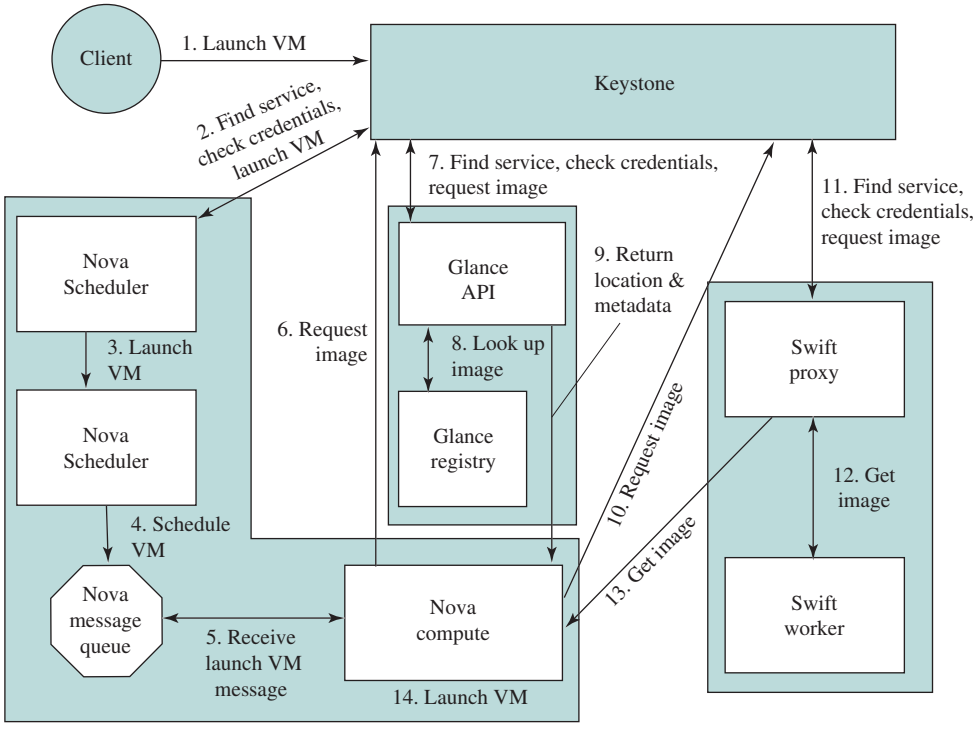



**Figure 16.11 Launching a Virtual Machine**

***DASHBOARD (HORIZON)*** The dashboard is the Web user interface for cloud infrastructure management. It provides administrators and users a graphical interface to access, provision, and automate cloud-based resources. The extensible design makes it easy to plug in and expose third-party products and services, such as billing, monitoring, and additional management tools. It interacts with the APIs of all the other software components. For example, Horizon enables a user or application to launch an instance, assign IP addresses, and configure access controls.

***MONITOR (CEILOMETER)*** Ceilometer provides a configurable collection of functions for metering data, such as processor and storage usage and network traffic. This is a unique point of contact for billing, benchmarking, scalability, and statistical purposes.

***ORCHESTRATION (HEAT)*** Heat orchestrates multiple cloud applications. The objective is to create a human- and machine-accessible service for managing the entire lifecycle of infrastructure and applications within OpenStack clouds. It implements an orchestration engine to launch multiple composite cloud applications based on templates in the form of text files that can be treated like code. Heat is compatible with Amazon Cloudformation, which is becoming a de facto standard.

***OTHER OPTIONAL SERVICES*** As the OpenStack project evolves, new components are being developed by various OpenStack members. As of this writing, the following components are available or in development:

- **Database (Trove)**: Trove is a database-as-a-service that provisions relational and nonrelational database engines. By default, Trove uses MySQL as its relational database management system, enabling the other services to store configurations and management information.
- **Messaging service (Zaqar)**: Zaqar is a multitenant cloud messaging service for Web and mobile developers. The service features an API that developers can use to send messages between various components of their SaaS and mobile applications, by using a variety of communication patterns. Underlying this API is an efficient messaging engine designed with scalability and security in mind.
- **Key management (Barbican)**: Barbican provides an API for the secure storage, provisioning, and management of secret values such as passwords, encryption keys, and X.509 Certificates.
- **Governance (Congress)**: Congress provides policy as a service across any collection of cloud services in order to offer governance and compliance for dynamic infrastructures.
- **Elastic map reduce (Sahara)**: Sahara aims to provide users with simple means to provision Hadoop clusters by specifying several parameters such as Hadoop version, cluster topology, and nodes hardware details. After a user fills all the parameters, Sahara deploys the cluster. Sahara also provides means to scale an already provisioned cluster by adding and removing worker nodes on demand.

- **Shared Filesystems (Manila)**: Manila provides coordinated access to shared or distributed file systems. While the primary consumption of file shares is across OpenStack Compute instances, the service is also intended to be accessible as an independent capability.
- **Containers (Magnum)**: Magnum provides an API service for making container orchestration engines such as Docker and Kubernetes available as resources in OpenStack.
- **Bare-metal provisioning (Ironic)**: Ironic provisions bare-metal machines instead of virtual machines, forked from the Nova baremetal driver. It is best thought of as a bare-metal hypervisor API and a set of plugins that interact with the bare-metal hypervisors.
- **DNS service (Designate)**: Designate provides DNS services for OpenStack users, including an API for domain/record management.
- **Application catalog (Murano)**: Murano introduces an application catalog to OpenStack, enabling application developers, and cloud administrators to publish various cloud-ready applications in a browsable categorized catalog.

These modular components are easily configured to enable an IaaS cloud service provider to tailor a cloud OS to its particular mission.

## 16.3 THE INTERNET OF THINGS

The Internet of Things is the latest development in the long and continuing revolution of computing and communications. Its size, ubiquity, and influence on everyday lives, business, and government dwarf any technical advance that has gone before. This section provides a brief overview of the Internet of Things, which is dealt with in greater detail later in the book.

### Things on the Internet of Things

The Internet of Things (IoT) is a term that refers to the expanding interconnection of smart devices, ranging from appliances to tiny sensors. A dominant theme is the embedding of short-range mobile transceivers into a wide array of gadgets and everyday items, enabling new forms of communication between people and things, and between things themselves. The Internet now supports the interconnection of billions of industrial and personal objects, usually through cloud systems. The objects deliver sensor information, act on their environment, and in some cases modify themselves, to create overall management of a larger system, like a factory or city.

The IoT is primarily driven by deeply embedded devices. These devices are low-bandwidth, low-repetition data capture, and low-bandwidth data-usage appliances that communicate with each other and provide data via user interfaces. Embedded appliances, such as high-resolution video security cameras, video VoIP phones, and a handful of others, require high-bandwidth streaming capabilities. Yet countless products simply require packets of data to be intermittently delivered.

## Evolution

With reference to the end systems supported, the Internet has gone through roughly four generations of deployment culminating in the IoT:

1. **Information technology (IT)**: PCs, servers, routers, firewalls, and so on, bought as IT devices by enterprise IT people, primarily using wired connectivity.
2. **Operational technology (OT)**: Machines/appliances with embedded IT built by non-IT companies, such as medical machinery, SCADA (supervisory control and data acquisition), process control, and kiosks, bought as appliances by enterprise OT people, primarily using wired connectivity.
3. **Personal technology**: Smartphones, tablets, and eBook readers bought as IT devices by consumers (employees) exclusively using wireless connectivity and often multiple forms of wireless connectivity.
4. **Sensor/actuator technology**: Single-purpose devices bought by consumers, IT, and OT people exclusively using wireless connectivity, generally of a single form, as part of larger systems.

It is the fourth generation that is usually thought of as the IoT, and which is marked by the use of billions of embedded devices.

## Components of IoT-Enabled Devices

The key components of an IoT-enabled device are the following:

- **Sensor**: A sensor measures some parameter of a physical, chemical, or biological entity and delivers an electronic signal proportional to the observed characteristic, either in the form of an analog voltage level or a digital signal. In both cases, the sensor output is typically input to a microcontroller or other management element.
- **Actuator**: An actuator receives an electronic signal from a controller and responds by interacting with its environment to produce an effect on some parameter of a physical, chemical, or biological entity.
- **Microcontroller**: The "smart" in a smart device is provided by a deeply embedded microcontroller.
- **Transceiver**: A transceiver contains the electronics needed to transmit and receive data. Most IoT devices contain a wireless transceiver, capable of communication using Wi-Fi, ZigBee, or some other wireless scheme.
- **Radio-Frequency Identification (RFID)**: (RFID) technology, which uses radio waves to identify items, is increasingly becoming an enabling technology for IoT. The main elements of an RFID system are tags and readers. RFID tags are small programmable devices used for object, animal, and human tracking. They come in a variety of shapes, sizes, functionalities, and costs. RFID readers acquire and sometimes rewrite information stored on RFID tags that come within operating range (a few inches up to several feet). Readers are usually connected to a computer system that records and formats the acquired information for further uses.

## IoT and Cloud Context

To better understand the function of an IoT, it is useful to view it in the context of a complete enterprise network that includes third-party networking and cloud computing elements. Figure 16.12 provides an overview illustration.

### *Edge*

At the edge of a typical enterprise network is a network of IoT-enabled devices, consisting of sensors and perhaps actuators. These devices may communicate with one another. For example, a cluster of sensors may all transmit their data to one sensor that aggregates the data to be collected by a higher-level entity. At this level also there may also be a number of **gateways**. A gateway interconnects the IoT-enabled devices with the higher-level communication networks. It performs the necessary translation between the protocols used in the communication networks and those used by devices. It may also perform a basic data aggregation function.

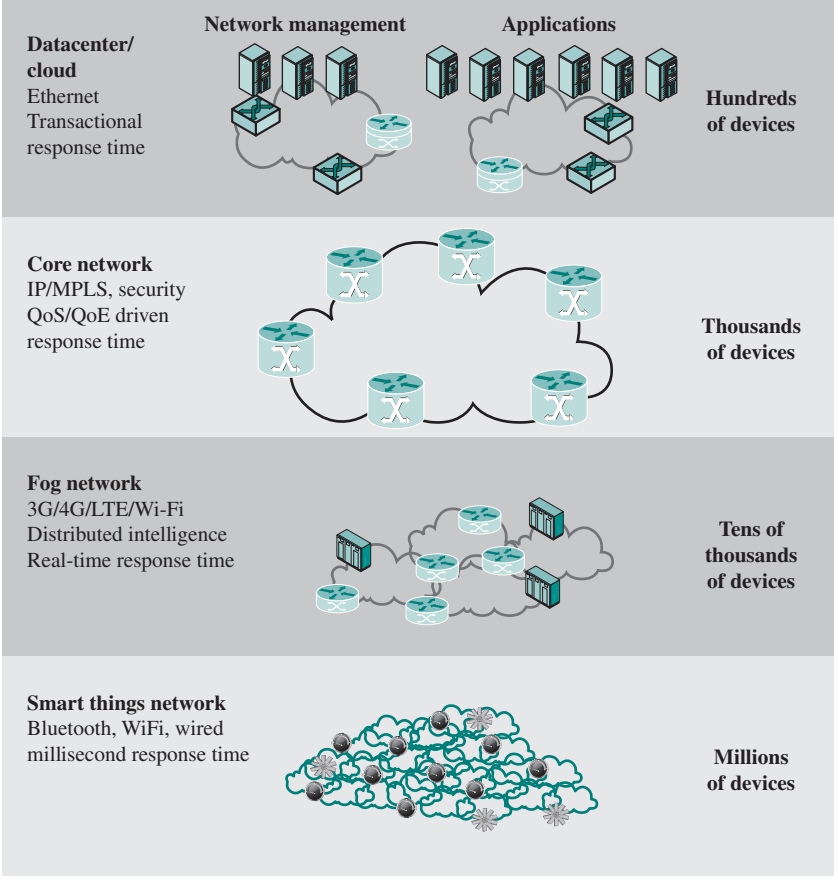



**Figure 16.12 The IoT/Cloud Context**

### *Fog*

In many IoT deployments, massive amounts of data may be generated by a distributed network of sensors. For example, offshore oil fields and refineries can generate a terabyte of data per day. An airplane can create multiple terabytes of data per hour. Rather than store all of that data permanently (or at least for a long period) in central storage accessible to IoT applications, it is often desirable to do as much data processing close to the sensors as possible. Thus, the purpose of what is sometimes referred to as the edge computing level is to convert network data flows into information that is suitable for storage and higher-level processing. Processing elements at these level may deal with high volumes of data and perform data transformation operations, resulting in the storage of much lower volumes of data. The following are examples of fog computing operations:

- **Evaluation**: Evaluating data for criteria as to whether it should be processed at a higher level.
- **Formatting**: Reformatting data for consistent higher-level processing.
- **Expanding/decoding**: Handling cryptic data with additional context (such as the origin).
- **Distillation/reduction**: Reducing and/or summarizing data to minimize the impact of data and traffic on the network and higher-level processing systems.
- **Assessment**: Determining whether data represents a threshold or alert; this could include redirecting data to additional destinations.

Generally, fog computing devices they are deployed physically near the edge of the IoT network; that is, near the sensors and other data-generating devices. Thus, some of the basic processing of large volumes of generated data is offloaded and outsourced from IoT application software located at the center.

Fog computing and fog services are expected to be a distinguishing characteristic of the IoT. Fog computing represents an opposite trend in modern networking from cloud computing. With cloud computing, massive, centralized storage and processing resources are made available to distributed customers over cloud networking facilities to a relatively small number of users. With fog computing, massive numbers of individual smart objects are interconnected with fog networking facilities that provide processing and storage resources close to the edge devices in an IoT. Fog computing addresses the challenges raised by the activity of thousands or millions of smart devices, including security, privacy, network capacity constraints, and latency requirements. The term *fog computing* is inspired by the fact that fog tends to hover low to the ground, whereas clouds are high in the sky.

### *Core*

The core network, also referred to as a **backbone network**, connects geographically dispersed fog networks as well as providing access to other networks that are not part of the enterprise network. Typically, the core network will use very high-performance routers, high-capacity transmission lines, and multiple interconnected routers for increased redundancy and capacity. The core network may also connect to high-performance, high-capacity servers such as large database servers and private cloud

**Table 16.4** Comparison of Cloud and Fog Features

| | Cloud | Fog |
|---|---|---|
| Location of processing/storage resources | Center | Edge |
| Latency | High | Low |
| Access | Fixed or wireless | Mainly wireless |
| Support for mobility | Not applicable | Yes |
| Control | Centralized/hierarchical (full control) | Distributed/hierarchical (partial control) |
| Service access | Through core | At the edge/on handheld device |
| Availability | 99.99% | Highly volatile/highly redundant |
| Number of users/devices | Tens/hundreds of millions | Tens of billions |
| Main content generator | Human | Devices/sensors |
| Content generation | Central location | Anywhere |
| Content consumption | End device | Anywhere |
| Software virtual infrastructure | Central enterprise servers | User devices |

facilities. Some of the core routers may be purely internal, providing redundancy and additional capacity without serving as edge routers.

### *Cloud*

The cloud network provides storage and processing capabilities for the massive amounts of aggregated data that originate in IoT-enabled devices at the edge. Cloud servers also host the applications that interact with and manage the IoT devices and that analyze the IoT-generated data.

Table 16.4 compares cloud and fog computing.

## 16.4 IoT OPERATING SYSTEMS

IoT devices are embedded devices, and so have an embedded OS. However, the vast majority of IoT devices have very limited resources including limited RAM and ROM, low-power requirements, no memory management unit, and limited processor performance. Thus, while some embedded OSs, such as TinyOS, are appropriate for IoT devices, many are simply too big and require too many resources to be used. In this section, we first define the types of devices that are usually considered as targets for an IoT OS, then examine the characteristics of an embedded OS suitable for such devices, and finally look at a popular open-source IoT OS, RIOT.

### Constrained Devices

Increasingly, the term *constrained device* is used to refer to the vast majority of IoT devices. In an IoT, a constrained device is a device with limited volatile and nonvolatile memory, limited processing power, and a low-data-rate transceiver. Many devices

**Table 16.5** Classes of Constrained Devices

| Class | Data Size (RAM) | Code Size (flash, ROM) |
|---|---|---|
| Class 0 | << 10 kB | << 100 kB |
| Class 1 | ~10 kB | ~100 kB |
| Class 2 | ~50 kB | ~250 kB |

in the IoT, particularly the smaller, more numerous devices, are resource constrained. As pointed out in [SEGH12], technology improvements following Moore's law continue to make embedded devices cheaper, smaller, and more energy efficient but not necessarily more powerful. Typical embedded IoT devices are equipped with 8- or 16-bit microcontrollers that possess very little RAM and storage capacities. Resource-constrained devices are often equipped with an IEEE 802.15.4 radio, which enables low-power low-data-rate wireless personal area networks (WPANs) with data rates of 20–250 kbps and frame sizes of up to 127 octets.

RFC 7228 (Terminology for Constrained-Node Networks) [BORM14] defines three classes of constrained devices (see Table 16.5):

- **Class 0**: These are very constrained devices, typically sensors, called *motes*, or *smart dust*. Motes can be implanted or scattered over a region to collect data and pass it on from one to another to some central collection point. For example, a farmer, vineyard owner, or ecologist could equip motes with sensors that detect temperature, humidity, etc., making each mote a mini weather station. Scattered throughout a field, vineyard or forest, these motes would allow the tracking of microclimates. Class 0 devices generally cannot be secured or managed comprehensively in the traditional sense. They will most likely be preconfigured (and will be reconfigured rarely, if at all) with a very small data set.
- **Class 1**: These are quite constrained in code space and processing capabilities, such that they cannot easily talk to other Internet nodes employing a full protocol stack. However, they are capable enough to use a protocol stack specifically designed for constrained nodes (such as the Constrained Application Protocol (CoAP)) and participate in meaningful conversations without the help of a gateway node.
- **Class 2**: These are less constrained and fundamentally capable of supporting most of the same protocol stacks as used on notebooks or servers. However, they are still very constrained compared to high-end IoT devices. Thus, they require lightweight and energy-efficient protocols and low transmission traffic.

Class 0 devices are so constrained that a conventional OS is not practical. These devices have a very limited, specialized function or set of functions that can be programmed directly onto the hardware. Class 1 and Class 2 devices are typically less specialized. An OS, with its kernel functions and support libraries, allow software developers to develop applications that make use of OS functionality and can be executed on a variety of devices. However, many embedded operating systems, such as $\mu$Clinux, consume too many resources and too much power to be usable for these constrained devices. Instead, an OS designed specifically for constrained devices is needed. Such an OS is typically referred to as an IoT OS.

## Requirements for an IoT OS

[HAHM15] lists the following as characteristics required of an IoT OS:

- **Small memory footprint**: Table 16.5 indicates the memory size limitations for constrained devices. This amount of memory is many orders of magnitude smaller than in smartphones, tablets, and a variety of larger embedded devices. Examples of the implications of this requirement are the need for libraries optimized in terms of both size and performance, and space-efficient data structures.
- **Support for heterogeneous hardware**: For the largest systems, such as servers, PCs, and laptops, the Intel x86 processor architecture dominates. For smaller systems, such as smartphones and a number of classes of IoT devices, the ARM architecture dominates. But constrained devices are based on various microcontroller architectures and families, especially 8-bit and 16-bit processors. A wide variety of communications technologies are also deployed on constrained devices.
- **Network connectivity**: Network connectivity is essential for data collection, development of distributed IoT applications, and remote system maintenance. A wide variety of communications techniques and protocols are used for low-power, minimal resource devices, including:
  - —IEEE 802.15.4 [low-rate wireless personal area network (WPAN)]
  - —Bluetooth Low Energy (BLE)
  - —6LoWPAN (IPv6 over Low-power Wireless Personal Area Networks)
  - —CoAP (Constrained Application Protocol)
  - —RPL (Routing Protocol for Low power and Lossy Networks)
- **Energy efficiency**: For any embedded device, and especially constrained devices, energy efficiency is of paramount importance. In a number of cases, IoT devices are required to work for years with a single battery charge [MIN02]. Chip manufacturers are addressing this requirement by making the processor as energy efficient as possible (e.g., [SHAH15]). In addition, a number of wireless transmission schemes have been developed that are designed to minimize power consumption [FREN16]. But there is also an important role to be played by the OS. [HAHM15] suggests that the key requirements for IoT OSs for the IoT are (i) provide energy saving options to upper layers, and (ii) make use of these functions itself as much as possible, for example by using techniques such as radio duty cycling, or by minimizing the number of periodic tasks that need to be executed.
- **Real-time capabilities**: A wide range of IoT devices require support for real-time operation [STAN14]. These include the following:
  - —real-time sensor data streams: for example, most sensornet applications (such as surveillance) tend to be time-sensitive in nature where packets must be relayed and forwarded on a timely basis; real-time guarantee is a necessary requirement for such applications [DONG10]
  - —2-way control on a wide scale: cars (and aircraft) talking to each other and controlling each other to avoid collisions, humans exchanging data automatically

when they meet and this possibly affecting their next actions, and physiological data uploaded to doctors in real time with real-time feedback from the doctor

—real-time response to security events

Thus, the IoT OS must be able to fulfill timely execution requirements and be designed to guarantee worst-case execution times and worst-case interrupt latencies.

- **Security**: IoT devices are numerous, often deployed in unsecured locations, have limited processing and memory resources to support sophisticated security protocols and mechanisms, and usually communicate wirelessly, increasing their vulnerability. Accordingly, IoT security is both a high priority and difficult to achieve [STAL16b]. ITU-T Recommendation Y-2060 (*Overview of the Internet of things*, June, 2–12) lists the following security capabilities desired in an IoT device:

  —**at the application layer:** authorization, authentication, application data confidentiality and integrity protection, privacy protection, security audit, and anti virus

  —**at the network layer:** authorization, authentication, use data and signaling data confidentiality, and signaling integrity protection

  —**at the device layer:** authentication, authorization, device integrity validation, access control, data confidentiality, and integrity protection

  Thus, an IoT OS needs to proved the necessary security mechanisms within the resource constraints of the device, and provide mechanisms for software updates on already-deployed IoT devices.

## IoT OS Architecture

There are a number of both embedded OSs that might be considered suitable for constrained IoT devices; [HAHM15] provides a useful survey. Two other surveys that focus on wireless sensor networks (sensornets) are [DONG10] and [SARA11]. While these systems differ from one another in many ways, the general structure shown in Figure 16.13 captures the key elements of a typical IoT OS. The main components are:

- **System and support libraries**: A streamlined set of libraries includes a shell, logging, and cryptographic functions.
- **Device drivers and logical file system**: Modular set of streamlined device drivers and file system support that can be minimally configured for a particular device and applications.
- **Low-power network stack**: There are differing requirements for network connectivity for various constrained IoT devices. For many sensor networks, the IoT devices require only limited communication capability that allows the sensor to communicate data to another sensor or a gateway that will pass the data along. In other cases, IoT devices (even constrained IoT devices) must seamlessly integrate with the Internet and communicate end-to-end with other machines on the Internet. Thus, the IoT OS needs to provide the ability to configure a network stack that supports protocols specifically designed for low-power requirements, but that also includes support up to the Internet Protocol level [PETE15].

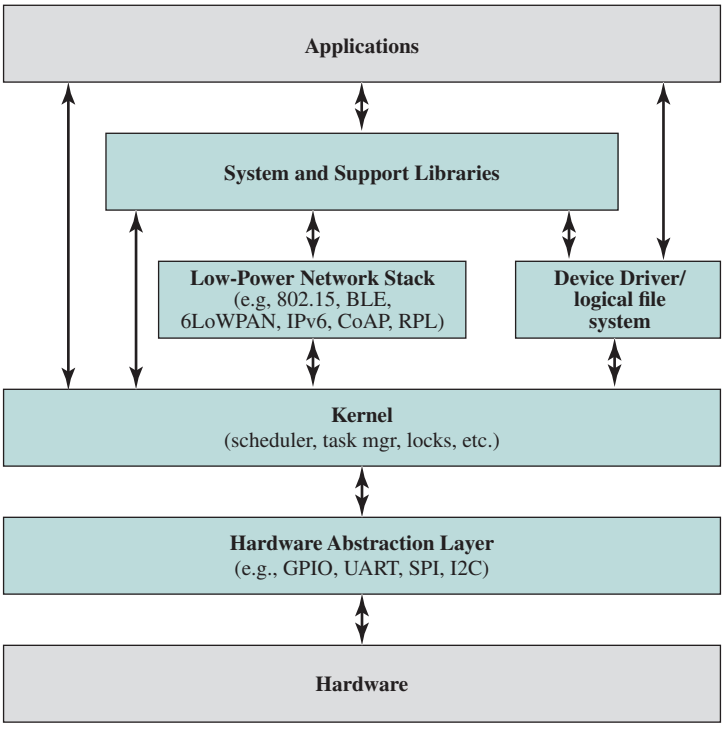



**Figure 16.13 Typical Structure for IoT OS**

- **Kernel**: Typically the kernel provides a scheduler, a model for tasks, mutual exclusion, and other forms of synchronization, and timers.
- **Hardware abstraction layer (HAL)**: The HAL is software that presents a consistent API to the upper layers and maps upper-layer operations onto a specific hardware platform. Thus, the HAL is different for each hardware platform. Common interfaces that could be supported include:

  —**General Purpose Input/Output (GPIO):** a generic pin that can be designated as input or output by the user at run time; useful when there is a scarcity of pin positions.

  —**Universal Asynchronous Receiver/Transmitter (UART):** an asynchronous serial digital data link.

  —**Serial Peripheral Interface (SPI):** a synchronous serial digital data link.

  — **Inter-Integrated Circuit** (**$I^2C$**): a serial computer bus typically used for attaching lower-speed peripheral ICs to processors and microcontrollers in short-distance, intra-board communication.

## RIOT

As was mentioned, not all embedded OSs are suitable for constrained IoT devices. For example, of the two OSs examined in Chapter 13, $\mu$Clinux requires too much memory, whereas TinyOS is suitable. In this section, we examine RIOT, an open-source

**Table 16.6** Comparison of $\mu$Clinux, TinyOS, and RIOT

| | $\mu$Clinux | TinyOS | RIOT |
|---|---|---|---|
| Minimum RAM | < 32 MB | < 1 kB | ~1.5 kB |
| Minimum ROM | < 2 MB | < 4 kB | ~5 kB |
| C Support | ✓ | ✗ | ✓ |
| C++ Support | ✓ | ✗ | ✓ |
| Multithreading | ✓ | ○ | ✓ |
| Microcontrollers without MMU | ✓ | ✓ | ✓ |
| Modularity | ○ | ✗ | ✓ |
| Real time | ○ | ✗ | ✓ |

✓ = full support
○ = partial support
✗ = no support

OS designed specifically for constrained IoT devices [BACC13]. Table 16.6 compares $\mu$Clinux, TinyOS, and RIOT.

Figure 16.14 illustrates the structure of RIOT.

***RIOT KERNEL*** RIOT uses a microkernel design structure, which means the kernel, referred to in RIOT as the core module, contains only the absolute features, such as scheduling, inter process communication (IPC), synchronization, and interrupt request (IRQ) handling. All other OS functions, including device drivers and system libraries run as threads. Because of this use of threads, applications and other parts of the system run in their own context, multiple of these contexts can run at the same time, and IPC provides a safe, synchronized way for communicating between them, with defined priorities.

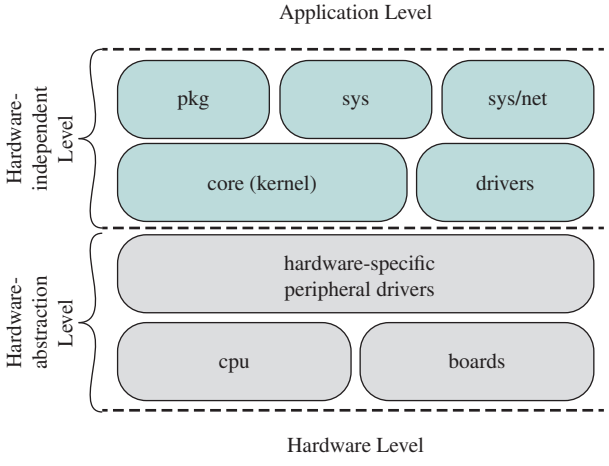



**Figure 16.14** **RIOT Structure**

The advantage of the microkernel approach is that it is easy to configure a system with only the minimum software necessary for the applications on a particular IoT device.

The modules in the kernel are:

- **IRQ Handling**: Provides an API to control interrupt processing.
- **Kernel utilities**: Utilities and data structures used by the kernel.
- **Mailboxes**: Mailbox implementation.
- **Messaging/IPC**: Messaging API for interprocess communication.
- **Power management**: The kernel's power management interface.
- **Scheduler**: The RIOT scheduler.
- **Startup and configuration**: Configuration data and startup code for the kernel.
- **Synchronization**: Mutex for thread synchronization.
- **Threading**: Support for multithreading.

One notable feature of RIOT is, in contrast to many other OSs, RIOT uses a tickless scheduler. When there are no pending tasks, RIOT will switch to the idle thread. The idle thread is to determine the deepest possible sleep mode, depending on the peripheral devices in use. The result is that the scheduler maximizes the time spent in sleep mode, which minimizes the energy consumption of the system. Only interrupts (external or kernel-generated) wake up the system from idle state. In addition, all kernel functions are kept as small as possible, which allows the kernel to run even on systems with a very low clock speed. The scheduler is designed to minimize the occurrences of thread switching to reduce overhead. This strategy is appropriate for IoT devices that do not have user interaction.

***Other Hardware-Independent Modules*** The sys library includes data structures (e.g., bloom, color), crypto libraries (e.g., hashes, AES), high-level APIs (e.g., Posix implementations), memory management (e.g., malloc), the RIOT shell, and other commonly used system library modules.

The sys/net sub-directory includes all the networking related software. This includes the network protocol stack, network APIs, and software related to specific network types.

The pkg library provides support for a number of external libraries (e.g., OpenWSN, microcoap). RIOT ships with a custom Makefile for each supported library, which downloads the library and optionally applies a number of patches to make it work with RIOT.

***Hardware Abstraction Layer*** The RIOT HAL consists of three sets of software. For each supported processor, the CPU directory contains a sub-directory with the name of the processor. These directories then contain all processor-specific configurations, such as implementations of power management, interrupt handling and vectors, startup code, clock initialization code, and thread handling (e.g., context switching) code.

The platform dependent code is split into two logic elements: processors and boards. While maintaining a strict 1-to-*n* relationship, a board has exactly one processor, while a processor can be part of *n* boards. The processor part contains all generic, processor-specific code.

The board part contains the specific configuration for the processor it contains. This configuration mainly includes the peripheral configuration and pin-mapping, the configuration of on-board devices, and the processor's clock configuration. On top of the source and header files needed for each board, the board's directory additionally may include some script and configuration files needed for interfacing with the board.

The hardware-specific peripheral drivers directory provides an API to the logical device driver software and is configured for the specific peripherals of the host system. The main goal of the separation of drivers from drivers/peripherals is to allow the writing of portable hardware-accessing code, which is one of the key aspects of RIOT. The drivers directory contains code for actual hardware drivers, such as sensors, radios. The drivers/peripherals directory contains the headers and some shared code for RIOT's hardware abstraction, which provides a unified API abstracting the I/O interfaces of microcontrollers, such as UART, $I^2C$, and SPI. The idea is that drivers (or applications) can be written once against the API provided by drivers/periph, and then run unmodified on all microcontrollers that provide an implementation for the needed interface.

## 16.5 KEY TERMS AND REVIEW QUESTIONS

### Key Terms

| | | |
|---|---|---|
| actuators | direct attached storage (DAS) | platform as a service (PaaS) |
| backbone network | file-based storage | private cloud |
| block storage | file storage | public cloud |
| cloud | gateways | radio-frequency identification (RFID) |
| cloud auditor | hybrid cloud | sensors |
| cloud broker | infrastructure as a service (IaaS) | service models |
| cloud carrier | Internet of Things (IoT) | software as a service (SaaS) |
| cloud computing | microcontroller | storage area network (SAN) |
| cloud service consumer (CSC) | network attached storage (NAS) | transceiver |
| cloud service provider (CSP) | object storage | |
| community cloud | OpenStack | |
| Constrained Application Protocol (CoAP) | | |
| constrained device | | |

### Review Questions

**16.1** Define cloud computing.

**16.2** List and briefly define three cloud service models.

**16.3** What is the cloud computing reference architecture?
**16.4** List and briefly define the key components of a cloud operating system.
**16.5** What is the relationship between a cloud OS and IaaS?
**16.6** What is OpenStack?
**16.7** Define the Internet of Things.
**16.8** List and briefly define the principal components of an IoT-enabled thing.
**16.9** What requirements should an IoT OS satisfy?
**16.10** What is RIOT?

# APPENDIX A

## TOPICS IN CONCURRENCY


**A.1 Race Conditions and Semaphores**
- Problem Statement
- First Attempt
- Second Attempt
- Third Attempt
- Fourth Attempt
- A Good Attempt

**A.2 A Barbershop Problem**
- An Unfair Barbershop
- A Fair Barbershop

**A.3 Problems**

## A.1 RACE CONDITIONS AND SEMAPHORES

Although the definition of a race condition provided in Section 5.1 seems straightforward, experience has shown that students usually have difficulty pinpointing race conditions in their programs. The purpose of this section, which is based on [CARR01],[1] is to step through a series of examples using semaphores that should help clarify the topic of race conditions.

### Problem Statement

Assume there are two processes, **A** and **B**, each of which consists of a number of concurrent threads. Each thread includes an infinite loop in which a message is exchanged with a thread in the other process. Each message consists of an integer placed in a shared global buffer. There are two requirements:

1. After a thread A1 of process **A** makes a message available to some thread B1 in **B**, A1 can only proceed after it receives a message from B1. Similarly, after B1 makes a message available to A1, it can only proceed after it receives a message from A1.
2. Once a thread A1 makes a message available, it must make sure that no other thread in **A** overwrites the global buffer before the message is retrieved by a thread in **B**.

In the remainder of this section, we show four attempts to implement this scheme using semaphores, each of which can result in a race condition. Finally, we show a correct solution.

### First Attempt

Consider this approach:

```
semaphore  a = 0, b = 0;
 int  buf_a, buf_b;
```

```
thread_A(...)
{
    int var_a;
    ...
    while (true) {
      . . .
       var a =...;
       semSignal(b);
       semWait(a);
       buf_a = var_a;
       var_a = buf_b;
       . . .;
    }
}
```

```
thread_B(...)
{
    int var_b;
    ...
    while (true) {
       . . .
       var_b =...;
       semSignal(a);
       semWait(b);
       buf_b = var_b;
       var_b = buf_a;
       . . .;
    }
}
```

[1] I am grateful to Professor Ching-Kuang Shene of Michigan Technological University for permission to use this example.

This is a simple handshaking protocol. When a thread A1 in **A** is ready to exchange messages, it sends a signal to a thread in **B** then waits for a thread B1 in **B** to be ready. Once a signal comes back from B1, which **A** perceives by performing `semWait(a)`, then A1 assumes that B1 is ready and performs the exchange. B1 behaves similarly, and the exchange happens regardless of which thread is ready first.

This attempt can lead to race conditions. For example, consider the following sequence, with time going vertically down the table:

| Thread A1 | Thread B1 |
|---|---|
| `semSignal(b)` | |
| `semWait(a)` | |
| | `semSignal(a)` |
| | `semWait(b)` |
| `buf_a = var_a` | |
| `var_a = buf_b` | |
| | `buf_b = var_b` |

In the preceding sequence, A1 reaches `semWait(a)` and is blocked. B1 reaches `semWait(b)` and is not blocked, but is switched out before it can update its `buf_b`. Meanwhile, A1 executes and reads from `buf_b` before it has the intended value. At this point, `buf_b` may have a value provided previously by another thread or provided by B1 in a previous exchange. This is a race condition.

A subtler race condition can be seen if two threads in **A** and **B** are active. Consider the following sequence:

| Thread A1 | Thread A2 | Thread B1 | Thread B2 |
|---|---|---|---|
| `semSignal(b)` | | | |
| `semWait(a)` | | | |
| | | `semSignal(a)` | |
| | | `semWait(b)` | |
| | `semSignal(b)` | | |
| | `semWait(a)` | | |
| | | `buf_b = var_b1` | |
| | | | `semSignal(a)` |
| `buf_a = var_a1` | | | |
| | `buf_a = var_a2` | | |

In this sequence, threads A1 and B1 attempt to exchange messages and go through the proper semaphore signaling instructions. However, immediately after the two `semWait` signals occur (in threads A1 and B1), thread A2 runs and executes `semSignal(b)` and `semWait(a)`, which causes thread B2 to execute `semSignal(a)` to release A2 from `semWait(a)`. At this point, either A1 or A2 could update `buf_a` next, and we have a race condition. By changing the sequence of execution among the threads, we can readily find other race conditions.

**Lesson Learned:** When a variable is shared by multiple threads, race conditions are likely to occur unless proper mutual exclusion protection is used.

## Second Attempt

For this attempt, we use a semaphore to protect the shared variable. The purpose is to ensure that access to `buf_a` and `buf_b` is mutually exclusive. The program is as follows:

```
semaphore a = 0, b = 0; mutex = 1;
int buf_a, buf_b;
```

```
thread_A(...)
{
    int var_a;
    . . .
    while (true) {
        . . .
        var_a =...;
        semSignal(b);
        semWait(a);
            semWait(mutex);
                buf_a = var_a;
            semSignal(mutex);
        semSignal(b);
        semWait(a);
            semWait(mutex);
                var_a = buf_b;
            semSignal(mutex);
        . . .;
    }
}
```

```
thread_B(...)
{
    int var_b;
    . . .
    while (true) {
        . . .
        var_b =...;
        semSignal(a);
        semWait(b);
            semWait(mutex);
                buf_b = var_b;
            semSignal(mutex);
        semSignal(a);
        semWait(b);
            semWait(mutex);
                var_b = buf_a;
            semSignal(mutex);
        . . .;
    }
}
```

Before a thread can exchange a message, it follows the same handshaking protocol as in the first attempt. The semaphore `mutex` protects `buf_a` and `buf_b` in an attempt to assure that update precedes reading. But the protection is not adequate. Once both threads complete the first handshaking stage, the values of semaphores `a` and `b` are both 1. There are three possibilities that could occur:

1. Two threads, say A1 and B1, complete the first handshaking and continue with the second stage of the exchange.
2. Another pair of threads starts the first stage.
3. One of the current pair will continue and exchange a message with a newcomer in the other pair.

All of these possibilities can lead to race conditions. As an example of a race condition based on the third possibility, consider the following sequence:

| Thread A1 | Thread A2 | Thread B1 |
|---|---|---|
| semSignal(b) | | |
| semWait(a) | | |
| | | semSignal(a) |
| | | semWait(b) |
| buf_a = var_a1 | | |
| | | buf_b = var_b1 |
| | semSignal(b) | |
| | semWait(a) | |
| | | semSignal(a) |
| | | semWait(b) |
| | buf_a = var_a2 | |

In this example, after A1 and B1 go through the first handshake, they both update the corresponding global buffers. Then A2 initiates the first handshaking stage. Following this, B1 initiates the second handshaking stage. At this point, A2 updates `buf_a` before B1 can retrieve the value placed in `buf_a` by A1. This is a race condition.

**Lesson Learned:** Protecting a single variable may be insufficient if the use of that variable is part of a long execution sequence. Protect the whole execution sequence.

## Third Attempt

For this attempt, we want to expand the critical section to include the entire message exchange (two threads each update one of two buffers and read from the other buffer). A single semaphore is insufficient because this could lead to deadlock, with each side waiting on the other. The program is as follows:

```
semaphore aready = 1, adone = 0, bready = 1 bdone = 0;
int buf_a, buf_b;
```

```
thread_A(...)
{
    int var_a;
    ...
    while (true) {
        . . .
        var_a =...;
        semWait(aready);
            buf_a = var_a;
            semSignal(adone);
            semWait(bdone);
            var_a = buf_b;
        semSignal(aready);
        . . .;
    }
}
```

```
thread_B(...)
{
    int var_b;
    ...
    while (true) {
        . . .
        var_b =...;
        semWait(bready);
            buf_b = var_b;
            semSignal(bdone);
            semWait(adone);
            var_b = buf_a;
        semSignal(bready);
        . . .;
    }
}
```

The semaphore aready is intended to insure that no other thread in **A** can update buf_a while one thread from **A** enters its critical section. The semaphore adone is intended to insure that no thread from **B** will attempt to read buf_a until buf_a has been updated. The same considerations apply to bready and bdone. However, this scheme does not prevent race conditions. Consider the following sequence:

| Thread A1 | Thread B1 |
|---|---|
| buf_a = var_a | |
| semSignal(adone) | |
| semWait(bdone) | |
| | buf_b = var_b |
| | semSignal(bdone) |
| | semWait(adone) |
| var_a = buf_b; | |
| semSignal(aready) | |
| ...loop back... | |
| semWait(aready) | |
| buf_a = var_a | |
| | var_b = buf_a |

In this sequence, both A1 and B1 enter their critical sections, deposit their messages, and reach the second wait. Then A1 copies the message from B1 and leaves its critical section. At this point, A1 could loop back in its program, generate a new message, and deposit it in buf_a, as shown in the preceding execution sequence. Another possibility is that at this same point, another thread of **A** could generate a message and put it in buf_a. In either case, a message is lost and a race condition occurs.

**Lesson Learned:** If we have a number of cooperating thread groups, mutual exclusion guaranteed for one group may not prevent interference from threads in other groups. Further, if a critical section is repeatedly entered by one thread, then the timing of the cooperation between threads must be managed properly.

## Fourth Attempt

The third attempt fails to force a thread to remain in its critical section until the other thread retrieves the message. Here is an attempt to achieve this objective:

```
semaphore aready = 1, adone = 0, bready = 1 bdone = 0;
int buf_a, buf_b;
```

```
thread_A(...)
{
    int var_a;
    ...
    while (true) {
        . . .
        var_a =...;
        semWait(bready);
            buf_a = var_a;
            semSignal(adone);
            semWait(bdone);
            var_a = buf_b;
        semSignal(aready);
        . . .;
    }
}
```

```
thread_B(...)
{
    int var_b;
    ...
    while (true) {
        . . .
        var_b =...;
        semWait(aready);
            buf_b = var_b;
            semSignal(bdone);
            semWait(adone);
            var_b = buf_a;
        semSignal(bready);
        . . .;
    }
}
```

In this case, the first thread in **A** to enter its critical section decrements `bready` to 0. No subsequent thread from **A** can attempt a message exchange until a thread from **B** completes the message exchange and increments `bready` to 1. This approach too can lead to race conditions, such as in the following sequence:

| Thread A1 | Thread A2 | Thread B1 |
|---|---|---|
| semWait(bready) | | |
| buf_a = var_a1 | | |
| semSignal(adone) | | |
| | | semWait(aready) |
| | | buf_b = var_b1 |
| | | semSignal(bdone) |
| | | semWait(adone) |
| | | var_b = buf_a |
| | | semSignal(bready) |
| | semWait(bready) | |
| | ... | |
| | semWait(bdone) | |
| | var_a2 = buf_b | |

In this sequence, threads A1 and B1 enter corresponding critical sections in order to exchange messages. Thread B1 retrieves its message and signals `bready`. This enables another thread from **A**, A2, to enter its critical section. If A2 is faster than A1, then A2 may retrieve the message that was intended for A1.

**Lesson Learned:** If the semaphore for mutual exclusion is not released by its owner, race conditions can occur. In this fourth attempt, a semaphore is locked by a thread in **A** and then unlocked by a thread in **B**. This is risky programming practice.

## A Good Attempt

The reader may notice the problem in this section is a variation of the bounded-buffer problem and can be approached in a manner similar to the discussion in Section 5.4. The most straightforward approach is to use two buffers, one for **B**-to-**A** messages and one for **A**-to-**B** messages. The size of each buffer needs to be one. To see the reason for this, consider that there is no ordering assumption for releasing threads from a synchronization primitive. If a buffer has more than one slot, then we cannot guarantee that the messages will be properly matched. For example, B1 could receive a message from A1 then send a message to A1. But if the buffer has multiple slots, another thread in **A** may retrieve the message from the slot intended for A1.

Using the same basic approach as was used in Section 5.4, we can develop the following program:

```
semaphore notFull_A = 1, notFull_B = 1;
semaphore notEmpty_A = 0, notEmpty_B = 0;
int buf_a, buf_b;
```

```
thread A(...)
{
    int var_a;
    ...
    while (true) {
        . . .
        var_a =...;
        semWait(notFull_A);
            buf_a = var_a;
            semSignal(notEmpty_A);
        semWait(notEmpty_B);
            var_a = buf_b;
            semSignal(notFull_B);
        . . .;
    }
}
```

```
thread_B(...)
{
    int var_b;
    ...
    while (true) {
        . . .
        var_b =...;
        semWait(notFull_B);
            buf_b = var_b;
            semSignal(notEmpty_B);
        semWait(notEmpty_A);
            var_b = buf_a;
            semSignal(notFull_A);
        . . .;
    }
}
```

To verify that this solution works, we need to address three issues:

1. The message exchange section is mutually exclusive within the thread group. Because the initial value of `notFull_A` is 1, only one thread in **A** can pass through `semWait(notFull_A)` until the exchange is complete as signaled by a thread in **B** that executes `semSignal(notFull_A)`. A similar reasoning applies to threads in **B**. Thus, this condition is satisfied.
2. Once two threads enter their critical sections, they exchange messages without interference from any other threads. No other thread in **A** can enter its critical

section until the thread in **B** is completely done with the exchange, and no other thread in **B** can enter its critical section until the thread in **A** is completely done with the exchange. Thus, this condition is satisfied.

3. After one thread exits its critical section, no thread in the same group can rush in and ruin the existing message. This condition is satisfied because a one-slot buffer is used in each direction. Once a thread in **A** has executed `semWait(notFull_A)` and entered its critical section, no other thread in **A** can update `buf_a` until the corresponding thread in **B** has retrieved the value in `buf_a` and issued a `semSignal(notFull_A)`.

**Lesson Learned:** It is well to review the solutions to well-known problems, because a correct solution to the problem at hand may be a variation of a solution to a known problem.

## A.2 A BARBERSHOP PROBLEM

As another example of the use of semaphores to implement concurrency, we consider a simple barbershop problem.[2] This example is instructive because the problems encountered when attempting to provide tailored access to barbershop resources are similar to those encountered in a real operating system.

Our barbershop has three chairs, three barbers, and a waiting area that can accommodate four customers on a sofa and that has standing room for additional customers (see Figure A.1). Fire codes limit the total number of customers in the shop to 20. In this example, we assume the barbershop will eventually process 50 customers.

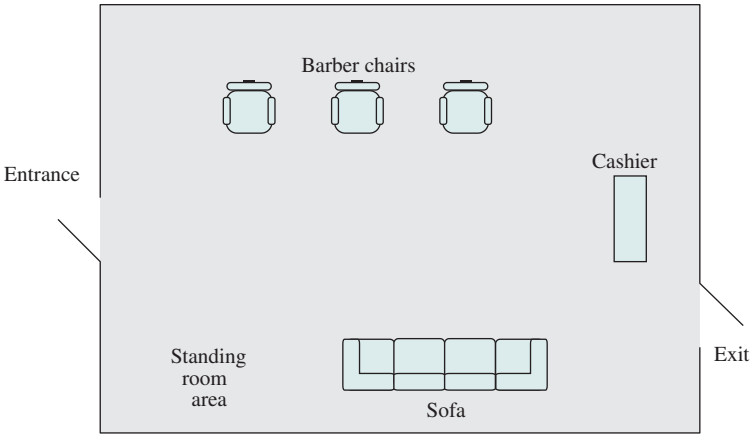



**Figure A.1 The Barbershop**

[2]I am indebted to Professor Ralph Hilzer of California State University at Chico for supplying this treatment of the problem.

A customer will not enter the shop if it is filled to capacity with other customers. Once inside, the customer takes a seat on the sofa or stands if the sofa is filled. When a barber is free, the customer that has been on the sofa the longest is served and, if there are any standing customers, the one that has been in the shop the longest takes a seat on the sofa. When a customer's haircut is finished, any barber can accept payment, but because there is only one cash register, payment is accepted for one customer at a time. The barbers divide their time among cutting hair, accepting payment, and sleeping in their chair waiting for a customer.

## An Unfair Barbershop

Figure A.2 shows an implementation using semaphores; the three procedures are listed side-by-side to conserve space. We assume all semaphore queues are handled with a first-in-first-out policy.

The main body of the program activates 50 customers, 3 barbers, and the cashier process. We now consider the purpose and positioning of the various synchronization operators:

- **Shop and sofa capacity:** The capacity of the shop and the capacity of the sofa are governed by the semaphores `max_capacity` and `sofa`, respectively. Every time a customer attempts to enter the shop, the `max_capacity` semaphore is decremented by 1; every time a customer leaves, the semaphore is incremented. If a customer finds the shop full, then that customer's process is blocked on `max_capacity` by the `semWait` function. Similarly, the `semWait` and `semSignal` operations surround the actions of sitting on and getting up from the sofa.
- **Barber chair capacity:** There are three barber chairs, and care must be taken that they are used properly. The semaphore `barber_chair` assures that no more than three customers attempt to obtain service at a time, trying to avoid the undignified occurrence of one customer sitting on the lap of another. A customer will not get up from the sofa until at least one chair is free [`semWait(barber_chair)`], and each barber signals when a customer has left that barber's chair [`semSignal(barber_chair)`]. Fair access to the barber chairs is guaranteed by the semaphore queue organization: The first customer to be blocked is the first one allowed into an available chair. Note that, in the customer procedure, if `semWait(barber_chair)` occurred after `semSignal(sofa)`, each customer would only briefly sit on the sofa then stand in line at the barber chairs, creating congestion and leaving the barbers with little elbow room.
- **Ensuring customers are in barber chair:** The semaphore `cust_ready` provides a wakeup signal for a sleeping barber, indicating that a customer has just taken a chair. Without this semaphore, a barber would never sleep but would begin cutting hair as soon as a customer left the chair; if no new customer had grabbed the seat, the barber would be cutting air.
- **Holding customers in barber chair:** Once seated, a customer remains in the chair until the barber gives the signal that the haircut is complete, using the semaphore `finished`.

```
/* program barbershop1 */
semaphore max_capacity = 20;
semaphore sofa = 4;
semaphore barber_chair = 3;
semaphore coord = 3;
semaphore cust_ready = 0, finished = 0, leave_b_chair = 0, payment= 0,
          receipt = 0;
```

```
void  customer  ()
{
   semWait(max_capacity);
   enter_shop();
   semWait(sofa);
   sit_on_sofa();
   semWait(barber_chair);
   get_up_from_sofa();
   semSignal(sofa);
   sit_in_barber_chair();
   semSignal(cust_ready);
   semWait(finished);
   leave_barber_chair();
   semSignal(leave_b_chair);
   pay();
   semSignal(payment);
   semWait(receipt);
   exit_shop();
   semSignal(max_capacity)
}
```

```
void  barber()
{
   while  (true)
   {
      semWait(cust_ready);
      semWait(coord);
      cut_hair();
      semSignal(coord);
      semSignal(finished);
      semWait(leave_b_chair);
      semSignal(barber_chair);
   }
}

void  cashier()
{
   while  (true)
   {  semWait(payment);
      semWait(coord);
      accept_pay();
      semSignal(coord);
      semSignal(receipt);
   }
}
```

```
void main()
{
   parbegin (customer, . . . 50 times, . . . customer, barber, barber,
             barber, cashier);
}
```

**Figure A.2 An Unfair Barbershop**

- **Limiting one customer to a barber chair:** The semaphore `barber_chair` is intended to limit the number of customers in barber chairs to three. However, by itself, `barber_chair` does not succeed in doing this. A customer that fails to get the processor immediately after his barber executes `semSignal(finished)` (i.e., one who falls into a trance or stops to chat with a neighbor) may still be in the chair when the next customer is given the go ahead to be seated. The semaphore `leave_b_chair` is intended to correct this problem by restraining the barber from inviting a new customer into the chair until the lingering one has announced his departure from it. In the problems at the end of this chapter, we will find that even this precaution fails to stop the mettlesome customer lap sittings.
- **Paying and receiving:** Naturally, we want to be careful when dealing with money. The cashier wants to be assured that each customer pays before leaving the shop, and the customer wants verification that payment was received (a receipt). This is accomplished, in effect, by a face-to-face transfer of the money. Each

**Table A.1** Purpose of Semaphores in Figure A.2

| Semaphore | Wait Operation | Signal Operation |
|---|---|---|
| `max_capacity` | Customer waits for space to enter shop. | Exiting customer signals customer waiting to enter. |
| `sofa` | Customer waits for seat on sofa. | Customer leaving sofa signals customer waiting for sofa. |
| `barber_chair` | Customer waits for empty barber chair. | Barber signals when that barber's chair is empty. |
| `cust_ready` | Barber waits until a customer is in the chair. | Customer signals barber that customer is in the chair. |
| `finished` | Customer waits until his haircut is complete. | Barber signals when cutting hair of this customer is done. |
| `leave_b_chair` | Barber waits until customer gets up from the chair. | Customer signals barber when customer gets up from chair. |
| `payment` | Cashier waits for a customer to pay. | Customer signals cashier that he has paid. |
| `receipt` | Customer waits for a receipt for payment. | Cashier signals that payment has been accepted. |
| `coord` | Wait for a barber resource to be free to perform either the hair cutting or cashiering function. | Signal that a barber resource is free. |

customer, upon arising from a barber chair, pays, alerts the cashier that money has been passed over [`semSignal(payment)`], then waits for a receipt [`semWait(receipt)`]. The cashier process repeatedly takes payments: It waits for a payment to be signaled, accepts the money, then signals acceptance of the money. Several programming errors need to be avoided here. If `semSignal(payment)` occurred just before the action `pay`, then a customer could be interrupted after signaling; this would leave the cashier free to accept payment even though none had been offered. An even more serious error would be to reverse the positions of the `semSignal(payment)` and `semWait(receipt)` lines. This would lead to deadlock because that would cause all customers and the cashier to block at their respective `semWait` operators.

- **Coordinating barber and cashier functions:** To save money, this barbershop does not employ a separate cashier. Each barber is required to perform that task when not cutting hair. The semaphore `coord` ensures that barbers perform only one task at a time.

Table A.1 summarizes the use of each of the semaphores in the program.

The cashier process could be eliminated by merging the payment function into the barber procedure. Each barber would sequentially `cut hair` and then `accept pay`. However, with a single cash register, it is necessary to limit access to the `accept pay` function to one barber at a time. This could be done by treating that function as a critical section and guarding it with a semaphore.

```
/* program barbershop2 */
semaphore max_capacity = 20;
semaphore sofa = 4;
semaphore barber_chair = 3, coord = 3;
semaphore mutex1 = 1, mutex2 = 1;
semaphore cust_ready = 0, leave_b_chair = 0, payment = 0, receipt = 0;
semaphore finished [50] = {0};
int count;
```

```
void customer()
{
   int  custnr;
   semWait(max_capacity);
   enter_shop();
   semWait(mutex1);
   custnr = count;
   count++;
   semSignal(mutex1);
   semWait(sofa);
   sit_on_sofa();
   semWait(barber_chair);
   get_up_from_sofa();
   semSignal(sofa);
   sit_in_barber_chair();
   semWait(mutex2);
   enqueue1(custnr);
   semSignal(cust_ready);
   semSignal(mutex2);
   semWait(finished[custnr]);
   leave_barber_chair();
   semSignal(leave_b_chair);
   pay();
   semSignal(payment);
   semWait(receipt);
   exit_shop();
   semSignal(max_capacity)
}
```

```
void barber()
{
   int  b_cust;
   while  (true)
   {
      semWait(cust_ready);
      semWait(mutex2);
      dequeue1(b_cust);
      semSignal(mutex2);
      semWait(coord);
      cut_hair();
      semSignal(coord);
      semSignal(finished[b_cust]);
      semWait(leave_b_chair);
      semSignal(barber_chair);
   }
}

void  cashier()
{
   while  (true)
   {
      semWait(payment);
      semWait(coord);
      accept_pay();
      semSignal(coord);
      semSignal(receipt);
   }
}
```

```
void  main()
{  count := 0;
   parbegin (customer, . . . 50 times, . . . customer, barber, barber,
             barber, cashier);
}
```

**Figure A.3** **A Fair Barbershop**

## A Fair Barbershop

Figure A.2 is a good effort, but some difficulties remain. One problem is solved in the remainder of this section; others are left as exercises for the reader (see Problem A.3).

There is a timing problem in Figure A.2 that could lead to unfair treatment of customers. Suppose three customers are currently seated in the three barber chairs. In that case, the customers would most likely be blocked on `semWait(finished)`, and due to the queue organization, they would be released in the order they entered the barber chair. However, what if one of the barbers is very fast or one of the customers is quite bald? Releasing the first customer to enter the chair could result in a situation where one customer is summarily ejected from his seat and forced to pay

full price for a partial haircut while another is restrained from leaving his chair even though his haircut is complete.

The problem is solved with more semaphores, as shown in Figure A.3. We assign a unique customer number to each customer; this is equivalent to having each customer take a number upon entering the shop. The semaphore `mutex1` protects access to the global variable `count` so each customer receives a unique number. The semaphore `finished` is redefined to be an array of 50 semaphores. Once a customer is seated in a barber chair, he executes `semWait(finished[custnr])` to wait on his own unique semaphore; when the barber is finished with that customer, the barber executes `semSignal(finished[b_cust])` to release the correct customer.

It remains to say how a customer's number is known to the barber. A customer places his number on the queue `enqueue1` just prior to signaling the barber with the semaphore `cust_ready`. When a barber is ready to cut hair, `dequeue1 (b_cust)` removes the top customer number from `queue1` and places it in the barber's local variable `b_cust`.

## A.3 PROBLEMS

**A.1.** Answer the following questions relating to the fair barbershop (see Figure A.3):

- **a.** Does the code require that the barber who finishes a customer's haircut collect that customer's payment?
- **b.** Do barbers always use the same barber chair?

**A.2.** A number of problems remain with the fair barbershop of Figure A.3. Modify the program to correct the following problems.

- **a.** The cashier may accept pay from one customer and release another if two or more are waiting to pay. Fortunately, once a customer presents payment, there is no way for him to un-present it, so in the end, the right amount of money ends up in the cash register. Nevertheless, it is desirable to release the right customer as soon as his payment is taken.
- **b.** The semaphore `leave_b_chair` supposedly prevents multiple access to a single barber chair. Unfortunately, this semaphore does not succeed in all cases. For example, suppose all three barbers have finished cutting hair and are blocked at `semWait(leave_b_chair)`. Two of the customers are in an interrupted state just prior to `leave barber chair`. The third customer leaves his chair and executes `semSignal (leave_b_chair)`. Which barber is released? Because the `leave_b_chair` queue is first-in-first-out, the first barber that was blocked is released. Is that the barber that was cutting the signaling customer's hair? Maybe, but maybe not. If not, then a new customer will come along and sit on the lap of a customer that was just about to get up.
- **c.** The program requires a customer first sits on the sofa even if a barber chair is empty. Granted, this is a rather minor problem, and fixing it makes code that is already a bit messy even messier. Nevertheless, give it a try.

# APPENDIX B

## PROGRAMMING AND OPERATING SYSTEM PROJECTS


**B.1 Semaphore Projects**

**B.2 File Systems Project**

**B.3 OS/161**

**B.4 Simulations**

**B.5 Programming Projects**

- Textbook-Defined Projects
- Additional Major Programming Projects
- Small Programming Projects

**B.6 Research Projects**

**B.7 Reading/Report Assignments**

**B.8 Writing Assignments**

**B.9 Discussion Topics**

**B.10 BACI**

Many instructors believe that implementation or research projects are crucial to the clear understanding of operating system concepts. Without projects, it may be difficult for students to grasp some of the basic OS abstractions and interactions among components; a good example of a concept that many students find difficult to master is that of semaphores. Projects reinforce the concepts introduced in this book, give the student a greater appreciation of how the different pieces of an OS fit together, and can motivate students and give them confidence that they are capable of not only understanding but also implementing the details of an OS.

In this text, I have tried to present the concepts of OS internals as clearly as possible and have provided numerous homework problems to reinforce those concepts. Many instructors will wish to supplement this material with projects. This appendix provides some guidance in that regard and describes support material available in the **Instructor's Resource Center (IRC)** for this book accessible from Pearson for instructors. The support material covers ten types of projects and other student exercises:

- Semaphore projects
- File systems project
- OS/161 projects
- Simulation projects
- Programming projects
- Research projects
- Reading/report assignments
- Writing assignments
- Discussion topics
- BACI

## B.1 SEMAPHORE PROJECTS

The ability to manage concurrency using semaphores is one of the most important topics of an operating systems or systems programming course, but it can be very difficult to teach. Concurrency problems, such as race conditions and deadlocks, are abstract concepts that can be difficult for a student to visualize. They can arise in many different situations from file locking to network communication. Not only is the material particularly difficult for students, but the complexity of the topic makes it difficult to develop projects which engage students and are appropriate for a single semester course. This problem is further complicated by the fact that concurrency problems which seem similar to students often have subtle differences that require very different approaches to their solution.

This IRC provides a set of hands-on activities which use an open-source train simulation game, OpenTTD, to visually represent concurrency problems. Each activity introduces one application of semaphores and allows the students to interact with the system by placing "real" semaphores along a train track carefully constructed to simulate a computing problem (such as a race condition).

These project assignments were developed by Professor Robert Marmorstein of Longwood University.

## B.2 FILE SYSTEMS PROJECT

Understanding file system implementation is another challenge for OS students. To support this goal, the IRC includes project in which, step-by-step, the student implements a simple file system in C++. This project assignment was developed by Professor Robert Marmorstein of Longwood University.

## B.3 OS/161

The **Instructor's Resource Center (IRC)** for this book provides support for using OS/161 as an active learning component.

OS/161 is an educational operating system developed at Harvard University [HOLL02]. It aims to strike a balance between giving students experience in working on a real operating system, and potentially overwhelming students with the complexity that exists in a fully fledged operating system, such as Linux. Compared to most deployed operating systems, OS/161 is quite small (approximately 20,000 lines of code and comments), and therefore it is much easier to develop an understanding of the entire code base.

The source code distribution contains a full operating system source tree, including the kernel, libraries, various utilities (ls, cat,...), and some test programs. OS/161 boots on the simulated machine in the same manner as a real system might boot on real hardware.

System/161 simulates a "real" machine to run OS/161 on. The machine features a MIPS R2000/R3000 CPU including an MMU, but no floating-point unit or cache. It also features simplified hardware devices hooked up to the system bus. These devices are much simpler than real hardware, and thus make it feasible for students to get their hands dirty without having to deal with the typical level of complexity of physical hardware. Using a simulator has several advantages: Unlike other software students write, buggy OS software may result in completely locking up the machine, making it difficult to debug and requiring a reboot. A simulator enables debuggers to access the machine below the software architecture level as if debugging was built into the CPU. In some senses, the simulator is similar to an in-circuit emulator (ICE) that you might find in industry, only it is implemented in software. The other major advantage is the speed of reboots. Rebooting real hardware takes minutes, and hence the development cycle can be frustratingly slow on real hardware. System/161 boots OS/161 in mere seconds.

The OS/161 and System/161 simulators can be hosted on a variety of platforms, including Unix, Linux, Mac OS X, and Cygwin (the free Unix environment for Windows).

The IRC includes the following:

- **Package for instructor's Web server:** A set of html and pdf files that can be easily uploaded to the instructor's site for the OS course, which provides all the

online resources for OS/161 and S/161 access, user's guides for students, assignments, and other useful material.

- **Getting started for instructors:** This guide lists all of the files that make up the website for the course and instructions on how to set up the website.
- **Getting started for students:** This guide explains to students step-by-step how to download and install OS/161 and S/161 on their PC.
- **Background material for students:** This consists of two documents that provide an overview of the architecture of S/161 and the internals of OS/161. These overviews are intended to be sufficient so that the student is not overwhelmed with figuring out what these systems are.
- **Student exercises:** A set of exercises that cover some of the key aspects of OS internals, including support for system calls, threading, synchronization, locks and condition variables, scheduling, virtual memory, files systems, and security.

The IRC OS/161 package was prepared by Andrew Peterson and other colleagues and students at the University of Toronto.

## B.4 SIMULATIONS

The IRC provides support for assigning projects based on a set of simulations developed at the University of Texas, San Antonio. Table B.1 lists the simulations by chapter. The simulators are all written in Java and can be run either locally as a Java application or online through a browser.

The IRC includes the following:

1. A brief overview of the simulations available.
2. How to port them to the local environment.
3. Specific assignments to give to students, telling them specifically what they are to do and what results are expected. For each simulation, this section provides one or two original assignments that the instructor can assign to students.

These simulation assignments were developed by Adam Critchley (University of Texas at San Antonio).

## B.5 PROGRAMMING PROJECTS

Three sets of programming projects are provided.

### Textbook-Defined Projects

Two major programming projects, one to build a shell, or command line interpreter, and one to build a process dispatcher, are described in the online portion of the textbook. The projects can be assigned after Chapter 3 and after Chapter 9, respectively. The IRC provides further information and step-by-step exercises for developing the programs.

**Table B.1** OS Simulations by Chapter

| | |
|---|---|
| **Chapter 5 – Concurrency: Mutual Exclusion and Synchronization** | |
| Producer-consumer | Allows the user to experiment with a bounded buffer synchronization problem in the context of a single producer and a single consumer |
| UNIX Fork-pipe | Simulates a program consisting of `pipe`, `dup2`, `close`, `fork`, `read`, `write`, and `print` instructions |
| **Chapter 6 – Concurrency: Deadlock and Starvation** | |
| Starving philosophers | Simulates the dining philosophers problem |
| **Chapter 8 – Virtual Memory** | |
| Address translation | Used for exploring aspects of address translation. It supports 1- and 2-level page tables and a translation lookaside buffer |
| **Chapter 9 – Uniprocessor Scheduling** | |
| Process scheduling | Allows users to experiment with various process scheduling algorithms on a collection of processes and to compare such statistics as throughput and waiting time |
| **Chapter 11 – I/O Management and Disk Scheduling** | |
| Disk head scheduling | Supports the standard scheduling algorithms such as FCFS, SSTF, SCAN, LOOK, C-SCAN, and C-LOOK as well as double buffered versions of these |
| **Chapter 12 – File Management** | |
| Concurrent I/O | Simulates a program consisting of `open`, `close`, `read`, `write`, `fork`, `wait`, `pthread_create`, `pthread_detach`, and `pthread_join` instructions |

These projects were developed by Ian G. Graham of Griffith University, Australia.

## Additional Major Programming Projects

A set of programming assignments, called machine problems (MPs), are available that are based on the Posix Programming Interface. The first of these assignments is a crash course in C, to enable the student to develop sufficient proficiency in C to be able to do the remaining assignments. The set consists of nine machine problems with different difficulty degrees. It may be advisable to assign each project to a team of two students.

Each MP includes not only a statement of the problem but a number of C files that are used in each assignment, step-by-step instructions, and a set of questions for each assignment that the student must answer that indicate a full understanding of each project. The scope of the assignments includes:

1. Create a program to run in a shell environment using basic I/O and string manipulation functions.
2. Explore and extend a simple Unix shell interpreter.
3. Modify faulty code that utilizes threads.
4. Implement a multithreaded application using thread synchronization primitives.

5. Write a user-mode thread scheduler.
6. Simulate a time-sharing system by using signals and timers.
7. A six-week project aimed at creating a simple yet functional networked file system. Covers I/O and file system concepts, memory management, and networking primitives.

The IRC provides specific instructions for setting up the appropriate support files on the instructor's website of local server.

These project assignments were developed at the University of Illinois at Urbana-Champaign, Department of Computer Science and adapted by Matt Sparks (University of Illinois at Urbana-Champaign) for use with this textbook.

### Small Programming Projects

The instructor can also assign a number of small programming projects described in the IRC. The projects can be programmed by the students on any available computer and in any appropriate language: They are platform and language independent.

These small projects have certain advantages over the larger projects. Larger projects usually give students more of a sense of achievement, but students with less ability or fewer organizational skills can be left behind. Larger projects usually elicit more overall effort from the best students. Smaller projects can have a higher concepts-to-code ratio, and because more of them can be assigned, the opportunity exists to address a variety of different areas. Accordingly, the IRC contains a series of small projects, each intended to be completed in a week or so, which can be very satisfying to both student and teacher. These projects were developed at Worcester Polytechnic Institute by Stephen Taylor, who has used and refined the projects in the course of teaching operating systems a dozen times.

## B.6 RESEARCH PROJECTS

An effective way of reinforcing basic concepts from the course and for teaching students research skills is to assign a research project. Such a project could involve a literature search as well as a Web search of vendor products, research lab activities, and standardization efforts. Projects could be assigned to teams or, for smaller projects, to individuals. In any case, it is best to require some sort of project proposal early in the term, giving the instructor time to evaluate the proposal for appropriate topic and appropriate level of effort. Student handouts for research projects should include:

- A format for the proposal.
- A format for the final report.
- A schedule with intermediate and final deadlines.
- A list of possible project topics.

The students can select one of the listed topics or devise their own comparable project. The IRC includes a list of possible research topics developed by Professor Tan N. Nguyen of George Mason University, and suggests the coverage to be provided in the proposal and final report.

## B.7 READING/REPORT ASSIGNMENTS

Another excellent way to reinforce concepts from the course and to give students research experience is to assign papers from the literature to be read and analyzed. The IRC includes a suggested list of papers to be assigned, organized by chapter. A PDF copy of each of the papers is available at box.com/OS8e. The IRC also includes a suggested assignment wording.

## B.8 WRITING ASSIGNMENTS

Writing assignments can have a powerful multiplier effect in the learning process in a technical discipline such as OS internals. Adherents of the Writing Across the Curriculum (WAC) movement (http://wac.colostate.edu/) report substantial benefits of writing assignments in facilitating learning. Writing assignments lead to more detailed and complete thinking about a particular topic. In addition, writing assignments help to overcome the tendency of students to pursue a subject with a minimum of personal engagement, just learning facts and problem-solving techniques without obtaining a deep understanding of the subject matter.

The IRC contains a number of suggested writing assignments, organized by chapter. Instructors may ultimately find this is an important part of their approach to teaching the material. I would greatly appreciate any feedback on this area, and any suggestions for additional writing assignments.

## B.9 DISCUSSION TOPICS

One way to provide a collaborative experience is discussion topics, a number of which are included in the IRC. Each topic relates to material in the book. The instructor can set it up so students can discuss a topic either in a class setting, an online chat room, or a message board. Again, I would greatly appreciate any feedback on this area, and any suggestions for additional discussion topics.

## B.10 BACI

In addition to all of the support provided at the IRC, the Ben-Ari Concurrent Interpreter (BACI) is a publicly available package that instructors may wish to use. BACI simulates concurrent process execution and supports binary and counting semaphores and monitors. BACI is accompanied by a number of project assignments to be used to reinforce concurrency concepts.

Appendix O provides a more detailed introduction to BACI, with information about how to obtain the system and the assignments.

*This page intentionally left blank*

# REFERENCES

**ABBREVIATIONS**

| | |
|---|---|
| ACM | Association for Computing Machinery |
| IBM | International Business Machines Corporation |
| IEEE | Institute of Electrical and Electronics Engineers |


**AGAR89** Agarwal, A. *Analysis of Cache Performance for Operating Systems and Multiprogramming*. Norwell, MA: Kluwer Academic Publishers, 1989.

**ANDE80** Anderson, J. *Computer Security Threat Monitoring and Surveillance.* Fort Washington, PA: James P. Anderson Co., April 1980.

**ANDE89** Anderson, T.; Lazowska, E.; and Levy, H. "The Performance Implications of Thread Management Alternatives for Shared-Memory Multiprocessors." *IEEE Transactions on Computers,* December 1989.

**ANDE04** Anderson, T.; Bershad, B.; Lazowska, E.; and Levy, H. "Thread Management for Shared-Memory Multiprocessors." In [TUCK04].

**ANDE05** Anderson, E. *μClibc.* Slide Presentation, Codepoet Consulting, January 26, 2005. http://www.codepoet-consulting.com/

**ARDE80** Arden, B., ed. *What Can Be Automated?* The Computer Science and Engineering Research Study, National Science Foundation, 1980.

**ATLA89** Atlas, A., and Blundon, B. "Time to Reach for It All." *UNIX Review*, January 1989.

**BACH86** Bach, M. *The Design of the UNIX Operating System.* Englewood Cliffs, NJ: Prentice Hall, 1986.

**BACO03** Bacon, J., and Harris, T. *Operating Systems: Concurrent and Distributed Software Design.* Reading, MA: Addison-Wesley, 2003.

**BAER80** Baer, J. *Computer Systems Architecture.* Rockville, MD: Computer Science Press, 1980.

**BACC13** Baccelli, E.; Hahm, O.; Wahlisch, M.; Gunes, M.; and Schmidt, T. "RIOT OS: Towards an OS for the Internet of Things." *Proceedings of IEEE INFOCOM, Demo/Poster for the 32nd IEEE International Conference on Computer Communications, Turin, Italy*, April 2013.

**BARK89** Barkley, R., and Lee, T. "A Lazy Buddy System Bounded by Two Coalescing Delays per Class." *Proceedings of the Twelfth ACM Symposium on Operating Systems Principles,* December 1989.

**BAYS77** Bays, C. "A Comparison of Next-Fit, First-Fit, and Best-Fit." *Communications of the ACM,* March 1977.

**BELA66** Belady, L. "A Study of Replacement Algorithms for a Virtual Storage Computer." *IBM Systems Journal,* No. 2, 1966.

**BLAC90** Black, D. "Scheduling Support for Concurrency and Parallelism in the Mach Operating System." *Computer,* May 1990.

**BOLO89** Bolosky, W.; Fitzgerald, R.; and Scott, M. "Simple but Effective Techniques for NUMA Memory Management." *Proceedings, Twelfth ACM Symposium on Operating Systems Principles,* December 1989.

**BONW94** Bonwick, J. "The Slab Allocator: An Object-Caching Kernel Memory Allocator." *Proceedings, USENIX Summer Technical Conference*, 1994.

**BORG90** Borg, A.; Kessler, R.; and Wall, D. "Generation and Analysis of Very Long Address Traces." *Proceedings of the 17th Annual International Symposium on Computer Architecture*, May 1990.

**BORM14** Bormann, C.; Ersue, M.; and Keranen, A. *Terminology for Constrained-Node Networks.* RFC 7228, May 2014.

**BRIA99** Briand, L., and Roy, D. *Meeting Deadlines in Hard Real-Time Systems: The Rate Monotonic Approach.* Los Alamitos, CA: IEEE Computer Society Press, 1999.

**BREN89** Brent, R. "Efficient Implementation of the First-Fit Strategy for Dynamic Storage Allocation." *ACM Transactions on Programming Languages and Systems*, July 1989.

**BRIN01** Brinch Hansen, P., ed. *Classic Operating Systems: From Batch Processing to Distributed Systems.* New York, NY: Springer-Verlag, 2001.

**BUON01** Buonadonna, P.; Hill, J.; and Culler, D. "Active Message Communication for Tiny Networked Sensors." *Proceedings, IEEE INFOCOM 2001*, April 2001.

**BUTT99** Buttazzo, G., Sensini, F. "Optimal Deadline Assignment for Scheduling Soft Aperiodic Tasks in Hard Real-Time Environments." *IEEE Transactions on Computers*, October 1999.

**CALL15** Callaway, B., and Esker, R. "OpenStack Deployment and Operations Guide." *NetApp White Paper*, May 2015.

**CARR84** Carr, R. *Virtual Memory Management.* Ann Arbor, MI: UMI Research Press, 1984.

**CARR89** Carriero, N., and Gelernter, D. "How to Write Parallel Programs: A Guide for the Perplexed." *ACM Computing Surveys*, September 1989.

**CARR01** Carr, S.; Mayo, J.; and Shene, C. "Race Conditions: A Case Study." *Journal of Computing in Colleges*, October 2001.

**CARR05** Carrier, B. *File System Forensic Analysis.* Upper Saddle River, NJ: Addison-Wesley, 2005.

**CHEN92** Chen, J.; Borg, A.; and Jouppi, N. "A Simulation-Based Study of TLB Performance." *Proceedings, 19th Annual International Symposium on Computer Architecture*, May 1992.

**CHOI05** Choi, H., and Yun, H. "Context Switching and IPC Performance Comparison between $\mu$Clinux and Linux on the ARM9 Based Processor." *Proceedings, Samsung Conference*, 2005.

**CHU72** Chu, W., and Opderbeck, H. "The Page Fault Frequency Replacement Algorithm." *Proceedings, Fall Joint Computer Conference,* 1972.

**CLAR85** Clark, D., and Emer, J. "Performance of the VAX-11/780 Translation Buffer: Simulation and Measurement." *ACM Transactions on Computer Systems,* February 1985.

**CLAR13** Clark, L. "Intro to Embedded Linux Part 1: Defining Android vs. Embedded Linux." *Libby Clark Blog*, Linux.com, March 6, 2013.

**COFF71** Coffman, E.; Elphick, M.; and Shoshani, A. "System Deadlocks." *Computing Surveys,* June 1971.

**COME79** Comer, D. "The Ubiquitous B-Tree." *Computing Surveys*, June 1979.

**CONW63** Conway, M. "Design of a Separable Transition-Diagram Compiler." *Communications of the ACM*, July 1963.

**CORB62** Corbato, F.; Merwin-Daggett, M.; and Daley, R. "An Experimental Time-Sharing System." *Proceedings of the 1962 Spring Joint Computer Conference*, 1962. Reprinted in [BRIN01].

**CORB68** Corbato, F. "A Paging Experiment with the Multics System." *MIT Project MAC Report MAC-M-384,* May 1968.

**CORB07** Corbet, J. "The SLUB Allocator." April 2007. http://lwn.net/Articles/229984/

**CORM09** Cormen, T., et al. *Introduction to Algorithms.* Cambridge, MA: MIT Press, 2009.

**COX89** Cox, A., and Fowler, R. "The Implementation of a Coherent Memory Abstraction on a NUMA Multiprocessor: Experiences with PLATINUM." *Proceedings, Twelfth ACM Symposium on Operating Systems Principles,* December 1989.

**DALE68** Daley, R., and Dennis, R. "Virtual Memory, Processes, and Sharing in MULTICS." *Communications of the ACM*, May 1968.

**DASG91** Dasgupta, P., et al. "The Clouds Distributed Operating System." *IEEE Computer*, November 1991.

**DENN68** Denning, P. "The Working Set Model for Program Behavior." *Communications of the ACM,* May 1968.

**DENN70** Denning, P. "Virtual Memory." *Computing Surveys*, September 1970.

**DENN71** Denning, P. "Third Generation Computer Systems." *ACM Computing Surveys*, December 1971.

**DENN80a** Denning, P.; Buzen, J.; Dennis, J.; Gaines, R.; Hansen, P.; Lynch, W.; and Organick, E. "Operating Systems." In [ARDE80].

**DENN80b** Denning, P. "Working Sets Past and Present." *IEEE Transactions on Software Engineering*, January 1980.

**DIJK65** Dijkstra, E. *Cooperating Sequential Processes*. Technological University, Eindhoven, The Netherlands, 1965. Reprinted [LAPL96] and in [BRIN01].

**DIJK71** Dijkstra, E. "Hierarchical Ordering of Sequential Processes." *Acta informatica*, Volume 1, Number 2, 1971. Reprinted in [BRIN01].

**DONG10** Dong, W., et al. "Providing OS Support for Wireless Sensor Networks: Challenges and Approaches." *IEEE Communications Surveys & Tutorials*, Fourth Quarter, 2010.

**DOWN16** Downey, A. *The Little Book of Semaphores Version 2.2.1.* 2016. www.greenteapress.com/semaphores/

**DUBE98** Dube, R. *A Comparison of the Memory Management Sub-Systems in FreeBSD and Linux*. Technical Report CS-TR-3929, University of Maryland, September 25, 1998.

**EISC07** Eischen, C. "RAID 6 Covers More Bases." *Network World*, April 9, 2007.

**EMCR15** EmCraft Systems. "What Is the Minimal Footprint of μClinux?" *EmCraft Documentation*, May 19, 2015. http://www.emcraft.com/stm32f429discovery/what-is-minimal-footprint

**ETUT16** eTutorials.org. *Embedded Linux Systems.* 2016. http://etutorials.org/Linux+systems/embedded+linux+systems/

**FEIT90a** Feitelson, D., and Rudolph, L. "Distributed Hierarchical Control for Parallel Processing." *Computer,* May 1990.

**FEIT90b** Feitelson, D., and Rudolph, L. "Mapping and Scheduling in a Shared Parallel Environment Using Distributed Hierarchical Control." *Proceedings, 1990 International Conference on Parallel Processing,* August 1990.

**FERR83** Ferrari, D., and Yih, Y. "VSWS: The Variable-Interval Sampled Working Set Policy." *IEEE Transactions on Software Engineering,* May 1983.

**FINK88** Finkel, R. *An Operating Systems Vade Mecum,* Second edition. Englewood Cliffs, NJ: Prentice Hall, 1988.

**FOST91** Foster, I. "Automatic Generation of Self-Scheduling Programs." *IEEE Transactions on Parallel and Distributed Systems*, January 1991.

**FRAN97** Franz, M. "Dynamic Linking of Software Components." *Computer*, March 1997.

**FREN16** Frenzel, L. "12 Wireless Options for IoT/M2M: Diversity or Dilemma?" *Electronic Design*, June 2016.

**GANA98** Ganapathy, N., and Schimmel, C. "General Purpose Operating System Support for Multiple Page Sizes." *Proceedings, USENIX Symposium*, 1998.

**GAY03** Gay, D., et al. "The nesC Language: A Holistic Approach to Networked Embedded Systems." *Proceedings of the ACM SIGPLAN 2003 Conference on Programming Language Design and Implementation*, 2003.

**GEHR87** Gehringer, E.; Siewiorek, D.; and Segall, Z. *Parallel Processing: The Cm* Experience.* Bedford, MA: Digital Press, 1987.

**GING90** Gingras, A. "Dining Philosophers Revisited." *ACM SIGCSE Bulletin*, September 1990.

**GOLD89** Goldman, P. "Mac VM Revealed." *Byte,* November 1989.

**GOYE99** Goyeneche, J., and Souse, E. "Loadable Kernel Modules." *IEEE Software*, January/February 1999.

**GRAH72** Graham, G., and Denning, P. "Protection—Principles and Practice." *Proceedings, AFIPS Spring Joint Computer Conference*, 1972.

**GROS86** Grosshans, D. *File Systems: Design and Implementation.* Englewood Cliffs, NJ: Prentice Hall, 1986.

**GUPT78** Gupta, R., and Franklin, M. "Working Set and Page Fault Frequency Replacement Algorithms: A Performance Comparison." *IEEE Transactions on Computers,* August 1978.

**HAHM15** Hahm, O.; Baccelli, E.; Petersen, H.; and Tsiftes, N. "Operating Systems for Low-End Devices in the Internet of Things: A Survey." *IEEE Internet of Things Journal*, December 2015.

**HALD91** Haldar, S., and Subramanian, D. "Fairness in Processor Scheduling in Time Sharing Systems." *Operating Systems Review*, January 1991.

**HAND98** Handy, J. *The Cache Memory Book, Second edition.* San Diego, CA: Academic Press, 1998.

**HARR06** Harris, W. "Multi-Core in the Source Engine." bit-tech.net technical paper, November 2, 2006. bit-tech.net/gaming/2006/11/02/Multi_core_in_the_Source_Engin/1

**HENR84** Henry, G. "The UNIX System: The Fair Share Scheduler." *AT&T Bell Laboratories Technical Journal*, October 1984.

**HERL90** Herlihy, M. "A Methodology for Implementing Highly Concurrent Data Structures." *Proceedings of the Second ACM SIGPLAN Symposium on Principles and Practices of Parallel Programming*, March 1990.

**HILL00** Hill, J., et al. "System Architecture Directions for Networked Sensors." *Proceedings, Architectural Support for Programming Languages and Operating Systems*, 2000.

**HOAR74** Hoare, C. "Monitors: An Operating System Structuring Concept." *Communications of the ACM*, October 1974.

**HOLL02** Holland, D.; Lim, A.; and Seltzer, M. "A New Instructional Operating System." *Proceedings of SIGCSE 2002*, 2002.

**HOLT72** Holt, R. "Some Deadlock Properties of Computer Systems." *Computing Surveys,* September 1972.

**HOWA73** Howard, J. "Mixed Solutions for the Deadlock Problem." *Communications of the ACM*, July 1973.

**HUCK83** Huck, T. *Comparative Analysis of Computer Architectures.* Stanford University Technical Report Number 83-243, May 1983.

**HUCK93** Huck, J., and Hays, J. "Architectural Support for Translation Table Management in Large Address Space Machines." *Proceedings of the 20th Annual International Symposium on Computer Architecture,* May 1993.

**HYMA66** Hyman, H. "Comments on a Problem in Concurrent Programming Control." *Communications of the ACM*, January 1966.

**ISLO80** Isloor, S., and Marsland, T. "The Deadlock Problem: An Overview." *Computer*, September 1980.

**IYER01** Iyer, S., and Druschel, P. "Anticipatory Scheduling: A Disk Scheduling Framework to Overcome Deceptive Idleness in Synchronous I/O." *Proceedings, 18th ACM Symposium on Operating Systems Principles*, October 2001.

**JACK10** Jackson, J. "Multicore Requires OS Rework, Windows Architect Advises." *Network World*, March 19 2010.

**JOHN92** Johnson, T., and Davis, T. "Space Efficient Parallel Buddy Memory Management." *Proceedings, Fourth International Conference on Computers and Information*, May 1992.

**JONE80** Jones, A., and Schwarz, P. "Experience Using Multiprocessor Systems—A Status Report." *Computing Surveys,* June 1980.

**JONE97** Jones, M. "What Really Happened on Mars?" http://research.microsoft.com/~mbj/Mars_Pathfinder/Mars_Pathfinder.html, 1997.

**KATZ89** Katz, R.; Gibson, G.; and Patterson, D. "Disk System Architecture for High Performance Computing." *Proceedings of the IEEE*, December 1989.

**KAY88** Kay, J., and Lauder, P. "A Fair Share Scheduler." *Communications of the ACM*, January 1988.

**KERN16** Kerner, S. " Inside the Box: Can Containers Simplify Networking?" *Network Evolution*, February 2016.

**KESS92** Kessler, R., and Hill, M. "Page Placement Algorithms for Large Real-Indexed Caches." *ACM Transactions on Computer Systems*, November 1992.

**KHAL93** Khalidi, Y.; Talluri, M.; Williams, D.; and Nelson, M. "Virtual Memory Support for Multiple Page Sizes." *Proceedings, Fourth Workshop on Workstation Operating Systems*, October 1993.

**KHUS12** Khusainov, V. "Practical Advice on Running $\mu$Clinux on Cortex-M3/M4." *Electronic Design*, September 17, 2012.

**KILB62** Kilburn, T.; Edwards, D.; Lanigan, M.; and Sumner, F. "One-Level Storage System." *IRE Transactions*, April 1962.

**KLEI95** Kleiman, S., Eykholt, J. "Interrupts as Threads." *Operating System Review*, April 1995.

**KLEI96** Kleiman, S.; Shah, D.; and Smallders, B. *Programming with Threads.* Upper Saddle River, NJ: Prentice Hall, 1996.

**KNUT71** Knuth, D. "An Experimental Study of FORTRAN Programs." *Software Practice and Experience,* Vol. 1, 1971.

**KNUT97** Knuth, D. *The Art of Computer Programming, Volume 1: Fundamental Algorithms.* Reading, MA: Addison-Wesley, 1997.

**KNUT98** Knuth, D. *The Art of Computer Programming, Volume 3: Sorting and Searching.* Reading, MA: Addison-Wesley, 1998.

**LAMP71** Lampson, B. "Protection." *Proceedings, Fifth Princeton Symposium on Information Sciences and Systems*, March 1971; Reprinted in *Operating Systems Review*, January 1974.

**LAMP74** Lamport, L. "A New Solution to Dijkstra's Concurrent Programming Problem." *Communications of the ACM,* August 1974.

**LAMP80** Lampson, B., and Redell D. "Experience with Processes and Monitors in Mesa." *Communications of the ACM*, February 1980.

**LAMP91** Lamport, L. "The Mutual Exclusion Problem Has Been Solved." *Communications of the ACM,* January 1991.

**LAPL96** Laplante, P., ed. *Great Papers in Computer Science*. New York, NY: IEEE Press, 1996.

**LARO92** LaRowe, R.; Holliday, M.; and Ellis, C. "An Analysis of Dynamic Page Placement in a NUMA Multiprocessor." *Proceedings, 1992 ACM SIGMETRICS and Performance '92,* June 1992.

**LEBL87** LeBlanc, T., and Mellor-Crummey, J. "Debugging Parallel Programs with Instant Replay." *IEEE Transactions on Computers,* April 1987.

**LEON07** Leonard, T. "Dragged Kicking and Screaming: Source Multicore." *Proceedings, Game Developers Conference 2007*, March 2007.

**LERO76** Leroudier, J., and Potier, D. "Principles of Optimality for Multiprogramming." *Proceedings, International Symposium on Computer Performance Modeling, Measurement, and Evaluation,* March 1976.

**LETW88** Letwin, G. *Inside OS/2.* Redmond, WA: Microsoft Press, 1988.

**LEUT90** Leutenegger, S., and Vernon, M. "The Performance of Multiprogrammed Multiprocessor Scheduling Policies." *Proceedings, Conference on Measurement and Modeling of Computer Systems,* May 1990.

**LEVI12** Levis, P. "Experiences from a Decade of TinyOS Development." *10th USENIX Symposium on Operating Systems Design and Implementation*, 2012.

**LEVI16** Levin, J. "GCD Internals." *Mac OS X and iOS Internals: To the Apple's Core.* newosxbook.com, 2016.

**LEWI96** Lewis, B., and Berg, D. *Threads Primer.* Upper Saddle River, NJ: Prentice Hall, 1996.

**LHEE03** Lhee, K., and Chapin, S., "Buffer Overflow and Format String Overflow Vulnerabilities." *Software: Practice and Experience*, Volume 33, 2003.

**LIGN05** Ligneris, B. "Virtualization of Linux Based Computers : The Linux-VServer Project." *Proceedings of the 19th International Symposium on High Performance Computing Systems and Applications*, 2005.

**LIU73** Liu, C., and Layland, J. "Scheduling Algorithms for Multiprogramming in a Hard Real-time Environment." *Journal of the ACM*, January 1973.

**LOVE04** Love, R. "I/O Schedulers." *Linux Journal*, February 2004.

**MACK05** Mackall, M. "Slob: Introduce the SLOB Allocator." November 2005. http://lwn.net/Articles/157944/

**MAEK87** Maekawa, M.; Oldehoeft, A.; and Oldehoeft, R. *Operating Systems: Advanced Concepts.* Menlo Park, CA: Benjamin Cummings, 1987.

**MAJU88** Majumdar, S.; Eager, D.; and Bunt, R. "Scheduling in Multiprogrammed Parallel Systems." *Proceedings, Conference on Measurement and Modeling of Computer Systems,* May 1988.

**MARW06** Marwedel, P. *Embedded System Design.* Dordrecht, The Netherlands: Springer, 2006.

**MCCU04** McCullough, D. "$\mu$Clinux for Linux Programmers." *Linux Journal*, July 2004.

**MCDO06** McDougall, R., and Laudon, J. "Multi-Core Microprocessors Are Here." *;login:*, October 2006.

**MCDO07** McDougall, R., and Mauro, J. *Solaris Internals: Solaris 10 and OpenSolaris Kernel Architecture.* Palo Alto, CA: Sun Microsystems Press, 2007.

**MCKU15** McKusick, M.; Neville-Neil, J.; and Watson, R. *The Design and Implementation of the FreeBSD Operating System.* Upper Saddle River, NJ: Addison-Wesley, 2015.

**MENA07** Menage, P. "Adding Generic Process Containers to the Linux Kernel." *Linux Symposium*, June 2007.

**MESN03** Mesnier, M.; Ganger, G.; and Riedel, E. "Object-Based Storage." *IEEE Communications Magazine*. August 2003.

**MIN02** Min, R., et al. "Energy-Centric Enabling Technologies for Wireless Wensor Networks." *IEEE wireless communications*, vol. 9, no. 4, 2002.

**MORG92** Morgan, K. "The RTOS Difference." *Byte*, August 1992.

**MORR16** Morra, J. "Google Rolls Out New Version of Android Operating System." Electronic Design, August 24, 2016.

**MOSB02** Mosberger, D., and Eranian, S. *IA-64 Linux Kernel: Design and Implementation.* Upper Saddle River, NJ: Prentice Hall, 2002.

**MS96** Microsoft Corp. *Microsoft Windows NT Workstation Resource Kit.* Redmond, WA: Microsoft Press, 1996.

**NELS91** Nelson, G. *Systems Programming with Modula-3.* Englewood Cliffs, NJ: Prentice Hall, 1991.

**NIST08** National Institute of Standards and Technology. *Guide to General Server Security.* Special Publication 800-124, July 2008.

**OUST85** Ousterhout, J., et al. "A Trace-Drive Analysis of the UNIX 4.2 BSD File System." *Proceedings, Tenth ACM Symposium on Operating System Principles*, 1985.

**PABL09** Pabla, C. "Completely Fair Scheduler." *Linux Journal*, August 2009.

**PARK13** Parker-Johnson, P. "Getting to Know OpenStack Neutron: Open Networking in Cloud Services." *TechTarget article*, December 13, 2013. http://searchtelecom.techtarget.com/tip/Getting-to-know-OpenStack-Neutron-Open-networking-in-cloud-services

**PATT82** Patterson, D., and Sequin, C. "A VLSI RISC." *Computer,* September 1982.

**PATT85** Patterson, D. "Reduced Instruction Set Computers." *Communications of the ACM*, January 1985.

**PATT88** Patterson, D.; Gibson, G.; and Katz, R. "A Case for Redundant Arrays of Inexpensive Disks (RAID)." *Proceedings, ACM SIGMOD Conference of Management of Data,* June 1988.

**PAZZ92** Pazzini, M., and Navaux, P. "TRIX, A Multiprocessor Transputer-Based Operating System." *Parallel Computing and Transputer Applications*, edited by M.Valero et al., Barcelona, Spain: IOS Press/CIMNE, 1992.

**PEIR99** Peir, J.; Hsu, W.; and Smith, A. "Functional Implementation Techniques for CPU Cache Memories." *IEEE Transactions on Computers*, February 1999.

**PETE15** Petersen, H., et al. "Old Wine in New Skins? Revisiting the Software Architecture for IP Network Stacks on Constrained IoT Devices." *ACM MobiSys Workshop on IoT Challenges in Mobile and Industrial Systems (IoTSys)*, May 2015.

**PIZZ89** Pizzarello, A. "Memory Management for a Large Operating System." *Proceedings, International Conference on Measurement and Modeling of Computer Systems,* May 1989.

**PETE77** Peterson, J., and Norman, T. "Buddy Systems." *Communications of the ACM*, June 1977.

**PETE81** Peterson, G. "Myths about the Mutual Exclusion Problem." *Information Processing Letters,* June 1981.

**PRZY88** Przybylski, S.; Horowitz, M.; and Hennessy, J. "Performance Trade-offs in Cache Design." *Proceedings, Fifteenth Annual International Symposium on Computer Architecture,* June 1988.

**RAMA94** Ramamritham, K., and Stankovic, J. "Scheduling Algorithms and Operating Systems Support for Real-Time Systems." *Proceedings of the IEEE*, January 1994.

**RASH88** Rashid, R., et al. "Machine-Independent Virtual Memory Management for Paged Uniprocessor and Multiprocessor Architectures." *IEEE Transactions on Computers,* August 1988.

**RAYN86** Raynal, M. *Algorithms for Mutual Exclusion.* Cambridge, MA: MIT Press, 1986.

**REIM06** Reimer, J. "Valve Goes Multicore." *Ars Technica*, November 5, 2006. arstechnica.com/articles/paedia/cpu/valve-multicore.ars

**RITC74** Ritchie, D., and Thompson, K. "The UNIX Time-Sharing System." *Communications of the ACM*, July 1974.

**RITC78** Ritchie, D. "UNIX Time-Sharing System: A Retrospective." *The Bell System Technical Journal,* July–August 1978.

**RITC84** Ritchie, D. "The Evolution of the UNIX Time-Sharing System." *AT&T Bell Labs Technical Journal*, October 1984.

**ROBE03** Roberson, J. "ULE: A Modern Scheduler for FreeBSD." *Proceedings of BSDCon '03*, September 2003.

**ROBI90** Robinson, J., and Devarakonda, M. "Data Cache Management Using Frequency-Based Replacement." *Proceedings, Conference on Measurement and Modeling of Computer Systems,* May 1990.

**ROME04** Romer, K., and Mattern, F. "The Design Space of Wireless Sensor Networks." *IEEE Wireless Communications*, December 2004.

**ROSA14** Rosado, T., and Bernardino, J. "An Overview of OpenStack Architecture." *ACM IDEAS '14*, July 2014.

**RUSS11** Russinovich, M.; Solomon, D.; and Ionescu, A. *Windows Internals: Covering Windows 7 and Windows Server 2008 R2.* Redmond, WA: Microsoft Press, 2011.

**SARA11** Saraswat, L., and Yadav, P. "A Comparative Analysis of Wireless Sensor Network Operating Systems." *The 5th National Conference; INDIACom*, 2011.

**SATY81** Satyanarayanan, M. and Bhandarkar, D. "Design Trade-Offs in VAX-11 Translation Buffer Organization." *Computer,* December 1981.

**SAUE81** Sauer, C., and Chandy, K. *Computer Systems Performance Modeling.* Englewood Cliffs, NJ: Prentice Hall, 1981.

**SEFR12** Serfaoui, O.; Aissaoui, M.; and Eleuldj, M. "OpenStack: Toward an Open-Source Solution for Cloud Computing." *International Journal of Computer Applications*, October 2012.

**SEGH12** Seghal, A., et al. "Management of Resource Constrained Devices in the Internet of Things." *IEEE Communications Magazine*, December 2012.

**SHA91** Sha, L.; Klein, M.; and Goodenough, J. "Rate Monotonic Analysis for Real-Time Systems." in [TILB91].

**SHA94** Sha, L.; Rajkumar, R.; and Sathaye, S. "Generalized Rate-Monotonic Scheduling Theory: A Framework for Developing Real-Time Systems." *Proceedings of the IEEE*, January 1994.

**SHAH15** Shah, A. "Smart Devices Could Get a Big Battery Boost from ARM's New Chip Design." *PC World*, June 1, 2015.

**SHEN02** Shene, C. "Multithreaded Programming Can Strengthen an Operating Systems Course." *Computer Science Education Journal*, December 2002.

**SHOR75** Shore, J. "On the External Storage Fragmentation Produced by First-Fit and Best-Fit Allocation Strategies." *Communications of the ACM*, August, 1975.

**SHUB90** Shub, C. "ACM Forum: Comment on a Self-Assessment Procedure on Operating Systems." *Communications of the ACM*, September 1990.

**SHUB03** Shub, C. "A Unified Treatment of Deadlock." *Journal of Computing in Small Colleges*, October 2003. Available through the ACM Digital Library.

**SILB04** Silberschatz, A.; Galvin, P.; and Gagne, G. *Operating System Concepts with Java.* Reading, MA: Addison-Wesley, 2004.

**SIRA09** Siracusa, J. "Grand Central Dispatch." *Ars Technica Review*, 2009. http://arstechnica.com/apple/reviews/2009/08/mac-os-x-10-6.ars/12

**SMIT82** Smith, A. "Cache Memories." *ACM Computing Surveys,* September 1982.

**SMIT85** Smith, A. "Disk Cache—Miss Ratio Analysis and Design Considerations." *ACM Transactions on Computer Systems*, August 1985.

**SOLT07** Soltesz, S., et al. "Container-Based Operating System Virtualization: A Scalable High-Performance Alternative to Hypervisors." *Proceedings of the EuroSys 2007 2*nd *EuroSys Conference, Operating Systems Review*, June 2007.

**STAL16a** Stallings, W. *Computer Organization and Architecture,* 10th ed. Upper Saddle River, NJ: Pearson, 2016.

**STAL16b** Stallings, W. *Foundations of Modern Networking: SDN, NFV, QoE, IoT and Cloud.* Upper Saddle River, NJ: Pearson, 2016.

**STAN14** Stankovic, J. "Research Directions for the Internet of Things." *Internet of Things Journal*, Volume 1, Number 1, 2014.

**STEE95** Steensgarrd, B., and Jul, E. "Object and Native Code Mobility among Heterogeneous Computers." *Proceedings, 15th ACM Symposium on Operating Systems Principles*, December 1995.

**STRE83** Strecker, W. "Transient Behavior of Cache Memories." *ACM Transactions on Computer Systems,* November 1983.

**TAKA01** Takada, H. "Real-Time Operating System for Embedded Systems." In Imai, M. and Yoshida, N. eds. *Asia South-Pacific Design Automation Conference*, 2001.

**TALL92** Talluri, M.; Kong, S.; Hill, M.; and Patterson, D. "Tradeoffs in Supporting Two Page Sizes." *Proceedings of the 19th Annual International Symposium on Computer Architecture*, May 1992.

**TAMI83** Tamir, Y., and Sequin, C. "Strategies for Managing the Register File in RISC." *IEEE Transactions on Computers*, November 1983.

**TANE78** Tanenbaum, A. "Implications of Structured Programming for Machine Architecture." *Communications of the ACM,* March 1978.

**TAUR12** Tauro, C.; Ganesan, N.; and Kumar, A. "A Study of Benefits in Object Based Storage Systems." *International Journal of Computer Applications*, March 2012.

**TEVA87** Tevanian, A., et al. "Mach Threads and the UNIX Kernel: The Battle for Control." *Proceedings, Summer 1987 USENIX Conference*, June 1987.

**TILB91** Tilborg, A., and Koob, G. eds. *Foundations of Real-Time Computing: Scheduling and Resource Management*. Boston: Kluwer Academic Publishers, 1991.

**TIME02** TimeSys Corp. "Priority Inversion: Why You Care and What to Do about It." *TimeSys White Paper*, 2002. https://linuxlink.timesys.com/docs/priority_inversion

**TUCK89** Tucker, A., and Gupta, A. "Process Control and Scheduling Issues for Multiprogrammed Shared-Memory Multiprocessors." *Proceedings, Twelfth ACM Symposium on Operating Systems Principles,* December 1989.

**TUCK04** Tucker, A. ed. *Computer Science Handbook,* Second Edition. Boca Raton, FL: CRC Press, 2004.

**VAHA96** Vahalia, U. *UNIX Internals: The New Frontiers.* Upper Saddle River, NJ: Prentice Hall, 1996.

**WARD80** Ward, S. "TRIX: A Network-Oriented Operating System." *Proceedings, COMPCON '80*, 1980.

**WARR91** Warren, C. "Rate Monotonic Scheduling." *IEEE Micro*, June 1991.

**WEIZ81** Weizer, N. "A History of Operating Systems." *Datamation,* January 1981.

**WEND89** Wendorf, J.; Wendorf, R.; and Tokuda, H. "Scheduling Operating System Processing on Small-Scale Microprocessors." *Proceedings, 22nd Annual Hawaii International Conference on System Science*, January 1989.

**WIED87** Wiederhold, G. *File Organization for Database Design.* New York, NY: McGraw-Hill, 1987.

**WOOD86** Woodside, C. "Controllability of Computer Performance Tradeoffs Obtained Using Controlled-Share Queue Schedulers." *IEEE Transactions on Software Engineering*, October 1986.

**WOOD89** Woodbury, P. et al. "Shared Memory Multiprocessors: The Right Approach to Parallel Processing." *Proceedings, COMPCON Spring '89,* March 1989.

**ZAHO90** Zahorjan, J., and McCann, C. "Processor Scheduling in Shared Memory Multiprocessors." *Proceedings, Conference on Measurement and Modeling of Computer Systems,* May 1990.

**ZHUR12** Zhuravlev, S., et al. "Survey of Scheduling Techniques for Addressing Shared Resources in Multicore Processors." *ACM Computing Surveys*, November 2012.

# CREDITS

**Chapter 2:** p. 102 Figure adapted from Russinovich, M.; Solomon, D.; and Ionescu, A. Windows Internals: Covering Windows 7 and Windows Server 2008 R2. Redmond, WA: Microsoft Press, 2011; p. 116 Figure adapted from Mosberger, David, Eranian, Stephane, IA-64 Linux Kernel: Design and Implementation, 1st Ed., (c) 2002. Reprinted and Electronically reproduced by permission of Pearson Education, Inc., Upper Saddle River, NJ 07458;

**Chapter 3:** p. 167 Figure adapted from Bach, Maurice J., Design of the UNIX Operating System, 1st Ed., (c) 1986.

**Chapter 4:** p. 182 Figure adapted from Kleiman, Steve; Shah, Devang; Smaalders, Bart, Programming with Threads, 1st Ed., ©1996; p. 163 Figure adapted from McDougall, R., and Mauro, J. Solaris Internals: Solaris 10 and OpenSolaris Kernel Architecture, 2nd Ed., ©2007. Reprinted and Electronically reproduced by permission of Pearson Education, Inc., Upper Saddle River, New Jersey; p. 198 Figure adapted from Russinovich, M.; Solomon, D.; and Ionescu, A. Windows Internals: Covering Windows 7 and Windows Server 2008 R2. Redmond, WA: Microsoft Press, 2011; p. 203 Figure adapted from McDougall, R., and Mauro, J. Solaris Internals: Solaris 10 and OpenSolaris Kernel Architecture, 2nd Ed., ©2007. Reprinted and Electronically reproduced by permission of Pearson Education, Inc., Upper Saddle River, New Jersey; p. 204 Figure adapted from Lewis, Bil; Berg, Daniel J., Threads Primer: A Guide To Multithreaded Programming, 1st Ed., © 1996. Reprinted and Electronically reproduced by permission of Pearson Education, Inc., Upper Saddle River, New Jersey;

**Chapter 5:** p. 250 Figure adapted from Bacon, J., and Harris, T. Operating Systems: Concurrent and Distributed Software Design. Reading, MA: Addison-Wesley, 2003; p. 276 Box adapted from Conway, M. "Design of a Separable Transition-Diagram Compiler." Communications of the ACM, July 1963; p. 287 Problem adapted from John Trono, St. Michael's College, Vermont.

**Chapter 6:** p. 294 Figure adapted from Bacon, Jean; Harris, Tim, Operating Systems: Concurrent and Distributed Software Design, 1st Ed., (c) 2003. Reprinted and Electronically reproduced by permission of Pearson Education, Inc., Upper Saddle River, NJ 07458;

**Chapter 8:** p. 402 Figure adapted from Denning, p. "Virtual Memory." Computing Surveys, September 1970. AND Denning, p. "Working Sets Past and Present." IEEE Transactions on Software Engineering, January 1980; p. 403 Figure adapted from Maekawa, Mamoru; Oldehoeft, Arthur; Oldehoeft, Rodney, Operating Systems: Advanced Concepts, 1st Ed., (c) 1987. Reprinted and Electronically reproduced by permission of Pearson Education, Inc., Upper Saddle River, NJ 07458;

**Chapter 9:** p. 437 Example adapted from Finkel, R. An Operating Systems Vade Mecum. Englewood Cliffs, NJ: Prentice Hall, 1988;

**Chapter 12:** p. 593 List adapted from Russinovich, M.; Solomon, D.; and Ionescu, A. Windows Internals: Covering Windows 7 and Windows Server 2008 R2. Redmond, WA: Microsoft Press, 2011;

**Chapter 13:** p. 617 List adapted from Hill, J., et al. "System Architecture Directions for Networked Sensors." Proceedings, Architectural Support for Programming Languages and Operating Systems. 2000;

**Chapter 16:** p. 714 Figure adapted based on Figures 2.1 from Callaway, B., and Esker, R. "OpenStack Deployment and Operations Guide." NetApp White Paper, May 2015. https://community.netapp.com/fukiw75442/attachments/fukiw75442/virtualization-and-cloud-articles-and-resources/450/1/openstack-deployment-ops-guide.pdf.

**Appendix B:** p. B-2 Section adapted from Professor Robert Marmorstein, Longwood University; p. B-3 adapted from Professor Robert Marmorstein, Longwood University; p. B-4 Section adapted from Adam Critchley, University of Texas, San Antonio; p. B-4 Section adapted from Ian G. Graham, Griffith University, Australia; p. B-5 Section was developed at the University of Illinois at Urbana-Champaign, Department of Computer Science and adapted by Matt Sparks (University of Illinois at Urbana-Champaign); p. B-6 Section adapted from Steven Taylor, Worcester Polytechnic Institute; p. B-6 Section adapted from Professor Tan N. Nguyen, George Mason University (The IRC includes a suggested format for the proposal and final report as well as a list of possible research topics)

# INDEX


**A**
Absolute loading, 364–365
Access
  efficiency, 65
  matrix, 670
  methods for file systems, 555
  rights for file sharing, 569–570
  security scheme, 687
  time, 518
  token, 687–688
Access control, 661, 670–678
  categories of, 672–673
  commands, 676
  discretionary, 672, 673–676
  file systems, 670–672
  function of, 675
  lists, 680–681
  mandatory, 672
  matrix of, 670
  policies, 672–678
  protection and, 87
  role-based, 673, 676–678
  structures of, 671
  UNIX systems, 678–681
Access control lists, 680–681
  discretionary, 689
  system, 689
  UNIX systems, 680–681
Access control policy, 672
Access matrix, 671
Accounting information, 132
Accumulator (AC), 34
Activity manager, Android, 119
Address binding, 365
Address translation
  for paging system, 376, 377
  in segmentation, 387, 388
Addresses. *See also* Address translation; Virtual addresses
  executable, space protection, 669
  logical, 353, 355
  physical, 353
  real, 371
  registers, 31, 32
  space, 207, 371
  space randomization, 669
Addressing, 89
  direct, 266
  indirect, 266
  indirect process communication, 266
  Linux virtual memory, 414
  many-to-one relationship, 266
  message passing, 266
  one-to-many relationship, 266
  one-to-one relationship, 266
  requirements of, 341
  translation of, 357
  virtual memory, 414
Advanced local procedure call (ALPC) facility, 35
Alignment check, 155
All users class, 570
Amazon Elastic Compute Cloud (Amazon EC2), 699, 716
AMD64, 417
Amdahl's law, 190
Analyzers for intrusion detection, 660
Android
  activities, 126, 211
  activity state, 213–214
  applications, 214
  architecture, 119–121
  file management, 594–595
  framework, 119–121
  interprocess communication, 330–331
  killing an application, 214
  and Linux, 614–615
  memory management, 419–420
  power management, 126
  process, 211–215
  runtime, 120
  services, 211
  system libraries, 120–128
  threads, 211–215

Anticipatory input/output scheduler, 542–543
Aperiodic tasks, 474, 480
API, 699, 716
Appending access rights, 569
Application binary interface (ABI), 71
Application catalog (Murano), 720
Application processors, 602
Application programming interface (API), 71
Architecture
  client/server model, 104–105
  file management systems, 554–555
  Linux VServer, 653
  microkernel, 92
  Microsoft Windows, 101–104
  UNIX systems, 109
Archive, 686
Assignment of processes to processors, 463–464
Associative lookup for page table, 382
Associative mapping, 380
Asynchronous input/output, Windows, 545–546
Asynchronous procedure call (APC), 545
Asynchronous processing, 180–181
Atomic bitmap operations, 317, 318
Atomic integer operations, 317
Atomic operations, 225, 316–318
AT&T, 109
Attribute definition table, 592
Authentication, 660–661
  computer security, 660–661
  steps of, 660
  of user's identification, 660, 661
  verification step of, 660
Authenticity of information, 90
Automatic allocation, 87
Automatic management, 87
Auxiliary carry flag, 155
Availability of information, 90
Available state, 417
Avoidance approaches for operating systems, 296
Awareness, degrees of, 236

**B**

Backbone network, 723
Background work, 180
Backup, 686
Balancing resources, 431
Ballooning, 645
Banker's algorithm, 302
Barbershop problem, A–37–42
Bare-metal provisioning (Ironic), 720
Basic buffer overflow, 663
Basic file systems, 551–552
Basic input/output supervisor, 555
Basic spinlocks, 318–319
Batch systems
  multiprogrammed, 77–80
  simple, 74–77
Bell Labs, 108
Berkeley Software Distribution (BSD), 109
Best fit strategy, 349, 573
_bh, 319
Binary semaphores, 244, 246, 247, 320
Bit tables, 577–578
Bitmap operations, Linux atomic, 316–317
Block device drivers, 118
Block diagram, 58, 510
Block operation, 181
Block Storage (Cinder), 711, 717
Blocked process, 144–145
Blocked state, 142, 166–167
Blocked/suspended process, 146
Blocked/waiting process state, 139
Blocking, 265
  fixed, 570
  permanent, 290
  record, 570–572
Block-oriented device, 514
Blocks, 50, 215, 573
  boot, 584
  data, 585
  defined, 215
  dispatched, 215

function of, 215
process control, 132–133
scheduled, 215
size of, 51
Boot block, 584
Boot loader, 607
Bottom half code, 318
Bottom-half kernel threads, 494
Bounded-buffer monitor code, 261
Bounded-buffer producer/consumer problem, 258, 260
Broad network access, 697
Broadcase receivers, Android, 212
B-trees, 561–564
characteristics of, 562
definition of, 562
nodes into, insertion of, 564
properties of, 562
rules for, 563
Buddy system, 351–352
algorithms of, 351
example of, 352
tree representation of, 353
Buffer cache, 588–589
UNIX system, 537–538
Buffer overflow, 662–670
attacks, 662–666
basic, example of, 663
examples, 663
runtime defenses, 668–670
stack values, 664
Buffer overflow attacks, 662–666
compile-time defenses, 666–668
dealing with, 666–670
defending against, 666
run-time defenses, 668–670
Buffer overrun. *See* Buffer overflow
Buffer registers
input/output buffer register (I/OBR), 31
memory buffer register (MBR), 31–32
Buffering, 397–398, 514–517
Busy waiting technique, 243

**C**

C implementation of UNIX systems, 108
Cache levels, 52
Cache manager, 103, 544, 593
Cache memory, 49–53, 533. *See also* Disk cache
block size, 52
blocks, 50
cache size, 52
categories of, 52
design of, 51–52
main memory and, 50–51
mapping function, 52
motivation, 49
principles of, 50–51
read operation of, 51–52
replacement algorithm, 52
slots, 50
write policy, 52
Cache operation, 382
Cache size, 52, 398
Canary value, 668
Capability tickets, 672
Carry flag, 155
Central processing unit (CPU), 30
Chain pointer, 378–379
Chained allocation, 575–576
Chained free portions, 578
Changing protection access rights, 569
Character device drivers, Linux, 118
Character queue, UNIX SVR 32, 540
Chbind, 652
Chcontext, 652
Child process, 138
Chip, 602
Chip multiprocessor, 30–58
Chroot, 652
Circular buffer, 516–517
Circular SCAN (C-SCAN) policy, 524
Circular wait process, deadlock prevention using, 298–300
Clandestine user, 659
Classes
all users, 570
availability, 96
of interrupts, 35
kernel (99-60), 493
objects, 106
priority, 494–495, 498
real time (159-100), 493

Classes (*continued*)
real-time priority, 498
specific user, 570
time-shared (50-0), 493
user groups, 570
variable priority, 498
Cleaning policy, 405
Client-server model, 104–105
Clock algorithm, 396–397, 410, 415
Clock interrupt, 160
Clock page, 410
Clock replacement policy, 394–395
Cloned () process, 208
Closing files, 552
Cloud auditor, 702, 703
Cloud broker, 702, 703
areas of support, 703
Cloud carrier, 702, 703
Cloud computing
cloud deployment models, 699–701
cloud operating systems, 704–720
cloud service models, 698–699
definition, 696
elements, 696–698
interactions between actors, 704
reference architecture, 701–704
Cloud computing elements, 696–698
models and characteristics, 697
Cloud context and IoT
cloud, 724
core, 723–724
edge, 722
fog, 723
Cloud deployment models, comparison, 701
community cloud, 700–701
hybrid cloud, 701
private cloud, 700
public cloud infrastructure, 699–700
Cloud operating systems, 704–720
definition, 704
general architecture of, 707–713
Infrastructure as a Service (IaaS), 705–706
OpenStack, 713–720
requirements for, 706–707
Cloud service consumer (CSC), 697, 698, 700, 702, 703
Cloud service models
Infrastructure as a Service (IaaS), 698–699
NIST, 698
Platform as a Service (PaaS), 698
Software as a Service (SaaS), 698
Cloud service provider (CSP), 702, 703
Cloud *vs.* fog features, 724
Clouds, 190
Cluster bit map, 591
Clusters, 461, 590
multiprocessor system, 461
sizes of, 591
Coarse parallelism, 462–463
Coarse threading, 193
Codecs, 32
Commands, TinyOS, 619
Commercial operating systems, 608
Committed state, 418–419
Communication
cooperation among processes by, 239–240
devices, 507
indirect process, 266, 267
interprocess, 264
lines, 539, 540
Community cloud, 700–701
Compaction of memory, 349
compare&swap instruction, 242–243
Compatible Time-Sharing System (CTSS), 81
Competition, 236
Compile-time defenses, 666–668
language extensions, safe libraries and, 667–668
programming language choices, 666
safe coding techniques, 666–667
stacking protection mechanisms, 668
Completely Fair Queuing input/output scheduler, 543
Completely Fair Scheduler (CFS), 491–492
Completion deadline, 480
Compute (Nova), 715–717
Computer systems. *See also* Operating systems (OS)
basic elements of, 30–32
cache memory, 49–53
direct memory access, 53–54

instruction execution, 32–35
interrupts, 35–45
memory hierarchy, 46–49
microprocessor, 32, 54–58
overview of, 29–67
top-level components of, 31
Computer-aided design (CAD), 63
Concurrency, 289–331, A–29–42
barbershop problem, A–37–42
contexts of, 224–225
deadlock, 289–331
Dekker's algorithm, 226–231
dining philosophers problem, 309–313
example of, 233–235
Linux kernel, mechanisms of, 315–323
message passing, 263–270
monitors, 257–263
mutual exclusion, 226–232
operating systems, concerns of, 235–236
Peterson's algorithm, 231–232
principles of, 232–240
process interaction, 236–240
race conditions of, 235
readers/writers problems, 270–274
semaphores, 244–257, A–37–42
Solaris thread synchronization, primitives of, 324–326
terms related to, 225
UNIX, mechanisms of, 313–315
Windows 7, mechanisms of, 326–329
Concurrent process, simultaneous, 98
Concurrent threads, simultaneous, 98
Condition codes, 155
Condition variables, 258, 326, 329
monitors, 257
Confidentiality, of information, 90
Configurability, 605
Configuration manager, Windows, 103
Consolidation ratio, 630
Constrained Application Protocol (CoAP), 725
Constrained device, 724–725
Consumable resources, deadlock and, 295–296
Container virtualization, 635–642
concepts, 636–639
Docker, 641–642
file system, 639–640
kernel control groups, 636
microservices, 641
Containers (Magnum), 720
Content providers, Android, 119, 211–212
Context data, 132
Contiguous allocation, 574–575
Control, 33
bits, 155, 378
complexity of, 507
load, 406–407
mode, 157
objects, Windows, 107
operating system, structures of, 149–150
process, 157–162
scheduling and, 512
status registers and, 153, 154
user, 475
Control bits, 155, 378
Control mode, 158
Control objects, Windows, 107
Cooperation, 237
Cores, 32, 57
Coroutines, 313
Countermeasures, intruders
access control, 661
authentication, 660–661
firewalls, 661–662
intrusion detection, 660
Counting (general) semaphores, 246, 320
Create file operation, 566
Creation of files, 552
Critical resource, 237
Critical sections, 225, 328–329
CSC. *See* Cloud service consumer (CSC)
C-SCAN (circular SCAN) policy, 524
csignal (c), 261
Currency mechanisms, 244
cwait (c), 258

**D**

Dashboard (Horizon), 719
Data
block, 585
Context, 132
directory, 594
integrity, 90

Data (*continued*)
memory, external fragmentation of, 349
processing, 33
rate, 507
semaphores and, 250
set of, 132
SIMD techniques, 32
table entry, page frame, 409–410
transfer capacity, RAID level 0 for high, 529
Data structure management, 711–712
Database, 553
Database (Trove), 719
DDR2 (double data rate) memory controller, 57
Deadline scheduler, 540–542
Deadline scheduling, 479–483
design issues, 480
real-time scheduling, 479–483
for tasks, 479–483
Deadlines, 431
Deadlock avoidance, 296, 300–306
logic of, 305
process initiation denial, 301–302
resource allocation denial, 302–306
restrictions of, 306
Deadlock detection, 296, 306–308
algorithm of, 306–307
recovery, 307–308
Deadlock prevention, 296, 299–300
circular wait condition, 300
hold and wait condition, 299–300
mutual exclusion, 299
no preemption condition, 297–298, 300
Deadlocks, 85, 225
conditions for, 297–299
consumable resources, 295–296
errors in process, 85
example of, 292
execution paths of, 293
illustration of, 291
integrated strategy for, 308
no, example of, 294
principles of, 290–299
resource allocation graphs, 296–297
reusable resources, 294–295
Decision mode, 434
Dedicated processor, 602
Dedicated processor assignment, 470–471
Dedicated resources, 623
Deeply embedded systems, 604–605
Default ACL, 688
Default owner, 688
Degrees of awareness, 236
Delay variable, 412
Delete access, 690
Delete file operation, 566
Deletion access rights, 569
Deletion of files, 552
Demand cleaning policy, 405
Demand paging, 390
Dentry object, Linux, 586, 588
Design issues
with deadline scheduling, 480
of disk cache, 533–536
for embedded operating systems, 605–606
of input/output, 511–513
with multiprocessor scheduling, 463–465
Determinism, 475
Device drivers, 102, 554
Device input/output, 512
Device list, 537
Die, 57
Differential responsiveness, 90
Digital Signal Processors (DSPs), 32
Dining philosophers problem, 309–313
dining arrangement, for philosophers, 310
monitors, solutions using, 310–313
semaphores, solutions using, 310
Direct addressing, 266
Direct Attached Storage (DAS), 709
Direct (hashed) file, 561
Direct lookup for page table, 382
Direct memory access (DMA)
block diagram, 510
configurations for, alternative, 511
input/output operations, techniques for, 509–511
Direction flag, 155
Directories
attributes, 592
cache, 594
file, 581
management, 512–513
system, 594

tree, Android, 594
UNIX, 584
Disabled interrupts, 44–45
Disabled/disabling, 241
Discretionary access control (DAC), 672, 673–676
Discretionary access control list (DACL), 689
Disk allocation tables, 576
Disk block descriptors, 409
Disk cache, 61, 533–536
design issues of, 533–535
performance issues of, 535–536
Disk drives, 539
Disk duplexing, 546
Disk performance parameters, 517–520
rotational delay, 518
seek time, 518
timing comparison, 519–520
Disk scheduling
algorithms for, 521, 522
anticipatory input/output scheduler, 542–543
Completely Fair Queuing input/output scheduler, 543
deadline scheduler, 540–542
disk performance parameters, 517–520
elevator scheduler, 540
input/output management and, 505–547
NOOP scheduler, 543
policies for, 520–524
Disk storage, 590–592
Dispatch queues, 493
Dispatched blocks, 215
Dispatcher objects, 107, 327–328
Dispatcher program, 133
Distributed multiprocessor system, 461
Distributed operating systems, 94
Distributed processing, 224
DMA. *See* Direct memory access (DMA)
DNS service (Designate), 720
Docker client, 641
Docker engine, 642
Docker host, 642
Docker hub, 642
Docker image, 641
Docker machine, 642
Docker registry, 642
Downtime, 95
Driver input/output queue, 537
Dynamic allocation, 572–573
Dynamic best effort scheduling, 479
Dynamic biometrics, 661
Dynamic link libraries (DLLs), 104, 368–369
Dynamic linker, 368–369
Dynamic linking, Linux, 114, 368–369
Dynamic partitioning for memory, 348–351
effect of, 348
placement algorithm, 349–350
replacement algorithm, 350–351
Dynamic planning-based scheduling, 476, 479
Dynamic run-time loading, 366–367
Dynamic scheduling, 472

**E**

Efficiency, 90, 511
EFLAGS register, Pentium, 155–156
Elastic map reduce (Sahara), 719
Elevator scheduler, 540
Embedded operating systems, 599–625
advantages, 612
Android, 614–615
application processors *vs.* dedicated processors, 602
boot loader, 607
characteristics of, 605–609
commercial operating systems, adapting to existing, 608
compilation, 608
deeply embedded systems, 604–605
definition of, 600
degree of user interaction, 611
design issues for, 605–606
development approaches, 608
elements of, 601
file system, 611–612
host and target environments, 606–608
kernel size, 609–610
memory size, 610
microcontrollers, 603–604
microprocessors, 602–603
networkability, 611
organization of, 601

Embedded (*continued*)
purpose-built, 608–609
requirements/constraints of, 601
root file system, 607
time constraints, 610
TinyOS, 615–625
Emerald system, 190
Encapsulation, 106
Encryption, volume, 546
Enforcing priorities, 431
Enterprise Edition (J2EE platform), 193
Environmental subsystems, Windows, 104
Errors in process, causes of, 84
deadlocks, 85
mutual exclusion, failed, 84–85
program operation, nondeterminate, 85
synchronization, improper, 84
Event flags, 244
Event object, Windows, 328, 545
Events, TinyOS, 620
Exchange instruction, 243
Executable address space protection, 669
Executable program, 85
Executables (EXEs), 104
Execution
access rights, 569
context (process state), 85
modules of, 101
of object-oriented design, 106
paths of deadlock, 293
of process, 177
process control, modes of, 157–162
of Solaris threads, 204–205
speed of, 181
stack, 179
state, 206
Executive stage, 34–35
Exit process state, 140
Exponential averaging, 440, 442
External fragmentation of memory data, 349

**F**

Fail-soft operation, 476
Fairness, 90, 431
Fair-share scheduling, 450–452
Fatal region, 293
Fault tolerance, 95–97
concepts, 95–96
faults, 96–97
OS mechanisms, 96, 97
Faulting processing, 407
Faults, 118
permanent, 96
spatial (partial) redundancy, 96–97
temporal redundancy, 97
temporary, 96
Feedback, 443–445
Feedback scheduling, 444
Fetch policy, 390–391
Fetch stage, 33–34
Fetches, 32
Fiber, 195
Field, input/output files, 552
File allocation, 572–579
dynamic allocation *vs.* preallocation, 572–573
methods of, 574–576
portion size, 573–574
UNIX, 583–584
File allocation table (FAT), 572
File directories, 564–568
contents of, 564–565
elements of, 565
naming, 567–568
structure of, 566–567
tree-structured, 567, 568
working, 568
File management systems, 550–596
Android, 594–595
architecture of, 554–555
elements of, 551
file sharing, 569–570
functions of, 555–557
Linux virtual file system (VFS), 585–589
objectives of, 554
overview of, 551–557
record blocking, 570–572
requirements of, minimal, 554
secondary storage management, 572–580
security. *See* File system security
UNIX, 580–585

File object, Linux, 545, 586, 588
File organization/access, 557–561
  criteria for, 557
  direct file, 561
  hash file, 561
  indexed file, 560–561
  indexed sequential file, 559–560
  pile, 558–559
  sequential file, 559
  types of, common, 557, 558
File system
  container, 639–640
File system security
  access control lists, 672
  access control structures, 671
  capability tickets, 672
File systems, 207, 513, 551–552
  drivers, 544
  isolation, 652
File tables, 149
  allocation table (FAT), 572
  volume master, 591
File-based storage, 711
Files, 551
  allocation. *See* File allocation
  closing, 552
  creation of, 552
  deletion of, 552
  direct, 561
  directories. *See* File directories
  field, input/output, 552
  indexed, 560–561
  indexed sequential, 559–560
  links, 581
  log, 591
  long-term existence of, 551
  management. *See* File management systems
  MFT 30, 591
  naming, 567–568
  object, Linux, 545, 586, 588
  opening, 552
  operations performed on, 552
  ordinary, 580
  organization/access. *See* File organization/access
  pile, 558–559
  properties of, 551–552
  reading, 552
  regular, 580
  sequential. *See* Sequential files
  sharing, 238–239, 342, 551, 569–570
  special, 581
  structure, 552–553, 590–592
  symbolic links, 581, 590
  tables, 149, 572, 591
  tree-structured, 567, 568
  UNIX, 580–585
  UNIX FreeBSD, structure of, 582
  writing, 552
Fine-grained parallelism, 463
Fine-grained threading, 193
Finish operation, 181
Finite circular buffer, for producer/consumer problem, 255
Firewall, 661–662
First fit strategy, 349, 573
First-come-first-served (FCFS), 435–437, 455, 468
First-in-first-out (FIFO) policy, 247, 394, 520
Five-state process model, 138–143
  states of, 139
  transitions of, 141–142
Fixed allocation
  local page, 396
  local scope, 400
  replacement policy, 399
Fixed blocking, 570
Fixed function units, 32
Fixed partitioning for memory, 344–347
  partition size, 344–345
  placement algorithm, 345–347
Flags, 688
Flexibility of input/output devices, 605
Foreground work, 180
FORTRAN programs, 62
Four page replacement algorithm, behavior of, 393
Frame, 340, 355, 356
  locking, 392
Free block list, 578–579
Free frame, 355
Free list, 537

Free Software Foundation (FSF), 113
Free space management, 576–579
  bit tables, 577–578
  chained free portions, 578
  free block list, 578–579
  indexing, 578
FREE state, 205
Frequency-based replacement, 534
FSCAN policy, 524
Functionally specialized multiprocessor system, 461
Functions
  access control, 675
  blocks, 215
  file management systems, 555–557
  kernel (nucleus), 158
  linking, 367
  loading, 364
  MAC OS Grand Central Dispatch (GCD), 217
  mapping, 52, 53
  Microsoft Windows input/output, 544–545
  operating systems (OS), 69–73
  processor, 31
  resource management in OS, scheduling and, 92
  selection, 433
  support, 157
  threads, 181–183
  wait, 326–327
Fuzzing, 665

**G**
Gang scheduling, 469–470
Gateways, 722
GCC (GNU Compiler Collection), 668
General message format, 267
General semaphores, 246
Generic_all access bits, 690
Glance, 717
Global replacement policy, 399
Global scope, 400
Google Compute Engine (GCE), 699
Governance (Congress), 719
Grand Central Dispatch (GCD), 99–100
Granularity, 461–463
Graphical Processing Units (GPUs), 32
Group, SIDs, 688
Guard pages, 670

**H**
Hamming code, 530
Handspread, 411
Hard affinity process, 499
Hard links, 590
Hard real-time task, 474
Hardware
  device drivers, 545
  interrupt processing, 41–42
  RAID, 546
  relocation, 354
  simple batch systems, 77
  virtual memory (paging), 367–369, 371–388
Hardware abstraction layer (HAL), 101
Hardware virtualization, 628
Hash table, 538
Hashed file, 561
Hexadecimal digit, 34
Highest response ratio next (HRRN), 443
High-level language (HLL), 71
Hit ratio *(H),* 46
Hold and wait process, deadlock prevention using, 299–300
Host-based IDS, 660
Hosting platform, 653
Human readable devices, 506
Hybrid cloud, 701
Hybrid public/private cloud, 701
Hybrid threading, 193–194
Hypervisor, 631–635
  and container, 638
  functions, 632
  hardware-assisted virtualization, 635
  paravirtualization, 634–635
  type-1, 632–633
  type-2, 633–634
  virtual appliance, 635

**I**
IBM personal computer (PC), 81, 113
Identification flag, 155
Identification step of authentication, 660
Identifiers, 132, 206

Identity (Keystone), 718
Idle user, 494
If statements, 262
Image (Glance), 717
In-circuit emulator (ICE), B–31
Incremental growth, 56
Independence, and conditional probability, 41–43
Independent parallelism, 462
Index register, 85
Indexed allocation, 576
Indexed files, 560–561
Indexed sequential files, 559–560
Indexing, 578
Indirect addressing, 266
Indirect process communication, 266–267
Individual processors, 464–465
Infinite buffer for producer/consumer problem, 251, 252, 254
Information, 89–90, 206
Information technology (IT), 721
Infrastructure as a Service (IaaS), 698–699, 705–706
  conceptual framework, 705
  CSP functional requirements for, 707
Inheritance, 106
Inode object, 586, 587–588
Inodes, UNIX, 581–583
  elements of, 581–583
  FreeBSD, structure of, 583
Input/output (I/O)
  address register (I/OAR), 31
  address registers, 31
  anticipatory scheduler, 542–543
  asynchronous, Windows, 545–546
  basic, 544–545
  channel, 509
  Completely Fair Queuing input/output scheduler, 543
  completion ports, 545
  design issues with, 511–513
  devices. *See* Input/output (I/O) devices
  direct memory access, 509–511
  disk cache, 533–536
  disk scheduling, 505–547
  driver queues, 537
  evolution of, 508–509
  field files, 552
  file system, logical, 512, 555
  function, organization of, 508–511
  interrupt, 160, 508
  Linux, 540–544
  logical structure of, 512–513
  management, 157
  manager, 103, 544, 592
  model of, 513
  modules, 31, 32
  NOOP scheduler, 543
  organization of, 508–511
  performing, techniques for, 508
  physical, 555
  processor, 32, 508
  program/programmed, 36, 53–54, 508
  RAID, 524–533
  scheduling, 426
  status information, 132
  supervisor, basic, 555
  tables, 149
  UNIX SVR 4 input/output, 537–540
  virtual machine, 645–647
  Windows, 544–546
Input/output buffer register (I/OBR), 32
Input/output (I/O) buffering, 514–517
  circular buffer, 516–517
  double buffer, 516
  single buffer, 514–516
  utility of, 517
Input/output (I/O) devices
  data rates of, 507
  flexibility of, 605
  types of, 506–508
Instantiation of objects, 106
Instruction cycle, 33, 37–40
Instruction execution, 32–35. *See also* Direct memory access (DMA)
  categories of, 33
  characteristics of, 34
  executive stage of, 33
  fetch stage of, 33
  partial program execution, 34–35
  steps of, 33
Instruction register (IR), 33
Instruction set architecture (ISA), 71
Instructor's Resource Center (IRC), B–31–32

Integer operations, atomic, 317
Integrated circuit, 602
Integrated strategy for deadlock, 308
Intel Core i7, 57, 58
Interactive scoring, 496–497
Interactive threads, 496
Interfaces
  application binary, 71
  native system, 104
  resource, 623–625
  TinyOS resource, 623–625
  of typical operating systems, 71
  user, in intrusion detection systems, 660
  user/computer, 70–71
Internal fragmentation, 373, 383
Internal registers of processor, 31–32
Internal resources, 309
Internet of Things (IoT)
  and cloud context, 722–724
  evolution, 721
  key components, 721–722
  operating systems, 724–731
    architecture, 727–728
    constrained devices, 724–725
    requirements, 726–727
    RIOT, 728–731
  things on, 720
  transceivers, 720
Internet-connected infrastructure, 696
Interprocess communication (IPC), 101, 207, 264
Interrupt processing, 41–43
  hardware events of, sequence of, 41–42
  memory for, changes in, 42–43
  operations of, 42
  registers for, changes in, 42–43
  simple, 41
Interrupt service routine (ISR), 45
Interrupt-driven input/output, 508
Interruptible state, 208
Interrupts, 35–45, 77, 160. *See also specific types of*
  classes of, 35
  direct use of, 606
  disabled/disabling, 44–45
  enable flag, 155
  handler, 39
  and instruction cycle, 37–41
  multiple, 43–45
  program flow of control with/without, 36–37
  request, 37
  Solaris threads, 205–206
  stage, 38
  WRITE call, 36–37
  WRITE instruction, 37
Intruders, 658–659
Intrusion, 660
Intrusion detection
  sensors for, 660
Intrusion detection systems (IDS), 660
  analyzers, 660
  host-based, 660
  network-based, 660
  user interface, 660
Inverted page tables, 377–379
I/O. *See* Input/output (I/O)
IOPL (I/O privilege level), 155
IoT. *See* Internet of Things (IoT)
IoT-enabled devices, 721
_irq, 318
_irqsave, 318

**J**

Jacketing, 187
Java 2 Platform, 192
Java Application Server, 192
Java Virtual Machine (JVM), 651–652
Java VM, 651–652
Job control language (JCL), 76
Job, serial processing, 74
Joint progress diagram, 291–292
Journaling, 590

**K**

Kernel memory allocation
  Linux, 416–417
  Solaris, 407, 411–413
  UNIX, 407, 411–413
Kernel-level threads (KLT), 187–188
Kernels, 72
  compilation, 607, 608
  control groups, 636
  control objects, 107
  functions of, 158

input/output manager, 544
Linux. *See* Linux kernels
memory allocation. *See* Kernel memory allocation
microkernels, 92
Microsoft Windows, 102
mode, 77, 157
modules, 114, 607
monolithic, 92
nonprocess, 163–164
RIOT structure, 729–730
UNIX systems, 109
Key field for sequential files, 559
Key management (Barbican), 719
Knowledge access rights, 569

**L**

Language extensions, 667–668
Largest process, 407
Last process activated, 407
Last-in-first-out (LIFO) implementation, 151, 153
Lazy buddy system algorithm, 412–413
Least frequently used policy (LFU), 415, 534
Least recently used (LRU) policy, 392–393, 533–534
Lightweight processes (LPW), 202, 203. *See also* Threads
Lines of memory, 50
Linkage editor, 368
Linking, 367–369
dynamic linker, 368–369
function of, 367
linkage editor, 368
Links, 207
Links file, 581
Linux, 113–118, 413–417, 613. *See also* Linux virtual file system (VFS); Linux VServer
2.4, 490
2.6, 491
and Android, 121, 122–123
character device drivers, 118
dentry object, 586, 588
dynamic linking, 114, 368–369
embedded, 611–612
file object, 545, 586, 588
history of, 113
input/output, 540–544
loadable modules, 114, 368
$\mu$Clinux, 612–614, 615
memory barrier operations, 322
modular structure of, 114–116
page cache, 543–544
scheduling. *See* Linux scheduling
semaphores, 321
spinlocks, 319
tasks, 206–208
threads, 208–209
virtual machine process scheduling, 653–654
VServer, architecture, 652–654
Linux kernels
concurrency mechanisms, 315–313
memory allocation, 416
Linux scheduling, 489–492
non-real-time scheduling, 490–492
real-time scheduling, 489–490
Linux virtual file system (VFS), 585–589
context of, 585
dentry object, 588
file object, 588
inode object, 587–588
object types in, 586
superblock object, 587
Linux virtual memory, 414–417
page allocation, 415
page replacement algorithm, 415
virtual memory addressing, 414
Linux VServer
applications running on, 653
architecture of, 652–653
chbind, 652
chcontext, 652
chroot, 652
file system isolation, 652
hosting platform, 653
network isolation, 652, 653
process isolation, 652
root isolation, 653
token bucket filter (TBF), 653–654
virtual machine architecture, 652–654
virtual platform, 653
virtual servers, 652, 653

List directory operation, 566
Livelocks, 225
Load control, 406–407
Load sharing, 467–469
Loadable modules, Linux, 114, 368
  absolute, 364–365
  characteristics of, 114
  kernel modules, 114, 115
  module table, elements of, 115
Loading, 364–367
  absolute, 364–365
  addressing binding, 365
  approaches to, 364
  dynamic run-time, 366–367
  function of, 363
  modules, 364
  relocatable, 365–366
Load-time dynamic linking, 368
Local replacement policy, 399
Local scope, 401–405
Locality of references, 48, 62–64, 373–374
  principle of, 374
  spatial, 63
  temporal, 63
Location manager, Android, 120
Lock-free synchronization, 329
Log file, NTFS, 591
Log file service, 592
Logging, 685–686
Logic bomb, 659
Logical address, 353, 355
Logical input/output file system, 512–513, 555
Logical organization, 343
Long-term existence of files, 551
Long-term scheduling, 426, 427–429
Long-term storage, 87
Loosely coupled multiprocessor system, 461
Loosely coupled service, 461
Lotus Domino, 192
Lowest-priority process, 407

**M**

MAC OS Grand Central Dispatch (GCD), 215–217
  blocks, 215
  codes for, 216
  functions of, 217
  purpose of, 215
Mac OS X, 99
Mach 3.0, 112
Machine readable devices, 506
Mailboxes, 244
Main memory, 30, 32, 49–50, 309
Main memory cache, 61
Malicious software, 659
Management and orchestration (MANO), 712–713
Mandatory access control (MAC), 672
Many-to-many relationships, 189–190
Many-to-one relationships, 266–267
Mapping function, cache memory, 52, 53
Marshalling, 331
Masquerader, 658
Master file table (MFT), 591
Matrix of access control, 673
$\mu$Clibc, 614, 615
$\mu$Clinux, 612–614, 615
Mean time to failure (MTTF), 95
Medium-grained parallelism, 463
Medium-term scheduling, 426, 429
Memory
  auxiliary, 49
  cache, 49–53, 533
  compaction of, 349
  dynamic partitioning for, 348–351
  fault, 160–161
  for interrupt processing, changes in, 42–43
  layout for resident monitor, 75
  Linux virtual, 414–416
  main, 31, 49–50, 309
  physical, 118
  processor, 32
  protection, 77
  real, 373
  secondary, 49
  shared, 314
  tables, 149–150
  two-level, 61–67
  virtual, 51, 87–88, 118, 370–420
Memory address register (MAR), 31
Memory buffer register (MBR), 31–32

Memory hierarchy, 46–49
auxiliary memory, 49
hit ratio, 46
levels of, 46–48
locality of reference, 48
secondary memory, 49
in software, 49
two-level memory, 46–47, 93
Memory management, 80, 157, 339–369
Android, 419
buffer overflow, 662–666
definition of, 340
formats for, typical, 375
Linux, 413–417
memory partitioning, 344–354
in OS, 87–89
paging, 355–358
read address, 88
requirements of, 340–344
security issues, 662–670
segmentation, 358–359
Solaris, 407–413
storage management responsibilities of, 87
UNIX, 407–413
UNIX SVR4, parameters of, 408–409
virtual address, 88
virtual machine, 644–645
virtual memory, 87–88
Windows, 417–419
Memory management unit (MMU), 669
Memory partitioning, 344–354
buddy system, 351–352
dynamic partitioning, 348–351
fixed partitioning, 344–347
relocation, 352–354
Mesa
resident, 79
security reference, 103
with signal, 257–261
simple batch systems, 74–77
structure of, 258
Mesa monitors, 261
Message passing, 263–270
addressing, 266–267
blocking, 265
distributed, 692–694
implementation of, 265
for interprocess communication, design characteristics of, 264
message format, 267–268
mutual exclusion, 268–270
nonblocking, 265
producer/consumer problem using, solution to bounded-buffer, 269
queuing discipline, 268
synchronization, 264–265
Messages, 314. *See also* Mailboxes
format, 267–268
mutual exclusion, 268–270
Messaging service (Zaqar), 719
MFT2 files, 591
Microcontrollers, 603–604
Micro-electromechanical sensors (MEMS), 616
Microkernels, 92
Microprocessor
cores, 32
Digital Signal Processors (DSPs), 32
evolution of, 32
Graphical Processing Units (GPUs), 32
multicore computer (chip multiprocessor), 57–58
and multicore organization, 54–57
sockets, 32
symmetric (SMP), 55–57
System on a Chip (SoC), 32
techniques, 32
Microprocessors, 602–603
Microservices, 641
Microsoft
DOS, 101
Xenix System V, 110
Microsoft Windows. *See also* Microsoft Windows 7*;* Microsoft Windows 8
architecture of, 101–104
asynchronous input/output, 545–546
client-server model, 104–105
input/output, 544–546
kernel-mode components of, 101–104
memory management, 417–419
object-oriented design, 106
scheduling, 498–500
symmetric multiprocessing (SMP), threads for, 105

Microsoft Windows 7
concurrency mechanisms of, 326–329
synchronization objects, 327
Microsoft Windows 8
characteristics of, 197
object-oriented design of, 198–199
subsystems of, support for, 201
thread objects, 198–199
Microsoft Windows Azure, 699
MIPS, 384
Misfeasor, 659
Modern operating systems (OS)
development leading to, 92–94
distributed operating system, 94
microkernel architecture, 92
monolithic kernel, 92
multiprocessing, 92–93
multiprogramming, 93
multithreading, 93
object-oriented design, 94
process, 93
symmetric multiprocessing (SMP), 93
Modes, 77
control, 157
decision, 62
kernel, 77, 158
nonpreemptive, 434
preemptive, 434
switching, 161–162
system, 158
user, 77, 158
Modular program structure, 181
Modular programming, 87
Modular structure of Linux, 114–115
Modules. *See also specific types of;*
of execution, 101
input/output, 31, 32
kernel, 114
loadable, Linux, 368
rendering, 193–194
stackable, 114
table, elements of, 115
Monitor (Ceilometer), 719
Monitor point of view, 75
Monitors, 75, 257–263
alternate model of, with notify and broadcast, 261–263
bounded-buffer producer/consumer problem, 256
characteristics of, 257
concurrency, 257–263
condition variables, 258
dining philosophers problem, solutions using, 310–312
Monolithic kernel, 92
Motherboard, 602
Motivation, 49, 203
MS-DOS, 101
Multicore computer
DDR3 (double data rate) memory controller, 57
elements of, 603
Intel Core i7, example of, 57–58
multicore computer (chip multiprocessor), 57–58
multithreading of, 190–195
operating systems, 98–100
QuickPath Interconnect (QPI), 57–58
software on, 190–195
support, 495–497
Valve game software, application example, 193–194
Multicore organization, 57–58
Multics, 83
Multiinstance applications, 193
Multilevel feedback, 444
Multiple applications, 224
Multiple interrupts, 43–45
approaches to, 44–45
control with, transfer of, 44
disable interrupt, 44–45
interrupt service routine (ISR), 45
time sequence of, 45
Multiprocess applications, 192
Multiprocessing, 92–93, 224
Multiprocessor operating system, 98–100
Multiprocessor scheduling, 461–474, 499–500
design issues, 463–465
granularity, 461–463

process scheduling, 465–466
thread scheduling, 467–472
Multiprocessor system, 461
Multiprogrammed batch systems
example of, 87
memory management, 87–89
multiprogramming (multitasking), 77–80
program execution attributes of, sample, 79
on resource utilization, effects of, 79
system utilization of, 78
time-sharing systems, differentiating between, 81
uniprogramming, 78, 79
utilization histograms, 80
Multiprogramming, 80, 224
processors, 464–465
Multiprogramming levels, 406–407
Multitasking. *See* Multiprogramming
Multithreading, 92, 93, 178–181
of multicore computer, 190–195
native applications, 192
process models, 179
on uniprocessor, 183
Windows, 200
Mutex, 244, 247, 327. *See also* Mutual exclusion
Mutex object, 328
Mutual exclusion, 226–232, 297, 299
failed, 84–85
illustration of, 238
interrupt disabling, 241
requirements for, 240
semaphores, 249
software approaches, 226–232
special machine instructions, 241–244
using messages, 268

**N**

*name, 115
Named pipes, 581
Naming files, 567–568
National Institute of Standards and Technology (NIST), 662, 676
Native system interfaces (NT API), 104
Nearest fit strategy, 573
Nested task flag, 155
Network (Neutron), 717
Network Attached Storage (NAS), 710
Network File System (NFS), 711
Networkability, 611
Network-based IDS, 660
Networks
device drivers, 118
drivers, 544
isolation, 652, 653
protocols, 117
Neutron, 717
New process state, 141
New Technology File System (NTFS)
cluster sizes, 591
components of, 593
directory attributes, types of, 592
disk storage, concepts of, 590
file structure, 590–592
hard links, 590
journaling, 590
large files, support for, 589
partition sizes, 591
recoverability, 592–593
symbolic links, 590
volume, 590–592
*next, 115
Next-fit, 349
NIST Cloud Computing Reference Architecture, 701–702
No access rights, 569
No deadlock, 294
No preemption deadlock prevention, 297–300
Nodes into B-trees, insertion of, 564
No-execute bit, 669
Nonblocking, 265
Nonpreemptive mode, 434
Nonprocess kernel, 163–164
Non-real-time scheduling, 490–492
disadvantages of, 490
Nonuniform memory access (NUMA), 391
NOOP scheduler, 543
Normalized response time, 447, 448
Notification manager, Android, 126, 120

Notify and broadcast, 261–263
Nova logical architecture, 715–717
N-step-SCAN policy, 524
NTFS. *See* New Technology File System (NTFS)
Nucleus. *See* Kernels
Null Fork, 187
num_syms, 115

**O**
Object Storage (Swift), 712, 717
Object-oriented design, 106
  categories of, 107
  concepts of, 106–108
  Executive of, 107
  kernel control objects, 107
  Security Descriptor (SD) of, 107
  Windows, 198–199
Objects
  access rights, 671
  classes, 106
  control, Windows, 108
  dentry, Linux, 586, 588
  dispatcher, 107, 327–328
  event, Windows, 328, 545
  file, Linux, 545, 586, 588
  inode, 586, 587–588
  instance, 106
  instantiation of, 106
  kernel control, 107
  manager, 103
  mutex, 327
  owner of, 689
  semaphore, 328
  superblock, 587
  thread, 198–199
  timer, 328
  types, 586
On-demand self-service, 697
One-to-many relationships, 189–190
One-to-one relationship, 266–267
ONPROC state, 204
Opcode, 34
Opening files, 552
Open-source Tomcat, 192
OpenStack
  functional interactions, 714–715
  high level architecture, 714
  network block storage, 713
  object storage, 713
  virtual machine image storage, 713–714
OpenVZ file scheme, 640
Operating systems (OS). *See also* Modern operating systems (OS)
  achievements of, major, 83–91
  aspects of, 69–73
  avoidance approaches for, 296
  central themes of, 224
  commercial, 608
  concurrency, concerns of, 235–236
  development of, 83–84
  distributed, 94
  embedded. *See* Embedded operating systems
  evolution of, 73–83
  functions, 69–73, 72–73
  information in, protection and security of, 89–90
  interfaces of, typical, 71
  Linux. *See* Linux
  Mac OS X, 112
  memory management in, 87–89
  Microsoft. *See* Microsoft Windows
  modern, development leading to, 92–94
  multiprocessor/multicore, 98–100
  objectives/functions of, 69–73
  organization of, 101–108
  overview of, 68–127
  process-based, 165–166
  processes, 83–87, 163–166
  real-time, 475–477, 605
  resource management in, 72–73, 90–91
  services provided by, 70
  structure, 224
  symmetric multiprocessor, considerations of, 93
  TinyOS. *See* TinyOS
  UNIX. *See* UNIX systems
  as user/computer interface, 70–71
  virtual machines (VM), 627–655

Operating systems (OS) control
file tables, 149
input/output tables, 149
memory tables, 149
process tables, 150
structures of, 149–150
Operating systems (OS) software
cleaning policy, 405
cloud, 704–720
definition, 704
general architecture of, 707–713
Infrastructure as a Service (IaaS), 705–706
OpenStack, 713–720
requirements for, 706–707
fetch policy, 390–391
IoT, 724–731
architecture, 727–728
constrained devices, 724–725
requirements, 726–727
RIOT, 728–731
load control, 406–407
placement policy, 391
policies for, 390
replacement policy, 391–398
resident set management, 398–405
virtual memory, 388–407
Operational technology (OT), 721
Optimal (OPT) replacement policy, 392
Oracle, 192
Orchestration (Heat), 719
Ordinary file, 580
Overall normalized response time, 447
Overcommit, memory, 645
Overflow flag, 155
Owner of object, 689

**P**

Package manager, Android, 119
Page tables, 355, 414
direct *vs.* associative lookup for, 380
inverted, 377–379
structure of, 376–377
two-level hierarchical, 377
Page/paging, 355–358
address translation in system for, 376, 378
allocation, 415
behavior, 383
buffering, 397–398
cache, Linux, 543–544
characteristics of, 373
demand, 390
directory, 414
fault, 379–380
fault frequency (PFF), 404
frame data table entry, 408
logical addresses, 357
middle directory, 414
numbers, 378
prepaging, 390
replacement algorithm, 408, 415
segmentation and, combining, 387–388
sharing, 644
simple, 373
size, 383–385
system, 408–411
table entry, 408–409
translation lookaside buffer (TLB), 379–382
virtual memory, 373, 374–375
Windows, 417–419
Parallelism, 99–100, 462–463
coarse, 462–463
fine-grained, 463
independent, 462
medium-grained, 463
synchronization, 462
very coarse-grained, 462–463
Parasitic, 659
Paravirtualization, 634–635
Parcel, 331
Parent process, 138
Parity flag, 155
Partial program execution, 34–35
Partition/partitioning
boot sector, 591
dynamic, 348–351
fixed, 344–347
memory, 344–354
size, 344–345, 591
Password, 661
Pathname, 567
Pentium EFLAGS Register bits, 155
PeopleSoft, 192

Performance
disk cache, issues of, 535–536
of software on multicore computer, 190–195
Performance comparison, 445–450
queuing analysis, 445–448
simulation modeling, 448–450
Periodic tasks, 474, 480
Permanent blocking, 290
Personal identification number (PIN), 661
Personal technology, 721
per-thread static storage, 179
Physical address, 353
Physical input/output, 555
Physical memory, Linux, 118
Physical organization, 343–344, 513
Pile files, 558–559
Pipes, UNIX, 313
Placement algorithm for memory, 345–347
Placement policy, 390, 391
Plain spinlocks, 318
Platform as a Service (PaaS), 698
Plug-and-play manager, Windows, 103
Poisson arrival rate, 446
Polymorphism, 106
Portion, 572, 573
Portion size, 573–574
POSIX, 104, 208
Power manager
Android, 126
Windows, 103
PowerPC, 377
Preallocation, 572–573
Predictability, 431
Preempted process, 141
Preemptive mode, 434
Preemptive smallest number of threads first, 468
Prepaging, 391
Printed circuit board (PCB), 602
Printer interrupt service routine (ISR), 45
Printers, 539
Priorities, 480
ceiling, 488
classes, 494–495, 498
enforcing, 431
inheritance, 487–488
level, 132
policy, 520–522
priority queuing, 432
process, 498–499
queuing, 432
thread, 498–499
use of, 432–433
Priority inversion, 486–489
priority ceiling, 488
priority inheritance, 487–488
unbounded, 486
Private cloud model, 700
Privileged instructions, batch systems, 77
Privileges, 688
Problem statement, A–30
Procedure call, asynchronous, 545
Process(es),
for addressing, requirements of, 341
affinity, 495, 497
attributes of, 152–157
characteristics of, 177
components of, 85
concept of, 83–87, 131, 177
creation of, 137–138, 159
definition of, 83, 131–133
description of, 148–157
dispatching, 465
elements of, 132
errors in, causes of, 84
execution of, mechanisms for interrupting, 160
identification, 152–153
identifier, 378
image, 151, 168–169
implementation of, 86
initiation denial, deadlock avoidance strategy, 301–302
input/output, 33
isolation, 87
with largest remaining execution window, 407
location of, 151–152
management of, 87–89
memory, 33
of operating systems (OS), 83–87, 163–166
priorities, 498–499

process control blocks and, 132–133, 156–157
processing time, 480
processor affinity, 198
queues, 272
scheduling, 434, 465–466
security issues, 658–662
with smallest resident set, 407
spawning, 138
state transitions, 427
suspension, 407
switching, 160–162
synchronization, 462
table entry, 169
tables, 150
termination of, 138
threads and, 87, 177–183, 188, 202
traces of, 133–135
UNIX SVR4 process management, 166–171
Process control, 157–162
execution, modes of, 157–159
information, 152–153, 154, 156
operating system, structures of, 149–150
process attributes, 152–157
process creation, 159
process location, 151–152
process switching, 160–162
structures of, 151–157
UNIX System V Release 4 (SVR4), 170–171
Process control blocks, 132–133
elements of, 152–153
role of, 156–157
simplified, 133
Process interaction, 236–240
awareness, 236
communication, 239–240
resources, 238
sharing, 238–239
Process operation latencies ($\mu$s), 187
Process state, 85, 133–148
changing of, 162
five-state model, 138–143
suspended processes, 143–147
two-state process model, 136–137
ULT, relationship with, 185
UNIX System V Release 4 (SVR4), 166–168
Process-based operating systems, 165–166
Processors, 30
internal registers of, 32
point of view, 75
scheduling, types of, 426–429
specific context, 207
state information, 152–153, 154
unit (CPU), 31, *see also specific types of functions*
utilization, 431
Process-thread manager, Windows, 103
Producer/consumer problem
bounded-buffer, 256, 260
semaphores, 250–256
Producer/consumer problem bounded-buffer, 260
Program code, 132
Program counter (PC), 33, 42, 132
Program execution attributes, 79
Program flow of control with/without interrupts, 36–37
Program operation, 85
Program status word (PSW), 41, 154
Programmed input/output, 508
Programming language, 666
Project MAC, 81
Protection, 342
access control and, 87
sharing and, 388
Pthread libraries, 208
Public cloud infrastructure, 699–700
Pull mechanism, 497
Purpose-built embedded operating systems, 608–609
Push mechanism, 497

**Q**

Quality of service (QoS), 711
Queues
character, UNIX SVR 32, 539
dispatch, 493
driver input/output, 537
process, 272
single-server, formulas for, 446
structure, 495–496

Queuing
diagram for scheduling, 429
discipline, 268
priority, 432
Queuing analysis, 445–448
QuickPath Interconnect (QPI), 57–58

**R**
Race conditions, 235, A–30–37
problem statement, A–30
Rackspace, 699
Radio-Frequency Identification (RFID), 721
RAID (redundant array of independent disks), 524–533
characteristics of, 525
for high data transfer capacity, 529
for high input/output request rate, 529
level 0, 528–529
level 1, 529–530
level 2, 530
level 3, 531
level 4, 531–532
level 5, 532
level 6, 532–533
proposal for, 525
software, 546
Random scheduling, 520
Rate monotonic scheduling, 482–486
Reactive operation, embedded systems, 605
Read operation, 51–52
Read_control access, 690
Readers/writers
lock, 325–326
mechanisms, 270–274
priorities of, 271–272
process queues, state of, 272
semaphores, 321
spinlocks, 319–320
using semaphores, solution to, 273
Reading access rights, 569
Reading files, 552
Ready process state, 144
Ready state, 139, 200
Ready time, 480
Ready/suspend : ready process, 146
Ready/suspend process, 145
Real address, 88, 371
Real memory, 373
Real time
class (159-100), 493
operating systems, 475–477, 605
priority classes, 498
user, 495
Real-time scheduling, 460–500
algorithms for, 477
deadline scheduling, 479–483
history of, 474
Linux, 489–490
and multiprocessor, 460–500
priority inversion, 486–489
rate monotonic scheduling, 482–486
real-time operating systems, characteristics of, 475–477
types of, 479
Receive primitive, 265
Record blocking, 570–572
fixed blocking, 571
methods of, 570–571
variable-length spanned, 570
variable-length unspanned, 571
Records, 552
Recoverability, 589, 592–593
Recovery, 307–308
Redundant arrays of independent disks (RAID), 710
refcnt, 115
Reference architecture, cloud computing
cloud service consumers, 703
cloud service provider, 702
NIST, 701
Registers
address, 31
context, 168
control and status, 153, 154
index, 85
input/output address, 51
instruction, 32–33
internal, of processor, 31–32
for interrupt processing, changes in, 41–43

memory address, 31
memory buffer, 31–32
Pentium EFLAGS, 154–155
Regular file, 580
Relative address, 353
Reliability, 95, 99, 476, 580
Relocatable loading, 365–366
Relocation, 341–342, 352–354
Remote procedure call (RPC), 182
Rendering module, 194
Replacement algorithms, 52, 53, 350–351, 391–398
clock page, 410
clock policy, 394–395
first-in-first-out (FIFO) policy, 394
fixed-allocation, local page, 396
four page, behavior of, 393
least recently used (LRU) policy, 393
optimal policy, 392
Replacement, frequency-based, 534
Replacement policies, 390, 392–398. *See also specific types of*
algorithms for, basic, 392–397
and cache size, 398
concepts of, 392
frame locking, 392
page buffering, 397–398
Replacement scope, 399–400
Reserved state, 417–418
Resident monitor, 75
Resident set, 372
size, 398–399
Resident set management, 390, 398–405
fixed allocation, local scope, 399
replacement scope, 399–400
resident set size, 398–399
variable allocation, 399–405
Resource pooling, 698
Resources
balancing, 431
competition among processes for, 237–238
configure interface, 624
interface, 624
manager, 72, 119
ownership, 177. *See also* Process(es).
requested interface, 624
requirements, 480
utilization, 79
Resources, allocation of
denial, 302–306
graphs, 296–297
Resources, management of, 90–91
Android, 119
elements of, major, 90
factors of, 90
functional description of, 91
round-robin, 91
Resource-specific interface, 624
Response time, 431
normalized, 447, 448
overall normalized, 447
Responsiveness, 475
Resume flag, 155
Reusable resources, deadlock and, 294–295
RIOT structure, 728–729
hardware abstraction layer, 730–731
hardware-independent modules, 730
Kernel, 729–730
microcontrollers, 731
Role-based access control (RBAC), 673, 676–678
Rotational delay, 518
Rotational positional sensing (RPS), 518
Round-robin techniques, 91, 138, 437–439
Running process state, 132, 142, 200, 204, 207
Run-time, Android, 121–124
Run-time defenses, 668–670
address space randomization, 669
executable address space protection, 669
guard pages, 670
Run-time dynamic linking, 368–369

**S**

Safe coding techniques, 666–667
Safe libraries, 667–668
Safe states, resource allocation, 302–304
Saved thread context, 179
Scaling, 93
SCAN policy, 523–524

Scanrate, 411
Scheduled blocks, 215
Scheduler, 116
Scheduling, 98, 177
  control and, 512
  criteria for, 431
  deadline, 479–483
  disk, 505–547
  dynamic, 467, 472
  dynamic best effort, 479
  dynamic planning-based, 479
  feedback, 444
  gang, 467
  input/output, 426
  levels of, 428
  Linux, 489–492
  long-term, 426, 427–429
  medium-term, 426, 429
  multiprocessor and multicore scheduling, 461–474
  non-real-time, 490–492
  process, 434, 465–466
  and process state transitions, 427
  processor, types of, 426–430
  queuing diagram for, 429
  random, 520
  rate monotonic, 482–486
  real-time, 460–500
  short-term, 426, 430
  static priority-driven preemptive, 477
  static table-driven, 477
  thread, 467–472
  types of, 426
  uniprocessor, 425–455
  UNIX FreeBSD, 494–497
  UNIX SVR 32, 492–494
  UNIX, traditional, 452–454
  Windows, 498–500
Scheduling algorithms, 430–452
  fair-share scheduling, 450–452
  performance comparison, 445–450
  priorities, use of, 432–433
  scheduling policies, alternative, 433–445
  short-term scheduling criteria, 430–432
Scheduling policies, 433–445
  feedback, 443–445
  first-come-first-served (FCFS), 435–437
  highest ratio next, 443
  round robin, 437–439
  shortest process next, 440–441
  shortest remaining time, 441–443
S.count value, 250
Search operation, 566
Secondary memory, 49
Secondary storage management, 572–580
  file allocation, 572–576
  free space management, 576–579
  reliability, 580
  volumes, 579
Sector, 590
Security Descriptor (SD), 107
  discretionary access control list (DACL), 689
  flags, 688
  owner, 689
  system access control list (SACL), 689
Security ID (SID), 687
Security maintenance, 685–686
  backup and archive, 686
  logging, 685–686
  Windows security, 686–691
Security, operating system, 657–692
  access control, 672–678
  additional controls, 684–685
  authentication, 660–661
  buffer overflow attacks, 662–666
  configuration, 683–684
  countermeasures for, 660–662
  installation, 682–683
  intrusion detection, 660
  maintenance, 685–686
  memory management, 662–666
  New Technology File System (NTFS), 589
  of process, 658–662
  system access threats, 658–659
  testing, 685
Security reference monitor, 103
Security Requirements for Cryptographic Modules, 676
Seek time, 518, 519
Segment pointers, 340, 358–359, 388
Segmentation, 358–359
  address translation in, 386, 387
  advantages of, 385

characteristics of, 373
implications of, 385
organization of, 386–387
paging and, combining, 387–388
segments, protection relationship between, 388
simple, 373
virtual memory, 373, 385
Selection function, 433
Semaphores, 244–257, 310–313, 314–315, 320–321, 325, A–30–37
binary, 246, 247, 320
counting, 247, 252, 320
currency mechanisms, common, 244
definition of, consequences of, 245–246
dining philosophers problem, solutions using, 311–312
first-in-first-out (FIFO) process, 247
general, 246
implementation of, 256
Linux, 321
mechanism of, example of, 248
mutex, 247
mutual exclusion, 249
object, Windows, 327
producer/consumer problem, 250–256
readers/writers, 271–273
reader-writer, 321
s.count, value of, 250
shared data protected by, process accessing, 250
strong, 247
as variable, operations of, 245
weak, 247
Sensor/actuator technology, 721
Sensors for intrusion detection, 660
Sequential files, 559
indexed, 559–560
key field for, 559
processing of, 553
Sequential search, 559
Serial processing, 74
Server Message Block (SMB), 711
Service(s)
processes, Windows, 104
Service-level agreements (SLAs), 706
Set of data, 132
Setup time, 74
Shadow copies, volume, 546
Shared data protected, 250
Shared Filesystems (Manila), 720
Shared memory multiprocessor, 314
Shared resources, 623
Sharing files, 238–239, 342
Shortest process next (SPN) scheduling, 440–441
Shortest remaining time (SRT) scheduling, 441–443
Shortest-service-time-first (SSTF) policy, 523
Short-term scheduling, 426, 430–432
Siebel CRM (Customer Relationship Manager), 192
Sign flag, 155
Signaling/signals, 84, 315
event object, 545
file object, 545
monitors with, 257–261
Signal-Wait, 188
Simple batch systems, 74–77
hardware features of, 77
job control language (JCL), 76
kernel mode, 77
monitor, 77
points of view of, 75
user mode, 77
Simple interrupt processing, 41
Simple paging, 373
Simple segmentation, 373
Simulation modeling for scheduling, 448–450
Simulation result, 449
Simultaneous access for file sharing, 570
Simultaneous concurrent process, 98
Simultaneous concurrent threads, 98
Single buffer, 514–516
Single-Instruction Multiple Data (SIMD) techniques, 32
Single-server queues, 446
Single-threaded process models, 179
Single-user multiprocessing system, 180–181
Slab allocation, 416
Slim read-writer locks, 329
Slots of memory, 50

Smallest number of threads first, 468
Sockets, 32
Soft affinity policy, 202, 499
Soft real-time task, 474
Software
  malicious, 658, 659–662
  memory hierarchy in, 49
  RAID, 546
  Valve game, 193–195
Software approaches, mutual exclusion, 226–232
Software as a Service (SaaS), 698
Solaris
  11, 112
  memory management, 407–413
  process structure of, 203
  three-level thread structure of, 203
Solaris, thread primitives, 324–326
  of threads, 183
Solaris threads
  SMP management of, 202–206
  states of, 204–205
  synchronization primitives, 324–326
Spanned blocking, variable-length, 570
SPARC, 669
Spatial locality, 63
Spawn state, 181
Special file, 581
Special machine instructions, 241–244
  compare&swap instruction, 242–243
  disadvantages of, 243–244
  exchange instruction, 243
  properties of, 243–244
Special system processes, Windows, 104
Specific user class, 570
Spin waiting, 243
Spinlocks, 244, 318–320
  basic, 318–319
  Linux, 319
  plain, 318
  reader-writer, 319–320
SQLite, 595
Stack overflow, 663
Stackable modules, Linux, 114
Stacking protection mechanisms, 668
Standby state, 200
Starting deadline, 480
Starvation, 238, 272
States, 302. *See also specific states*
  available, 417
  blocked, 141–142
  blocked/waiting process, 140
  committed, 418
  execution, 206
  exit process, 140
  interruptible, 208
  new process, 140
  ONPROC, 204
  process, 86, 133–148
  of processes, 87, 133–145
  ready, 139, 200
  ready process, 144
  reserved, 417
  running process, 132, 139, 201, 204, 207
  safe, resource allocation, 302–303
  SLEEP, 205
  spawn, 181
  standby, 200–201
  stopped, 205, 208
  system operational, 95
  terminated, 201
  thread, 181–183
  thread execution, 179
  transition, 201
  uninterruptible, 208
  unsafe, 302
  waiting, 200
Static biometrics, 661
Static priority-driven preemptive scheduling, 477, 479
Static table-driven scheduling, 477, 479
Storage Area Network (SAN), 709, 710
Storage management, 87
  access control, protection and, 87
  automatic allocation/management, 87
  long-term storage, 93, 87
  modular programming, support of, 87
  process isolation, 87
Streamlined protection mechanisms, 605–606
Stream-oriented device, 514
Stripe, 528
Strong semaphores, 247
Structured applications, 224

Structured programming (SAL), 62
Subject access rights, 671
Subtask structure, 480
Sun Microsystems, 110
SunOS, 110
Superblock object, 587
Superblocks, 585
Supervisor call, 161
Support functions, 158
Suspended processes states, 143–147
  characteristics of, 147
  purposes of, 148
  states of, 144–145
  swapping, 143–147
  transitions of, 145, 146–147
Swap, 242
Swappable space, 308
Swapping process states, 143–147
Swap-use table entry, 408–409
Switching process, 160–162
Symbolic links file, 581, 590
Symmetric multiprocessor (SMP), 55–57, 93
  advantages of, 55–56
  availability, 55
  characteristics of, 55
  definition of, 55
  incremental growth, 93
  multicore support and, 495–497
  organization of, 55–56
  OS considerations of, 98–99
  scaling, 93
  threads for, 105
*syms, 115
Synchronization, 84, 94, 264–265
  design characteristics of, 264
  granularity, 461–463
  improper, 84
  lock-free, 329
  message passing, 263–264, 264–265
  processes, 462
Synchronized access, 689
Synchronous input/output, Windows, 545–546
System(s)
  access control list (SACL), 689
  access threats, 658–659
  bus, 31
  calls, Linux, 116
  files, 591
  ISA, 71
  mode, 158
  response time, 84
  utilization of, 74
System access control list (SACL), 689
System directory, 594
System libraries, Android
  bionic LibC, 121
  browser engine, 121
  media framework, 121
  openGL, 121
  SQL database, 121
  surface manager, 121
System on a Chip (SoC), 32
System operational states, 95
System oriented, other criteria, 431
System oriented, performance related criteria, 431
System-level context, 168

**T**

Tape drives, 539
Tasks, 618, 619
  aperiodic, 474
  deadline scheduling for, 479–483
  hard real-time, 474
  Linux, 206–208
  periodic, 474
  soft real-time, 474
Telephony manager, Android, 119
Temporal locality, 63
Terminals, 539
Termination of process states, 201
Thrashing, load control, 374
Thread scheduling, 467–472
  approaches to, 467
  dedicated processor assignment, 470–471
  dynamic scheduling, 472
  gang scheduling, 467
  load sharing, 467–469
Thread states, 181–183
  of Microsoft Windows 36, 200–201
  of Solaris, 204–205

Threading granularity options, 193
Threads, 87, 176–217. *See also specific types of*
  Android, 211–215
  benefits of, 180
  bottom-half kernel, 494
  execution state, 179
  functionality of, 181–183
  interactive, 496
  kernel-level (KLT), 187–188, 202
  Linux process and, management of, 206–210
  MAC OS Grand Central Dispatch (GCD), 215–217
  management of, 206–210
  many-to-many relationships of, 189–190
  migration, 497
  multithreaded process models, 179
  multithreading, 178–181, 190–195
  objects, 198–199
  one-to-many relationships of, 189–190
  operations associated with change in, 181
  pool, 195
  priorities, 498–499
  process operation latencies ($\mu$s), 187
  processes and, 177–183, 188, 202
  processor affinity, 198
  remote procedure call (RPC) using, 182
  single-threaded process models, 179
  in single-user multiprocessing system, 180–181
  for SMP, 105–106
  Solaris, and SMP management, 202–206
  states of, 181–183
  synchronization, 183
  top-half kernel, 494
  types of, 183–190
  user-level (ULT), 183–188, 202
  Windows 36, 195
Three-level thread structure, Solaris, 203
Throughput, 431
Tightly coupled multiprocessor system, 461
Time, creation of, 207
Timeliness, 561
Timers, batch systems, 77, 207
Time-shared (59-0) class, 493
Time-sharing systems, 81–83
  batch multiprogramming, differentiating between, 81
  Compatible Time-Sharing System (CTSS), 81
  memory requirements of, 82
  time sharing, 82–83
  time slicing, 81
Time-sharing user, 495
Timeslices/timeslicing, 82–83, 160
Timing comparison, 519–520
TinyOS, 615–625
  components of, 618–620
  configurations for, examples of, 621–623
  goals of, 617–618
  resource interface, 623–625
  scheduler, 621
  wireless sensor networks, 616–617
Token, 661
Token bucket filter (TBF), 653–654
Top-half kernel threads, 494
Torvalds, Linus, 113
Trace of process, 135
Transfer time, 518
Transition of process state, 201
Translation lookaside buffer (TLB), 379–383
  cache operation and, 382
  operation of, 381
Trap flag, 155
Traps, 118
Tree representation of buddy system, 353
Tree-structured file directory, 567, 568
TRIX, 189
Turnaround time (TAT), 431, 434, 449
Two-handed clock algorithm, 410
Two-level hierarchical page table, 377
Two-level memory
  characteristics of, 61–67
  locality, 62–64
  operation of, 62
  performance of, 46–47, 64–67
Two-priority categories, 446
Two-state process model, 136–137

**U**

U area, 169–170
Unblock state, 181
Unbounded priority inversion, 486

Unbuffered input/output, 539
Uninterruptible state, 208
Uniprocessor
  multithreading on, 183
  scheduling, 425–455
Uniprogramming systems, 80
University of California at Berkeley, 525
UNIX BSD (Berkeley Software Distribution), 109
UNIX FreeBSD, 112
  files, structure of, 582
  inodes, structure of, 582
  scheduling, 494–497
UNIX System V Release 4
  process control of, 170–171
  process description of, 168–170
  process image of, 168
  process management, 166–171
  process states of, 166–168
  process table entry of, 169
  scheduling, 493
  U area, 169–170
  unbuffered input/output, 539
UNIX System V Release 4 (SVR4), 110–111
  buffer cache, 537–538
  character queue, 539
  devices, types of, 539–540
  dispatch queues, 493
  input/output, 537–540
  parameters of, 408–409
UNIX systems, 108–110, 580–585. *See also specific systems*
  access control lists, 678–681
  architecture of, 109
  Berkeley Software Distribution (BSD), 111–112
  buffer cache, organization of, 538
  C implementation of, 108
  concurrency mechanisms of, 313–315
  description of, general, 109–110
  devices, types of, 539–540
  directories, 584
  file access control, 678–680
  file allocation, 583–584
  files, 580–581
  history of, 108–109
  inodes, 581–583
  input/output, structure of, 537
  kernel, 109
  license for, 109
  memory management, 407–413
  modern, 110–112
  process structure of, 203
  scheduling, traditional, 452–454
  signals of, 315
  System III, 109
  System V, 109
  traditional, 109–110
  traditional, file access control, 678–680
  Version 34, 109
  Version 35, 109
  volume structure, 584–585
Unmarshalling, 331
Unsafe state, resource allocation, 302, 304
Unspanned blocking, variable-length, 571–572
Update directory operation, 566
Updating access rights, 569
User applications, Windows, 104
User control, 475–476
User groups class, 570
User identification (ID), 678
User interfaces, 70–71
User ISA, 71
User mode, 104, 158
User-level context, 168
User-level threads (ULT), 183–187
  advantages of, 186–187
  and KLT, combined with, 187–188
  occurrences of, 184, 186
  process states, relationship with, 185
User-mode processes, 104
  environmental subsystems, 104
  execution within, 164–165
  service processes, 104
  special system processes, 104
  user applications, 104
  in virtual memory, 156
User-mode scheduling (UMS), 196
User-oriented, other criteria, 431
User-oriented, performance related criteria, 431

User's identity authentication, 661
User-visible registers, 153
Utilization histograms, 80

**V**
Valve game software, 193–195
Variable, operations of, 245
Variable priority classes, 498
Variable-allocation replacement policy, 399
  global scope, 400–401
  local scope, 401–405
Variable-interval sampled working set (VSWS) policy, 404–405
Variable-length spanned, 570
Variable-length unspanned, 571
VAX/VMS, 109
Verification step of authentication, 660
Very coarse-grained parallelism, 462–463
View system, Android, 120
Virtual 8086 mode, 155
Virtual addresses
  map, 417
  memory management, 88
  space, 179, 371
Virtual computing, 709
Virtual interrupt flag, 155
Virtual interrupt pending, 155
Virtual LANs (VLANs), 717
Virtual machine technology, 707–708
Virtual machines (VM)
  aggregating, 630
  availability, 630
  concepts of, 628–631
  consolidation, 630
  container virtualization, 635–642
  devices emulation and access control, 632
  dynamics, 630
  execution management, 632
  Hyper V, 650–651
  hypervisors, 631–635
  input and output management, 645–647
  Java VM, 651–652
  legacy hardware, 630
  lifecycle management, 632
  Linux VServer architecture, 652–654
  management, 630
  memory management, 644–645
  monitor, 629
  rapid deployment, 630
  versatility, 630
  VMware ESXi, 647–649
Virtual memory, 61, 87–88, 370–420
  addressing, 89, 413–414
  concepts of, 88
  hardware/control structures of, 371–388
  locality and, 373–374
  management, 63–417
  manager, 103, 593
  operating system software, 388–407
  paging, 373, 374–385
  protection, sharing and, 388
  segmentation, 385–387
  terminology of, 371
  user-mode processes in, 156
Virtual network, 711
Virtual platform, 653
Virtual private networks (VPNs), 700, 717
Virtual runtime, 491
Virtual servers, 652, 653
Virtual storage
  SAN and NAS, 710–711
  storage services, 709
  topologies of, 709–710
Virtualization, 628
  container virtualization, 635–642
  hardware-assisted virtualization, 635
  paravirtualization, 634–635
Virtualized resources, 623
Volume, 579, 590–592
  layout, 591–592
  master file table, 592
  shadow copies, 546
  structure, UNIX, 584–585
VPNs. *See* Virtual private networks (VPNs)

**W**
Wait functions, Windows, 326–327
Waitable timer object, Window, 327, 328
Waiting state, 201
Waiting time, 449
Weak semaphores, 247
Weblogic, 192
Websphere, 192
While loops, 262

Win 60, 104
Window manager, Android, 119
Windowing/graphics system, 103
Windows security, 686–691
  access control scheme, 687
  access mask, 690
  access token, 687–688
  security descriptors, 688–691
Wireless personal area networks (WPANs), 725, 726
Wireless sensor networks (WSN), 616–617
Working directories, 568
Working set strategy, 401
WRITE call, 36–37, 40
WRITE instruction, 37
Write policy, cache memory, 52, 53
Write_DAC access, 690
Write_owner access, 689

**X**
XMPP, Android, 120

**Z**
ZF (zero flag), 155
Zombie state, 205, 208
Zombies, 167, 208

*This page intentionally left blank*

CHAPTER 17

# Network Protocols


**17.1 The Need for a Protocol Architecture**

**17.2 The TCP/IP Protocol Architecture**
- TCP/IP Layers
- TCP and UDP
- IP and IPv6
- Operation of TCP/IP
- TCP/IP Applications

**17.3 Sockets**
- The Socket
- Socket Interface Calls

**17.4 Linux Networking**
- Sending Data
- Receiving Data

**17.5 Summary**

**17.6 Key Terms, Review Questions, and Problems**

**APPENDIX 17A The Trivial File Transfer Protocol**
- Introduction to TFTP
- TFTP Packets
- Overview of a Transfer
- Errors and Delays
- Syntax, Semantics, and Timing

### Learning Objectives

After studying this chapter, you should be able to:

- Explain the motivation for organizing communication functions into a layered protocol architecture.
- Describe the TCP/IP protocol architecture.
- Understand the purpose of the Sockets facility and how to use it.
- Describe the networking features in Linux.
- Understand how TFTP works.

With the increasing availability of inexpensive yet powerful personal computers and servers, there has been an increasing trend toward distributed data processing (DDP), in which processors, data, and other aspects of a data processing system may be dispersed within an organization. A DDP system involves a partitioning of the computing function and may also involve a distributed organization of databases, device control, and interaction (network) control.

In many organizations, there is heavy reliance on personal computers coupled with servers. Personal computers are used to support a variety of user-friendly applications, such as word processing, spreadsheet, and presentation graphics. The servers house the corporate database plus sophisticated database management and information systems software. Linkages are needed among the personal computers and between each personal computer and the server. Various approaches are in common use, ranging from treating the personal computer as a simple terminal to implementing a high degree of integration between personal computer applications and the server database.

These application trends have been supported by the evolution of distributed capabilities in the operating system and supporting utilities. A spectrum of distributed capabilities has been explored:

- **Communications architecture:** This is software that supports a group of networked computers. It provides support for distributed applications, such as electronic mail, file transfer, and remote terminal access. However, each computer retains a distinct identity to the user and to the applications, which must communicate with other computers by explicit reference. Each computer has its own separate operating system, and a heterogeneous mix of computers and operating systems is possible, as long as all machines support the same communications architecture. The most widely used communications architecture is the TCP/IP protocol suite, examined in this chapter.
- **Network operating system:** This is a configuration in which there is a network of application machines, usually single-user workstations and one or more "server" machines. The server machines provide networkwide services or applications, such as file storage and printer management. Each computer has its own private operating system. The network operating system is simply an adjunct to the local operating system that allows application machines to interact with server machines. The user is aware that there are multiple independent computers and must deal with them explicitly. Typically, a common communications architecture is used to support these network applications.

- **Distributed operating system:** A common operating system shared by a network of computers. It looks to its users like an ordinary centralized operating system but provides the user with transparent access to the resources of a number of machines. A distributed operating system may rely on a communications architecture for basic communications functions; more commonly, a stripped-down set of communications functions is incorporated into the operating system to provide efficiency.

The technology of the communications architecture is well-developed and is supported by all vendors. Network operating systems are a more recent phenomena, but a number of commercial products exist. The leading edge of research and development for distributed systems is in the area of distributed operating systems. Although some commercial systems have been introduced, fully functional distributed operating systems are still at the experimental stage.

In this chapter and the next, we will provide a survey of distributed processing capabilities. This chapter focuses on the underlying network protocol software.

## 17.1 THE NEED FOR A PROTOCOL ARCHITECTURE

When computers, terminals, and/or other data processing devices exchange data, the procedures involved can be quite complex. Consider, for example, the transfer of a file between two computers. There must be a data path between the two computers, either directly or via a communication network. But more is needed. Typical tasks to be performed include the following:

1. The source system must either activate the direct data communication path or inform the communication network of the identity of the desired destination system.
2. The source system must ascertain that the destination system is prepared to receive data.
3. The file transfer application on the source system must ascertain that the file management program on the destination system is prepared to accept and store the file for this particular user.
4. If the file formats or data representations used on the two systems are incompatible, one or the other system must perform a format translation function.

The exchange of information between computers for the purpose of cooperative action is generally referred to as *computer communications*. Similarly, when two or more computers are interconnected via a communication network, the set of computer stations is referred to as a *computer network*. Because a similar level of cooperation is required between a terminal and a computer, these terms are often used when some of the communicating entities are terminals.

In discussing computer communications and computer networks, two concepts are paramount:

- Protocols
- Computer communications architecture, or protocol architecture

A **protocol** is used for communication between entities in different systems. The terms *entity* and *system* are used in a very general sense. Examples of entities are user application programs, file transfer packages, database management systems, electronic mail facilities, and terminals. Examples of systems are computers, terminals, and remote sensors. Note in some cases the entity and the system in which it resides are coextensive (e.g., terminals). In general, an entity is anything capable of sending or receiving information, and a system is a physically distinct object that contains one or more entities. For two entities to communicate successfully, they must "speak the same language." What is communicated, how it is communicated, and when it is communicated must conform to mutually agreed conventions between the entities involved. The conventions are referred to as a protocol, which may be defined as a set of rules governing the exchange of data between two entities. The key elements of a protocol are as follows:

- **Syntax:** Includes such things as data format and signal levels
- **Semantics:** Includes control information for coordination and error handling
- **Timing:** Includes speed matching and sequencing

Appendix 17A provides a specific example of a protocol, the Internet standard Trivial File Transfer Protocol (TFTP).

Having introduced the concept of a protocol, we can now introduce the concept of a **protocol architecture**. It is clear there must be a high degree of cooperation between the two computer systems. Instead of implementing the logic for this as a single module, the task is broken up into subtasks, each of which is implemented separately. As an example, Figure 17.1 suggests the way in which a file transfer facility could be implemented. Three modules are used. Tasks 3 and 4 in the preceding list could be performed by a file transfer module. The two modules on the two systems exchange files and commands. However, rather than requiring the file transfer module to deal with the details of actually transferring data and commands, the file transfer modules each rely on a communications service module. This module is responsible for making sure the file transfer commands and

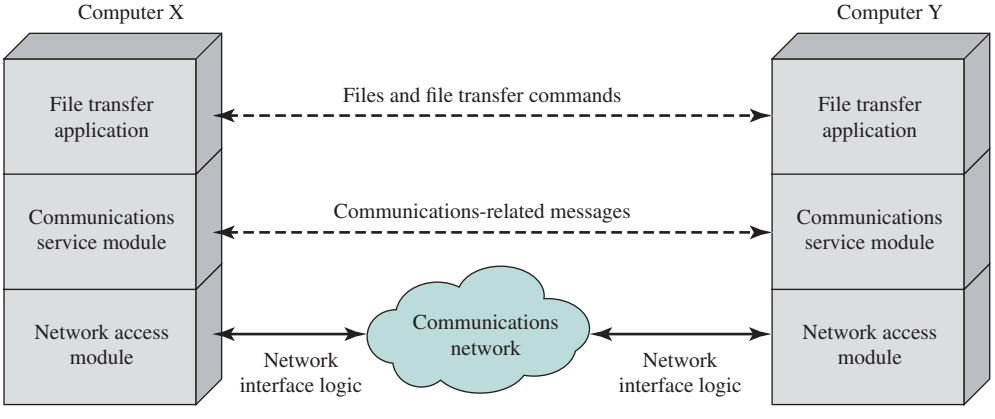



**Figure 17.1 A Simplified Architecture for File Transfer**

data are reliably exchanged between systems. The manner in which a communications service module functions will be explored subsequently. Among other things, this module would perform task 2. Finally, the nature of the exchange between the two communications service modules is independent of the nature of the network that interconnects them. Therefore, rather than building details of the network interface into the communications service module, it makes sense to have a third module, a network access module, that performs task 1 by interacting with the network.

To summarize, the file transfer module contains all the logic that is unique to the file transfer application, such as transmitting passwords, file commands, and file records. These files and commands must be transmitted reliably. However, the same sorts of reliability requirements are relevant to a variety of applications (e.g., electronic mail, document transfer). Therefore, these requirements are met by a separate communications service module that can be used by a variety of applications. The communications service module is concerned with assuring that the two computer systems are active and ready for data transfer, and for keeping track of the data that are being exchanged to assure delivery. However, these tasks are independent of the type of network that is being used. Therefore, the logic for actually dealing with the network is put into a separate network access module. If the network to be used is changed, only the network access module is affected.

Thus, instead of a single module for performing communications, there is a structured set of modules that implements the communications function. That structure is referred to as a protocol architecture. An analogy might be useful at this point. Suppose an executive in office X wishes to send a document to an executive in office Y. The executive in X prepares the document and perhaps attaches a note. This corresponds to the actions of the file transfer application in Figure 17.1. Then the executive in X hands the document to a secretary or administrative assistant (AA). The AA in X puts the document in an envelope and puts Y's address and X's return address on the outside. Perhaps the envelope is also marked "confidential." The AA's actions correspond to the communications service module in Figure 17.1. The AA in X then gives the package to the shipping department. Someone in the shipping department decides how to send the package: mail, UPS, or express courier. The shipping department attaches the appropriate postage or shipping documents to the package and ships it out. The shipping department corresponds to the network access module of Figure 17.1. When the package arrives at Y, a similar layered set of actions occurs. The shipping department at Y receives the package and delivers it to the appropriate AA or secretary based on the name on the package. The AA opens the package and hands the enclosed document to the executive to whom it is addressed.

## 17.2 THE TCP/IP PROTOCOL ARCHITECTURE

The TCP/IP protocol architecture is a result of protocol research and development conducted on the experimental packet-switched network, ARPANET, funded by the Defense Advanced Research Projects Agency (DARPA), and is generally referred to as the TCP/IP protocol suite. This protocol suite consists of a large collection of

protocols that have been issued as Internet standards by the Internet Activities Board (IAB). Appendix L provides a discussion of Internet standards.

## TCP/IP Layers

In general terms, computer communications can be said to involve three agents: applications, computers, and networks. Examples of applications include file transfer and electronic mail. The applications with which we are concerned here are distributed applications that involve the exchange of data between two computer systems. These applications and others execute on computers that can often support multiple simultaneous applications. Computers are connected to networks, and the data to be exchanged are transferred by the network from one computer to another. Thus, the transfer of data from one application to another involves first getting the data to the computer in which the application resides, then getting the data to the intended application within the computer.

There is no official TCP/IP protocol model. However, based on the protocol standards that have been developed, we can organize the communication task for TCP/IP into five relatively independent layers, from bottom to top:

- Physical layer
- Network access layer
- Internet layer
- Host-to-host, or transport layer
- Application layer

The **physical layer** covers the physical interface between a data transmission device (e.g., workstation, computer) and a transmission medium or network. This layer is concerned with specifying the characteristics of the transmission medium, the nature of the signals, the data rate, and related matters.

The **network access layer** is concerned with the exchange of data between an end system (server, workstation, etc.) and the network to which it is attached. The sending computer must provide the network with the address of the destination computer, so the network may route the data to the appropriate destination. The sending computer may wish to invoke certain services, such as priority, that might be provided by the network. The specific software used at this layer depends on the type of network to be used; different standards have been developed for circuit switching, packet switching (e.g., frame relay), LANs (e.g., Ethernet), and others. Thus, it makes sense to separate those functions having to do with network access into a separate layer. By doing this, the remainder of the communications software, above the network access layer, need not be concerned about the specifics of the network to be used. The same higher-layer software should function properly regardless of the particular network to which the computer is attached.

The network access layer is concerned with access to and routing data across a network for two end systems attached to the same network. In those cases where two devices are attached to different networks, procedures are needed to allow data to traverse multiple interconnected networks. This is the function of the Internet layer. The **Internet Protocol (IP)** is used at this layer to provide the routing function across multiple networks. This protocol is implemented not only in the end systems but also

in routers. A **router** is a processor that connects two networks and whose primary function is to relay data from one network to the other on a route from the source to the destination end system.

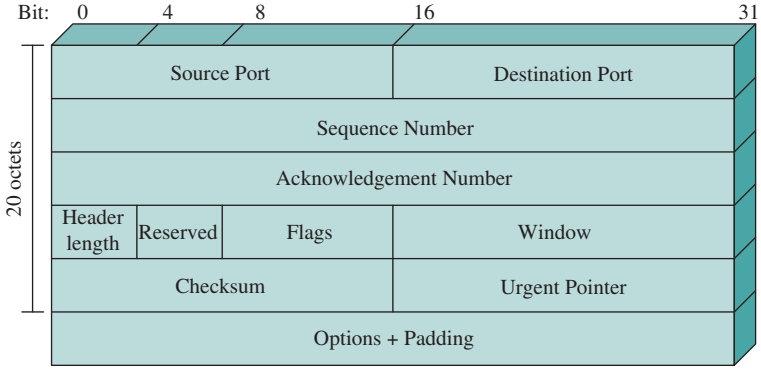
Regardless of the nature of the applications that are exchanging data, there is usually a requirement that data be exchanged reliably. That is, we would like to be assured that all the data arrive at the destination application, and that the data arrive in the same order in which they were sent. As we shall see, the mechanisms for providing reliability are essentially independent of the nature of the applications. Thus, it makes sense to collect those mechanisms in a common layer shared by all applications; this is referred to as the host-to-host layer, or **transport layer**. The Transmission Control Protocol (TCP) is the most commonly used protocol to provide this functionality.

Finally, the **application layer** contains the logic needed to support the various user applications. For each different type of application, such as file transfer, a separate module is needed that is peculiar to that application.

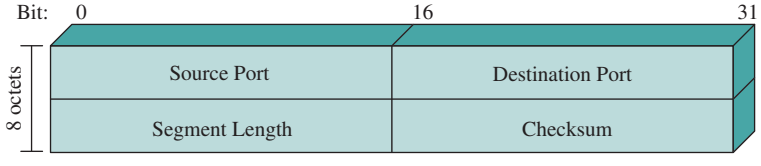
## TCP and UDP

For most applications running as part of the TCP/IP protocol architecture, the transport layer protocol is TCP. TCP provides a reliable connection for the transfer of data between applications. A connection is simply a temporary logical association between two entities in different systems. For the duration of the connection, each entity keeps track of segments coming and going to the other entity, in order to regulate the flow of segments and to recover from lost or damaged segments.

Figure 17.2a shows the header format for TCP, which is a minimum of 20 octets, or 160 bits. The Source Port and Destination Port fields identify the applications at




(a) TCP Header




(b) UDP Header

**Figure 17.2 TCP and UDP Headers**

the source and destination systems that are using this connection. The Sequence Number, Acknowledgment Number, and Window fields provide flow control and error control. The checksum is a 16-bit code based on the contents of the segment used to detect errors in the TCP segment.

In addition to TCP, there is one other transport-level protocol that is in common use as part of the TCP/IP protocol suite: the User Datagram Protocol (UDP). UDP does not guarantee delivery, preservation of sequence, or protection against duplication. UDP enables a process to send messages to other processes with a minimum of protocol mechanism. Some transaction-oriented applications make use of UDP; one example is SNMP (Simple Network Management Protocol), the standard network management protocol for TCP/IP networks. Because it is connectionless, UDP has very little to do. Essentially, it adds a port addressing capability to IP. This is best seen by examining the UDP header, shown in Figure 17.2b.

## IP and IPv6

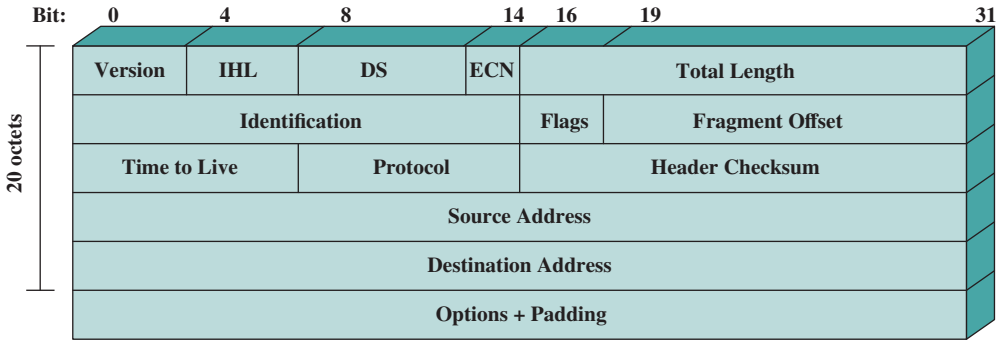
For decades, the keystone of the TCP/IP protocol architecture has been IP. Figure 17.3a shows the IP header format, which is a minimum of 20 octets, or 160 bits. The header, together with the segment from the transport layer, forms an IP-level block referred to as an IP datagram or an IP packet. The header includes 32-bit source and destination addresses. The Header Checksum field is used to detect errors in the header to avoid misdelivery. The Protocol field indicates whether TCP, UDP, or some other higher-layer protocol is using IP. The ID, Flags, and Fragment Offset fields are used in the fragmentation and reassembly process, in which a single IP datagram is divided into multiple IP datagrams on transmission then reassembled at the destination.

In 1995, the Internet Engineering Task Force (IETF), which develops protocol standards for the Internet, issued a specification for a next-generation IP, known then as IPng. This specification was turned into a standard in 1996 known as IPv6. IPv6 provides a number of functional enhancements over the existing IP, designed to accommodate the higher speeds of today's networks and the mix of data streams, including graphic and video, which are becoming more prevalent. But the driving force behind the development of the new protocol was the need for more addresses. The current IP uses a 32-bit address to specify a source or destination. With the explosive growth of the Internet and of private networks attached to the Internet, this address length became insufficient to accommodate all the systems needing addresses. As Figure 17.3b shows, IPv6 includes 128-bit source and destination address fields.

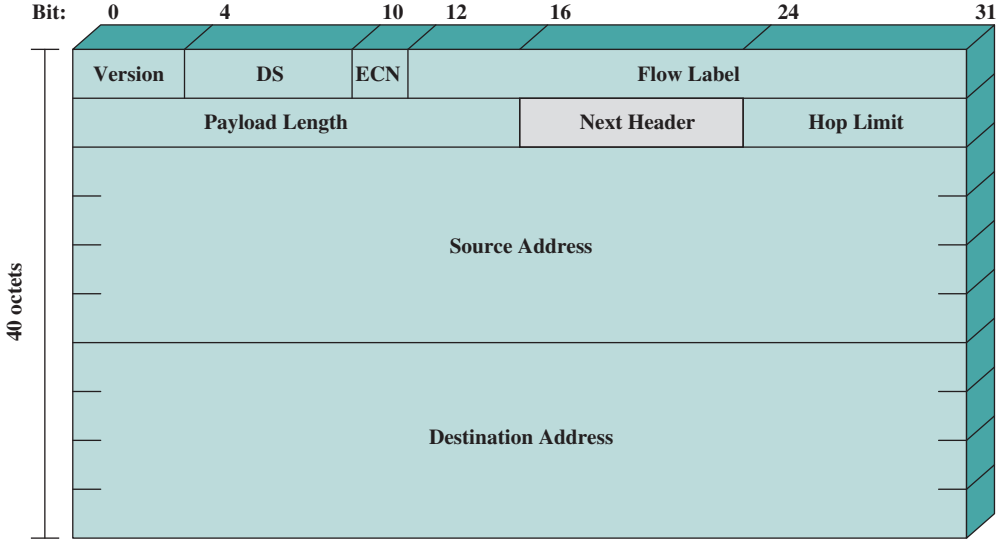
Ultimately, all installations using TCP/IP are expected to migrate from the current IP to IPv6, but this process will take many years, if not decades.

## Operation of TCP/IP

Figure 17.4 indicates how these protocols are configured for communications. Some sort of network access protocol, such as the Ethernet logic, is used to connect a computer to a network. This protocol enables the host to send data across the network to another host or, in the case of a host on another network, to a router. IP is implemented in all end systems and routers. It acts as a relay to move a block of data from




(a) IPv4 Header




(b) IPv6 Header

**DS = Differentiated services field**
**ECN = Explicit congestion notification field**

Note: The 8-bit DS/ECN fields were formerly known as the Type of Service field in the IPv4 header and the Traffic Class field in the IPv6 header.

**Figure 17.3 IP Headers**

one host, through one or more routers, to another host. TCP is implemented only in the end systems; it keeps track of the blocks of data being transferred to assure that all are delivered reliably to the appropriate application.

For successful communication, every entity in the overall system must have a unique address. In fact, two levels of addressing are needed. Each host on a network must have a unique global Internet address; this allows the data to be delivered to

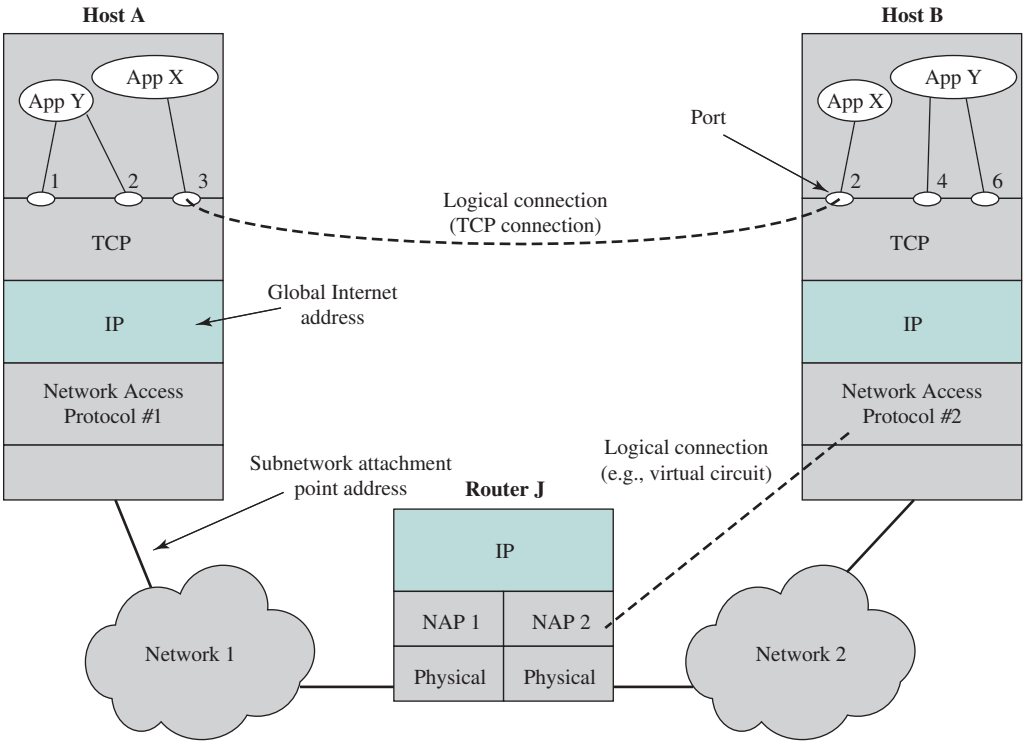



**Figure 17.4 TCP/IP Concepts**

the proper host. This address is used by IP for routing and delivery. Each application within a host must have an address that is unique within the host; this allows the host-to-host protocol (TCP) to deliver data to the proper process. These latter addresses are known as **ports**.

Let us trace a simple operation. Suppose a process, associated with port 3 at host A, wishes to send a message to another process, associated with port 2 at host B. The process at A hands the message down to TCP with instructions to send it to host B, port 2. TCP hands the message down to IP with instructions to send it to host B. Note IP need not be told the identity of the destination port. All it needs to know is that the data are intended for host B. Next, IP hands the message down to the network access layer (e.g., Ethernet logic) with instructions to send it to router J (the first hop on the way to B).

To control this operation, control information as well as user data must be transmitted, as suggested in Figure 17.5. Let us say that the sending process generates a block of data and passes this to TCP. TCP may break this block into smaller pieces to make it more manageable. To each of these pieces, TCP appends control information known as the TCP header (see Figure 17.2a), forming a **TCP segment**. The control information is to be used by the peer TCP protocol entity at host B.

Next, TCP hands each segment over to IP, with instructions to transmit it to B. These segments must be transmitted across one or more networks and relayed

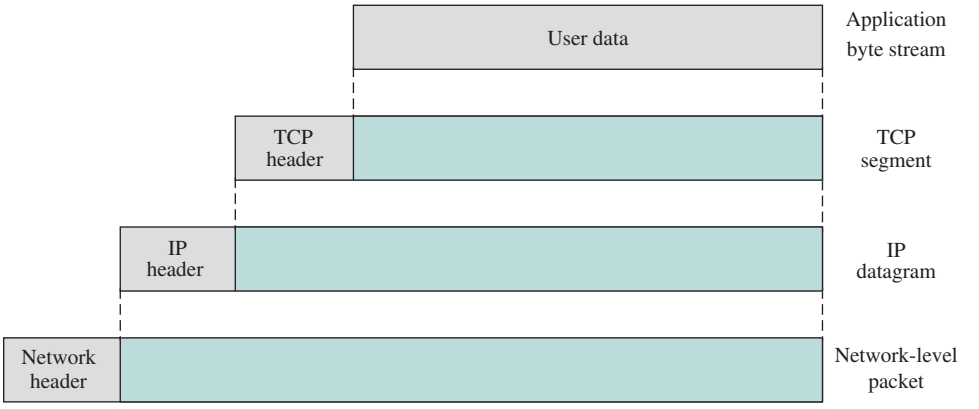



**Figure 17.5 Protocol Data Units (PDUs) in the TCP/IP Architecture**

through one or more intermediate routers. This operation, too, requires the use of control information. Thus IP appends a header of control information (see Figure 17.3) to each segment to form an **IP datagram**. An example of an item stored in the IP header is the destination host address (in this example, B).

Finally, each IP datagram is presented to the network access layer for transmission across the first network in its journey to the destination. The network access layer appends its own header, creating a packet, or frame. The packet is transmitted across the network to router J. The packet header contains the information that the network needs in order to transfer the data across the network. Examples of items that may be contained in this header include:

- **Destination network address:** The network must know to which attached device the packet is to be delivered, in this case router J.
- **Facilities requests:** The network access protocol might request the use of certain network facilities, such as priority.

At router J, the packet header is stripped off and the IP header examined. On the basis of the destination address information in the IP header, the IP module in the router directs the datagram across network 2 to B. To do this, the datagram is again augmented with a network access header.

When the data are received at B, the reverse process occurs. At each layer, the corresponding header is removed, and the remainder is passed on to the next higher layer, until the original user data are delivered to the destination process.

## TCP/IP Applications

A number of applications have been standardized to operate on top of TCP. We mention three of the most common here.

The **Simple Mail Transfer Protocol (SMTP)** provides a basic electronic mail facility. It provides a mechanism for transferring messages among separate hosts. Features of SMTP include mailing lists, return receipts, and forwarding. The SMTP protocol does not specify the way in which messages are to be created; some local

editing or native electronic mail facility is required. Once a message is created, SMTP accepts the message and makes use of TCP to send it to an SMTP module on another host. The target SMTP module will make use of a local electronic mail package to store the incoming message in a user's mailbox.

The **File Transfer Protocol (FTP)** is used to send files from one system to another under user command. Both text and binary files are accommodated, and the protocol provides features for controlling user access. When a user wishes to engage in file transfer, FTP sets up a TCP connection to the target system for the exchange of control messages. This connection allows user ID and password to be transmitted and allows the user to specify the file and file actions desired. Once a file transfer is approved, a second TCP connection is set up for the data transfer. The file is transferred over the data connection, without the overhead of any headers or control information at the application level. When the transfer is complete, the control connection is used to signal the completion and to accept new file transfer commands.

**SSH (Secure Shell)** provides a secure remote logon capability, which enables a user at a terminal or personal computer to log on to a remote computer and function as if directly connected to that computer. SSH also supports file transfer between the local host and a remote server. SSH enables the user and the remote server to authenticate each other; it also encrypts all traffic in both directions. SSH traffic is carried on a TCP connection.

## 17.3 SOCKETS[1]

The concept of sockets and sockets programming was developed in the 1980s in the UNIX environment as the Berkeley Sockets Interface. In essence, a socket enables communication between a client and server process and may be either connection oriented or connectionless. A socket can be considered an endpoint in a communication. A client socket in one computer uses an address to call a server socket on another computer. Once the appropriate sockets are engaged, the two computers can exchange data.

Typically, computers with server sockets keep a TCP or UDP port open, ready for unscheduled incoming calls. The client typically determines the socket identification of the desired server by finding it in a Domain Name System (DNS) database. Once a connection is made, the server switches the dialogue to a different port number to free up the main port number for additional incoming calls.

Internet applications, such as TELNET and remote login (rlogin), make use of sockets, with the details hidden from the user. However, sockets can be constructed from within a program (in a language such as C or Java), enabling the programmer to easily support networking functions and applications. The sockets programming mechanism includes sufficient semantics to permit unrelated processes on different hosts to communicate.

The Berkeley Sockets Interface is the de facto standard **application programming interface (API)** for developing networking applications, spanning a wide range

[1]This section provides a Sockets overview. Appendix M contains a more detailed treatment.

of operating systems. Windows Sockets (WinSock) is based on the Berkeley specification. The sockets API provide generic access to interprocess communications services. Thus, the sockets capability is ideally suited for students to learn the principles of protocols and distributed applications by hands-on program development.

## The Socket

Recall that each TCP and UDP header includes source port and destination port fields (see Figure 17.2). These port values identify the respective users (applications) of the two TCP entities. Also, each IPv4 and IPv6 header includes source address and destination address fields (see Figure 17.3); these **IP addresses** identify the respective host systems. The concatenation of a port value and an IP address forms a **socket**, which is unique throughout the Internet. Thus, in Figure 17.4, the combination of the IP address for host B and the port number for application X uniquely identifies the socket location of application X in host B. As the figure indicates, an application may have multiple socket addresses, one for each port into the application.

The socket is used to define an API, which is a generic communication interface for writing programs that use TCP or UDP. In practice, when used as an API, a socket is identified by the triple (protocol, local address, and local process). The local address is an IP address and the local process is a port number. Because port numbers are unique within a system, the port number implies the protocol (TCP or UDP). However, for clarity and ease of implementation, sockets used for an API include the protocol as well as the IP address and port number in defining a unique socket.

Corresponding to the two protocols, the Sockets API recognizes two types of sockets: stream sockets and datagram sockets. **Stream sockets** make use of TCP, which provides a connection-oriented reliable data transfer. Therefore, with stream sockets, all blocks of data sent between a pair of sockets are guaranteed for delivery and arrive in the order in which they were sent. **Datagram sockets** make use of UDP, which does not provide the connection-oriented features of TCP. Therefore, with datagram sockets, delivery is not guaranteed, nor is order necessarily preserved.

There is a third type of socket provided by the Sockets API: raw sockets. **Raw sockets** allow direct access to lower-layer protocols, such as IP.

## Socket Interface Calls

This subsection summarizes the key system calls.

***SOCKET SETUP*** The first step in using Sockets is to create a new socket using the `socket()` command. This command includes three parameters, the protocol family is always `PF_INET`, for the TCP/IP protocol suite. *Type* specifies whether this is a stream or datagram socket, and *protocol* specifies either TCP or UDP. The reason that both *type* and *protocol* need to be specified is to allow additional transport-level protocols to be included in a future implementation. Thus, there might be more than one datagram-style transport protocol, or more than one connection-oriented transport protocol. The `socket()` command returns an integer result that identifies this socket; it is similar to a UNIX file descriptor. The exact socket data structure depends on the implementation. It includes the source port and IP address and, if a

connection is open or pending, the destination port and IP address and various options and parameters associated with the connection.

After a socket is created, it must have an address to which to listen. The bind() function binds a socket to a socket address. The address has the structure

```
struct sockaddr_in {
    short int sin_family;         // Address family (TCP/IP)
    unsigned short int sin_port;// Port number
    struct in_addr sin_addr;     // Internet address
    unsigned char sin_zero[8];  // Same size as struct
};                                  sockaddr
```

***SOCKET CONNECTION*** For a stream socket, once the socket is created, a connection must be set up to a remote socket. One side functions as a client and requests a connection to the other side, which acts as a server.

The server side of a connection setup requires two steps. First, a server application issues a `listen()`, indicating the given socket is ready to accept incoming connections. The parameter *backlog* is the number of connections allowed on the incoming queue. Each incoming connection is placed in this queue until a matching `accept()` is issued by the server side. Next, the `accept()` call is used to remove one request from the queue. If the queue is empty, the `accept()` blocks the process until a connection request arrives. If there is a waiting call, then `accept()` returns a new file descriptor for the connection. This creates a new socket, which has the IP address and port number of the remote party, the IP address of this system, and a new port number. The reason that a new socket with a new port number is assigned is that this enables the local application to continue to listen for more requests. As a result, an application may have multiple connections active at any time, each with a different local port number. This new port number is returned across the TCP connection to the requesting system.

A client application issues a `connect()` that specifies both a local socket and the address of a remote socket. If the connection attempt is unsuccessful `connect()` returns the value −1. If the attempt is successful, `connect()` returns a 0 and fills in the file descriptor parameter to include the IP address and port number of the local and foreign sockets. Recall that the remote port number may differ from that specified in the `foreignAddress` parameter because the port number is changed on the remote host.

Once a connection is set up, `getpeername()` can be used to find out who is on the other end of the connected stream socket. The function returns a value in the `sockfd` parameter.

***SOCKET COMMUNICATION*** For **stream communication**, the functions `send()` and `recv()` are used to send or receive data over the connection identified by the sockfd parameter. In the `send()` call, the `*msg` parameter points to the block of data to be sent, and the `len` parameter specifies the number of bytes to be sent. The `flags` parameter contains control flags, typically set to 0. The `send()` call returns the number of bytes sent, which may be less than the number specified in the len parameter. In the `recv()` call, the `*buf` parameter points to the buffer for storing incoming data, with an upper limit on the number of bytes set by the `len` parameter.

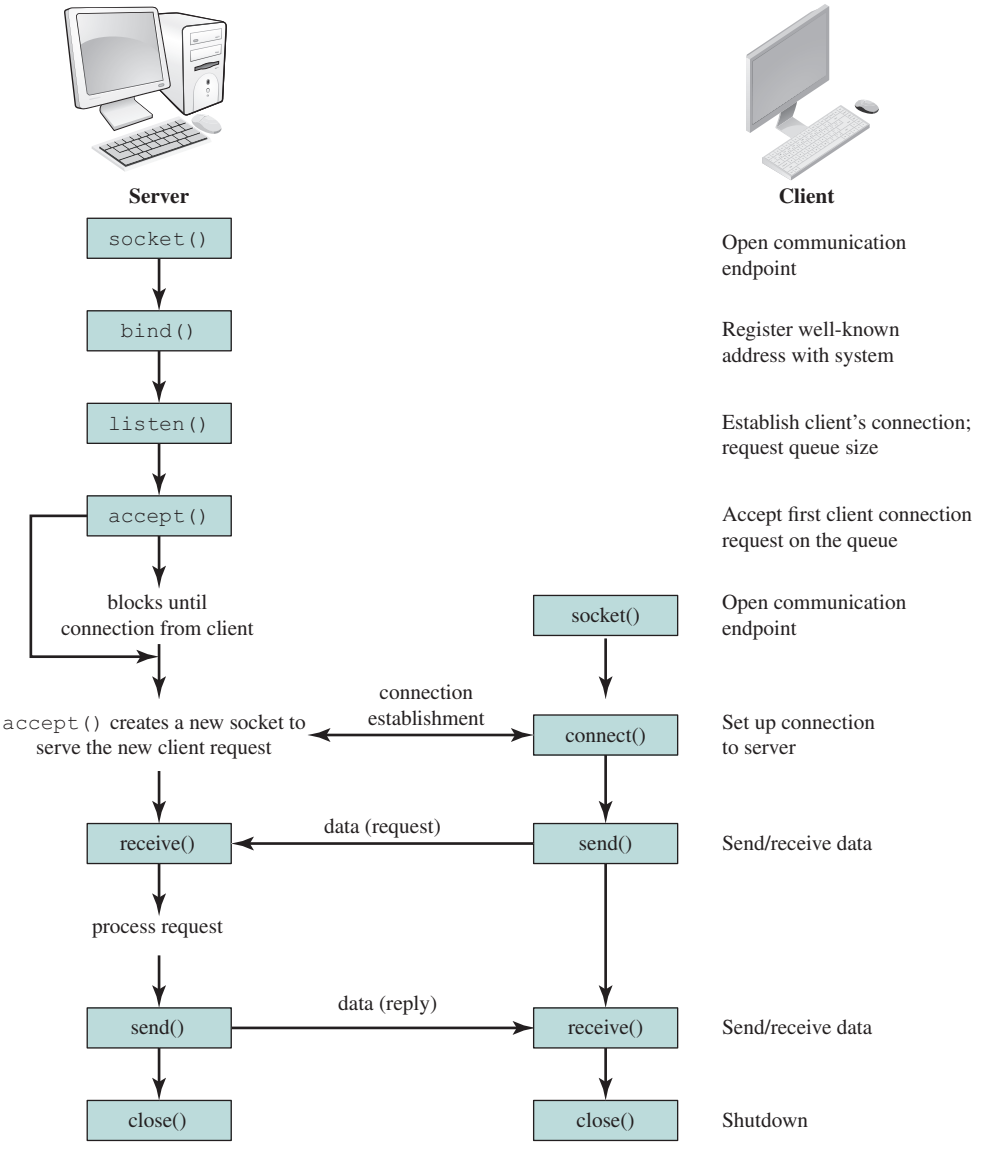



**Figure 17.6 Socket System Calls for Connection-Oriented Protocol**

At any time, either side can close the connection with the `close()` call, which prevents further sends and receives. The `shutdown()` call allows the caller to terminate sending or receiving or both.

Figure 17.6 shows the interaction of the clients and server sides in setting up, using, and terminating a connection.

For **datagram communication**, the functions `sendto()` and `recvfrom()` are used. The `sendto()` call includes all the parameters of the `send()` call plus a specification of the destination address (IP address and port). Similarly, the `recvfrom()` call includes an address parameter, which is filled in when data are received.

## 17.4 LINUX NETWORKING

Linux supports a variety of networking architectures, in particular TCP/IP by means of Berkeley Sockets. Figure 17.7 shows the overall structure of Linux support for TCP/IP. User-level processes interact with networking devices by means of system calls to the Sockets interface. The Sockets module in turn interacts with a software package in the kernel that handles transport-layer (TCP and UDP) and IP protocol operations. This software package exchanges data with the device driver for the network interface card.

Linux implements sockets as special files. Recall from Chapter 12 that, in UNIX systems, a special file is one that contains no data but provides a mechanism to map

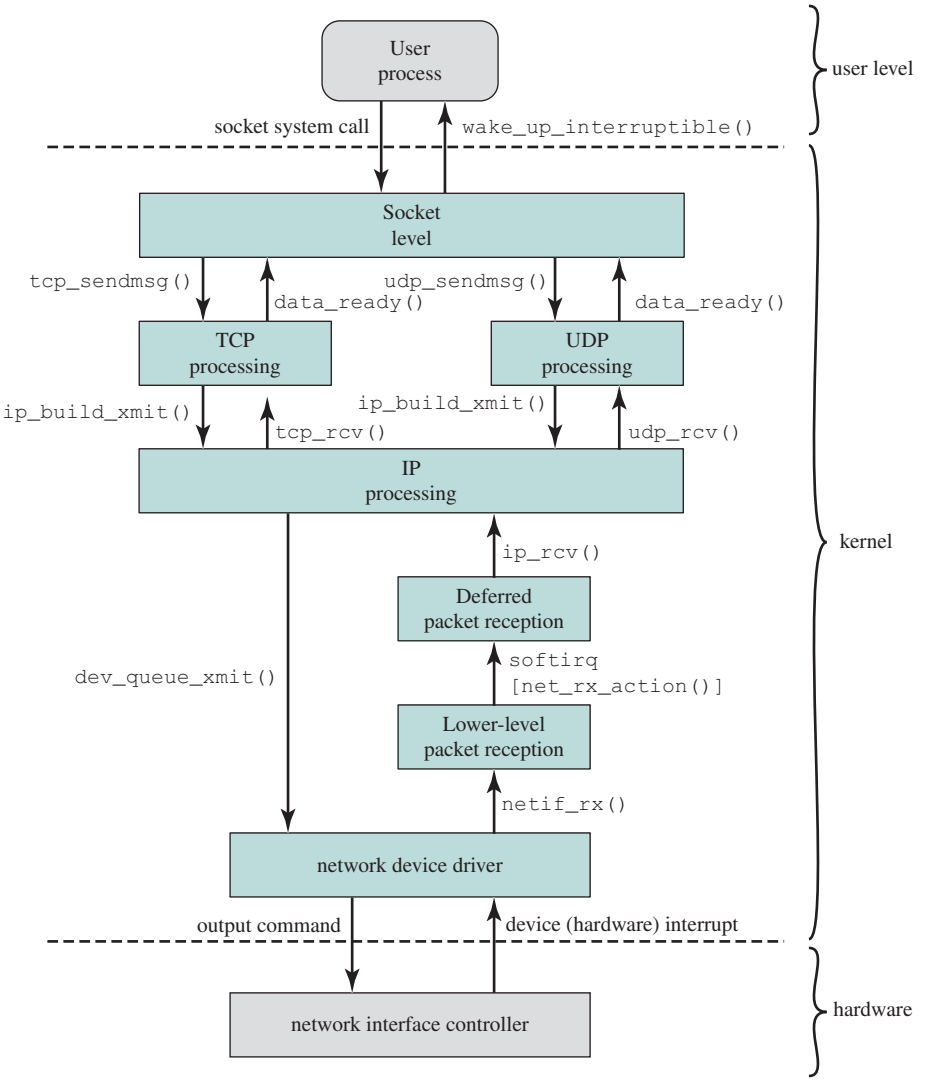



**Figure 17.7 Linux Kernel Components for TCP/IP Processing**

physical devices to file names. For every new socket, the Linux kernel creates a new inode in the *sockfs* special file system.

Figure 17.7 depicts the relationships among various kernel modules involved in sending and receiving TCP/IP-based data blocks. The remainder of this section looks at the sending and receiving facilities.

## Sending Data

A user process uses the sockets calls described in Section 17.3 to create new sockets, set up connections to remote sockets, and send and receive data. To send data, the user process writes data to the socket with the following file system call:

```
write(sockfd, mesg, mesglen)
```

where `mesglen` is the length of the `mesg` buffer in bytes.

This call triggers the `write` method of the file object associated with the `sockfd` file descriptor. The file descriptor indicates whether this is a socket set up for TCP or UDP. The kernel allocates the appropriate data structures and invokes the appropriate sockets-level function to pass data to either a TCP module or a UDP module. The corresponding functions are `tcp_sendmsg()` and `udp_sendmsg()`, respectively. The transport-layer module allocates a data structure of the TCP or UPD header and performs `ip_build_xmit()` to invoke the IP-layer processing module. This module builds an IP datagram for transmission and places it in a transmission buffer for this socket. The IP-layer module then performs `dev_queue_xmit()` to queue the socket buffer for later transmission via the network device driver. When it is available, the network device driver will transmit buffered packets.

## Receiving Data

Data reception is an unpredictable event and so involves the use of interrupts and deferrable functions. When an IP datagram arrives, the network interface controller issues a hardware interrupt to the corresponding network device driver. The interrupt triggers an interrupt service routine that handles the interrupt as part of the network device driver module. The driver allocates a kernel buffer for the incoming data block and transfers the data from the device controller to the buffer. The driver then performs `netif_rx()` to invoke a lower-level packet reception routine. In essence, the `netif_rx()` function places the incoming data block in a queue then issues a soft interrupt request (`softirq`) so the queued data will eventually be processed. The action to be performed when the `softirq` is processed is the `net_rx_action()` function.

Once a `softirq` has been queued, processing of this packet is halted until the kernel executes the `softirq` function, which is equivalent to saying until the kernel responds to this soft interrupt request and executes the function (in this case, `net_rx_action()`) associated with this soft interrupt. There are three places in the kernel, where the kernel checks to see if any `softirqs` are pending: when a hardware interrupt has been processed, when an application-level process invokes a system call, and when a new process is scheduled for execution.

When the `net_rx_action()` function is performed, it retrieves the queued packet and passes it on to the IP packet handler by means of an `ip_rcv` call. The IP packet handler processes the IP header then uses `tcp_rcv` or `udp_rcv` to invoke the transport-layer processing module. The transport-layer module processes the

transport-layer header and passes the data to the user through the sockets interface by means of a `wake_up_interruptible()` call, which awakens the receiving process.

## 17.5 SUMMARY

The communication functionality required for distributed applications is quite complex. This functionality is generally implemented as a structured set of modules. The modules are arranged in a vertical, layered fashion, with each layer providing a particular portion of the needed functionality and relying on the next lower layer for more primitive functions. Such a structure is referred to as a protocol architecture.

One motivation for the use of this type of structure is that it eases the task of design and implementation. It is standard practice for any large software package to break up the functions into modules that can be designed and implemented separately. After each module is designed and implemented, it can be tested. Then the modules can be combined and tested together. This motivation has led computer vendors to develop proprietary layered-protocol architectures. An example of this is the Systems Network Architecture (SNA) of IBM.

A layered architecture can also be used to construct a standardized set of communication protocols. In this case, the advantages of modular design remain. But, in addition, a layered architecture is particularly well-suited to the development of standards. Standards can be developed simultaneously for protocols at each layer of the architecture. This breaks down the work to make it more manageable and speeds up the standards-development process. The TCP/IP protocol architecture is the standard architecture used for this purpose. This architecture contains five layers. Each layer provides a portion of the total communications function required for distributed applications. Standards have been developed for each layer. Development work continues, particularly at the top (application) layer, where new distributed applications are still being defined.

## 17.6 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

### Key Terms

| | | |
|---|---|---|
| application layer | protocol | syntax |
| application programming interface (API) | protocol architecture | network access layer |
| datagram communication | raw socket | TCP segment |
| datagram socket | router | TELNET |
| File Transfer Protocol (FTP) | semantics | timing |
| Internet Protocol (IP) | Simple Mail Transfer Protocol (SMTP) | Transmission Control Protocol (TCP) |
| IP addresses | socket | transport layer |
| IP datagram | SSH (secure shell) | User Datagram Protocol (UDP) |
| physical layer | stream communication | |
| port | stream socket | |

## Review Questions

**17.1.** What is the major function of the network access layer?
**17.2.** What tasks are performed by the transport layer?
**17.3.** What is a protocol?
**17.4.** What is a protocol architecture?
**17.5.** What is TCP/IP?
**17.6.** What is the purpose of the Sockets interface?

## Problems

**17.1.** For this problem, first consider the case where you wish to order pizza for a party of guests. The layer models in Figure 17.8 can be used to describe the ordering and delivery of a pizza. The guest effectively places the order with the cook. The host communicates this order to the clerk, who places the order with the cook. The phone system provides the physical means for the order to be transported from host to clerk. The cook gives the pizza to the clerk with the order form (acting as a "header" to the pizza). The clerk boxes the pizza with the delivery address, and the delivery van encloses all of the orders to be delivered. The road provides the physical path for the delivery.

**a.** The French and Chinese prime ministers need to come to an agreement by telephone, but neither speaks the other's language. Further, neither has on hand a translator that can translate to the language of the other. However, both prime ministers have English translators on their staffs. Draw a diagram similar to Figure 17.8 to depict the situation, and describe the interaction at each layer.

**b.** Now suppose the Chinese prime minister's translator can translate only into Japanese and the French prime minister has a German translator available. A translator between German and Japanese is available in Germany. Draw a new diagram that reflects this arrangement, and describe the hypothetical phone conversation.

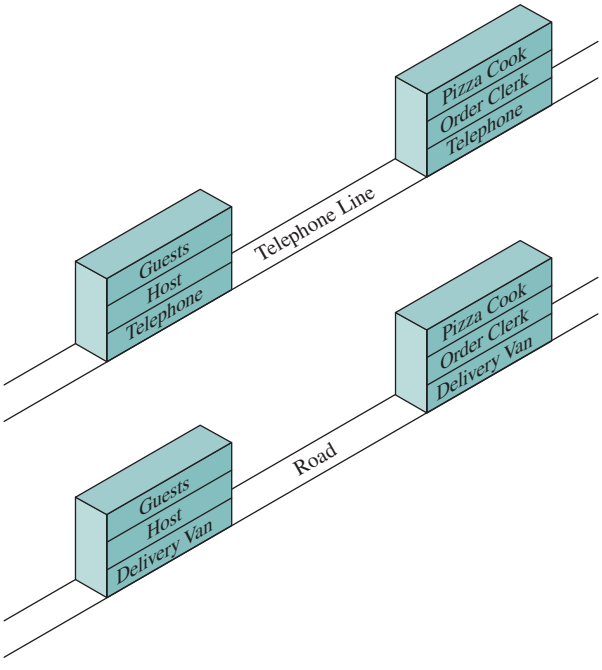



**Figure 17.8 Architecture for Problem 17.1**

**17.2.** List the major disadvantages of the layered approach to protocols.

**17.3.** A TCP segment consisting of 1,500 bits of data and 160 bits of header is sent to the IP layer, which appends another 160 bits of header. This is then transmitted through two networks, each of which uses a 24-bit packet header. The destination network has a maximum packet size of 800 bits. How many bits, including headers, are delivered to the network layer protocol at the destination?

**17.4.** Why does the TCP header have a header length field while the UDP header does not?

**17.5.** The previous version of the TFTP specification, RFC 783, included the following statement:

*All packets other than those used for termination are acknowledged individually unless a timeout occurs.*

The new specification revises this to say

*All packets other than duplicate ACKs and those used for termination are acknowledged unless a timeout occurs.*

The change was made to fix a problem referred to as the "Sorcerer's Apprentice." Deduce and explain the problem.

**17.6.** What is the limiting factor in the time required to transfer a file using TFTP?

**17.7.** A user on a UNIX host wants to transfer a 4,000-byte text file to a Microsoft Windows host. In order to do this, he transfers the file by means of TFTP, using the netascii transfer mode. Even though the transfer was reported as being performed successfully, the Windows host reports the resulting file size is 4,050 bytes, rather than the original 4,000 bytes. Does this difference in the file sizes imply an error in the data transfer? Why or why not?

**17.8.** The TFTP specification (RFC 1350) states that the transfer identifiers (TIDs) chosen for a connection should be randomly chosen, so the probability that the same number is chosen twice in immediate succession is very low. What would be the problem of using the same TIDs twice in immediate succession?

**17.9.** In to retransmit lost packets, TFTP must keep a copy of the data it sends. How many packets of data must TFTP keep at a time to implement this retransmission mechanism?

**17.10.** TFTP, like most protocols, will never send an error packet in response to an error packet it receives. Why?

**17.11.** We have seen that in order to deal with lost packets, TFTP implements a time-out-and-retransmit scheme, by setting a retransmission timer when it transmits a packet to the remote host. Most TFTP implementations set this timer to a fixed value of about five seconds. Discuss the advantages and the disadvantages of using a fixed value for the retransmission timer.

**17.12.** TFTP's time-out-and-retransmission scheme implies that all data packets will eventually be received by the destination host. Will these data also be received uncorrupted? Why or why not?

**17.13.** This chapter mentions the use of Frame Relay as a specific protocol or system used to connect to a wide area network. Each organization will have a certain collection of services available (like Frame Relay) but this is dependent upon provider provisioning, cost and customer premises equipment. What are some of the services available to you in your area?

**17.14.** Wireshark is a free packet sniffer that allows you to capture traffic on a local area network. It can be used on a variety of operating systems and is available at www.ethereal.com. You must also install the WinPcap packet capture driver, which can be obtained from www.wireshark.org/.

After starting a capture from Wireshark, start a TCP-based application like TELNET, FTP, or HTTP (Web browser). Can you determine the following from your capture?

**a.** Source and destination layer 2 addresses (MAC).

**b.** Source and destination layer 3 addresses (IP).

**c.** Source and destination layer 4 addresses (port numbers).

**17.15.** Packet capture software or sniffers can be powerful management and security tools. By using the filtering capability that is built in, you can trace traffic based on several different criteria and eliminate everything else. Use the filtering capability built into Ethereal to do the following;

**a.** Capture only traffic coming from your computer's MAC address.

**b.** Capture only traffic coming from your computer's IP address.

**c.** Capture only UDP-based transmissions.

# APPENDIX 17A THE TRIVIAL FILE TRANSFER PROTOCOL

This appendix provides an overview of the Internet standard Trivial File Transfer Protocol (TFTP), defined in RFC 1350. Our purpose is to give the reader some flavor for the elements of a protocol. TFTP is simple enough to provide a concise example but includes most of the significant elements found in other, more complex, protocols.

## Introduction to TFTP

TFTP is far simpler than the Internet standard File Transfer Protocol (FTP). There are no provisions for access control or user identification, so TFTP is only suitable for public access file directories. Because of its simplicity, TFTP is easily and compactly implemented. For example, some diskless devices use TFTP to download their firmware at boot time.

TFTP runs on top of UDP. The TFTP entity that initiates the transfer does so by sending a read or write request in a UDP segment with a destination port of 69 to the target system. This port is recognized by the target UDP module as the identifier of the TFTP module. For the duration of the transfer, each side uses a transfer identifier (TID) as its port number.

## TFTP Packets

TFTP entities exchange commands, responses, and file data in the form of packets, each of which is carried in the body of a UDP segment. TFTP supports five types of packets (see Figure 17.9); the first two bytes contain an opcode that identifies the packet type:

- **RRQ:** The read request packet requests permission to transfer a file from the other system. The packet includes a file name, which is a sequence of ASCII[2] bytes terminated by a zero byte. The zero byte is the means by which the receiving TFTP entity knows when the file name is terminated. The packet also includes a mode field, which indicates whether the data file is to be interpreted as a string of ASCII bytes (netascii mode) or as raw 8-bit bytes (octet mode) of data. In netascii mode, the file is transferred as lines of characters, each terminated by a carriage return, line feed. Each system must translate between its own format for character files and the TFTP format.

[2]ASCII is the American Standard Code for Information Interchange, a standard of the American National Standards Institute. It designates a unique 7-bit pattern for each letter, with an eighth bit used for parity. ASCII is equivalent to the International Reference Alphabet (IRA), defined in ITU-T Recommendation T.50. See Appendix N for a discussion.

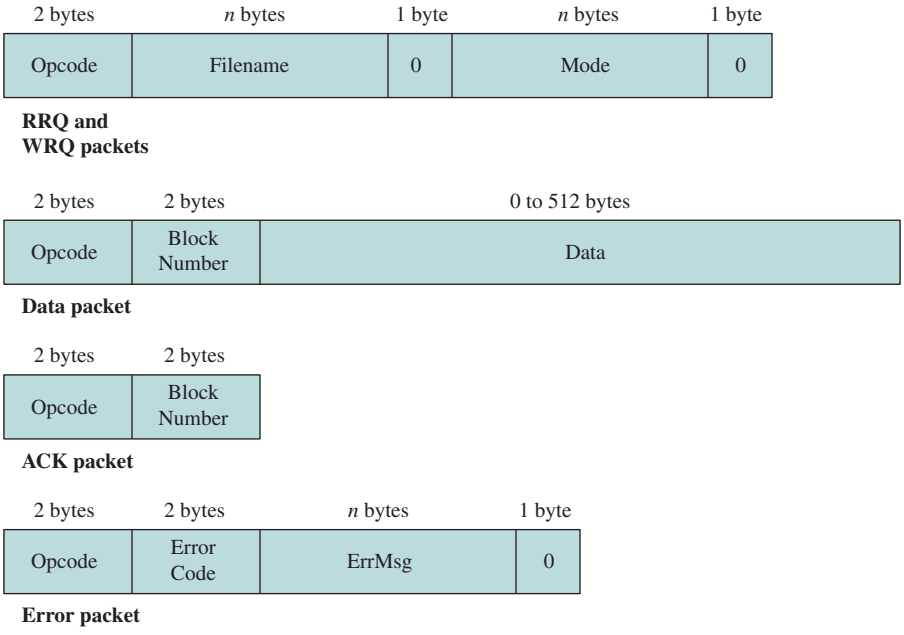



**Figure 17.9 TFTP Packet Formats**

- **WRQ:** The write request packet requests permission to transfer a file to the other system.
- **Data:** The block numbers on data packets begin with one and increase by one for each new block of data. This convention enables the program to use a single number to discriminate between new packets and duplicates. The data field is from 0 to 512 bytes long. If it is 512 bytes long, the block is not the last block of data; if it is from 0 to 511 bytes long, it signals the end of the transfer.
- **ACK:** This packet is used to acknowledge receipt of a data packet or a WRQ packet. An ACK of a data packet contains the block number of the data packet being acknowledged. An ACK of a WRQ contains a block number of zero.
- **Error:** An error packet can be the acknowledgment of any other type of packet. The error code is an integer indicating the nature of the error (see Table 17.1). The error message is intended for human consumption and should be in ASCII. Like all other strings, it is terminated with a zero byte.

All packets other than duplicate ACKs (explained subsequently) and those used for termination are to be acknowledged. Any packet can be acknowledged by an error packet. If there are no errors, then the following conventions apply. A WRQ or a data packet is acknowledged by an ACK packet. When an RRQ is sent, the other side responds (in the absence of error) by beginning to transfer the file; thus, the first data block serves as an acknowledgment of the RRQ packet. Unless a file transfer is complete, each ACK packet from one side is followed by a data packet from the other, so the data packet functions as an acknowledgment. An error packet can be acknowledged by any other kind of packet, depending on the circumstance.

**Table 17.1** TFTP Error Codes

| Value | Meaning |
|---|---|
| 0 | Not defined, see error message (if any) |
| 1 | File not found |
| 2 | Access violation |
| 3 | Disk full or allocation exceeded |
| 4 | Illegal TFTP operation |
| 5 | Unknown transfer ID |
| 6 | File already exists |
| 7 | No such user |

Figure 17.10 shows a TFTP data packet in context. When such a packet is handed down to UDP, UDP adds a header to form a UDP segment. This is then passed to IP, which adds an IP header to form an IP datagram.

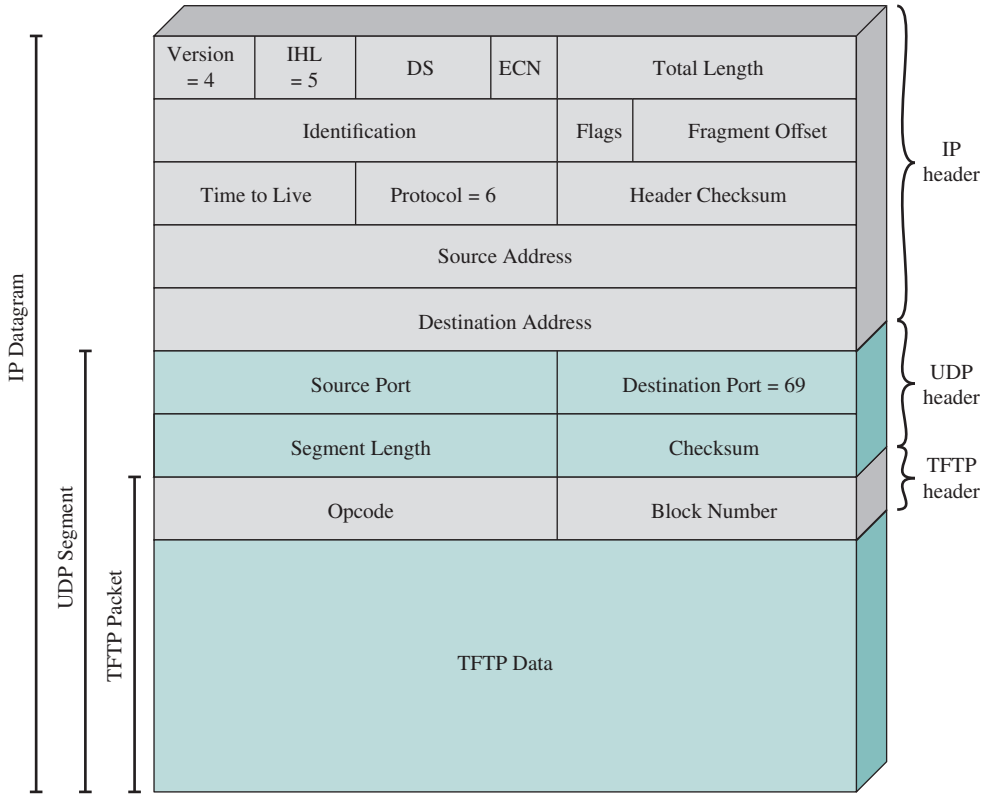



**Figure 17.10 A TFTP Packet in Context**

## Overview of a Transfer

The example illustrated in Figure 17.11 is of a simple file transfer operation from A to B. No errors occur and the details of the option specification are not explored.

The operation begins when the TFTP module in system A sends a WRQ to the TFTP module in system B. The WRQ packet is carried as the body of a UDP segment. The WRQ includes the name of the file (in this case, XXX) and a mode of octet, or raw data. In the UDP header, the destination port number is 69, which alerts the receiving UDP entity that this message is intended for the TFTP application. The source port number is a TID selected by A, in this case 1511. System B is prepared to accept the file and so responds with an ACK with a block number of 0. In the UDP header, the destination port is 1511, which enables the UDP entity at A to route the incoming packet to the TFTP module, which can match this TID with

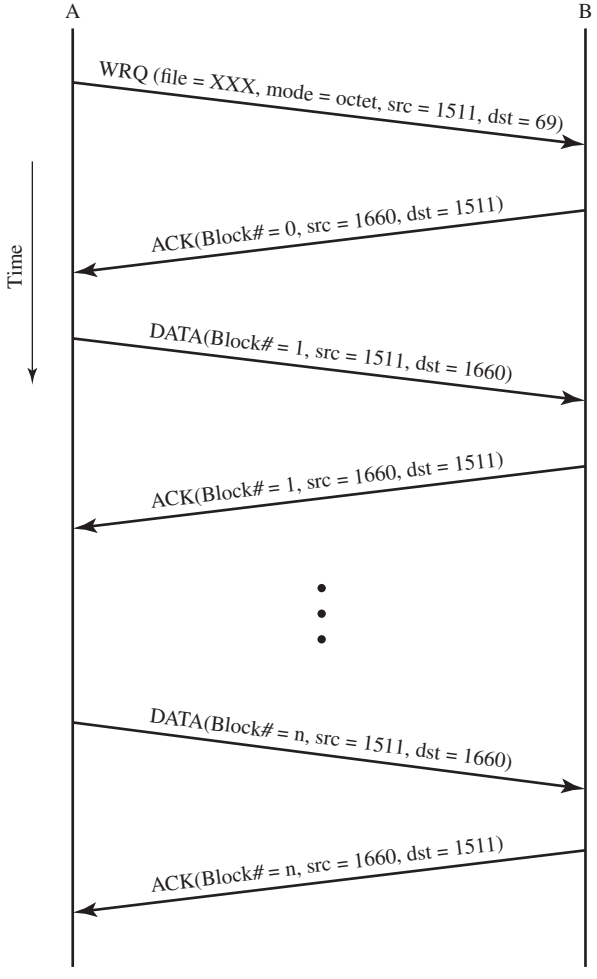



**Figure 17.11 Example TFTP Operation**

the TID in the WRQ. The source port is a TID selected by B for this file transfer, in this case 1660.

Following this initial exchange, the file transfer proceeds. The transfer consists of one or more data packets from A, each of which is acknowledged by B. The final data packet contains less than 512 bytes of data, which signals the end of the transfer.

## Errors and Delays

If TFTP operates over a network or the Internet (as opposed to a direct data link), it is possible for packets to be lost. Because TFTP operates over UDP, which does not provide a reliable delivery service, there needs to be some mechanism in TFTP to deal with lost packets. TFTP uses the common technique of a time-out mechanism. Suppose A sends a packet to B that requires an acknowledgment (i.e., any packet other than duplicate ACKs and those used for termination). When A has transmitted the packet, it starts a timer. If the timer expires before the acknowledgment is received from B, A retransmits the same packet. If in fact the original packet was lost, then the retransmission will be the first copy of this packet received by B. If the original packet was not lost but the acknowledgment from B was lost, then B will receive two copies of the same packet from A and simply acknowledges both copies. Because of the use of block numbers, this causes no confusion. The only exception to this rule is for duplicate ACK packets. The second ACK is ignored.

## Syntax, Semantics, and Timing

In Section 17.1, it was mentioned that the key features of a protocol can be classified as syntax, semantics, and timing. These categories are easily seen in TFTP. The formats of the various TFTP packets determine the **syntax** of the protocol. The **semantics** of the protocol are shown in the definitions of each of the packet types and the error codes. Finally, the sequence in which packets are exchanged, the use of block numbers, and the use of timers are all aspects of the **timing** of TFTP.

CHAPTER 18

# Distributed Processing, Client/Server, and Clusters


18.1 **Client/Server Computing**
- What Is Client/Server Computing?
- Client/Server Applications
- Middleware

18.2 **Distributed Message Passing**
- Reliability versus Unreliability
- Blocking versus Nonblocking

18.3 **Remote Procedure Calls**
- Parameter Passing
- Parameter Representation
- Client/Server Binding
- Synchronous versus Asynchronous
- Object-Oriented Mechanisms

18.4 **Clusters**
- Cluster Configurations
- Operating System Design Issues
- Cluster Computer Architecture
- Clusters Compared to SMP

18.5 **Windows Cluster Server**

18.6 **Beowulf and Linux Clusters**
- Beowulf Features
- Beowulf Software

18.7 **Summary**

18.8 **References**

18.9 **Key Terms, Review Questions, and Problems**

**Learning Objectives**

After studying this chapter, you should be able to:

- Present a summary of the key aspects of client/server computing.
- Understand the principle design issues for distributed message passing.
- Understand the principle design issues for remote procedure calls.
- Understand the principle design issues for clusters.
- Describe the cluster mechanisms in Windows 7 and Beowulf.

In this chapter, we begin with an examination of some of the key concepts in distributed software, including client/server architecture, message passing, and remote procedure calls. Then we examine the increasingly important cluster architecture.

Chapters 17 and 18 complete our discussion of distributed systems.

## 18.1 CLIENT/SERVER COMPUTING

The concept of client/server computing, and related concepts, has become increasingly important in information technology systems. This section begins with a description of the general nature of client/server computing. This is followed by a discussion of alternative ways of organizing the client/server functions. The issue of **file cache consistency**, raised by the use of file servers, is then examined. Finally, this section introduces the concept of middleware.

### What Is Client/Server Computing?

As with other new waves in the computer field, client/server computing comes with its own set of jargon words. Table 18.1 lists some of the terms that are commonly found in descriptions of client/server products and applications.

**Table 18.1** Client/Server Terminology

| |
|---|
| **Applications Programming Interface (API)**<br>A set of function and call programs that allow clients and servers to intercommunicate. |
| **Client**<br>A networked information requester, usually a PC or workstation, that can query a database and/or other information from a server. |
| **Middleware**<br>A set of drivers, APIs, or other software that improves connectivity between a client application and a server. |
| **Relational Database**<br>A database in which information access is limited to the selection of rows that satisfy all search criteria. |
| **Server**<br>A computer, usually a high-powered workstation, a minicomputer, or a mainframe, that houses information for manipulation by networked clients. |
| **Structured Query Language (SQL)**<br>A language developed by IBM and standardized by ANSI for addressing, creating, updating, or querying relational databases. |

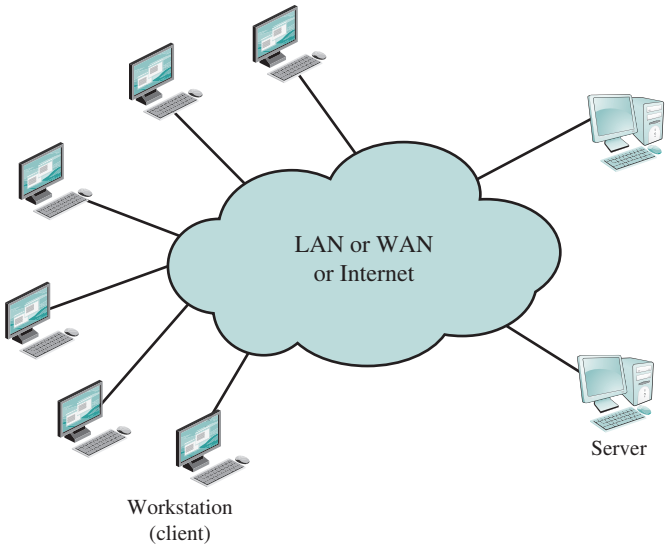



**Figure 18.1 Generic Client/Server Environment**

Figure 18.1 attempts to capture the essence of the client/server concept. As the term suggests, a *client/server environment* is populated by clients and servers. The **client** machines are generally single-user PCs or workstations that provide a user-friendly interface to the end user. The client-based station generally presents the type of graphical interface that is most comfortable to users, including the use of windows and a mouse. Microsoft Windows and Macintosh OS provide examples of such interfaces. Client-based applications are tailored for ease of use and include such familiar tools as the spreadsheet.

Each **server** in the client/server environment provides a set of shared services to the clients. The most common type of server currently is the database server, usually controlling a relational database. The server enables many clients to share access to the same database and enables the use of a high-performance computer system to manage the database.

In addition to clients and servers, the third essential ingredient of the client/server environment is the **network**. Client/server computing is typically distributed computing. Users, applications, and resources are distributed in response to business requirements and linked by a single LAN or WAN or by an internet of networks.

How does a client/server configuration differ from any other distributed processing solution? There are a number of characteristics that stand out and together, make client/server distinct from other types of distributed processing:

- There is a heavy reliance on bringing user-friendly applications to the user on his or her system. This gives the user a great deal of control over the timing and style of computer usage, and gives department-level managers the ability to be responsive to their local needs.
- Although applications are dispersed, there is an emphasis on centralizing corporate databases and many network management and utility functions. This

enables corporate management to maintain overall control of the total capital investment in computing and information systems and to provide interoperability so systems are tied together. At the same time, it relieves individual departments and divisions of much of the overhead of maintaining sophisticated computer-based facilities, but enables them to choose just about any type of machine and interface they need to access data and information.

- There is a commitment, both by user organizations and vendors, to open and modular systems. This means that the user has more choice in selecting products and in mixing equipment from a number of vendors.
- Networking is fundamental to the operation. Thus, network management and network security have a high priority in organizing and operating information systems.

## Client/Server Applications

The key feature of a client/server architecture is the allocation of application-level tasks between clients and servers. Figure 18.2 illustrates the general case. In both client and server, of course, the basic software is an operating system running on the hardware platform. The platforms and the operating systems of client and server may differ. Indeed, there may be a number of different types of client platforms and operating systems and a number of different types of server platforms in a single environment. As long as a particular client and server share the same communications protocols and support the same applications, these lower-level differences are irrelevant.

It is the communications software that enables client and server to interoperate. The principal example of such software is TCP/IP. Of course, the point of all of this support software (communications and operating system) is to provide a base for distributed applications. Ideally, the actual functions performed by the application can be split up between client and server in a way that optimizes the use of resources. In some cases, depending on the application needs, the bulk of the applications

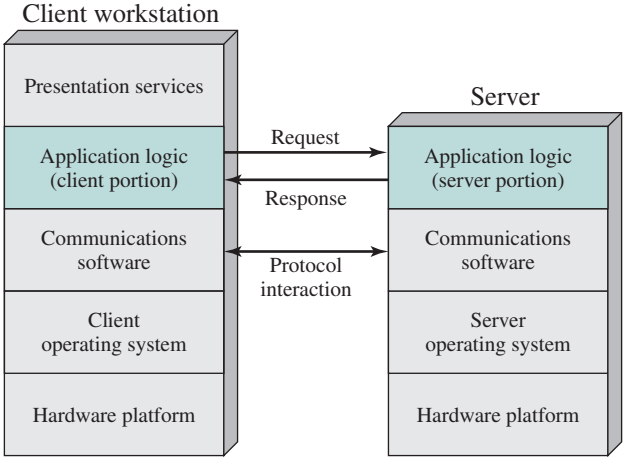



**Figure 18.2 Generic Client/Server Architecture**

software executes at the server, while in other cases, most of the application logic is located at the client.

An essential factor in the success of a client/server environment is the way in which the user interacts with the system as a whole. Thus, the design of the user interface on the client machine is critical. In most client/server systems, there is heavy emphasis on providing a **graphical user interface (GUI)** that is easy to use, easy to learn, yet powerful and flexible. Thus, we can think of a presentation services module in the client workstation that is responsible for providing a user-friendly interface to the distributed applications available in the environment.

***DATABASE APPLICATIONS*** As an example that illustrates the concept of splitting application logic between client and server, let us consider one of the most common families of client/server applications: those that use relational databases. In this environment, the server is essentially a database server. Interaction between client and server is in the form of transactions in which the client makes a database request and receives a database response.

Figure 18.3 illustrates, in general terms, the architecture of such a system. The server is responsible for maintaining the database, for which purpose a complex database management system software module is required. A variety of different applications that make use of the database can be housed on client machines. The "glue" that ties client and server together is software that enables the client to make requests for access to the server's database. A popular example of such logic is the structured query language (SQL).

Figure 18.3 suggests that all of the application logic—the software for "number crunching" or other types of data analysis—is on the client side, while the server is only concerned with managing the database. Whether such a configuration is appropriate depends on the style and intent of the application. For example, suppose the primary

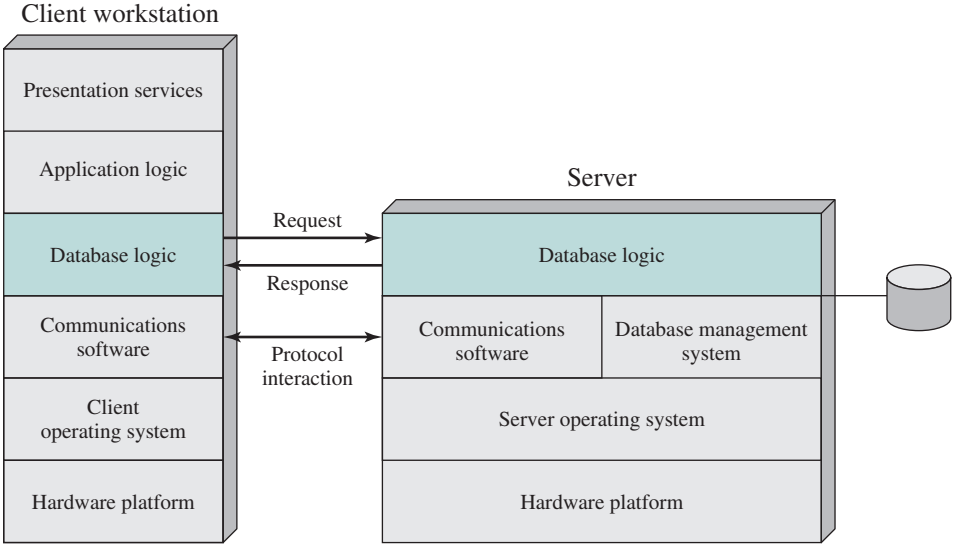
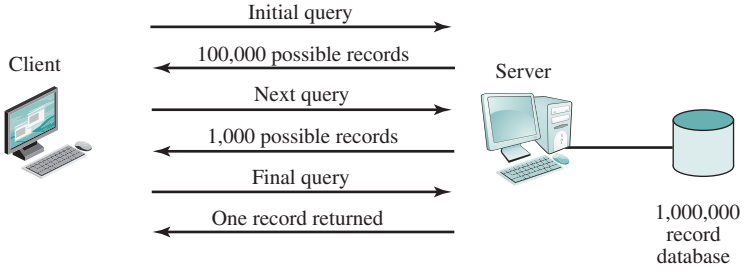



**Figure 18.3 Client/Server Architecture for Database Applications**

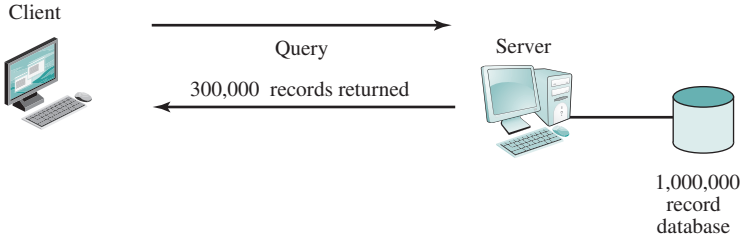
purpose is to provide online access for record lookup. Figure 18.4a suggests how this might work. Suppose the server is maintaining a database of 1 million records (called rows in relational database terminology), and the user wants to perform a lookup that should result in zero, one, or at most a few records. The user could search for these records using a number of search criteria (e.g., records older than 1992, records referring to individuals in Ohio, records referring to a specific event or characteristic, etc.). An initial client query may yield a server response that there are 100,000 records that satisfy the search criteria. The user then adds additional qualifiers and issues a new query. This time, a response indicating that there are 1,000 possible records is returned. Finally, the client issues a third request with additional qualifiers. The resulting search criteria yield a single match, and the record is returned to the client.

The preceding application is well-suited to a client/server architecture for two reasons:

1. There is a massive job of sorting and searching the database. This requires a large disk or bank of disks, a high-speed CPU, and a high-speed I/O architecture. Such capacity and power is not needed and is too expensive for a single-user workstation or PC.
2. It would place too great a traffic burden on the network to move the entire 1-million-record file to the client for searching. Therefore, it is not enough for the server just to be able to retrieve records on behalf of a client; the server needs to have database logic that enables it to perform searches on behalf of a client.




(a) Desirable client/server use




(b) Misused client/server

**Figure 18.4 Client/Server Database Usage**

Now consider the scenario of Figure 18.4b, which has the same 1-million-record database. In this case, a single query results in the transmission of 300,000 records over the network. This might happen if, for example, the user wishes to find the grand total or mean value of some field across many records or even the entire database.

Clearly, this latter scenario is unacceptable. One solution to this problem, which maintains the client/server architecture with all of its benefits, is to move part of the application logic over to the server. That is, the server can be equipped with application logic for performing data analysis as well as data retrieval and data searching.

***CLASSES OF CLIENT/SERVER APPLICATIONS*** Within the general framework of client/server, there is a spectrum of implementations that divide the work between client and server differently. Figure 18.5 illustrates in general terms some of the major options for database applications. Other splits are possible, and the options may have a different characterization for other types of applications. In any case, it is useful to examine this figure to get a feel for the kind of trade-offs possible.

Figure 18.5 depicts four classes:

- **Host-based processing:** *Host-based processing* is not true client/server computing as the term is generally used. Rather, host-based processing refers to the

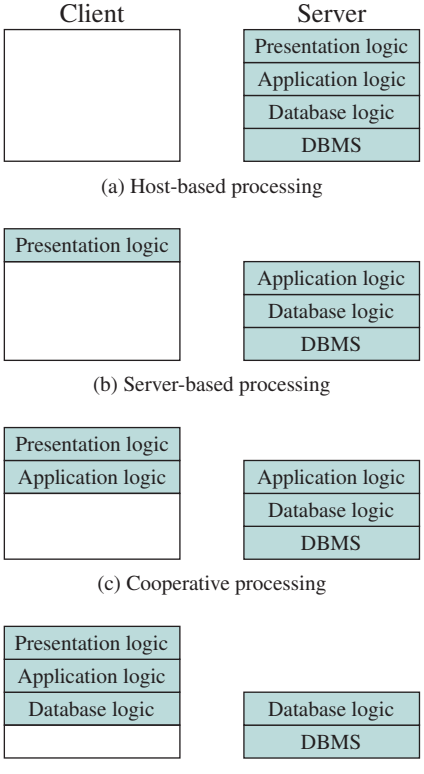



**Figure 18.5 Classes of Client/Server Applications**

traditional mainframe environment in which all or virtually all of the processing is done on a central host. Often the user interface is via a dumb terminal. Even if the user is employing a microcomputer, the user's station is generally limited to the role of a terminal emulator.

- **Server-based processing:** The most basic class of client/server configuration is one in which the client is principally responsible for providing a graphical user interface, while virtually all of the processing is done on the server. This configuration is typical of early client/server efforts, especially departmental-level systems. The rationale behind such configurations is that the user workstation is best suited to providing a user-friendly interface, and that databases and applications can easily be maintained on central systems. Although the user gains the advantage of a better interface, this type of configuration does not generally lend itself to any significant gains in productivity, or to any fundamental changes in the actual business functions that the system supports.
- **Client-based processing:** At the other extreme, virtually all application processing may be done at the client, with the exception of data validation routines and other database logic functions that are best performed at the server. Generally, some of the more sophisticated database logic functions are housed on the client side. This architecture is perhaps the most common client/server approach in current use. It enables the user to employ applications tailored to local needs.
- **Cooperative processing:** In a cooperative processing configuration, the application processing is performed in an optimized fashion, taking advantage of the strengths of both client and server machines and of the distribution of data. Such a configuration is more complex to set up and maintain but, in the long run, this type of configuration may offer greater user productivity gains and greater network efficiency than other client/server approaches.

Figures 18.5c and 18.5d correspond to configurations in which a considerable fraction of the load is on the client. This so-called **fat client** model has been popularized by application development tools such as Sybase Inc.'s PowerBuilder and Gupta Corp.'s SQL Windows. Applications developed with these tools are typically departmental in scope. The main benefit of the fat client model is that it takes advantage of desktop power, offloading application processing from servers and making them more efficient and less likely to be bottlenecks.

There are, however, several disadvantages to the fat client strategy. The addition of more functions rapidly overloads the capacity of desktop machines, forcing companies to upgrade. If the model extends beyond the department to incorporate many users, the company must install high-capacity LANs to support the large volumes of transmission between the thin servers and the fat clients. Finally, it is difficult to maintain, upgrade, or replace applications distributed across tens or hundreds of desktops.

Figure 18.5b is representative of a **thin client** approach. This approach more nearly mimics the traditional host-centered approach and is often the migration path for evolving corporate-wide applications from the mainframe to a distributed environment.

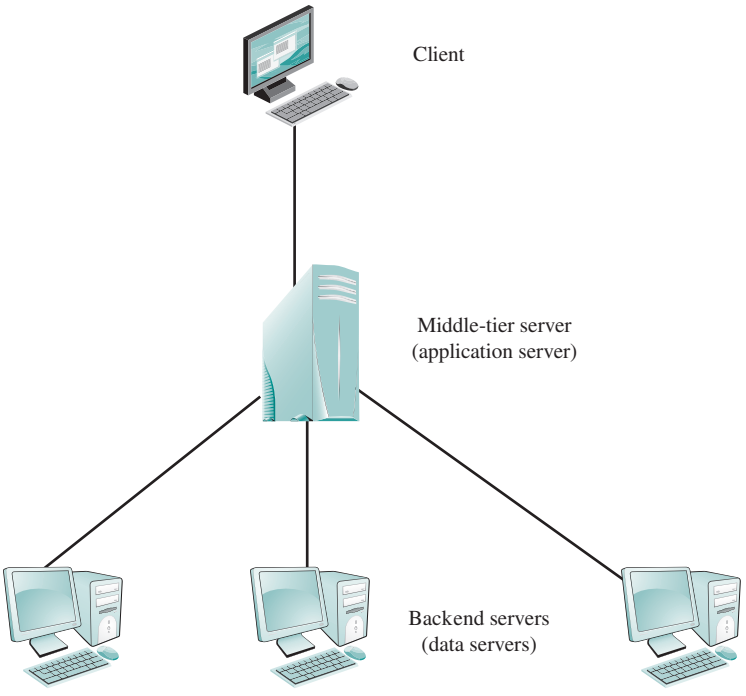



**Figure 18.6 Three-Tier Client/Server Architecture**

***THREE-TIER CLIENT/SERVER ARCHITECTURE*** The traditional client/server architecture involves two levels, or tiers: a client tier and a server tier. A three-tier architecture is also common (see Figure 18.6). In this architecture, the application software is distributed among three types of machines: a user machine, a middle-tier server, and a backend server. The user machine is the client machine we have been discussing and, in the three-tier model, is typically a thin client. The middle-tier machines are essentially gateways between the thin user clients and a variety of backend database servers. The middle-tier machines can convert protocols and map from one type of database query to another. In addition, the middle-tier machine can merge/integrate results from different data sources. Finally, the middle-tier machine can serve as a gateway between the desktop applications and the backend legacy applications by mediating between the two worlds.

The interaction between the middle-tier server and the backend server also follows the client/server model. Thus, the middle-tier system acts as both a client and a server.

***FILE CACHE CONSISTENCY*** When a file server is used, performance of file I/O can be noticeably degraded relative to local file access because of the delays imposed by the network. To reduce this performance penalty, individual systems can use file caches to hold recently accessed file records. Because of the principle of locality, use of a local file cache should reduce the number of remote server accesses that must be made.

Figure 18.7 illustrates a typical distributed mechanism for caching files among a networked collection of workstations. When a process makes a file access, the request is presented first to the cache of the process's workstation ("file traffic"). If not satisfied there, the request is passed either to the local disk, if the file is stored there ("disk traffic"), or to a file server, where the file is stored ("server traffic"). At the server, the server's cache is first interrogated and, if there is a miss, then the server's disk is accessed. The dual caching approach is used to reduce communications traffic (client cache) and disk I/O (server cache).

When caches always contain exact copies of remote data, we say the caches are **consistent**. It is possible for caches to become inconsistent when the remote data are changed and the corresponding obsolete local cache copies are not discarded. This can happen if one client modifies a file that is also cached by other clients. The difficulty is actually at two levels. If a client adopts a policy of immediately writing any changes to a file back to the server, then any other client that has a cache copy of the relevant portion of the file will have obsolete data. The problem is made even worse if the client delays writing back changes to the server. In that case, the server itself has an obsolete version of the file, and new file read requests to the server might obtain obsolete data. The problem of keeping local cache copies up to date to changes in remote data is known as the **cache consistency** problem.

The simplest approach to cache consistency is to use file-locking techniques to prevent simultaneous access to a file by more than one client. This guarantees consistency at the expense of performance and flexibility. A more powerful approach is provided with the facility in Sprite [NELS88, OUST88]. Any number of remote processes may open a file for read and create their own client cache. But when an open file request to a server requests write access and other processes have the file open for read access, the server takes two actions. First, it notifies the writing process that, although it may maintain a cache, it must write back all altered blocks immediately

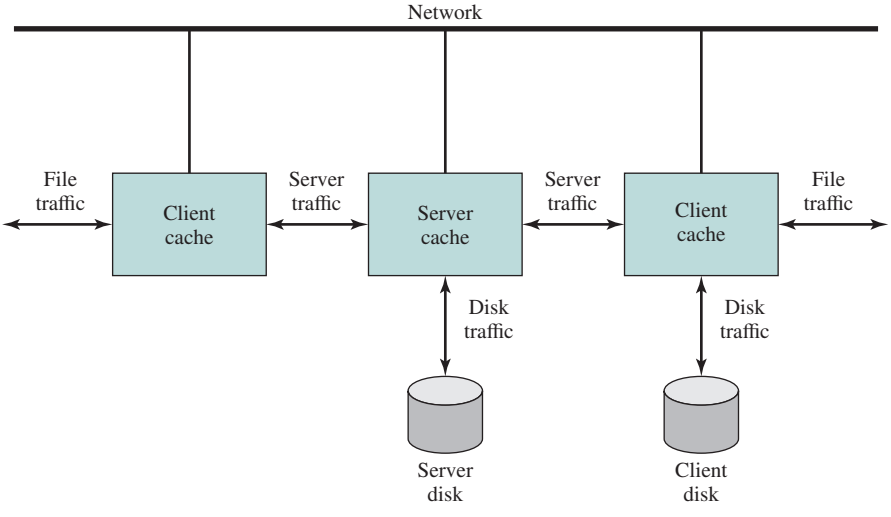



**Figure 18.7 Distributed File Caching in Sprite**

upon update. There can be at most one such client. Second, the server notifies all reading processes that have the file open that the file is no longer cacheable.

## Middleware

The development and deployment of client/server products has far outstripped efforts to standardize all aspects of distributed computing, from the physical layer up to the application layer. This lack of standards makes it difficult to implement an integrated, multivendor, enterprise-wide client/server configuration. Because much of the benefit of the client/server approach is tied up with its modularity and the ability to mix and match platforms and applications to provide a business solution, this interoperability problem must be solved.

To achieve the true benefits of the client/server approach, developers must have a set of tools that provide a uniform means and style of access to system resources across all platforms. This will enable programmers to build applications that not only look and feel the same on various PCs and workstations, but that use the same method to access data regardless of the location of that data.

The most common way to meet this requirement is by the use of standard programming interfaces and protocols that sit between the application above and communications software and operating system below. Such standardized interfaces and protocols have come to be referred to as middleware. With standard programming interfaces, it is easy to implement the same application on a variety of server types and workstation types. This obviously benefits the customer, but vendors are also motivated to provide such interfaces. The reason is that customers buy applications, not servers; customers will only choose among those server products that run the applications they want. The standardized protocols are needed to link these various server interfaces back to the clients that need access to them.

There is a variety of middleware packages ranging from the very simple to the very complex. What they all have in common is the capability to hide the complexities and disparities of different network protocols and operating systems. Client and server vendors generally provide a number of the more popular middleware packages as options. Thus, a user can settle on a particular middleware strategy then assemble equipment from various vendors that support that strategy.

***MIDDLEWARE ARCHITECTURE*** Figure 18.8 suggests the role of middleware in a client/server architecture. The exact role of the middleware component will depend on the style of client/server computing being used. Referring back to Figure 18.5, recall that there are a number of different client/server approaches, depending on the way in which application functions are split up. In any case, Figure 18.8 gives a good general idea of the architecture involved.

Note there is both a client and server component of middleware. The basic purpose of middleware is to enable an application or a user at a client to access a variety of services on servers without being concerned about differences among servers. To look at one specific application area, the structured query language (SQL) is supposed to provide a standardized means for access to a relational database by either a local or remote user or application. However, many relational database vendors, although they support SQL, have added their own proprietary extensions to

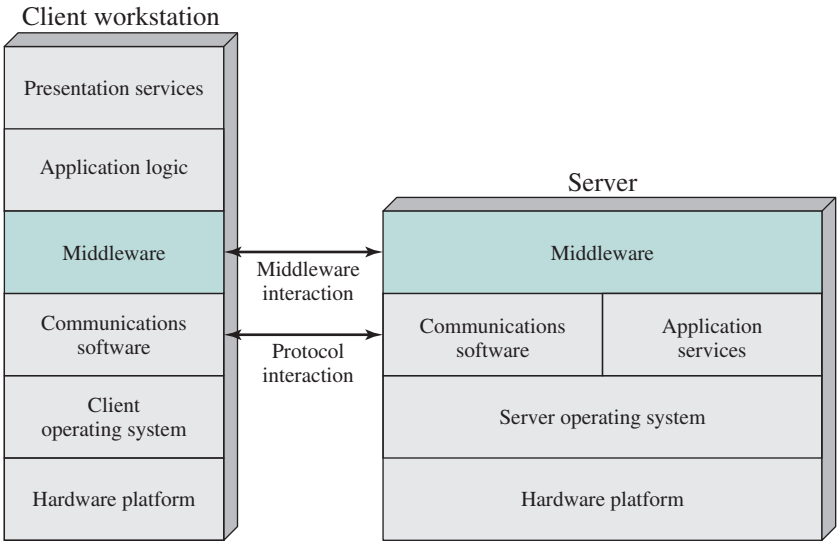



**Figure 18.8 The Role of Middleware in Client/Server Architecture**

SQL. This enables vendors to differentiate their products but also creates potential incompatibilities.

As an example, consider a distributed system used to support, among other things, the personnel department. The basic employee data, such as employee name and address, might be stored on a Gupta database, whereas salary information might be contained on an Oracle database. When a user in the personnel department requires access to particular records, that user does not want to be concerned with which vendor's database contains the records needed. Middleware provides a layer of software that enables uniform access to these differing systems.

It is instructive to look at the role of middleware from a logical, rather than an implementation, point of view. This viewpoint is illustrated in Figure 18.9. Middleware enables the realization of the promise of distributed client/server computing. The entire distributed system can be viewed as a set of applications and resources available to users. Users need not be concerned with the location of data or indeed the location of applications. All applications operate over a uniform applications programming interface (API). The middleware, which cuts across all client and server platforms, is responsible for routing client requests to the appropriate server.

Although there is a wide variety of middleware products, these products are typically based on one of two underlying mechanisms: message passing or remote procedure calls. These two methods are examined in the next two sections.

## 18.2 DISTRIBUTED MESSAGE PASSING

It is usually the case in a distributed processing systems that the computers do not share main memory; each is an isolated computer system. Thus, interprocessor communication techniques that rely on shared memory, such as semaphores, cannot be

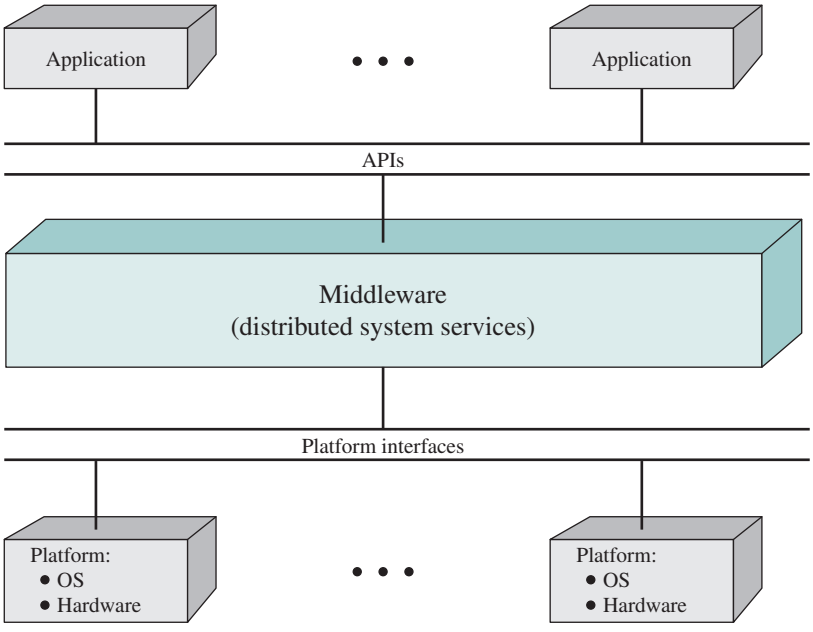



**Figure 18.9** **Logical View of Middleware**

used. Instead, techniques that rely on message passing are used. In this section and the next, we look at the two most common approaches. The first is the straightforward application of messages as they are used in a single system. The second is a separate technique that relies on message passing as a basic function: the remote procedure call.

Figure 18.10a shows the use of message passing to implement client/server functionality. A client process requires some service (e.g., read a file, print) and sends a message containing a request for service to a server process. The server process honors the request and sends a message containing a reply. In its simplest form, only two functions are needed: Send and Receive. The Send function specifies a destination and includes the message content. The Receive function tells from whom a message is desired (including "all") and provides a buffer where the incoming message is to be stored.

Figure 18.11 suggests an implementation for message passing. Processes make use of the services of a message-passing module. Service requests can be expressed in terms of primitives and parameters. A primitive specifies the function to be performed, and the parameters are used to pass data and control information. The actual form of a primitive depends on the message-passing software. It may be a procedure call, or it may itself be a message to a process that is part of the operating system.

The Send primitive is used by the process that desires to send the message. Its parameters are the identifier of the destination process and the contents of the message. The message-passing module constructs a data unit that includes these two elements. This data unit is sent to the machine that hosts the destination process, using some sort of communications facility, such as TCP/IP. When the data unit is received in the target system, it is routed by the communications facility to the message-passing module. This module examines the process ID field and stores the message in the buffer for that process.

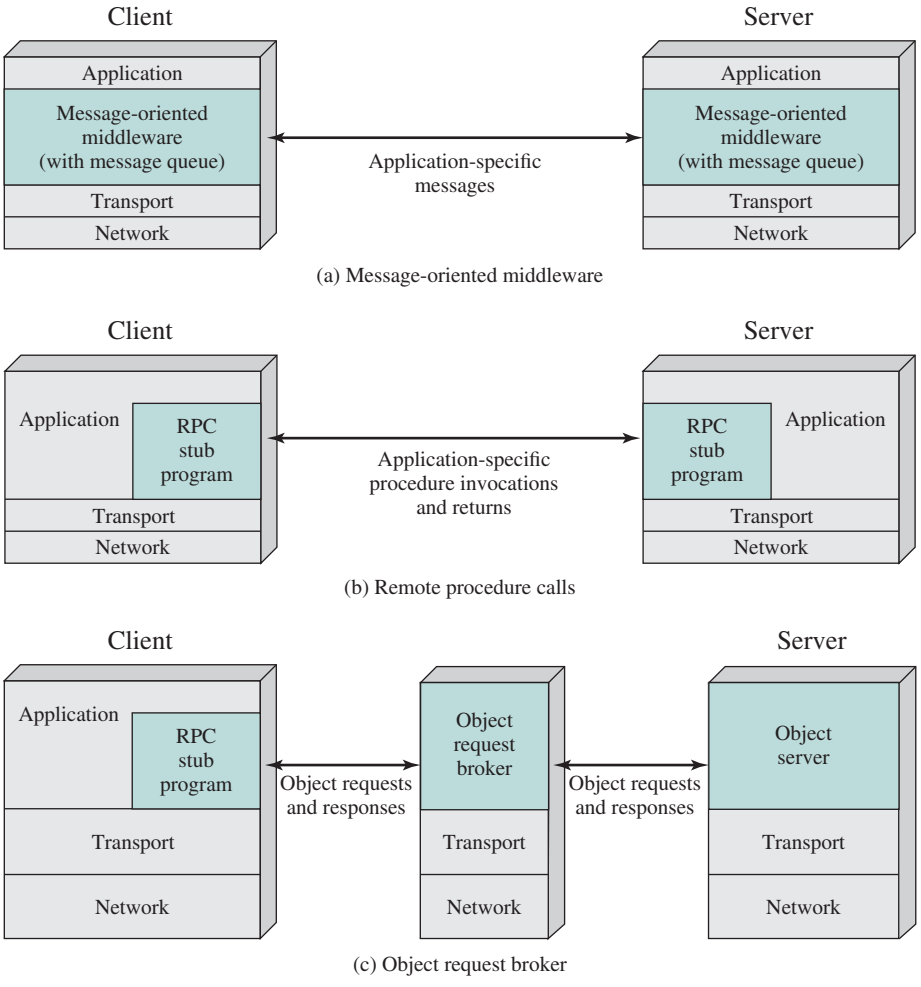



**Figure 18.10** **Middleware Mechanisms**

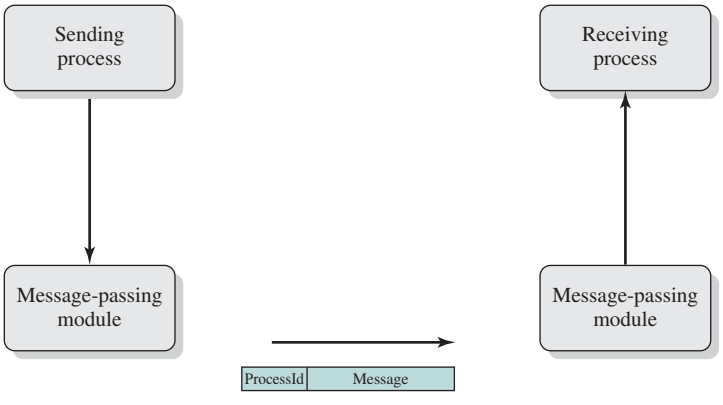



**Figure 18.11** **Basic Message-Passing Primitives**

In this scenario, the receiving process must announce its willingness to receive messages by designating a buffer area and informing the message-passing module by a Receive primitive. An alternative approach does not require such an announcement. Instead, when the message-passing module receives a message, it signals the destination process with some sort of Receive signal then makes the received message available in a shared buffer.

Several design issues are associated with distributed message passing, and these are addressed in the remainder of this section.

## Reliability versus Unreliability

A reliable message-passing facility is one that guarantees delivery if possible. Such a facility makes use of a reliable transport protocol or similar logic and performs error checking, acknowledgment, retransmission, and reordering of misordered messages. Because delivery is guaranteed, it is not necessary to let the sending process know the message was delivered. However, it might be useful to provide an acknowledgment back to the sending process so it knows that delivery has already taken place. In either case, if the facility fails to achieve delivery (e.g., persistent network failure, crash of destination system), the sending process is notified of the failure.

At the other extreme, the message-passing facility may simply send the message out into the communications network but will report neither success nor failure. This alternative greatly reduces the complexity and processing and communications overhead of the message-passing facility. For those applications that require confirmation that a message has been delivered, the applications themselves may use request and reply messages to satisfy the requirement.

## Blocking versus Nonblocking

With nonblocking, or asynchronous, primitives, a process is not suspended as a result of issuing a Send or Receive. Thus, when a process issues a Send primitive, the operating system returns control to the process as soon as the message has been queued for transmission or a copy has been made. If no copy is made, any changes made to the message by the sending process before or even while it is being transmitted are made at the risk of the process. When the message has been transmitted or copied to a safe place for subsequent transmission, the sending process is interrupted to be informed that the message buffer may be reused. Similarly, a nonblocking Receive is issued by a process that then proceeds to run. When a message arrives, the process is informed by interrupt, or it can poll for status periodically.

Nonblocking primitives provide for efficient, flexible use of the message-passing facility by processes. The disadvantage of this approach is that it is difficult to test and debug programs that use these primitives. Irreproducible, timing-dependent sequences can create subtle and difficult problems.

The alternative is to use blocking, or synchronous, primitives. A blocking Send does not return control to the sending process until the message has been transmitted (unreliable service) or until the message has been sent and an acknowledgment received (reliable service). A blocking Receive does not return control until a message has been placed in the allocated buffer.

## 18.3 REMOTE PROCEDURE CALLS

A variation on the basic message-passing model is the remote procedure call. This is now a widely accepted and common method for encapsulating communication in a distributed system. The essence of the technique is to allow programs on different machines to interact using simple procedure call/return semantics, just as if the two programs were on the same machine. That is, the procedure call is used for access to remote services. The popularity of this approach is due to the following advantages.

1. The procedure call is a widely accepted, used, and understood abstraction.
2. The use of remote procedure calls enables remote interfaces to be specified as a set of named operations with designated types. Thus, the interface can be clearly documented, and distributed programs can be statically checked for type errors.
3. Because a standardized and precisely defined interface is specified, the communication code for an application can be generated automatically.
4. Because a standardized and precisely defined interface is specified, developers can write client and server modules that can be moved among computers and operating systems with little modification and recoding.

The remote procedure call mechanism can be viewed as a refinement of reliable, blocking message passing. Figure 18.10b illustrates the general architecture, and Figure 18.12 provides a more detailed look. The calling program makes a normal procedure call with parameters on its machine. For example,

CALL P(X, Y)

where

P = procedure name
X = passed arguments
Y = returned values

It may or may not be transparent to the user that the intention is to invoke a remote procedure on some other machine. A dummy or stub procedure P must be included in the caller's address space or be dynamically linked to it at call time. This procedure creates a message that identifies the procedure being called and includes the parameters. It then sends this message to a remote system and waits for a reply. When a reply is received, the stub procedure returns to the calling program, providing the returned values.

At the remote machine, another stub program is associated with the called procedure. When a message comes in, it is examined and a local CALL P(X, Y) is generated. This remote procedure is thus called locally, so its normal assumptions about where to find parameters, the state of the stack, and so on are identical to the case of a purely local procedure call.

Several design issues are associated with remote procedure calls, and these are addressed in the remainder of this section.

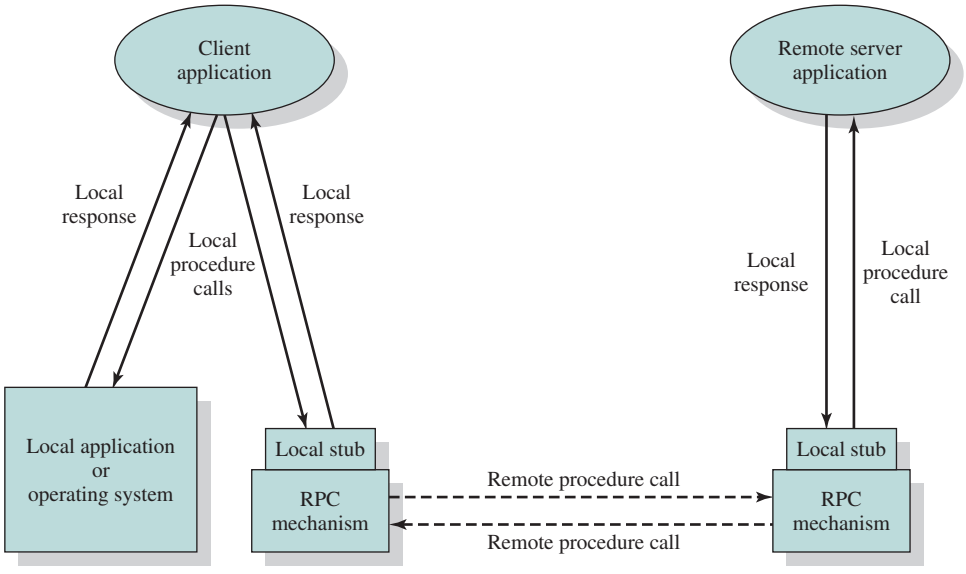



**Figure 18.12** **Remote Procedure Call Mechanism**

## Parameter Passing

Most programming languages allow parameters to be passed as values (call by value) or as pointers to a location that contains the value (call by reference). Call by value is simple for a remote procedure call: The parameters are simply copied into the message and sent to the remote system. It is more difficult to implement call by reference. A unique, system-wide pointer is needed for each object. The overhead for this capability may not be worth the effort.

## Parameter Representation

Another issue is how to represent parameters and results in messages. If the called and calling programs are in identical programming languages on the same type of machines with the same operating system, then the representation requirement may present no problems. If there are differences in these areas, then there will probably be differences in the ways in which numbers and even text are represented. If a full-blown communications architecture is used, then this issue is handled by the presentation layer. However, the overhead of such an architecture has led to the design of remote procedure call facilities that bypass most of the communications architecture and provide their own basic communications facility. In that case, the conversion responsibility falls on the remote procedure call facility (e.g., see [GIBB87]).

The best approach to this problem is to provide a standardized format for common objects, such as integers, floating-point numbers, characters, and character strings. Then the native parameters on any machine can be converted to and from the standardized representation.

## Client/Server Binding

Binding specifies how the relationship between a remote procedure and the calling program will be established. A binding is formed when two applications have made a logical connection and are prepared to exchange commands and data.

**Nonpersistent binding** means that a logical connection is established between the two processes at the time of the remote procedure call, and that as soon as the values are returned, the connection is dismantled. Because a connection requires the maintenance of state information on both ends, it consumes resources. The nonpersistent style is used to conserve those resources. On the other hand, the overhead involved in establishing connections makes nonpersistent binding inappropriate for remote procedures that are called frequently by the same caller.

With **persistent binding**, a connection that is set up for a remote procedure call is sustained after the procedure return. The connection can then be used for future remote procedure calls. If a specified period of time passes with no activity on the connection, then the connection is terminated. For applications that make many repeated calls to remote procedures, persistent binding maintains the logical connection and allows a sequence of calls and returns to use the same connection.

## Synchronous versus Asynchronous

The concepts of synchronous and asynchronous remote procedure calls are analogous to the concepts of blocking and nonblocking messages. The traditional remote procedure call is synchronous, which requires that the calling process wait until the called process returns a value. Thus, the **synchronous RPC** behaves much like a subroutine call.

The synchronous RPC is easy to understand and program because its behavior is predictable. However, it fails to exploit fully the parallelism inherent in distributed applications. This limits the kind of interaction the distributed application can have, resulting in lower performance.

To provide greater flexibility, various **asynchronous RPC** facilities have been implemented to achieve a greater degree of parallelism while retaining the familiarity and simplicity of the RPC [ANAN92]. Asynchronous RPCs do not block the caller; the replies can be received as and when they are needed, thus allowing client execution to proceed locally in parallel with the server invocation.

A typical asynchronous RPC use is to enable a client to invoke a server repeatedly so the client has a number of requests in the pipeline at one time, each with its own set of data. Synchronization of client and server can be achieved in one of two ways:

1. A higher-layer application in the client and server can initiate the exchange then check at the end that all requested actions have been performed.
2. A client can issue a string of asynchronous RPCs followed by a final synchronous RPC. The server will respond to the synchronous RPC only after completing all of the work requested in the preceding asynchronous RPCs.

In some schemes, asynchronous RPCs require no reply from the server and the server cannot send a reply message. Other schemes either require or allow a reply, but the caller does not wait for the reply.

### Object-Oriented Mechanisms

As object-oriented technology becomes more prevalent in operating system design, client/server designers have begun to embrace this approach. In this approach, clients and servers ship messages back and forth between objects. Object communications may rely on an underlying message or RPC structure or be developed directly on top of object-oriented capabilities in the operating system.

A client that needs a service sends a request to an object request broker, which acts as a directory of all the remote service available on the network (see Figure 18.10c). The broker calls the appropriate object and passes along any relevant data. Then the remote object services the request and replies to the broker, which returns the response to the client.

The success of the object-oriented approach depends on standardization of the object mechanism. Unfortunately, there are several competing designs in this area. One is Microsoft's Component Object Model (COM), the basis for Object Linking and Embedding (OLE). A competing approach, developed by the Object Management Group, is the Common Object Request Broker Architecture (CORBA), which has wide industry support. IBM, Apple, Sun, and many other vendors support the CORBA approach.

## 18.4 CLUSTERS

Clustering is an alternative to symmetric multiprocessing (SMP) as an approach to providing high performance and high availability and is particularly attractive for server applications. We can define a cluster as a group of interconnected, whole computers working together as a unified computing resource that can create the illusion of being one machine. The term *whole computer* means a system that can run on its own, apart from the cluster; in the literature, each computer in a cluster is typically referred to as a *node*.

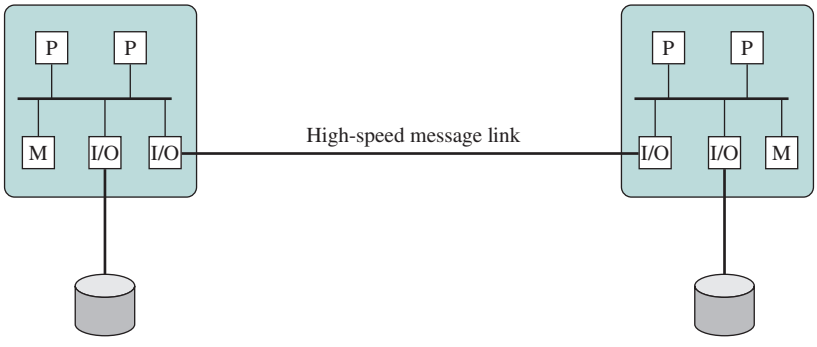
[BREW97] lists four benefits that can be achieved with clustering. These can also be thought of as objectives or design requirements:

- **Absolute scalability:** It is possible to create large clusters that far surpass the power of even the largest stand-alone machines. A cluster can have dozens or even hundreds of machines, each of which is a multiprocessor.
- **Incremental scalability:** A cluster is configured in such a way that it is possible to add new systems to the cluster in small increments. Thus, a user can start out with a modest system and expand it as needs grow, without having to go through a major upgrade in which an existing small system is replaced with a larger system.
- **High availability:** Because each node in a cluster is a stand-alone computer, the failure of one node does not mean loss of service. In many products, fault tolerance is handled automatically in software.
- **Superior price/performance:** By using commodity building blocks, it is possible to put together a cluster with equal or greater computing power than a single large machine, at much lower cost.

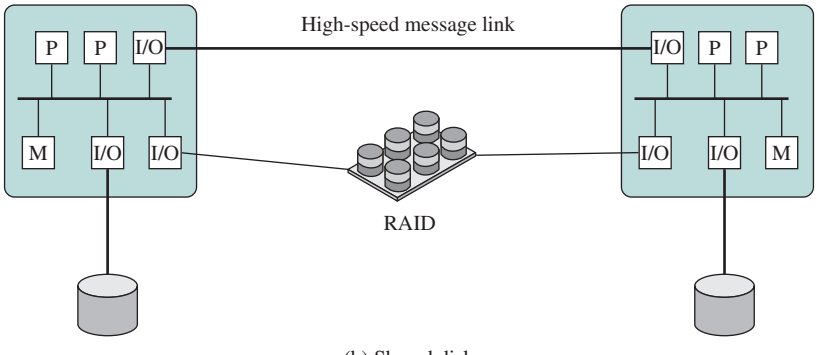
## Cluster Configurations

In the literature, clusters are classified in a number of different ways. Perhaps the simplest classification is based on whether the computers in a cluster share access to the same disks. Figure 18.13a shows a two-node cluster in which the only interconnection is by means of a high-speed link that can be used for message exchange to coordinate cluster activity. The link can be a LAN that is shared with other computers that are not part of the cluster, or the link can be a dedicated interconnection facility. In the latter case, one or more of the computers in the cluster will have a link to a LAN or WAN so there is a connection between the server cluster and remote client systems. Note in the figure, each computer is depicted as being a multiprocessor. This is not necessary but does enhance both performance and availability.

In the simple classification depicted in Figure 18.13, the other alternative is a shared disk cluster. In this case, there generally is still a message link between nodes. In addition, there is a disk subsystem that is directly linked to multiple computers within the cluster. In Figure 18.13b, the common disk subsystem is a RAID system. The use of RAID or some similar redundant disk technology is common in clusters so the high availability achieved by the presence of multiple computers is not compromised by a shared disk that is a single point of failure.




(a) Standby server with no shared disk




(b) Shared disk

**Figure 18.13 Cluster Configurations**

**Table 18.2** Clustering Methods: Benefits and Limitations

| Clustering Method | Description | Benefits | Limitations |
|---|---|---|---|
| **Passive Standby** | A secondary server takes over in case of primary server failure. | Easy to implement. | High cost because the secondary server is unavailable for other processing tasks. |
| **Active Secondary** | The secondary server is also used for processing tasks. | Reduced cost because secondary servers can be used for processing. | Increased complexity. |
| **Separate Servers** | Separate servers have their own disks. Data are continuously copied from primary to secondary server. | High availability. | High network and server overhead due to copying operations. |
| **Servers Connected to Disks** | Servers are cabled to the same disks, but each server owns its disks. If one server fails, its disks are taken over by the other server. | Reduced network and server overhead due to elimination of copying operations. | Usually requires disk mirroring or RAID technology to compensate for risk of disk failure. |
| **Servers Share Disks** | Multiple servers simultaneously share access to disks. | Low network and server overhead. Reduced risk of downtime caused by disk failure. | Requires lock manager software. Usually used with disk mirroring or RAID technology. |

A clearer picture of the range of clustering approaches can be gained by looking at functional alternatives. A white paper from Hewlett Packard [HP96] provides a useful classification along functional lines (see Table 18.2), which we now discuss.

A common, older method, known as **passive standby**, is simply to have one computer handle all of the processing load while the other computer remains inactive, standing by to take over in the event of a failure of the primary. To coordinate the machines, the active, or primary, system periodically sends a "heartbeat" message to the standby machine. Should these messages stop arriving, the standby assumes that the primary server has failed and puts itself into operation. This approach increases availability but does not improve performance. Further, if the only information that is exchanged between the two systems is a heartbeat message, and if the two systems do not share common disks, then the standby provides a functional backup but has no access to the databases managed by the primary.

The passive standby is generally not referred to as a cluster. The term cluster is reserved for multiple interconnected computers that are all actively doing processing while maintaining the image of a single system to the outside world. The term **active secondary** is often used in referring to this configuration. Three classifications of clustering can be identified: separate servers, shared nothing, and shared memory.

In one approach to clustering, each computer is a **separate server** with its own disks and there are no disks shared between systems (see Figure 18.13a). This arrangement provides high performance as well as high availability. In this case, some type of management or scheduling software is needed to assign incoming client requests to servers so the load is balanced and high utilization is achieved. It is desirable to

have a failover capability, which means that if a computer fails while executing an application, another computer in the cluster can pick up and complete the application. For this to happen, data must constantly be copied among systems so each system has access to the current data of the other systems. The overhead of this data exchange ensures high availability at the cost of a performance penalty.

To reduce the communications overhead, most clusters now consist of servers connected to common disks (see Figure 18.13b). In one variation of this approach, called **shared nothing**, the common disks are partitioned into volumes, and each volume is owned by a single computer. If that computer fails, the cluster must be reconfigured so some other computer has ownership of the volumes of the failed computer.

It is also possible to have multiple computers share the same disks at the same time (called the **shared disk** approach), so each computer has access to all of the volumes on all of the disks. This approach requires the use of some type of locking facility to ensure data can only be accessed by one computer at a time.

## Operating System Design Issues

Full exploitation of a cluster hardware configuration requires some enhancements to a single-system operating system.

***Failure Management*** How failures are managed by a cluster depends on the clustering method used (see Table 18.2). In general, two approaches can be taken to dealing with failures: highly available clusters and fault-tolerant clusters. A highly available cluster offers a high probability that all resources will be in service. If a failure occurs, such as a node goes down or a disk volume is lost, then the queries in progress are lost. Any lost query, if retried, will be serviced by a different computer in the cluster. However, the cluster operating system makes no guarantee about the state of partially executed transactions. This would need to be handled at the application level.

A fault-tolerant cluster ensures all resources are always available. This is achieved by the use of redundant shared disks and mechanisms for backing out uncommitted transactions and committing completed transactions.

The function of switching an application and data resources over from a failed system to an alternative system in the cluster is referred to as **failover**. A related function is the restoration of applications and data resources to the original system once it has been fixed; this is referred to as **failback**. Failback can be automated, but this is desirable only if the problem is truly fixed and unlikely to recur. If not, automatic failback can cause subsequently failed resources to bounce back and forth between computers, resulting in performance and recovery problems.

***Load Balancing*** A cluster requires an effective capability for balancing the load among available computers. This includes the requirement that the cluster be incrementally scalable. When a new computer is added to the cluster, the load-balancing facility should automatically include this computer in scheduling applications. Middleware mechanisms need to recognize that services can appear on different members of the cluster and may migrate from one member to another.

***PARALLELIZING COMPUTATION*** In some cases, effective use of a cluster requires executing software from a single application in parallel. [KAPP00] lists three general approaches to the problem:

- **Parallelizing compiler:** A parallelizing compiler determines, at compile time, which parts of an application can be executed in parallel. These are then split off to be assigned to different computers in the cluster. Performance depends on the nature of the problem and how well the compiler is designed.
- **Parallelized application:** In this approach, the programmer writes the application from the outset to run on a cluster and uses message passing to move data, as required, between cluster nodes. This places a high burden on the programmer but may be the best approach for exploiting clusters for some applications.
- **Parametric computing:** This approach can be used if the essence of the application is an algorithm or program that must be executed a large number of times, each time with a different set of starting conditions or parameters. A good example is a simulation model, which will run a large number of different scenarios, then develop statistical summaries of the results. For this approach to be effective, parametric processing tools are needed to organize, run, and manage the jobs in an orderly manner.

## Cluster Computer Architecture

Figure 18.14 shows a typical cluster architecture. The individual computers are connected by some high-speed LAN or switch hardware. Each computer is capable of operating independently. In addition, a middleware layer of software is installed in each computer to enable cluster operation. The cluster middleware provides a unified system image to the user, known as a **single-system image**. The middleware may also be responsible for providing high availability, by means of load balancing and responding to failures in individual components. [HWAN99] lists the following as desirable cluster middleware services and functions:

- **Single entry point:** A user logs on to the cluster rather than to an individual computer
- **Single file hierarchy:** The user sees a single hierarchy of file directories under the same root directory.
- **Single control point:** There is a default node used for cluster management and control.
- **Single virtual networking:** Any node can access any other point in the cluster, even though the actual cluster configuration may consist of multiple interconnected networks. There is a single virtual network operation.
- **Single memory space:** Distributed shared memory enables programs to share variables.
- **Single job-management system:** Under a cluster job scheduler, a user can submit a job without specifying the host computer to execute the job.

- **Single-user interface:** A common graphic interface supports all users, regardless of the workstation from which they enter the cluster.
- **Single I/O space:** Any node can remotely access any I/O peripheral or disk device without knowledge of its physical location.
- **Single process space:** A uniform process-identification scheme is used. A process on any node can create or communicate with any other process on a remote node.
- **Checkpointing:** This function periodically saves the process state and intermediate computing results, to allow rollback recovery after a failure.
- **Process migration:** This function enables load balancing.

The last four items on the preceding list enhance the availability of the cluster. The remaining items are concerned with providing a single-system image.

Returning to Figure 18.14, a cluster will also include software tools for enabling the efficient execution of programs that are capable of parallel execution.

## Clusters Compared to SMP

Both clusters and symmetric multiprocessors provide a configuration with multiple processors to support high-demand applications. Both solutions are commercially available, although SMP has been around far longer.

The main strength of the SMP approach is that an SMP is easier to manage and configure than a cluster. The SMP is much closer to the original single-processor model for which nearly all applications are written. The principal change required in going from a uniprocessor to an SMP is to the scheduler function. Another benefit of the SMP is that it usually takes up less physical space and draws less power than

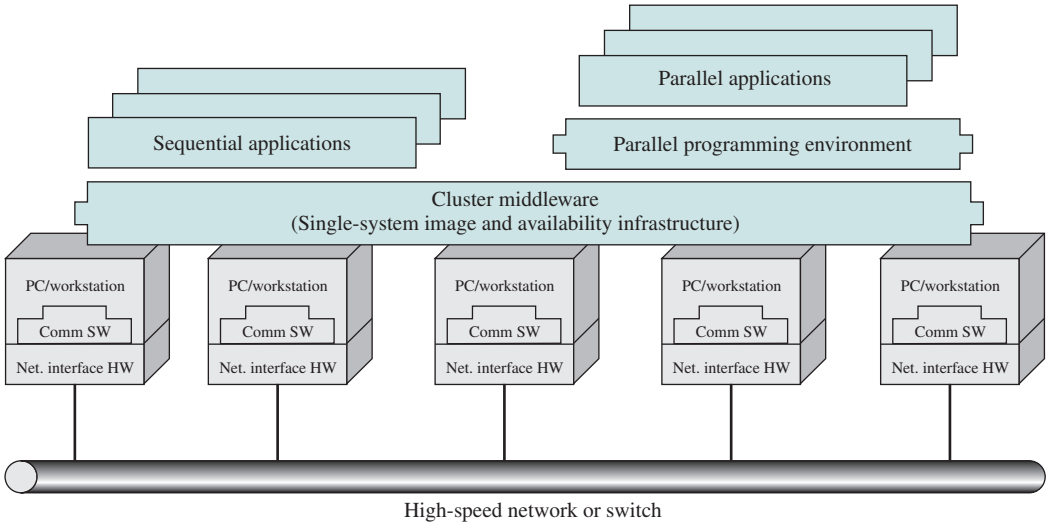



**Figure 18.14 Cluster Computer Architecture**

a comparable cluster. A final important benefit is that the SMP products are well established and stable.

Over the long run, however, the advantages of the cluster approach are likely to result in clusters dominating the high-performance server market. Clusters are far superior to SMPs in terms of incremental and absolute scalability. Clusters are also superior in terms of availability, because all components of the system can readily be made highly redundant.

## 18.5 WINDOWS CLUSTER SERVER

Windows Failover Clustering is a shared-nothing cluster, in which each disk volume and other resources are owned by a single system at a time.

The Windows cluster design makes use of the following concepts:

- **Cluster Service:** The collection of software on each node that manages all cluster-specific activity.
- **Resource:** An item managed by the cluster service. All resources are objects representing actual resources in the system, including hardware devices such as disk drives and network cards and logical items such as logical disk volumes, TCP/IP addresses, entire applications, and databases.
- **Online:** A resource is said to be online at a node when it is providing service on that specific node.
- **Group:** A collection of resources managed as a single unit. Usually, a group contains all of the elements needed to run a specific application, and for client systems to connect to the service provided by that application.

The concept of *group* is of particular importance. A group combines resources into larger units that are easily managed, both for failover and load balancing. Operations performed on a group, such as transferring the group to another node, automatically affect all of the resources in that group. Resources are implemented as dynamically linked libraries (DLLs) and managed by a resource monitor. The resource monitor interacts with the cluster service via remote procedure calls and responds to cluster service commands to configure and move resource groups.

Figure 18.15 depicts the Windows clustering components and their relationships in a single system of a cluster. The **node manager** is responsible for maintaining this node's membership in the cluster. Periodically, it sends heartbeat messages to the node managers on other nodes in the cluster. In the event that one node manager detects a loss of heartbeat messages from another cluster node, it broadcasts a message to the entire cluster, causing all members to exchange messages to verify their view of current cluster membership. If a node manager does not respond, it is removed from the cluster and its active groups are transferred to one or more other active nodes in the cluster.

The **configuration database manager** maintains the cluster configuration database. The database contains information about resources and groups and node

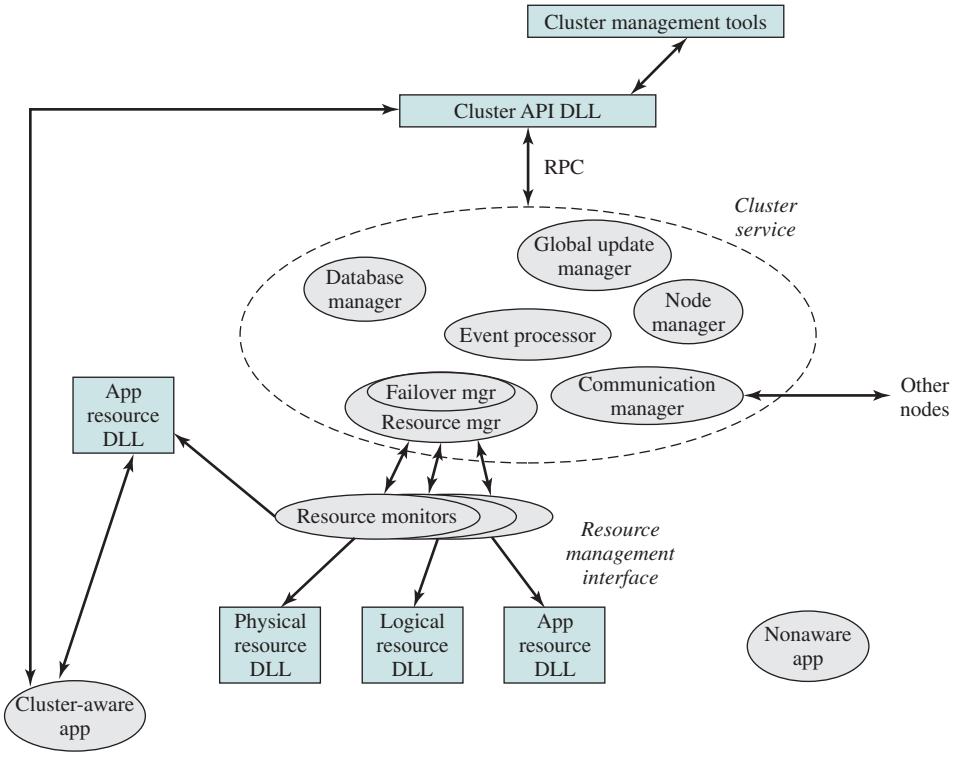



**Figure 18.15 Windows Cluster Server Block Diagram**

ownership of groups. The database managers on each of the cluster nodes cooperate to maintain a consistent picture of configuration information. Fault-tolerant transaction software is used to assure that changes in the overall cluster configuration are performed consistently and correctly.

The **resource manager/failover manager** makes all decisions regarding resource groups and initiates appropriate actions such as startup, reset, and failover. When failover is required, the failover managers on the active node cooperate to negotiate a distribution of resource groups from the failed system to the remaining active systems. When a system restarts after a failure, the failover manager can decide to move some groups back to this system. In particular, any group may be configured with a preferred owner. If that owner fails and then restarts, the group is moved back to the node in a rollback operation.

The **event processor** connects all of the components of the cluster service, handles common operations, and controls cluster service initialization. The communications manager manages message exchange with all other nodes of the cluster. The global update manager provides a service used by other components within the cluster service.

Microsoft is continuing to ship their cluster product, but they have also developed virtualization solutions based on efficient live migration of virtual

machines between hypervisors running on different computer systems as part of Windows Server 2008 R2. For new applications, live migration offers many benefits over the cluster approach, such as simpler management, and improved flexibility.

## 18.6 BEOWULF AND LINUX CLUSTERS

In 1994, the Beowulf project was initiated under the sponsorship of the NASA High Performance Computing and Communications (HPCC) project. Its goal was to investigate the potential of clustered PCs for performing important computation tasks beyond the capabilities of contemporary workstations at minimum cost. Today, the Beowulf approach is widely implemented and is perhaps the most important cluster technology available.

### Beowulf Features

Key features of Beowulf include the following [RIDG97]:

- Mass market commodity components
- Dedicated processors (rather than scavenging cycles from idle workstations)
- A dedicated, private network (LAN or WAN or internetted combination)
- No custom components
- Easy replication from multiple vendors
- Scalable I/O
- A freely available software base
- Use of freely available distribution computing tools with minimal changes
- Return of the design and improvements to the community

Although elements of Beowulf software have been implemented on a number of different platforms, the most obvious choice for a base is Linux, and most Beowulf implementations use a cluster of Linux workstations and/or PCs. Figure 18.16 depicts a representative configuration. The cluster consists of a number of workstations, perhaps of differing hardware platforms, all running the Linux operating system. Secondary storage at each workstation may be made available for distributed access (for distributed file sharing, distributed virtual memory, or other uses). The cluster nodes (the Linux systems) are interconnected with a commodity networking approach, typically Ethernet. The Ethernet support may be in the form of a single Ethernet switch or an interconnected set of switches. Commodity Ethernet products at the standard data rates (10 Mbps, 100 Mbps, 1 Gbps) are used.

### Beowulf Software

The Beowulf software environment is implemented as an add-on to commercially available, royalty-free base Linux distributions. The principal source of open-source Beowulf software is the Beowulf site at www.beowulf.org, but numerous other organizations also offer free Beowulf tools and utilities.

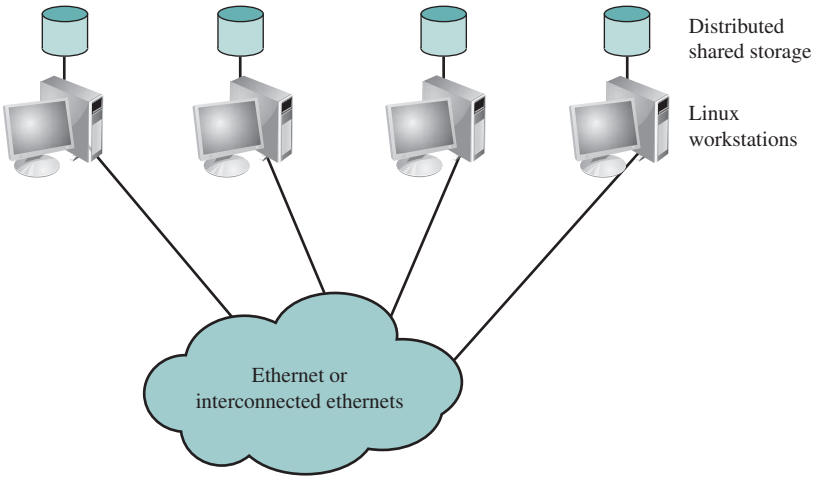



**Figure 18.16 Generic Beowulf Configuration**

Each node in the Beowulf cluster runs its own copy of the Linux kernel and can function as an autonomous Linux system. To support the Beowulf cluster concept, extensions are made to the Linux kernel to allow the individual nodes to participate in a number of global namespaces. The following are examples of Beowulf system software:

- **Beowulf distributed process space (BPROC):** This package allows a process ID space to span multiple nodes in a cluster environment and also provides mechanisms for starting processes on other nodes. The goal of this package is to provide key elements needed for a single-system image on Beowulf cluster. BPROC provides a mechanism to start processes on remote nodes without ever logging into another node, and by making all the remote processes visible in the process table of the cluster's front-end node.
- **Beowulf Ethernet channel bonding:** This is a mechanism that joins multiple low-cost networks into a single logical network with higher bandwidth. The only additional work over using single network interface is the computationally simple task of distributing the packets over the available device transmit queues. This approach allows load balancing over multiple Ethernets connected to Linux workstations.
- **Pvmsync:** This is a programming environment that provides synchronization mechanisms and shared data objects for processes in a Beowulf cluster.
- **EnFuzion:** EnFuzion consists of a set of tools for doing parametric computing. Parametric computing involves the execution of a program as a large number of jobs, each with different parameters or starting conditions. EnFusion emulates a set of robot users on a single root node machine, each of which will log into one of the many clients that form a cluster. Each job is set up to run with a unique, programmed scenario, with an appropriate set of starting conditions [KAPP00].

## 18.7 SUMMARY

Client/server computing is the key to realizing the potential of information systems and networks to improve productivity significantly in organizations. With client/server computing, applications are distributed to users on single-user workstations and personal computers. At the same time, resources that can and should be shared are maintained on server systems that are available to all clients. Thus, the client/server architecture is a blend of decentralized and centralized computing.

Typically, the client system provides a graphical user interface (GUI) that enables a user to exploit a variety of applications with minimal training and relative ease. Servers support shared utilities, such as database management systems. The actual application is divided between client and server in a way intended to optimize ease of use and performance.

The key mechanism required in any distributed system is interprocess communication. Two techniques are in common use. A message-passing facility generalizes the use of messages within a single system. The same sorts of conventions and synchronization rules apply. Another approach is the use of the remote procedure call. This is a technique by which two programs on different machines interact using procedure call/return syntax and semantics. Both the called and calling program behave as if the partner program were running on the same machine.

A cluster is a group of interconnected, whole computers working together as a unified computing resource that can create the illusion of being one machine. The term *whole computer* means a system that can run on its own, apart from the cluster.

## 18.8 REFERENCES

**ANAN92** Ananda, A.; Tay, B.; and Koh, E. "A Survey of Asynchronous Remote Procedure Calls." *Operating Systems Review*, April 1992.

**BREW97** Brewer, E. "Clustering: Multiply and Conquer." *Data Communications*, July 1997.

**GIBB87** Gibbons, P. "A Stub Generator for Multilanguage RPC in Heterogeneous Environments." *IEEE Transactions on Software Engineering*, January 1987.

**HP96** Hewlett Packard. *White Paper on Clustering*. June 1996.

**HWAN99** Hwang, K., et al. "Designing SSI Clusters with Hierarchical Checkpointing and Single I/O Space." *IEEE Concurrency*, January–March 1999.

**KAPP00** Kapp, C. "Managing Cluster Computers." *Dr. Dobb's Journal*, July 2000.

**NELS88** Nelson, M.; Welch, B.; and Ousterhout, J. "Caching in the Sprite Network File System." *ACM Transactions on Computer Systems*, February 1988.

**OUST88** Ousterhout, J., et al. "The Sprite Network Operating System." *Computer*, February 1988.

**RIDG97** Ridge, D., et al. "Beowulf: Harnessing the Power of Parallelism in a Pile-of-PCs." *Proceedings, IEEE Aerospace Conference*, 1997.

## 18.9 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

### Key Terms

| | | |
|---|---|---|
| applications programming interface | failback | middleware |
| Beowulf | failover | remote procedure call (RPC) |
| client | fat client | server |
| cluster | file cache consistency | thin client |
| distributed message passing | graphical user interface (GUI) | |
| | message | |

### Review Questions

**18.1.** What is client/server computing?

**18.2.** What distinguishes client/server computing from any other form of distributed data processing?

**18.3.** What is the role of a communications architecture such as TCP/IP in a client/server environment?

**18.4.** Discuss the rationale for locating applications on the client, the server, or split between client and server.

**18.5.** What are fat clients and thin clients, and what are the differences in philosophy of the two approaches?

**18.6.** Suggest pros and cons for fat client and thin client strategies.

**18.7.** Explain the rationale behind the three-tier client/server architecture.

**18.8.** What is middleware?

**18.9.** Because we have standards such as TCP/IP, why is middleware needed?

**18.10.** List some benefits and disadvantages of blocking and nonblocking primitives for message passing.

**18.11.** List some benefits and disadvantages of nonpersistent and persistent binding for RPCs.

**18.12.** List some benefits and disadvantages of synchronous and asynchronous RPCs.

**18.13.** List and briefly define four different clustering methods.

### Problems

**18.1.** Let $\alpha$ be the percentage of program code that can be executed simultaneously by $n$ computers in a cluster, each computer using a different set of parameters or initial conditions. Assume the remaining code must be executed sequentially by a single processor. Each processor has an execution rate of $x$ MIPS.

**a.** Derive an expression for the effective MIPS rate when using the system for exclusive execution of this program, in terms of $n$, $\alpha$, and $x$.

**b.** If $n = 16$ and $x = 4$ MIPS, determine the value of $\alpha$ that will yield a system performance of 40 MIPS.

**18.2.** An application program is executed on a nine-computer cluster. A benchmark program takes time $T$ on this cluster. Further, 25% of $T$ is time in which the application is running simultaneously on all nine computers. The remaining time, the application has to run on a single computer.

**a.** Calculate the effective speedup under the aforementioned condition as compared to executing the program on a single computer. Also calculate, the percentage of code that has been parallelized (programmed or compiled so as to use the cluster mode) in the preceding program.

**b.** Suppose we are able to effectively use 18 computers rather than 9 computers on the parallelized portion of the code. Calculate the effective speedup that is achieved.

**18.3.** The following FORTRAN program is to be executed on a computer, and a parallel version is to be executed on a 32-computer cluster:

```
L1:      DO 10 I = 1,1024
L2:      SUM(I) = 0
L3:      DO 20 J = 1, I
L4: 20 SUM(I) = SUM(I) + I
L5: 10 CONTINUE
```

Suppose lines 2 and 4 each take two machine cycle times, including all processor and memory-access activities. Ignore the overhead caused by the software loop control statements (lines 1, 3, 5) and all other system overhead and resource conflicts.

**a.** What is the total execution time (in machine cycle times) of the program on a single computer?

**b.** Divide the I-loop iterations among the 32 computers as follows: Computer 1 executes the first 32 iterations (I = 1 to 32), processor 2 executes the next 32 iterations, and so on. What are the execution time and speedup factor compared with part (a)? (Note the computational workload, dictated by the J-loop, is unbalanced among the computers.)

**c.** Explain how to modify the parallelizing to facilitate a balanced parallel execution of all the computational workload over 32 computers. A balanced load means an equal number of additions assigned to each computer with respect to both loops.

**d.** What is the minimum execution time resulting from the parallel execution on 32 computers? What is the resulting speedup over a single computer?

CHAPTER 19

# Distributed Process Management


19.1 **Process Migration**
- Motivation
- Process Migration Mechanisms
- Negotiation of Migration
- Eviction
- Preemptive versus Nonpreemptive Transfers

19.2 **Distributed Global States**
- Global States and Distributed Snapshots
- The Distributed Snapshot Algorithm

19.3 **Distributed Mutual Exclusion**
- Distributed Mutual Exclusion Concepts
- Ordering of Events in a Distributed System
- Distributed Queue
- A Token-Passing Approach

19.4 **Distributed Deadlock**
- Deadlock in Resource Allocation
- Deadlock in Message Communication

19.5 **Summary**

19.6 **References**

19.7 **Key Terms, Review Questions, And Problems**

**LEARNING OBJECTIVES**

After studying this chapter, you should be able to:

- Give an explanation of process migration.
- Understand the concept of distributed global states.
- Analyze distributed mutual exclusion algorithms.
- Analyze distributed deadlock algorithms.

This chapter examines key mechanisms used in distributed operating systems. First we look at process migration, which is the movement of an active process from one machine to another. Next, we examine the question of how processes on different systems can coordinate their activities when each is governed by a local clock and when there is a delay in the exchange of information. Finally, we explore two key issues in distributed process management: mutual exclusion and deadlock.

## 19.1 PROCESS MIGRATION

Process migration is the transfer of a sufficient amount of the state of a process from one computer to another for the process to execute on the target machine. Interest in this concept grew out of research into methods of load balancing across multiple networked systems, although the application of the concept now extends beyond that one area.

In the past, only a few of the many papers on load distribution were based on true implementations of process migration, which includes the ability to preempt a process on one machine and reactivate it later on another machine. Experience showed that preemptive process migration is possible, although with higher overhead and complexity than originally anticipated [ARTS89a]. This cost led some observers to conclude that process migration was not practical. Such assessments have proved too pessimistic. New implementations, including those in commercial products, have fueled a continuing interest and new developments in this area. This section provides an overview.

### Motivation

Process migration is desirable in distributed systems for a number of reasons [SMIT88, JUL88], including:

- **Load sharing:** By moving processes from heavily loaded to lightly loaded systems, the load can be balanced to improve overall performance. Empirical data suggest that significant performance improvements are possible [LELA86, CABR86]. However, care must be taken in the design of load-balancing algorithms. [EAGE86] points out that the more communication necessary for the distributed system to perform the balancing, the worse the performance becomes. A discussion of this issue, with references to other studies, can be found in [ESKI90].

- **Communications performance:** Processes that interact intensively can be moved to the same node to reduce communications cost for the duration of their interaction. Also, when a process is performing data analysis on some file or set of files larger than the process's size, it may be advantageous to move the process to the data rather than vice versa.
- **Availability:** Long-running processes may need to move to survive in the face of faults for which advance notice is possible or in advance of scheduled downtime. If the operating system provides such notification, a process that wants to continue can either migrate to another system or ensure that it can be restarted on the current system at some later time.
- **Utilizing special capabilities:** A process can move to take advantage of unique hardware or software capabilities on a particular node.

## Process Migration Mechanisms

A number of issues need to be addressed in designing a process migration facility. Among these are the following:

- Who initiates the migration?
- What portion of the process is migrated?
- What happens to outstanding messages and signals?

***Initiation of Migration*** Who initiates migration will depend on the goal of the migration facility. If the goal is load balancing, then some module in the operating system that is monitoring system load will generally be responsible for deciding when migration should take place. The module will be responsible for preempting or signaling a process to be migrated. To determine where to migrate, the module will need to be in communication with peer modules in other systems so the load patterns on other systems can be monitored. If the goal is to reach particular resources, then a process may migrate itself as the need arises. In this latter case, the process must be aware of the existence of a distributed system. In the former case, the entire migration function, and indeed the existence of multiple systems, may be transparent to the process.

***What Is Migrated?*** When a process is migrated, it is necessary to destroy the process on the source system and create it on the target system. This is a movement of a process, not a replication. Thus, the process image, consisting of at least the process control block, must be moved. In addition, any links between this process and other processes, such as for passing messages and signals, must be updated. Figure 19.1 illustrates these considerations. Process 3 has migrated out of machine S to become Process 4 in machine D. All link identifiers held by processes (denoted in lowercase letters) remain the same as before. It is the responsibility of the operating system to move the process control block and to update link mappings. The transfer of the process of one machine to another is invisible to both the migrated process and its communication partners.

The movement of the process control block is straightforward. The difficulty, from a performance point of view, concerns the process address space and any open

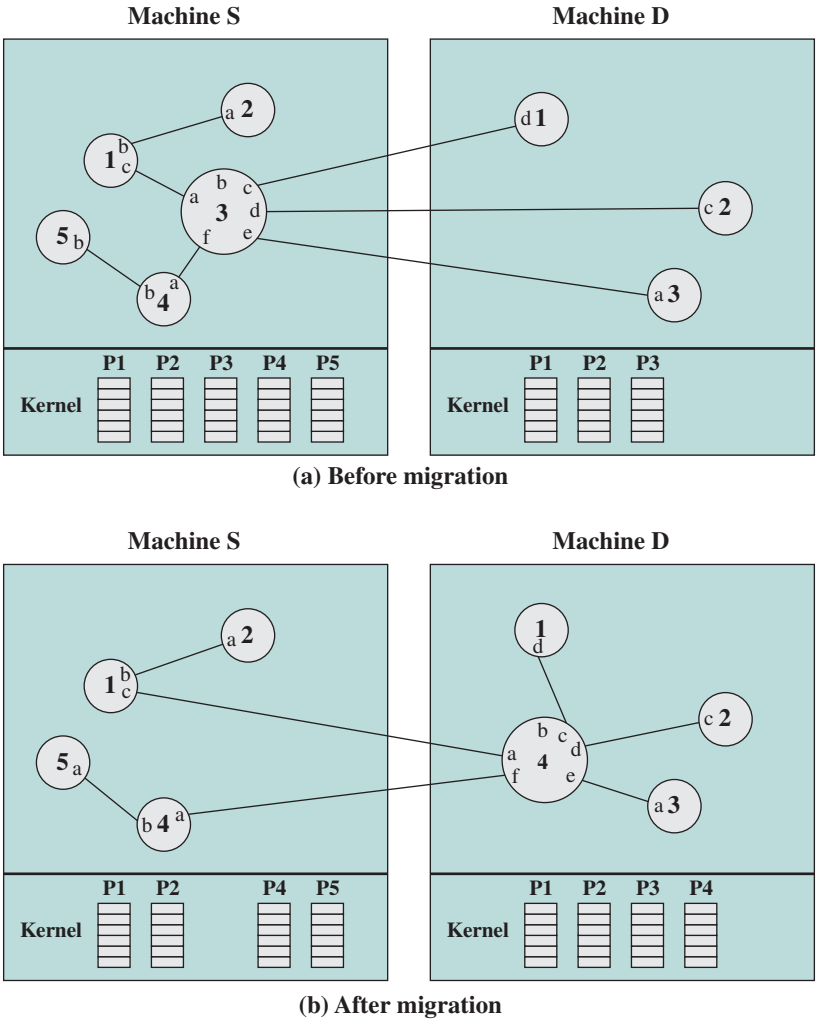



**Figure 19.1** Example of Process Migration

files assigned to the process. Consider first the process address space and let us assume that a virtual memory scheme (paging or paging/segmentation) is being used. The following strategies have been considered [MILO00]:

- **Eager (all):** Transfer the entire address space at the time of migration. This is certainly the cleanest approach. No trace of the process need to be left behind at the old system. However, if the address space is very large and if the process is likely not to need most of it, then this may be unnecessarily expensive. Initial costs of migration may be on the order of minutes. Implementations that provide a checkpoint/restart facility are likely to use this approach, because it is simpler to do the checkpointing and restarting if all of the address space is localized.

- **Precopy:** The process continues to execute on the source node while the address space is copied to the target node. Pages modified on the source during the precopy operation have to be copied a second time. This strategy reduces the time that a process is frozen and cannot execute during migration.
- **Eager (dirty):** Transfer only those pages of the address space that are in main memory and have been modified. Any additional blocks of the virtual address space will be transferred on demand only. This minimizes the amount of data that are transferred. It does require, however, that the source machine continue to be involved in the life of the process by maintaining page and/or segment table entries and it requires remote paging support.
- **Copy-on-reference:** This is a variation of eager (dirty) in which pages are only brought over when referenced. This has the lowest initial cost of process migration, ranging from a few tens to a few hundreds of microseconds.
- **Flushing:** The pages of the process are cleared from the main memory of the source by flushing dirty pages to disk. Then pages are accessed as needed from disk instead of from memory on the source node. This strategy relieves the source of the need to hold any pages of the migrated process in main memory, immediately freeing a block of memory to be used for other processes.

If it is likely that the process will not use much of its address space while on the target machine (e.g., the process is only temporarily going to another machine to work on a file and will soon return), then one of the last three strategies makes sense. On the other hand, if much of the address space will eventually be accessed while on the target machine, then the piecemeal transfer of blocks of the address space may be less efficient than simply transferring all of the address space at the time of migration, using one of the first two strategies.

In many cases, it may not be possible to know in advance whether or not much of the nonresident address space will be needed. However, if processes are structured as threads, and if the basic unit of migration is the thread rather than the process, then a strategy based on remote paging would seem to be the best. Indeed, such a strategy is almost mandated, because the remaining threads of the process are left behind and also need access to the address space of the process. Thread migration is implemented in the Emerald operating system [JUL89].

Similar considerations apply to the movement of open files. If the file is initially on the same system as the process to be migrated, and if the file is locked for exclusive access by that process, then it may make sense to transfer the file with the process. The danger here is that the process may only be gone temporarily and may not need the file until its return. Therefore, it may make sense to transfer the entire file only after an access request is made by the migrated process. If a file is shared by multiple distributed processes, then distributed access to the file should be maintained without moving the file.

If caching is permitted, as in the Sprite system (see Figure 16.7), then an additional complexity is introduced. For example, if a process has a file open for writing and it forks and migrates a child, the file would then be open for writing on two different hosts; Sprite's cache consistency algorithm dictates that the file be made noncacheable on the machines on which the two processes are executing [DOUG89, DOUG91].

***Messages and Signals*** The final issue listed previously, the fate of messages and signals, is addressed by providing a mechanism for temporarily storing outstanding messages and signals during the migration activity then directing them to the new destination. It may be necessary to maintain forwarding information at the initial site for some time to assure that all outstanding messages and signals get through.

***A Migration Scenario*** As a representative example of self-migration, let us consider the facility available on IBM's AIX operating system [WALK89], which is a distributed UNIX operating system. A similar facility is available on the LOCUS operating system [POPE85], and in fact the AIX system is based on the LOCUS development. This facility has also been ported to the OSF/1 AD operating system, under the name TNC [ZAJC93].

The following sequence of events occurs:

1. When a process decides to migrate itself, it selects a target machine and sends a remote tasking message. The message carries a part of the process image and open file information.
2. At the receiving site, a kernel server process forks a child, giving it this information.
3. The new process pulls over data, environment, arguments, or stack information as needed to complete its operation. Program text is copied over if it is dirty or demand paged from the global file system if it is clean.
4. The originating process is signaled on the completion of the migration. This process sends a final done message to the new process and destroys itself.

A similar sequence would be followed when another process initiates the migration. The principal difference is that the process to be migrated must be suspended so it can be migrated in a nonrunning state. This procedure is followed in Sprite, for example [DOUG89].

In the foregoing scenario, migration is a dynamic activity involving a number of steps for moving the process image over. When migration is initiated by another process, rather than self-migration, another approach is to copy the process image and its entire address space into a file, destroy the process, copy the file to another machine using a file transfer facility, then recreate the process from the file on the target machine. [SMIT89] describes such an approach.

## Negotiation of Migration

Another aspect of process migration relates to the decision about migration. In some cases, the decision is made by a single entity. For example, if load balancing is the goal, a load-balancing module monitors the relative load on various machines and performs migration as necessary to maintain a load balance. If self-migration is used to allow a process access to special facilities or to large remote files, then the process itself may make the decision. However, some systems allow the designated target system to participate in the decision. One reason for this could be to preserve response time for users. A user at a workstation, for example, might suffer noticeable response time degradation if processes migrate to the user's system, even if such migration served to provide better overall balance.

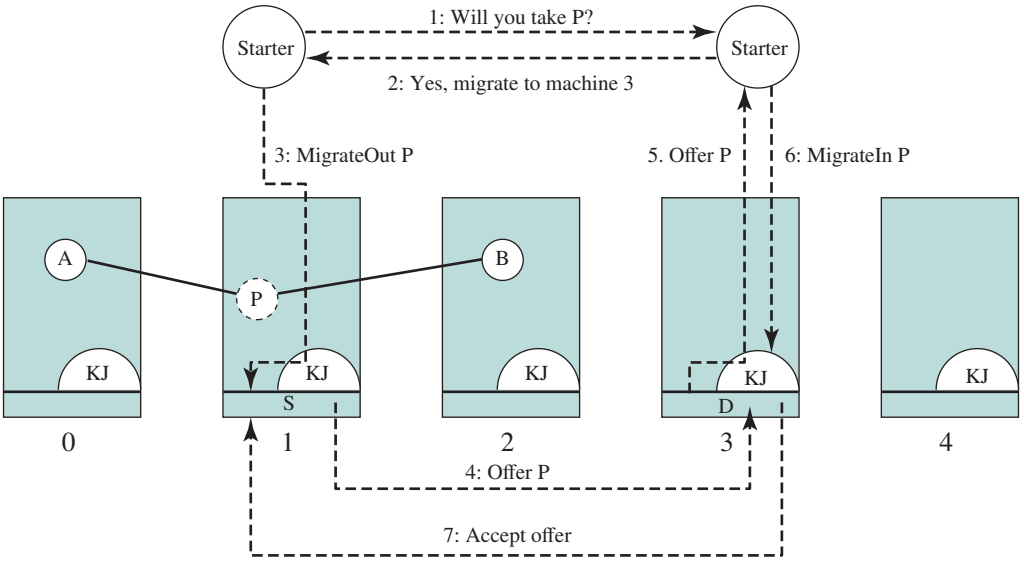



**Figure 19.2 Negotiation of Process Migration**

An example of a negotiation mechanism is that found in Charlotte [FINK89, ARTS89b]. Migration policy (when to migrate which process to what destination) is the responsibility of the Starter utility, which is a process that is also responsible for long-term scheduling and memory allocation. The Starter can therefore coordinate policy in these three areas. Each Starter process may control a cluster of machines. The Starter receives timely and fairly elaborate load statistics from the kernel of each of its machines.

The decision to migrate must be reached jointly by two Starter processes (one on the source node and one on the destination node), as illustrated in Figure 19.2. The following steps occur:

1. The Starter that controls the source system (S) decides that a process P should be migrated to a particular destination system (D). It sends a message to D's Starter, requesting the transfer.
2. If D's Starter is prepared to receive the process, it sends back a positive acknowledgment.
3. S's Starter communicates this decision to S's kernel via service call (if the starter runs on S) or a message to the KernJob (KJ) of machine S (if the starter runs on another machine). KJ is a process used to convert messages from remote processes into service calls.
4. The kernel on S then offers to send the process to D. The offer includes statistics about P, such as its age and processor and communication loads.
5. If D is short of resources, it may reject the offer. Otherwise, the kernel on D relays the offer to its controlling Starter. The relay includes the same information as the offer from S.

6. The Starter's policy decision is communicated to D by a MigrateIn call.
7. D reserves necessary resources to avoid deadlock and flow-control problems and then sends an acceptance to S.

Figure 19.2 also shows two other processes, A and B, that have links open to P. Following the foregoing steps, machine 1, where S resides, must send a link update message to both machines 0 and 2 to preserve the links from A and B to P. Link update messages tell the new address of each link held by P and are acknowledged by the notified kernels for synchronization purposes. After this point, a message sent to P on any of its links will be sent directly to D. These messages can be exchanged concurrently with the steps just described. Finally, after step 7 and after all links have been updated, S collects all of P's context into a single message and sends it to D.

Machine 4 is also running Charlotte but is not involved in this migration and therefore has no communication with the other systems in this episode.

## Eviction

The negotiation mechanism allows a destination system to refuse to accept the migration of a process to itself. In addition, it might also be useful to allow a system to evict a process that has been migrated to it. For example, if a workstation is idle, one or more processes may be migrated to it. Once the user of that workstation becomes active, it may be necessary to evict the migrated processes to provide adequate response time.

An example of an eviction capability is that found in Sprite [DOUG89]. In Sprite, which is a workstation operating system, each process appears to run on a single host throughout its lifetime. This host is known as the home node of the process. If a process is migrated, it becomes a foreign process on the destination machine. At any time the destination machine may evict the foreign process, which is then forced to migrate back to its home node.

The elements of the Sprite eviction mechanism are as follows:

1. A monitor process at each node keeps track of current load to determine when to accept new foreign processes. If the monitor detects activity at the workstation's console, it initiates an eviction procedure on each foreign process.
2. If a process is evicted, it is migrated back to its home node. The process may be migrated again if another node is available.
3. Although it may take some time to evict all processes, all processes marked for eviction are immediately suspended. Permitting an evicted process to execute while it is waiting for eviction would reduce the time during which the process is frozen, but also reduce the processing power available to the host while evictions are underway.
4. The entire address space of an evicted process is transferred to the home node. The time to evict a process and migrate it back to its home node may be reduced substantially by retrieving the memory image of an evicted process from its previous foreign host as referenced. However, this compels the foreign host to dedicate resources and honor service requests from the evicted process for a longer period of time than necessary.

### Preemptive versus Nonpreemptive Transfers

The discussion in this section has dealt with preemptive process migration, which involves transferring a partially executed process, or at least a process whose creation has been completed. A simpler function is nonpreemptive process transfer, which involves only processes that have not begun execution and hence do not require transferring the state of the process. In both types of transfer, information about the environment in which the process will execute must be transferred to the remote node. This may include the user's current working directory, the privileges inherited by the process, and inherited resources such as file descriptions.

Nonpreemptive process migration can be useful in load balancing (e.g., see [SHIV92]). It has the advantage that it avoids the overhead of full-blown process migration. The disadvantage is that such a scheme does not react well to sudden changes in load distribution.

## 19.2 DISTRIBUTED GLOBAL STATES

### Global States and Distributed Snapshots

All of the concurrency issues that are faced in a tightly coupled system, such as mutual exclusion, deadlock, and starvation, are also faced in a distributed system. Design strategies in these areas are complicated by the fact that there is no global state to the system. That is, it is not possible for the operating system, or any process, to know the current state of all processes in the distributed system. A process can only know the current state of all the processes on the local system, by access to process control blocks in memory. For remote processes, a process can only know state information that is received via messages, which represent the state of the remote process sometime in the past. This is analogous to the situation in astronomy: Our knowledge of a distant star or galaxy consists of light and other electromagnetic waves arriving from the distant object, and these waves provide a picture of the object sometime in the past. For example, our knowledge of an object at a distance of five light-years is five years old.

The time lags imposed by the nature of distributed systems complicate all issues relating to concurrency. To illustrate this, we present an example taken from [ANDR90]. We will use process/event graphs (see Figures 19.3 and 19.4) to illustrate the problem. In these graphs, there is a horizontal line for each process representing the time axis. A point on the line corresponds to an event (e.g., internal process event, message send, message receive). A box surrounding a point represents a snapshot of the local process state taken at that point. An arrow represents a message between two processes.

In our example, an individual has a bank account distributed over two branches of a bank. To determine the total amount in the customer's account, the bank must determine the amount in each branch. Suppose the determination is to be made at exactly 3:00 p.m. Figure 19.3a shows an instance in which a balance of $100.00 in the combined account is found. But the situation in Figure 19.3b is also possible. Here, the balance from branch A is in transit to branch B at the time of observation; the result is a false reading of $0.00. This particular problem can be solved by examining all messages in transit at the time of observation. Branch A will keep a record of all

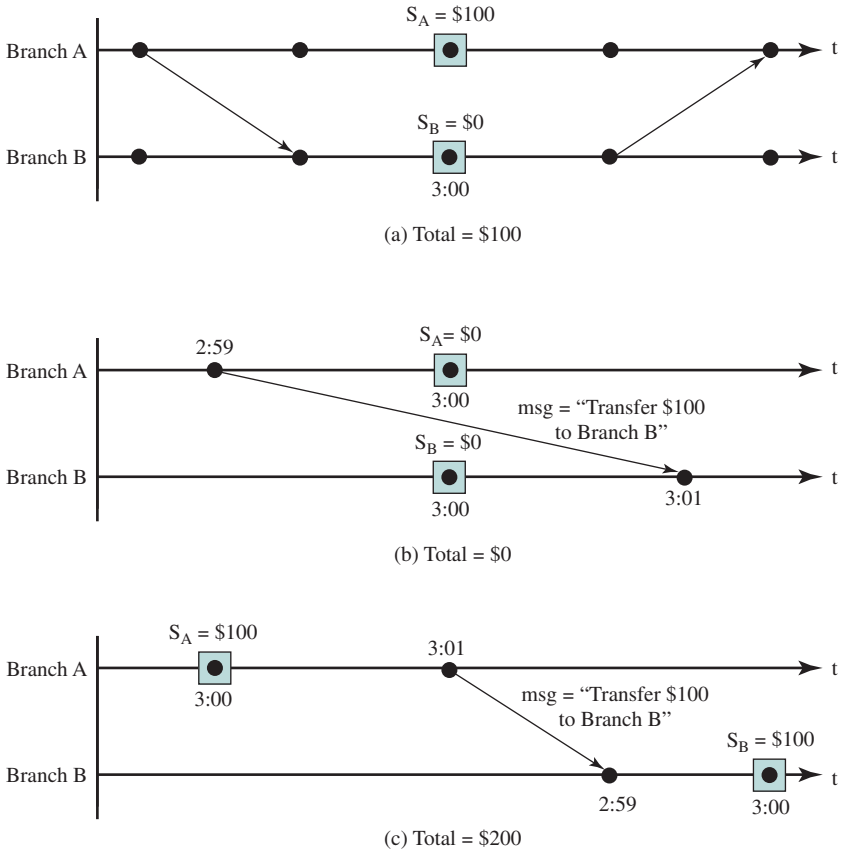



**Figure 19.3 Example of Determining Global States**

transfers out of the account, together with the identity of the destination of the transfer. Therefore, we will include in the "state" of a branch A account both the current balance and a record of transfers. When the two accounts are examined, the observer finds a transfer that has left branch A destined for the customer's account in branch B. Because the amount has not yet arrived at branch B, it is added into the total balance. Any amount that has been both transferred and received is counted only once, as part of the balance at the receiving account.

This strategy is not foolproof, as shown in Figure 19.3c. In this example, the clocks at the two branches are not perfectly synchronized. The state of the customer account at branch A at 3:00 p.m. indicates a balance of $100.00. However, this amount is subsequently transferred to branch B at 3:01 according to the clock at A but arrives at B at 2:59 according to B's clock. Therefore, the amount is counted twice for a 3:00 observation.

To understand the difficulty we face and to formulate a solution, let us define the following terms:

- **Channel:** A channel exists between two processes if they exchange messages. We can think of the channel as the path or means by which the messages are

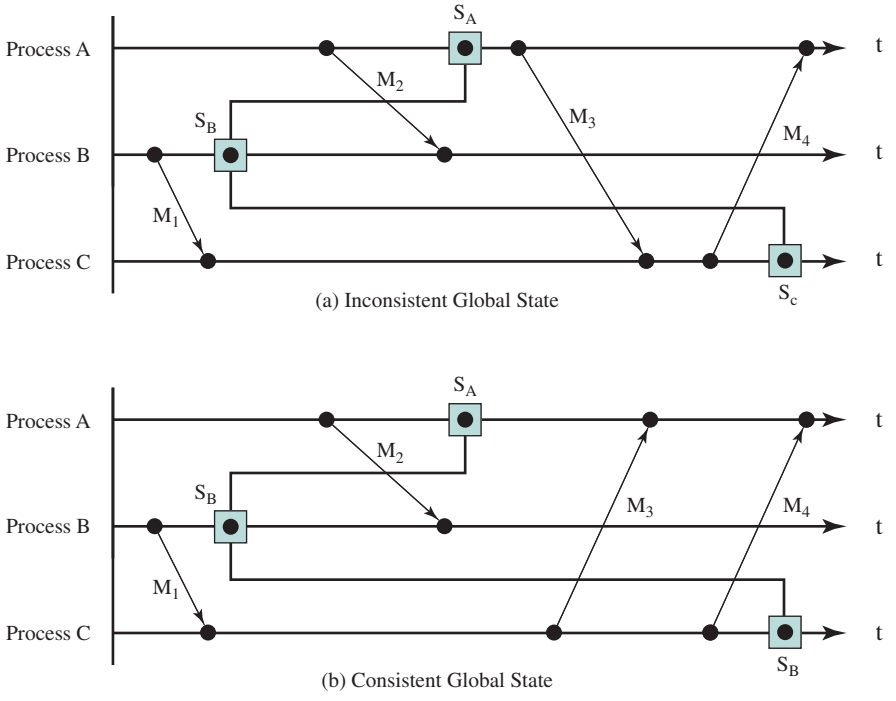



**Figure 19.4 Inconsistent and Consistent Global States**

transferred. For convenience, channels are viewed as unidirectional. Thus, if two processes exchange messages, two channels are required, one for each direction of message transfer.

- **State:** The state of a process is the sequence of messages that have been sent and received along channels incident with the process.
- **Snapshot:** A snapshot records the state of a process. Each snapshot includes a record of all messages sent and received on all channels since the last snapshot.
- **Global state:** The combined state of all processes.
- **Distributed snapshot:** A collection of snapshots, one for each process.

The problem is that a true global state cannot be determined because of the time lapse associated with message transfer. We can attempt to define a global state by collecting snapshots from all processes. For example, the global state of Figure 19.4a at the time of the taking of snapshots shows a message in transit on the <A,B> channel, one in transit on the <A,C> channel, and one in transit on the <C,A> channel. Messages 2 and 4 are represented appropriately, but message 3 is not. The distributed snapshot indicates that this message has been received but not yet sent.

We desire that the distributed snapshot record a consistent global state. A global state is consistent if for every process state that records the receipt of a

message, the sending of that message is recorded in the process state of the process that sent the message. Figure 19.4b gives an example. An inconsistent global state arises if a process has recorded the receipt of a message but the corresponding sending process has not recorded that the message has been sent (see Figure 19.4a).

## The Distributed Snapshot Algorithm

A distributed snapshot algorithm that records a consistent global state has been described in [CHAN85]. The algorithm assumes that messages are delivered in the order in which they are sent, and that no messages are lost. A reliable transport protocol (e.g., TCP) satisfies these requirements. The algorithm makes use of a special control message, called a **marker**.

Some process initiates the algorithm by recording its state and sending a marker on all outgoing channels before any more messages are sent. Each process $p$ then proceeds as follows. Upon the first receipt of the marker (say from process $q$), receiving process $p$ performs the following:

1. $p$ records its local state $S_p$.
2. $p$ records the state of the incoming channel from $q$ to $p$ as empty.
3. $p$ propagates the marker to all of its neighbors along all outgoing channels.

These steps must be performed atomically; that is, no messages can be sent or received by $p$ until all 3 steps are performed.

At any time after recording its state, when $p$ receives a marker from another incoming channel (say from process $r$), it performs the following:

- $p$ records the state of the channel from $r$ to $p$ as the sequence of messages $p$ has received from $r$ from the time $p$ recorded its local state $S_p$ to the time it received the marker from $r$.

The algorithm terminates at a process once the marker has been received along every incoming channel.

[ANDR90] makes the following observations about the algorithm:

1. Any process may start the algorithm by sending out a marker. In fact, several nodes could independently decide to record the state and the algorithm would still succeed.
2. The algorithm will terminate in finite time if every message (including marker messages) is delivered in finite time.
3. This is a distributed algorithm: Each process is responsible for recording its own state and the state of all incoming channels.
4. Once all of the states have been recorded (the algorithm has terminated at all processes), the consistent global state obtained by the algorithm can be assembled at every process by having every process send the state data that it has recorded along every outgoing channel, and having every process forward the state data that it receives along every outgoing channel. Alternatively, the initiating process could poll all processes to acquire the global state.
5. The algorithm does not affect and is not affected by any other distributed algorithm that the processes are participating in.

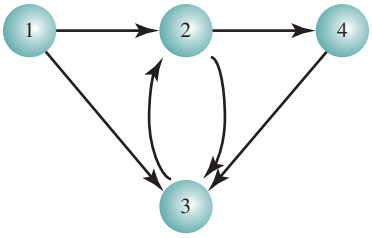



**Figure 19.5** **Process and Channel Graph**

As an example of the use of the algorithm (taken from [BEN06]), consider the set of processes illustrated in Figure 19.5. Each process is represented by a node, and each unidirectional channel is represented by a line between two nodes, with the direction indicated by an arrowhead. Suppose the snapshot algorithm is run, with nine messages being sent along each of its outgoing channels by each process. Process 1 decides to record the global state after sending six messages, and process 4 independently decides to record the global state after sending three messages. Upon termination, the snapshots are collected from each process; the results are shown in Figure 19.6. Process 2 sent four messages on each of the two outgoing channels to processes 3 and 4 prior to the recording of the state. It received four messages from process 1 before recording its state, leaving messages 5 and 6 to be associated with the channel. The reader should check the snapshot for consistency: Each message sent was either received at the destination process or recorded as being in transit in the channel.

The distributed snapshot algorithm is a powerful and flexible tool. It can be used to adapt any centralized algorithm to a distributed environment, because the basis of any centralized algorithm is knowledge of the global state. Specific examples include detection of deadlock and detection of process termination (e.g., see [BEN06], [LYNC96]). It can also be used to provide a checkpoint of a distributed algorithm to allow rollback and recovery if a failure is detected.

| | |
|---|---|
| **Process 1**<br>Outgoing channels<br>2 sent 1,2,3,4,5,6<br>3 sent 1,2,3,4,5,6<br>Incoming channels | **Process 3**<br>Outgoing channels<br>2 sent 1,2,3,4,5,6,7,8<br>Incoming channels<br>1 received 1,2,3 stored 4,5,6<br>2 received 1,2,3 stored 4<br>4 received 1,2,3 |
| **Process 2**<br>Outgoing channels<br>3 sent 1,2,3,4<br>4 sent 1,2,3,4<br>Incoming channels<br>1 received 1,2,3,4 stored 5,6<br>3 received 1,2,3,4,5,6,7,8 | **Process 4**<br>Outgoing channels<br>3 sent 1,2,3<br>Incoming channels<br>2 received 1,2 stored 3,4 |

**Figure 19.6** **An Example of a Snapshot**

## 19.3 DISTRIBUTED MUTUAL EXCLUSION

Recall that in Chapters 5 and 6, we addressed issues relating to the execution of concurrent processes. Two key problems that arose were those of mutual exclusion and deadlock. Chapters 5 and 6 focused on solutions to this problem in the context of a single system, with one or more processors but with a common main memory. In dealing with a distributed operating system and a collection of processors that do not share common main memory or clock, new difficulties arise and new solutions are called for. Algorithms for mutual exclusion and deadlock must depend on the exchange of messages and cannot depend on access to common memory. In this section and the next, we examine mutual exclusion and deadlock in the context of a distributed operating system.

### Distributed Mutual Exclusion Concepts

When two or more processes compete for the use of system resources, there is a need for a mechanism to enforce mutual exclusion. Suppose two or more processes require access to a single nonsharable resource, such as a printer. During the course of execution, each process will be sending commands to the I/O device, receiving status information, sending data, and/or receiving data. We will refer to such a resource as a critical resource, and the portion of the program that uses it as a critical section of the program. It is important that only one program at a time be allowed in its critical section. We cannot simply rely on the operating system to understand and enforce this restriction, because the detailed requirement may not be obvious. In the case of the printer, for example, we wish any individual process to have control of the printer while it prints an entire file. Otherwise, lines from competing processes will be interleaved.

The successful use of concurrency among processes requires the ability to define critical sections and enforce mutual exclusion. This is fundamental for any concurrent processing scheme. Any facility or capability that is to provide support for mutual exclusion should meet the following requirements:

1. Mutual exclusion must be enforced: Only one process at a time is allowed into its critical section, among all processes that have critical sections for the same resource or shared object.
2. A process that halts in its noncritical section must do so without interfering with other processes.
3. It must not be possible for a process requiring access to a critical section to be delayed indefinitely: no deadlock or starvation.
4. When no process is in a critical section, any process that requests entry to its critical section must be permitted to enter without delay.
5. No assumptions are made about relative process speeds or number of processors.
6. A process remains inside its critical section for a finite time only.

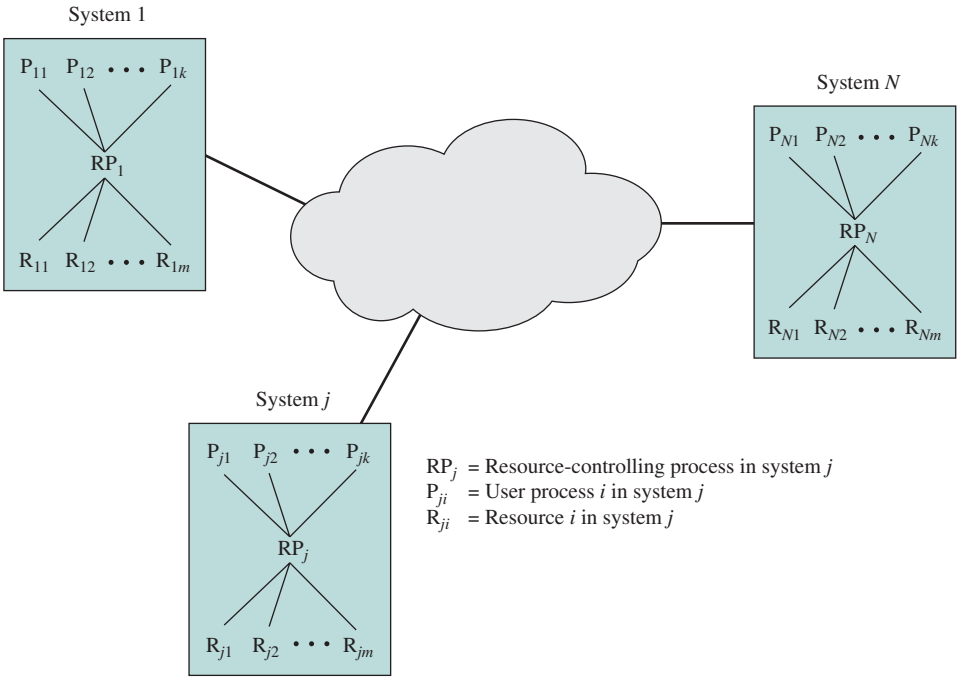



**Figure 19.7 Model for Mutual Exclusion Problem in Distributed Process Management**

Figure 19.7 shows a model that we can use for examining approaches to mutual exclusion in a distributed context. We assume some number of systems interconnected by some type of networking facility. Within each system, we assume some function or process within the operating system is responsible for resource allocation. Each such process controls a number of resources and serves a number of user processes. The task is to devise an algorithm by which these processes may cooperate in enforcing mutual exclusion.

Algorithms for mutual exclusion may be either centralized or distributed. In a fully **centralized algorithm**, one node is designated as the control node and controls access to all shared objects. When any process requires access to a critical resource, it issues a Request to its local resource-controlling process. This process, in turn, sends a Request message to the control node, which returns a Reply (permission) message when the shared object becomes available. When a process has finished with a resource, a Release message is sent to the control node. Such a centralized algorithm has two key properties:

1. Only the control node makes resource-allocation decisions.
2. All necessary information is concentrated in the control node, including the identity and location of all resources and the allocation status of each resource.

The centralized approach is straightforward, and it is easy to see how mutual exclusion is enforced: The control node will not satisfy a request for a resource until

that resource has been released. However, such a scheme suffers several drawbacks. If the control node fails, then the mutual exclusion mechanism breaks down, at least temporarily. Furthermore, every resource allocation and deallocation requires an exchange of messages with the control node. Thus, the control node may become a bottleneck.

Because of the problems with centralized algorithms, there has been more interest in the development of distributed algorithms. A fully **distributed algorithm** is characterized by the following properties:

1. All nodes have an equal amount of information, on average.
2. Each node has only a partial picture of the total system and must make decisions based on this information.
3. All nodes bear equal responsibility for the final decision.
4. All nodes expend equal effort, on average, in effecting a final decision.
5. Failure of a node, in general, does not result in a total system collapse.
6. There exists no system-wide common clock with which to regulate the timing of events.

Points 2 and 6 may require some elaboration. With respect to point 2, some distributed algorithms require that all information known to any node be communicated to all other nodes. Even in this case, at any given time, some of that information will be in transit and will not have arrived at all of the other nodes. Thus, because of time delays in message communication, a node's information is usually not completely up to date and is in that sense only partial information.

With respect to point 6, because of the delay in communication among systems, it is impossible to maintain a system-wide clock that is instantly available to all systems. Furthermore, it is also technically impractical to maintain one central clock and to keep all local clocks synchronized precisely to that central clock; over a period of time, there will be some drift among the various local clocks that will cause a loss of synchronization.

It is the delay in communication, coupled with the lack of a common clock, that makes it much more difficult to develop mutual exclusion mechanisms in a distributed system compared to a centralized system. Before looking at some algorithms for distributed mutual exclusion, we examine a common approach to overcoming the clock inconsistency problem.

## Ordering of Events in a Distributed System

Fundamental to the operation of most distributed algorithms for mutual exclusion and deadlock is the temporal ordering of events. The lack of a common clock or a means of synchronizing local clocks is thus a major constraint. The problem can be expressed in the following manner. We would like to be able to say that an event $a$ at system $i$ occurred before (or after) event $b$ at system $j$, and we would like to be able to arrive consistently at this conclusion at all systems in the network. Unfortunately, this statement is not precise for two reasons. First, there may be a delay between the actual occurrence of an event and the time that it is observed on some other system. Second, the lack of synchronization leads to a variance in clock readings on different systems.

To overcome these difficulties, a method referred to as timestamping has been proposed by Lamport [LAMP78], which orders events in a distributed system without using physical clocks. This technique is so efficient and effective that it is used in the great majority of algorithms for distributed mutual exclusion and deadlock.

To begin, we need to decide on a definition of the term *event*. Ultimately, we are concerned with actions that occur at a local system, such as a process entering or leaving its critical section. However, in a distributed system, the way in which processes interact is by means of messages. Therefore, it makes sense to associate events with messages. A local event can be bound to a message very simply; for example, a process can send a message when it desires to enter its critical section or when it is leaving its critical section. To avoid ambiguity, we associate events with the sending of messages only, not with the receipt of messages. Thus, each time that a process transmits a message, an event is defined that corresponds to the time that the message leaves the process.

The timestamping scheme is intended to order events consisting of the transmission of messages. Each system $i$ in the network maintains a local counter, $C_i$, which functions as a clock. Each time a system transmits a message, it first increments its clock by 1. The message is sent in the form

$$(m, T_i, i)$$

where

$m$ = contents of the message
$T_i$ = timestamp for this message, set to equal $C_i$
$i$ = numerical identifier of this system in the distributed system

When a message is received, the receiving system $j$ sets its clock to one more than the maximum of its current value and the incoming timestamp:

$$C_j \leftarrow 1 + \max[C_j, T_i]$$

At each site, the ordering of events is determined by the following rules. For a message $x$ from site $i$ and a message $y$ from site $j$, $x$ is said to precede $y$ if one of the following conditions holds:

1. If $T_i < T_j$, or
2. If $T_i = T_j$ and $i < j$

The time associated with each message is the timestamp accompanying the message, and the ordering of these times is determined by the two foregoing rules. That is, two messages with the same timestamp are ordered by the numbers of their sites. Because the application of these rules is independent of site, this approach avoids any problems of drift among the various clocks of the communicating processes.

An example of the operation of this algorithm is shown in Figure 19.8. There are three sites, each of which is represented by a process that controls the timestamping algorithm. Process $P_1$ begins with a clock value of 0. To transmit message $a$, it increments its clock by 1 and transmits $(a, 1, 1)$, where the first numerical value is the timestamp and the second is the identity of the site. This message is received by

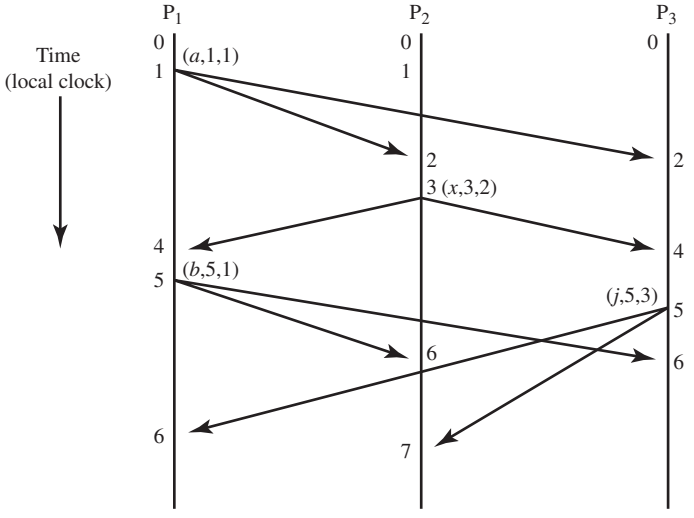



**Figure 19.8 Example of Operation of Timestamping Algorithm**

processes at sites 2 and 3. In both cases, the local clock has a value of zero and is set to a value of $2 = 1 + \max[0, 1]$. $P_2$ issues the next message, first incrementing its clock to 3. Upon receipt of this message, $P_1$ and $P_3$ increment their clocks to 4. Then $P_1$ issues message *b* and $P_3$ issues message *j* at about the same time and with the same timestamp. Because of the ordering principle outlined previously, this causes no confusion. After all of these events have taken place, the ordering of messages is the same at all sites, namely $\{a, x, b, j\}$.

The algorithm works in spite of differences in transmission times between pairs of systems, as illustrated in Figure 19.9. Here, $P_1$ and $P_4$ issue messages with the same timestamp. The message from $P_1$ arrives earlier than that of $P_4$ at site 2, but later than

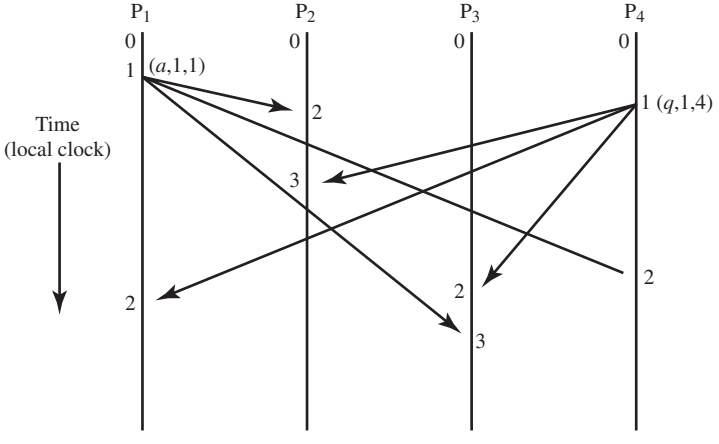



**Figure 19.9 Another Example of Operation of Timestamping Algorithm**

that of $P_4$ at site 3. Nevertheless, after all messages have been received at all sites, the ordering of messages is the same at all sites: $\{a, q\}$.

Note the ordering imposed by this scheme does not necessarily correspond to the actual time sequence. For the algorithms based on this timestamping scheme, it is not important which event actually happened first. It is only important that all processes that implement the algorithm agree on the ordering that is imposed on the events.

In the two examples just discussed, each message is sent from one process to all other processes. If some messages are not sent this way, some sites do not receive all of the messages in the system, and it is therefore impossible that all sites have the same ordering of messages. In such a case, a collection of partial orderings exist. However, we are primarily concerned with the use of timestamps in distributed algorithms for mutual exclusion and deadlock detection. In such algorithms, a process usually sends a message (with its timestamp) to every other process, and the timestamps are used to determine how the messages are processed.

## Distributed Queue

***First Version*** One of the earliest proposed approaches to providing distributed mutual exclusion is based on the concept of a distributed queue [LAMP78]. The algorithm is based on the following assumptions:

1. A distributed system consists of $N$ nodes, uniquely numbered from 1 to $N$. Each node contains one process that makes requests for mutually exclusive access to resources on behalf of other processes; this process also serves as an arbitrator to resolve incoming requests from other nodes that overlap in time.
2. Messages sent from one process to another are received in the same order in which they are sent.
3. Every message is correctly delivered to its destination in a finite amount of time.
4. The network is fully connected; this means that every process can send messages directly to every other process, without requiring an intermediate process to forward the message.

Assumptions 2 and 3 can be realized by the use of a reliable transport protocol, such as TCP (Chapter 13).

For simplicity, we describe the algorithm for the case in which each site only controls a single resource. The generalization to multiple resources is trivial.

The algorithm attempts to generalize an algorithm that would work in a straightforward manner in a centralized system. If a single central process managed the resource, it could queue incoming requests and grant requests in a first-in-first-out manner. To achieve this same algorithm in a distributed system, all of the sites must have a copy of the same queue. Timestamping can be used to assure that all sites agree on the order in which resource requests are to be granted. One complication arises: Because it takes some finite amount of time for messages to transit a network, there is a danger that two different sites will not agree on which process is at the head of the queue. Consider Figure 19.9. There is a point at which message $a$ has arrived at $P_2$

and message $q$ has arrived at $P_3$, but both messages are still in transit to other processes. Thus, there is a period of time in which $P_1$ and $P_2$ consider message $a$ to be the head of the queue and in which $P_3$ and $P_4$ consider message $q$ to be the head of the queue. This could lead to a violation of the mutual exclusion requirement. To avoid this, the following rule is imposed: For a process to make an allocation decision based on its own queue, it needs to have received a message from each of the other sites such that the process is guaranteed that no message earlier than its own head of queue is still in transit. This rule is explained in part 3b of the algorithm described subsequently.

At each site, a data structure is maintained that keeps a record of the most recent message received from each site (including the most recent message generated at this site). Lamport refers to this structure as a queue; actually it is an array with one entry for each site. At any instant, entry $q[j]$ in the local array contains a message from $P_j$. The array is initialized as follows:

$$q[j] = (\text{Release}, 0, j) \quad j = 1, \ldots, N$$

Three types of messages are used in this algorithm:

- (Request, $T_i$, $i$): A request for access to a resource is made by $P_i$.
- (Reply, $T_j$, $j$): $P_j$ grants access to a resource under its control.
- (Release, $T_k$, $k$): $P_k$ releases a resource previously allocated to it.

The algorithm is as follows:

1. When $P_i$ requires access to a resource, it issues a request (Request, $T_i$, $i$), time-stamped with the current local clock value. It puts this message in its own array at $q[i]$ and sends the message to all other processes.
2. When $P_j$ receives (Request, $T_i$, $i$), it puts this message in its own array at $q[i]$. If $q[j]$ does not contain a request message, then $P_j$ transmits (Reply, $T_j$, $j$) to $P_i$. It is this action that implements the rule described previously, which assures that no earlier Request message is in transit at the time of a decision.
3. $P_i$ can access a resource (enter its critical section) when both of these conditions hold:
   a. $P_i$'s own Request message in array $q$ is the earliest Request message in the array; because messages are consistently ordered at all sites, this rule permits one and only one process to access the resource at any instant.
   b. All other messages in the local array are later than the message in $q[i]$; this rule guarantees that $P_i$ has learned about all requests that preceded its current request.
3. $P_i$ releases a resource by issuing a release (Release, $T_i$, $i$), which it puts in its own array and transmits to all other processes.
4. When $P_i$ receives (Release, $T_j$, $j$), it replaces the current contents of $q[j]$ with this message.
5. When $P_i$ receives (Reply, $T_j$, $j$), it replaces the current contents of $q[j]$ with this message.

It is easily shown that this algorithm enforces mutual exclusion, is fair, avoids deadlock, and avoids starvation:

- **Mutual exclusion:** Requests for entry into the critical section are handled according to the ordering of messages imposed by the timestamping mechanism. Once $P_i$ decides to enter its critical section, there can be no other Request message in the system that was transmitted before its own. This is true because $P_i$ has by then necessarily received a message from all other sites and these messages from other sites date from later than its own Request message. We can be sure of this because of the Reply message mechanism; remember that messages between two sites cannot arrive out of order.
- **Fair:** Requests are granted strictly on the basis of timestamp ordering. Therefore, all processes have equal opportunity.
- **Deadlock free:** Because the timestamp ordering is consistently maintained at all sites, deadlock cannot occur.
- **Starvation free:** Once $P_i$ has completed its critical section, it transmits the Release message. This has the effect of deleting $P_i$'s Request message at all other sites, allowing some other process to enter its critical section.

As a measure of efficiency of this algorithm, note to guarantee exclusion, $3 \times (N - 1)$ messages are required: $(N - 1)$ Request messages, $(N - 1)$ Reply messages, and $(N - 1)$ Release messages.

***Second Version*** A refinement of the Lamport algorithm was proposed in [RICA81]. It seeks to optimize the original algorithm by eliminating Release messages. The same assumptions as before are in force, except that it is not necessary that messages sent from one process to another are received in the same order in which they are sent.

As before, each site includes one process that controls resource allocation. This process maintains an array $q$ and obeys the following rules:

1. When $P_i$ requires access to a resource, it issues a request (Request, $T_i$, $i$), timestamped with the current local clock value. It puts this message in its own array at $q[i]$ and sends the message to all other processes.
2. When $P_j$ receives (Request, $T_i$, $i$), it obeys the following rules:
   a. If $P_j$ is currently in its critical section, it defers sending a Reply message (see Rule 4, which follows)
   b. If $P_j$ is not waiting to enter its critical section (has not issued a Request that is still outstanding), it transmits (Reply, $T_j$, $j$) to $P_i$.
   c. If $P_j$ is waiting to enter its critical section and if the incoming message follows $P_j$'s request, then it puts this message in its own array at $q[i]$ and defers sending a Reply message.
   d. If $P_j$ is waiting to enter its critical section and if the incoming message precedes $P_j$'s request, then it puts this message in its own array at $q[i]$ and transmits (Reply, $T_j$, $j$) to $P_i$.

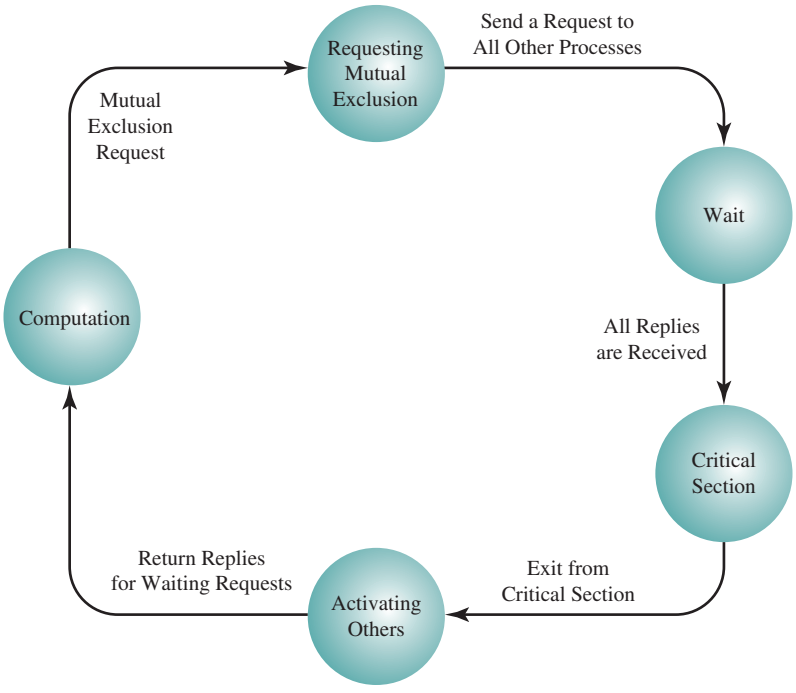



**Figure 19.10 State Diagram for Algorithm in [RICA81]**

5. $P_i$ can access a resource (enter its critical section) when it has received a Reply message from all other processes.
6. When $P_i$ leaves its critical section, it releases the resource by sending a Reply message to each pending Request.

The state transition diagram for each process is shown in Figure 19.10.

To summarize, when a process wishes to enter its critical section, it sends a time-stamped Request message to all other processes. When it receives a Reply from all other processes, it may enter its critical section. When a process receives a Request from another process, it must eventually send a matching Reply. If a process does not wish to enter its critical section, it sends a Reply at once. If it wants to enter its critical section, it compares the timestamp of its Request with that of the last Request received, and if the latter is more recent, it defers its Reply; otherwise Reply is sent at once.

With this method, $2 \times (N - 1)$ messages are required: $(N - 1)$ Request messages to indicate $P_i$'s intention of entering its critical section, and $(N - 1)$ Reply messages to allow the access it has requested.

The use of timestamping in this algorithm enforces mutual exclusion. It also avoids deadlock. To prove the latter, assume the opposite: It is possible that, when there are no more messages in transit, we have a situation in which each process has transmitted a Request and has not received the necessary Reply. This situation cannot arise, because a decision to defer a Reply is based on a relation that orders Requests. There is therefore one Request that has the earliest timestamp and that will receive all the necessary Replies. Deadlock is therefore impossible.

Starvation is also avoided because Requests are ordered. Because Requests are served in that order, every Request will at some stage become the oldest and will then be served.

### A Token-Passing Approach

A number of investigators have proposed a quite different approach to mutual exclusion, which involves passing a token among the participating processes. The token is an entity that at any time is held by one process. The process holding the token may enter its critical section without asking permission. When a process leaves its critical section, it passes the token to another process.

In this subsection, we look at one of the most efficient of these schemes. It was first proposed in [SUZU82]; a logically equivalent proposal also appeared in [RICA83]. For this algorithm, two data structures are needed. The token, which is passed from process to process, is actually an array, `token`, whose $k$th element records the timestamp of the last time that the token visited process $P_k$. In addition, each process maintains an array, request, whose $j$th element records the timestamp of the last Request received from $P_j$.

The procedure is as follows. Initially, the token is assigned arbitrarily to one of the processes. When a process wishes to use its critical section, it may do so if it currently possesses the token; otherwise it broadcasts a timestamped request message to all other processes and waits until it receives the token. When process $P_j$ leaves its critical section, it must transmit the token to some other process. It chooses the next process to receive the token by searching the request array in the order $j + 1, j + 2, \ldots, 1, 2, \ldots, j - 1$ for the first entry request $[k]$ such that the timestamp for $P_k$'s last request for the token is greater than the value recorded in the token for $P_k$'s last holding of the token, that is, request $[k] >$ token $[k]$.

Figure 19.11 depicts the algorithm, which is in two parts. The first part deals with the use of the critical section and consists of a prelude, followed by the critical section, followed by a postlude. The second part concerns the action to be taken upon receipt of a request. The variable clock is the local counter used for the timestamp function. The operation wait (access, token) causes the process to wait until a message of the type "access" is received, which is then put into the variable array token.

The algorithm requires either of the following:

- $N$ messages ($N - 1$ to broadcast the request and 1 to transfer the token) when the requesting process does not hold the token
- No messages, if the process already holds the token

## 19.4 DISTRIBUTED DEADLOCK

In Chapter 6, we defined deadlock as the permanent blocking of a set of processes that either compete for system resources or communicate with one another. This definition is valid for a single system as well as for a distributed system. As with mutual exclusion, deadlock presents more complex problems in a distributed system, compared with a shared memory system. Deadlock handling is complicated in a distributed system

```
if  (!token_present) {
  clock++;                /* Prelude */
  broadcast (Request, clock, i);
  wait (access, token);
  token_present = true;
}

token_held = true;
<critical section>;

token[i] = clock;         /* Postlude */
token_held = false;
 for  (int  j = i + 1; j < n; j++) {
  if  (request(j) > token[j] && token_present) {
    token_present = false;
    send (access, token[j]);
  }
}
```

**(a) First Part**

```
if  (received (Request, k, j)) {
  request (j) = max(request(j), k);
  if  (token_present && !token_held)
    <text of postlude>;
}
```

**(b) Second Part**

| Notation | |
|---|---|
| send (j, access, token) | end message of type access, with token, by process j |
| broadcast (request, clock, i) | send message from process i of type request, with timestamp clock, to all other processes |
| received (request, t, j) | receive message from process j of type request, with timestamp t |

**Figure 19.11** Token-Passing Algorithm (for process $P_i$)

because no node has accurate knowledge of the current state of the overall system and because every message transfer between processes involves an unpredictable delay.

Two types of distributed deadlock have received attention in the literature: those that arise in the allocation of resources, and those that arise with the communication of messages. In resource deadlocks, processes attempt to access resources, such as data objects in a database or I/O resources on a server; deadlock occurs if each process in a set of processes requests a resource held by another process in the set. In communications deadlocks, messages are the resources for which processes wait; deadlock occurs if each process in a set is waiting for a message from another process in the set, and no process in the set ever sends a message.

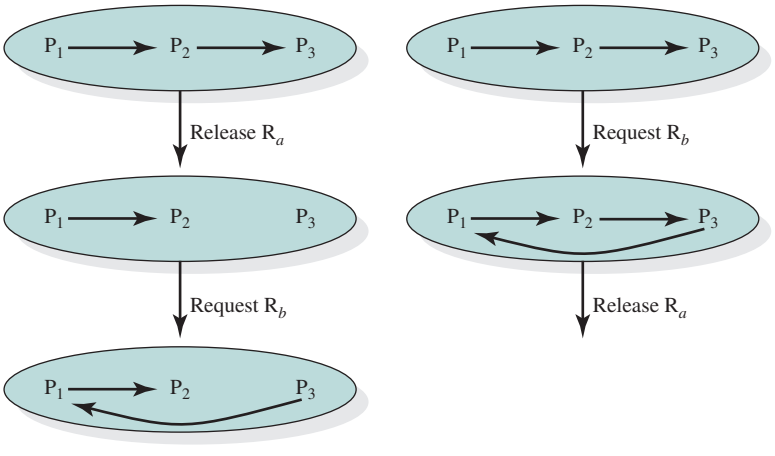
## Deadlock in Resource Allocation

Recall from Chapter 6 that a deadlock in resource allocation exists only if all of the following conditions are met:

- **Mutual exclusion:** Only one process may use a resource at a time. No process may access a resource unit that has been allocated to another process.
- **Hold and wait:** A process may hold allocated resources while awaiting assignment of others.
- **No preemption:** No resource can be forcibly removed from a process holding it.
- **Circular wait:** A closed chain of processes exists, such that each process holds at least one resource needed by the next process in the chain.

The aim of an algorithm that deals with deadlock is either to prevent the formation of a circular wait, or to detect its actual or potential occurrence. In a distributed system, the resources are distributed over various sites and access to them is regulated by control processes that do not have complete, up-to-date knowledge of the global state of the system and must therefore make their decisions on the basis of local information. Thus, new deadlock algorithms are required.

One example of the difficulty faced in distributed deadlock management is the phenomenon of phantom deadlock. An example of phantom deadlock is illustrated in Figure 19.12. The notation $P_1 \rightarrow P_2 \rightarrow P_3$ means that $P_1$ is halted waiting for a resource held by $P_2$, and $P_2$ is halted waiting for a resource held by $P_3$. Let us say that at the beginning of the example, $P_3$ owns resource $R_a$ and $P_1$ owns resource $R_b$. Suppose now that $P_3$ issues first a message releasing $R_a$ then a message requesting $R_b$. If the first message reaches a cycle-detecting process before the second, the sequence of Figure 19.12a results, which properly reflects resource requirements. If, however, the second message arrives before the first message, a deadlock is registered (see Figure 19.12b). This is a false detection, not a real deadlock, due to the lack of a global state, such as would exist in a centralized system.




(a) Release arrives before request (b) Request arrives before release

**Figure 19.12 Phantom Deadlock**

***DEADLOCK PREVENTION*** Two of the deadlock prevention techniques discussed in Chapter 6 can be used in a distributed environment.

1. The circular-wait condition can be prevented by defining a linear ordering of resource types. If a process has been allocated resources of type R, then it may subsequently request only those resources of types following R in the ordering. A major disadvantage of this method is that resources may not be requested in the order in which they are used; thus, resources may be held longer than necessary.
2. The hold-and-wait condition can be prevented by requiring that a process request all of its required resources at one time, and blocking the process until all requests can be granted simultaneously. This approach is inefficient in two ways. First, a process may be held up for a long time waiting for all of its resource requests to be filled, when in fact it could have proceeded with only some of the resources. Second, resources allocated to a process may remain unused for a considerable period, during which time they are denied to other processes.

Both of these methods require that a process determine its resource requirements in advance. This is not always the case; an example is a database application in which new items can be added dynamically. As an example of an approach that does not require this foreknowledge, we consider two algorithms proposed in [ROSE78]. These were developed in the context of database work, so we shall speak of transactions rather than processes.

The proposed methods make use of timestamps. Each transaction carries throughout its lifetime the timestamp of its creation. This establishes a strict ordering of the transactions. If a resource R already being used by transaction T1 is requested by another transaction T2, the conflict is resolved by comparing their timestamps. This comparison is used to prevent the formation of a circular-wait condition. Two variations of this basic method are proposed by the authors, referred to as the "wait-die" method and the "wound-wait" method.

Let us suppose T1 currently holds R and T2 issues a request. For the **wait-die method**, Figure 19.13a shows the algorithm used by the resource allocator at the site of R. The timestamps of the two transactions are denoted as e(T1) and e(T2). If T2 is older, it is blocked until T1 releases R, either by actively issuing a release or by being "killed" when requesting another resource. If T2 is younger, then T2 is restarted but with the same timestamp as before.

Thus, in a conflict, the older transaction takes priority. Because a killed transaction is revived with its original timestamp, it grows older and therefore gains increased priority. No site needs to know the state of allocation of all resources. All that are required are the timestamps of the transactions that request its resources.

```
if (e(T2) < e(T1))
   halt_T2 ('wait');
else
   kill_T2 ('die');
```

(a) Wait-die method

```
if (e(T2) < e(T1))
   kill_T1 ('wound');
else
   halt_T2 ('wait');
```

(b) Wound-wait method

**Figure 19.13 Deadlock Prevention Methods**

The **wound-wait method** immediately grants the request of an older transaction by killing a younger transaction that is using the required resource. This is shown in Figure 19.13b. In contrast to the wait-die method, a transaction never has to wait for a resource being used by a younger transaction.

***DEADLOCK AVOIDANCE*** Deadlock avoidance is a technique in which a decision is made dynamically whether a given resource allocation request could, if granted, lead to a deadlock. [SING94] points out that distributed deadlock avoidance is impractical for the following reasons:

1. Every node must keep track of the global state of the system; this requires substantial storage and communications overhead.
2. The process of checking for a safe global state must be mutually exclusive. Otherwise, two nodes could each be considering the resource request of a different process and concurrently reach the conclusion that it is safe to honor the request, when in fact if both requests are honored, deadlock will result.
3. Checking for safe states involves considerable processing overhead for a distributed system with a large number of processes and resources.

***DEADLOCK DETECTION*** With deadlock detection, processes are allowed to obtain free resources as they wish, and the existence of a deadlock is determined after the fact. If a deadlock is detected, one of the *constituent* processes is selected and required to release the resources necessary to break the deadlock.

The difficulty with distributed deadlock detection is that each site only knows about its own resources, whereas a deadlock may involve distributed resources. Several approaches are possible, depending on whether the system control is centralized, hierarchical, or distributed (see Table 19.1).

With **centralized control**, one site is responsible for deadlock detection. All request and release messages are sent to the central process as well as to the process that controls the particular resource. Because the central process has a complete picture, it is in a position to detect a deadlock. This approach requires a lot of messages and is vulnerable to a failure of the central site. In addition, phantom deadlocks may be detected.

With **hierarchical control**, the sites are organized in a tree structure, with one site serving as the root of the tree. At each node, other than leaf nodes, information about the resource allocation of all dependent nodes is collected. This permits deadlock detection to be done at lower levels than the root node. Specifically, a deadlock that involves a set of resources will be detected by the node that is the common ancestor of all sites whose resources are among the objects in conflict.

With **distributed control**, all processes cooperate in the deadlock detection function. In general, this means that considerable information must be exchanged, with timestamps; thus the overhead is significant. [RAYN88] cites a number of approaches based on distributed control, and [DATT90] provides a detailed examination of one approach.

We now give an example of a distributed deadlock detection algorithm ([DATT92], [JOHN91]). The algorithm deals with a distributed database system in which each site maintains a portion of the database and transactions may be initiated from each site. A transaction can have at most one outstanding resource request. If

**Table 19.1** Distributed Deadlock Detection Strategies

| Centralized Algorithms | | Hierarchical Algorithms | | Distributed Algorithms | |
|---|---|---|---|---|---|
| **Strengths** | **Weaknesses** | **Strengths** | **Weaknesses** | **Strengths** | **Weaknesses** |
| • Algorithms are conceptually simple and easy to implement.<br>• Central site has complete information and can optimally resolve deadlocks. | • Considerable communications overhead; every node must send state information to central node.<br>• Vulnerable to failure of central node. | • Not vulnerable to single point of failure.<br>• Deadlock resolution activity is limited if most potential deadlocks are relatively localized. | • May be difficult to configure system so that most potential deadlocks are localized; otherwise there may actually be more overhead than in a distributed approach. | • Not vulnerable to single point of failure.<br>• No node is swamped with deadlock detection activity. | • Deadlock resolution is cumbersome because several sites may detect the same deadlock and may not be aware of other nodes involved in the deadlock.<br>• Algorithms are difficult to design because of timing considerations. |

a transaction needs more than one data object, the second data object can be requested only after the first data object has been granted.

Associated with each data object *i* at a site are two parameters: a unique identifier $D_i$ and the variable Locked_by ($D_i$). This latter variable has the value nil if the data object is not locked by any transaction; otherwise its value is the identifier of the locking transaction.

Associated with each transaction *j* at a site are four parameters:

- A unique identifier $T_j$
- The variable Held_by ($T_j$), which is set to nil if transaction $T_j$ is executing or in a Ready state. Otherwise, its value is the transaction that is holding the data object required by transaction $T_j$.
- The variable Wait_for ($T_j$), which has the value nil if transaction $T_i$ is not waiting for any other transaction. Otherwise, its value is the identifier of the transaction that is at the head of an ordered list of transactions that are blocked.
- A queue Request_Q ($T_j$), which contains all outstanding requests for data objects being held by $T_j$. Each element in the queue is of the form ($T_k$, $D_k$), where $T_k$ is the requesting transaction and $D_k$ is the data object held by $T_j$.

For example, suppose transaction $T_2$ is waiting for a data object held by $T_1$, which is, in turn, waiting for a data object held by $T_0$. Then the relevant parameters have the following values:

| Transaction | Wait_for | Held_by | Request_Q |
|---|---|---|---|
| $T_0$ | nil | nil | $T_1$ |
| $T_1$ | $T_0$ | $T_0$ | $T_2$ |
| $T_2$ | $T_0$ | $T_1$ | nil |

This example highlights the difference between Wait_for ($T_i$) and Held_by ($T_i$). Neither process can proceed until $T_0$ releases the data object needed by $T_1$, which can then execute and release the data object needed by $T_2$.

Figure 19.14 shows the algorithm used for deadlock detection. When a transaction makes a lock request for a data object, a server process associated with that data object either grants or denies the request. If the request is not granted, the server process returns the identity of the transaction holding the data object.

When the requesting transaction receives a granted response, it locks the data object. Otherwise, the requesting transaction updates its Held_by variable to the identity of the transaction holding the data object. It adds its identity to the Request_Q of the holding transaction. It updates its Wait_for variable either to the identity of the holding transaction (if that transaction is not waiting) or to the identity of the Wait_for variable of the holding transaction. In this way, the Wait_for variable is set to the value of the transaction that ultimately is blocking execution. Finally, the requesting transaction issues an update message to all of the transactions in its own Request_Q to modify all the Wait_for variables that are affected by this change.

When a transaction receives an update message, it updates its Wait_for variable to reflect the fact that the transaction on which it had been ultimately waiting is now blocked by yet another transaction. Then it does the actual work of deadlock detection by checking to see if it is now waiting for one of the processes that is waiting for it. If not, it forwards the update message. If so, the transaction sends a clear message to the transaction holding its requested data object and allocates every data object that it holds to the first requester in its Request_Q and enqueues remaining requesters to the new transaction.

An example of the operation of the algorithm is shown in Figure 19.15. When $T_0$ makes a request for a data object held by $T_3$, a cycle is created. $T_0$ issues an update message that propagates from $T_1$ to $T_2$ to $T_3$. At this point, $T_3$ discovers that the intersection of its Wait_for and Request_Q variables is not empty. $T_3$ sends a clear message to $T_2$ so $T_3$ is purged from Request_Q ($T_2$), and it releases the data objects it held, activating $T_4$ and $T_6$.

## Deadlock in Message Communication

***Mutual Waiting*** Deadlock occurs in message communication when each of a group of processes is waiting for a message from another member of the group and there are no messages in transit.

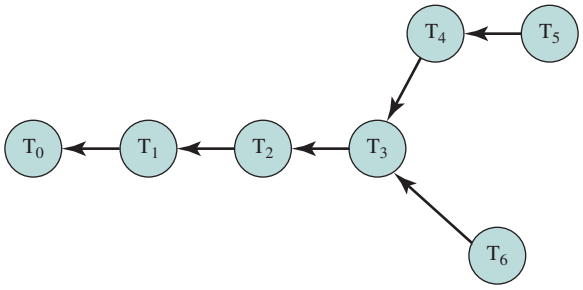
To analyze this situation in more detail, we define the dependence set (DS) of a process. For a process $P_i$ that is halted, waiting for a message, DS($P_i$) consists of all processes from which $P_i$ is expecting a message. Typically, $P_i$ can proceed if any of the expected messages arrives. An alternative formulation is that $P_i$ can proceed only after all of the expected messages arrive. The former situation is the more common one and is considered here.

With the preceding definition, a deadlock in a set S of processes can be defined as follows:

1. All the processes in S are halted, waiting for messages.
2. S contains the dependence set of all processes in S.
3. No messages are in transit between members of S.

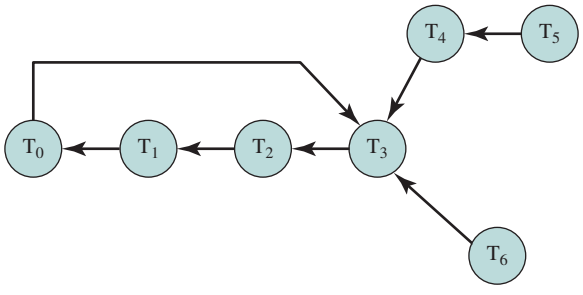
```
/* Data object Dj receiving a lock_request(Ti) */
  if (Locked_by(Dj) == null)
    send(granted);
  else {
    send not granted to Ti;
    send Locked_by(Dj) to Ti
  }
/* Transaction Ti makes a lock request for data object Dj  */
  send lock_request(Ti) to Dj;
  wait for granted/not granted;
  if (granted) {
    Locked_by(Dj) = Ti;
    Held_by(Ti) = f;
  }
  else { /* suppose Dj is being used by transaction Tj  */
    Held_by(Ti) = Tj;
    Enqueue(Ti, Request_Q(Tj));
    if (Wait_for(Tj) == null)
      Wait_for(Ti) = Tj  ;
    else
      Wait_for(Ti) = Wait_for(Tj);
      update(Wait_for(Ti), Request_Q(Ti));
  }
/* Transaction Tj receiving an update message */
  if (Wait_for(Tj) != Wait_for(Ti))
    Wait_for(Tj) = Wait_for(Ti);
  if (intersect(Wait_for(Tj), Request_Q(Tj)) = null)
    update(Wait_for(Ti), Request_Q(Tj);
  else  {
    DECLARE DEADLOCK;
      /* initiate deadlock resolution as follows */
      /* Tj is chosen as the transaction to be aborted */
      /* Tj releases all the data objects it holds */
      send_clear(Tj, Held_by(Tj));
      allocate each data object Di held by Tj to the first requester Tk
      in Request_Q(Tj);
      for (every transaction Tn  in Request_Q(Tj) requesting data object
          Di  held by Tj)
      {
        Enqueue(Tn, Request_Q(Tk));
      }
  }
/* Transaction Tk  receiving a clear(Tj, Tk) message */
    purge the tuple having Tj as the requesting transaction from
    Request_Q(Tk);
```

**Figure 19.14 A Distributed Deadlock Detection Algorithm**




| Transaction | Wait_for | Held_by | Request_Q |
|---|---|---|---|
| $T_0$ | nil | nil | $T_1$ |
| $T_1$ | $T_0$ | $T_0$ | $T_2$ |
| $T_2$ | $T_0$ | $T_1$ | $T_3$ |
| $T_3$ | $T_0$ | $T_2$ | $T_4$, $T_6$ |
| $T_4$ | $T_0$ | $T_3$ | $T_5$ |
| $T_5$ | $T_0$ | $T_4$ | nil |
| $T_6$ | $T_0$ | $T_3$ | nil |

(a) State of system before request




| Transaction | Wait_for | Held_by | Request_Q |
|---|---|---|---|
| $T_0$ | $T_0$ | $T_3$ | $T_1$ |
| $T_1$ | $T_0$ | $T_0$ | $T_2$ |
| $T_2$ | $T_0$ | $T_1$ | $T_3$ |
| $T_3$ | $T_0$ | $T_2$ | $T_4$, $T_6$, $T_0$ |
| $T_4$ | $T_0$ | $T_3$ | $T_5$ |
| $T_5$ | $T_0$ | $T_4$ | NIL |
| $T_6$ | $T_0$ | $T_3$ | NIL |

(b) State of system after $T_0$ makes a request to $T_3$

**Figure 19.15 Example of Distributed Deadlock Detection Algorithm of Figure 19.14**

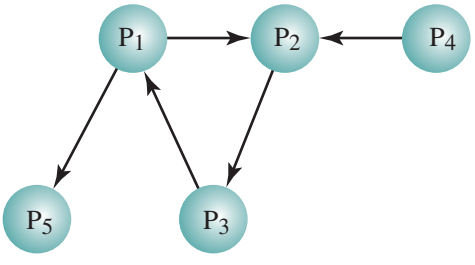
Any process in S is deadlocked because it can never receive a message that will release it.

In graphical terms, there is a difference between message deadlock and resource deadlock. With resource deadlock, a deadlock exists if there is a closed loop, or cycle, in the graph that depicts process dependencies. In the resource case, one process is dependent on another if the latter holds a resource that the former requires. With message deadlock, the condition for deadlock is that all successors of any member of S are themselves in S.

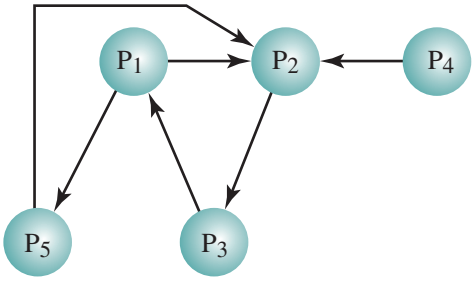
Figure 19.16 illustrates the point. In Figure 19.16a, $P_1$ is waiting for a message from either $P_2$ or $P_5$; $P_5$ is not waiting for any message and so can send a message to $P_1$, which is therefore released. As a result, the links ($P_1$, $P_5$) and ($P_1$, $P_2$) are deleted. Figure 19.16b adds a dependency: $P_5$ is waiting for a message from $P_2$, which is waiting for a message from $P_3$, which is waiting for a message from $P_1$, which is waiting for a message from $P_2$. Thus, deadlock exists.

As with resource deadlock, message deadlock can be attacked by either prevention or detection. [RAYN88] gives some examples.

***Unavailability of Message Buffers*** Another way in which deadlock can occur in a message-passing system has to do with the allocation of buffers for the storage of messages in transit. This kind of deadlock is well known in packet-switching data networks. We first examine this problem in the context of a data network, then view it from the point of view of a distributed operating system.




(a) No deadlock



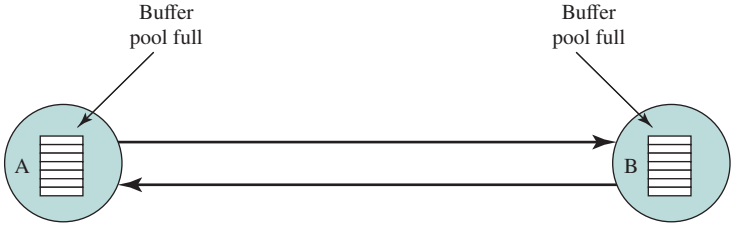

(b) Deadlock

**Figure 19.16 Deadlock in Message Communication**

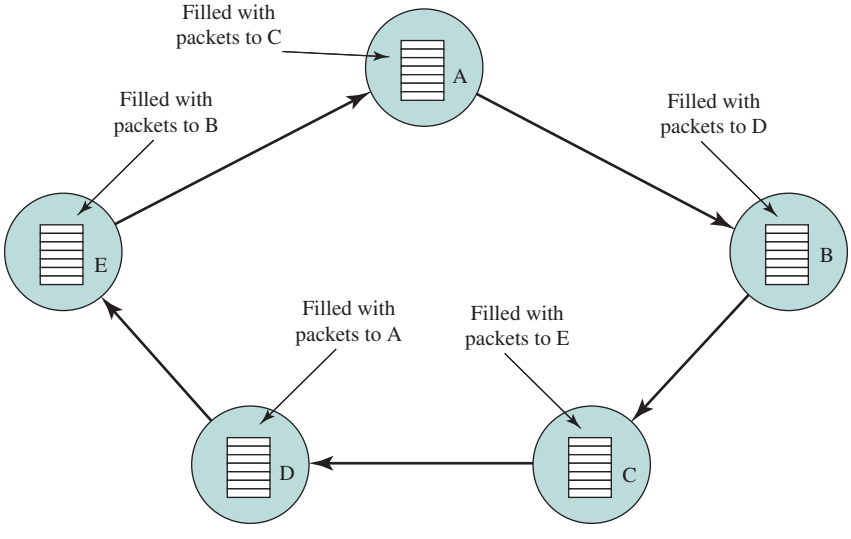
The simplest form of deadlock in a data network is direct store-and-forward deadlock and can occur if a packet-switching node uses a common buffer pool from which buffers are assigned to packets on demand. Figure 19.17a shows a situation in which all of the buffer space in node A is occupied with packets destined for B. The reverse is true at B. Neither node can accept any more packets because their buffers are full. Thus neither node can transmit or receive on any link.

Direct store-and-forward deadlock can be prevented by not allowing all buffers to end up dedicated to a single link. Using separate fixed-size buffers, one for each link, will achieve this prevention. Even if a common buffer pool is used, deadlock is avoided if no single link is allowed to acquire all of the buffer space.

A more subtle form of deadlock, indirect store-and-forward deadlock, is illustrated in Figure 19.17b. For each node, the queue to the adjacent node in one direction is full with packets destined for the next node beyond. One simple way to prevent this type of deadlock is to employ a structured buffer pool (see Figure 19.18). The buffers are organized in a hierarchical fashion. The pool of memory at level 0 is




(a) Direct store-and-forward deadlock




(b) Indirect store-and-forward deadlock

**Figure 19.17 Store-and-Forward Deadlock**

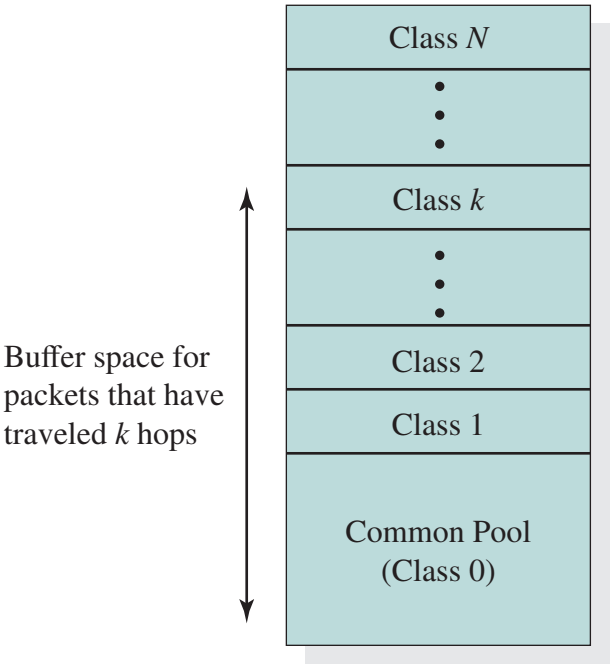



**Figure 19.18** **Structured Buffer Pool for Deadlock Prevention**

unrestricted; any incoming packet can be stored there. From level 1 to level $N$ (where $N$ is the maximum number of hops on any network path), buffers are reserved in the following way: Buffers at level $k$ are reserved for packets that have traveled at least $k$ hops so far. Thus, in heavy load conditions, buffers fill up progressively from level 0 to level $N$. If all buffers up through level $k$ are filled, arriving packets that have covered $k$ or less hops are discarded. It can be shown [GOPA85] that this strategy eliminates both direct and indirect store-and-forward deadlocks.

The deadlock problem just described would be dealt with in the context of communications architecture, typically at the network layer. The same sort of problem can arise in a distributed operating system that uses message passing for interprocess communication. Specifically, if the send operation is nonblocking, then a buffer is required to hold outgoing messages. We can think of the buffer used to hold messages to be sent from process X to process Y to be a communications channel between X and Y. If this channel has finite capacity (finite buffer size), then it is possible for the send operation to result in process suspension. That is, if the buffer is of size $n$ and there are currently $n$ messages in transit (not yet received by the destination process), then the execution of an additional send will block the sending process until a receive has opened up space in the buffer.

Figure 19.19 illustrates how the use of finite channels can lead to deadlock. The figure shows two channels, each with a capacity of four messages, one from process X to process Y and one from Y to X. If exactly four messages are in transit in each of the channels, and both X and Y attempt a further transmission before executing a receive, then both are suspended and a deadlock arises.

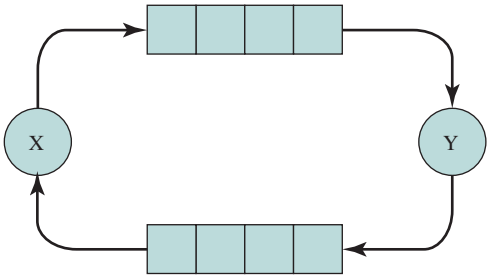



**Figure 19.19** **Communication Deadlock in a Distributed System**

If it is possible to establish upper bounds on the number of messages that will ever be in transit between each pair of processes in the system, then the obvious prevention strategy would be to allocate as many buffer slots as needed for all these channels. This might be extremely wasteful, and of course requires this foreknowledge. If requirements cannot be known ahead of time, or if allocating based on upper bounds is deemed too wasteful, then some estimation technique is needed to optimize the allocation. It can be shown that this problem is unsolvable in the general case; some heuristic strategies for coping with this situation are suggested in [BARB90].

## 19.5 SUMMARY

A distributed operating system may support process migration. This is the transfer of a sufficient amount of the state of a process from one machine to another for the process to execute on the target machine. Process migration may be used for load balancing, to improve performance by minimizing communication activity, to increase availability, or to allow processes access to specialized remote facilities.

With a distributed system, it is often important to establish global state information, to resolve contention for resources, and to coordinate processes. Because of the variable and unpredictable time delay in message transmission, care must be taken to assure that different processes agree on the order in which events have occurred.

Process management in a distributed system includes facilities for enforcing mutual exclusion and for taking action to deal with deadlock. In both cases, the problems are more complex than those in a single system.

## 19.6 REFERENCES

**ANDR90** Andrianoff, S. "A Module on Distributed Systems for the Operating System Course." *Proceedings, Twenty-First SIGCSE Technical Symposium on Computer Science Education, SIGSCE Bulletin,* February 1990.

**ARTS89a** Artsy, Y., ed. "Special Issue on Process Migration." *Newsletter of the IEEE Computer Society Technical Committee on Operating Systems,* Winter 1989.

**ARTS89b** Artsy, Y. "Designing a Process Migration Facility: The Charlotte Experience." *Computer,* September 1989.

**BARB90** Barbosa, V. "Strategies for the Prevention of Communication Deadlocks in Distributed Parallel Programs." *IEEE Transactions on Software Engineering,* November 1990.

**BEN06** Ben-Ari, M. *Principles of Concurrent and Distributed Programming*. Harlow, England: Addison-Wesley, 2006.

**CABR86** Cabrear, L. "The Influence of Workload on Load Balancing Strategies." *USENIX Conference Proceedings,* Summer 1986.

**CASA94** Casavant, T., and Singhal, M. *Distributed Computing Systems.* Los Alamitos, CA: IEEE Computer Society Press, 1994.

**CHAN90** Chandras, R. "Distributed Message Passing Operating Systems." *Operating Systems Review*, January 1990.

**DATT90** Datta, A., and Ghosh, S. "Deadlock Detection in Distributed Systems." *Proceedings, Phoenix Conference on Computers and Communications,* March 1990.

**DATT92** Datta, A.; Javagal, R.; and Ghosh, S. "An Algorithm for Resource Deadlock Detection in Distributed Systems," *Computer Systems Science and Engineering*, October 1992.

**DOUG89** Douglas, F., and Ousterhout, J. "Process Migration in Sprite: A Status Report." *Newsletter of the IEEE Computer Society Technical Committee on Operating Systems,* Winter 1989.

**DOUG91** Douglas, F., and Ousterhout, J. "Transparent Process Migration: Design Alternatives and the Sprite Implementation." *Software Practice and Experience*, August 1991.

**EAGE86** Eager, D.; Lazowska, E.; and Zahnorjan, J. "Adaptive Load Sharing in Homogeneous Distributed Systems." *IEEE Transactions on Software Engineering,* May 1986.

**ESKI90** Eskicioglu, M. "Design Issues of Process Migration Facilities in Distributed Systems." *Newsletter of the IEEE Computer Society Technical Committee on Operating Systems and Application Environments*, Summer 1990.

**FINK89** Finkel, R. "The Process Migration Mechanism of Charlotte." *Newsletter of the IEEE Computer Society Technical Committee on Operating Systems,* Winter 1989.

**GOPA85** Gopal, I. "Prevention of Store-and-Forward Deadlock in Computer Networks." *IEEE Transactions on Communications,* December 1985.

**JOHN91** Johnston, B.; Javagal, R.; Datta, A.; and Ghosh, S. "A Distributed Algorithm for Resource Deadlock Detection." *Proceedings, Tenth Annual Phoenix Conference on Computers and Communications*, March 1991.

**JUL88** Jul, E.; Levy, H.; Hutchinson, N.; and Black, A. "Fine-Grained Mobility in the Emerald System." *ACM Transactions on Computer Systems,* February 1988.

**JUL89** Jul, E. "Migration of Light-Weight Processes in Emerald." *Newsletter of the IEEE Computer Society Technical Committee on Operating Systems*, Winter 1989.

**LAMP78** Lamport, L. "Time, Clocks, and the Ordering of Events in a Distributed System." *Communications of the ACM,* July 1978.

**LELA86** Leland, W., and Ott, T. "Load-Balancing Heuristics and Process Behavior." *Proceedings, ACM SigMetrics Performance 1986 Conference*, 1986.

**LYNC96** Lynch, N. *Distributed Algorithms.* San Francisco, CA: Morgan Kaufmann, 1996.

**MILO00** Milojicic, D.; Douglis, F.; Paindaveine, Y.; Wheeler, R.; and Zhou, S. "Process Migration." *ACM Computing Surveys*, September 2000.

**POPE85** Popek, G., and Walker, B. *The LOCUS Distributed System Architecture*, Cambridge, MA: MIT Press, 1985.

**RAYN88** Raynal, M. *Distributed Algorithms and Protocols.* New York: Wiley, 1988.

**RICA81** Ricart, G., and Agrawala, A. "An Optimal Algorithm for Mutual Exclusion in Computer Networks." *Communications of the ACM,* January 1981 (Corrigendum in *Communications of the ACM,* September 1981).

**RICA83** Ricart, G., and Agrawala, A. "Author's Response to 'On Mutual Exclusion in Computer Networks' by Carvalho and Roucairol." *Communications of the ACM,* February 1983.

**ROSE78** Rosenkrantz, D.; Stearns, R.; and Lewis, P. "System Level Concurrency Control in Distributed Database Systems." *ACM Transactions on Database Systems,* June 1978.

**SHIV92** Shivaratri, N.; Krueger, P.; and Singhal, M. "Load Distributing for Locally Distributed Systems." *Computer*, December 1992.

**SING94** Singhal, M. "Deadlock Detection in Distributed Systems." In [CASA94].

**SMIT88** Smith, J. "A Survey of Process Migration Mechanisms." *Operating Systems Review*, July 1988.

**SMIT89** Smith, J. "Implementing Remote *fork()* with Checkpoint/restart." *Newsletter of the IEEE Computer Society Technical Committee on Operating Systems,* Winter 1989.

**SUZU82** Suzuki, I., and Kasami, T. "An Optimality Theory for Mutual Exclusion Algorithms in Computer Networks." *Proceedings of the Third International Conference on Distributed Computing Systems,* October 1982.

**WALK89** Walker, B., and Mathews, R. "Process Migration in AIX's Transparent Computing Facility." *Newsletter of the IEEE Computer Society Technical Committee on Operating Systems,* Winter 1989.

**ZAJC93** Zajcew, R., et al. "An OSF/1 UNIX for Massively Parallel Multicomputers." *Proceedings, Winter USENIX Conference*, January 1993.

## 19.7 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

### Key Terms

| | | |
|---|---|---|
| channel | eviction | preemptive transfer |
| distributed deadlock | global state | process migration |
| distributed mutual exclusion | nonpreemptive transfer | snapshot |

### Review Questions

**19.1.** Discuss some of the reasons for implementing process migration.

**19.2.** How is the process address space handled during process migration?

**19.3.** What are the motivations for preemptive and nonpreemptive process migration?

**19.4.** Why is it impossible to determine a true global state?

**19.5.** What is the difference between distributed mutual exclusion enforced by a centralized algorithm and enforced by a distributed algorithm?

**19.6.** Define the two types of distributed deadlock.

### Problems

**19.1.** The flushing policy is described in the subsection on process migration strategies in Section 19.1.

**a.** From the perspective of the source, which other strategy does flushing resemble?

**b.** From the perspective of the target, which other strategy does flushing resemble?

**19.2.** For Figure 19.9, it is claimed that all four processes assign an ordering of $\{a, q\}$ to the two messages, even though $q$ arrives before $a$ at $P_3$. Work through the algorithm to demonstrate the truth of the claim.

**19.3.** For Lamport's algorithm, are there any circumstances under which $P_i$ can save itself the transmission of a Reply message?

**19.4.** For the mutual exclusion algorithm of [RICA81],

**a.** Prove that mutual exclusion is enforced.

**b.** If messages do not arrive in the order that they are sent, the algorithm does not guarantee that critical sections are executed in the order of their requests. Is starvation possible?

**19.5.** In the token-passing mutual exclusion algorithm, is the timestamping used to reset clocks and correct drifts, as in the distributed queue algorithms? If not, what is the function of the timestamping?

**19.6.** For the token-passing mutual exclusion algorithm, prove that it:

**a.** guarantees mutual exclusion.

**b.** avoids deadlock.

**c.** is fair.

**19.7.** In Figure 19.11b, explain why the second line cannot simply read "request $(j) = t$."

CHAPTER 20

# Overview of Probability and Stochastic Processes


**20.1 Probability**
- Definitions of Probability
- Conditional Probability and Independence
- Bayes's Theorem

**20.2 Random Variables**
- Distribution and Density Functions
- Important Distributions
- Multiple Random Variables

**20.3 Elementary Concepts of Stochastic Processes**
- First- and Second-Order Statistics
- Stationary Stochastic Processes
- Spectral Density
- Independent Increments
- Ergodicity

**20.4 Problems**

**LEARNING OBJECTIVES**

After studying this chapter, you should be able to:

- Understand the basic concepts of probability.
- Explain the concept of random variable.
- Understand some of the important basic concepts of stochastic processes.

Before setting out on our exploration of queueing analysis, we review background on probability and stochastic processes. The reader familiar with these topics can safely skip this chapter.

The chapter begins with an introduction to some elementary concepts from probability theory and random variables; this material is needed for Chapter 21, on queueing analysis. Following this, we look at stochastic processes, which are also relevant to queueing analysis.

## 20.1 PROBABILITY

We give here the barest outline of probability theory, but enough to support the rest of this chapter.

### Definitions of Probability

Probability is concerned with the assignment of numbers to events. The probability $\Pr[A]$ of an event $A$ is a number between 0 and 1 that corresponds to the likelihood that the event $A$ will occur. Generally, we talk of performing an experiment and obtaining an **outcome**. The **event** $A$ is a particular outcome or set of outcomes, and a probability is assigned to that event.

It is difficult to get a firm grip on the concept of probability. Different applications of the theory present probability in different lights. In fact, there are a number of different definitions of probability. We highlight three here.

***AXIOMATIC DEFINITION*** A formal approach to probability is to state a number of axioms that define a probability measure and, from them, to derive laws of probability that can be used to perform useful calculations. The axioms are simply assertions that must be accepted. Once the axioms are accepted, it is possible to prove each of the laws.

The axioms and laws make use of the following concepts from set theory. The **certain event** $\Omega$ is the event that occurs in every experiment; it consists of the universe, or **sample space**, of all possible outcomes. The **union** $A \cup B$ of two events $A$ and $B$ is the event that occurs when either $A$ or $B$ or both occur. The **intersection** $A \cap B$, also written $AB$, is the event that occurs when both events $A$ and $B$ occur. The events $A$ and $B$ are **mutually exclusive** if the occurrence of one of them excludes the occurrence of the other; that is, there is no outcome that is included in both $A$ and $B$. The event $\overline{A}$, called the **complement** of $A$, is the event that occurs when $A$ does

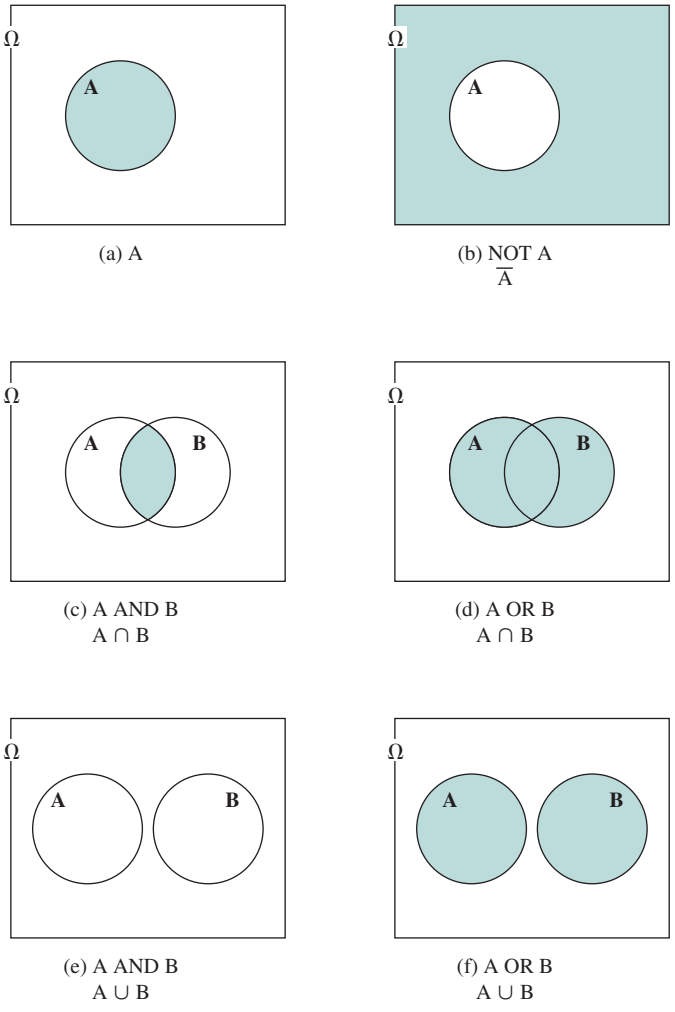



**Figure 20.1 Venn Diagrams**

not occur—that is, all outcomes in the sample space not included in $A$. These concepts are easily visualized with Venn diagrams, such as those shown in Figure 20.1. In each diagram, the shaded part corresponds to the expression below the diagram. Parts (c) and (d) correspond to cases in which $A$ and $B$ are not mutually exclusive; that is, some outcomes are defined as part of both events $A$ and $B$. Parts (e) and (f) correspond to cases in which $A$ and $B$ are mutually exclusive. Note in these cases, the intersection of the two events is the empty set.

The common set of axioms used to define probability is as follows:

1. $0 \leq \Pr[A] \leq 1$ for each event $A$
2. $\Pr[\Omega] = 1$
3. $\Pr[A \cup B] = \Pr[A] + \Pr[B]$ if $A$ and $B$ are mutually exclusive

Axiom 3 can be extended to many events. For example, $\Pr[A \cup B \cup C] = \Pr[A] + \Pr[B] + \Pr[C]$ if $A$, $B$, and $C$ are mutually exclusive. Note the axioms do not say anything about how probabilities are to be assigned to individual outcomes or events.

Based on these axioms, many laws can be derived. Here are some of the most important:

$$\Pr[\overline{A}) = 1 - \Pr[A]$$
$$\Pr[A \cap B] = 0 \text{ if } A \text{ and } B \text{ are mutually exclusive}$$
$$\Pr[A \cup B] = \Pr[A] + \Pr[B] - \Pr[A \cap B]$$
$$\Pr[A \cup B \cup C] = \Pr[A] + \Pr[B] + \Pr[C] - \Pr[A \cap B] - \Pr[A \cap C] - \Pr[B \cap C] + \Pr[A \cap B \cap C]$$

As an example, consider the throwing of a single die. This has six possible outcomes. The certain event is the event that occurs when any of the six die faces is on top. The union of the events {even} and {less than three} is the event {1 or 2 or 4 or 6}; the intersection of these events is the event {2}. The events {even} and {odd} are mutually exclusive. If we assume each of the six outcomes is equally likely and assign a probability of 1/6 to each outcome, it is easy to see that the three axioms are satisfied. We can apply the laws of probability as follows:

$$\Pr\{\text{even}\} = \Pr\{2\} + \Pr\{4\} + \Pr\{6\} = 1/2$$
$$\Pr\{\text{less than three}\} = \Pr\{1\} + \Pr\{2\} = 1/3$$
$$\Pr[\{\text{even}\} \cup \{\text{less than three}\}] = \Pr\{\text{even}\} + \Pr\{\text{less than three}\} - \Pr\{2\}$$
$$= 1/2 + 1/3 - 1/6 = 2/3$$

***Relative Frequency Definition*** The relative frequency approach uses the following definition of probability. Perform an experiment a number of times; each time is called a **trial**. For each trial, observe whether the event $A$ occurs. Then the probability $\Pr[A]$ of an event $A$ is the limit:

$$\Pr[A] = \lim_{n \to \infty} \frac{n_A}{n}$$

where $n$ is the number of trials, and $n_A$ is the number of occurrences of $A$.

For example, we could toss a coin many times. If the ratio of heads to total tosses hovers around 0.5 after a very large number of tosses, then we can assume that this is a fair coin, with equal probability of heads and tails.

***Classical Definition*** For the classical definition, let $N$ be the number of possible outcomes, with the restriction that all outcomes are equally likely, and $N_A$ the number of outcomes in which event $A$ occurs. Then the probability of $A$ is defined as:

$$\Pr[A] = \frac{N_A}{N}$$

For example, if we throw one die, then $N$ is 6 and there are three outcomes that correspond to the event {even}; hence $\Pr\{\text{even}\} = 3/6 = 0.5$. Here's a more complicated example: We roll two dice and want to determine the probability $p$ that the sum is 7. You could consider the number of different sums that could be produced $(2, 3, \ldots, 12)$, which is 11, and conclude incorrectly that the probability is 1/11. We need to consider equally likely outcomes. For this purpose, we need to consider each combination of die faces, and we must distinguish between the first and second die. For example, the outcome $(3,4)$ must be counted separately from the outcome $(4,3)$. With this approach, there are 36 equally likely outcomes, and the favorable outcomes are the six pairs $(1,6)$, $(2,5)$, $(3,4)$, $(4,3)$, $(5,2)$, and $(6,1)$. Thus, $p = 6/36 = 1/6$.

## Conditional Probability and Independence

We often want to know a probability that is conditional on some event. The effect of the condition is to remove some of the outcomes from the sample space. For example, what is the probability of getting a sum of 8 on the roll of two dice, if we know that the face of at least one die is an even number? We can reason as follows. Because one die is even and the sum is even, the second die must show an even number. Thus, there are three equally likely successful outcomes: (2, 6), (4, 4), and (6, 2), out of a total set of possibilities of $[36 - (\text{number of events with both faces odd})] = 36 - 3 \times 3 = 27$. The resulting probability is $3/27 = 1/9$.

Formally, the **conditional probability** of an event $A$ assuming the event $B$ has occurred, denoted by $\Pr[A \mid B]$, is defined as the ratio:

$$\Pr[A \mid B] = \frac{\Pr[AB]}{\Pr[B]}$$

where we assume $\Pr[B]$ is not zero.

In our example, $A = \{\text{sum of } 8\}$ and $B = \{\text{at least one die even}\}$. The quantity $\Pr[AB]$ encompasses all of those outcomes in which the sum is 8 and at least one die is even. As we have seen, there are three such outcomes. Thus, $\Pr[AB] = 3/36 = 1/12$. A moment's thought should convince you that $\Pr[B] = 3/4$. We can now calculate:

$$\Pr[A \mid B] = \frac{1/12}{3/4} = \frac{1}{9}$$

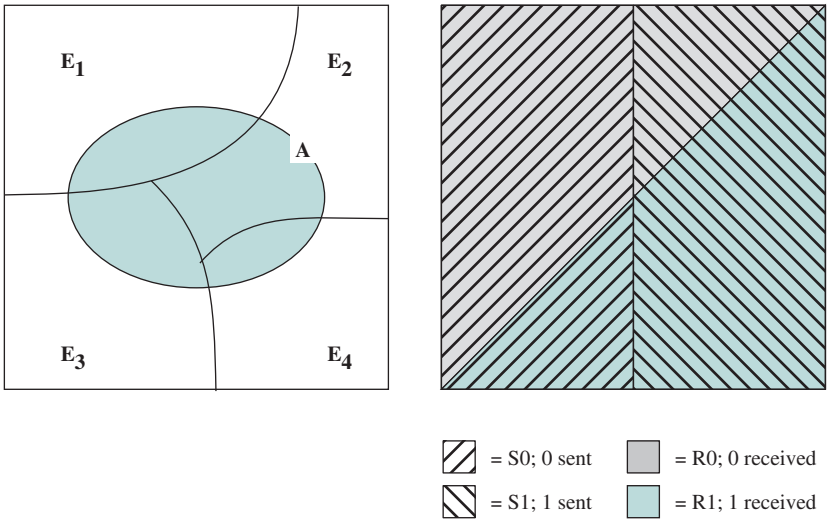
This agrees with our previous reasoning.

Two events $A$ and $B$ are called **independent** if $\Pr[AB] = \Pr[A]\Pr[B]$. It can easily be seen that if $A$ and $B$ are independent, $\Pr[A \mid B] = \Pr[A]$ and $\Pr[B \mid A] = \Pr[B]$.

## Bayes's Theorem

We close this section with one of the most important results from probability theory, known as Bayes's Theorem. First, we need to state the total probability formula. Given a set of mutually exclusive events $E_1, E_2, \ldots, E_n$, such that the union of these events covers all possible outcomes, and given an arbitrary event $A$, then it can be shown that

$$\Pr[A] = \sum_{i=1}^{n} \Pr[A \mid E_i]\Pr[E_i] \qquad \textbf{(20.1)}$$




(a) Diagram to illustrate concepts (b) Example

**Figure 20.2 Illustration of Total Probability and Bayes' Theorem**

Bayes's Theorem may be stated as follows:

$$\Pr[E_i|A] = \frac{\Pr[A|E_i]\mathrm{P}[E_i]}{\Pr[A]} = \frac{\Pr[A|E_i]\mathrm{P}[E_i]}{\sum_{j=1}^{n}\Pr[A|E_j]\Pr[E_j]}$$

Figure 20.2a illustrates the concepts of total probability and Bayes's Theorem.

Bayes's Theorem is used to calculate *posterior odds*, that is, the probability that something really is the case, given evidence in favor of it. For example, suppose we are transmitting a sequence of 0s and 1s over a noisy transmission line. Let S0 and S1 be the events that a 0 is sent at a given time and a 1 is sent, respectively, and R0 and R1 be the events that a 0 is received and a 1 is received. Suppose we know the probabilities of the source, namely $\Pr[\mathrm{S1}] = p$ and $\Pr[\mathrm{S0}] = 1 - p$. Now the line is observed to determine how frequently an error occurs when a 1 is sent and when a 0 is sent, and the following probabilities are calculated: $\Pr[\mathrm{R0}|\mathrm{S1}] = p_a$ and $\Pr[\mathrm{R1}|\mathrm{S0}] = p_b$. If a 0 is received, we can then calculate the conditional probability of an error, namely the conditional probability that a 1 was sent given that a 0 was received, using Bayes's Theorem:

$$\Pr[\mathrm{S1}|\mathrm{R0}] = \frac{\Pr[\mathrm{R0}|\mathrm{S1}]\Pr[\mathrm{S1}]}{\Pr[\mathrm{R0}|\mathrm{S1}]\Pr[\mathrm{S1}] + \Pr[\mathrm{R0}|\mathrm{S0}]\Pr[\mathrm{S0}]} = \frac{p_a p}{p_a p + (1 - p_b)(1 - p)}$$

Figure 20.2b illustrates the preceding equation. In the figure, the sample space is represented by a unit square. Half of the square corresponds to S0 and half to S1, so $\Pr[\mathrm{S0}] = \Pr[\mathrm{S1}] = 0.5$. Similarly, half of the square corresponds to R0 and half to R1, so $\Pr[\mathrm{R0}] = \Pr[\mathrm{R1}] = 0.5$. Within the area representing S0, 1/4 of that area corresponds to R1, so $\Pr[\mathrm{R1}|\mathrm{S0}] = 0.25$. Other conditional probabilities are similarly evident.

## 20.2 RANDOM VARIABLES

A **random variable** is a mapping from the set of all possible events in a sample space under consideration to the real numbers. That is, a random variable associates a real number with each event. This concept is sometimes expressed in terms of an experiment with many possible outcomes; a random variable assigns a value to each such outcome. Thus, the value of a random variable is a random quantity. We give the following formal definition. A random variable $X$ is a function that assigns a number to every outcome in a sample space and satisfies the following conditions:

1. The set $\{X \leq x\}$ is an event for every $x$.
2. $\Pr[X = \infty] = \Pr[X = -\infty] = 0$.

A random variable is **continuous** if it takes on an uncountably infinite number of distinct values. A random variable is **discrete** if it takes on a finite or countably infinite number of values.

### Distribution and Density Functions

A continuous random variable $X$ can be described by either its **distribution function** $F(x)$ or **density function** $f(x)$:

$$\text{Distribution function: } F(x) = \Pr[X \leq x] \quad F(-\infty) = 0; \quad F(\infty) = 1$$

$$\text{Density function: } f(x) = \frac{d}{dx}F(x) \qquad F(x) = \int_{-\infty}^{x} f(y)dy \qquad \int_{-\infty}^{\infty} f(y)dy = 1$$

For a discrete random variable, its probability distribution is characterized by

$$P_X(k) = \Pr[X = k] \sum_{\text{all } k} P_X(k) = 1$$

We are often concerned with some characteristic of a random variable rather than the entire distribution, such as shown in Table 20.1:

**Table 20.1** Random Variable Characteristics

| | |
|---|---|
| **Mean value** (also known as expected value or first moment) | $\begin{cases} \mathrm{E}[X] = \mu_X = \int_{-\infty}^{\infty} xf(x)dx & \text{continuous case} \\ \mathrm{E}[X] = \mu_X = \sum_{\text{all } k} k \Pr[x = k] & \text{discrete case} \end{cases}$ |
| **Second moment** | $\begin{cases} \mathrm{E}[X^2] = \int_{-\infty}^{\infty} x^2 f(x)dx & \text{continuous case} \\ \mathrm{E}[X^2] = \sum_{\text{all } k} k^2 \Pr[x = k] & \text{discrete case} \end{cases}$ |
| **Variance** | $\mathrm{Var}[X] = \mathrm{E}[(X - \mu_X)^2] = \mathrm{E}[X^2] - \mu_X^2$ |
| **Standard deviation** | $\sigma_X = \sqrt{\mathrm{Var}[x]}$ |

The variance and standard deviation are measures of the dispersion of values around the mean. A high variance means the variable takes on more values relatively farther from the mean than for a low variance. It is easy to show that for any constant $a$:

$$\mathrm{E}[aX] = a\mathrm{E}[X]; \qquad \mathrm{Var}[aX] = a^2\mathrm{Var}[X]$$

The mean is known as a first-order statistic; the second moment and variance are second-order statistics. Higher-order statistics can also be derived from the probability density function.

## Important Distributions

Several distributions that play an important role in queueing analysis are described next.

***EXPONENTIAL DISTRIBUTION*** The exponential distribution with parameter $\lambda > 0$ is given by (see Figures 20.3a and 20.3b) and has the following distribution and density functions:

$$F(x) = 1 - e^{-\lambda x} \qquad f(x) = \lambda e^{-\lambda x} \qquad x \geq 0$$

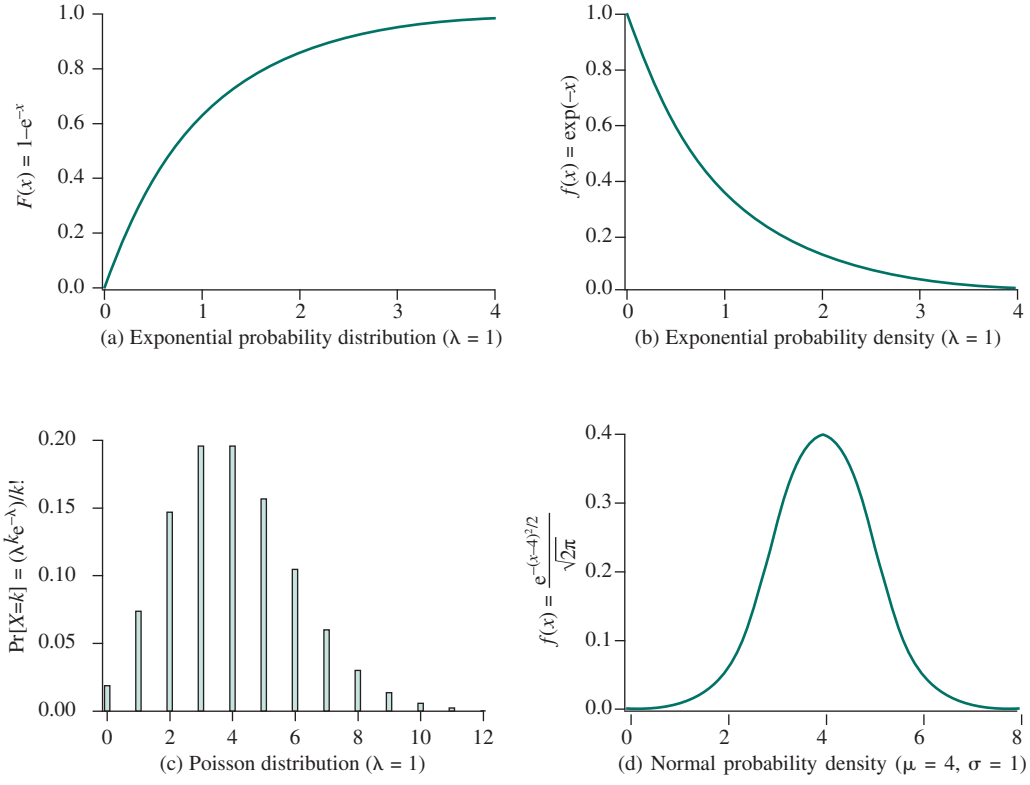



**Figure 20.3 Some Probability Functions**

The exponential distribution has the interesting property that its mean is equal to its standard deviation:

$$\mathrm{E}[X] = \sigma_X = \frac{1}{\lambda}$$

When used to refer to a time interval, such as a service time, this distribution is sometimes referred to as a random distribution. This is because, for a time interval that has already begun, each time at which the interval may finish is equally likely.

This distribution is important in queueing theory because we can often assume that the service time of a server in a queueing system is exponential. In the case of telephone traffic, the service time is the time for which a subscriber engages the equipment of interest. In a packet-switching network, the service time is the transmission time and is therefore proportional to the packet length. It is difficult to give a sound theoretical reason why service times should be exponential, but in many cases they are very nearly exponential. This is good news because it simplifies the queueing analysis immensely.

***POISSON DISTRIBUTION*** Another important distribution is the Poisson distribution (see Figure 20.3c), with parameter $\lambda > 0$, which takes on values at the points $0, 1, \ldots$:

$$\Pr[X = k] = \frac{\lambda^k}{k!}\mathrm{e}^{-\lambda} \qquad k = 0, 1, 2 \ldots$$
$$\mathrm{E}[X] = \mathrm{Var}[X] = \lambda$$

If $\lambda < 1$, then $\Pr[X = k]$ is maximum for $k = 0$. If $\lambda > 1$ but not an integer, then $\Pr[X = k]$ is maximum for the largest integer smaller than $\lambda$; if $\lambda$ is a positive integer, then there are two maxima at $k = \lambda$ and $k = \lambda - 1$.

The Poisson distribution is also important in queueing analysis because we must assume a Poisson arrival pattern to be able to develop the queueing equations (discussed in Chapter 21). Fortunately, the assumption of Poisson arrivals is usually valid.

The way in which the Poisson distribution can be applied to arrival rate is as follows. If items arrive at a queue according to a Poisson process, this may be expressed as:

$$\Pr[k \text{ items arrive in time interval } T] = \frac{(\lambda T)^k}{k!}\mathrm{e}^{-\lambda T}$$

$$\mathrm{E}[\text{number of items to arrive in time interval } T] = \lambda T$$

$$\text{Mean arrival rate, in items per second} = \lambda$$

Arrivals occurring according to a Poisson process are often referred to as random arrivals. This is because the probability of arrival of an item in a small interval is proportional to the length of the interval, and is independent of the amount of elapsed time since the arrival of the last item. That is, when items are arriving according to a Poisson process, an item is as likely to arrive at one instant as any other, regardless of the instants at which the other items arrive.

Another interesting property of the Poisson process is its relationship to the exponential distribution. If we look at the times between arrivals of items $T_a$ (called

the interarrival times), then we find that this quantity obeys the exponential distribution:

$$\Pr[T_a < t] = 1 - e^{-\lambda t}$$
$$E[T_a] = \frac{1}{\lambda}$$

Thus, the mean interarrival time is the reciprocal of the arrival rate, as we would expect.

***NORMAL DISTRIBUTION*** The normal distribution with parameters $\mu > 0$ and $\sigma$ has the following density function (see Figure 20.3d) and distribution function:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/2\sigma^2} \qquad F(x) = \frac{1}{\sigma\sqrt{2\pi}} \int_{-\infty}^{x} e^{-(y-\mu)^2/2\sigma^2} dy$$

with

$$E[X] = \mu$$
$$\text{Var}[X] = \sigma^2$$

An important result is the central limit theorem, which states that the distribution of the average of a large number of independent random variables will be approximately normal, almost regardless of their individual distributions. One key requirement is finite mean and variance. The central limit theorem plays a key role in statistics.

## Multiple Random Variables

With two or more random variables, we are often concerned whether variations in one are reflected in the other. This subsection defines some important measures of dependence.

In general, the statistical characterization of multiple random variables requires a definition of their joint probability density function or joint probability distribution function:

Distribution: $F(x_1, x_2, \ldots x_n) = \Pr[X_1 \leq x_1, X_2 \leq x_2, \ldots, X_n \leq x_n]$

Density: $f(x_1, x_2, \ldots, x_n) = \dfrac{\partial^n}{\partial x_1 \partial x_2 \cdots \partial x_n} F(x_1, x_2, \ldots, x_n)$

Discrete distribution: $P(x_1, x_2, \ldots x_n) = \Pr[X_1 = x_1, X_2 = x_2, \ldots, X_n = x_n]$

For any two random variables $X$ and $Y$, we have

$$E[X + Y] = E[X] + E[Y]$$

Two continuous random variables $X$ and $Y$ are called (statistically) **independent** if $F(x, y) = F(x)F(y)$, and therefore $f(x, y) = f(x)f(y)$. If the random variables $X$ and $Y$ are discrete, then they are independent if $P(x, y) = P(x)P(y)$.

For independent random variables, the following relationships hold:

$$E[XY] = E[X] \times E[Y]$$
$$\text{Var}[X + Y] = \text{Var}[X] + \text{Var}[Y]$$

The **covariance** of two random variables $X$ and $Y$ is defined as follows:

$$\text{Cov}(X, Y) = \text{E}[(X - \mu_X)(Y - \mu_Y)] = \text{E}[XY] - \text{E}[X]\text{E}[Y]$$

If the variances of $X$ and $Y$ are finite, then their covariance is finite but may be positive, negative, or zero.

For finite variances of $X$ and $Y$, the **correlation coefficient** of $X$ and $Y$ is defined as:

$$r(X, Y) = \frac{\text{Cov}(X,Y)}{\sigma_X \sigma_Y} \tag{20.2}$$

We can think of this as a measure of the linear dependence between $X$ and $Y$, normalized to be relative to the amount of variability in $X$ and $Y$. The following relationship holds:

$$-1 \leq r(X, Y) \leq 1$$

It is said $X$ and $Y$ are **positively correlated** if $r(X, Y) > 0$, that $X$ and $Y$ are **negatively correlated** if $r(X, Y) < 0$, and $X$ and $Y$ are **uncorrelated** if $r(X, Y) = \text{Cov}(X, Y) = 0$. If $X$ and $Y$ are independent random variables, then they are uncorrelated and $r(X, Y) = 0$. However, it is possible for $X$ and $Y$ to be uncorrelated but not independent (see Problem 20.12).

The correlation coefficient provides a measure of the extent to which two random variables are linearly related. If the joint distribution of $X$ and $Y$ is relatively concentrated around a straight line in the $xy$-plane that has a positive slope, then $r(X, Y)$ will typically be close to 1. This indicates that a movement in $X$ will be matched by a movement of relatively similar magnitude and direction in $Y$. If the joint distribution of $X$ and $Y$ is relatively concentrated around a straight line that has a negative slope, then $r(X, Y)$ will typically be close to $-1$.

The following relationship is easily demonstrated:

$$\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y) + 2\text{Cov}(X, Y)$$

If $X$ and $Y$ have the same variance $\sigma^2$, then the preceding can be rewritten as:

$$\text{Var}(X + Y) = 2\sigma^2(1 + r(X, Y))$$

If $X$ and $Y$ are uncorrelated [$r(X, Y) = 0$], then $\text{Var}(X + Y) = 2\sigma^2$. These results easily generalize to more than two variables: Consider a set of random variables $X_1, \ldots, X_N$, such that each has the same variance $\sigma^2$. Then

$$\text{Var}\left(\sum_{i=1}^{N} X_i\right) = \sigma^2\left(N + 2\sum_i \sum_{j<i} r(i, j)\right)$$

where $r(i, j)$ is shorthand for $r(X_i, X_j)$. Using the relationship $\text{Var}(X/N) = \text{Var}(X)/N^2$, we can develop an equation for the variance of the sample mean of a set of random variables:

$$\overline{X} = \frac{1}{N}\sum_{i=1}^{N} X_i$$

$$\text{Var}(\overline{X}) = \frac{\sigma^2}{N}\left(1 + \sum_i \sum_{j<i} r(i, j)\right)$$

If the $X_i$ are mutually independent, then we have $\text{Var}(\overline{X}) = \dfrac{\sigma^2}{N}$.

## 20.3 ELEMENTARY CONCEPTS OF STOCHASTIC PROCESSES

A **stochastic process**, also called a **random process**, is a family of random variables $\{\mathbf{x}(t), t \in T\}$ indexed by a parameter $t$ over some index set $T$. Typically, the index set is interpreted as the time dimension, and $\mathbf{x}(t)$ is a function of time. Another way to say this is a stochastic process is a random variable that is a function of time. A **continuous-time stochastic process** is one in which $t$ varies continuously, typically over the nonnegative real line $\{\mathbf{x}(t), 0 \leq t < \infty\}$, although sometimes over the entire real line; whereas a **discrete-time stochastic process** is one in which $t$ takes on discrete values, typically the positive integers $\{\mathbf{x}(t), t = 1, 2, \ldots\}$, although in some cases the range is the integers from $-\infty$ to $+\infty$.

Recall a random variable is defined as a function that maps the outcome of an experiment into a given value. With that in mind, the expression $\mathbf{x}(t)$ can be interpreted in several ways:

1. A family of time functions ($t$ variable; all possible outcomes)
2. A single time function ($t$ variable; one outcome)
3. A random variable ($t$ fixed; all possible outcomes)
4. A single number ($t$ fixed; one outcome)

The specific interpretation of $\mathbf{x}(t)$ is usually clear from the context.

A word about terminology. A **continuous-value stochastic process** is one in which the random variable $\mathbf{x}(t)$ with $t$ fixed (case 3) takes on continuous values, whereas a **discrete-value stochastic process** is one in which the random variable at any time $t$ takes on a finite or countably infinite number of values. A continuous-time stochastic process may be either continuous value or discrete value, and a discrete-time stochastic process may be either continuous value or discrete value.

As with any random variable, $\mathbf{x}(t)$ for a fixed value of $t$ can be characterized by a probability distribution and a probability density. For continuous-value stochastic processes, these functions take the following form:

$$\text{Distribution function: } (x; t)F = \Pr[\mathbf{x}(t) \leq x] \quad F(-\infty; t) = 0; \quad F(\infty; t) = 1$$

$$\text{Density function: } f(x; t) = \frac{\partial}{\partial x}\mathrm{F}(x; t) \qquad F(x; t) = \int_{-\infty}^{x} f(y; t)dy \int_{-\infty}^{\infty} f(y; t)dy = 1$$

For discrete-value stochastic processes:

$$P_{\mathbf{x}(t)}(k) = \Pr[\mathbf{x}(t) = k] \qquad \sum_{\text{all } k} P_{\mathbf{x}(t)}(k) = 1$$

A full statistical characterization of a stochastic process must take into account the time variable. Using the first interpretation in the preceding list, a stochastic process $\mathbf{x}(t)$ comprises an infinite number of random variables, one for each $t$. To specify fully the statistics of the process, we would need to specify the joint probability density function of the variables $\mathbf{x}(t_1), \mathbf{x}(t_2), \ldots, \mathbf{x}(t_n)$ for all values of $n$ $(1 \leq n < \infty)$ and all possible sampling times $(t_1, t_2, \ldots, t_n)$. For our purposes, we need not pursue this topic.

## First- and Second-Order Statistics

The mean and variance of a stochastic process are defined in the usual way:

$$\mathrm{E}[\mathbf{x}(t)] = \mu(t) = \int_{-\infty}^{\infty} x f(x;t)dx \quad \text{continuous-value case}$$

$$\mathrm{E}[\mathbf{x}(t)] = \mu(t) = \sum_{\text{all } k} k \Pr[x(t) = k] \quad \text{discrete-value case}$$

$$\mathrm{E}[\mathbf{x}^2(t)] = \int_{-\infty}^{\infty} x^2 f(x;t)dx \quad \text{continuous-value case}$$

$$\mathrm{E}[\mathbf{x}^2(t)] = \sum_{\text{all } k} k^2 \Pr[x(t) = k] \quad \text{discrete-value case}$$

$$Var[\mathbf{x}(t)] = \sigma^2_{\mathbf{x}(t)} = \mathrm{E}[(\mathbf{x}(t) - \mu(t))^2] = \mathrm{E}[\mathbf{x}^2(t)] - \mu^2(t)$$

Note that, in general, the mean and variance of a stochastic process are functions of time. An important concept for our discussion is the **autocorrelation function** $R(t_1, t_2)$, which is the joint moment of the random variables $\mathbf{x}(t_1)$ and $\mathbf{x}(t_2)$:

$$R(t_1, t_2) = \mathrm{E}[\mathbf{x}(t_1)\mathbf{x}(t_2)]$$

As with the correlation function for two random variables introduced earlier, the autocorrelation is a measure of the relationship between the two time instances of a stochastic process. A related quantity is the **autocovariance**:

$$C(t_1, t_2) = \mathrm{E}[(\mathbf{x}(t_1) - \mu(t_1))(\mathbf{x}(t_2) - \mu(t_2))] = R(t_1, t_2) - \mu(t_1)\mu(t_2) \qquad \textbf{(20.3)}$$

Note the variance of $\mathbf{x}(t)$ is given by:

$$\mathrm{Var}[\mathbf{x}(t)] = C(t, t) = R(t, t) - \mu^2(t)$$

Finally, the **correlation coefficient** (see Equation 20.2) of $\mathbf{x}(t_1)$ and $\mathbf{x}(t_2)$ is called the normalized autocorrelation function of the stochastic process and can be expressed as:

$$\rho(t_1, t_2) = \frac{\mathrm{E}[(x(t_1) - \mu(t_1))(x(t_2) - \mu(t_2))]}{\sigma_1\sigma_2} = \frac{C(t_1, t_2)}{\sigma_1\sigma_2} \qquad \textbf{(20.4)}$$

Unfortunately, some texts and some of the literature refer to $\rho(t_1, t_2)$ as the autocorrelation function, so the reader must beware.

## Stationary Stochastic Processes

In general terms, a **stationary stochastic process** is one in which the probability characteristics of the process do not vary as a function of time. There are several different precise definitions of this concept, but the one of most interest here is the concept of

**wide sense stationary**. A process is stationary in the wide sense (or weakly stationary) if its expected value is a constant and its autocorrelation function depends only on the time difference:

$$\begin{aligned} \mathrm{E}[\mathbf{x}(t)] &= \mu \\ R(t, t+\tau) &= R(t+\tau, t) = R(\tau) = R(-\tau) \quad \text{for all } t \end{aligned}$$

From these equalities, the following can be derived:

$$\begin{aligned} \mathrm{Var}[\mathbf{x}(t)] &= R(t,t) - \mu^2(t) = R(0) - \mu^2 \\ C(t, t+\tau) &= R(t, t+\tau) - \mu(t)\,\mu(t+\tau) = R(\tau) - \mu^2 = C(\tau) \end{aligned}$$

An important characteristic of $R(\tau)$ is that it measures the degree of dependence of one time instant of a stochastic process on other time instants. If $R(\tau)$ goes to zero exponentially fast as $\tau$ becomes large, then there is little dependence of one instant of a stochastic process on instants far removed in time. Such a process is called a **short memory process**, whereas if $R(\tau)$ remains substantial for large values of $\tau$ (decays to zero at a slower than exponential rate), the stochastic process is a **long memory process**.

## Spectral Density

The **power spectrum**, or **spectral density**, of a stationary random process is the Fourier transform of its autocorrelation function:

$$S(w) = \int_{-\infty}^{\infty} R(\tau) e^{-\mathrm{j}w\tau} d\tau$$

where $w$ is the frequency in radians ($w = 2\pi f$) and $\mathrm{j} = \sqrt{-1}$.

For a deterministic time function, the spectral density gives the distribution frequency of the power of the signal. For a stochastic process, $S(w)$ is the average density of power in the frequency components of $\mathbf{x}(t)$ in the neighborhood of $w$. Recall that one interpretation of $\mathbf{x}(t)$ is that of a single time function ($t$ variable; one outcome). For that interpretation, the time function, as with any time function, is made up of a summation of frequency components, and its spectral density gives the relative power contributed by each component. If we view $\mathbf{x}(t)$ as a family of time functions ($t$ variable; all possible outcomes), then the spectral density gives the average power in each frequency component, averaged over all possible time functions $\mathbf{x}(t)$.

The Fourier inversion formula gives the time function in terms of its Fourier transform:

$$R(\tau) = \frac{1}{2\pi} \int_{-\infty}^{\infty} S(w) e^{\mathrm{j}w\tau} dw$$

With $\tau = 0$, the preceding yields:

$$\frac{1}{2\pi} \int_{-\infty}^{\infty} S(w)\, dw = R(0) = \mathrm{E}[|\mathbf{x}(t)|^2]$$

Thus, the total area under $S(w)/2\pi$ equals the average power of the process $\mathbf{x}(t)$. Also note:

$$S(0) = \int_{-\infty}^{\infty} R(\tau)d\tau$$

$S(0)$ represents the direct-current (dc) component of the power spectrum and corresponds to the integral of the autocorrelation function. This component will be finite only if $R(\tau)$ decays as $\tau \rightarrow \infty$ sufficiently rapidly for the integral of $R(\tau)$ to be finite.

We can also express the power spectrum for a stochastic process that is defined at discrete points in time (discrete-time stochastic process). In this case, we have:

$$S(w) = \sum_{k=-\infty}^{\infty} R(k)e^{-\mathrm{j}kw} \quad S(0) = \sum_{k=-\infty}^{\infty} R(k)$$

Again, $S(0)$ represents the dc component of the power spectrum and corresponds to the infinite sum of the autocorrelation function. This component will be finite only if $R(\tau)$ decays as $\tau \rightarrow \infty$ sufficiently rapidly for the summation to be finite.

Table 20.2 shows some interesting correspondences between the autocorrelation function and the power spectral density.

## Independent Increments

A continuous-time stochastic process $\{\mathbf{x}(t), 0 \leq t < \infty\}$ is said to have independent increments if $\mathbf{x}(0) = 0$ and, for all choices of indexes $t_0 < t_1 < \ldots < t_n$, the $n$ random variables

$$\mathbf{x}(t_1) - \mathbf{x}(t_0), \mathbf{x}(t_2) - \mathbf{x}(t_1), \ldots, \mathbf{x}(t_n) - \mathbf{x}(t_{n-1})$$

are independent. Thus, the amount of "movement" in a stochastic process in one time interval is independent of the movement in any other nonoverlapping time interval. The process is said to have stationary independent increments if, in addition, $\mathbf{x}(t_2 + h) - \mathbf{x}(t_1 + h)$ has the same distribution as $\mathbf{x}(t_2) - \mathbf{x}(t_1)$ for all choices of $t_2 > t_1$ and every $h > 0$.

Two properties of processes with stationary independent increments are noteworthy. If $\mathbf{x}(t)$ has stationary independent increments and $\mathrm{E}[\mathbf{x}(t)] = \mu(t)$ is a continuous function of time, then $\mu(t) = \mathrm{a} + \mathrm{b}t$, where a and b are constants. Also, if $\mathrm{Var}[\mathbf{x}(t) - \mathbf{x}(0)]$ is a continuous function of time, then for all $s$, $\mathrm{Var}[\mathbf{x}(s + t) - \mathbf{x}(s)] = \sigma^2 t$, where $\sigma^2$ is a constant.

**Table 20.2** Autocorrelation Functions and Spectral Densities

| Stationary Random Process | Autocorrelation Function | Power Spectral Density |
|---|---|---|
| $X(t)$ | $R_X(\tau)$ | $S_X(w)$ |
| $aX(t)$ | $a^2R_X(\tau)$ | $\mathrm{a}^2S_X(w)$ |
| $X'(t)$ | $-\mathrm{d}^2R_X(\tau)/\mathrm{d}\tau^2$ | $w^2S_X(w)$ |
| $X^{(n)}(t)$ | $(-1)^n\mathrm{d}^{2n}R_X(\tau)/\mathrm{d}\tau^{2n}$ | $w^{2n}S_X(w)$ |
| $X(t)\exp(\mathrm{j}w_0t)$ | $\exp(\mathrm{j}w_0\tau)R_X(\tau)$ | $S_X(w - w_0)$ |

Two processes that play a central role in the theory of stochastic processes, the Brownian motion process and the Poisson process, have independent increments. A brief introduction to both follows.

***BROWNIAN MOTION PROCESS*** Brownian motion is the random movement of microscopic particles suspended in a liquid or gas, caused by collisions with molecules of the surrounding medium. This physical phenomenon is the basis for the definition of the Brownian motion stochastic process, also known as the Wiener process and the Wiener-Levy process.

Let us consider the function $B(t)$ for a particle in Brownian motion as denoting the displacement from a starting point in one dimension after time $t$. Consider the net movement of the particle in a time interval $(s, t)$, which is long compared to the time between impacts. The quantity $B(t) - B(s)$ can be viewed as the sum of a large number of small displacements. By the central limit theorem, we can assume this quantity has a normal probability distribution.

If we assume the medium is in equilibrium, it is reasonable to assume the net displacement depends only on the length of the time interval and not on the time at which the interval begins. That is, the probability distribution of $B(t) - B(s)$ should be the same as $B(t + h) - B(s + h)$ for any $h > 0$. Finally, if the motion of the particle is due entirely to frequent random collisions, then the net displacements in nonoverlapping time intervals should be independent, and therefore $B(t)$ has independent increments.

With the foregoing reasoning in mind, we define a Brownian motion process $B(t)$ as one that satisfies the following conditions:

1. $\{B(t), 0 \leq t < \infty\}$ has stationary independent increments.
2. For every $t > 0$, the random variable $B(t)$ has a normal distribution.
3. For all $t > 0$, $\mathrm{E}[B(t)] = 0$.
4. $B(0) = 0$.

The probability density of a Brownian motion process has the form:

$$f_B(x, t) = \frac{1}{\sigma\sqrt{2\pi t}} \mathrm{e}^{-x^2/2\sigma^2 t}$$

From this we have:

$$\mathrm{Var}[B(t)] = t; \qquad \mathrm{Var}[B(t) - B(s)] = |t - s|$$

Another important quantity is the autocorrelation of $B(t)$, expressed as $R_B(t_1, t_2)$. We derive this quantity in the following way. First, observe that for $t_4 > t_3 > t_2 > t_1$:

$$\begin{aligned}\mathrm{E}[(B(t_4) - B(t_3))(B(t_2) - B(t_1))] &= \mathrm{E}[B(t_4) - B(t_3)] \times \mathrm{E}[B(t_2) - B(t_1)] \\ &= (\mathrm{E}[B(t_4)] - \mathrm{E}[B(t_3)]) \times (\mathrm{E}[B(t_2)] - \mathrm{E}[B(t_1)]) \\ &= (0 - 0) \times (0 - 0) = 0\end{aligned}$$

The first line of the preceding equation is true because the two intervals are nonoverlapping, and therefore the quantities $(B(t_4) - B(t_3))$ and $(B(t_2) - B(t_1))$ are independent, due to the assumption of independent increments. Recall that for

independent random variables $X$ and $Y$, $\mathrm{E}[XY] = \mathrm{E}[X]\mathrm{E}[Y]$. Now consider the two intervals $(0, t_1)$ and $(t_1, t_2)$, for $0 < t_1 < t_2$. These are nonoverlapping intervals, so

$$\begin{aligned} 0 &= \mathrm{E}[(B(t_2) - B(t_1))(B(t_1) - B(0))] \\ &= \mathrm{E}[(B(t_2) - B(t_1))B(t_1)] \\ &= \mathrm{E}[B(t_2)B(t_1)] - \mathrm{E}[B^2(t_1)] \\ &= \mathrm{E}[B(t_2)B(t_1)] - \mathrm{Var}[B(t_1)] \\ &= \mathrm{E}[B(t_2)B(t_1)] - t_1 \end{aligned}$$

Therefore,

$$R_B(t_1, t_2) = \mathrm{E}[B(t_1)B(t_2)] = t_1 \text{ where } t_1 < t_2$$

In general, then, the autocorrelation of $B(t)$ can be expressed as $R_B(t, s) = \min[t, s]$. Because $B(t)$ has zero mean, the autocovariance is the same as the autocorrelation. Thus, $C_B(t, s) = \min[t, s]$.

For any $t \geq 0$ and $\delta > 0$, the increment of a Brownian motion process, $B(t + \delta) - B(t)$, is normally distributed with mean 0 and variance $\delta$. Thus,

$$\Pr[(B(t + \delta) - B(t)) \leq x] = \frac{1}{\sqrt{2\pi\delta}} \int_{-\infty}^{x} \mathrm{e}^{-y^2/2\delta} dy \tag{20.5}$$

Note this distribution is independent of $t$ and depends only on $\delta$, consistent with the fact that $B(t)$ has stationary increments.

One useful way to visualize the Brownian motion process is as the limit of a discrete-time process. Let us consider a particle performing a random walk on the real line. At small time intervals $\tau$, the particle randomly jumps a small distance $\delta$ to the left or right. We denote the position of the particle at time $k\tau$ as $X_\tau(k\tau)$. If positive and negative jumps are equally likely, then $X_\tau((k + 1)\tau)$ equals $X_\tau(k\tau) + \delta$ or $X_\tau(k\tau) - \delta$ with equal probability. If we assume $X_\tau(0) = 0$, then the position of the particle at time $t$ is

$$X_\tau(t) = \delta(Y_1 + Y_2 \ldots + Y_{\lfloor t/\tau \rfloor})$$

where $Y_1, Y_2, \ldots$ are independent random variables with equal probability of being 1 or $-1$ and $\lfloor t/\tau \rfloor$ denotes the largest integer less than or equal to $t/\tau$. It is convenient to normalize the step length $\delta$ as $\sqrt{\tau}$ so

$$X_\tau(t) = \sqrt{\tau}(Y_1 + Y_2 \ldots + Y_{\lfloor t/\tau \rfloor})$$

By the central limit theorem, for fixed $t$, if $\tau$ is sufficiently small then the sum in the preceding equation consists of many random variables, and therefore the distribution of $X_\tau(t)$ is approximately normal with mean 0 and variance $t$, because the $Y_i$ have mean 0 and variance 1. Also, for fixed $t$ and $h$, if $\tau$ is sufficiently small, then $X_\tau(t + h) - X_\tau(t)$ is approximately normal with mean 0 and variance $h$. Finally, we note the increments of $X_\tau(t)$ are independent. Thus, $X_\tau(t)$ is a discrete-time function that approximates Brownian motion. If we divide the time axis more finely, we improve the approximation. In the limit, this becomes a continuous-time Brownian motion process.

***POISSON AND RELATED PROCESSES*** Recall that for random arrivals in time, we have the Poisson distribution:

$$\Pr[k \text{ items arrive in time interval } T] = \frac{(\lambda T)^k}{k!} e^{-\lambda T}$$

We can define a **Poisson counting process** $\{N(t), t \geq 0\}$ as follows:

1. $N(t)$ has stationary independent increments.
2. $N(0) = 0$.
3. For $0 < t_1 < t_2$, the quantity $N(t_2) - N(t_1)$ equals the number of points in the interval $(t_1, t_2)$ and is Poisson distributed with mean $\lambda(t_1 - t_2)$.

Then we have the following probability functions for $N(t)$:

$$\Pr[N(t) = k] = \frac{(\lambda t)^k}{k!} e^{-\lambda t}$$

$$\mathrm{E}[N(t)] = \mathrm{Var}[N(t)] = \lambda t$$

Clearly, $N(t)$ is not stationary, because its mean is a function of time. Every time function of this stochastic process (one outcome) has the form of an increasing staircase with steps equal to 1, occurring at the random points $t_i$. Figure 20.4a gives an example of $N(t)$ for a specific outcome.

A stationary process related to the Poisson counting process is the **Poisson increment process**. For a Poisson counting process $N(t)$ with mean $\lambda t$, and for a constant $L$ ($L > 0$), we can define the Poisson increment process $\mathrm{X}(t)$ as follows:

$$X(t) = \frac{N(t + L) - N(t)}{L}$$

$X(t)$ equals $k/L$, where $k$ is the number of points in the interval $(t, t + L)$. The increment process derived from the counting process in Figure 20.4a is shown in Figure 20.4b. The following relationship holds.

$$\mathrm{E}[X(t)] = \frac{1}{L}\mathrm{E}[N(t + L)] - \frac{1}{L}\mathrm{E}[N(t)] = \lambda$$

With a constant mean, $X(t)$ is a wide-sense stationary process and therefore has an autocorrelation function of a single variable, $R(\tau)$. It can be shown that this function is:

$$R(\tau) = \begin{cases} \lambda^2 & |\tau| > L \\ \lambda^2 + \dfrac{\lambda^2}{L}\left(1 - \dfrac{|\tau|}{L}\right) & |\tau| < L \end{cases} \tag{20.6}$$

Thus, the correlation is greatest if the two time instants are within the interval length of each other, and it is a small constant value for greater time differences.

## Ergodicity

For a stochastic process $\mathbf{x}(t)$, there are two types of "averaging" functions that can be performed: ensemble averages and time averages.

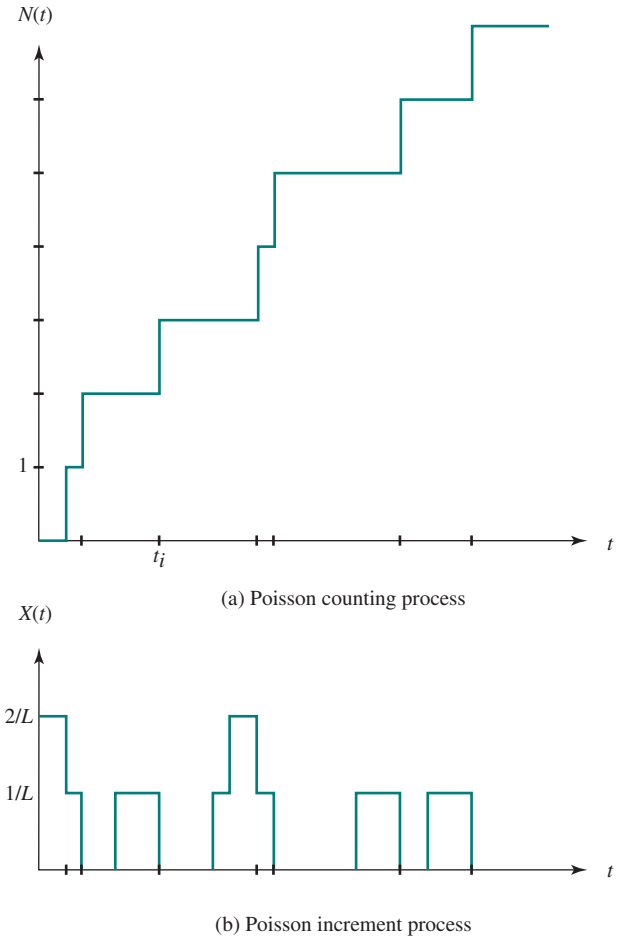



**Figure 20.4 Poisson Processes**

First, consider **ensemble averages**. For a constant value of $t$, $\mathbf{x}(t)$ is a single random variable with a mean, variance, and other distributional properties. For a given constant value C of $t$, the following measures exist:

$$\mathrm{E}[\mathbf{x}(\mathrm{C})] = \mu_{\mathbf{x}}(\mathrm{C}) = \int_{-\infty}^{\infty} x f(x; \mathrm{C}) dx \qquad \text{continuous-value case}$$

$$\mathrm{E}[\mathbf{x}(\mathrm{C})] = \mu_{\mathbf{x}}(\mathrm{C}) = \sum_{\text{all } k} k \Pr[\mathbf{x}(\mathrm{C}) = k] \quad \text{discrete-value case}$$

$$Var[\mathbf{x}(\mathrm{C})] = \sigma^2_{\mathbf{x}(\mathrm{C})} = \mathrm{E}[(\mathbf{x}(\mathrm{C}) - \mu_{\mathbf{x}}(\mathrm{C}))^2] = \mathrm{E}[\mathbf{x}(\mathrm{C})^2] - \mu^2_{\mathbf{x}}(\mathrm{C})$$

Each of these quantities is calculated over all values of $\mathbf{x}(t)$ for all possible outcomes. For a given random variable, the set of all possible outcomes is called an *ensemble*, and hence these are referred to as ensemble averages.

For time averages, consider a single outcome of $\mathbf{x}(t)$. This is a single deterministic function of $t$. Looking at $\mathbf{x}(t)$ in this way, we can consider what is the average value of the function over time. This **time average** is generally expressed as follows:

$$M_T = \frac{1}{2T}\int_{-T}^{T} \mathbf{x}(t)dt \quad \text{continuous-time case}$$

$$M_T = \frac{1}{T}\sum_{t=1}^{T} \mathbf{x}(t) \quad \text{discrete-time case}$$

Note $M_T$ is a random variable, because the calculation of $M_T$ for a single time function is a calculation for a single outcome.

A stationary process is said to be **ergodic** if time averages equal ensemble averages. Because $\mathrm{E}[\mathbf{x}(t)]$ is a constant for a stationary process, we have

$$\mathrm{E}[M_T] = \mathrm{E}[\mathbf{x}(t)] = \mu$$

Thus, we can say that a stationary process is ergodic if

$$\lim_{T\to\infty} \mathrm{Var}(M_T) = 0$$

In words, as the time average is taken over larger and larger time intervals, the value of the time average approaches the ensemble average.

The conditions under which a stochastic process is ergodic are beyond the scope of this book, but the assumption is generally made. Indeed, the assumption of ergodicity is essential to almost any mathematical model used for stationary stochastic processes. The practical importance of ergodicity is that in most cases, one does not have access to the ensemble of outcomes of a stochastic process or even to more than one outcome. Thus, the only means of obtaining estimates of the probabilistic parameters of the stochastic process is to analyze a single time function over a long period of time.

## 20.4 PROBLEMS

**20.1** You are asked to play a game in which I hide a prize in one of three boxes (with equal probability for all three boxes) while you are out of the room. When you return, you have to guess which box hides the prize. There are two stages to the game. First, you indicate one of the three boxes as your choice. As soon as you do that, I open the lid of one of the other two boxes and I will always open an empty box. I can do this because I know where the prize is hidden. At this point, the prize must be in the box that you have chosen or in the other unopened box. You are now free to stick with your original choice or to switch to the other unopened box. You win the prize if your final selection is the box containing the prize. What is your best strategy? Should you (a) stay with your original choice, (b) switch to the other box, or (c) do either because it does not matter?

**20.2** A patient has a test for some disease that comes back positive (indicating he has the disease). You are told that

- the accuracy of the test is 87% (i.e., if a patient has the disease, 87% of the time, the test yields the correct result, and if the patient does not have the disease, 87% of the time, the test yields the correct result)
- the incidence of the disease in the population is 1%

Given that the test is positive, how probable is it that the patient really has the disease?

**20.3** A taxicab was involved in a fatal hit-and-run accident at night. Two cab companies, the Green and the Blue, operate in the city. You are told that:

- 85% of the cabs in the city are Green and 15% are Blue
- A witness identified the cab as Blue

The court tested the reliability of the witness under the same circumstances that existed on the night of the accident and concluded that the witness was correct in identifying the color of the cab 80% of the time. What is the probability that the cab involved in the incident was Blue rather than Green?

**20.4** The birthday paradox is a famous problem in probability that can be stated as follows: What is the minimum value of $K$ such that the probability is greater than 0.5 that at least two people in a group of $K$ people have the same birthday? Ignore February 29 and assume each birthday is equally likely. We will do the problem in two parts:

**a.** Define $Q(K)$ as the probability that there are no duplicate birthdays in a group of $K$ people. Derive a formula for $Q(K)$. *Hint:* First determine the number of different ways, $N$, that we can have $K$ values with no duplicates.

**b.** Define $P(K)$ as the probability that there is at least one duplicate birthday in a group of $K$ people. Derive this formula. What is the minimum value of $K$ such that $P(K) > 0.5$? It may help to plot $P(K)$.

**20.5** A pair of fair dice (the probability of each outcome is 1/6) is thrown. Let $X$ be the maximum of the two numbers that comes up.

**a.** Find the distribution of $X$.

**b.** Find the expectation $\mathrm{E}[X]$, the variance $\mathrm{Var}[X]$, and the standard deviation $\sigma_X$.

**20.6** A player tosses a fair die. If a prime number greater than 1 appears, he wins that number of dollars, but if a nonprime number appears, he loses that number of dollars.

**a.** Denote the player's gain or loss on one toss by the random variable $X$. Enumerate the distribution of $X$.

**b.** Is the game fair (i.e., $\mathrm{E}[X] = 0$)?

**20.7** In the carnival game known as *chuck-a-luck*, a player pays an amount $E$ as an entrance fee, selects a number between one and six, then rolls three dice. If all three dice show the number selected, the player is paid four times the entrance fee; if two dice show the number, the player is paid three times the entrance fee; and if only one die shows the number, the player is paid twice the entrance fee. If the selected number does not show up, the player is paid nothing. Let $X$ denote the player's gain in a single play of this game, and assume the dice are fair.

**a.** Determine the probability function of $X$.

**b.** Compute $\mathrm{E}[X]$.

**20.8** The mean and variance of $X$ are 50 and 4, respectively. Evaluate the following:

**a.** The mean of $X^2$

**b.** The variance and standard deviation of $2X + 3$

**c.** The variance and standard deviation of $-X$

**20.9** The continuous random variable $R$ has a uniform density between 900 and 1,100, and 0 elsewhere. Find the probability that $R$ is between 950 and 1,050.

**20.10** Show that, all other things being equal, the greater the correlation coefficient of two random variables is, the greater the variance of their sum and the less the variance of their difference will be.

**20.11** Suppose $X$ and $Y$ each have only two possible values, 0 and 1. Prove if $X$ and $Y$ are uncorrelated, then they are also independent.

**20.12** Consider a random variable $X$ with the following distribution: $\Pr[X = -1] = 0.25$; $\Pr[X = 0] = 0.5$; $\Pr[X = 1] = 0.25$. Let $Y = X^2$.

**a.** Are $X$ and $Y$ independent random variables? Justify your answer.

**b.** Calculate the covariance $\mathrm{Cov}(X, Y)$.

**c.** Are $X$ and $Y$ uncorrelated? Justify your answer.

**20.13** An artificial example of a stochastic process is a deterministic signal $\mathbf{x}(t) = g(t)$. Determine the mean, variance, and autocorrelation of $\mathbf{x}(t)$.

**20.14** Suppose $\mathbf{x}(t)$ is a stochastic process with

$$\mu(t) = 3 \quad R(t_1, t_2) = 9 + 4e^{-0.2|t_1 - t_2|}$$

Determine the mean, variance, and covariance of the following random variables: $Z = \mathbf{x}(5)$ and $W = \mathbf{x}(8)$.

**20.15** Let $\{\mathbf{Z}_n\}$ be a set of uncorrelated real-valued random variables, each with a mean of 0 and a variance of 1. Define the moving average

$$\mathbf{Y}_n = \sum_{i=0}^{K} \alpha_i Z_{n-i}$$

for constants $\alpha_0, \alpha_1, \ldots, \alpha_K$. Show that $\mathbf{Y}$ is stationary and find its autocovariance function.

**20.16** Let $\mathbf{X}_n = \mathbf{A}\cos(n\lambda) + \mathbf{B}\sin(n\lambda)$ where $\mathbf{A}$ and $\mathbf{B}$ are uncorrelated random variables, each with a mean of 0 and a variance of 1. Show that $\mathbf{X}$ is stationary with a spectrum containing exactly one point.

CHAPTER 21

# QUEUEING ANALYSIS


21.1 **How Queues Behave—A Simple Example**

21.2 **Why Queueing Analysis?**

21.3 **Queueing Models**
- The Single-Server Queue
- The Multiserver Queue
- Basic Queueing Relationships
- Assumptions

21.4 **Single-Server Queues**

21.5 **Multiserver Queues**

21.6 **Examples**
- Database Server
- Calculating Percentiles
- Tightly-Coupled Multiprocessor
- A Multiserver Problem

21.7 **Queues with Priorities**

21.8 **Networks of Queues**
- Partitioning and Merging of Traffic Streams
- Queues in Tandem
- Jackson's Theorem
- Application to a Packet-Switching Network

21.9 **Other Queueing Models**

21.10 **Estimating Model Parameters**
- Sampling
- Sampling Errors

21.11 **References**

21.12 **Problems**

**Learning Objectives**

After studying this chapter, you should be able to:

- Understand the characteristic behavior of queueing systems.
- Explain the value of queueing analysis.
- Explain the key features of single-server and multiserver queues.
- Analyze single-server queueing models.
- Analyze multiserver queueing models.
- Describe the effect of priorities on queueing performance.
- Understand the key concept relating to queueing networks.
- Understand the issues involved in estimating queueing model parameters.

Queueing[1] analysis is one of the most important tools for those involved with computer and network analysis. It can be used to provide approximate answers to a host of questions, such as:

- What happens to file retrieval time when disk I/O utilization goes up?
- Does response time change if both processor speed and the number of users on the system are doubled?
- How will performance be affected if the process scheduling algorithm includes priorities?
- Which disk scheduling algorithm produces the best average performance?

The number of questions that can be addressed with a queueing analysis is endless and touches on virtually every area in computer science. The ability to make such an analysis is an essential tool for those involved in this field.

Although the theory of queueing is mathematically complex, the application of queueing theory to the analysis of performance is, in many cases, remarkably straightforward. A knowledge of elementary statistical concepts (means and standard deviations) and a basic understanding of the applicability of queueing theory is all that is required. Armed with these, the analyst can often make a queueing analysis on the back of an envelope using readily available queueing tables, or with the use of simple computer programs that occupy only a few lines of code.

The purpose of this chapter is to provide a practical guide to queueing analysis. A subset, although a very important subset, of the subject is addressed. In the final section, pointers to additional references are provided. An annex to this paper reviews some elementary concepts in probability and statistics.

This chapter provides a practical guide to queueing analysis. A subset, although a very important subset, of the subject is addressed.

[1]Two spellings are in use: queueing and queueing. The vast majority of queueing theory researchers use *queueing*. The premier journal in this field is *Queueing systems: Theory and Applications*. On the other hand, most American dictionaries and spell checkers prefer the spelling *queueing*.

## 21.1 HOW QUEUES BEHAVE—A SIMPLE EXAMPLE

Before getting into the details of queueing analysis, let us look at a crude example that will give some feel for the topic. Consider a Web server that is capable of handling an individual request in an average of 1 millisecond. In fact, to make things simple, assume that the server handles each request in exactly 1 millisecond. Now, if the rate of arriving requests is 1 per millisecond (1,000 per second), then it seems sensible to state that the server can keep up with the load.

Suppose the requests arrive at a uniform rate of exactly one request each millisecond. When a request comes in, the server immediately handles the request. Just as the server completes the current request, a new request arrives and the server goes to work again.

Now let's take a more realistic approach and suppose the average arrival rate for requests is 1 per millisecond but that there is some variability. During any given 1 millisecond period, there may be no requests, or one, or multiple requests, but the average is still 1 per millisecond. Again, common sense would seem to indicate that the server could keep up. During busy times, when lots of requests bunch up, the server can store outstanding requests in a buffer. Another way of putting this is to say that arriving requests enter a queue to await service. During quiet times, the server can catch up and clear the buffer. In this case, the interesting design issue would seem to be: How big should the buffer be?

Tables 21.1 through 21.3 give a very rough idea of the behavior of this system. In Table 21.1, we assume an average arrival rate of 500 requests per second, which is half the capacity of the server. The entries in the table show the number of requests

**Table 21.1** Queue Behavior with Normalized Arrival Rate of 0.5

| Time | Input | Output | Queue |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 1 | 88 | 88 | 0 |
| 2 | 796 | 796 | 0 |
| 3 | 1627 | 1000 | 627 |
| 4 | 51 | 678 | 0 |
| 5 | 34 | 34 | 0 |
| 6 | 966 | 966 | 0 |
| 7 | 714 | 714 | 0 |
| 8 | 1276 | 1000 | 276 |
| 9 | 494 | 769 | 0 |
| 10 | 933 | 933 | 0 |
| 11 | 107 | 107 | 0 |
| 12 | 241 | 241 | 0 |
| 13 | 16 | 16 | 0 |
| 14 | 671 | 671 | 0 |

**Table 21.1** Queue Behavior with Normalized Arrival Rate of 0.5 (*Continued*)

| Time | Input | Output | Queue |
|---|---|---|---|
| 15 | 643 | 643 | 0 |
| 16 | 812 | 812 | 0 |
| 17 | 262 | 262 | 0 |
| 18 | 218 | 218 | 0 |
| 19 | 1378 | 1000 | 378 |
| 20 | 507 | 885 | 0 |
| 21 | 15 | 15 | 0 |
| 22 | 820 | 820 | 0 |
| 23 | 1253 | 1000 | 253 |
| 24 | 307 | 559 | 0 |
| 25 | 540 | 540 | 0 |
| 26 | 190 | 190 | 0 |
| 27 | 500 | 500 | 0 |
| 28 | 96 | 96 | 0 |
| 29 | 943 | 943 | 0 |
| 30 | 105 | 105 | 0 |
| 31 | 183 | 183 | 0 |
| 32 | 447 | 447 | 0 |
| 33 | 542 | 542 | 0 |
| 34 | 166 | 166 | 0 |
| 35 | 165 | 165 | 0 |
| 36 | 490 | 490 | 0 |
| 37 | 510 | 510 | 0 |
| 38 | 877 | 877 | 0 |
| 39 | 37 | 37 | 0 |
| 40 | 163 | 163 | 0 |
| 41 | 104 | 104 | 0 |
| 42 | 42 | 42 | 0 |
| 43 | 291 | 291 | 0 |
| 44 | 645 | 645 | 0 |
| 45 | 363 | 363 | 0 |
| 46 | 134 | 134 | 0 |
| 47 | 920 | 920 | 0 |
| 48 | 1507 | 1000 | 507 |
| 49 | 598 | 1000 | 105 |
| 50 | 172 | 277 | 0 |
| **Average** | 499 | 499 | 43 |

that arrive each second, the number of requests served during that second, and the number of outstanding requests waiting in the buffer at the end of the second. After 50 seconds, the table shows an average buffer contents of 43 requests, with a peak of over 600 requests. In Table 21.2, the average arrival rate is increased to 95% of the server's capacity, that is, 950 requests per second, and the average buffer contents rises to 1859. This seems a little surprising: the arrival rate has gone up by less than a

**Table 21.2** Queue Behavior with Normalized Arrival Rate of 0.95

| Time | Input | Output | Queue |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 1 | 167 | 167 | 0 |
| 2 | 1512 | 1000 | 512 |
| 3 | 3091 | 1000 | 2603 |
| 4 | 97 | 1000 | 1700 |
| 5 | 65 | 1000 | 765 |
| 6 | 1835 | 1000 | 1600 |
| 7 | 1357 | 1000 | 1957 |
| 8 | 2424 | 1000 | 3381 |
| 9 | 939 | 1000 | 3320 |
| 10 | 1773 | 1000 | 4093 |
| 11 | 203 | 1000 | 3296 |
| 12 | 458 | 1000 | 2754 |
| 13 | 30 | 1000 | 1784 |
| 14 | 1275 | 1000 | 2059 |
| 15 | 1222 | 1000 | 2281 |
| 16 | 1543 | 1000 | 2824 |
| 17 | 498 | 1000 | 2322 |
| 18 | 414 | 1000 | 1736 |
| 19 | 2618 | 1000 | 3354 |
| 20 | 963 | 1000 | 3317 |
| 21 | 29 | 1000 | 2346 |
| 22 | 1558 | 1000 | 2904 |
| 23 | 2381 | 1000 | 4285 |
| 24 | 583 | 1000 | 3868 |
| 25 | 1026 | 1000 | 3894 |
| 26 | 361 | 1000 | 3255 |
| 27 | 950 | 1000 | 3205 |
| 28 | 182 | 1000 | 2387 |
| 29 | 1792 | 1000 | 3179 |
| 30 | 200 | 1000 | 2379 |

**Table 21.2** Queue Behavior with Normalized Arrival Rate of 0.95 (*Continued*)

| Time | Input | Output | Queue |
|---|---|---|---|
| 31 | 348 | 1000 | 1727 |
| 32 | 849 | 1000 | 1576 |
| 33 | 1030 | 1000 | 1606 |
| 34 | 315 | 1000 | 921 |
| 35 | 314 | 1000 | 235 |
| 36 | 931 | 1000 | 166 |
| 37 | 969 | 1000 | 135 |
| 38 | 1666 | 1000 | 801 |
| 39 | 70 | 871 | 0 |
| 40 | 310 | 310 | 0 |
| 41 | 198 | 198 | 0 |
| 42 | 80 | 80 | 0 |
| 43 | 553 | 553 | 0 |
| 44 | 1226 | 1000 | 226 |
| 45 | 690 | 916 | 0 |
| 46 | 255 | 255 | 0 |
| 47 | 1748 | 1000 | 748 |
| 48 | 2863 | 1000 | 2611 |
| 49 | 1136 | 1000 | 2747 |
| 50 | 327 | 1000 | 2074 |
| **Average** | 948 | 907 | 1859 |

factor of 2, but the average buffer contents has gone up by more than a factor of 40. In Table 21.3, the average arrival rate is increased slightly, to 99% of capacity, which yields an average buffer contents of 2583. Thus, a tiny increase in average arrival rate results in an increase of almost 40% in the average buffer contents.

This crude example suggests that the behavior of a system with a queue may not accord with our intuition.

**Table 21.3** Queue Behavior with Normalized Arrival Rate of 0.99

| Time | Input | Output | Queue |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 1 | 174 | 174 | 0 |
| 2 | 1576 | 1000 | 576 |
| 3 | 3221 | 1000 | 2797 |
| 4 | 101 | 1000 | 1898 |
| 5 | 67 | 1000 | 965 |
| 6 | 1913 | 1000 | 1878 |

| Time | Input | Output | Queue |
|---|---|---|---|
| 7 | 1414 | 1000 | 2292 |
| 8 | 2526 | 1000 | 3818 |
| 9 | 978 | 1000 | 3796 |
| 10 | 1847 | 1000 | 4643 |
| 11 | 212 | 1000 | 3855 |
| 12 | 477 | 1000 | 3332 |
| 13 | 32 | 1000 | 2364 |
| 14 | 1329 | 1000 | 2693 |
| 15 | 1273 | 1000 | 2966 |
| 16 | 1608 | 1000 | 3574 |
| 17 | 519 | 1000 | 3093 |
| 18 | 432 | 1000 | 2525 |
| 19 | 2728 | 1000 | 4253 |
| 20 | 1004 | 1000 | 4257 |
| 21 | 30 | 1000 | 3287 |
| 22 | 1624 | 1000 | 3911 |
| 23 | 2481 | 1000 | 5392 |
| 24 | 608 | 1000 | 5000 |
| 25 | 1069 | 1000 | 5069 |
| 26 | 376 | 1000 | 4445 |
| 27 | 990 | 1000 | 4435 |
| 28 | 190 | 1000 | 3625 |
| 29 | 1867 | 1000 | 4492 |
| 30 | 208 | 1000 | 3700 |
| 31 | 362 | 1000 | 3062 |
| 32 | 885 | 1000 | 2947 |
| 33 | 1073 | 1000 | 3020 |
| 34 | 329 | 1000 | 2349 |
| 35 | 327 | 1000 | 1676 |
| 36 | 970 | 1000 | 1646 |
| 37 | 1010 | 1000 | 1656 |
| 38 | 1736 | 1000 | 2392 |
| 39 | 73 | 1000 | 1465 |
| 40 | 323 | 1000 | 788 |
| 41 | 206 | 994 | 0 |
| 42 | 83 | 83 | 0 |
| 43 | 576 | 576 | 0 |
| 44 | 1277 | 1000 | 277 |

**Table 21.3** Queue Behavior with Normalized Arrival Rate of 0.99 (*Continued*)

| Time | Input | Output | Queue |
|---|---|---|---|
| 45 | 719 | 996 | 0 |
| 46 | 265 | 265 | 0 |
| 47 | 1822 | 1000 | 822 |
| 48 | 2984 | 1000 | 2806 |
| 49 | 1184 | 1000 | 2990 |
| 50 | 341 | 1000 | 2331 |
| **Average** | 988 | 942 | 2583 |

## 21.2 WHY QUEUEING ANALYSIS?

There are many cases when it is important to be able to project the effect of some change in a design: Either the load on a system is expected to increase, or a design change is contemplated. For example, an organization supports a number of terminals, personal computers, and workstations on a 100-Mbps local area network (LAN). An additional department in the building is to be cut over onto the network. Can the existing LAN handle the increased workload, or would it be better to provide a second LAN with a bridge between the two? There are other cases in which no facility exists but, on the basis of expected demand, a system design needs to be created. For example, a department intends to equip all of its personnel with a personal computer and to configure these into a LAN with a file server. Based on experience elsewhere in the company, the load generated by each PC can be estimated.

The concern is system performance. In an interactive or real-time application, often the parameter of concern is response time. In other cases, throughput is the principal issue. In any case, projections of performance are to be made on the basis of existing load information, or on the basis of estimated load for a new environment. A number of approaches are possible:

1. Do an after-the-fact analysis based on actual values.
2. Make a simple projection by scaling up from existing experience to the expected future environment.
3. Develop an analytic model based on queueing theory.
4. Program and run a simulation model.

Option 1 is no option at all: We will wait and see what happens. This leads to unhappy users and to unwise purchases. Option 2 sounds more promising. The analyst may take the position that it is impossible to project future demand with any degree of certainty. Therefore, it is pointless to attempt some exact modeling procedure. Rather, a rough-and-ready projection will provide ballpark estimates. The problem with this approach is that the behavior of most systems under a changing load is not what one would intuitively expect, as Section 21.1 suggests. If there is an environment in which there is a shared facility (e.g., a network, a transmission line, a time-sharing system), then the performance of that system typically responds in an exponential way to increases in demand.

Figure 21.1 is a representative example. The upper line shows what typically happens to user response time on a shared facility as the load on that facility increases. The load is expressed as a fraction of capacity. Thus, if we are dealing with an input from a disk that is capable of transferring 1,000 blocks per second, then a load of 0.5 represents a transfer of 500 blocks per second, and the response time is the amount of time it takes to retransmit any incoming block. The lower line is a simple projection based on a knowledge of the behavior of the system up to a load of 0.5. Note while things appear rosy when the simple projection is made, performance on the system will in fact collapse beyond a load of about 0.8–0.9.

Thus, a more exact prediction tool is needed. Option 3 is to make use of an analytic model, which is one that can be expressed as a set of equations that can be solved to yield the desired parameters (response time, throughput, etc.). For computer, operating system, and networking problems, and indeed for many practical real-world problems, analytic models based on queueing theory provide a reasonably good fit to reality. The disadvantage of queueing theory is that a number of simplifying assumptions must be made to derive equations for the parameters of interest.

The final approach is a simulation model. Here, given a sufficiently powerful and flexible simulation programming language, the analyst can model reality in great detail and avoid making many of the assumptions required of queueing theory. However, in most cases, a simulation model is not needed or at least is not advisable as a first step in the analysis. For one thing, both existing measurements and projections of future load carry with them a certain margin of error. Thus, no matter how good

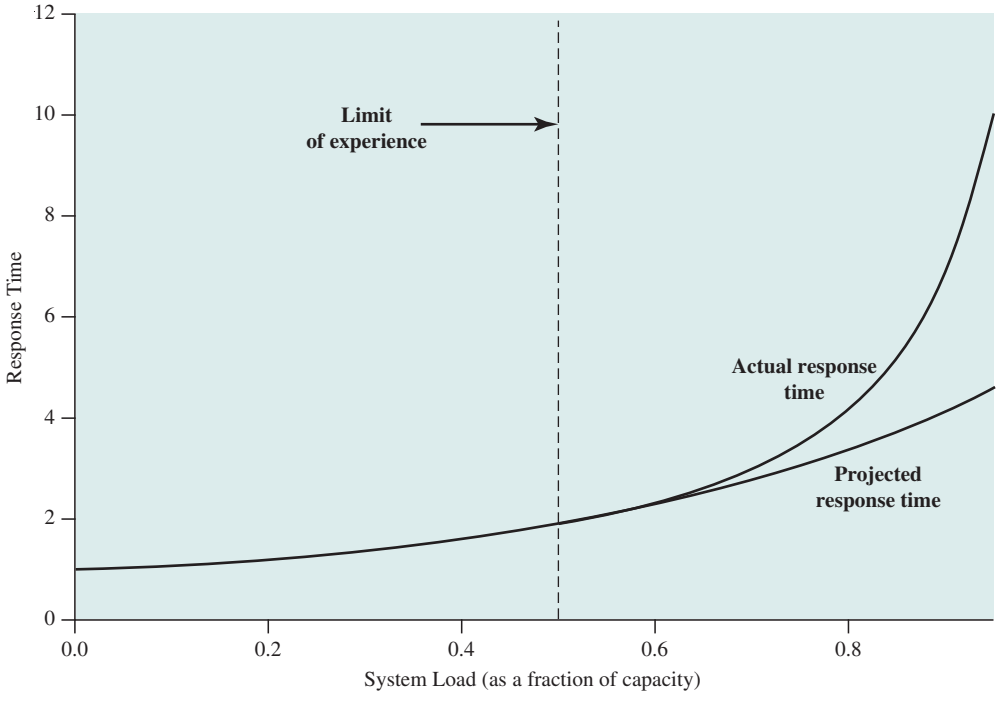



**Figure 21.1** **Projected Versus Actual Response Time**

the simulation model, the value of the results is limited by the quality of the input. For another, despite the many assumptions required of queueing theory, the results that are produced often come quite close to those that would be produced by a more careful simulation analysis. Furthermore, a queueing analysis can literally be accomplished in a matter of minutes for a well-defined problem, whereas simulation exercises can take days, weeks, or longer to program and run.

Accordingly, it behooves the analyst to master the basics of queueing theory.

## 21.3 QUEUEING MODELS

### The Single-Server Queue

The simplest queueing system is depicted in Figure 21.2. The central element of the system is a server, which provides some service to items. Items from some population of items arrive at the system to be served. If the server is idle, an item is served immediately. Otherwise, an arriving item joins a waiting line.[2] When the server has completed serving an item, the item departs. If there are items waiting in the queue, one is immediately dispatched to the server. The server in this model can represent anything that performs some function or service for a collection of items. For example, a processor provides service to processes, a transmission line provides a transmission service to packets or frames of data, and an I/O device provides a read or write service for I/O requests.

***QUEUE PARAMETERS*** Figure 21.2 also illustrates some important parameters associated with a queueing model. Items arrive at the facility at some average rate $\lambda$ (items arriving per second). Some examples of items arriving include packets arriving at a router and calls arriving at a telephone exchange. At any given time, a certain number of items will be waiting in the waiting line (zero or more); the average number waiting is $w$, and the mean time that an item must wait is $T_w$. $T_w$ is averaged

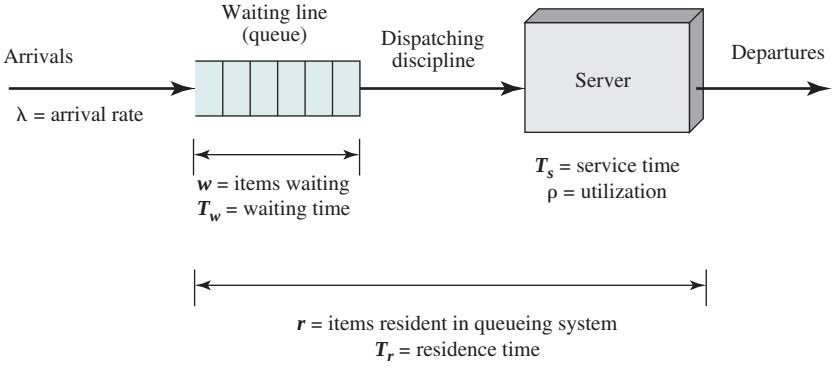



**Figure 21.2 Queueing System Structure and Parameters for Single-Server Queue**

[2]The waiting line is referred to as a queue in some treatments in the literature; it is also common to refer to the entire system as a queue. Unless otherwise noted, we use the term *queue* to mean waiting line.

over all incoming items, including those that do not wait at all. The server handles incoming items with an average service time $T_s$; this is the time interval between the dispatching of an item to the server and the departure of that item from the server. Utilization, $\rho$, is the fraction of time that the server is busy, measured over some interval of time. Finally, two parameters apply to the system as a whole. The average number of items resident in the system, including the item being served (if any) and the items waiting (if any), is $r$; and the average time that an item spends in the system, waiting and being served, is $T_r$; we refer to this as the *mean residence time*.[3]

If we assume the capacity of the queue is infinite, then no items are ever lost from the system; they are just delayed until they can be served. Under these circumstances, the departure rate equals the arrival rate. As the arrival rate, which is the rate of traffic passing through the system, increases, the utilization increases and with it, congestion. The queue becomes longer, increasing waiting time. At $\rho = 1$, the server becomes saturated, working 100% of the time. So long as utilization is less than 100%, the server can keep up with arrivals, so the average departure rate equals the average arrival rate. Once the server is saturated, working 100% of the time, the departure rate remains constant, no matter how great the arrival rate becomes. Thus, the theoretical maximum input rate that can be handled by the system is:

$$\lambda_{\max} = \frac{1}{T_s}$$

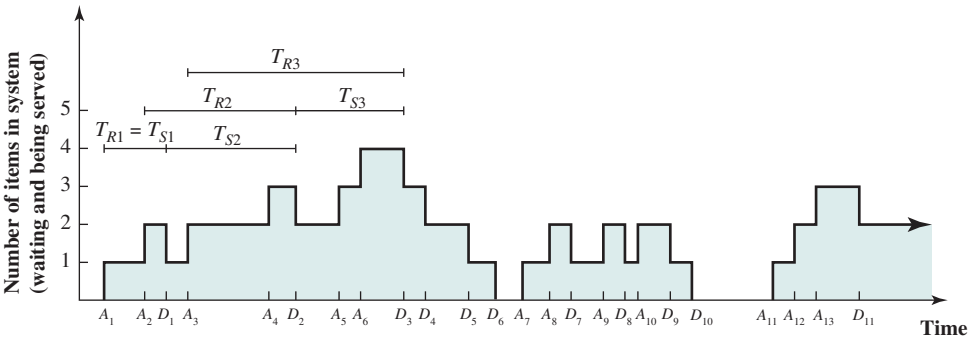
However, queues become very large near system saturation, growing without bound when $\rho = 1$. Practical considerations, such as response time requirements or buffer sizes, usually limit the input rate for a single server to 70–90% of the theoretical maximum.

***ILLUSTRATION OF KEY FEATURES*** It is helpful to have an illustration of the processes involved in queueing. Figure 21.3 shows an example realization of a queueing process, with the total number of items in the system plotted against time. The shaded areas represent time periods in which the server is busy. On the time axis are marked two types of events: the arrival of item $j$ at time $A_j$ and the completion of service of item $j$ at time $D_j$, when the item departs the system. The time that item $j$ spends in the system is $T_{Rj} = D_j - A_j$; the actual service time for item $j$ is denoted by $T_{Sj}$.

In this example, $T_{R1}$ is composed entirely of the service time $T_{S1}$ for the first item, because when item 1 arrives the system is empty and it can go straight into service. $T_{R2}$ is composed of the time that item 2 waits for service $(D_1 - A_2)$ plus its service time $T_{S2}$. Similarly, $T_{R3} = (D_3 - A_3) = (D_3 - D_2) + (D_2 - A_3) = T_{S3} + (D_2 - A_3)$. However, item $n$ may depart before the arrival of item $n + 1$, (e.g., $D_6 < A_7$), so the general expression is $T_{Rn+1} = T_{Sn+1} + \text{MAX}[0, D_n - A_{n+1}]$.

***MODEL CHARACTERISTICS*** Before deriving any analytic equations for the queueing model, certain key characteristics of the model must be chosen. The following are the typical choices, usually reasonable in a data communications context:

[3]Again, in some of the literature, this is referred to as the mean queueing time, while other treatments use mean queueing time to mean the average time spent waiting in the waiting line (before being served).




For item $i$:

$A_i$ = Arrival time
$D_i$ = Departure time
$T_{Ri}$ = Residence time
$T_{Si}$ = Service time

**Figure 21.3 Example of a Queueing Process**

- **Item population:** We assume items arrive from a source population so large that it can be viewed as infinite. The effect of this assumption is that the arrival rate is not altered as items enter the system. If the population is finite, then the population available for arrival is reduced by the number of items currently in the system; this would typically reduce the arrival rate proportionally. Networking and server problems can usually be handled with an infinite-population assumption.
- **Queue size:** We assume an infinite queue size. Thus, the queue can grow without bound. With a finite queue, items can be lost from the system; that is, if the queue is full and additional items arrive, some items must be discarded. In practice, any queue is finite, but in many cases, this makes no substantive difference to the analysis. We will address this issue briefly later in this chapter.
- **Dispatching discipline:** When the server becomes free, and if there is more than one item waiting, a decision must be made as to which item to dispatch next. The simplest approach is first-in-first-out (FIFO), also known as first-come-first-served (FCFS); this discipline is what is normally implied when the term *queue* is used. Another possibility is last-in-last-out (LIFO). A common approach is a dispatching discipline based on relative priority. For example, a router may use QoS (quality of service) information to give preferential treatment to some packets. We will discuss dispatching based on priority subsequently. One dispatching discipline that you might encounter in practice is based on service time. For example, a process scheduler may choose to dispatch processes on the basis of shortest first (to allow the largest number of processes to be granted time in a short interval) or longest first (to minimize processing time relative to service time). Unfortunately, a discipline based on service time is very difficult to model analytically.

Table 21.4 summarizes the notation that is used in Figure 21.2 and introduces some other useful parameters. In particular, we are often interested in the variability of various parameters, and this is neatly captured in the standard deviation.

**Table 21.4** Notation for Queueing Systems

| | |
|---|---|
| $\lambda$ | = arrival rate; mean number of arrivals per second |
| $T_s$ | = mean service time for each arrival; amount of time being served, not counting time waiting in the queue |
| $\sigma_{Ts}$ | = standard deviation of service time |
| $\rho$ | = utilization; fraction of time facility (server or servers) is busy |
| $u$ | = traffic intensity |
| $r$ | = mean number of items in system, waiting and being served |
| $R$ | = number of items in system, waiting and being served |
| $T_r$ | = mean time an item spends in system (residence time) |
| $T_R$ | = time an item spends in system (residence time) |
| $\sigma_r$ | = standard deviation of $r$ |
| $\sigma_{Tr}$ | = standard deviation of $T_r$ |
| $w$ | = mean number of items waiting to be served |
| $\sigma_w$ | = standard deviation of $w$ |
| $T_w$ | = mean waiting time (including items that have to wait and items with waiting time = 0) |
| $T_d$ | = mean waiting time for items that have to wait |
| $N$ | = number of servers |
| $m_x(y)$ | = the $y$th percentile; that value of $y$ below which $x$ occurs $y$ percent of the time |

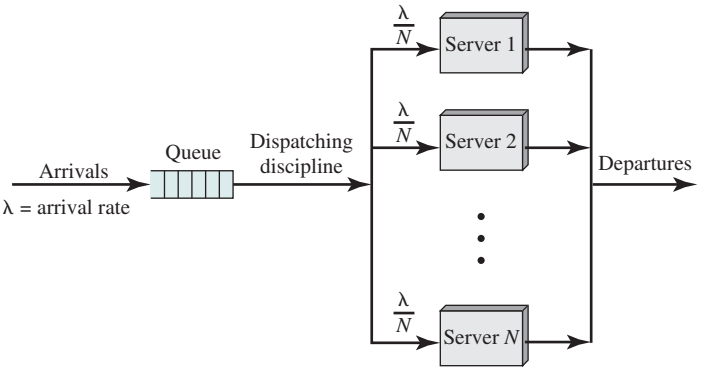
## The Multiserver Queue

Figure 21.4a shows a generalization of the simple model we have been discussing for multiple servers, all sharing a common queue. If an item arrives and at least one server is available, then the item is immediately dispatched to that server. It is assumed all servers are identical; thus, if more than one server is available, the selection of a particular server for a waiting item has no effect on service time. If all servers are busy, a queue begins to form. As soon as one server becomes free, an item is dispatched from the queue using the dispatching discipline in force.

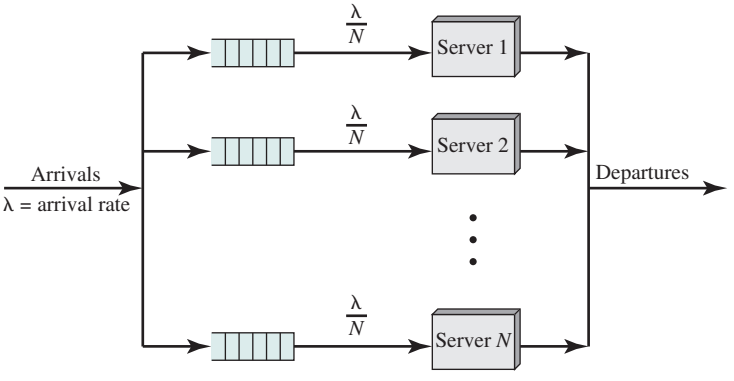
With the exception of utilization, all of the parameters illustrated in Figure 21.2 carry over to the multiserver case with the same interpretation. If we have $N$ identical servers, then $\rho$ is the utilization of each server, and we can consider $N\rho$ to be the utilization of the entire system; this latter term is often referred to as the *traffic intensity*, $u$. Thus, the theoretical maximum utilization is $N \times 100\%$, and the theoretical maximum input rate is:

$$\lambda_{\max} = \frac{N}{T_s}$$

The key characteristics typically chosen for the multiserver queue correspond to those for the single-server queue. That is, we assume an infinite population and an infinite queue size, with a single infinite queue shared among all servers. Unless otherwise stated in this discussion, the dispatching discipline is FIFO. For the multiserver case, if all servers are assumed identical, the selection of a particular server for a waiting item has no effect on service time.




(a) Multiserver queue




(b) Multiple Single-server queues

**Figure 21.4 Multiserver Versus Multiple Single-Server Queues**

By way of contrast, Figure 21.4b shows the structure of multiple single-server queues. As we shall see, this apparently minor change in structure has significant impact on performance.

## Basic Queueing Relationships

To proceed much further, we have to make some simplifying assumptions. These assumptions risk making the models less valid for various real-world situations. Fortunately, in most cases, the results will be sufficiently accurate for planning and design purposes.

There are, however, some relationships that are true in the general case, and these are illustrated in Table 21.5. By themselves, these relationships are not particularly helpful, although they can be used to answer a few basic questions. For example, consider a spy from Burger King trying to figure out how many people are inside the McDonald's across the way. He can't sit inside the McDonald's all day, so he has to determine an answer just based on observing the traffic in and out of the building.

Over the course of the day, he observes that on average 32 customers per hour go into the restaurant. He notes certain people and finds that on average a customer stays inside 12 minutes. Using Little's formula, the spy deduces that there are on average 6.4 customers in McDonald's at any given time (6.4 = 32 customers per hour × 0.2 hours per customer).

It would be useful at this point to gain an intuitive grasp of the equations in Table 21.5. For the equation $\rho = \lambda T_s$, consider that for an arrival rate of $\lambda$, the average time between arrivals is $1/\lambda = T$. If $T$ is greater than $T_s$, then during a time interval $T$, the server is only busy for a time $T_s$ for a utilization of $T_s/T = \lambda T_s$. Similar reasoning applies in the multiserver case to yield $\rho = (\lambda T_s)/N$.

To understand Little's formula, consider the following argument, which focuses on the experience of a single item. When the item arrives, it will find on average $w$ items waiting ahead of it. When the item leaves the queue behind it to be serviced, it will leave behind on average the same number of items in the queue, namely $w$. To see this, note while the item is waiting, the line in front of it shrinks until the item is at the front of the line; meanwhile, additional items arrive and get in line behind this item. When the item leaves the queue to be serviced, the number of items behind it, on average, is $w$, because $w$ is defined as the average number of items waiting. Further, the average time that the item was waiting for service is $T_w$. Since items arrive at a rate of $\lambda$, we can reason that in the time $T_w$, a total of $\lambda T_w$ items must have arrived. Thus, $w = \lambda T_w$. Similar reasoning can be applied to the relationship $r = \lambda T_r$.

Turning to the last equation in the first column of Table 21.5, it is easy to observe that the time that an item spends in the system is the sum of the time waiting for service plus the time being served. Thus, on average, $T_r = T_w + T_s$. The last equations in the second and third columns are easily justified. At any time, the number of items in the system is the sum of the number of items waiting for service plus the number of items being served. For a single server, the average number of items being served is $\rho$. Therefore, $r = w + \rho$ for a single server. Similarly, $r = w + N\rho$ for $N$ servers.

## Assumptions

The fundamental task of a queueing analysis is as follows: Given the following information as input:

- Arrival rate
- Service time
- Number of servers

**Table 21.5** Some Basic Queueing Relationships

| General | Single Server | Multiserver |
|---|---|---|
| $r = \lambda T_r$ Little's formula<br>$w = \lambda T_w$ Little's formula<br>$T_r = T_w + T_s$ | $\rho = \lambda T_s$<br>$r = w + \rho$ | $\rho = \frac{\lambda T_s}{N}$<br>$u = \lambda T_s = \rho N$<br>$r = w + N\rho$ |

provide as output information concerning:

- Items waiting
- Waiting time
- Items in residence
- Residence time

What specifically would we like to know about these outputs? Certainly we would like to know their average values ($w$, $T_w$, $r$, $T_r$). In addition, it would be useful to know something about their variability. Thus, the standard deviation of each would be useful ($\sigma_r$, $\sigma_{T_r}$, $\sigma_w$, $\sigma_{T_w}$). Other measures may also be useful. For example, to design a buffer associated with a router or multiplexer, it might be useful to know for what buffer size the probability of overflow is less than 0.001. That is, what is the value of $N$ such that Pr[items waiting $< N$] $= 0.999$?

To answer such questions in general requires complete knowledge of the probability distribution of the interarrival times (time between successive arrivals) and service time. Furthermore, even with that knowledge, the resulting formulas are exceedingly complex. Thus, to make the problem tractable, we need to make some simplifying assumptions.

The most important of these assumptions concerns the arrival rate. We assume the interarrival times are exponential, which is equivalent to saying that the number of arrivals in a period $t$ obeys the Poisson distribution, which is equivalent to saying that the arrivals occur randomly and independent of one another. This assumption is almost invariably made. Without it, most queueing analysis is impractical, or at least quite difficult. With this assumption, it turns out that many useful results can be obtained if only the mean and standard deviation of the arrival rate and service time are known. Matters can be made even simpler and more detailed results can be obtained if it is assumed the service time is exponential or constant.

A convenient notation, called **Kendall's notation**, has been developed for summarizing the principal assumptions that are made in developing a queueing model. The notation is X/Y/N, where X refers to the distribution of the interarrival times, Y refers to the distribution of service times, and N refers to the number of servers. The most common distributions are denoted as follows:

G = general distribution of interarrival times or service times

GI = general distribution of interarrival times with the restriction that interarrival times are independent

M = negative exponential distribution

D = deterministic arrivals or fixed-length service

Thus, M/M/1 refers to a single-server queueing model with Poisson arrivals (exponential interarrival times) and exponential service times.

## 21.4 SINGLE-SERVER QUEUES

Table 21.6a provides some equations for single-server queues that follow the M/G/1 model. That is, the arrival rate is Poisson and the service time is general. Making use of a scaling factor, $A$, the equations for some of the key output variables are straightforward. Note the key factor in the scaling parameter is the ratio of the standard deviation of service time to the mean. No other information about the service time is needed. Two special cases are of some interest. When the standard deviation is equal to the mean, the service time distribution is exponential (M/M/1). This is the simplest case, and the easiest one for calculating results. Table 21.6b shows the simplified versions of equations for the standard deviation of $r$ and $T_r$, plus some other parameters of interest. The other interesting case is a standard deviation of service time equal to zero, that is, a constant service time (M/D/1). The corresponding equations are shown in Table 21.6c.

Figures 21.5 and 21.6 plot values of average queue size and residence time versus utilization for three values of $\sigma_{T_s}/T_s$. This latter quantity is known as the **coefficient of variation** and gives a normalized measure of variability. Note the poorest performance is exhibited by the exponential service time, and the best by a constant service time. In many cases, one can consider the exponential service time to be a worst case, so an analysis based on this assumption will give conservative results. This is nice, because tables are available for the M/M/1 case and values can be looked up quickly.

What value of $\sigma_{T_s}/T_s$ is one likely to encounter? We can consider four regions:

- **Zero:** This is the rare case of constant service time. For example, if all transmitted packets are of the same length, they would fit this category.
- **Ratio less than 1:** Because this ratio is better than the exponential case, using M/M/1 tables will give queue sizes and times that are slightly larger than they should be. Using the M/M/1 model would give answers on the safe side. An example of this category might be a data entry application for a particular form.
- **Ratio close to 1:** This is a common occurrence and corresponds to exponential service time. That is, service times are essentially random. Consider message lengths to a computer terminal: A full screen might be 1920 characters, with message sizes varying over the full range. Airline reservations, file lookups on inquiries, shared LAN, and packet-switching networks are examples of systems that often fit this category.
- **Ratio greater than 1:** If you observe this, you need to use the M/G/1 model and not rely on the M/M/1 model. A common occurrence of this is a bimodal distribution, with a wide spread between the peaks. An example is a system that experiences many short messages, many long messages, and few in between.

The same consideration applies to the arrival rate. For a Poisson arrival rate, the interarrival times are exponential, and the ratio of standard deviation to mean is 1. If the observed ratio is much less than one, then arrivals tend to be evenly spaced (not

**Table 21.6** Formulas for Single-Server Queues

| Assumptions: | 1. Poisson arrival rate.<br>2. Dispatching discipline does not give preference to items based on service times.<br>3. Formulas for standard deviation assume first-in-first-out dispatching.<br>4. No items are discarded from the queue. |
|---|---|

**(a) General Service Times (M/G/1)**

$$A = \frac{1}{2}\left[1 + \left(\frac{\sigma_{T_s}}{T_s}\right)^2\right]$$

$$r = \rho + \frac{\rho^2 A}{1-\rho}$$

$$w = \frac{\rho^2 A}{1-\rho}$$

$$T_r = T_s + \frac{\rho T_s A}{1-\rho}$$

$$T_w = \frac{\rho T_s A}{1-\rho}$$

**(b) Exponential Service Times (M/M/1)**

$$r = \frac{\rho}{1-\rho} \qquad w = \frac{\rho^2}{1-\rho}$$

$$T_r = \frac{T_s}{1-\rho} \qquad T_w = \frac{\rho T_s}{1-\rho}$$

$$\sigma_r = \frac{\sqrt{\rho}}{1-\rho} \qquad \sigma_{T_r} = \frac{T_s}{1-\rho}$$

$$\Pr[R = N] = (1-\rho)\rho^N$$

$$\Pr[R \le N] = \sum_{i=0}^{N} (1-\rho)\rho^i$$

$$\Pr[T_R \le T] = 1 - e^{-(1-\rho)t/T_s}$$

$$m_{T_r}(y) = T_r \times \ln\left(\frac{100}{100-y}\right)$$

$$m_{T_w}(y) = \frac{T_w}{\rho} \times \ln\left(\frac{100\rho}{100-y}\right)$$

**(c) Constant Service Times (M/D/1)**

$$r = \frac{\rho^2}{2(1-\rho)} + \rho$$

$$w = \frac{\rho^2}{2(1-\rho)}$$

$$T_r = \frac{T_s(2-\rho)}{2(1-\rho)}$$

$$T_w = \frac{\rho T_s}{2(1-\rho)}$$

$$\sigma_r = \frac{1}{1-\rho}\sqrt{\rho - \frac{3\rho^2}{2} + \frac{5\rho^3}{6} - \frac{\rho^4}{12}}$$

$$\sigma_{T_r} = \frac{T_s}{1-\rho}\sqrt{\frac{\rho}{3} - \frac{\rho^2}{12}}$$

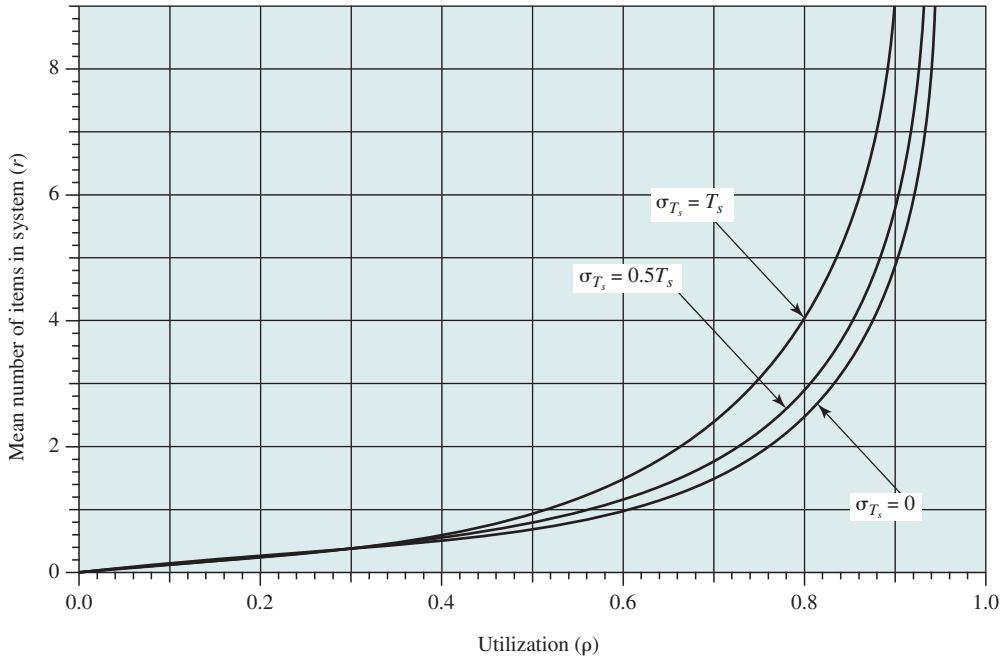



**Figure 21.5** **Mean Number of Items in System for Single-Server Queue**

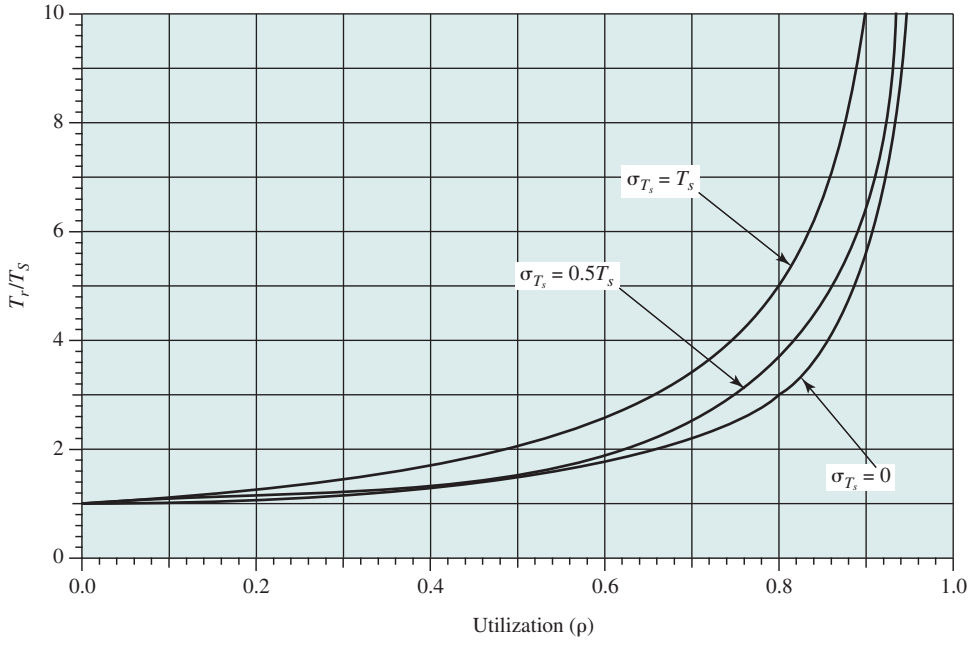



**Figure 21.6** **Mean Residence Time for Single-Server Queue**

much variability), and the Poisson assumption will overestimate queue sizes and delays. On the other hand, if the ratio is greater than 1, then arrivals tend to cluster and congestion becomes more acute.

## 21.5 MULTISERVER QUEUES

Table 21.7 lists formulas for some key parameters for the multiserver case. Note the restrictiveness of the assumptions. Useful congestion statistics for this model have been obtained only for the case of M/M/N, where the exponential service times are identical for the N servers.

Note the presence of the Erlang C function in nearly all of the equations. This is the probability that all servers are busy at a given instant; equivalently, this is the probability that the number of items in the system (waiting and being served) is greater than or equal to the number of servers. The equation has the form

$$C(N, \rho) = \frac{1 - K(N, \rho)}{1 - \rho K(N, \rho)}$$

where $K$ is known as the Poisson ratio function. Because $C$ is a probability, its value is always between zero and one. As can be seen, this quantity is a function of the number of servers and the utilization. This expression turns up frequently in queueing calculations. Tables of values are readily found, or a computer program must be used. Note for a single-server system, this equation simplifies to $C(1, \rho) = \rho$.

## 21.6 EXAMPLES

Let us look at a few examples to get some feel for the use of these equations.

### Database Server

Consider a LAN with 100 personal computers and a server that maintains a common database for a query application. The average time for the server to respond to a query is 0.6 seconds, and the standard deviation is estimated to equal the mean. At peak times, the query rate over the LAN reaches 20 queries per minute. We would like to answer the following questions:

- What is the average response time ignoring line overhead?
- If a 1.5-second response time is considered the maximum acceptable, what percent growth in message load can occur before the maximum is reached?
- If 20% more utilization is experienced, will response time increase by more or less than 20%?

**Table 21.7** Formulas for Multiserver Queues (M/M/N)

| Assumptions: | 1. Poisson arrival rate.<br>2. Exponential service times.<br>3. All servers equally loaded.<br>4. All servers have same mean service time.<br>5. First-in-first-out dispatching.<br>6. No items are discarded from the queue. |
|---|---|

$$K = \frac{\sum_{I=0}^{N-1} \frac{(N\rho)^I}{I!}}{\sum_{I=0}^{N} \frac{(N\rho)^I}{I!}} \qquad \text{Poisson ratio function}$$

$$\text{Erlang C function} = \text{Probability that all servers are busy} = C = \frac{1 - K}{1 - \rho K}$$

$$r = C\frac{\rho}{1-\rho} + N\rho \qquad w = C\frac{\rho}{1-\rho}$$

$$T_r = \left(\frac{C}{N}\right)\frac{T_s}{1-\rho} + T_s \qquad T_w = \left(\frac{C}{N}\right)\frac{T_s}{1-\rho}$$

$$\sigma_{T_r} = \frac{T_s}{N(1-\rho)}\sqrt{C(2-C) + N^2(1-\rho)^2}$$

$$\sigma_w = \frac{1}{1-\rho}\sqrt{C\rho(1+\rho-C\rho)}$$

$$\Pr[T_W > t] = Ce^{-N(1-\rho)t/T_s}$$

$$m_{T_w}(y) = \frac{T_s}{N(1-\rho)}\ln\left(\frac{100C}{100-y}\right)$$

$$T_d = \frac{T_s}{N(1-\rho)}$$

Assume an M/M/1 model, with the database server being the server in the model. We ignore the effect of the LAN, assuming that its contribution to the delay is negligible. Facility utilization is calculated as:

$$\begin{aligned}\rho &= \lambda T_s \\ &= (20 \text{ arrivals per minute})(0.6 \text{ seconds per transmission})/(60 \text{ s/min}) \\ &= 0.2\end{aligned}$$

The first value, average response time, is easily calculated:

$$\begin{aligned}T_r &= T_s/(1-\rho) \\ &= 0.6/(1-0.2) = 0.75 \text{ seconds}\end{aligned}$$

The second value is more difficult to obtain. Indeed, as worded, there is no answer because there is a nonzero probability that some instances of response time will exceed 1.5 seconds for any value of utilization. Instead, let us say we would like

90% of all responses to be less than 1.5 seconds. Then, we can use the equation from Table 21.6b:

$$m_{T_r}(y) = T_r \times \ln(100/(100 - y))$$

$$m_{T_r}(90) = T_r \times \ln(10) = \frac{T_s}{1 - \rho} \times 2.3 = 1.5 \text{ seconds}$$

We have $T_s = 0.6$. Solving for $\rho$ yields $\rho = 0.08$. In fact, utilization would have to decline from 20% to 8% to put 1.5 seconds at the 90th percentile.

The third part of the question is to find the relationship between increases in load versus response time. Because a facility utilization of 0.2 is down in the flat part of the curve, response time will increase more slowly than utilization. In this case, if facility utilization increases from 20% to 40%, which is a 100% increase, the value of $T_r$ goes from 0.75 seconds to 1.0 second, which is an increase of only 33%.

## Calculating Percentiles

Consider a configuration in which packets are sent from computers on a LAN to systems on other networks. All of these packets must pass through a router that connects the LAN to a wide area network and hence to the outside world. Let us look at the traffic from the LAN through the router. Packets arrive with a mean arrival rate of 5 per second. The average packet length is 144 octets, and it is assumed packet length is exponentially distributed. Line speed from the router to the wide area network is 9,600 bps. The following questions are asked:

1. What is the mean residence time for the router?
2. How many packets are in the router, including those waiting for transmission and the one currently being transmitted (if any), on the average?
3. Same question as (2), for the 90th percentile.
4. Same question as (2), for the 95th percentile.

$$\begin{aligned}
\lambda &= 5 \text{ packets per second} \\
T_s &= (144 \text{ octets} \times 8 \text{ bits per octet})/9{,}600 \text{ bps} = 0.12 \text{ seconds} \\
\rho &= \lambda T_s = 5 \times 0.12 = 0.6 \\
T_r &= T_s/(1 - \rho) = 0.3 \text{ seconds} \quad \text{Mean residence time} \\
r &= \rho/(1 - \rho) = 1.5 \text{ packets} \quad \text{Mean number of resident items}
\end{aligned}$$

To obtain the percentiles, we use the equation from Table 21.6b:

$$\Pr[R = N] = (1 - \rho)\rho^N$$

To calculate the $y$th percentile of queue size, we write the preceding equation in cumulative form:

$$\frac{y}{100} = \sum_{k=0}^{m_r(y)} (1 - \rho)\rho^k = 1 - \rho^{1+m_r(y)}$$

Here $m_r(y)$ represents the maximum number of packets in the queue expected $y$ percent of the time. That is, $m_r(y)$ is that value below which $R$ occurs $y$ percent of the

time. In the form given, we can determine the percentile for any queue size. We wish to do the reverse: Given $y$, find $m_r(y)$. So, taking the logarithm of both sides:

$$m_r(y) = \frac{\ln\left(1 - \frac{y}{100}\right)}{\ln \rho} - 1$$

If $m_r(y)$ is fractional, take the next higher integer; if it is negative, set it to zero. For our example, $\rho = 0.6$ and we wish to find $m_r(90)$ and $m_r(95)$:

$$m_r(90) = \frac{\ln(1 - 0.90)}{\ln(0.6)} - 1 = 3.5$$

$$m_r(95) = \frac{\ln(1 - 0.95)}{\ln(0.6)} - 1 = 4.8$$

Thus, 90% of the time there are fewer than 4 packets in the queue, and 95% of the time there are fewer than 5 packets. If we were designing to a 95th percentile criterion, a buffer would have to be provided to store at least 5 packets.

## Tightly-Coupled Multiprocessor

Let us consider the use of multiple tightly-coupled processors in a single computer system. One of the design decisions had to do with whether processes are dedicated to processors. If a process is permanently assigned to one processor from activation until its completion, then a separate short-term queue is kept for each processor. In this case, one processor can be idle, with an empty queue, while another processor has a backlog. To prevent this situation, a common queue can be used. All processes go into one queue and are scheduled to any available processor. Thus, over the life of a process, the process may be executed on different processors at different times.

Let us try to get a feel for the performance speed-up to be achieved by using a common queue. Consider a system with five processors, and the average amount of processor time provided to a process while in the Running state is 0.1 second. Assume the standard deviation of service time is observed to be 0.094 second. Because the standard deviation is close to the mean, we will assume exponential service time. Also assume processes are arriving at the Ready state at the rate of 40 per second.

***Single-Server Approach*** If processes are evenly distributed among the processors, then the load for each processor is 40/5 = 8 processes per second. Thus,

$$\rho = \lambda T_s$$
$$= 8 \times 0.1 = 0.8$$

The residence time is then easily calculated:

$$t_r = \frac{T_s}{1 - \rho} = \frac{0.1}{0.2} = 0.5 \text{ sec}$$

***MULTISERVER APPROACH*** Now assume a single Ready queue is maintained for all processors. We now have an aggregate arrival rate of 40 processes per second. However, the facility utilization is still 0.8 ($\lambda T_s/M$). To calculate the residence time from the formula in Table 21.7, we need to first calculate the Erlang C function. If you have not programmed the parameter, it can be looked up in a table under a facility utilization of 0.8 for five servers to yield $C = 0.554$. Substituting,

$$T_r = (0.1) + \frac{(0.544)(0.1)}{5(1 - 0.8)} = 0.1544$$

So the use of multiserver queue has reduced average residence time from 0.5 seconds down to 0.1544 seconds, which is greater than a factor of 3. If we look at just the waiting time, the multiserver case is 0.0544 seconds compared to 0.4 seconds, which is a factor of 7.

Although you may not be an expert in queueing theory, you now know enough to be annoyed when you have to wait in a line at a multiple single-server queue facility.

## A Multiserver Problem

An engineering firm provides each of its analysts with a personal computer, all of which are hooked up over a LAN to a database server. In addition, there is an expensive, stand-alone graphics workstation that is used for special-purpose design tasks. During the course of a typical eight-hour day, 10 engineers will make use of the workstation and spend an average of 30 minutes at a session.

***SINGLE-SERVER MODEL*** The engineers complain to their manager that the wait for using the workstation is long, often an hour or more, and are asking for more workstations. This surprises the manager since the utilization of the workstation is only 5/8 ($10 \times 1/2 = 5$ hours out of 8). To convince the manager, one of the engineers performs a queueing analysis. The engineer makes the usual assumptions of an infinite population, random arrivals, and exponential service times, none of which seem unreasonable for rough calculations. Using the equations in Tables 21.5 and 21.6b, the engineer gets:

| | |
|---|---|
| $T_w = \frac{\rho T_s}{1 - \rho} = 50$ minutes | Average time an engineer spends waiting for the workstation |
| $m_{T_w}(90) = \frac{T_w}{\rho} \times \ln(10\rho) = 146.6$ minutes | 90th percentile waiting time |
| $\lambda = \frac{10}{8 \times 60} = 0.021$ engineers/minute | Arrival rate of engineers |
| $w = \lambda T_w = 1.0416$ engineers | Average number of engineers waiting |

These figures show that indeed the engineers do have to wait an average of almost an hour to use the workstation, and that in 10% of the cases, an engineer has to wait well over two hours. Even if there is a significant error in the estimate, say

20%, the waiting time is still far too long. Furthermore, if an engineer can do no useful work while waiting for the workstation, then a little over one engineer-day is being lost per day.

***MULTISERVER MODEL*** The engineers have convinced the manager of the need for more workstations. They would like the mean waiting time not to exceed 10 minutes, with the 90th percentile value not to exceed 15 minutes. This concerns the manager, who reasons that if one workstation results in a waiting time of 50 minutes, then five workstations will be required to get the average down to 10 minutes.

The engineers set to work to determine how many workstations are required. There are two possibilities: Put additional workstations in the same room as the original one (multiserver queue) or scatter the workstations to various rooms on various floors (multiple single-server queues). First, we look at the multiserver case and consider the addition of a second workstation in the same room. Let's assume that the addition of the new workstation, which reduces waiting time, does not affect the arrival rate (10 engineers per day). Then the available service time is 16 hours in an eight-hour day with a demand of five hours (10 engineers $\times$ 0.5 hours), giving a utilization of $5/16 = 0.3125$. Using the equations in Table 21.7:

| | |
|---|---|
| $C(2, \rho) = C(2, 0.3125) = 0.1488$ | Probability that both servers are busy |
| $T_w = \dfrac{CT_s}{N(1 - \rho)} = 3.247$ minutes | Average time an engineer spends waiting for a workstation |
| $m_{T_w}(90) = \dfrac{T_s}{2(1 - \rho)} \ln(10C) = 8.67$ minutes | 90th percentile waiting time |
| $w = \lambda T_w = 0.07$ engineers | Average number of engineers waiting |

With this arrangement, the probability that an engineer who wishes to use a workstation must wait is less than 0.15 and the average wait is just a little over three minutes, with the 90th percentile wait of less than nine minutes. Despite the manager's doubts, the multiserver arrangement with two workstations easily meets the design requirement.

All of the engineers are housed on two floors of the building, so the manager wonders whether it might be more convenient to place one workstation on each floor. If we assume the traffic to the two workstations is about evenly split, then there are two M/M/1 queues, each with a $\lambda$ of five engineers per eight-hour day. This yields:

| | |
|---|---|
| $\rho = \lambda T_s = 0.3125$ | Utilization of one server |
| $T_w = \dfrac{\rho T_s}{1 - \rho} = 13.64$ minutes | Average time an engineer spends waiting for the workstation |
| $m_{T_w}(90) = \dfrac{T_w}{\rho} \times \ln(10\rho) = 49.73$ minutes | 90th percentile waiting time |
| $w = \lambda T_w = 0.142$ engineers | Average number of engineers waiting |

**Table 21.8** Summary of Calculations for Multiserver Example

| Workstations | System | $\rho$ | $T_w$ | $m_{Tw}(90)$ |
|---|---|---|---|---|
| 1 | M/M/1 | 0.625 | 50 | 146.61 |
| 2 | M/M/2 | 0.3125 | 3.25 | 8.67 |
| 3 | M/M/1's | 0.3125 | 13.64 | 49.73 |
| 4 | M/M/1's | 0.15625 | 5.56 | 15.87 |
| 5 | M/M/1's | 0.125 | 4.29 | 7.65 |

This performance is significantly worse than the multiserver model and does not meet the design criteria. Table 21.8 summarizes the results and also shows the results for four and five separate workstations. Note to meet the design goal, five separate workstations are needed compared to only two multiserver workstations.

## 21.7 QUEUES WITH PRIORITIES

So far, we have considered queues in which items are treated in a first-come-first-served basis. There are many cases in both networking and operating system design in which it is desirable to use priorities. Priorities may be assigned in a variety of ways. For example, priorities may be assigned on the basis of traffic type. If it turns out that the average service time for the various traffic types is identical, then the overall equations for the system are not changed, although the performance seen by the different traffic classes will differ.

An important case is one in which priority is assigned on the basis of average service time. Often, items with shorter expected service times are given priority over items with longer service times. For example, a router may assign a higher priority to a stream of voice packets than a stream of data packets, and typically, the voice packets would be much shorter than the data packets. With this kind of scheme, performance is improved for higher-priority traffic.

Table 21.9 shows the formulas that apply when we assume two priority classes with different service times for each class. These results are easily generalized to any number of priority classes.

To see the effects of the use of priority, let us consider a simple example of a data stream consisting of a mixture of long and short packets being transmitted by a packet-switching node and that the rate of arrival of the two types of packets is equal. Suppose both packets have lengths that are exponentially distributed, and the long packets have a mean packet length of 10 times the short packets. In particular, let us assume a 64-Kbps transmission link and the mean packet lengths are 80 and 800 octets. Then the two service times are 0.01 and 0.1 seconds. Also assume the arrival rate for each type is 8 packets per second. So the shorter packets are not held up by the longer packets, let us assign the shorter packets a higher priority. Then:

$$\rho_1 = 8 \times 0.01 = 0.08 \qquad \rho_2 = 8 \times 0.1 = 0.8 \qquad \rho = 0.88$$

**Table 21.9** Formulas for Single-Server Queues with Two Priority Categories

| Assumptions: | **1.** Poisson arrival rate.<br>**2.** Priority 1 items are serviced before priority 2 items.<br>**3.** First-in-first-out dispatching for items of equal priority.<br>**4.** No item is interrupted while being served.<br>**5.** No items leave the queue (lost calls delayed). |
|---|---|

**(a) General Formulas**

$$\lambda = \lambda_1 + \lambda_2$$
$$\rho_1 = \lambda_1 T_{s1}; \rho_2 = \lambda_2 T_{s2}$$
$$\rho = \rho_1 + \rho_2$$
$$T_s = \frac{\lambda_1}{\lambda} T_{s1} + \frac{\lambda_2}{\lambda} T_{s2}$$
$$T_r = \frac{\lambda_1}{\lambda} T_{r1} + \frac{\lambda_2}{\lambda} T_{r2}$$

**(b) Exponential Service Times**

$$w_1 = \frac{\rho_1(\rho_1 T_{s1} + \rho_2 T_{s2})}{T_{s1}(1 - \rho_1)}$$
$$w_2 = w_1 \frac{\lambda_2}{\lambda_1(1 - \rho)}$$
$$T_{r1} = T_{s1} + \frac{\rho_1 T_{s1} + \rho_2 T_{s2}}{1 - \rho_1}$$
$$T_{r2} = T_{s2} + \frac{T_{r1} - T_{s1}}{1 - \rho}$$

$$T_{r1} = 0.01 + \frac{0.08 \times 0.01 + 0.8 \times 0.1}{1 - 0.08} = 0.098 \text{ seconds}$$
$$T_{r2} = 0.1 + \frac{0.098 - 0.01}{1 - 0.88} = 0.833 \text{ seconds}$$
$$T_r = 0.5 \times 0.098 + 0.5 \times 0.833 = 0.4655 \text{ seconds}$$

So we see the higher-priority packets get considerably better service than the lower-priority packets.

## 21.8 NETWORKS OF QUEUES

In a distributed environment, isolated queues are unfortunately not the only problem presented to the analyst. Often, the problem to be analyzed consists of several interconnected queues. Figure 21.7 illustrates this situation, using nodes to represent queues and the interconnecting lines to represent traffic flow.

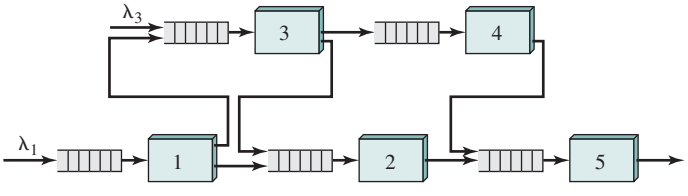



**Figure 21.7** Example of a Network of Queues

Two elements of such a network complicate the methods shown so far:

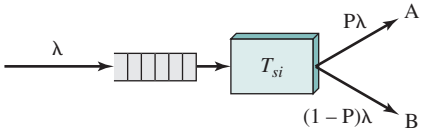
- The partitioning and merging of traffic, as illustrated by nodes 1 and 5, respectively, in the figure
- The existence of queues in tandem, or series, as illustrated by nodes 3 and 4

No exact method has been developed for analyzing general queueing problems that have the aforementioned elements. However, if the traffic flow is Poisson and the service times are exponential, an exact and simple solution exists. In this section, we first examine the two elements listed previously, then present the approach to queueing analysis.

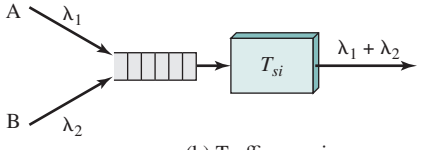
## Partitioning and Merging of Traffic Streams

Suppose traffic arrives at a queue with a mean arrival rate of $\lambda$, and that there are two paths, A and B, by which an item may depart (see Figure 21.8a). When an item is serviced and departs the queue, it does so via path A with probability $P$ and via path B with probability $(1 - P)$. In general, the traffic distribution of streams A and B will differ from the incoming distribution. However, if the incoming distribution is Poisson, then the two departing traffic flows also have Poisson distributions, with mean rates of $P\lambda$ and $(1 - P)\lambda$.

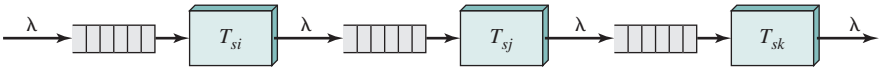
A similar situation exists for traffic merging (see Figure 21.8b). If two Poisson streams with mean rates of $\lambda_1$ and $\lambda_2$ are merged, the resulting stream is Poisson with a mean rate of $\lambda_1 + \lambda_2$.




(a) Traffic partitioning




(b) Traffic merging




(c) Simple tandem queue

**Figure 21.8 Elements of Queueing Networks**

Both of these results generalize to more than two departing streams for partitioning and more than two arriving streams for merging.

## Queues in Tandem

Figure 21.8c is an example of a set of single-server queues in tandem: The input for each queue except the first is the output of the previous queue. Assume the input to the first queue is Poisson. Then, if the service time of each queue is exponential and the queues are of infinite capacity, the output of each queue is a Poisson stream statistically identical to the input. When this stream is fed into the next queue, the delays at the second queue are the same as if the original traffic had bypassed the first queue and fed directly into the second queue. Thus, the queues are independent and may be analyzed one at a time. Therefore, the mean total delay for the tandem system is equal to the sum of the mean delays at each stage.

This result can be extended to the case where some or all of the nodes in tandem are multiserver queues.

## Jackson's Theorem

Jackson's theorem can be used to analyze a network of queues. The theorem is based on three assumptions:

1. The queueing network consists of $m$ nodes, each of which provides an independent exponential service.
2. Items arriving from outside the system to any one of the nodes arrive with a Poisson rate.
3. Once served at a node, an item goes (immediately) to one of the other nodes with a fixed probability, or out of the system.

Jackson's theorem states that in such a network of queues, each node is an independent queueing system, with a Poisson input determined by the principles of partitioning, merging, and tandem queueing. Thus, each node may be analyzed separately from the others using the M/M/1 or M/M/N model, and the results may be combined by ordinary statistical methods. Mean delays at each node may be added to derive system delays, but nothing can be said about the higher moments of system delays (e.g., standard deviation).

Jackson's theorem appears attractive for application to packet-switching networks. One can model the packet-switching network as a network of queues. Each packet represents an individual item. We assume each packet is transmitted separately and, at each packet-switching node in the path from source to destination, the packet is queued for transmission on the next length. The service at a queue is the actual transmission of the packet and is proportional to the length of the packet.

The flaw in this approach is that a condition of the theorem is violated: Namely, it is not the case that the service distributions are independent. Because the length of a packet is the same at each transmission link, the arrival process to each queue is correlated to the service process. However, Kleinrock [KLEI76] has demonstrated that, because of the averaging effect of merging and partitioning, assuming independent service times provides a good approximation.

## Application to a Packet-Switching Network[4]

Consider a packet-switching network, consisting of nodes interconnected by transmission links, with each node acting as the interface for zero or more attached systems, each of which functions as a source and destination of traffic. The external workload that is offered to the network can be characterized as:

$$\gamma = \sum_{j=1}^{N}\sum_{k=1}^{N}\gamma_{jk}$$

where

$\gamma$ = total workload in packets per second

$\gamma_{jk}$ = workload between source $j$ and destination $k$

$N$ = total number of sources and destinations

Because a packet may traverse more than one link between source and destination, the total internal workload will be higher than the offered load:

$$\lambda = \sum_{i=1}^{L}\lambda_i$$

where

$\lambda$ = total load on all of the links in the network

$\lambda_i$ = load on link $i$

$L$ = total number of links

The internal load will depend on the actual path taken by packets through the network. We will assume a routing algorithm is given such that the load on the individual links, $\lambda_i$, can be determined from the offered load, $\gamma_{jk}$. For any particular routing assignment, we can determine the average number of links that a packet will traverse from these workload parameters. Some thought should convince you that the average length for all paths is given by:

$$\text{E[number of links in a path]} = \frac{\lambda}{\gamma}$$

Now, our objective is to determine the average delay, $T$, experienced by a packet through the network. For this purpose, it is useful to apply Little's formula (see Table 21.5). For each link in the network, the average number of items waiting and being served for that link is given by:

$$r_i = \lambda_i T_{ri}$$

where $T_{ri}$ is the yet-to-be-determined queueing delay at each queue. Suppose we sum these quantities. That would give us the average total number of packets waiting in all of the queues of the network. It turns out that Little's formula works in the aggregate as well.[5] Thus, the number of packets waiting and being served in the network can be expressed as $\gamma T$. Combining the two:

$$T = \frac{1}{\gamma}\sum_{i=1}^{L}\lambda_i T_{ri}$$

[4] This discussion is based on the development in [KLEI76].

[5] In essence, this statement is based on the fact that the sum of the averages is the average of the sums.

To determine the value of $T$, we need to determine the values of the individual delays, $T_{ri}$. Because we are assuming each queue can be treated as an independent M/M/1 model, this is easily determined:

$$T_{ri} = \frac{T_{si}}{1 - \rho_i} = \frac{T_{si}}{1 - \lambda_i T_{si}}$$

The service time $T_{si}$ for link $i$ is just the ratio of the average packet length in bits ($M$) to the data rate on the link in bits per second ($R_i$). Then:

$$T_{ri} = \frac{\frac{M}{R_i}}{1 - \frac{M\lambda_i}{R_i}} = \frac{M}{R_i - M\lambda_i}$$

Putting all of the elements together, we can calculate the average delay of packets sent through the network:

$$T = \frac{1}{\gamma} \sum_{i=1}^{L} \frac{M\lambda_i}{R_i - M\lambda_i}$$

## 21.9 OTHER QUEUEING MODELS

In this chapter, we have concentrated on one type of queueing model. There are in fact a number of models, based on two key factors:

- The manner in which blocked items are handled
- The number of traffic sources

When an item arrives at a server and finds that server busy, or arrives at a multiple-server facility and finds all servers busy, that item is said to be blocked. Blocked items can be handled in a number of ways. First, the item can be placed in a queue awaiting a free server. This policy is referred to in the telephone traffic literature as *lost calls delayed*, although in fact the call is not lost. Alternatively, no queue is provided. This in turn leads to two assumptions about the action of the item. The item may wait some random amount of time then try again; this is known as *lost calls cleared*. If the item repeatedly attempts to gain service, with no pause, it is referred to as *lost calls held*. The lost calls delayed model is the most appropriate for most computer and data communications problems. Lost calls cleared is usually the most appropriate in a telephone-switching environment.

The second key element of a traffic model is whether the number of sources is assumed infinite or finite. For an infinite source model, there is assumed to be a fixed arrival rate. For the finite source case, the arrival rate will depend on the number of sources already engaged. Thus, if each of $L$ sources generates arrivals at a rate $\lambda/L$, then when the queueing facility is unoccupied, the arrival rate is $\lambda$. However, if $K$ sources are in the queueing facility at a particular time, then the instantaneous arrival rate at that time is $\lambda(L - K)/L$. Infinite source models are easier to deal with. The infinite source assumption is reasonable when the number of sources is at least 5–10 times the capacity of the system.

## 21.10 ESTIMATING MODEL PARAMETERS

To perform a queueing analysis, we need to estimate the values of the input parameters, specifically the mean and standard deviation of the arrival rate and service time. If we are contemplating a new system, these estimates may have to be based on judgment and an assessment of the equipment and work patterns likely to prevail. However, it will often be the case that an existing system is available for examination. For example, a collection of terminals, personal computers, and host computers are interconnected in a building by direct connection and multiplexers, and it is desired to replace the interconnection facility with a LAN. To be able to size the network, it is possible to measure the load currently generated by each device.

### Sampling

The measurements that are taken are in the form of samples. A particular parameter, for example, the rate of packets generated by a terminal or the size of packets, is estimated by observing the number of packets generated during a period of time.

The most important quantity to estimate is the mean. For many of the equations in Tables 21.6 and 21.7, this is the only quantity that need be estimated. The estimate is referred to as the sample mean $\overline{X}$ and is calculated as follows:

$$\overline{X} = \frac{1}{N}\sum_{i=1}^{N} X_i$$

where

$N$ = sample size

$X_i$ = $i$th item in the sample

It is important to note the sample mean is itself a random variable. For example, if you take a sample from some population and calculate the sample mean, and do this a number of times, the calculated values will differ. Thus, we can talk of the mean and standard deviation of the sample mean, or even of the entire probability distribution of the sample mean. To distinguish the concepts, it is common to refer to the probability distribution of the original random variable $X$ as the *underlying distribution*, and the probability distribution of the sample mean $\overline{X}$ as the *sampling distribution of the mean*.

The remarkable thing about the sample mean is that its probability distribution tends to the normal distribution as $N$ increases for virtually all underlying distributions. The assumption of normality breaks down only if $N$ is very small or if the underlying distribution is highly abnormal.

The mean and variance of $\overline{X}$ are as follows:

$$\mathrm{E}[\overline{X}] = \mathrm{E}[X] = \mu$$

$$\mathrm{Var}[\overline{X}] = \frac{\sigma_X^2}{N}$$

Thus, if a sample mean is calculated, its expected value is the same as that of the underlying random variable and the variability of the sample mean around this

expected value decreases as $N$ increases. These characteristics are illustrated in Figure 21.9. The figure shows an underlying exponential distribution with mean value $\mu = 1$. This could be the distribution of service times of a server, or of the interarrival times of a Poisson arrival process. If a sample of size 10 is used to estimate the value of $\mu$, then the expected value is indeed $\mu$, but the actual value could easily be off by as much as 50%. If the sample size is 100, the spread among possible calculated values is considerably tightened, so that we would expect the actual sample mean for any given sample to be much closer to $\mu$.

The sample mean as defined previously can be used directly to estimate the service time of a server. For arrival rate, one can observe the interarrival times for a sequence of $N$ arrivals, calculate the sample mean, then calculate the estimated arrival rate. An equivalent and simpler approach is to use the following estimate:

$$\overline{\lambda} = \frac{N}{T}$$

where $N$ is the number of items observed in a period of time of duration $T$.

For much of queueing analysis, it is only an estimate of the mean that is required. But for a few important equations, an estimate of the variance of the

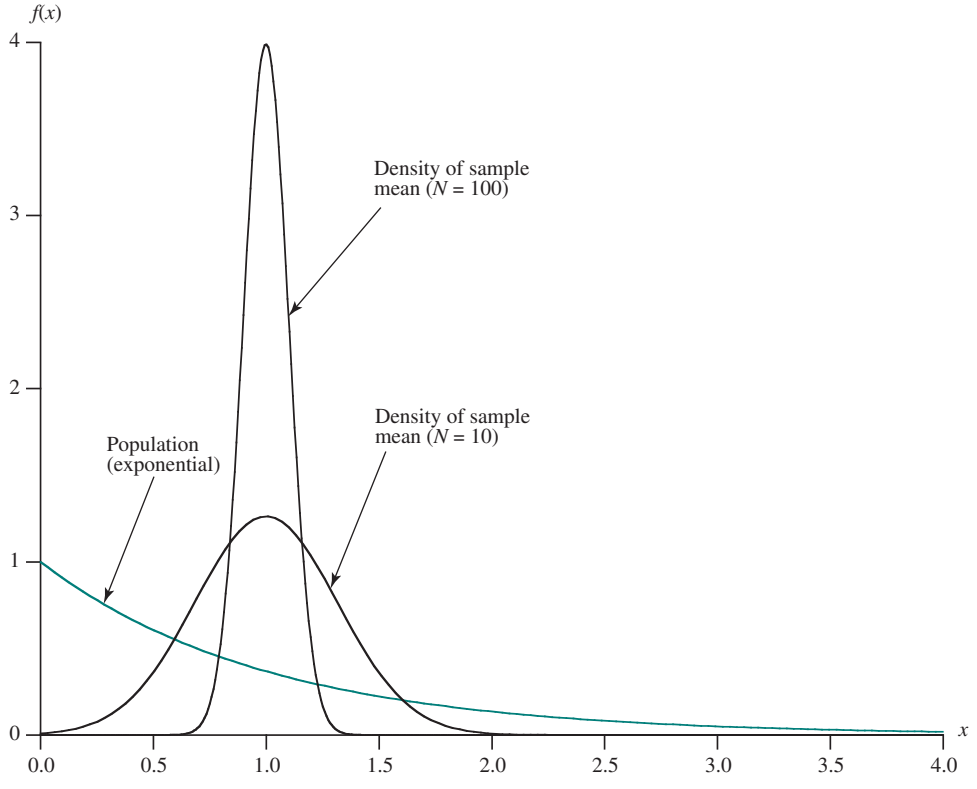



**Figure 20.9 Sample Means for an Exponential Population**

underlying random variable, $\sigma_X^2$, is also needed. The sample variance is calculated as follows:

$$S^2 = \frac{1}{N-1}\sum_{i=1}^{N}(X_i - \overline{X})^2$$

The expected value of $S^2$ has the desired value:

$$\mathrm{E}[S^2] = \sigma_X^2$$

The variance of $S^2$ depends on the underlying distribution and is, in general, difficult to calculate. However, as you would expect, the variance of $S^2$ decreases as $N$ increases.

Table 21.10 summarizes the concepts discussed in this section.

## Sampling Errors

When we estimate values such as the mean and standard deviation on the basis of a sample, we leave the realm of probability and enter that of statistics. This is a complex topic that will not be explored here, except to provide a few comments.

The probabilistic nature of our estimated values is a source of error, known as **sampling error**. In general, the greater the size of the sample taken, the smaller the standard deviation of the sample mean or other quantity, and therefore the closer that our estimate is likely to be to the actual value. By making certain reasonable assumptions about the nature of the random variable being tested and the randomness of the sampling procedure, one can in fact determine the probability that a sample mean or sample standard deviation is within a certain distance from the actual mean or standard deviation. This concept is often reported with the results of a sample. For example, it is common for the result of an opinion poll to include a comment such as, "The result is within 5% of the true value with a confidence (probability) of 99%."

There is, however, another source of error, which is less widely appreciated among nonstatisticians: **bias**. For example, if an opinion poll is conducted and only members of a certain socioeconomic group are interviewed, the results are not necessarily representative of the entire population. In a communications context, sampling done during one time of day may not reflect the activity at another time of day. If we are concerned with designing a system that will handle the peak load that is likely to be experienced, then we should observe the traffic during the time of day that is most likely to produce the greatest load.

**Table 21.10** Statistical Parameters

| | Population | Sample Mean | Sample Variance |
|---|---|---|---|
| **Random variable** | $X$ | $\overline{X} = \frac{1}{N}\sum_{i=1}^{N} X_i$ | $S^2 = \frac{1}{N-1}\sum_{i=1}^{N}(X_i - \overline{X})^2$ |
| **Expected value** | $\mathrm{E}[X] = \mu$ | $\mathrm{E}[\overline{X}] = \mu$ | $\mathrm{E}[S^2] = \sigma_X^2$ |
| **Variance** | $\mathrm{Var}[X] = \mathrm{E}[(X-\mu)^2] = \sigma_X^2$ | $\mathrm{Var}[\overline{X}] = \frac{\sigma_X^2}{N}$ | |

## 21.11 REFERENCES

**KLEI76** Kleinrock, L. *Queueing Systems, Volume II: Computer Applications.* New York: Wiley, 1976.

## 21.12 PROBLEMS

**21.1** Section 21.3 provided an intuitive argument to justify Little's formula. Develop a similar argument to justify the relationship $r = \lambda T_r$.

**21.2** Figure 21.3 shows the number of items in a system as a function of time. This can be viewed as the difference between an arrival process and a departure process, of the form $n(t) = a(t) - d(t)$.

**a.** On one graph, show the functions $a(t)$ and $d(t)$ that produce the $n(t)$ shown in Figure 21.3.

**b.** Using the graph from (a), develop an intuitive argument to justify Little's formula. *Hint*: Consider the area between the two step functions, computed first by adding vertical rectangles and second by adding horizontal rectangles.

**21.3** The owner of a shop observes that on average 18 customers per hour arrive and there are typically 8 customers in the shop. What is the average length of time each customer spends in the shop?

**21.4** A simulation program of a multiprocessor system starts running with no jobs in the queue and ends with no jobs in the queue. The simulation program reports the average number of jobs in the system over the simulation run as 12.356, the average arrival rate as 25.6 jobs per minute, and the average delay for a job as 8.34 minutes. Was the simulation correct?

**21.5** Section 21.3 provided an intuitive argument to justify the single-server relationship $\rho = \lambda T_s$. Develop a similar argument to justify the multiserver relationship $\rho = \lambda T_s/N$.

**21.6** If an M/M/1 queue has arrivals at a rate of two per minute and serves at a rate of four per minute, how many customers are found in the system on average? How many customers are found in service on average?

**21.7** What is the utilization of an M/M/1 queue that has four people waiting on average?

**21.8** At an ATM machine in a supermarket, the average length of a transaction is two minutes, and on average, customers arrive to use the machine once every five minutes. How long is the average time that a person must spend waiting and using the machine? What is the 90th percentile of residence time? On average, how many people are waiting to use the machine? Assume M/M/1.

**21.9** Messages arrive at random to be sent across a communications link with a data rate of 9,600 bps. The link is 70% utilized, and the average message length is 1,000 octets. Determine the average waiting time for constant-length messages and for exponentially distributed length messages.

**21.10** Messages of three different sizes flow through a message switch. Seventy percent of the messages take 1 millisecond to serve, 20% take 3 millisecond, and 10% take 10 millisecond. Calculate the average time spent in the switch, and the average number of messages in the switch, when messages arrive at an average rate of:

**a.** one per 3 milliseconds.

**b.** one per 4 milliseconds.

**c.** one per 5 milliseconds.

**21.11** Messages arrive at a switching center for a particular outgoing communications line in a Poisson manner with a mean arrival rate of 180 messages per hour. Message length

is distributed exponentially with a mean length of 14,400 characters. Line speed is 9,600 bps.

**a.** What is the mean waiting time in the switching center?

**b.** How many messages will be waiting in the switching center for transmission on the average?

**21.12** Often inputs to a queueing system are not independent and random, but occur in clusters. Mean waiting delays are greater for this type of arrival pattern than for Poisson arrivals. This problem demonstrates the effect with a simple example. Assume items arrive at a queue in fixed-size batches of $M$ items. The batches have a Poisson arrival distribution with mean rate $\lambda/M$, yielding a customer arrival rate of $\lambda$. For each item, the service time is $T_s$, and the standard deviation of service time of $\sigma_{Ts}$.

**a.** If we treat the batches as large-size items, what is the mean and variance of batch service time? What is the mean batch waiting time?

**b.** What is the mean waiting time for service for an item once its batch begins service? Assume an item may be in any of the $M$ positions in a batch with equal probability. What is the total mean waiting time for an item?

**c.** Verify the results of (b) by showing that for $M = 1$, the results reduce to the M/G/1 case. How do the results vary for values of $M > 1$?

**21.13** Consider a single queue with a constant service time of four seconds and a Poisson input with mean rate of 0.20 items per second.

**a.** Find the mean and standard deviation of queue size.

**b.** Find the mean and standard deviation of residence time.

**21.14** Consider a frame relay node that is handling a Poisson stream of incoming frames to be transmitted on a particular 1-Mbps outgoing link. The stream consists of two types of frames. Both types of frames have the same exponential distribution of frame length with a mean of 1,000 bits.

**a.** Assume priorities are not used. The combined arrival rate of frame of both types is 800 frames per second. What is the mean residence time ($T_r$) for all frames?

**b.** Now assume the two types are assigned different priorities, with the arrival rate of type 1 of 200 frames per second and the arrival rate of type 2 of 600 frames per second. Calculate the mean residence time for type 1, type 2, and overall.

**c.** Repeat (b) for $\lambda_1 = \lambda_2 = 400$ frames per second.

**d.** Repeat (b) for $\lambda_1 = 600$ frames per second and $\lambda_2 = 200$ frames per second.

**21.15** The Multilink Protocol (MLP) is part of X.25; a similar facility is used in IBM's System Network Architecture (SNA). With MLP, a set of data links exists between two nodes and is used as a pooled resource for transmitting packets, regardless of virtual circuit number. When a packet is presented to MLP for transmission, any available link can be chosen for the job. For example, if two LANs at different sites are connected by a pair of bridges, there may be multiple point-to-point links between the bridges to increase throughput and availability.

The MLP approach requires extra processing and frame overhead compared to a simple link protocol. A special MLP header is necessary for the protocol. An alternative is to assign each of the arriving packets to the queue for a single outgoing link in round-robin fashion. This would simplify processing, but what kind of effect would it have on performance?

Let us consider a concrete example. Suppose there are five 9,600-bps links connecting two nodes, the average packet size is 100 octets with an exponential distribution, and packets arrive at a rate of 48 per second.

**a.** For a single-server design, calculate $\rho$ and $T_r$.

**b.** For a multiserver design, it can be calculated that the Erlang C function has a value of 0.554. Determine $T_r$.

**21.16** A supplement to the X.25 packet-switching standard is a set of standards for a packet assembler-disassembler (PAD), defined in standards X.3, X.28, and X.29. A PAD is used to connect asynchronous terminals to a packet-switching network. Each terminal attached to a PAD sends characters one at a time. These are buffered in the PAD then assembled into an X.25 packet that is transmitted to the packet-switching network. The buffer length is equal to the maximum data field size for an X.25 packet. A packet is formed from assembled characters and transmitted whenever the buffer is full, a special control character such as a carriage return is received, or when a timeout occurs. For this problem, we ignore the last two conditions. Figure 21.10 illustrates the queueing model for the PAD. The first queue models the delay for characters waiting to be put into a packet; this queue is completely emptied when it is filled. The second queue models the delay waiting to transmit packets. Use the following notation:

$\lambda$ = Poisson input rate of characters from each termina.l
$C$ = Rate of transmission on the output channel in characters per second.
$M$ = Number of data characters in a packet.
$H$ = Number of overhead characters in a packet.
$K$ = Number of terminals.

**a.** Determine the average waiting time for a character in the input queue.

**b.** Determine the average waiting time for a packet in the output queue.

**c.** Determine the average time spent by a character from when it leaves the terminal to when it leaves the PAD. Plot the result as a function of normalized load.

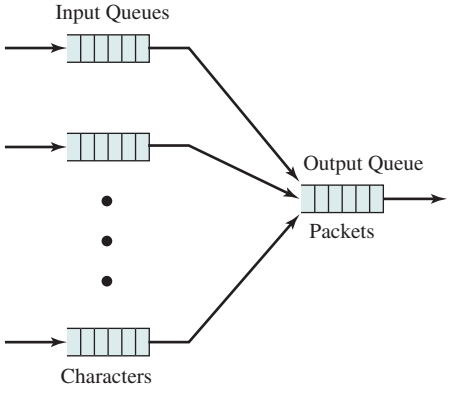



**Figure 21.10 Queueing Model for a Packet Assembler/Disassembler (PAD)**

**21.17** A fraction $P$ of the traffic from a single exponential server is fed back into the input as shown in Figure 21.11. In the figure, $\Lambda$ denotes the system throughput, which is the output rate from the server.

**a.** Determine the system throughput and the server utilization and the mean residence time for one pass through the server.

**b.** Determine the mean number of passes that an item makes through the system and the mean total time spent in the system.

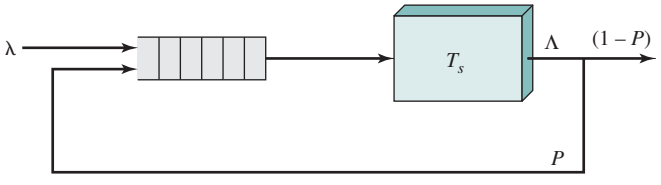



**Figure 21.11 Feedback Queue**

# PROGRAMMING PROJECT ONE

## DEVELOPING A SHELL

The Shell or Command Line Interpreter is the fundamental User interface to an operating system. Your first project is to write a simple shell—`myshell`—that has the following properties:

1. The shell must support the following internal commands:
   - i. `cd <directory>`—Change the current default directory to `<directory>`. If the `<directory>` argument is not present, report the current directory. If the directory does not exist, an appropriate error should be reported. This command should also change the `PWD` environment variable.
   - ii. `clr`—Clear the screen.
   - iii. `dir <directory>`—List the contents of directory `<directory>`.
   - iv. `environ`—List all the environment strings.
   - v. `echo <comment>`—Display `<comment>` on the display followed by a new line (multiple spaces/tabs may be reduced to a single space).
   - vi. `help`—Display the user manual using the `more` filter.
   - vii. `pause`—Pause operation of the shell until "`Enter`" is pressed.
   - viii. `quit`—Quit the shell.
   - ix. The shell environment should contain `shell=<pathname>/myshell` where `<pathname>/myshell` is the full path for the shell executable (not a hardwired path back to your directory, but the one from which it was executed).
2. All other command line input is interpreted as program invocation, which should be done by the shell `fork`ing and `exec`ing the programs as its own child processes. The programs should be executed with an environment that contains the entry: `parent=<pathname>/myshell` where `<pathname>/myshell` is as described in 1.ix above.
3. The shell must be able to take its command line input from a file. That is, if the shell is invoked with a command line argument:

   ```
   myshell batchfile
   ```

   then `batchfile` is assumed to contain a set of command lines for the shell to process. When the end-of-file is reached, the shell should exit. Obviously, if the shell is invoked without a command line argument, it solicits input from the user via a prompt on the display.
4. The shell must support I/O redirection on either or both *stdin* and/or *stdout*. That is, the command line

   ```
   programname arg1 arg2 < inputfile > outputfile
   ```

   will execute the program `programname` with arguments `arg1` and `arg2`, the *stdin FILE stream* replaced by `inputfile` and the *stdout FILE stream* replaced by `outputfile`.

`stdout` redirection should also be possible for the internal commands `dir`, `environ`, `echo`, and `help`.

With output redirection, if the redirection character is > then the `outputfile` is created if it does not exist, and truncated if it does. If the redirection token is >> then `outputfile` is created if it does not exist, and appended to if it does.

5. The shell must support background execution of programs. An ampersand (`&`) at the end of the command line indicates that the shell should return to the command line prompt immediately after launching that program.
6. The command line prompt must contain the pathname of the current directory.

*Note:* You can assume all command line arguments (including the redirection symbols, <, > & >> and the background execution symbol, `&`) will be delimited from other command line arguments by white space—one or more spaces and/or tabs (see the command line in 4. above).

## PROJECT REQUIREMENTS

1. Design a simple command line shell that satisfies the above criteria and implement it on the specified UNIX platform.
2. Write a simple manual describing how to use the shell. The manual should contain enough detail for a beginner to UNIX to use it. For example, you should explain the concepts of I/O redirection, the program environment, and background program execution. The manual MUST be named `readme` and must be a simple text document capable of being read by a standard Text Editor.

   For an example of the sort of depth and type of description required, you should have a look at the online manuals for `csh` and `tcsh` (`man csh`, `man tcsh`). These shells obviously have much more functionality than yours and thus, your manuals don't have to be quite so large.

   You should NOT include building instructions, included file lists, or source code—we can find that out from the other files you submit. This should be an Operator's manual not a Developer's manual.
3. The source code **MUST** be extensively commented and appropriately structured to allow your peers to understand and easily maintain the code. Properly commented and laid out code is much easier to interpret, and it is in your interests to ensure the person marking your project is able to understand your coding without having to perform mental gymnastics!
4. Details of submission procedures will be supplied well before the deadline.
5. The submission should contain only source code file(s), include file(s), a `makefile` (all lowercase please), and the `readme` file (all lowercase, please). No executable program should be included. The person marking your project will be automatically rebuilding your shell program from the source code provided. If the submitted code does not compile, it cannot be marked!

6. The `makefile` (all lowercase, please) **MUST** generate the binary file `myshell` (all lowercase please). A sample `makefile` would be

```
# Joe Citizen, s1234567 - Operating Systems Project 1
# CompLab1/01 tutor: Fred Bloggs
myshell: myshell.c utility.c myshell.h
    gcc -Wall myshell.c utility.c -o myshell
```

   The program `myshell` is then generated by just typing `make` at the command line prompt.

   *Note:* The fourth line in the above `makefile` **MUST** begin with a tab.

7. In the instance shown above, the files in the submitted directory would be:

```
makefile
myshell.c
utility.c
myshell.h
readme
```

## SUBMISSION

A `makefile` is required. All files in your submission will be copied to the same directory, therefore, do not include any paths in your `makefile`. The `makefile` should include all dependencies that build your program. If a library is included, your `makefile` should also build the library.

**Do not hand in any binary or object code files**. All that is required is your source code, a `makefile`, and a `readme` file. Test your project by copying the source code only into an empty directory then compile it by entering the command `make`.

We shall be using a shell script that copies your files to a test directory, deletes any preexisting `myshell`, `*.a`, and/or `*.o` files, performs a `make`, copies a set of test files to the test directory, and then exercises your shell with a standard set of test scripts through *stdin* and command line arguments. If this sequence fails due to wrong names, wrong case for names, wrong version of source code that fails to compile, nonexistence of files, and so on, then the marking sequence will also stop. In this instance, the only marks that can be awarded will be for the tests completed at that point, and the source code and manual.

## REQUIRED DOCUMENTATION

Your source code will be assessed and marked as well as the `readme` manual. Commenting is definitely required in your source code. The user manual can be presented in a format of your choice (within the limitations of being displayable by a simple Text Editor). Again, the manual should contain enough detail for a beginner to UNIX to use the shell. For example, you should explain the concepts of I/O redirection, the program environment, and background program execution. The manual **MUST** be named `readme` (all lowercase, please, and **NO** `.txt` extension).

# PROGRAMMING PROJECT TWO

## THE HOST DISPATCHER SHELL

The Hypothetical Operating System Testbed (HOST) is a multiprogramming system with a four-level priority process dispatcher operating within the constraints of finite available resources.

## FOUR-LEVEL PRIORITY DISPATCHER

The dispatcher operates at four priority levels:

1. Real-Time processes must be run immediately on a first-come-first-served (FCFS) basis, preempting any other processes running with lower priority. These processes are run until completion.
2. Normal user processes are run on a three-level feedback dispatcher (see Figure PP2.1). The basic timing quantum of the dispatcher is one second. This is also the value for the time quantum of the feedback scheduler.

The dispatcher needs to maintain two submission queues—Real-Time and User priority—fed from the job dispatch list. The dispatch list is examined at every dispatcher tick and jobs that "have arrived" are transferred to the appropriate submission queue. The submission queues are then examined; any Real-Time jobs are run to completion, preempting any other jobs currently running.

The Real-Time priority job queue must be empty before the lower-priority feedback dispatcher is reactivated. Any User priority jobs in the User job queue that can run within available resources (memory and I/O devices) are transferred to the appropriate priority queue. Normal operation of a feedback queue will accept all

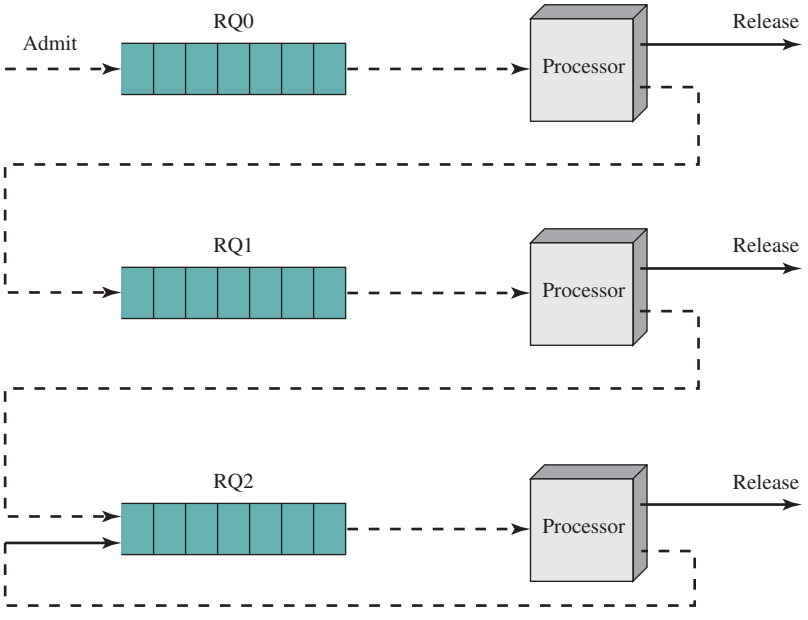



**Figure PP2.1**




**Figure PP2.2**

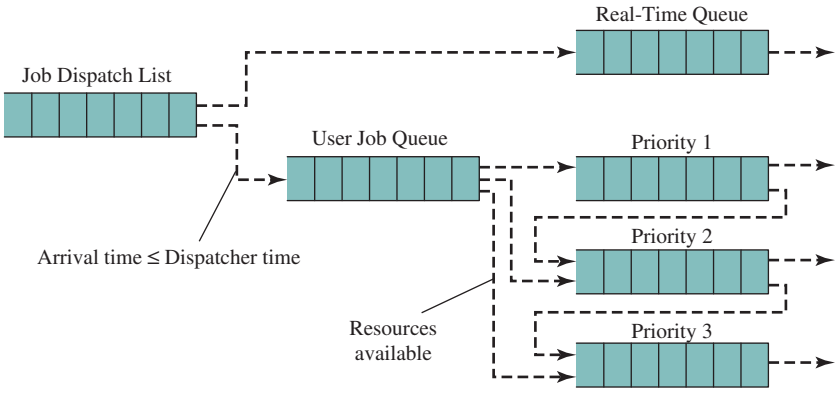



**Figure PP2.3**

jobs at the highest-priority level and degrade the priority after each completed time quantum. However, this dispatcher has the ability to accept jobs at a lower priority, inserting them in the appropriate queue. This enables the dispatcher to emulate a simple round-robin dispatcher (see Figure PP2.2) if all jobs are accepted at the lowest priority.

When all "ready" higher-priority jobs have been completed, the feedback dispatcher resumes by starting or resuming the process at the head of the highest-priority nonempty queue. At the next tick the current job is suspended (or terminated and its resources released) if there are any other jobs "ready" of an equal or higher priority.

The logic flow should be as shown in Figure PP2.3 (and as discussed subsequently in this project assignment).

## RESOURCE CONSTRAINTS

The HOST has the following resources:

- 2 Printers
- 1 Scanner
- 1 Modem
- 2 CD drives
- 1024 Mbytes of memory available for processes

Low-priority processes can use any or all of these resources, but the HOST dispatcher is notified of which resources the process will use when the process is submitted. The dispatcher ensures that each requested resource is solely available to that process throughout its lifetime in the "ready-to-run" dispatch queues: from the initial transfer from the job queue to the Priority 1–3 queues through to process completion, including intervening idle time quanta.

Real-Time processes will not need any I/O resources (Printer, Scanner, Modem, and CD), but will obviously require memory allocation—this memory requirement will always be 64 Mbytes or less for Real-Time jobs.

## MEMORY ALLOCATION

For each process, a **contiguous** block of memory must be assigned. The memory block must remain assigned to the process for the lifetime of the process.

Enough contiguous spare memory must be left so the Real-Time processes are not blocked from execution—64 Mbytes for a running Real-Time job, leaving 960 Mbytes to be shared among "active" User jobs.

The HOST hardware MMU cannot support virtual memory, so no swapping of memory to disk is possible. Neither is it a paged system.

Within these constraints, any suitable variable partition memory allocation scheme (First Fit, Next Fit, Best Fit, Worst Fit, Buddy, and so on) may be used.

## PROCESSES

Processes on HOST are simulated by the dispatcher creating a new process for each dispatched process. This process is a generic process (supplied as `process`—source: `sigtrap.c`) that can be used for any priority process. It actually runs itself at very low priority, sleeping for one-second periods and displaying the following:

1. A message displaying the process ID when the process starts;
2. A regular message every second the process is executed, and;
3. A message when the process is Suspended, Continued, or Terminated.

The process will terminate of its own accord after 20 seconds if it is not terminated by your dispatcher. The process prints out using a randomly generated color scheme for each unique process, so individual "slices" of processes can be easily distinguishable. Use this process rather than your own.

The life cycle of a process is as follows:

1. The process is submitted to the dispatcher input queues via an initial process list that designates the arrival time, priority, processor time required (in seconds), memory block size, and other resources requested.
2. A process is "ready-to-run" when it has "arrived" and all required resources are available.
3. Any pending Real-Time jobs are submitted for execution on a FCFS basis.

4. If enough resources and memory are available for a lower-priority User process, the process is transferred to the appropriate priority queue within the feedback dispatcher unit, and the remaining resource indicators (memory list and I/O devices) are updated.
5. When a job is started (`fork` and `exec("process",...)`), the dispatcher will display the job parameters (Process ID, priority, processor time remaining (in seconds), memory location and block size, and resources requested) before performing the `exec`.
6. A Real-Time process is allowed to run until its time has expired when the dispatcher kills it by sending a `SIGINT` signal to it.
7. A low-priority User job is allowed to run for one dispatcher tick (one second) before it is suspended (`SIGTSTP`) or terminated (`SIGINT`) if its time has expired. If suspended, its priority level is lowered (if possible) and it is requeued on the appropriate priority queue as shown in Figures P2.1 and P2.3. To retain synchronization of output between your dispatcher and the child process, your dispatcher should wait for the process to respond to a `SIGTSTP` or `SIGINT` signal before continuing (`waitpid(p->pid, &status, WUNTRACED)`). To match the performance sequence indicated in the comparison of scheduling policies (see Figure 9.5), the User job should not be suspended and moved to a lower-priority level unless another process is waiting to be (re)started.
8. Provided no higher-priority Real-Time jobs are pending in the submission queue, the highest-priority pending process in the feedback queues is started or restarted (`SIGCONT`).
9. When a process is terminated, the resources it used are returned to the dispatcher for reallocation to further processes.
10. When there are no more processes in the dispatch list—the input queues and the feedback queues—the dispatcher exits.

## DISPATCH LIST

The Dispatch List is the list of processes to be processed by the dispatcher. The list is contained in a text file that is specified on the command line. That is,

```
>hostd dispatchlist
```

Each line of the list describes one process with the following data as a *"comma-space"* delimited list:

```
<arrival time>, <priority>, <processor time>, <mbytes>,
<#printers>, <#scanners>, <#modems>, <#CDs>
```

Thus,

```
12, 0, 1, 64, 0, 0, 0, 0
12, 1, 2, 128, 1, 0, 0, 1
13, 3, 6, 128, 1, 0, 1, 2
```

would indicate the following:

| | |
|---|---|
| **1st Job:** | Arrival at time 12, priority 0 (Real-Time), requiring 1 second of processor time and 64 Mbytes of memory—no I/O resources required. |
| **2nd Job:** | Arrival at time 12, priority 1 (high-priority User job), requiring 2 seconds of processor time, 128 Mbytes of memory, 1 printer, and 1 CD drive. |
| **3rd Job:** | Arrival at time 13, priority 3 (lowest-priority User job), requiring 6 seconds of processor time, 128 Mbytes of memory, 1 printer, 1 modem, and 2 CD drives. |

The submission text file can be of any length, containing up to 1000 jobs. It will be terminated with an end-of-line followed by an end-of-file marker.

Dispatcher input lists to test the operation of the individual features of the dispatcher are described subsequently in this project assignment. It should be noted that these lists will almost certainly form the basis of tests that will be applied to your dispatcher during marking. Operation as described in the exercises will be expected.

Obviously, your submitted dispatcher will be tested with more complex combinations as well!

A fully functional working example of the dispatcher will be presented during the course. If in any doubt as to the manner of operation or format of output, you should refer to this program to observe how your dispatcher is expected to operate.

## PROJECT REQUIREMENTS

1. Design a dispatcher that satisfies the above criteria. In a formal design document,

   a. Describe and discuss what memory allocation algorithms you could have used and justify your final design choice.

   b. Describe and discuss the structures used by the dispatcher for queueing, dispatching, and allocating memory and other resources.

   c. Describe and justify the overall structure of your program, describing the various modules and major functions (descriptions of the function "interfaces" are expected).

   d. Discuss why such a multilevel dispatching scheme would be used, comparing it with schemes used by "real" operating systems. Outline shortcomings in such a scheme, suggesting possible improvements. Include the memory and resource allocation schemes in your discussions.

   The formal design document is expected to have in-depth discussions, descriptions, and arguments. The design document is to be submitted separately as a physical paper document. The design document should NOT include any source code.

2. Implement the dispatcher using the C language.
3. The source code MUST be extensively commented and appropriately structured to allow your peers to understand and easily maintain the code. Properly commented and laid out code is much easier to interpret, and it is in your interests to ensure the person marking your project is able to understand your coding without having to perform mental gymnastics.
4. Details of submission procedures will be supplied well before the deadline.
5. The submission should contain only source code file(s), include file(s), and a `makefile`. No executable program should be included. The marker will be automatically rebuilding your program from the source code provided. If the submitted code does not compile, it cannot be marked.
6. The `makefile` should generate the binary executable file `hostd` (all lowercase please). A sample `makefile` would be as follows:

```
# Joe Citizen, s1234567 - Operating Systems Project 2
# CompLab1/01 tutor: Fred Bloggs
hostd: hostd.c utility.c hostd.h
gcc hostd.c utility.c -o hostd
```

The program `hostd` is then generated by typing `make` at the command line prompt. Note: The fourth line in the above `makefile` **MUST** begin with a `tab`.

## DELIVERABLES

1. Source code file(s), include file(s), and a `makefile`.
2. The design document as outlined in Project Requirements section 1 above.

## SUBMISSION OF CODE

A `makefile` is required. All files will be copied to the same directory; therefore, *do not include any paths in your makefile*. The `makefile` should include all dependencies that build your program. If a library is included, your `makefile` should also build the library.

*Do not submit any binary or object code files*. All that is required is your source code and a `makefile`. Test your project by copying the source code only into an *empty* directory then compile it with your `makefile`.

The marker will be using a shell script that copies your files to a test directory, performs a `make`, then exercises your dispatcher with a standard set of test files. If this sequence fails due to wrong names, wrong case for names, wrong version of source code that fails to compile, nonexistence of files, etc., then the marking sequence will also stop. In this instance, the only further marks that can be awarded will be for the source code and design document.

# APPENDIX C

## TOPICS IN CONCURRENCY


**C.1 Processor Registers**
- User-Visible Registers
- Control and Status Registers

**C.2 Instruction Execution For I/O Functions**

**C.3 I/O Communication Techniques**
- Programmed I/O
- Interrupt-Driven I/O
- Direct Memory Access

**C.4 Hardware Performance Issues For Multicore**
- Increase in Parallelism
- Power Consumption

**C.5 Reference**

This appendix provides additional details to supplement Chapter 1.

## C.1 PROCESSOR REGISTERS

A processor includes a set of registers that provide memory that is faster and smaller than main memory. Processor registers serve two functions:

- **User-visible registers:** Enable the machine or assembly language programmer to minimize main memory references by optimizing register use. For high-level languages, an optimizing compiler will attempt to make intelligent choices of which variables to assign to registers and which to main memory locations. Some high-level languages such as C allow the programmer to suggest to the compiler which variables should be held in registers.
- **Control and status registers:** Used by the processor to control the operation of the processor, and by privileged OS routines to control the execution of programs.

There is not a clean separation of registers into these two categories. For example, on some processors, the program counter is user visible, but on many it is not. For purposes of the following discussion, however, it is convenient to use these categories.

### User-Visible Registers

A user-visible register may be referenced by means of the machine language that the processor executes and is generally available to all programs, including application programs as well as system programs. Types of registers that are typically available are data, address, and condition code registers.

**Data registers** can be assigned to a variety of functions by the programmer. In some cases, they are general purpose in nature and can be used with any machine instruction that performs operations on data. Often, however, there are restrictions. For example, there may be dedicated registers for floating-point operations, and others for integer operations.

**Address registers** contain main memory addresses of data and instructions, or they contain a portion of the address that is used in the calculation of the complete or effective address. These registers may themselves be general purpose, or may be devoted to a particular way, or mode, of addressing memory. Examples include the following:

- **Index register:** Indexed addressing is a common mode of addressing that involves adding an index to a base value to get the effective address.
- **Segment pointer:** With segmented addressing, memory is divided into segments, which are variable-length blocks of words.[1] A memory reference consists of a reference to a particular segment and an offset within the segment;

[1] There is no universal definition of the term *word*. In general, a **word** is an ordered set of bytes or bits that is the normal unit in which information may be stored, transmitted, or operated on within a given computer. Typically, if a processor has a fixed-length instruction set, then the instruction length equals the word length.

this mode of addressing is important in our discussion of memory management in Chapter 7. In this mode of addressing, a register is used to hold the base address (starting location) of the segment. There may be multiple registers; for example, one for the OS (i.e., when OS code is executing on the processor) and one for the currently executing application.

- **Stack pointer:** If there is user-visible stack[2] addressing, then there is a dedicated register that points to the top of the stack. This allows the use of instructions that contain no address field, such as push and pop.

For some processors, a procedure call will result in automatic saving of all user-visible registers, to be restored on return. Saving and restoring is performed by the processor as part of the execution of the call and return instructions. This allows each procedure to use these registers independently. On other processors, the programmer must save the contents of the relevant user-visible registers prior to a procedure call, by including instructions for this purpose in the program. Thus, the saving and restoring functions may be performed in either hardware or software, depending on the processor.

## Control and Status Registers

A variety of processor registers are employed to control the operation of the processor. On most processors, most of these are not visible to the user. Some of them may be accessible by machine instructions executed in what is referred to as a control or kernel mode.

Of course, different processors will have different register organizations and use different terminology. We provide here a reasonably complete list of register types, with a brief description. In addition to the MAR, MBR, I/OAR, and I/OBR registers mentioned in Chapter 1 (see Figure 1.1), the following are essential to instruction execution:

- **Program counter (PC):** Contains the address of the next instruction to be fetched
- **Instruction register (IR):** Contains the instruction most recently fetched

All processor designs also include a register or set of registers, often known as the program status word (PSW) that contains status information. The PSW typically contains condition codes plus other status information, such as an interrupt enable/disable bit and a kernel/user mode bit.

**Condition codes** (also referred to as *flags*) are bits typically set by the processor hardware as the result of operations. For example, an arithmetic operation may produce a positive, negative, zero, or overflow result. In addition to the result itself being stored in a register or memory, a condition code is also set following the execution of the arithmetic instruction. The condition code may subsequently be tested as part of a conditional branch operation. Condition code bits are collected into one or more registers. Usually, they form part of a control register. Generally, machine instructions

[2]A stack is located in main memory and is a sequential set of locations that are referenced similarly to a physical stack of papers, by putting on and taking away from the top. See Appendix P for a discussion of stack processing.

allow these bits to be read by implicit reference, but they cannot be altered by explicit reference because they are intended for feedback regarding the results of instruction execution.

In processors with multiple types of interrupts, a set of interrupt registers may be provided, with one pointer to each interrupt-handling routine. If a stack is used to implement certain functions (e.g., procedure call), then a stack pointer is needed (see Appendix 1B). Memory management hardware, discussed in Chapter 7, requires dedicated registers. Finally, registers may be used in the control of I/O operations.

A number of factors go into the design of the control and status register organization. One key issue is OS support. Certain types of control information are of specific utility to the OS. If the processor designer has a functional understanding of the OS to be used, then the register organization can be designed to provide hardware support for particular features such as memory protection and switching between user programs.

Another key design decision is the allocation of control information between registers and memory. It is common to dedicate the first (lowest) few hundred or thousand words of memory for control purposes. The designer must decide how much control information should be in more expensive, faster registers and how much in less expensive, slower main memory.

## C.2 INSTRUCTION EXECUTION FOR I/O FUNCTIONS

This section supplements the information in Section 1.3.

Data can be exchanged directly between an I/O module (e.g., a disk controller) and the processor. Just as the processor can initiate a read or write with memory, specifying the address of a memory location, the processor can also read data from or write data to an I/O module. In this latter case, the processor identifies a specific device that is controlled by a particular I/O module. Thus, an instruction sequence similar in form to that of Figure 1.4 could occur, with I/O instructions rather than memory-referencing instructions.

In some cases, it is desirable to allow I/O exchanges to occur directly with main memory to relieve the processor of the I/O task. In such a case, the processor grants to an I/O module the authority to read from or write to memory, so the I/O-memory transfer can occur without tying up the processor. During such a transfer, the I/O module issues read or write commands to memory, relieving the processor of responsibility for the exchange. This operation, known as direct memory access (DMA), is examined in Section 1.7.

## C.3 I/O COMMUNICATION TECHNIQUES

Three techniques are possible for I/O operations:

- Programmed I/O
- Interrupt-driven I/O
- Direct memory access (DMA)

## Programmed I/O

When the processor is executing a program and encounters an instruction relating to I/O, it executes that instruction by issuing a command to the appropriate I/O module. In the case of programmed I/O, the I/O module performs the requested action then sets the appropriate bits in the I/O status register, but takes no further action to alert the processor. In particular, it does not interrupt the processor. Thus, after the I/O instruction is invoked, the processor must take some active role in determining when the I/O instruction is completed. For this purpose, the processor periodically checks the status of the I/O module until it finds that the operation is complete.

With this technique, the processor is responsible for extracting data from main memory for output, and storing data in main memory for input. I/O software is written in such a way that the processor executes instructions that give it direct control of the I/O operation, including sensing device status, sending a read or write command, and transferring the data. Thus, the instruction set includes I/O instructions in the following categories:

- **Control:** Used to activate an external device and tell it what to do. For example, a magnetic-tape unit may be instructed to rewind or to move forward one record.
- **Status:** Used to test various status conditions associated with an I/O module and its peripherals.
- **Transfer:** Used to read and/or write data between processor registers and external devices.

Figure C.1a gives an example of the use of programmed I/O to read in a block of data from an external device (e.g., a record from tape) into memory. Data are read in one word (e.g., 16 bits) at a time. For each word that is read in, the processor must remain in a status-checking loop until it determines that the word is available in the I/O module's data register. This flowchart highlights the main disadvantage of this technique: It is a time-consuming process that keeps the processor busy needlessly.

## Interrupt-Driven I/O

With programmed I/O, the processor has to wait a long time for the I/O module of concern to be ready for either reception or transmission of more data. The processor, while waiting, must repeatedly interrogate the status of the I/O module. As a result, the performance level of the entire system is severely degraded.

An alternative is for the processor to issue an I/O command to a module then go on to do some other useful work. The I/O module will then interrupt the processor to request service when it is ready to exchange data with the processor. The processor then executes the data transfer, as before, and resumes its former processing.

Let us consider how this works, first from the point of view of the I/O module. For input, the I/O module receives a READ command from the processor. The I/O module then proceeds to read data in from an associated peripheral. Once the data are in the module's data register, the module signals an interrupt to the processor over a control line. The module then waits until its data are requested by the

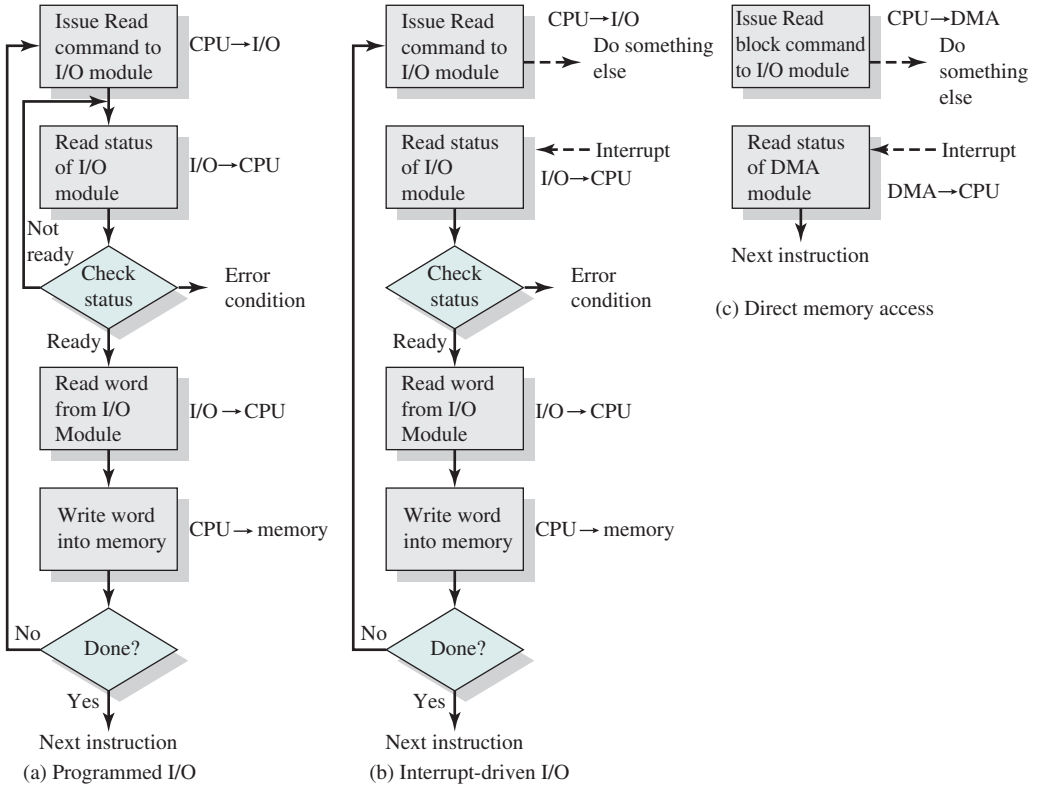



**Figure C.1 Three Techniques for Input of a Block of Data**

processor. When the request is made, the module places its data on the data bus and is then ready for another I/O operation.

From the processor's point of view, the action for input is as follows. The processor issues a READ command. It then saves the context (e.g., program counter and processor registers) of the current program, and goes off and does something else (e.g., the processor may be working on several different programs at the same time). At the end of each instruction cycle, the processor checks for interrupts (see Figure 1.7). When the interrupt from the I/O module occurs, the processor saves the context of the program it is currently executing and begins to execute an interrupt-handling program that processes the interrupt. In this case, the processor reads the word of data from the I/O module and stores it in memory. It then restores the context of the program that had issued the I/O command (or some other program) and resumes execution.

Figure C.1b shows the use of interrupt-driven I/O for reading in a block of data. Interrupt-driven I/O is more efficient than programmed I/O because it eliminates needless waiting. However, interrupt-driven I/O still consumes a lot of processor time, because every word of data that goes from memory to I/O module, or from I/O module to memory, must pass through the processor.

Almost invariably, there will be multiple I/O modules in a computer system, so mechanisms are needed to enable the processor to determine which device caused the interrupt and to decide, in the case of multiple interrupts, which one to handle first. In some systems, there are multiple interrupt lines, so that each I/O module signals on a different line. Each line will have a different priority. Alternatively, there can be a single interrupt line, but additional lines are used to hold a device address. Again, different devices are assigned different priorities.

## Direct Memory Access (DMA)

Interrupt-driven I/O, though more efficient than simple programmed I/O, still requires the active intervention of the processor to transfer data between memory and an I/O module, and any data transfer must traverse a path through the processor. Thus, both of these forms of I/O suffer from two inherent drawbacks:

1. The I/O transfer rate is limited by the speed with which the processor can test and service a device.
2. The processor is tied up in managing an I/O transfer; a number of instructions must be executed for each I/O transfer.

When large volumes of data are to be moved, a more efficient technique is required: direct memory access (DMA). The DMA function can be performed by a separate module on the system bus or it can be incorporated into an I/O module. In either case, the technique works as follows. When the processor wishes to read or write a block of data, it issues a command to the DMA module, by sending the following information to the DMA module:

- Whether a read or write is requested
- The address of the I/O device involved
- The starting location in memory to read data from or write data to
- The number of words to be read or written

The processor then continues with other work. It has delegated this I/O operation to the DMA module, and that module will take care of it. The DMA module transfers the entire block of data, one word at a time, directly to or from memory without going through the processor. When the transfer is complete, the DMA module sends an interrupt signal to the processor. Thus the processor is involved only at the beginning and end of the transfer (see Figure C.1c).

The DMA module needs to take control of the bus to transfer data to and from memory. Because of this competition for bus usage, there may be times when the processor needs the bus and must wait for the DMA module. Note this is not an interrupt; the processor does not save a context and do something else. Rather, the processor pauses for one bus cycle (the time it takes to transfer one word across the bus). The overall effect is to cause the processor to execute more slowly during a DMA transfer when processor access to the bus is required. Nevertheless, for a multiple-word I/O transfer, DMA is far more efficient than interrupt-driven or programmed I/O.

## C.4 HARDWARE PERFORMANCE ISSUES FOR MULTICORE

Microprocessor systems have experienced a steady, exponential increase in execution performance for decades. This increase is due partly to refinements in the organization of the processor on the chip, and partly to the increase in the clock frequency.

### Increase in Parallelism

The organizational changes in processor design have primarily been focused on increasing instruction-level parallelism, so more work could be done in each clock cycle. These changes include, in chronological order (see Figure C.2):

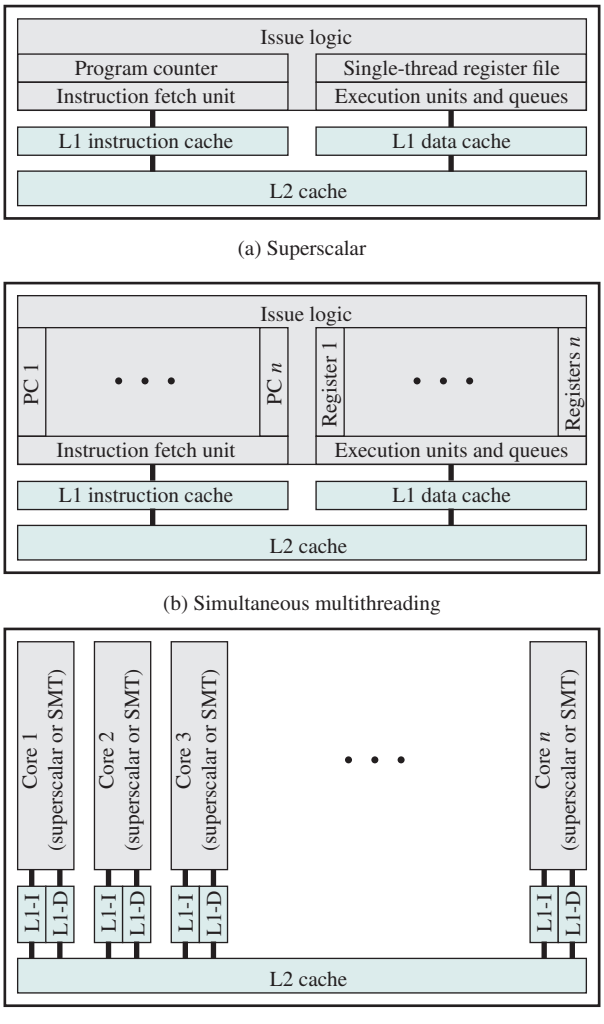



**Figure C.2 Alternative Chip Organizations**

- **Pipelining:** Individual instructions are executed through a pipeline of stages so while one instruction is executing in one stage of the pipeline, another instruction is executing in another stage of the pipeline.
- **Superscalar:** Multiple pipelines are constructed by replicating execution resources. This enables parallel execution of instructions in parallel pipelines, so long as hazards are avoided.
- **Simultaneous multithreading (SMT):** Register banks are replicated so multiple threads can share the use of pipeline resources.

For each of these innovations, designers have over the years attempted to increase the performance of the system by adding complexity. In the case of pipelining, simple three-stage pipelines were replaced by pipelines with five stages, then many more stages, with some implementations having over a dozen stages. There is a practical limit to how far this trend can be taken, because with more stages, there is the need for more logic, more interconnections, and more control signals. With superscalar organization, performance increases can be achieved by increasing the number of parallel pipelines. Again, there are diminishing returns as the number of pipelines increases. More logic is required to manage hazards and to stage instruction resources. Eventually, a single thread of execution reaches the point where hazards and resource dependencies prevent the full use of the multiple pipelines available. This same point of diminishing returns is reached with SMT, as the complexity of managing multiple threads over a set of pipelines limits the number of threads and number of pipelines that can be effectively utilized.

There is a related set of problems dealing with the design and fabrication of the computer chip. The increase in complexity to deal with all of the logical issues related to very long pipelines, multiple superscalar pipelines, and multiple SMT register banks means that increasing amounts of the chip area is occupied with coordinating and signal transfer logic. This increases the difficulty of designing, fabricating, and debugging the chips. The increasingly difficult engineering challenge related to processor logic is one of the reasons that an increasing fraction of the processor chip is devoted to the simpler memory logic. Power issues, discussed next, provide another reason.

## Power Consumption

To maintain the trend of higher performance as the number of transistors per chip rise, designers have resorted to more elaborate processor designs (pipelining, superscalar, and SMT) and to high clock frequencies. Unfortunately, power requirements have grown exponentially as chip density and clock frequency have risen.

One way to control power density is to use more of the chip area for cache memory. Memory transistors are smaller and have a power density an order of magnitude lower than that of logic (see Figure C.3). The percentage of the chip area devoted to memory has grown to exceed 50% as the chip transistor density has increased.

How to use all those logic transistors is a key design issue. As discussed earlier in this section, there are limits to the effective use of such techniques as superscalar and SMT. In general terms, the experience of recent decades has been encapsulated

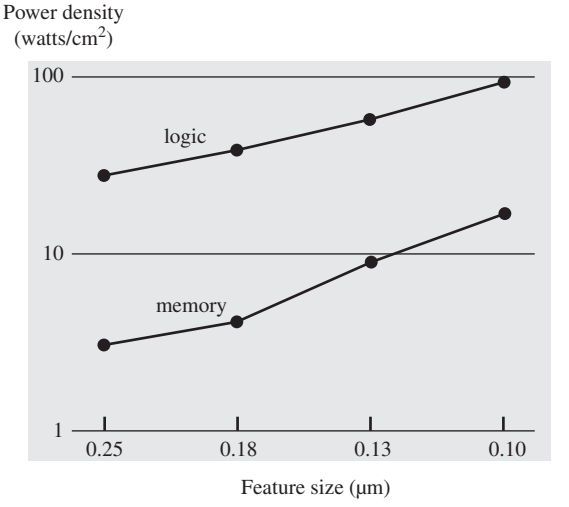



**Figure C.3 Power and Memory Considerations**

in a rule of thumb known as **Pollack's rule** [POLL99], which states that performance increase is roughly proportional to square root of increase in complexity. In other words, if you double the logic in a processor core, then it delivers only 40% more performance. In principle, the use of multiple cores has the potential to provide near-linear performance improvement with the increase in the number of cores.

Power considerations provide another motive for moving toward a multicore organization. Because the chip has such a huge amount of cache memory, it becomes unlikely that any one thread of execution can effectively use all that memory. Even with SMT, you are multithreading in a relatively limited fashion and cannot therefore fully exploit a gigantic cache, whereas a number of relatively independent threads or processes has a greater opportunity to take full advantage of the cache memory.

## C.5 REFERENCE

**POLL99** Pollack, F. "New Microarchitecture Challenges in the Coming Generations of CMOS Process Technologies (keynote address)." *Proceedings of the 32nd annual ACM/IEEE International Symposium on Microarchitecture*, 1999.

# APPENDIX D

## OBJECT-ORIENTED DESIGN


**D.1 Motivation**

**D.2 Object-Oriented Concepts**
- Object Structure
- Object Classes
- Containment

**D.3 Benefits of Object-Oriented Design**

**D.4 CORBA**

**D.5 Recommended Reading and Website**

Windows and several other contemporary operating systems rely heavily on object-oriented design principles. This appendix provides a brief overview of the main concepts of object-oriented design.

## D.1 MOTIVATION

Object-oriented concepts have become quite popular in the area of computer programming, with the promise of interchangeable, reusable, easily updated, and easily interconnected software parts. More recently, database designers have begun to appreciate the advantages of an object orientation, with the result that object-oriented database management systems (OODBMS) are beginning to appear. Operating systems designers have also recognized the benefits of the object-oriented approach.

Object-oriented programming and object-oriented database management systems are in fact different things, but they share one key concept: that software or data can be "containerized." Everything goes into a box, and there can be boxes within boxes. In the simplest conventional program, one program step equates to one instruction; in an object-oriented language, each step might be a whole boxful of instructions. Similarly, with an object-oriented database, one variable, instead of equating to a single data element, may equate to a whole boxful of data.

Table D.1 introduces some of the key terms used in object-oriented design.

**Table D.1** Key Object-Oriented Terms

| Term | Definition |
|---|---|
| Attribute | Data variables contained within an object. |
| Containment | A relationship between two object instances in which the containing object includes a pointer to the contained object. |
| Encapsulation | The isolation of the attributes and services of an object instance from the external environment. Services may only be invoked by name and attributes may only be accessed by means of the services. |
| Inheritance | A relationship between two object classes in which the attributes and services of a parent class are acquired by a child class. |
| Interface | A description closely related to an object class. An interface contains method definitions (without implementations) and constant values. An interface cannot be instantiated as an object. |
| Message | The means by which objects interact. |
| Method | A procedure that is part of an object and that can be activated from outside the object to perform certain functions. |
| Object | An abstraction of a real-world entity. |
| Object class | A named set of objects that share the same names, sets of attributes, and services. |
| Object instance | A specific member of an object class, with values assigned to the attributes. |
| Polymorphism | Refers to the existence of multiple objects that use the same names for services and present the same interface to the external world but that represent different types of entities. |
| Service | A function that performs an operation on an object. |

## D.2 OBJECT-ORIENTED CONCEPTS

The central concept of object-oriented design is the object. An object is a distinct software unit that contains a collection of related variables (data) and methods (procedures). Generally, these variables and methods are not directly visible outside the object. Rather, well-defined interfaces exist that allow other software to have access to the data and the procedures.

An object represents some thing, be it a physical entity, a concept, a software module, or some dynamic entity such as a TCP connection. The values of the variables in the object express the information that is known about the thing that the object represents. The methods include procedures whose execution affect the values in the object and possibly also affect that thing being represented.

Figures D.1 and D.2 illustrate key object-oriented concepts.

### Object Structure

The data and procedures contained in an object are generally referred to as variables and methods, respectively. Everything that an object "knows" can be expressed in its variables, and everything it can do is expressed in its methods.

The **variables** in an object, also called **attributes**, are typically simple scalars or tables. Each variable has a type, possibly a set of allowable values, and may either be constant or variable (by convention, the term *variable* is used even for constants).

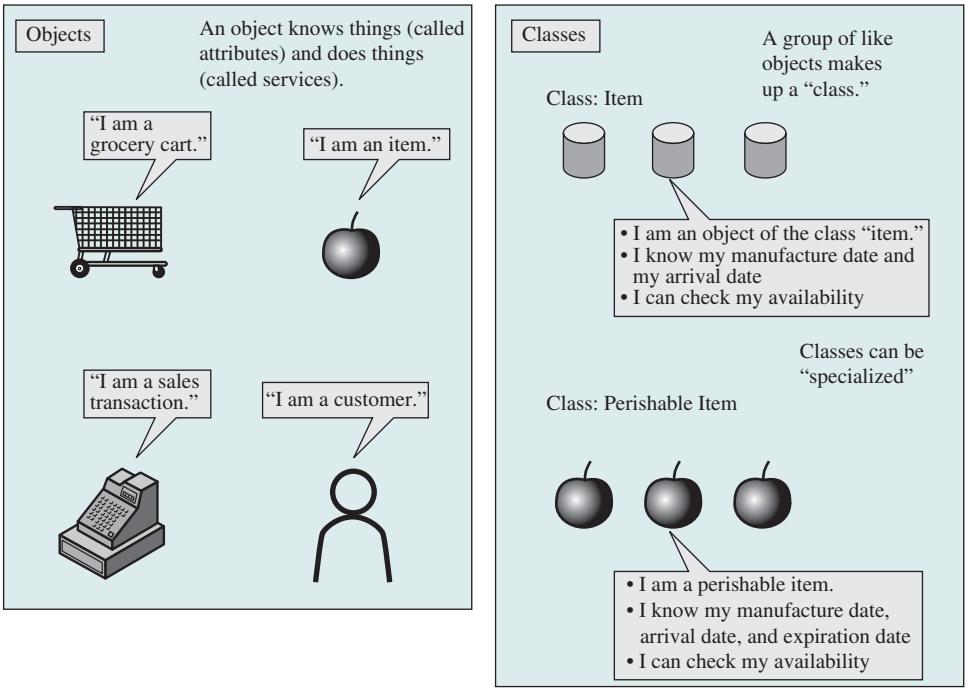



**Figure D.1 Objects**

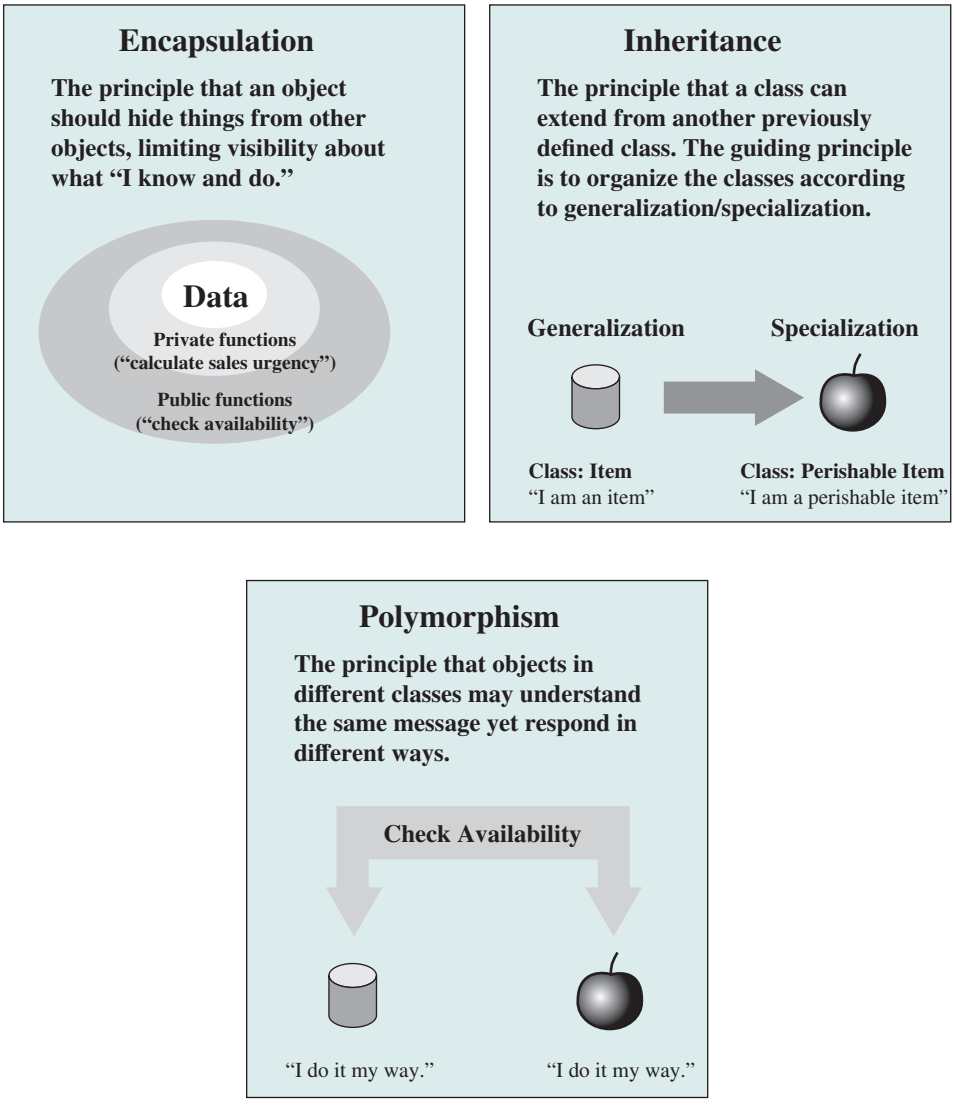



**Figure D.2** **Object Concepts**

Access restrictions may also be imposed on variables for certain users, classes of users, or situations.

The **methods** in an object are procedures that can be triggered from outside to perform certain functions. The method may change the state of the object, update some of its variables, or act on outside resources to which the object has access.

Objects interact by means of **messages**. A message includes the name of the sending object, the name of the receiving object, the name of a method in the receiving object, and any parameters needed to qualify the execution of the method. A message can only be used to invoke a method within an object. The only

way to access the data inside an object is by means of the object's methods. Thus, a method may cause an action to be taken, or for the object's variables to be accessed, or both. For local objects, passing a message to an object is the same as calling an object's method. When objects are distributed, passing a message is exactly what it sounds like.

The interface of an object is a set of public methods that the object supports. An interface says nothing about implementation; objects in different classes may have different implementations of the same interfaces.

The property of an object that its only interface with the outside world is by means of messages is referred to as **encapsulation**. The methods and variables of an object are encapsulated and available only via message-based communication. Encapsulation offers two advantages:

1. It protects an object's variables from corruption by other objects. This protection may include protection from unauthorized access and protection from the types of problems that arise from concurrent access, such as deadlock and inconsistent values.
2. It hides the internal structure of the object so that interaction with the object is relatively simple and standardized. Furthermore, if the internal structure or procedures of an object are modified without changing its external functionality, other objects are unaffected.

## Object Classes

In practice, there will typically be a number of objects representing the same types of things. For example, if a process is represented by an object, then there will be one object for each process present in a system. Clearly, every such object needs its own set of variables. However, if the methods in the object are reentrant procedures, then all similar objects could share the same methods. Furthermore, it would be inefficient to redefine both methods and variables for every new but similar object.

The solution to these difficulties is to make a distinction between an object class and an object instance. An **object class** is a template that defines the methods and variables to be included in a particular type of object. An **object instance** is an actual object that includes the characteristics of the class that defines it. The object contains values for the variables defined in the object class. **Instantiation** is the process of creating a new object instance for an object class.

***INHERITANCE*** The concept of an object class is powerful because it allows for the creation of many object instances with a minimum of effort. This concept is made even more powerful by the use of the mechanism of inheritance [TAIV96].

Inheritance enables a new object class to be defined in terms of an existing class. The new (lower level) class, called the **subclass**, or the **child class**, automatically includes the methods and variable definitions in the original (higher-level) class, called the **superclass**, or **parent class**. A subclass may differ from its superclass in a number of ways:

1. The subclass may include additional methods and variables not found in its superclass.

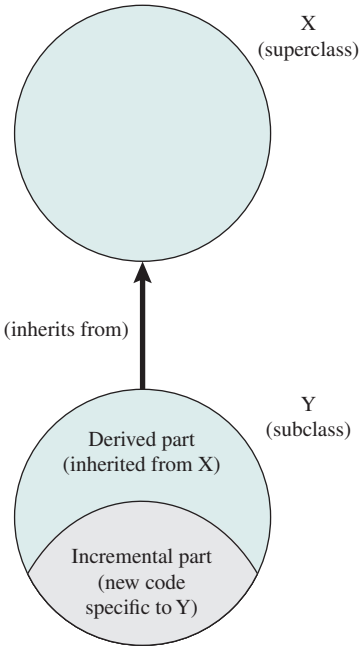



**Figure D.3 Inheritance**

2. The subclass may override the definition of any method or variable in its superclass by using the same name with a new definition. This provides a simple and efficient way of handling special cases.
3. The subclass may restrict a method or variable inherited from its superclass in some way.

Figure D.3, based on one in [KORS90], illustrates the concept.

The inheritance mechanism is recursive, allowing a subclass to become the superclass of its own subclasses. In this way, an **inheritance hierarchy** may be constructed. Conceptually, we can think of the inheritance hierarchy as defining a search technique for methods and variables. When an object receives a message to carry out a method that is not defined in its class, it automatically searches up the hierarchy until it finds the method. Similarly, if the execution of a method results in the reference to a variable not defined in that class, the object searches up the hierarchy for the variable name.

***POLYMORPHISM*** Polymorphism is an intriguing and powerful characteristic that makes it possible to hide different implementations behind a common interface. Two objects that are polymorphic to each other utilize the same names for methods and present the same interface to other objects. For example, there may be a number of print objects for different output devices, such as `printDotmatrix`, `printLaser`, `printScreen`, and so forth, or for different types of documents, such as `printText`, `printDrawing`, and `printCompound`. If each such object includes a method called `print`, then any document could be printed by sending

the message print to the appropriate object, without concern for how that method is actually carried out. Typically, polymorphism is used to allow you have the same method in multiple subclasses of the same superclass, each with a different detailed implementation.

It is instructive to compare polymorphism to the usual modular programming techniques. An objective of top-down modular design is to design lower-level modules of general utility with a fixed interface to higher-level modules. This allows the one lower-level module to be invoked by many different higher-level modules. If the internals of the lower-level module are changed without changing its interface, then none of the upper-level modules that use it are affected. By contrast, with polymorphism, we are concerned with the ability of one higher-level object to invoke many different lower-level objects using the same message format to accomplish similar functions. With polymorphism, new lower-level objects can be added with minimal changes to existing objects.

***INTERFACES*** Inheritance enables a subclass object to use functionality of a superclass. There may be cases when you wish to define a subclass that has functionality of more than one superclass. This could be accomplished by allowing a subclass to inherit from more than one superclass. C++ is one language that allows such multiple inheritance. However, for simplicity, most modern object-oriented languages including Java, C\#, and Visual Basic .NET limit a class to inheriting from only one superclass. Instead, a feature known as *interfaces* is used to enable a class to borrow some functionality from one class and other functionality from a completely different class.

Unfortunately, the term *interface* is used in much of the literature on objects with both a general-purpose and a specific functional meaning. An interface, as we are discussing it here, specifies an application-programming interface (API) for certain functionality. It does not define any implementation for that API. The syntax for an interface definition typically looks similar to a class definition, except that there is no code defined for the methods, just the method names, the arguments passed, and the type of the value returned. An interface may be implemented by a class. This works in much the same way that inheritance works. If a class implements an interface, it must have the properties and methods of the interface defined in the class. The methods that are implemented can be coded in any fashion, so long as the name, arguments, and return type of each method from the interface are identical to the definition in the interface.

## Containment

Object instances that contain other objects are called **composite objects**. Containment may be achieved by including the pointer to one object as a value in another object. The advantage of composite objects is that they permit the representation of complex structures. For example, an object contained in a composite object may itself be a composite object.

Typically, the structures built up from composite objects are limited to a tree topology; that is, no circular references are allowed and each "child" object instance may have only one "parent" object instance.

It is important to be clear about the distinction between an inheritance hierarchy of object classes, and a containment hierarchy of object instances. The two are not related. The use of inheritance simply allows many different object types to be defined with a minimum of efforts. The use of containment allows the construction of complex data structures.

## D.3 BENEFITS OF OBJECT-ORIENTED DESIGN

[CAST92] lists the following benefits of object-oriented design:

- **Better organization of inherent complexity:** Through the use of inheritance, related concepts, resources, and other objects can be efficiently defined. Through the use of containment, arbitrary data structures, which reflect the underlying task at hand, can be constructed. Object-oriented programming languages and data structures enable designers to describe operating system resources and functions in a way that reflects the designer's understanding of those resources and functions.
- **Reduced development effort through reuse:** Reusing object classes that have been written, tested, and maintained by others reduces development, testing, and maintenance time.
- **More extensible and maintainable systems:** Maintenance, including product enhancements and repairs, traditionally consumes about 65% of the cost of any product life cycle. Object-oriented design drives that percentage down. The use of object-based software helps limit the number of potential interactions of different parts of the software, ensuring changes to the implementation of a class can be made with little impact on the rest of the system.

These benefits are driving operating system design in the direction of object-oriented systems. Objects enable programmers to customize an operating system to meet new requirements without disrupting system integrity. Objects also pave the road to distributed computing. Because objects communicate by means of messages, it matters not whether two communicating objects are on the same system or on two different systems in a network. Data, functions, and threads can be dynamically assigned to workstations and servers as needed. Accordingly, the object-oriented approach to the design of operating systems is becoming increasingly evident in PC and workstation operating systems.

## D.4 CORBA

As we have seen in this book, object-oriented concepts have been used to design and implement operating system kernels, bringing benefits of flexibility, manageability, and portability. The benefits of using object-oriented techniques extend with equal or greater benefit to the realm of distributed software, including distributed operating systems. The application of object-oriented techniques to the design and implementation of distributed software is referred to as distributed object computing (DOC).

The motivation for DOC is the increasing difficulty in writing distributed software: while computing and network hardware get smaller, faster, and cheaper, distributed software gets larger, slower, and more expensive to develop and maintain. [SCHM97] points out that the challenge of distributed software stems from two types of complexity:

- **Inherent:** Inherent complexities arise from fundamental problems of distribution. Chief among these are detecting and recovering from network and host failures, minimizing the impact of communication latency, and determining an optimal partitioning of service components and workload onto computers throughout a network. In addition, concurrent programming, with issues of resource locking and deadlocks, is still difficult, and distributed systems are inherently concurrent.
- **Accidental:** Accidental complexities arise from limitations with tools and techniques used to build distributed software. A common source of accidental complexity is the widespread use of functional design, which results in nonextensible and nonreusable systems.

DOC is a promising approach to managing both types of complexity. The centerpiece of the DOC approach are object request brokers (ORBs), which act as intermediaries for communication between local and remote objects. ORBs eliminate some of the tedious, error-prone, and nonportable aspects of designing and implementing distributed applications. Supplementing the ORB must be a number of conventions and formats for message exchange and interface definition between applications and the object-oriented infrastructure.

There are three main competing technologies in the DOC market: the object management group (OMG) architecture, called Common Object Request Broker Architecture (CORBA); the Java remote method invocation (RMI) system; and Microsoft's distributed component object model (DCOM). CORBA is the most advanced and well-established of the three. A number of industry leaders, including IBM, Sun, Netscape, and Oracle, support CORBA, and Microsoft has announced that it will link its Windows-only DCOM with CORBA. The remainder of this appendix provides a brief overview of CORBA.

Table D.2 defines some key terms used in CORBA. The main features of CORBA are as follows (see Figure D.4):

- **Clients:** Clients generate requests and access object services through a variety of mechanisms provided by the underlying ORB.
- **Object implementations:** These implementations provide the services requested by various clients in the distributed system. One benefit of the CORBA architecture is that both clients and object implementations can be written in any number of programming languages and can still provide the full range of required services.
- **ORB core:** The ORB core is responsible for communication between objects. The ORB finds an object on the network, delivers requests to the object, activates the object (if not already active), and returns any message back to the sender. The ORB core provides **access transparency** because programmers use

**Table D.2** Key Concepts in a Distributed CORBA System

| CORBA Concept | Definition |
|---|---|
| Client application | Invokes requests for a server to perform operations on objects. A client application uses one or more interface definitions that describe the objects and operations the client can request. A client application uses object references, not objects, to make requests. |
| Exception | Contains information that indicates whether a request was successfully performed. |
| Implementation | Defines and contains one or more methods that do the work associated with an object operation. A server can have one or more implementations. |
| Interface | Describes how instances of an object will behave, such as what operations are valid on those objects. |
| Interface definition | Describes the operations that are available on a certain type of object. |
| Invocation | The process of sending a request. |
| Method | The server code that does the work associated with an operation. Methods are contained within implementations. |
| Object | Represents a person, place, thing, or piece of software. An object can have operations performed on it, such as the promote operation on an employee object. |
| Object instance | An occurrence of one particular kind of object. |
| Object reference | An identifier of an object instance. |
| OMG Interface Definition Language (IDL) | A definition language for defining interfaces in CORBA. |
| Operation | The action that a client can request a server to perform on an object instance. |
| Request | A message sent between a client and a server application. |
| Server application | Contains one or more implementations of objects and their operations. |

exactly the same method with the same parameters when invoking a local method or a remote method. The ORB core also provides **location transparency**: Programmers do not need to specify the location of an object.

- **Interface:** An object's interface specifies the operations and types supported by the object, and thus defines the requests that can be made on the object. CORBA interfaces are similar to classes in C++ and interfaces in Java. Unlike C++ classes, a CORBA interface specifies methods and their parameters and return values, but is silent about their implementation. Two objects of the same C++ class have the same implementation of their methods.
- **OMG interface definition language (IDL):** IDL is the language used to define objects. An example IDL interface definition is:

```
//OMG IDL
interface Factory
    { Object create ( ) ;
} ;
```

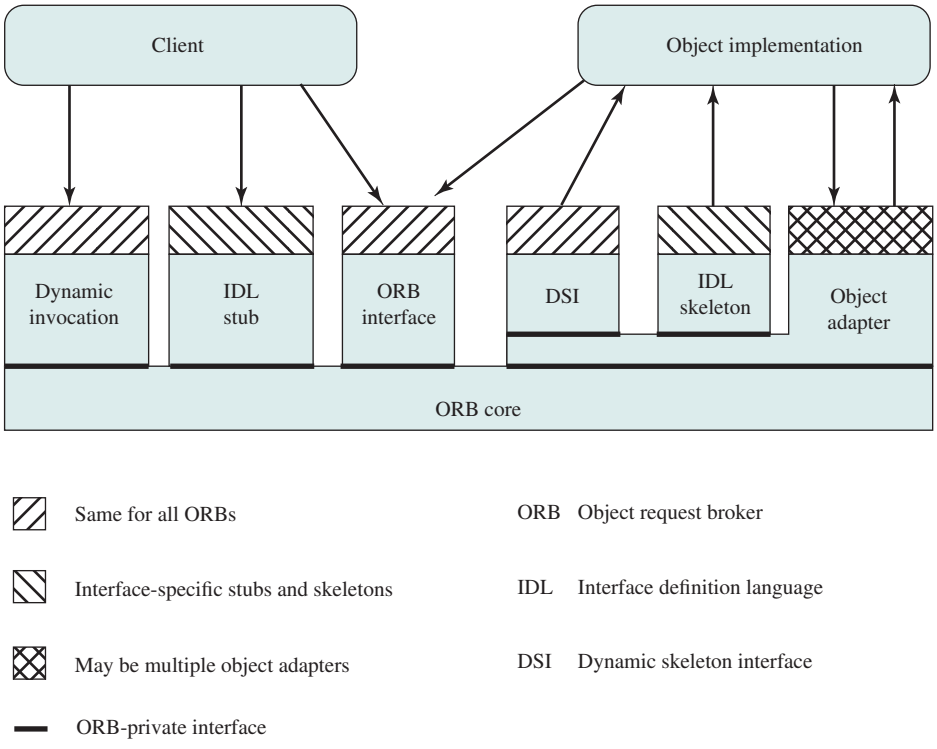



**Figure D.4 Common Object Request Broker Architecture**

This definition specifies an interface named `Factory` that supports one operation, `create`. The create operation takes no parameters and returns an object reference of type `Object`. Given an object reference for an object of type `Factory`, a client could invoke it to create a new CORBA object. IDL is a programming-independent language and, for this reason, a client does not invoke directly any object operation. It needs a mapping to the client programming language to do that. It is also possible, that the server and the client are programmed in different programming languages. The use of a specification language is a way to deal with heterogeneous processing across multiple languages and platform environments. Thus, IDL enables **platform independence**.

- **Language binding creation:** IDL compilers map one OMG IDL file to different programming languages, which may or may not be object oriented, such as Java, Smalltalk, Ada, C, C++, and COBOL. That mapping includes the definition of the language-specific data types and procedure interfaces to access service objects, the IDL client stub interface, the IDL skeleton, the object adapters, the dynamic skeleton interface, and the direct ORB interface. Usually, clients have a compile-time knowledge of the object interface and use client stubs to do a static invocation; in certain cases, clients do not have that knowledge and they must do a dynamic invocation.

- **IDL stub:** Makes calls to the ORB core on behalf of a client application. IDL stubs provide a set of mechanisms that abstract the ORB core functions into direct RPC (remote procedure call) mechanisms that can be employed by the end-client applications. These stubs make the combination of the ORB and remote object implementation appear as if they were tied to the same in-line process. In most cases, IDL compilers generate language-specific interface libraries that complete the interface between the client and object implementations.
- **IDL skeleton:** Provides the code that invokes specific server methods. Static IDL skeletons are the server-side complements to the client-side IDL stubs. They include the bindings between the ORB core and the object implementations that complete the connection between the client and object implementations.
- **Dynamic invocation:** Using the dynamic invocation interface (DII), a client application can invoke requests on any object without having compile-time knowledge of the object's interfaces. The interface details are filled in by consulting with an interface repository and/or other run-time sources. The DII allows a client to issue one-way commands (for which there is no response).
- **Dynamic skeleton interface (DSI):** Similar to the relationship between IDL stubs and static IDL skeletons, the DSI provides dynamic dispatch to objects. Equivalent to dynamic invocation on the server side.
- **Object adapter:** An object adapter is CORBA system component provided by the CORBA vendor to handle general ORB-related tasks, such as activating objects and activating implementations. The adapter takes these general tasks and ties them to particular implementations and methods in the server.

## D.5 RECOMMENDED READING AND WEBSITE

[KORS90] is a good overview of object-oriented concepts. [STRO88] is a clear description of object-oriented programming. An interesting perspective on object-oriented concepts is provided in [SYND93]. [VINO97] is an overview of CORBA.

**KORS90** Korson, T., and McGregor, J. "Understanding Object-Oriented: A Unifying Paradigm." *Communications of the ACM*, September 1990.

**STRO88** Stroustrup, B. "What is Object-Oriented Programming?" *IEEE Software*, May 1988.

**SNYD93** Snyder, A. "The Essence of Objects: Concepts and Terms." *IEEE Software*, January 1993.

**VINO97** Vinoski, S. "CORBA: Integrating Diverse Applications Within Distributed Heterogeneous Environments." *IEEE Communications Magazine*, February 1997.



Recommended Website:

**Object Management Group:** Industry consortium that promotes CORBA and related object technologies

# APPENDIX E

## AMDAHL'S LAW

E.1 Implications of Amdahl's Law

E.2 References

## E.1 IMPLICATIONS OF AMDAHL'S LAW

When considering system performance, computer system designers look for ways to improve performance by improvement in technology or change in design. Examples include the use of parallel processors, the use of a memory cache hierarchy, and speedup in memory access time and I/O transfer rate due to technology improvements. In all of these cases, it is important to note that a speedup in one aspect of the technology or design does not result in a corresponding improvement in performance. This limitation is succinctly expressed by Amdahl's law.

Amdahl's law was first proposed by Gene Amdahl in 1967 [AMDA67] and deals with the potential speedup of a program using multiple processors compared to a single processor. Consider a program running on a single processor such that a fraction $(1 - f)$ of the execution time involves code that is inherently serial, and a fraction $f$ that involves code that is infinitely parallelizable with no scheduling overhead. Let $T$ be the total execution time of the program using a single processor. Then the speedup using a parallel processor with $N$ processors that fully exploits the parallel portion of the program is as follows:

$$\text{Speedup} = \frac{\text{time to execute program on a single processor}}{\text{time to execute program on } N \text{ parallel processors}}$$

$$= \frac{T(1 - f) + Tf}{T(1 - f) + \frac{Tf}{N}} = \frac{1}{(1 - f) + \frac{f}{N}}$$

This equation is illustrated in Figure E.1. Two important conclusions can be drawn:

1. When $f$ is small, the use of parallel processors has little effect.
2. As $N$ approaches infinity, speedup is bound by $1/(1 - f)$, so there are diminishing returns for using more processors.

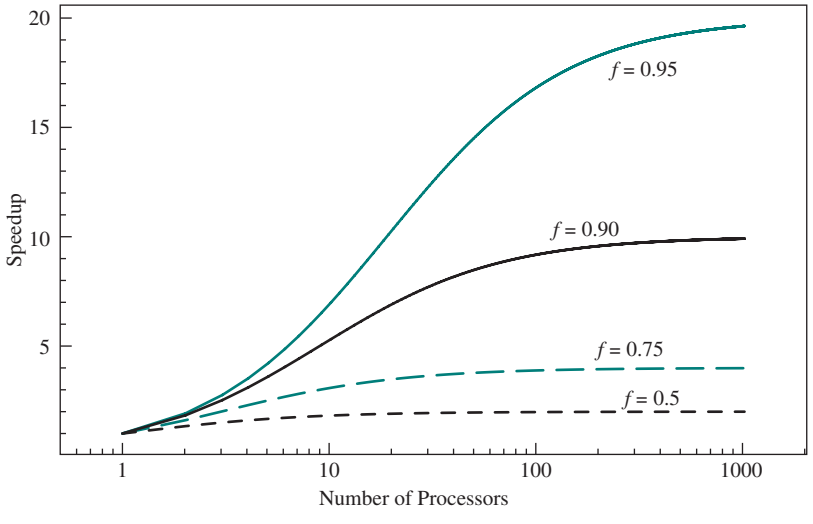



**Figure E.1 Amdahl's Law for Multiprocessors**

These conclusions are too pessimistic, an assertion first put forward in [GUST88]. For example, a server can maintain multiple threads or multiple tasks to handle multiple clients and execute the threads or tasks in parallel up to the limit of the number of processors. Many database applications involve computations on massive amounts of data that can be split up into multiple parallel tasks. Nevertheless, Amdahl's law illustrates the problems facing industry in the development of multicore machines with an ever-growing number of cores: The software that runs on such machines must be adapted to a highly parallel execution environment to exploit the power of parallel processing.

Amdahl's law can be generalized to evaluate any design or technical improvement in a computer system. Consider any enhancement to a feature of a system that results in a speedup. The speedup can be expressed as follows:

$$\text{Speedup} = \frac{\text{Performance after enhancement}}{\text{Performance before enhancement}} = \frac{\text{Execution time before enhancement}}{\text{Execution time after enhancement}}$$

Suppose a feature of the system is used during execution a fraction of the time $f$, before enhancement, and the speedup of that feature after enhancement is $SU_f$. Then the overall speedup of the system is

$$Speedup = \frac{1}{(1 - f) + \frac{f}{SU_f}}$$

For example, suppose a task makes extensive use of floating-point operations, with 40% of the time is consumed by floating-point operations. With a new hardware design, the floating-point module is speeded up by a factor of $K$. Then, the overall speedup is:

$$Speedup = \frac{1}{0.6 + \frac{0.4}{K}}$$

Thus, independent of $K$, the maximum speedup is 1.67.

## E.2 REFERENCES

**AMDA67** Amdahl, G. "Validity of the Single-Processor Approach to Achieving Large-Scale Computing Capability." *Proceedings, of the AFIPS Conference*, 1967.

**GUST88** Gustafson, J. "Reevaluating Amdahl's Law." *Communications of the ACM*, May 1988.

# APPENDIX F

## HASH TABLES

Consider the following problem. A set of $N$ items is to be stored in a table. Each item consists of a label plus some additional information, which we can refer to as the value of the item. We would like to be able to perform a number of ordinary operations on the table, such as insertion, deletion, and searching for a given item by label.

If the labels of the items are numeric, in the range 0 to $M - 1$, then a simple solution would be to use a table of length $M$. An item with label $i$ would be inserted into the table at location $i$. As long as items are of fixed length, table lookup is trivial and involves indexing into the table based on the numeric label of the item. Furthermore, it is not necessary to store the label of an item in the table, because this is implied by the position of the item. Such a table is known as a **direct access table**.

If the labels are nonnumeric, then it is still possible to use a direct access approach. Let us refer to the items as A[1], . . . A[$N$]. Each item A[$i$] consists of a label, or key, $k_i$, and a value $v_i$. Let us define a mapping function I($k$) such that I($k$) takes a value between 1 and $M$ for all keys, and I($k_i$) $\neq$ I($k_j$) for any $i$ and $j$. In this case, a direct access table can also be used, with the length of the table equal to $M$.

The one difficulty with these schemes occurs if $M$ is much greater than $N$. In this case, the proportion of unused entries in the table is large, and this is an inefficient use of memory. An alternative would be to use a table of length $N$ and store the $N$ items (label plus value) in the $N$ table entries. In this scheme, the amount of memory is minimized, but there is now a processing burden to do table lookup. There are several possibilities:

- **Sequential search**: This brute-force approach is time consuming for large tables.
- **Associative search**: With the proper hardware, all of the elements in a table can be searched simultaneously. This approach is not general purpose and cannot be applied to any and all tables of interest.
- **Binary search**: If the labels or the numeric mapping of the labels are arranged in ascending order in the table, then a binary search is much quicker than sequential (see Table F.1) and requires no special hardware.

The binary search looks promising for table lookup. The major drawback with this method is that adding new items is not usually a simple process and will require reordering of the entries. Therefore, binary search is usually used only for reasonably static tables that are seldom changed.

**Table F.1** Average Search Length for One of $N$ items in a Table of Length $M$

| Technique | Search Length |
|---|---|
| Direct | 1 |
| Sequential | $\frac{M + 1}{2}$ |
| Binary | $\log_2 M$ |
| Linear hashing | $\frac{2 - N/M}{2 - 2N/M}$ |
| Hash (overflow with chaining) | $1 + \frac{N - 1}{2M}$ |

We would like to avoid the memory penalties of a simple direct access approach and the processing penalties of the alternatives listed previously. The most frequently used method to achieve this compromise is **hashing**. Hashing, which was developed in the 1950s, is simple to implement and has two advantages. First, it can find most items with a single seek, as in direct accessing. Second, insertions and deletions can be handled without added complexity.

The hashing function can be defined as follows. Assume up to $N$ items are to be stored in a **hash table** of length $M$, with $M \geq N$, but not much larger than $N$. To insert an item in the table:

**I1.** Convert the label of the item to a near-random number $n$ between 0 and $M - 1$. For example, if the label is numeric, a popular mapping function is to divide the label by $M$ and take the remainder as the value of $n$.

**I2.** Use $n$ as the index into the hash table.

- **a.** If the corresponding entry in the table is empty, store the item (label and value) in that entry.
- **b.** If the entry is already occupied, then store the item in an overflow area, as discussed subsequently.

To perform table lookup of an item whose label is known:

**L1.** Convert the label of the item to a near-random number $n$ between 0 and $M - 1$, using the same mapping function as for insertion.

**L2.** Use $n$ as the index into the hash table.

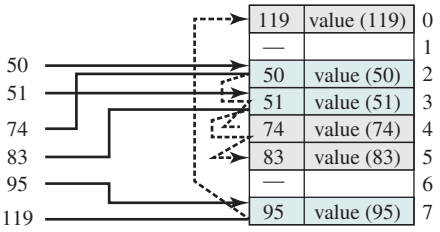
- **a.** If the corresponding entry in the table is empty, then the item has not previously been stored in the table.
- **b.** If the entry is already occupied and the labels match, then the value can be retrieved.
- **c.** If the entry is already occupied and the labels do not match, then continue the search in the overflow area.

Hashing schemes differ in the way in which the overflow is handled. One common technique is referred to as the **linear hashing** technique and is commonly used in compilers. In this approach, rule I2.b becomes

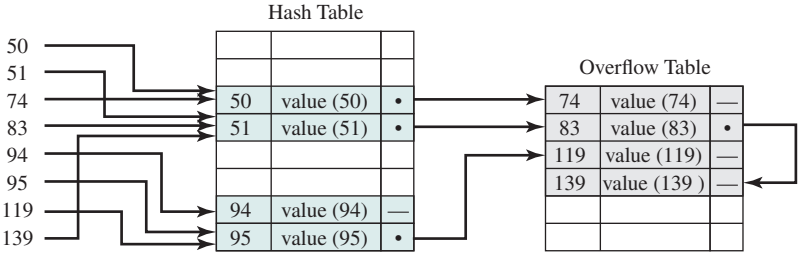
**I2.b.** If the entry is already occupied, set $n = n + 1 \pmod{M}$ and go back to step I2.a.

Rule L2.c is modified accordingly.

Figure F.1a is an example. In this case, the labels of the items to be stored are numeric, and the hash table has eight positions ($M = 8$). The mapping function is to take the remainder upon division by 8. The figure assumes the items were inserted in ascending numerical order, although this is not necessary. Thus, items 50 and 51 map into positions 2 and 3, respectively, and as these are empty, they are inserted there. Item 74 also maps into position 2, but as it is not empty, position 3 is tried. This is also occupied, so the position 4 is ultimately used.




(a) Linear rehashing




(b) Overflow with chaining

**Figure F.1 Hashing**

It is not easy to determine the average length of the search for an item in an open hash table because of the clustering effect. An approximate formula was obtained by Schay and Spruth:[1]

$$\text{Average search length} = \frac{2 - r}{2 - 2r}$$

where $r = N/M$. Note the result is independent of table size and depends only on how full the table is. The surprising result is with the table 80% full, the average length of the search is still around 3.

Even so, a search length of 3 may be considered long, and the linear hashing table has the additional problem that it is not easy to delete items. A more attractive approach, which provides shorter search lengths (see Table F.1) and allows deletions as well as additions, is **overflow with chaining**. This technique is illustrated in Figure F.1b. In this case, there is a separate table into which overflow entries are inserted. This table includes pointers passing down the chain of entries associated with any position in the hash table. In this case, the average search length, assuming randomly distributed data, is

$$\text{Average search length} = 1 + \frac{N - 1}{2M}$$

For large values of $N$ and $M$, this value approaches 1.5 for $N = M$. Thus, this technique provides for compact storage with rapid lookup.

[1]Schay, G., and Spruth, W. "Analysis of a File Addressing Method." *Communications of the ACM,* August 1962.

# APPENDIX G

## RESPONSE TIME

G.1 Response Time Considerations

G.2 References

## G.1 RESPONSE TIME CONSIDERATIONS

Response time is the time taken by a system to react to a given input. In an interactive transaction, it may be defined as the time between the last keystroke by the user and the beginning of the display of a result by the computer. For different types of applications, a slightly different definition is needed. In general, it is the time taken for the system to respond to a request to perform a particular task.

Ideally, one would like the response time for any application to be short. However, it is almost invariably the case that shorter response time imposes greater cost. This cost comes from two sources:

- **Computer processing power:** The faster the processor is, the shorter the response time will be. Of course, increased processing power means increased cost.
- **Competing requirements:** Providing rapid response time to some processes may penalize other processes.

Thus the value of a given level of response time must be assessed versus the cost of achieving that response time.

Table G.1, from [MART88] lists six general ranges of response times. Design difficulties are faced when a response time of less than 1 second is required. A requirement for a sub-second response time is generated by a system that controls or in some other way interacts with an ongoing external activity, such as an assembly line. Here the requirement is straightforward. When we consider human-computer interaction, such as in a data entry application, then we are in the realm of conversational response time. In this case, there is still a requirement for a short response time, but the acceptable length of time may be difficult to assess.

That rapid response time is the key to productivity in interactive applications has been confirmed in a number of studies [SHNE84; THAD81; GUYN88]. These studies show that when a computer and a user interact at a pace that ensures neither has to wait on the other, productivity increases significantly, the cost of the work done on the computer therefore drops, and quality tends to improve. It used to be widely accepted that a relatively slow response, up to 2 seconds, was acceptable for most interactive applications because the person was thinking about the next task. However, it now appears that productivity increases as rapid response times are achieved.

The results reported on response time are based on an analysis of online transactions. A transaction consists of a user command from a terminal and the system's reply. It is the fundamental unit of work for online system users. It can be divided into two time sequences:

- **User response time:** The time span between the moment the user receives a complete reply to one command and enters the next command. People often refer to this as think time.
- **System response time:** The time span between the moment the user enters a command and the moment a complete response is displayed on the terminal.

**Table G.1** Response Time Ranges

| |
|---|
| **Greater than 15 seconds** |
| This rules out a conversational interaction. For certain types of applications, certain types of users may be content to sit at a terminal for more than 15 seconds waiting for the answer to a single simple inquiry. However, for a busy person, captivity for more than 15 seconds seems intolerable. If such delays will occur, the system should be designed so the user can turn to other activities and request the response at some later time. |
| **Greater than 4 seconds** |
| These are generally too long for a conversation requiring the operator to retain information in short-term memory (the operator's memory, not the computer's!). Such delays would be very inhibiting in problem-solving activity and frustrating in data entry activity. However, after a major closure, such as the end of a transaction, delays from 4 to 15 seconds can be tolerated. |
| **2 to 4 seconds** |
| A delay longer than 2 seconds can be inhibiting to terminal operations demanding a high level of concentration. A wait of 2–4 seconds at a terminal can seem surprisingly long when the user is absorbed and emotionally committed to complete what he or she is doing. Again, a delay in this range may be acceptable after a minor closure has occurred. |
| **Less than 2 seconds** |
| When the terminal user has to remember information throughout several responses, the response time must be short. The more detailed the information remembered, the greater the need for responses of less than 2 seconds. For elaborate terminal activities, 2 seconds represents an important response-time limit. |
| **Sub-second response time** |
| Certain types of thought-intensive work, especially with graphics applications, require very short response times to maintain the user's interest and attention for long periods of time. |
| **Decisecond response time** |
| A response to pressing a key and seeing the character displayed on the screen or clicking a screen object with a mouse needs to be almost instantaneous—less than 0.1 second after the action. Interaction with a mouse requires extremely fast interaction if the designer is to avoid the use of alien syntax (one with commands, mnemonics, punctuation, etc.). |

As an example of the effect of reduced system response time, Figure G.1 shows the results of a study carried out on engineers using a computer-aided design graphics program for the design of integrated circuit chips and boards [SMIT83]. Each transaction consists of a command by the engineer that in some way alters the graphic image being displayed on the screen. The results show that the rate of transactions increases as system response time falls and rises dramatically once system response time falls below 1 second. What is happening is that as the system response time falls, so does the user response time. This has to do with the effects of short-term memory and human attention span.

Another area where response time has become critical is the use of the World Wide Web, either over the Internet or over a corporate intranet. The time taken for a typical Web page to come up on the user's screen varies greatly. Response times can be gauged based on the level of user involvement in the session; in particular, systems with vary fast response times tend to command more user attention. In a study by Sevcik [SEVC96, SEVC02], illustrated in Figure G.2, Web systems with a 3-second or

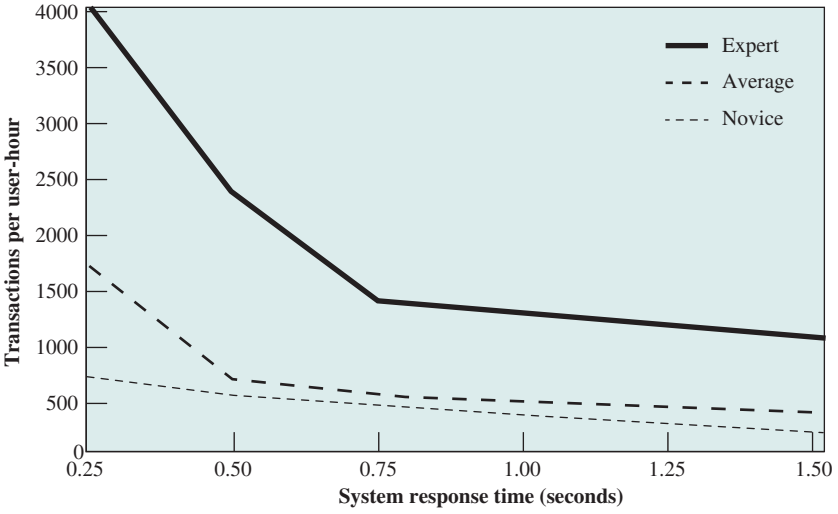



**Figure G.1 Response Time Results for High-Function Graphics**

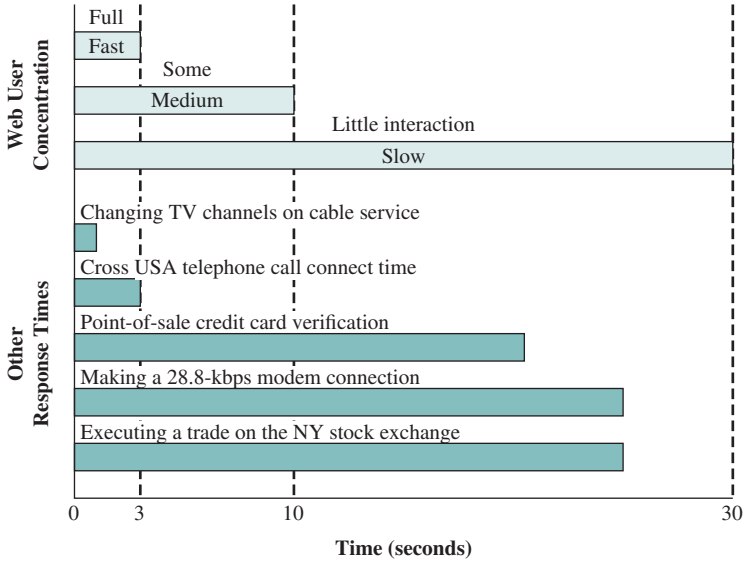



**Figure G.2 Response Time Requirements**

better response time maintain a high level of user attention. With a response time of between 3 and 10 seconds, some user concentration is lost, and response times above 10 seconds discourage the user, who may simply abort the session. Other studies of Web response time generally confirm these findings [BHAT01].

## G.2 REFERENCES

**BHAT01** Bhatti, N.; Bouch, A.; and Kuchinsky, A. "Integrated User-Perceived Quality into Web Server Design." *Proceedings, 9th International World Wide Web Conference*, May 2000.

**GUYN88** Guynes, J. "Impact of System Response Time on State Anxiety." *Communications of the ACM,* March 1988.

**MART88** Martin, J. *Principles of Data Communication.* Englewood Cliffs, NJ: Prentice Hall, 1988.

**SELV99** Selvidge, P. "How Long Is Too Long to Wait for a Webpage to Load." *Usability News*, Wichita State University, July 1999.

**SEVC96** Sevcik, P. "Designing a High-Performance Web Site." *Business Communications Review*, March 1996.

**SEVC02** Sevcik, P. "Understanding How Users View Application Performance." *Business Communications Review*, July 2002.

**SHNE84** Shneiderman, B. "Response Time and Display Rate in Human Performance with Computers." *ACM Computing Surveys*, September 1984.

**SMIT83** Smith, D. "Faster Is Better: A Business Case for Subsecond Response Time." *Computerworld*, April 18, 1983.

**THAD81** Thadhani, A. "Interactive User Productivity." *IBM Systems Journal*, No. 1, 1981.

# APPENDIX H

## QUEUEING SYSTEM CONCEPTS


H.1 Why Queueing Analysis?

H.2 The Single-Server Queue

H.3 The Multiserver Queue

H.4 Poisson Arrival Rate

In a number of chapters in this book, results from queueing theory are used. Chapter 21 provides a detailed discussion of queueing analysis. For purposes of understanding the description of the results in the book, however, the brief overview in this appendix should suffice. In this appendix, we present a brief definition of queueing systems and define key terms.

## H.1 WHY QUEUEING ANALYSIS?

It is often necessary to make projections of performance on the basis of existing load information or on the basis of estimated load for a new environment. A number of approaches are possible:

1. Do an after-the-fact analysis based on actual values.
2. Make a simple projection by scaling up from existing experience to the expected future environment.
3. Develop an analytic model based on queueing theory.
4. Program and run a simulation model.

Option 1 is no option at all: we will wait and see what happens. This leads to unhappy users and to unwise purchases. Option 2 sounds more promising. The analyst may take the position that it is impossible to project future demand with any degree of certainty. Therefore, it is pointless to attempt some exact modeling procedure. Rather, a rough-and-ready projection will provide ballpark estimates. The problem with this approach is that the behavior of most systems under a changing load is not what one would intuitively expect. If there is an environment in which there is a shared facility (e.g., a network, a transmission line, and a time-sharing system), then the performance of that system typically responds in an exponential way to increases in demand.

Figure H.1 is a representative example. The upper line shows what typically happens to user response time on a shared facility as the load on that facility increases. The load is expressed as a fraction of capacity. Thus, if we are dealing with a router that is capable of processing and forwarding 1000 packets per second, then a load of 0.5 represents an arrival rate of 500 packets per second, and the response time is the amount of time it takes to retransmit any incoming packet. The lower line is a simple projection[1] based on knowledge of the behavior of the system up to a load of 0.5. Note while things appear rosy when the simple projection is made, performance on the system will in fact collapse beyond a load of about 0.8 to 0.9.

Thus, a more exact prediction tool is needed. Option 3 is to make use of an analytic model, which is one that can be expressed as a set of equations that can be solved to yield the desired parameters (response time, throughput, etc.). For computer, operating system, and networking problems, and indeed for many practical real-world problems, analytic models based on queueing theory provide a reasonably good fit to reality. The disadvantage of queueing theory is that a number of simplifying assumptions must be made to derive equations for the parameters of interest.

[1]The lower line is based on fitting a third-order polynomial to the data available up to a load of 0.5.

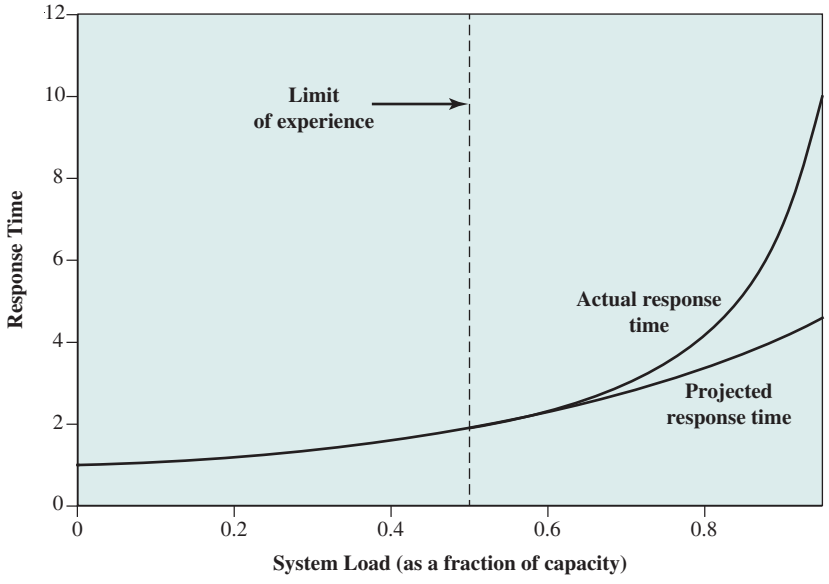



**Figure H.1** **Projected versus Actual Response Time**

The final approach is a simulation model. Here, given a sufficiently powerful and flexible simulation programming language, the analyst can model reality in great detail and avoid making many of the assumptions required of queueing theory. However, in most cases, a simulation model is not needed or at least is not advisable as a first step in the analysis. For one thing, both existing measurements and projections of future load carry with them a certain margin of error. Thus, no matter how good the simulation model, the value of the results is limited by the quality of the input. For another, despite the many assumptions required of queueing theory, the results that are produced often come quite close to those that would be produced by a more careful simulation analysis. Furthermore, a queueing analysis can literally be accomplished in a matter of minutes for a well-defined problem, whereas simulation exercises can take days, weeks, or longer to program and run.

Accordingly, it behooves the analyst to master the basics of queueing theory.

## H.2 THE SINGLE-SERVER QUEUE

The simplest queueing system is depicted in Figure H.2. The central element of the system is a server, which provides some service to items. Items from some population of items arrive at the system to be served. If the server is idle, an item is served immediately. Otherwise, an arriving item joins a waiting line.[2] When the server has completed serving an item, the item departs. If there are items waiting in the queue, one

[2]The waiting line is referred to as a queue in some treatments in the literature; it is also common to refer to the entire system as a queue. Unless otherwise noted, we use the term *queue* to mean waiting line.

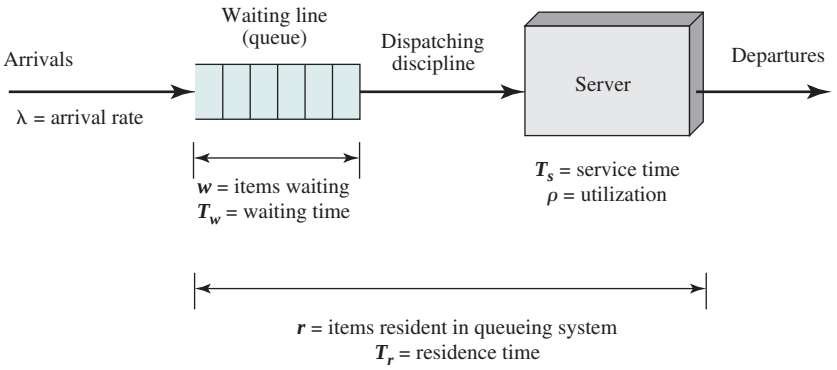



**Figure H.2 Queueing System Structure and Parameters for Single-Server Queue**

**Table H.1** Notation for Queueing Systems

| |
|---|
| $\lambda$ = arrival rate; mean number of arrivals per second |
| $T_s$ = mean service time for each arrival; amount of time being served, not counting time waiting in the queue |
| $\rho$ = utilization; fraction of time facility (server or servers) is busy |
| $w$ = mean number of items waiting to be served |
| $T_w$ = mean waiting time (including items that have to wait and items with waiting time = 0) |
| $r$ = mean number of items resident in system (waiting and being served) |
| $T_r$ = mean residence time; time an item spends in system (waiting and being served) |

is immediately dispatched to the server. The server in this model can represent anything that performs some function or service for a collection of items. Some examples are: a processor provides service to processes; a transmission line provides a transmission service to packets or frames of data; an I/O device provides a read or write service for I/O requests.

Table H.1 summarizes some important parameters associated with a queueing model. Items arrive at the facility at some average rate (items arriving per second) $\lambda$. At any given time, a certain number of items will be waiting in the queue (zero or more); the average number waiting is $w$, and the mean time that an item must wait is $T_w$. $T_w$ is averaged over all incoming items, including those that do not wait at all. The server handles incoming items with an average service time $T_s$; this is the time interval between the dispatching of an item to the server, and the departure of that item from the server. Utilization, $\rho$, is the fraction of time that the server is busy, measured over some interval of time. Finally, two parameters apply to the system as a whole. The average number of items resident in the system, including the item being served (if any) and the items waiting (if any), is $r$; and the average time that an item spends in the system, waiting and being served, is $T_r$; we refer to this as the mean residence time.[3]

[3]Again, in some of the literature, this is referred to as the mean queueing time, while other treatments use mean queueing time to mean the average time spent waiting in the queue (before being served).

If we assume that the capacity of the queue is infinite, then no items are ever lost from the system; they are just delayed until they can be served. Under these circumstances, the departure rate equals the arrival rate. As the arrival rate increases, the utilization increases and with it, congestion. The queue becomes longer, increasing waiting time. At $\rho = 1$, the server becomes saturated, working 100% of the time. Thus, the theoretical maximum input rate that can be handled by the system is

$$\lambda_{\max} = \frac{1}{T_s}$$

However, queues become very large near system saturation, growing without bound when $\rho = 1$. Practical considerations, such as response time requirements or buffer sizes, usually limit the input rate for a single server to between 70 and 90% of the theoretical maximum.

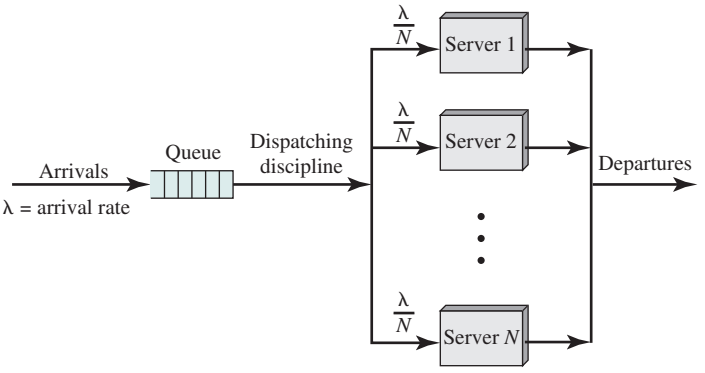
The following assumptions are typically made:

- **Item population:** Typically, we assume an infinite population. This means the arrival rate is not altered by the loss of population. If the population is finite, then the population available for arrival is reduced by the number of items currently in the system; this would typically reduce the arrival rate proportionally.
- **Queue size:** Typically, we assume an infinite queue size. Thus, the waiting line can grow without bound. With a finite queue, it is possible for items to be lost from the system. In practice, any queue is finite. In many cases, this will make no substantive difference to the analysis.
- **Dispatching discipline:** When the server becomes free, and if there is more than one item waiting, a decision must be made as to which item to dispatch next. The simplest approach is first-in-first-out; this discipline is what is normally implied when the term *queue* is used. Another possibility is last-in-first-out. One that you might encounter in practice is a dispatching discipline based on service time. For example, a packet-switching node may choose to dispatch packets on the basis of shortest first (to generate the most outgoing packets) or longest first (to minimize processing time relative to transmission time). Unfortunately, a discipline based on service time is very difficult to model analytically.

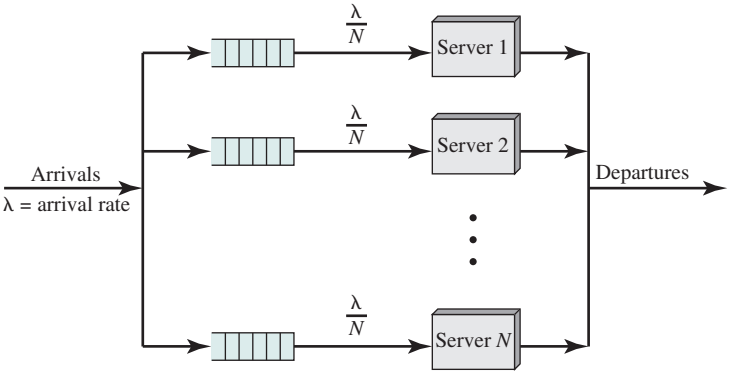
## H.3 THE MULTISERVER QUEUE

Figure H.3 shows a generalization of the simple model we have been discussing for multiple servers, all sharing a common queue. If an item arrives and at least one server is available, then the item is immediately dispatched to that server. It is assumed all servers are identical; thus, if more than one server is available, it makes no difference which server is chosen for the item. If all servers are busy, a queue begins to form. As soon as one server becomes free, an item is dispatched from the queue using the dispatching discipline in force.

With the exception of utilization, all of the parameters illustrated in Figure H.2 carry over to the multiserver case with the same interpretation. If we have $N$ identical




(a) Multiserver queue




(b) Multiple single-server queues

**Figure H.3 Multiserver Versus Multiple Single-Server Queues**

servers, then $\rho$ is the utilization of each server, and we can consider $N_\rho$ to be the utilization of the entire system; this latter term is often referred to as the traffic intensity, $u$. Thus, the theoretical maximum utilization is $N \times 100\%$, and the theoretical maximum input rate is:

$$\lambda_{\max} = \frac{N}{T_s}$$

The key characteristics typically chosen for the multiserver queue correspond to those for the single-server queue. That is, we assume an infinite population and an infinite queue size, with a single infinite queue shared among all servers. Unless otherwise stated, the dispatching discipline is FIFO. For the multiserver case, if all servers are assumed identical, the selection of a particular server for a waiting item has no effect on service time.

By way of contrast, Figure H.3b shows the structure of multiple single-server queues.

## H.4 POISSON ARRIVAL RATE

Typically, analytic queueing models assume the arrival rate obeys a Poisson distribution. This is what is assumed in the results of Table 9.6. We define this distribution as follows. If items arrive at a queue according to a Poisson distribution, this may be expressed as

$$\Pr[k \text{ items arrive in time interval } T] = \frac{(\lambda T)^k}{k!} e^{-\lambda T}$$

$$\mathrm{E}[\text{number of items to arrive in time interval } T] = \lambda T$$

$$\text{Mean arrival rate, in items per second} = \lambda$$

Arrivals occurring according to a Poisson process are often referred to as **random arrivals**. This is because the probability of arrival of an item in a small interval is proportional to the length of the interval, and is independent of the amount of elapsed time since the arrival of the last item. That is, when items are arriving according to a Poisson process, an item is as likely to arrive at one instant as any other, regardless of the instants at which the other customers arrive.

Another interesting property of the Poisson process is its relationship to the exponential distribution. If we look at the times between arrivals of items $T_a$ (called the interarrival times), then we find that this quantity obeys the exponential distribution:

$$\Pr[T_a < t] = 1 - e^{-\lambda t}$$

$$\mathrm{E}[T_a] = \frac{1}{\lambda}$$

Thus, the mean interarrival time is the reciprocal of the arrival rate, as we would expect.

# APPENDIX I

## THE COMPLEXITY OF ALGORITHMS

I.1 Complexity Overview

I.2 References

## I.1 COMPLEXITY OVERVIEW

A central issue in assessing the practicality of an algorithm is the relative amount of time it takes to execute the algorithm. Typically, one cannot be sure that one has found the most efficient algorithm for a particular function. The most that one can say is that for a particular algorithm, the level of effort for execution is of a particular order of magnitude. One can then compare that order of magnitude to the speed of current or predicted processors to determine the level of practicality of a particular algorithm.

A common measure of the efficiency of an algorithm is its time complexity. We define the **time complexity** of an algorithm to be $f(n)$ if, for all $n$ and all inputs of length $n$, the execution of the algorithm takes at most $f(n)$ steps. Thus, for a given size of input and a given processor speed, the time complexity is an upper bound on the execution time.

There are several ambiguities here. First, the definition of a step is not precise. A step could be a single operation of a Turing machine, a single processor machine instruction, a single high-level language machine instruction, and so on. However, these various definitions of step should all be related by simple multiplicative constants. For very large values of $n$, these constants are not important. What is important is how fast the relative execution time is growing.

A second issue is that, generally speaking, we cannot pin down an exact formula for $f(n)$. We can only approximate it. But again, we are primarily interested in the rate of change of $f(n)$ as $n$ becomes very large.

There is a standard mathematical notation, known as the "big-O" notation, for characterizing the time complexity of algorithms that is useful in this context. The definition is as follows: $f(n) = O(g(n))$ if and only if there exist two numbers $a$ and $M$ such that

$$|f(n)| \leq a \times |g(n)|, \qquad n \geq M \tag{I.1}$$

An example helps clarify the use of this notation. Suppose we wish to evaluate a general polynomial of the form:

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

Consider the following simple-minded algorithm from [POHL81]:

```
algorithm P1;
    n, i, j: integer; x, polyval: real;
    a, S: array [0..100] of real;
    begin
     read(x, n);
     for i := 0 upto n do
     begin
          S[i] := 1; read(a[i]);
          for j := 1 upto i do S[i] := x × S[i];
          S[i] := a[i] × S[i]
     end;
```

```
    polyval := 0;
    for i := 0 upto n do polyval := polyval + S[i];
    write ('value at', x, 'is', polyval)
end.
```

In this algorithm, each sub-expression is evaluated separately. Each S[$i$] requires ($i$ + 1) multiplications: $i$ multiplications to compute S[$i$] and one to multiply by a[$i$]. Computing all $n$ terms requires

$$\sum_{i=0}^{n}(i+1) = \frac{(n+2)(n+1)}{2}$$

multiplications. There are also ($n$ + 1) additions, which we can ignore relative to the much larger number of multiplications. Thus, the time complexity of this algorithm is $f(n) = (n+2)(n+1)/2$. We now show $f(n) = O(n^2)$. From the definition of Equation (I.1), we want to show that for $a = 1$ and $M = 4$, the relationship holds for $g(n) = n^2$. We do this by induction on $n$. The relationship holds for $n = 4$ because $(4+2)(4+1)/2 = 15 < 4^2 = 16$. Now assume it holds for all values of $n$ up to $k$ [i.e., $(k+2)(k+1)/2 < k^2$]. Then, with $n = k + 1$:

$$\begin{aligned}\frac{(n+2)(n+1)}{2} &= \frac{(k+3)(k+2)}{2}\\ &= \frac{(k+2)(k+1)}{2} + k + 2\\ &\le k^2 + k + 2\\ &\le k^2 + 2k + 1 = (k+1)^2 = n^2\end{aligned}$$

Therefore, the result is true for $n = k + 1$.

In general, the big-O notation makes use of the term that grows the fastest. For example:

1. $O[ax^7 + 3x^3 + \sin(x)] = O(ax^7) = O(x^7)$
2. $O(e^n + an^{10}) = O(e^n)$
3. $O(n! + n^{50}) = O(n!)$

There is much more to the big-O notation, with fascinating ramifications. For the interested reader, two of the best accounts are in [GRAH94] and [KNUT97].

An algorithm with an input of size $n$ is said to be:

1. **Linear:** if the running time is $O(n)$
2. **Polynomial:** if the running time is $O(n^t)$ for some constant $t$
3. **Exponential:** if the running time is $O(t^{h(n)})$ for some constant $t$ and polynomial $h(n)$

Generally, a problem that can be solved in polynomial time is considered feasible, whereas anything larger than polynomial time, especially exponential time, is considered infeasible. But you must be careful with these terms. First, if the size of the input is small enough, even very complex algorithms become feasible. Suppose, for example, you have a system that can execute $10^{12}$ operations per unit time.

**Table I.1** Level of Effort for Various Levels of Complexity

| Complexity | Size of Input | Operations |
|---|---|---|
| $\log_2 n$ | $2^{10^{12}} = 10^{3\times10^{11}}$ | $10^{12}$ |
| $n$ | $10^{12}$ | $10^{12}$ |
| $n^2$ | $10^6$ | $10^{12}$ |
| $n^6$ | $10^2$ | $10^{12}$ |
| $2^n$ | 39 | $10^{12}$ |
| $n!$ | 15 | $10^{12}$ |

Table I-1 shows the size of input that can be handled in one time unit for algorithms of various complexities. For algorithms of exponential or factorial time, only very small inputs can be accommodated.

The second thing to be careful about is the way in which the input is characterized. For example, the complexity of cryptanalysis of an encryption algorithm can be characterized equally well in terms of the number of possible keys or the length of the key. For the Advanced Encryption Standard (AES), for example, the number of possible keys is $2^{128}$, and the length of the key is 128 bits. If we consider a single encryption to be a "step" and the number of possible keys to be $N = 2^n$, then the time complexity of the algorithm is linear in terms of the number of keys [$O(N)$] but exponential in terms of the length of the key [$O(2^n)$].

## I.2 REFERENCES

**GRAH94** Graham, R.; Knuth, D.; and Patashnik, O. *Concrete Mathematics: A Foundation for Computer Science.* Reading, MA: Addison-Wesley, 1994.

**KNUT97** Knuth, D. *The Art of Computer Programming, Volume 1: Fundamental Algorithms.* Reading, MA: Addison-Wesley, 1997.

**POHL81** Pohl, I., and Shaw, A. *The Nature of Computation: An Introduction to Computer Science.* Rockville, MD: Computer Science Press, 1981.

# APPENDIX J

# DISK STORAGE DEVICES


**J.1 Magnetic Disk**
- Data Organization and Formatting
- Physical Characteristics

**J.2 Optical Memory**
- CD-ROM
- CD Recordable
- CD Rewritable
- Digital Versatile Disk
- High-Definition Optical Disks

## J.1 MAGNETIC DISK

A disk is a circular platter constructed of metal or of plastic coated with a magnetizable material. Data are recorded on and later retrieved from the disk via a conducting coil named the **head**. During a read or write operation, the head is stationary while the platter rotates beneath it.

The write mechanism exploits the fact that electricity flowing through a coil produces a magnetic field. Electric pulses are sent to the head, and magnetic patterns are recorded on the surface below, with different patterns for positive and negative currents. The read mechanism is based on the fact that a magnetic field moving relative to a coil produces an electrical current in the coil. When the surface of the disk passes under the head, it generates a current of the same polarity as the one already recorded.

### Data Organization and Formatting

The head is a relatively small device capable of reading from or writing to a portion of the platter rotating beneath it. This gives rise to the organization of data on the platter in a concentric set of rings, called **tracks**. Each track is the same width as the head. There are thousands of tracks per surface.

Figure J.1 depicts this data layout. Adjacent tracks are separated by **gaps**. This prevents, or at least minimizes, errors due to misalignment of the head or simply interference of magnetic fields.

Data are transferred to and from the disk in **sectors** (see Figure J.1). There are typically hundreds of sectors per track, and these may be of either fixed or variable length. In most contemporary systems, fixed-length sectors are used, with 512 bytes being the nearly universal sector size. To avoid imposing unreasonable precision requirements on the system, adjacent sectors are separated by intratrack (intersector) gaps.

A bit near the center of a rotating disk travels past a fixed point (such as a read-write head) slower than a bit on the outside. Therefore, some way must be found to compensate for the variation in speed so the head can read all the bits at the same rate. This can be done by increasing the spacing between bits of information recorded in segments of the disk. The information can then be scanned at the same rate by rotating the disk at a fixed speed, known as the **constant angular velocity (CAV)**.

Figure J.2a shows the layout of a disk using CAV. The disk is divided into a number of pie-shaped sectors and into a series of concentric tracks. The advantage of using CAV is that individual blocks of data can be directly addressed by track and sector. To move the head from its current location to a specific address, it only takes a short movement of the head to a specific track and a short wait for the proper sector to spin under the head. The disadvantage of CAV is that the amount of data that can be stored on the long outer tracks is the same as what can be stored on the short inner tracks.

Because the **density**, in bits per linear inch, increases in moving from the outermost track to the innermost track, disk storage capacity in a straightforward CAV system is limited by the maximum recording density that can be achieved on the innermost track. To increase density, modern hard disk systems use a technique known as **multiple zone recording**, in which the surface is divided into a number of concentric zones (16 is typical). Within a zone, the number of bits per track is

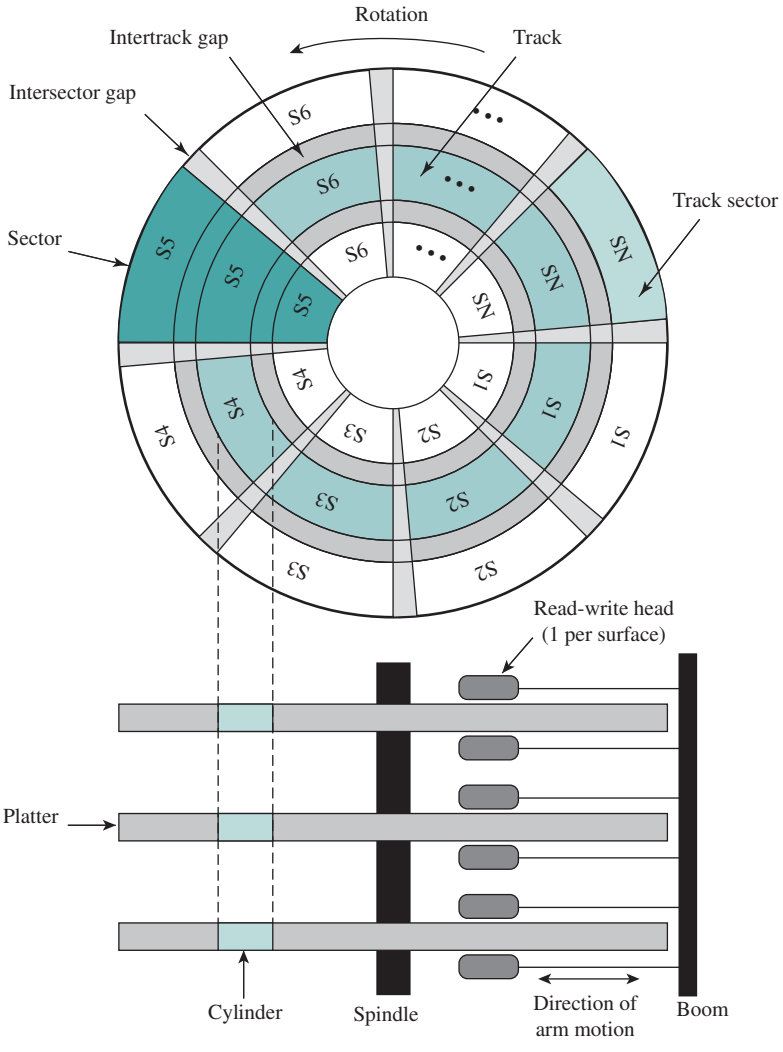
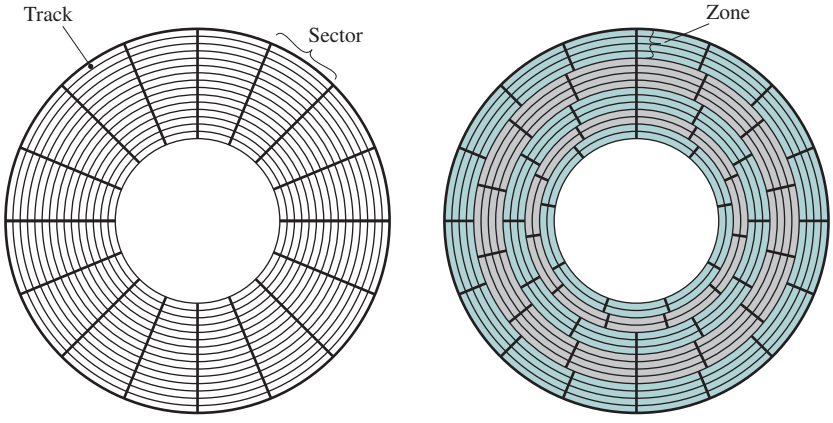



**Figure J.1** **Disk Data Layout**

constant. Zones farther from the center contain more bits (more sectors) than zones closer to the center. This allows for greater overall storage capacity at the expense of somewhat more complex circuitry. As the disk head moves from one zone to another, the length (along the track) of individual bits changes, causing a change in the timing for reads and writes. Figure J.2b suggests the nature of multiple zone recording; in this illustration, each zone is only a single track wide.

Some means is needed to locate sector positions within a track. Clearly, there must be some starting point on the track, and a way of identifying the start and end of each sector. These requirements are handled by means of control data recorded on the disk. Thus, the disk is formatted with some extra data used only by the disk drive and not accessible to the user.




(a) Constant angular velocity (b) Multiple zone recording

**Figure J.2 Comparison of Disk Layout Methods**

## Physical Characteristics

Table J.1 lists the major characteristics that differentiate among the various types of magnetic disks. First, the head may either be fixed or movable with respect to the radial direction of the platter. In a **fixed-head disk**, there is one read/write head per track. All of the heads are mounted on a rigid arm that extends across all tracks; such systems are rare today. In a **movable-head disk**, there is only one read/write head. Again, the head is mounted on an arm. Because the head must be able to be positioned above any track, the arm can be extended or retracted for this purpose.

The disk itself is mounted in a disk drive, which consists of the arm, a spindle that rotates the disk, and the electronics needed for input and output of binary data. A **nonremovable disk** is permanently mounted in the disk drive; the hard disk in a personal computer is a nonremovable disk. A **removable disk** can be removed and replaced with another disk. The advantage of the latter type is that unlimited amounts of data are available with a limited number of disk systems. Furthermore, such a disk may be moved from one computer system to another.

For most disks, the magnetizable coating is applied to both sides of the platter, which is then referred to as **double-sided**. Some less expensive disk systems use **single-sided** disks.

**Table J.1** Physical Characteristics of Disk Systems

| | |
|---|---|
| **Head Motion** | **Platters** |
| Fixed head (one per track) | Single platter |
| Movable head (one per surface) | Multiple platter |
| **Disk Portability** | **Head Mechanism** |
| Nonremovable disk | Contact (floppy) |
| Removable disk | Fixed gap |
| **Sides** | Aerodynamic gap (Winchester) |
| Single sided | |
| Double sided | |

Some disk drives accommodate **multiple platters** stacked vertically a fraction of an inch apart. Multiple arms are provided (Figure J.3). Multiple-platter disks employ a movable head, with one read-write head per platter surface. All of the heads are mechanically fixed so all are at the same distance from the center of the disk and move together. Thus, at any time, all of the heads are positioned over tracks that are of equal distance from the center of the disk. The set of all the tracks in the same relative position on the platter is referred to as a **cylinder**. For example, all of the shaded tracks in Figure J.4 are part of one cylinder.

Finally, the head mechanism provides a classification of disks into three types. Traditionally, the read/write head has been positioned at a fixed distance above the platter, allowing an air gap. At the other extreme is a head mechanism that actually comes into physical contact with the medium during a read or write operation. This mechanism is used with the **floppy disk**, which is a small, flexible platter and the least expensive type of disk.

To understand the third type of disk, we need to comment on the relationship between data density and the size of the air gap. The head must generate or sense an electromagnetic field of sufficient magnitude to write and read properly. The narrower the head is, the closer it must be to the platter surface to function. A narrower head means narrower tracks and therefore greater data density, which is desirable.

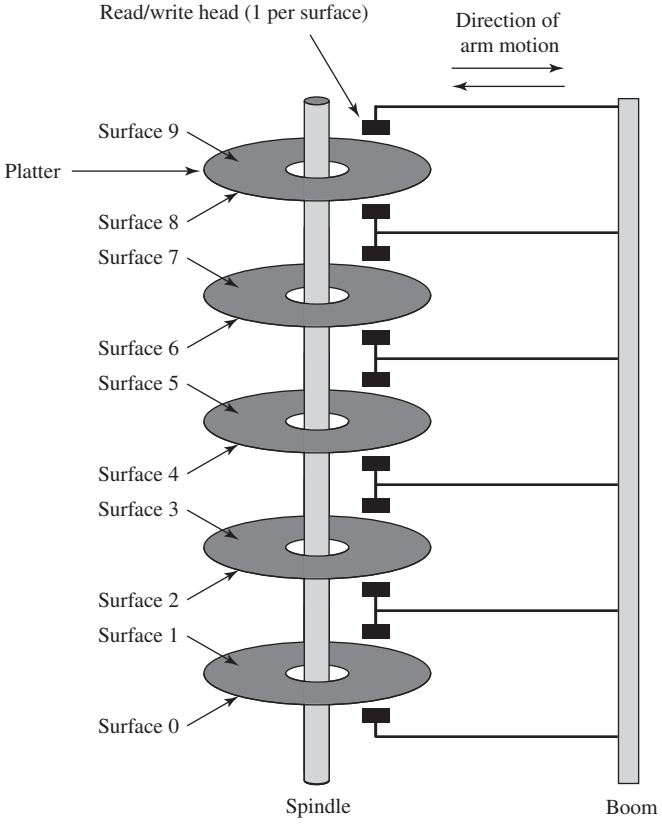



**Figure J.3 Components of a Disk Drive**

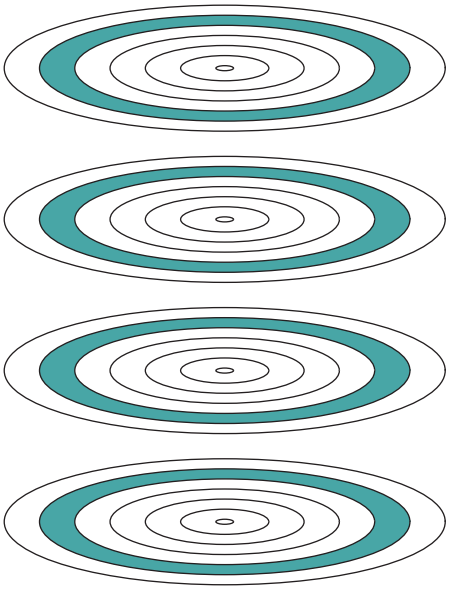


**Figure J.4 Tracks and Cylinders**

However, the closer the head is to the disk, the greater the risk of error from impurities or imperfections. To push the technology further, the **Winchester disk** was developed. Winchester heads are used in sealed drive assemblies that are almost free of contaminants. They are designed to operate closer to the disk's surface than conventional rigid disk heads, thus allowing greater data density. The head is actually an aerodynamic foil that rests lightly on the platter's surface when the disk is motionless. The air pressure generated by a spinning disk is enough to make the foil rise above the surface. The resulting noncontact system can be engineered to use narrower heads that operate closer to the platter's surface than conventional rigid disk heads.

Table J.2 gives disk parameters for typical contemporary high-performance disks.

## J.2 OPTICAL MEMORY

In 1983, one of the most successful consumer products of all time was introduced: the compact disk (CD) digital audio system. The CD is a nonerasable disk that can store more than 60 minutes of audio information on one side. The huge commercial success of the CD enabled the development of low-cost optical-disk storage technology that has revolutionized computer data storage. A variety of optical-disk systems are in use (see Table J.3). We briefly review each of these.

### CD-ROM

The audio CD and the CD-ROM (compact disk read-only memory) share a similar technology. The main difference is that CD-ROM players are more rugged and have error-correction devices to ensure data are properly transferred from disk to

**Table J.2** Typical Hard Disk Drive Parameters

| Characteristics | Seagate Barracuda ES.2 | Seagate Barracuda 7200.10 | Seagate Barracuda 7200.9 | Seagate | Hitachi Microdrive |
|---|---|---|---|---|---|
| Application | High-capacity server | High-performance desktop | Entry-level desktop | Laptop | Handheld devices |
| Capacity | 1 TB | 750 GB | 160 GB | 120 GB | 8 GB |
| Minimum track-to-track seek time | 0.8 ms | 0.3 ms | 1.0 ms | — | 1.0 ms |
| Average seek time | 8.5 ms | 3.6 ms | 9.5 ms | 12.5 ms | 12 ms |
| Spindle speed | 7200 rpm | 7200 rpm | 7200 | 5400 rpm | 3600 rpm |
| Average rotational delay | 4.16 ms | 4.16 ms | 4.17 ms | 5.6 ms | 8.33 ms |
| Maximum transfer rate | 3 GB/s | 300 MB/s | 300 MB/s | 150 MB/s | 10 MB/s |
| Bytes per sector | 512 | 512 | 512 | 512 | 512 |
| Tracks per cylinder (number of platter surfaces) | 8 | 8 | 2 | 8 | 2 |

computer. Both types of disk are made in the same way. The disk is formed from a resin, such as polycarbonate. Digitally recorded information (either music or computer data) is imprinted as a series of microscopic pits on the surface of the polycarbonate. This is done, first of all, with a finely focused, high-intensity laser to create a master disk. The master is used, in turn, to make a die to stamp out copies onto polycarbonate. The pitted surface is then coated with a highly reflective surface, usually aluminum or gold. This shiny surface is protected against dust and scratches by a top coat of clear acrylic. Finally, a label can be silkscreened onto the acrylic.

Information is retrieved from a CD or CD-ROM by a low-powered laser housed in an optical-disk player, or drive unit. The laser shines through the clear polycarbonate while a motor spins the disk past it (see Figure J.5). The intensity of the reflected light of the laser changes as it encounters a pit. Specifically, if the laser beam falls on a pit, which has a somewhat rough surface, the light scatters and a low intensity is reflected back to the source. The areas between pits are called *lands*. A land is a smooth surface, which reflects back at higher intensity. The change between pits and lands is detected by a photosensor and converted into a digital signal. The sensor tests the surface at regular intervals. The beginning or end of a pit represents a 1; when no change in elevation occurs between intervals, a 0 is recorded.

Recall that on a magnetic disk, information is recorded in concentric tracks. With the simplest CAV system, the number of bits per track is constant. An increase in density is achieved with multiple zoned recording, in which the surface is divided into a number of zones, with zones farther from the center containing more bits than zones closer to the center. Although this technique increases capacity, it is still not optimal.

To achieve greater capacity, CDs and CD-ROMs do not organize information on concentric tracks. Instead, the disk contains a single spiral track, beginning near

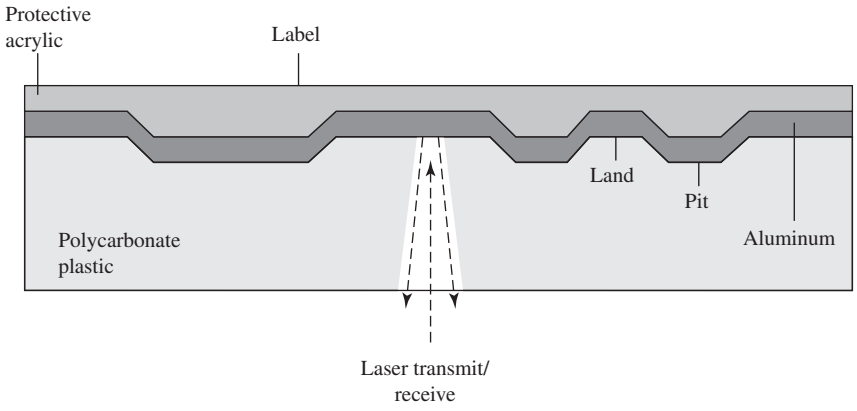



**Figure J.5 CD Operation**

the center and spiraling out to the outer edge of the disk. Sectors near the outside of the disk are the same length as those near the inside. Thus, information is packed evenly across the disk in segments of the same size, and these are scanned at the same rate by rotating the disk at a variable speed. The pits are then read by the laser at a **constant linear velocity (CLV)**. The disk rotates more slowly for accesses near the outer edge than for those near the center. Thus, the capacity of a track and the rotational delay both increase for positions nearer the outer edge of the disk. The data capacity for a CD-ROM is about 680 MB.

CD-ROM is appropriate for the distribution of large amounts of data to a large number of users. Because of the expense of the initial writing process, it is not appropriate for individualized applications. Compared with traditional magnetic disks, the CD-ROM has three major advantages:

- The information-storage capacity is much greater on the optical disk.
- The optical disk together with the information stored on it can be mass replicated inexpensively—unlike a magnetic disk. The data on a magnetic disk has to be reproduced by copying one disk at a time using two disk drives.
- The optical disk is removable, allowing the disk itself to be used for archival storage. Most magnetic disks are nonremovable. The information on nonremovable magnetic disks must first be copied to some other storage device before the disk drive/disk can be used to store new information.

The disadvantages of CD-ROM are as follows:

- It is read-only and cannot be updated.
- It has an access time much longer than that of a magnetic disk drive, as much as half a second.

## CD Recordable

To accommodate applications in which only one or a small number of copies of a set of data is needed, the write-once read-many CD, known as the CD recordable (CD-R) has been developed. For CD-R, a disk is prepared in such a way that it can

be subsequently written once with a laser beam of modest intensity. Thus, with a somewhat more expensive disk controller than for CD-ROM, the customer can write once as well as read the disk.

The CD-R medium is similar to, but not identical to, that of a CD or CD-ROM. For CDs and CD-ROMs, information is recorded by the pitting of the surface of the medium, which changes reflectivity. For a CD-R, the medium includes a dye layer. The dye is used to change reflectivity and is activated by a high-intensity laser. The resulting disk can be read on a CD-R drive or a CD-ROM drive.

The CD-R optical disk is attractive for archival storage of documents and files. It provides a permanent record of large volumes of user data.

### CD Rewritable

The CD-RW optical disk can be repeatedly written and overwritten, as with a magnetic disk. Although a number of approaches have been tried, the only pure optical approach that has proved attractive is called phase change. The phase change disk uses a material that has two significantly different reflectivities in two different phase states. There is an amorphous state, in which the molecules exhibit a random orientation that reflects light poorly; and a crystalline state, which has a smooth surface that reflects light well. A beam of laser light can change the material from one phase to the other. The primary disadvantage of phase change optical disks is that the material eventually and permanently loses its desirable properties. Current materials can be used for between 500,000 and 1,000,000 erase cycles.

The CD-RW has the obvious advantage over CD-ROM and CD-R that it can be rewritten and thus used as a true secondary storage. As such, it competes with magnetic disk. A key advantage of the optical disk is that the engineering tolerances for optical disks are much less severe than for high-capacity magnetic disks. Thus, optical disks exhibit higher reliability and longer life.

### Digital Versatile Disk

With the capacious digital versatile disk (DVD), the electronics industry has at last found an acceptable replacement for the analog VHS video tape. The DVD will replace the video tape used in video cassette recorders (VCRs) and, more important for this discussion, replace the CD-ROM in personal computers and servers. The DVD takes video into the digital age. It delivers movies with impressive picture quality, and it can be randomly accessed like audio CDs, which DVD machines can also play. Vast volumes of data can be crammed onto the disk, currently seven times as much as a CD-ROM. With DVD's huge storage capacity and vivid quality, PC games will become more realistic, and educational software will incorporate more video. Following in the wake of these developments will be a new crest of traffic over the Internet and corporate intranets, as this material is incorporated into websites.

The DVD's greater capacity is due to three differences from CDs:

1. Bits are packed more closely on a DVD. The spacing between loops of a spiral on a CD is 1.6 $\mu$m and the minimum distance between pits along the spiral is 0.834 $\mu$m. The DVD uses a laser with shorter wavelength and achieves a loop

spacing of 0.74 $\mu$m and a minimum distance between pits of 0.4 $\mu$m. The result of these two improvements is about a sevenfold increase in capacity, to about 4.7 GB.

2. The DVD employs a second layer of pits and lands on top of the first layer. A dual-layer DVD has a semireflective layer on top of the reflective layer, and by adjusting focus, the lasers in DVD drives can read each layer separately. This technique almost doubles the capacity of the disk, to about 8.5 GB. The lower reflectivity of the second layer limits its storage capacity so a full doubling is not achieved.
3. The DVD-ROM can be two sided, whereas data are recorded on only one side of a CD. This brings total capacity up to 17 GB.

As with the CD, DVDs come in writeable as well as read-only versions (see Table J.3).

## High-Definition Optical Disks

High-definition optical disks are designed to store high-definition videos and to provide significantly greater storage capacity compared to DVDs. The higher bit density is achieved by using a laser with a shorter wavelength, in the blue-violet range. The data pits, which constitute the digital 1s and 0s, are smaller on the high-definition optical disks compared to DVD because of the shorter laser wavelength.

**Table J.3** Optical Disk Products

| |
|---|
| **CD** |
| Compact Disk. A nonerasable disk that stores digitized audio information. The standard system uses 12-cm disks and can record more than 60 minutes of uninterrupted playing time. |
| **CD-ROM** |
| Compact Disk Read-Only Memory. A nonerasable disk used for storing computer data. The standard system uses 12-cm disks and can hold more than 650 Mbytes. |
| **CD-R** |
| CD Recordable. Similar to a CD-ROM. The user can write to the disk only once. |
| **CD-RW** |
| CD Rewritable. Similar to a CD-ROM. The user can erase and rewrite to the disk multiple times. |
| **DVD** |
| Digital Versatile Disk. A technology for producing digitized, compressed representation of video information, as well as large volumes of other digital data. Both 8- and 12-cm diameters are used, with a double-sided capacity of up to 17 GB. The basic DVD is read-only (DVD-ROM). |
| **DVD-R** |
| DVD Recordable. Similar to a DVD-ROM. The user can write to the disk only once. Only one-sided disks can be used. |
| **DVD-RW** |
| DVD Rewritable. Similar to a DVD-ROM. The user can erase and rewrite to the disk multiple times. Only one-sided disks can be used. |
| **Blu-Ray DVD** |
| High definition video disk. Provides considerably greater data storage density than DVD, using a 405-nm (blue-violet) laser. A single layer on a single side can store 25 GB. |

Two disk formats and technologies initially competed for market acceptance: HD DVD and Blu-ray DVD. The Blu-ray scheme ultimately achieved market dominance. The HD DVD scheme can store 15 GB on a single layer on a single side. Blu-ray positions the data layer on the disk closer to the laser (shown on the right-hand side of each diagram in Figure J.6). This enables a tighter focus and less distortion and thus smaller pits and tracks. Blu-ray can store 25 GB on a single layer. Three versions are available: read only (BD-ROM), recordable once (BD-R), and rerecordable (BD-RE).

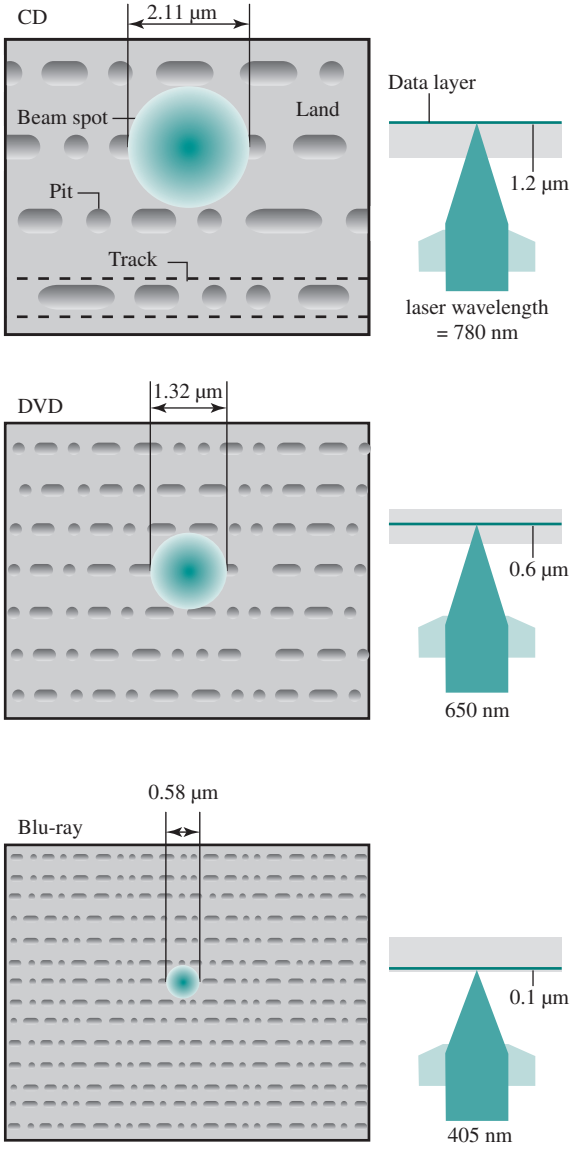



**Figure J.6 Optical Memory Characteristics**

# APPENDIX K

## CRYPTOGRAPHIC ALGORITHMS


**K.1 Symmetric Encryption**
- The Data Encryption Standard (DES)
- Advanced Encryption Standard (AES)

**K.2 Public-Key Cryptography**
- Rivest-Shamir-Adleman (RSA) Algorithm

**K.3 Message Authentication and Hash Functions**
- Authentication Using Symmetric Encryption
- Message Authentication without Message Encryption
- Message Authentication Code
- One-Way Hash Function

**K.4 Secure Hash Functions**

The essential technology underlying virtually all automated network and computer security applications is cryptography. Two fundamental approaches are in use: symmetric encryption, also known as conventional encryption, and public-key encryption, also known as asymmetric encryption. This appendix provides an overview of both types of encryption, together with a brief discussion of some important encryption algorithms.

## K.1 SYMMETRIC ENCRYPTION

Symmetric encryption was the only type of encryption in use prior to the introduction of public-key encryption in the late 1970s. Symmetric encryption has been used for secret communication by countless individuals and groups, from Julius Caesar to the German U-boat force to present-day diplomatic, military, and commercial users. It remains by far the more widely used of the two types of encryption.

A symmetric encryption scheme has five ingredients (see Figure K.1):

- **Plaintext:** This is the original message or data that is fed into the algorithm as input.
- **Encryption algorithm:** The encryption algorithm performs various substitutions and transformations on the plaintext.
- **Secret key:** The secret key is also input to the encryption algorithm. The exact substitutions and transformations performed by the algorithm depend on the key.
- **Ciphertext:** This is the scrambled message produced as output. It depends on the plaintext and the secret key. For a given message, two different keys will produce two different ciphertexts.
- **Decryption algorithm:** This is essentially the encryption algorithm run in reverse. It takes the ciphertext and the secret key and produces the original plaintext.

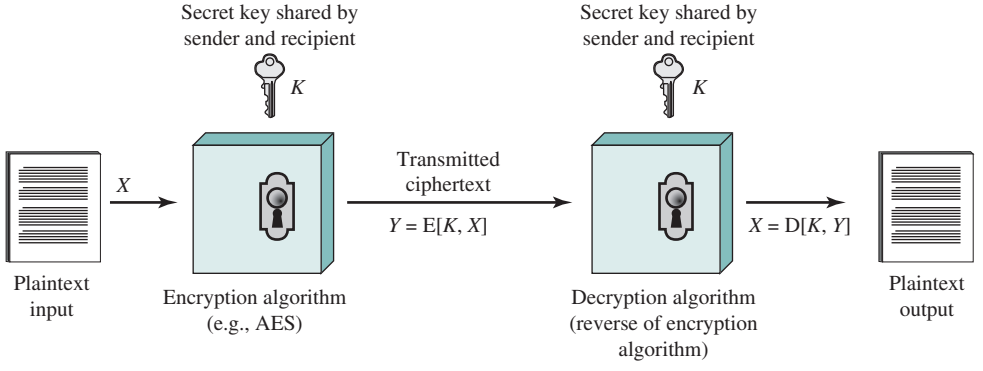



**Figure K.1 Simplified Model of Symmetric Encryption**

There are two requirements for secure use of symmetric encryption:

1. We need a strong encryption algorithm. At a minimum, we would like the algorithm to be such that an opponent who knows the algorithm and has access to one or more ciphertexts would be unable to decipher the ciphertext or figure out the key. This requirement is usually stated in a stronger form: The opponent should be unable to decrypt ciphertext or discover the key, even if he or she is in possession of a number of ciphertexts together with the plaintext that produced each ciphertext.
2. Sender and receiver must have obtained copies of the secret key in a secure fashion and must keep the key secure. If someone can discover the key and knows the algorithm, all communication using this key is readable.

There are two general approaches to attacking a symmetric encryption scheme. The first attack is known as **cryptanalysis**. Cryptanalytic attacks rely on the nature of the algorithm plus perhaps some knowledge of the general characteristics of the plaintext or even some sample plaintext-ciphertext pairs. This type of attack exploits the characteristics of the algorithm to attempt to deduce a specific plaintext or to deduce the key being used. If the attack succeeds in deducing the key, the effect is catastrophic: All future and past messages encrypted with that key are compromised.

The second method, known as the **brute-force** attack, is to try every possible key on a piece of ciphertext until an intelligible translation into plaintext is obtained. On average, half of all possible keys must be tried to achieve success. Table K.1 shows how much time is involved for various key sizes. The table shows results for each key size, assuming it takes 1 $\mu$s to perform a single decryption, a reasonable order of magnitude for today's computers. With the use of massively parallel organizations of microprocessors, it may be possible to achieve processing rates many orders of magnitude greater. The final column of the table considers the results for a system that can process 1 million keys per microsecond. As one can see, at this performance level, a 56-bit key can no longer be considered computationally secure.

The most commonly used symmetric encryption algorithms are block ciphers. A block cipher processes the plaintext input in fixed-size blocks and produces a block of ciphertext of equal size for each plaintext block. The two most important symmetric algorithms, both of which are block ciphers, are the Data Encryption Standard (DES) and the Advanced Encryption Standard (AES).

**Table K.1** Average Time Required for Exhaustive Key Search

| Key Size (bits) | Number of Alternative Keys | Time Required at 1 Decryption/$\mu$s | Time Required at $10^6$ Decryptions/$\mu$s |
|---|---|---|---|
| 32 | $2^{32} = 4.3 \times 10^9$ | $2^{31}\ \mu s = 35.8$ minutes | 2.15 milliseconds |
| 56 | $2^{56} = 7.2 \times 10^{16}$ | $2^{55}\ \mu s = 1142$ years | 10.01 hours |
| 128 | $2^{128} = 3.4 \times 10^{38}$ | $2^{127}\ \mu s = 5.4 \times 10^{24}$ years | $5.4 \times 10^{18}$ years |
| 168 | $2^{168} = 3.7 \times 10^{50}$ | $2^{167}\ \mu s = 5.9 \times 10^{36}$ years | $5.9 \times 10^{30}$ years |
| 26 characters (permutation) | $26! = 4 \times 10^{26}$ | $2 \times 10^{26}\ \mu s = 6.4 \times 10^{12}$ years | $6.4 \times 10^6$ years |

### The Data Encryption Standard (DES)

DES has been the dominant encryption algorithm since its introduction in 1977. However, because DES uses only a 56-bit key, it was only a matter of time before computer processing speed made DES obsolete. In 1998, the Electronic Frontier Foundation (EFF) announced that it had broken a DES challenge using a special-purpose "DES cracker" machine that was built for less than $250,000. The attack took less than three days. The EFF has published a detailed description of the machine, enabling others to build their own cracker. And, of course, hardware prices continue to drop as speeds increase, making DES worthless.

The life of DES was extended by the use of triple DES (3DES), which involves repeating the basic DES algorithm three times, using either two or three unique keys, for a key size of 112 or 168 bits.

The principal drawback of 3DES is that the algorithm is relatively sluggish in software. A secondary drawback is that both DES and 3DES use a 64-bit block size. For reasons of both efficiency and security, a larger block size is desirable.

### Advanced Encryption Standard

Because of these drawbacks, 3DES is not a reasonable candidate for long-term use. As a replacement, the National Institute of Standards and Technology (NIST) in 1997 issued a call for proposals for a new Advanced Encryption Standard (AES), which should have a security strength equal to or better than 3DES and significantly improved efficiency. In addition to these general requirements, NIST specified that AES must be a symmetric block cipher with a block length of 128 bits and support for key lengths of 128, 192, and 256 bits. Evaluation criteria include security, computational efficiency, memory requirements, hardware and software suitability, and flexibility. In 2001, NIST issued AES as a federal information processing standard (FIPS 197).

## K.2 PUBLIC-KEY CRYPTOGRAPHY

Public-key encryption, first publicly proposed by Diffie and Hellman in 1976, is the first truly revolutionary advance in encryption in literally thousands of years. For one thing, public-key algorithms are based on mathematical functions rather than on simple operations on bit patterns. More important, public-key cryptography is asymmetric, involving the use of two separate keys, in contrast to symmetric encryption, which uses only one key. The use of two keys has profound consequences in the areas of confidentiality, key distribution, and authentication.

Before proceeding, we should first mention several common misconceptions concerning public-key encryption. One is that public-key encryption is more secure from cryptanalysis than symmetric encryption. In fact, the security of any encryption scheme depends on the length of the key and the computational work involved in breaking a cipher. There is nothing in principle about either symmetric or public-key encryption that makes one superior to another from the point of view of resisting cryptanalysis. A second misconception is that public-key encryption is a general-purpose technique that has made symmetric encryption obsolete. On the contrary, because of the computational overhead of current public-key encryption schemes,

there seems no foreseeable likelihood that symmetric encryption will be abandoned. Finally, there is a feeling that key distribution is trivial when using public-key encryption, compared to the rather cumbersome handshaking involved with key distribution centers for symmetric encryption. In fact, some form of protocol is needed, often involving a central agent, and the procedures involved are no simpler or any more efficient than those required for symmetric encryption.

A public-key encryption scheme has six ingredients (see Figure K.2):

- **Plaintext:** This is the readable message or data that is fed into the algorithm as input.
- **Encryption algorithm:** The encryption algorithm performs various transformations on the plaintext.
- **Public and private key:** This is a pair of keys that have been selected so if one is used for encryption, the other is used for decryption. The exact transformations performed by the encryption algorithm depend on the public or private key that is provided as input.
- **Ciphertext:** This is the scrambled message produced as output. It depends on the plaintext and the key. For a given message, two different keys will produce two different ciphertexts.
- **Decryption algorithm:** This algorithm accepts the ciphertext and the matching key and produces the original plaintext.

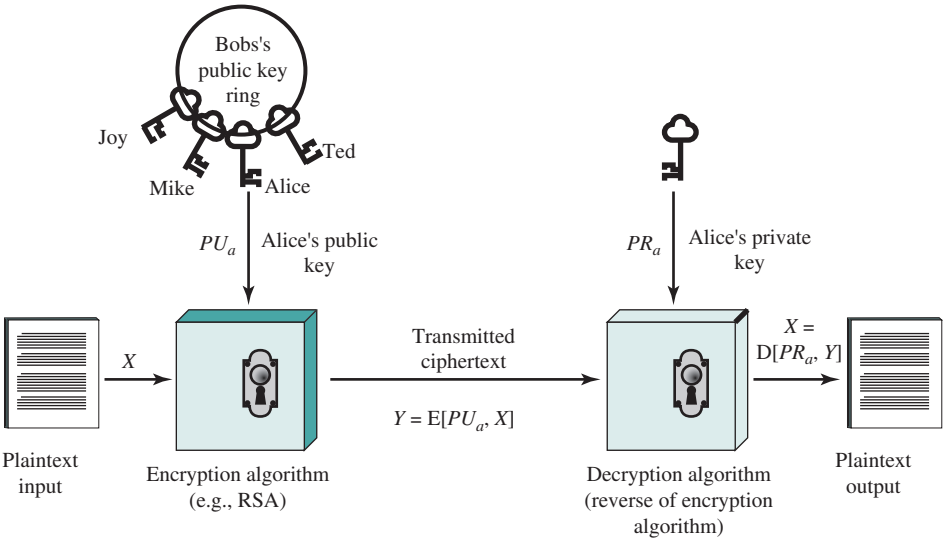
The process works (produces the correct plaintext on output) regardless of the order in which the pair of keys is used. As the names suggest, the public key of the pair is made public for others to use, while the private key is known only to its owner.

Now, say Bob wants to send a private message to Alice, and suppose that he has Alice's public key and Alice has the matching private key (see Figure K.2a). Using Alice's public key, Bob encrypts the message to produce ciphertext. The ciphertext is then transmitted to Alice. When Alice gets the ciphertext, she decrypts it using her private key. Because only Alice has a copy of her private key, no one else can read the message.

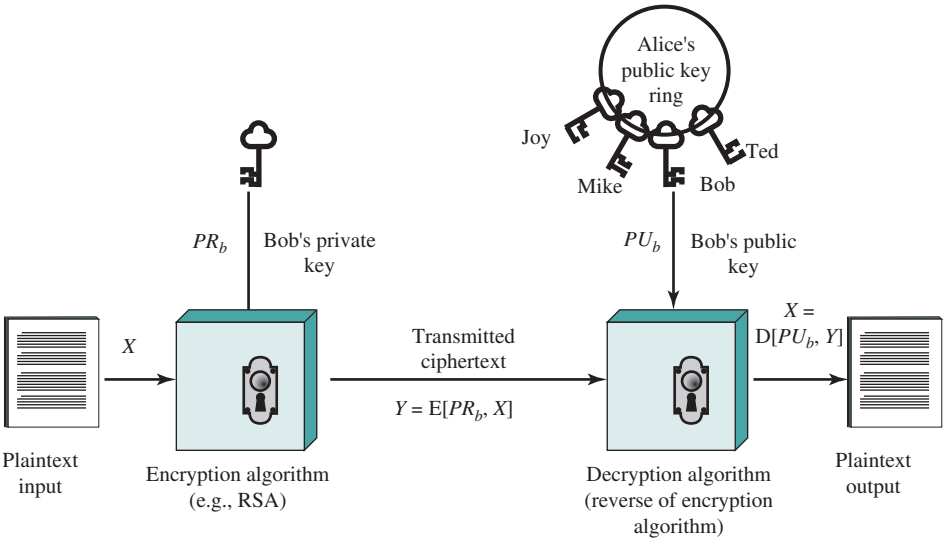
Public-key encryption can be used in another way, as illustrated in Figure K.2b. Suppose Bob wants to send a message to Alice and, although it isn't important that the message be kept secret, he wants Alice to be certain that the message is indeed from him. In this case Bob uses his own private key to encrypt the message. When Alice receives the ciphertext, she finds that she can decrypt it with Bob's public key, thus proving that the message must have been encrypted by Bob: No one else has Bob's private key, and therefore no one else could have created a ciphertext that could be decrypted with Bob's public key.

A general-purpose public-key cryptographic algorithm relies on one key for encryption and a different but related key for decryption. Furthermore, these algorithms have the following important characteristics:

- It is computationally infeasible to determine the decryption key given only knowledge of the cryptographic algorithm and the encryption key.
- Either of the two related keys can be used for encryption, with the other used for decryption.




(a) Encryption with public key




(b) Encryption with private key

**Figure K.2 Public-Key Cryptography**

The essential steps are the following:

1. Each user generates a pair of keys to be used for the encryption and decryption of messages.
2. Each user places one of the two keys in a public register or other accessible file. This is the public key. The companion key is kept private. As Figure K.2a suggests, each user maintains a collection of public keys obtained from others.

3. If Bob wishes to send a private message to Alice, Bob encrypts the message using Alice's public key.
4. When Alice receives the message, she decrypts it using her private key. No other recipient can decrypt the message because only Alice knows her own private key.

With this approach, all participants have access to public keys, and private keys are generated locally by each participant and therefore need never be distributed. As long as a user protects his or her private key, incoming communication is secure. At any time, a user can change the private key and publish the companion public key to replace the old public key.

The key used in symmetric encryption is typically referred to as a **secret key**. The two keys used for public-key encryption are referred to as the **public key** and the **private key**. Invariably, the private key is kept secret, but it is referred to as a private key rather than a secret key to avoid confusion with symmetric encryption.

### Rivest-Shamir-Adleman (RSA) Algorithm

One of the first public-key schemes was developed in 1977 by Ron Rivest, Adi Shamir, and Len Adleman at MIT. The RSA scheme has since that time reigned supreme as the only widely accepted and implemented approach to public-key encryption. RSA is a cipher in which the plaintext and ciphertext are integers between 0 and $n - 1$ for some $n$. Encryption involves modular arithmetic. The strength of the algorithm is based on the difficulty of factoring numbers into their prime factors.

## K.3 MESSAGE AUTHENTICATION AND HASH FUNCTIONS

Encryption protects against passive attack (eavesdropping). A different requirement is to protect against active attack (falsification of data and transactions). Protection against such attacks is known as message authentication.

A message, file, document, or other collection of data is said to be authentic when it is genuine and came from its alleged source. Message authentication is a procedure that allows communicating parties to verify that received messages are authentic. The two important aspects are to verify that the contents of the message have not been altered, and that the source is authentic. We may also wish to verify a message's timeliness (it has not been artificially delayed and replayed) and sequence relative to other messages flowing between two parties.

### Authentication Using Symmetric Encryption

It is possible to perform authentication simply by the use of symmetric encryption. If we assume only the sender and receiver share a key (which is as it should be), then only the genuine sender would be able successfully to encrypt a message for the other participant. Furthermore, if the message includes an error-detection code and a sequence number, the receiver is assured no alterations have been made and sequencing is proper. If the message also includes a timestamp, the receiver is assured that the message has not been delayed beyond that normally expected for network transit.

## Message Authentication without Message Encryption

In this section, we examine several approaches to message authentication that do not rely on message encryption. In all of these approaches, an authentication tag is generated and appended to each message for transmission. The message itself is not encrypted and can be read at the destination independent of the authentication function at the destination.

Because the approaches discussed in this section do not encrypt the message, message confidentiality is not provided. Because symmetric encryption will provide authentication, and because it is widely used with readily available products, why not simply use such an approach, which provides both confidentiality and authentication? Here are three situations in which message authentication without confidentiality is preferable:

1. There are a number of applications in which the same message is broadcast to a number of destinations. An example is the notification to users that the network is now unavailable or an alarm signal in a control center. It is cheaper and more reliable to have only one destination responsible for monitoring authenticity. Thus, the message must be broadcast in plaintext with an associated message authentication tag. The responsible system performs authentication. If a violation occurs, the other destination systems are alerted by a general alarm.
2. Another possible scenario is an exchange in which one side has a heavy load and cannot afford the time to decrypt all incoming messages. Authentication is carried out on a selective basis, with messages chosen at random for checking.
3. Authentication of a computer program in plaintext is an attractive service. The computer program can be executed without having to decrypt it every time, which would be wasteful of processor resources. However, if a message authentication tag were attached to the program, it could be checked whenever assurance is required of the integrity of the program.

Thus, there is a place for both authentication and encryption in meeting security requirements.

## Message Authentication Code

One authentication technique involves the use of a secret key to generate a small block of data, known as a message authentication code, that is appended to the message. This technique assumes that two communicating parties, say A and B, share a common secret key $K_{AB}$. When A has a message $M$ to send to B, it calculates the message authentication code as a function of the message and the key: $MAC_M = F(K_{AB}, M)$. The message plus code are transmitted to the intended recipient. The recipient performs the same calculation on the received message, using the same secret key, to generate a new message authentication code. The received code is compared to the calculated code (see Figure K.3). If we assume only the receiver

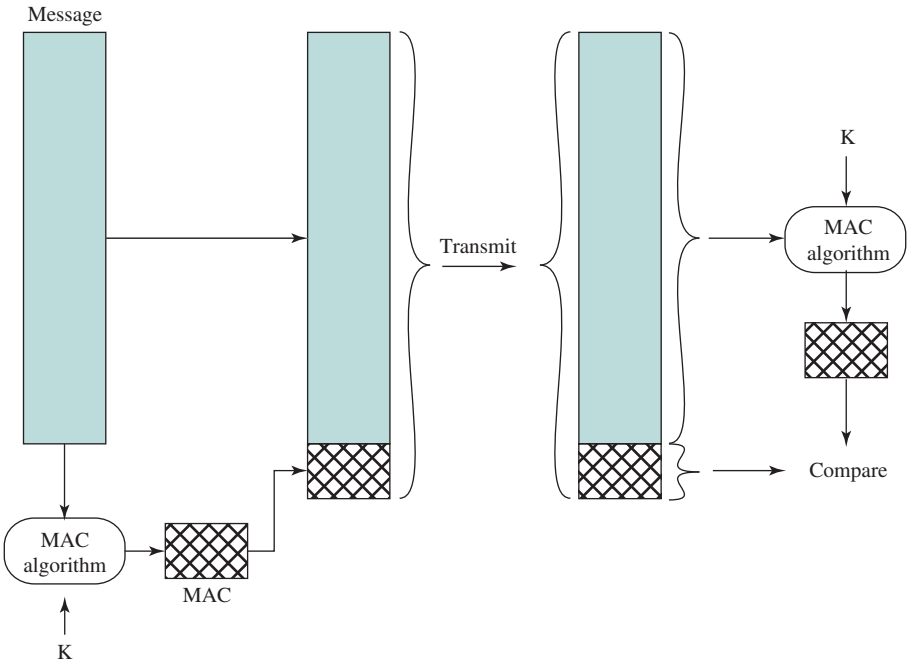



**Figure K.3 Message Authentication Using a Message Authentication Code (MAC)**

and the sender know the identity of the secret key, and if the received code matches the calculated code, then:

1. The receiver is assured the message has not been altered. If an attacker alters the message but does not alter the code, then the receiver's calculation of the code will differ from the received code. Because the attacker is assumed not to know the secret key, the attacker cannot alter the code to correspond to the alterations in the message.
2. The receiver is assured the message is from the alleged sender. Because no one else knows the secret key, no one else could prepare a message with a proper code.
3. If the message includes a sequence number (such as is used with X.25, HDLC, and TCP), then the receiver can be assured of the proper sequence, because an attacker cannot successfully alter the sequence number.

A number of algorithms could be used to generate the code. The National Bureau of Standards, in its publication *DES Modes of Operation*, recommends the use of DES. DES is used to generate an encrypted version of the message, and the last number of bits of ciphertext are used as the code. A 16- or 32-bit code is typical.

The process just described is similar to encryption. One difference is that the authentication algorithm need not be reversible, as it must for decryption. It turns out that because of the mathematical properties of the authentication function, it is less vulnerable to being broken than encryption.

## One-Way Hash Function

A variation on the message authentication code that has received much attention is the one-way hash function. As with the message authentication code, a hash function accepts a variable-size message $M$ as input and produces a fixed-size message digest $H(M)$ as output. Unlike the MAC, a hash function does not also take a secret key as input. To authenticate a message, the message digest is sent with the message in such a way that the message digest is authentic.

Figure K.4 illustrates three ways in which the message can be authenticated. The message digest can be encrypted using symmetric encryption (part a); if it is assumed only the sender and receiver share the encryption key, then authenticity is assured. The message digest can also be encrypted using public-key encryption (part b). The public-key approach has two advantages: it provides a digital signature as well as message authentication, and it does not require the distribution of keys to communicating parties.

These two approaches have an advantage over approaches that encrypt the entire message in that less computation is required. Nevertheless, there has been interest in developing a technique that avoids encryption altogether. Several reasons for this interest are as follows:

- Encryption software is somewhat slow. Even though the amount of data to be encrypted per message is small, there may be a steady stream of messages into and out of a system.
- Encryption hardware costs are nonnegligible. Low-cost chip implementations of DES are available, but the cost adds up if all nodes in a network must have this capability.
- Encryption hardware is optimized toward large data sizes. For small blocks of data, a high proportion of the time is spent in initialization/invocation overhead.
- Encryption algorithms may be covered by patents and must be licensed, adding a cost.
- Encryption algorithms may be subject to export control.

Figure K.4c shows a technique that uses a hash function but no encryption for message authentication. This technique assumes that two communicating parties, say A and B, share a common secret value $S_{AB}$. When A has a message to send to B, it calculates the hash function over the concatenation of the secret value and the message: $MD_M = H(S_{AB}||M)$.[1] It then sends $[M||MD_M]$ to B. Because B possesses $S_{AB}$, it can recompute $H(S_{AB}||M)$ and verify $MD_M$. Because the secret value itself is not sent, it is not possible for an attacker to modify an intercepted message. As long as the secret value remains secret, it is also not possible for an attacker to generate a false message.

This third technique, using a shared secret value, is the one adopted for IP security; it has also been specified for SNMPv3.

[1] || denotes concatenation.

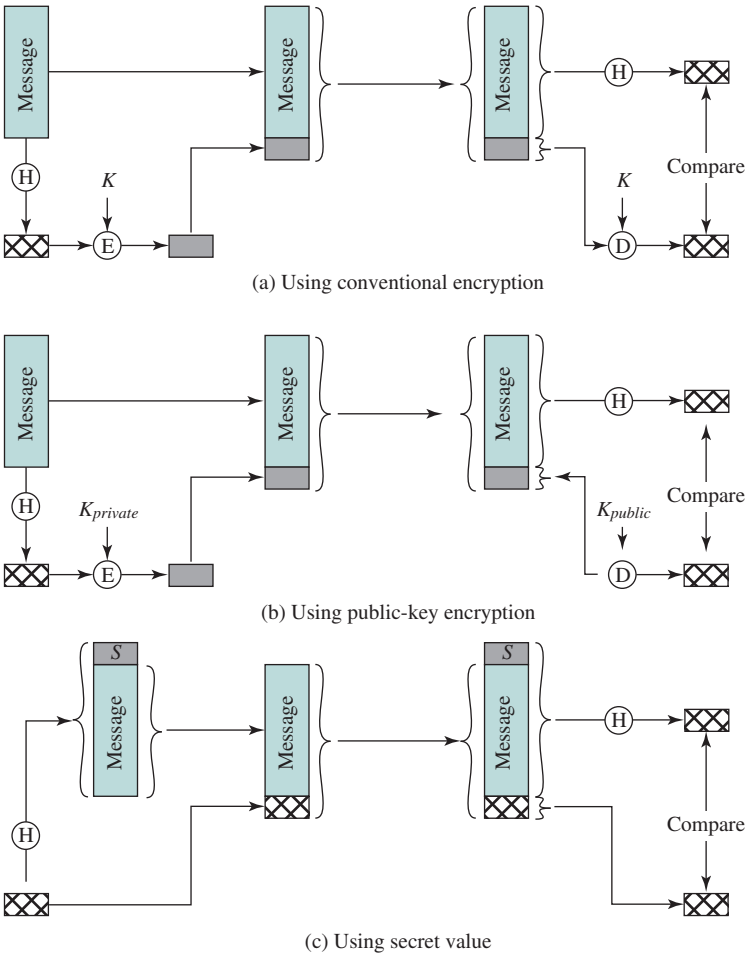



**Figure K.4 Message Authentication Using a One-Way Hash Function**

## K.4 SECURE HASH FUNCTIONS

An essential element of many security services and applications is a secure hash function. A hash function accepts a variable-size message $M$ as input and produces a fixed-size tag H($M$), sometimes called a message digest, as output. For a digital signature, a hash code is generated for a message, encrypted with the sender's private key, and sent with the message. The receiver computes a new hash code for the incoming message, decrypts the hash code with the sender's public key and compares. If the message has been altered in transit, there will be a mismatch.

To be useful for security applications, a hash function H must have the following properties:

1. H can be applied to a block of data of any size.
2. H produces a fixed-length output.
3. $H(x)$ is relatively easy to compute for any given $x$, making both hardware and software implementations practical.
4. For any given value $h$, it is computationally infeasible to find $x$ such that $H(x) = h$. This is sometimes referred to in the literature as the **one-way property**.
5. For any given block $x$, it is computationally infeasible to find $y \neq x$ such that $H(y) = H(x)$. This is sometimes referred to as **weak collision resistance**.
6. It is computationally infeasible to find any pair $(x, y)$ such that $H(x) = H(y)$. This is sometimes referred to as **strong collision resistance**.

In recent years, the most widely used hash function has been the Secure Hash Algorithm (SHA). SHA was developed by the NIST and published as a federal information processing standard (FIPS 180) in 1993. When weaknesses were discovered in SHA, a revised version was issued as FIPS 180-1 in 1995 and is generally referred to as SHA-1. SHA-1 produces a hash value of 160 bits. In 2002, NIST produced a revised version of the standard, FIPS 180-2, that defined three new versions of SHA, with hash value lengths of 256, 384, and 512 bits, known as SHA-256, SHA-384, and SHA-512. These new versions have the same underlying structure and use the same types of modular arithmetic and logical binary operations as SHA-1. In 2005, NIST announced the intention to phase out approval of SHA-1 and move to a reliance on the other SHA versions by 2010. Researchers have demonstrated that SHA-1 is far weaker than its 160-bit hash length suggests, necessitating the move to the newer versions of SHA.

# APPENDIX L

## STANDARDS ORGANIZATIONS


**L.1 The Importance of Standards**

**L.2 Standards and Regulation**

**L.3 Standards-Setting Organizations**

Internet Standards and the Internet Society
The International Telecommunication Union
IEEE 802 Committee
The International Organization for Standardization

An important concept that recurs frequently in this book is standards. This appendix provides some background on the nature and relevance of standards and looks at the key organizations involved in developing standards for networking and communications.

## L.1 THE IMPORTANCE OF STANDARDS

It has long been accepted in the telecommunications industry that standards are required to govern the physical, electrical, and procedural characteristics of communication equipment. In the past, this view has not been embraced by the computer industry. Whereas communication equipment vendors recognize that their equipment will generally interface to and communicate with other vendors' equipment, computer vendors have traditionally attempted to monopolize their customers. The proliferation of computers and distributed processing has made that an untenable position. Computers from different vendors must communicate with each other and, with the ongoing evolution of protocol standards, customers will no longer accept special-purpose protocol conversion software development. The result is that standards now permeate all the areas of technology discussed in this book.

There are a number of advantages and disadvantages to the standards-making process. The principal advantages of standards are:

- A standard assures there will be a large market for a particular piece of equipment or software. This encourages mass production and, in some cases, the use of large-scale-integration (LSI) or very-large-scale-integration (VLSI) techniques, resulting in lower costs.
- A standard allows products from multiple vendors to communicate, giving the purchaser more flexibility in equipment selection and use.

The principal disadvantages of standards are:

- A standard tends to freeze the technology. By the time a standard is developed, subjected to review and compromise, and promulgated, more efficient techniques are possible.
- There are multiple standards for the same thing. This is not a disadvantage of standards per se, but of the current way things are done. Fortunately, in recent years the various standards-making organizations have begun to cooperate more closely. Nevertheless, there are still areas where multiple conflicting standards exist.

## L.2 STANDARDS AND REGULATION

It is helpful for the reader to distinguish three concepts:

- Voluntary standards
- Regulatory standards
- Regulatory use of voluntary standards

Voluntary standards are developed by standards-making organizations, such as those described in the next section. They are voluntary in that the existence of the standard does not compel its use. That is, manufacturers voluntarily implement a product that conforms to a standard if they perceive a benefit to themselves; there is no legal requirement to conform. These standards are also voluntary in the sense that they are developed by volunteers who are not paid for their efforts by the standards-making organization that administers the process. These volunteers are generally employees of interested organizations, such as manufacturers and government agencies.

Voluntary standards work because they are generally developed on the basis of broad consensus, and because the customer demand for standard products encourages the implementation of these standards by the vendors.

In contrast, a regulatory standard is developed by a government regulatory agency to meet some public objective, such as economic, health, and safety objectives. These standards have the force of regulation behind them and must be met by providers in the context in which the regulations apply. Familiar examples of regulatory standards are in areas such as fire codes and health codes. But regulations can apply to a wide variety of products, including those related to computers and communications. For example, the Federal Communications Commission regulates electromagnetic emissions.

A relatively new, or at least newly prevalent, phenomenon is the regulatory use of voluntary standards. A typical example of this is a regulation that requires that the government purchase of a product be limited to those that conform to some referenced set of voluntary standards. This approach has a number of benefits:

- It reduces the rule-making burden on government agencies.
- It encourages cooperation between government and standards organizations to produce standards of broad applicability.
- It reduces the variety of standards that providers must meet.

## L.3 STANDARDS-SETTING ORGANIZATIONS

Various organizations have been involved in the development of standards related to data and computer communications. The remainder of this document provides an overview of some of the most important of these organizations:

- Internet Society
- ITU
- IEEE 802
- ISO

### Internet Standards and the Internet Society

Many of the protocols that make up the TCP/IP protocol suite have been standardized or are in the process of standardization. By universal agreement, an organization known as the Internet Society is responsible for the development and publication of these standards. The Internet Society is a professional membership organization that

oversees a number of boards and task forces involved in Internet development and standardization.

This section provides a brief description of the way in which standards for the TCP/IP protocol suite are developed.

***The Internet Organizations and RFC Publication*** The Internet Society is the coordinating committee for Internet design, engineering, and management. Areas covered include the operation of the Internet itself and the standardization of protocols used by end systems on the Internet for interoperability. Three organizations under the Internet Society are responsible for the actual work of standards development and publication:

- **Internet Architecture Board (IAB):** Responsible for defining the overall architecture of the Internet, providing guidance and broad direction to the IETF
- **Internet Engineering Task Force (IETF):** The protocol engineering and development arm of the Internet
- **Internet Engineering Steering Group (IESG):** Responsible for technical management of IETF activities and the Internet standards process

Working groups chartered by the IETF carry out the actual development of new standards and protocols for the Internet. Membership in a working group is voluntary; any interested party may participate. During the development of a specification, a working group will make a draft version of the document available as an Internet Draft, which is placed in the IETF's "Internet Drafts" online directory. The document may remain as an Internet Draft for up to six months, and interested parties may review and comment on the draft. During that time, the IESG may approve publication of the draft as an RFC (Request for Comment). If the draft has not progressed to the status of an RFC during the six-month period, it is withdrawn from the directory. The working group may subsequently publish a revised version of the draft.

The IETF is responsible for publishing the RFCs, with approval of the IESG. The RFCs are the working notes of the Internet research and development community. A document in this series may be on essentially any topic related to computer communications and may be anything from a meeting report to the specification of a standard.

The work of the IETF is divided into eight areas, each with an area director and each composed of numerous working groups. Table L.1 shows the IETF areas and their focus.

***The Standardization Process*** The decision of which RFCs become Internet standards is made by the IESG, on the recommendation of the IETF. To become a standard, a specification must meet the following criteria:

- Be stable and well understood
- Be technically competent
- Have multiple, independent, and interoperable implementations with substantial operational experience

**Table L.1** IETF Areas

| IETF Area | Theme | Example Working Groups |
|---|---|---|
| **Applications** | Internet applications | Web-related protocols (HTTP)<br>EDI-Internet integration<br>LDAP |
| **General** | IETF processes and procedures | Policy Framework<br>Process for Organization of Internet Standards |
| **Internet** | Internet infrastructure | IPv6<br>PPP extensions |
| **Operations and management** | Standards and definitions for network operations | SNMPv3<br>Remote Network Monitoring |
| **Real-time applications and infrastructure** | Protocols and applications for real-time requirements | Real-time Transport Protocol (RTP)<br>Session Initiation Protocol (SIP) |
| **Routing** | Protocols and management for routing information | Multicast routing<br>OSPF<br>QoS routing |
| **Security** | Security protocols and technologies | Kerberos<br>IPSec<br>X.509<br>S/MIME<br>TLS |
| **Transport** | Transport layer protocols | Differentiated services<br>IP telephony<br>NFS<br>RSVP |

- Enjoy significant public support
- Be recognizably useful in some or all parts of the Internet

The key difference between these criteria and those used for international standards from ITU is the emphasis here on operational experience.

The left-hand side of Figure L.1 shows the series of steps, called the *standards track*, that a specification goes through to become a standard; this process is defined in RFC 2026. The steps involve increasing amounts of scrutiny and testing. At each step, the IETF must make a recommendation for advancement of the protocol, and the IESG must ratify it. The process begins when the IESG approves the publication of an Internet Draft document as an RFC with the status of Proposed Standard.

The white boxes in the diagram represent temporary states, which should be occupied for the minimum practical time. However, a document must remain a Proposed Standard for at least six months and a Draft Standard for at least four months to allow time for review and comment. The shaded boxes represent long-term states that may be occupied for years.

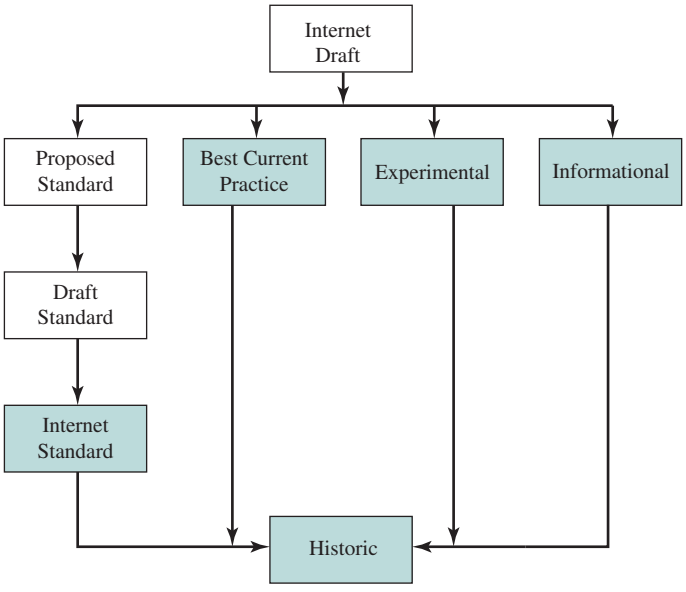



**Figure L.1 Internet RFC Publication Process**

For a specification to be advanced to Draft Standard status, there must be at least two independent and interoperable implementations from which adequate operational experience has been obtained.

After significant implementation and operational experience has been obtained, a specification may be elevated to Internet Standard. At this point, the Specification is assigned an STD number as well as an RFC number.

Finally, when a protocol becomes obsolete, it is assigned to the Historic state.

***INTERNET STANDARDS CATEGORIES*** All Internet standards fall into one of the two categories:

- **Technical specification (TS):** A TS defines a protocol, service, procedure, convention, or format. The bulk of the Internet standards are TSs.
- **Applicability statement (AS):** An AS specifies how, and under what circumstances, one or more TSs may be applied to support a particular Internet capability. An AS identifies one or more TSs that are relevant to the capability, and may specify values or ranges for particular parameters associated with a TS or functional subsets of a TS that are relevant for the capability.

***OTHER RFC TYPES*** There are numerous RFCs that are not destined to become Internet standards. Some RFCs standardize the results of community deliberations about statements of principle or conclusions about what is the best way to perform some operations or IETF process function. Such RFCs are designated as Best Current Practice (BCP). Approval of BCPs follows essentially the same process for approval of Proposed Standards. Unlike standards-track documents, there is not a

three-stage process for BCPs; a BCP goes from Internet draft status to approved BCP in one step.

A protocol or other specification that is not considered ready for standardization may be published as an Experimental RFC. After further work, the specification may be resubmitted. If the specification is generally stable, has resolved known design choices, is believed to be well understood, has received significant community review, and appears to enjoy enough community interest to be considered valuable, then the RFC will be designated a Proposed Standard.

Finally, an Informational Specification is published for the general information of the Internet community.

## The International Telecommunication Union

The International Telecommunication Union (ITU) is a United Nations specialized agency. Hence the members of ITU-T are governments. The U.S. representation is housed in the Department of State. The charter of the ITU is that it "is responsible for studying technical, operating, and tariff questions and issuing Recommendations on them with a view to standardizing telecommunications on a worldwide basis." Its primary objective is to standardize, to the extent necessary, techniques and operations in telecommunications to achieve end-to-end compatibility of international telecommunication connections, regardless of the countries of origin and destination.

***ITU Radio Communication Sector*** The ITU Radiocommunication (ITU-R) Sector was created on March 1, 1993 and comprises the former CCIR and IFRB (founded 1927 and 1947, respectively). ITU-R is responsible for all ITU's work in the field of radio communications. The main activities of ITU-R are:

- Develop draft ITU-R Recommendations on the technical characteristics of, and operational procedures for, radiocommunication services and systems.
- Compile Handbooks on spectrum management and emerging radiocommunication services and systems.

ITU-R is organized into the following study groups:

- SG 1 Spectrum management
- SG 3 Radiowave propagation
- SG 4 Fixed-satellite service
- SG 6 Broadcasting service (terrestrial and satellite)
- SG 7 Science services
- SG 8 Mobile, radiodetermination, amateur and related satellite services
- SG 9 Fixed service
- SC Special Committee on Regulatory/Procedural Matters
- CCV Coordination Committee for Vocabulary
- CPM Conference Preparatory Meeting

***ITU TELECOMMUNICATION STANDARDIZATION SECTOR*** The ITU-T was created on March 1, 1993 as one consequence of a reform process within the ITU. It replaces the International Telegraph and Telephone Consultative Committee (CCITT), which had essentially the same charter and objectives as the new ITU-T. The ITU-T fulfills the purposes of the ITU relating to telecommunications standardization by studying technical, operating and tariff questions, and adopting Recommendations on them with a view to standardizing telecommunications on a worldwide basis.

ITU-T is organized into 13 study groups that prepare Recommendations, numbered as follows:

1. Network and service operation
2. Tariff and accounting principles
3. Telecommunications management network and network maintenance
4. Protection against electromagnetic environment effects
5. Outside plant
6. Integrated broadband cable networks and television and sound transmission
7. Signaling requirements and protocols
8. Performance and quality of service
9. Next generation networks
10. Optical and other transport networks infrastructures
11. Multimedia terminals, systems, and applications
12. Security, languages, and telecommunication software
13. Mobile telecommunications networks

***SCHEDULE*** Work within ITU-R and ITU-T is conducted in four-year cycles. Every four years, a World Telecommunications Standardization Conference is held. The work program for the next four years is established at the assembly in the form of questions submitted by the various study groups, based on requests made to the study groups by their members. The conference assesses the questions, reviews the scope of the study groups, creates new or abolishes existing study groups, and allocates questions to them.

Based on these questions, each study group prepares draft Recommendations. A draft Recommendation may be submitted to the next conference, four years hence, for approval. Increasingly, however, Recommendations are approved when they are ready, without having to wait for the end of the four-year study period. This accelerated procedure was adopted after the study period that ended in 1988. Thus, 1988 was the last time that a large batch of documents was published at one time as a set of Recommendations.

## IEEE 802 Committee

The key to the development of the LAN market is the availability of a low-cost interface. The cost to connect equipment to a LAN must be much less than the cost of the equipment alone. This requirement, plus the complexity of the LAN logic, dictates a

solution based on the use of chips and very-large-scale integration (VLSI). However, chip manufacturers will be reluctant to commit the necessary resources unless there is a high-volume market. A widely accepted LAN standard assures volume and also enables equipment from a variety of manufacturers to intercommunicate. This is the rationale of the IEEE 802 committee.

The committee issued a set of standards, which were adopted in 1985 by the American National Standards Institute (ANSI) as American National Standards. The standards were subsequently revised and reissued as international standards by the International Organization for Standardization (ISO) in 1987, with the designation ISO 8802. Since then, the IEEE 802 committee has continued to revise and extend the standards, which are ultimately then adopted by ISO.

The committee quickly reached two conclusions. First, the task of communication across the local network is sufficiently complex that it needs to be broken up into more manageable subtasks. Accordingly, the standards are organized as a three-layer protocol hierarchy: Logical Link Control (LLC), medium access control (MAC), and physical.

Second, no single technical approach will satisfy all requirements. The second conclusion was reluctantly reached when it became apparent that no single standard would satisfy all committee participants. There was support for various topologies, access methods, and transmission media. The response of the committee was to standardize all serious proposals rather than to attempt to settle on just one. The current state of standardization is reflected by the various working groups in IEEE 802 and the work that each is doing (see Table L.2).

## The International Organization for Standardization

The International Organization for Standardization, or ISO,[1] is an international agency for the development of standards on a wide range of subjects. It is a voluntary, nontreaty organization whose members are designated standards bodies of participating nations, plus nonvoting observer organizations. Although ISO is not a governmental body, more than 70% of ISO member bodies are governmental standards institutions or organizations incorporated by public law. Most of the remainder have close links with the public administrations in their own countries. The U.S. member body is the American National Standards Institute.

ISO was founded in 1946 and has issued more than 12,000 standards in a broad range of areas. Its purpose is to promote the development of standardization and related activities to facilitate international exchange of goods and services and to develop cooperation in the sphere of intellectual, scientific, technological, and economic activity. Standards have been issued to cover everything from screw threads to solar energy. One important area of standardization deals with the Open Systems Interconnection (OSI) communications architecture and the standards at each layer of the OSI architecture.

[1] ISO is not an acronym (in which case it would be IOS), but a word, derived from the Greek *isos*, meaning "equal."

**Table L.2** IEEE 802 Active Working Groups

| Number | Name | Charter |
|---|---|---|
| 802.1 | Higher Layer LAN Protocols | Standards and recommended practices for: 802 LAN/MAN architecture, Internet working among 802 LANs, MANs, and other wide area networks, 802 overall network management, and protocol layers above the MAC and LLC layers |
| 802.3 | Ethernet | Standards for CSMA/CD (Ethernet) based LANs |
| 802.11 | Wireless LAN | Standards for wireless LANs |
| 802.15 | Wireless Personal Area Networks | Personal area network standards for short distance wireless networks |
| 802.16 | Broadband Wireless Access | Standards for broadband wireless access |
| 802.17 | Resilient Packet Ring | Standards for RPR LAN/MAN for rates up to many gigabits per second |
| 802.18 | Radio Regulatory TAG | Monitor regulations that may affect 802.11, 802.15, and 802.16 |
| 802.19 | Coexistence TAG | Standards for coexistence between wireless standards of unlicensed devices. |
| 802.20 | Mobile Broadband Wireless Access | Standards for mobile broadband wireless access |
| 802.21 | Media Independent Handoff | Standards to enable handover and interoperability between heterogeneous network types including both 802 and non-802 networks |
| 802.22 | Wireless Regional Area Networks | Standards for regional wireless networks using unused frequencies in the broadcast television band |
| 82.23 | Emergency Services | Media independent framework to provide consistent access and data that facilitate compliance to applicable civil authority requirements for communications systems that include IEEE 802 networks. |

In the areas of data communications and networking, ISO standards are actually developed in a joint effort with another standards body, the International Electrotechnical Commission (IEC). IEC is primarily concerned with electrical and electronic engineering standards. In the area of information technology, the interests of the two groups overlap, with IEC emphasizing hardware and ISO focusing on software. In 1987, the two groups formed the Joint Technical Committee 1 (JTC 1). This committee has the responsibility of developing the documents that ultimately become ISO (and IEC) standards in the area of information technology.

The development of an ISO standard from first proposal to actual publication of the standard follows a six-step process. The objective is to ensure the final result is acceptable to as many countries as possible. Briefly, the steps are:

1. **Proposal stage:** A new work item is assigned to the appropriate technical committee, and within that technical committee, to the appropriate working group.
2. **Preparatory stage:** The working group prepares a working draft. Successive working drafts may be considered until the working group is satisfied that it has

developed the best technical solution to the problem being addressed. At this stage, the draft is forwarded to the working group's parent committee for the consensus-building phase.

3. **Committee stage:** As soon as a first committee draft is available, it is registered by the ISO Central Secretariat. It is distributed among interested members for balloting and technical comment. Successive committee drafts may be considered until consensus is reached on the technical content. Once consensus has been attained, the text is finalized for submission as a Draft International Standard (DIS).
4. **Enquiry stage:** The DIS is circulated to all ISO member bodies by the ISO Central Secretariat for voting and commenting within a period of five months. It is approved for submission as a Final Draft International Standard (FDIS) if a two-thirds majority is in favor and not more than one-quarter of the total number of votes cast are negative. If the approval criteria are not met, the text is returned to the originating working group for further study, and a revised document will again be circulated for voting and comment as a DIS.
5. **Approval stage:** The Final Draft International Standard (FDIS) is circulated to all ISO member bodies by the ISO Central Secretariat for a final yes/no vote within a period of two months. If technical comments are received during this period, they are no longer considered at this stage, but registered for consideration during a future revision of the International Standard. The text is approved as an International Standard if a two-thirds majority is in favor and not more than one-quarter of the total number of votes cast are negative. If these approval criteria are not met, the standard is referred back to the originating working group for reconsideration in the light of the technical reasons submitted in support of the negative votes received.
6. **Publication stage:** Once a FDIS has been approved, only minor editorial changes, if and where necessary, are introduced into the final text. The final text is sent to the ISO Central Secretariat, which publishes the International Standard.

The process of issuing an ISO standard can be a slow one. Certainly, it would be desirable to issue standards as quickly as the technical details can be worked out, but ISO must ensure the standard will receive widespread support.

# APPENDIX M

## SOCKETS: A PROGRAMMER'S INTRODUCTION


**M.1 Sockets, Socket Descriptors, Ports, and Connections**

**M.2 The Client/Server Model of Communication**
- Running a Sockets Program on a Windows Machine Not Connected to a Network
- Running a Sockets Program on a Windows Machine Connected to a Network, When Both Server and Client Reside on the Same Machine

**M.3 Sockets Elements**
- Socket Creation
- The Socket Address
- Bind to a Local Port
- Data Representation and Byte Ordering
- Connecting a Socket
- The `gethostbyname()` Function Call
- Listening for an Incoming Client Connection
- Accepting a Connection from a Client
- Sending and Receiving Messages on a Socket
- Closing a Socket
- `Report errors`
- Example TCP/IP Client Program (Initiating Connection)
- Example TCP/IP Server Program (Passively Awaiting Connection)

**M.4 Stream and Datagram Sockets**
- Example UDP Client Program (Initiate Connections)
- Example UDP Server Program (Passively Await Connection)

**M.5 Run-Time Program Control**
- Nonblocking Socket Calls
- Asynchronous I/O (Signal Driven I/O)

**M.6 Remote Execution of a Windows Console Application**
- Local Code
- Remote Code

The concept of sockets and sockets programming was developed in the 1980s in the Unix environment as the Berkeley Sockets Interface. In essence, a socket enables communications between a client and server process and may be either connection-oriented or connectionless. A socket can be considered an endpoint in a communication. A client socket in one computer uses an address to call a server socket on another computer. Once the appropriate sockets are engaged, the two computers can exchange data.

Typically, computers with server sockets keep a TCP or UDP port open, ready for unscheduled incoming calls. The client typically determines the socket identification of the desired server by finding it in a Domain Name System (DNS) database. Once a connection is made, the server switches the dialogue to a different port number to free up the main port number for additional incoming calls.

Internet applications, such as TELNET and remote login (rlogin) make use of sockets, with the details hidden from the user. However, sockets can be constructed from within a program (in a language such as C or Java), enabling the programmer to easily support networking functions and applications. The sockets programming mechanism includes sufficient semantics to permit unrelated processes on different hosts to communicate.

The Berkeley Sockets Interface is the de facto standard application programming interface (API) for developing networking applications, spanning a wide range of operating systems. The sockets API provides generic access to interprocess communications services. Thus, the sockets capability is ideally suited for students to learn the principles of protocols and distributed applications by hands-on program development.

The Sockets Application Program Interface (API) provides a library of functions that programmers can use to develop network aware applications. It has the functionality of identifying endpoints of the connection, establishing the communication, allowing messages to be sent, waiting for incoming messages, terminating the communication, and error handling. The operating system used and the programming language both determine the specific Sockets API.

We concentrate on only two of the most widely used interfaces—the Berkley Software Distribution Sockets (BSD) as introduced for UNIX, and its slight modification the Windows Sockets (WinSock) API from Microsoft.

This sockets material is intended for the C language programmer. (It provides external references for the C++, Visual Basic, and PASCAL languages.) The Windows operating system is in the center of our discussion. At the same time, topics from the original BSD UNIX specification are introduced in order to point out (usually minor) differences in the sockets specifications for the two operating systems. Basic knowledge of the TCP/IP and UDP network protocols is assumed. Most of the code would compile on both Windows and UNIX-like systems.

We cover C language sockets exclusively, but most other programming languages, such as C++, Visual Basic, and PASCAL, can take advantage of the Winsock API, as well. The only requirement is that the language has to recognize dynamic link libraries (DLLs). In a 32-bit Windows environment, you will need to import the `wsock32.lib` to take advantage of the WinSock API. This library has to be linked, so at run time the dynamic link library `wsock32.dll` gets loaded. `wsock32.dll` runs over the TCP/IP stack. Windows NT, Windows 2000, and Windows 95 include

the file `wsock32.dll` by default. When you create your executables, if you link with `wsock32.lib` library, you will implicitly link the `wsock32.dll` at run time, without adding lines of code to your source file.

The website for this book provides links to useful Sockets websites.

## M.1 SOCKETS, SOCKET DESCRIPTORS, PORTS, AND CONNECTIONS

Sockets are endpoints of communication referred to by their corresponding socket descriptors, or natural language words describing the socket's association with a particular machine or application (e.g., we will refer to a server socket as `server_s`). A connection (or socket pair) consists of the pair of IP addresses that are communicating with each other, as well a pair of port numbers, where a port number is a 32-bit positive integer usually denoted in its decimal form. Some destination port numbers are well known and indicate the type of service being connected to.

For many applications, the TCP/IP environment expects that applications use well-known ports to communicate with each other. This is done so client applications assume the corresponding server application is listening on the well-known port associated with that application. For example, the port number for HTTP, the protocol used to transfer HTML pages across the World Wide Web, is TCP port 80. By default, a Web browser will attempt to open a connection on the destination host's TCP port 80 unless another port number is specified in the URL (such as 8000 or 8080).

A *port* identifies a connection point in the local stack (i.e., port number 80 is typically used by a Web server). A *socket* identifies an IP address and port number pair (i.e., port 192.168.1.20:80 would be the Web server port 80 on host 192.168.1.20. The two together are considered a socket.). A *socket pair* identifies all four components (source address and port, and destination address and port). Since well-known ports are unique, they are sometimes used to refer to a specific application on any host that might be running the application. Using the word socket, however, would imply a specific application on some specific host. Connection, or a *socket pair*, stands for the sockets connection between two specific systems that are communicating. TCP allows multiple simultaneous connections involving the same local port number as long as the remote IP addresses or port numbers are different for each connection.

Port numbers are divided into three ranges:

- Ports 0 through 1023 are well known. They are associated with services in a static manner. For example, HTTP servers would always accept requests at port 80.
- Port numbers from 1024 through 49151 are registered. They are used for multiple purposes.
- Dynamic and private ports are those from 49152 through 65535 and services should not be associated with them.

In reality, machines start assigning dynamic ports starting at 1024. If you are developing a protocol or application that will require the use of a link, socket, port,

| Proto | Local Address | Foreign Address | State |
|---|---|---|---|
| TCP | Mycomp:1025 | Mycomp:0 | LISTENING |
| TCP | Mycomp:1026 | Mycomp:0 | LISTENING |
| TCP | Mycomp:6666 | Mycomp:0 | LISTENING |
| TCP | Mycomp:6667 | Mycomp:0 | LISTENING |
| TCP | Mycomp:1234 | mycomp:1234 | TIME_WAIT |
| TCP | Mycomp:1025 | 2hfc327.any.com:6667 | ESTABLISHED |
| TCP | Mycomp:1026 | 46c311.any.com:6668 | ESTABLISHED |
| UDP | Mycomp:6667 | *.* | |

**Figure M.1 Sample Netstat Output**

protocol, etc., please contact the Internet Assigned Numbers Authority (IANA) to receive a port number assignment. The IANA is located at and operated by the Information Sciences Institute (ISI) of the University of Southern California. The Assigned Numbers request for comments (RFC) published by IANA is the official specification that lists port assignments. You can access it at http://www.iana .org/assignments/port-numbers.

On both UNIX and Windows, the *netstat* command can be used to check the status of all active local sockets. Figure M.1 is a sample netstat output.

## M.2 THE CLIENT/SERVER MODEL OF COMMUNICATION

A socket application consists of code, executed on both communication ends. The program initiating transmission is often referred to as the client. The server, on the other hand, is a program that passively awaits incoming connections from remote clients. Server applications typically load during system startup and actively listen for incoming connections on their well-known port. Client applications will then attempt to connect to the server, and a TCP exchange will then take place. When the session is complete, usually the client will be the one to terminate the connection. Figure M.2 depicts the basic model of stream-based (or TCP/IP sockets) communication.

### Running a Sockets Program on a Windows Machine Not Connected to a Network

As long as TCP/IP is installed on one machine, you can execute both the server and client code on it. (If you do not have the TCP/IP protocol stack installed, you can expect socket operations to throw exceptions such as `BindException`, `Connect-Exception`, `ProtocolException`, `SocketException`, etc. ) You will have to use `localhost` as the hostname or `127.0.0.1` as the IP address.

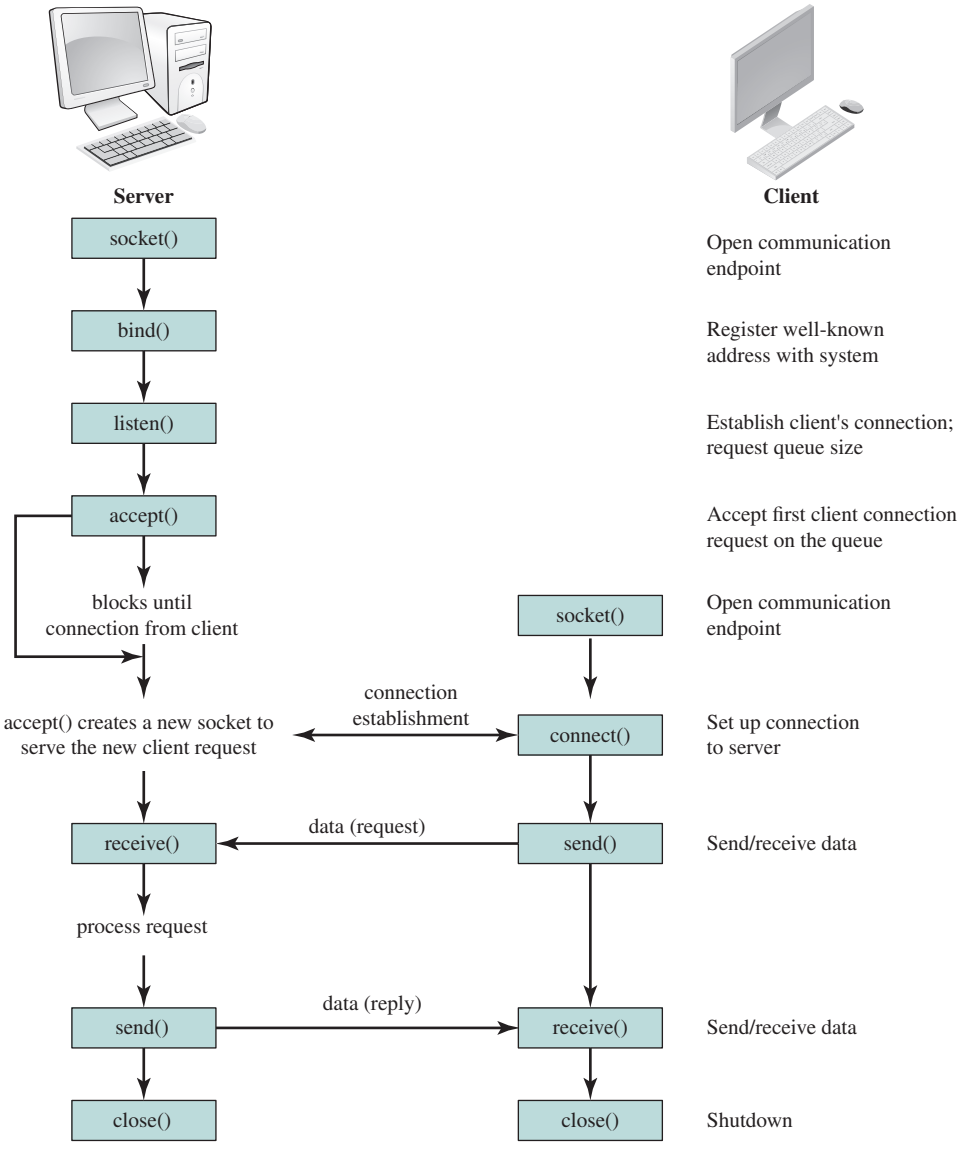



**Figure M.2 Socket System Calls for Connection-Oriented Protocol**

## Running a Sockets Program on a Windows Machine Connected to a Network, When Both Server and Client Reside on the Same Machine

In such a case, you will be communicating with yourself. It is important to know whether your machine is attached to an Ethernet or communicates with the network through a telephone modem. In the first case, you will have an IP address assigned

to your machine, without efforts on your part. When communicating via a modem, you need to dial in, grab an IP address, and then be able to "talk to yourself." In both cases you can find out the IP address of the machine you are using with the `winipcfg` command for Win9X, and `ipconfig` for WinNT/2K and UNIX.

## M.3 SOCKETS ELEMENTS

### Socket Creation

```
#include <sys/types.h>
#include <sys/socket.h>

int socket(int  domain, int  type, int  protocol)
```

- *domain* is `AF_UNIX, AF_INET, AF_OSI`, etc. `AF_INET` is for communication on the Internet to IP addresses. We will only use `AF_INET`.
- *type* is either `SOCK_STREAM` (TCP, connection-oriented, reliable), or `SOCK_DGRAM` (UDP, datagram, unreliable), or `SOCK_RAW` (IP level).
- *protocol* specifies the protocol used. It is usually 0 to say we want to use the default protocol for the chosen domain and type. We always use 0.

If successful, `socket()` returns a socket descriptor, which is an integer, and −1 in the case of a failure. An example call:

```
if ((sd = socket(AF_INET, SOCK_DGRAM, 0) < 0)
  {
      printf(socket() failed.);
   exit(1);
  }
```

### The Socket Address

The structures to store socket addresses as used in the domain **`AF_INET`**:

```
struct in_addr {
  unsigned long s_addr;
};
```

**`in_addr`** just provides a name **`(s_addr)`** for the C language type to be associated with IP addresses.

```
struct sockaddr_in {
 unsigned short   sin_family; // AF_INET identifiers
 unsigned short   sin_port;   // port number,
                              // if 0 then kernel chosen
struct in_addr sin_addr;      // IP address
```

```
                                // INADDR_ANY refers to the IP
                                // addresses of the current host
    char   sin_zero[8];         // Unused, always zero
};
```

Both local and remote addresses will be declared as a **sockaddr_in** structure. Depending on this declaration, **sin_addr** will represent a local or remote IP address. (On a UNIX like system, you need to include the file **<netinet/in.h>** for both structures.)

## Bind to a Local Port

```
#define WIN   // WIN for Winsock and BSD for BSD sockets
#ifdef WIN
 . . .
#include <windows.h> // for all Winsock functions
 . . .
#endif
#ifdef BSD
 . . .
#include <sys/types.h>
#include <sys/socket.h> // for struct sockaddr
 . . .
#endif
int bind(int  local_s, const struct sockaddr *addr,
int  addrlen);
```

- ***local_s*** is a socket descriptor of the local socket, as created by the **socket()** function;
- ***addr*** is a pointer to the (local) address structure of this socket;
- ***addrlen*** is the length (in bytes) of the structure referenced by ***addr***.

`bind()` returns the integer 0 on success, and −1 on failure. After a call to `bind()`, a local port number is associated with the socket, but no remote destination is yet specified.

An example call:

```
struct sockaddr_in name;
...
name.sin_family = AF_INET;    // use the Internet domain
name.sin_port = htons(0);     // kernel provides a port
name.sin_addr.s_addr = htonl(INADDR_ANY); // use all IPs
                                                of host
if (bind(local_socket, (struct sockaddr *)&name,
       sizeof(name)) != 0)
                                // print error and exit
```

A call to bind is optional on the client side, but it is required on the server side. After `bind()` is called on a socket, we can retrieve its address structure, given the socket file descriptor, by using the function `getsockname()`.

## Data Representation and Byte Ordering

Some computers are big endian. This refers to the representation of objects such as integers within a word. A big endian machine stores them in the expected way: the high byte of an integer is stored in the leftmost byte, while the low byte of an integer is stored in the rightmost byte. So the number $5 \times 2^{16} + 6 \times 2^8 + 4$ would be stored as:

| | | | | |
|---|---|---|---|---|
| *Big endian* representation | | 5 | 6 | 4 |
| *Little endian* representation | 4 | 6 | 5 | |
| Memory (byte) address | 0 | 1 | 2 | 3 |

As you can see, reading a value of the wrong word size will result in an incorrect value; when done on big endian architecture, on a little endian machine it can sometimes return the correct result. The big endian ordering is somewhat more natural to humans, because we are used to reading numbers from left to right.

A Sun SPARC is a big endian machine. When it communicates with an i-386 PC (which is a little endian), the following discrepancy will exist: The i-386 will interpret $5 \times 2^{16} + 6 \times 2^8 + 4$ as $4 \times 2^{16} + 6 \times 2^8 + 5$. To avoid this situation from occurring, the TCP/IP protocol defines a machine independent standard for byte order—network byte ordering. In a TCP/IP packet, the first transmitted data is the most significant byte. Because big endian refers to storing the most significant byte in the lowest memory address, which is the address of the data, TCP/IP defines network byte order as big endian.

Winsock uses network byte order for various values. The functions `htonl()`, `htons()`,`ntohl()`,`ntohs()` ensure the proper byte order is being used in Winsock calls, regardless of whether the computer normally uses little endian or big endian ordering.

The following functions are used to convert from host to network ordering before transmission, and from network to host form after reception:

- unsigned long *`htonl`*(unsigned long *`n`*)—host to network conversion of a 32-bit value;
- unsigned short *`htons`*(unsigned short *`n`*)—host to network conversion of a 16-bit value;
- unsigned long *`ntohl`*(unsigned long *`n`*)—network to host conversion of a 32-bit value;
- unsigned short *`ntohs`*(unsigned short *`n`*)—network to host conversion of a 16-bit value.

## Connecting a Socket

A remote process is identified by an IP address and a port number. The `connect()` call evoked on the local site attempts to establish the connection to the remote destination. It is required in the case of connection-oriented communication such as

stream-based sockets (TCP/IP). Sometimes we call `connect()` on datagram sockets, as well. The reason is that this stores the destination address locally, so we do not need to specify the destination address every time when we send datagram message and thus can use the `send()` and `recv()` system calls instead of `sendto()` and `recvfrom()`. Such sockets, however, cannot be used to accept datagrams from other addresses.

```
#define WIN   // WIN for Winsock and BSD for BSD sockets
#ifdef WIN
#include <windows.h> // Needed for all Winsock functions
#endif
#ifdef BSD
#include <sys/types.h> // Needed for system defined
identifiers
#include <netinet/in.h> // Needed for Internet address
structure
#include <sys/socket.h> // Needed for socket(), bind(),
                            etc...
#endif
int  connect(int  local_s, const struct sockaddr
       *remote_addr, int  rmtaddr_len)
```

- ***local_s*** is a local socket descriptor;
- ***remote_addr*** is a pointer to protocol address of other socket;
- ***rmtaddr_len*** is the length in bytes of the address structure.

Returned is an integer 0 (on success). The Windows connect function returns a nonzero value to indicate an error, while the UNIX connection function returns a negative value in such case.

An example call:

```
#define PORT_NUM 1050          // Arbitrary port number
struct sockaddr_in serv_addr; // Server Internet address
int   rmt_s;                  // Remote socket descriptor
         // Fill-in the server (remote) socket's address
            information and connect
         // with the listening server.
server_addr.sin_family = AF_INET; // Address family to use
server_addr.sin_port = htons(PORT_NUM); // Port num to use
server_addr.sin_addr.s_addr
        = inet_addr(inet_ntoa(address));  // IP address
if (connect(rmt_s,(struct sockaddr *)&serv_addr,
        sizeof(serv_addr)) != 0)
         // print error and exit
```

## The gethostbyname() Function Call

The function **gethostbyname()** is supplied a host name argument and returns NULL in case of failure, or a pointer to a **struct hostent** instance on success. It gives information about the host names, aliases, and IP addresses. This information is obtained from the DNS or a local configuration database. The **getservbyname()** will determine the port number associated with a named service. If a numeric value is supplied instead, it is converted directly to binary and used as a port number.

```
#define struct hostent {
  char  *h_name;        // official name of host
  char  **h_aliases;    // null terminated list of alias
                           names
                        // for this host
  int   h_addrtype;     // host address type,
                           e.g. AF_INET
  int   h_length;       // length of address structure
  char  **h_addr_list;  // null terminated list of
                           addresses
                        // in network byte order
};
```

Note `h_addr_list` refers to the IP address associated with the host.

```
#define WIN // WIN for Winsock and BSD for BSD sockets
#ifdef WIN
#include <windows.h> // for all Winsock functions
#endif
#ifdef BSD
#include <netdb.h> // for struct hostent
#endif
struct hostent *gethostbyname (const char *hostname);
```

Other functions that can be used to find hosts, services, protocols, or networks are: `getpeername()`, `gethostbyaddr()`, `getprotobyname()`, `getprotobynumber()`, `getprotoent()`, `getservbyname()`, `getservbyport()`, `getservent()`, `getnetbyname()`, `getnetbynumber()`, `getnetent()`.

An example call:

```
#ifdef BSD
 . . .
#include <sys\types.h> // for caddr_t type
 . . .
#endif
```

```
#define SERV_NAME somehost.somecompany.com
#define PORT_NUM 1050          // Arbitrary port number
#define h_addr h_addr_list[0]  // To hold host Internet
                                  address
 . . .
struct sockaddr_in myhost_addr; // This Internet address
struct hostent *hp;       // buffer information about
remote host
int rmt_s; // Remote socket descriptor
           // UNIX specific part
bzero( (char *)&myhost_addr, sizeof(myhost_addr) );
           // Winsock specific
memset( &myhost_addr, 0, sizeof(myhost_addr) );
           // Fill-in the server (remote) socket's
              address information and connect
           // with the listening server.
myhost_addr.sin_family = AF_INET;    // Address family
                                        to use
myhost_addr.sin_port = htons(PORT_NUM); // Port num to use
if (hp = gethostbyname(MY_NAME)== NULL)
           // print error and exit
           // UNIX specific part
bcopy(hp->h_name, (char *)&myhost_addr.sin_addr,
      hp->h_length );
           // Winsock specific
memcpy( &myhost_addr.sin_addr, hp->h_addr, hp->h_length );
if(connect(rmt_s,(struct sockaddr *)&myhost_addr,
     sizeof(myhost_addr))!=0)
           // print error and exit
```

The UNIX function `bzero()` zeroes out a buffer of specified length. It is one of a group of functions for dealing with arrays of bytes. `bcopy()` copies a specified number of bytes from a source to a target buffer. `bcmp()` compares a specified number of bytes of two byte buffers. The UNIX `bzero()` and `bcopy()` functions are not available in Winsock, so the ANSI functions `memset()` and `memcpy()` have to be used instead.

An example sockets program to get a host IP address for a given host name:

```
#define WIN   // WIN for Winsock and BSD for BSD sockets
#include <stdio.h>    // Needed for printf()
#include <stdlib.h>   // Needed for exit()
#include <string.h>   // Needed for memcpy() and strcpy()
#ifdef WIN
#include <windows.h>  // Needed for all Winsock stuff
#endif
```

```
#ifdef BSD
#include <sys/types.h>      // Needed for system defined
                               identifiers.
#include <netinet/in.h>     // Needed for Internet
                               address structure.
#include <arpa/inet.h>      // Needed for inet_ntoa.
#include <sys/socket.h>     // Needed for socket(),
                               bind(), etc...
#include <fcntl.h>
#include <netdb.h>
#endif
void main(int argc, char *argv[])
{
#ifdef WIN
WORD wVersionRequested = MAKEWORD(1,1);
                            // Stuff for WSA functions
WSADATA wsaData;            // Stuff for WSA functions
#endif
struct hostent *host;   // Structure for gethostbyname()
struct in_addr address; // Structure for Internet
address
char host_name[256];    // String for host name
if (argc != 2)
{
printf(*** ERROR - incorrect number of command line
arguments \n);
printf( usage is 'getaddr host_name' \n);
exit(1);
}
#ifdef WIN
                        // Initialize winsock
WSAStartup(wVersionRequested, &wsaData);
#endif
                        // Copy host name into host_name
strcpy(host_name, argv[1]);
                        // Do a gethostbyname()
printf(Looking for IP address for '%s'... \n,
       host_name);
host = gethostbyname(host_name);
                        // Output address if host found
if (host == NULL)
printf( IP address for '%s' could not be found \n,
        host_name);
else
```

```
{
memcpy(&address, host->h_addr, 4);
printf( IP address for '%s' is %s \n, host_name,
        inet_ntoa(address));
}
#ifdef WIN
                                // Cleanup winsock
WSACleanup();
#endif
}
```

## Listening for an Incoming Client Connection

The `listen()` function is used on the server in the case of connection-oriented communication to prepare a socket to accept messages from clients. It has the prototype:

```
int listen(int  sd, int  qlen);
```

- ***sd*** is a socket descriptor of a socket after a **`bind()`** call
- ***qlen*** specifies the maximum number of incoming connection requests that can wait to be processed by the server while the server is busy.

The call to **`listen()`** returns an integer: 0 on success, and −1 on failure. For example:

```
if (listen(sd, 5) < 0) {
       // print error and exit
```

## Accepting a Connection from a Client

The `accept()` function is used on the server in the case of connection-oriented communication (after a call to `listen()`) to accept a connection request from a client.

```
#define WIN   // WIN for Winsock and BSD for BSD sockets
#ifdef WIN
 . . .
#include <windows.h>        // for all Winsock functions
 . . .
#endif
#ifdef BSD
 . . .
#include <sys/types.h>
#include <sys/socket.h>     // for struct sockaddr
 . . .
#endif
int accept(int  server_s, struct sockaddr *  client_addr,
int *  clntaddr_len)
```

- ***server_s*** is a socket descriptor the server is listening on
- ***client_addr*** will be filled with the client address
- ***clntaddr_len*** contains the length of the client address structure.

The `accept()` function returns an integer representing a new socket (−1 in case of failure).

Once executed, the first queued incoming connection is accepted, and a new socket with the same properties as *sd* is created and returned. It is the socket that the server will use from now on to communicate with this client. Multiple successful calls to `connect()` will result in multiple new sockets returned.

An example call:

```
struct sockaddr_in client_addr;
int server_s, client_s, clntaddr_len;
 ...
if ((client_s = accept(server_s, (struct sockaddr *)&
client_addr, &clntaddr_len) < 0)
                  // print error and exit
                  // at this stage a thread or a process
                     can take over and handle
                 // communication with the client
```

Successive calls to accept on the same listening socket return different connected sockets. These connected sockets are multiplexed on the same port of the server by the running TCP stack functions.

## Sending and Receiving Messages on a Socket

We will present only four function calls in this section. There are, however, more than four ways to send and receive data through sockets. Typical functions for TCP/IP sockets are **`send()`** and **`recv()`**.

```
int send(int  socket, const void  *msg, unsigned
     int  msg_length, int  flags);
int recv(int  socket, void  *rcv_buff, unsigned
     int buff_length, int  flags);
```

- ***socket*** is the local socket used to send and receive.
- ***msg*** is the pointer to a message.
- ***msg_length*** is the message length.
- ***rcv_buff*** is a pointer to the receive buffer.
- ***buff_length*** is its length.
- ***flags*** changes the default behavior of the call.

For example, a particular value of flags will be used to specify that the message is to be sent without using local routing tables (they are used by default).

Typical functions for UDP sockets are:

```
int sendto(int  socket, const void  *msg, unsigned
     int  msg_length, int  flags, struct sockaddr
     *dest_addr, unsigned int addr_length);
int recvfrom(int  socket, void  *rcv_buff, unsigned
     int buff_length, int  flags, struct sockaddr
     *src_addr, unsigned int addr_length);
```

Most parameters are the same as for **`send()`** and **`recv()`**, except ***`dest_addr/src_addr`*** and ***`addr_length`***. Unlike with stream sockets, datagram callers of **`sendto()`** need to be informed of the destination address to send the message to, and callers of **`recvfrom()`** need to distinguish between different sources sending datagram messages to the caller. We provide code for TCP/IP and UDP client and server applications in the following sections, where you can find the sample calls of all four functions.

## Closing a Socket

The prototype:

```
int closesocket(int  sd); // Windows prototype
int close(int  fd); // BSD UNIX prototype
```

*fd* and *sd* are a file descriptor (same as socket descriptor in UNIX) and a socket descriptor.

When a socket on some reliable protocol, such as TCP/IP is closed, the kernel will still retry to send any outstanding data, and the connection enters a `TIME_WAIT` state (see Figure M.1). If an application picks the same port number to connect to, the following situation can occur. When this remote application calls `connect()`, the local application assumes that the existing connection is still active and sees the incoming connection as an attempt to duplicate an existing connection. As a result, `[WSA]ECONNREFUSED` error is returned. The operating system keeps a reference counter for each active socket. A call to `close()` is essentially decrementing this counter on the argument socket. This is important to keep in mind when we are using the same socket in multiple processes. We will provide a couple of example calls in the code segments presented in the next subsections.

### Report errors

All the preceding operations on sockets can exhibit a number of different failures at execution time. It is considered a good programming practice to report the returned error. Most of these errors are designed to assist the developer in the debugging process, and some of them can be displayed to the user, as well. In a Windows environment, all of the returned errors are defined in `winsock.h`. On an UNIX-like system, you can find these definitions in `socket.h`. The Windows codes are

computed by adding 10,000 to the original BSD error number and adding the prefix `WSA` in front of the BSD error name. For example:

| Windows name | BSD name | Windows value | BSD value |
|---|---|---|---|
| `WSAEPROTOTYPE` | `EPROTOTYPE` | 10041 | 41 |

There are a few Windows-specific errors not present in a UNIX system:

| | | |
|---|---|---|
| `WSASYSNOTREADY` | *10091* | Returned by `WSAStartup()` indicating that the network subsystem is unusable. |
| `WSAVERNOTSUPPORTED` | *10092* | Returned by `WSAStartup()` indicating that the Windows Sockets DLL cannot support this app. |
| `WSANOTINITIALISED` | *10093* | Returned by any function except `WSAStartup()`, when a successful `WSAStartup()` has not yet been performed. |

An example error-catching source file, responsible for displaying an error and exiting:

```
#ifdef WIN
#include <stdio.h>      // for fprintf()
#include <winsock.h>    // for WSAGetLastError()
#include <stdlib.h>     // for exit()
#endif
#ifdef BSD
#include <stdio.h>      // for fprintf() and perror()
#include <stdlib.h>     // for exit()
#endif

void catch_error(char * program_msg)
{
char err_descr[128];    // to hold error description
int err;
err = WSAGetLastError();
                        // record the winsock.h error
                           description
if (err == WSANO_DATA)
   strcpy(err_descr, WSANO_DATA (11004) Valid name, no
"       data record of requested type.);
if (err == WSANO_RECOVERY)
   strcpy(err_descr, WSANO_RECOVERY (11003) This is a
      non-recoverable error.);
if (err == WSATRY_AGAIN)
```

```
 . . .
fprintf(stderr,%s: %s\n, program_msg, err_descr);
exit(1);
}
```

You can extend the list of errors to be used in your Winsock application by looking at http://www.sockets.com.

## Example TCP/IP Client Program (Initiating Connection)

This client program is designed to receive a single message from a server (lines 39–41) then terminate itself (lines 45–56). It sends a confirmation to the server after the message is received (lines 42–44).

```
#define WIN   // WIN for Winsock and BSD for BSD sockets
#include      // Needed for printf()
#include      // Needed for memcpy() and strcpy()
#ifdef WIN
#include      // Needed for all Winsock stuff
#endif
#ifdef BSD
#include      // Needed for system defined identifiers.
#include      // Needed for Internet address structure.

#include      // Needed for socket(), bind(), etc...
#include      // Needed for inet_ntoa()
#include
#include
#endif
#define PORT_NUM 1050  // Port number used at the server
#define IP_ADDR 131.247.167.101  // IP address of server
                                    (*** HARDWIRED ***)
void main(void)
{
#ifdef WIN
WORD wVersionRequested = MAKEWORD(1,1); // WSA functions
WSADATA wsaData;       // WSA functions
#endif
unsigned int server_s; // Server socket descriptor
struct sockaddr_in server_addr; // Server Internet address
char out_buf[100];     // 100-byte output buffer for data
char in_buf[100];      // 100-byte input buffer for data
#ifdef WIN             // Initialize Winsock
WSAStartup(wVersionRequested, &wsaData);
#endif
```

```
      // Create a socket
server_s = socket(AF_INET, SOCK_STREAM, 0);
      // Fill-in the server socket's address and do a
         connect with
      // the listening server. The connect() will block.
Server_addr.sin_family = AF_INET;      // Address family
Server_addr.sin_port = htons(PORT_NUM); // Port num
Server_addr.sin_addr.s_addr = inet_addr(IP_ADDR);
                           // IP address
Connect(server_s, (struct sockaddr *)&server_addr,
sizeof(server_addr));
                           // Receive from the server
recv(server_s, in_buf, sizeof(in_buf), 0);
printf(Received from server... data = '%s' \n, in_buf);
                           // Send to the server
strcpy(out_buf, Message -- client to server);
send(server_s, out_buf, (strlen(out_buf) + 1), 0);
                           // Close all open sockets
#ifdef WIN
closesocket(server_s);
#endif
#ifdef BSD
close(server_s);
#endif
#ifdef WIN                // Cleanup winsock
WSACleanup();
#endif
}
```

## Example TCP/IP Server Program (Passively Awaiting Connection)

All the following server program does is serving a message to a client running on another host. It creates one socket in line 37 and listens for a single incoming service request from the client through this single socket. When the request is satisfied, this server terminates (lines 62–74).

```
#define WIN // WIN for Winsock and BSD for BSD sockets
#include <stdio.h> // Needed for printf()
#include <string.h> // Needed for memcpy() and strcpy()
#ifdef WIN
#include <windows.h> // Needed for all Winsock calls
#endif
#ifdef BSD
```

```
#include <sys/types.h>   // Needed for system defined
                            identifiers.
#include <netinet/in.h>  // Needed for Internet address
structure.
#include <sys/socket.h>  // Needed for socket(), bind(),
                             etc...
#include <arpa/inet.h>   // Needed for inet_ntoa()
#include <fcntl.h>
#include <netdb.h>
#endif
#define PORT_NUM 1050    // Arbitrary port number for
                            the server
#define MAX_LISTEN 3     // Maximum number of listens
                            to queue
void main(void)
{
#ifdef WIN
WORD wVersionRequested = MAKEWORD(1,1);
                          // for WSA functions
WSADATA wsaData;         // for WSA functions
#endif
unsigned int server_s;   // Server socket descriptor
struct sockaddr_in server_addr;
                          // Server Internet address
unsigned int client_s;   // Client socket descriptor
struct sockaddr_in client_addr;
                          // Client Internet address
struct in_addr client_ip_addr; // Client IP address
int addr_len;             // Internet address length
char out_buf[100];       // 100-byte output buffer for data
char in_buf[100];        // 100-byte input buffer for data
#ifdef WIN               // Initialize Winsock
WSAStartup(wVersionRequested, &wsaData);
#endif                  // Create a socket
                       // - AF_INET is Address Family
                    Internet and SOCK_STREAM is streams
server_s = socket(AF_INET, SOCK_STREAM, 0);
                        // Fill-in my socket's address
                          information and bind the socket
                        // - See winsock.h for a
                           description of struct
                           sockaddr_in
server_addr.sin_family = AF_INET; // Address family
                                     to use
```

```
server_addr.sin_port = htons(PORT_NUM);
                    // Port number to use
server_addr.sin_addr.s_addr = htonl(INADDR_ANY);
                    // Listen on any IP addr.
bind(server_s, (struct sockaddr *)&server_addr,
sizeof(server_addr));
  // Listen for connections (queueing up to MAX_LISTEN)
listen(server_s, MAX_LISTEN);
  // Accept a connection. The accept() will block and
     then return with
  // client_addr filled-in.
addr_len = sizeof(client_addr);
client_s = accept(server_s, (struct sockaddr *)&client_
addr, &addr_len);
  // Copy the four-byte client IP address into an IP
     address structure
  // - See winsock.h for a description of struct in_addr
memcpy(&client_ip_addr, &client_addr.sin_addr.s_addr, 4);
  // Print an informational message that accept completed
printf(Accept completed!!! IP address of client = %s
port = %d \n,
inet_ntoa(client_ip_addr), ntohs(client_addr.sin_port));
                    // Send to the client
strcpy(out_buf, Message -- server to client);
send(client_s, out_buf, (strlen(out_buf) + 1), 0);
                    // Receive from the client
recv(client_s, in_buf, sizeof(in_buf), 0);
printf(Received from client... data = '%s' \n, in_buf);
                    // Close all open sockets
#ifdef WIN
closesocket(server_s);
closesocket(client_s);
#endif
#ifdef BSD
close(server_s);
close(client_s);
#endif
#ifdef WIN
                    // Cleanup Winsock
WSACleanup();
#endif
}
```

This is not a very realistic implementation. More often, server applications will contain some indefinite loop and be able to accept multiple requests. The preceding code can be easily converted into such more realistic server by inserting lines 46–61 into a loop in which the termination condition is never satisfied (e.g., **`while(1){ . . . }`**). Such servers will create one permanent socket through the **`socket()`** call (line 37), while a temporary socket gets spun off every time when a request is accepted (line 49). In this manner, each temporary socket will be responsible of handling a single incoming connection. If a server gets killed eventually, the permanent socket will be closed, as will each of the active temporary sockets. The TCP implementation determines when the same port number will become available for reuse by other applications. The status of such port will be in TIME-WAIT state for some predetermined period of time, as shown in Figure M.1 for port number 1234.

## M.4 STREAM AND DATAGRAM SOCKETS

When sockets are used to send a connection-oriented, reliable stream of bytes across machines, they are of SOCK_STREAM type. As we previously discussed, in such cases sockets have to be connected before being used. The data are transmitted through a bidirectional stream of bytes and are guaranteed to arrive in the order they were sent.

Sockets of SOCK_DGRAM type (or datagram sockets) support a bidirectional flow of data, as well, but data may arrive out of order, and possibly duplicated (i.e., it is not guaranteed to be arriving in sequence or to be unique). Datagram sockets also do not provide reliable service since they can fail to arrive at all. It is important to note, though, that the data record boundaries are preserved, as long as the records are no longer than the receiver could handle. Unlike stream sockets, datagram sockets are connectionless; hence, they do not need to be connected before being used. Figure M.3 shows the basic flowchart of datagram sockets communication. Taking the stream-based communication model as a base, as one can easily notice, the calls to `listen()` and `accept()` are dropped, and the calls to `send()` and `recv()` are replaced by calls to `sendto()` and `recvfrom()`.

### Example UDP Client Program (Initiate Connections)

```
#define WIN   // WIN for Winsock and BSD for BSD sockets
#include <stdio.h>     // Needed for printf()
#include <string.h>    // Needed for memcpy() and strcpy()
#ifdef WIN
#include <windows.h> // Needed for all Winsock stuff
#endif
#ifdef BSD
```

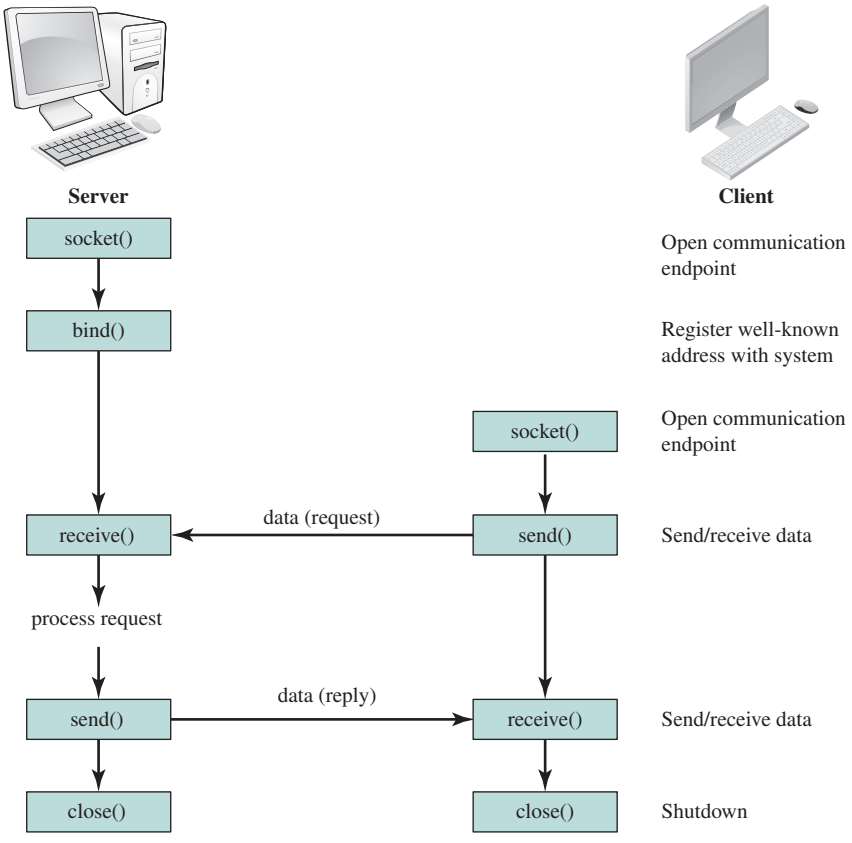



**Figure M.3 Socket System Calls for Connectionless Protocol**

```
#include <sys/types.h>   // Needed for system defined
                            identifiers.
#include <netinet/in.h> // Needed for Internet address
structure.
#include <sys/socket.h> // Needed for socket(), bind(),
                           etc...
#include <arpa/inet.h>  // Needed for inet_ntoa()
#include <fcntl.h>
#include <netdb.h>
#endif
#define PORT_NUM 1050    // Port number used
#define IP_ADDR 131.247.167.101
            // IP address of server1 (*** HARDWIRED ***)
void main(void)
{
#ifdef WIN
```

```
WORD wVersionRequested = MAKEWORD(1,1);
                         // Stuff for WSA functions
WSADATA wsaData;          // Stuff for WSA functions
#endif
unsigned int server_s;   // Server socket descriptor
struct sockaddr_in server_addr;
                         // Server Internet address
int addr_len;             // Internet address length
char out_buf[100];    // 100-byte buffer for output data
char in_buf[100];     // 100-byte buffer for input data
#ifdef WIN            // This stuff initializes winsock
WSAStartup(wVersionRequested, &wsaData);
#endif                // Create a socket
                      // - AF_INET is Address Family
                         Internet and SOCK_DGRAM is
                         datagram
server_s = socket(AF_INET, SOCK_DGRAM, 0);
                      // Fill-in server1 socket's
                         address information
server_addr.sin_family = AF_INET;
                     // Address family to use
 server_addr.sin_port = htons(PORT_NUM);
                      // Port num to use
 server_addr.sin_addr.s_addr = inet_addr(IP_ADDR);
                      // IP address to use
                   // Assign a message to buffer out_buf
 strcpy(out_buf, Message from client1 to server1);
                   // Now send the message to server1.
                      The + 1 includes the end-of-string
                   // delimiter
 sendto(server_s, out_buf, (strlen(out_buf) + 1), 0,
 (struct sockaddr *)&server_addr, sizeof(server_addr));
                   // Wait to receive a message
 addr_len = sizeof(server_addr);
 recvfrom(server_s, in_buf, sizeof(in_buf), 0,
 (struct sockaddr *)&server_addr, &addr_len);
                   // Output the received message
 printf(Message received is: '%s' \n, in_buf);
                   // Close all open sockets
#ifdef WIN
 closesocket(server_s);
#endif
#ifdef BSD
 close(server_s);
```

```
#endif
#ifdef WIN
 // Cleanup Winsock
 WSACleanup();
#endif
}
```

## Example UDP Server Program (Passively Await Connection)

```
#define WIN   // WIN for Winsock and BSD for BSD sockets
#include <stdio.h>    // Needed for printf()
#include <string.h>   // Needed for memcpy() and strcpy()
#ifdef WIN
#include <windows.h>  // Needed for all Winsock stuff
#endif
#ifdef BSD
#include <sys/types.h>    // Needed for system defined
                             identifiers.
#include <netinet/in.h>   // Needed for Internet address
                             structure.
#include <sys/socket.h>   // Needed for socket(),
                             bind(), etc...
#include <arpa/inet.h>    // Needed for inet_ntoa()
#include <fcntl.h>
#include <netdb.h>
#endif
#define PORT_NUM 1050     // Port number used
#define IP_ADDR 131.247.167.101 // IP address of client1
void main(void)
{
#ifdef WIN
WORD wVersionRequested = MAKEWORD(1,1);
                          // Stuff for WSA functions
WSADATA wsaData;          // Stuff for WSA functions
#endif
unsigned int server_s;    // Server socket descriptor
struct sockaddr_in server_addr;
                          // Server1 Internet address
struct sockaddr_in client_addr; // Client1 Internet
address
int addr_len;             // Internet address length
char out_buf[100];    // 100-byte buffer for output data
char in_buf[100];     // 100-byte buffer for input data
long int i;           // Loop counter
```

```
#ifdef WIN    // This stuff initializes winsock
WSAStartup(wVersionRequested, &wsaData);
#endif        // Create a socket
              // AF_INET is Address Family Internet and
                 SOCK_DGRAM is datagram
server_s = socket(AF_INET, SOCK_DGRAM, 0);
             // Fill-in my socket's address information
server_addr.sin_family = AF_INET;     // Address family
server_addr.sin_port = htons(PORT_NUM);   // Port number
server_addr.sin_addr.s_addr = htonl(INADDR_ANY);
             // Listen on any IP address
bind(server_s, (struct sockaddr *)&server_addr,
sizeof(server_addr));
        // Fill-in client1 socket's address information
client_addr.sin_family = AF_INET;
        // Address family to use
client_addr.sin_port = htons(PORT_NUM); // Port num to use
client_addr.sin_addr.s_addr = inet_addr(IP_ADDR);
        // IP address to use
        // Wait to receive a message from client1
addr_len = sizeof(client_addr);
recvfrom(server_s, in_buf, sizeof(in_buf), 0,
(struct sockaddr *)&client_addr, &addr_len);
        // Output the received message
printf(Message received is: '%s' \n, in_buf);
        // Spin-loop to give client1 time to turn-around
for (i=0; i>> Step #5 <<<
        // Now send the message to client1. The + 1
           includes the end-of-string
        // delimiter
sendto(server_s, out_buf, (strlen(out_buf) + 1), 0,
(struct sockaddr *)&client_addr, sizeof(client_addr));
        // Close all open sockets
#ifdef WIN
closesocket(server_s);
#endif
#ifdef BSD
close(server_s);
#endif
#ifdef WIN
        // Cleanup Winsock
WSACleanup();
#endif
}
```

## M.5 RUN-TIME PROGRAM CONTROL

### Nonblocking Socket Calls

By default, a socket is created as blocking, (i.e., it blocks until the current function call is completed). For example, if we execute an `accept()` on a socket, the process will block until there is an incoming connection from a client. In UNIX, two functions are involved in turning a blocking socket into a nonblocking one: `ioctl()` and `select()`. The first facilitates input/output control on a file descriptor or socket. The `select()` function is then used to determine the socket status—ready or not ready to perform action.

```
// change the blocking state of a socket
unsigned long unblock = TRUE;
            // TRUE for nonblocking, FALSE for blocking
ioctl(s, FIONBIO, &unblock);
```

We then call accept() periodically:

```
while(client_s = accept(s, NULL, NULL) > 0)
{
 if ( client_s == EWOULDBLOCK)
      // wait until a client connection arrives,
      // while executing useful tasks
 else
      // process accepted connection
}     // display error and exit
```

or use the `select()` function call to query the socket status, as in the following segment from a nonblocking socket program:

```
if (select(max_descr + 1, &sockSet, NULL, NULL,
&sel_timeout) == 0)
              // print a message for the user
else
{ . . .
client_s = accept(s, NULL, NULL);
  . . .
}
```

In this way, when some socket descriptor is ready for I/O, the process has to be constantly polling the OS with `select()` calls, until the socket is ready. Although the process executing a `select()` call would suspend the program until the socket is ready or until the `select()` function times out (as opposed to suspending it until the socket is ready, if the socket were blocking), this solution is still inefficient. Just like calling a nonblocking `accept()` within a loop, calling `select()` within a loop results in wasting CPU cycles.

## Asynchronous I/O (Signal Driven I/O)

A better solution is to use asynchronous I/O (i.e., when I/O activity is detected on the socket, the OS informs the process immediately and thus relieves it from the burden of polling all the time). In the original BSD UNIX, this involves the use of calls to `sigaction()` and `fcntl()`. An alternative to poll for the status of a socket through the `select()` call is to let the kernel inform the application about events via a `SIGIO` signal. In order to do that, a valid signal handler for `SIGIO` must be installed with `sigaction()`. The following program does not involve sockets, it merely provides a simple example on how to install a signal handler. It catches an interrupt char (Cntrl-C) input by setting the signal handling for `SIGINT` (interrupt signal) via `sigaction()`:

```
#include <stdio.h>          // for printf()
#include <sys/signal.h>     // for sigaction()
#include <unistd.h>         // for pause()
void catch_error(char *errorMessage);
                            // for error handling
void InterruptSignalHandler(int signalType);
                            // handle interr. signal
int main(int argc, char *argv[])
{
 struct sigaction handler;
                           // Signal handler specification
   // Set InterruptSignalHandler() as a handler function
 handler.sa_handler = InterruptSignalHandler;
                            // Create mask for all signals
 if (sigfillset(&handler.sa_mask) < 0)
 catch_error(sigfillset() failed);

                            // No flags
 handler.sa_flags = 0;
            // Set signal handling for interrupt signals
 if (sigaction(SIGINT, &handler, 0) < 0)
 catch_error(sigaction() failed);
 for(;;)
 pause();   // suspend program until signal received
 exit(0);
}
void InterruptSignalHandler(int signalType)
{
 printf(Interrupt Received. Program terminated.\n);
 exit(1);
}
```

A **FASYNC** flag must be set on a socket file descriptor via **fcntl()**. In more detail, first we notify the OS about our desire to install a new disposition for **SIGIO**, using **sigaction()**; then we force the OS to submit signals to the current process by using **fcntl()**. This call is needed to ensure that among all processes that access the socket, the signal is delivered to the current process (or process group); next, we use **fcntl()** again to set the status flag on the same socket descriptor for asynchronous **FASYNC**. The following segment of a datagram sockets program follows this scheme. Note all the unnecessary details are omitted for clarity:

```
int main()
{
 . . .
        // Create socket for sending/receiving datagrams
        // Set up the server address structure
        // Bind to the local address
        // Set signal handler for SIGIO
        // Create mask that mask all signals
 if (sigfillset(&handler.sa_mask) < 0)
        // print error and exit
        // No flags
 handler.sa_flags = 0;
 if (sigaction(SIGIO, &handler, 0) < 0)
        // print error and exit
        // We must own the socket to receive the SIGIO
           message
 if (fcntl(s_socket, F_SETOWN, getpid()) < 0)
        //print error and exit
        // Arrange for asynchronous I/O and SIGIO
           delivery
 if (fcntl(s_socket, F_SETFL, FASYNC | O_NONBLOCK) < 0)
        // print error and exit
 for (;;)
 pause();
 . . .
}
```

Under Windows, the `select()` function is not implemented. The `WSAAsyncSelect()` is used to request notification of network events (i.e., request that `Ws2_32.dll` sends a message to the window `hWnd`):

```
WSAAsyncSelect (SOCKET  socket, HWND  hWnd,
unsigned int  wMsg, long  lEvent)
```

The SOCKET type is defined in `winsock.h`. *socket* is a socket descriptor, *hWnd* is the window handle, *wMsg* is the message, *lEvent* is usually a logical OR of all events we are expecting to be notified of, when completed. Some of the event values

are FD_CONNECT (connection completed), FD_ACCEPT (ready to accept), FD_READ (ready to read), FD_WRITE (ready to write), FD_CLOSE (connection closed). You can easily incorporate the following stream sockets program segment into the previously presented programs or your own application. (Again, details are omitted.):

```
        // the message for the asynchronous notification
 #define wMsg (WM_USER + 4)
 ...
       // socket_s has already been created and bound to
          a name
       // listen for connections
if (listen(socket_s, 3) == SOCKET_ERROR)
       // print error message
       // exit after cleanup
       // get notification on connection accept
       // to this window
if (WSAAsyncSelect(s, hWnd, wMsg, FD_ACCEPT)==
SOCKET_ERROR)
       // print cannot process asynchronously
       // exit after cleanup
 else  // accept the incoming connection
```

Further references on Asynchronous I/O are "The Pocket Guide to TCP/IP Sockets—C version" by Donahoo and Calvert (for UNIX), and "Windows Sockets Network Programming" (for Windows), by Bob Quinn.

## M.6 REMOTE EXECUTION OF A WINDOWS CONSOLE APPLICATION

Simple sockets operations can be used to accomplish tasks that are otherwise hard to achieve. For example, by using sockets we can remotely execute an application. The sample code[1] is presented. Two sockets programs, a local and remote, are used to transfer a Windows console application (an `.exe` file) from the local host to the remote host. The program is executed on the remote host, then `stdout` is returned to the local host.

### Local Code

```
#include <stdio.h>   // Needed for printf()
#include <stdlib.h>  // Needed for exit()
#include <string.h>  // Needed for memcpy() and strcpy()
#include <windows.h> // Needed for Sleep() and Winsock
                        stuff
```

[1] This and other code presented is in part written by Ken Christensen and Karl S. Lataxes at the Computer Science Department of the University of South Florida, http://www.csee.usf.edu/~christen/tools/.

```
#include <fcntl.h>     // Needed for file i/o constants
#include <sys\stat.h>  // Needed for file i/o constants
#include <io.h> // Needed for open(), close(), and eof()
#define PORT_NUM 1050  // Arbitrary port number for the
                          server
#define MAX_LISTEN 1   // Maximum number of listens to
                          queue
#define SIZE 256       // Size in bytes of transfer
                          buffer

void main(int argc, char *argv[])
{
WORD wVersionRequested = MAKEWORD(1,1); // WSA functions
WSADATA wsaData;       // Winsock API data structure
unsigned int remote_s; // Remote socket descriptor
struct sockaddr_in remote_addr;
                       // Remote Internet address
struct sockaddr_in server_addr;
                       // Server Internet address
unsigned char bin_buf[SIZE]; // Buffer for file transfer
unsigned int fh;        // File handle
unsigned int length;    // Length of buffers transferred
struct hostent *host;   // Structure for gethostbyname()
struct in_addr address; // Structure for Internet
                           address
char host_name[256];    // String for host name
int addr_len; // Internet address length
unsigned int local_s;   // Local socket descriptor
struct sockaddr_in local_addr; // Local Internet address
struct in_addr remote_ip_addr; // Remote IP address
   // Check if number of command line arguments is valid
if (argc !=4)
{
printf( *** ERROR - Must be 'local (host) (exefile)
        (outfile)' \n);
printf( where host is the hostname *or* IP address \n);
printf( of the host running remote.c, exefile is the \n);
printf( name of the file to be remotely run, and \n);
printf( outfile is the name of the local output file. \n);
exit(1);
}
        // Initialization of winsock
WSAStartup(wVersionRequested, &wsaData);
        // Copy host name into host_name
```

```
strcpy(host_name, argv[1]);
        // Do a gethostbyname()
host = gethostbyname(argv[1]);
if (host == NULL)
{
printf( *** ERROR - IP address for '%s' not be found \n,
host_name);
exit(1);
}         // Copy the four-byte client IP address into
             an IP address structure
memcpy(&address, host->h_addr, 4);
        // Create a socket for remote
remote_s = socket(AF_INET, SOCK_STREAM, 0);
        // Fill-in the server (remote) socket's address
information and connect
        // with the listening server.
server_addr.sin_family = AF_INET; // Address family to use
server_addr.sin_port = htons(PORT_NUM); // Port num to use
server_addr.sin_addr.s_addr = inet_addr(inet_
ntoa(address)); // IP address
connect(remote_s, (struct sockaddr *)&server_addr,
sizeof(server_addr));
        // Open and read *.exe file
if((fh = open(argv[2], O_RDONLY | O_BINARY, S_IREAD |
     S_IWRITE)) == -1)
{
printf( ERROR - Unable to open file '%s'\n, argv[2]);
exit(1);
}
        // Output message stating sending executable file
printf(Sending '%s' to remote server on '%s' \n,
argv[2], argv[1]);
        // Send *.exe file to remote
while(!eof(fh))
{
length = read(fh, bin_buf, SIZE);
send(remote_s, bin_buf, length, 0);
}
        // Close the *.exe file that was sent to the
           server (remote)
close(fh);
        // Close the socket
closesocket(remote_s);
        // Cleanup Winsock
```

```
WSACleanup();
        // Output message stating remote is executing
printf( '%s' is executing on remote server \n, argv[2]);
        // Delay to allow everything to cleanup
Sleep(100);
        // Initialization of winsock
WSAStartup(wVersionRequested, &wsaData);

        // Create a new socket to receive output file
           from remote server
local_s = socket(AF_INET, SOCK_STREAM, 0);
        // Fill-in the socket's address information and
           bind the socket
local_addr.sin_family = AF_INET; // Address family to use
local_addr.sin_port = htons(PORT_NUM); // Port num to use
local_addr.sin_addr.s_addr = htonl(INADDR_ANY);
        // Listen on any IP addr
bind(local_s, (struct sockaddr *)&local_addr,
sizeof(local_addr));
        // Listen for connections (queueing up to
           MAX_LISTEN)
listen(local_s, MAX_LISTEN);
        // Accept a connection, the accept will block
           and then return with
        // remote_addr filled in.
addr_len = sizeof(remote_addr);
remote_s = accept(local_s, (struct sockaddr*)
      &remote_addr, &addr_len);
        // Copy the four-byte client IP address into an
           IP address structure
memcpy(&remote_ip_addr, &remote_addr.sin_addr.s_addr, 4);
        // Create and open the output file for writing
if ((fh=open(argv[3], O_WRONLY | O_CREAT | O_TRUNC |
      O_BINARY,
S_IREAD | S_IWRITE)) == - 1)
{
printf( *** ERROR - Unable to open '%s'\n, argv[3]);
exit(1);
}
        // Receive output file from server
length = SIZE;
while(length > 0)
{
length = recv(remote_s, bin_buf, SIZE, 0);
```

```
write(fh, bin_buf, length);
}
  // Close output file that was received from the remote
close(fh);
  // Close the sockets
closesocket(local_s);
closesocket(remote_s);
  // Output final status message
printf(Execution of '%s' and transfer of output
     to '%s' done! \n,
argv[2], argv[3]);
  // Cleanup Winsock
WSACleanup();
}
```

## Remote Code

```
#include <stdio.h>     // Needed for printf()
#include <stdlib.h>    // Needed for exit()

#include <string.h>    // Needed for memcpy() and strcpy()
#include <windows.h>  // Needed for Sleep() and Winsock
                         stuff
#include <fcntl.h>     // Needed for file i/o constants
#include <sys\stat.h> // Needed for file i/o constants
#include <io.h>  // Needed for open(), close(), and eof()
#define PORT_NUM 1050 // Arbitrary port number for the
                         server
#define MAX_LISTEN 1   // Maximum number of listens to
                         queue
#define IN_FILE run.exe // Name given to transferred
                           *.exe file
#define TEXT_FILE output
                       // Name of output file for stdout
#define SIZE 256       // Size in bytes of transfer
                         buffer
void main(void)
{
WORD wVersionRequested = MAKEWORD(1,1); // WSA functions
WSADATA wsaData;        // WSA functions
unsigned int remote_s; // Remote socket descriptor
struct sockaddr_in remote_addr;
                       // Remote Internet address
struct sockaddr_in server_addr;
                       // Server Internet address
```

```
unsigned int local_s;          // Local socket descriptor
struct sockaddr_in local_addr; // Local Internet address
struct in_addr local_ip_addr;  // Local IP address
int addr_len;                  // Internet address length
unsigned char bin_buf[SIZE]; // File transfer buffer
unsigned int fh;               // File handle
unsigned int length;         // Length of transf. buffers
                               // Do forever
while(1)
{                              // Winsock initialization
WSAStartup(wVersionRequested, &wsaData);
                              // Create a socket
remote_s = socket(AF_INET, SOCK_STREAM, 0);
              // Fill-in my socket's address information
                 and bind the socket
remote_addr.sin_family = AF_INET; // Address family to use
remote_addr.sin_port = htons(PORT_NUM);
              // Port number to use
remote_addr.sin_addr.s_addr = htonl(INADDR_ANY);
              // Listen on any IP addr
bind(remote_s,  (struct  sockaddr  *)&remote_addr,
sizeof(remote_addr));
              // Output waiting message
printf(Waiting for a connection... \n);
   // Listen for connections (queueing up to MAX_LISTEN)
listen(remote_s, MAX_LISTEN);
   // Accept a connection, accept() will block and return
      with local_addr
addr_len = sizeof(local_addr);
local_s = accept(remote_s, (struct sockaddr *)&local_
addr, &addr_len);
   // Copy the four-byte client IP address into an IP
      address structure
memcpy(&local_ip_addr, &local_addr.sin_addr.s_addr, 4);
   // Output message acknowledging receipt, saving of *.exe
printf( Connection established, receiving remote executable
file \n);
   // Open IN_FILE for remote executable file
if((fh = open(IN_FILE, O_WRONLY | O_CREAT | O_TRUNC |
O_BINARY,
S_IREAD | S_IWRITE)) == - 1)
{
printf( *** ERROR - unable to open executable file \n);
exit(1);
```

```
}
     // Receive executable file from local
length = 256;
while(length > 0)
{
length = recv(local_s, bin_buf, SIZE, 0);
write(fh, bin_buf, length);
}
     // Close the received IN_FILE
close(fh);
     // Close sockets
closesocket(remote_s);
closesocket(local_s);
     // Cleanup Winsock
WSACleanup();
     // Print message acknowledging execution of *.exe
printf( Executing remote executable (stdout to output
file) \n);
     // Execute remote executable file (in IN_FILE)
system(IN_FILE > TEXT_FILE);
     // Winsock initialization to reopen socket to send
        output file to local
WSAStartup(wVersionRequested, &wsaData);
     // Create a socket
     // - AF_INET is Address Family Internet and SOCK_
        STREAM is streams
local_s = socket(AF_INET, SOCK_STREAM, 0);
     // Fill in the server's socket address information
        and connect with
     // the listening local
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(PORT_NUM);
server_addr.sin_addr.s_addr = inet_addr(inet_ntoa
        (local_ip_addr));
connect(local_s, (struct sockaddr *)&server_addr,
sizeof(server_addr));
     // Print message acknowledging transfer of output to
        client
printf( Sending output file to local host \n);
     // Open output file to send to client
if((fh = open(TEXT_FILE, O_RDONLY |
        O_BINARY, S_IREAD | S_IWRITE)) == - 1)
{
printf( *** ERROR - unable to open output file \n);
```

```
exit(1);
}
                // Send output file to client
while(!eof(fh))
{
length = read(fh, bin_buf, SIZE);
send(local_s, bin_buf, length, 0);
}
                // Close output file
close(fh);
                // Close sockets
closesocket(remote_s);
closesocket(local_s);
                // Cleanup Winsock
WSACleanup();
                // Delay to allow everything to cleanup
Sleep(100);
}
}
```

# APPENDIX N

## THE INTERNATIONAL REFERENCE ALPHABET

A familiar example of data is **text** or character strings. While textual data are most convenient for human beings, they cannot, in character form, be easily stored or transmitted by data processing and communications systems. Such systems are designed for binary data. Thus, a number of codes have been devised by which characters are represented by a sequence of bits. Perhaps the earliest common example of this is the Morse code. Today, the most commonly used text code is the International Reference Alphabet (IRA).[1] Each character in this code is represented by a unique 7-bit binary code; thus, 128 different characters can be represented. Table N.1 lists all of the code values. In the table, the bits of each character are labeled from $b_7$, which is the most significant bit, to $b_1$, the least significant bit. Characters are of two types: printable and control (see Table N.2). Printable characters are the alphabetic, numeric, and special characters that can be printed on paper or displayed on a screen. For example, the bit representation of the character "K" is $b_7b_6b_5b_4b_3b_2b_1 = 1001011$. Some of the control characters have to do with controlling the printing or displaying

**Table N.1** The International Reference Alphabet (IRA)

| Bit Position | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | $b_7$ | | | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| | | $b_6$ | | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 |
| | | | $b_5$ | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |
| $b_4$ | $b_3$ | $b_2$ | $b_1$ | | | | | | | | |
| 0 | 0 | 0 | 0 | NUL | DLE | SP | 0 | @ | P | ‘ | p |
| 0 | 0 | 0 | 1 | SOH | DC1 | ! | 1 | A | Q | a | q |
| 0 | 0 | 1 | 0 | STX | DC2 | ” | 2 | B | R | b | r |
| 0 | 0 | 1 | 1 | ETX | DC3 | # | 3 | C | S | c | s |
| 0 | 1 | 0 | 0 | EOT | DC4 | $ | 4 | D | T | d | t |
| 0 | 1 | 0 | 1 | ENQ | NAK | % | 5 | E | U | e | u |
| 0 | 1 | 1 | 0 | ACK | SYN | & | 6 | F | V | f | v |
| 0 | 1 | 1 | 1 | BEL | ETB | ’ | 7 | G | W | g | w |
| 1 | 0 | 0 | 0 | BS | CAN | ( | 8 | H | X | h | x |
| 1 | 0 | 0 | 1 | HT | EM | ) | 9 | I | Y | i | y |
| 1 | 0 | 1 | 0 | LF | SUB | * | : | J | Z | j | z |
| 1 | 0 | 1 | 1 | VT | ESC | + | ; | K | [ | k | { |
| 1 | 1 | 0 | 0 | FF | FS | , | < | L | \ | l | \| |
| 1 | 1 | 0 | 1 | CR | GS | – | = | M | ] | m | } |
| 1 | 1 | 1 | 0 | SO | RS | . | > | N | ^] | n | ~ |
| 1 | 1 | 1 | 1 | SI | US | / | ? | O | _ | o | DEL |

[1] IRA is defined in ITU-T Recommendation T.50 and was formerly known as International Alphabet Number 5 (IA5). The U.S. national version of IRA is referred to as the American Standard Code for Information Interchange (ASCII).

**Table N.2** IRA Control Characters

| **Format Control** | |
|---|---|
| **BS** (Backspace): Indicates movement of the printing mechanism or display cursor backward one position.<br>**HT** (Horizontal Tab): Indicates movement of the printing mechanism or display cursor forward to the next preassigned "tab" or stopping position.<br>**LF** (Line Feed): Indicates movement of the printing mechanism or display cursor to the start of the next line. | **VT** (Vertical Tab): Indicates movement of the printing mechanism or display cursor to the next of a series or preassigned printing lines.<br>**FF** (Form Feed): Indicates movement of the printing mechanism or display cursor to the starting position of the next page, form, or screen.<br>**CR** (Carriage Return): Indicates movement of the printing mechanism or display cursor to the starting position of the same line. |
| **Transmission Control** | |
| **SOH** (Start of Heading): Used to indicate the start of a heading, which may contain address or routing information.<br>**STX** (Start of Text): Used to indicate the start of the text and so also indicates the end of the heading.<br>**ETX** (End of Text): Used to terminate the text that was started with STX.<br>**EOT** (End of Transmission): Indicates the end of a transmission, which may have included one or more "texts" with their headings.<br>**ENQ** (Enquiry): A request for a response from a remote station. It may be used as a "WHO ARE YOU" request for a station to identify itself. | **ACK** (Acknowledge): A character transmitted by a receiving device as an affirmation response to a sender. It is used as a positive response to polling messages.<br>**NAK** (Negative Acknowledgment): A character transmitted by a receiving device as a negative response to a sender. It is used as a negative response to polling messages.<br>**SYN** (Synchronous/Idle): Used by a synchronous transmission system to achieve synchronization. When no data is being sent a synchronous transmission system may send SYN characters continuously.<br>**ETB** (End of Transmission Block): Indicates the end of a block of data for communication purposes. It is used for blocking data where the block structure is not necessarily related to the processing format. |
| **Information Separator** | |
| **FS** (File Separator)<br>**GS** (Group Separator)<br>**RS** (Record Separator)<br>**US** (Unit Separator) | Information separators to be used in an optional manner except that their hierarchy shall be FS (the most inclusive) to US (the least inclusive) |
| **Miscellaneous** | |
| **NUL** (Null): No character. Used for filling in time or filling space on tape when there are no data.<br>**BEL** (Bell): Used when there is need to call human attention. It may control alarm or attention devices.<br>**SO** (Shift Out): Indicates the code combinations that follow shall be interpreted as outside of the standard character set until a SI character is reached.<br>**SI** (Shift In): Indicates the code combinations that follow shall be interpreted according to the standard character set.<br>**DEL** (Delete): Used to obliterate unwanted characters; for example by overwriting.<br>**SP** (Space): A nonprinting character used to separate words, or to move the printing mechanism or display cursor forward by one position. | **DLE** (Data Link Escape): A character that shall change the meaning of one or more contiguously following characters. It can provide supplementary controls, or permits the sending of data characters having any bit combination.<br>**DC1, DC2, DC3, DC4** (Device Controls): Characters for the control of ancillary devices or special terminal features.<br>**CAN** (Cancel): Indicates the data that precedes it in a message or block should be disregarded (usually because an error has been detected).<br>**EM** (End of Medium): Indicates the physical end of a tape or other medium, or the end of the required or used portion of the medium.<br>**SUB** (Substitute): Substituted for a character that is found to be erroneous or invalid.<br>**ESC** (Escape): A character intended to provide code extension in that it gives a specified number of continuously following characters an alternate meaning. |

of characters; an example is carriage return. Other control characters are concerned with communications procedures.

IRA-encoded characters are almost always stored and transmitted using 8 bits per character. In that case, the eighth bit is a parity bit used for error detection. The parity bit is the most significant bit and is therefore labeled $b_8$. This bit is set such that the total number of binary 1s in each octet is always odd (odd parity) or always even (even parity). Thus a transmission error that changes a single bit, or any odd number of bits, can be detected.

# APPENDIX O

## BACI: THE BEN-ARI CONCURRENT PROGRAMMING SYSTEM


O.1 **Introduction**

O.2 **BACI**
- System Overview
- Concurrency Constructs in BACI
- How to Obtain BACI

O.3 **Examples Of BACI Programs**

O.4 **BACI Projects**
- Implementation of Synchronization Primitives
- Semaphores, Monitors, and Implementations

O.5 **Enhancements To The BACI System**

## O.1 INTRODUCTION

In Chapter 5, concurrency concepts are introduced (e.g., mutual exclusion and the critical section problem) and synchronization techniques are proposed (e.g., semaphores, monitors, and message passing). Deadlock and starvation issues for concurrent programs are discussed in Chapter 6. Due to the increasing emphasis on parallel and distributed computing, understanding concurrency and synchronization is more necessary than ever. To obtain a thorough understanding of these concepts, practical experience writing concurrent programs is needed.

Three options exist for this desired "hands-on" experience. First, we can write concurrent programs with an established concurrent programming language such as Concurrent Pascal, Modula, Ada, or the SR Programming Language. To experiment with a variety of synchronization techniques, however, we must learn the syntax of many concurrent programming languages. Second, we can write concurrent programs using system calls in an operating system such as UNIX. It is easy, however, to be distracted from the goal of understanding concurrent programming by the details and peculiarities of a particular operating system (e.g., details of the semaphore system calls in UNIX). Lastly, we can write concurrent programs with a language developed specifically for giving experience with concurrency concepts such as the Ben-Ari Concurrent Interpreter (BACI). Using such a language offers a variety of synchronization techniques with a syntax that is usually familiar. Languages developed specifically for giving experience with concurrency concepts are the best option to obtain the desired hands-on experience.

Section O.2 contains a brief overview of the BACI system and how to obtain the system. Section O.3 contains examples of BACI programs, and Section O.4 contains a discussion of projects for practical concurrency experience at the implementation and programming levels. Lastly, Section O.5 contains a description of enhancements to the BACI system that have been made.

## O.2 BACI

### System Overview

BACI is a direct descendant of Ben-Ari's modification to sequential Pascal (Pascal-S). Pascal-S is a subset of standard Pascal by Wirth, without files, except INPUT and OUTPUT, sets, pointer variables, and goto statements. Ben-Ari took the Pascal-S language and added concurrent programming constructs such as the `cobegin... coend` construct and the semaphore variable type with `wait` and `signal` operations. BACI is Ben-Ari's modification to Pascal-S with additional synchronization features (e.g., monitors) as well as encapsulation mechanisms to ensure that a user is prevented from modifying a variable inappropriately (e.g., a semaphore variable should only be modified by semaphore functions).

BACI simulates concurrent process execution and supports the following synchronization techniques: general semaphores, binary semaphores, and monitors. The BACI system is composed of two subsystems, as illustrated in Figure O.1. The first

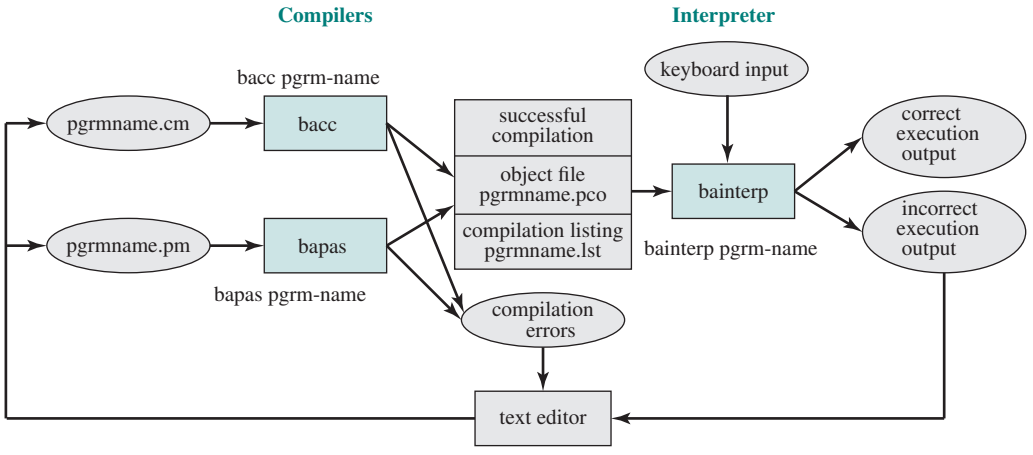



**Figure O.1**

subsystem, the compiler, compiles a user's program into intermediate object code, called PCODE. There are two compilers available with the BACI system, corresponding to two popular languages taught in introductory programming courses. The syntax of one compiler is similar to standard Pascal; BACI programs that use the Pascal syntax are denoted as pgrm-name.pm. The syntax of the other compiler is similar to standard C++; these BACI programs are denoted as pgrm-name.cm. Both compilers create two files during the compilation: pgrm-name.lst and pgrm-name.pco.

The second subsystem in the BACI system, the interpreter, executes the object code created by the compiler. In other words, the interpreter executes pgrm-name.pco. The core of the interpreter is a preemptive scheduler; during execution, this scheduler randomly swaps between concurrent processes, thus simulating a parallel execution of the concurrent processes. The interpreter offers a number of different debug options, such as single-step execution, disassembly of PCODE instructions, and display of program storage locations.

## Concurrency Constructs in BACI

In the rest of this appendix, we focus on the compiler similar to standard C++. We call this compiler C−−; although the syntax is similar to C++, it does not include inheritance, encapsulation, or other object-oriented programming features. In this section, we give an overview of the BACI concurrency constructs; see the user's guides at the BACI website for further details of the required Pascal or C−− BACI syntax.

***COBEGIN*** A list of processes to be run concurrently is enclosed in a `cobegin` block. Such blocks cannot be nested and must appear in the main program.

```
cobegin { proc1(...); proc2(...); ... ; procN(...); }
```

The PCODE statements created by the compiler for the above block are interleaved by the interpreter in an arbitrary, "random" order; multiple executions of the same program containing a `cobegin` block will appear to be nondeterministic.

***SEMAPHORES*** A semaphore in BACI is a nonnegative-valued `int` variable, which can only be accessed by the semaphore calls defined subsequently. A binary semaphore in BACI, one that only assumes the values 0 and 1, is supported by the `binarysem` subtype of the `semaphore` type. During compilation and execution, the compiler and interpreter enforce the restrictions that a `binarysem` variable can only have the values 0 or 1 and that `semaphore` type can only be nonnegative. BACI semaphore calls include

- `initialsem(semaphore sem, int expression)`
- `p(semaphore sem)`: If the value of `sem` is greater than zero, then the interpreter decrements `sem` by one and returns, allowing p's caller to continue. If the value of `sem` is equal to zero, then the interpreter puts `p`'s caller to sleep. The command `wait` is accepted as a synonym for `p`.
- `v(semaphore sem)`: If the value of `sem` is equal to zero and one or more processes are sleeping on `sem`, then wake up one of these processes. If no processes are waiting on `sem`, then increment `sem` by one. In any event, `v`'s caller is allowed to continue. (BACI conforms to Dijkstra's original semaphore proposal by randomly choosing which process to wake up when a signal arrives.) The command `signal` is accepted as a synonym for `v`.

***MONITORS*** BACI supports the monitor concept, with some restrictions. A monitor is a C++ block, like a block defined by a procedure or function, with some additional properties (e.g., conditional variables). In BACI, a monitor must be declared at the outermost, global level and it cannot be nested with another monitor block. Three constructs are used by the procedures and functions of a monitor to control concurrency: condition variables, `waitc` (wait on a condition), and `signalc` (signal a condition). A condition never actually has a value; it is somewhere to wait or something to signal. A monitor process can wait for a condition to hold or signal that a given condition now holds through the `waitc` and `signalc` calls. `waitc` and `signalc` calls have the following syntax and semantics:

- `waitc(condition cond, int prio)`: The monitor process (and hence the outside process calling the monitor process) is blocked on the condition `cond` and assigned the priority `prio`.
- `waitc(condition cond)`: This call has the same semantics as the `waitc` call, but the `wait` is assigned a default priority of 10.
- `signalc(condition cond)`: Wake some process waiting on `cond` with the smallest (highest) priority; if no process is waiting on `cond`, do nothing.

BACI conforms to the immediate resumption requirement. In other words, a process waiting on a condition has priority over a process trying to enter the monitor, if the process waiting on a condition has been signaled.

***OTHER CONCURRENCY CONSTRUCTS*** The C−− BACI compiler provides several low-level concurrency constructs that can be used to create new concurrency control primitives. If a function is defined as atomic, then the function is nonpreemptible. In other words, the interpreter will not interrupt an atomic function with a context switch. In BACI, the suspend function puts the calling process to sleep and the revive function revives a suspended process.

### How to Obtain BACI

The BACI system, with two user guides (one for each of the two compilers) and detailed project descriptions, is available at the BACI website at http://inside.mines.edu/fs_home/tcamp/baci/baci_index.html. The BACI system is written in both C and Java. The C version of the BACI system can be compiled in Linux, RS/6000 AIX, Sun OS, DOS, and CYGWIN on Windows with minimal modifications to the Makefile file. (See the README file in the distribution for installation details for a given platform.)

## O.3 EXAMPLES OF BACI PROGRAMS

In Chapters 5 and 6, a number of the classical synchronization problems were discussed (e.g., the readers/writers problem and the dining philosophers problem). In this section, we illustrate the BACI system with three BACI programs. Our first example illustrates the nondeterminism in the execution of concurrent processes in the BACI system. Consider the following program:

```
const int m = 5;
int n;
void incr(char id)
{
 int i;
      for(i = 1; i <= m; i = i + 1)
       {
         n = n + 1;
         cout << id << " n =" << n << " i =";
         cout << i << " " << id << endl;
       }
}
main()
{
      n = 0;
      cobegin {
```

```
        incr( 'A'); incr( 'B' ); incr('C');
        }
      cout << "The sum is " << n << endl;
}
```

Note in the preceding program that if each of the three processes created (A, B, and C) executed sequentially, the output sum would be 15. Concurrent execution of the statement n = n + 1; however, can lead to different values of the output sum. After we compiled the preceding program with BACI, we executed the PCODE file with `bainterp` a number of times. Each execution produced output sums between 9 and 15. One sample execution produced by the BACI interpreter is the following.

```
Source file: incremen.cm Fri Aug 1 16:51:00 1997
CB n = 2 i =1 C n =2
A n = 2 i =1 i =1 A
CB
     n = 3 i = 2 C
A n = 4 i = 2 C n = 5 i = 3 C
A
B n = 6C i = 2 B
     n = 7 i = 4 C
A n = 8 i = 3 A
BC n = 10 n = 10 i = 5 C
A n = i = 311 i = 4 A
 B
A n = 12 i = B5 n = 13A
     i = 4 B
B n = 14 i = 5 B
The sum is 14
```

Special machine instructions are needed to synchronize the access of processes to a common main memory. Mutual exclusion protocols, or synchronization primitives, are then built on top of these special instructions. In BACI, the interpreter will not interrupt a function defined as atomic with a context switch. This feature allows users to implement these low-level special machine instructions. For example, the following program is a BACI implementation of the testset function. A testset instruction tests the value of the function's argument i. If the value of i is zero, the function replaces it with 1 and returns true; otherwise, the function does not change the value of i and returns false. As discussed in Section 5.2, special machine instructions (e.g., testset) allow more than one action to occur without interruption. BACI has an atomic keyword defined for this purpose.

```
// Test and set instruction
//
atomic int testset(int& i)
```

```
{
      if (i == 0) {
       i = 1;
       return 1;
      }
      else
       return 0;
}
```

We can use testset to implement mutual exclusion protocols, as shown in the following program. This program is a BACI implementation of a mutual exclusion program based on the test and set instruction. The program assumes three concurrent processes; each process requests mutual exclusion 10 times.

```
int bolt = 0;
const int RepeatCount = 10;
void proc(int id)
{
      int i = 0;
      while(i < RepeatCount) {
       while (testset(bolt)); // wait
             // enter critical section
       cout << id;
             // leave critical section
       bolt = 0;
       i++;
      }
}
main()
{
      cobegin {
       proc(0); proc(1); proc(2);
      }
}
```

The following two programs are a BACI solution to the bounded-buffer producer/consumer problem with semaphores (see Figure 5.13). In this example, we have two producers, three consumers, and a buffer size of five. We first list the program details for this problem. We then list the included file that defines the bounded-buffer implementation.

```
// A solution to the bounded-buffer producer/consumer
problem
// Stallings,  Figure 5.13
// bring in the bounded-buffer machinery
```

```
#include "boundedbuff.inc"
const int ValueRange = 20; // integers in 0..19 will be
                                produced
semaphore to; // for exclusive access to terminal output
semaphore s; // mutual exclusion for the buffer
semaphore n; // # consumable items in the buffer
semaphore e; // # empty spaces in the buffer
int produce(char id)
{
     int tmp;
     tmp = random(ValueRange);
     wait(to);
     cout << "Producer " << id << " produces " << tmp
       << endl;
     signal(to);
     return tmp;
}
void consume(char id, int i)
{
     wait(to);
     cout << "Consumer " << id << " consumes " << i
       << endl;
     signal(to);
}
void producer(char id)
{
     int i;
     for (;;) {
      i = produce(id);
      wait(e);
      wait(s);
      append(i);
      signal(s);
      signal(n);
     }
}
void consumer(char id)
{
     int i;
     for (;;) {
      wait(n);
      wait(s);
      i = take();
      signal(s);
```

```
        signal(e);
        consume(id,i);
       }
}
main()
{
      initialsem(s,1);
      initialsem(n,0);
      initialsem(e,SizeOfBuffer);
      initialsem(to,1);
      cobegin {
       producer('A'); producer('B');
       consumer('x'); consumer('y'); consumer('z');
      }
}
```

```
// boundedbuff.inc -- bounded buffer include file
const int SizeOfBuffer = 5;
int buffer[SizeOfBuffer];
int in = 0; // index of buffer to use for next append
int out = 0; // index of buffer to use for next take
void append(int v)
      // add v to the buffer
      // overrun is assumed to be taken care of
      // externally through semaphores or conditions
{
      buffer[in] = v;
      in = (in + 1) % SizeOfBuffer;
}
int take()
      // return an item from the buffer
 // underrun is assumed to be taken care of
 // externally through a semaphore or condition
{
      int tmp;
      tmp = buffer[out];
      out = (out + 1) % SizeOfBuffer;
      return tmp;
}
```

One sample execution of the preceding bounded-buffer solution in BACI is the following.

```
Source file: semprodcons.cm Fri Aug 1 12:36:55 1997
Producer B produces 4
```

```
Producer A produces 13
Producer B produces 12
Producer A produces 4
Producer B produces 17
Consumer x consumes 4
Consumer y consumes 13
Producer A produces 16
Producer B produces 11
Consumer z consumes 12
Consumer x consumes 4
Consumer y consumes 17
Producer B produces 6
...
```

## O.4 BACI PROJECTS

In this section, we discuss two general types of projects one can implement in BACI. We first discuss projects that involve the implementation of low-level operations (e.g., special machine instructions that are used to synchronize the access of processes to a common main memory). We then discuss projects that are built on top of these low-level operations (e.g., classical synchronization problems). For more information on these projects, see the project descriptions included in the BACI distribution. For solutions to some of these projects, teachers should contact the authors. In addition to the projects discussed in this section, many of the problems at the end of Chapter 5 and Appendix A can be implemented in BACI.

### Implementation of Synchronization Primitives

***Implementation of Machine Instructions*** There are numerous machine instructions that one can implement in BACI. For example, one can implement the compare-and-swap or the exchange instruction discussed in Figure 5.2. The implementation of these instructions should be based on an atomic function that returns an int value. You can test your implementation of the machine instruction by building a mutual exclusion protocol on top of your low-level operation.

***Implementation of Fair Semaphores (FIFO)*** The semaphore operation in BACI is implemented with a random wake up order, which is how semaphores were originally defined by Dijkstra. As discussed in Section 5.3, however, the fairest policy is FIFO. One can implement semaphores with this FIFO wake up order in BACI. At least the following four procedures should be defined in the implementation:

- `CreateSemaphores()` to initialize the program code
- `InitSemaphore(int sem-index)` to initialize the semaphore represented by `sem-index`
- `FIFOP(int sem-index)`
- `FIFOV(int sem-index)`

This code needs to be written as a system implementation and, as such, should handle all possible errors. In other words, the semaphore designer is responsible for producing code that is robust in the presence of ignorant, stupid, or even malicious use by the user community.

## Semaphores, Monitors, and Implementations

There are many classical concurrent programming problems: the producer/consumer problem, the dining philosophers, the reader/writer problem with different priorities, the sleeping barber problem, and the cigarette smoker's problem. All of these problems can be implemented in BACI. In this section, we discuss nonstandard semaphore/monitor projects that one can implement in BACI to further aid the understanding of concurrency and synchronization concepts.

***A'S AND B'S AND SEMAPHORES*** For the following program outline in BACI,

```
// global semaphore declarations here
void A()
{
     p()'s and v()'s ONLY
}
void B()
{
     p()'s and v()'s ONLY
}
main()
{
     // semaphore initialization here
     cobegin {
      A(); A(); A(); B(); B();
     }
}
```

complete the program using the least number of general semaphores, such that the processes ALWAYS terminate in the order A (any copy), B (any copy), A (any copy), A, B. Use the -t option of the interpreter to display process termination. (Many variations of this project exist. For example, have four concurrent processes terminated in the order ABAA or eight concurrent processes terminated in the order AABABABB.)

***USING BINARY SEMAPHORES*** Repeat the previous project using binary semaphores. Evaluate why assignment and IF-THEN-ELSE statements are necessary in this solution, although they were not necessary in solutions to the previous project. In other words, explain why you cannot use only Ps and Vs in this case.

***BUSY WAITING VERSUS SEMAPHORES*** Compare the performance of a solution to mutual exclusion that uses busy waiting (e.g., the testset instruction) to a solution that uses semaphores. For example, compare a semaphore solution and a testset solution

to the ABAAB project discussed previously. In each case, use a large number of executions (say, 1000) to obtain better statistics. Discuss your results, explaining why one implementation is preferred over another.

***SEMAPHORES AND MONITORS*** In the spirit of Problem 5.17, implement a monitor using general semaphores, then implement a general semaphore using a monitor in BACI.

***GENERAL AND BINARY SEMAPHORES*** Prove that general semaphores and binary semaphores are equally powerful, by implementing one type of semaphore with the other type of semaphore and vice versa.

***TIME TICKS: A MONITOR PROJECT*** Write a program containing a monitor AlarmClock. The monitor must have an `int` variable `theClock` (initialized to zero) and two functions:

- `Tick()`: This function increments `theClock` each time that it is called. It can do other things, like `signalc`, if needed.
- `int Alarm(int id, int delta)`: This function blocks the caller with identifier `id` for at least `delta` ticks of `theClock`.

The main program should have two functions as well:

- `void Ticker()`: This procedure calls `Tick()` in a repeat-forever loop.
- `void Thread(int id, int myDelta)`: This function calls `Alarm` in a repeat-forever loop.

You may endow the monitor with any other variables that it needs. The monitor should be able to accommodate up to five simultaneous alarms.

***A PROBLEM OF A POPULAR BAKER*** Due to the recent popularity of a bakery, almost every customer needs to wait for service. To maintain service, the baker wants to install a ticket system that will ensure customers are served in turn. Construct a BACI implementation of this ticket system.

## O.5 ENHANCEMENTS TO THE BACI SYSTEM

We have enhanced the BACI System in several ways:

1. We have implemented the BACI system in Java (JavaBACI). This, along with our original C implementation of BACI, is available from: http://inside.mines.edu/fs_home/tcamp/baci/baci_index.html. The JavaBACI classes and source files are stored in self-extracting Java .jar files. JavaBACI includes all BACI applications: C and Pascal compilers, disassembler, archiver, linker, and command-line and GUI PCODE interpreters. The input, behavior, and output of programs in JavaBACI are identical to the input, behavior, and output of programs in our C implementation of BACI; we note that, in JavaBACI, students

continue to write concurrency programs in C−− or Pascal (not Java). JavaBACI will execute on any computer that has an installation of the Java Virtual Machine.

2. We have added graphical user interfaces (GUIs) for JavaBACI and the UNIX version of BACI in C. The windowing environments of these GUIs allow a user to monitor all aspects of the execution of a BACI program; specifically, a user can set and remove breakpoints (either by PCODE address or source line), view variables values, runtime stacks, and process tables, and examine interleaved PCODE execution. The BACI GUIs are available at the BACI GUI Web site http://inside.mines.edu/fs_home/tcamp/baci/index_gui.html. For an alternative GUI, see below.
3. We have created a distributed version of BACI. Similar to concurrent programs, it is difficult to prove the correctness of distributed programs without an implementation. Distributed BACI allows distributed programs to be easily implemented. In addition to proving the correctness of a distributed program, one can use distributed BACI to test the program's performance. Distributed BACI is available at the following website: http://inside.mines.edu/fs_home/tcamp/baci/dbaci.html.
4. We have a PCODE disassembler that will provide the user with an annotated listing of a PCODE file, showing the mnemonics for each PCODE instruction and, if available, the corresponding program source that generated the instruction. This PCODE disassembler is included in the BACI System.
5. We have added the capability of separate compilation and external variables to both compilers (C and Pascal). The BACI System includes an archiver and a linker that enable the creation and use of libraries of BACI PCODE. For more details, see the BACI Separate Compilation User's Guide.

The BACI system has also been enhanced by others.

1. David Strite, an M.S. student who worked with Linda Null from the Pennsylvania State University, created a BACI Debugger: A GUI Debugger for the BACI System. This GUI is available at http://cs.hbg.psu.edu/~null/baci.
2. Using BACI and the BACI GUI from Pennsylvania State University, Moti Ben-Ari from the Weizmann Institute of Science in Israel created an integrated development environment for learning concurrent programming by simulating concurrency called jBACI. jBACI is available at: https://code.google.com/archive/p/jbaci/.

# APPENDIX P

## PROCEDURE CONTROL

P.1 **Stack Implementation**

P.2 **Procedure Calls and Returns**

P.3 **Reentrant Procedures**

A common technique for controlling the execution of procedure calls and returns makes use of a stack. This appendix summarizes the basic properties of stacks and looks at their use in procedure control.

## P.1 STACK IMPLEMENTATION

A stack is an ordered set of elements, only one of which (the most recently added) can be accessed at a time. The point of access is called the *top* of the stack. The number of elements in the stack, or length of the stack, is variable. Items may only be added to or deleted from the top of the stack. For this reason, a stack is also known as a *pushdown list* or a *last-in-first-out (LIFO) list*.

The implementation of a stack requires that there be some set of locations used to store the stack elements. A typical approach is illustrated in Figure P.1. A contiguous block of locations is reserved in main memory (or virtual memory) for the stack. Most of the time, the block is partially filled with stack elements and the remainder is available for stack growth. Three addresses are needed for proper operation, and these are often stored in processor registers:

- **Stack pointer:** Contains the address of the current top of the stack. If an item is appended to (PUSH) or deleted from (POP) the stack, the pointer is decremented or incremented to contain the address of the new top of the stack.

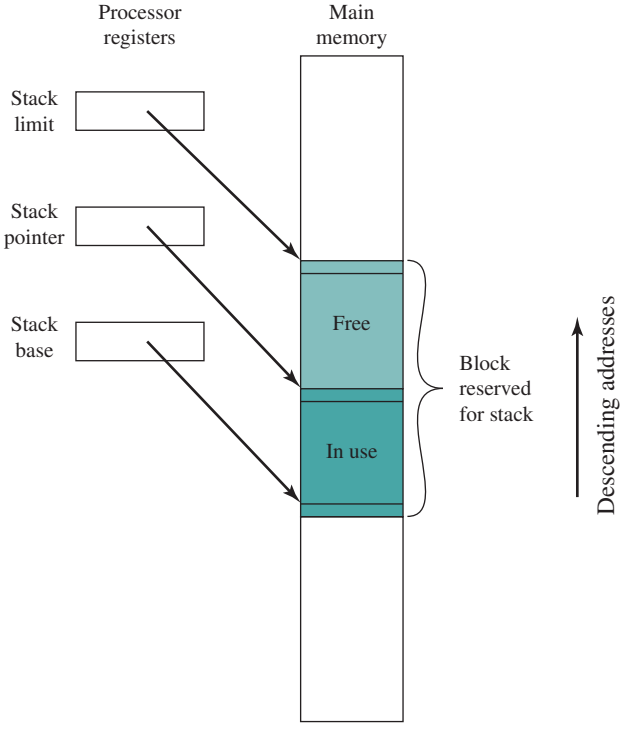



**Figure P.1 Typical Stack Organization (full/descending)**

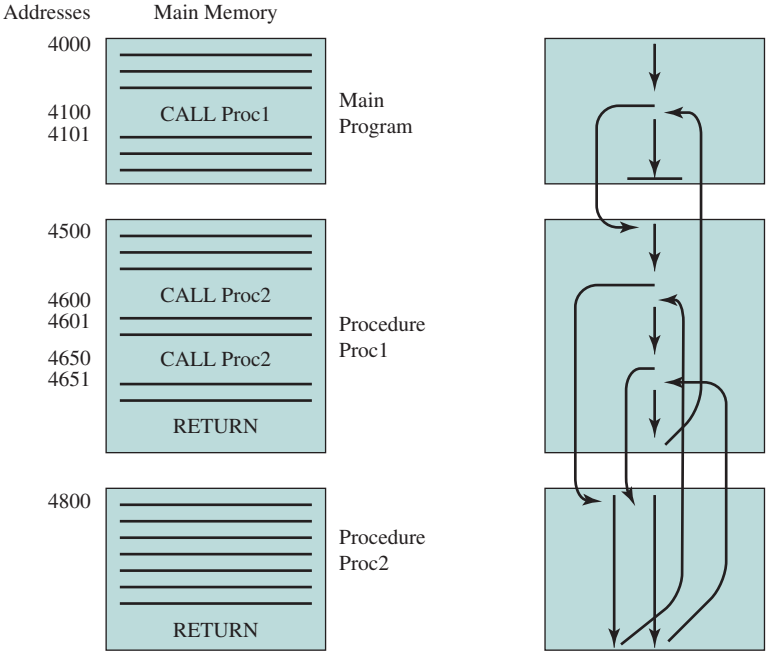
- **Stack base:** Contains the address of the bottom location in the reserved block. This is the first location to be used when an item is added to an empty stack. If an attempt is made to POP an element when the stack is empty, an error is reported.
- **Stack limit:** Contains the address of the other end, or top, of the reserved block. If an attempt is made to PUSH an element when the stack is full, an error is reported.

Traditionally, and on most processors today, the base of the stack is at the high-address end of the reserved stack block, and the limit is at the low-address end. Thus, the stack grows from higher addresses to lower addresses.

## P.2 PROCEDURE CALLS AND RETURNS

A common technique for managing procedure calls and returns makes use of a stack. When the processor executes a call, it places (pushes) the return address on the stack. When it executes a return, it uses the address on top of the stack and removes (pops) that address from the stack. For the nested procedures of Figure P.2, Figure P.3 illustrates the use of a stack.

It is also often necessary to pass parameters with a procedure call. These could be passed in registers. Another possibility is to store the parameters in memory just




(a) Calls and returns (b) Execution sequence

**Figure P.2 Nested Procedures**

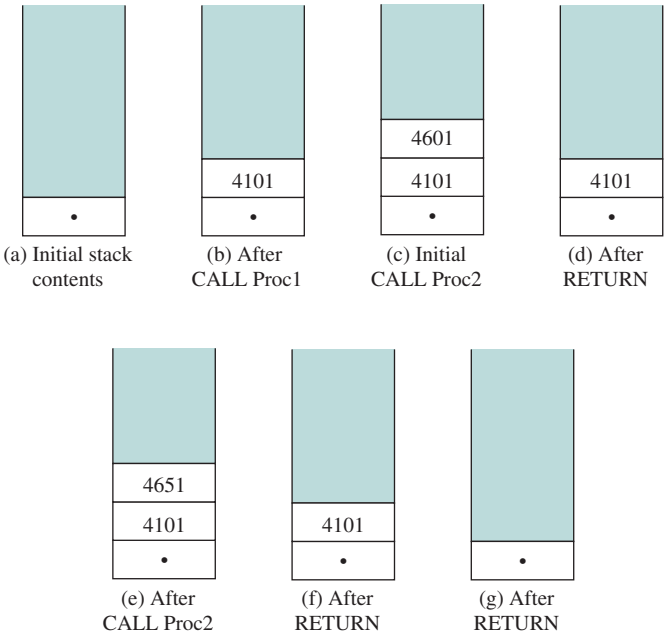



**Figure P.3 Use of Stack to Implement Nested Procedures of Figure P.2**

after the Call instruction. In this case, the return must be to the location following the parameters. Both of these approaches have drawbacks. If registers are used, the called program and the calling program must be written to assure that the registers are used properly. The storing of parameters in memory makes it difficult to exchange a variable number of parameters.

A more flexible approach to parameter passing is the stack. When the processor executes a call, it not only stacks the return address, it stacks parameters to be passed to the called procedure. The called procedure can access the parameters from the stack. Upon return, return parameters can also be placed on the stack, *under* the return address. The entire set of parameters, including return address, that is stored for a procedure invocation is referred to as a **stack frame**.

An example is provided in Figure P.4. The example refers to procedure P in which the local variables $x1$ and $x2$ are declared, and procedure Q, which can be called by P and in which the local variables $y1$ and $y2$ are declared. The first item stored in each stack frame is a pointer to the beginning of the previous frame. This is needed if the number or length of parameters to be stacked is variable. Next is stored the return point for the procedure that corresponds to this stack frame. Finally, space is allocated at the top of the stack frame for local variables. These local variables can be used for parameter passing. For example, suppose when P calls Q, it passes one parameter value. This value could be stored in variable $y1$. Thus, in a high-level language, there would be an instruction in the P routine that looks like this:

CALL Q($y1$)

When this call is executed, a new stack frame is created for Q (see Figure P.4b), which includes a pointer to the stack frame for P, the return address to P, and two local

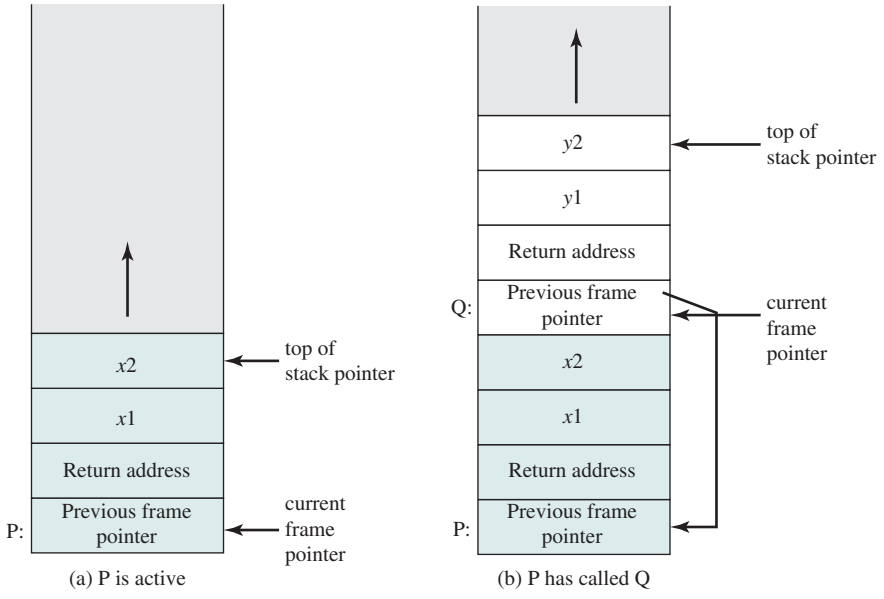



**Figure P.4 Stack Frame Growth Using Sample Procedures P and Q**

variables for Q, one of which is initialized to the passed parameter value from P. The other local variable, *y*2, is simply a local variable used by Q in its calculations. The need to include such local variables in the stack frame is discussed in the next subsection.

## P.3 REENTRANT PROCEDURES

A useful concept, particularly in a system that supports multiple users at the same time, is that of the reentrant procedure. A reentrant procedure is one in which a single copy of the program code can be shared by multiple users during the same period of time. Reentrancy has two key aspects: The program code cannot modify itself and the local data for each user must be stored separately. A reentrant procedure can be interrupted and called by an interrupting program and still execute correctly upon return to the procedure. In a shared system, reentrancy allows more efficient use of main memory: One copy of the program code is kept in main memory, but more than one application can call the procedure.

Thus, a reentrant procedure must have a permanent part (the instructions that make up the procedure) and a temporary part (a pointer back to the calling program as well as memory for local variables used by the program). Each execution instance, called activation, of a procedure will execute the code in the permanent part but must have its own copy of local variables and parameters. The temporary part associated with a particular activation is referred to as an *activation record*.

The most convenient way to support reentrant procedures is by means of a stack. When a reentrant procedure is called, the activation record of the procedure can be stored on the stack. Thus, the activation record becomes part of the stack frame that is created on procedure call.

# APPENDIX Q

## ECOS


**Q.1 Configurability**

**Q.2 Ecos Components**
- Hardware Abstraction Layer (HAL)
- eCos Kernel
- I/O System
- Standard C Libraries

**Q.3 Ecos Scheduler**
- Bitmap Scheduler
- Multilevel Queue Scheduler

**Q.4 Ecos Thread Synchronization**
- Mutexes
- Semaphores
- Condition Variables
- Event Flags
- Mailboxes
- Spinlocks

The Embedded Configurable Operating System (eCos) is an open source, royalty-free, real-time OS intended for embedded applications. The system is targeted at high-performance small embedded systems. For such systems, an embedded form of Linux or other commercial OS would not provide the streamlined software required. The eCos software has been implemented on a wide variety of processor platforms, including Intel IA32, PowerPC, SPARC, ARM, CalmRISC, MIPS, and NEC V8xx. It is one of the most widely used embedded operating systems. It is implemented in C/C++.

## Q.1 CONFIGURABILITY

An embedded OS that is flexible enough to be used in a wide variety of embedded applications and on a wide variety of embedded platforms must provide more functionality than will be needed for any particular application and platform. For example, many real-time operating systems support task switching, concurrency controls, and a variety of priority scheduling mechanisms. A relatively simple embedded system would not need all these features.

The challenge is to provide an efficient, user-friendly mechanism for configuring selected components and for enabling and disabling particular features within components. The eCos configuration tool, which runs on Windows or Linux, is used to configure an eCos package to run on a target embedded system. The complete eCos package is structured hierarchically, making it easy (using the configuration tool) to assemble a target configuration. At a top level, eCos consists of a number of components, and the configuration user may select only those components needed for the target application. For example, a system might have a particular serial I/O device. The configuration user would select serial I/O for this configuration, then select one or more specific I/O devices to be supported. The configuration tool would include the minimum necessary software for that support. The configuration user can also select specific parameters, such as default data rate and the size of I/O buffers to be used.

This configuration process can be extended down to finer levels of detail, even to the level of individual lines of code. For example, the configuration tool provides the option of including or omitting a priority inheritance protocol.

Figure Q.1 shows the top level of the eCos configuration tool as seen by the tool user. Each of the items on the list in the left-hand window can be selected or deselected. When an item is highlighted, the lower right-hand window provides a description, and the upper right-hand window provides a link to further documentation plus additional information about the highlighted item. Items on the list can be expanded to provide a finer-grained menu of options. Figure Q.2 illustrates an expansion of the eCos kernel option. In this figure, note exception handling has been selected for inclusion, but SMP (symmetric multiprocessing) has been omitted. In general, components and individual options can be selected or omitted. In some cases, individual values can be set; for example, a minimum acceptable stack size is an integer value that can be set or left to a default value.

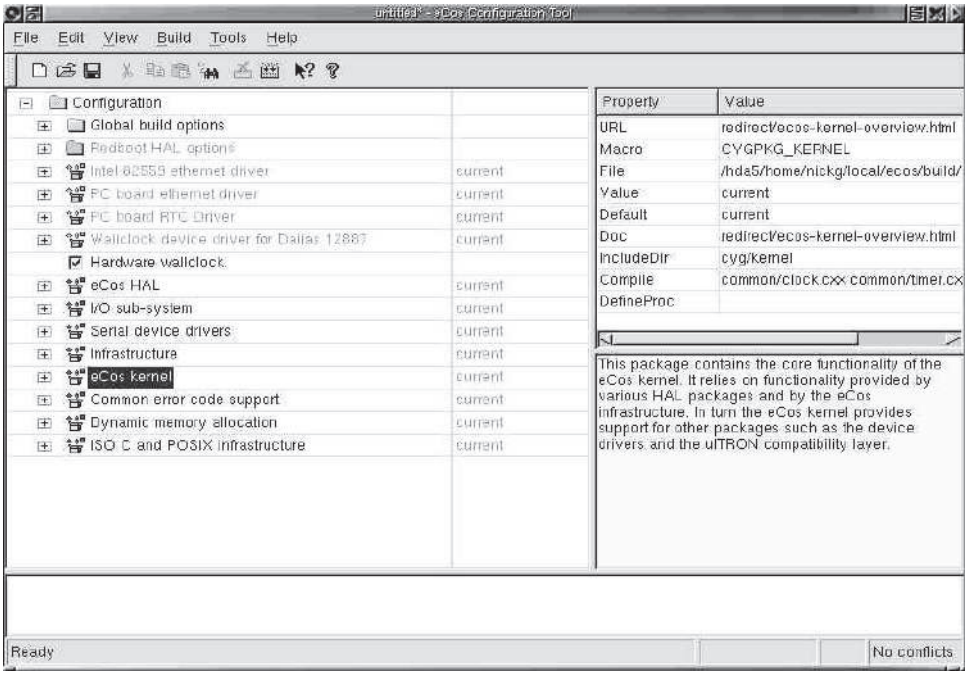



**Figure Q.1** eCos Configuration Tool - Top Level. Courtesy of eCosCentric Limited. Used with permission.

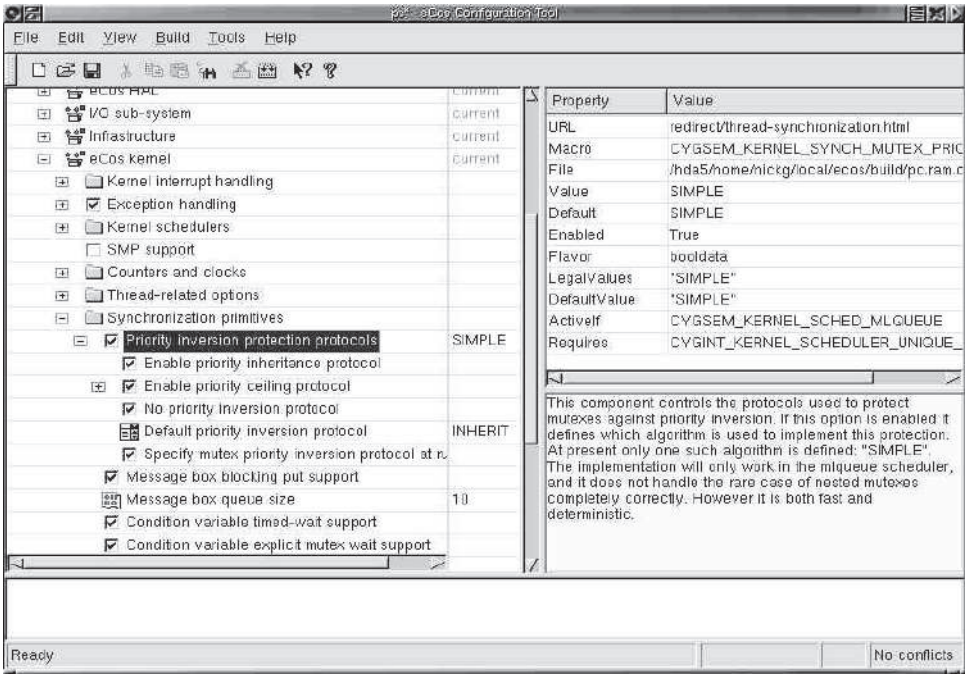



**Figure Q.2** eCos Configuration Tool - Kernel Details. Courtesy of eCosCentric Limited. Used with permission.

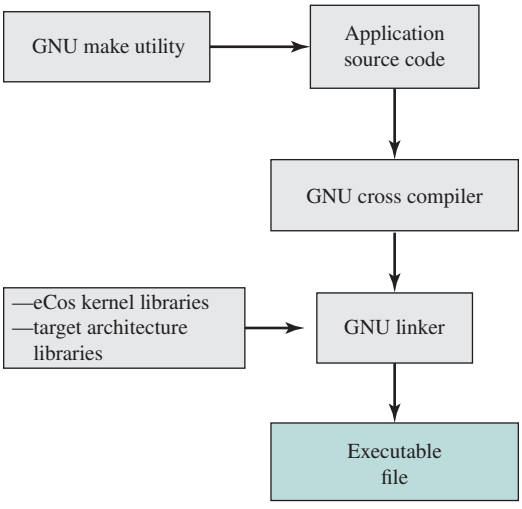



**Figure Q.3 Loading an eCos Configuration**

Figure Q.3 shows a typical example of the overall process of creating the binary image to execute in the embedded system. This process is run on a source system, such as a Windows or Linux platform, and the executable image is destined to execute on a target embedded system, such as a sensor in an industrial environment. At the highest software level is the application source code for the particular embedded application. This code is independent of eCos but makes use of application programming interfaces (API) to sit on top of the eCos software. There may be only one version of the application source code, or there may be variations for different versions of the target embedded platform. In this example, the GNU make utility is used to selectively determine which pieces of a program need to be compiled or recompiled (in the case of a modified version of the source code) and issues the commands to recompile them. The GNU cross compiler, executing on the source platform, then generates the binary executable code for the target embedded platform. The GNU linker links the application object code with the code generated by the eCos configuration tool. This latter set of software includes selected portions of the eCos kernel plus selected software for the target embedded system. The result can then be loaded into the target system.

## Q.2 ECOS COMPONENTS

A key design requirement for eCos is portability to different architectures and platforms with minimal effort. To meet this requirement, eCos consists of a layered set of components (see Figure Q.4).

### Hardware Abstraction Layer (HAL)

At the bottom is the hardware abstraction layer (HAL). The HAL is software that presents a consistent API to the upper layers and maps upper-layer operations onto a specific hardware platform. Thus, the HAL is different for each hardware

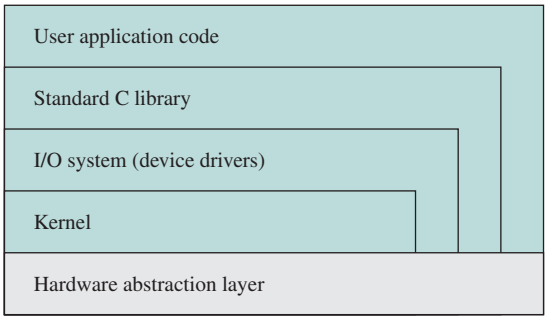



**Figure Q.4 eCos Layered Structure**

platform. Figure Q.5 is an example that demonstrates how the HAL abstracts hardware-specific implementations for the same API call on two different platforms. As this example shows, the call from an upper layer to enable interrupts is the same on both platforms, but the C code implementation of the function is specific to each platform.

```
1 #define HAL_ENABLE_INTERRUPTS()          \
2     asm volatile (                       \
3         "mrs r3, cpsr;"                  \
4         "bic r3, r3, #0xC0;"             \
5         "mrs cpsr, r3;"                  \
6         :                                \
7         :                                \
8         :  "r3"                          \
9         );                               \
```

**(a) ARM architecture**

```
1 #define HAL_ENABLE_INTERRUPTS()          \
2     CYG_MACRO_START                      \
3     cyg_uint32 tmp1, tmp2                \
4     asm volatile (                       \
5         "mfmsr   %0;"                    \
6         "ori     %1,%1,0x800;"           \
7         "r1wimi  %0,%1,0,16,16;"         \
8         "mtmsr   %0;"                    \
9         : "=r" (tmp1), "=r" (tmp2));     \
10    CYG_MACRO_END                        \
```

**(b) PowerPC architecture**

**Figure Q.5 Two Implementations of HAL_ENABLE_INTERRUPTS() Macro**

The HAL is implemented as three separate modules:

- **Architecture:** Defines the processor family type. This module contains the code necessary for processor startup, interrupt delivery, context switching, and other functionality specific to the instruction set architecture of that processor family.
- **Variant:** Supports the features of the specific processor in the family. An example of a supported feature is an on-chip module such as a memory management unit (MMU).
- **Platform:** Extends the HAL support to tightly coupled peripherals such as interrupt controllers and timer devices. This module defines the platform or board that includes the selected processor architecture and variant. It includes code for startup, chip selection configuration, interrupt controllers, and timer devices.

Note the HAL interface can be directly used by any of the upper layers, promoting efficient code.

## eCos Kernel

The eCos kernel was designed to satisfy four main objectives:

- **Low interrupt latency:** The time it takes to respond to an interrupt and begin executing an ISR.
- **Low task switching latency:** The time it takes from when a thread becomes available to when actual execution begins.
- **Small memory footprint:** Memory resources for both program and data are kept to a minimum by allowing all components to configure memory as needed.
- **Deterministic behavior:** Throughout all aspect of execution, the kernels performance must be predictable and bounded to meet real-time application requirements.

The eCos kernel provides the core functionality needed for developing multithreaded applications:

1. The ability to create new threads in the system, either during startup or when the system is already running
2. Control over the various threads in the system: for example, manipulating their priorities
3. A choice of schedulers, determining which thread should currently be running
4. A range of synchronization primitives, allowing threads to interact and share data safely
5. Integration with the system's support for interrupts and exceptions

Some functionality that is typically included in the kernel of an OS is not included in the eCos kernel. For example, memory allocation is handled by a separate package. Similarly, each device driver is a separate package. Various packages are

combined and configured using the eCos configuration technology to meet the requirements of the application. This makes for a lean kernel. Further, the minimal nature of the kernel means that for some embedded platforms, the eCos kernel is not used at all. Simple single-threaded applications can be run directly on HAL. Such configurations can incorporate needed C library functions and device drivers, but avoid the space and time overhead of the kernel.

There are two different techniques for utilizing kernel functions in eCos. One way to employ kernel functionality is by using the C API of kernel. Examples of such functions are `cyg_thread_create` and `cyg_mutex_lock`. These functions can be invoked directly from application code. On the other hand, kernel functions can also be invoked by using compatibility packages for existing API's, for example, POSIX threads or μITRON. The compatibility packages allow application code to call standard functions like `pthread_create`, and those functions are implemented using the basic functions provided by the eCos kernel. Code sharing and reusability of already developed code is easily achieved by use of compatibility packages.

## I/O System

The eCos I/O system is a framework for supporting device drivers. A variety of drivers for a variety of platforms are provided in the eCos configuration package. These include drivers for serial devices, Ethernet, flash memory interfaces, and various I/O interconnects such as PCI (Peripheral Component Interconnect) and USB (Universal Serial Bus). In addition, users can develop their own device drivers.

The principal objective for the I/O system is efficiency, with no unnecessary software layering or extraneous functionality. Device drivers provide the necessary functions for input, output, buffering, and device control.

As mentioned, device drivers and other higher-layer software may be implemented directly on the HAL if this is appropriate. If specialized kernel-type functions are needed, then the device driver is implemented using kernel APIs. The kernel provides a three-level interrupt model:

- **Interrupt service routines (ISRs):** Invoked in response to a hardware interrupt. Hardware interrupts are delivered with minimal intervention to an ISR. The HAL decodes the hardware source of the interrupt and calls the ISR of the attached interrupt object. This ISR may manipulate the hardware but is only allowed to make a restricted set of calls on the driver API. When it returns, an ISR may request that its deferred service routine (DSR) should be scheduled to run.
- **Deferred service routines (DSRs):** Invoked in response to a request by an ISR. A DSR will be run when it is safe to do so without interfering with the scheduler. Most of the time the DSR will run immediately after the ISR, but if the current thread is in the scheduler, it will be delayed until the thread is finished. A DSR is allowed to make a larger set of driver API calls, including, in particular, being able to call `cyg_drv_cond_signal()` to wake up waiting threads.
- **Threads:** The clients of the driver. Threads are able to make all API calls and in particular are allowed to wait on mutexes and condition variables.

Tables Q.1 and Q.2 show the device driver interface to the kernel. These tables give a good feel for the type of functionality available in the kernel to support device

**Table Q.1** Device Driver Interface to the eCos Kernel: Concurrency

| |
|---|
| `cyg_drv_spinlock_init` Initialize a spinlock in a locked or unlocked state.<br>`cyg_drv_spinlock_destroy` Destroy a spinlock that is no longer of use.<br>`cyg_drv_spinlock_spin` Claim a spinlock, waiting in a busy loop until it is available.<br>`cyg_drv_spinlock_clear` Clear a spinlock. This clears the spinlock and allows another CPU to claim it. If there is more than one CPU waiting in `cyg_drv_spinlock_spin`, then just one of them will be allowed to proceed.<br>`cyg_drv_spinlock_test` Inspect the state of the spinlock. If the spinlock is not locked, then the result is TRUE. If it is locked then the result will be FALSE.<br>`cyg_drv_spinlock_spin_intsave` This function behaves like `cyg_drv_spinlock_spin` except that it also disables interrupts before attempting to claim the lock. The current interrupt enable state is saved in *istate. Interrupts remain disabled once the spinlock has been claimed and must be restored by calling `cyg_drv_spinlock_clear_intsave`. Device drivers should use this function to claim and release spinlocks rather than the `non-_intsave()` variants, to ensure proper exclusion with code running on both other CPUs and this CPU. |
| `cyg_drv_mutex_init` Initialize a mutex.<br>`cyg_drv_mutex_destroy` Destroy a mutex. The mutex should be unlocked and there should be no threads waiting to lock it when this call is made.<br>`cyg_drv_mutex_lock` Attempt to lock the mutex pointed to by the mutex argument. If the mutex is already locked by another thread, then this thread will wait until that thread is finished. If the result from this function is FALSE, then the thread was broken out of its wait by some other thread. In this case the mutex will not have been locked.<br>`cyg_drv_mutex_trylock` Attempt to lock the mutex pointed to by the mutex argument without waiting. If the mutex is already locked by some other thread then this function returns FALSE. If the function can lock the mutex without waiting, then TRUE is returned.<br>`cyg_drv_mutex_unlock` Unlock the mutex pointed to by the mutex argument. If there are any threads waiting to claim the lock, one of them is woken up to try and claim it.<br>`cyg_drv_mutex_release` Release all threads waiting on the mutex. |
| `cyg_drv_cond_init` Initialize a condition variable associated with a mutex. A thread may only wait on this condition variable when it has already locked the associated mutex. Waiting will cause the mutex to be unlocked, and when the thread is reawakened, it will automatically claim the mutex before continuing.<br>`cyg_drv_cond_destroy` Destroy the condition variable.<br>`cyg_drv_cond_wait` Wait for a signal on a condition variable.<br>`cyg_drv_cond_signal` Signal a condition variable. If there are any threads waiting on this variable, at least one of them will all be awakened.<br>`cyg_drv_cond_broadcast` Signal a condition variable. If there are any threads waiting on this variable, they will all be awakened. |

**Table Q.2** Device Driver Interface to the eCos Kernel: Interrupts

| |
|---|
| `cyg_drv_isr_lock` Disable delivery of interrupts, preventing all ISRs running. This function maintains a counter of the number of times it is called.<br>`cyg_drv_isr_unlock` Reenable delivery of interrupts, allowing ISRs to run. This function decrements the counter maintained by `cyg_drv_isr_lock`, and only reallows interrupts when it goes to zero.<br>`cyg_ISR_t` Define ISR. |
| `cyg_drv_dsr_lock` Disable scheduling of DSRs. This function maintains a counter of the number of times it has been called.<br>`cyg_drv_dsr_unlock` Reenable scheduling of DSRs. This function decrements the counter incremented by `cyg_drv_dsr_lock`. DSRs are only allowed to be delivered when the counter goes to zero.<br>`cyg_DSR_t Define` DSR prototype. |
| `cyg_drv_interrupt_create` Create an interrupt object and returns a handle to it.<br>`cyg_drv_interrupt_delete` Detach the interrupt from the vector and free the memory for reuse.<br>`cyg_drv_interrupt_attach` Attach an interrupt to a vector so that interrupts will be delivered to the ISR when the interrupt occurs.<br>`cyg_drv_interrupt_detach` Detach the interrupt from the vector so that interrupts will no longer be delivered to the ISR.<br>`cyg_drv_interrupt_mask` Program the interrupt controller to stop delivery of interrupts on the given vector.<br>`cyg_drv_interrupt_mask_intunsafe` Program the interrupt controller to stop delivery of interrupts on the given vector. This version differs from `cyg_drv_interrupt_mask` in not being interrupt safe. So in situations where, for example, interrupts are already known to be disabled, this may be called to avoid the extra overhead.<br>`cyg_drv_interrupt_unmask`, `cyg_drv_interrupt_unmask_intunsafe` Program the interrupt controller to reallow delivery of interrupts on the given vector.<br>`cyg_drv_interrupt_acknowledge` Perform any processing required at the interrupt controller and in the CPU to cancel the current interrupt request.<br>`cyg_drv_interrupt_configure` Program the interrupt controller with the characteristics of the interrupt source.<br>`cyg_drv_interrupt_level` Program the interrupt controller to deliver the given interrupt at the supplied priority level.<br>`cyg_drv_interrupt_set_cpu` On multiprocessor systems, this function causes all interrupts on the given vector to be routed to the specified CPU. Subsequently, all such interrupts will be handled by that CPU.<br>`cyg_drv_interrupt_get_cpu` On multiprocessor systems, this function returns the ID of the CPU to which interrupts on the given vector are currently being delivered. |

drivers. Note the device driver interface can be configured for one or more of the following concurrency mechanisms: spinlocks, condition variables, and mutexes. These are described in a subsequent portion of this discussion.

### Standard C Libraries

A complete Standard C run-time library is provided. Also included is a complete math run-time library for high-level mathematics functions, including a complete IEEE-754 floating-point library for those platforms without hardware floating points.

## Q.3 ECOS SCHEDULER

The eCos kernel can be configured to provide one of two scheduler designs: the bitmap scheduler and a multilevel queue scheduler. The configuration user selects the appropriate scheduler for the environment and the application. The bitmap scheduler provides efficient scheduling for a system with a small number of threads that may be active at any point in time. The multiqueue scheduler is appropriate if the number of threads is dynamic or if it is desirable to have multiple threads at the same priority level. The multilevel scheduler is also needed if time slicing is desired.

### Bitmap Scheduler

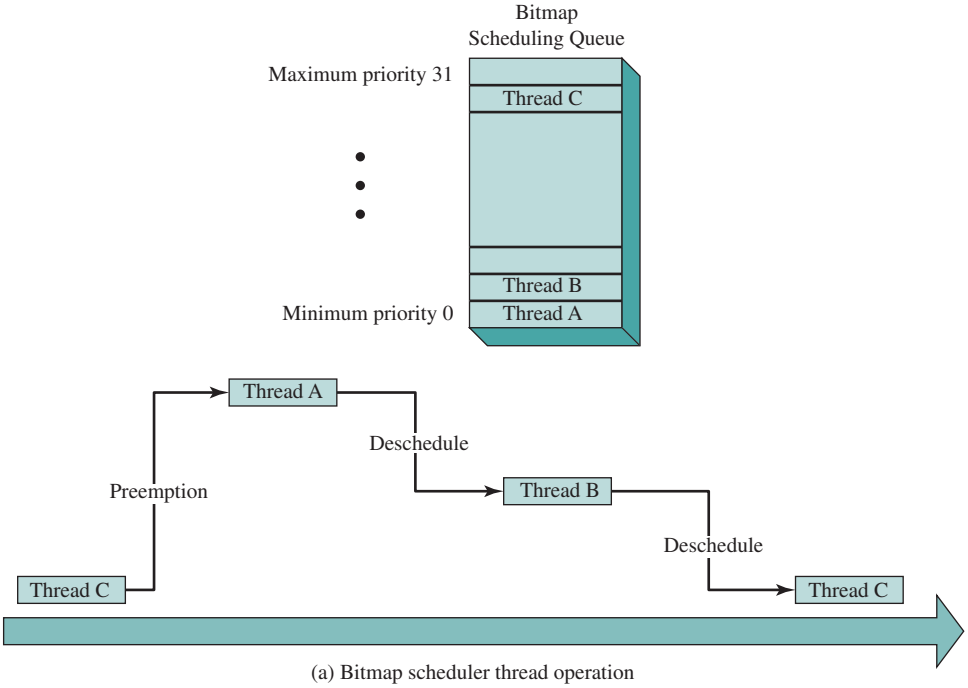
A bitmap scheduler supports multiple priority levels, but only one thread can exist at each priority level at any given time. Scheduling decisions are quite simple with this scheduler (see Figure Q.6a). When a blocked thread become ready to run, it may preempt a thread of lower priority. When a running thread suspends, the ready thread with the highest priority is dispatched. A thread can be suspended because it is blocked on a synchronization primitive, because it is interrupted, or because it relinquishes control. Because there is only one thread, at most, at each priority level, the scheduler does not have to make a decision as to which thread at a given priority level should be dispatched next.

The bitmap scheduler is configured with 8, 16, or 32 priority levels. A simple bitmap is kept of the threads that are ready to execute. The scheduler need only to determine the position of the most significant one bit in the bitmap to make a scheduling decision.

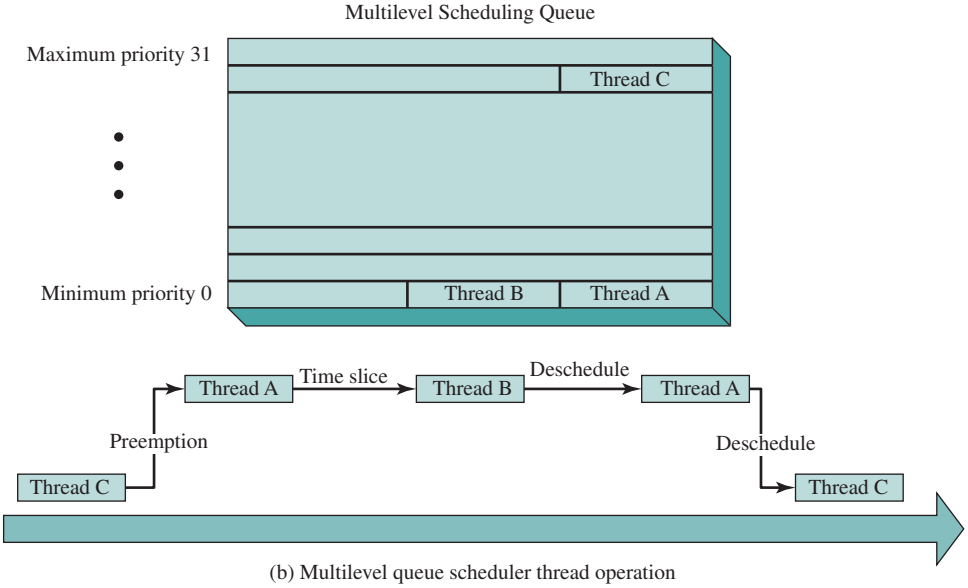
### Multilevel Queue Scheduler

As with the bitmap scheduler, the multilevel queue scheduler supports up to 32 priority levels. The multilevel queue scheduler allows for multiple active threads at each priority level, limited only by system resources.

Figure Q.6b illustrates the nature of the multilevel queue scheduler. A data structure represents the number of ready threads at each priority level. When a blocked thread become ready to run, it may preempt a thread of lower priority. As with the bitmap scheduler, a running thread may be blocked on a synchronization primitive, because it is interrupted, or because it relinquishes control. When




(a) Bitmap scheduler thread operation




(b) Multilevel queue scheduler thread operation

**Figure Q.6 eCos Scheduler Options**

a thread is blocked, the scheduler must first determine if one or more threads at the same priority level as the blocked thread is ready. If so, the scheduler chooses the one at the front of the queue. Otherwise, the scheduler looks for the next highest priority level with one or more ready threads and dispatches one of these threads.

In addition, the multilevel queue scheduler can be configured for time slicing. Thus, if a thread is running and there is one or more ready threads at the same priority level, the scheduler will suspend the running thread after one time slice and choose the next thread in the queue at that priority level. This is a round-robin policy within one priority level. Not all applications require time slicing. For example, an application may contain only threads that block regularly for some other reason. For these applications, the user can disable time slicing, which reduces the overhead associated with timer interrupts.

## Q.4 ECOS THREAD SYNCHRONIZATION

The eCos kernel can be configured to include one or more of six different thread synchronization mechanisms. These include the classic synchronization mechanisms: mutexes, semaphores, and condition variables. In addition, eCos supports two synchronization/communication mechanisms that are common in real-time systems, namely event flags and mailboxes. Finally, the eCos kernel supports spinlocks, which are useful in SMP (symmetric multiprocessing) systems.

### Mutexes

The mutex (mutual exclusion lock) was introduced in Chapter 6. Recall that a mutex is used to enforce mutually exclusive access to a resource, allowing only one thread at a time to gain access. The mutex has only two states: locked and unlocked. This is similar to a binary semaphore: When a mutex is locked by one thread, any other thread attempting to lock the mutex is blocked; when the mutex is unlocked, then one of the threads blocked on this mutex is unblocked and allowed to lock the mutex and gain access to the resource.

The mutex differs from a binary semaphore in two respects. First, the thread that locks the mutex must be the one to unlock it. In contrast, it is possible for one thread to lock a binary semaphore and for another to unlock it. The other difference is that a mutex provides protection against priority inversion, whereas a semaphore does not.

The eCos kernel can be configured to support either a priority inheritance protocol or a priority ceiling protocol. These are described in Chapter 10.

### Semaphores

The eCos kernel provides support for a counting semaphore. Recall from Chapter 5 that a counting semaphore is an integer value used for signaling among threads. The `cyg_semaphore_init` is used to initialize a semaphore. The `cyg_semaphore_post` command increments the semaphore count when an event occurs. If

the new count is less than or equal to zero, then a thread is waiting on this semaphore and is awakened. The `cyg_semaphore_wait` function checks the value of a semaphore count. If the count is zero, the thread calling this function will wait for the semaphore. If the count is nonzero, the count is decremented and the thread continues.

Counting semaphores are suited to enabling threads to wait until an event has occurred. The event may be generated by a producer thread, or by a DSR in response to a hardware interrupt. Associated with each semaphore is an integer counter that keeps track of the number of events that have not yet been processed. If this counter is zero, an attempt by a consumer thread to wait on the semaphore will block until some other thread or a DSR posts a new event to the semaphore. If the counter is greater than zero, then an attempt to wait on the semaphore will consume one event (in other words decrement the counter) and return immediately. Posting to a semaphore will wake up the first thread that is currently waiting, which will then resume inside the semaphore wait operation and decrement the counter again.

Another use of semaphores is for certain forms of resource management. The counter would correspond to how many of a certain type of resource are currently available, with threads waiting on the semaphore to claim a resource and posting to release the resource again. In practice, condition variables are usually much better suited for operations like this.

## Condition Variables

A condition variable is used to block a thread until a particular condition is true. Condition variables are used with mutexes to allow multiple threads to access shared data. They can be used to implement monitors of the type discussed in Chapter 6 (e.g., Figure 6.14). The basic commands are as follows:

`cyg_cond_wait` Causes the current thread to wait on the specified condition variable and simultaneously unlocks the mutex attached to the condition variable.

`cyg_cond_signal` Wakes up one of the threads waiting on this condition variable, causing that thread to become the owner of the mutex.

`cyg_cond_broadcast` Wakes up all the threads waiting on this condition variable. Each thread that was waiting on the condition variable becomes the owner of the mutex when it runs.

In eCos, condition variables are typically used in conjunction with mutexes to implement long-term waits for some condition to become true. Consider the following example. Figure Q.7 defines a set of functions to control access to a pool of resources using mutexes. The mutex is used to make the allocation and freeing of resources from a pool atomic. The function `res_t res_allocate` checks to see if one or more units of a resource are available and, if so, takes one unit. This operation is protected by a mutex so no other thread can check or alter the resource pool while this thread has control of the mutex. The function `res_free(res_t res)` enables a thread to release one unit of a resource that it had previously acquired. Again, this operation is made atomic by a mutex.

In this example, if a thread attempts to access a resource and none are available, the function returns `RES_NONE`. Suppose, however, we want the thread to be blocked and wait for a resource to become available, rather than returning `RES_NONE`.

```
cyg_mutex_t res_lock;
res_t res_pool[RES_MAX];
int res_count = RES_MAX;

void res_init(void)
{
    cyg_mutex_init(&res_lock);
     <fill pool with resources>
}
res_t res_allocate(void)
{
    res_t res;
    cyg_mutex_lock(&res_lock);        // lock the mutex
    if( res_count == 0 )              // check for free resource
        res =RES_NONE;                // return RES_NONE if none
    else  {
        res_count--;                  // allocate a resources
        res =res_pool[res_count];
}
    cyg_mutex_unlock(&res_lock);      // unlock the mutex
    return res;
}
void res_free(res_t res)
{
    cyg_mutex_lock(&res_lock);        // lock the mutex
    res_pool[res_count] =res;         // free the resource
    res_count++;
    cyg_mutex_unlock(&res_lock);      // unlock the mutex
}
```

**Figure Q.7 Controlling Access to a Pool of Resources Using Mutexes**

Figure Q.8 accomplishes this with the use of a condition variable associated with the mutex. When `res_allocate` detects that there are no resources, it calls `cyg_cond_wait`. This latter function unlocks the mutex and puts the calling thread to sleep on the condition variable. When `res_free` is eventually called, it puts a resource back into the pool and calls `cyg_cond_signal` to wake up any thread waiting on the condition variable. When the waiting thread eventually gets to run again, it will relock the mutex before returning from `cyg_cond_wait`.

There are two significant features of this example, and of the use of condition variables in general. First, the mutex unlock and wait in `cyg_cond_wait` are atomic: No other thread can run between the unlock and the wait. If this were not the case, then a call to `res_free` by some other thread would release the resource, but the call to `cyg_cond_signal` would be lost, and the first thread would end up waiting when there were resources available.

```
cyg_mutex_t res_lock;
cyg_cond_t res_wait;
res_t res_pool[RES_MAX];
int res_count =RES_MAX;
void res_init(void)
{
    cyg_mutex_init(&res_lock);
    cyg_cond_init(&res_wait, &res_lock);
    <fill pool with resources>
}
res_t res_allocate(void)
{
    res_t res;
    cyg_mutex_lock(&res_lock);        // lock the mutex
    while( res_count == 0 )           // wait for a resources
       cyg_cond_wait(&res_wait);
    res_count--;                      // allocate a resource
    res =res_pool[res_count];
    cyg_mutex_unlock(&res_lock);      // unlock the mutex
    return res;
}
void res_free(res_t res)
{
    cyg_mutex_lock(&res_lock);         // lock the mutex
    res_pool[res_count] =res;          // free the resource
    res_count++;
    cyg_cond_signal(&res_wait);        // wake up any waiting
                                          allocators
    cyg_mutex_unlock(&res_lock);       // unlock the mutex
}
```

**Figure Q.8 Controlling Access to a Pool of Resources Using Mutexes and Condition Variables**

The second feature is that the call to `cyg_cond_wait` is in a `while` loop and not a simple `if` statement. This is because of the need to relock the mutex in `cyg_cond_wait` when the signaled thread reawakens. If there are other threads already queued to claim the lock, then this thread must wait. Depending on the scheduler and the queue order, many other threads may have entered the critical section before this one gets to run. So the condition that it was waiting for may have been rendered false. Using a loop around all condition variable wait operations is the only way to guarantee that the condition being waited for is still true after waiting.

## Event Flags

An event flag is a 32-bit word used as a synchronization mechanism. Application code may associate a different event with each bit in a flag. A thread can wait for either a single event or a combination of events by checking one or multiple bits in the

corresponding flag. The thread is blocked until all of the required bits are set (AND) or until at least one of the bits is set (OR). A signaling thread can set or reset bits based on specific conditions or events so that the appropriate thread is unblocked. For example, bit 0 could represent completion of a specific I/O operation, making data available, and bit 1 could indicate that the user has pressed a start button. A producer thread or DSR could set these two bits, and a consumer thread waiting on these two events will be woken up.

A thread can wait on one or more events using the `cyg_flag_wait` command, which takes three arguments: a particular event flag, a combination of bit positions in the flag, and a mode parameter. The mode parameter specifies whether the thread will block until all the bits are set (AND) or until at least one of the bits is set (OR). The mode parameter may also specify that when the wait succeeds, the entire event flag is cleared (set to all zeros).

## Mailboxes

Mailboxes, also called message boxes, are an eCos synchronization mechanism that provides a means for two threads to exchange information. Section 5.5 provides a general discussion of message-passing synchronization. Here, we look at the specifics of the eCos version.

The eCos mailbox mechanism can be configured for blocking or nonblocking on both the send and receive side. The maximum size of the message queue associated with a given mailbox can also be configured.

The send message primitive, called put, includes two arguments: a handle to the mailbox and a pointer for the message itself. There are three variants to this primitive:

`cyg_mbox_put` If there is a spare slot in the mailbox, then the new message is placed there; if there is a waiting thread, it will be woken up so it can receive the message. If the mailbox is currently full, `cyg_mbox_put` blocks until there has been a corresponding get operation and a slot is available.

`cyg_mbox_timed_put` Same as `cyg_mbox_put` if there is a spare slot. Otherwise, the function will wait a specified time limit and place the message if a slot becomes available. If the time limit expires, the operation returns false. Thus, `cyg_mbox_timed_put` is blocking only for less than or equal to a specified time interval.

`cyg_mbox_tryput` This is a nonblocking version, which returns true if the message is sent successfully and false if the mailbox is full.

Similarly, there are three variants to the get primitive.

`cyg_mbox_get` If there is a pending message in the specified mailbox, `cyg_mbox_get` returns with the message that was put into the mailbox. Otherwise this function blocks until there is a put operation.

`cyg_mbox_timed_get` Immediately returns a message if one is available. Otherwise, the function will wait until either a message is available or until a number of clock ticks have occurred. If the time limit expires, the operation returns a null pointer. Thus, `cyg_mbox_timed_get` is blocking only for less than or equal to a specified time interval.

`cyg_mbox_tryget` This is a nonblocking version, which returns a message if one is available and a null pointer if the mailbox is empty.

## Spinlocks

A spinlock is a flag that a thread can check before executing a particular piece of code. Recall from our discussion of Linux spinlocks in Chapter 6 the basic operation of the spinlock: Only one thread at a time can acquire a spinlock. Any other thread attempting to acquire the same lock will keep trying (spinning) until it can acquire the lock. In essence, a spinlock is built on an integer location in memory that is checked by each thread before it enters its critical section. If the value is 0, the thread sets the value to 1 and enters its critical section. If the value is nonzero, the thread continually checks the value until it is zero.

A spinlock should not be used on a single-processor system, which is why it is compiled away on Linux. As an example of the danger, consider a uniprocessor system with preemptive scheduling, in which a higher-priority thread attempts to acquire a spinlock already held by a lower-priority thread. The lower-priority thread cannot execute so as to finish its work and release the spinlock, because the higher-priority thread preempts it. The higher-priority thread can execute but is stuck checking the spinlock. As a result, the higher-priority thread will just loop forever and the lower-priority thread will never get another chance to run and release the spinlock. On an SMP system, the current owner of a spinlock can continue running on a different processor.

# GLOSSARY

**access method** The method that is used to find a file, a record, or a set of records.

**address space** The range of addresses available to a computer program.

**address translator** A functional unit that transforms virtual addresses to real addresses.

**application programming interface (API)** A standardized library of programming tools used by software developers to write applications that are compatible with a specific operating system or graphic user interface.

**asynchronous operation** An operation that occurs without a regular or predictable time relationship to a specified event, for example, the calling of an error diagnostic routine that may receive control at any time during the execution of a computer program.

**base address** An address that is used as the origin in the calculation of addresses in the execution of a computer program.

**batch processing** Pertaining to the technique of executing a set of computer programs such that each is completed before the next program of the set is started.

**Beowulf** Defines a class of clustered computing that focuses on minimizing the price-to-performance ratio of the overall system without compromising its ability to perform the computation work for which it is being built. Most Beowulf systems are implemented on Linux computers.

**binary semaphore** A semaphore that takes on only the values 0 and 1. A binary semaphore allows only one process or thread to have access to a shared critical resource at a time.

**block** (1) A collection of contiguous records that are recorded as a unit; the units are separated by interblock gaps. (2) A group of bits that are transmitted as a unit.

**B-tree** A technique for organizing indexes. In order to keep access time to a minimum, it stores the data keys in a balanced hierarchy that continually realigns itself as items are inserted and deleted. Thus, all nodes always have a similar number of keys.

**busy waiting** The repeated execution of a loop of code while waiting for an event to occur.

**cache memory** A memory that is smaller and faster than main memory and that is interposed between the processor and main memory. The cache acts as a buffer for recently used memory locations.

**central processing unit (CPU)** That portion of a computer that fetches and executes instructions. It consists of an Arithmetic and Logic Unit (ALU), a control unit, and registers. Often simply referred to as a *processor*.

**chained list** A list in which data items may be dispersed but in which each item contains an identifier for locating the next item.

**client** A process that requests services by sending messages to server processes.

**cluster** A group of interconnected, whole computers working together as a unified computing resource that can create the illusion of being one machine. The term *whole computer* means a system that can run on its own, apart from the cluster.

**communications architecture** The hardware and software structure that implements the communications function.

**compaction** A technique used when memory is divided into variable-size partitions. From time to time, the operating system shifts the partitions so they are contiguous and so all of the free memory is together in one block. See *external fragmentation*.

**concurrent** Pertaining to processes or threads that take place within a common interval of time during which they may have to alternately share common resources.

**consumable resource** A resource that can be created (produced) and destroyed (consumed). When a resource is acquired by a process, the resource ceases to exist. Examples of consumable resources are interrupts, signals, messages, and information in I/O buffers.

**critical section** In an asynchronous procedure of a computer program, a part that cannot be executed simultaneously with an associated critical section of another asynchronous procedure. See *mutual exclusion*.

**database** A collection of interrelated data, often with controlled redundancy, organized according to a schema to serve one or more applications; the data are stored so they can be used by different programs without concern for the data structure or organization. A common approach is used to add new data, and to modify and retrieve existing data.

**deadlock** (1) An impasse that occurs when multiple processes are waiting for the availability of a resource that will not become available because it is being held by another process that is in a similar wait state. (2) An impasse that occurs when multiple processes are waiting for an action by or a response from another process that is in a similar wait state.

**deadlock avoidance** A dynamic technique that examines each new resource request for deadlock. If the new request could lead to a deadlock, then the request is denied.

**deadlock detection** A technique in which requested resources are always granted when available. Periodically, the operating system tests for deadlock.

**deadlock prevention** A technique that guarantees that a deadlock will not occur. Prevention is achieved by assuring that one of the necessary conditions for deadlock is not met.

**demand paging** The transfer of a page from secondary memory to main memory storage at the moment of need. Compare *prepaging*.

**device driver** An operating system module (usually in the kernel) that deals directly with a device or I/O module.

**direct access** The capability to obtain data from a storage device or to enter data into a storage device in a sequence independent of their relative position, by means of addresses that indicate the physical location of the data.

**direct memory access (DMA)** A form of I/O in which a special module, called a DMA module, controls the exchange of data between main memory and an I/O device. The processor sends a request for the transfer of a block of data to the DMA module, and is interrupted only after the entire block has been transferred.

**disabled interrupt** A condition, usually created by the operating system, during which the processor will ignore interrupt request signals of a specified class.

**disk allocation table** A table that indicates which blocks on secondary storage are free and available for allocation to files.

**disk cache** A buffer, usually kept in main memory, that functions as a cache of disk blocks between disk memory and the rest of main memory.

**dispatch** To allocate time on a processor to jobs or tasks that are ready for execution.

**distributed operating system** A common operating system shared by a network of computers. The distributed operating system provides support for interprocess communication, process migration, mutual exclusion, and the prevention or detection of deadlock.

**dynamic relocation** A process that assigns new absolute addresses to a computer program during execution so the program may be executed from a different area of main storage.

**enabled interrupt** A condition, usually created by the operating system, during which the processor will respond to interrupt request signals of a specified class.

**encryption** The conversion of plain text or data into unintelligible form by means of a reversible mathematical computation.

**execution context** Same as *process state*.

**external fragmentation** Occurs when memory is divided into variable-size partitions corresponding to the blocks of data assigned to the memory (e.g., segments in main memory). As segments are moved into and out of the memory, gaps will occur between the occupied portions of memory.

**field** (1) Defined logical data that are part of a record. (2) The elementary unit of a record that may contain a data item, a data aggregate, a pointer, or a link.

**file** A set of related records treated as a unit.

**file allocation table (FAT)** A table that indicates the physical location on secondary storage of the space allocated to a file. There is one file allocation table for each file.

**file management system** A set of system software that provides services to users and applications in the use of files, including file access, directory maintenance, and access control.

**file organization** The physical order of records in a file, as determined by the access method used to store and retrieve them.

**first-come-first-served (FCFS)** Same as *FIFO*.

**first-in-first-out (FIFO)** A queueing technique in which the next item to be retrieved is the item that has been in the queue for the longest time.

**frame** In paged virtual storage, a fixed-length block of main memory that is used to hold one page of virtual memory.

**gang scheduling** The scheduling of a set of related threads to run on a set of processors at the same time, on a one-to-one basis.

**hash file** A file in which records are accessed according to the values of a key field. Hashing is used to locate a record on the basis of its key value.

**hashing** The selection of a storage location for an item of data by calculating the address as a function of the contents of the data. This technique complicates the storage allocation function but results in rapid random retrieval.

**hit ratio** In a two-level memory, the fraction of all memory accesses that are found in the faster memory (e.g., the cache).

**indexed access** Pertaining to the organization and accessing of the records of a storage structure through a separate index to the locations of the stored records.

**indexed file** A file in which records are accessed according to the value of key fields. An index is required that indicates the location of each record on the basis of each key value.

**indexed sequential access** Pertaining to the organization and accessing of the records of a storage structure through an index of the keys that are stored in arbitrarily partitioned sequential files.

**indexed sequential file** A file in which records are ordered according to the values of a key field. The main file is supplemented with an index file that contains a partial list of key values; the index provides a lookup capability to quickly reach the vicinity of a desired record.

**instruction cycle** The time period during which one instruction is fetched from memory and executed when a computer is given an instruction in machine language.

**internal fragmentation** Occurs when memory is divided into fixed-size partitions (e.g., page frames in main memory, physical blocks on disk). If a block of data is assigned to one or more partitions, then there may be wasted space in the last partition. This will occur if the last portion of data is smaller than the last partition.

**interrupt** A suspension of a process, such as the execution of a computer program, caused by an event external to that process and performed in such a way that the process can be resumed.

**interrupt handler** A routine, generally part of the operating system. When an interrupt occurs, control is transferred to the corresponding interrupt handler, which takes some action in response to the condition that caused the interrupt.

**job** A set of computational steps packaged to run as a unit.

**job control language (JCL)** A problem-oriented language that is designed to express statements in a job that are used to identify the job or to describe its requirements to an operating system.

**kernel** A portion of the operating system that includes the most heavily used portions of software. Generally, the kernel is maintained permanently in main memory. The kernel runs in a privileged mode and responds to calls from processes and interrupts from devices.

**kernel mode** A privileged mode of execution reserved for the kernel of the operating system. Typically, kernel mode allows access to regions of main memory that are unavailable to processes executing in a less-privileged mode, and also enables execution of certain machine instructions that are restricted to the kernel mode. Also referred to as *system mode* or *privileged mode*.

**last-in-first-out (LIFO)** A queueing technique in which the next item to be retrieved is the item most recently placed in the queue.

**lightweight process** A thread.

**livelock** A condition in which two or more processes continuously change their state in response to changes in the other process(es) without doing any useful work. This is similar to deadlock in that no progress is made, but it differs in that neither process is blocked or waiting for anything.

**locality of reference** The tendency of a processor to access the same set of memory locations repetitively over a short period of time.

**logical address** A reference to a memory location independent of the current assignment of data to memory. A translation must be made to a physical address before the memory access can be achieved.

**logical record** A record independent of its physical environment; portions of one logical record may be located in different physical records or several logical records or parts of logical records may be located in one physical record.

**macrokernel** A large operating system core that provides a wide range of services.

**mailbox** A data structure shared among a number of processes that is used as a queue for messages. Messages are sent to the mailbox and retrieved from the mailbox rather than passing directly from sender to receiver.

**main memory** Memory that is internal to the computer system, is program addressable, and can be loaded into registers for subsequent execution or processing.

**malicious software** Any software designed to cause damage to or use up the resources of a target computer. Malicious software (malware) is frequently concealed within or masquerades as legitimate software. In some cases, it spreads itself to other computers via e-mail or infected disks. Types of malicious software include viruses, Trojan horses, worms, and hidden software for launching denial-of-service attacks.

**memory cycle time** The time it takes to read one word from or write one word to memory. This is the inverse of the rate at which words can be read from or written to memory.

**memory partitioning** The subdividing of storage into independent sections.

**message** A block of information that may be exchanged between processes as a means of communication.

**microkernel** A small, privileged operating system core that provides process scheduling, memory management, and communication services and relies on other processes to perform some of the functions traditionally associated with the operating system kernel.

**mode switch** A hardware operation that occurs that causes the processor to execute in a different mode (kernel or process). When the mode switches from

process to kernel, the program counter, processor status word, and other registers are saved. When the mode switches from kernel to process, this information is restored.

**monitor** A programming language construct that encapsulates variables, access procedures, and initialization code within an abstract data type. The monitor's variable may only be accessed via its access procedures and only one process may be actively accessing the monitor at any one time. The access procedures are *critical sections*. A monitor may have a queue of processes that are waiting to access it.

**monolithic kernel** A large kernel containing virtually the complete operating system, including scheduling, file system, device drivers, and memory management. All the functional components of the kernel have access to all of its internal data structures and routines. Typically, a monolithic kernel is implemented as a single process, with all elements sharing the same address space.

**multilevel security** A capability that enforces access control across multiple levels of classification of data.

**multiprocessing** A mode of operation that provides for parallel processing by two or more processors of a multiprocessor.

**multiprocessor** A computer with two or more processors that have common access to a main storage.

**multiprogramming** A mode of operation that provides for the interleaved execution of two or more computer programs by a single processor. The same as multitasking, using different terminology.

**multiprogramming level** The number of processes that are partially or fully resident in main memory.

**multithreading** Multitasking within a single program. It allows multiple streams of instructions (threads) to execute concurrently within the same program, each stream processing a different transaction or message. Each stream is a "subprocess," and the operating system typically cooperates with the application to handle the threads.

**multitasking** A mode of operation that provides for the concurrent performance or interleaved execution of two or more computer tasks. The same as multiprogramming, using different terminology.

**mutex** A programming flag used to grab and release an object. When data are acquired that cannot be shared or processing is started that cannot be performed simultaneously elsewhere in the system, the mutex is set to "lock," which blocks other attempts to use it. The mutex is set to "unlock" when the data are no longer needed or the routine is finished. Similar to a *binary semaphore*. A key difference between the two is that the process that locks the mutex (sets the value to zero) must be the one to unlock it (sets the value to 1). In contrast, it is possible for one process to lock a binary semaphore and for another to unlock it.

**mutual exclusion** A condition in which there is a set of processes, only one of which is able to access a given resource or perform a given function at any time. See *critical section*.

**nonprivileged state** An execution context that does not allow sensitive hardware instructions to be executed, such as the halt instruction and I/O instructions.

**nonuniform memory access (NUMA) multiprocessor** A shared-memory multiprocessor in which the access time from a given processor to a word in memory varies with the location of the memory word.

**object request broker** An entity in an object-oriented system that acts as an intermediary for requests sent from a client to a server.

**operating system** Software that controls the execution of programs and provides services such as resource allocation, scheduling, input/output control, and data management.

**page** In virtual storage, a fixed-length block that has a virtual address and is transferred as a unit between main memory and secondary memory.

**page fault** Occurs when the page containing a referenced word is not in main memory. This causes an interrupt and requires the proper page be brought into main memory.

**page frame** A fixed-size contiguous block of main memory used to hold a page.

**paging** The transfer of pages between main memory and secondary memory.

**physical address** The absolute location of a unit of data in memory (e.g., word or byte in main memory, block on secondary memory).

**pipe** A circular buffer allowing two processes to communicate on the producer–consumer model. Thus, it is a first-in-first-out queue, written by one process and read by another. In some systems, the pipe is generalized to allow any item in the queue to be selected for consumption.

**preemption** Reclaiming a resource from a process before the process has finished using it.

**prepaging** The retrieval of pages other than the one demanded by a page fault. The hope is that the additional pages will be needed in the near future, conserving disk I/O. Compare *demand paging*.

**priority inversion** A circumstance in which the operating system forces a higher-priority task to wait for a lower-priority task.

**privileged instruction** An instruction that can be executed only in a specific mode, usually by a supervisory program.

**privileged mode** Same as *kernel mode*.

**process** A program in execution. A process is controlled and scheduled by the operating system. Same as *task*.

**process control block** The manifestation of a process in an operating system. It is a data structure containing information about the characteristics and state of the process.

**process descriptor** Same as process control block.

**process image** All of the ingredients of a process, including program, data, stack, and process control block.

**process migration** The transfer of a sufficient amount of the state of a process from one machine to another for the process to execute on the target machine.

**process spawning** The creation of a new process by another process.

**process state** All of the information the operating system needs to manage a process and the processor needs to properly execute the process. The process state includes the contents of the various processor registers, such as the program counter and data registers; it also includes information of use to the operating system, such as the priority of the process and whether the process is waiting for the completion of a particular I/O event. Same as *execution context*.

**process switch** An operation that switches the processor from one process to another by saving all the process control block, registers, and other information for the first and replacing them with the process information for the second.

**processor** In a computer, a functional unit that interprets and executes instructions. A processor consists of at least an instruction control unit and an arithmetic unit.

**program counter** Instruction address register.

**program status word (PSW)** A register or set of registers that contains condition codes, execution mode, and other status information that reflects the state of a process.

**programmed I/O** A form of I/O in which the CPU issues an I/O command to an I/O module and must then wait for the operation to be complete before proceeding.

**race condition** Situation in which multiple processes access and manipulate shared data with the outcome dependent on the relative timing of the processes.

**real address** A physical address in main memory.

**real-time system** An operating system that must schedule and manage real-time tasks.

**real-time task** A task that is executed in connection with some process or function or set of events external to the computer system, and must meet one or more deadlines to interact effectively and correctly with the external environment.

**record** A group of data elements treated as a unit.

**reentrant procedure** A routine that may be entered before the completion of a prior execution of the same routine and execute correctly.

**registers** High-speed memory internal to the CPU. Some registers are user visible; that is, available to the programmer via the machine instruction set. Other registers are used only by the CPU, for control purposes.

**relative address** An address calculated as a displacement from a base address.

**remote procedure call (RPC)** A technique by which two programs on different machines interact using procedure call/return syntax and semantics. Both the called and calling program behave as if the partner program were running on the same machine.

**rendezvous** In message passing, a condition in which both the sender and receiver of a message are blocked until the message is delivered.

**resident set** That portion of a process that is actually in main memory at a given time. Compare *working set*.

**response time** In a data system, the elapsed time between the end of transmission of an enquiry message and the beginning of the receipt of a response message, measured at the enquiry terminal.

**reusable resource** A resource that can be safely used by only one process at a time and is not depleted by that use. Processes obtain reusable resource units that they later release for reuse by other processes. Examples of reusable resources include processors, I/O channels, main and secondary memory, devices, and data structures such as files, databases, and semaphores.

**round robin** A scheduling algorithm in which processes are activated in a fixed cyclic order; that is, all processes are in a circular queue. A process that cannot proceed because it is waiting for some event (e.g., termination of a child process or an input/output operation) returns control to the scheduler.

**scheduling** To select jobs or tasks that are to be dispatched. In some operating systems, other units of work, such as input/output operations, may also be scheduled.

**secondary memory** Memory located outside the computer system itself; that is, it cannot be processed directly by the processor. It must first be copied into main memory. Examples include disk and tape.

**segment** In virtual memory, a block that has a virtual address. The blocks of a program may be of unequal length, and may even be of dynamically varying lengths.

**segmentation** The division of a program or application into segments as part of a virtual memory scheme.

**semaphore** An integer value used for signaling among processes. Only three operations may be performed on a semaphore, all of which are atomic: initialize, decrement, and increment. Depending on the exact definition of the semaphore, the decrement operation may result in the blocking of a process, and the increment operation may result in the unblocking of a process. Also known as a *counting semaphore* or a *general semaphore*.

**sequential access** The capability to enter data into a storage device or a data medium in the same sequence as the data are ordered, or to obtain data in the same order as they were entered.

**sequential file** A file in which records are ordered according to the values of one or more key fields and processed in the same sequence from the beginning of the file.

**server** (1) A process that responds to request from clients via messages. (2) In a network, a data station that provides facilities to other stations; for example, a file server, a print server, and a mail server.

**session** A collection of one or more processes that represents a single interactive user application or operating system function. All keyboard and mouse input is directed to the foreground session, and all output from the foreground session is directed to the display screen.

**shell** The portion of the operating system that interprets interactive user commands and job control language commands. It functions as an interface between the user and the operating system.

**spin lock** Mutual exclusion mechanism in which a process executes in an infinite loop waiting for the value of a lock variable to indicate availability.

**spooling** The use of secondary memory as buffer storage to reduce processing delays when transferring data between peripheral equipment and the processors of a computer.

**stack** An ordered list in which items are appended to and deleted from the same end of the list, known as the top. That is, the next item appended to the list is put on the top, and the next item to be removed from the list is the item that has been in the list the shortest time. This method is characterized as last in first out.

**starvation** A condition in which a process is indefinitely delayed because other processes are always given preference.

**strong semaphore** A semaphore in which all processes waiting on the same semaphore are queued and will eventually proceed in the same order as they executed the wait (P) operations (FIFO order).

**swapping** A process that interchanges the contents of an area of main storage with the contents of an area in secondary memory.

**symmetric multiprocessing (SMP)** A form of multiprocessing that allows the operating system to execute on any available processor or on several available processors simultaneously.

**synchronous operation** An operation that occurs regularly or predictably with respect to the occurrence of a specified event in another process, for example, the calling of an input/output routine that receives control at a precoded location in a computer program.

**synchronization** Situation in which two or more processes coordinate their activities based on a condition.

**system bus** A bus used to interconnect major computer components (CPU, memory, I/O).

**system mode** Same as *kernel mode*.

**task** Same as *process*.

**thrashing** A phenomenon in virtual memory schemes, in which the processor spends most of its time swapping pieces rather than executing instructions.

**thread** A dispatchable unit of work. It includes a processor context (which includes the program counter and stack pointer) and its own data area for a stack (to enable subroutine branching). A thread executes sequentially and is interruptible so the processor can turn to another thread. A process may consist of multiple threads.

**thread switch** The act of switching processor control from one thread to another within the same process.

**time sharing** The concurrent use of a device by a number of users.

**time slice** The maximum amount of time that a process can execute before being interrupted.

**time slicing** A mode of operation in which two or more processes are assigned quanta of time on the same processor.

**trace** A sequence of instructions that are executed when a process is running.

**translation lookaside buffer (TLB)** A high-speed cache used to hold recently referenced page table entries as part of a paged virtual memory scheme. The TLB reduces the frequency of access to main memory to retrieve page table entries.

**trap** An unprogrammed conditional jump to a specified address that is automatically activated by hardware; the location from which the jump was made is recorded.

**trap door** Secret undocumented entry point into a program, used to grant access without normal methods of access authentication.

**Trojan horse** A computer program that appears to have a useful function, but also has a hidden and potentially malicious function that evades security mechanisms, sometimes by exploiting legitimate authorizations of a system entity that invokes the Trojan horse program.

**trusted system** A computer and operating system that can be verified to implement a given security policy.

**user mode** The least-privileged mode of execution. Certain regions of main memory and certain machine instructions cannot be used in this mode.

**virtual address** The address of a storage location in virtual memory.

**virtual machine** One instance of an operating system along with one or more applications running in an isolated partition within the computer. It enables different operating systems to run in the same computer at the same time as well as prevents applications from interfering with each other.

**virtual memory** The storage space that may be regarded as addressable main storage by the user of a computer system in which virtual addresses are mapped into real addresses. The size of virtual storage is limited by the addressing scheme of the computer system and by the amount of secondary memory available and not by the actual number of main storage locations.

**virus** Software that, when executed, tries to replicate itself into other executable code; when it succeeds the code is said to be infected. When the infected code is executed, the virus also executes.

**weak semaphore** A semaphore in which all processes waiting on the same semaphore proceed in an unspecified order (i.e., the order is unknown or indeterminate).

**word** An ordered set of bytes or bits that is the normal unit in which information may be stored, transmitted, or operated on within a given computer. Typically, if a processor has a fixed-length instruction set, then the instruction length equals the word length.

**working set** The working set with parameter $\Delta$ for a process at virtual time $t$, $W(t, \Delta)$ is the set of pages of that process that have been referenced in the last $\Delta$ time units. Compare *resident set*.

**worm** A destructive program that replicates itself throughout a single computer or across a network, both wired and wireless. It can do damage by sheer reproduction, consuming internal disk and memory resources within a single computer or by exhausting network bandwidth. It can also deposit a Trojan that turns a computer into a zombie for spam and other malicious purposes. Very often, the terms "worm" and "virus" are used synonymously; however, worm implies an automatic method for reproducing itself in other computers.